排序

排序

简单的排序算法

堆与堆排序

高效的排序算法

基数排序

外部排序

排序

将一组数据元素按照某种规则（通常是关键字的大小）重新排列成有序的过程。

排序

将一组数据元素按照某种规则（通常是关键字的大小）重新排列成有序的过程。

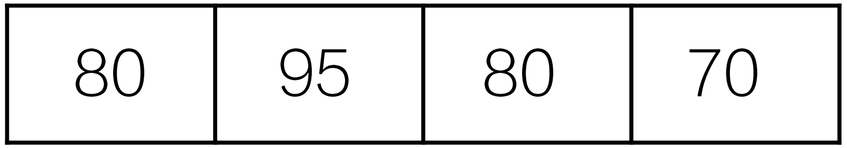
对于不同的排序算法，除了要掌握算法的实现思想及具体实现外，我们需要关注排序算法的三个核心指标：

时间复杂度：排序一组规模为 n 的数据，算法需要执行多少次基本操作？这会直接影响排序效率。

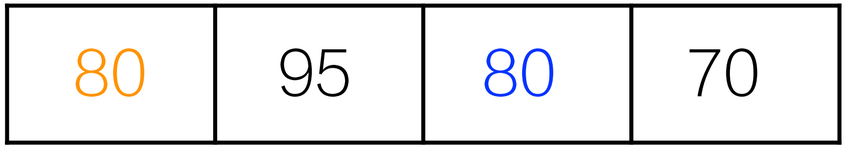
空间复杂度：排序过程中需要使用多少额外的内存空间？

排序的稳定性：排序前后，相等元素之间的相对顺序是否保持不变？

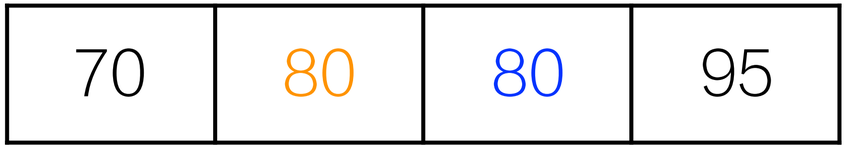
排序的稳定性



排序的稳定性



排序的稳定性



简单的排序算法

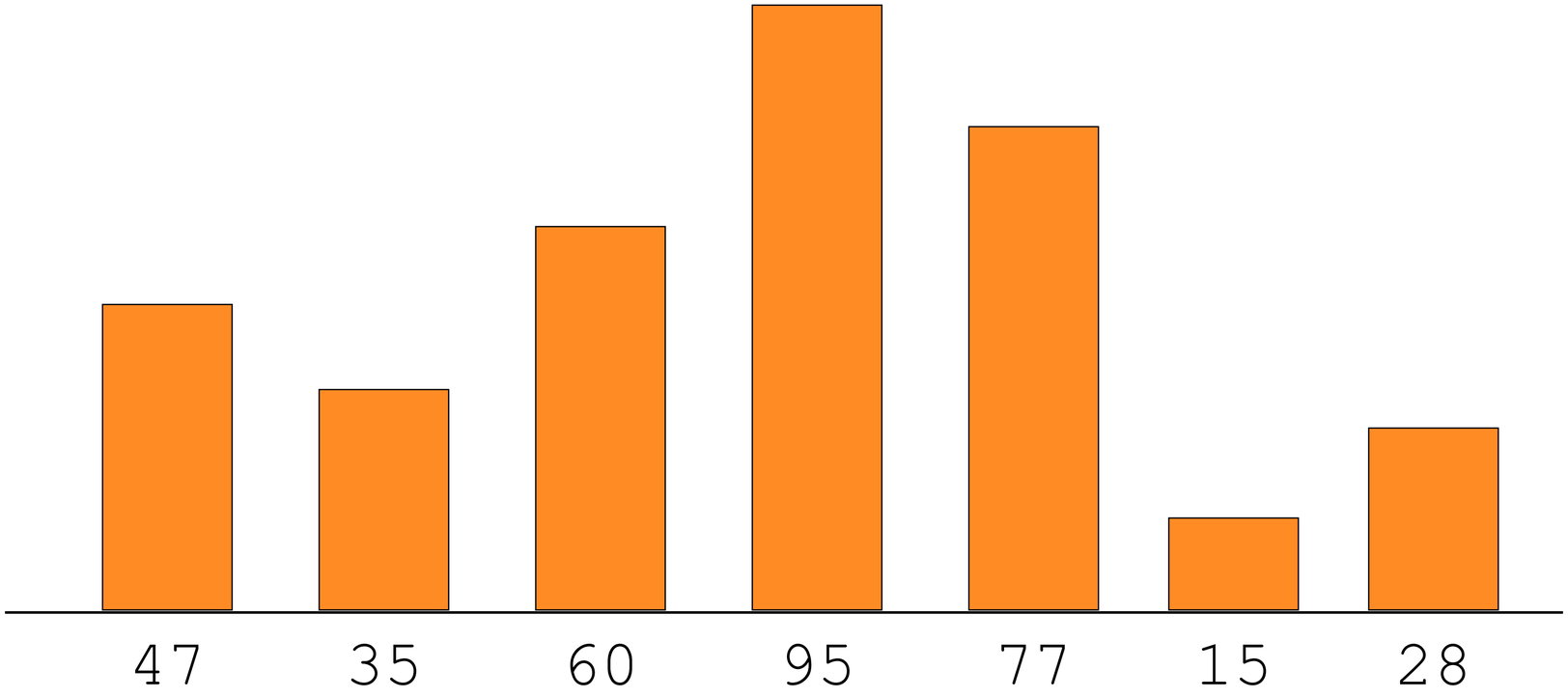
冒泡排序

插入排序

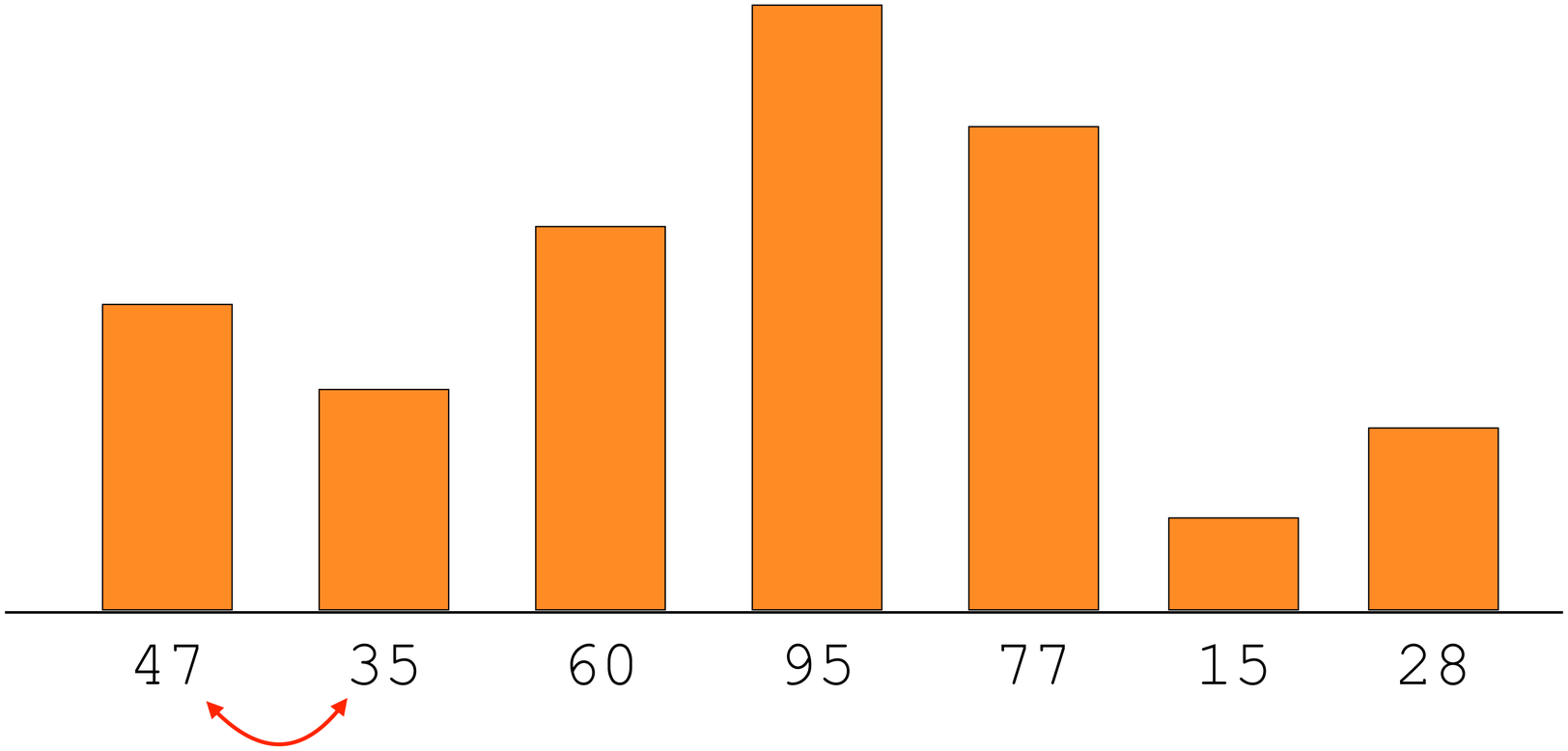
折半插入排序

简单选择排序

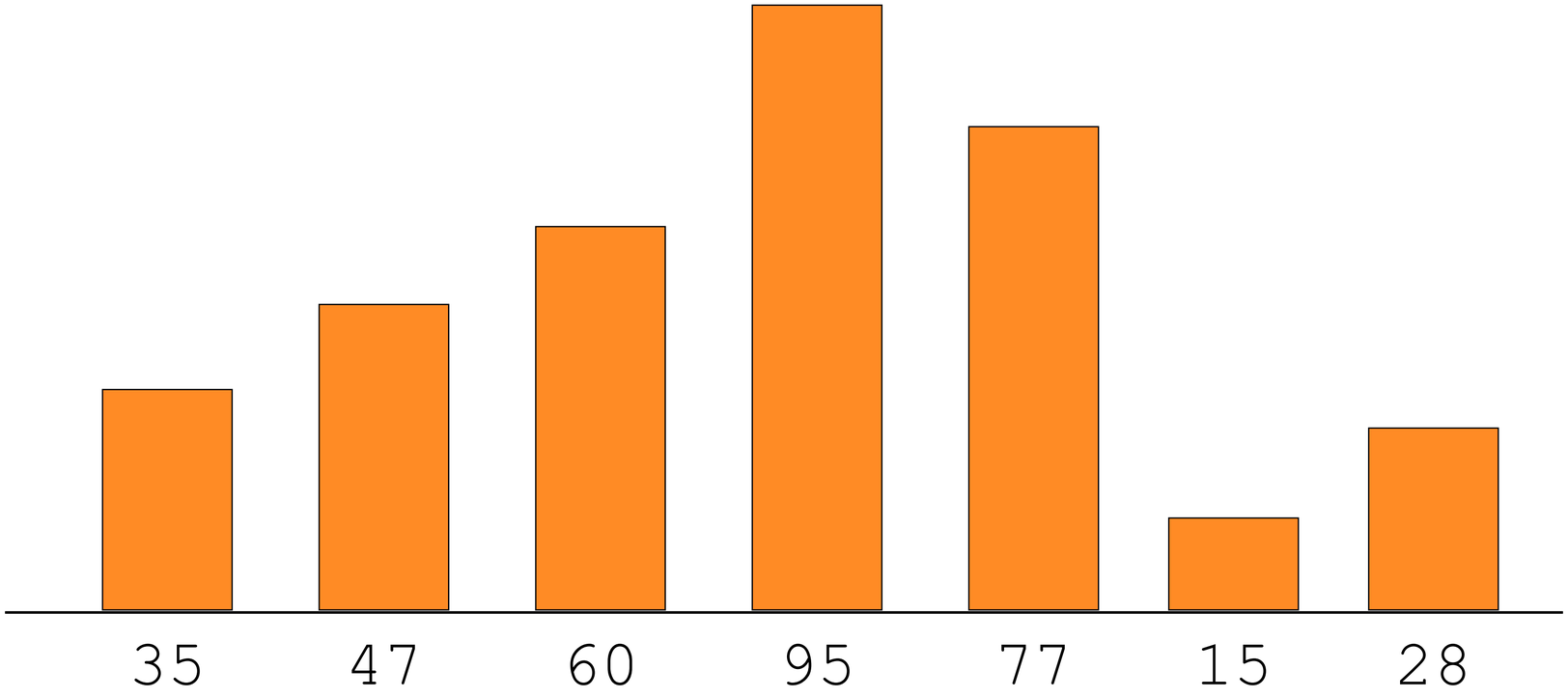
冒泡排序



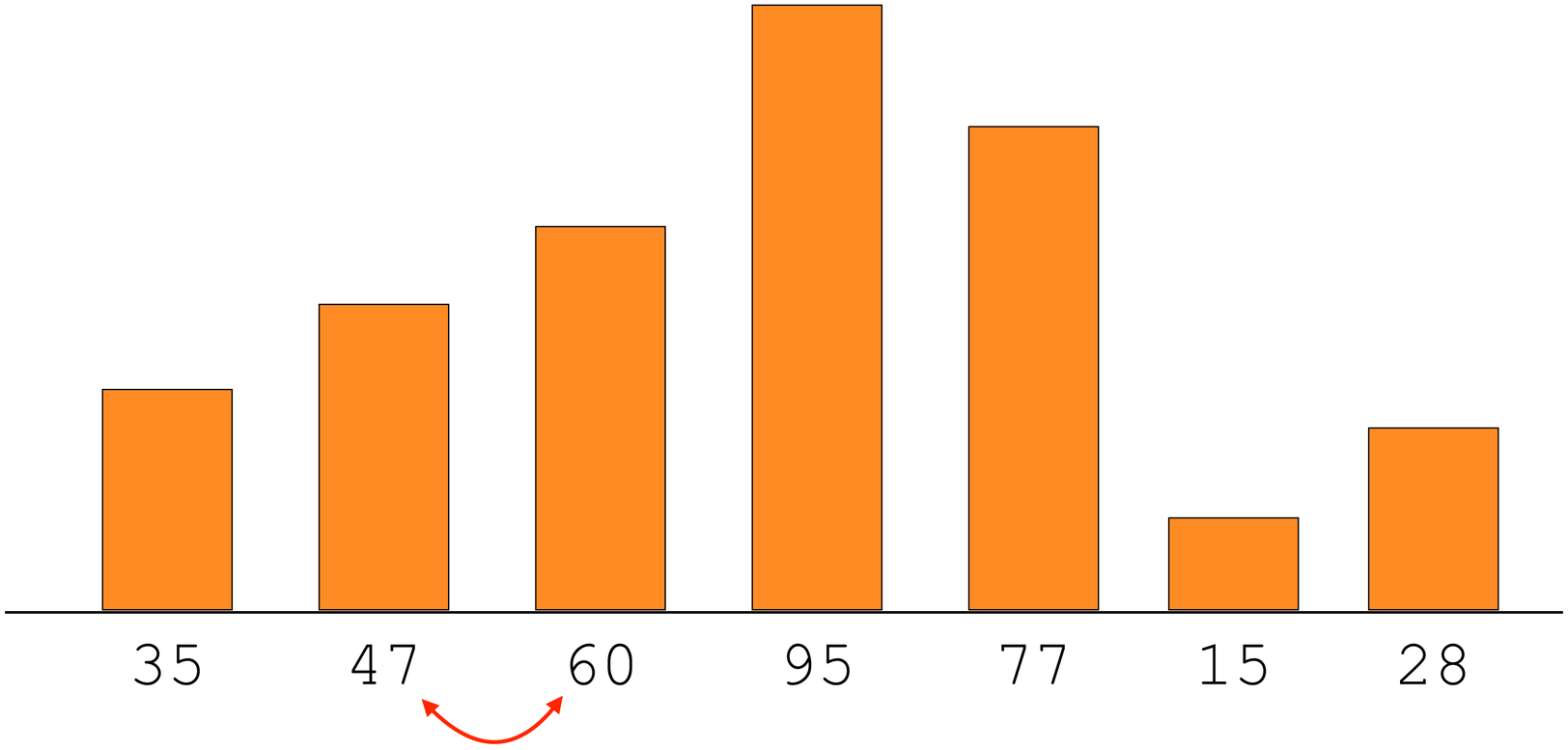
冒泡排序



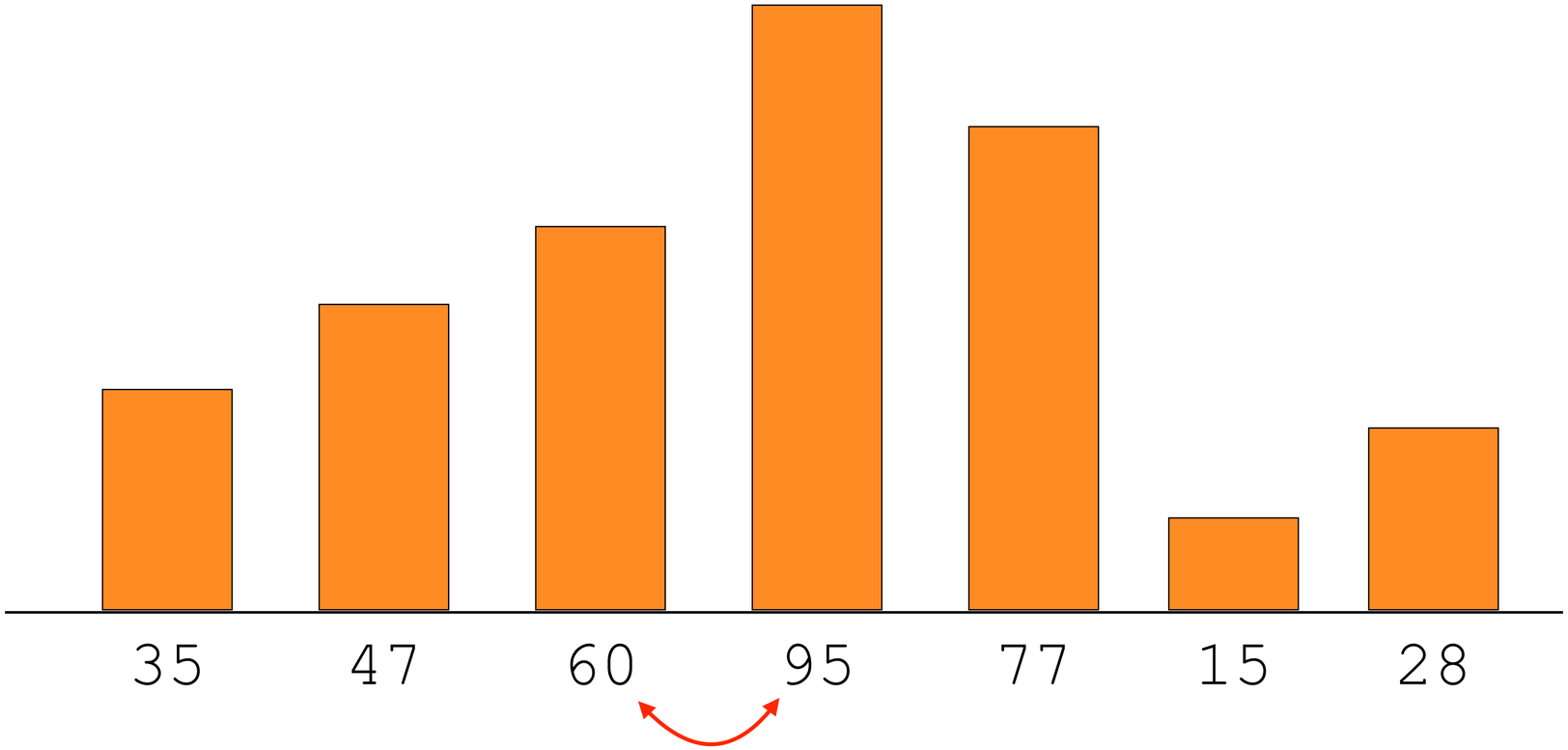
冒泡排序



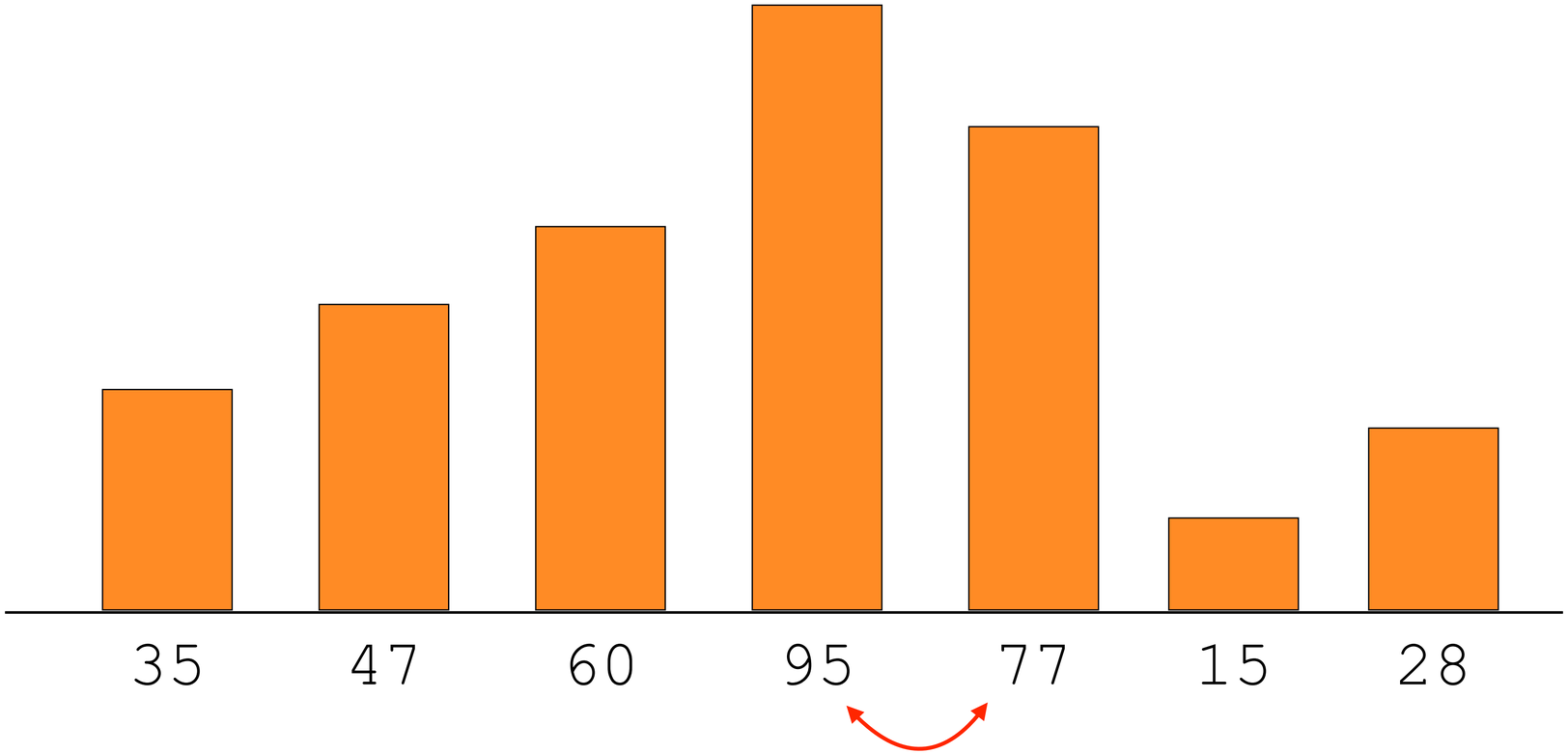
冒泡排序



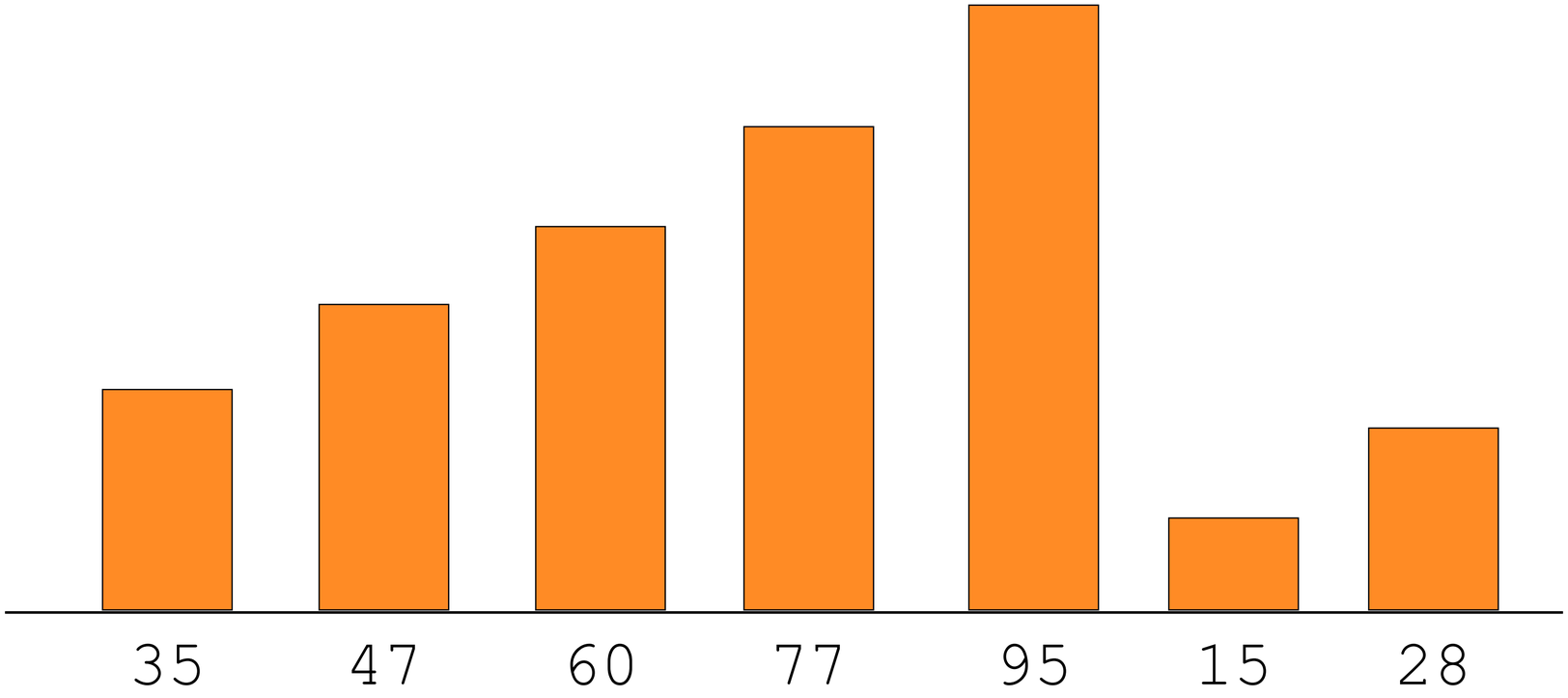
冒泡排序



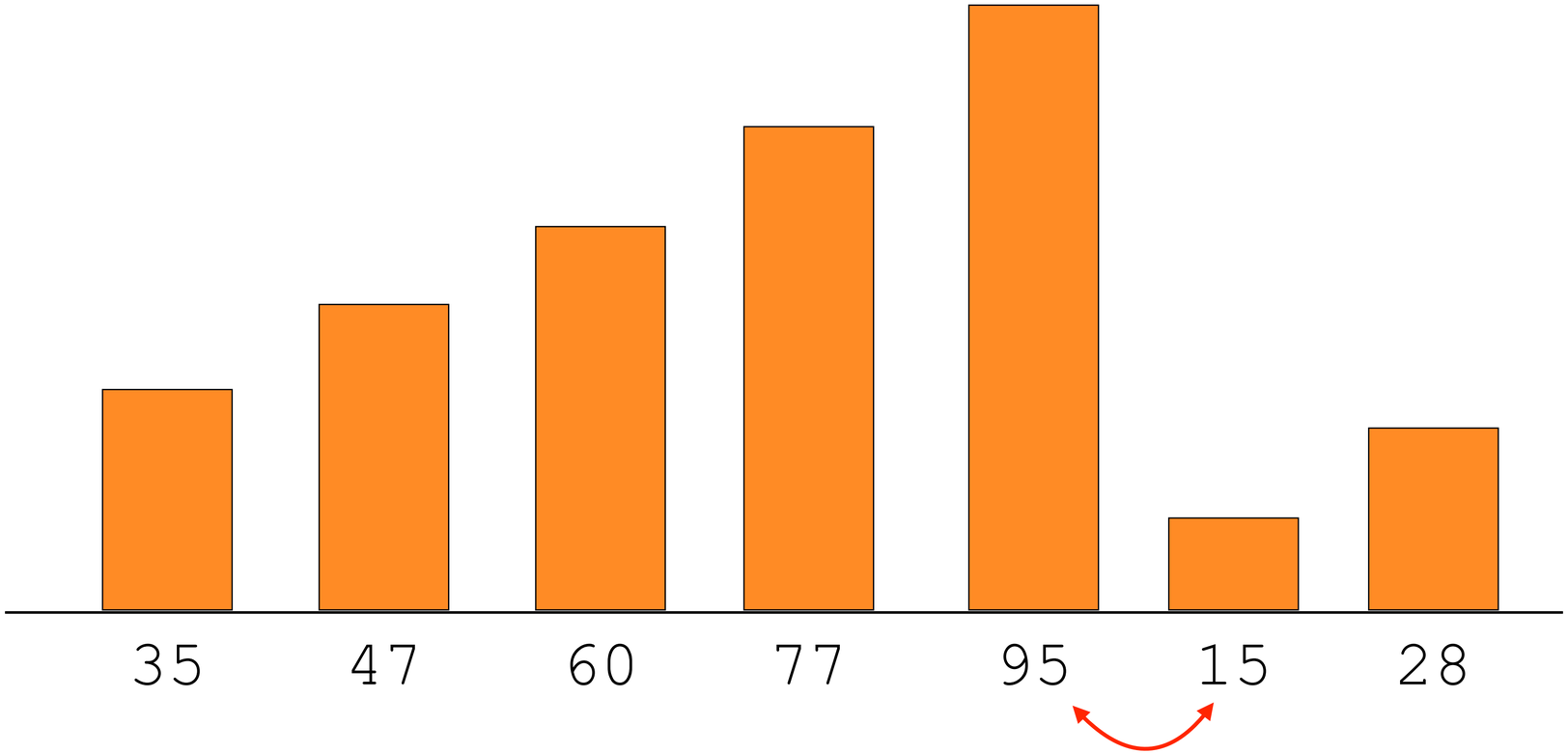
冒泡排序



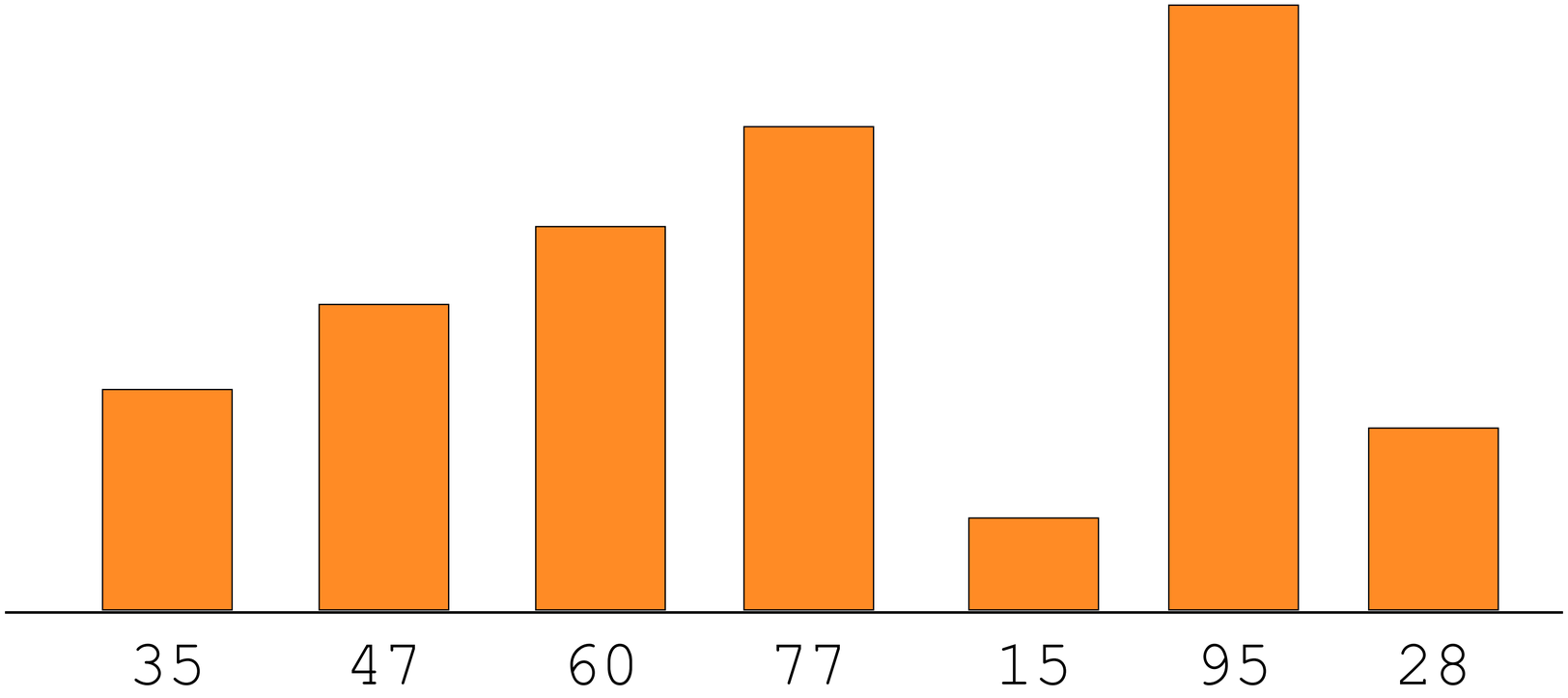
冒泡排序



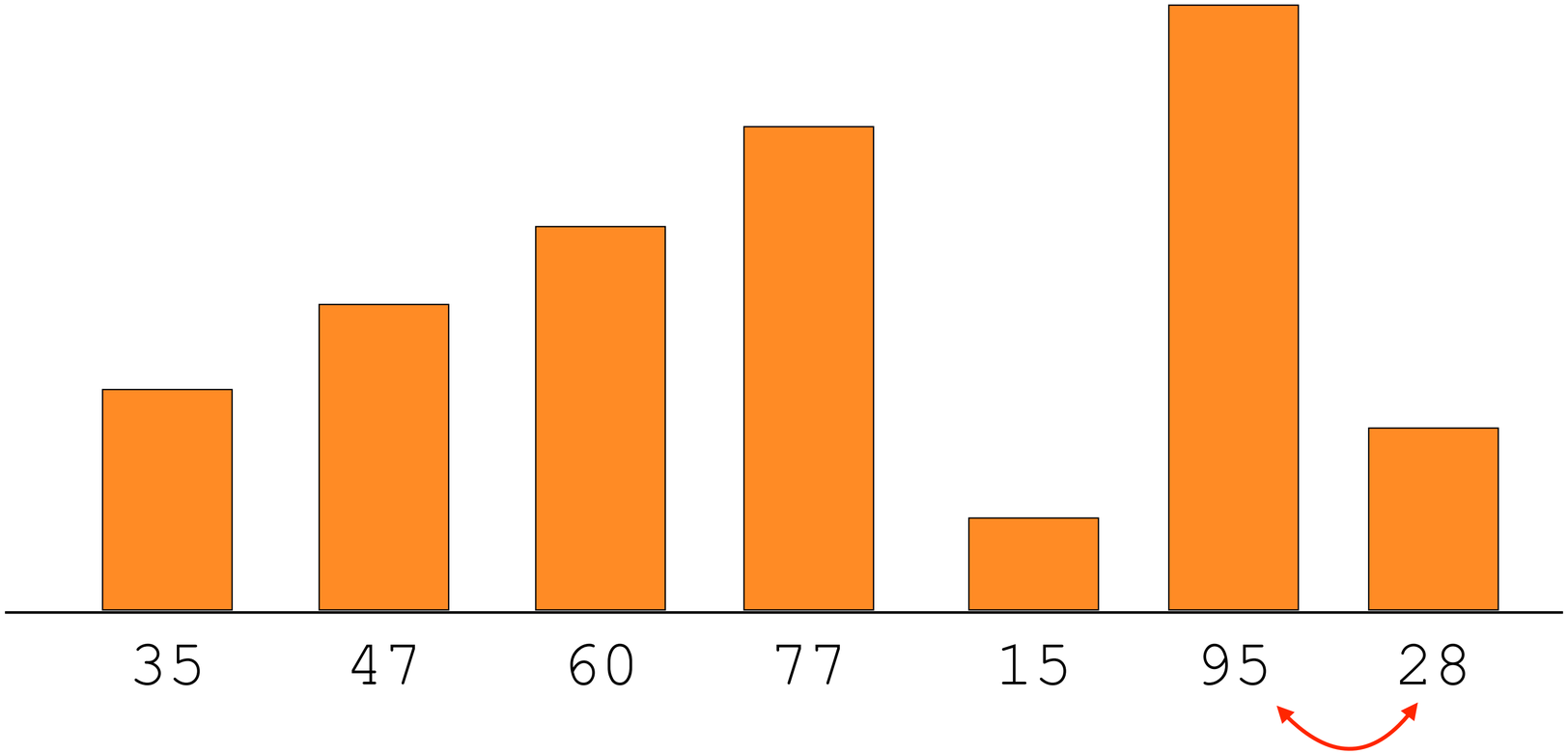
冒泡排序



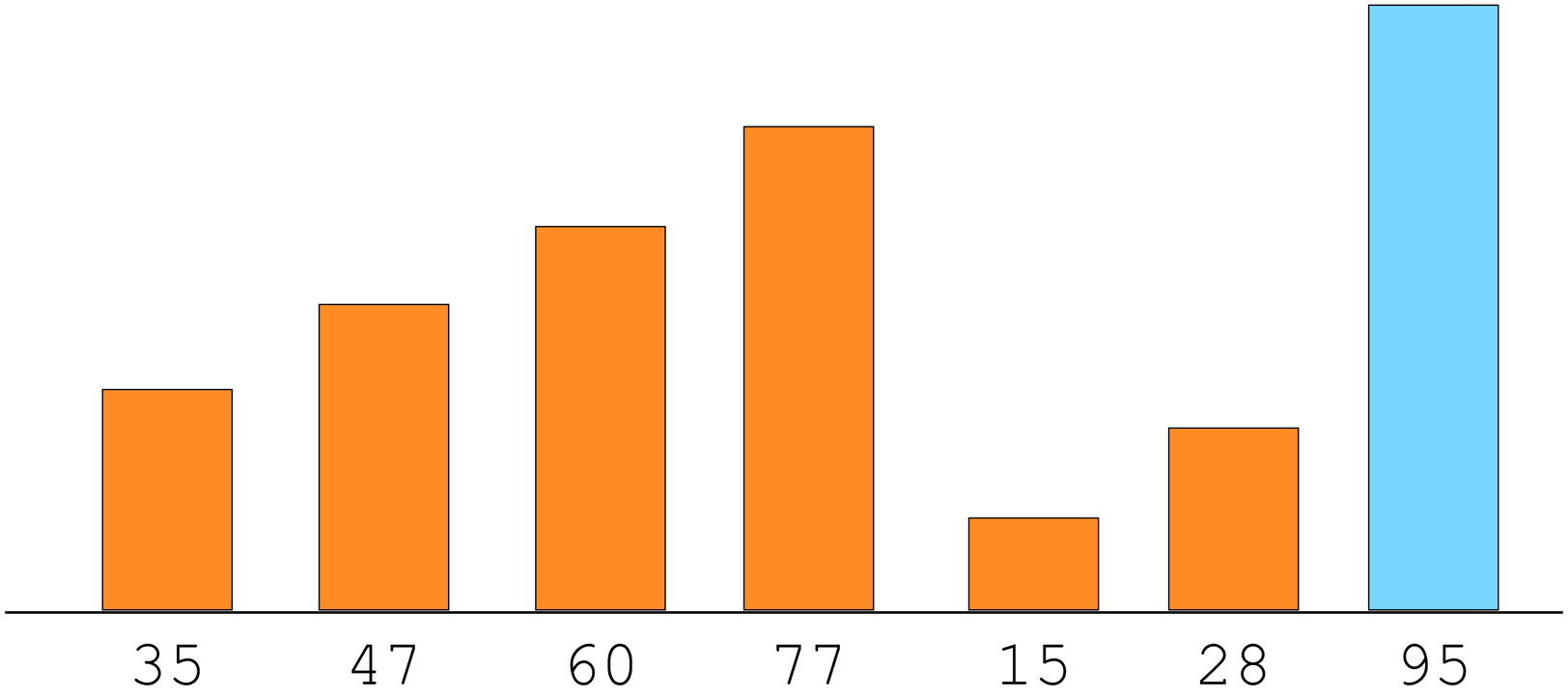
冒泡排序



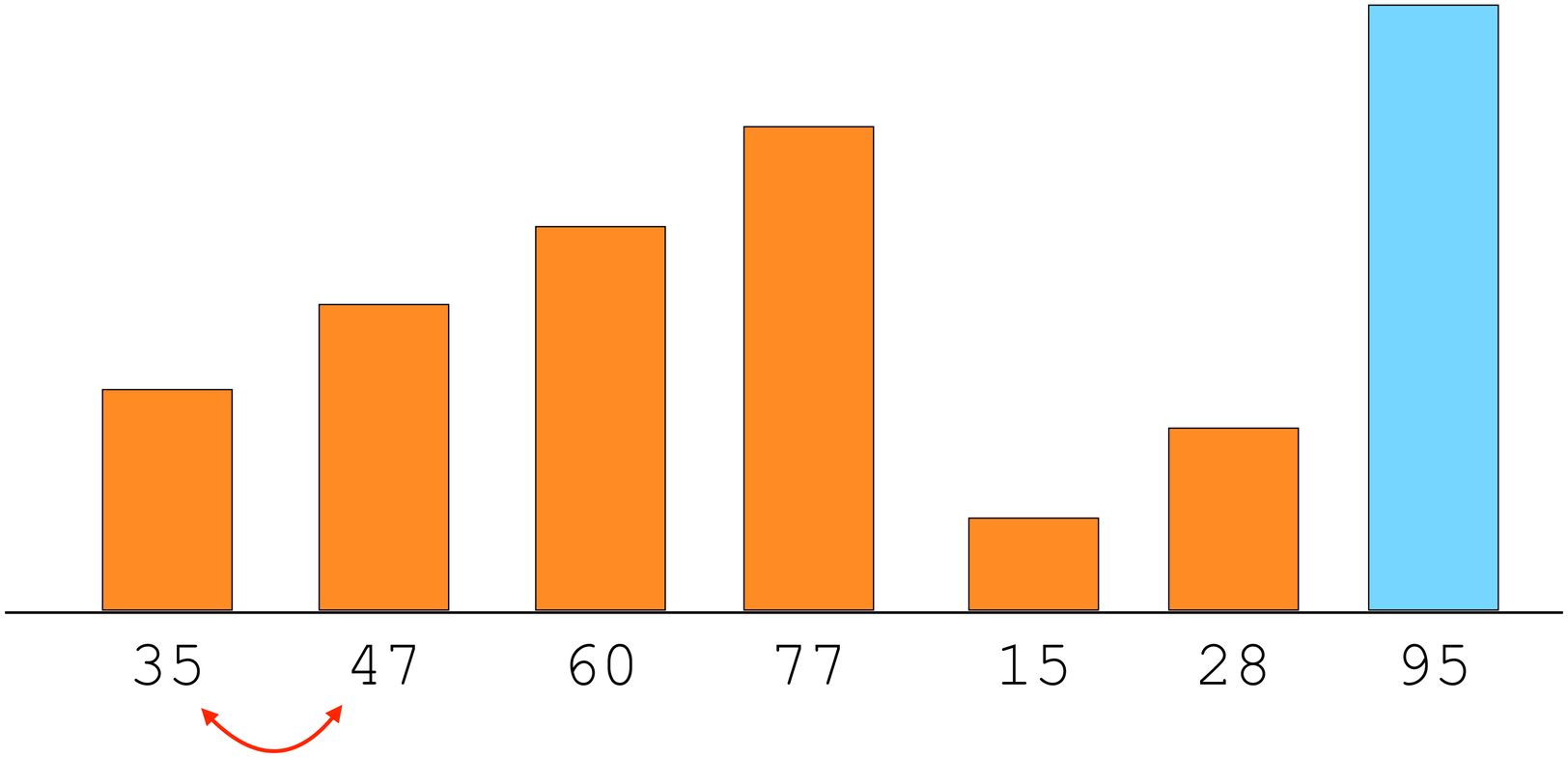
冒泡排序



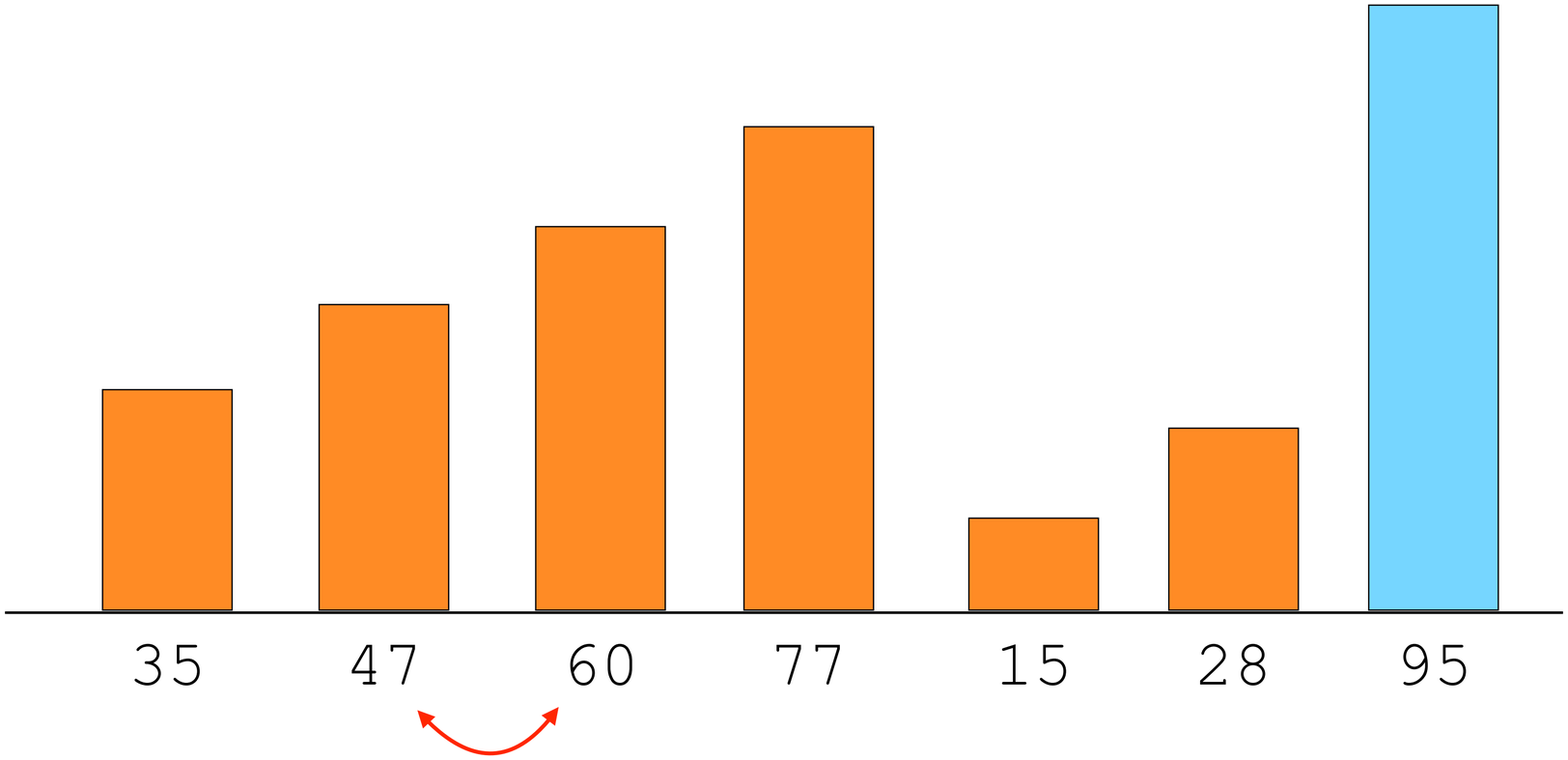
冒泡排序



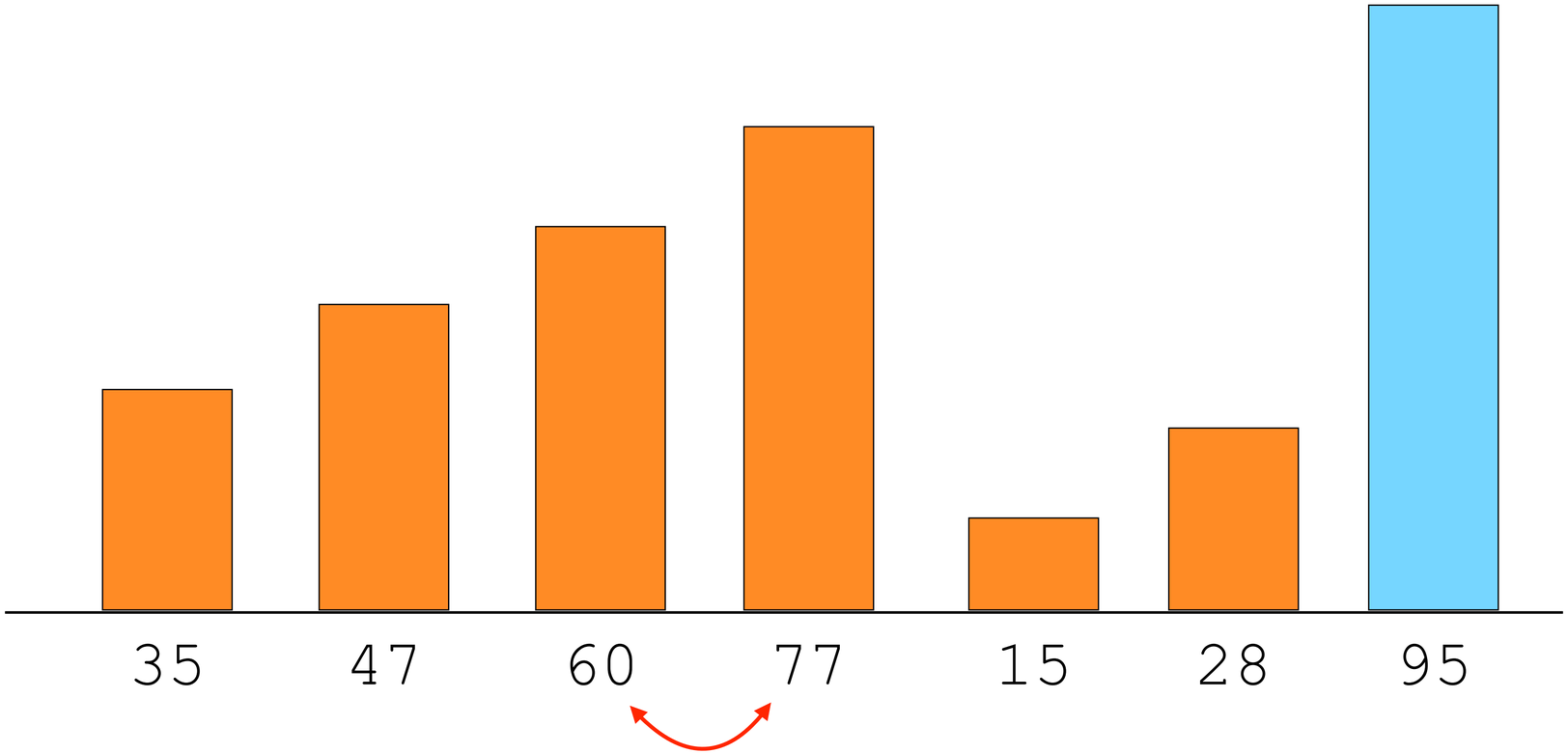
冒泡排序



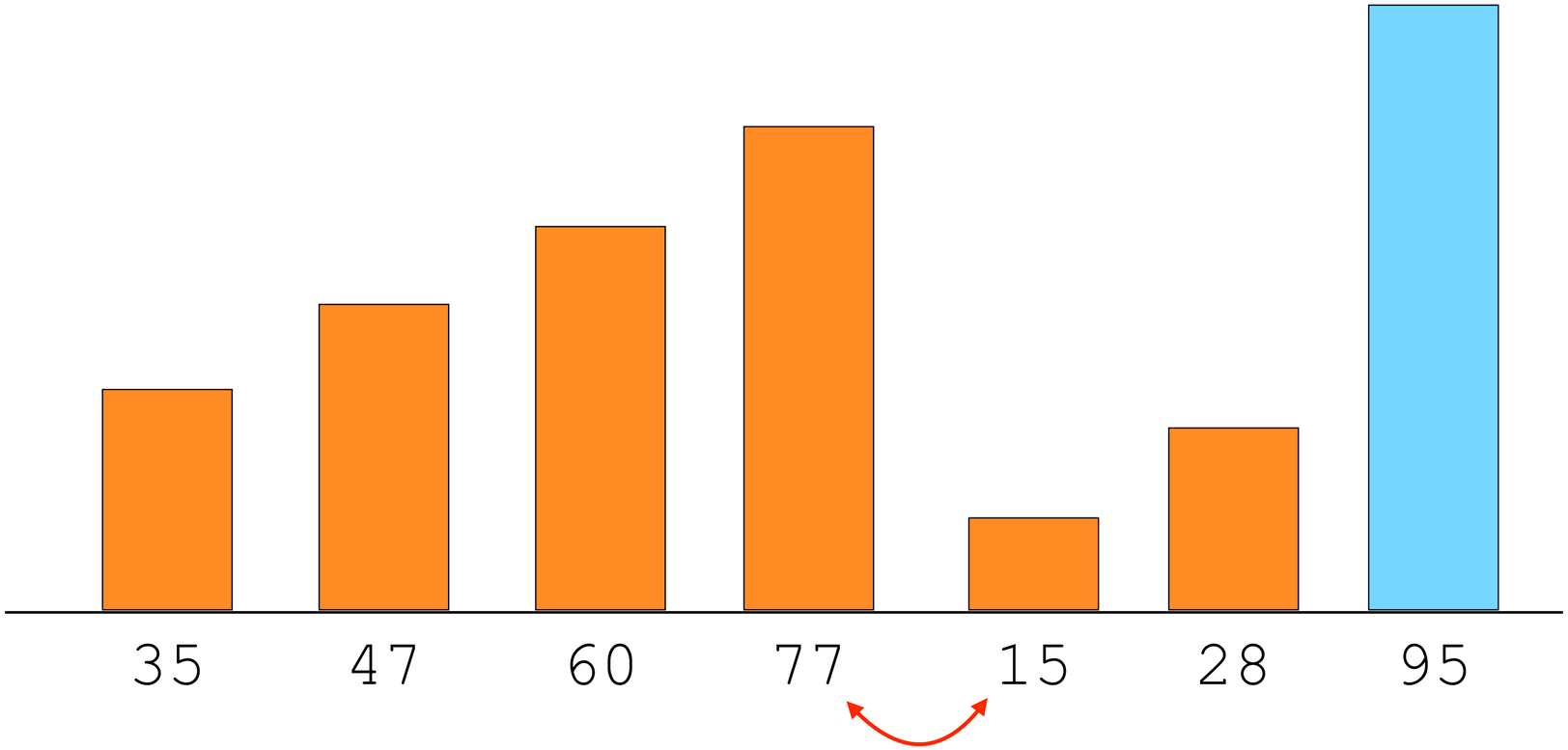
冒泡排序



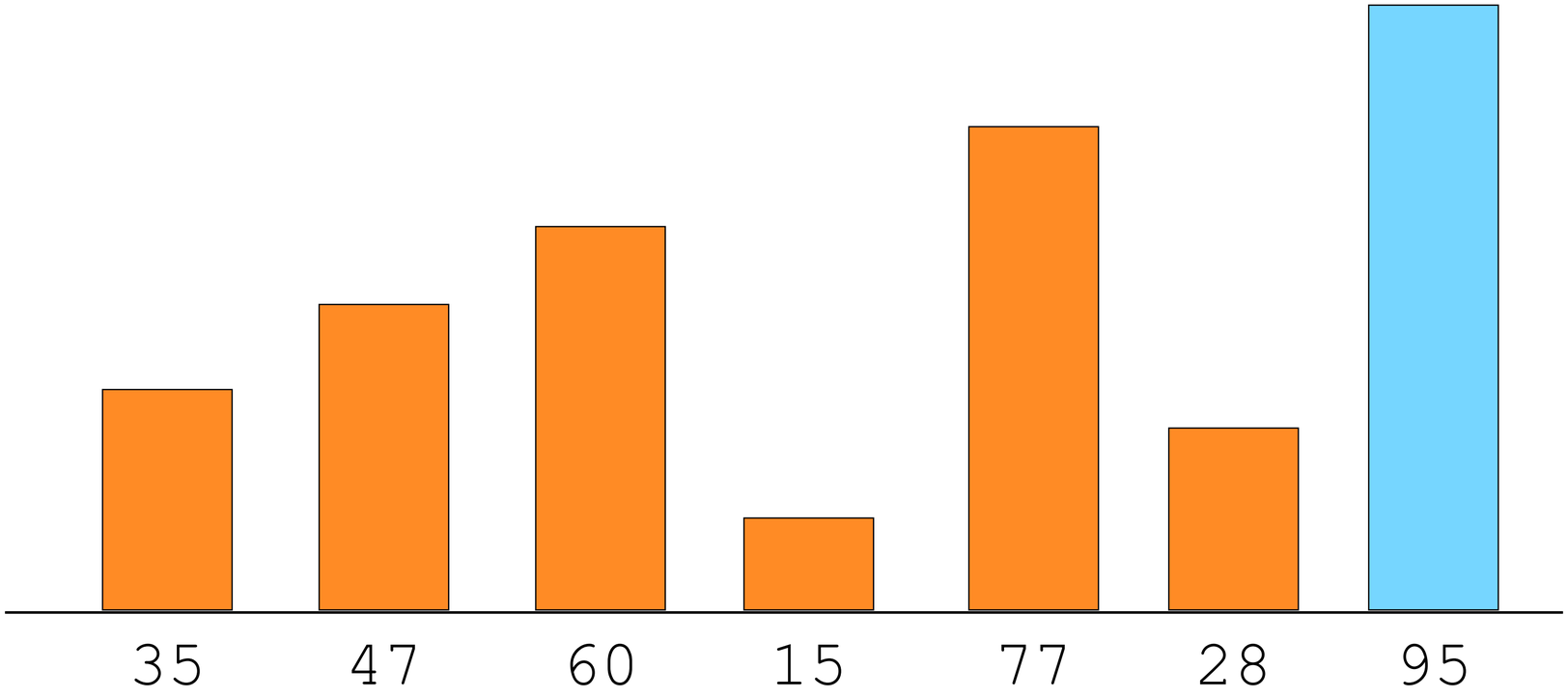
冒泡排序



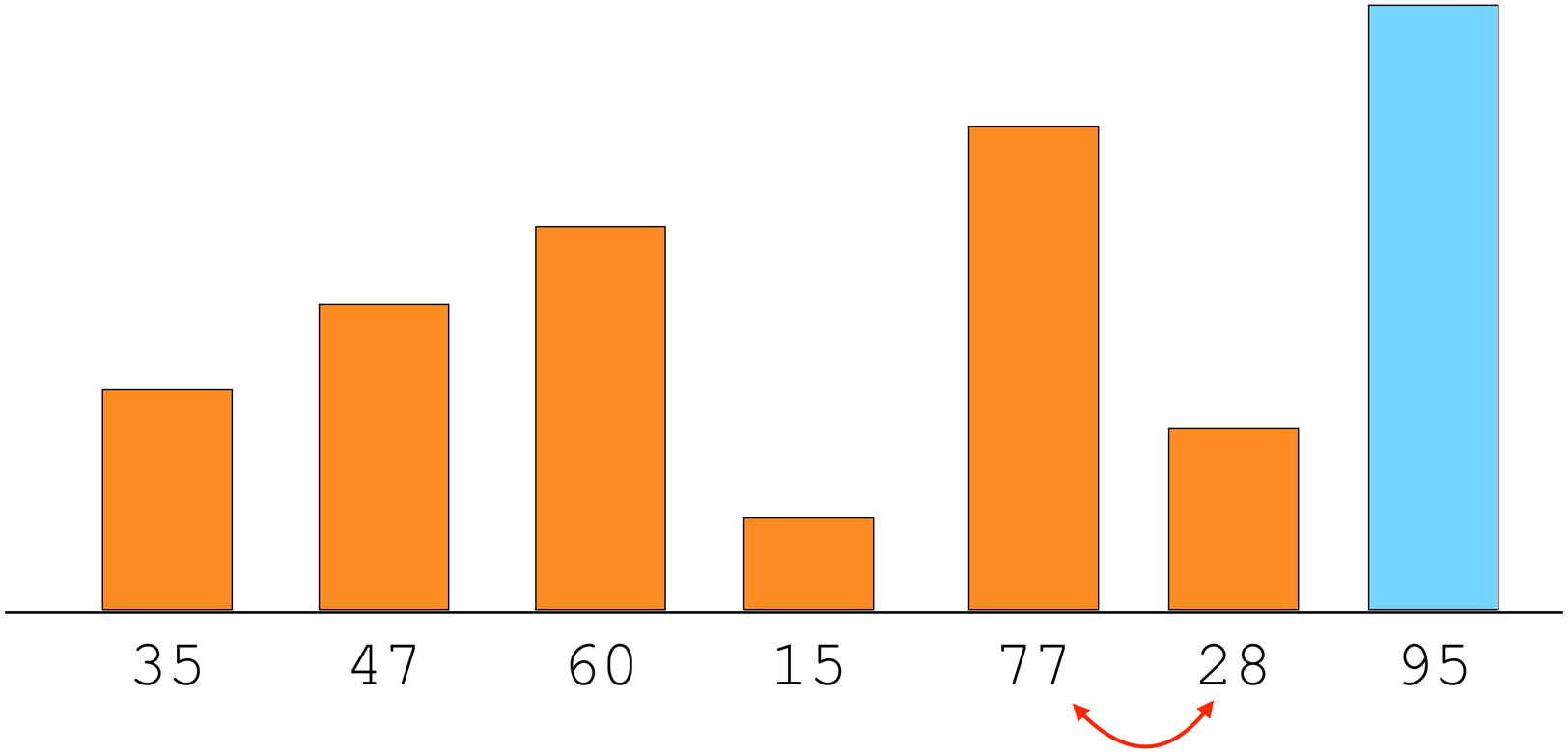
冒泡排序



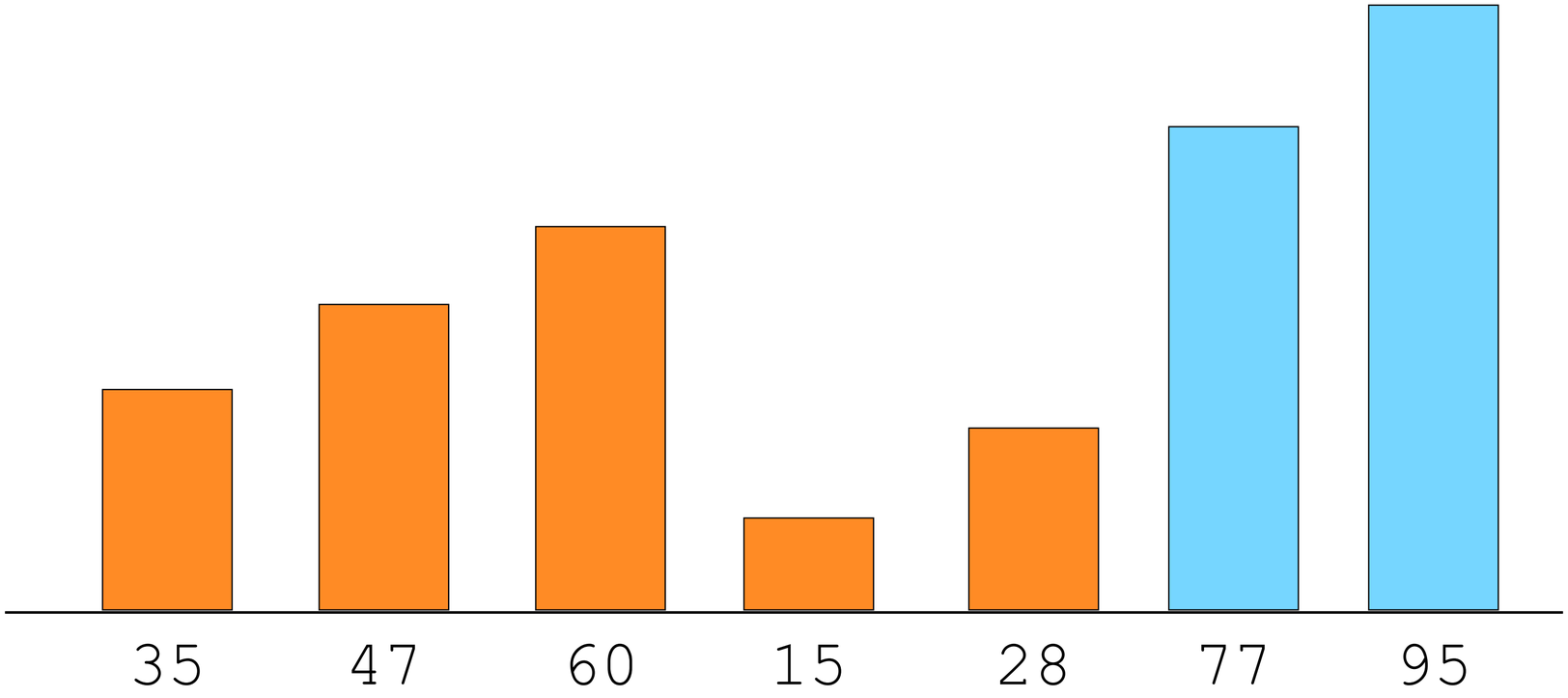
冒泡排序



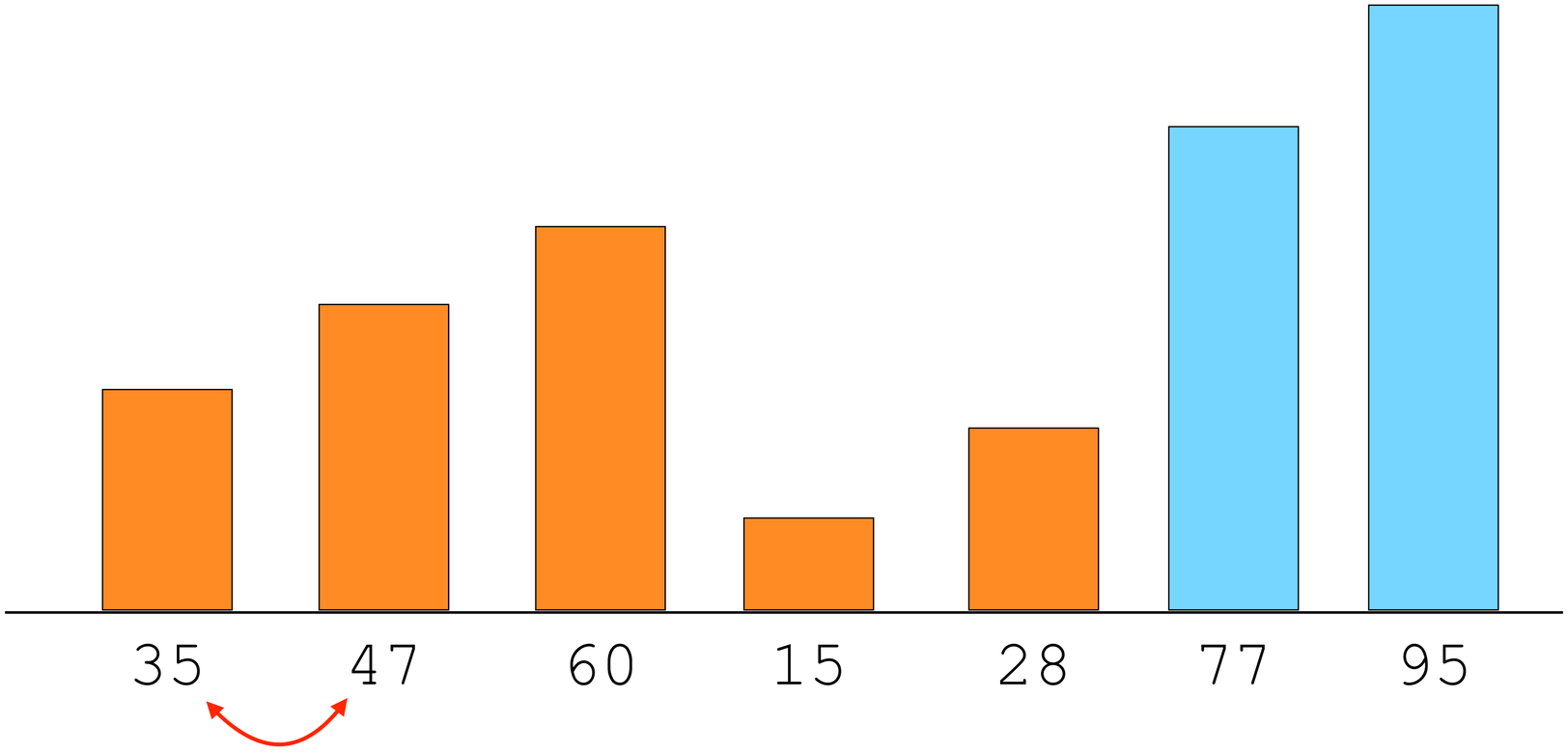
冒泡排序



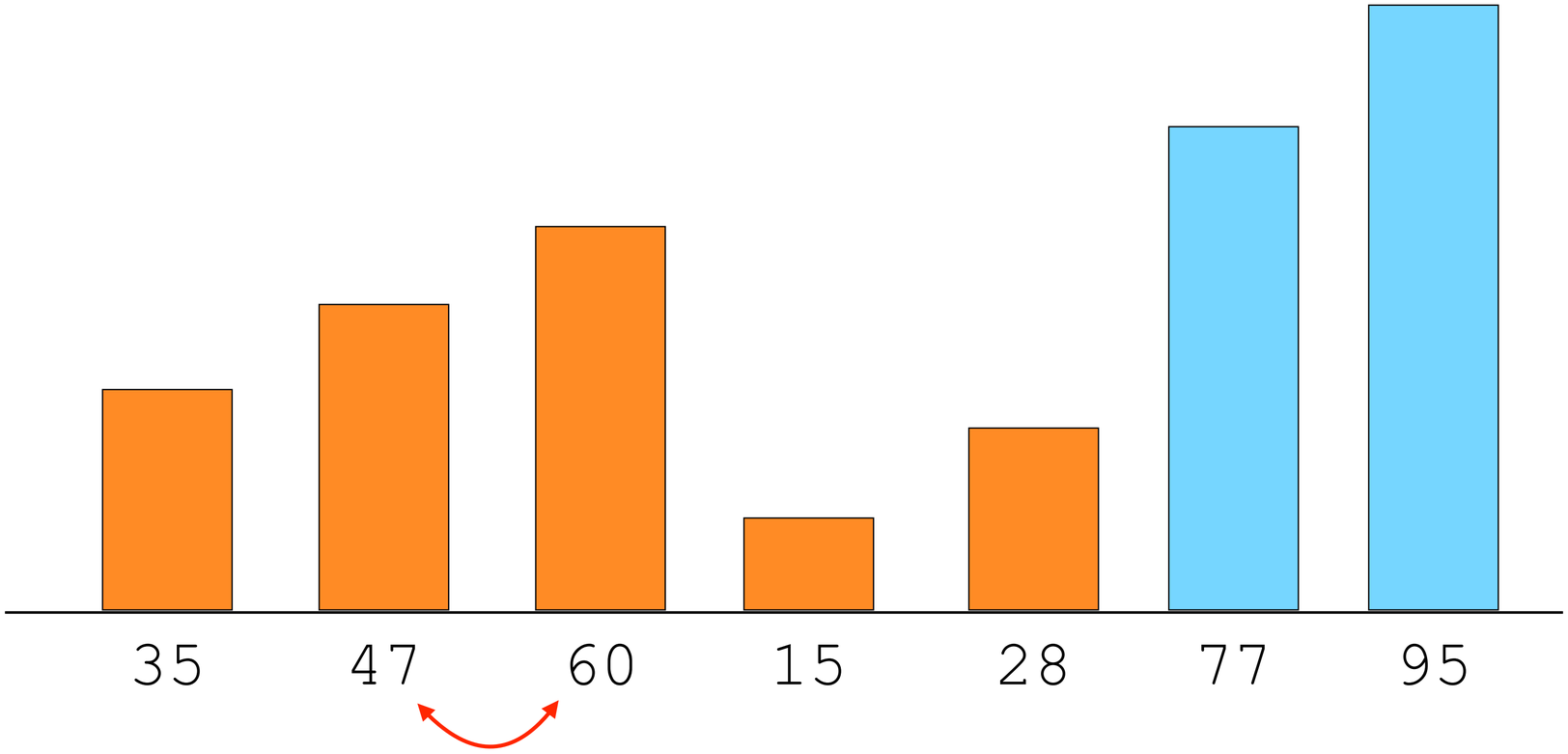
冒泡排序



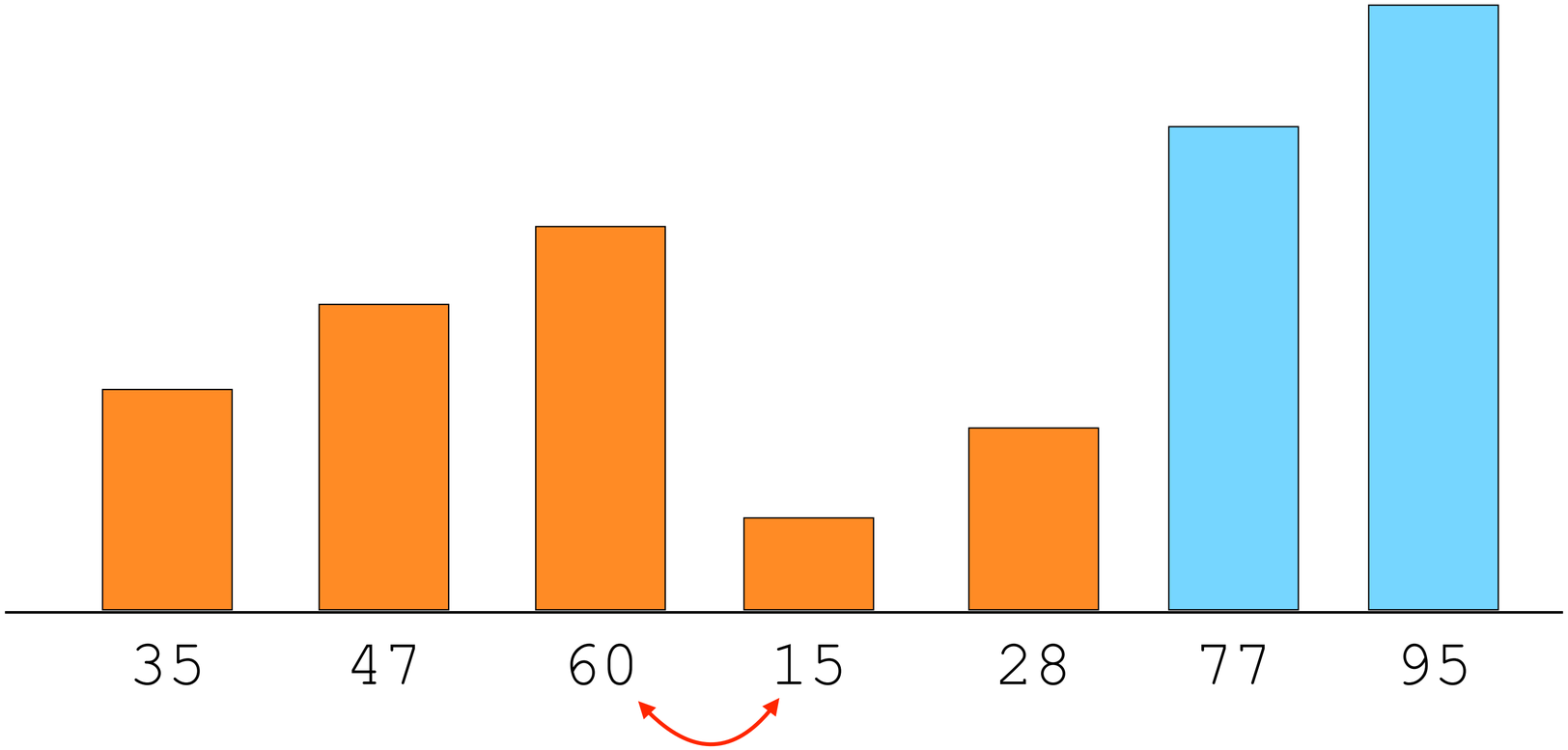
冒泡排序



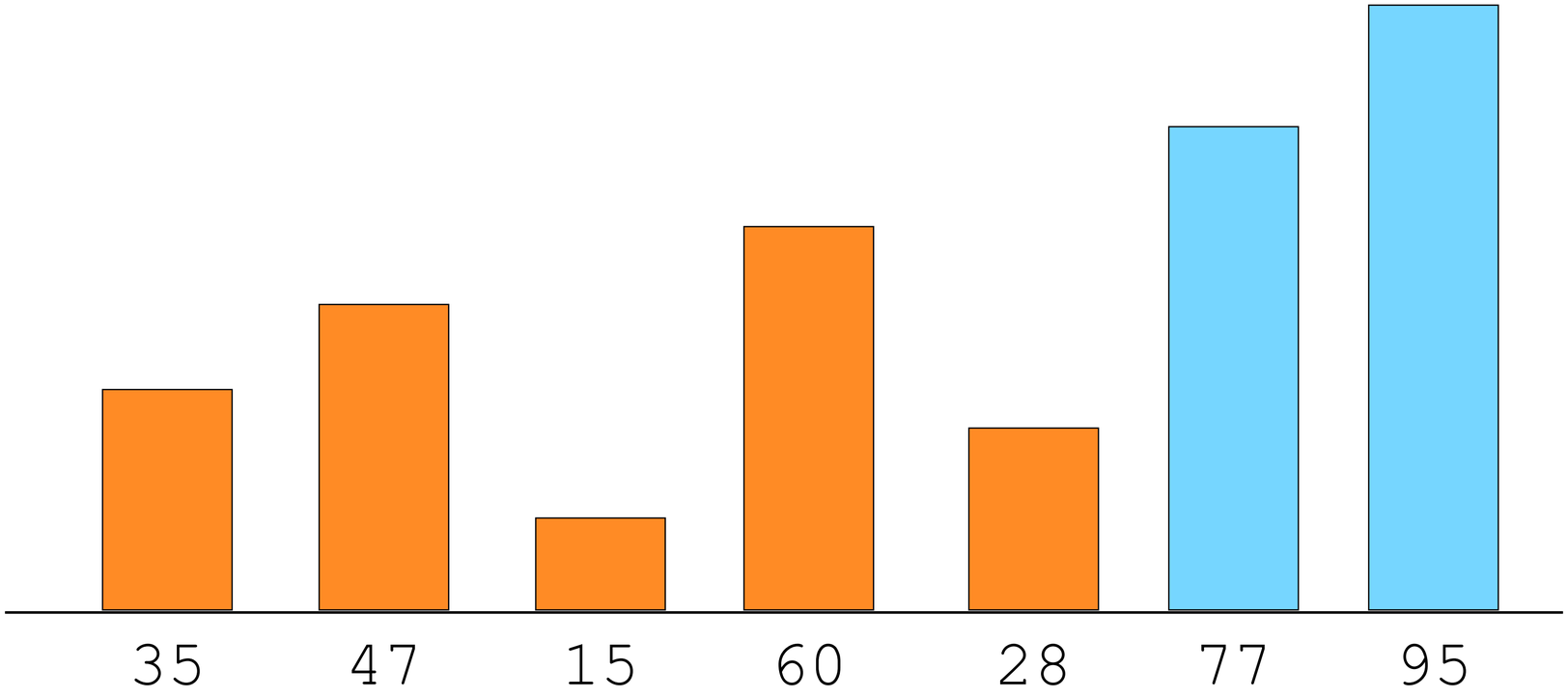
冒泡排序



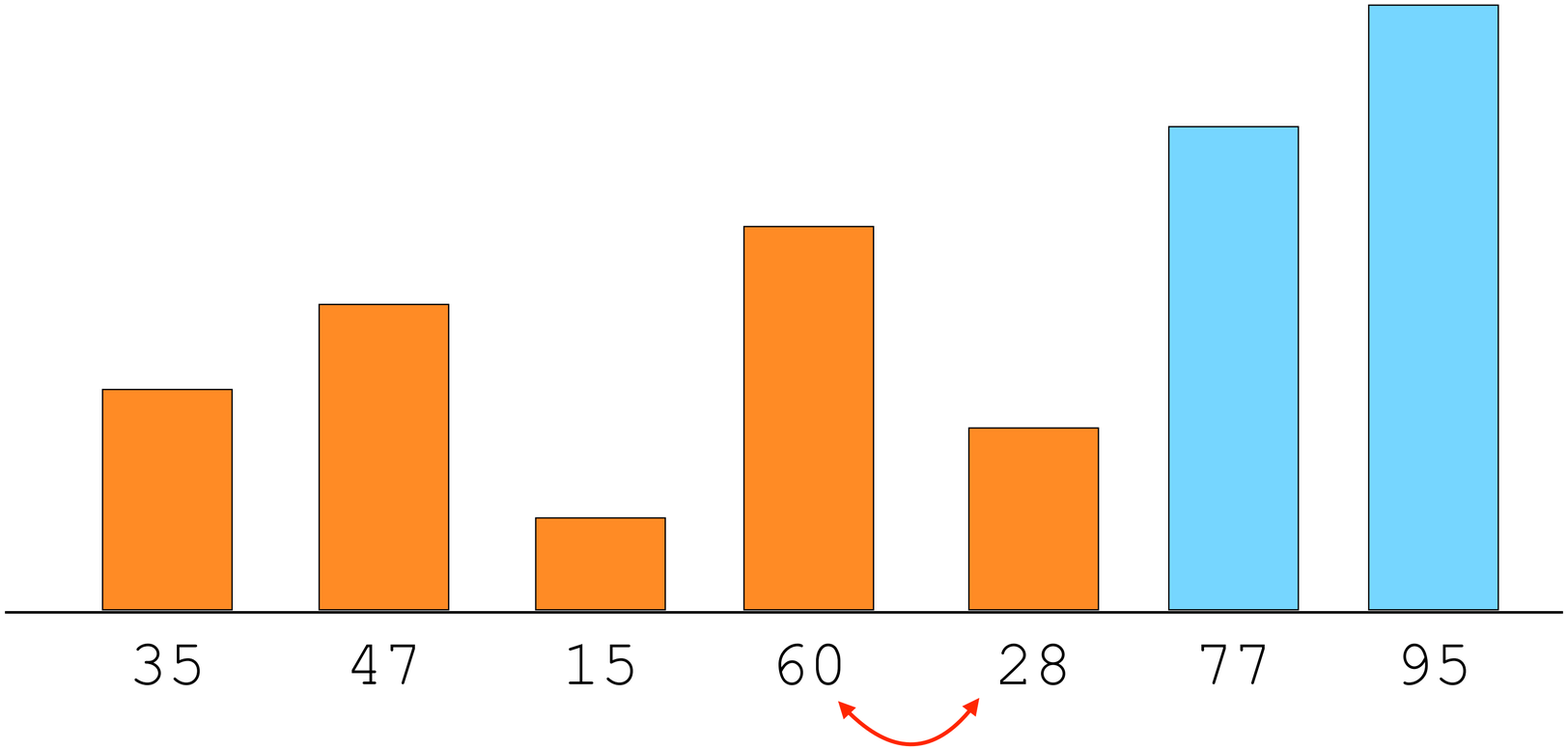
冒泡排序



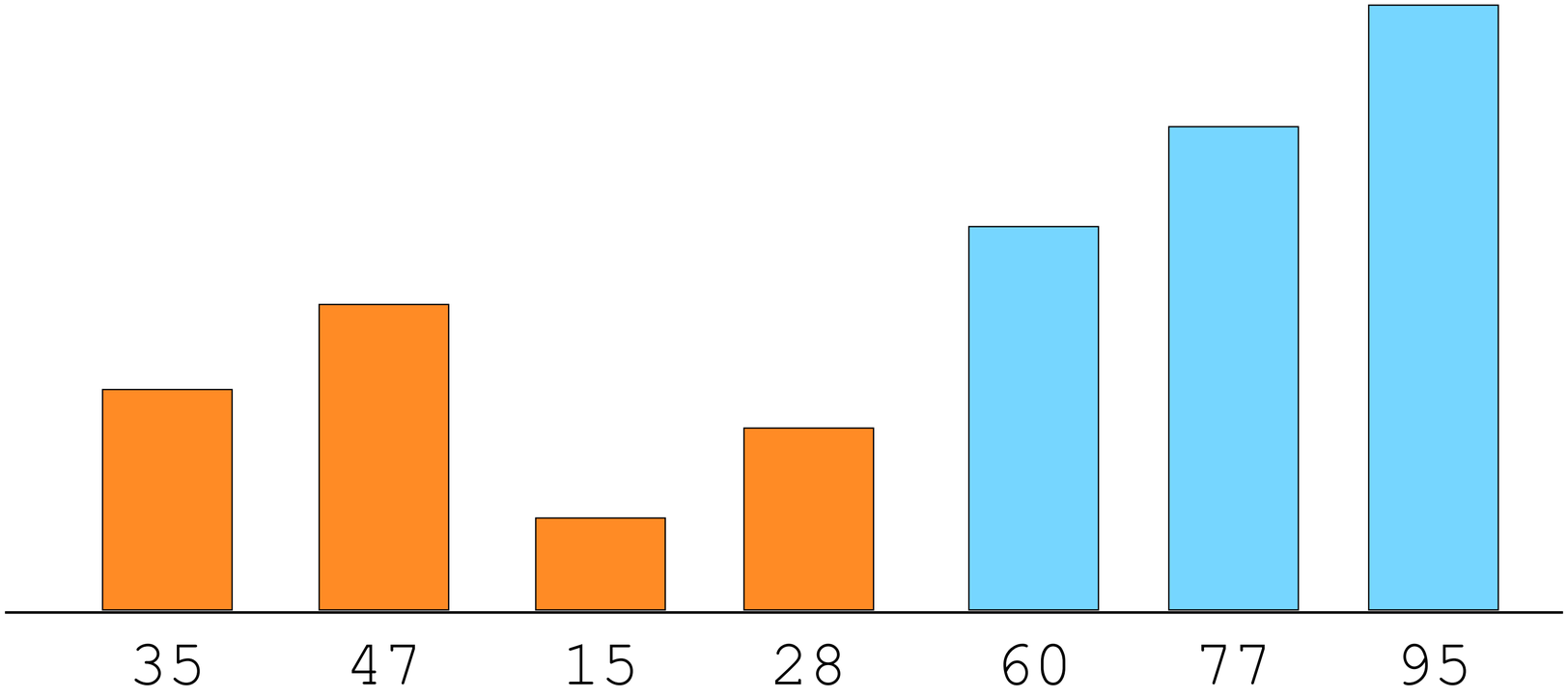
冒泡排序



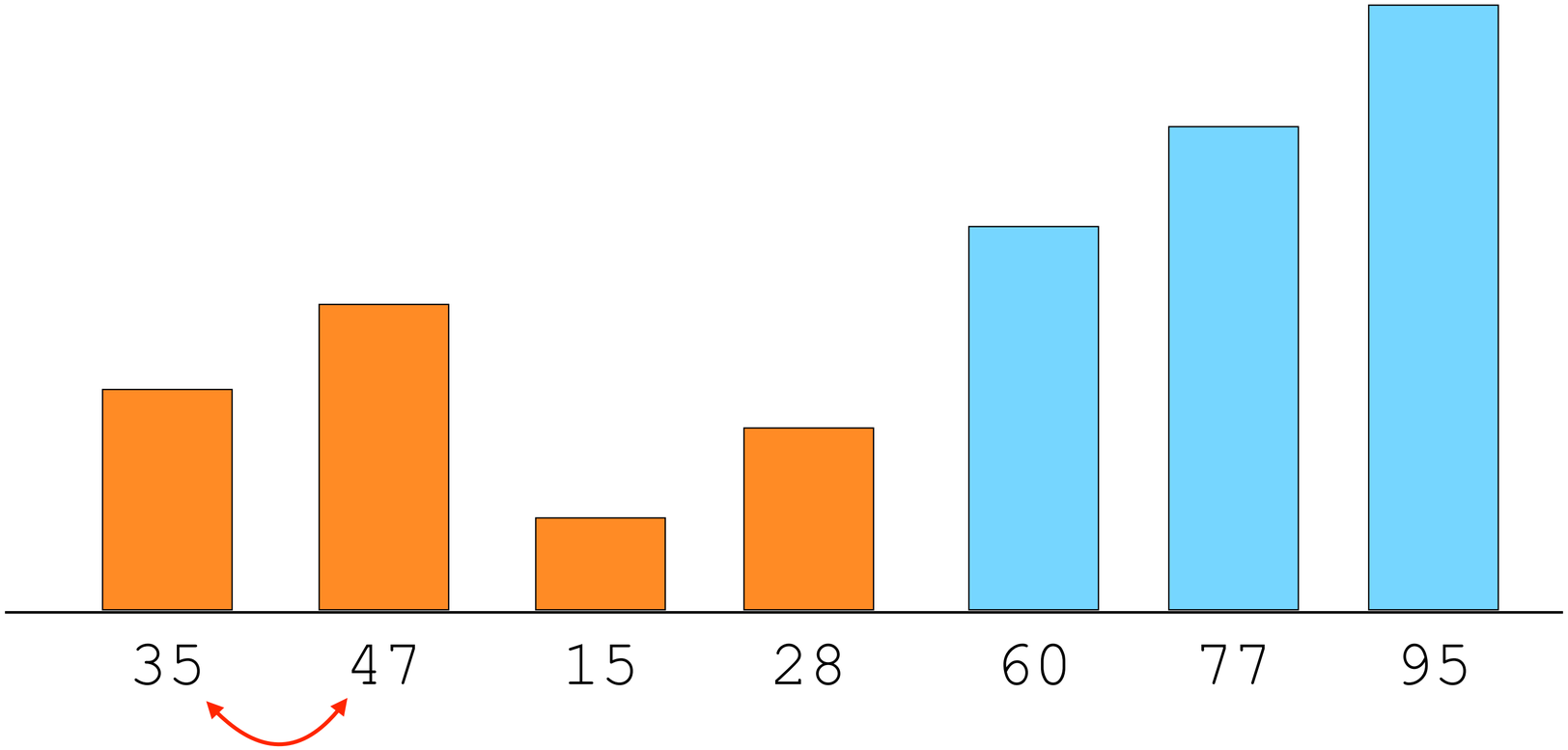
冒泡排序



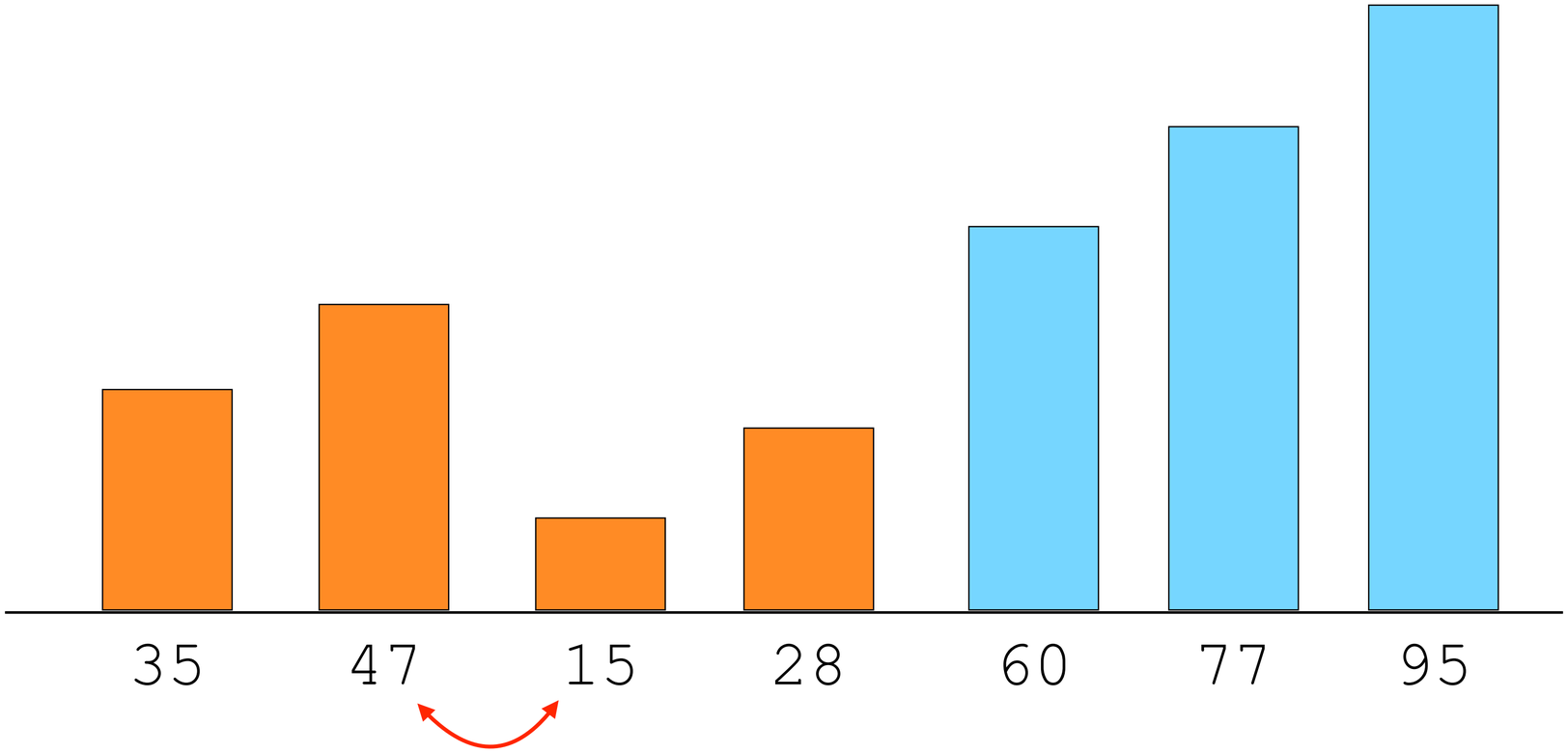
冒泡排序



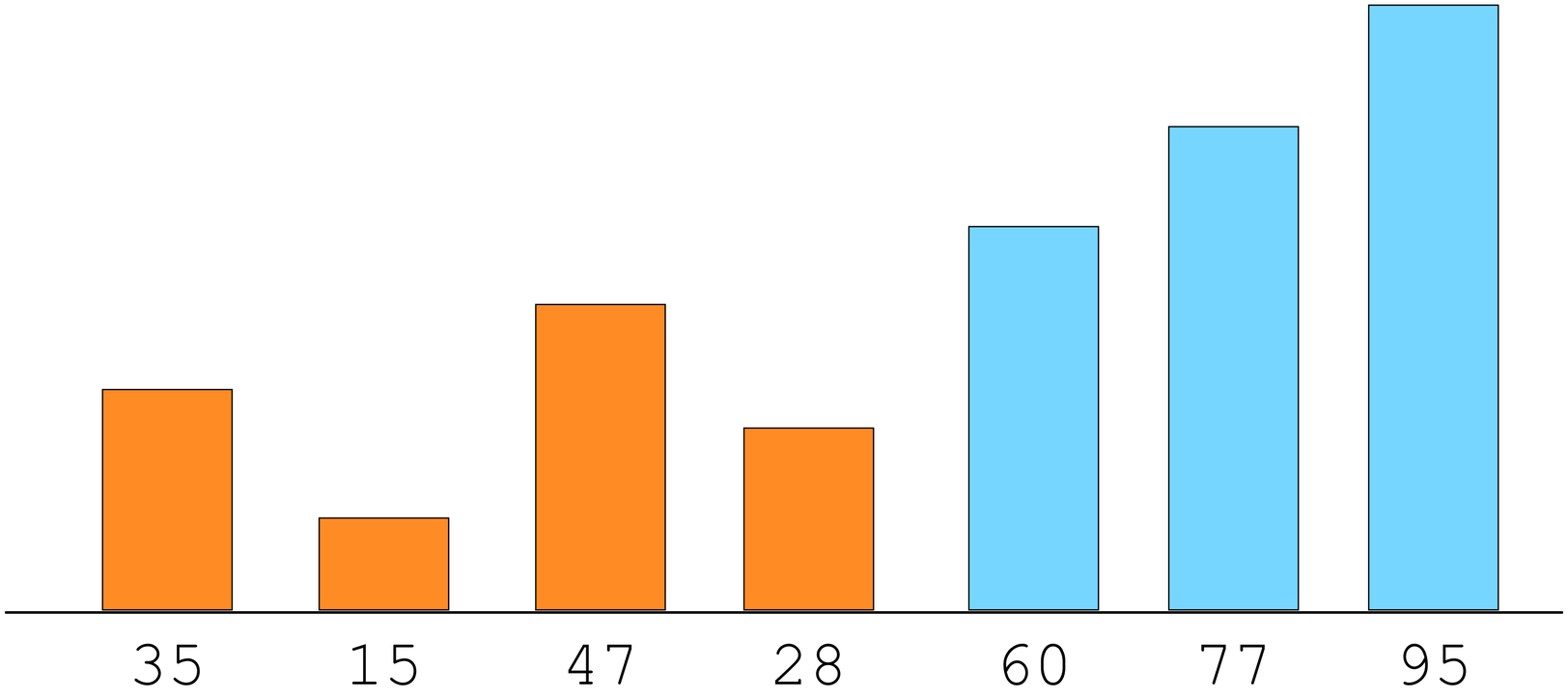
冒泡排序



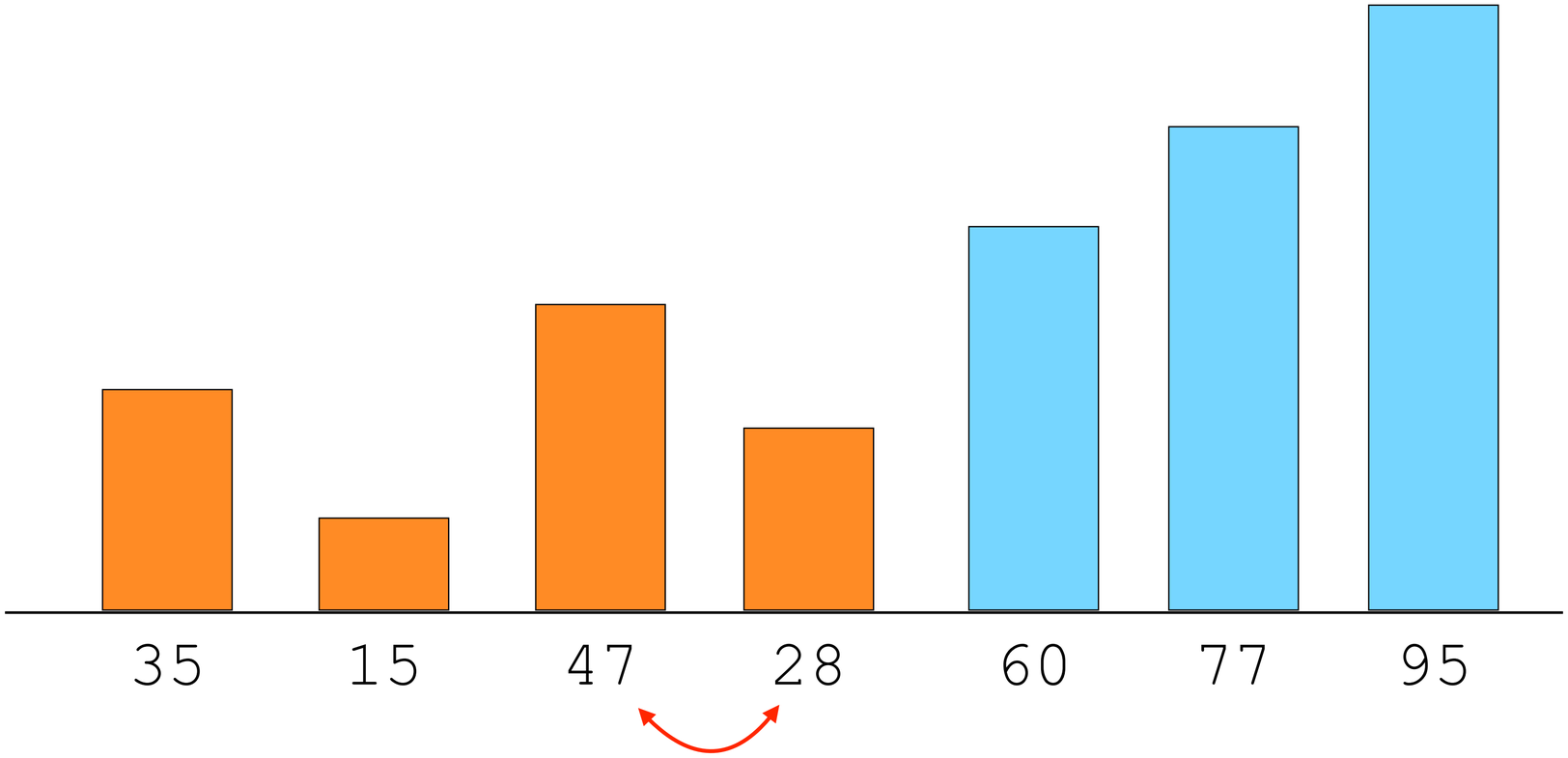
冒泡排序



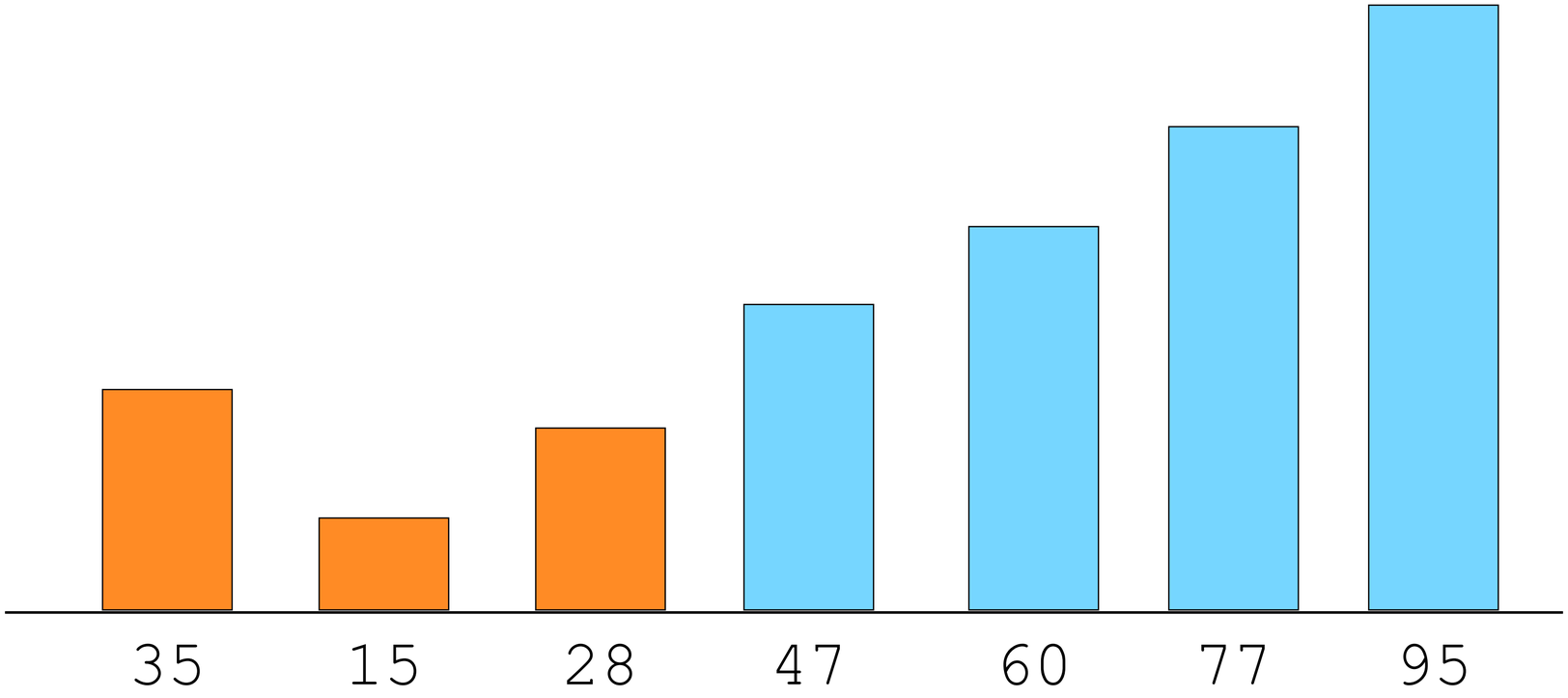
冒泡排序



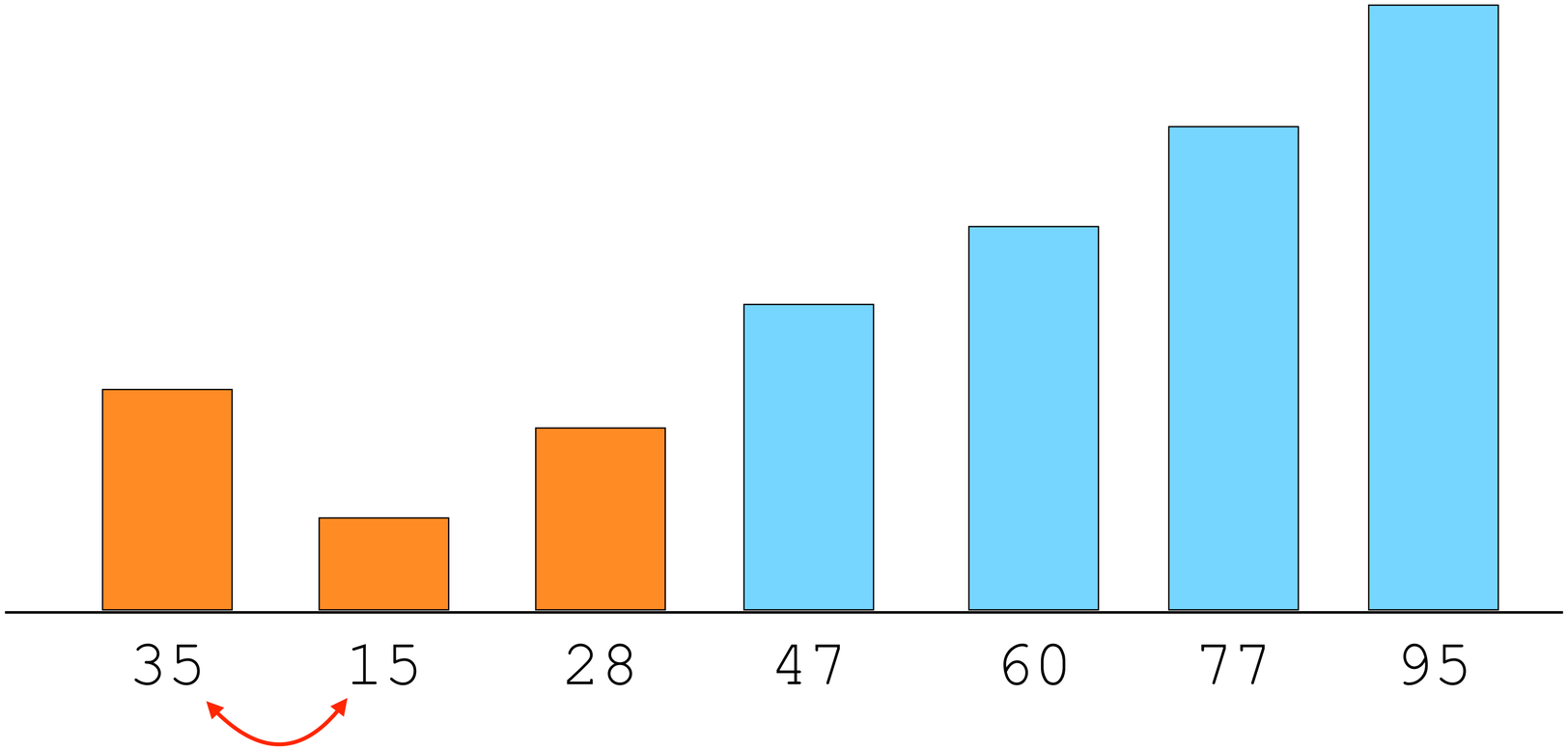
冒泡排序



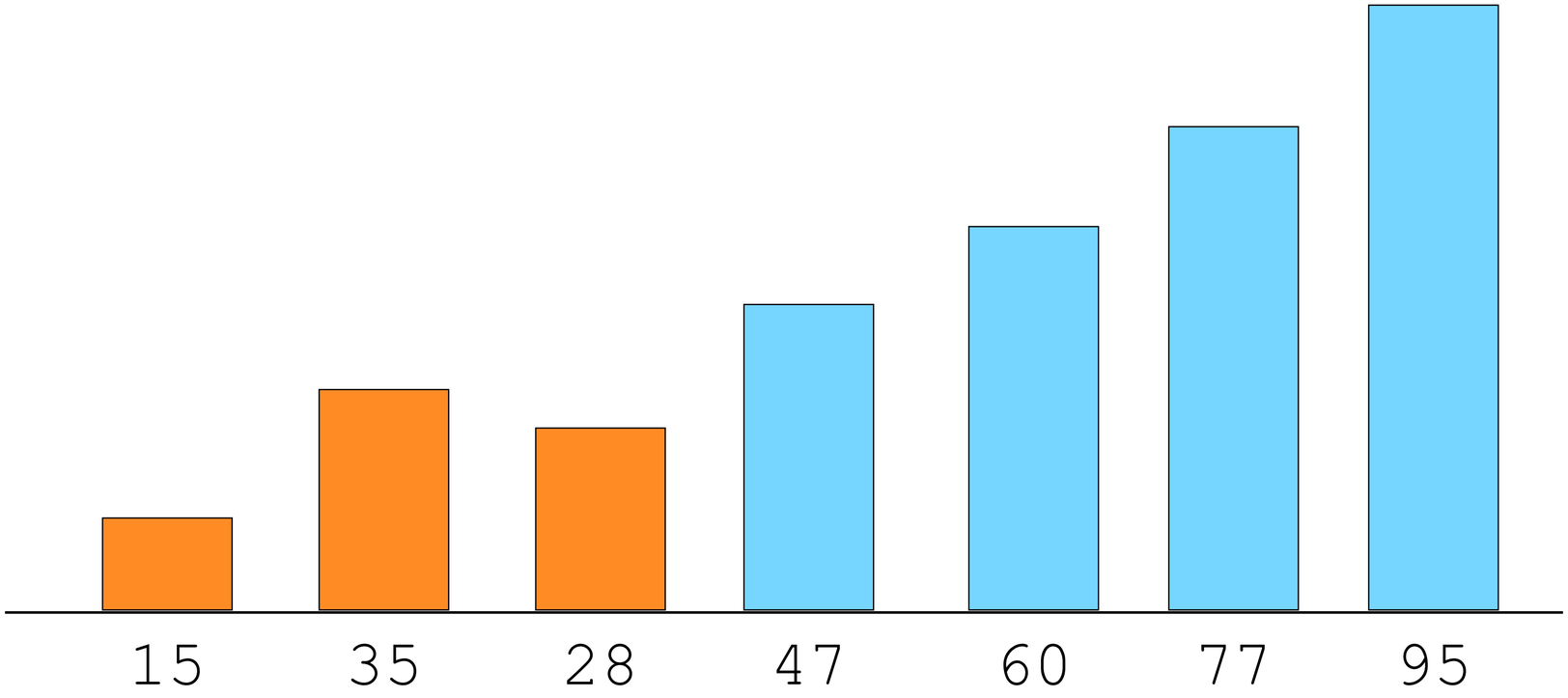
冒泡排序



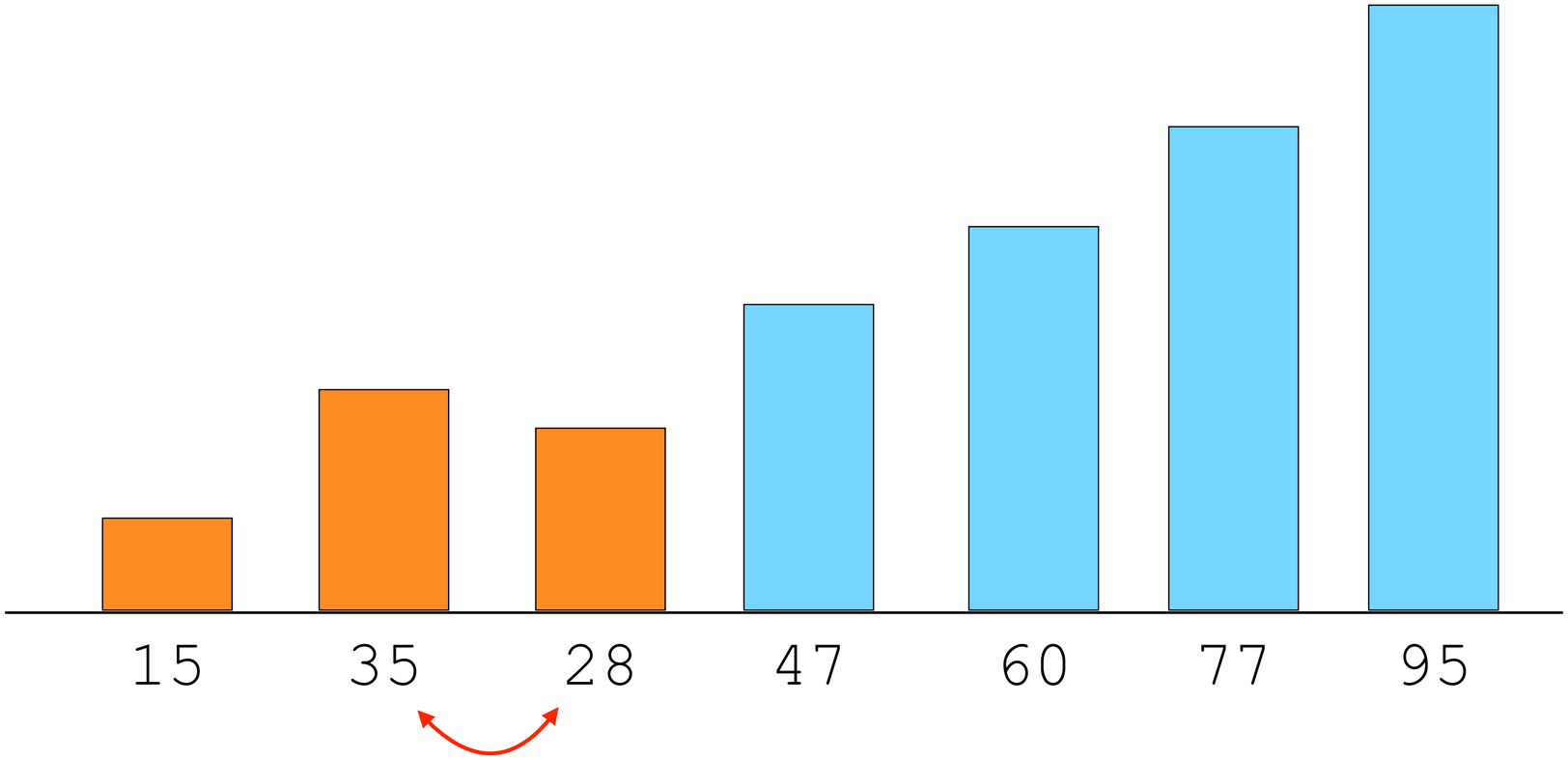
冒泡排序



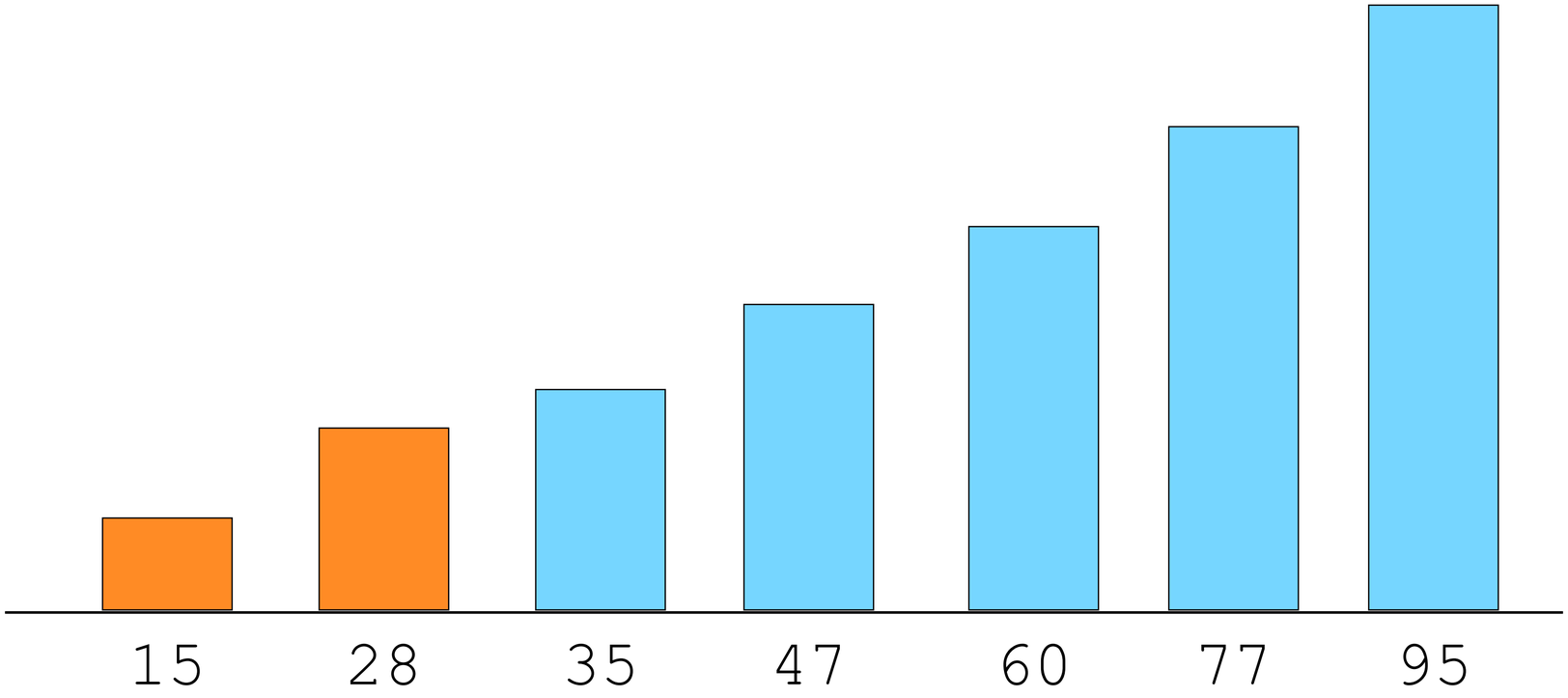
冒泡排序



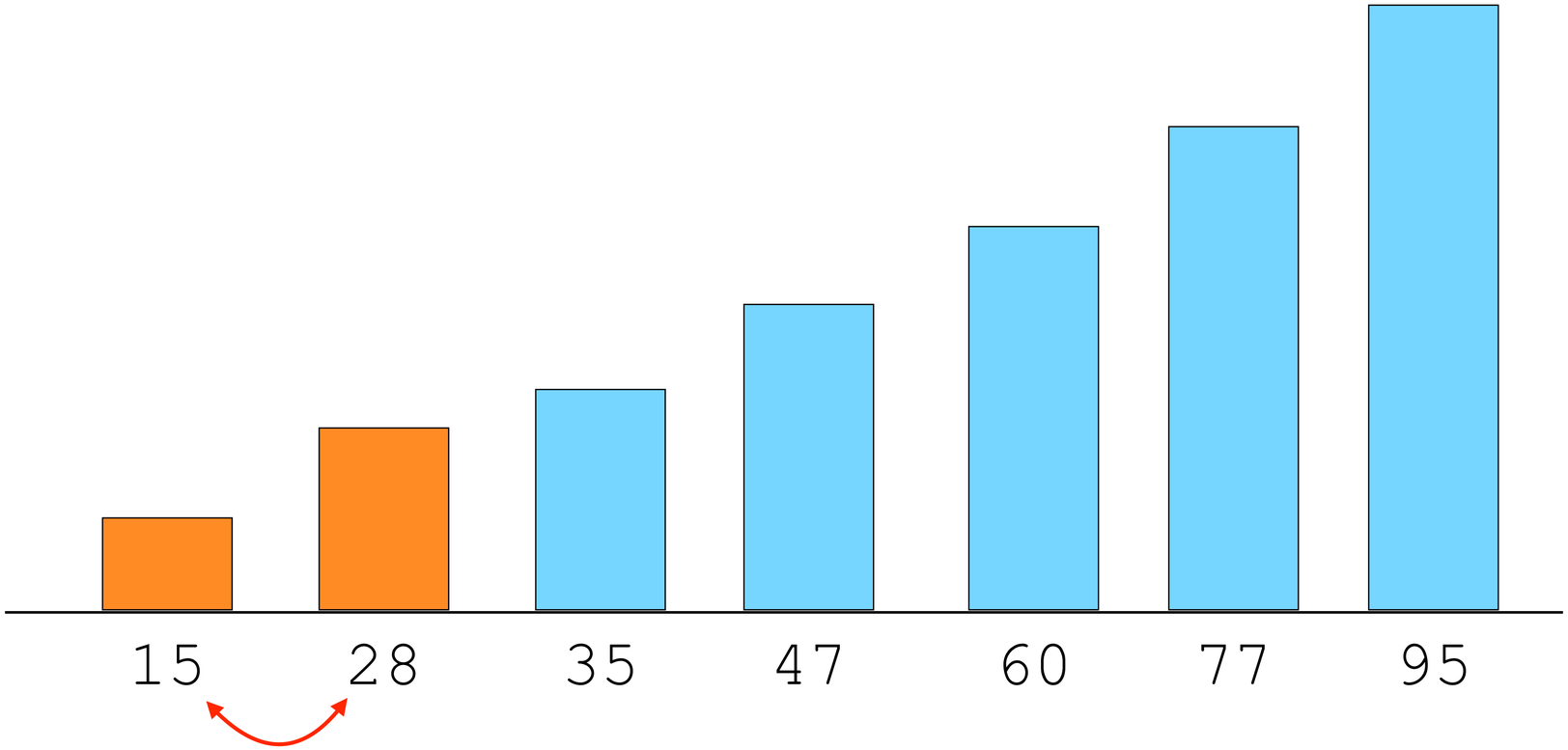
冒泡排序



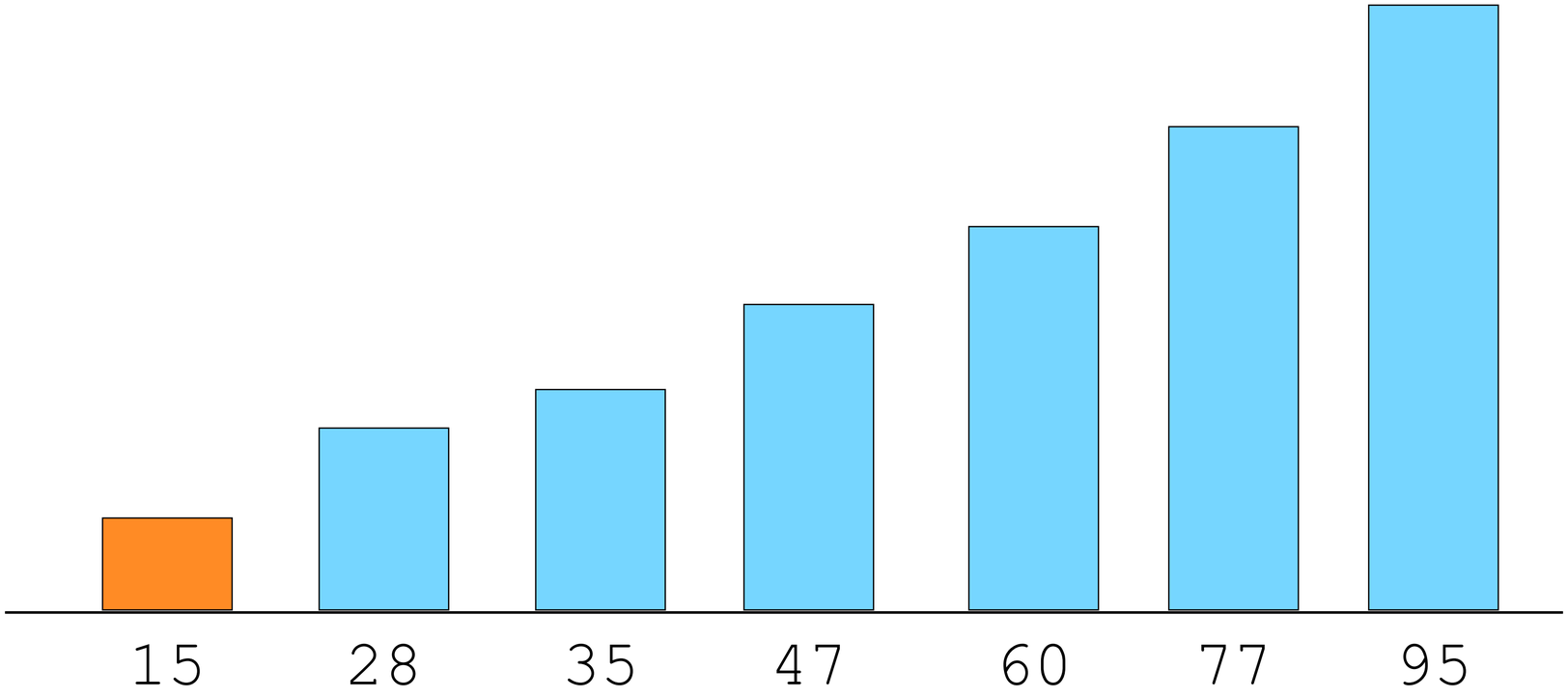
冒泡排序



冒泡排序

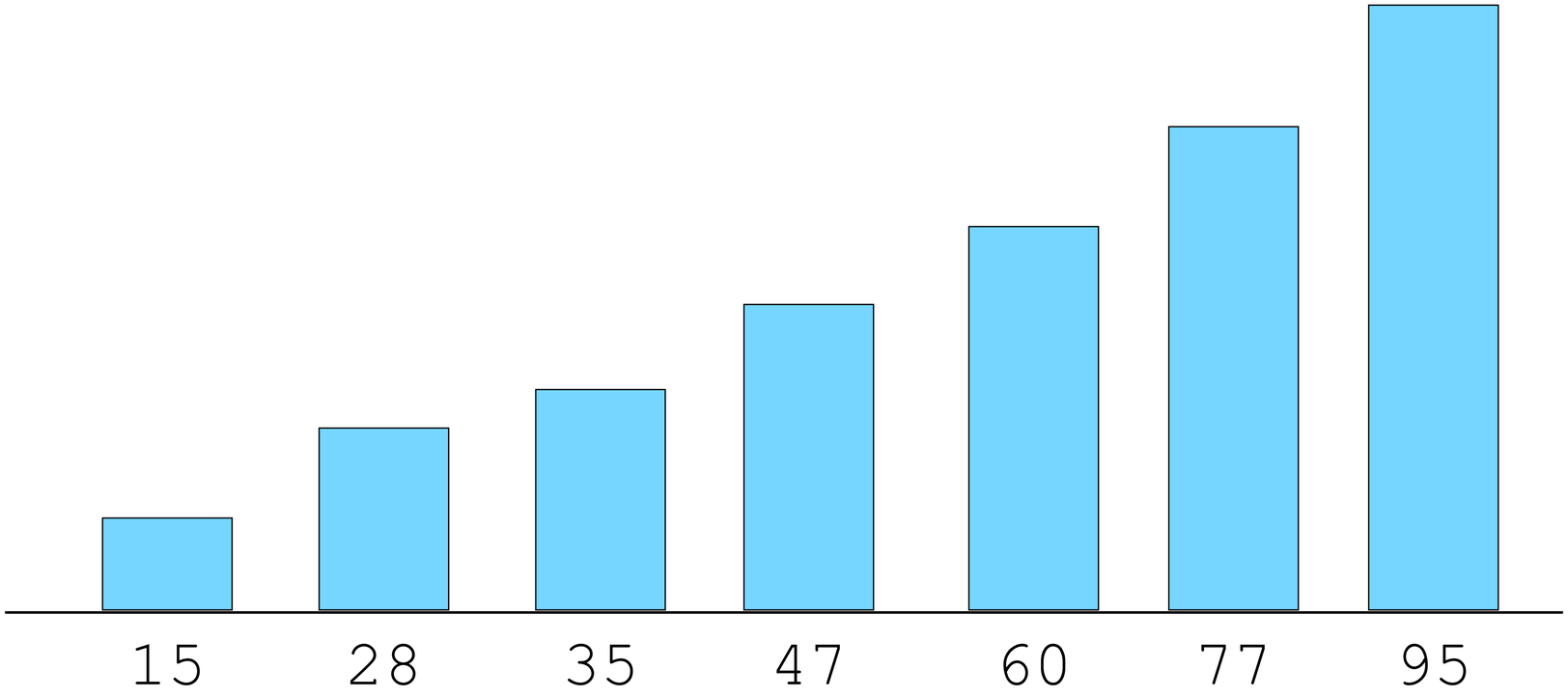


冒泡排序



冒泡排序

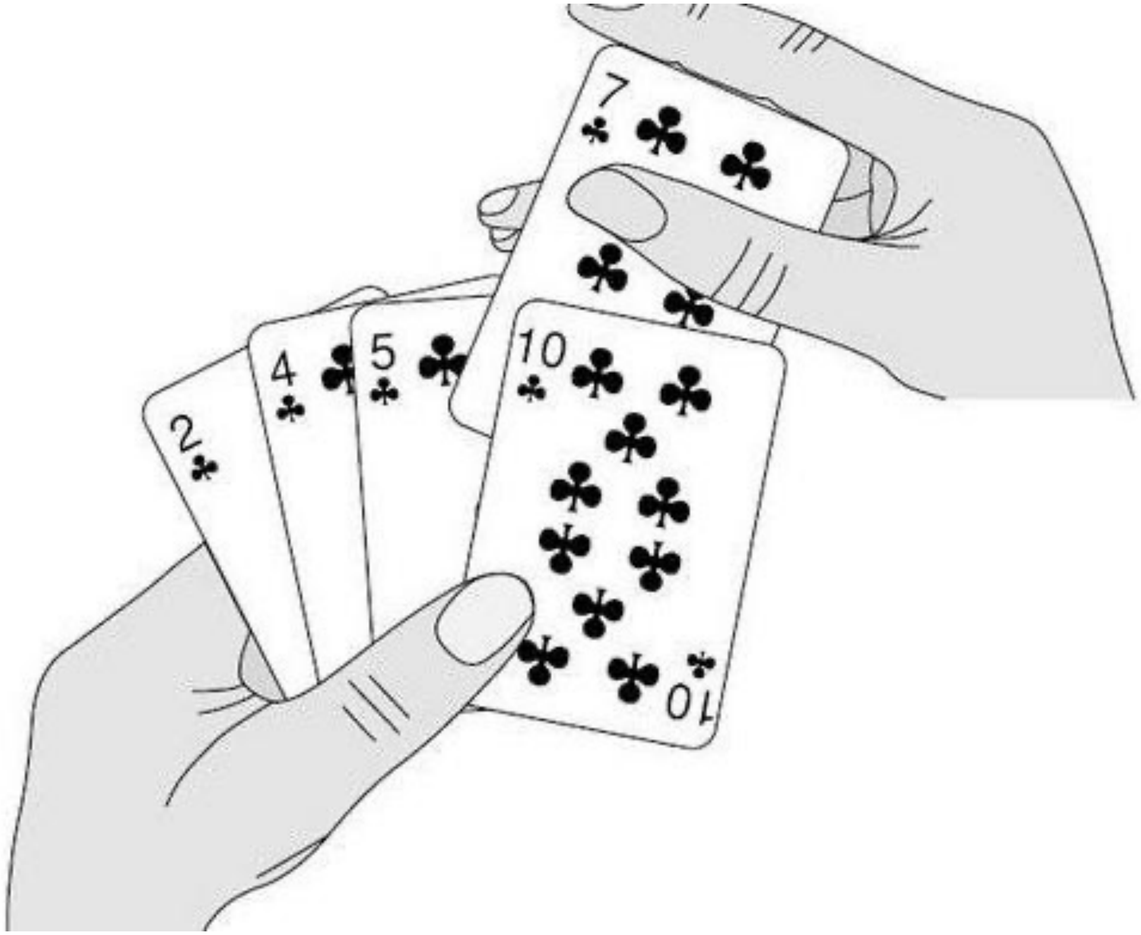
排序 / test1.c



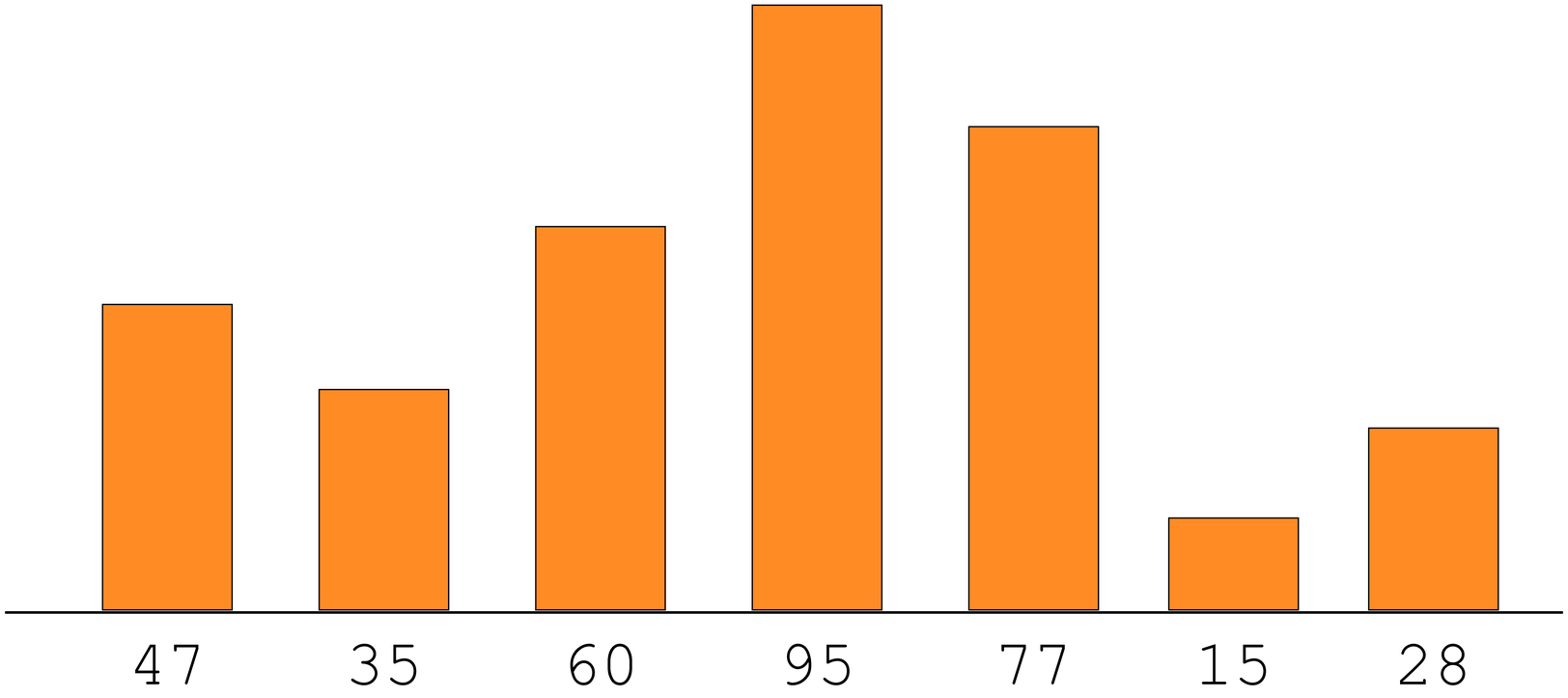
冒泡排序

维度	描述
时间复杂度	最坏：$O(n^2)$，最好：$O(n)$，平均：$O(n^2)$
空间复杂度	$O(1)$
算法稳定性	稳定
适用场景	数据量较小、对稳定性有要求的场景

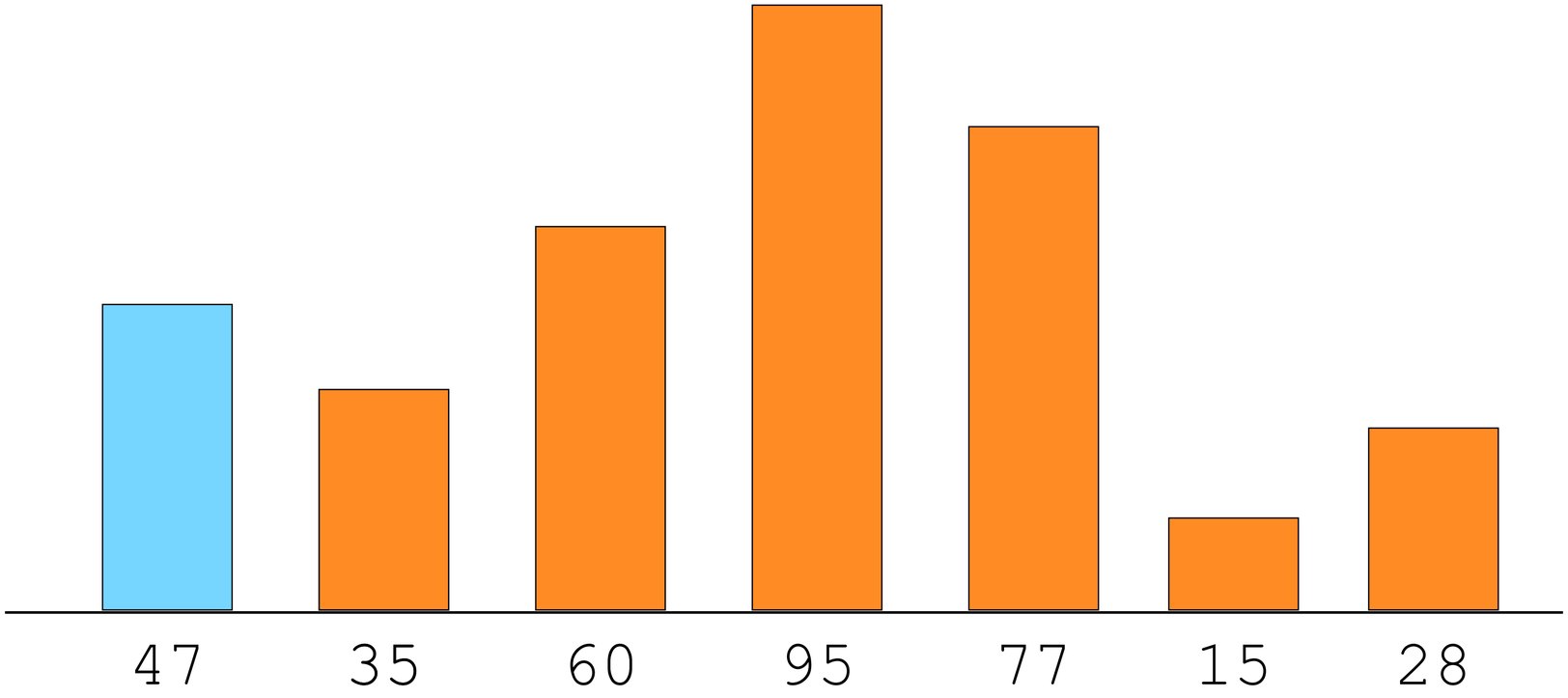
插入排序



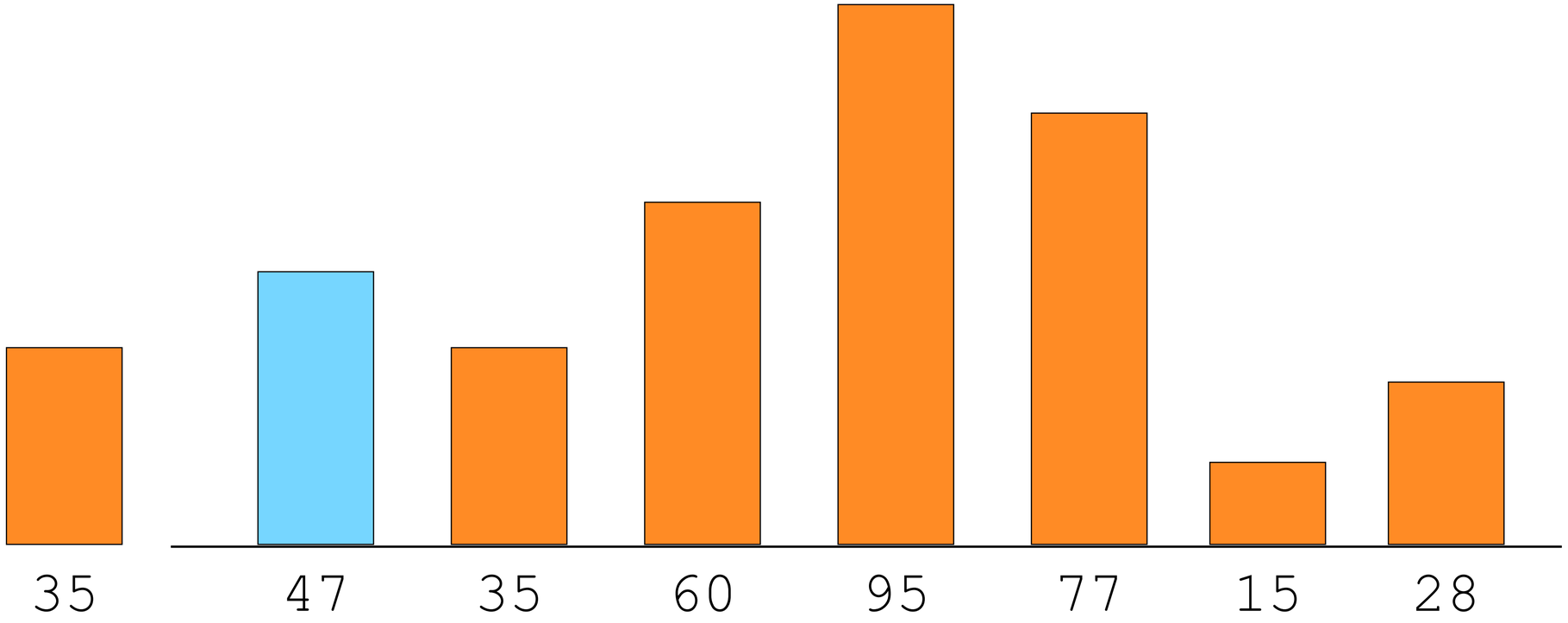
插入排序



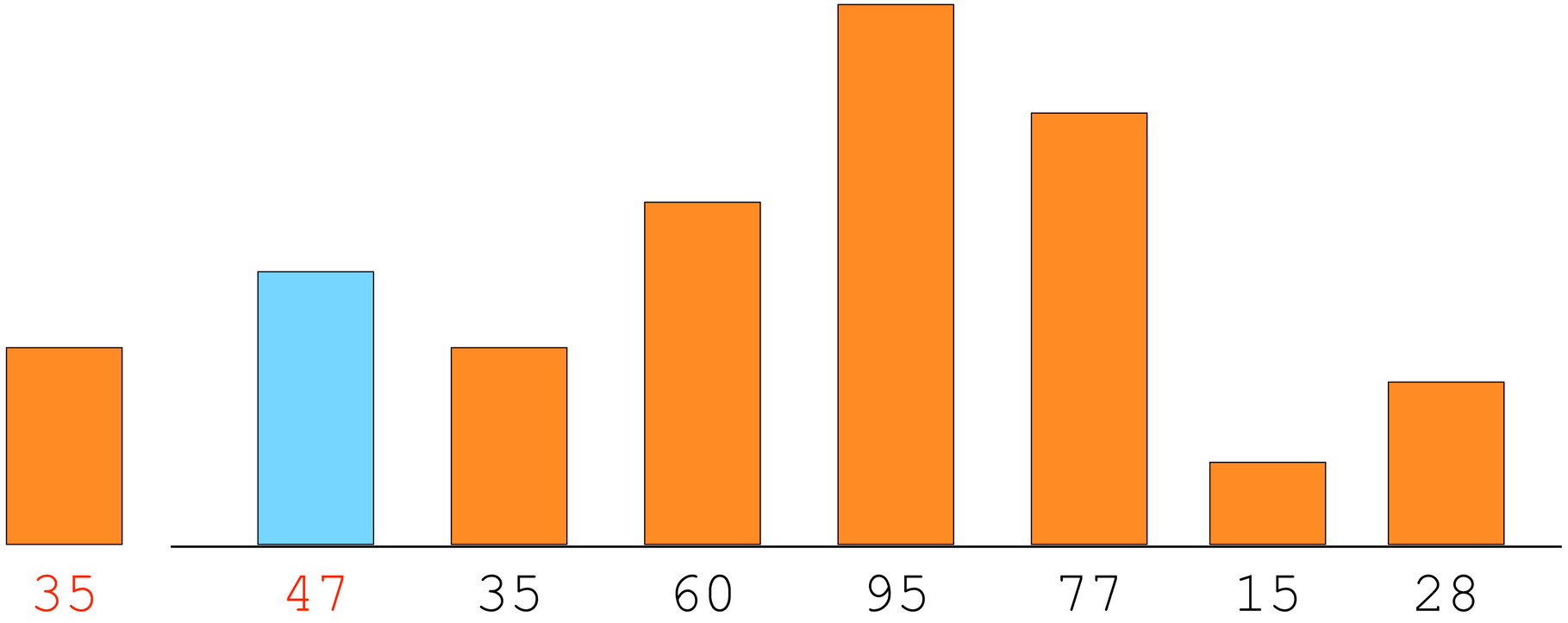
插入排序



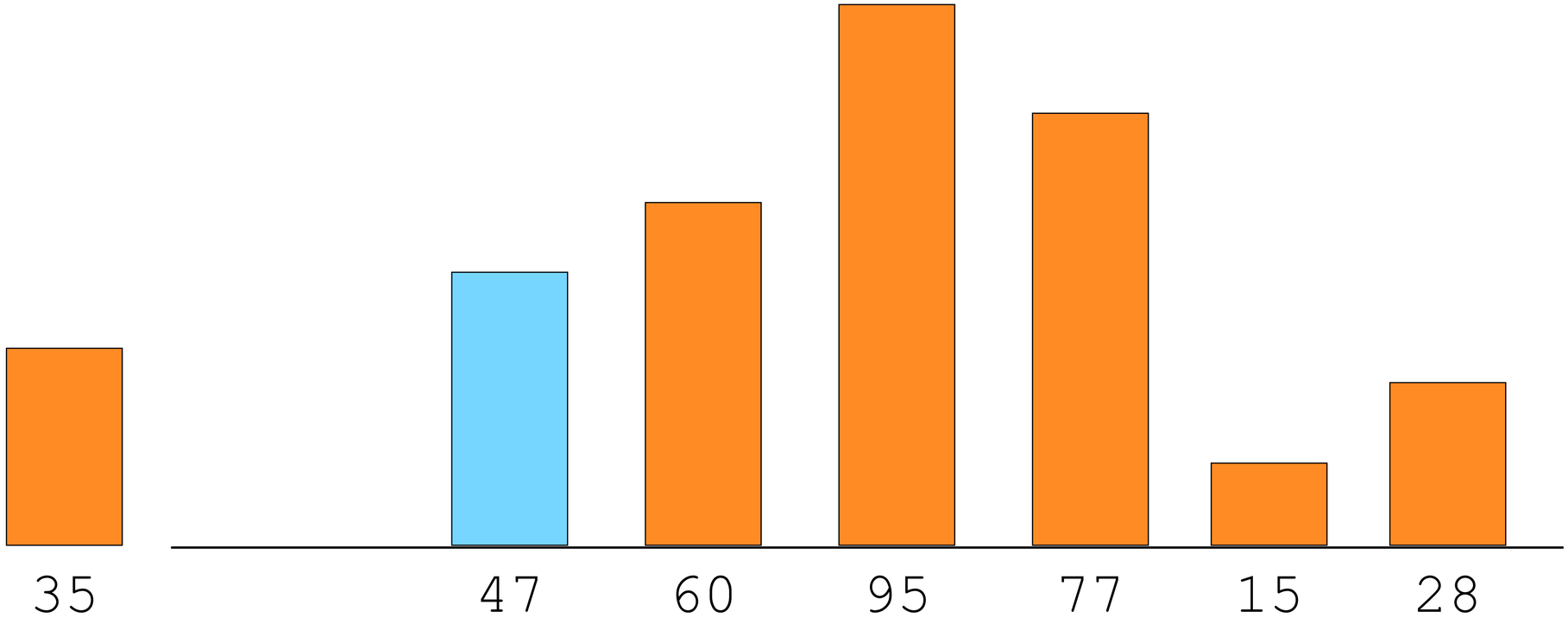
插入排序



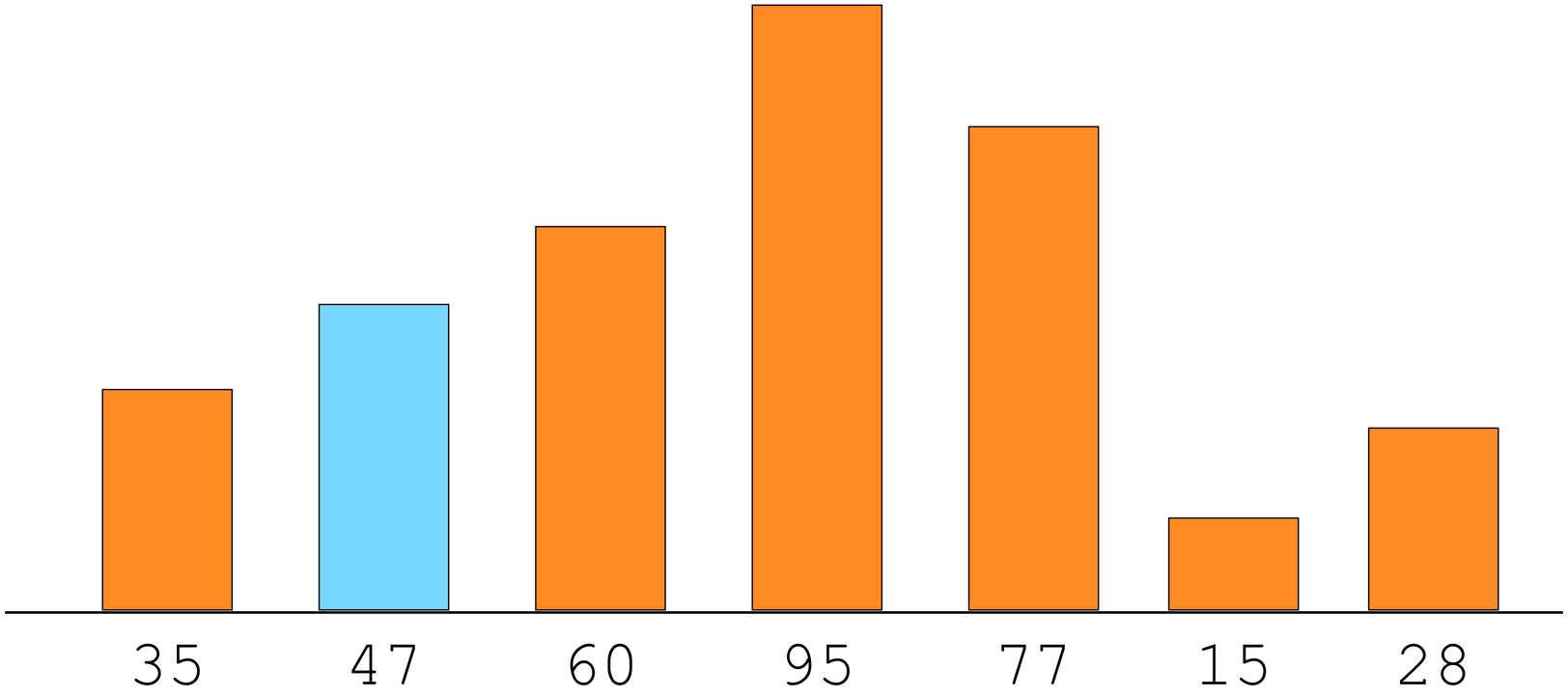
插入排序



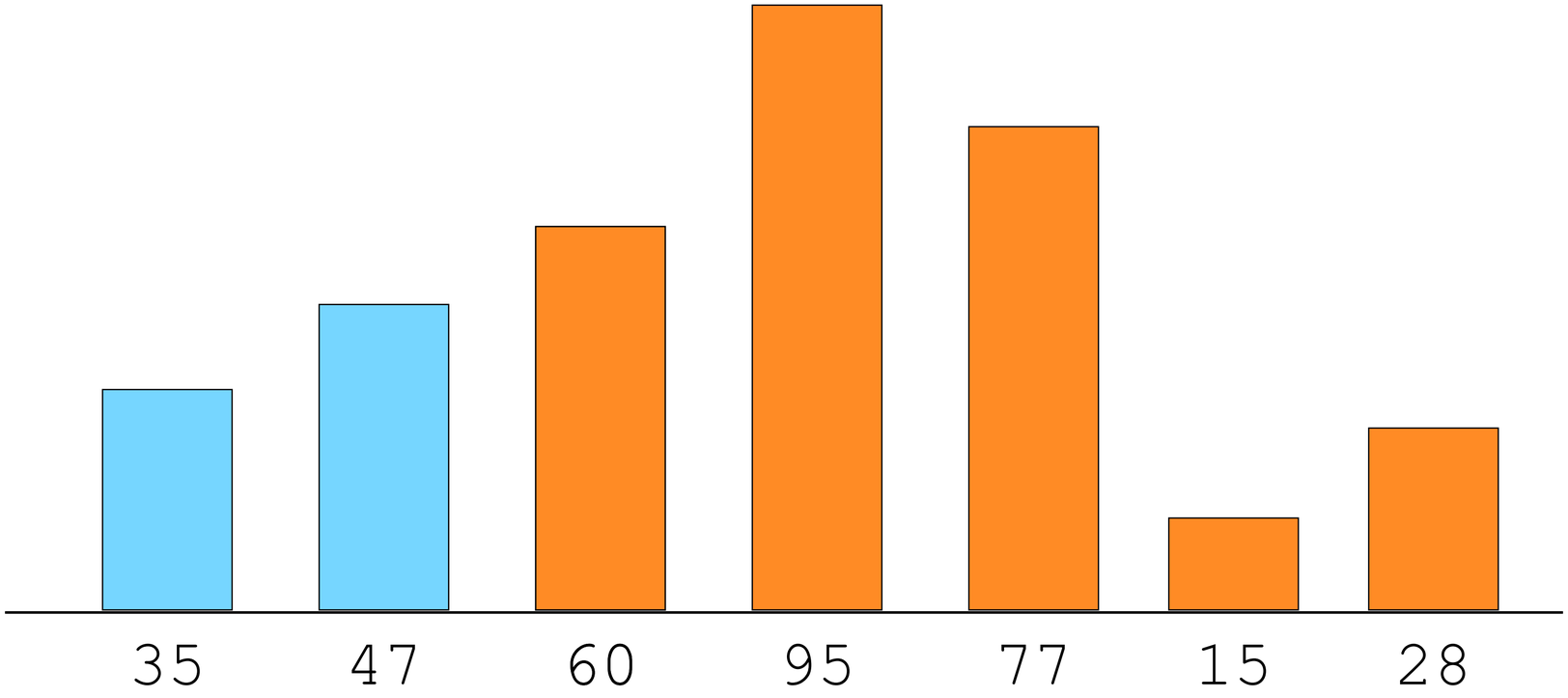
插入排序



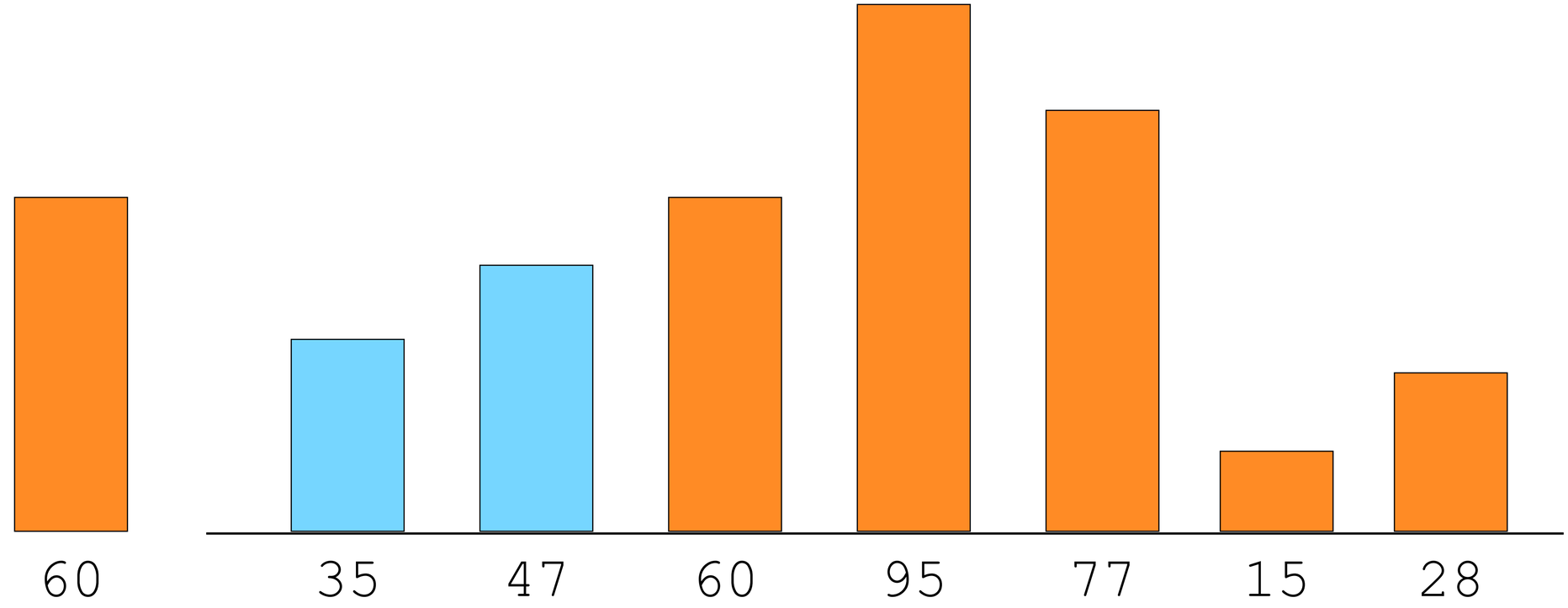
插入排序



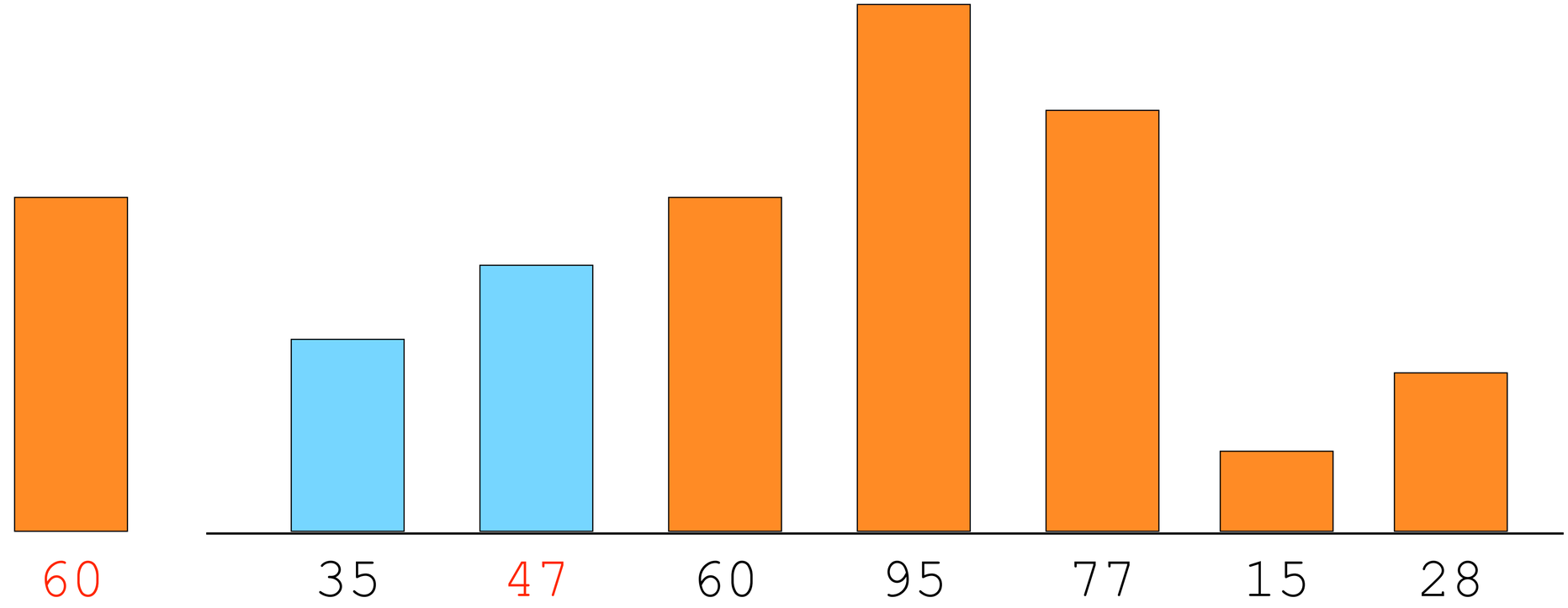
插入排序



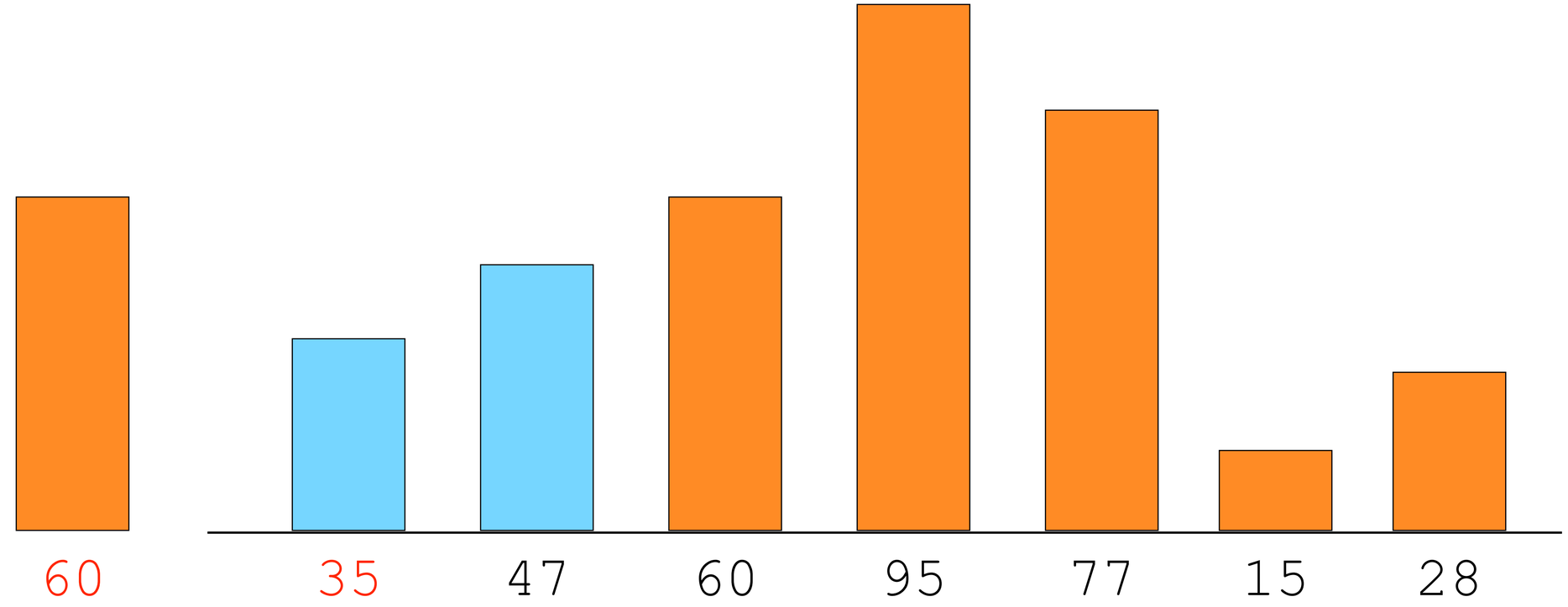
插入排序



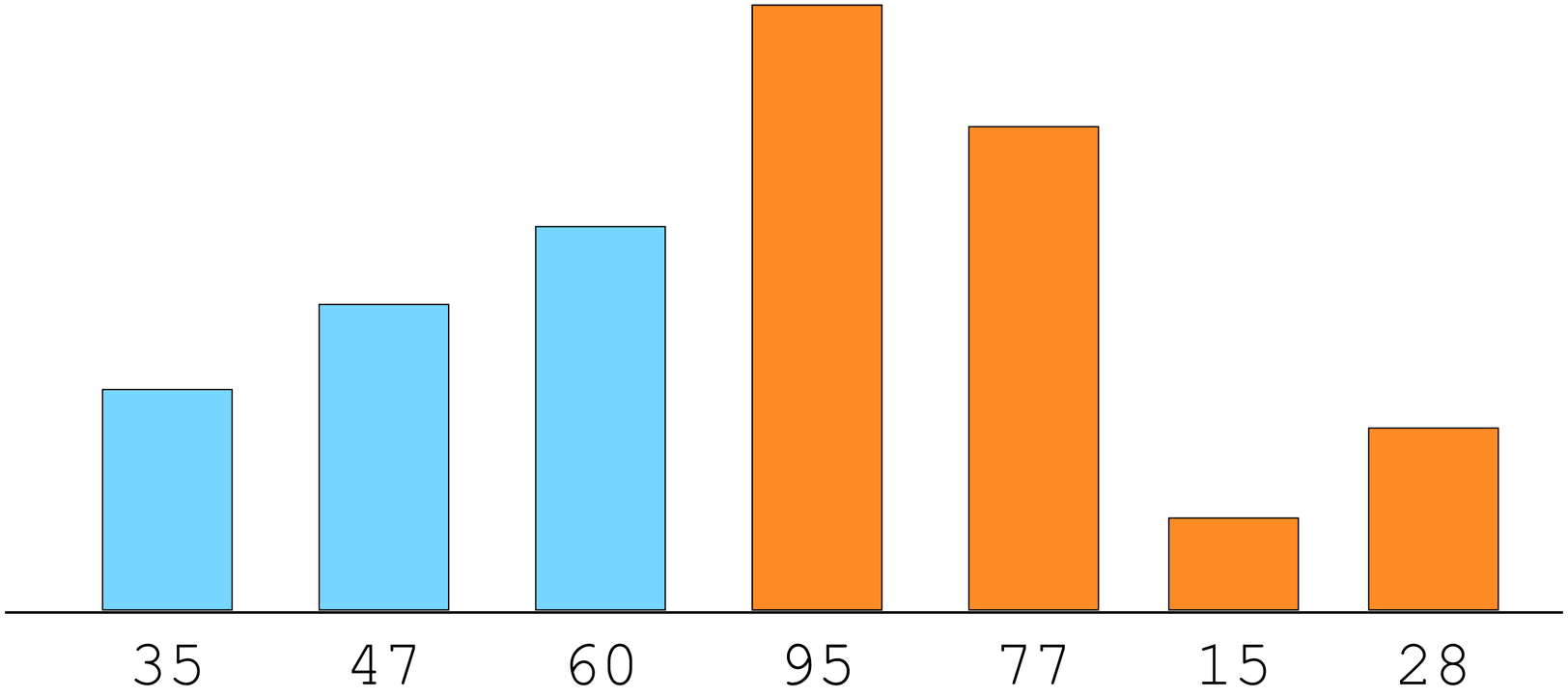
插入排序



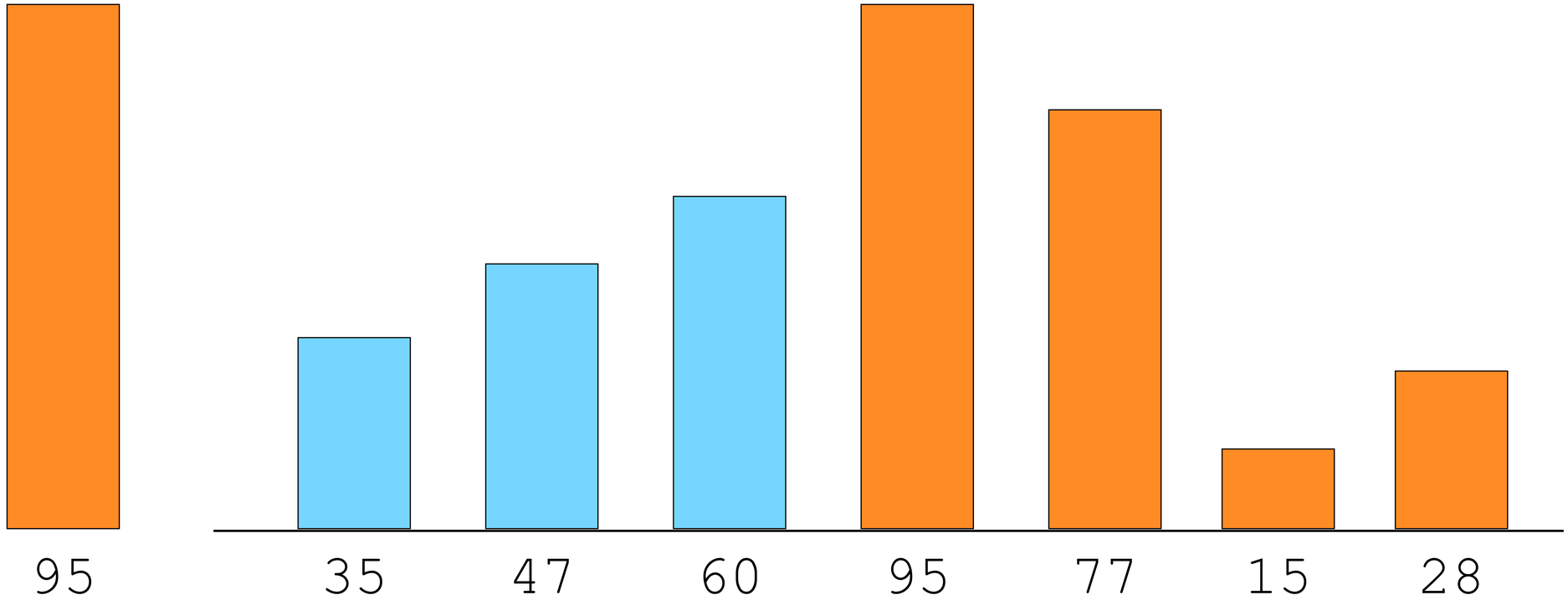
插入排序



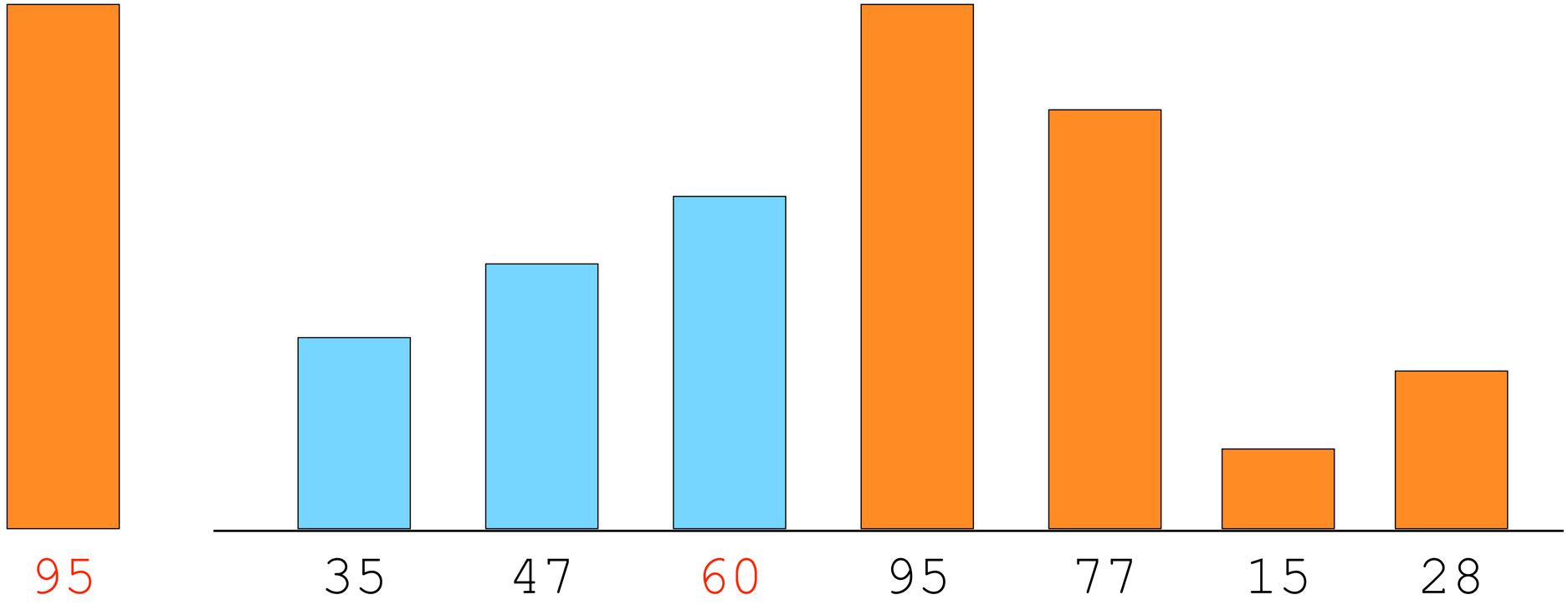
插入排序



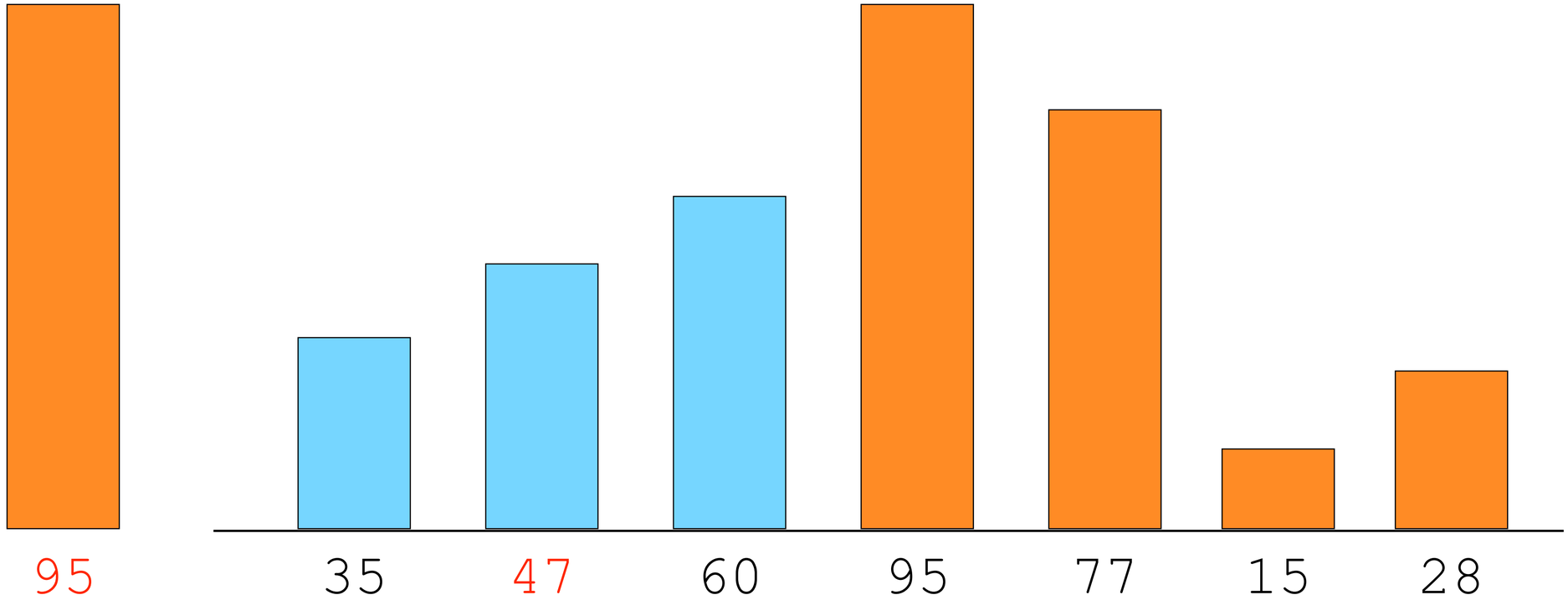
插入排序



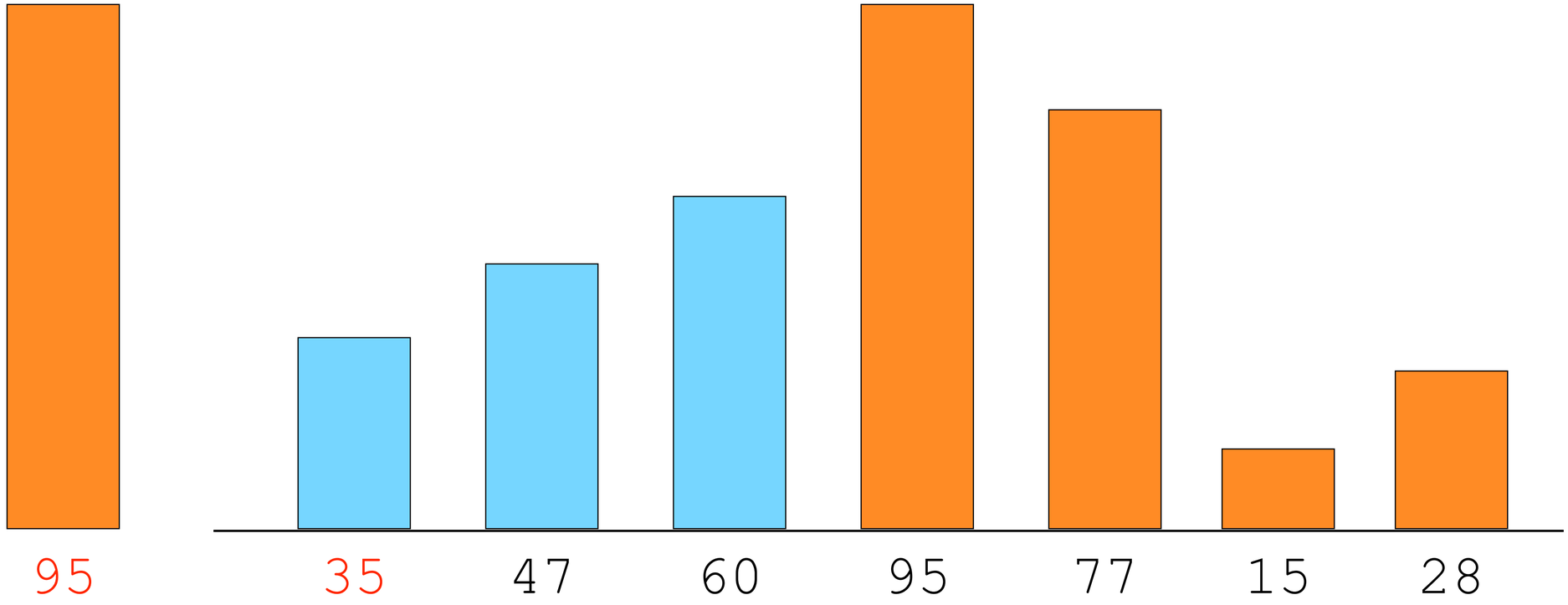
插入排序



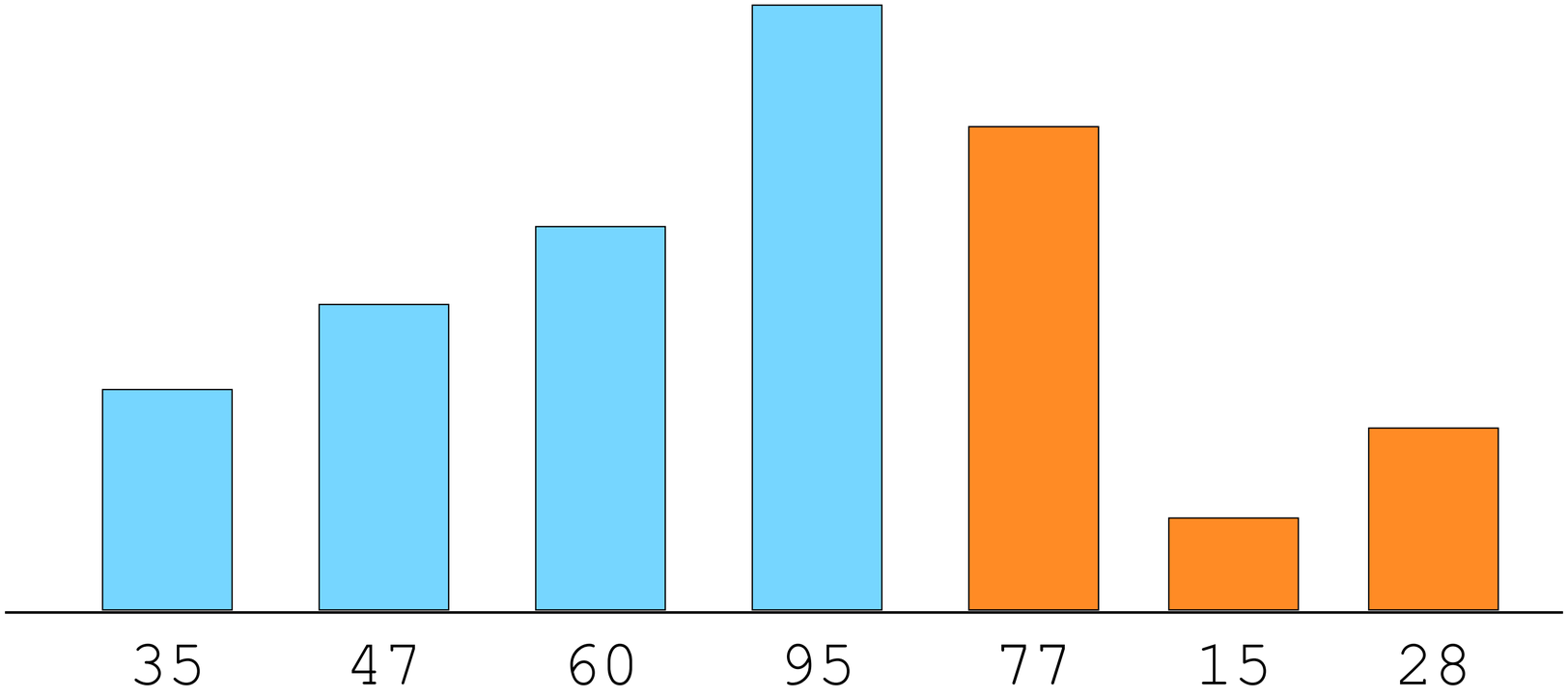
插入排序



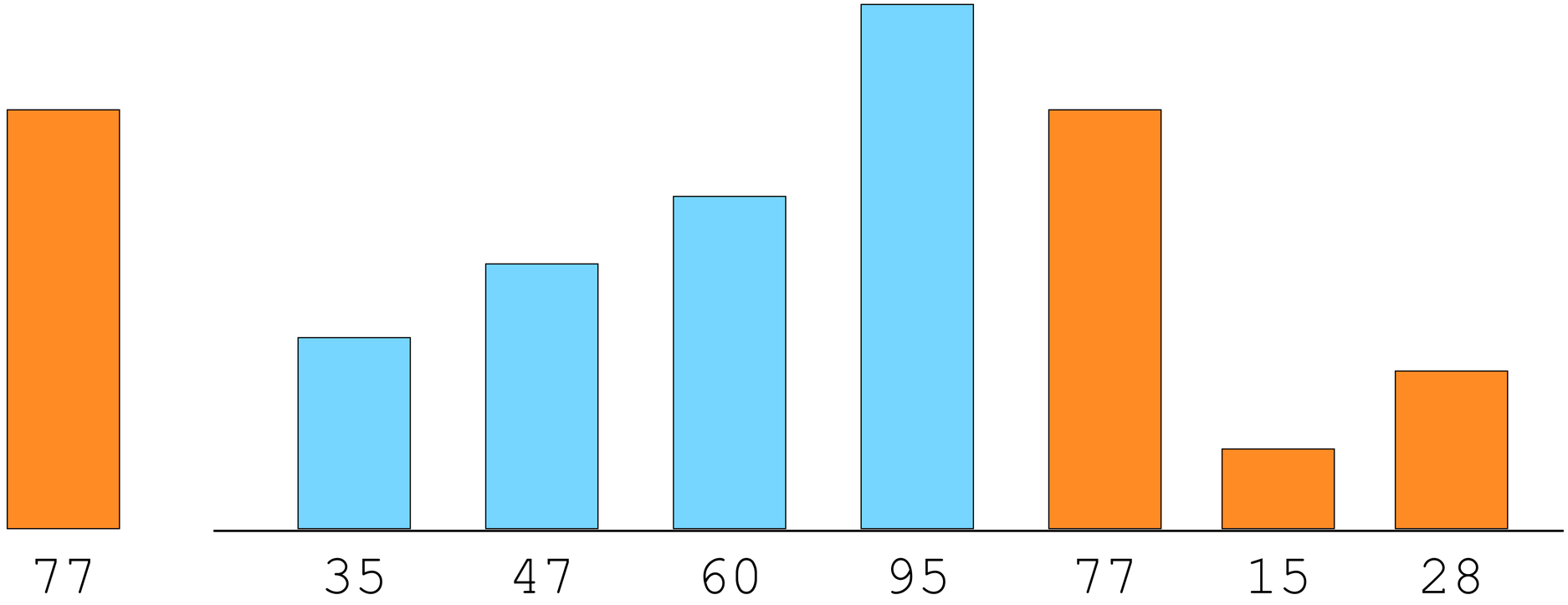
插入排序



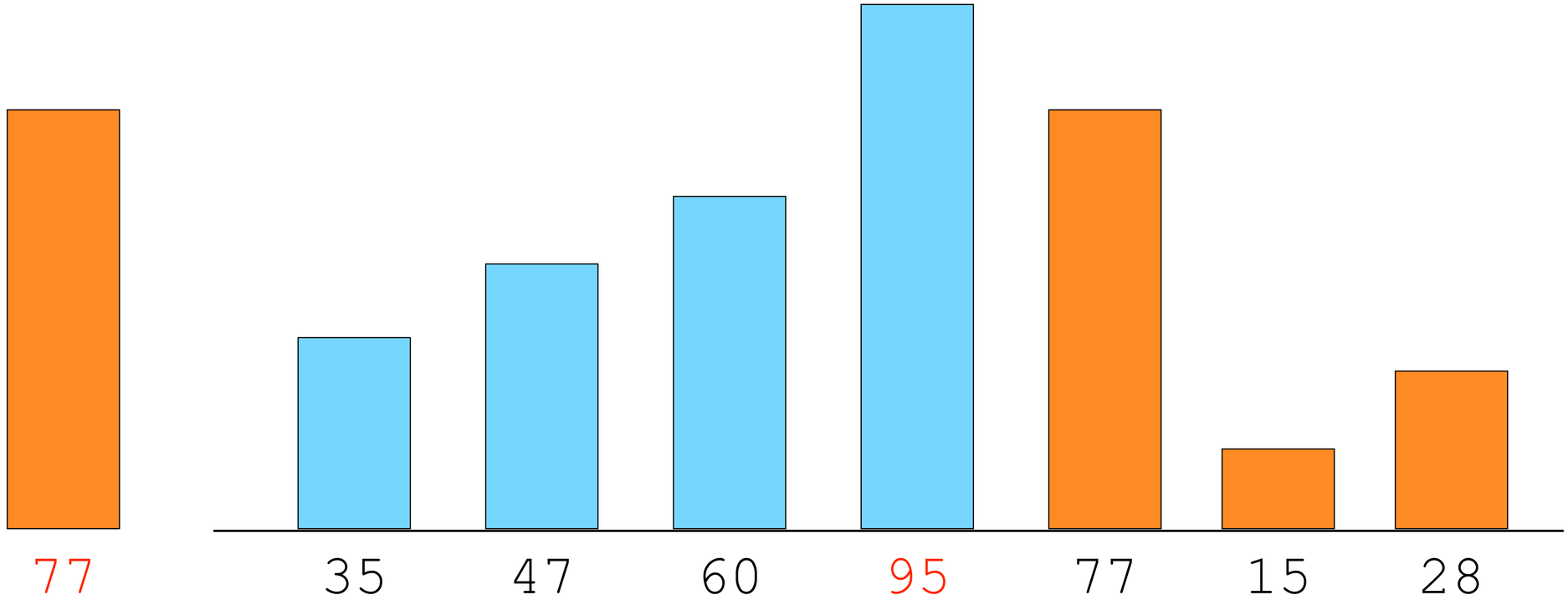
插入排序



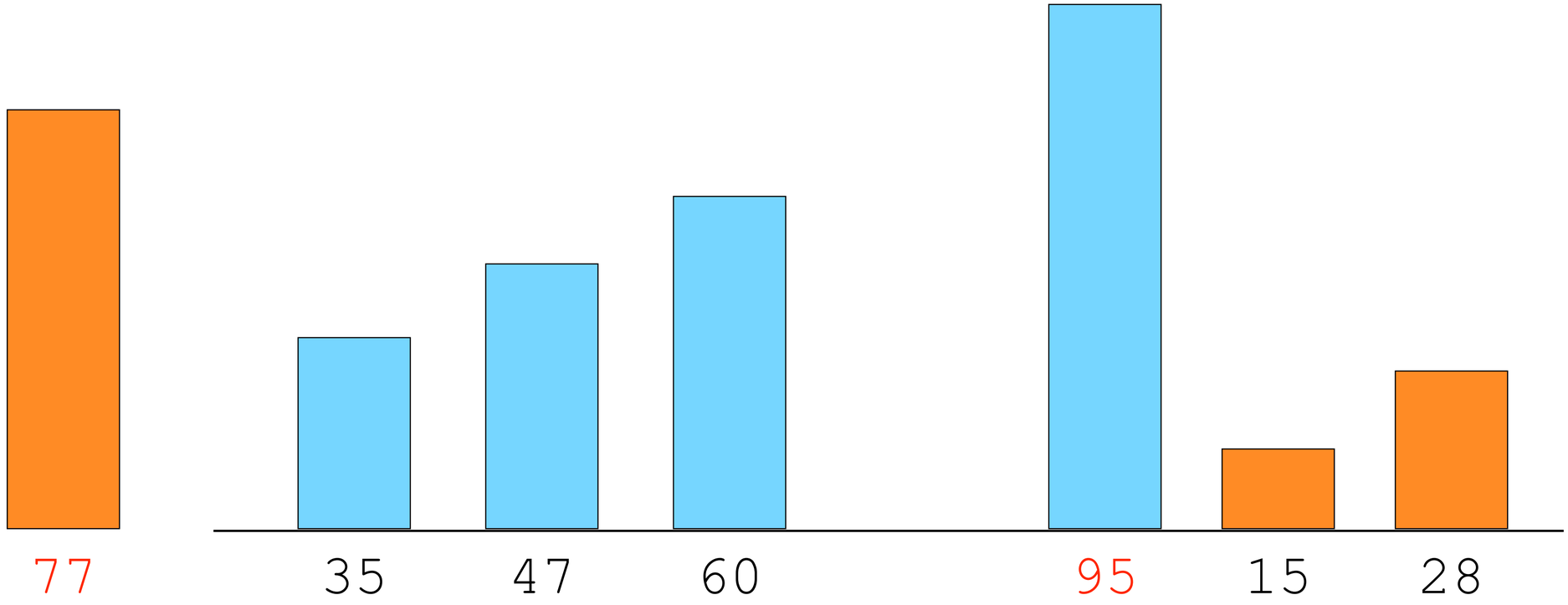
插入排序



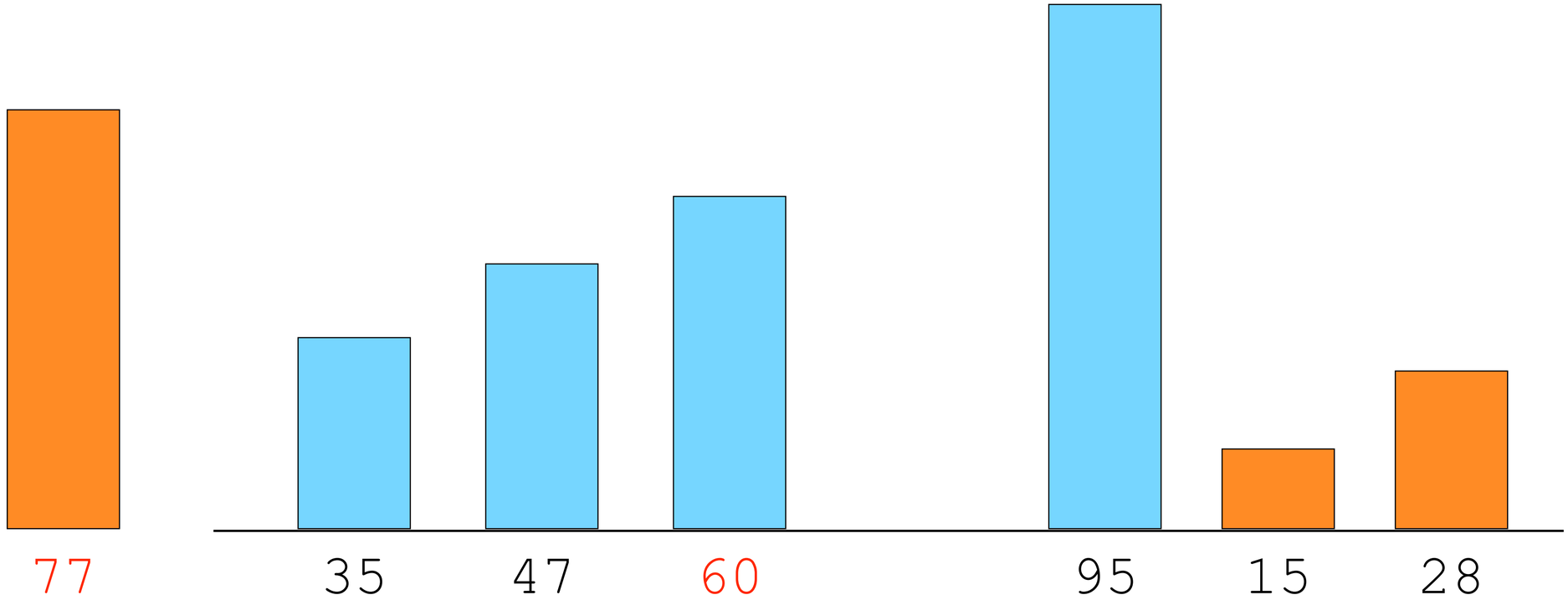
插入排序



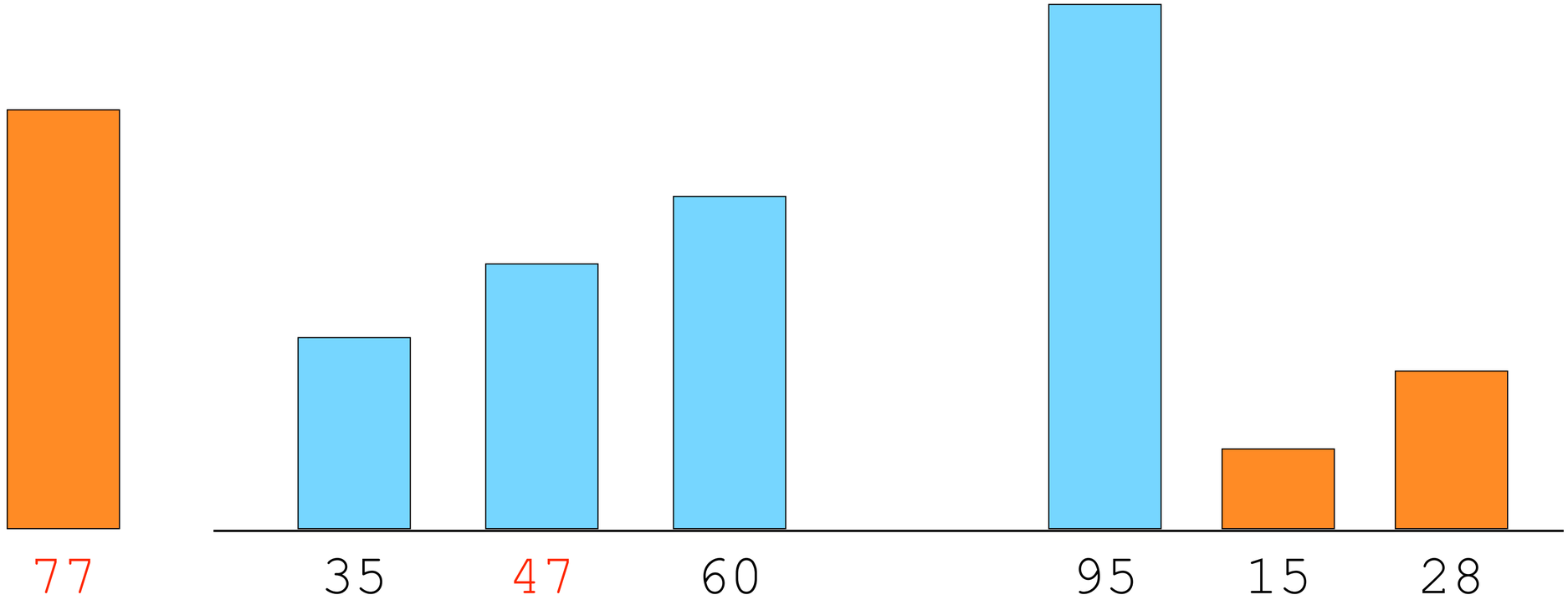
插入排序



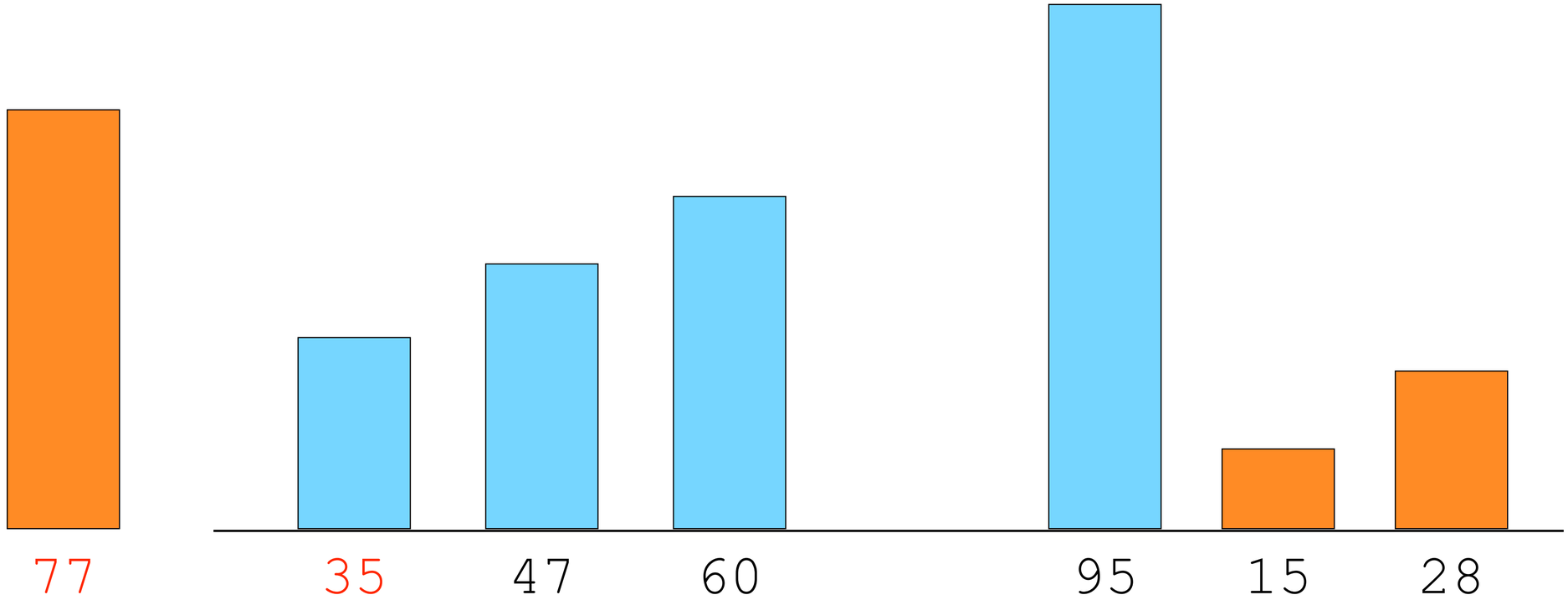
插入排序



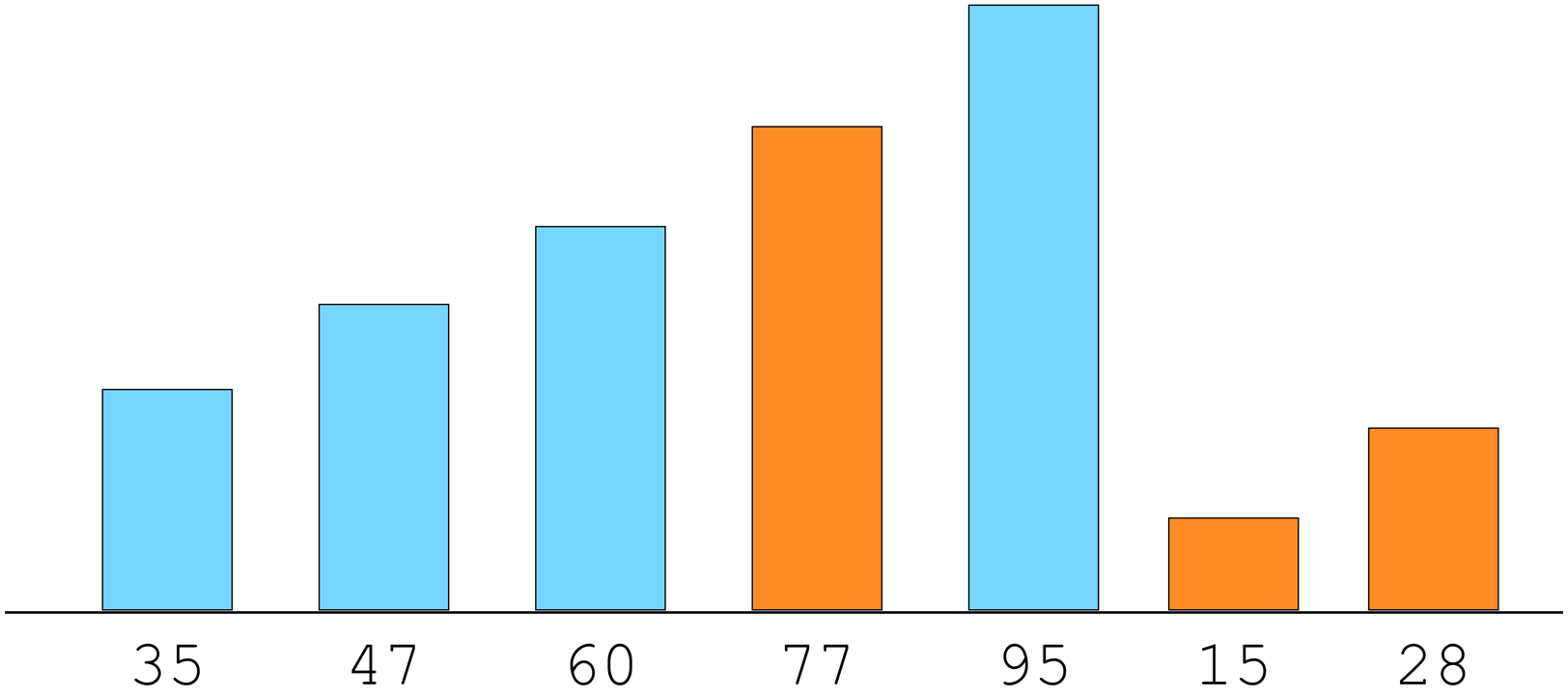
插入排序



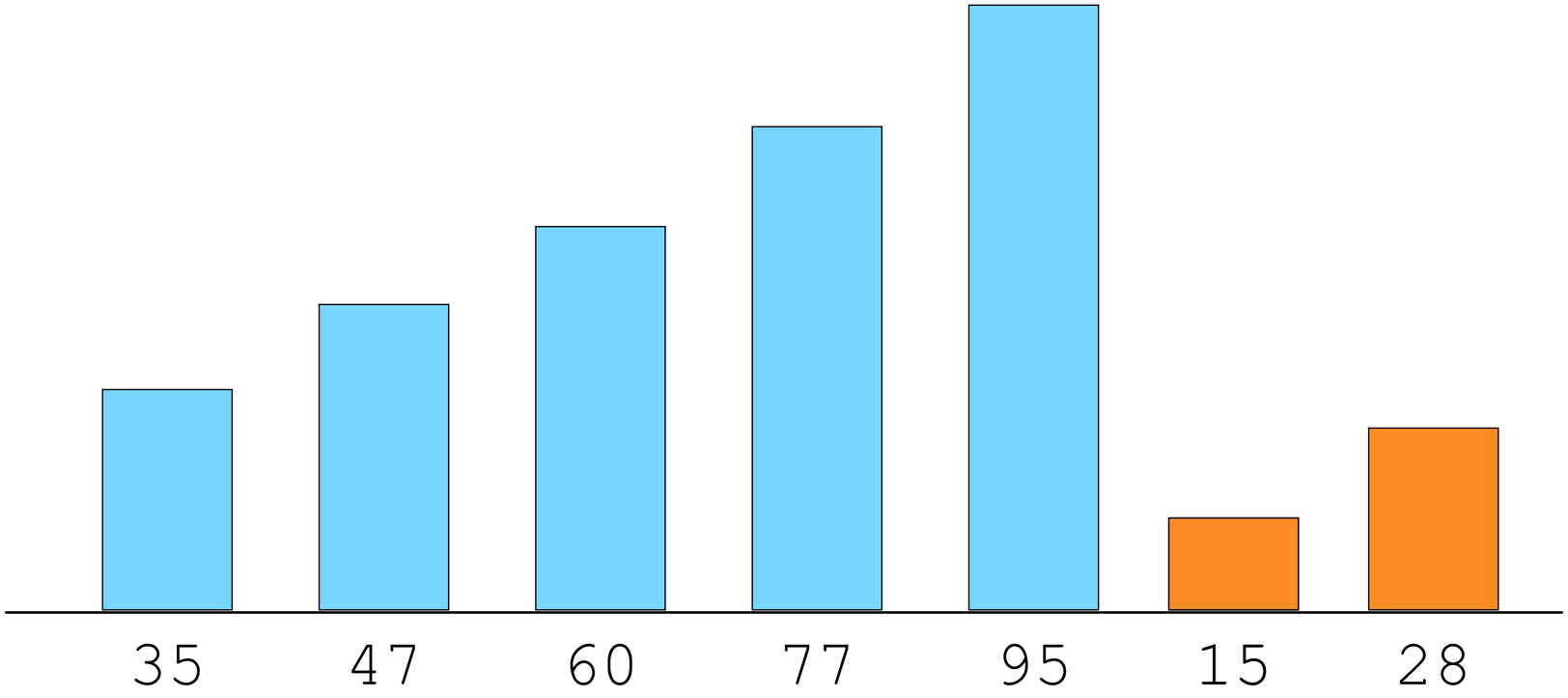
插入排序



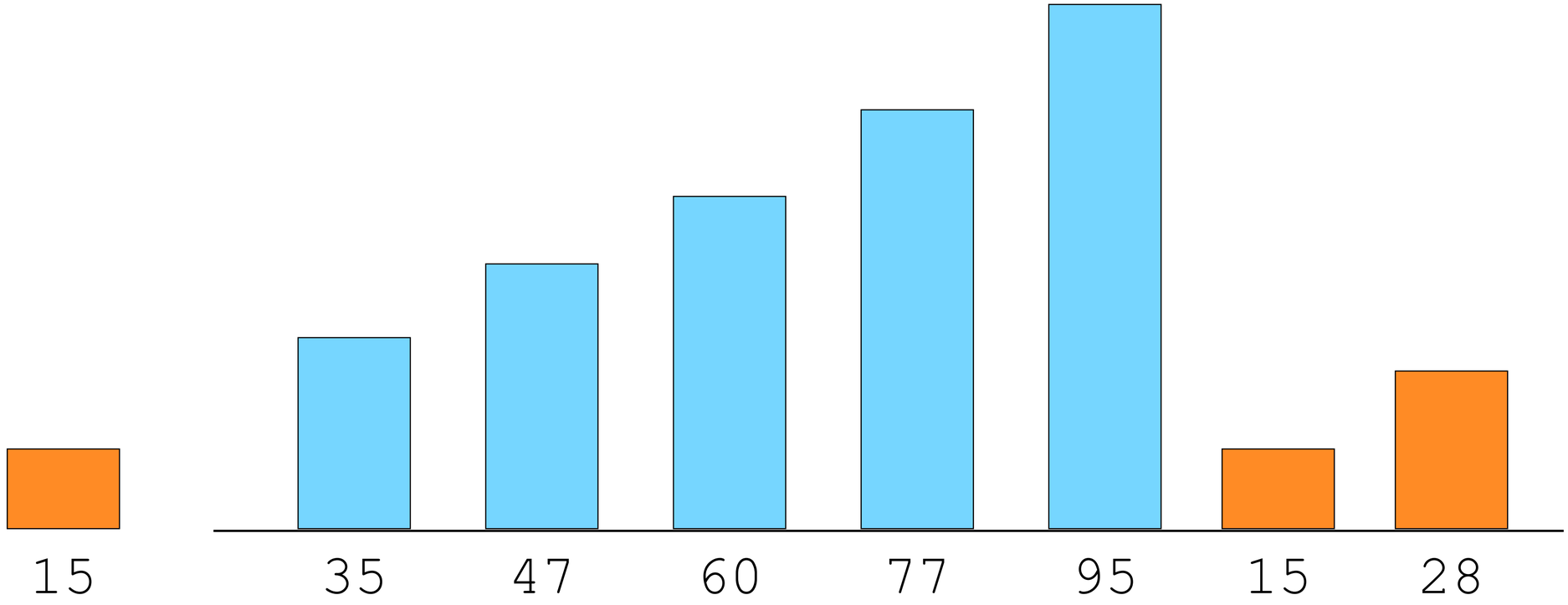
插入排序



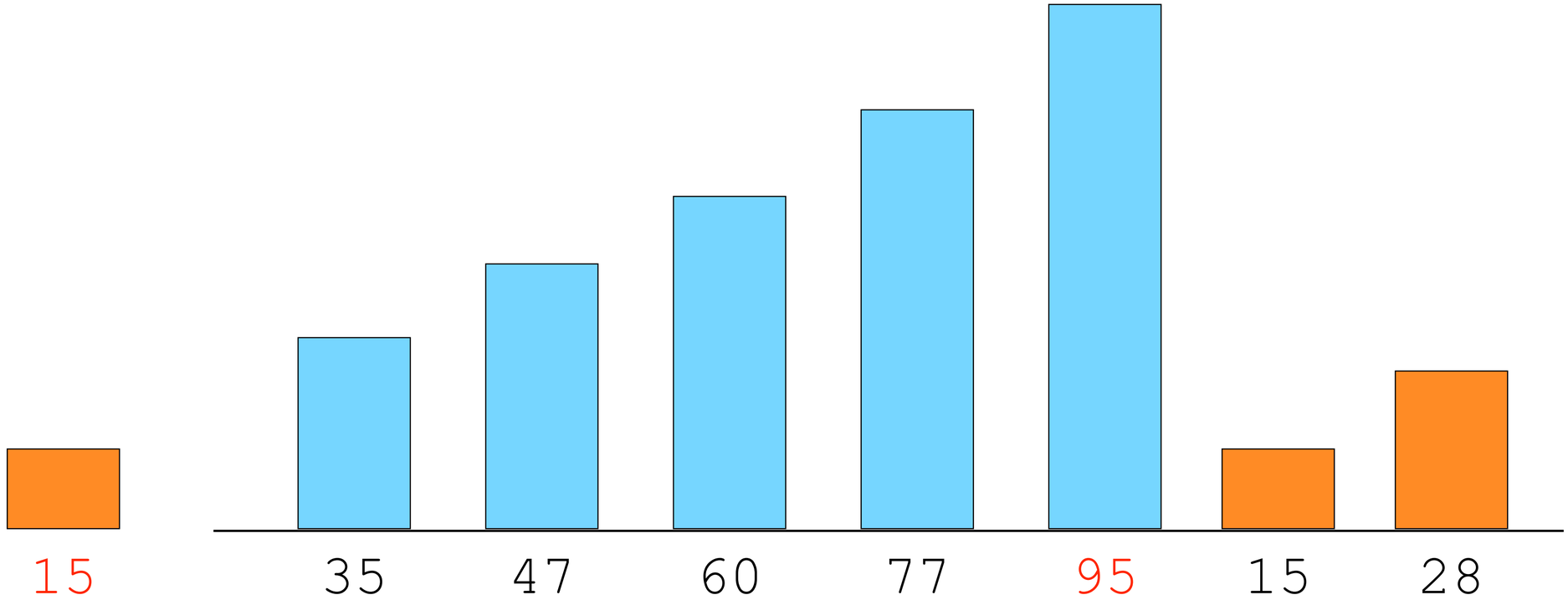
插入排序



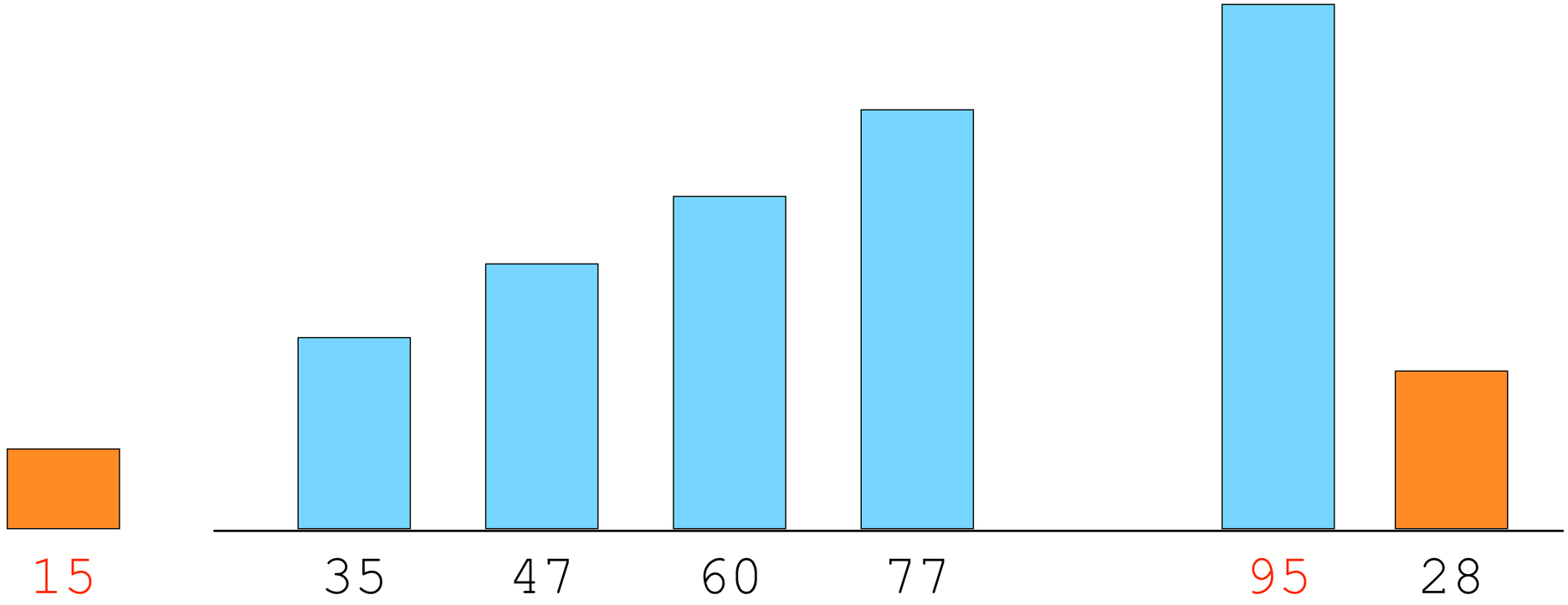
插入排序



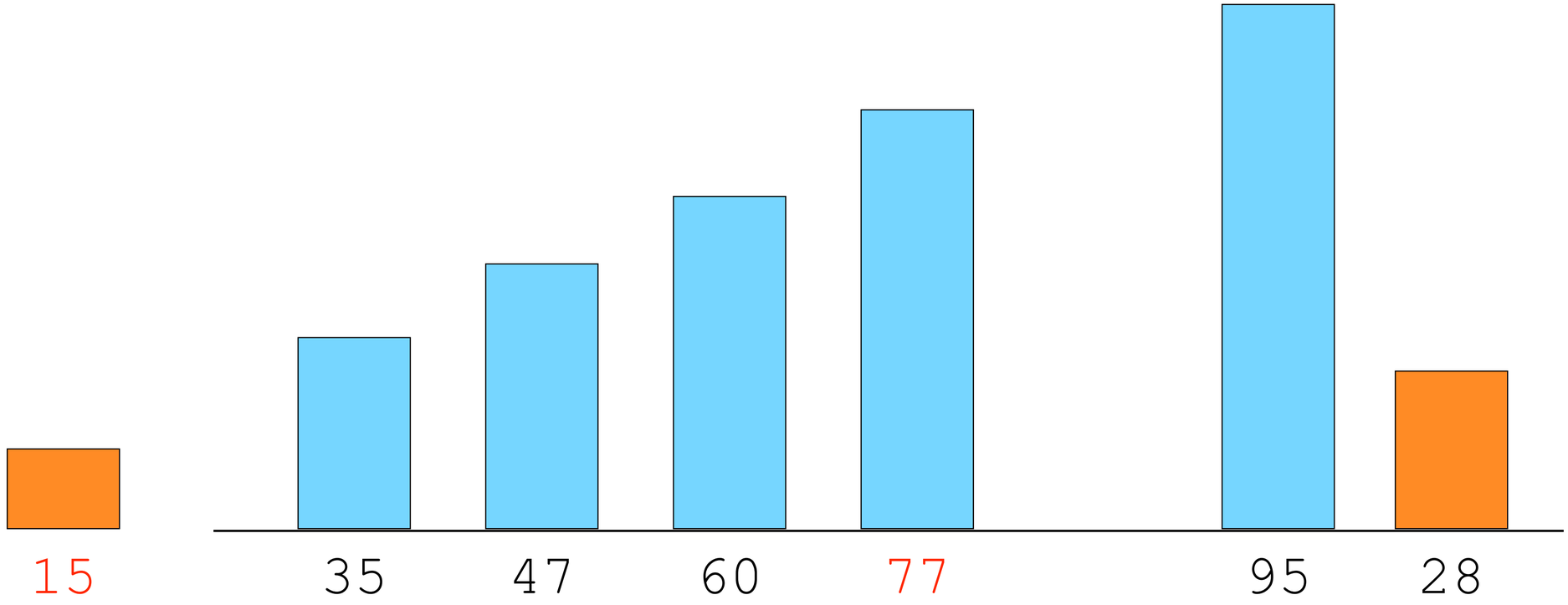
插入排序



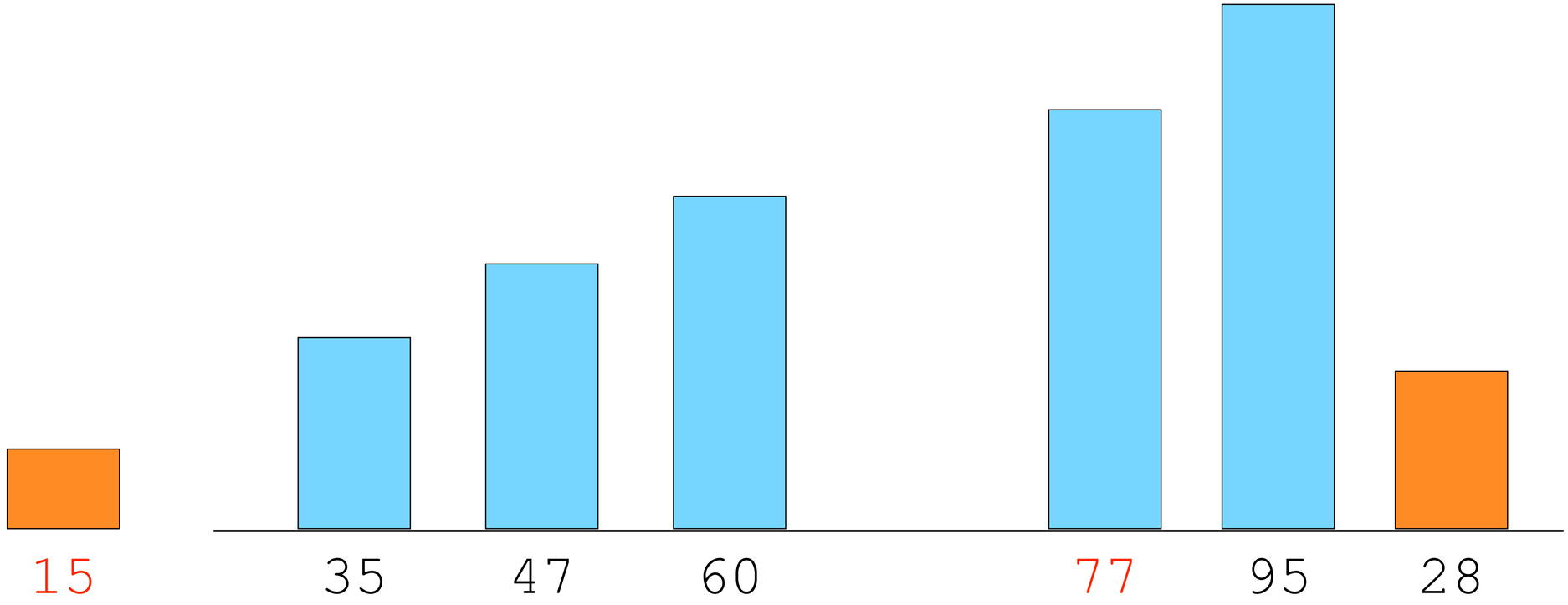
插入排序



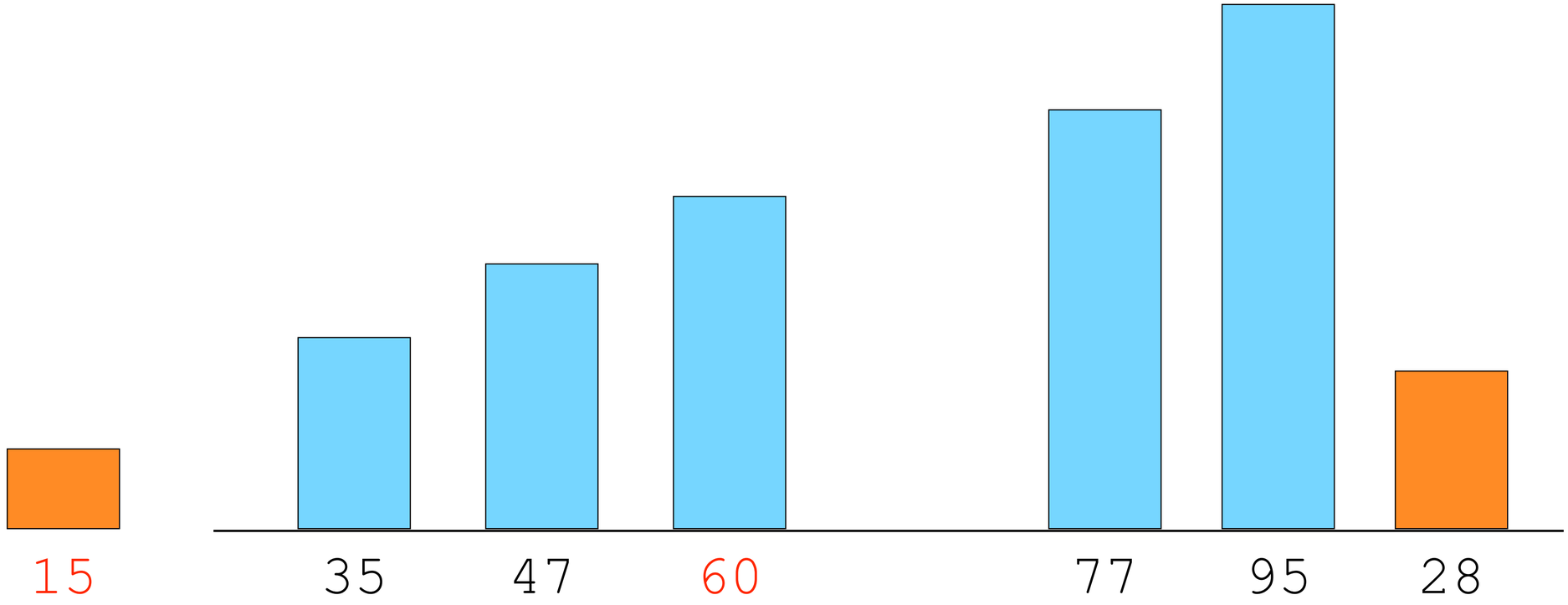
插入排序



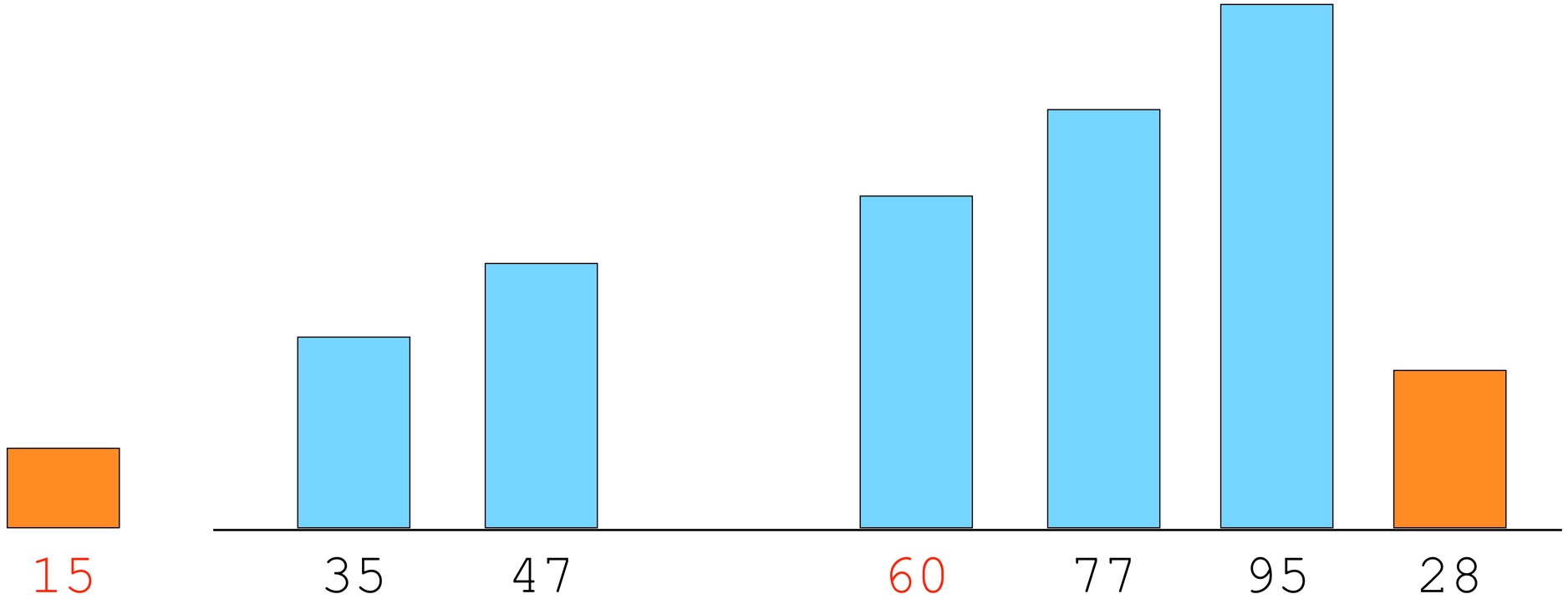
插入排序



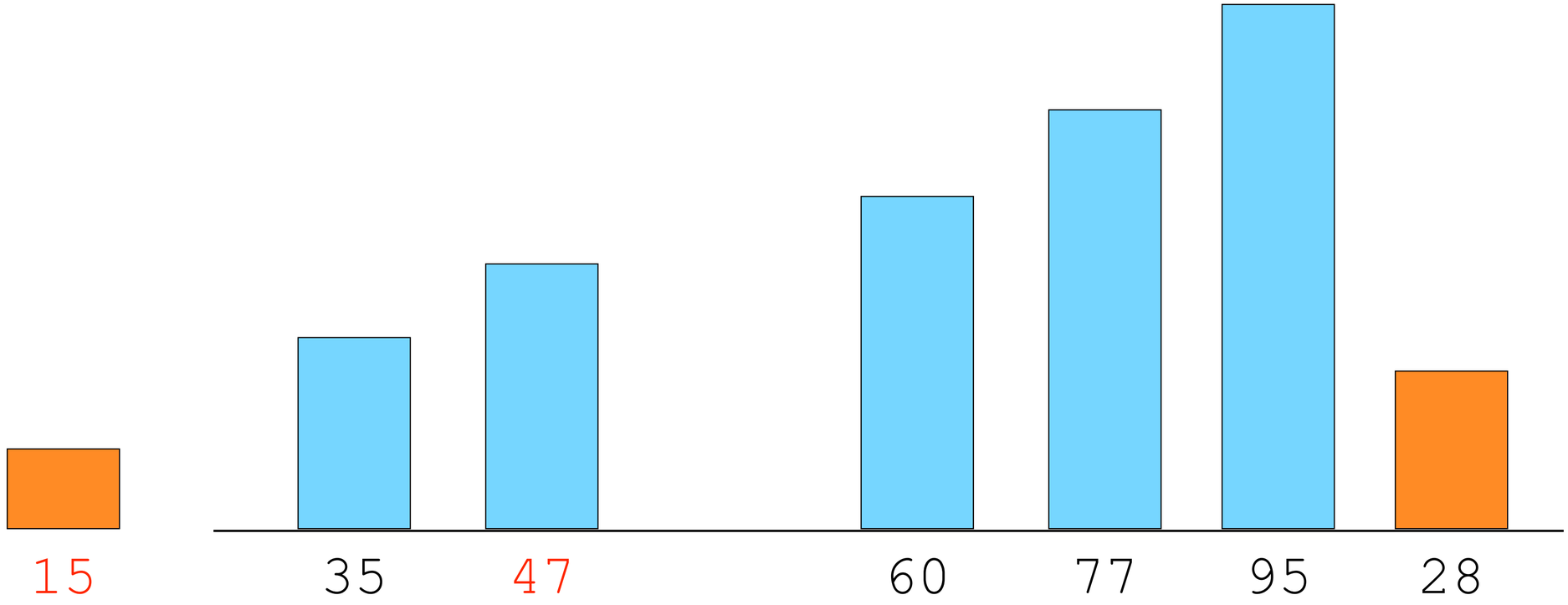
插入排序



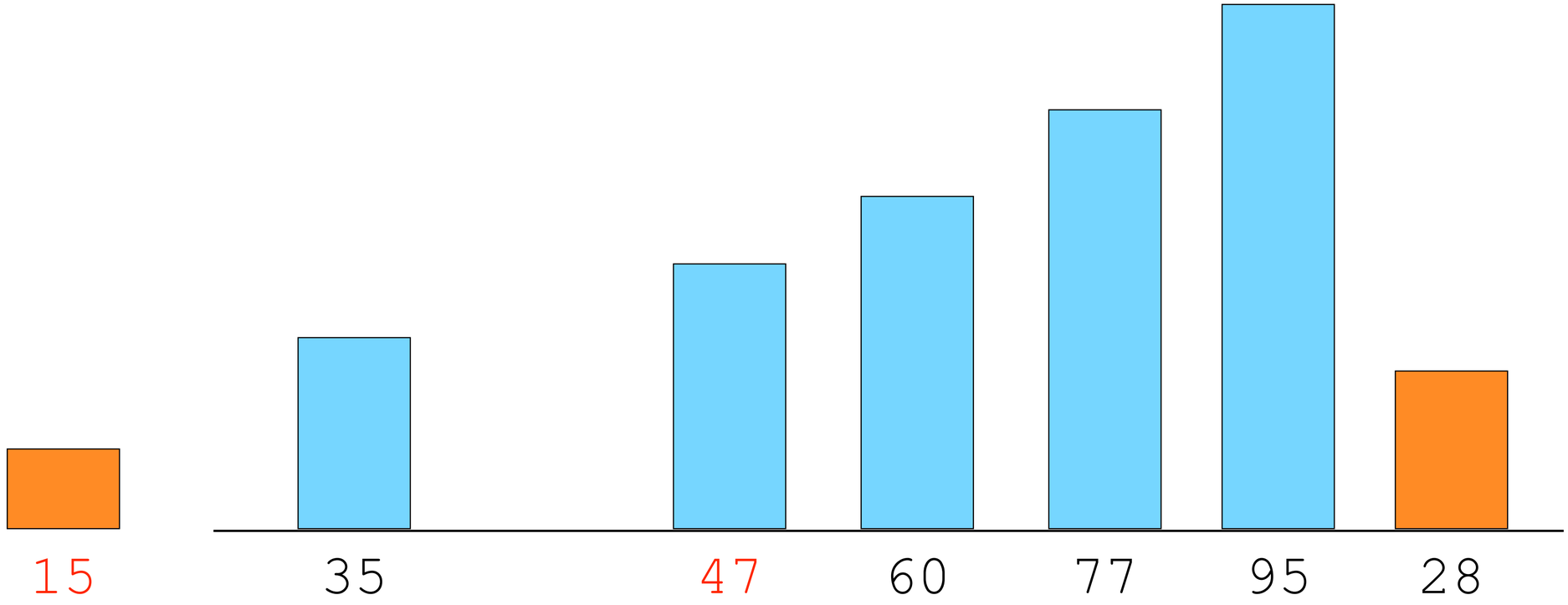
插入排序



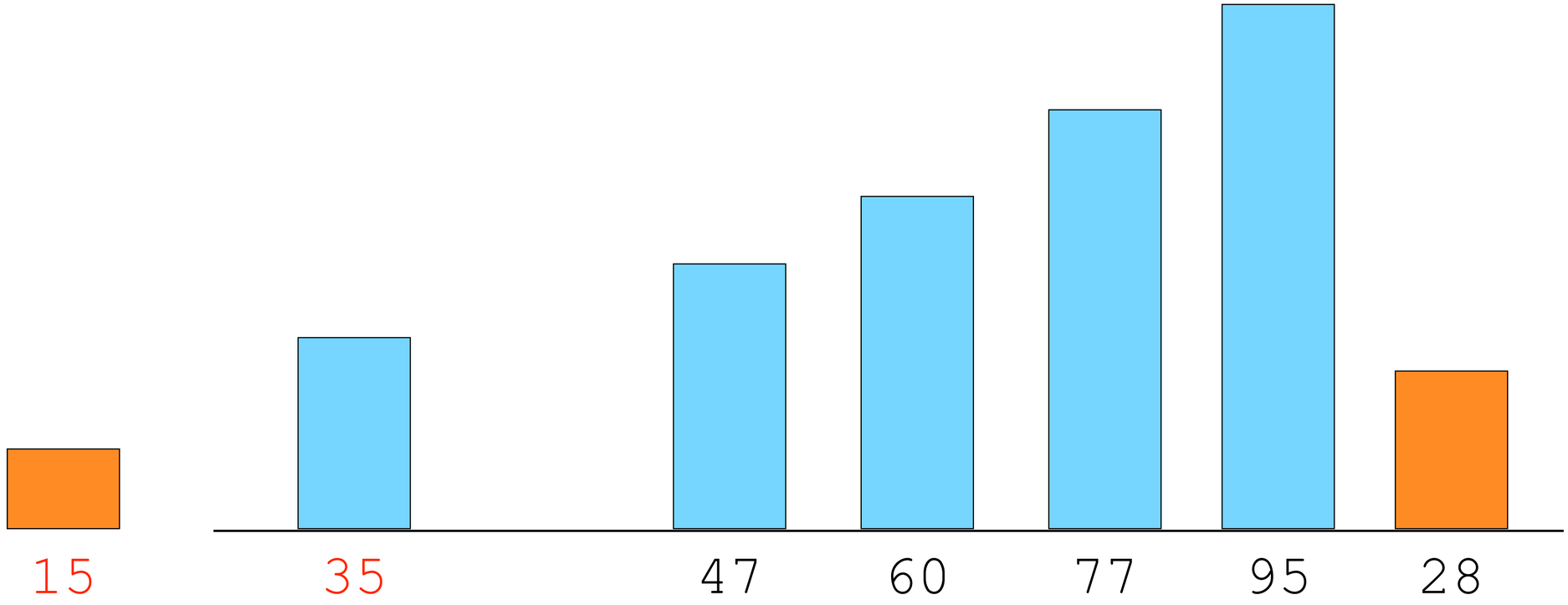
插入排序



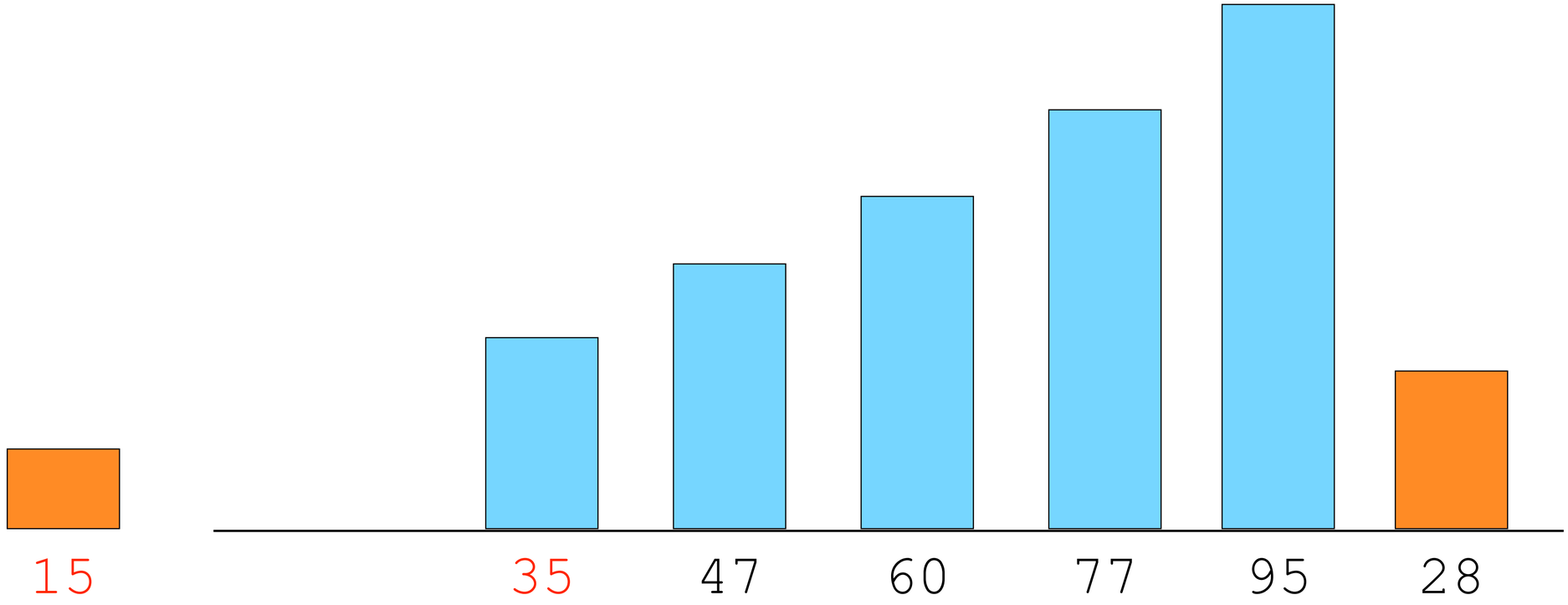
插入排序



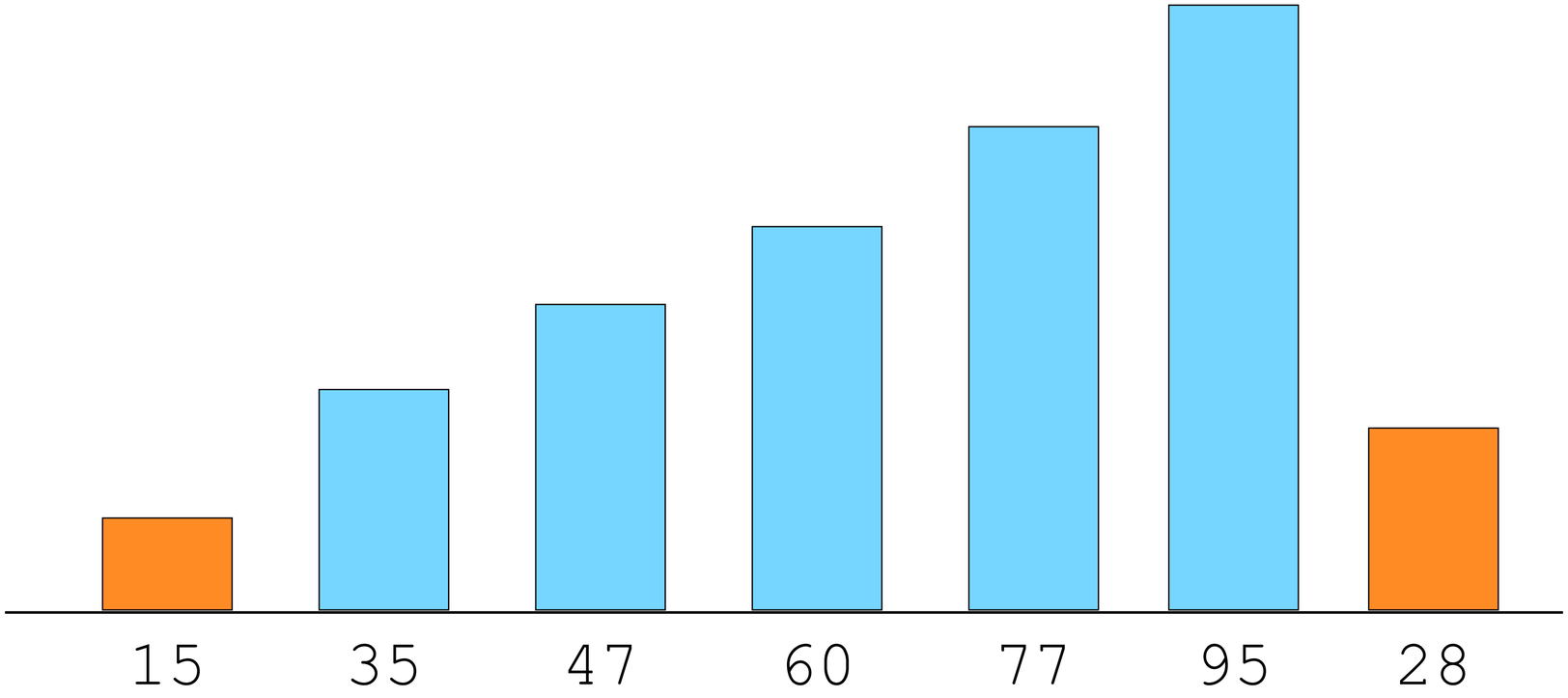
插入排序



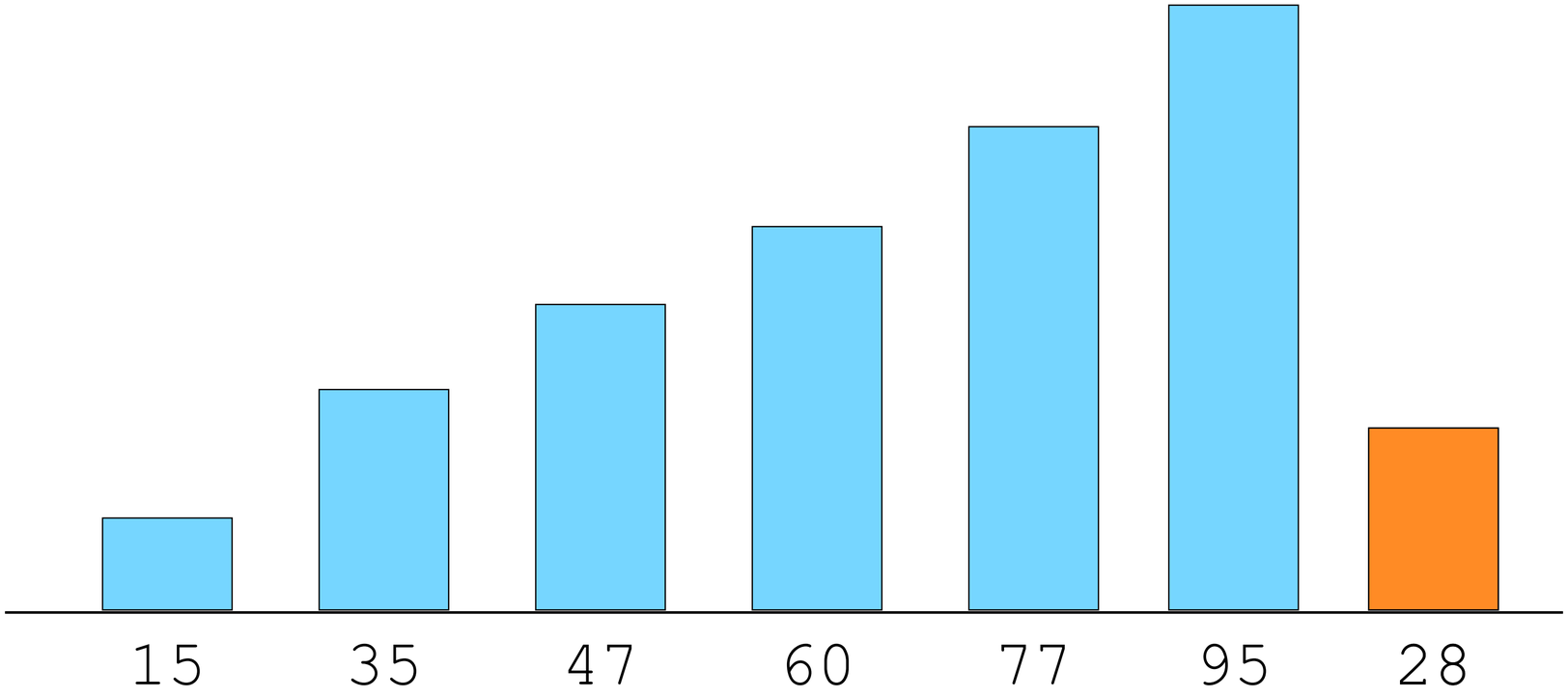
插入排序



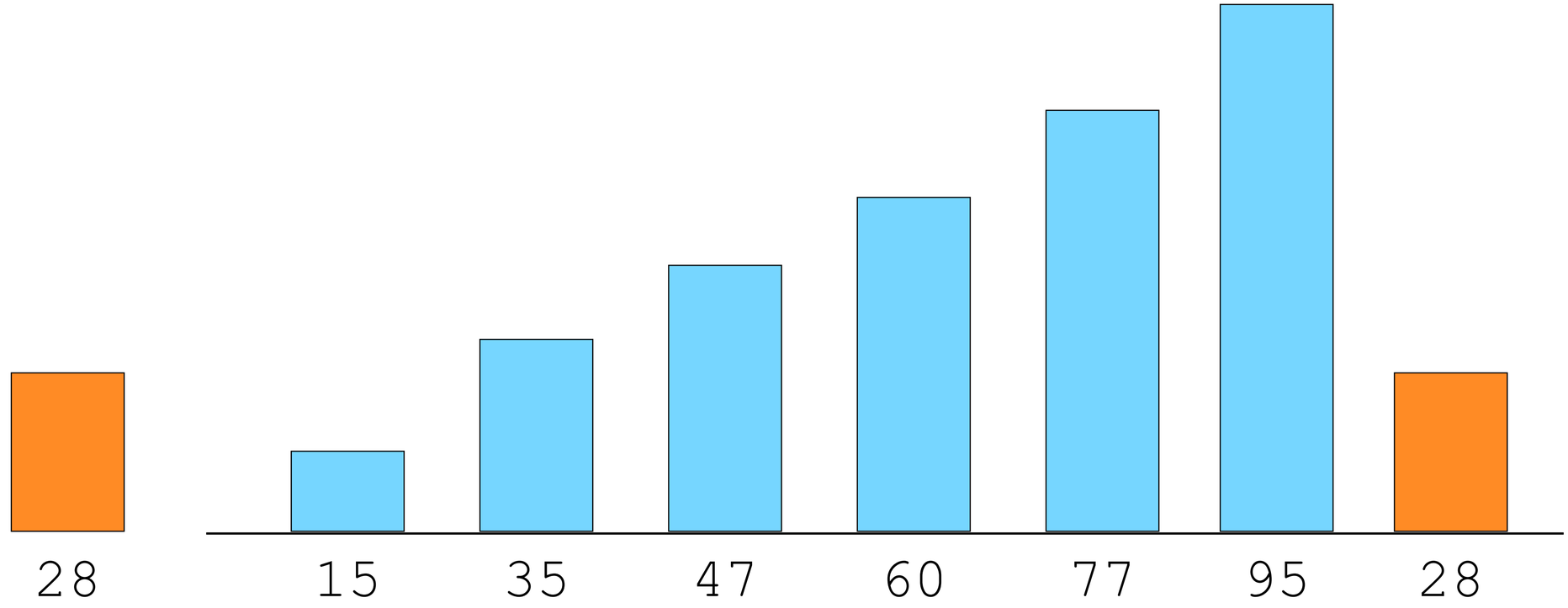
插入排序



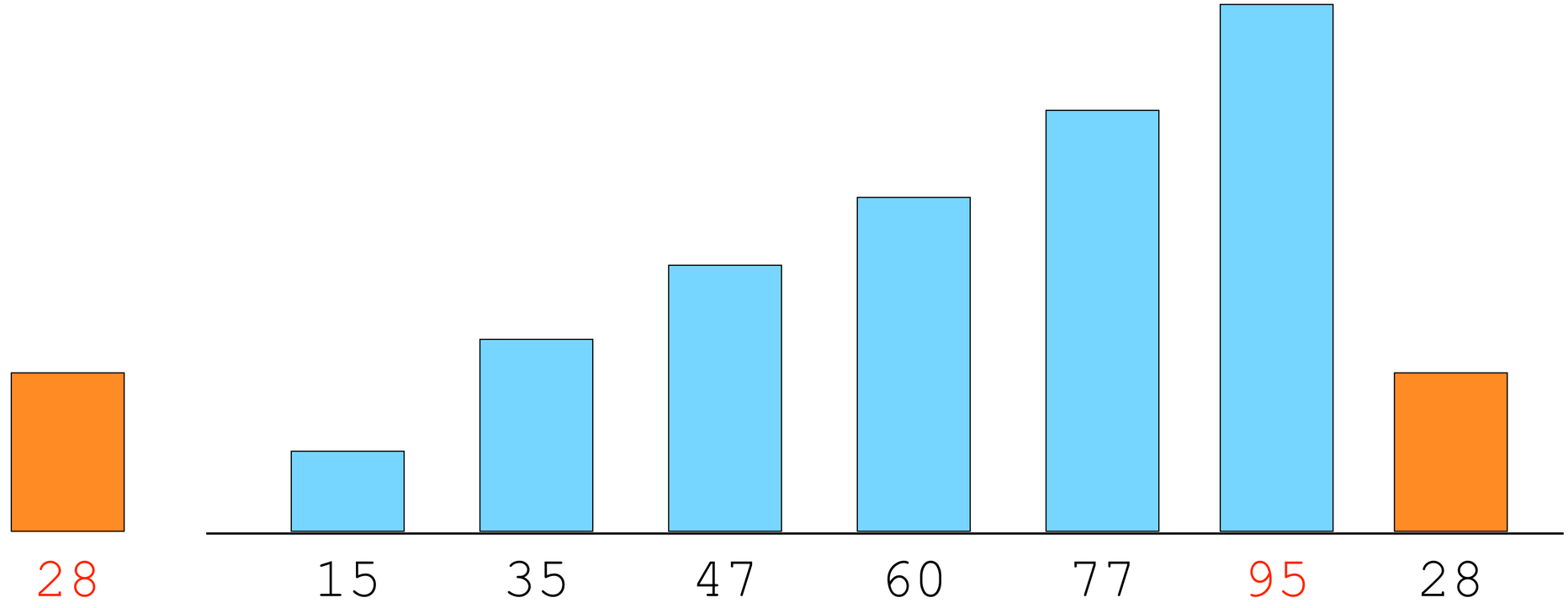
插入排序



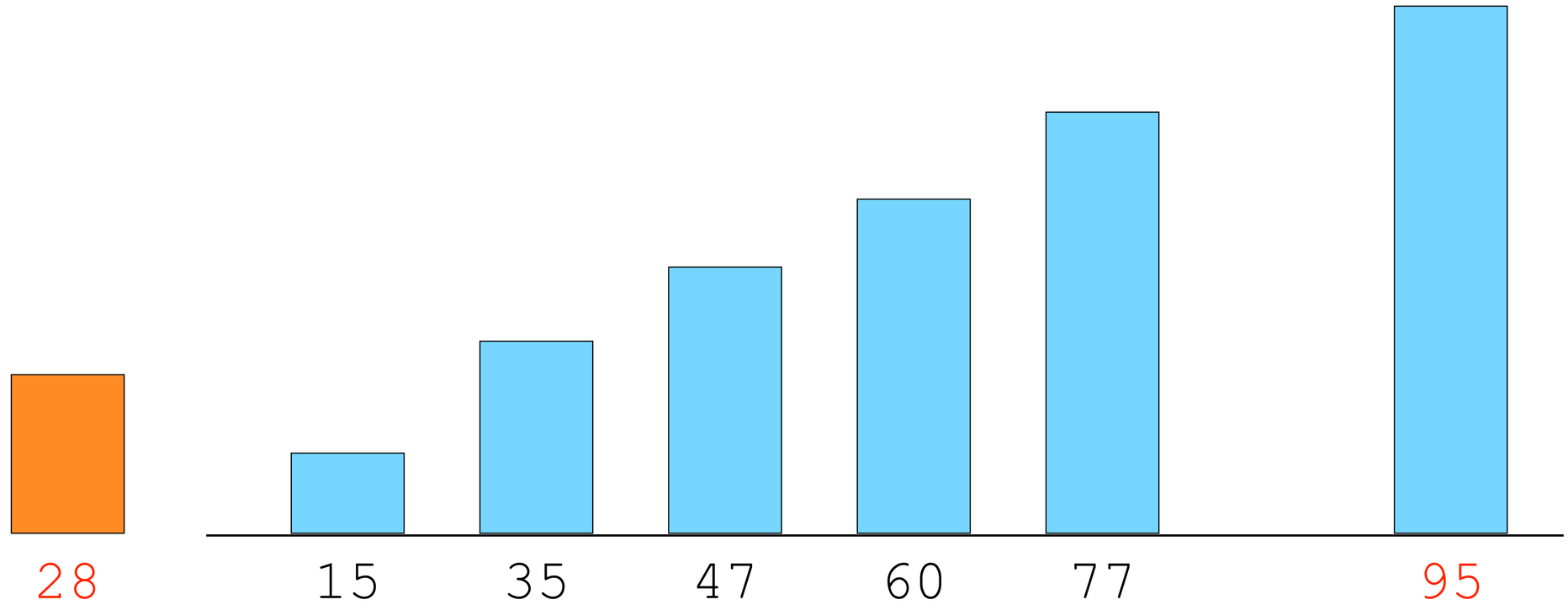
插入排序



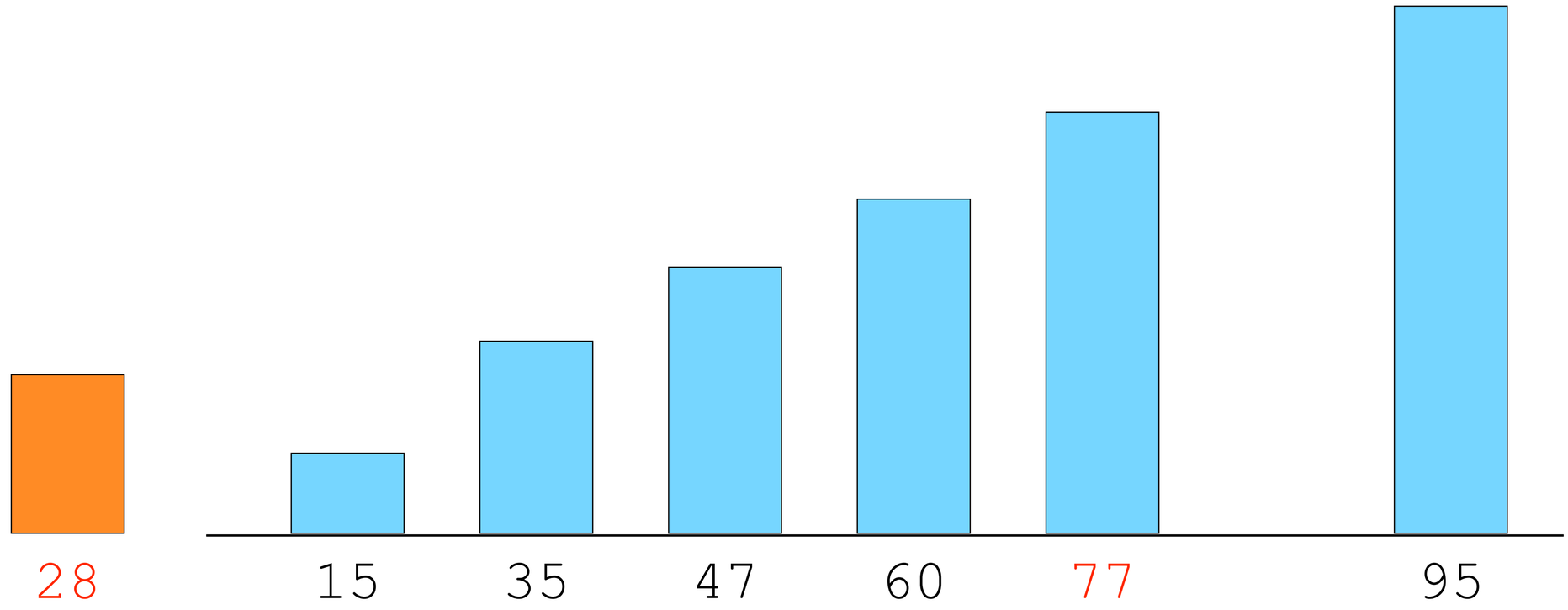
插入排序



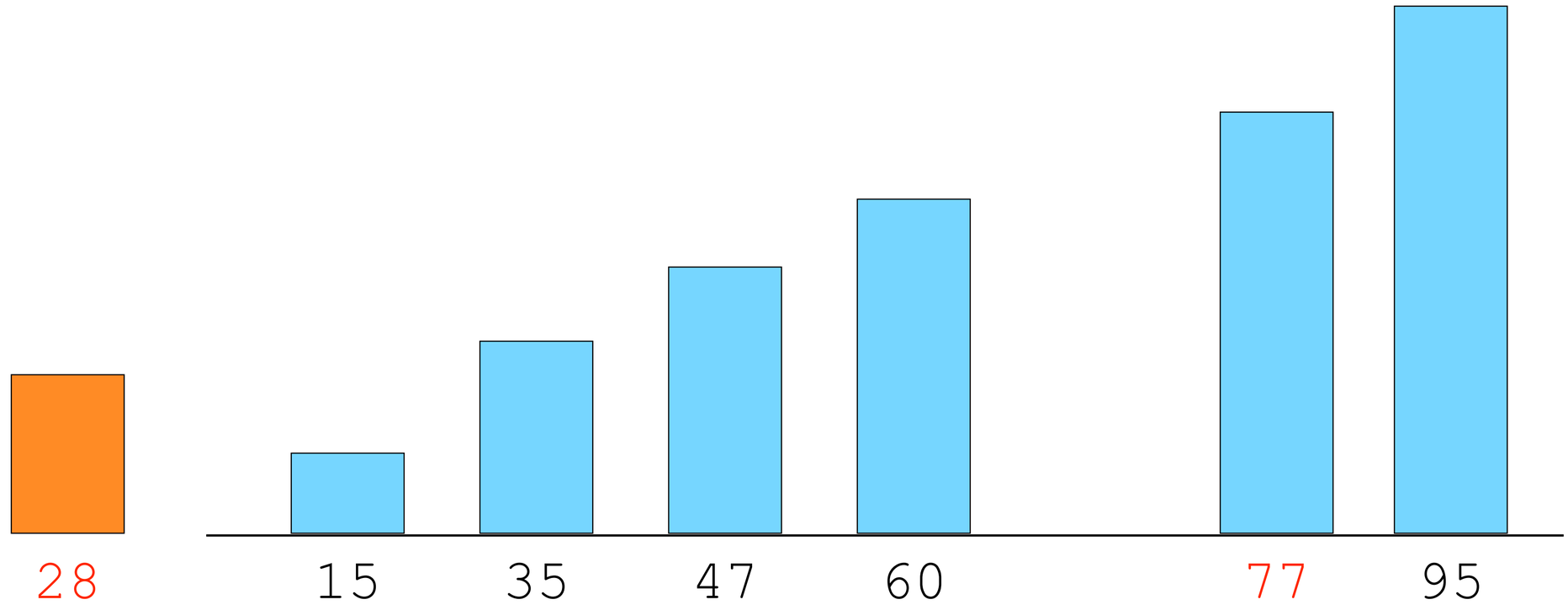
插入排序



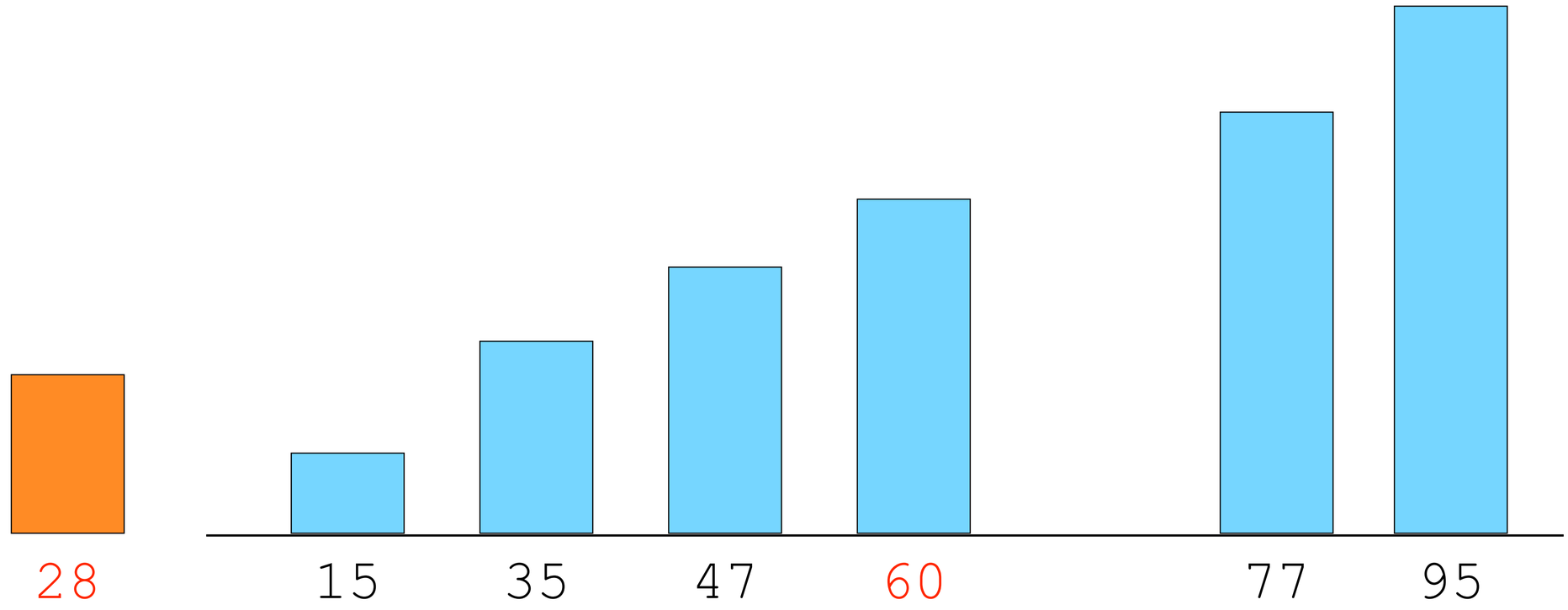
插入排序



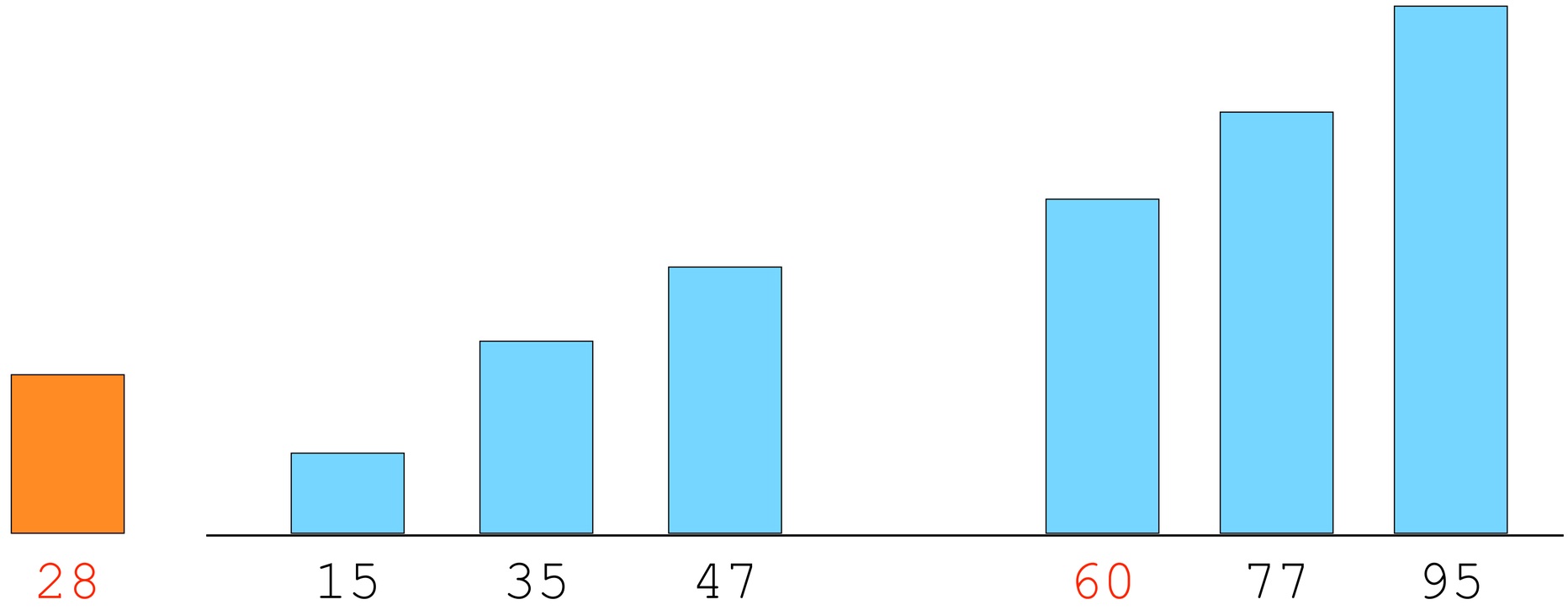
插入排序



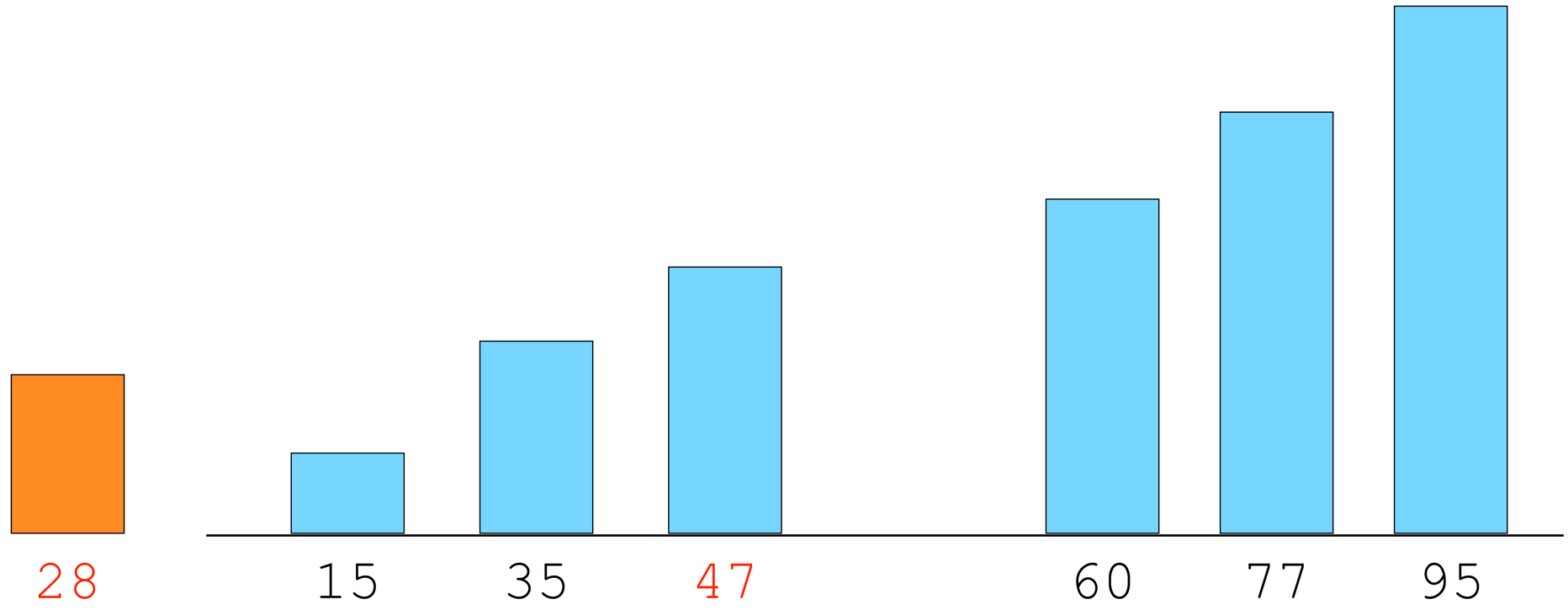
插入排序



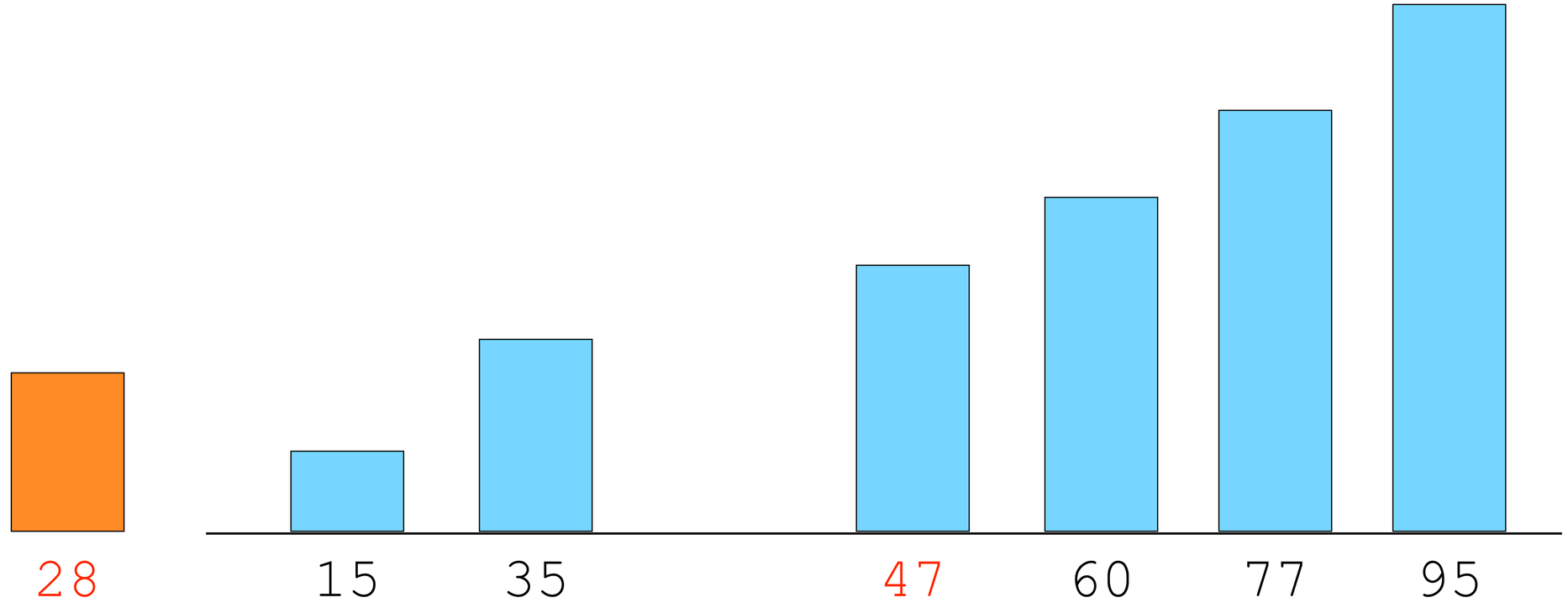
插入排序



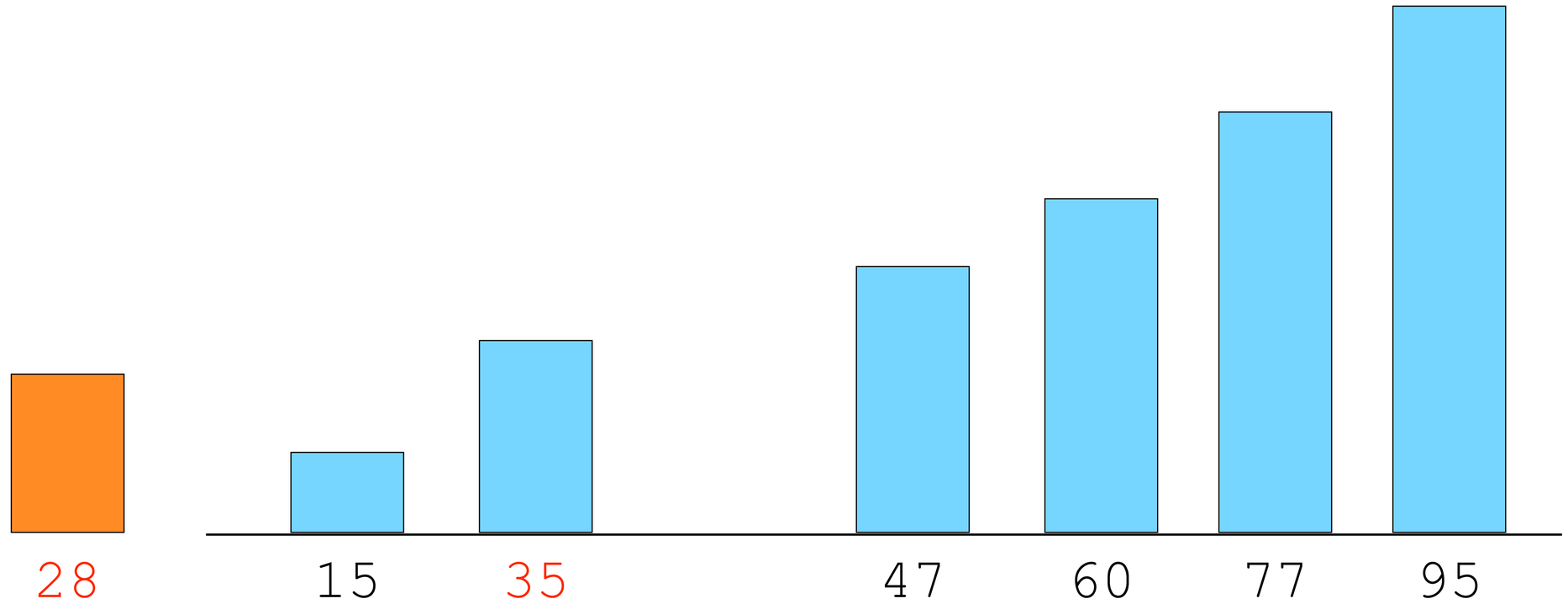
插入排序



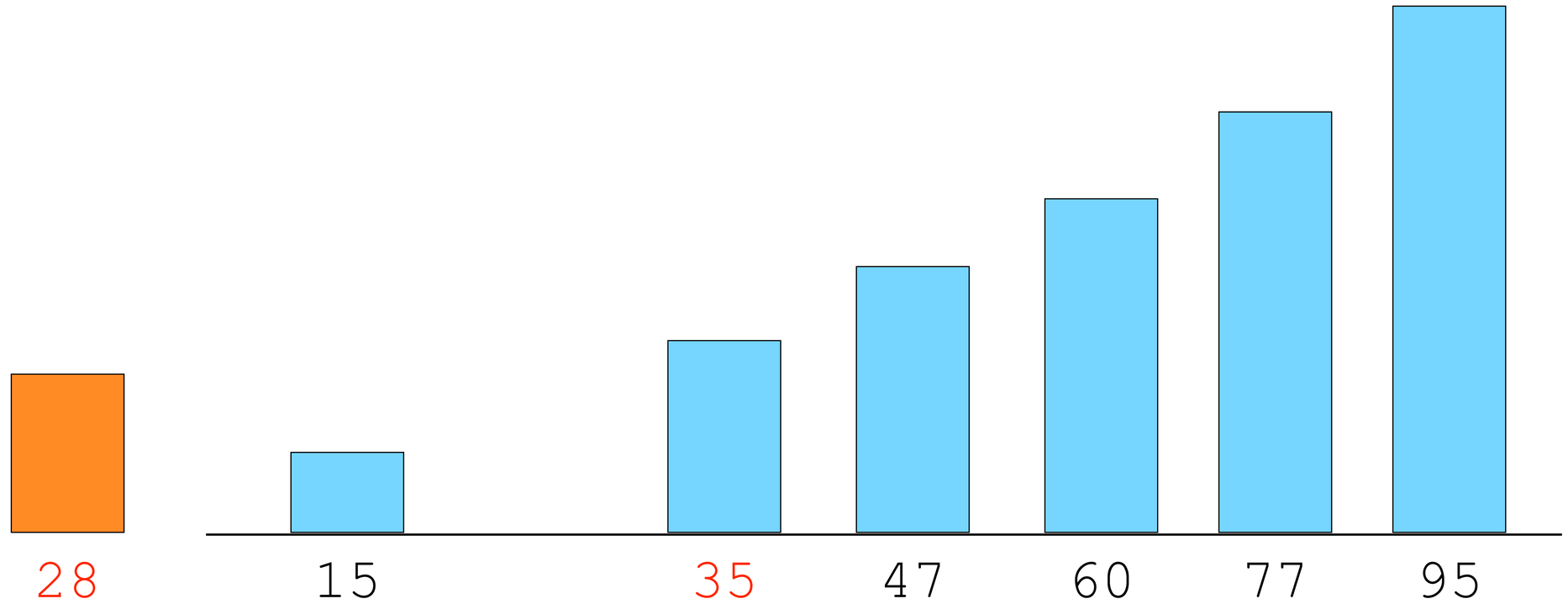
插入排序



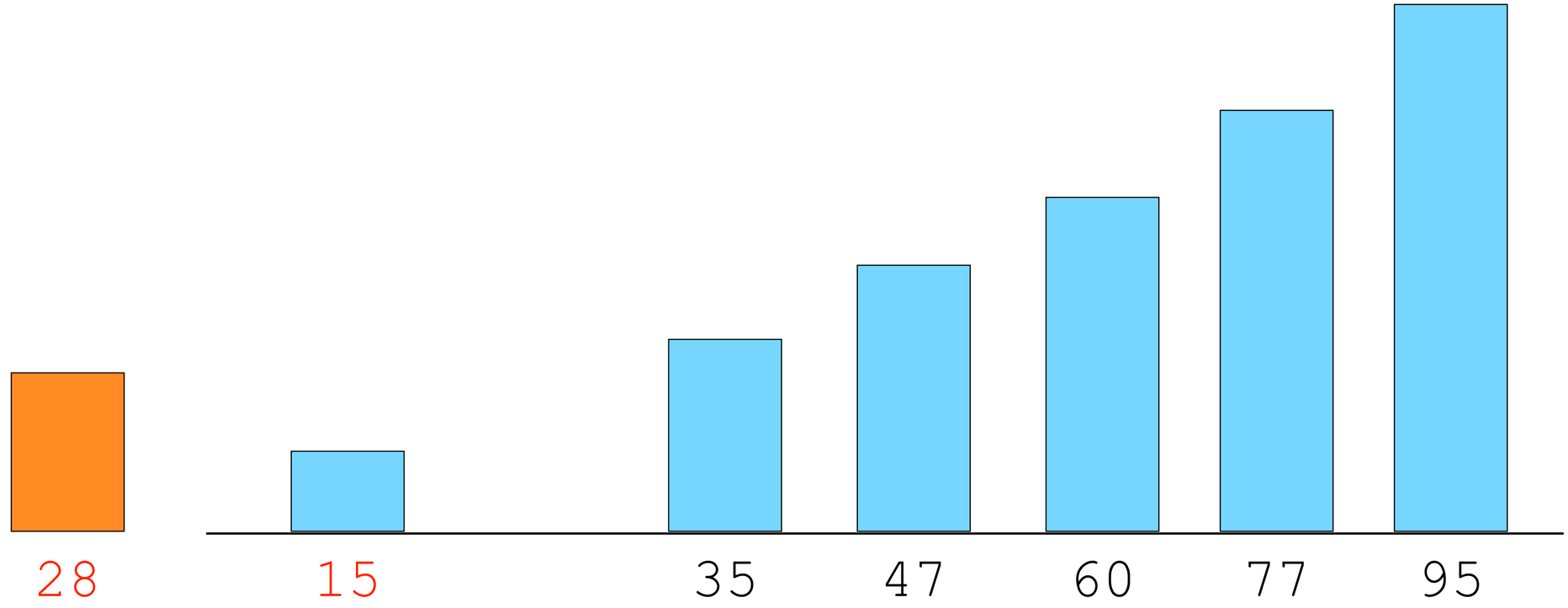
插入排序



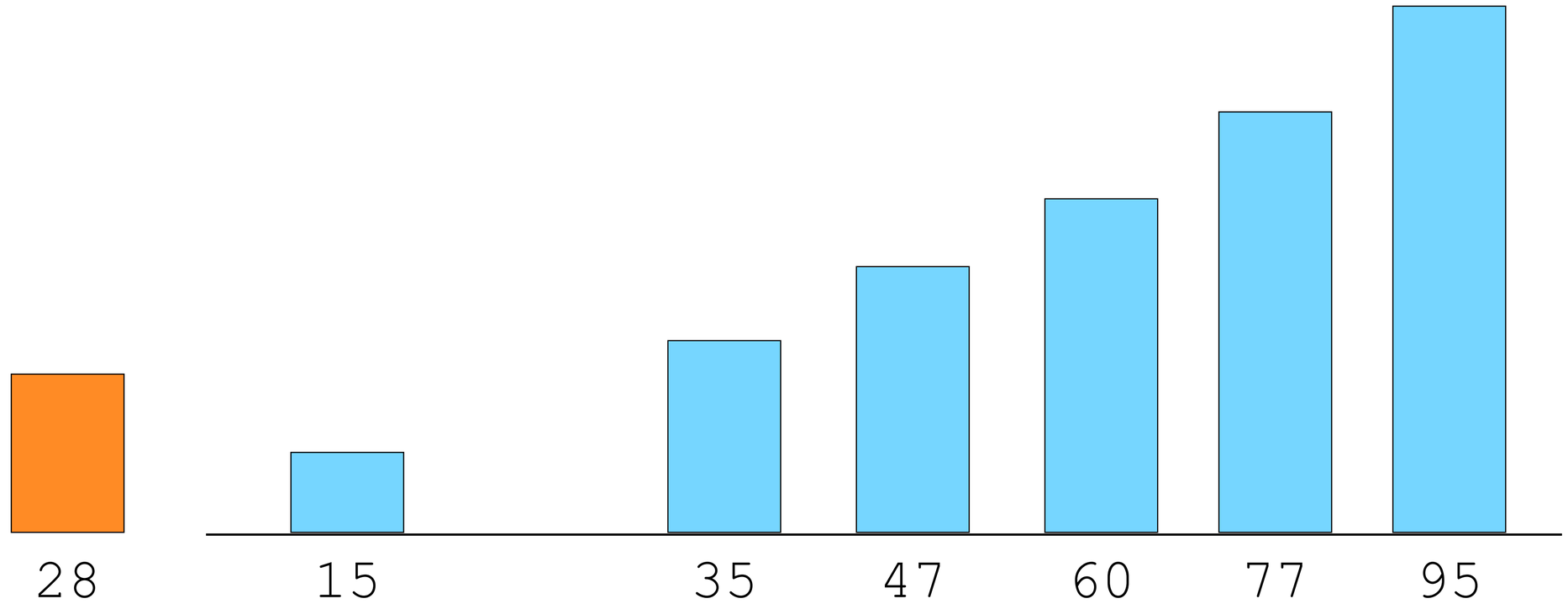
插入排序



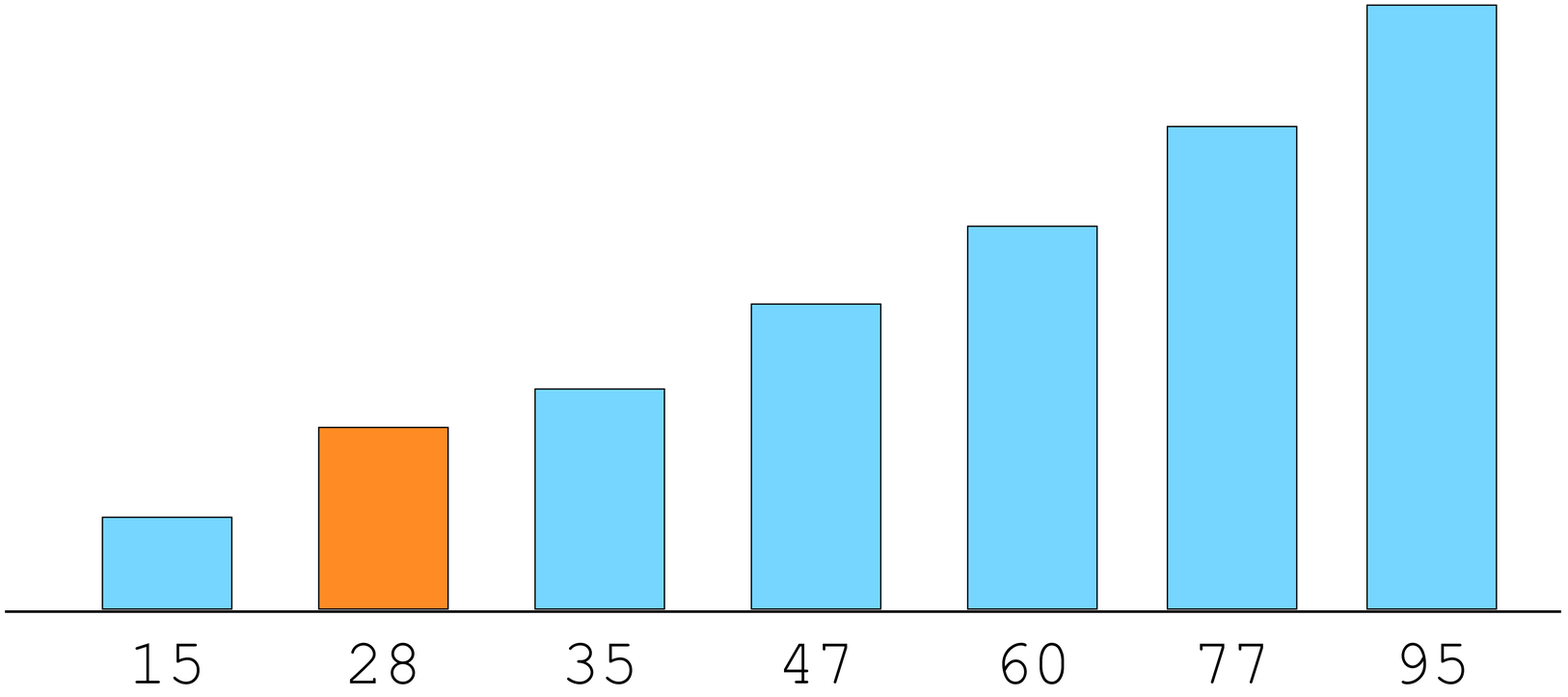
插入排序



插入排序

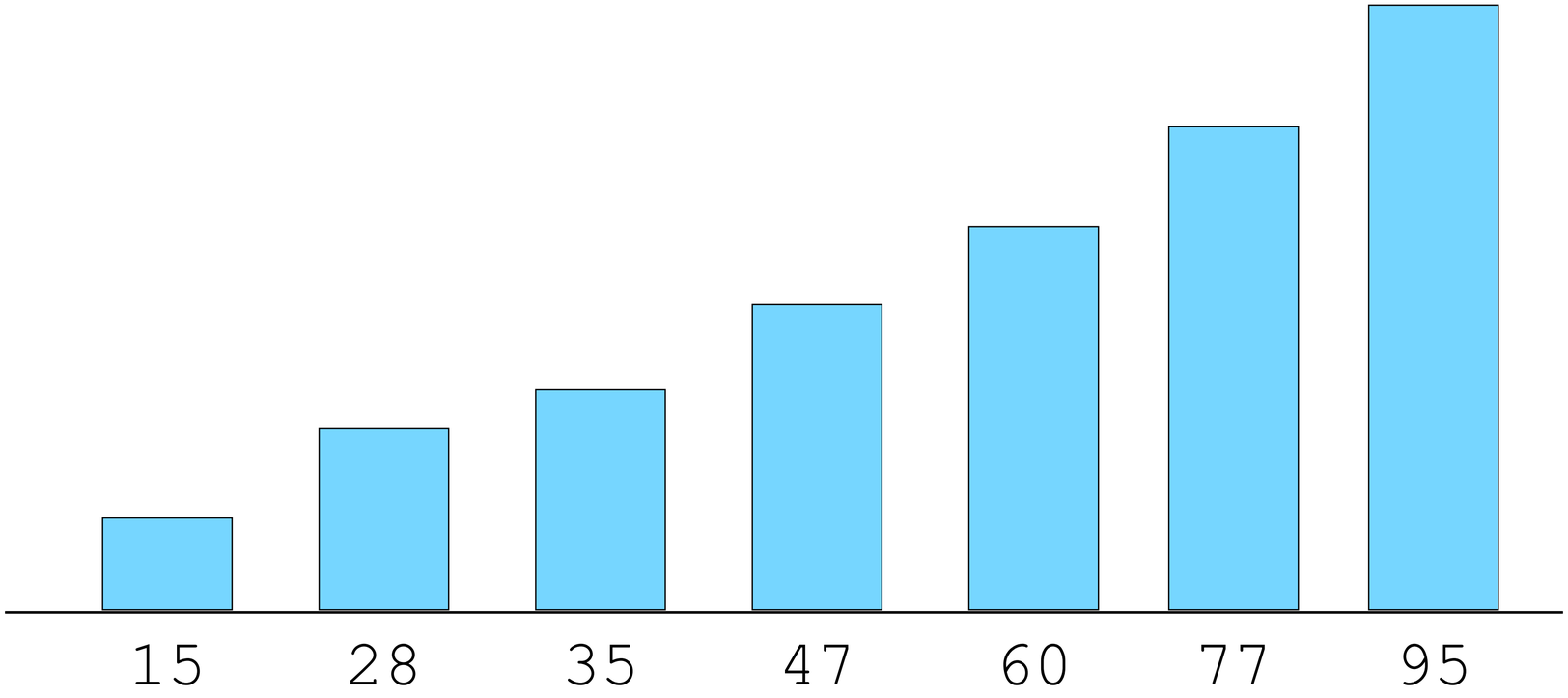


插入排序



插入排序

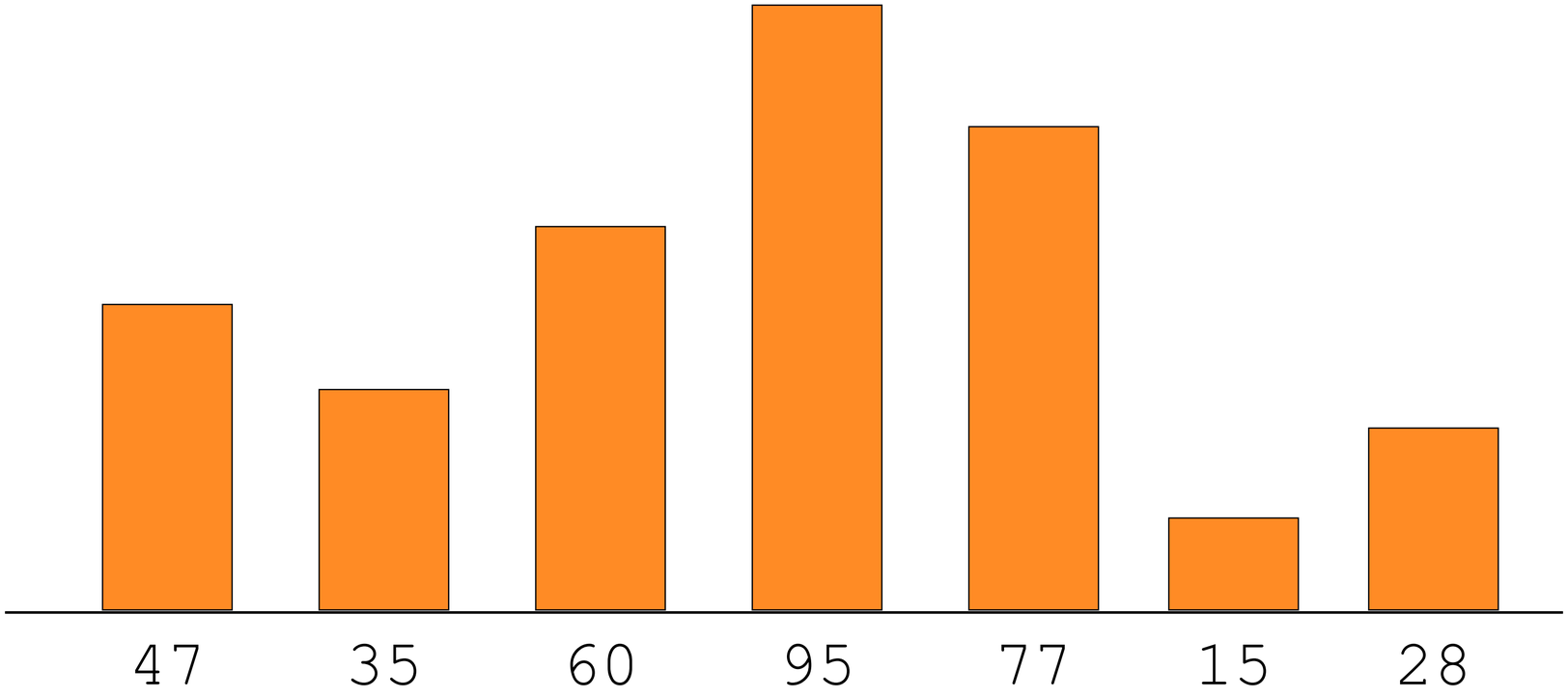
排序 / test2.c



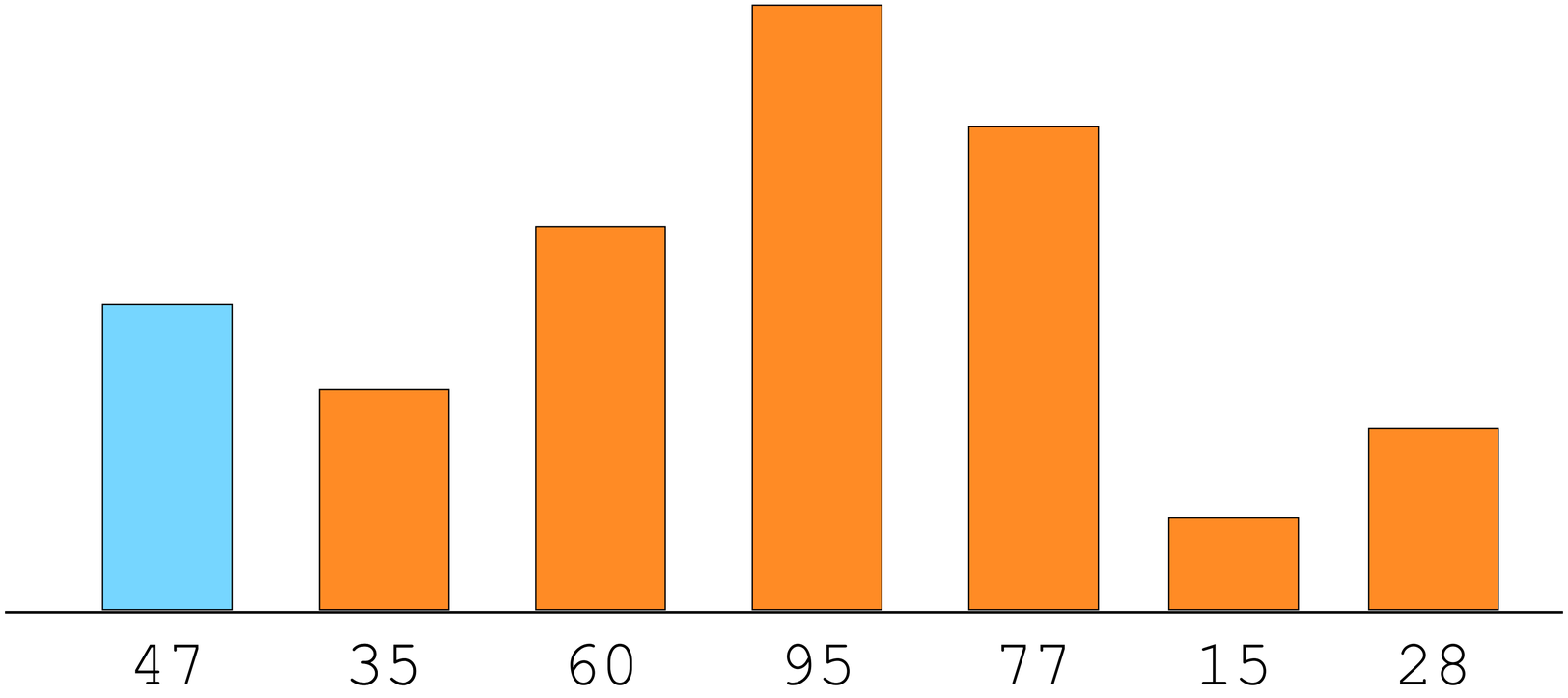
插入排序

维度	描述
时间复杂度	最坏：$O(n^2)$，最好：$O(n)$，平均：$O(n^2)$
空间复杂度	$O(1)$
算法稳定性	稳定
适用场景	适合数据规模小或部分有序的情况，常用于教学或小规模数据的处理

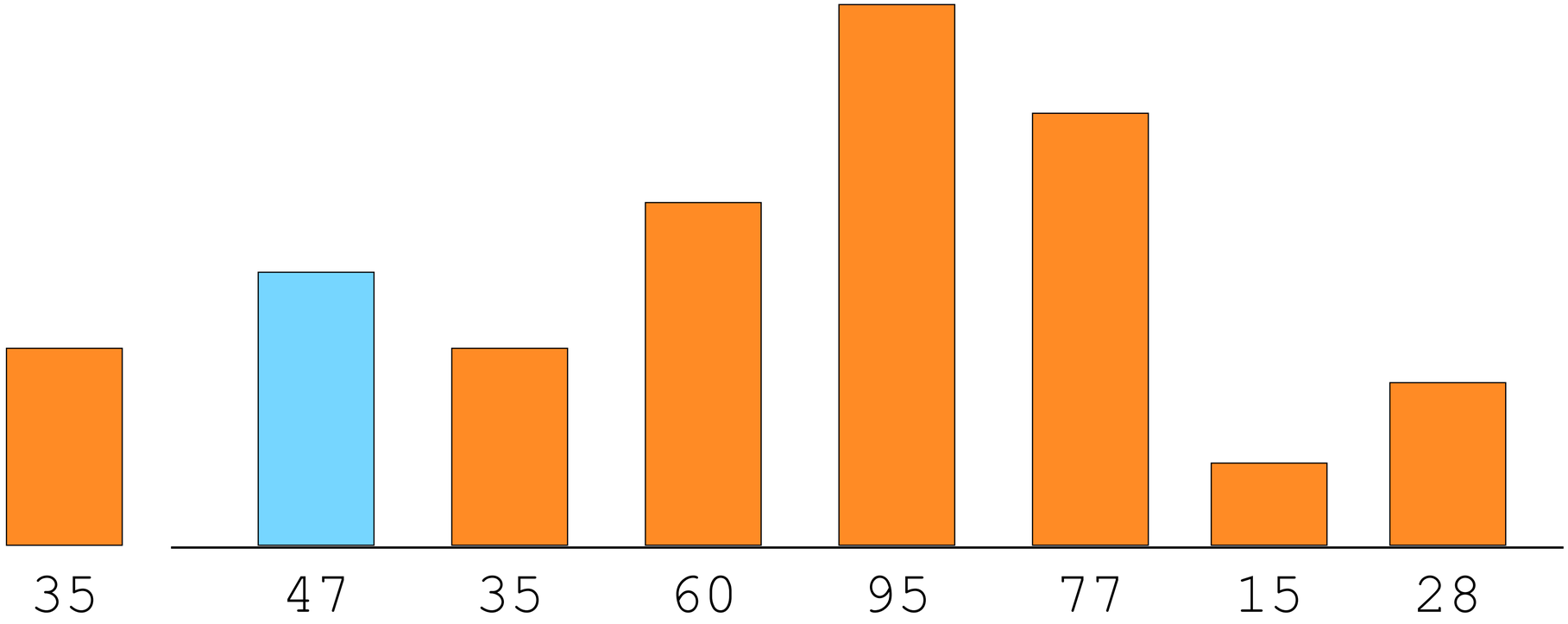
折半插入排序



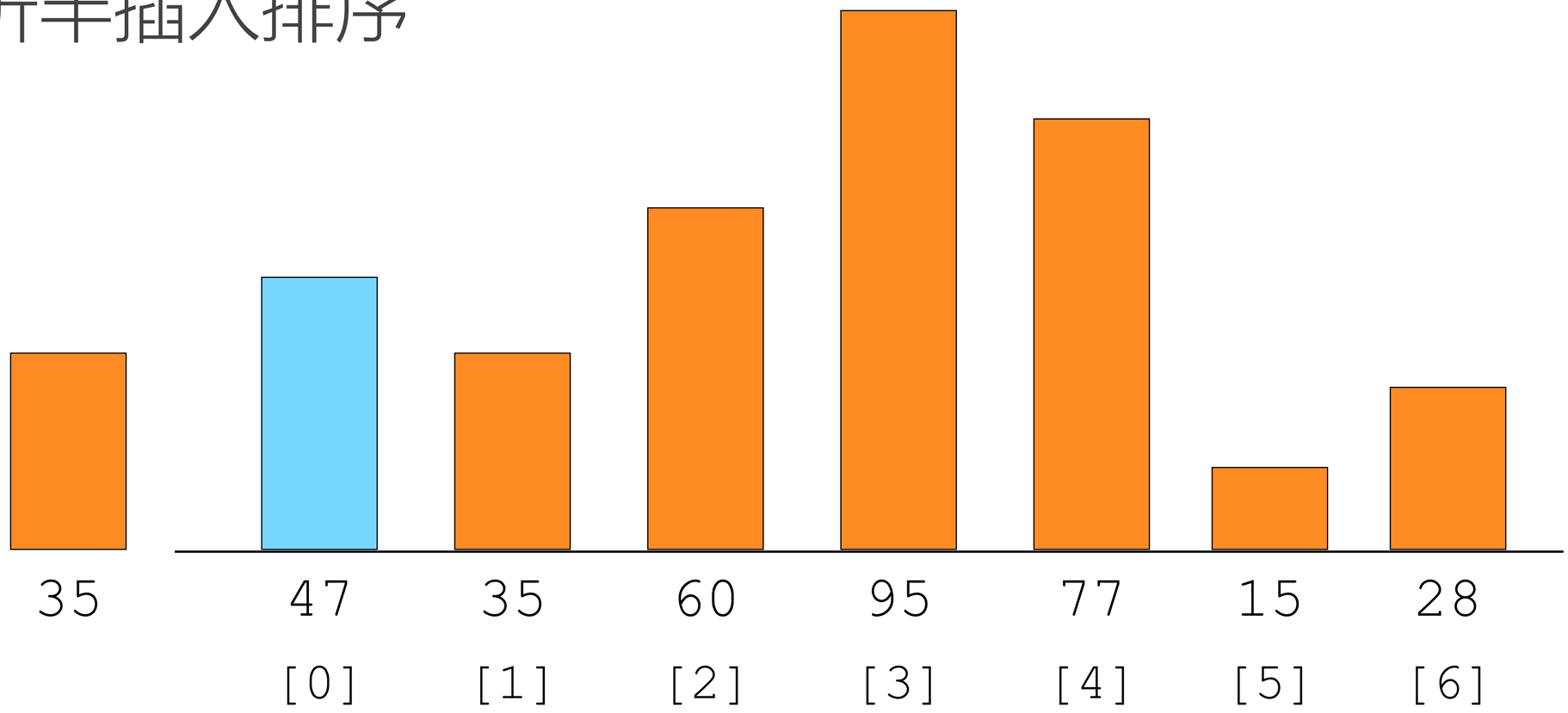
折半插入排序



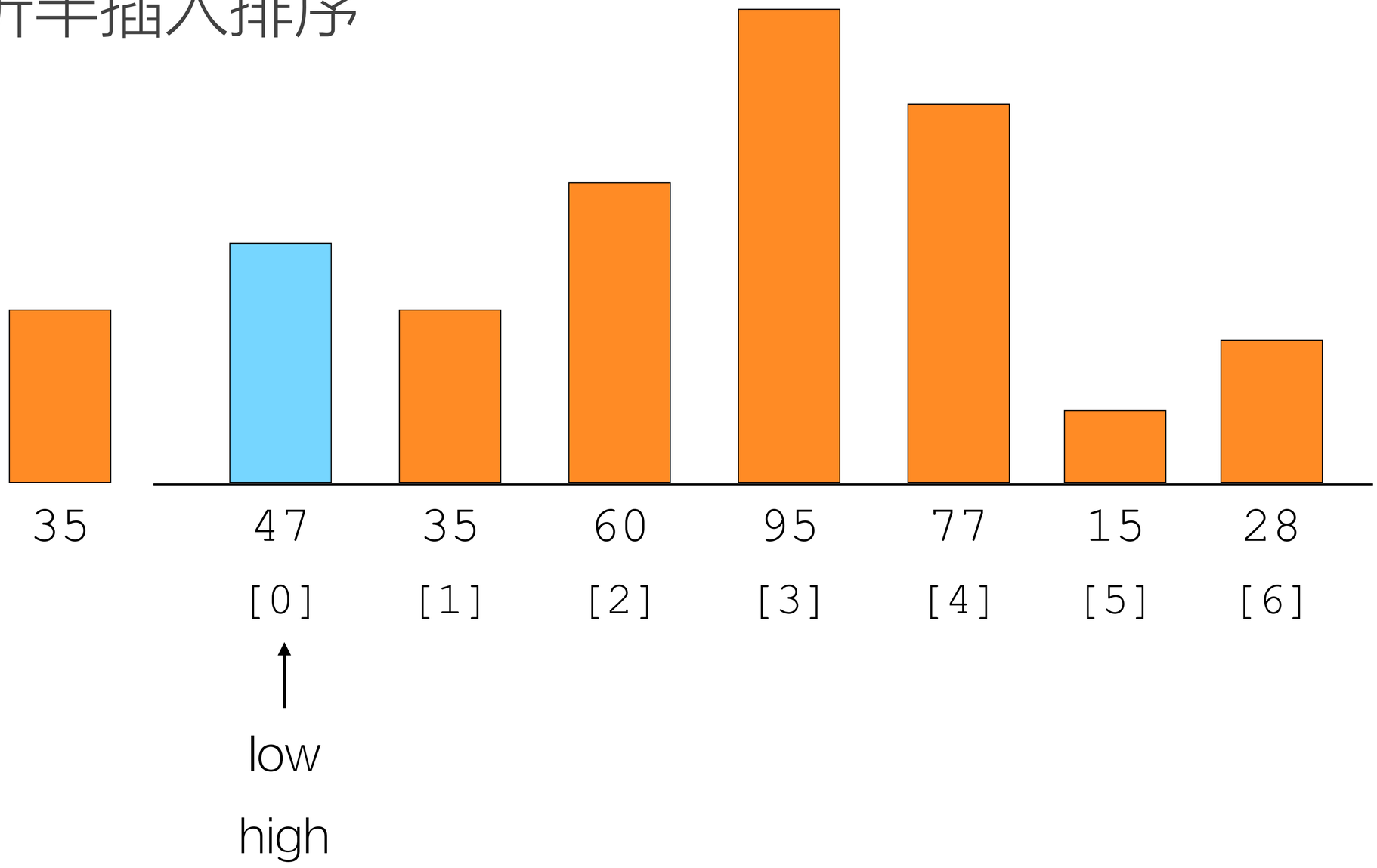
折半插入排序



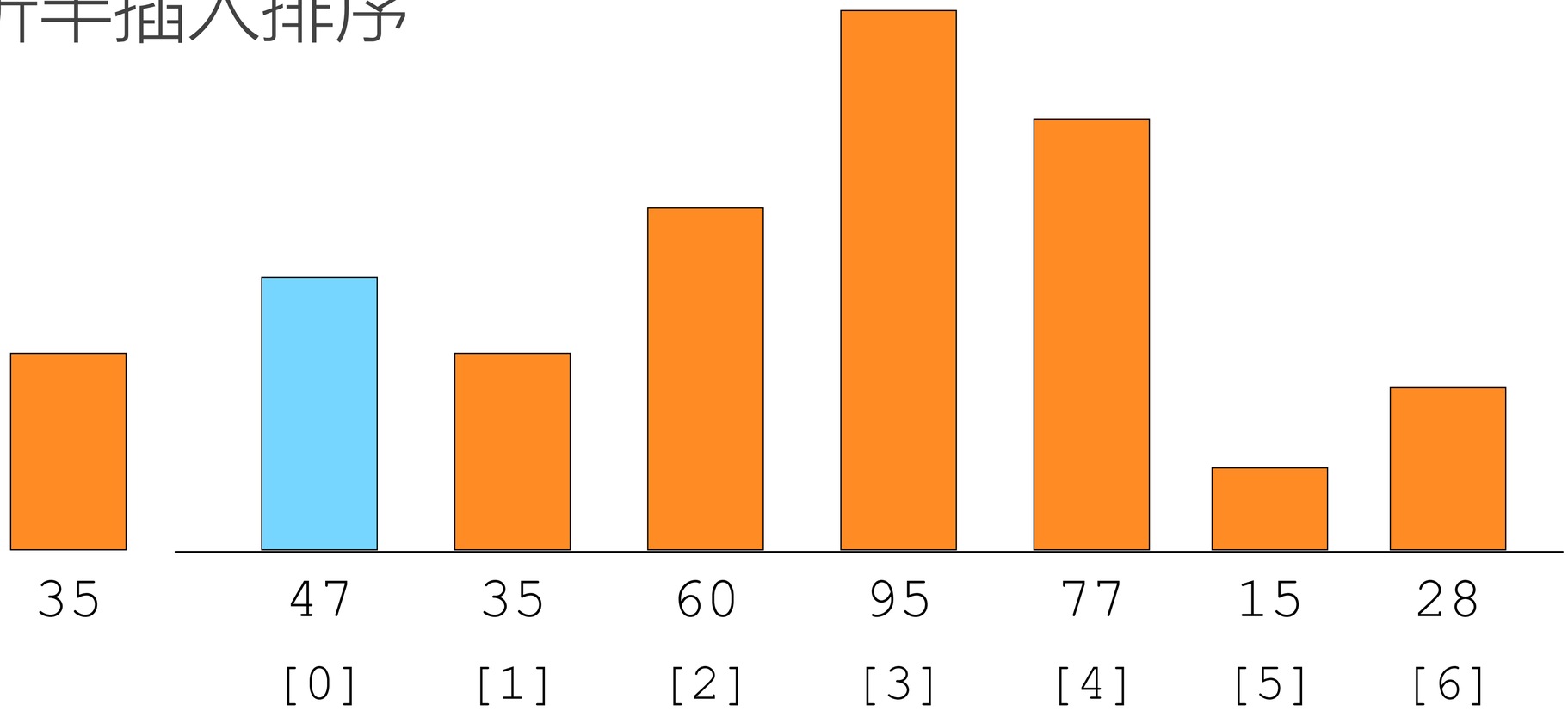
折半插入排序



折半插入排序

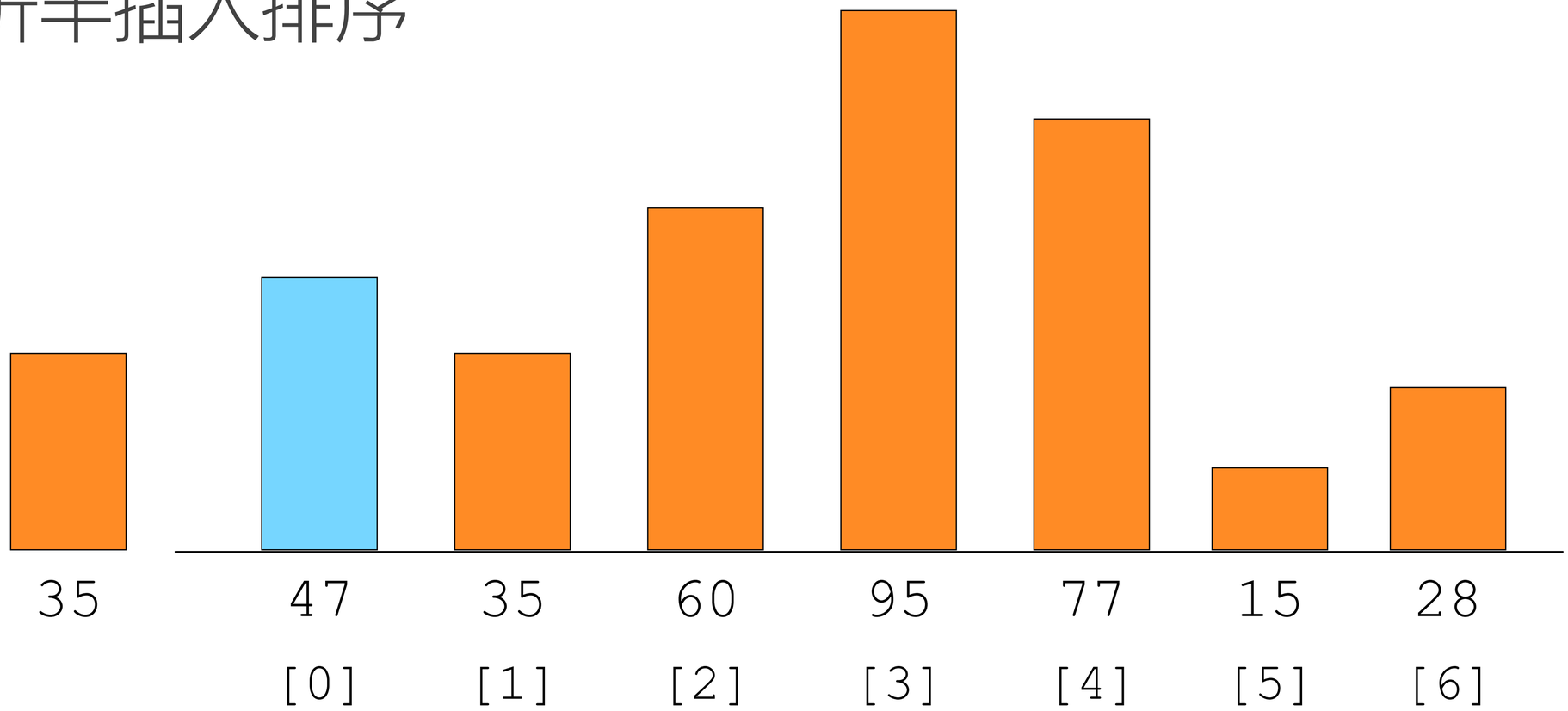


折半插入排序



mid = (low + high) / 2

折半插入排序



low

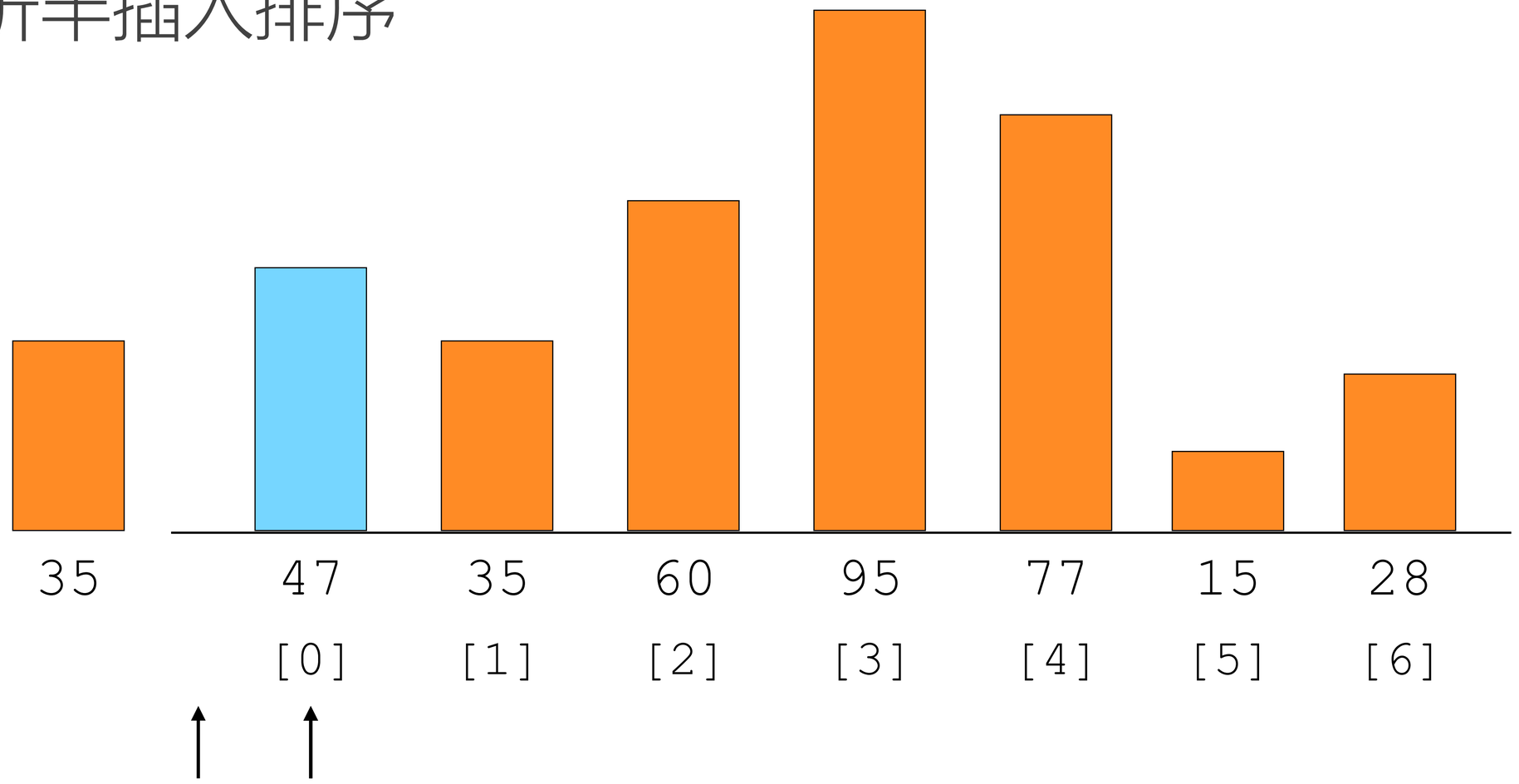
high

mid

mid = (low + high) / 2

data[mid] > 35 ? high = mid − 1 : low = mid + 1

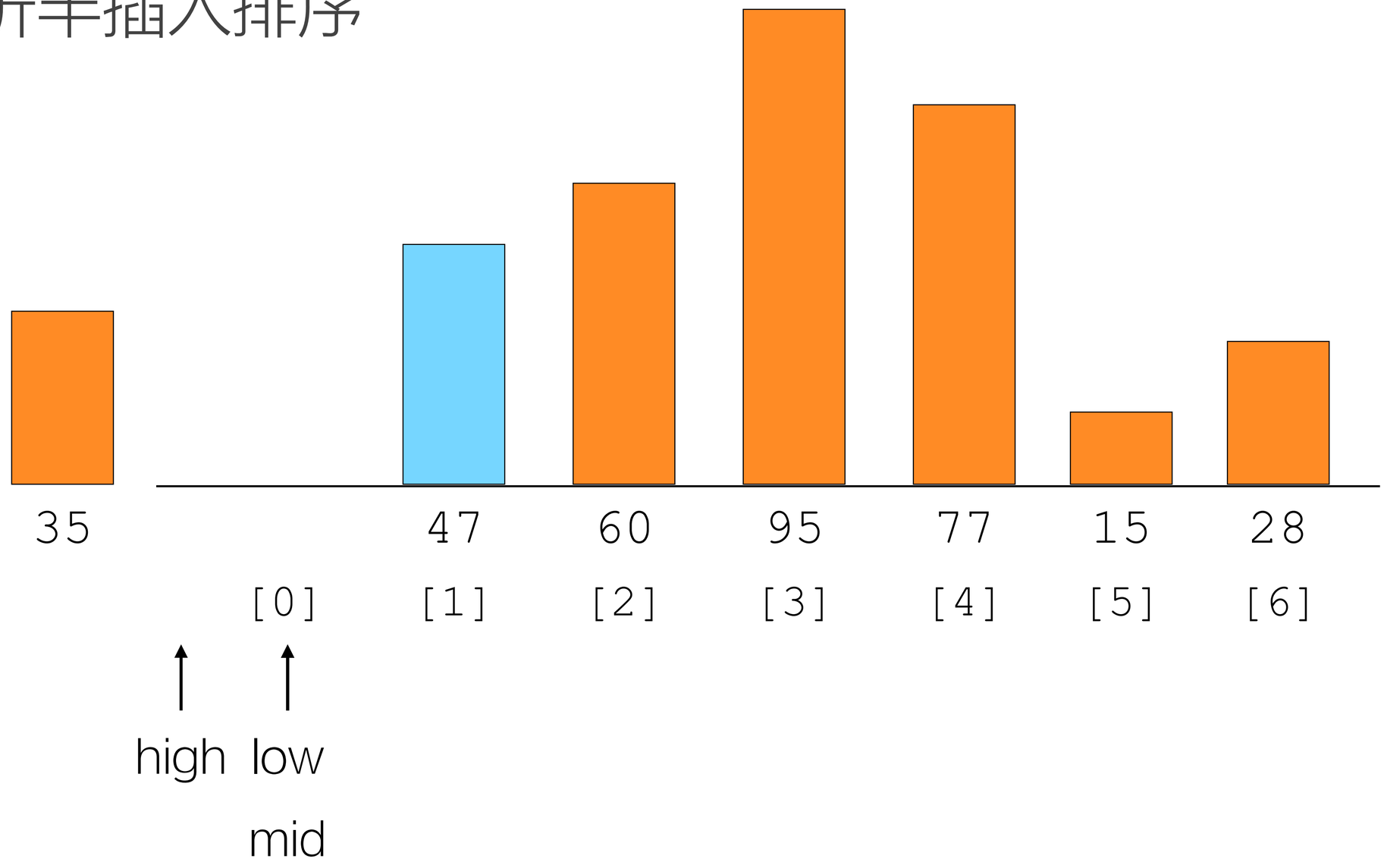
折半插入排序



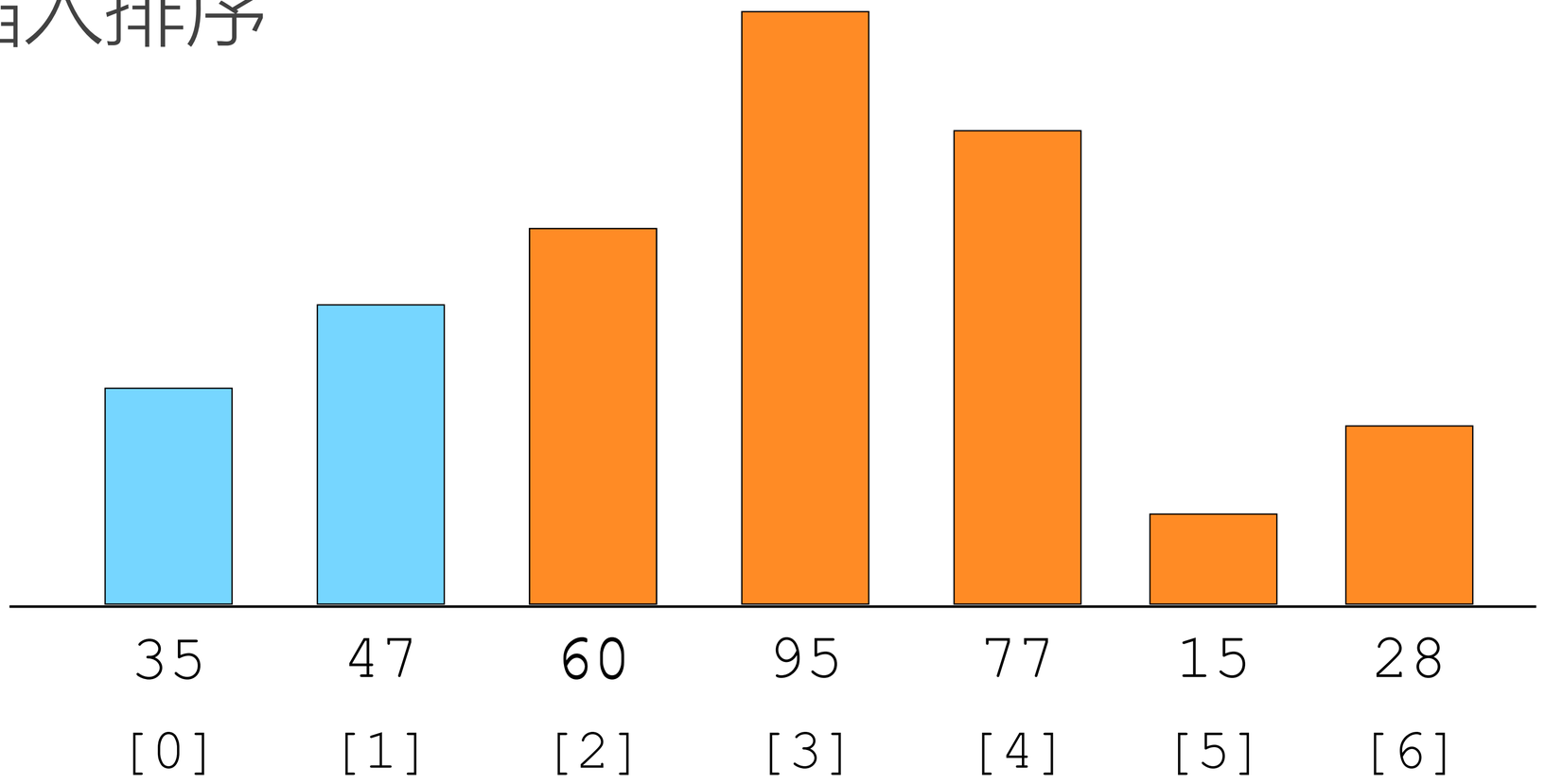
mid = (low + high) / 2

data[mid] > 35 ? high = mid − 1 : low = mid + 1

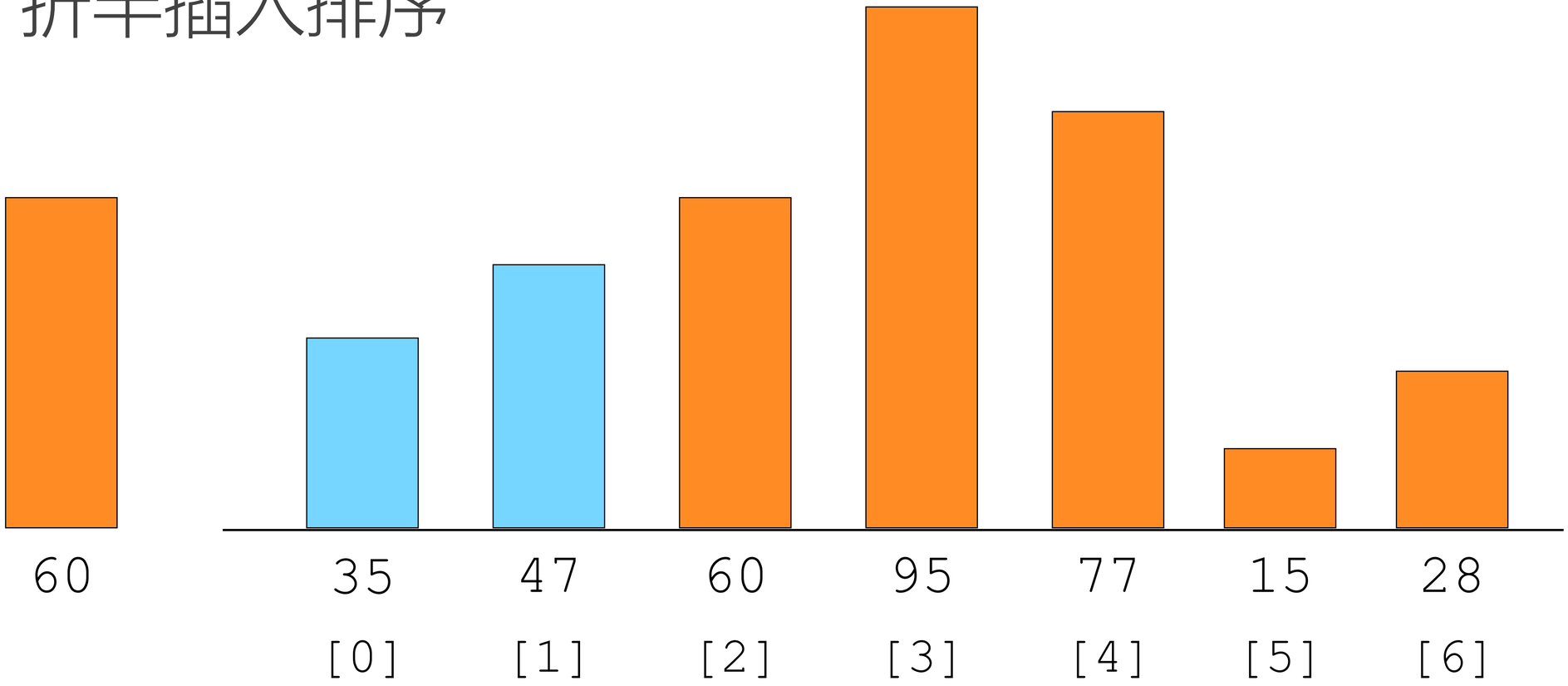
折半插入排序



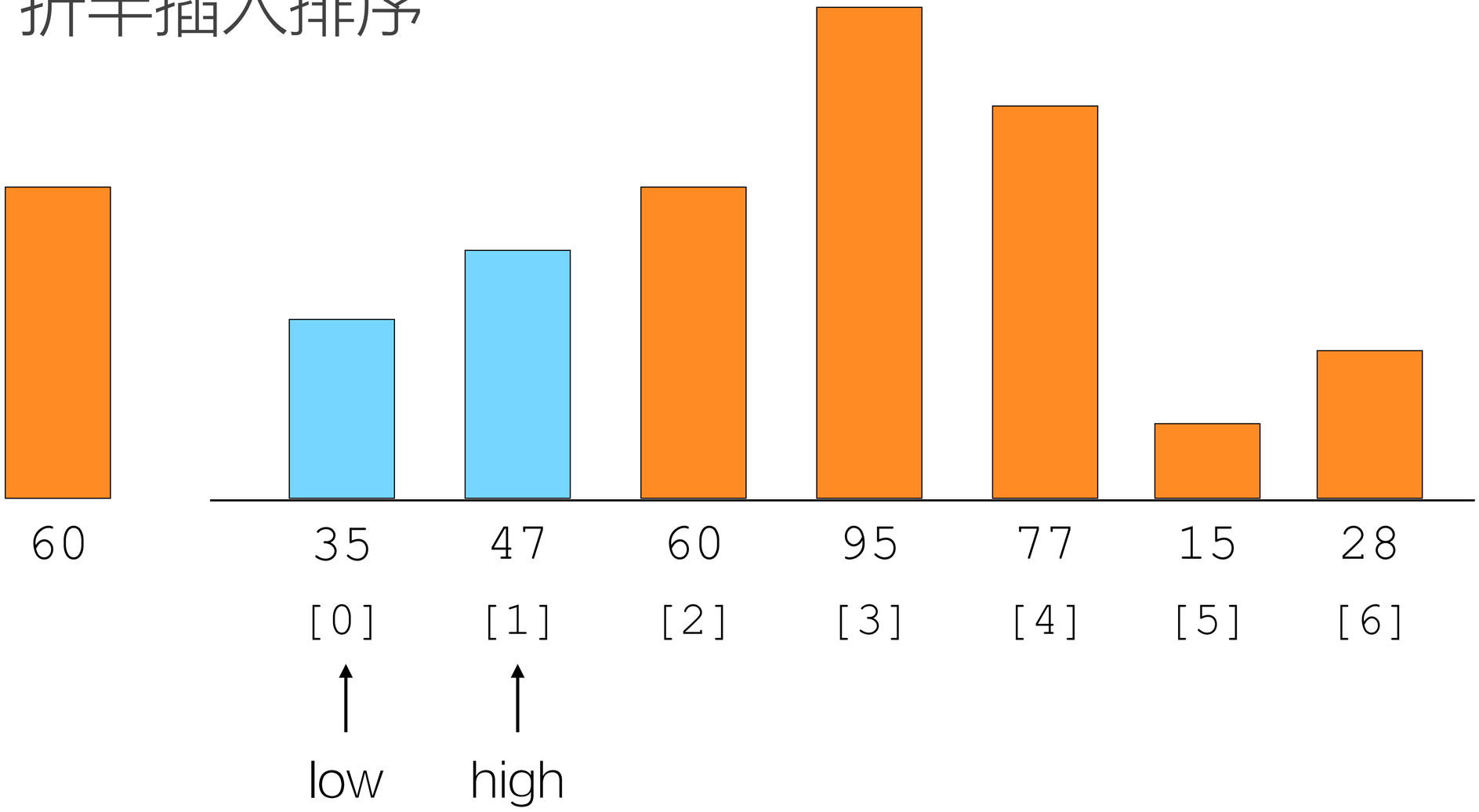
折半插入排序



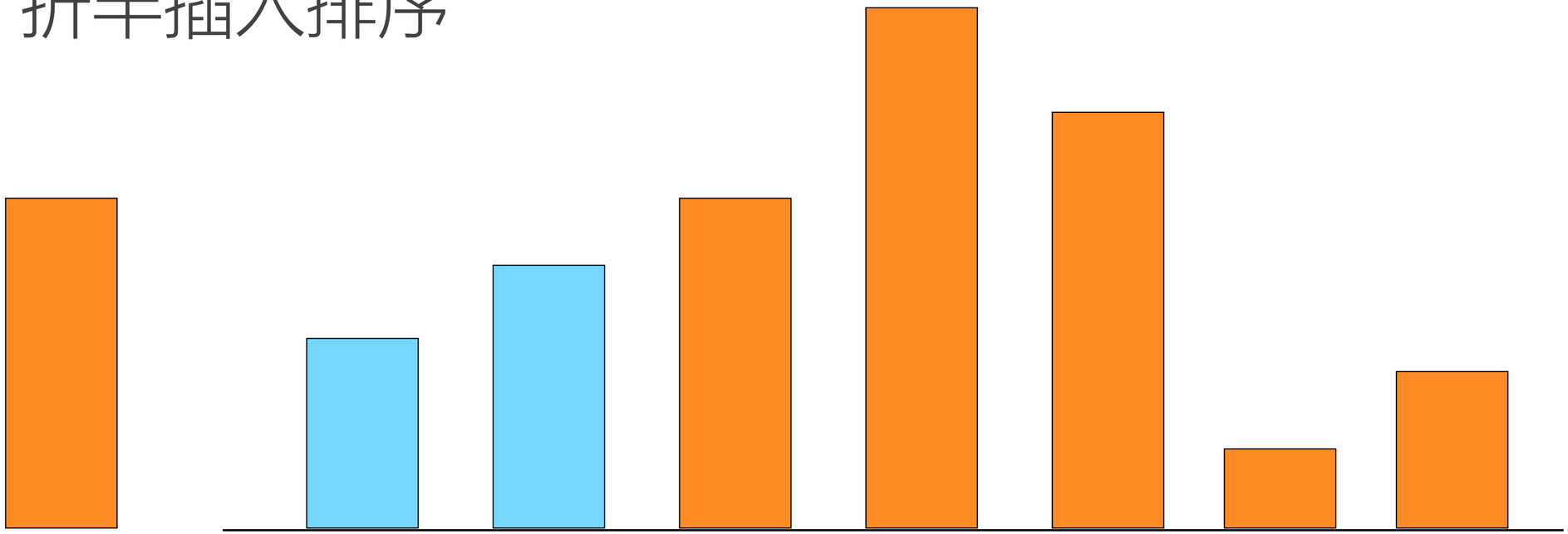
折半插入排序



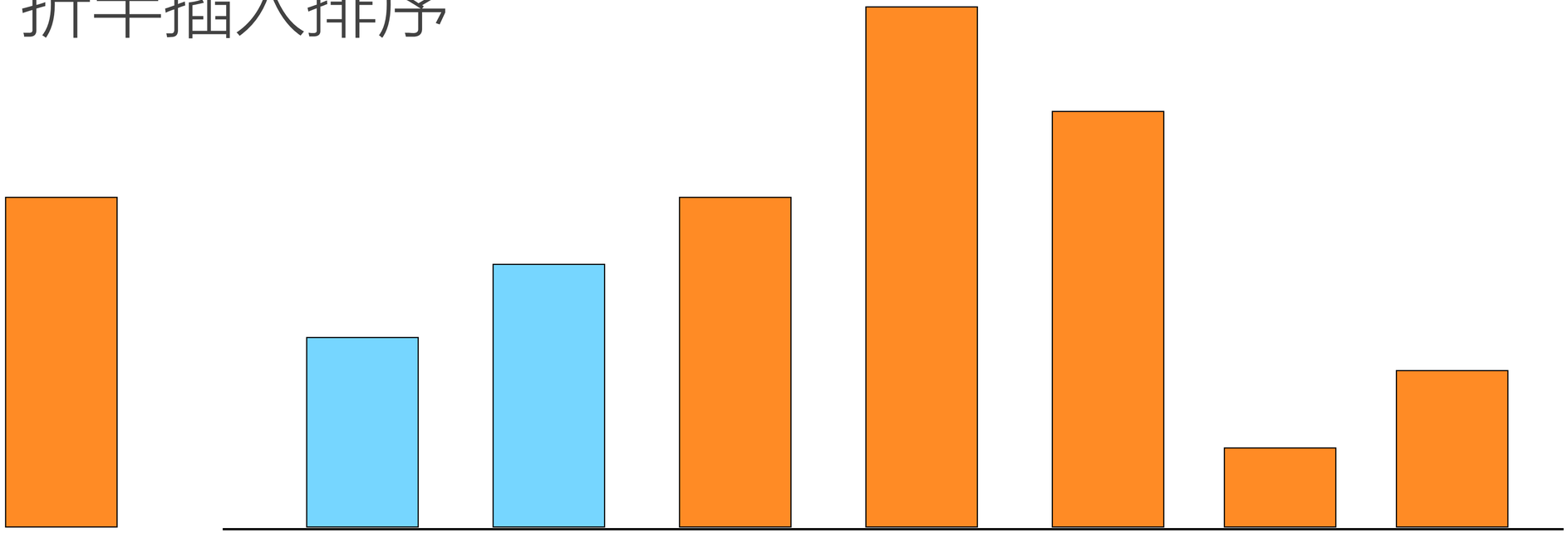
折半插入排序



折半插入排序



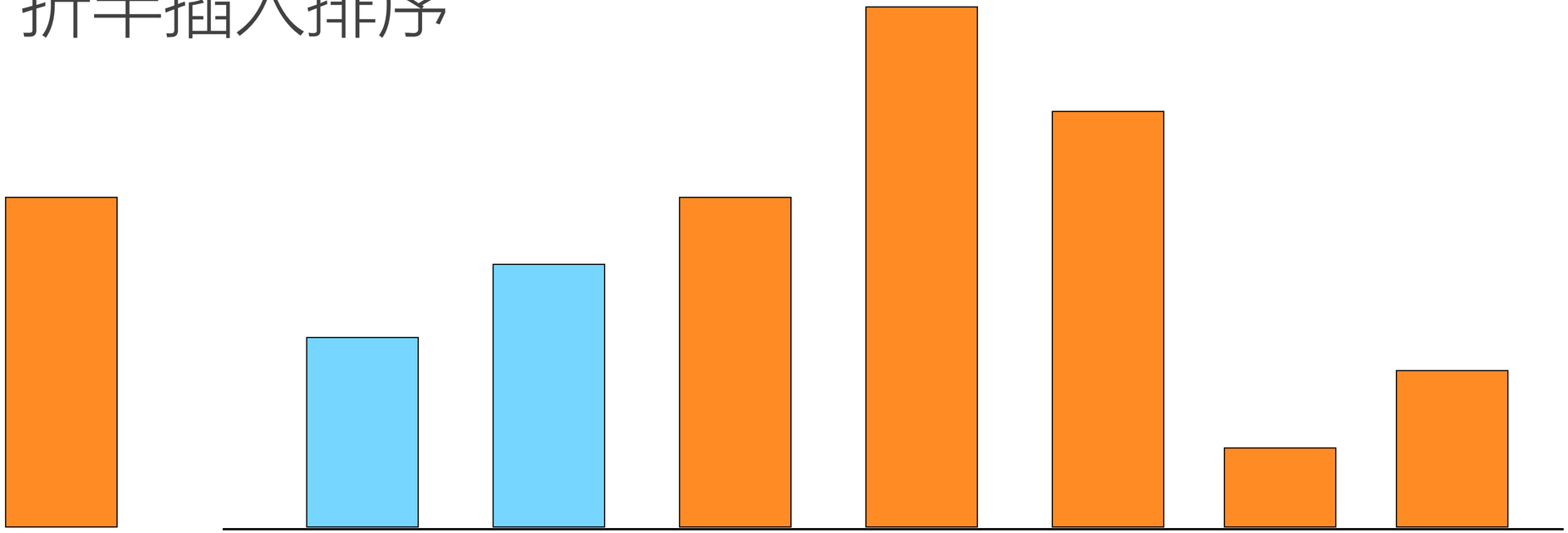
折半插入排序



mid = (low + high) / 2

data[mid] > 60 ? high = mid − 1 : low = mid + 1

折半插入排序



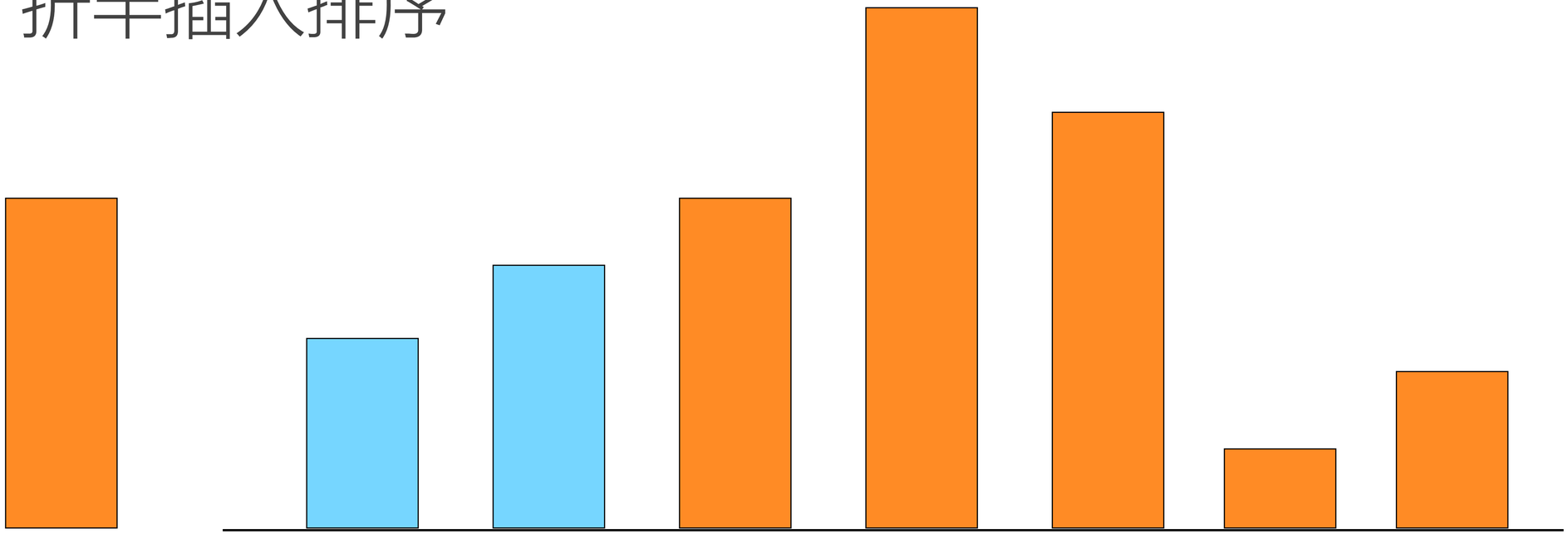
mid high

low

mid = (low + high) / 2

data[mid] > 60 ? high = mid − 1 : low = mid + 1

折半插入排序



high

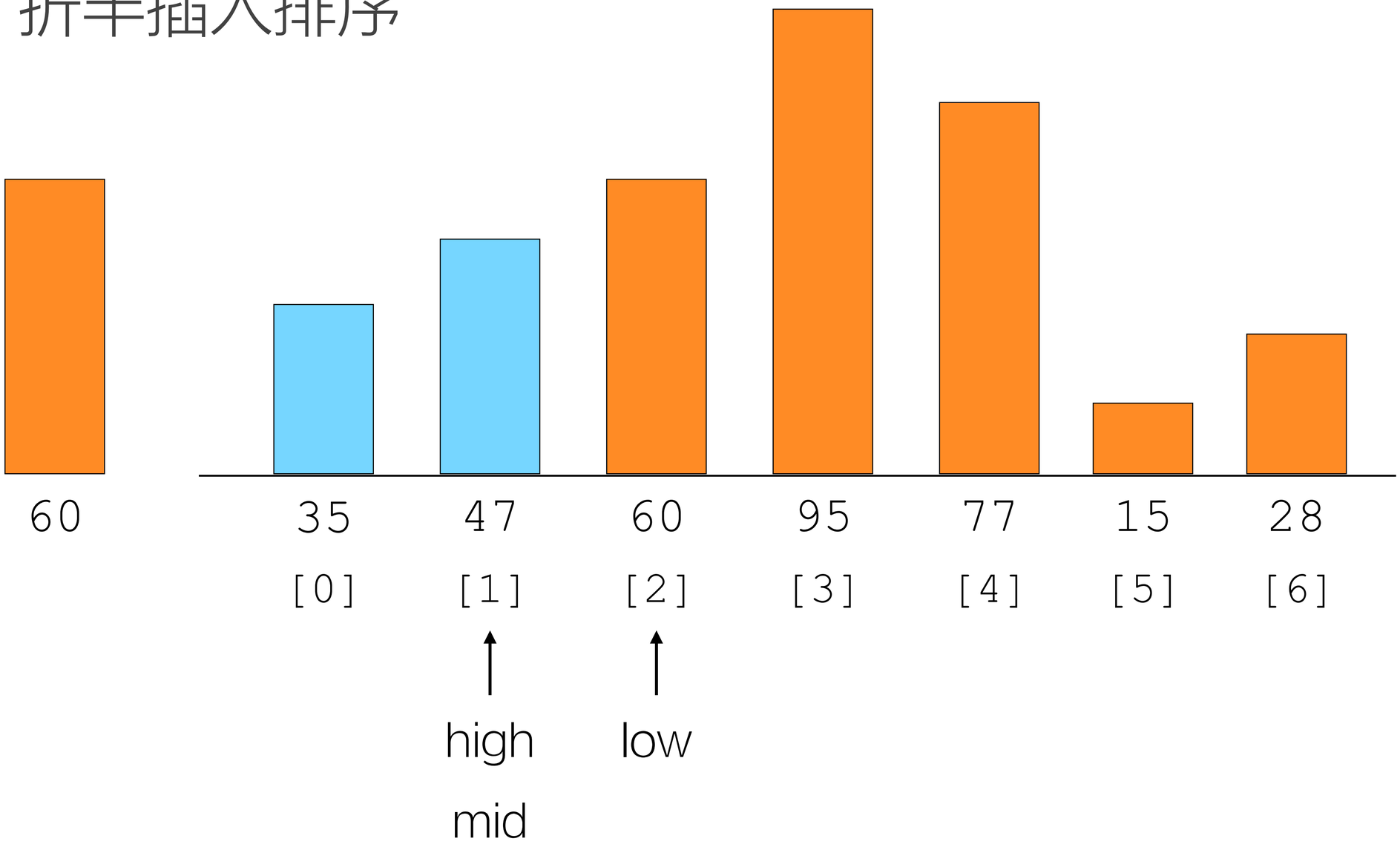
low

mid

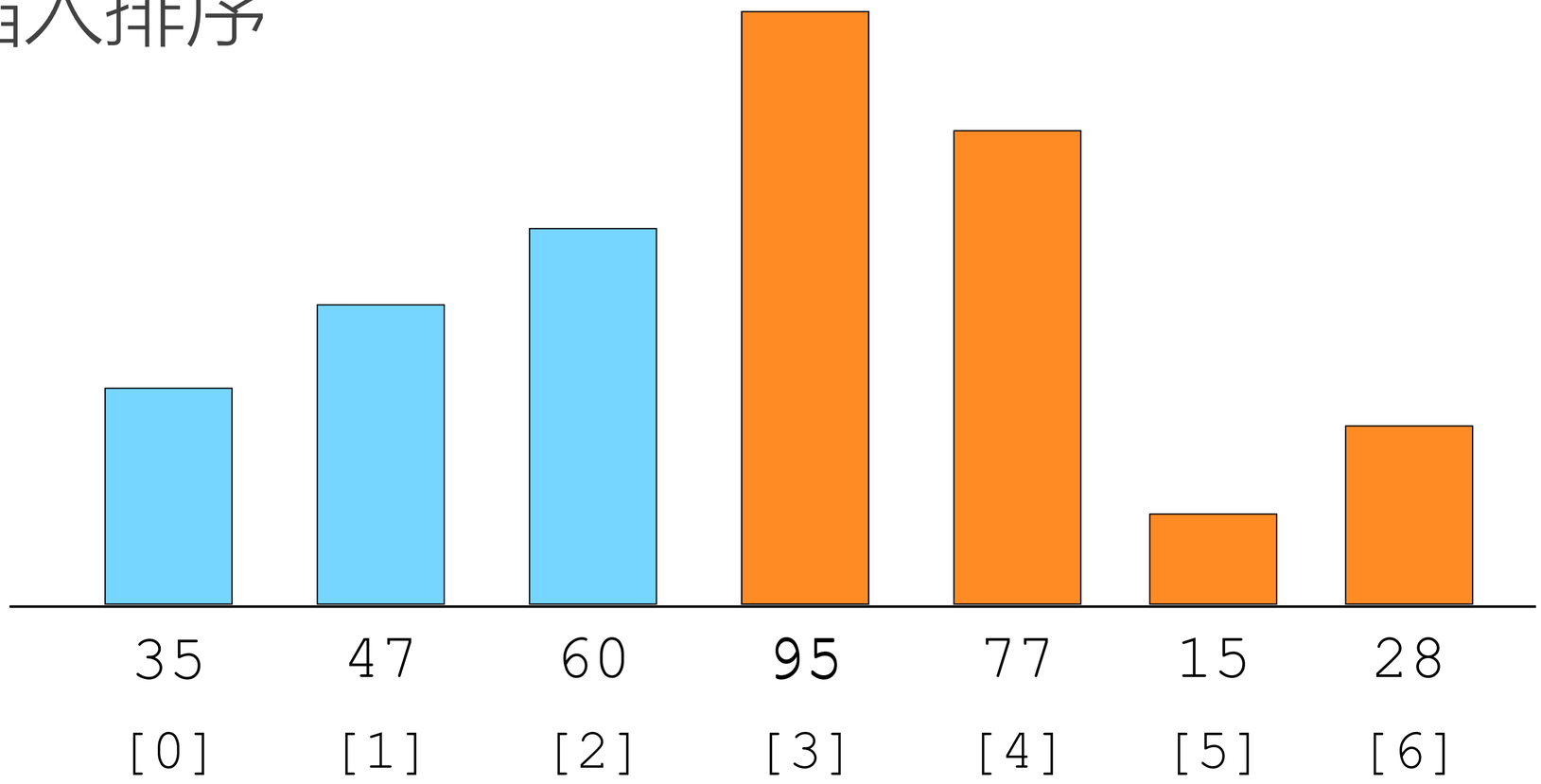
mid = (low + high) / 2

data[mid] > 60 ? high = mid − 1 : low = mid + 1

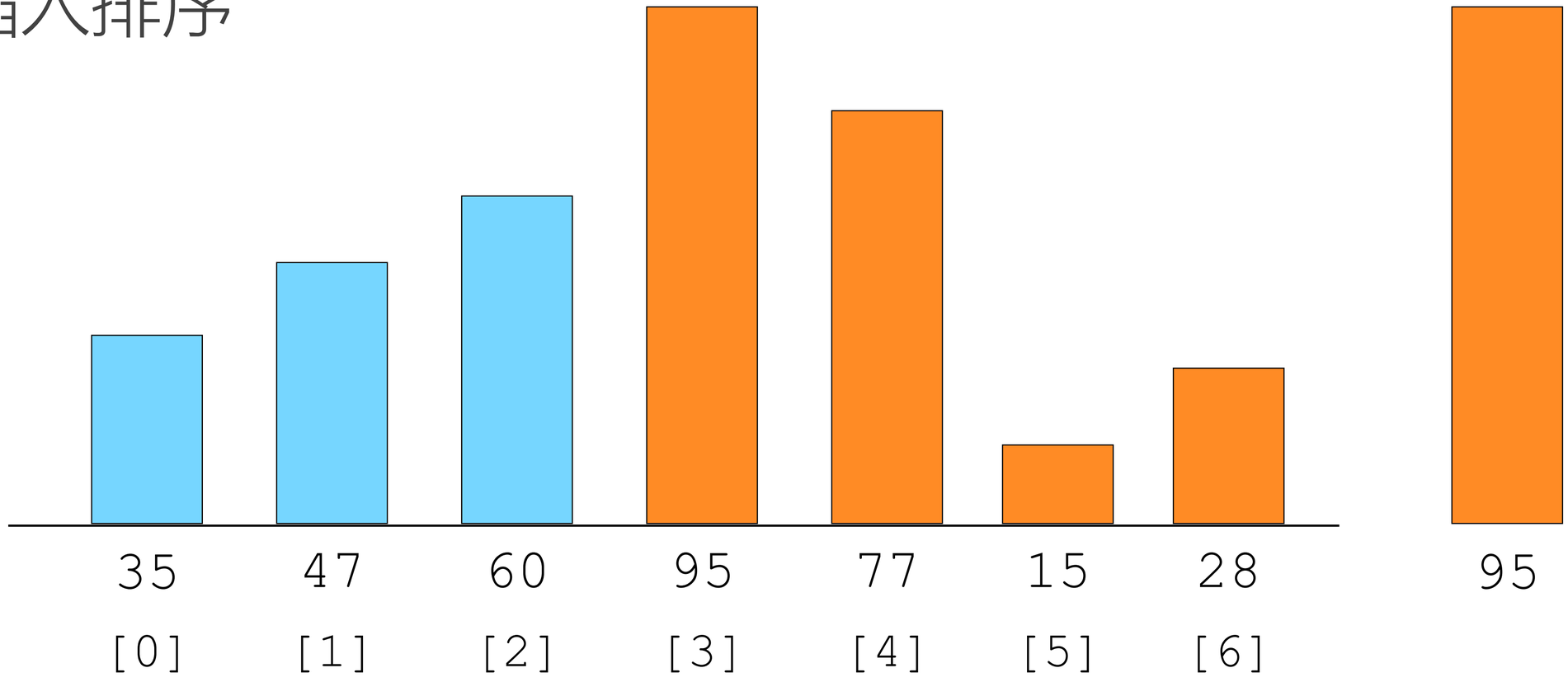
折半插入排序



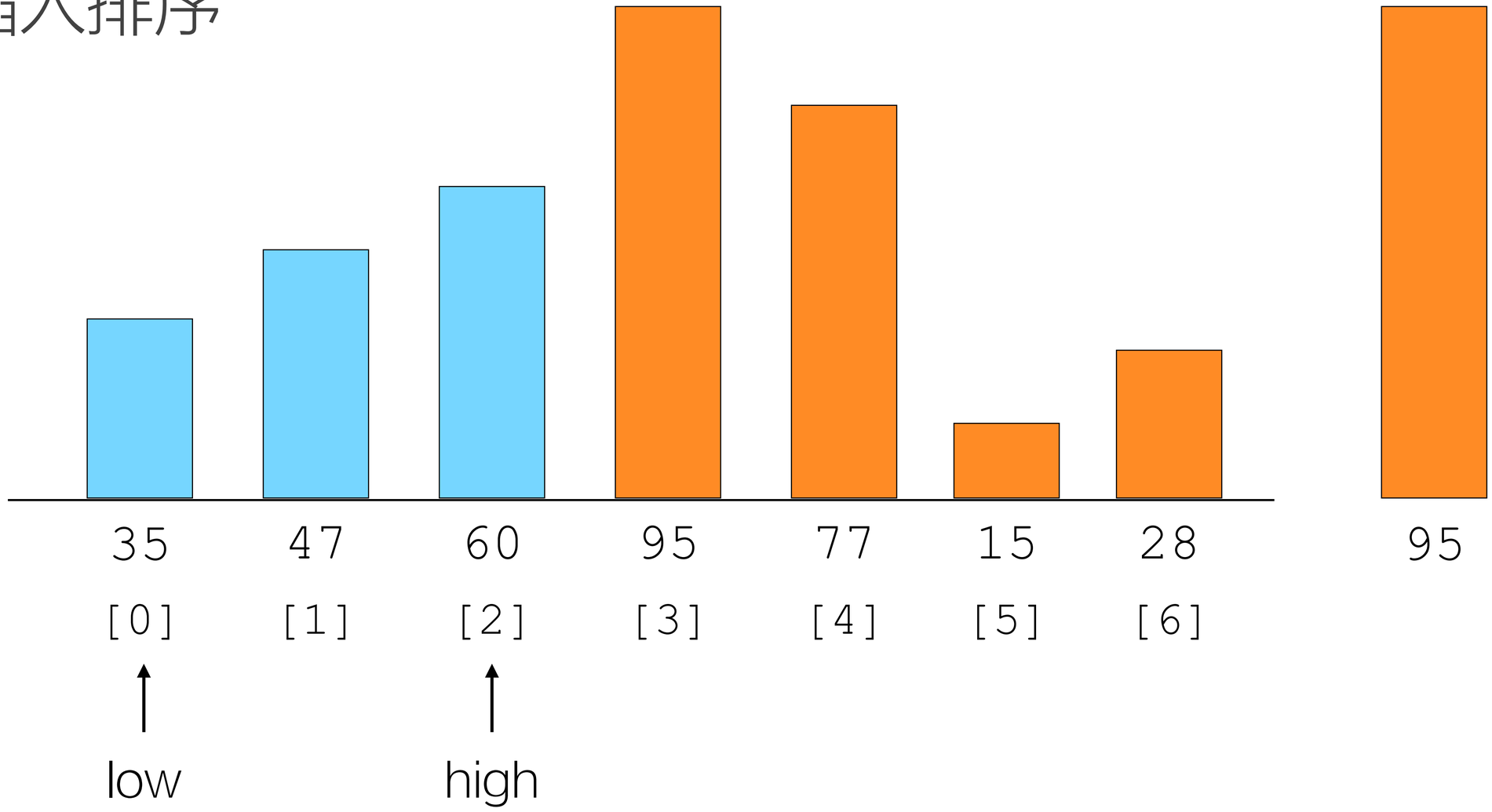
折半插入排序



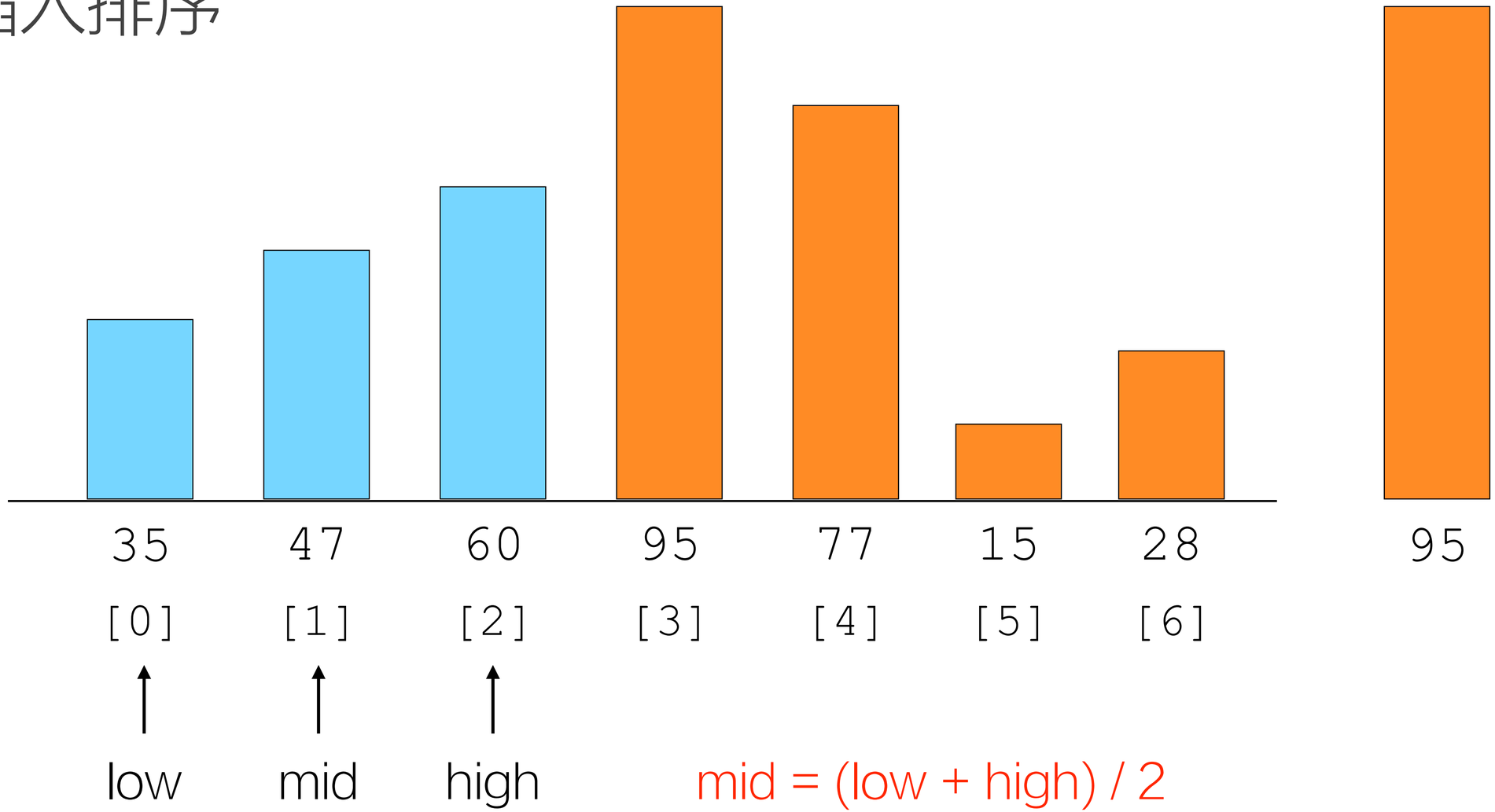
折半插入排序



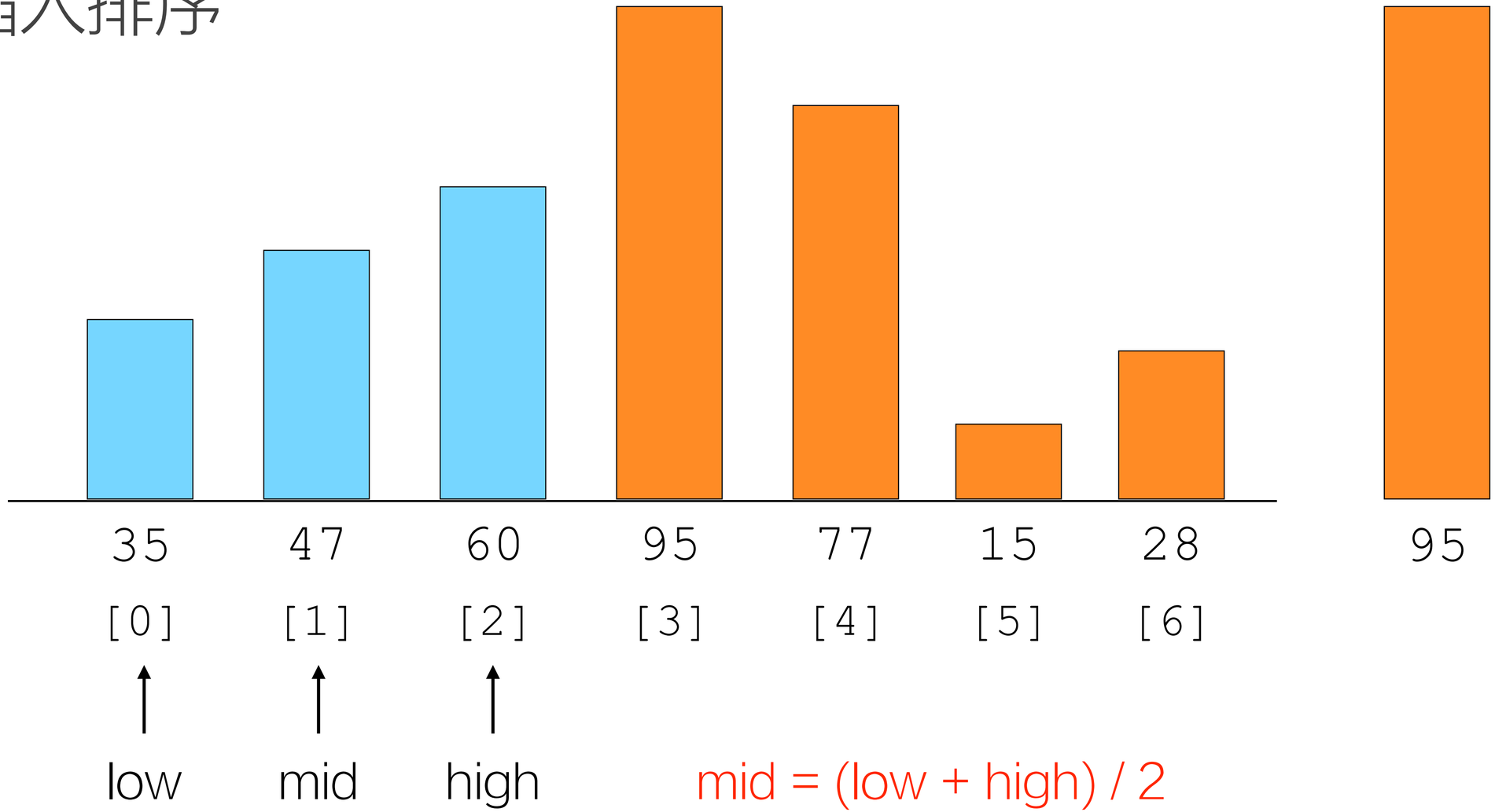
折半插入排序



折半插入排序



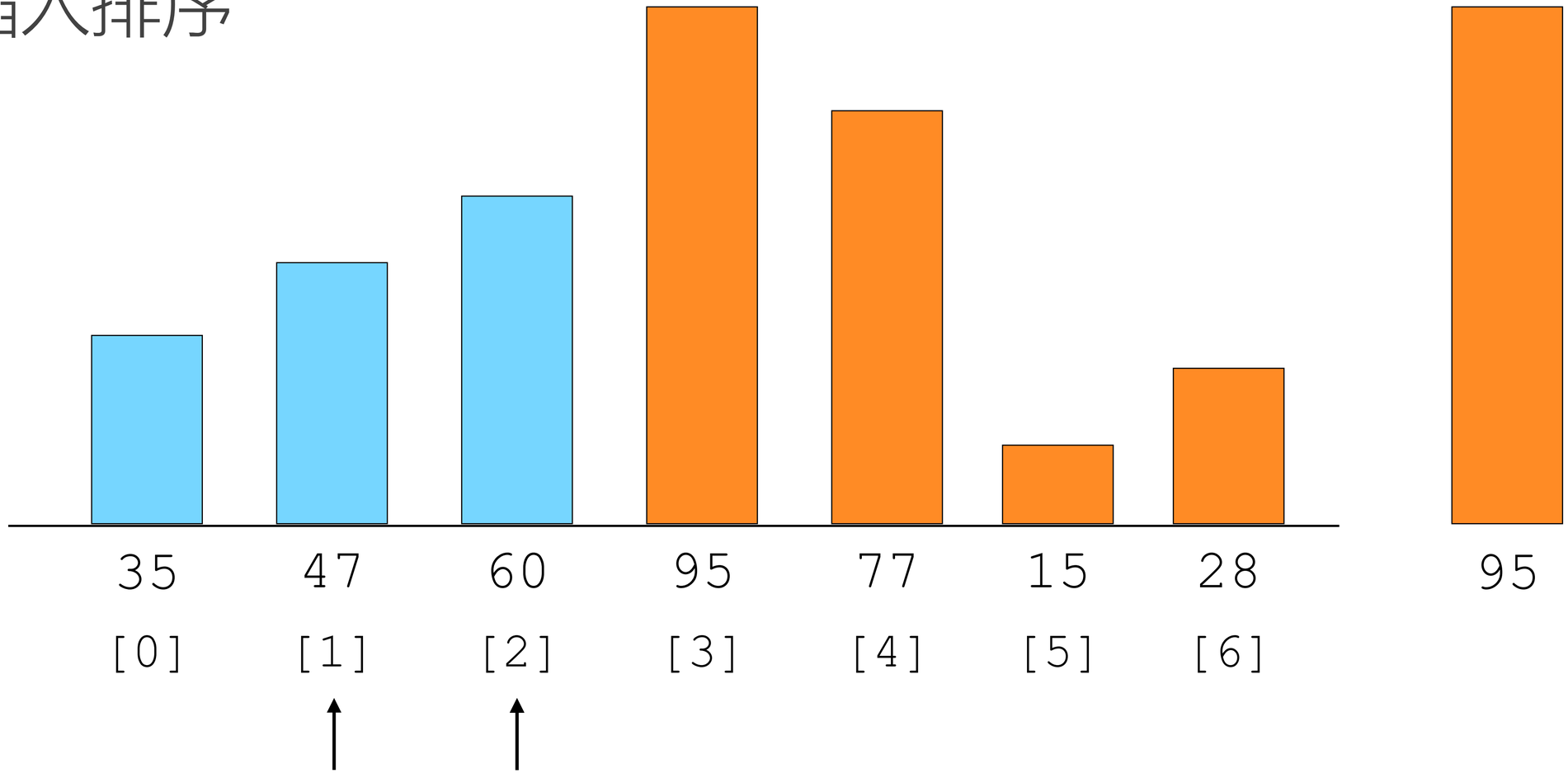
折半插入排序



mid = (low + high) / 2

data[mid] > 95 ? high = mid − 1 : low = mid + 1

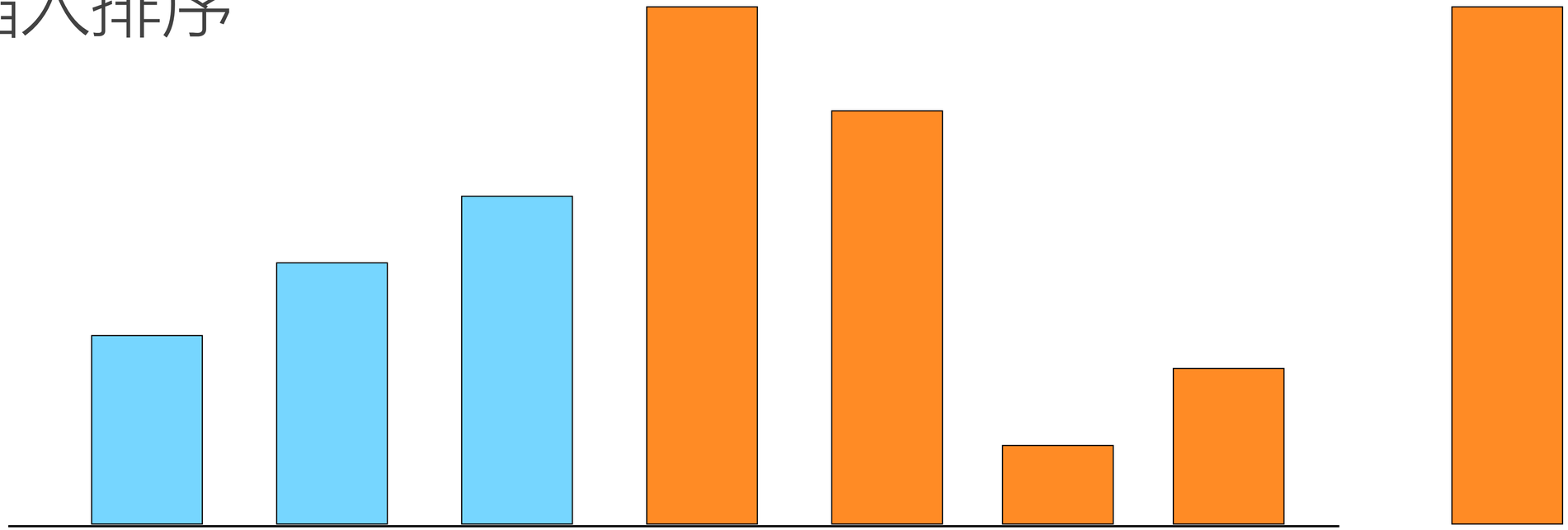
折半插入排序



mid = (low + high) / 2

data[mid] > 95 ? high = mid − 1 : low = mid + 1

折半插入排序



high

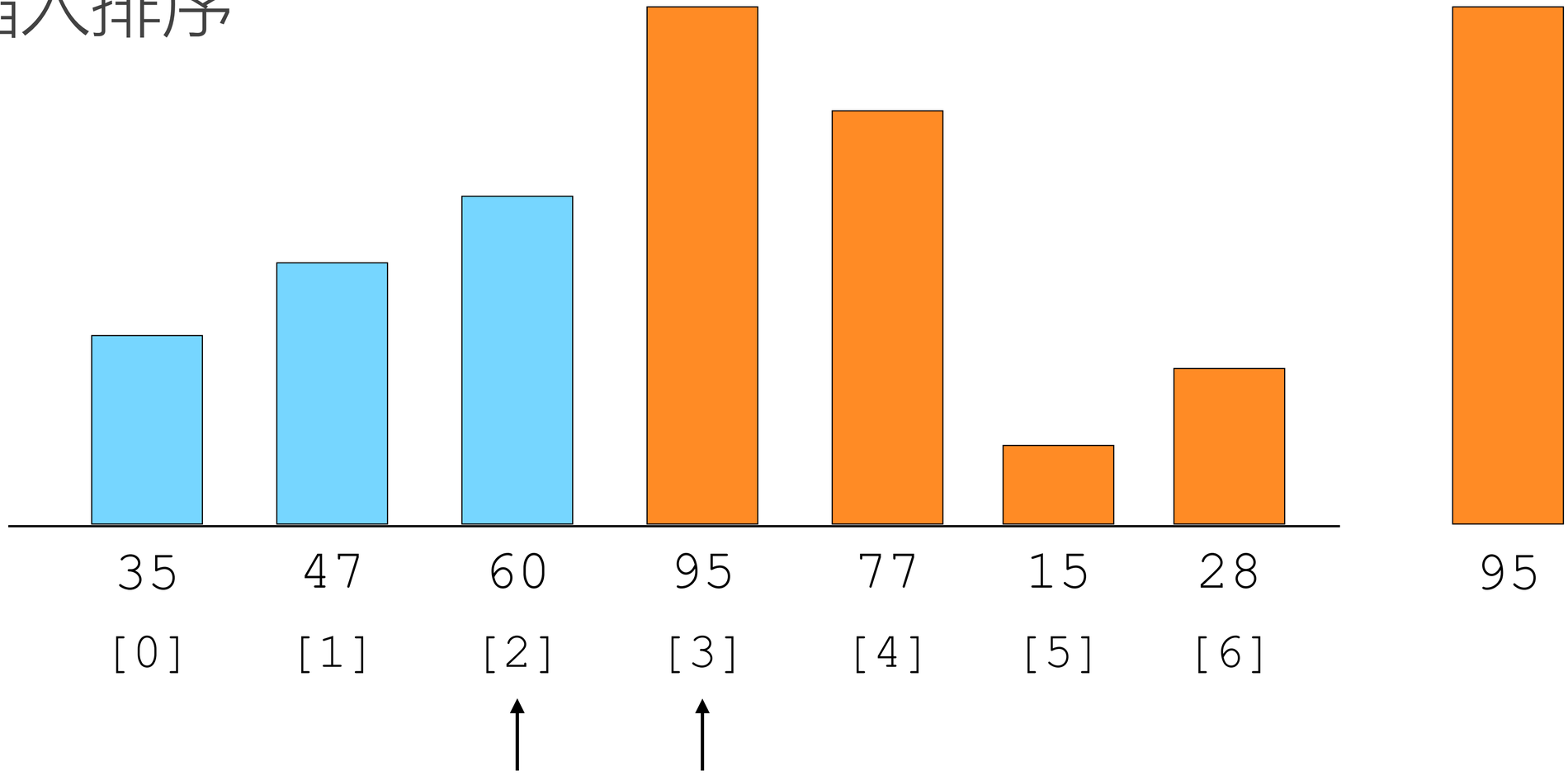
low

mid

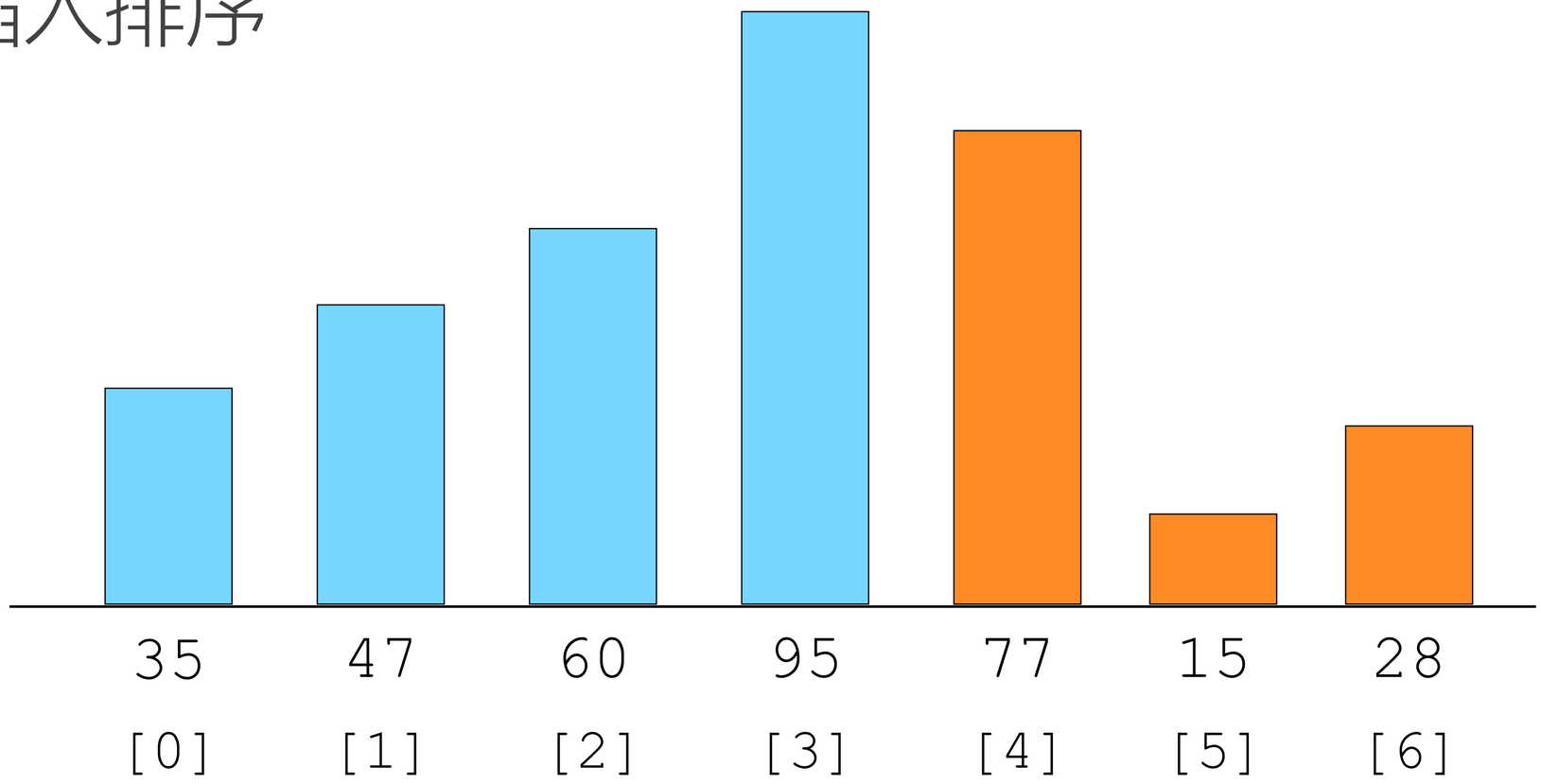
mid = (low + high) / 2

data[mid] > 95 ? high = mid − 1 : low = mid + 1

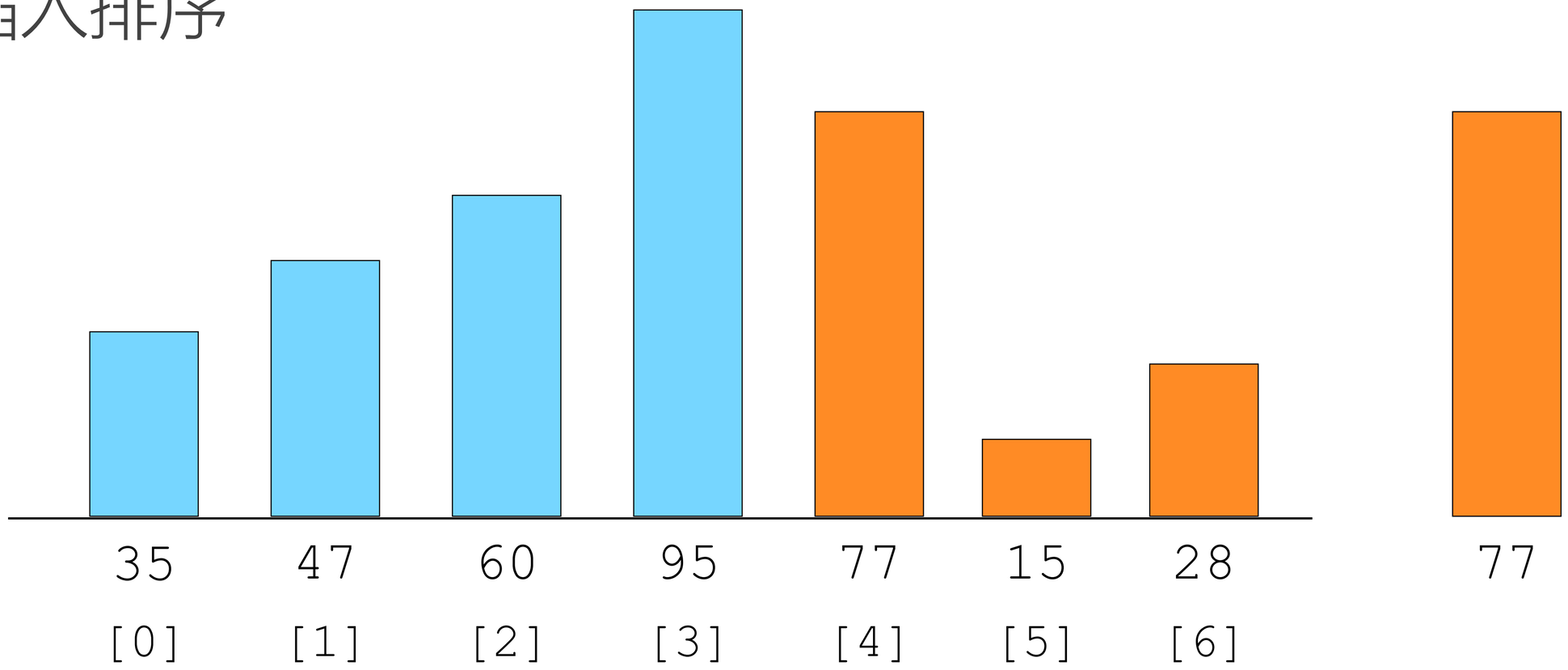
折半插入排序


35
47
60
95
77
15
28
95
[0]
[1]
[2]
[3]
[4]
[5]
[6]
high
low
mid
mid = (low + high) / 2
data[mid] > 95 ? high = mid − 1 : low = mid + 1

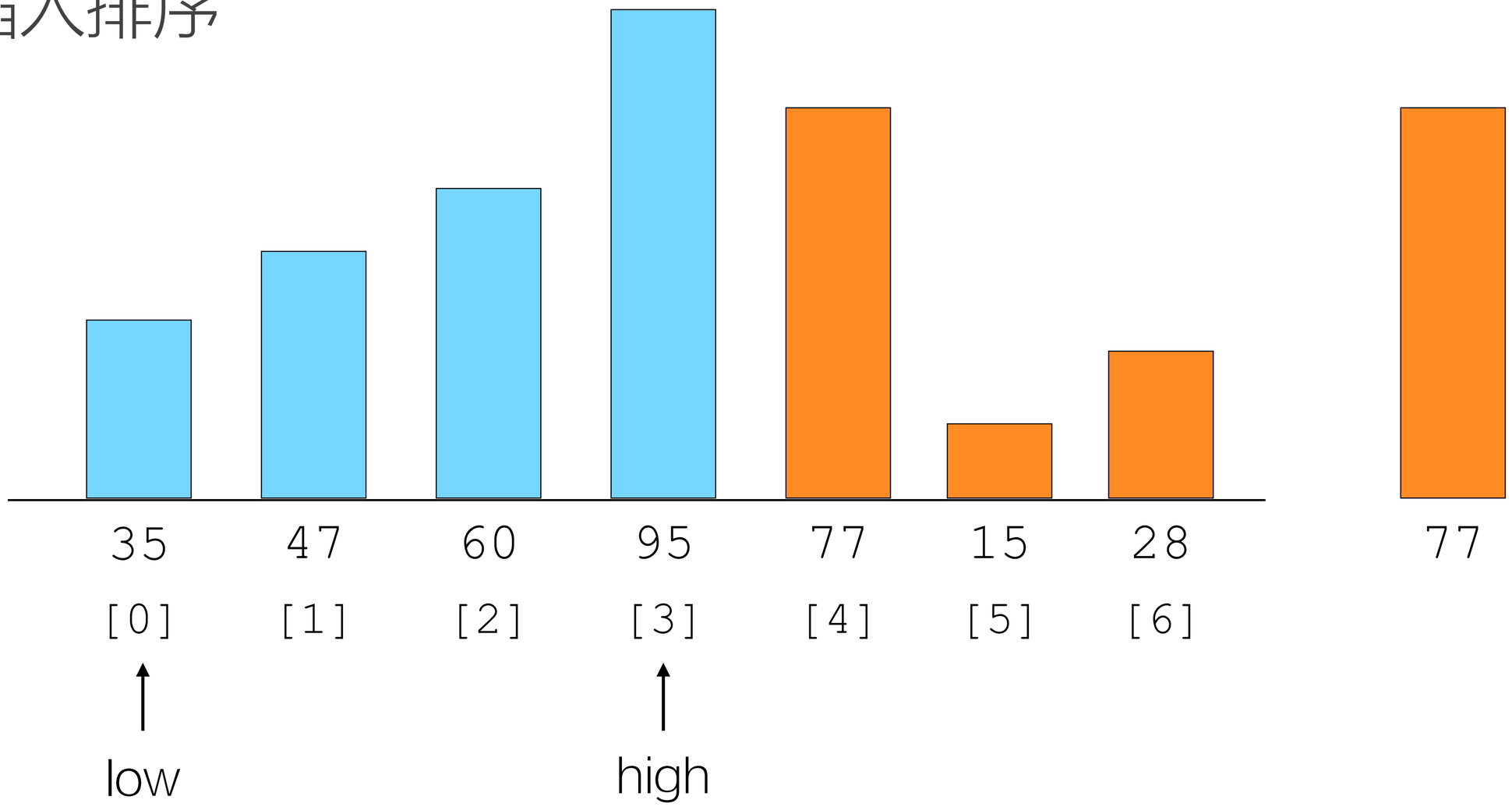
折半插入排序



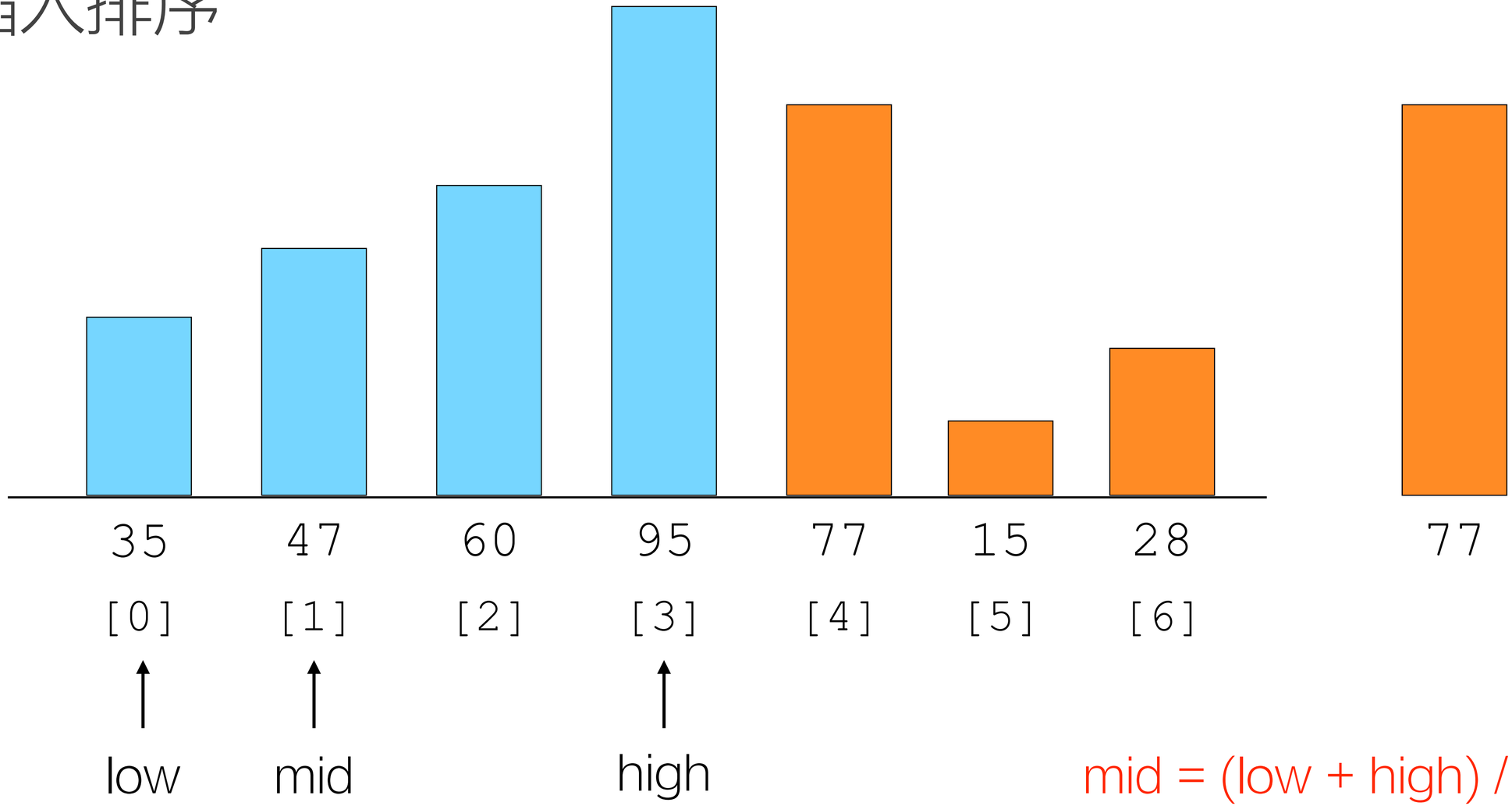
折半插入排序



折半插入排序

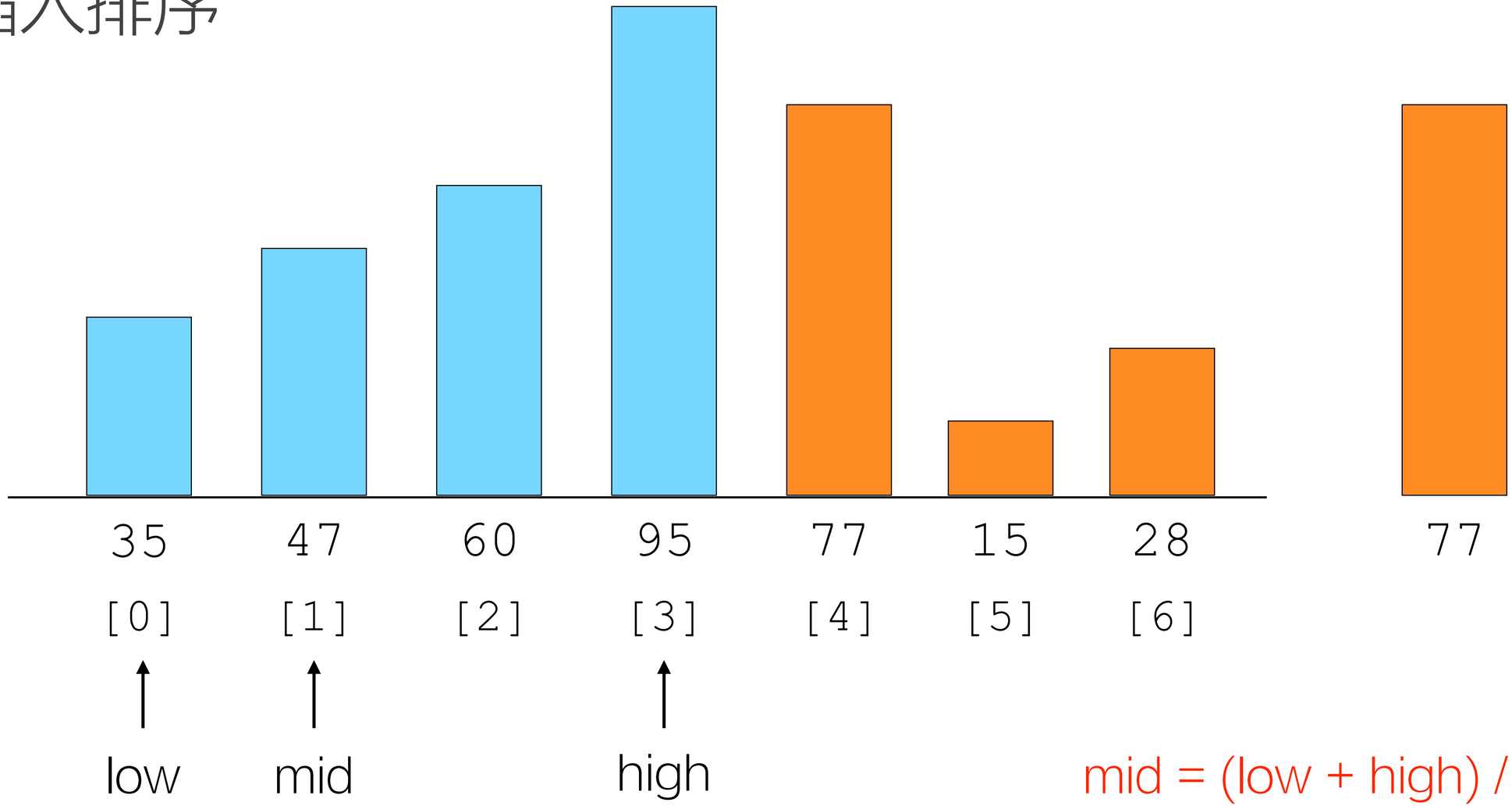


折半插入排序



mid = (low + high) / 2

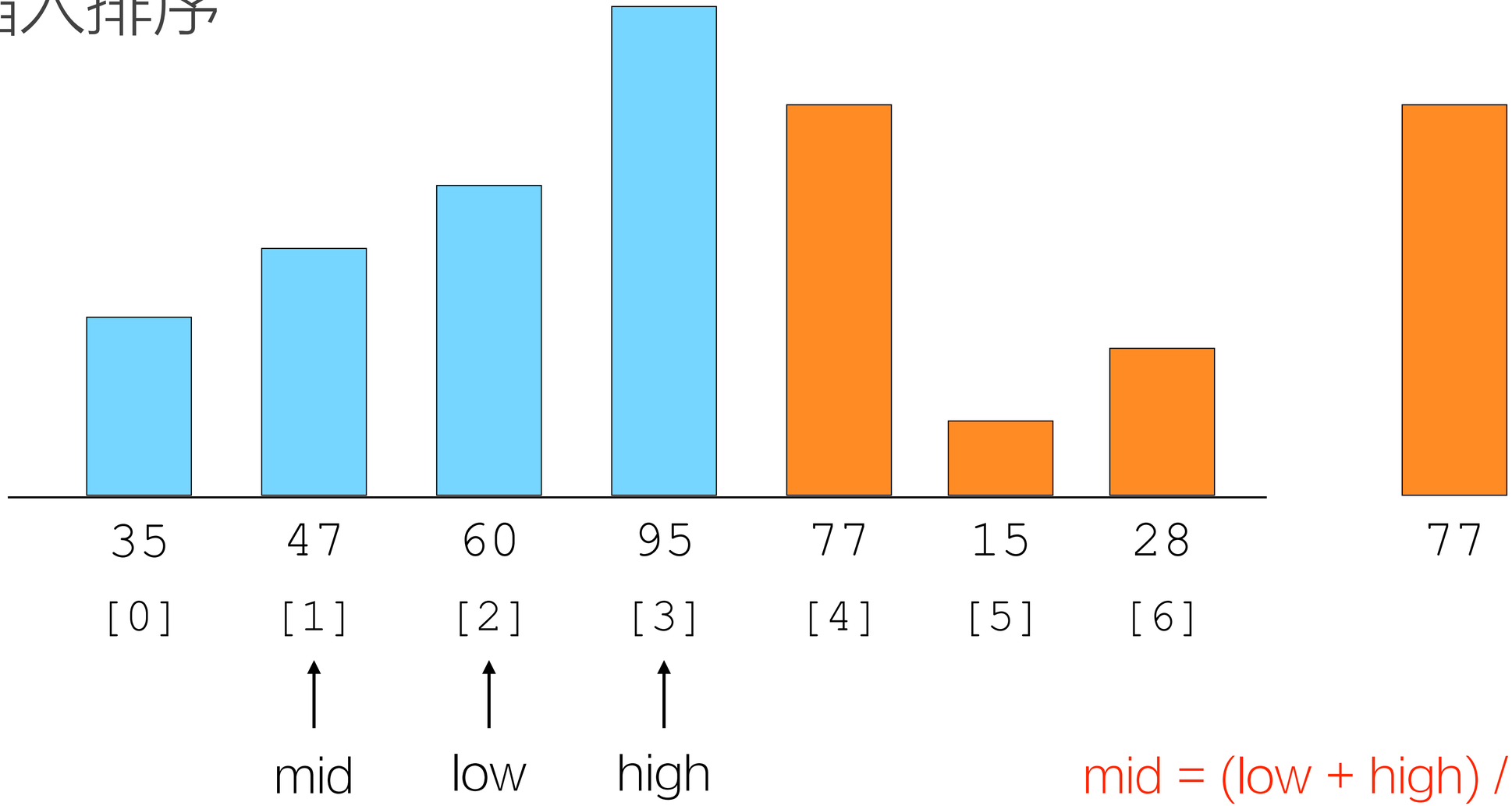
折半插入排序



mid = (low + high) / 2

data[mid] > 77 ? high = mid − 1 : low = mid + 1

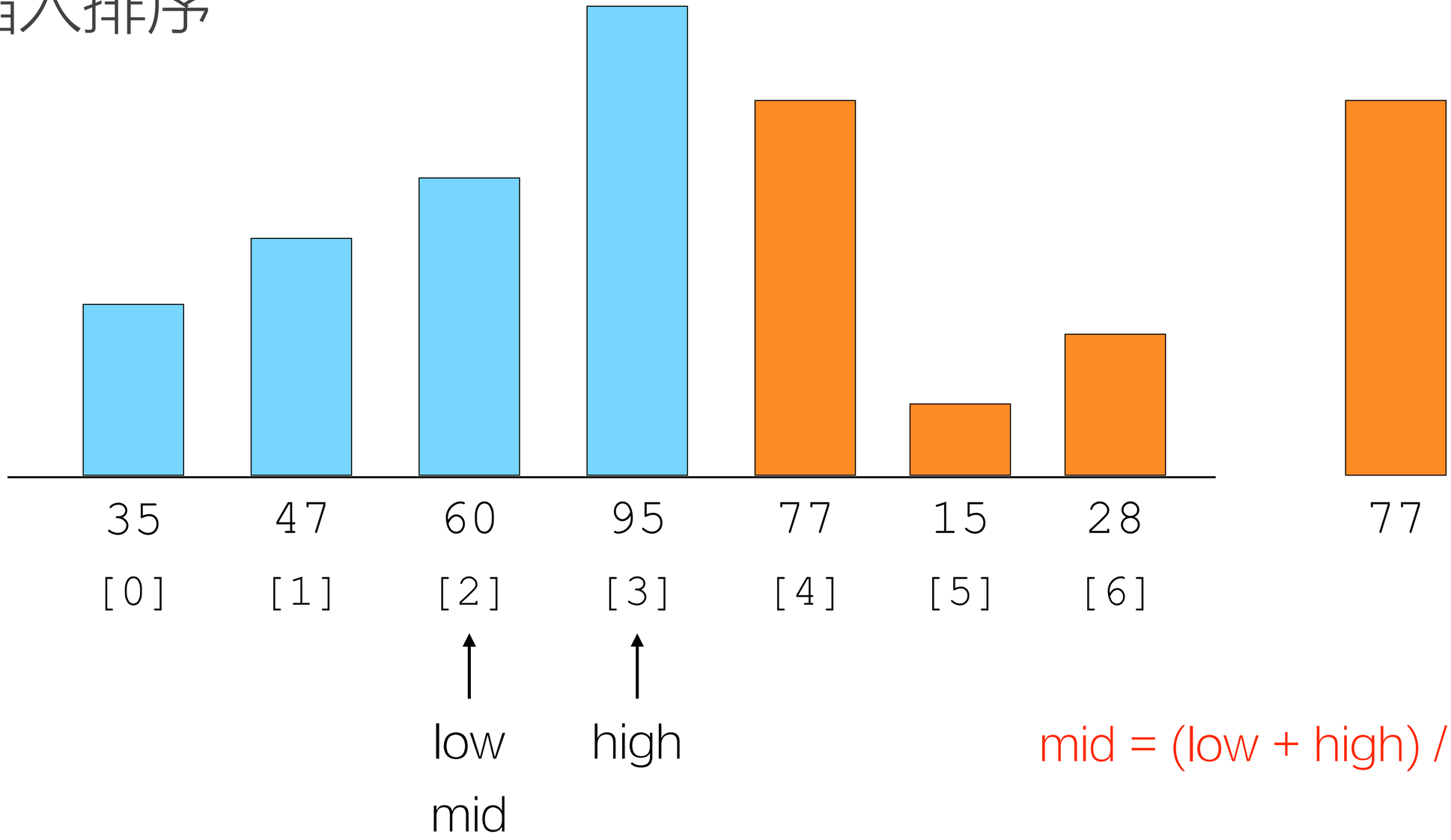
折半插入排序



mid = (low + high) / 2

data[mid] > 77 ? high = mid − 1 : low = mid + 1

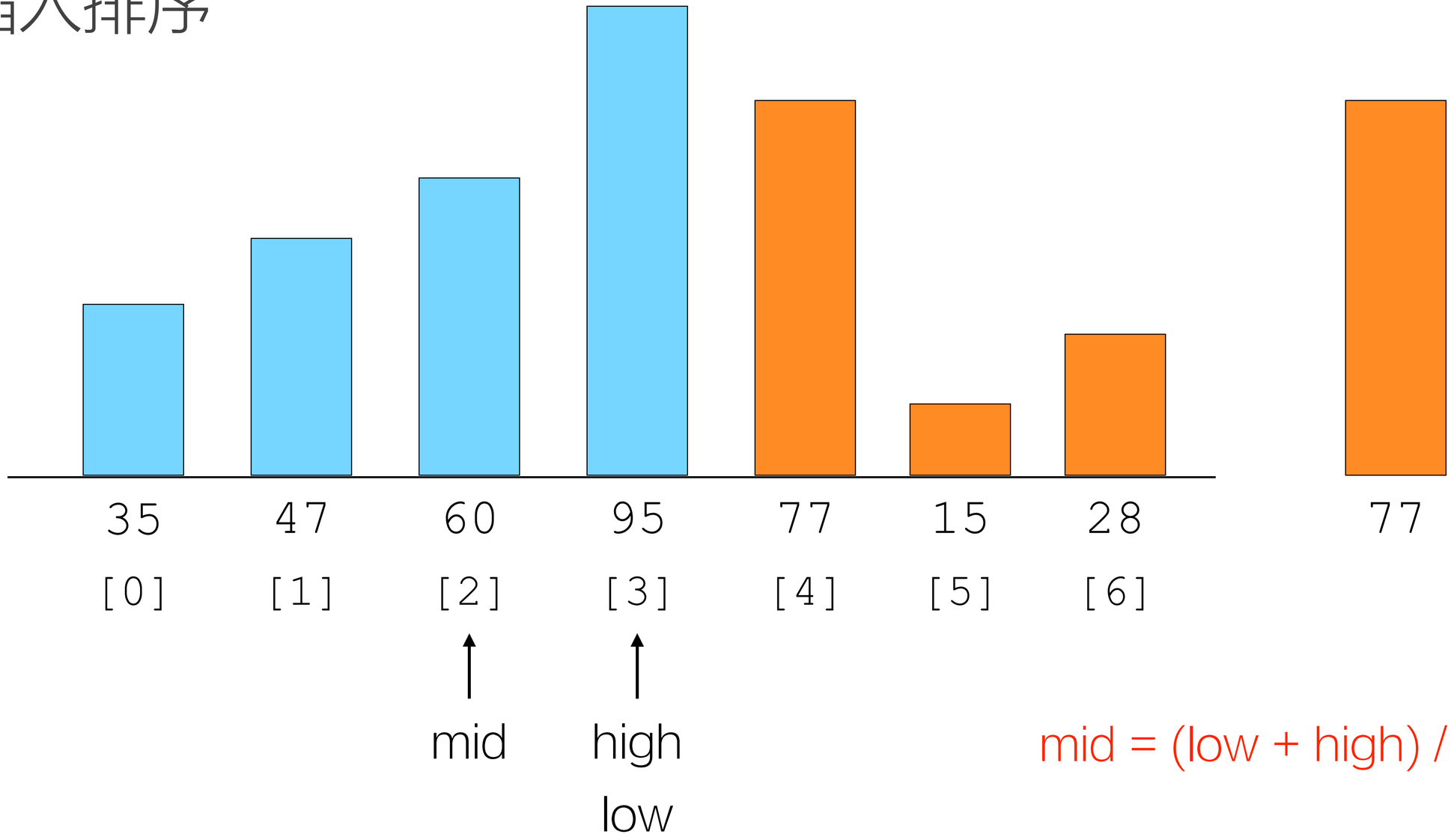
折半插入排序



mid = (low + high) / 2

data[mid] > 77 ? high = mid − 1 : low = mid + 1

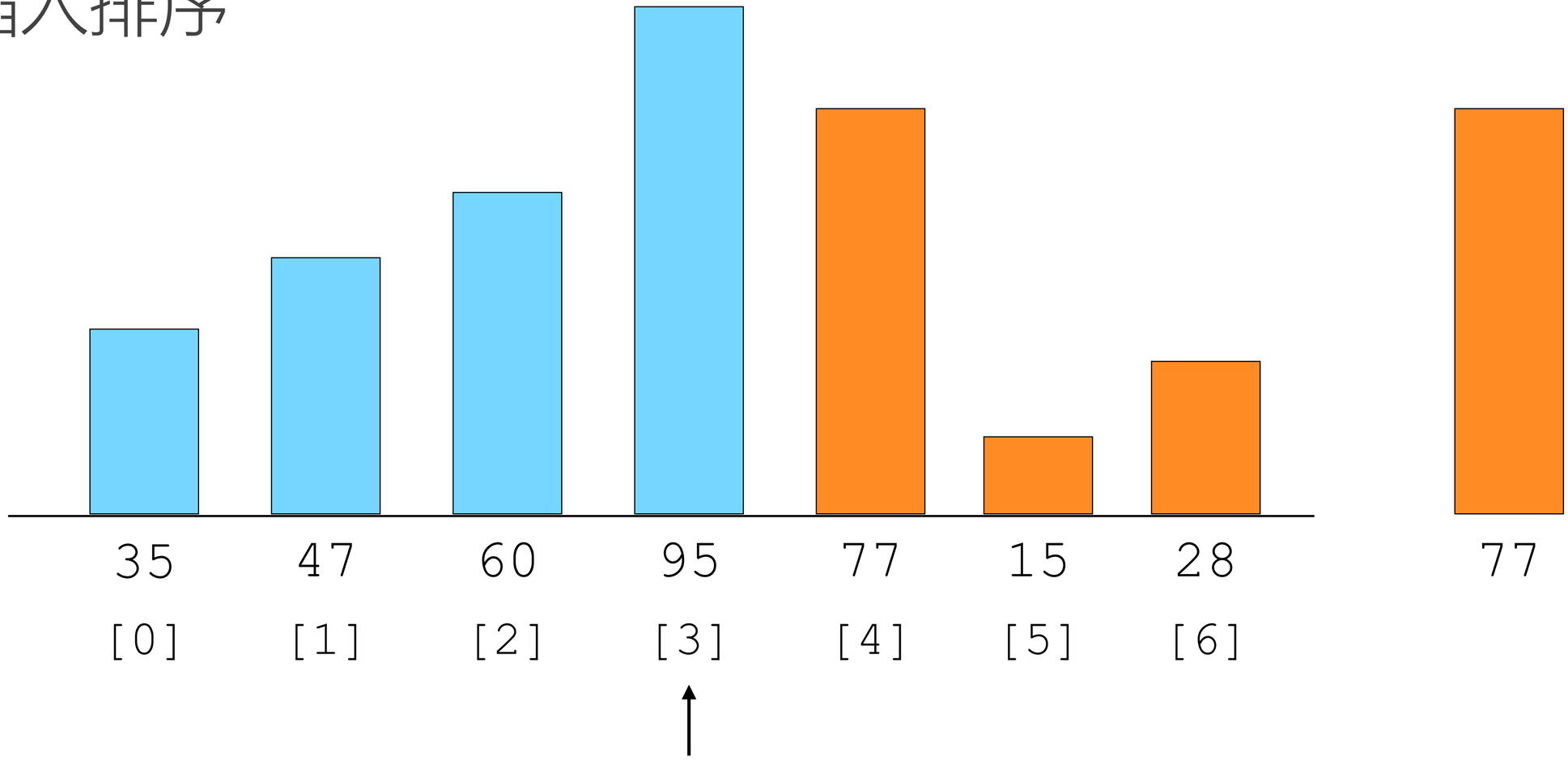
折半插入排序



mid = (low + high) / 2

data[mid] > 77 ? high = mid − 1 : low = mid + 1

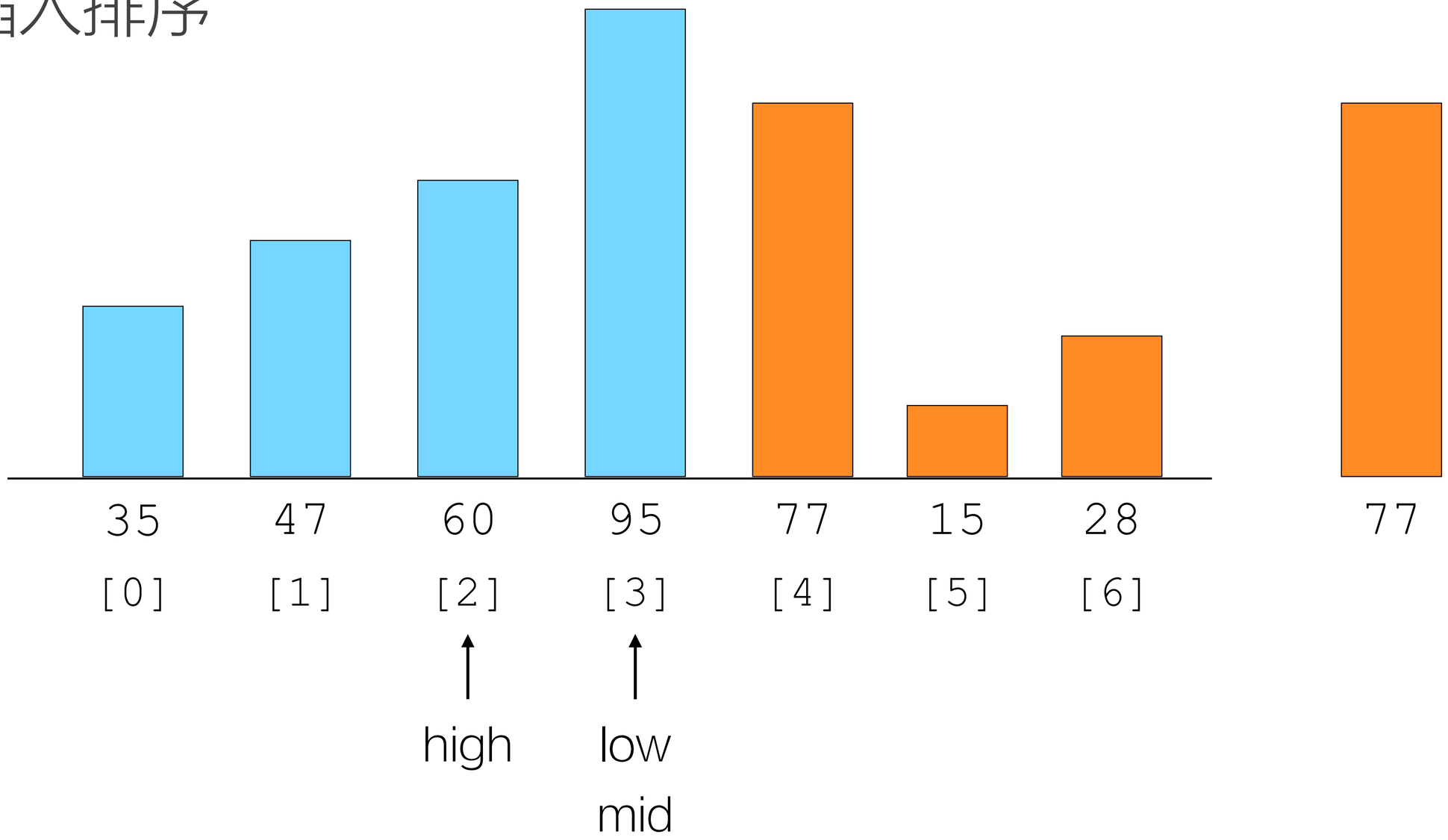
折半插入排序



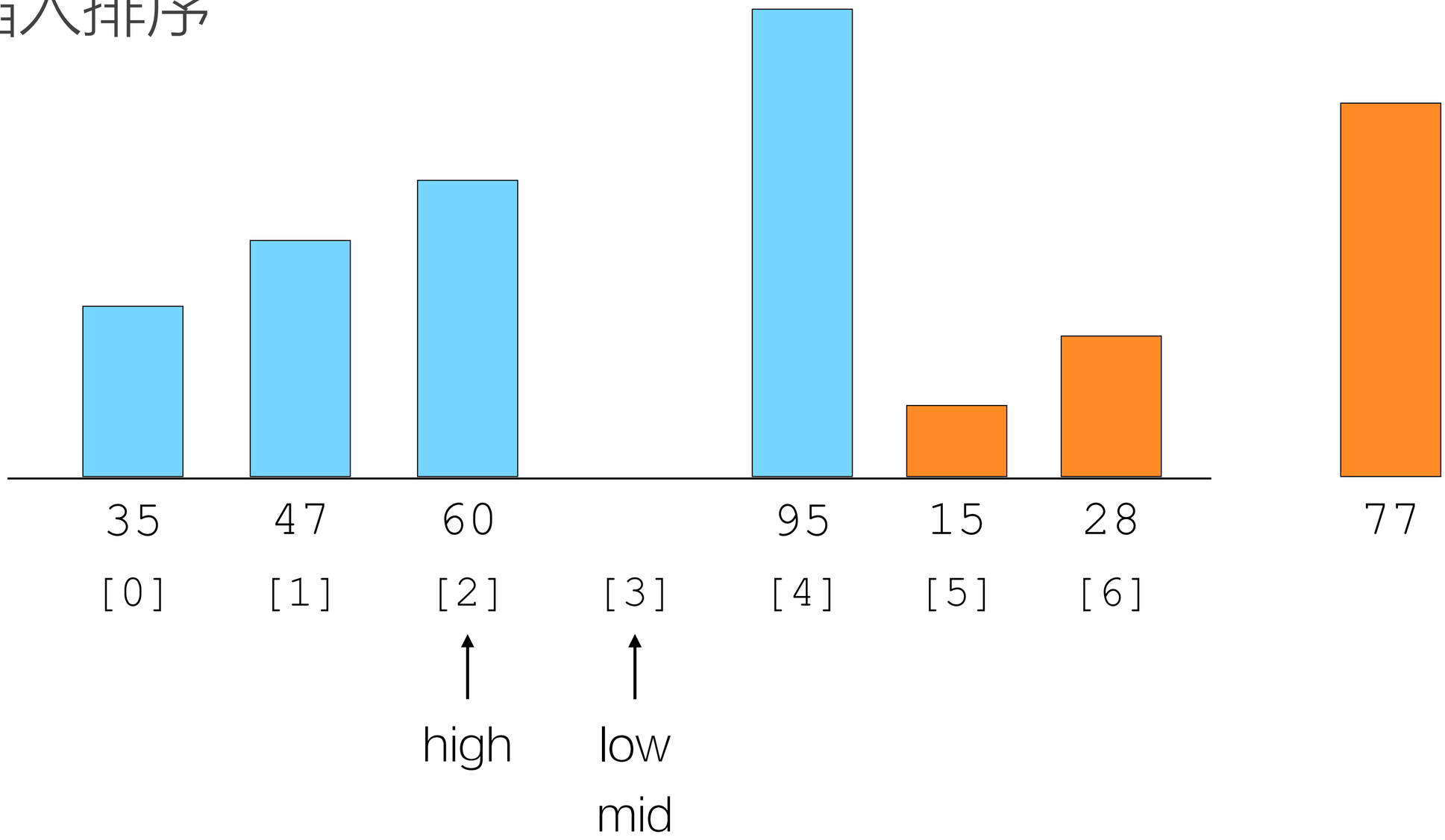
mid = (low + high) / 2

data[mid] > 77 ? high = mid − 1 : low = mid + 1

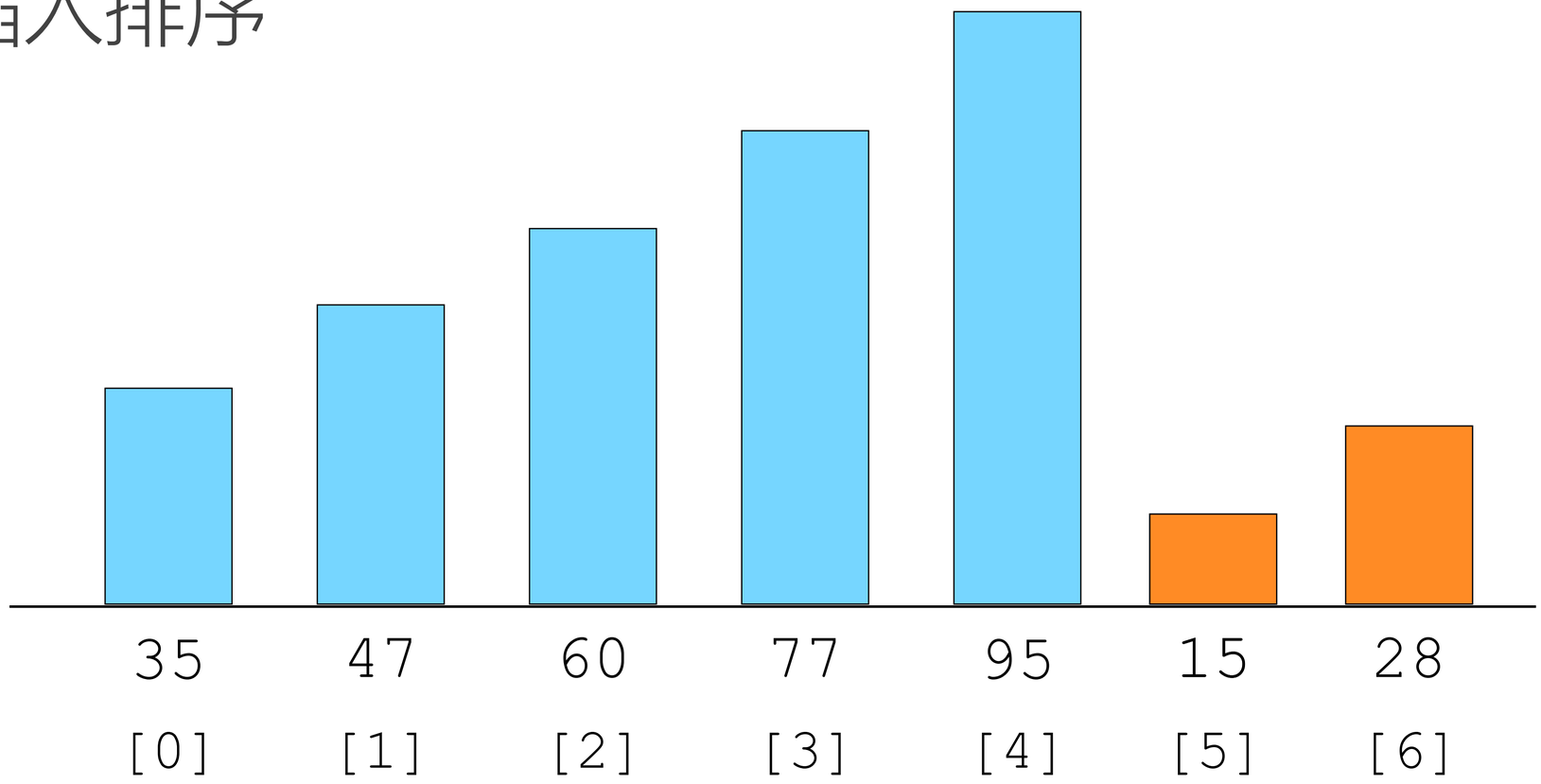
折半插入排序



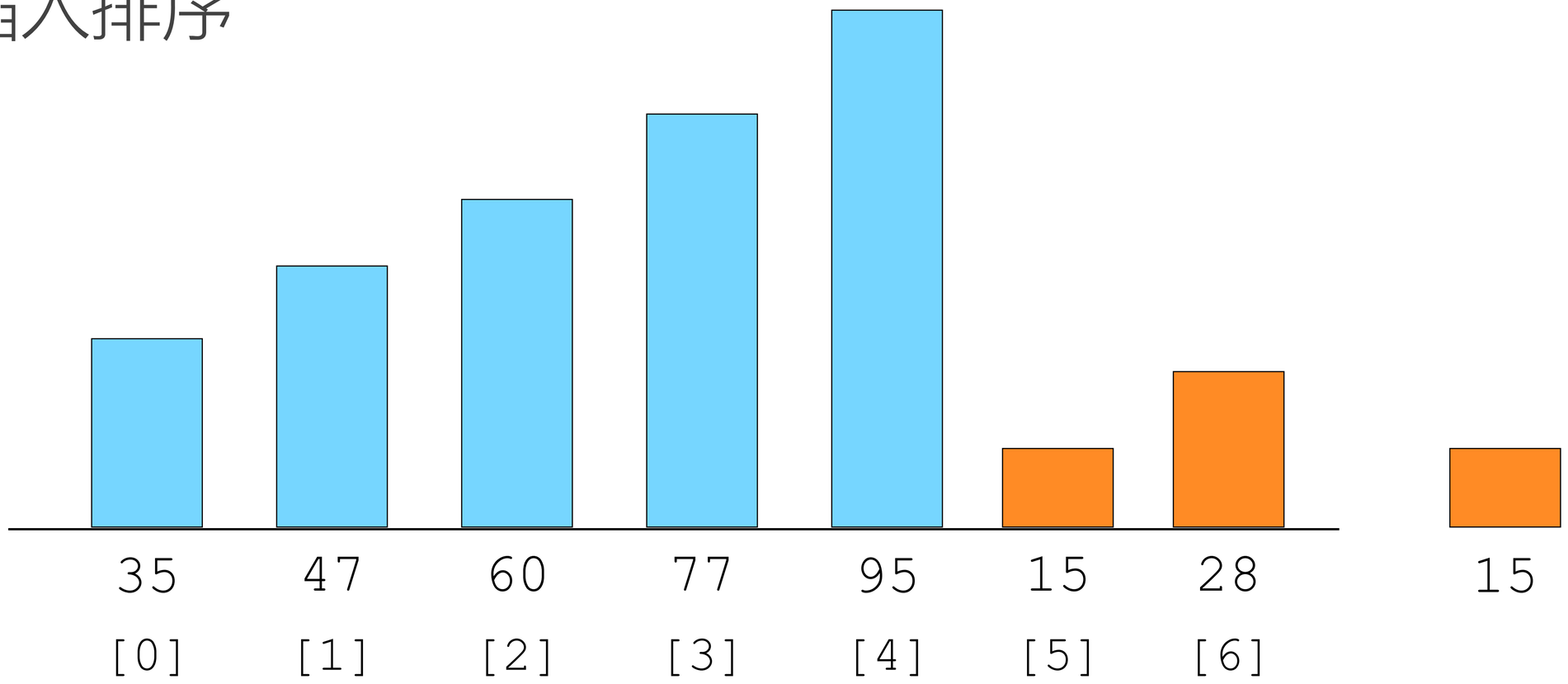
折半插入排序



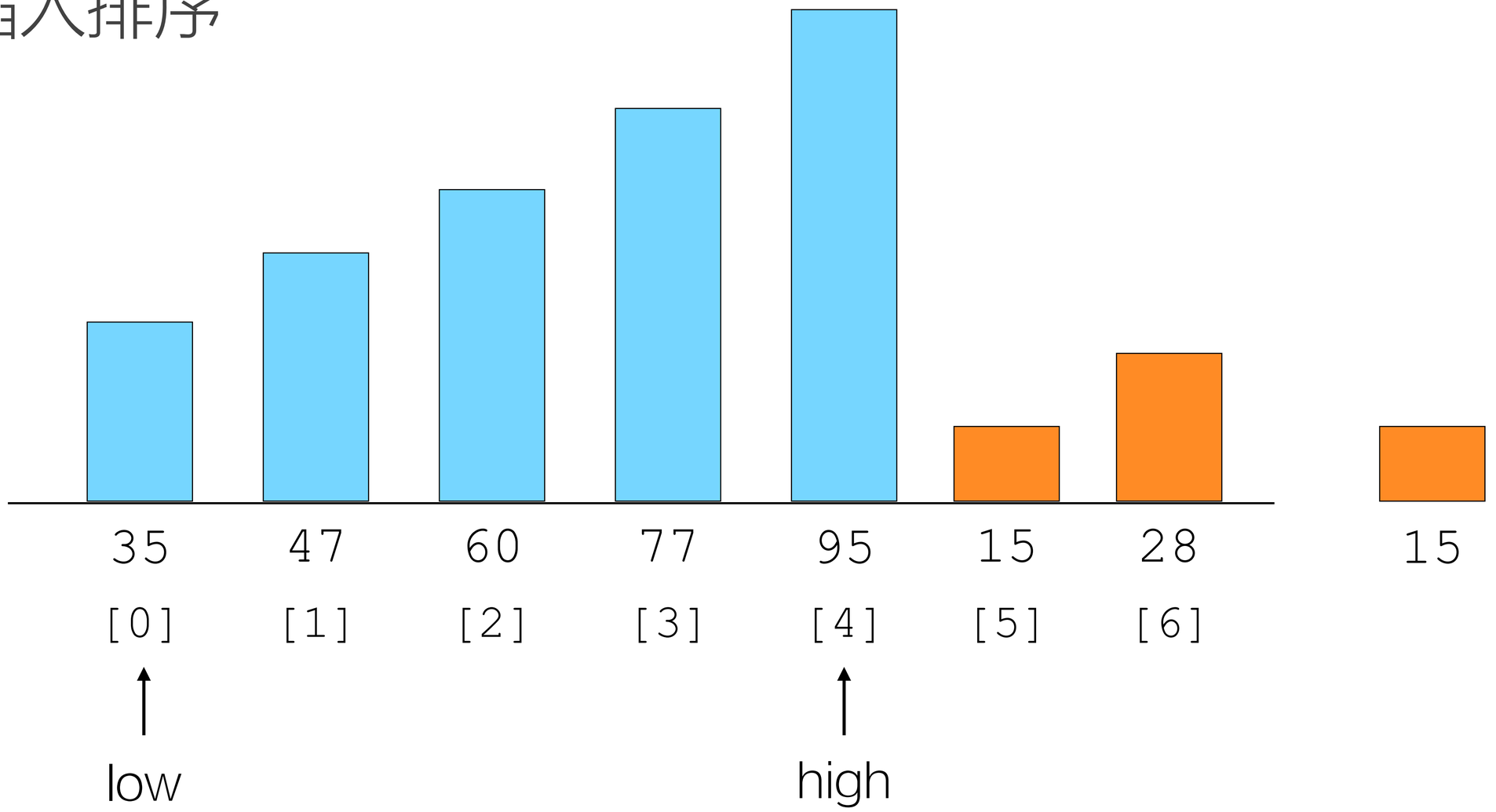
折半插入排序



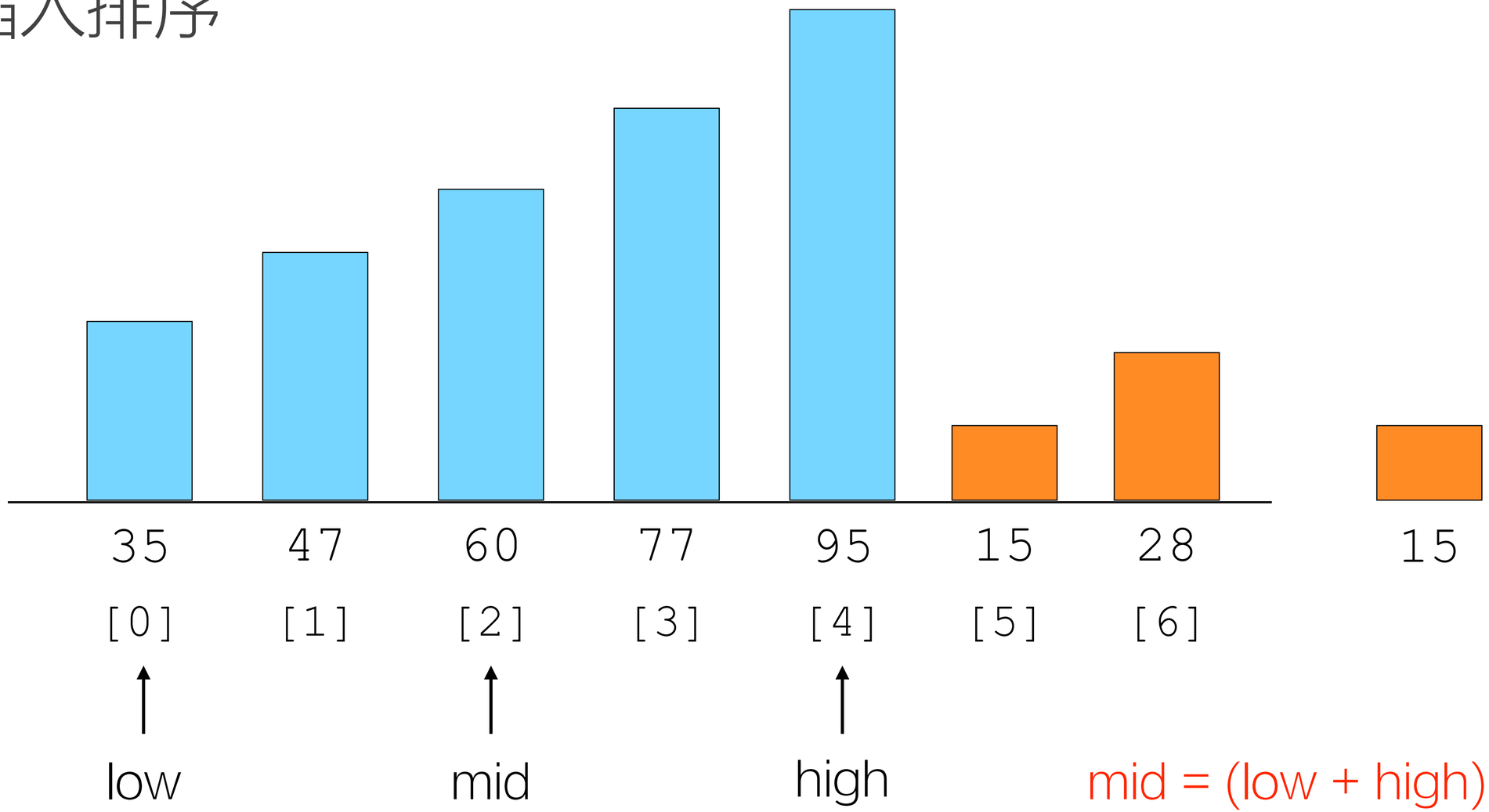
折半插入排序



折半插入排序

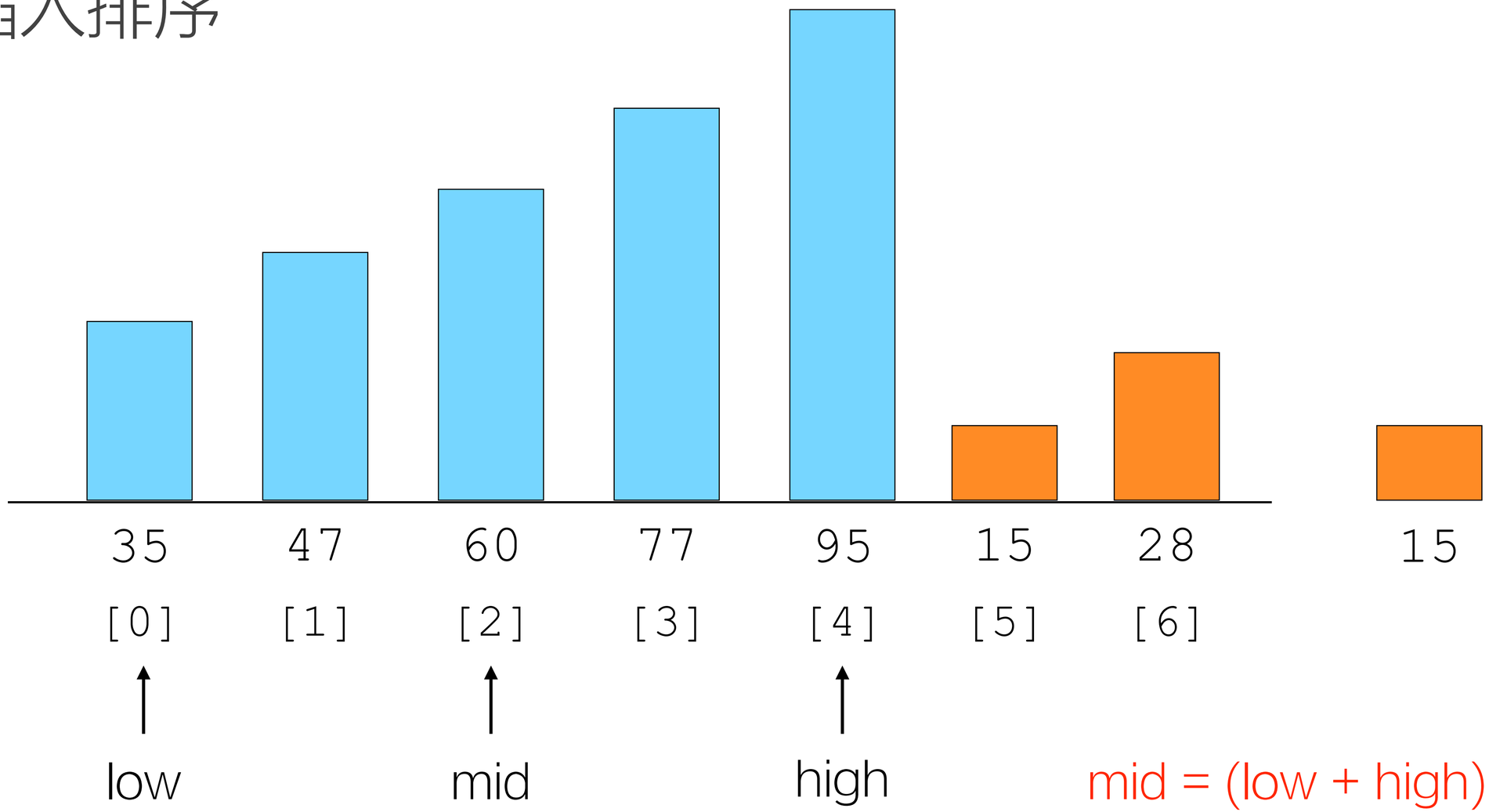


折半插入排序



mid = (low + high) / 2

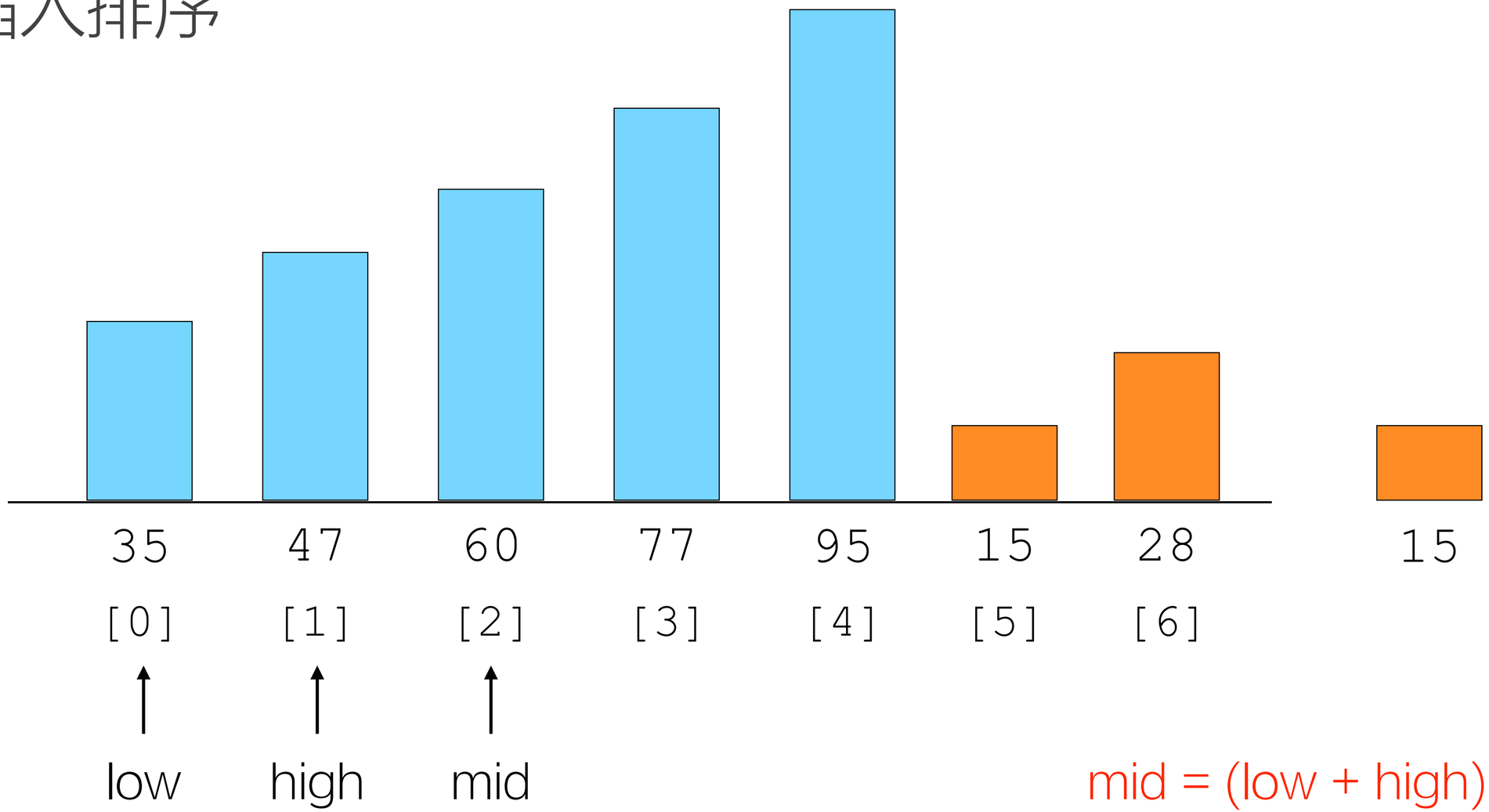
折半插入排序



mid = (low + high) / 2

data[mid] > 15 ? high = mid − 1 : low = mid + 1

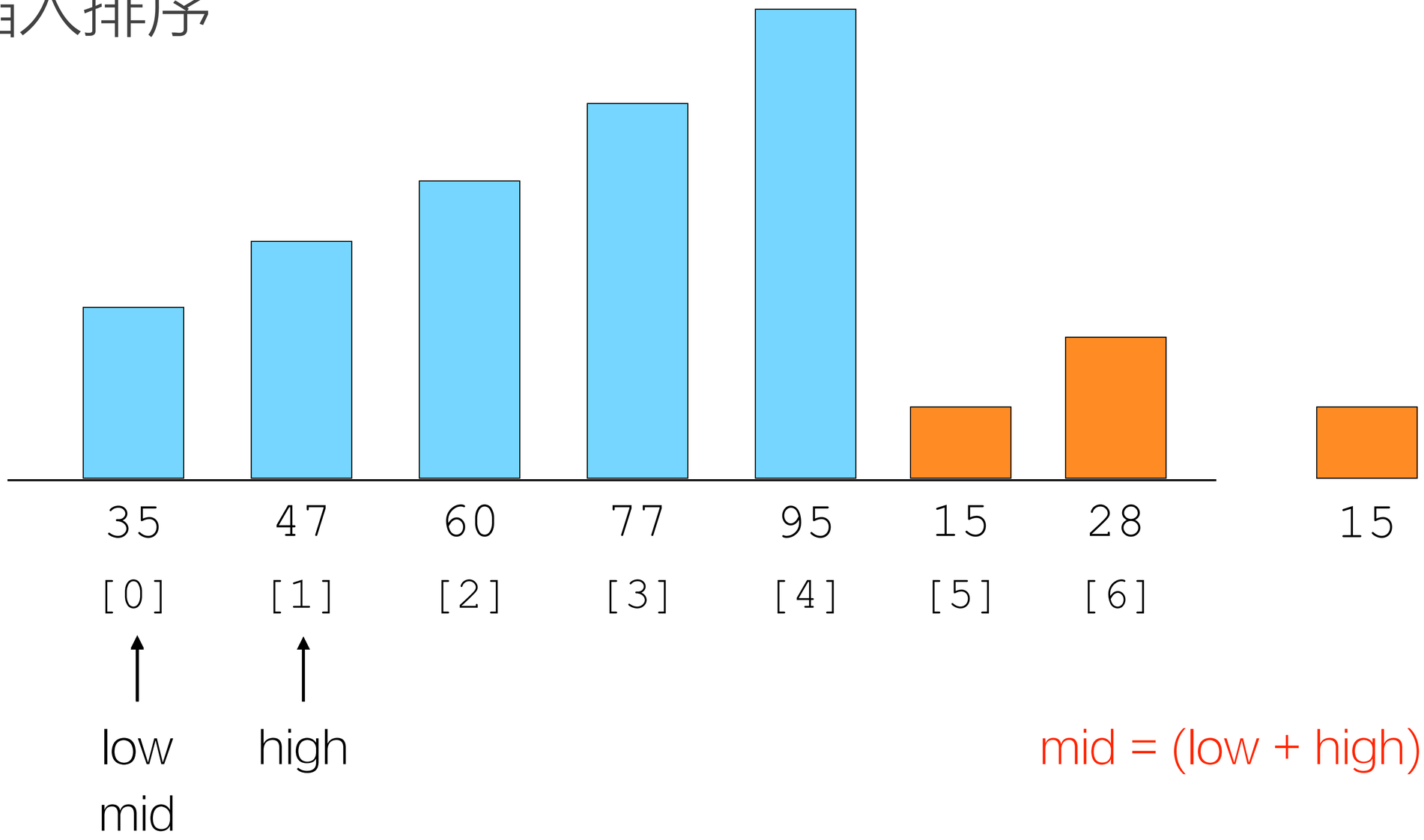
折半插入排序



mid = (low + high) / 2

data[mid] > 15 ? high = mid − 1 : low = mid + 1

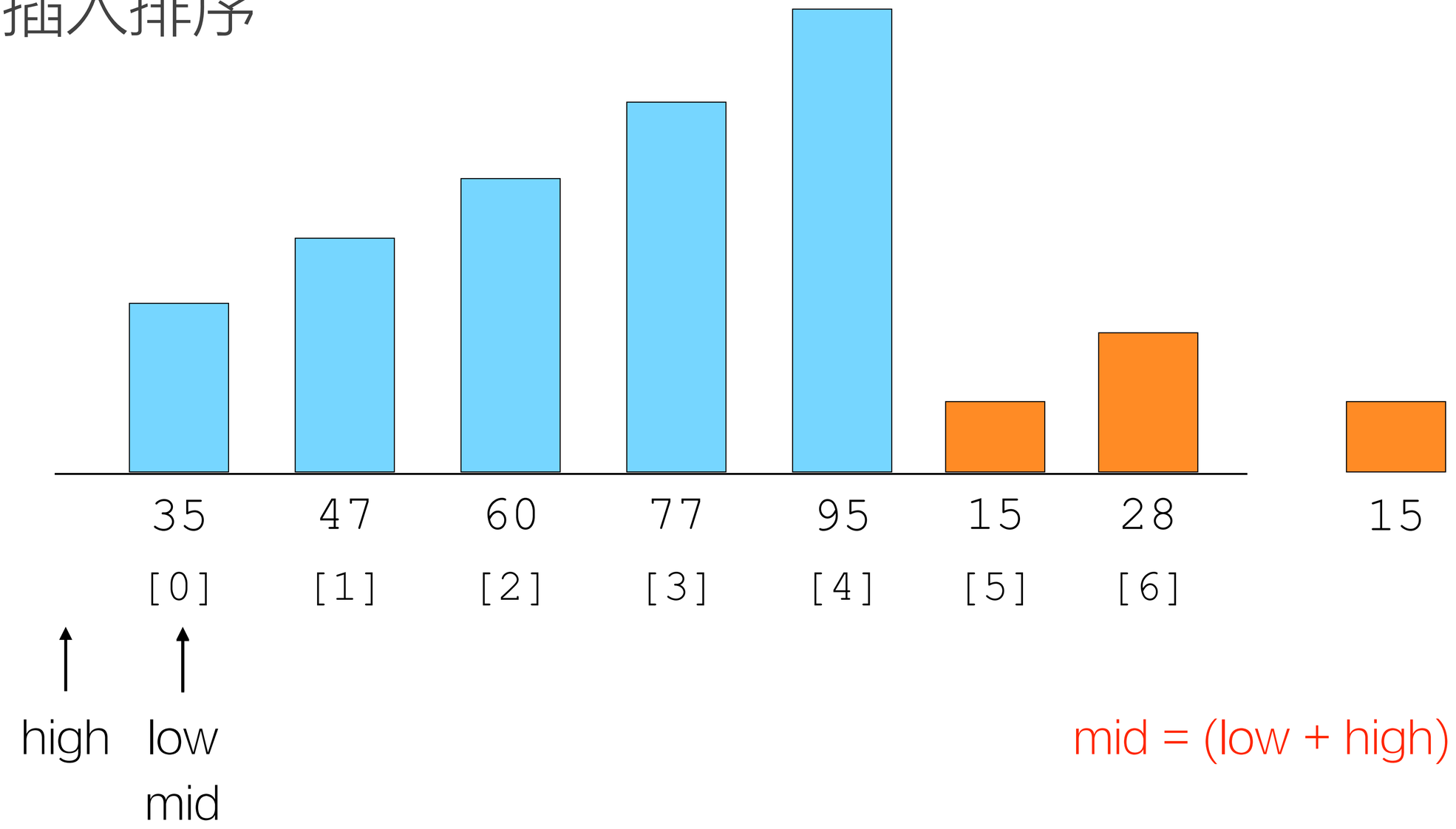
折半插入排序



mid = (low + high) / 2

data[mid] > 15 ? high = mid − 1 : low = mid + 1

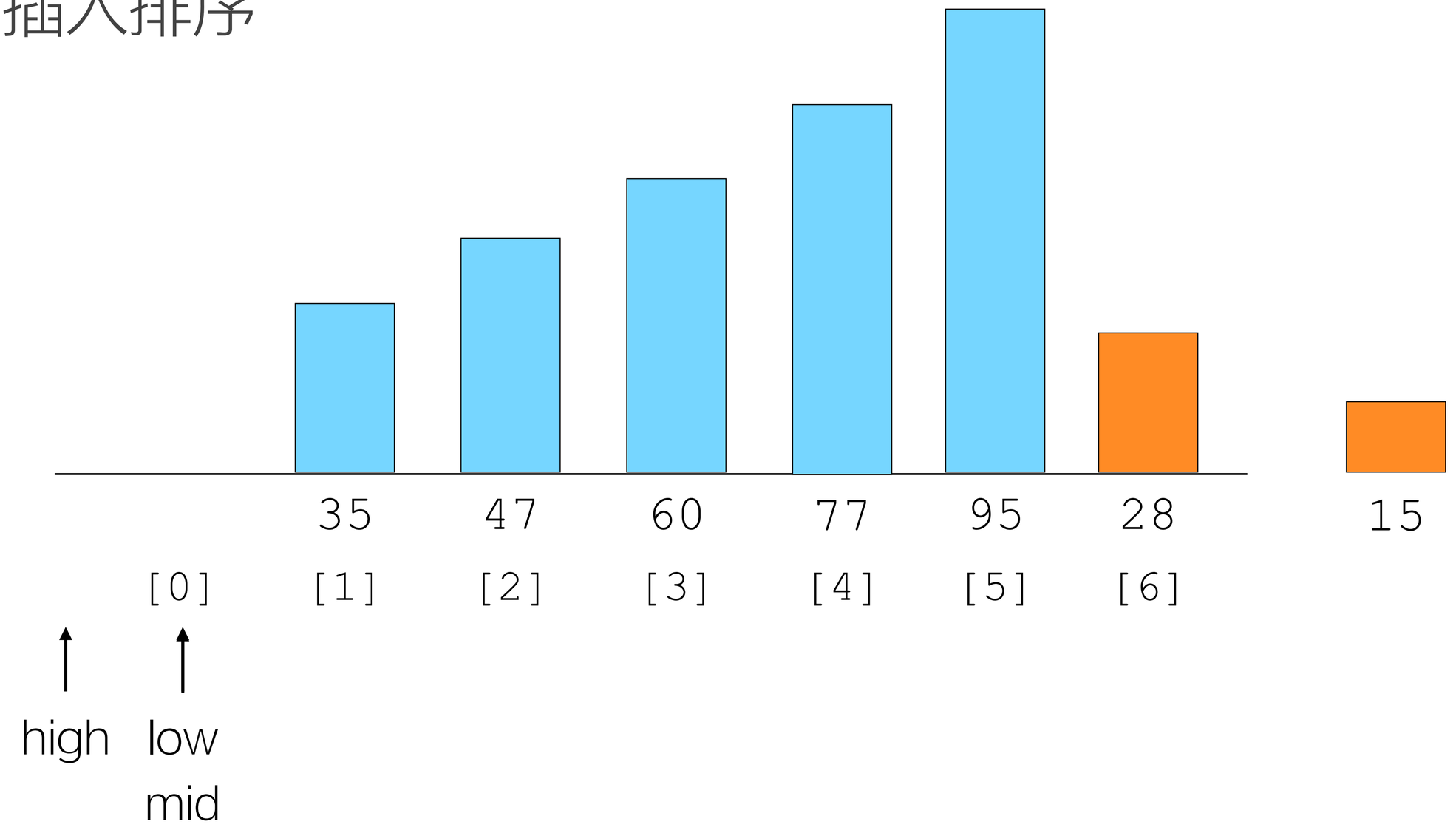
折半插入排序



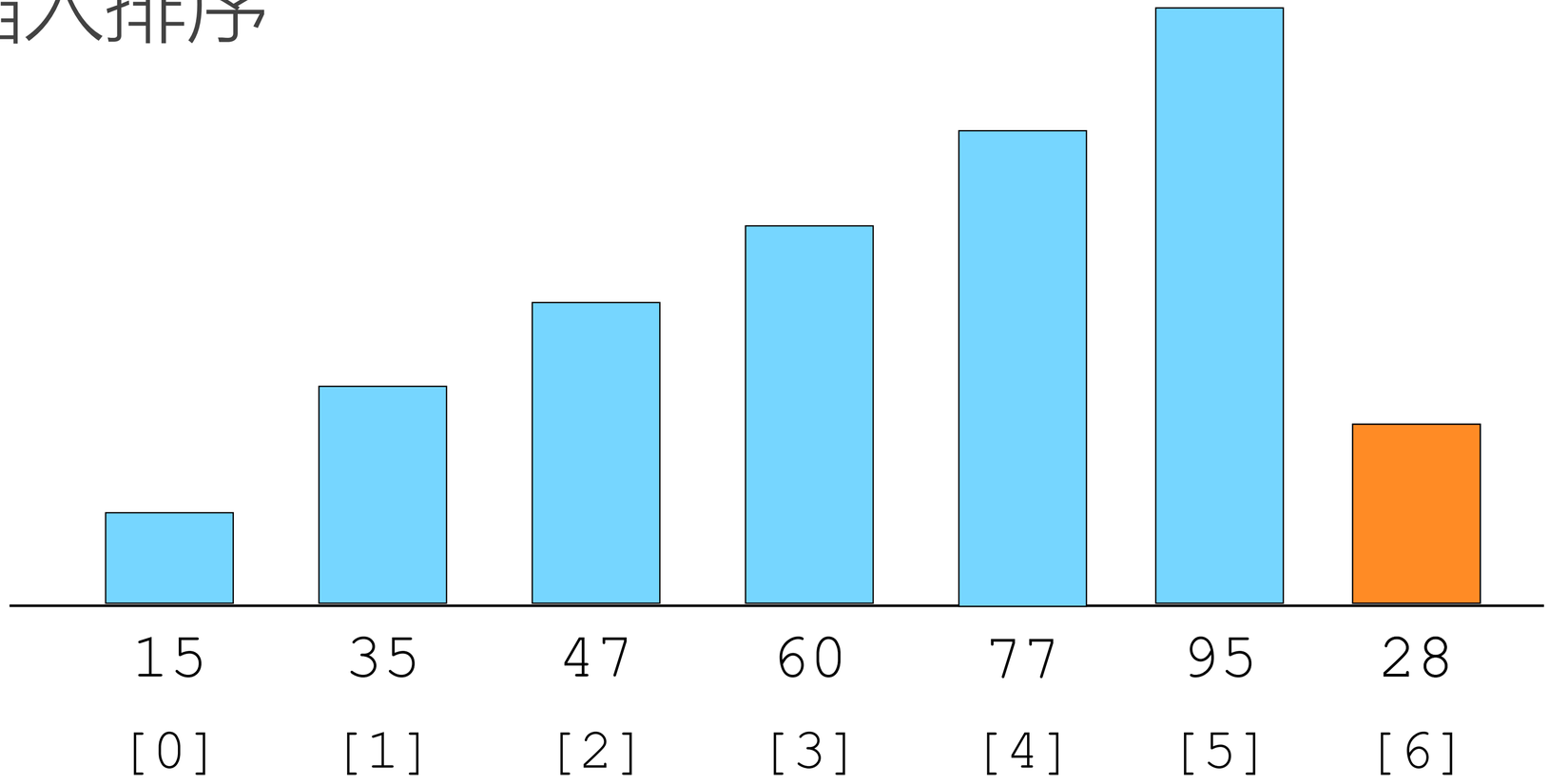
mid = (low + high) / 2

data[mid] > 15 ? high = mid − 1 : low = mid + 1

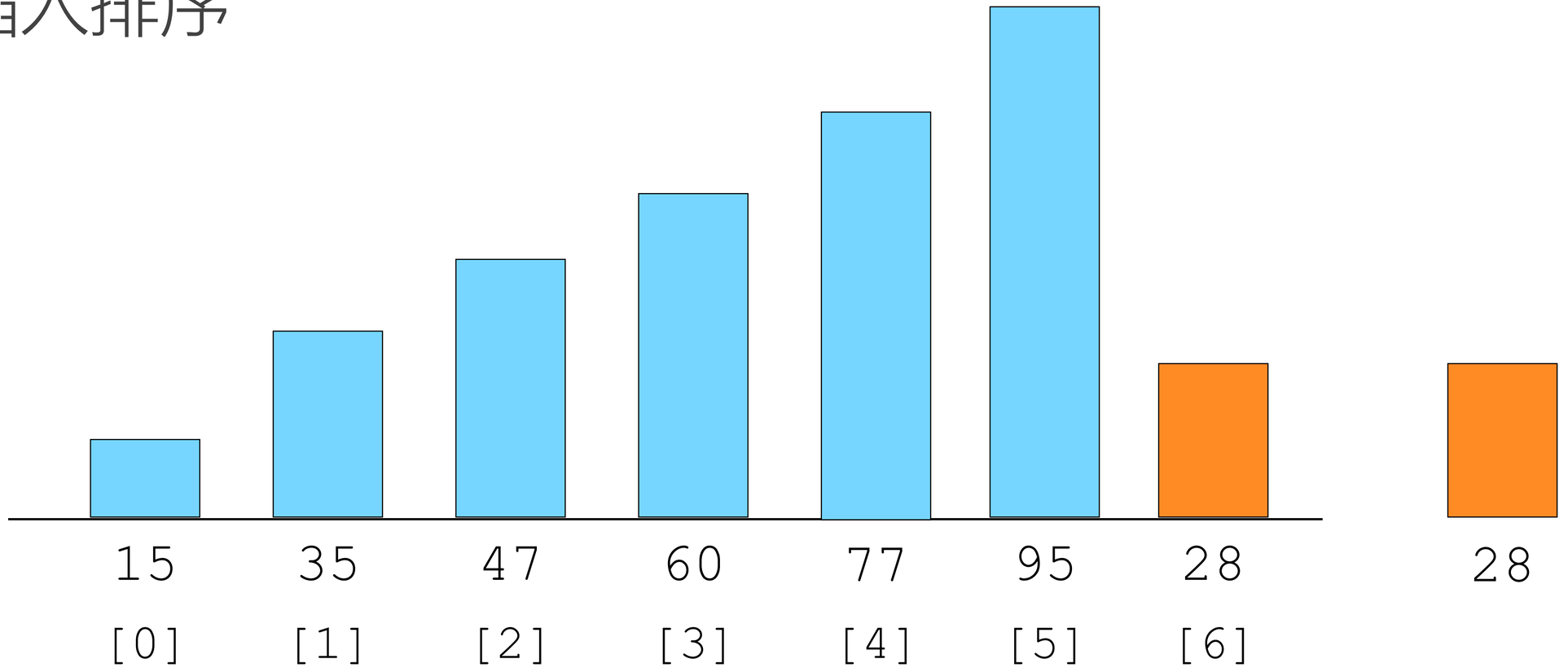
折半插入排序



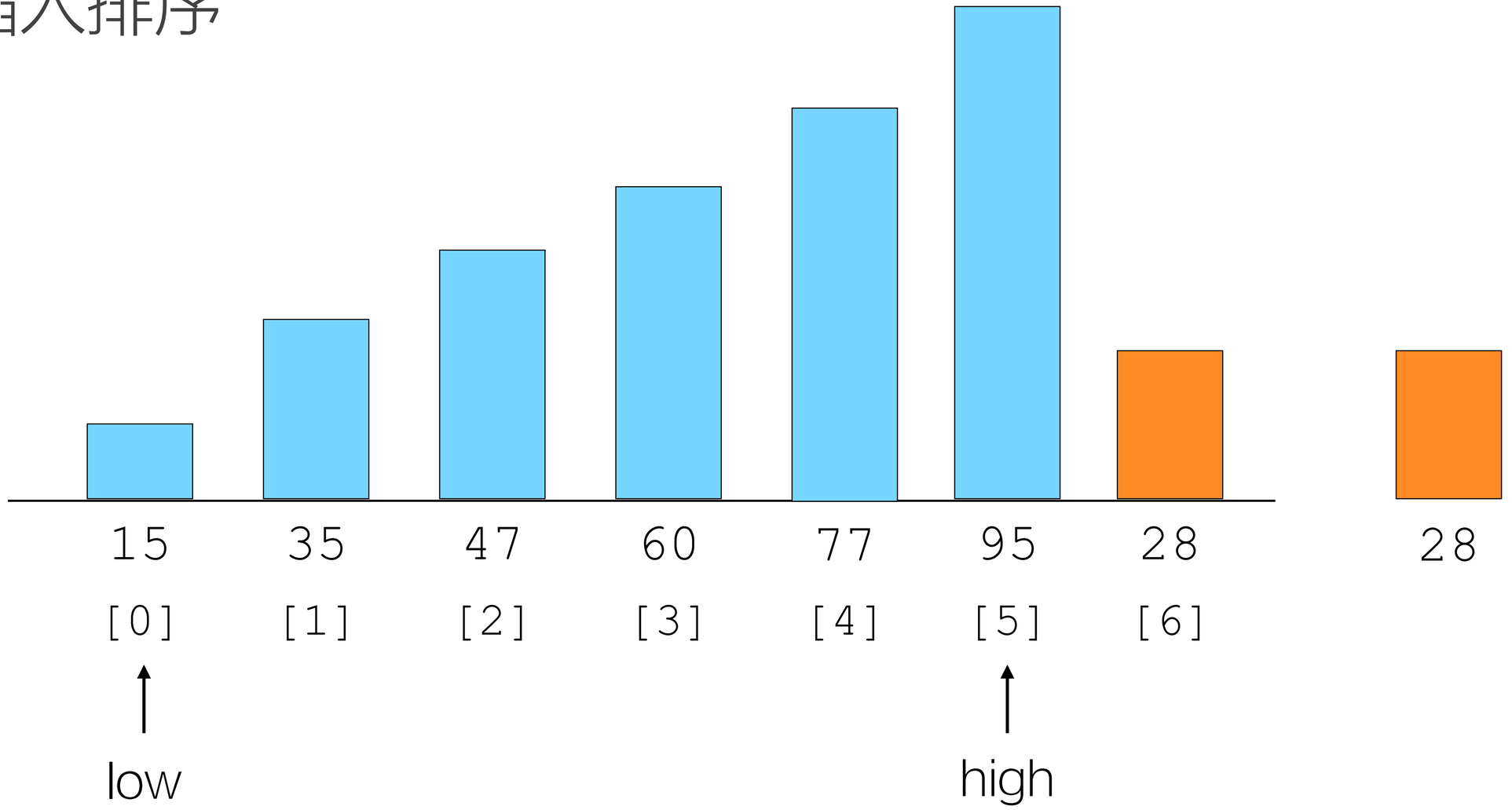
折半插入排序



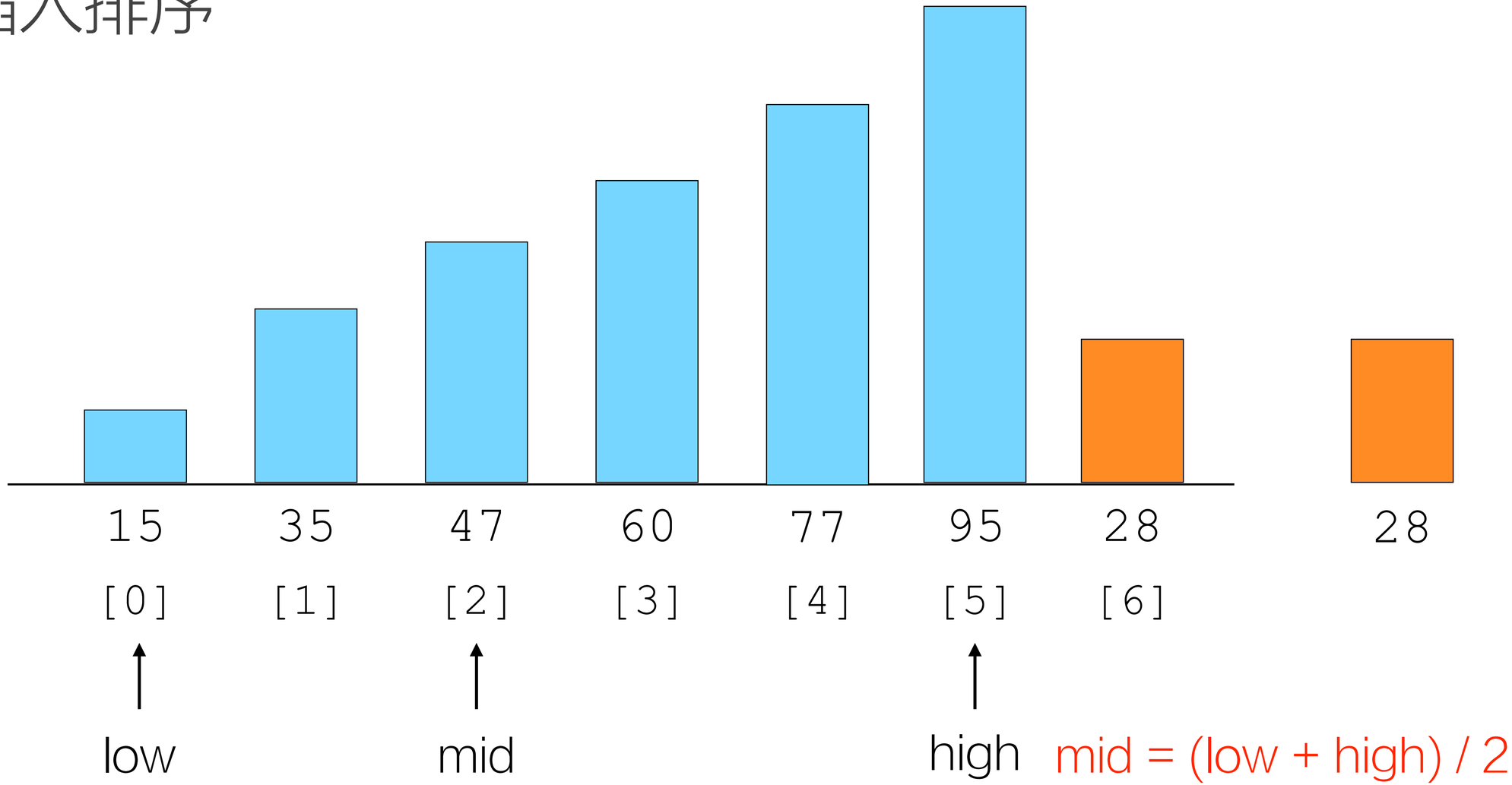
折半插入排序



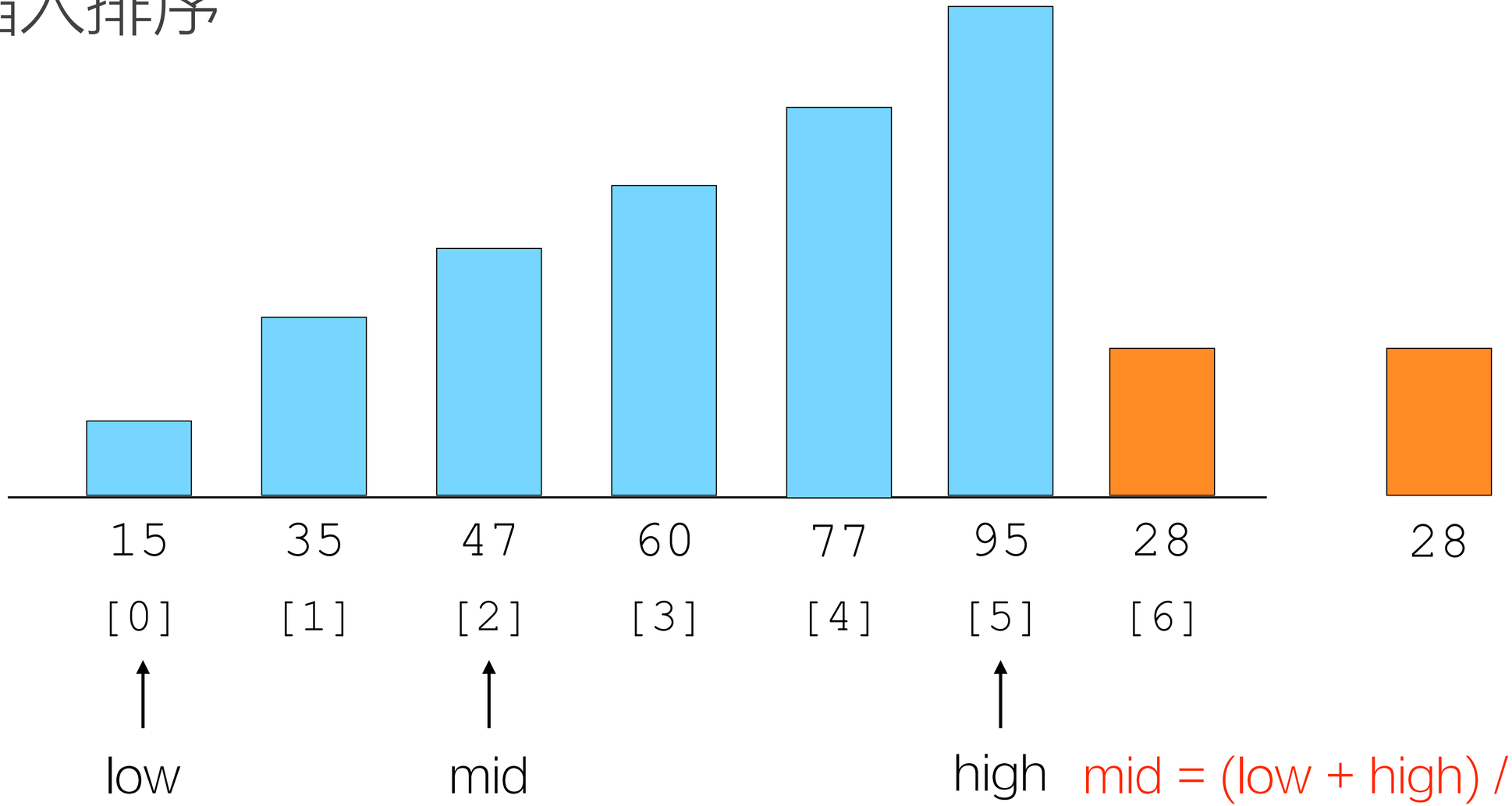
折半插入排序



折半插入排序



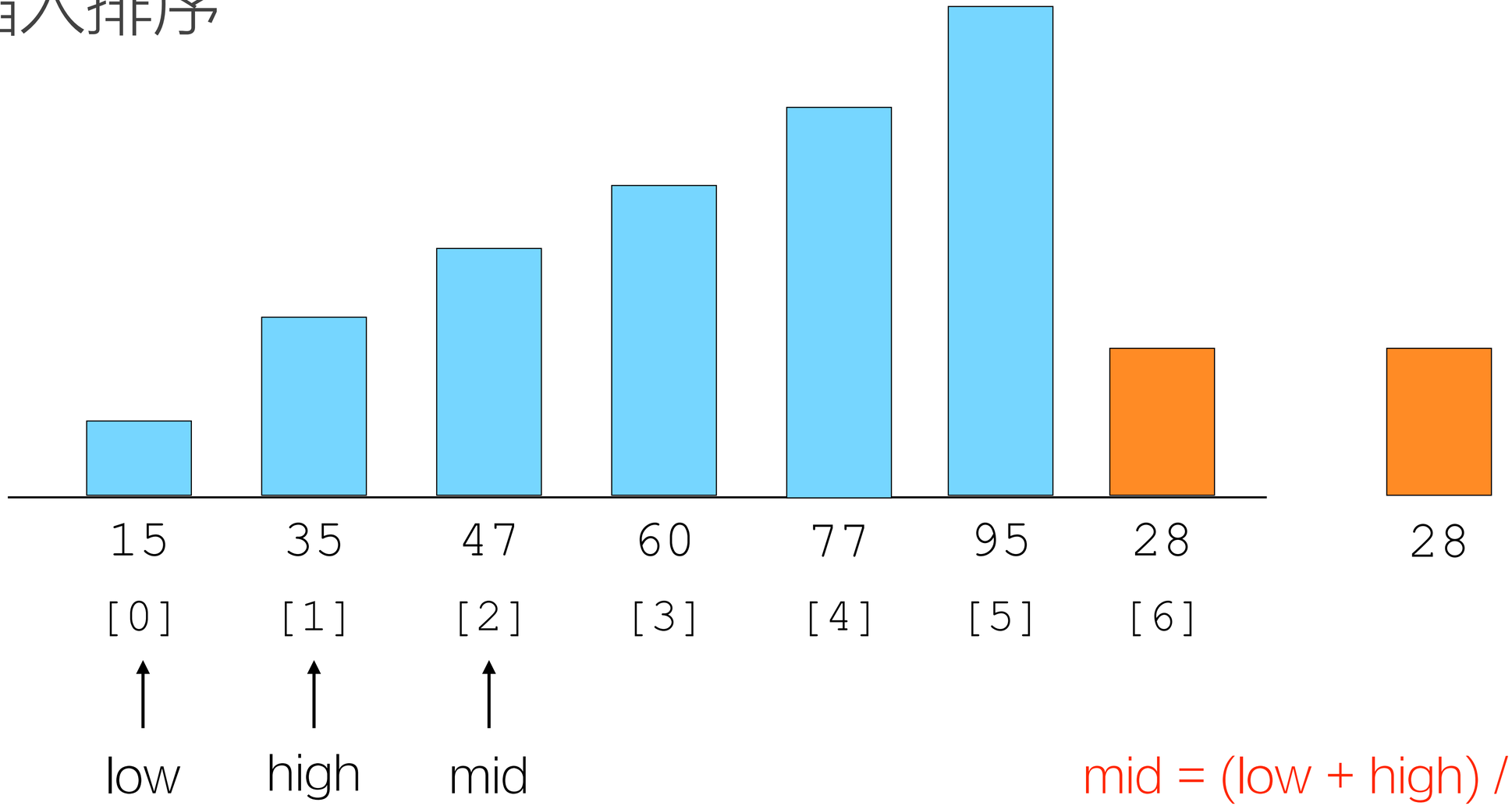
折半插入排序



mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

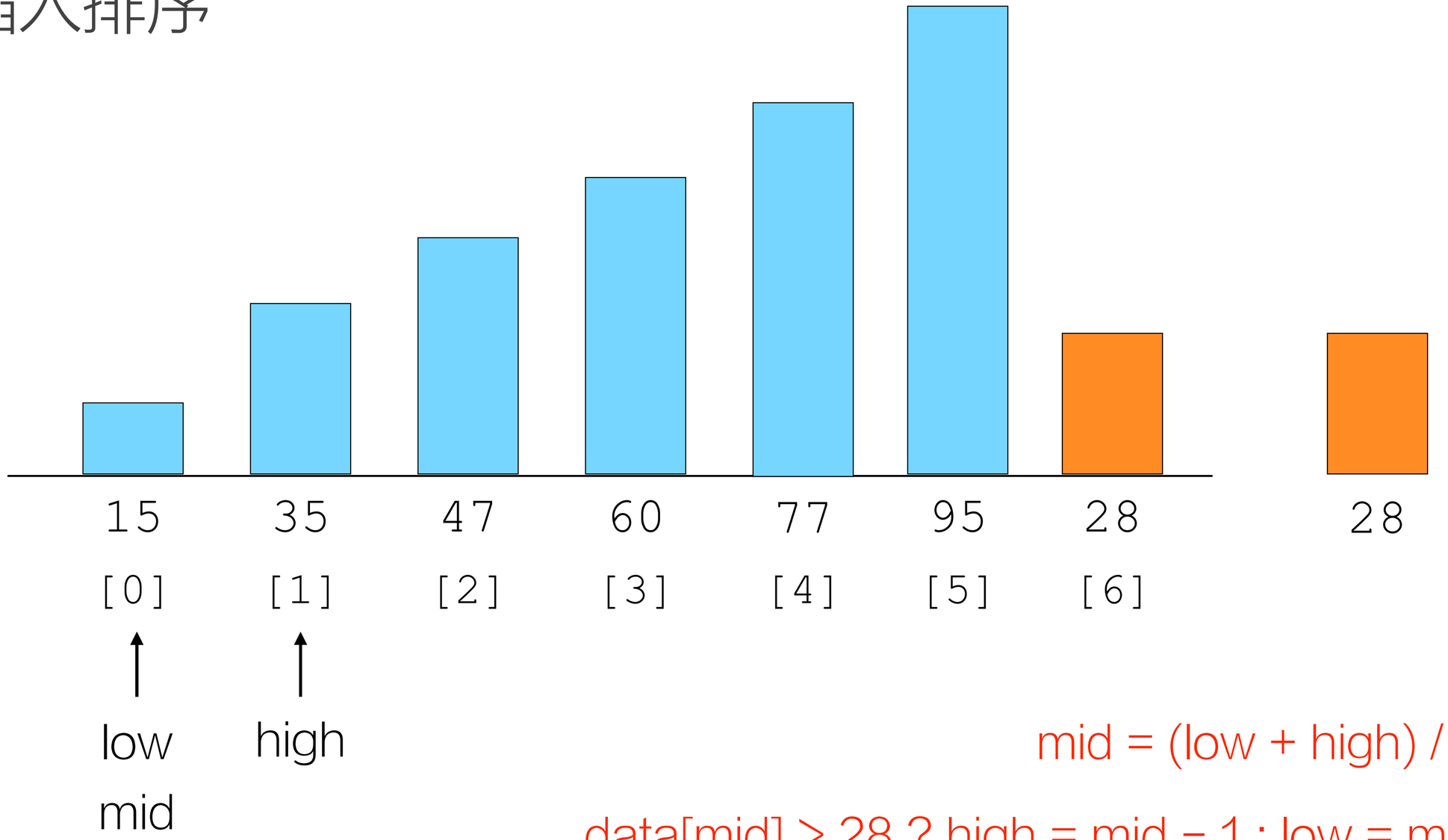
折半插入排序



mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

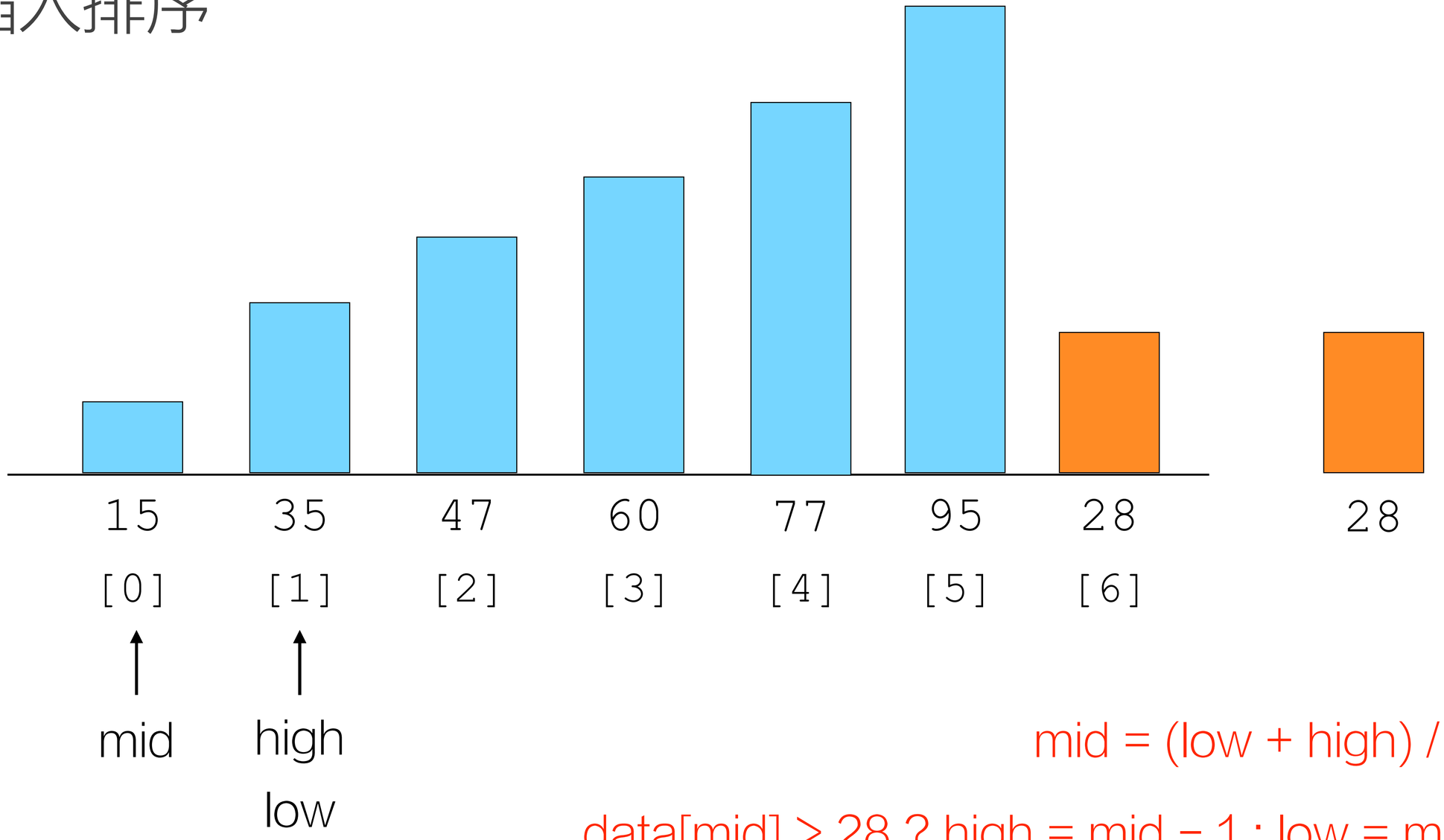
折半插入排序



mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

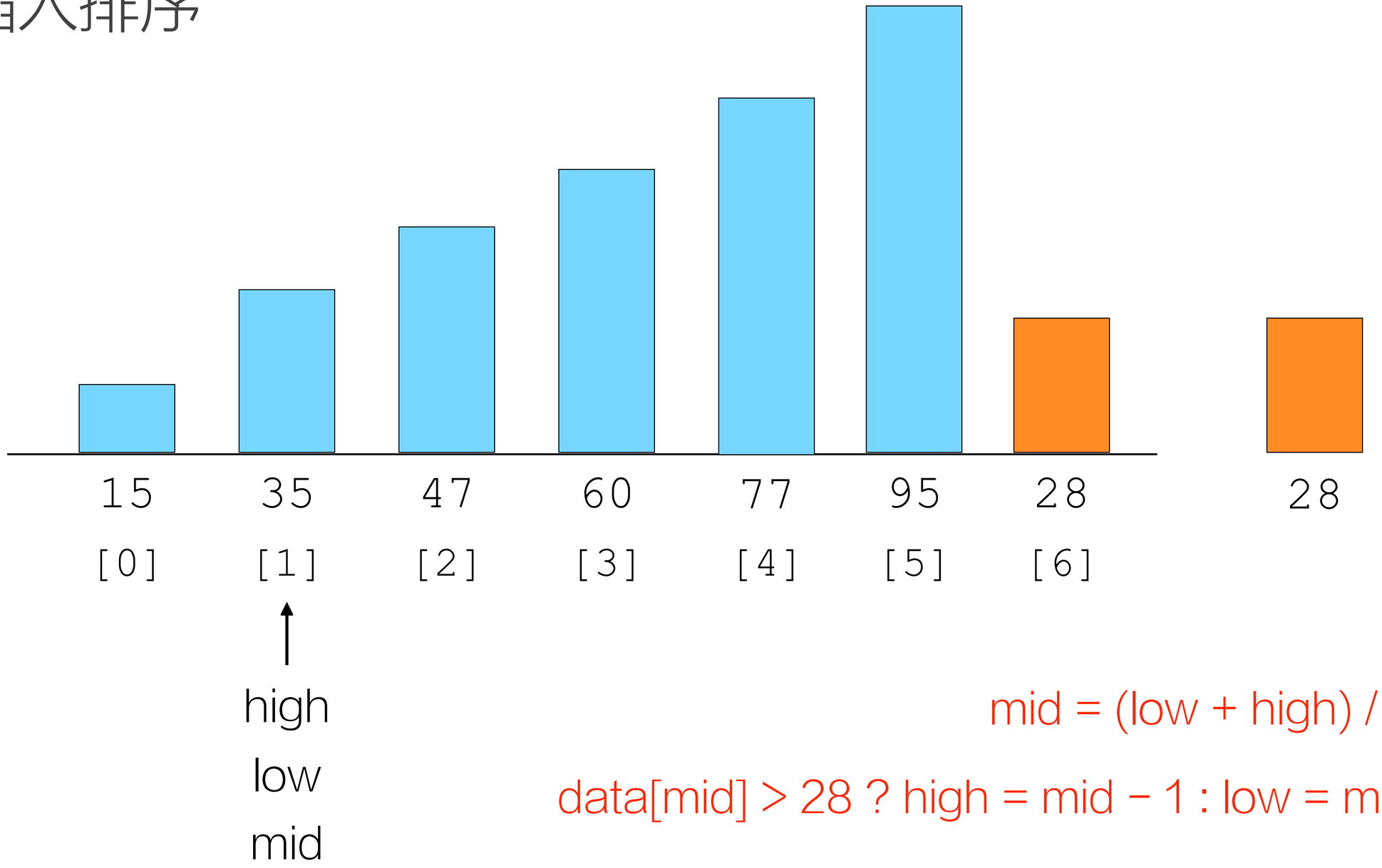
折半插入排序



mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

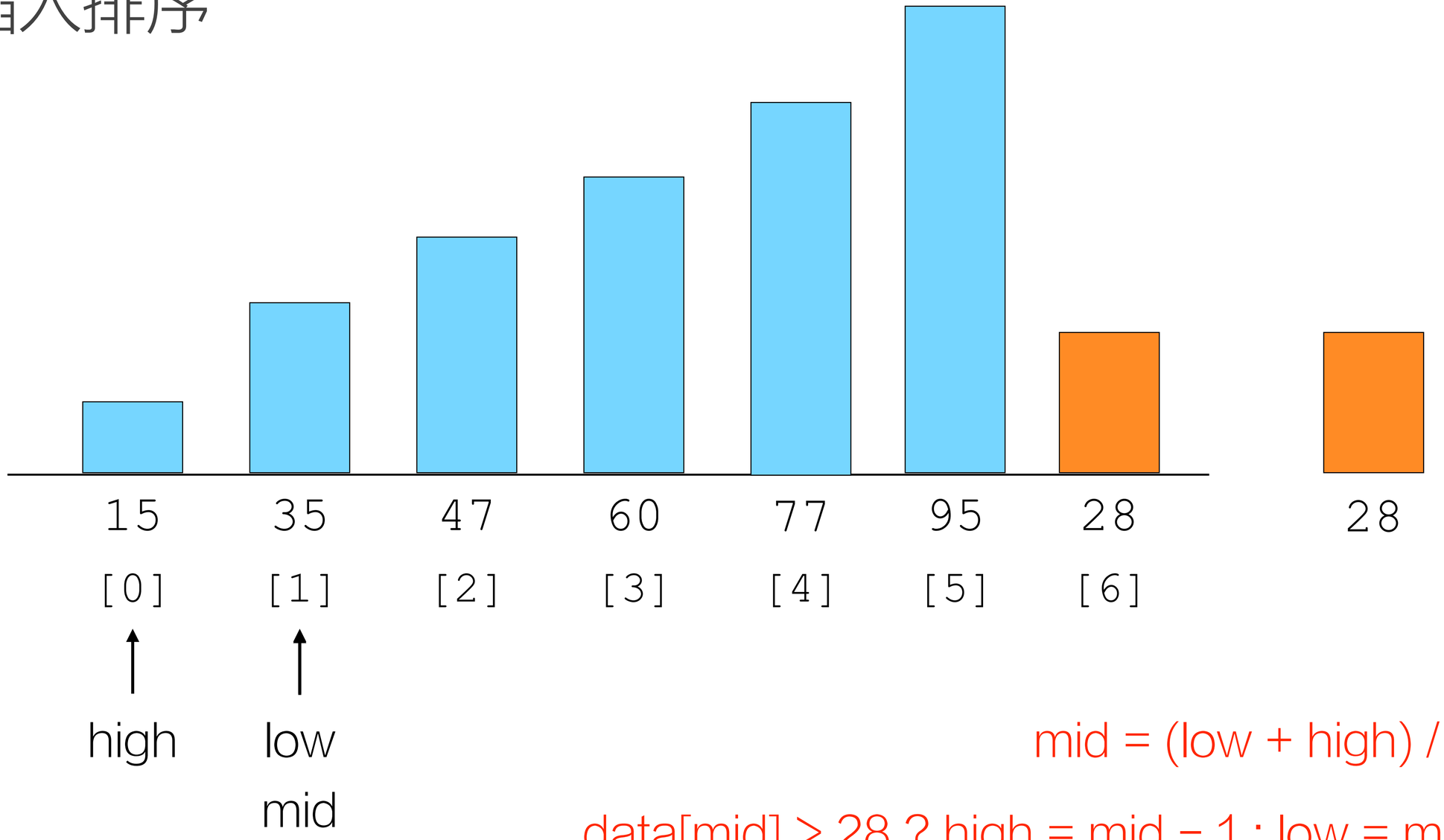
折半插入排序



mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

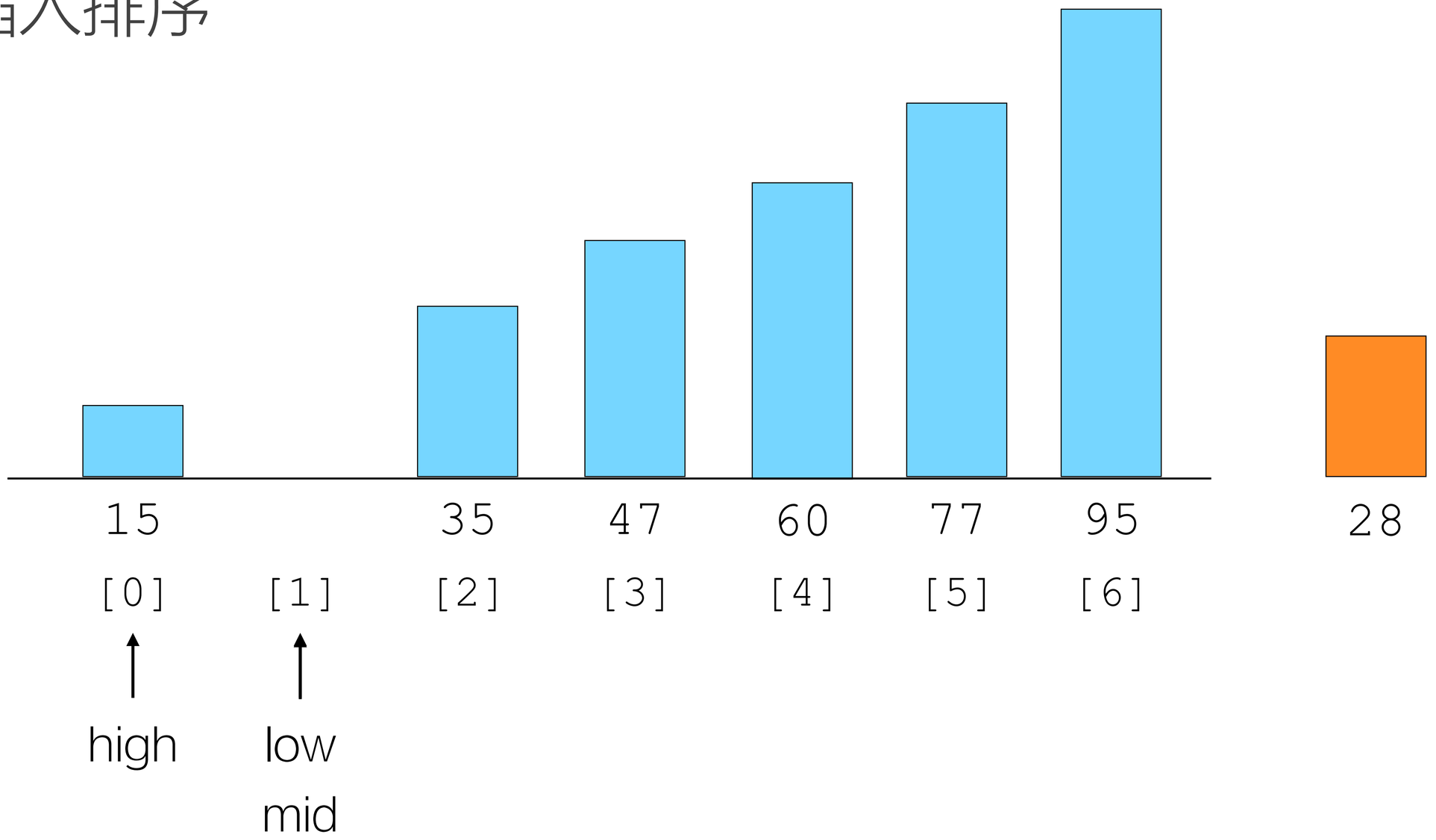
折半插入排序



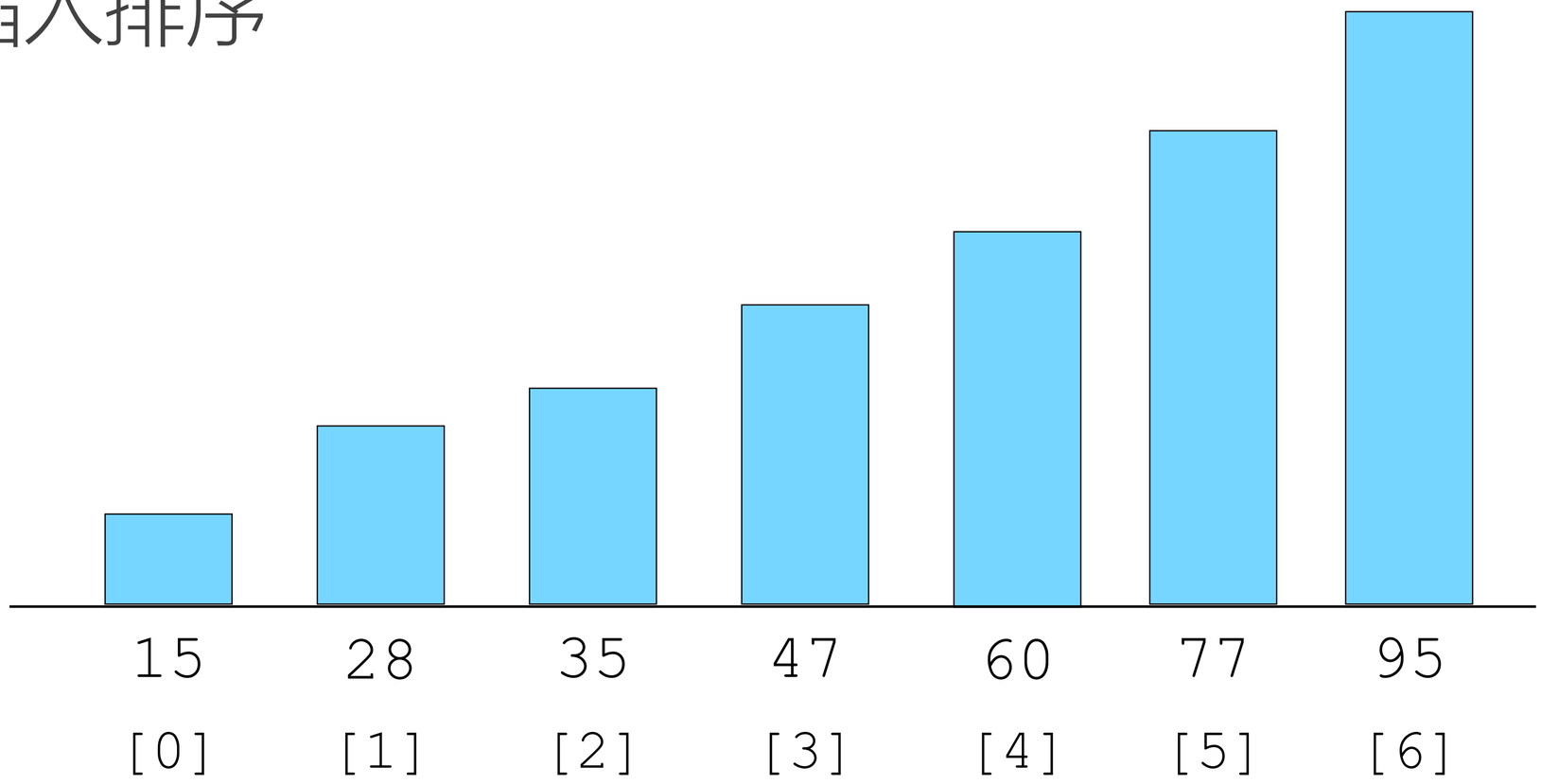
mid = (low + high) / 2

data[mid] > 28 ? high = mid − 1 : low = mid + 1

折半插入排序



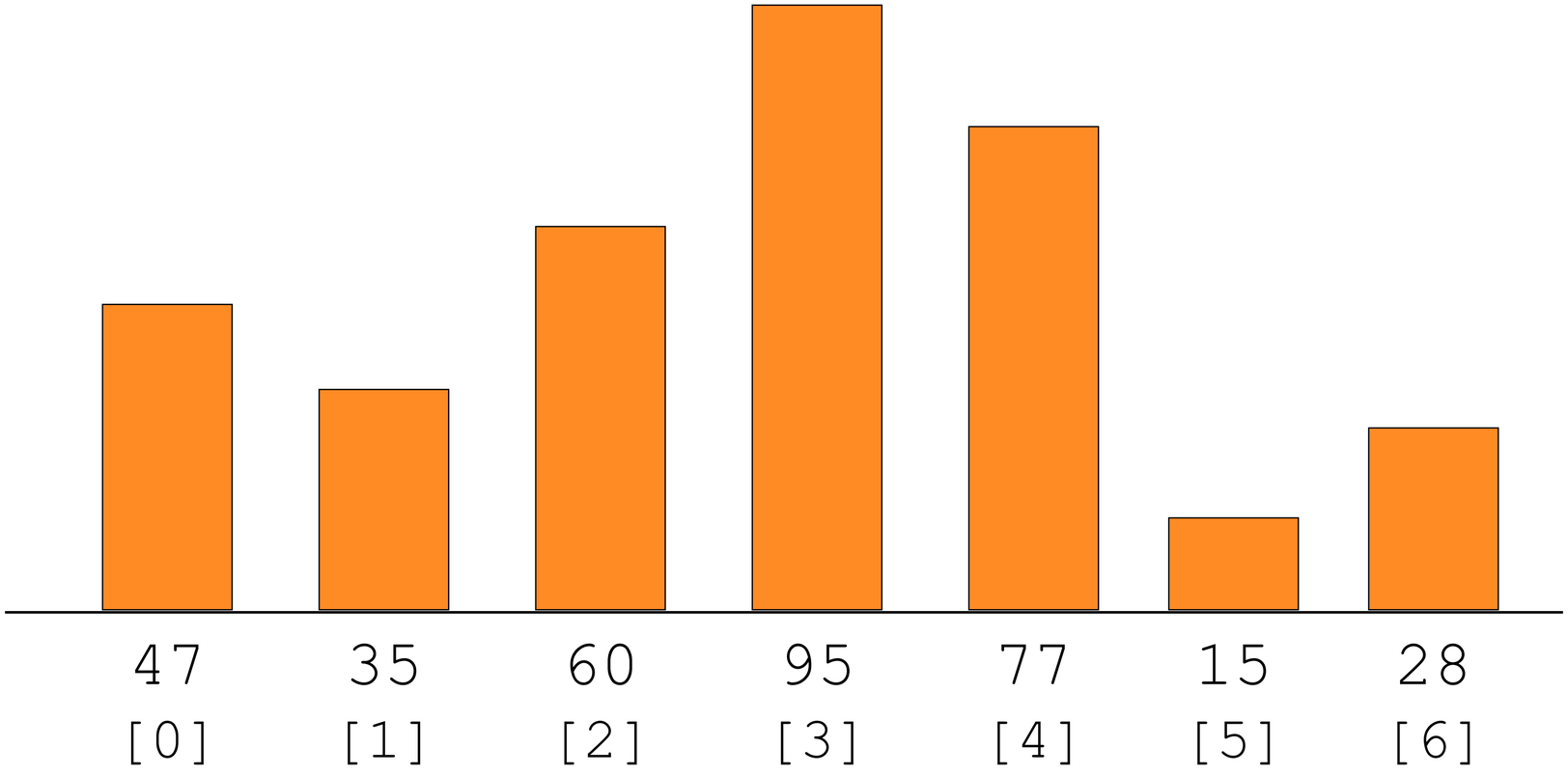
折半插入排序



折半插入排序

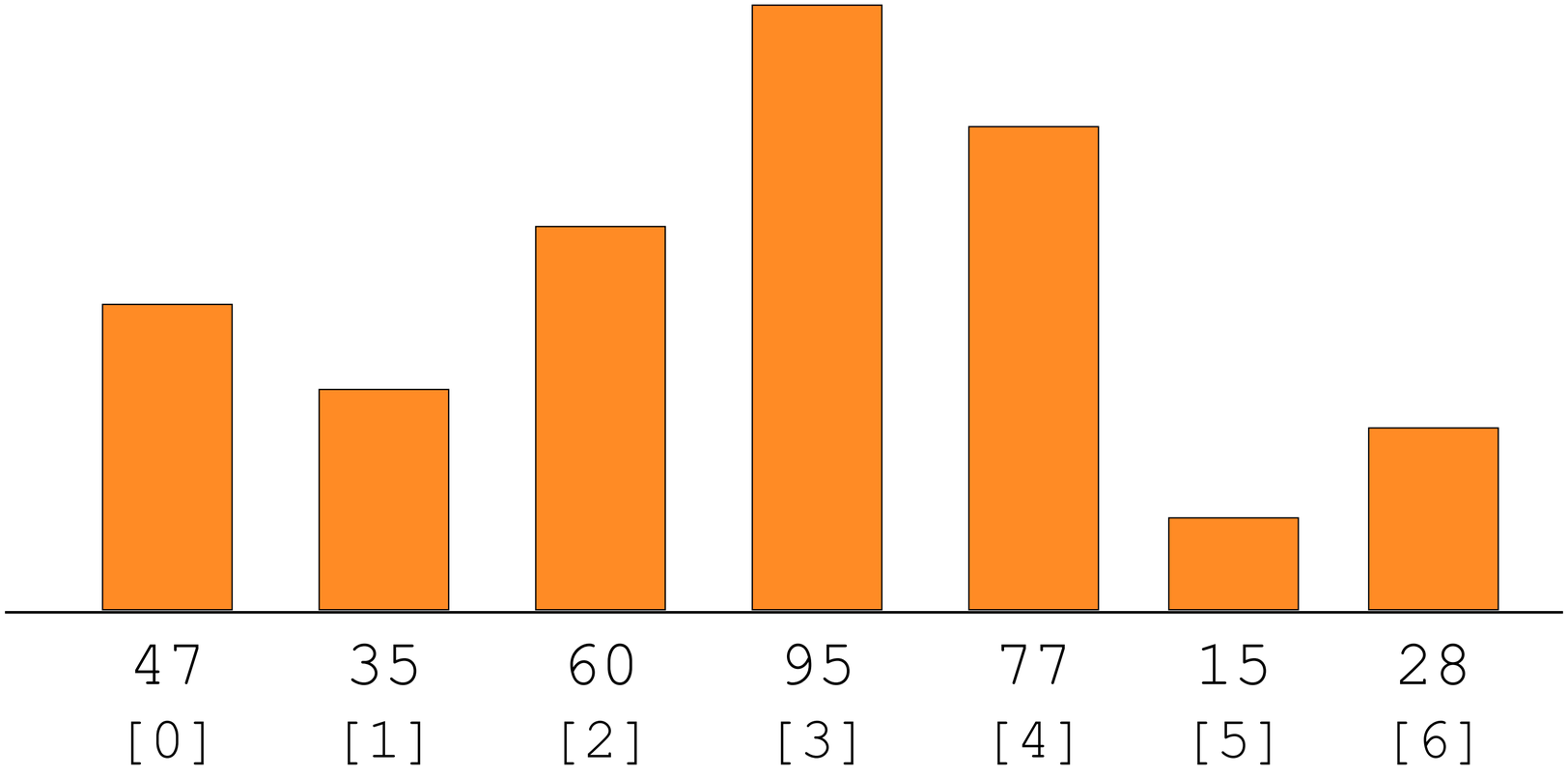
维度	描述
时间复杂度	最好：O(n log n)最坏：O(n^2)平均：O(n^2)
空间复杂度	O(1)
算法稳定性	稳定
适用场景	数据基本有序或对稳定性有要求时效果较好

简单选择排序



简单选择排序

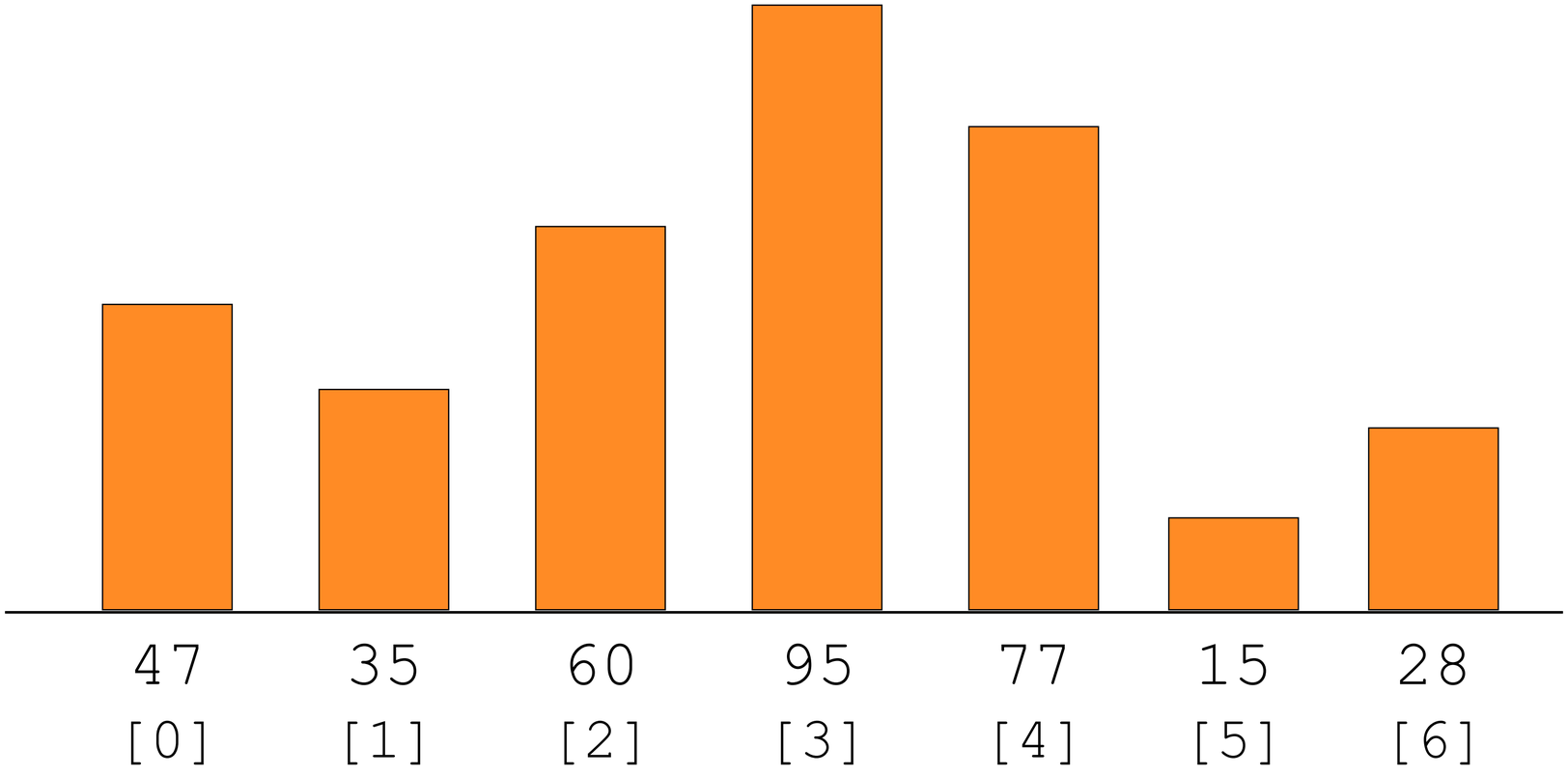
min



简单选择排序

min

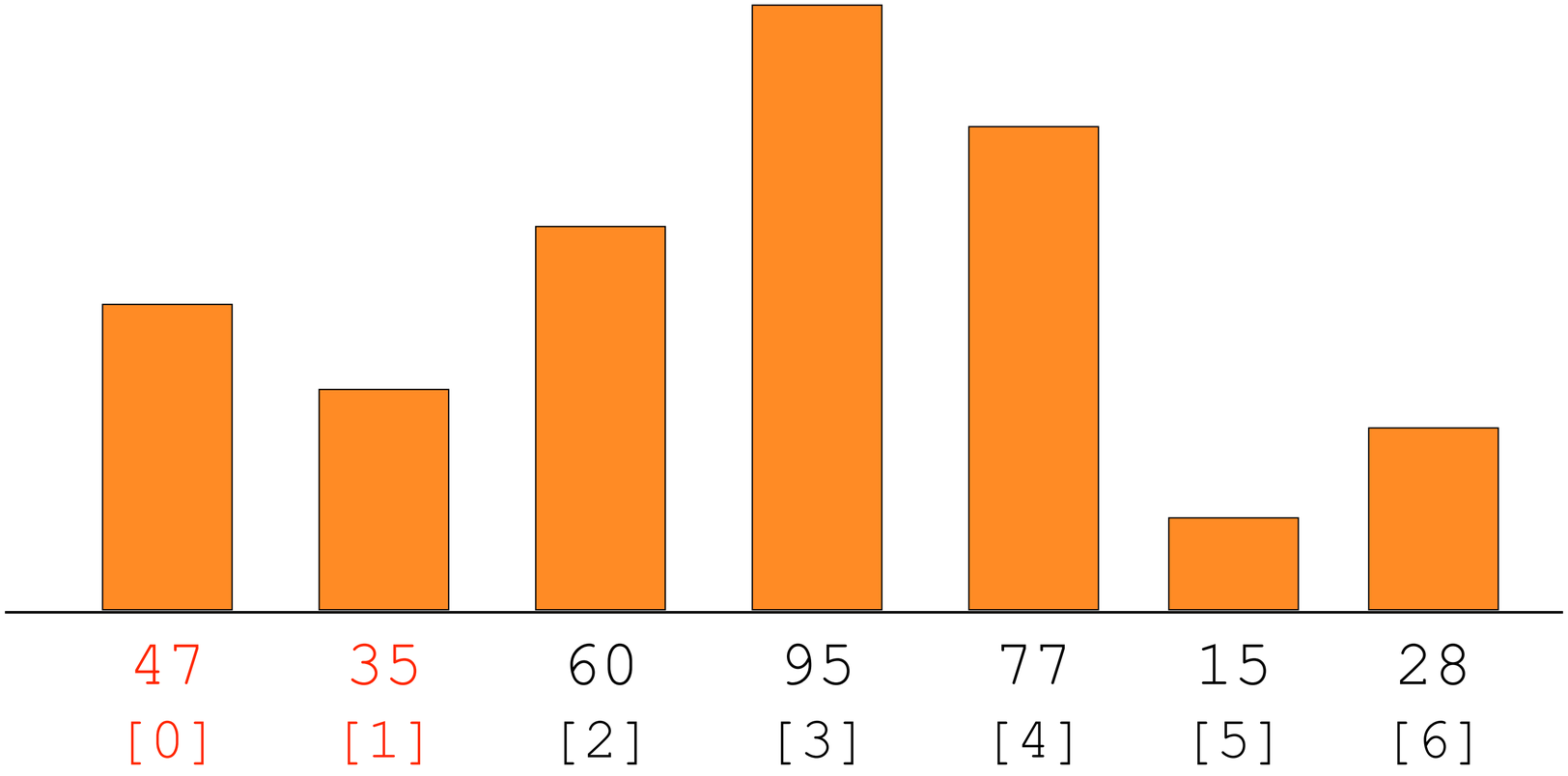
0



简单选择排序

min

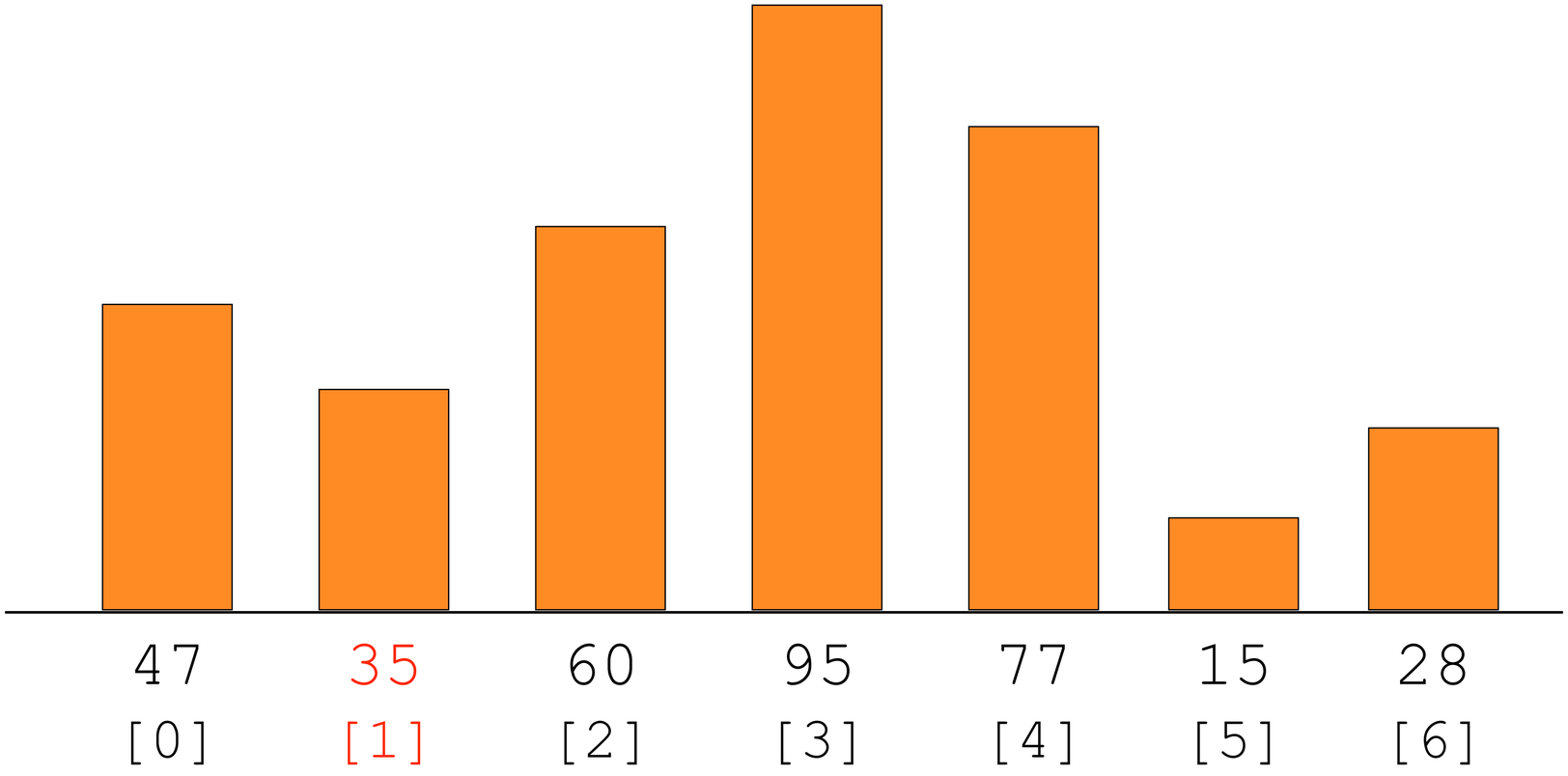
0



简单选择排序

min

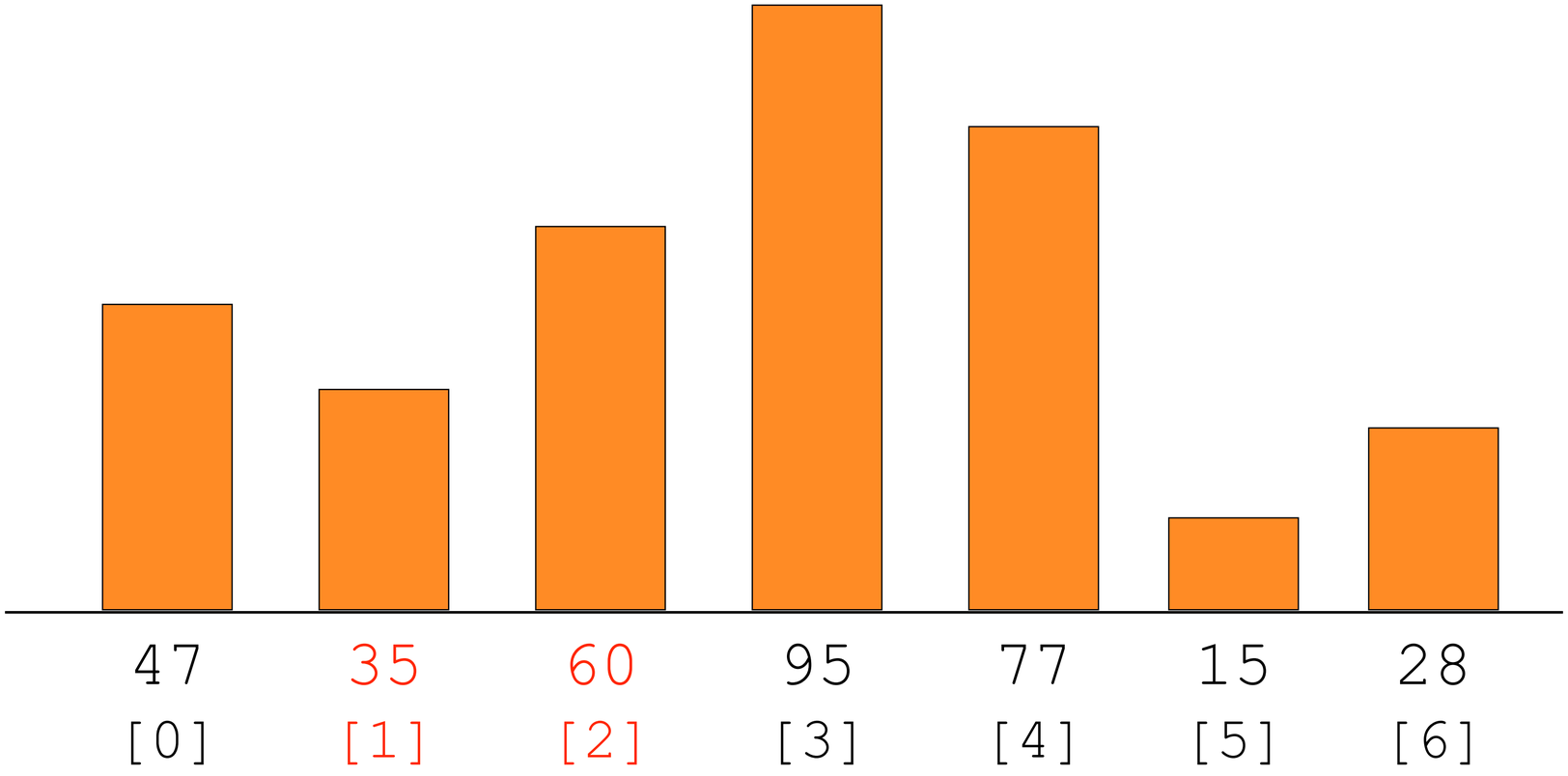
1



简单选择排序

min

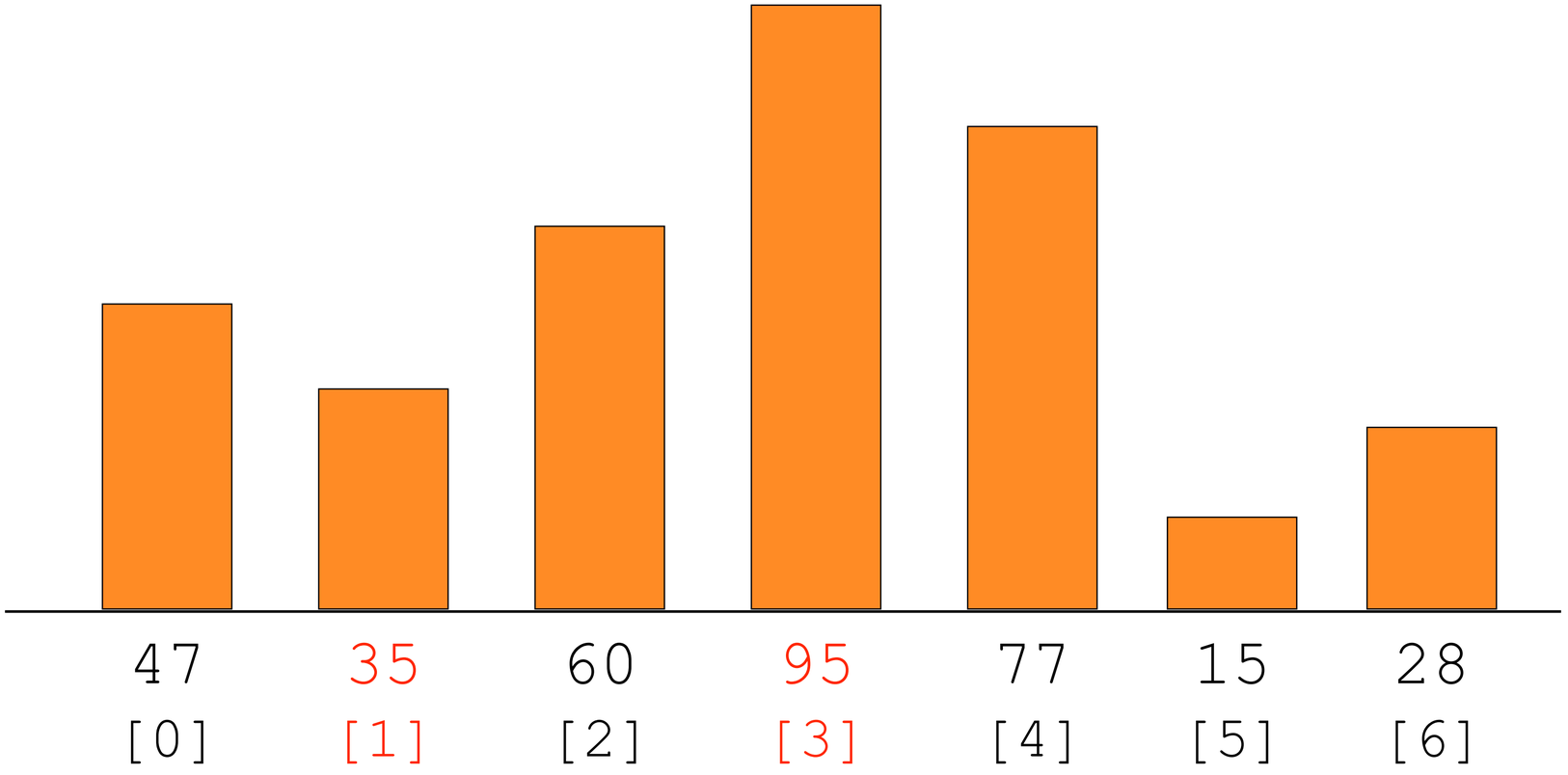
1



简单选择排序

min

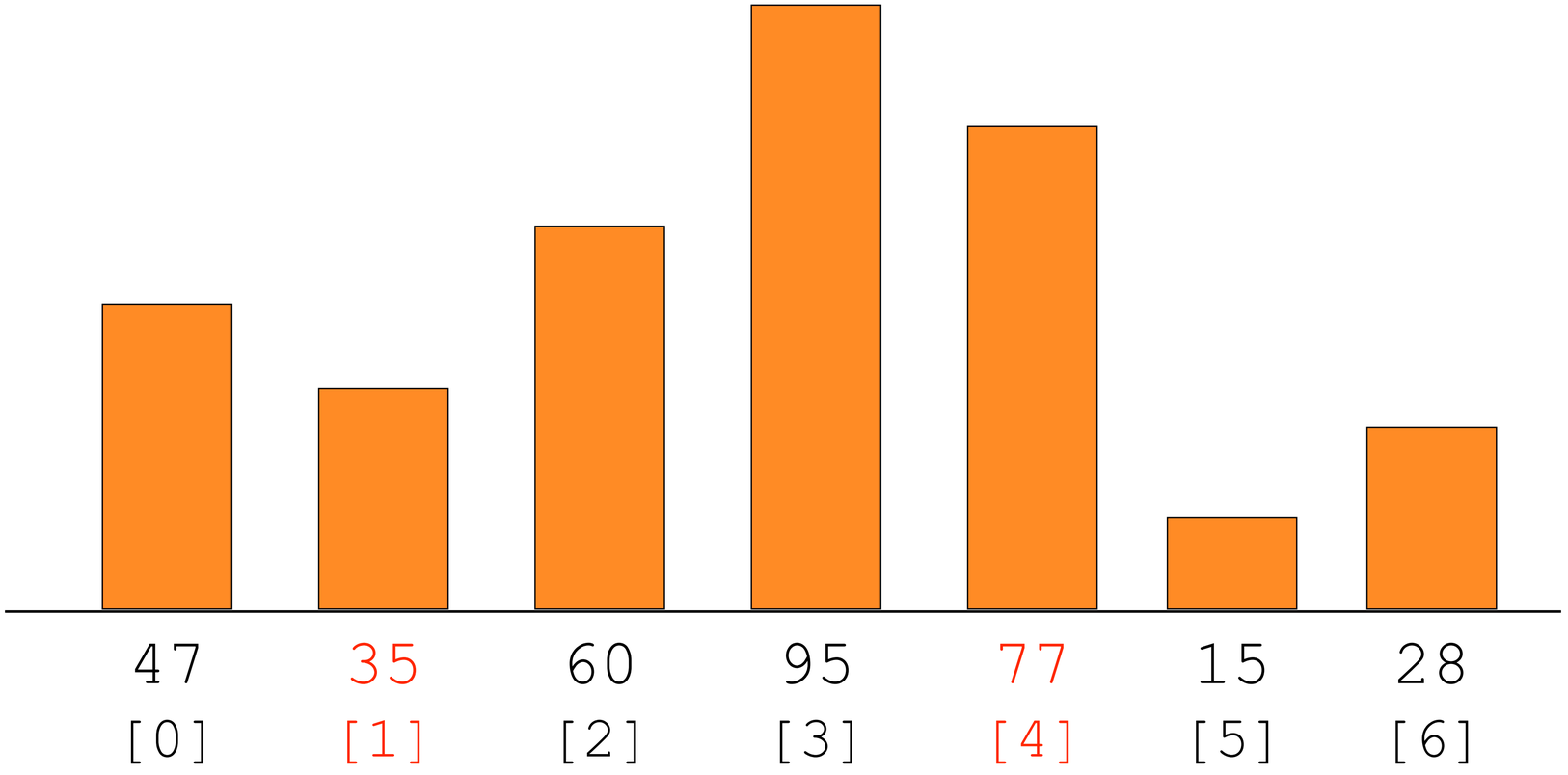
1



简单选择排序

min

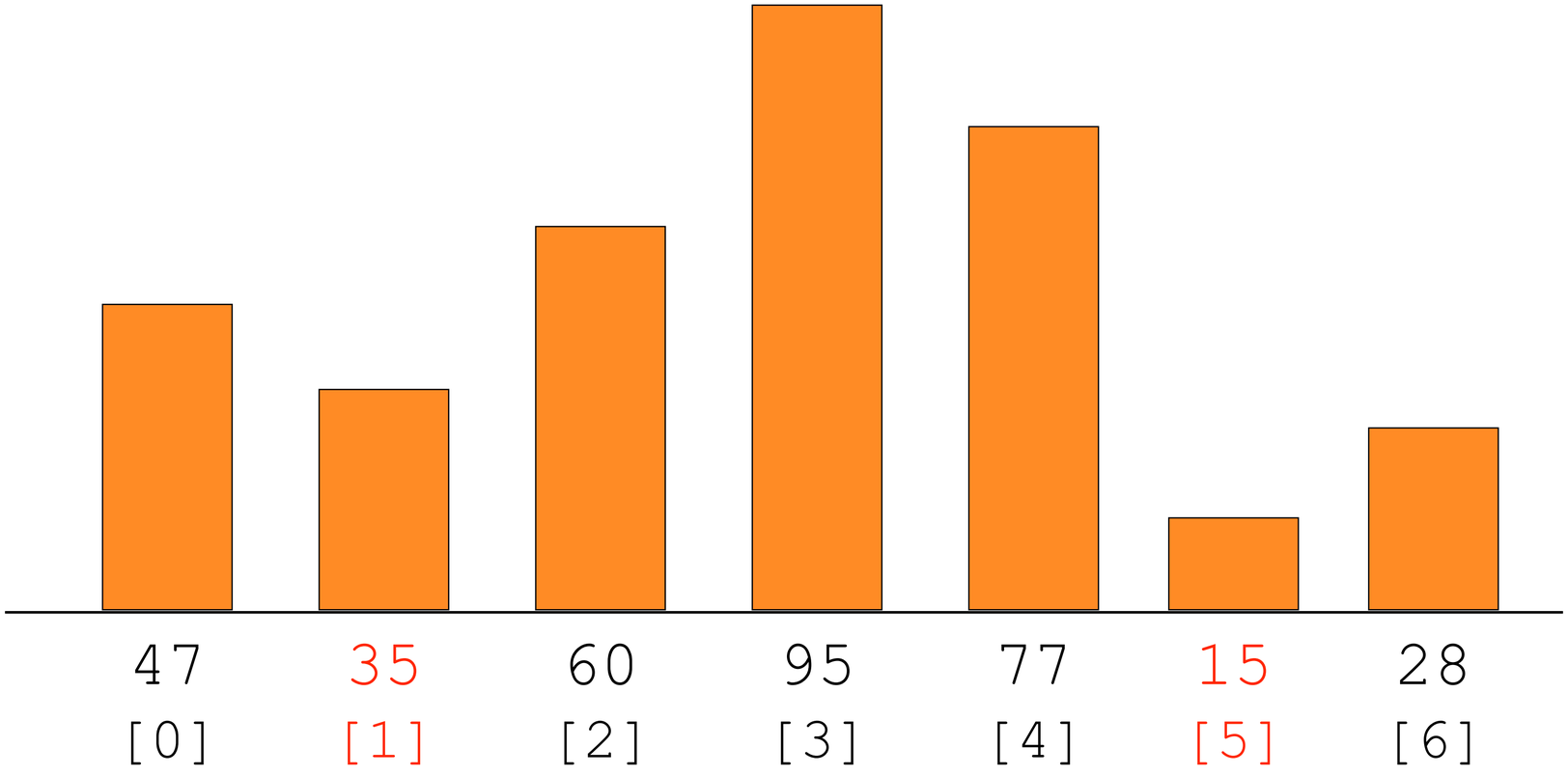
1



简单选择排序

min

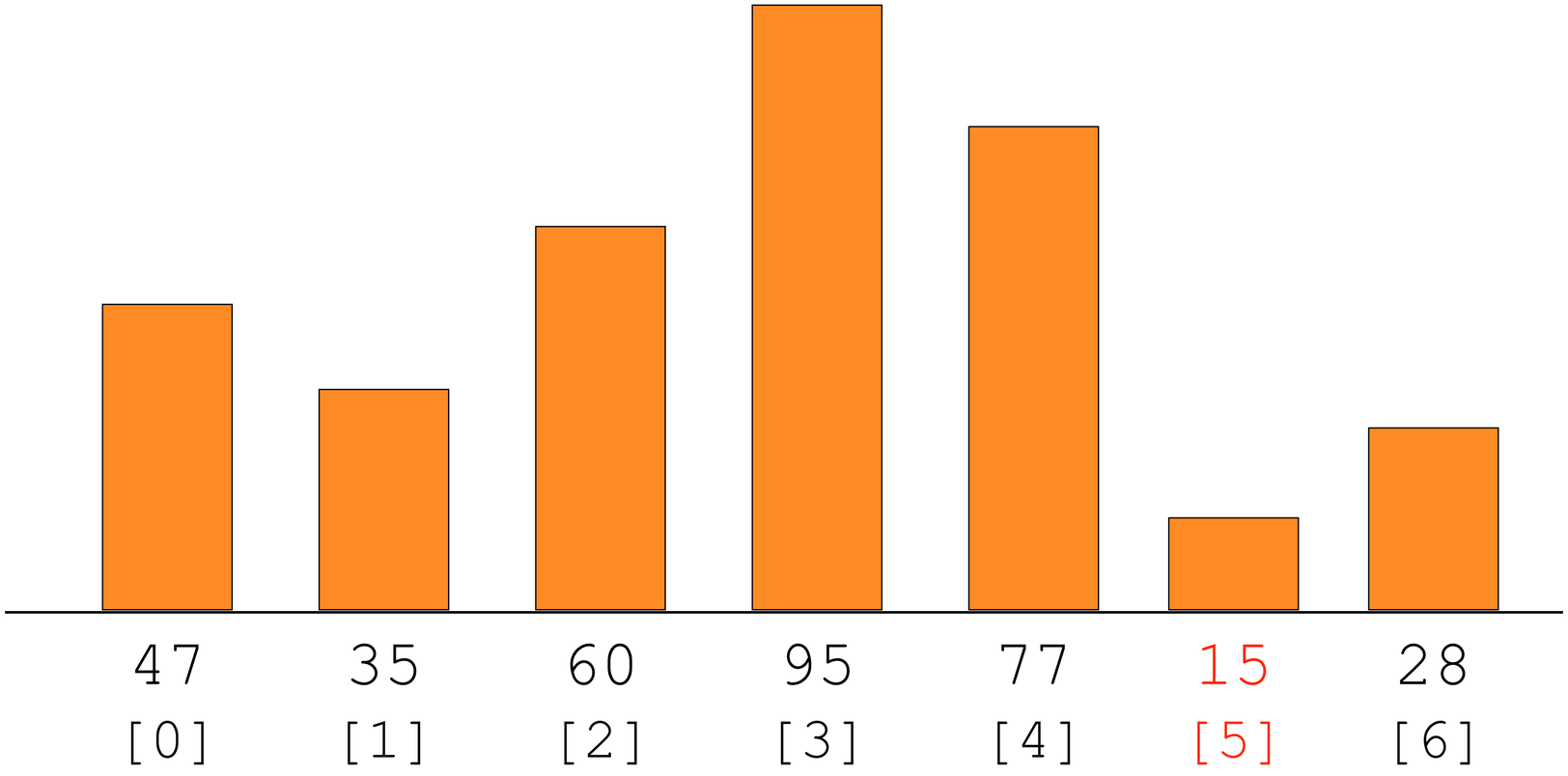
1



简单选择排序

min

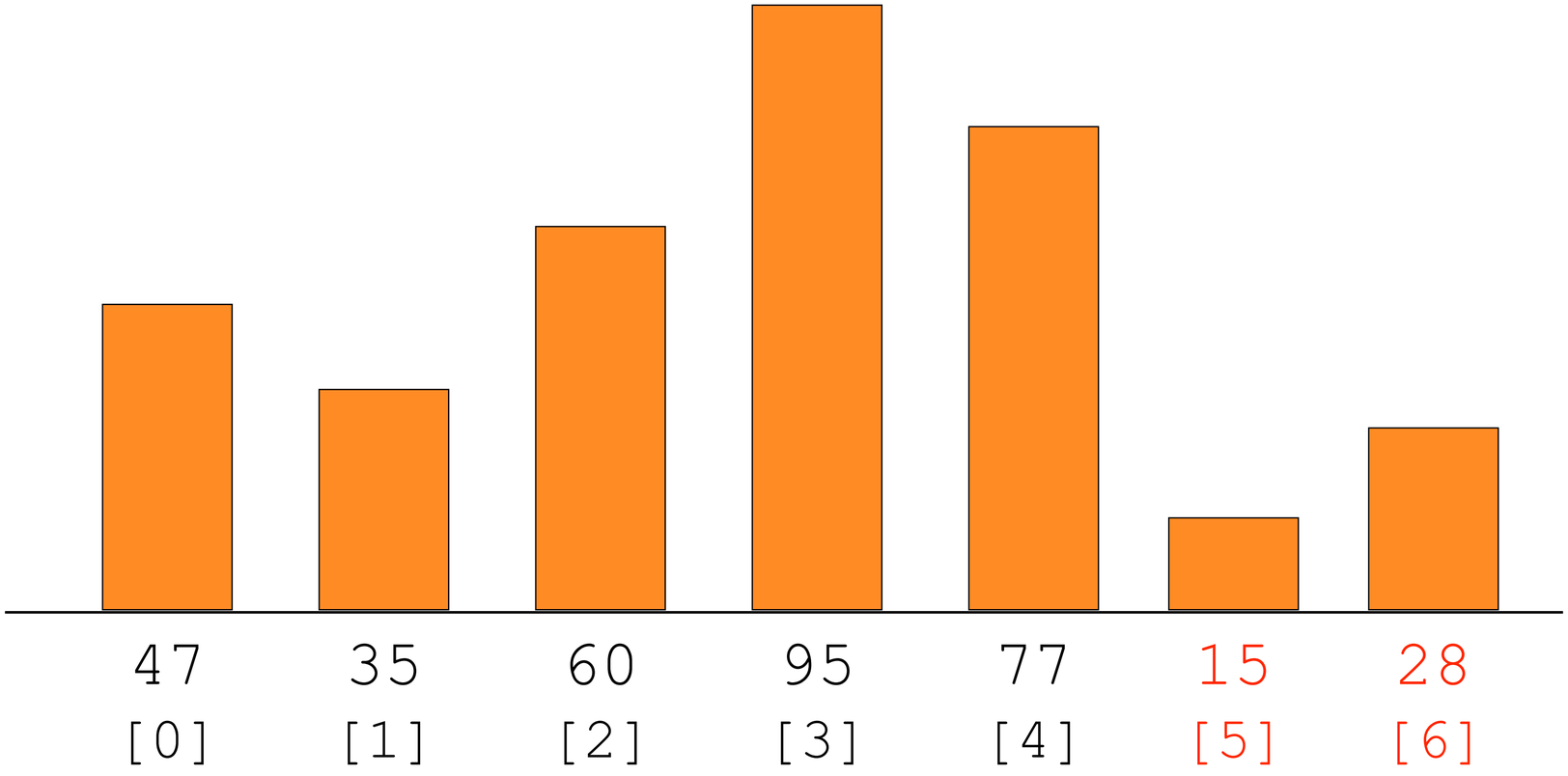
5



简单选择排序

min

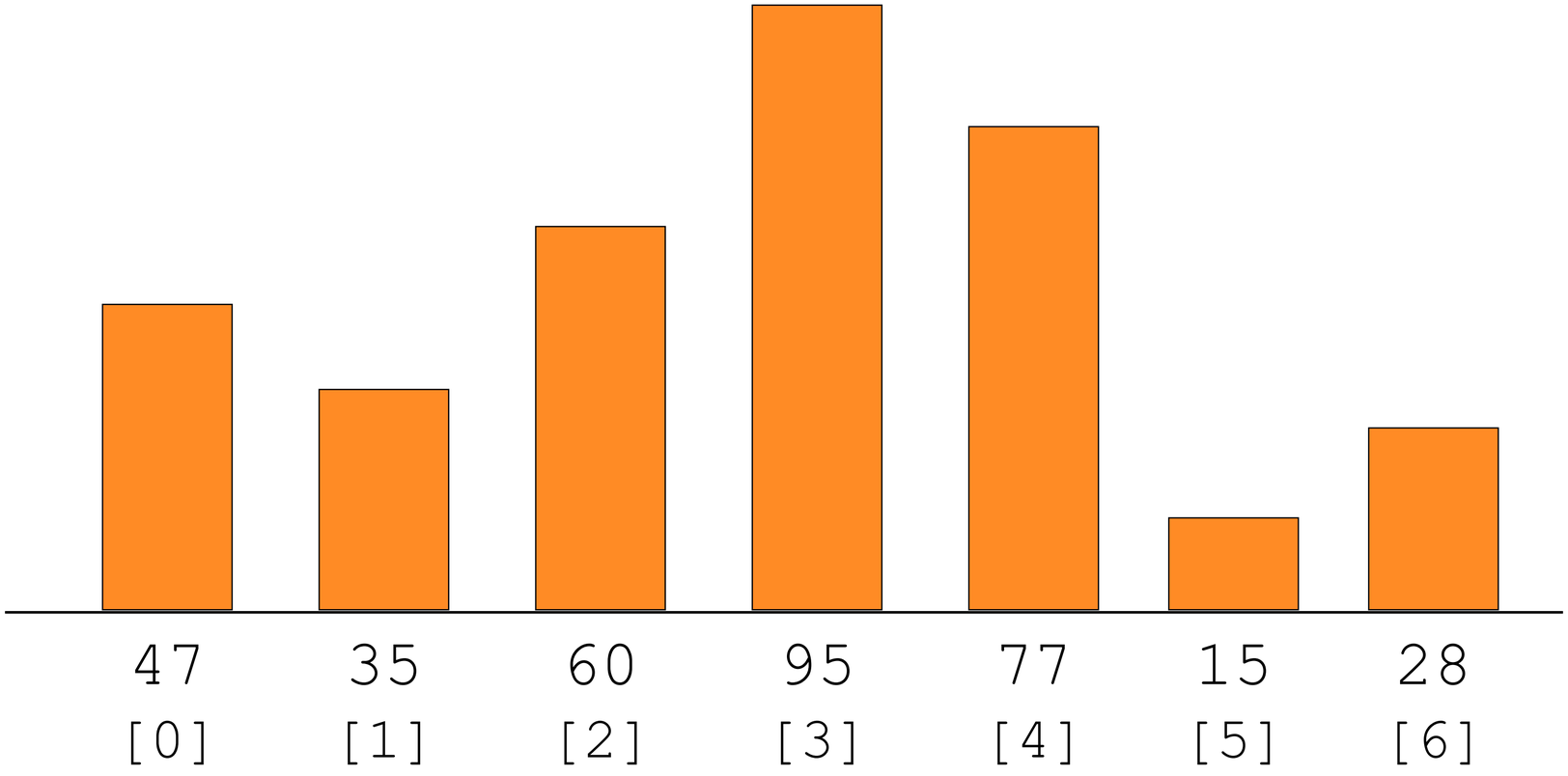
5



简单选择排序

min

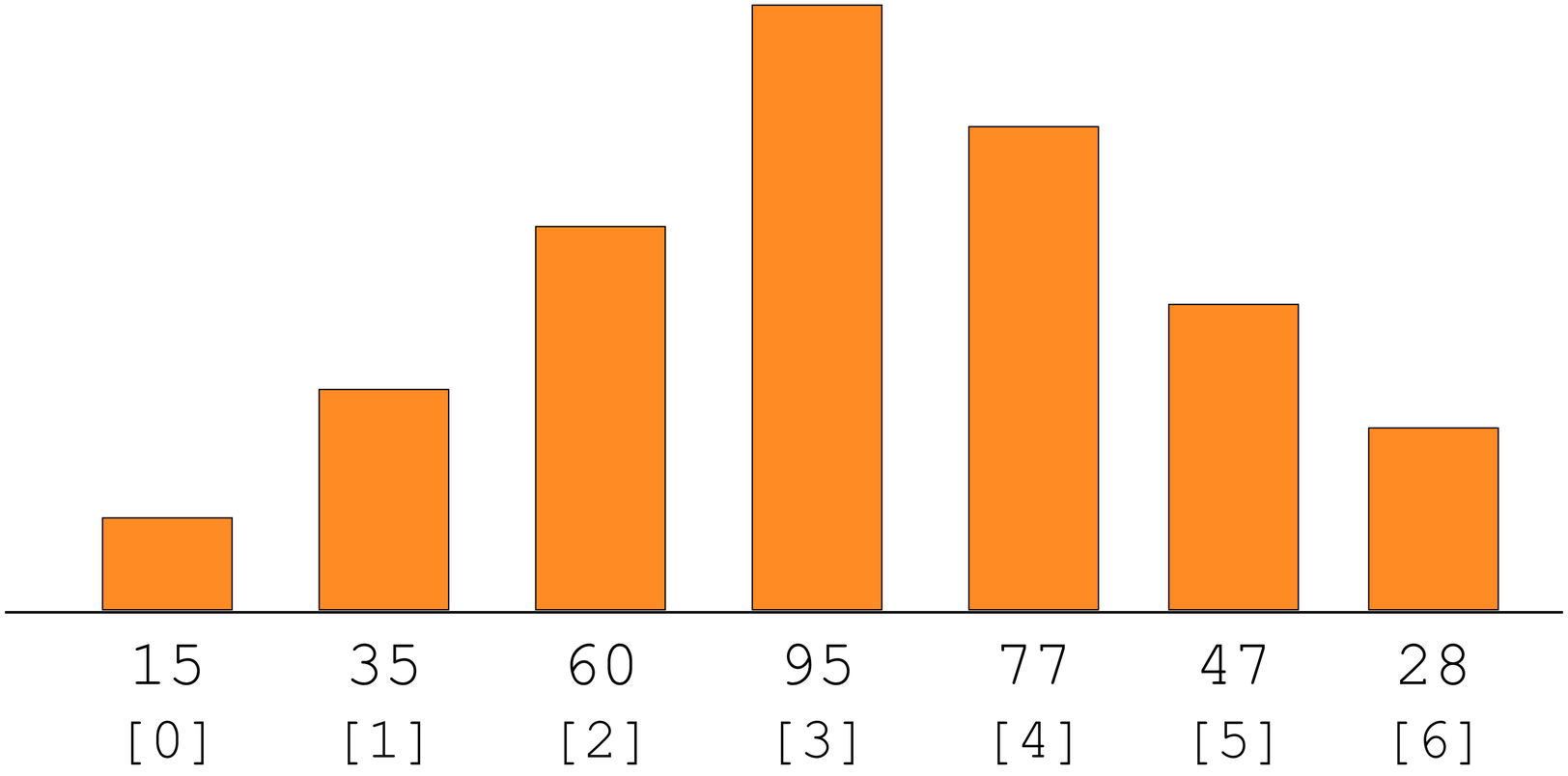
5



简单选择排序

min

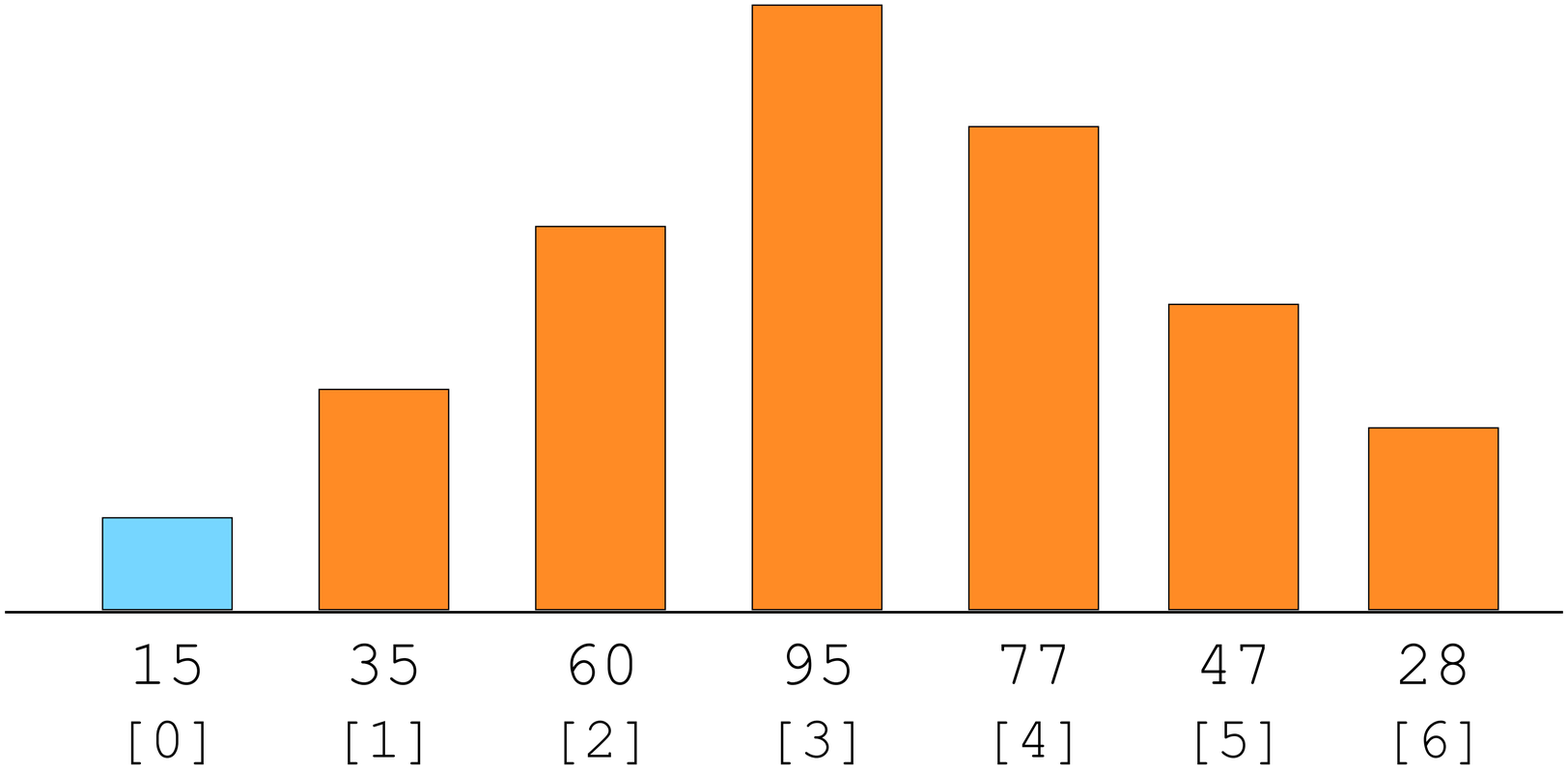
5



简单选择排序

min

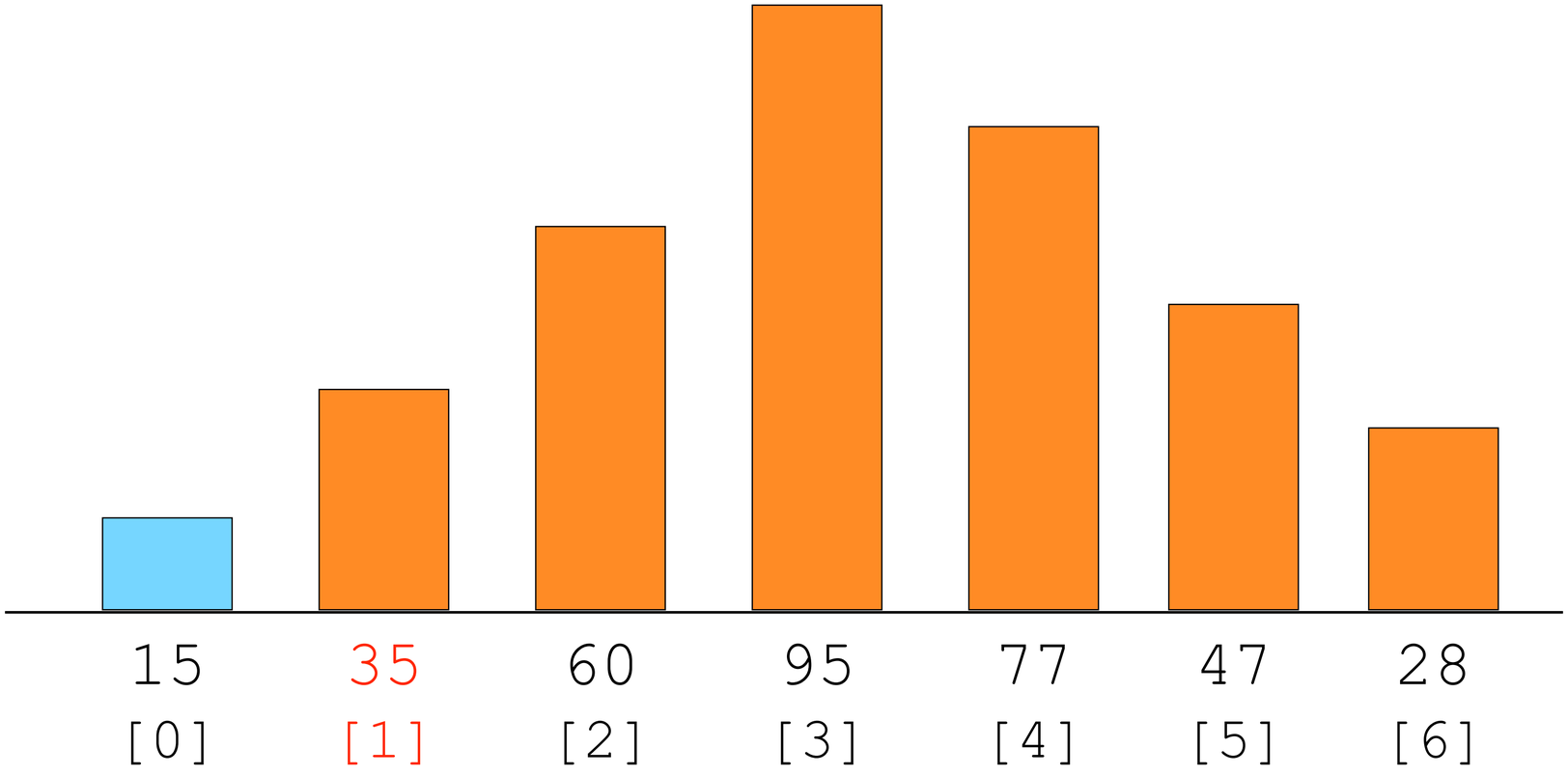
5



简单选择排序

min

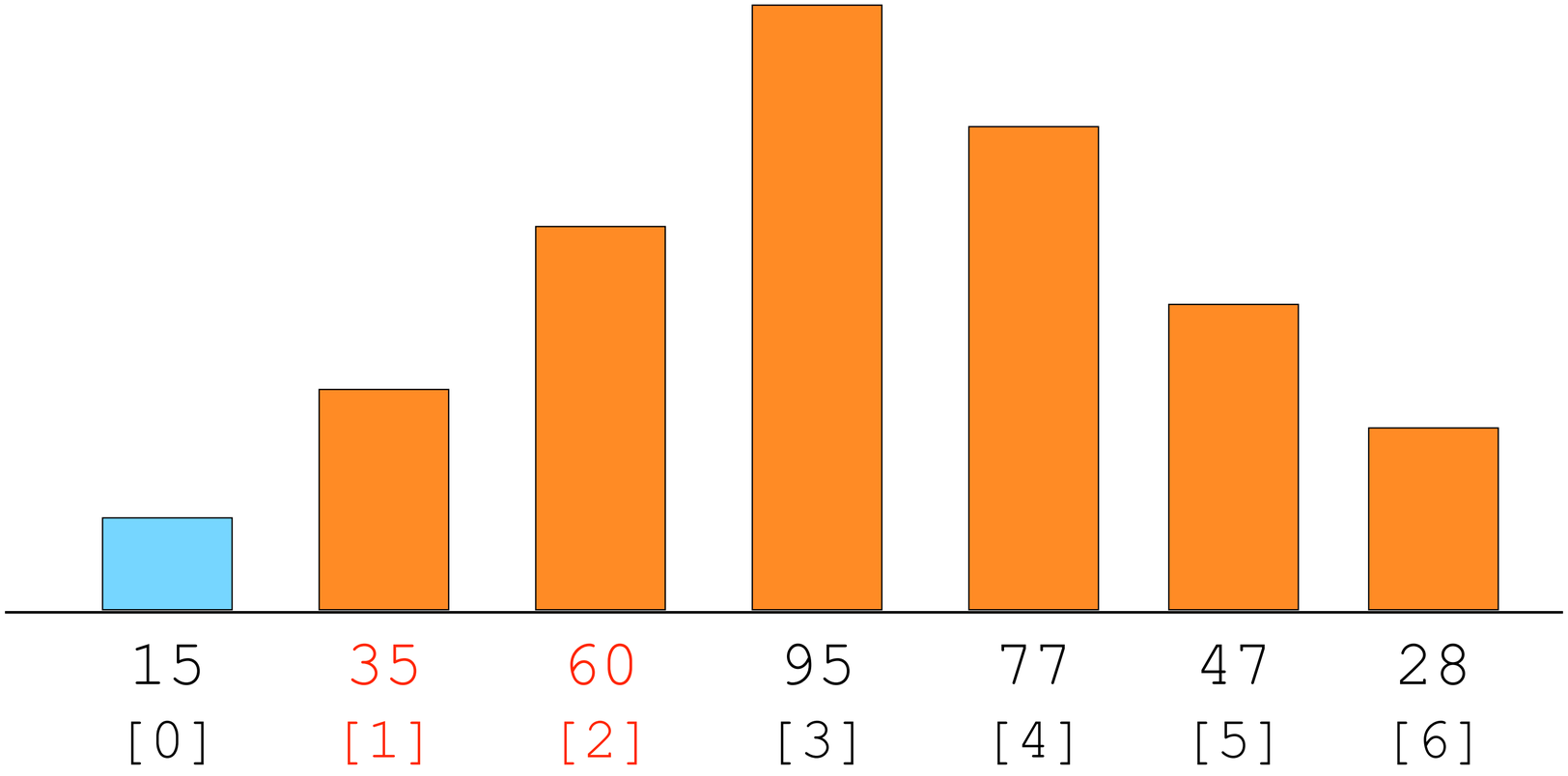
1



简单选择排序

min

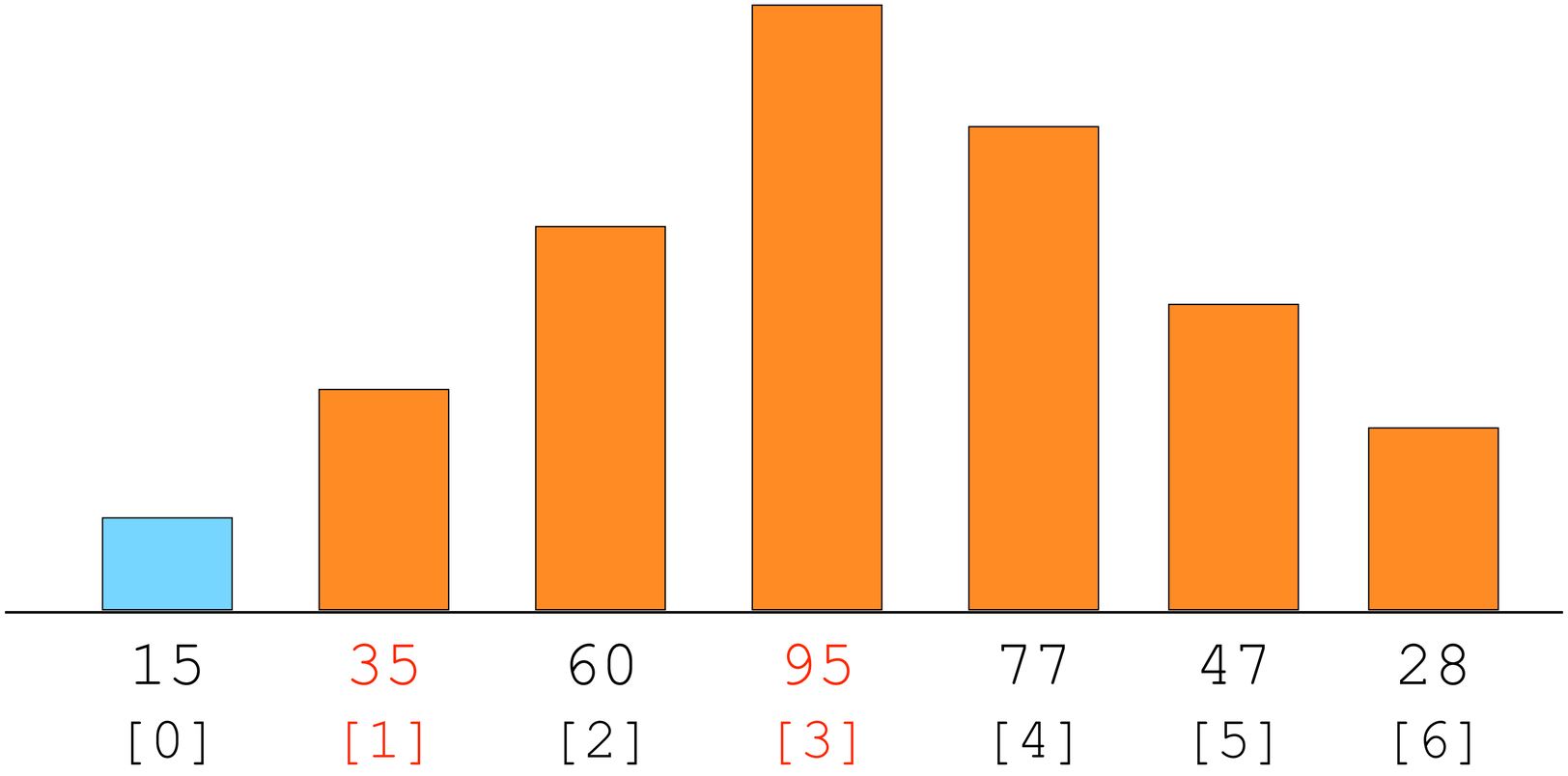
1



简单选择排序

min

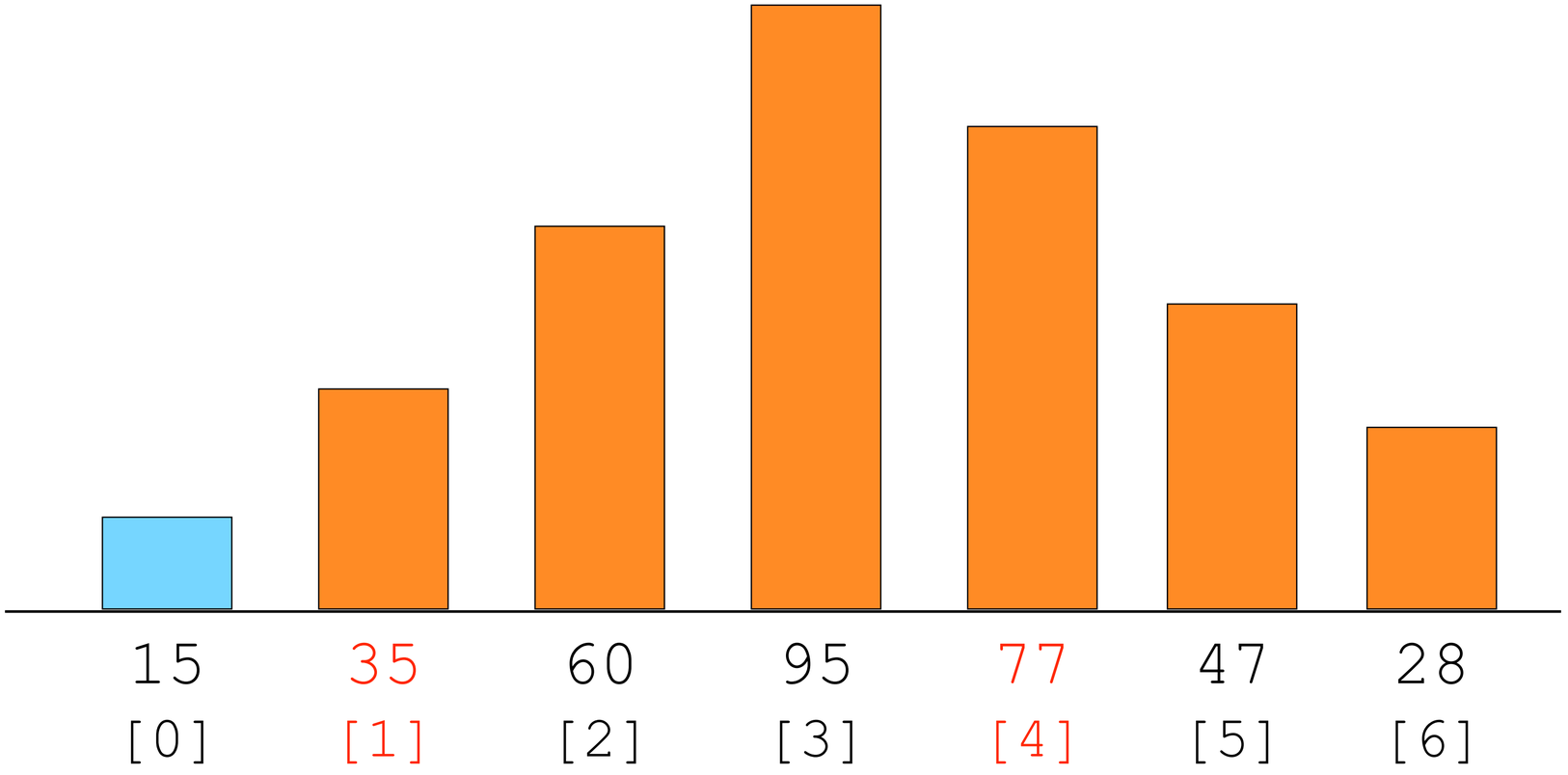
1



简单选择排序

min

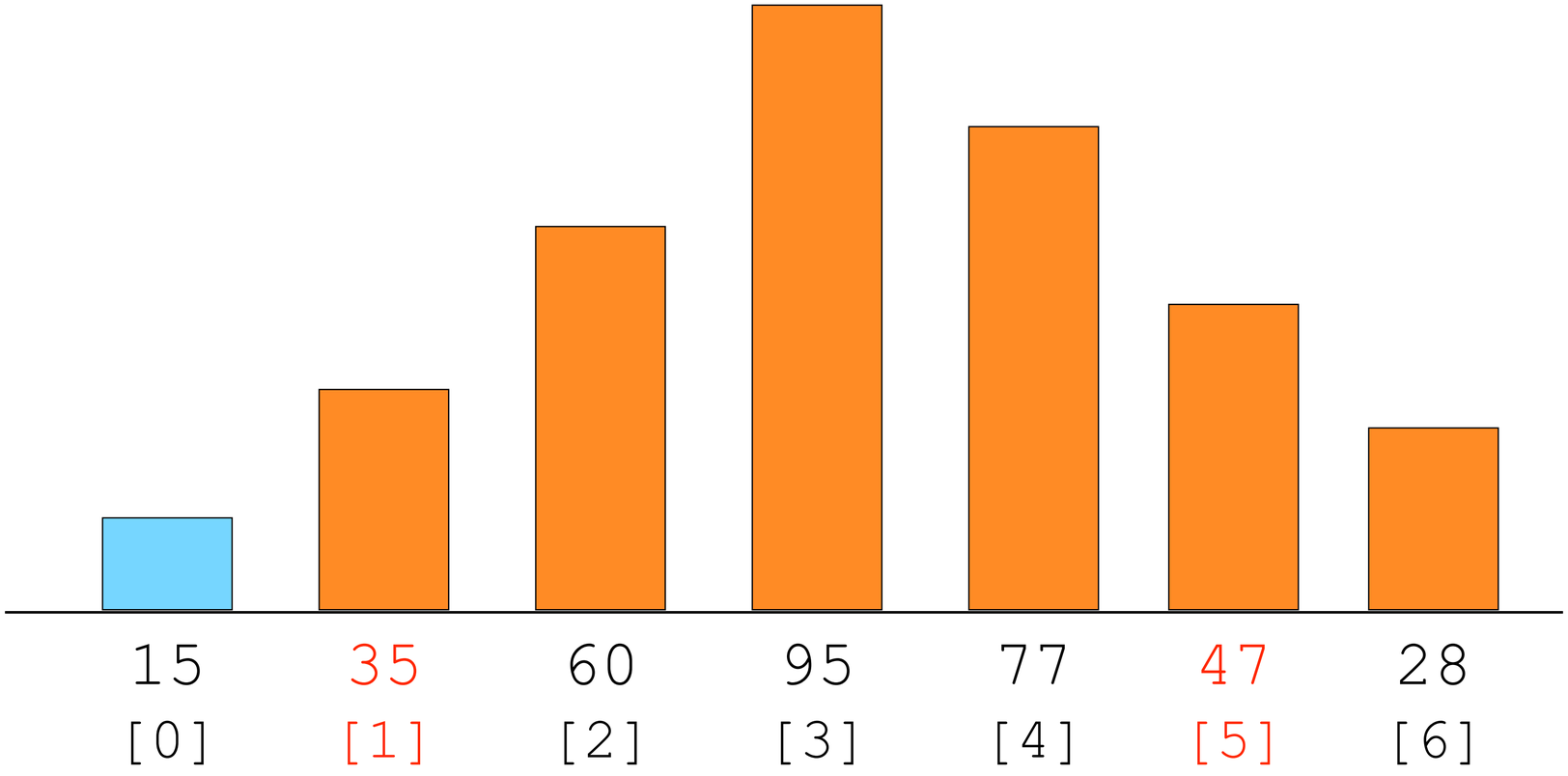
1



简单选择排序

min

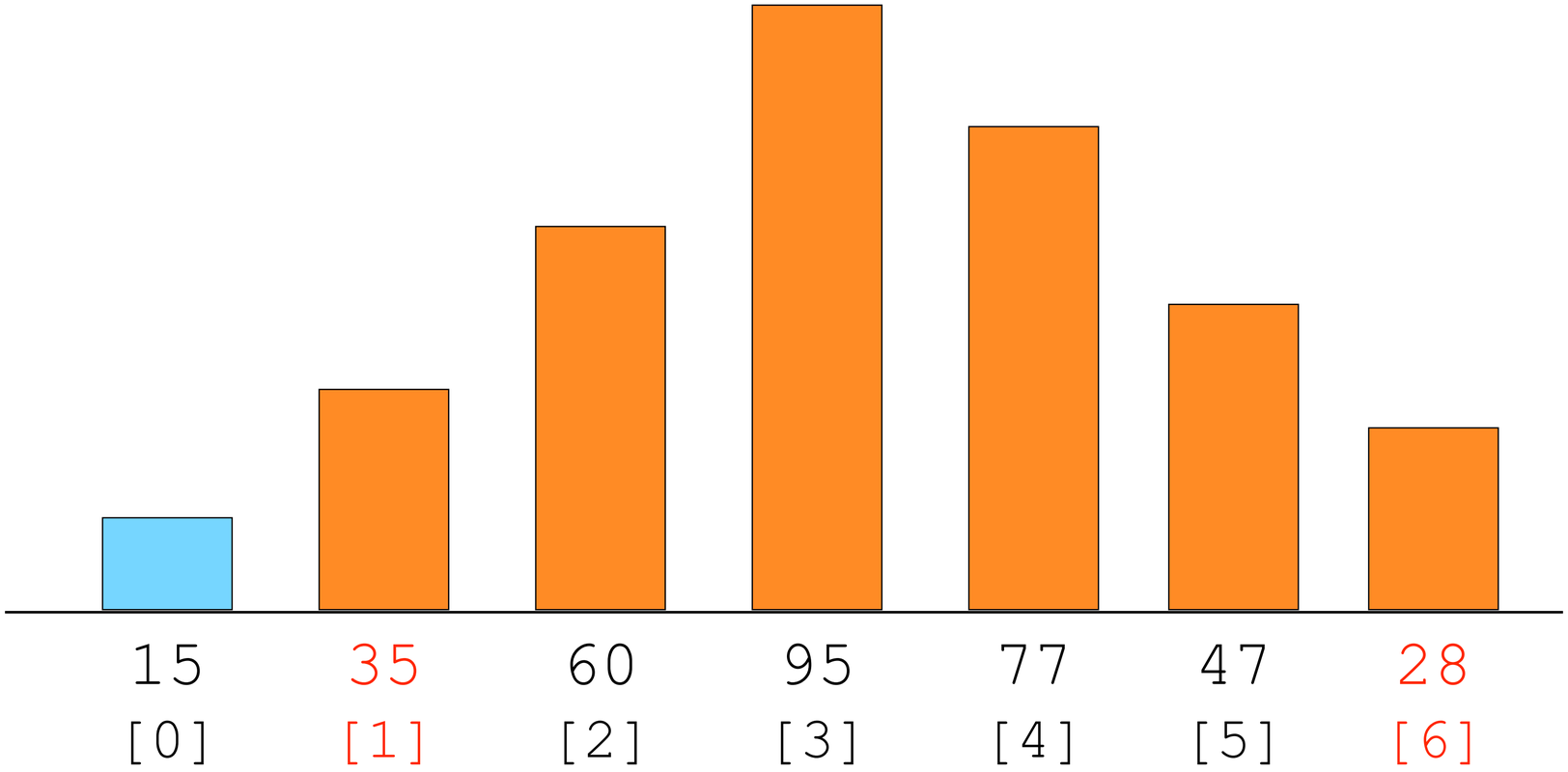
1



简单选择排序

min

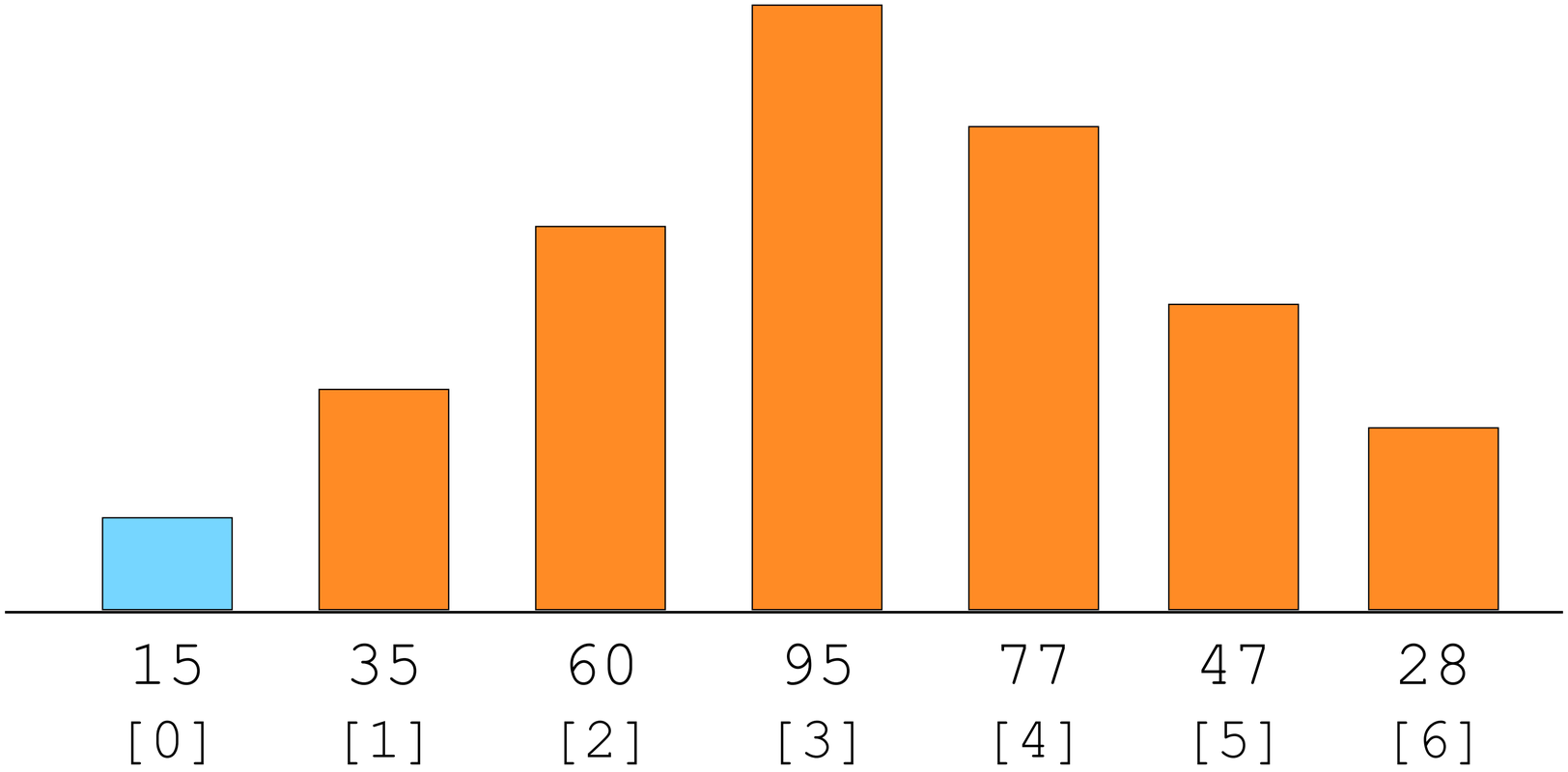
1



简单选择排序

min

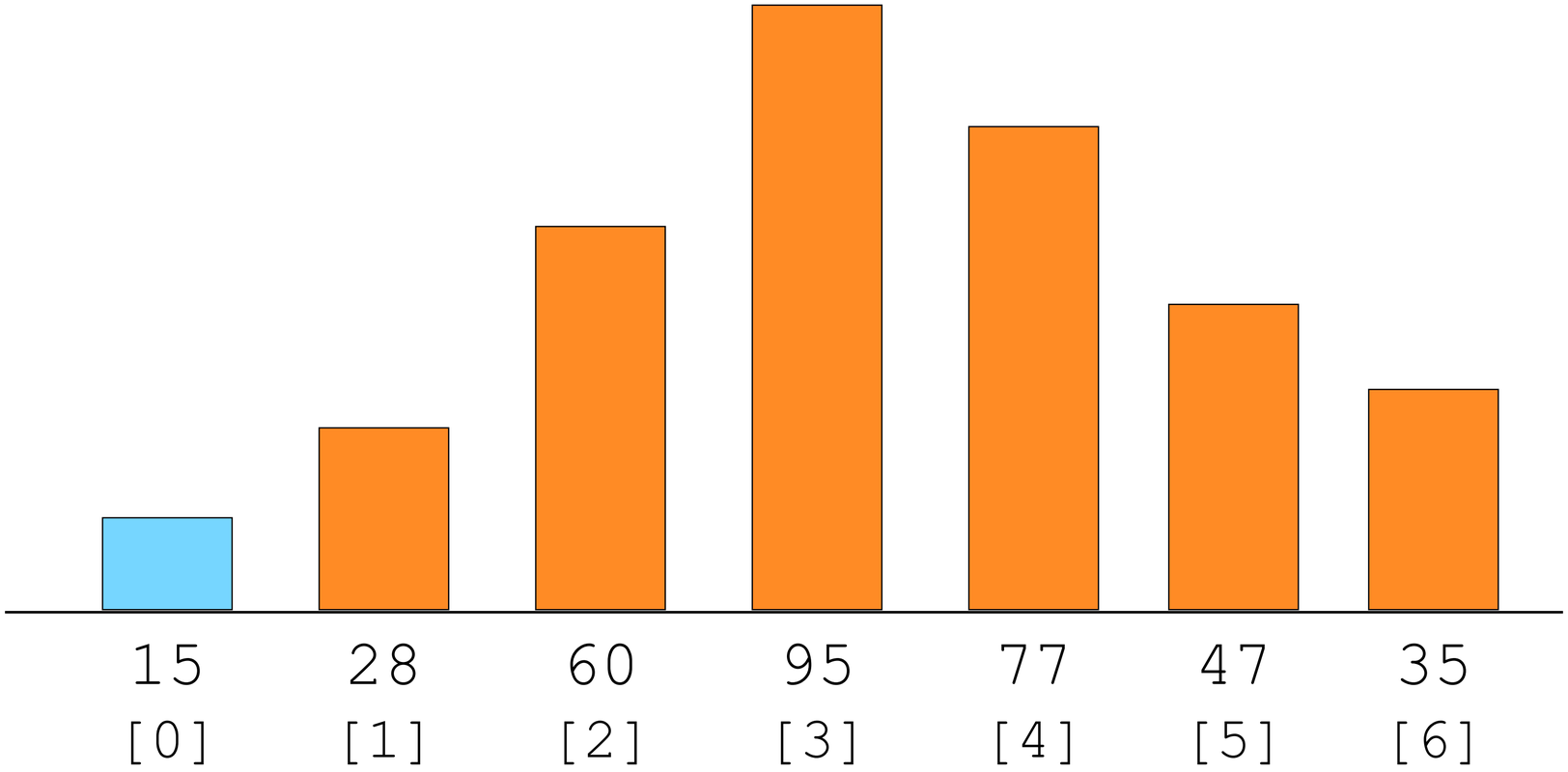
6



简单选择排序

min

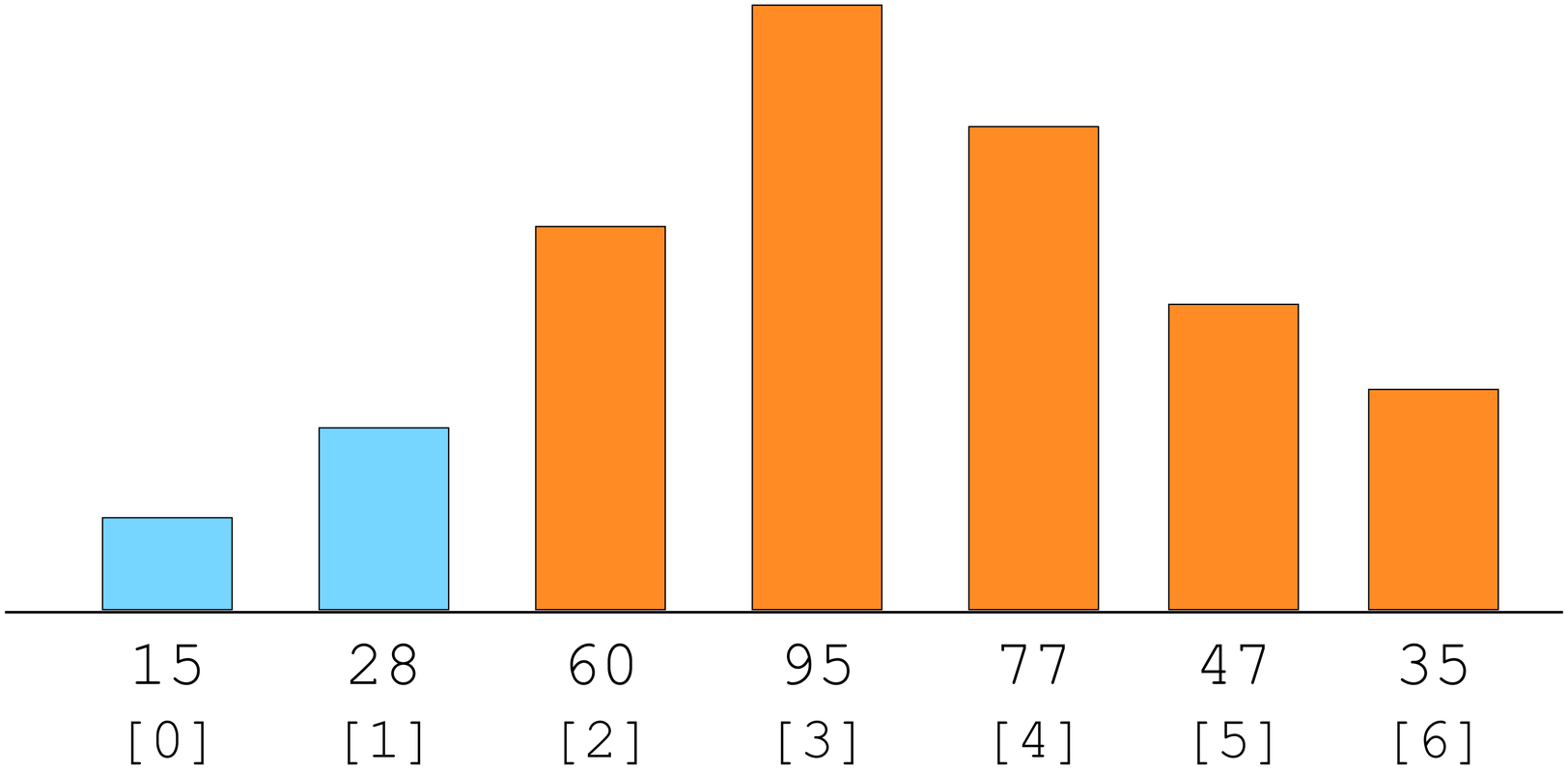
6



简单选择排序

min

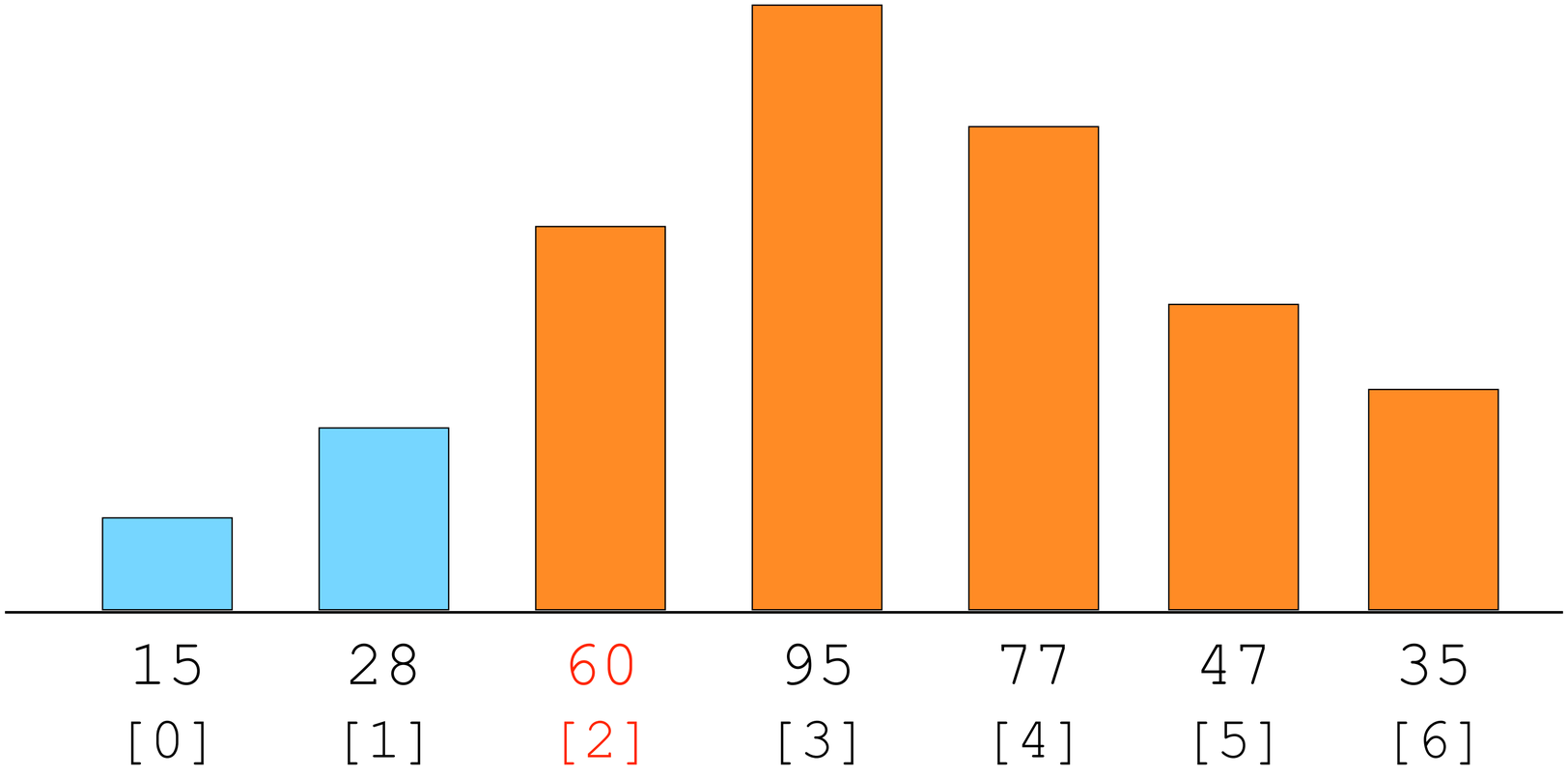
6



简单选择排序

min

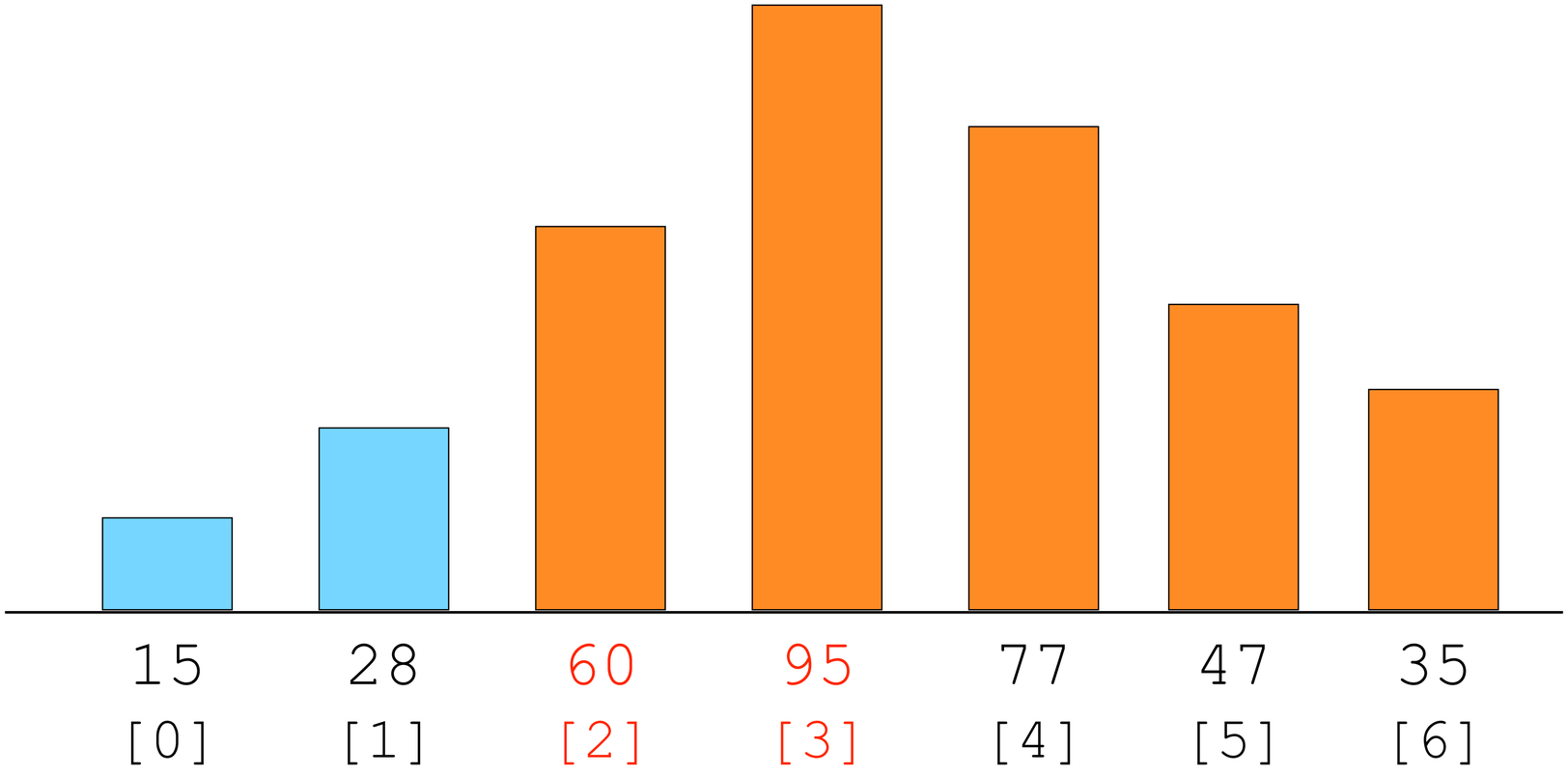
2



简单选择排序

min

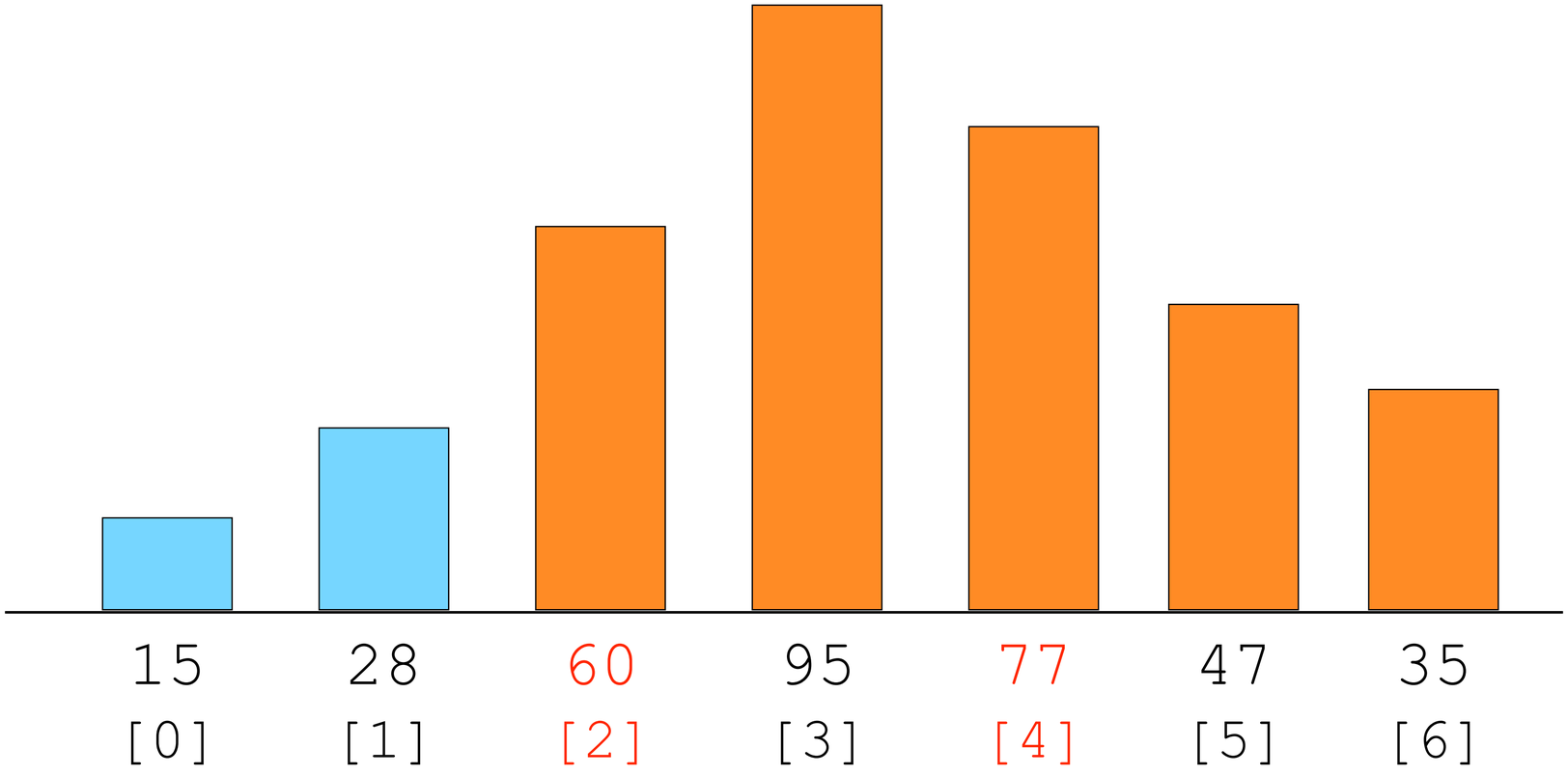
2



简单选择排序

min

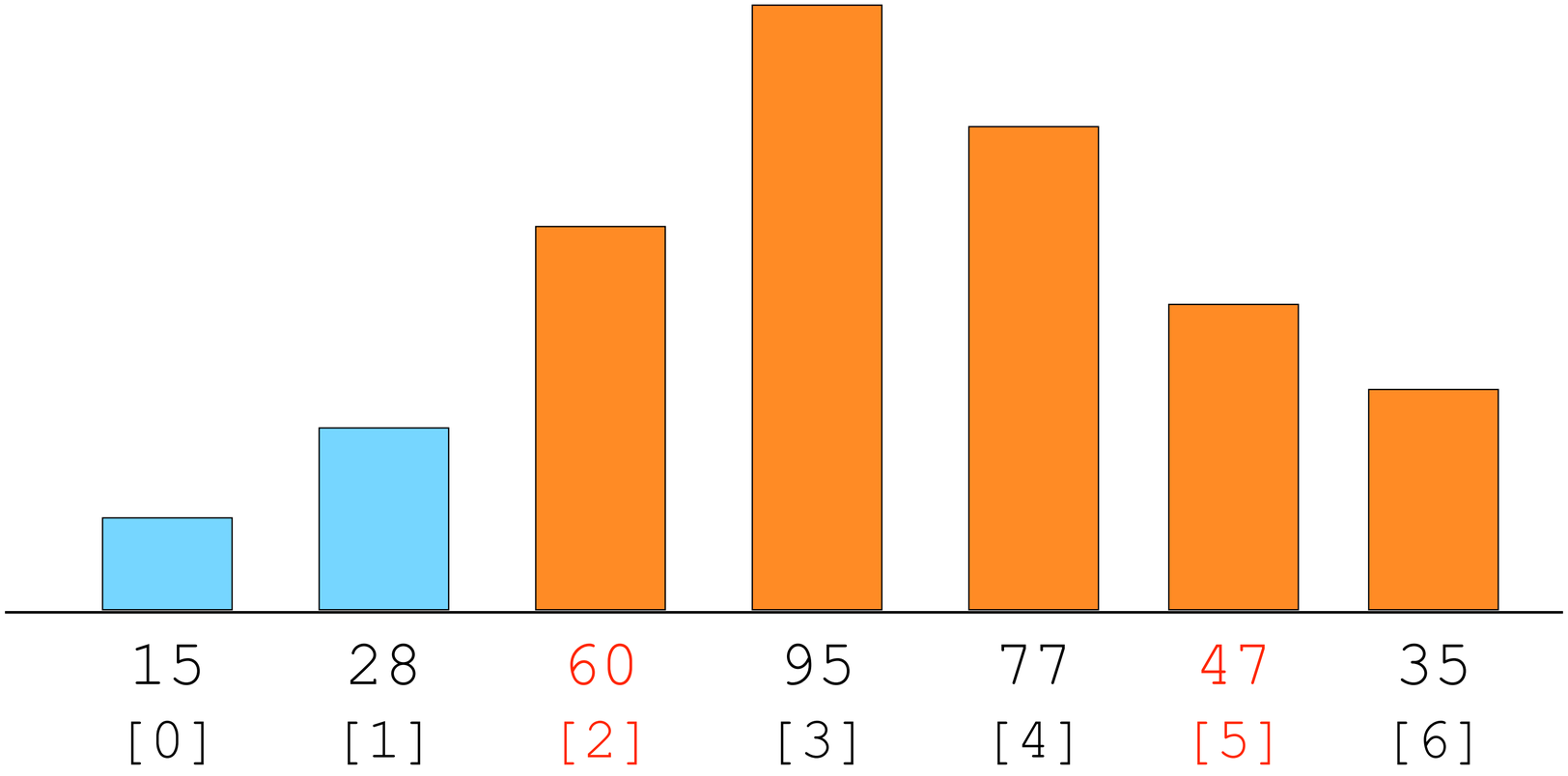
2



简单选择排序

min

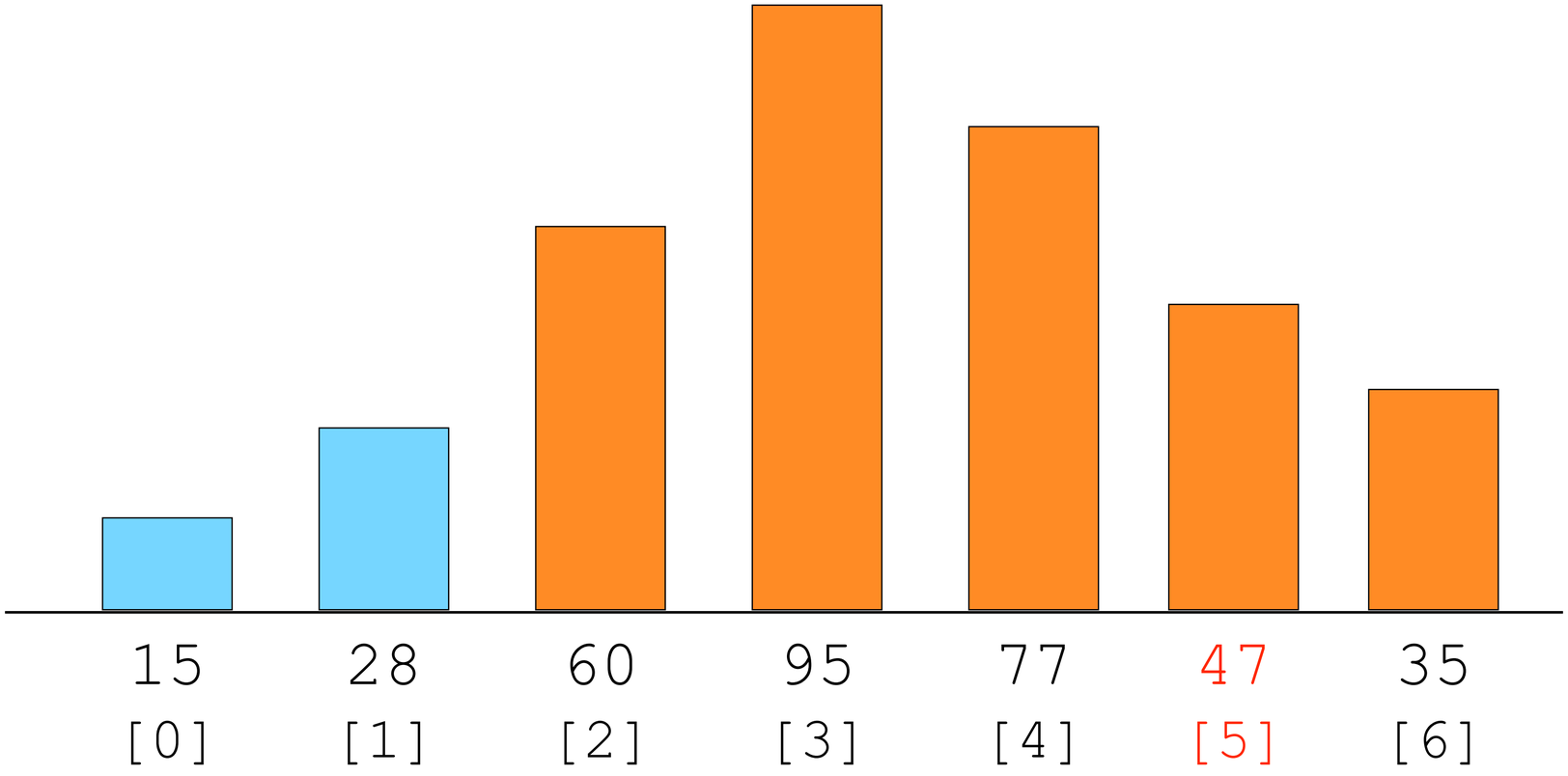
2



简单选择排序

min

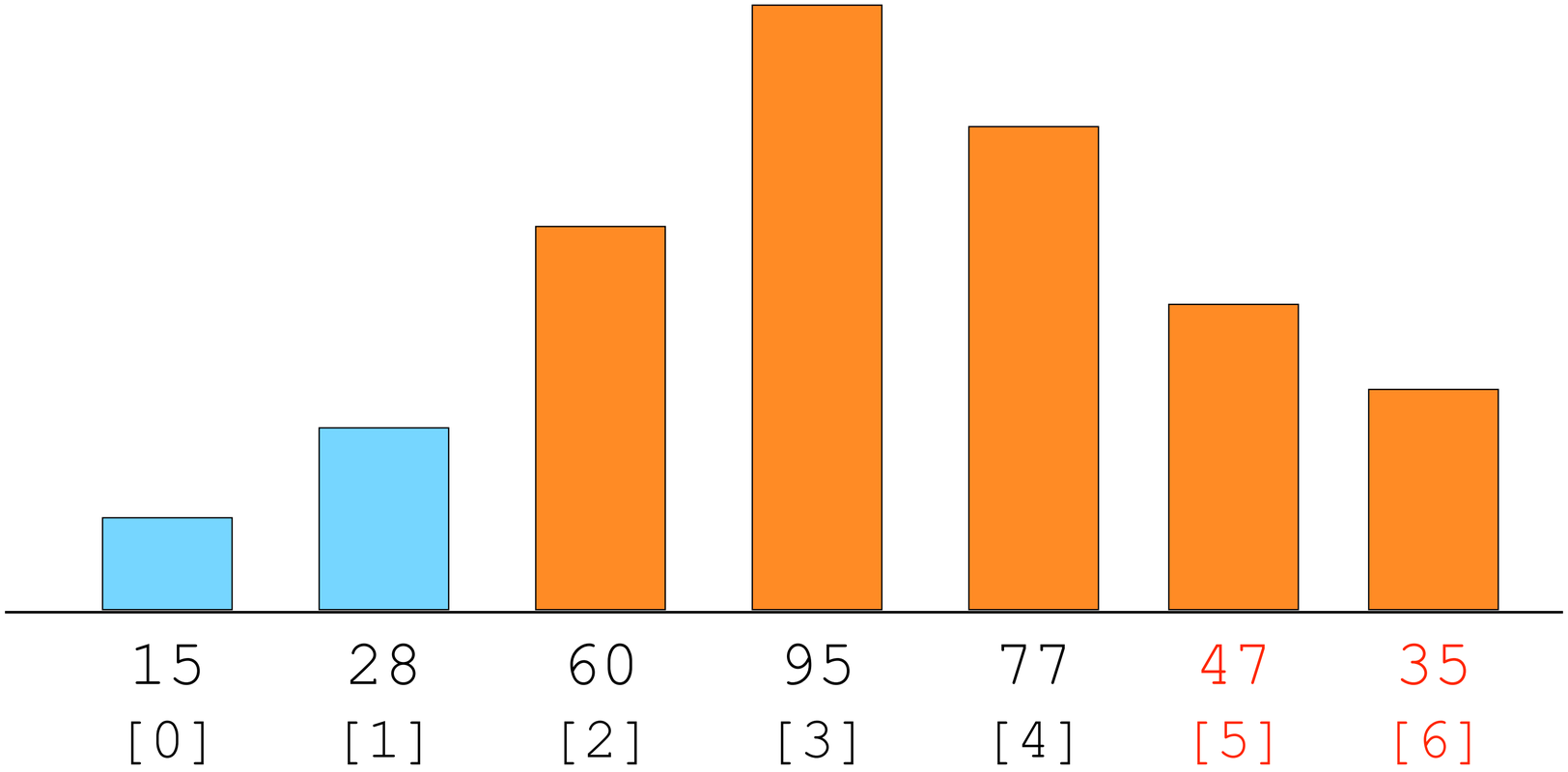
5



简单选择排序

min

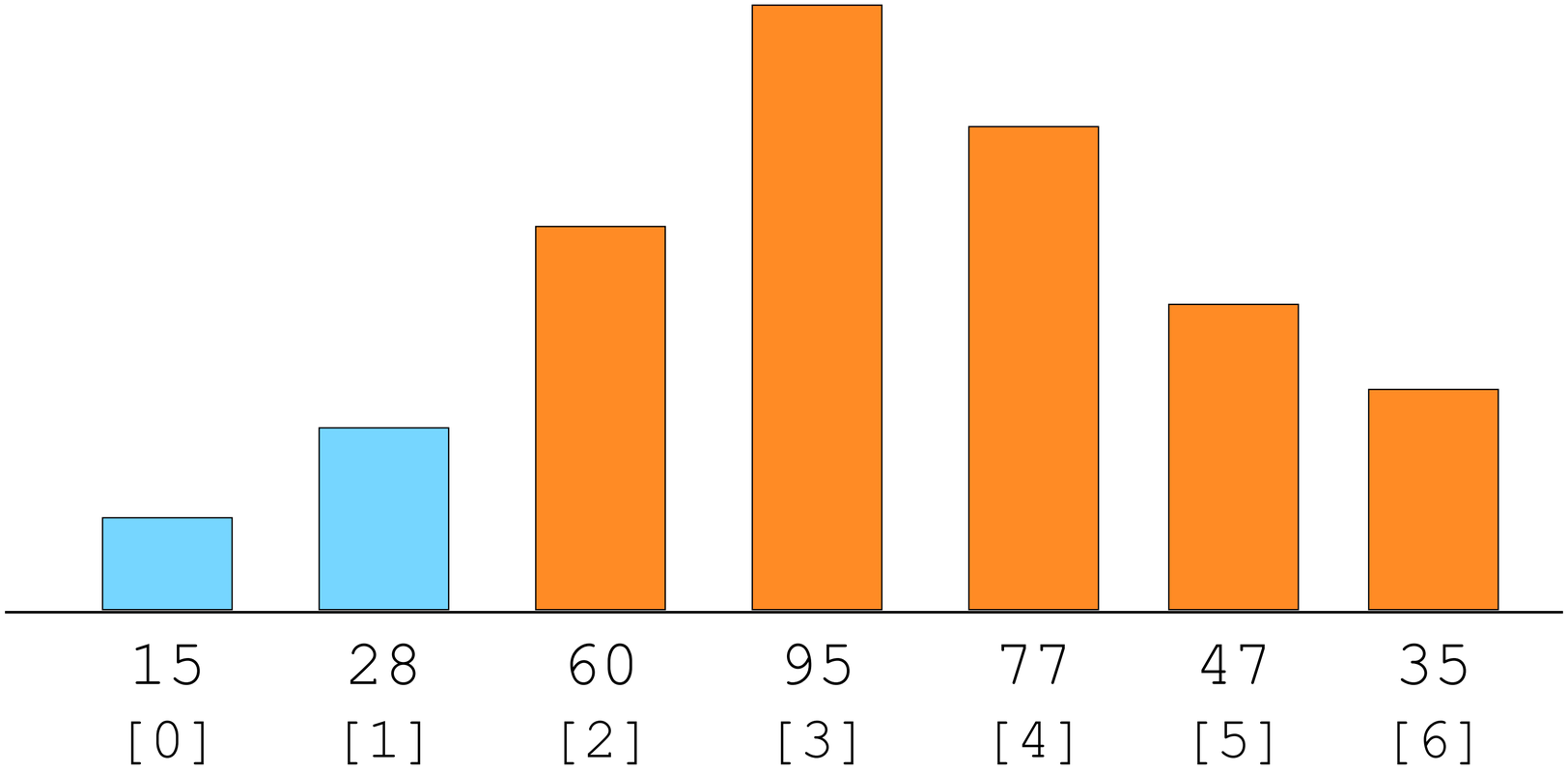
5



简单选择排序

min

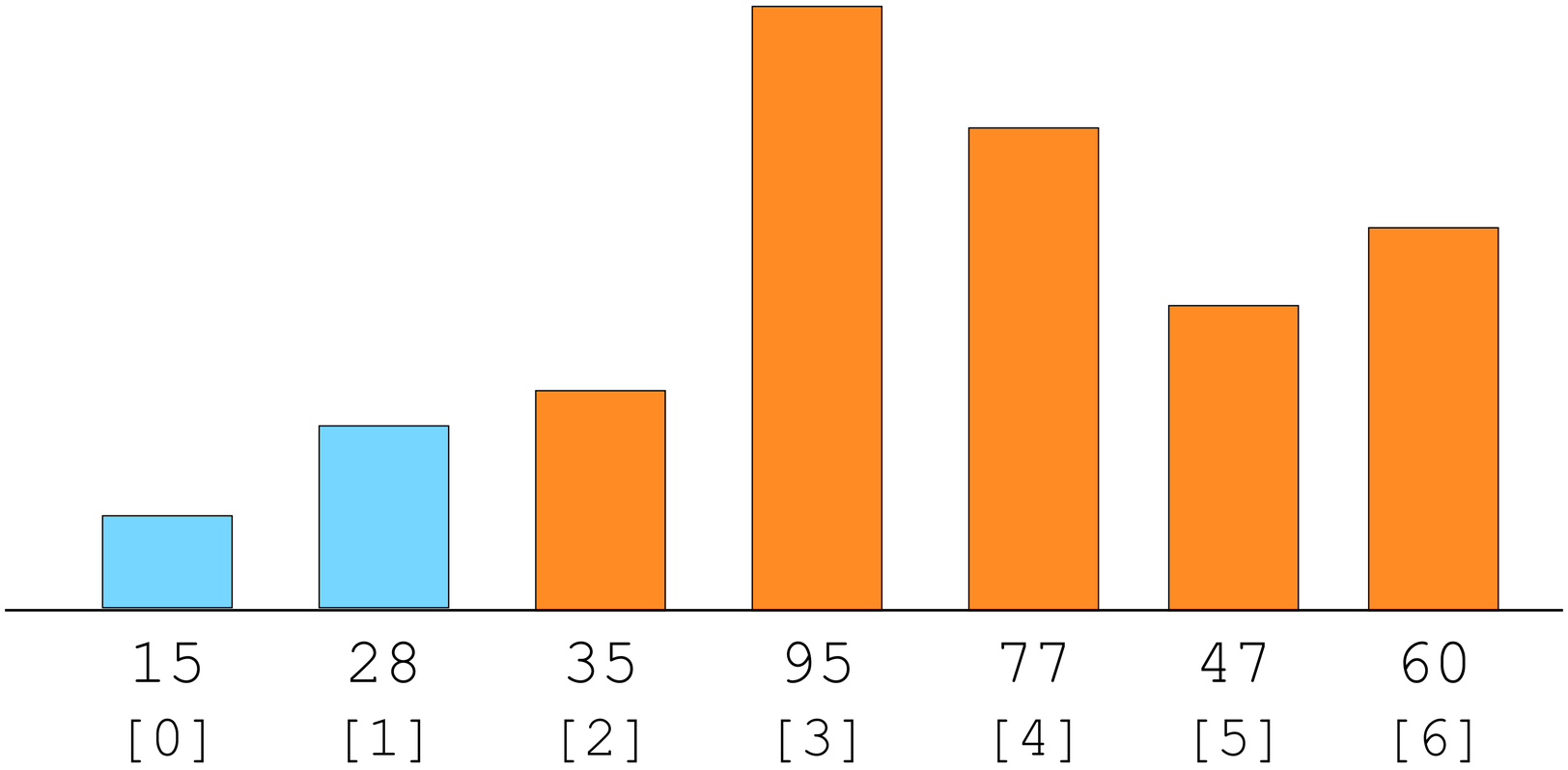
6



简单选择排序

min

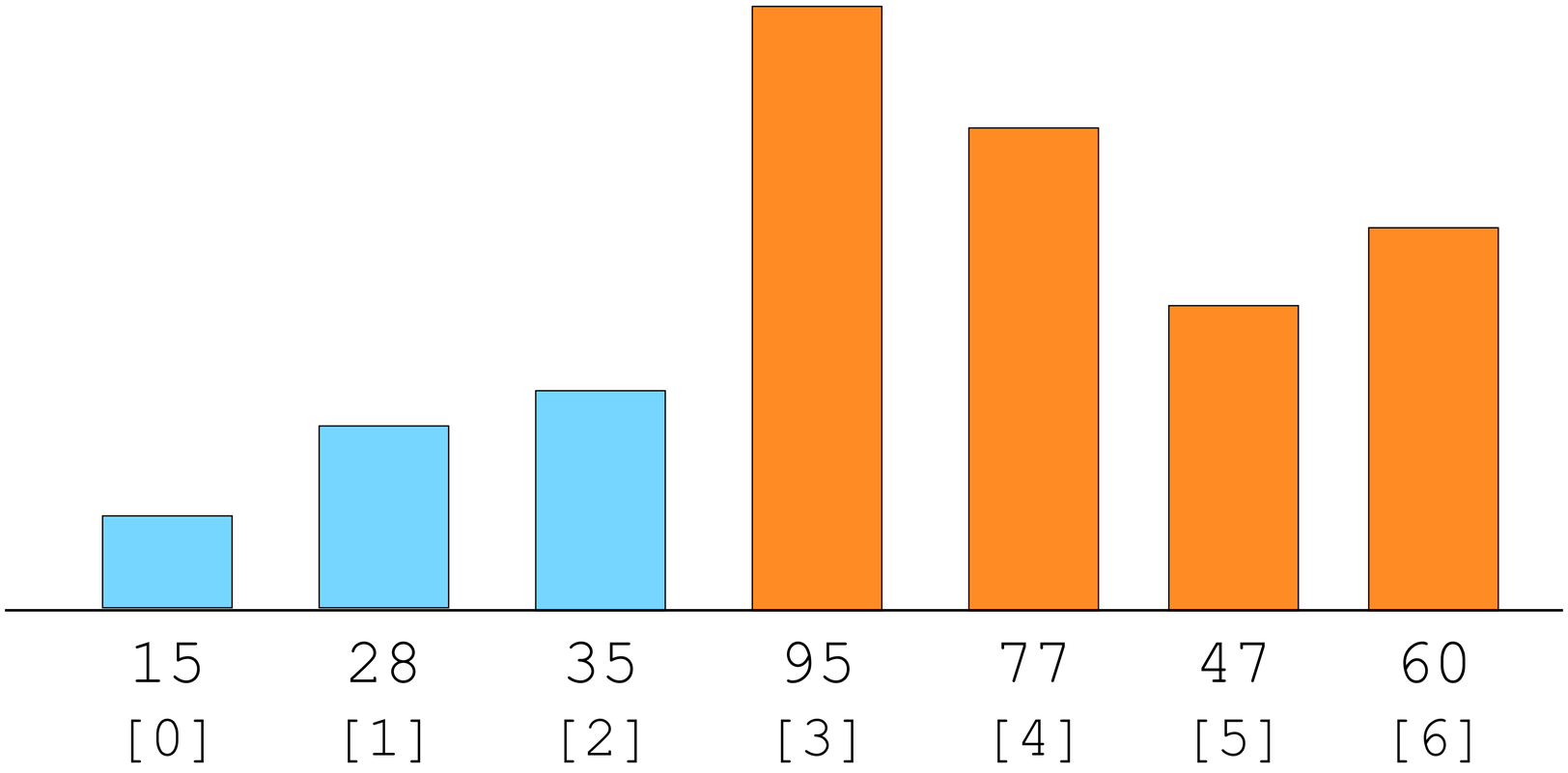
6



简单选择排序

min

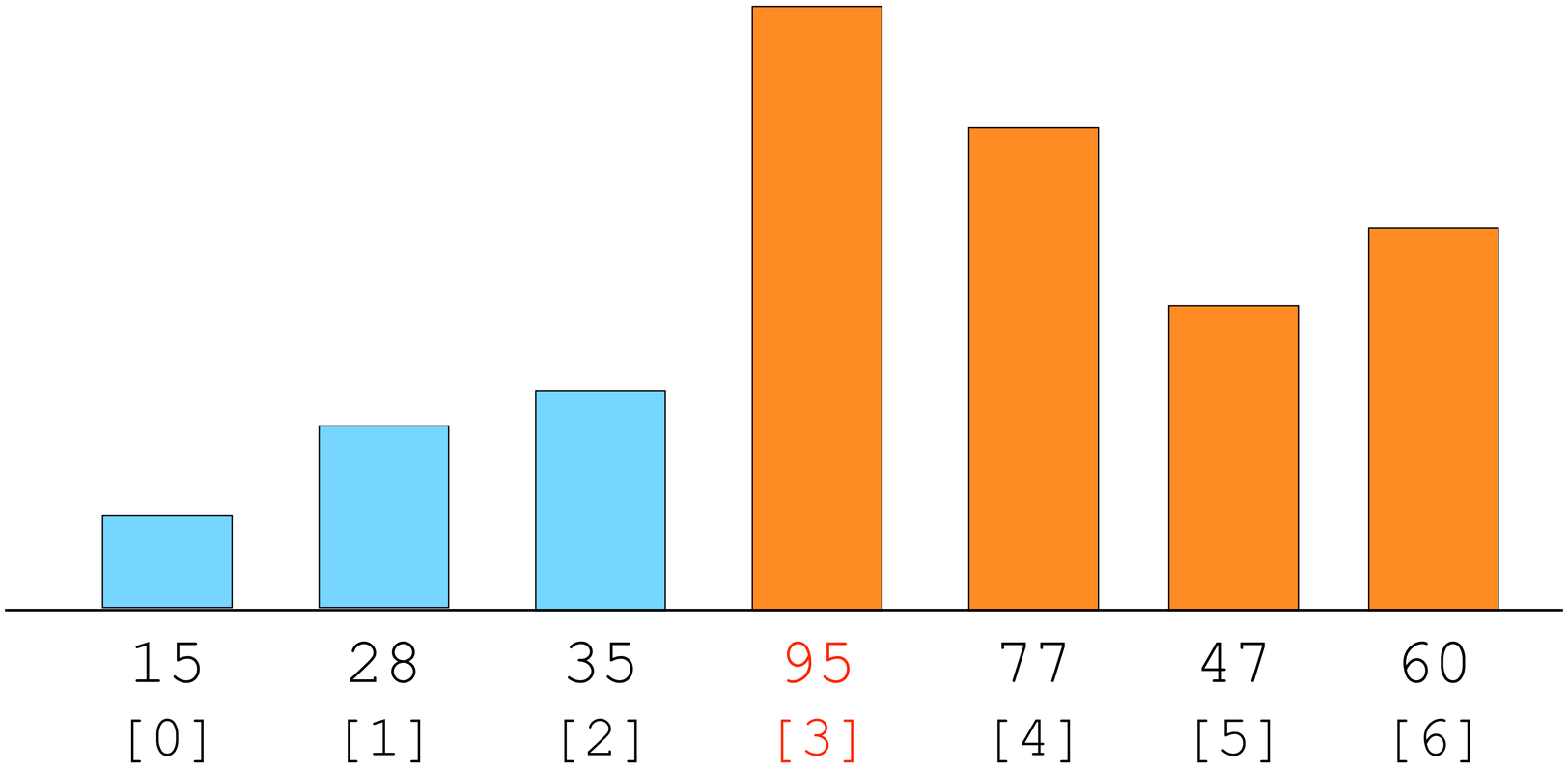
6



简单选择排序

min

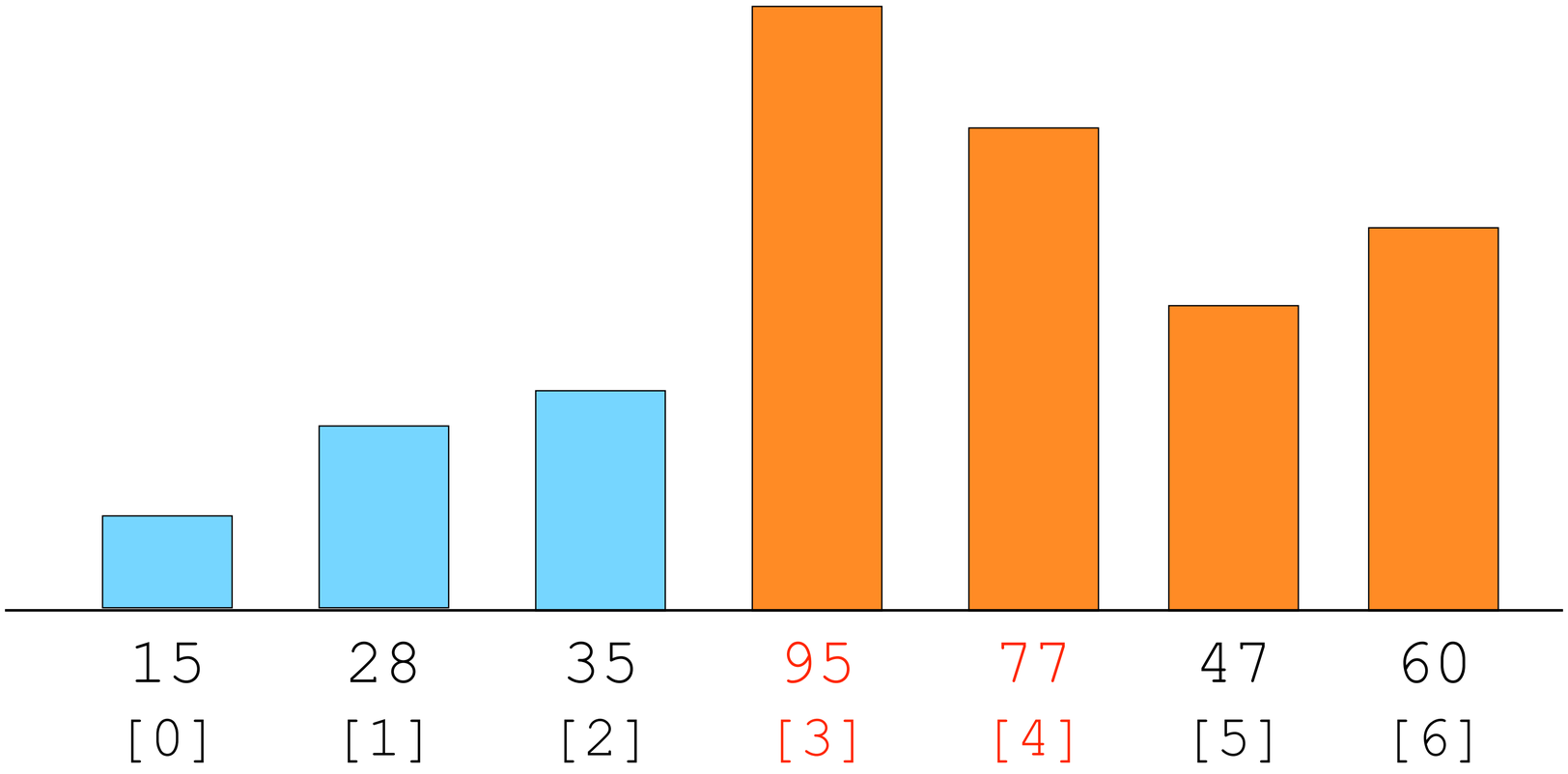
3



简单选择排序

min

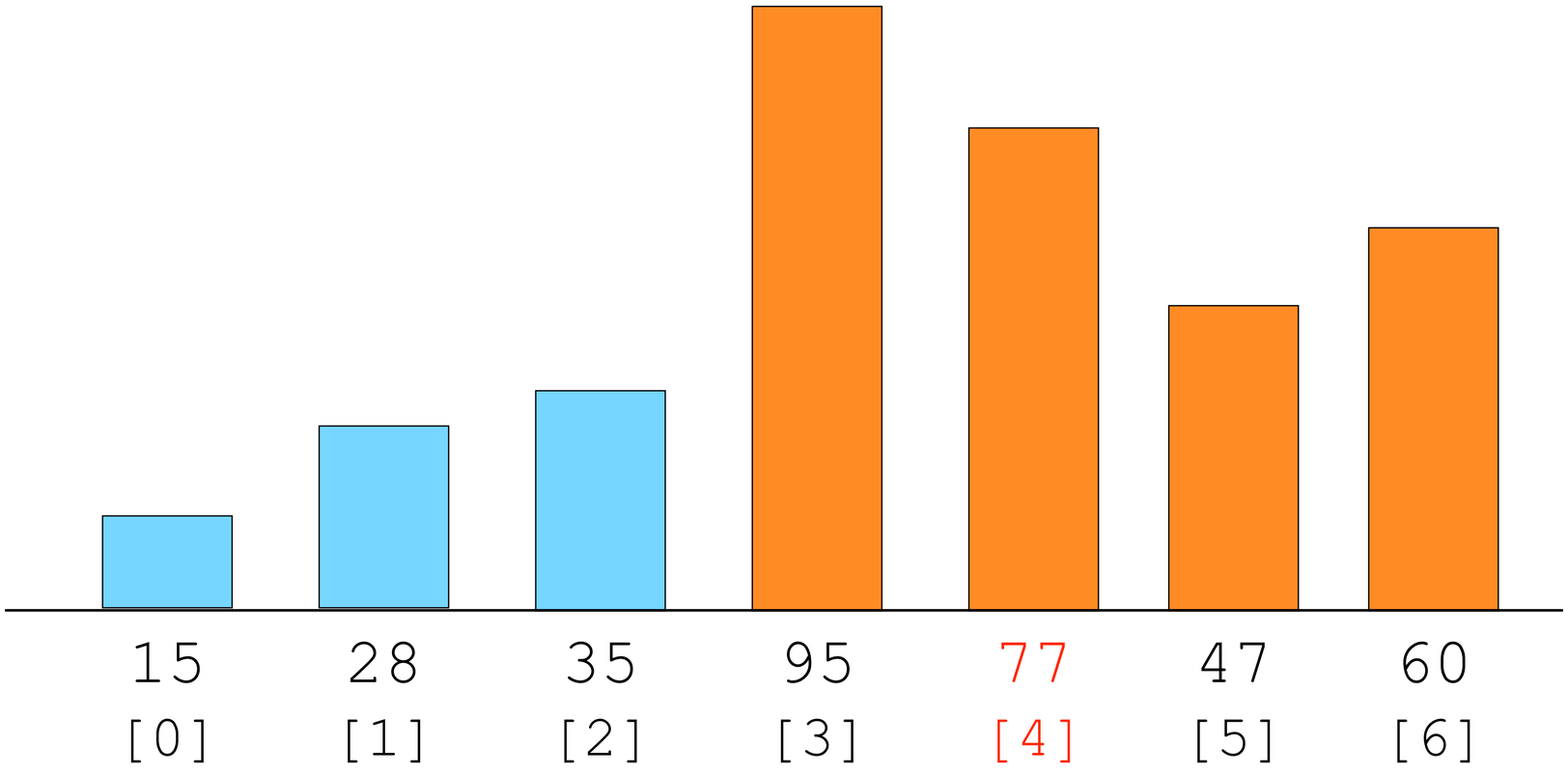
3



简单选择排序

min

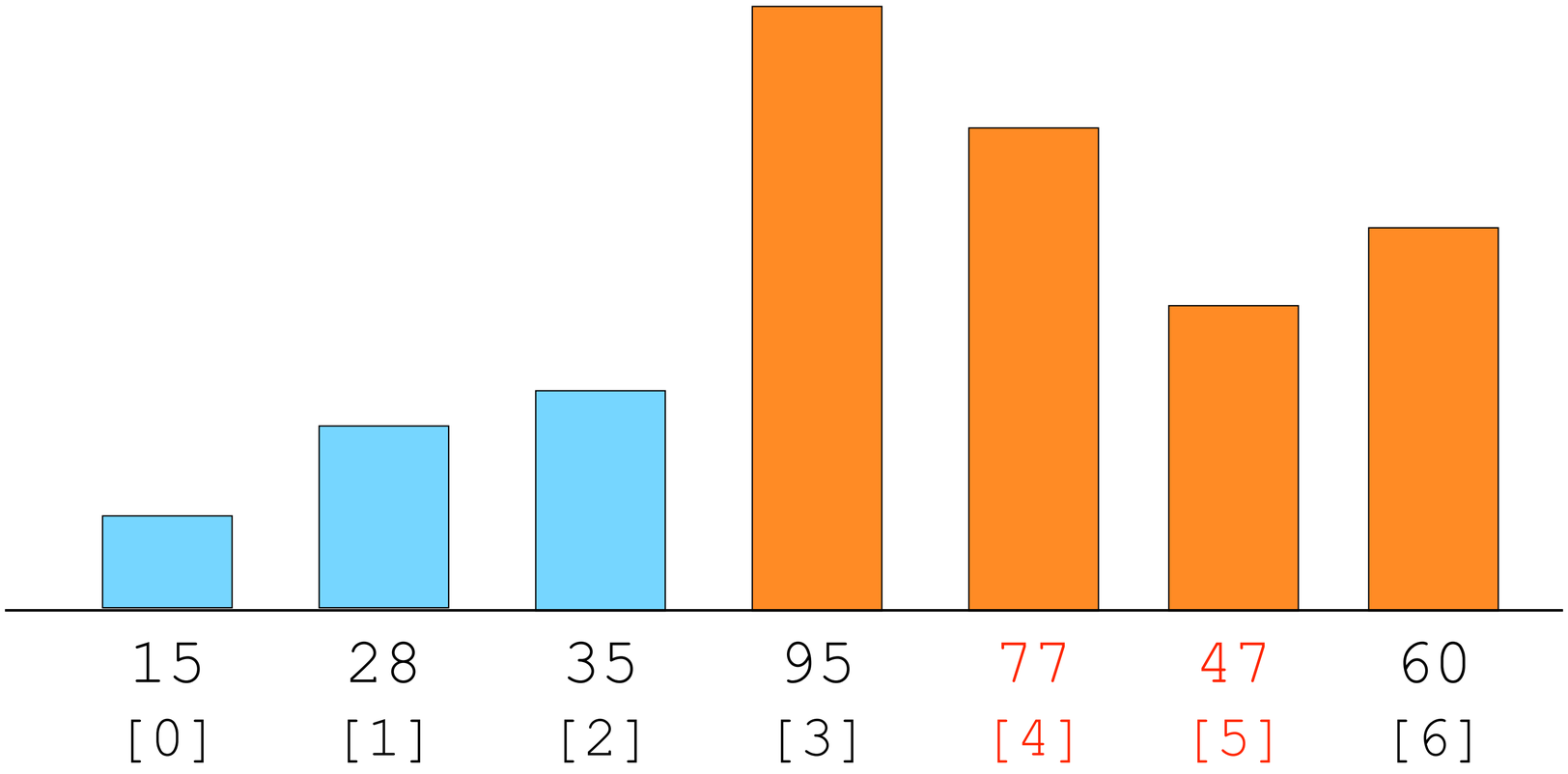
4



简单选择排序

min

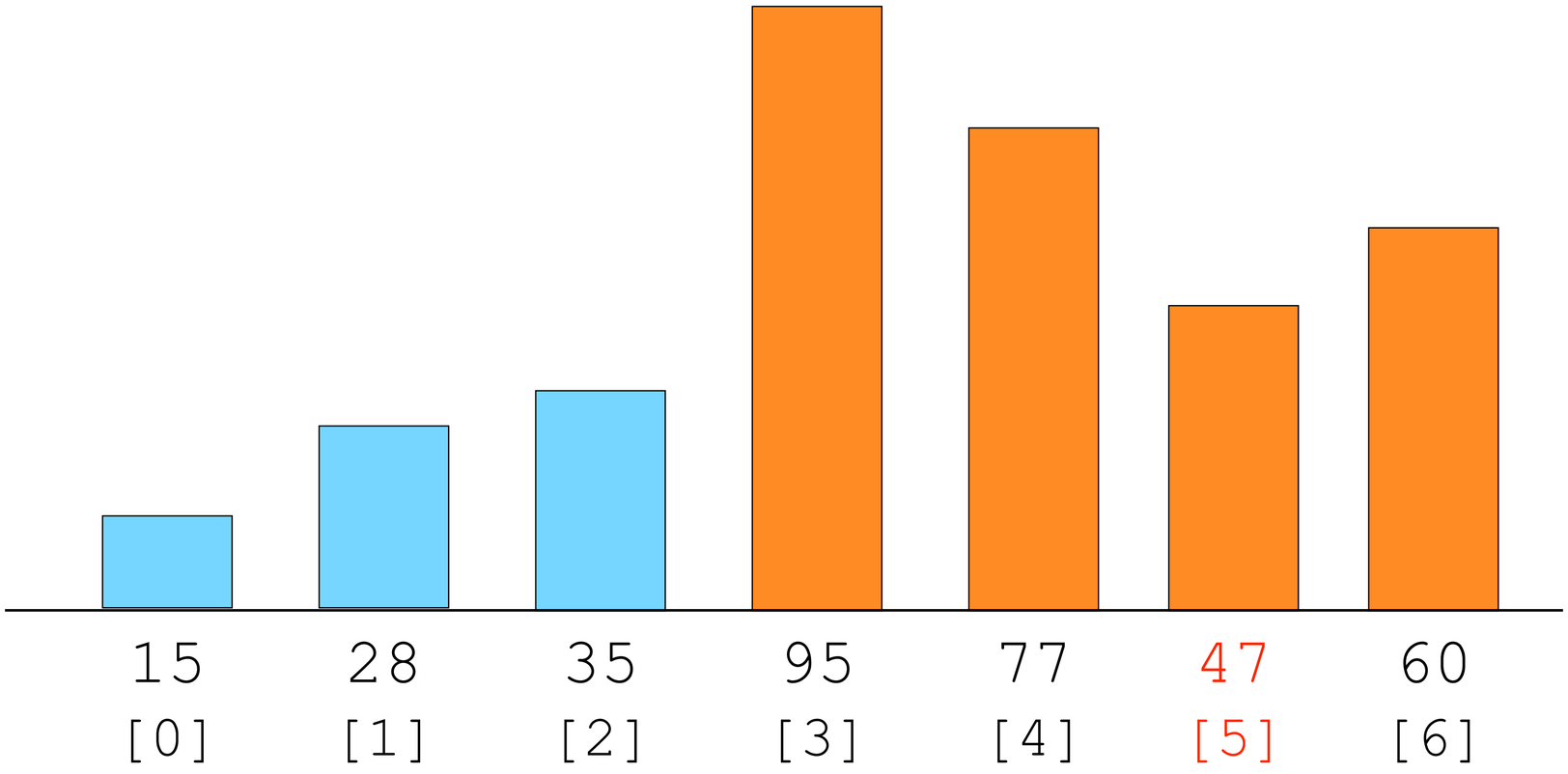
4



简单选择排序

min

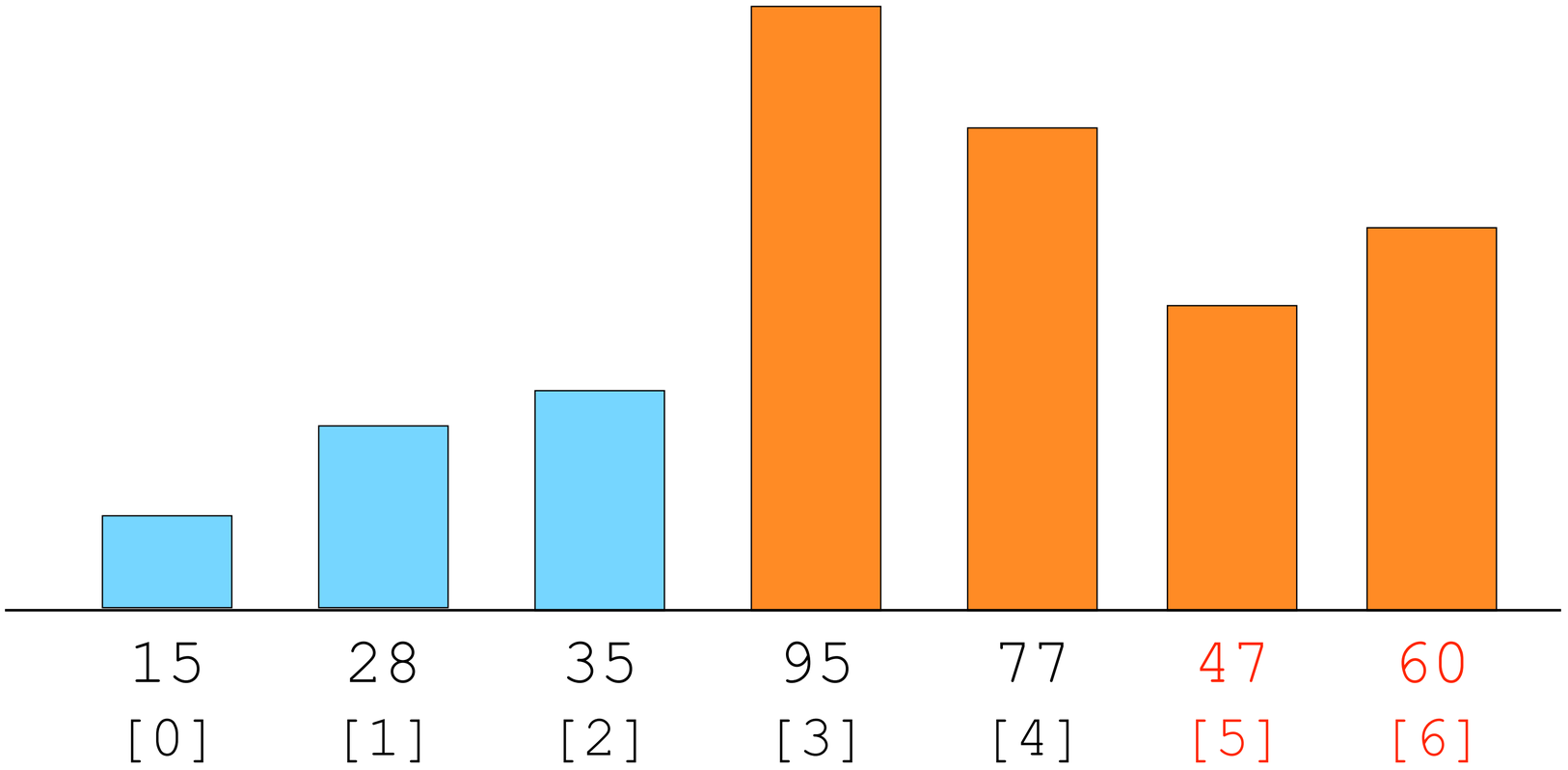
5



简单选择排序

min

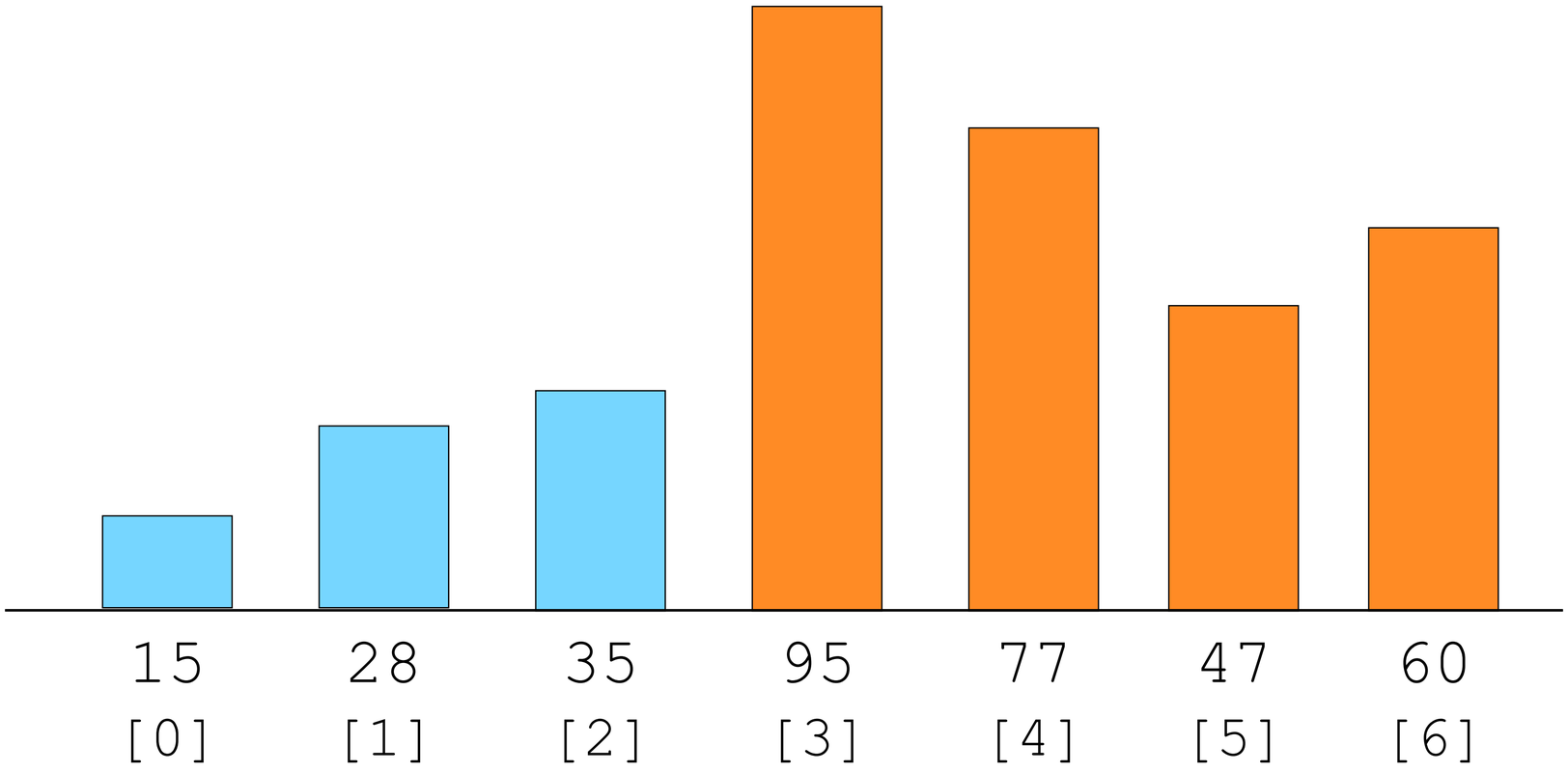
5



简单选择排序

min

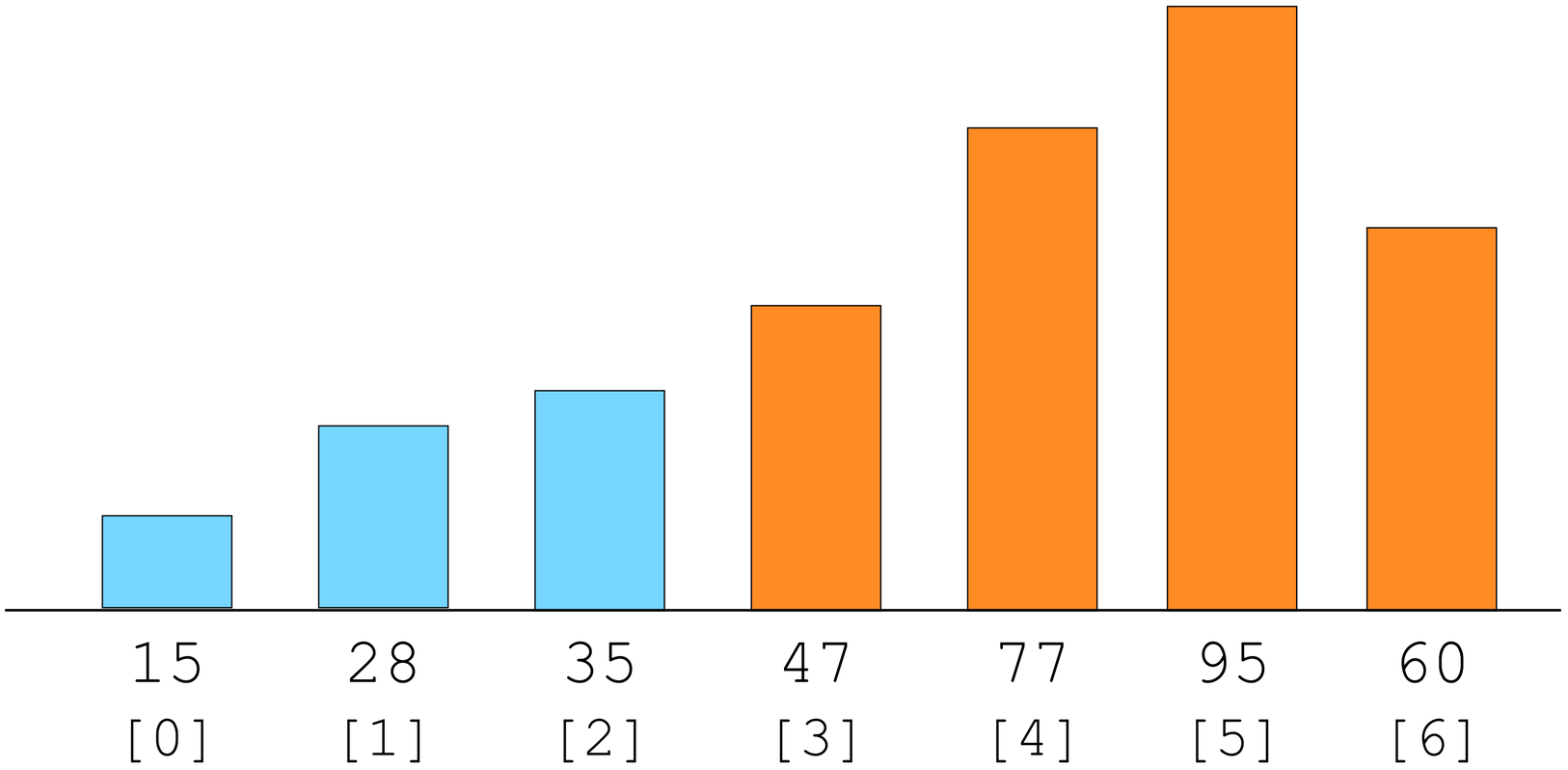
5



简单选择排序

min

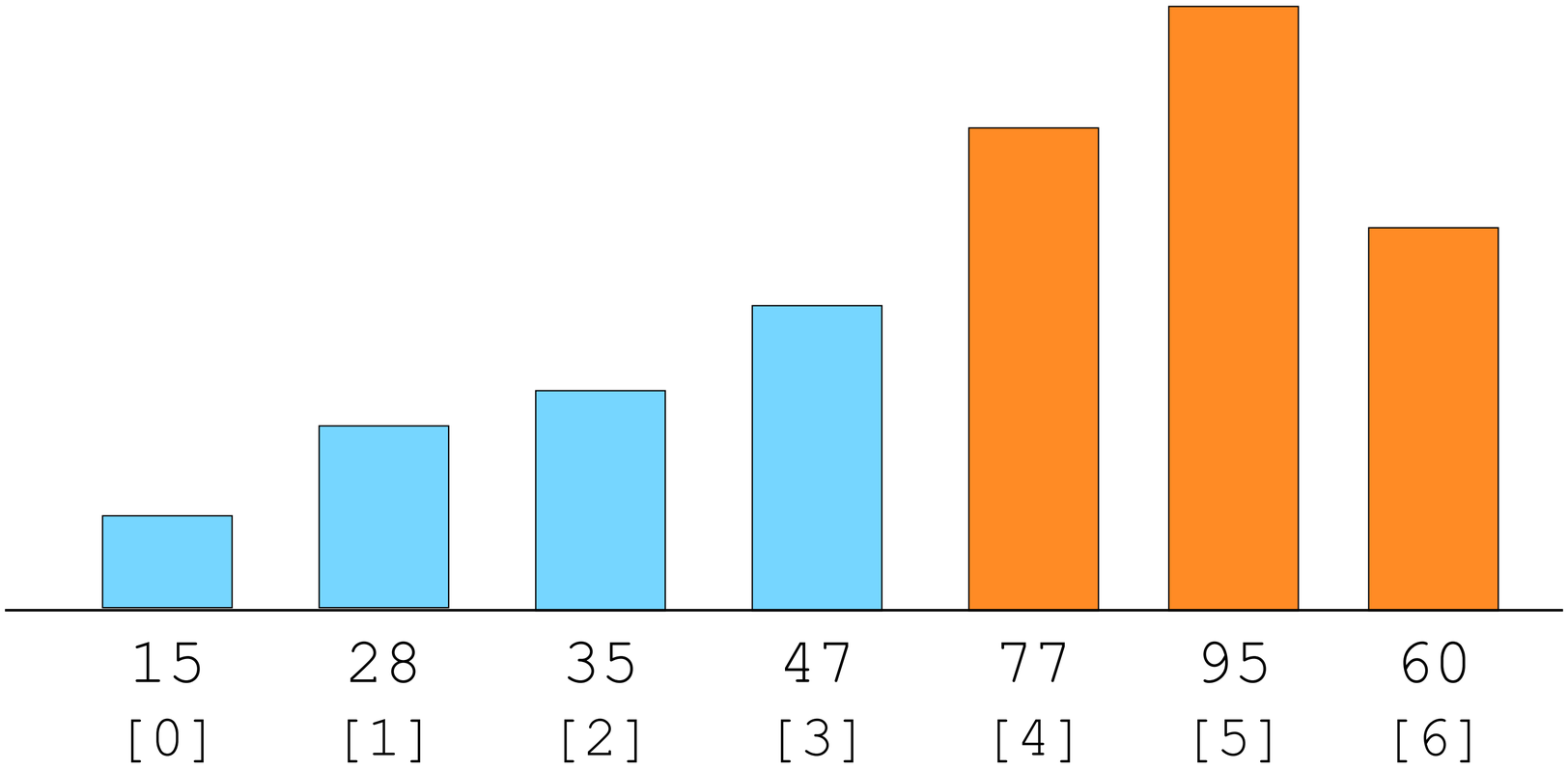
5



简单选择排序

min

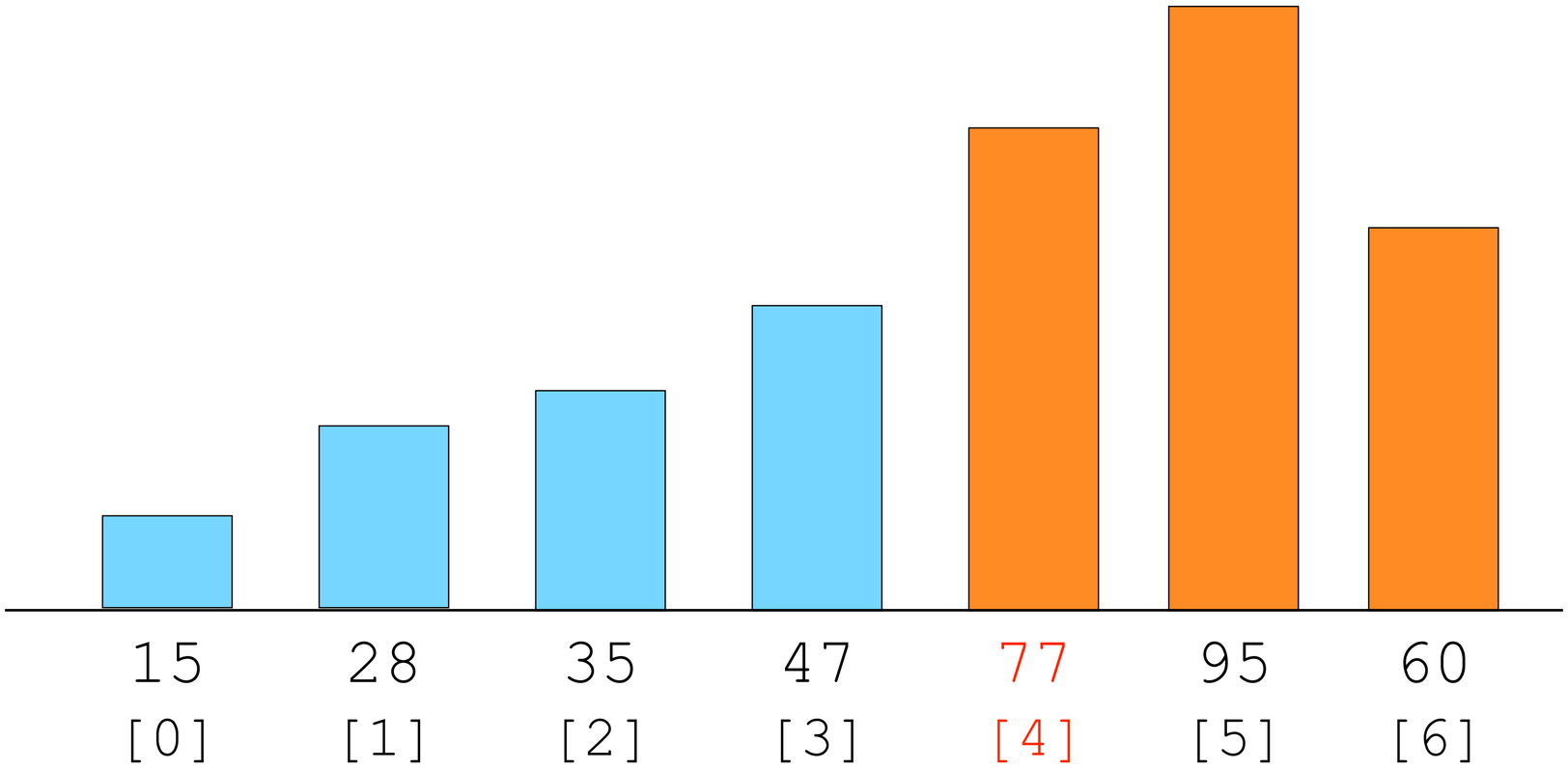
5



简单选择排序

min

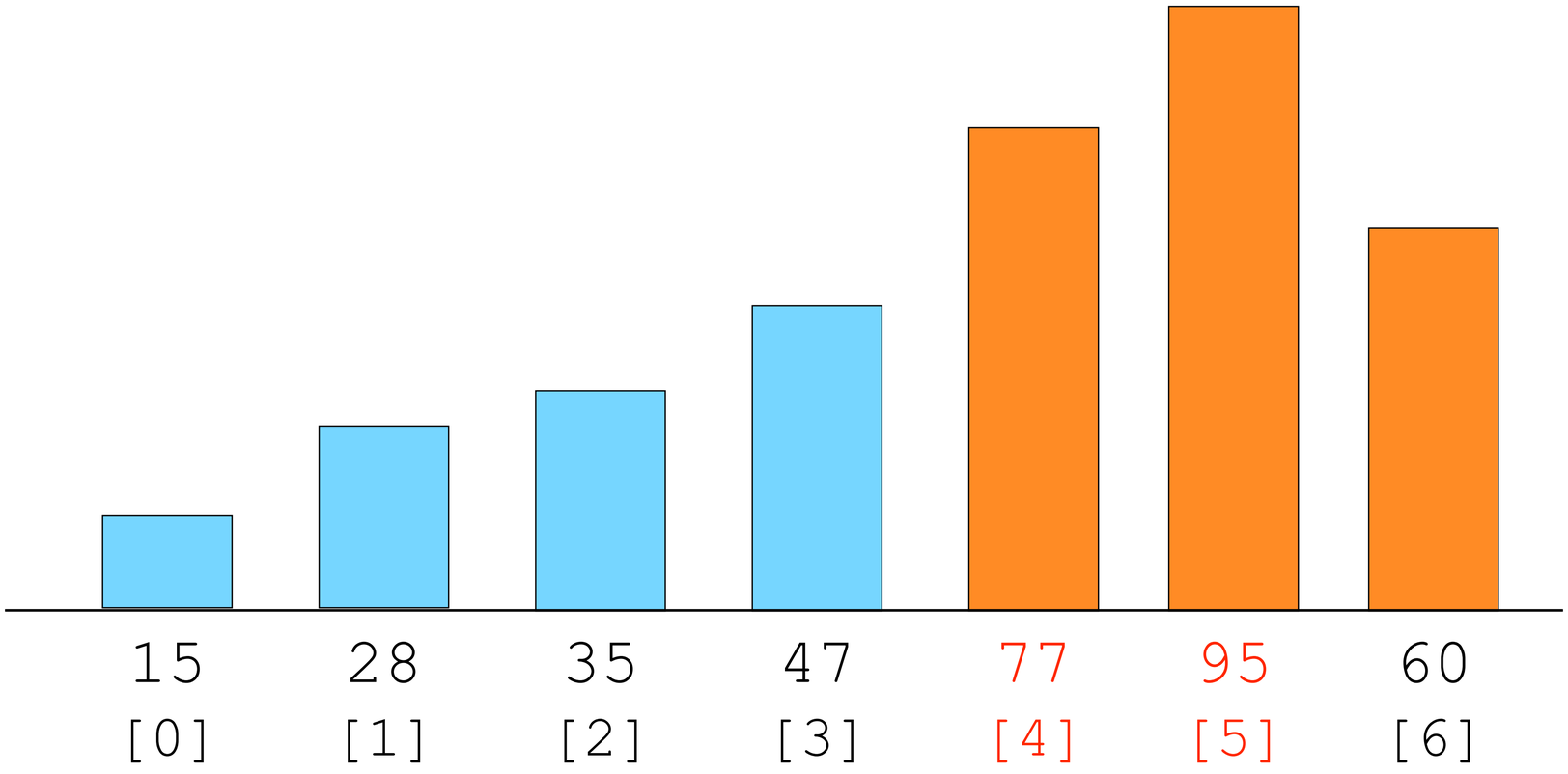
4



简单选择排序

min

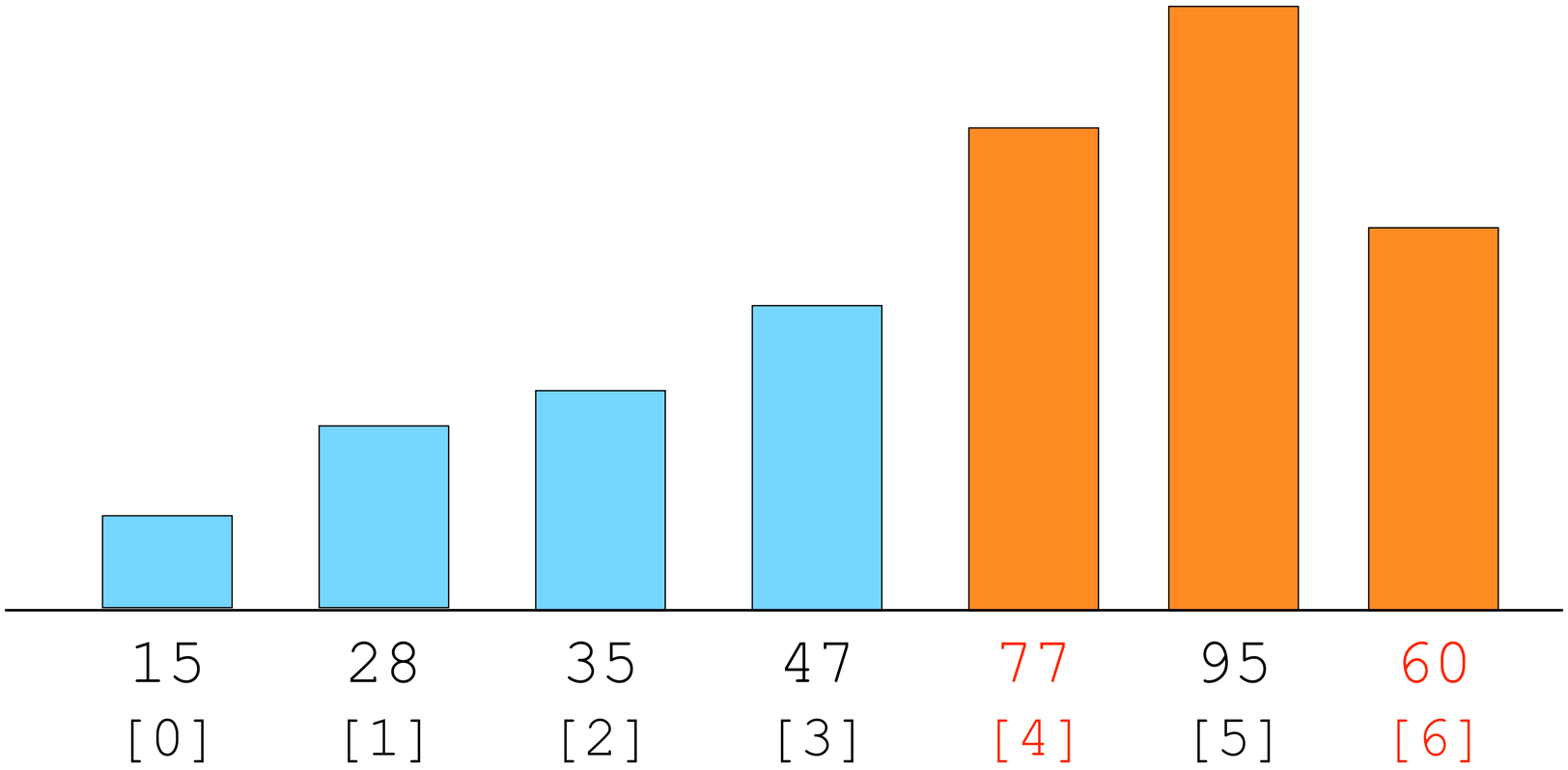
4



简单选择排序

min

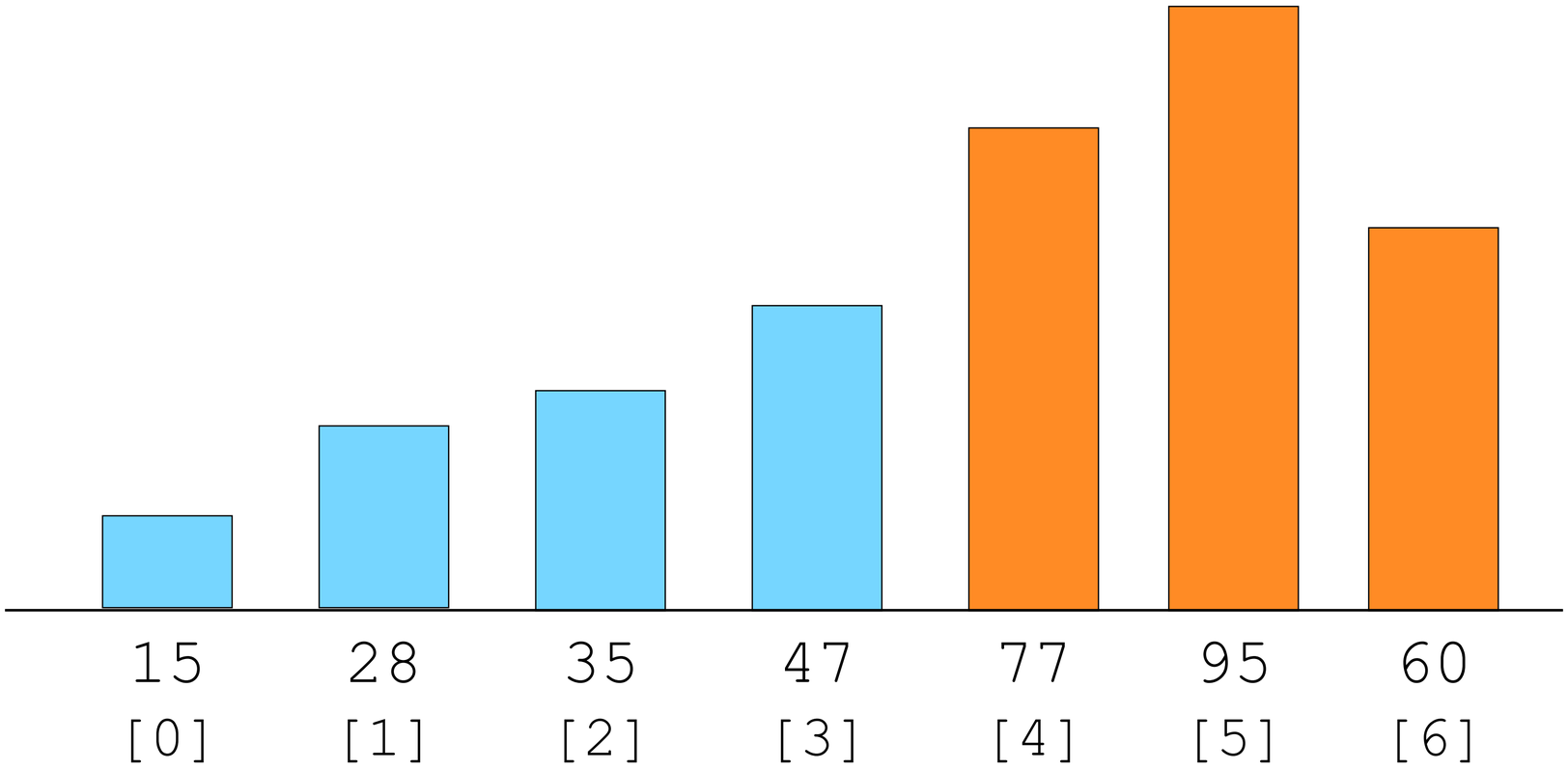
4



简单选择排序

min

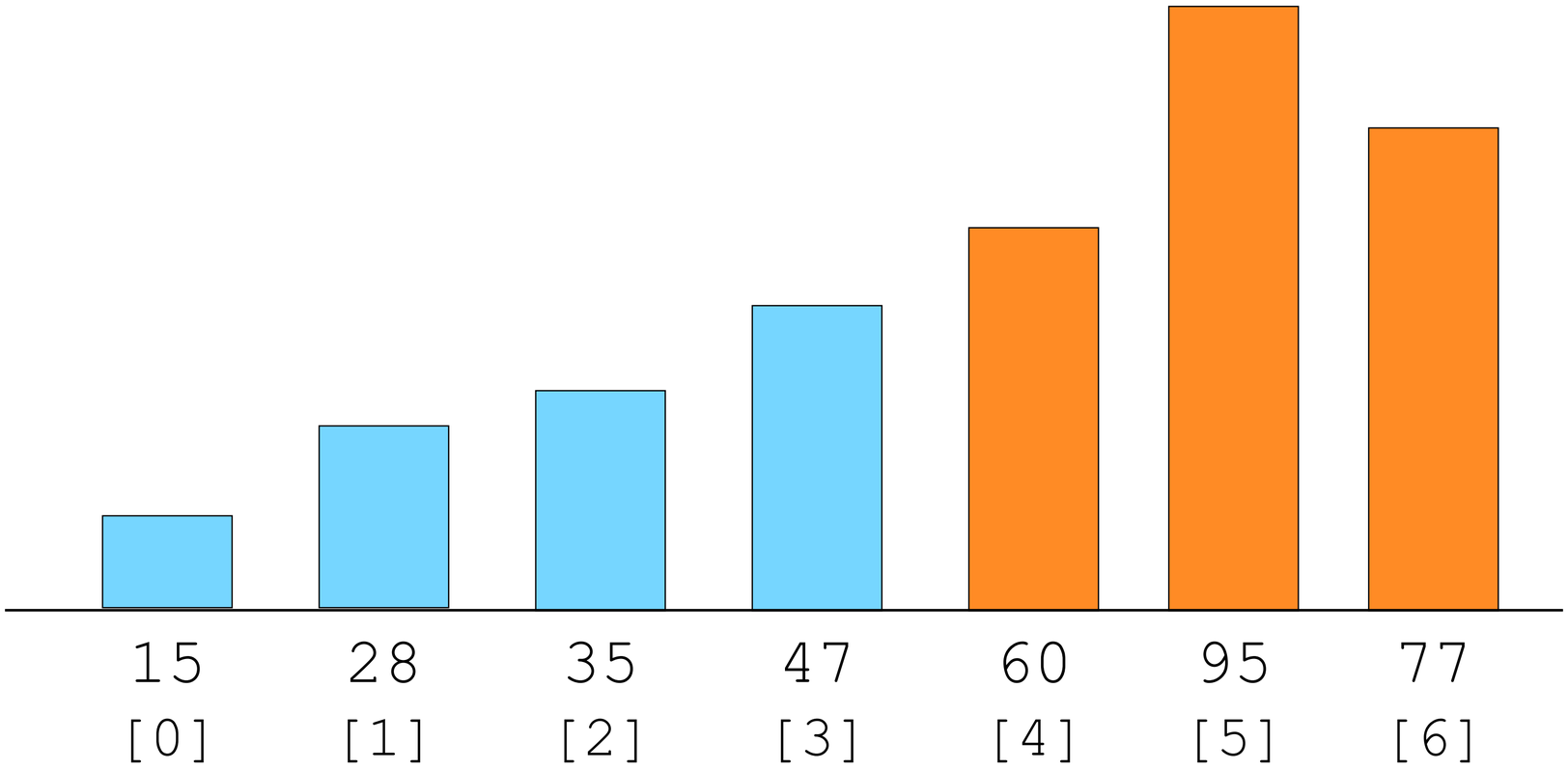
6



简单选择排序

min

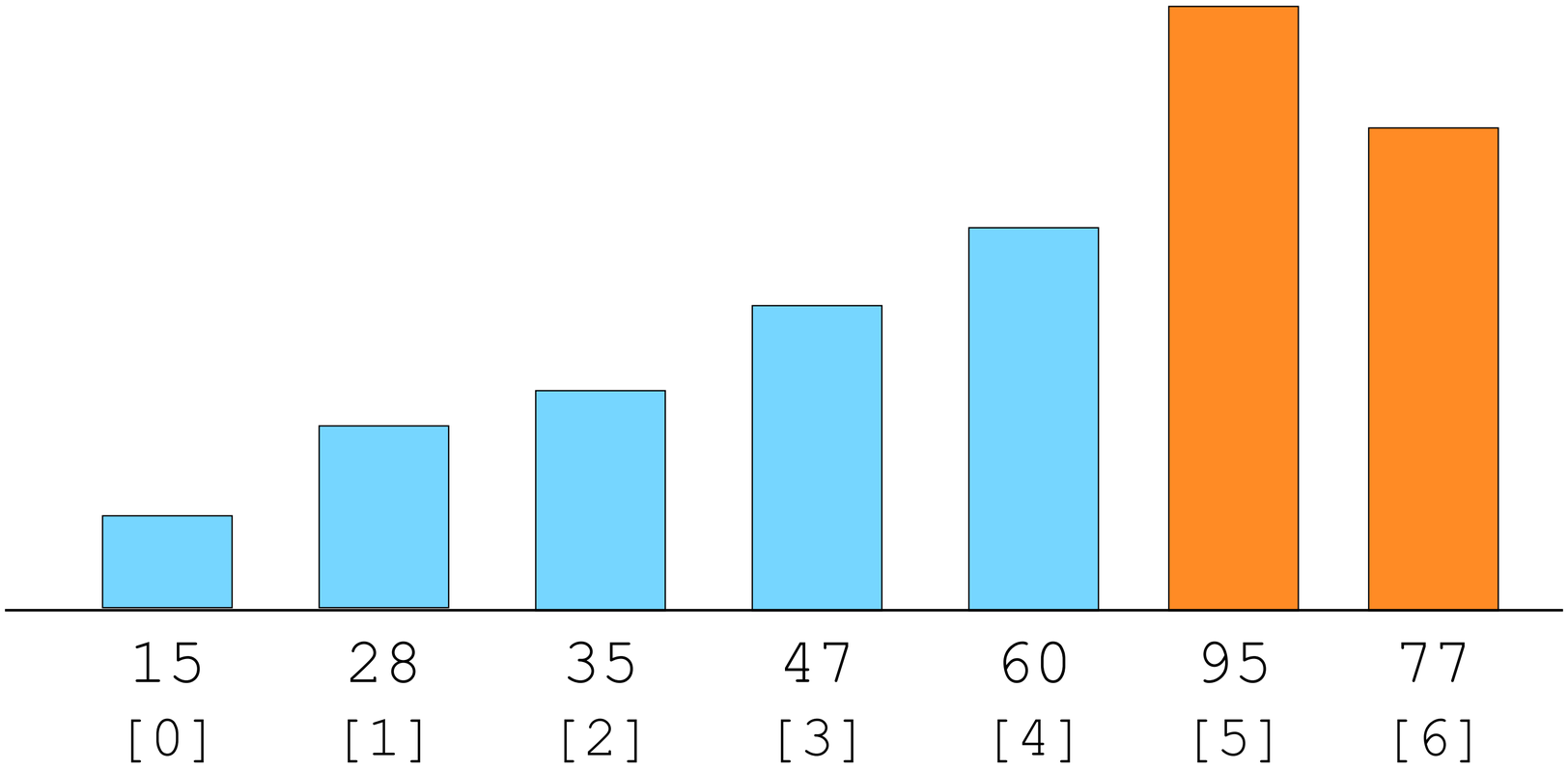
6



简单选择排序

min

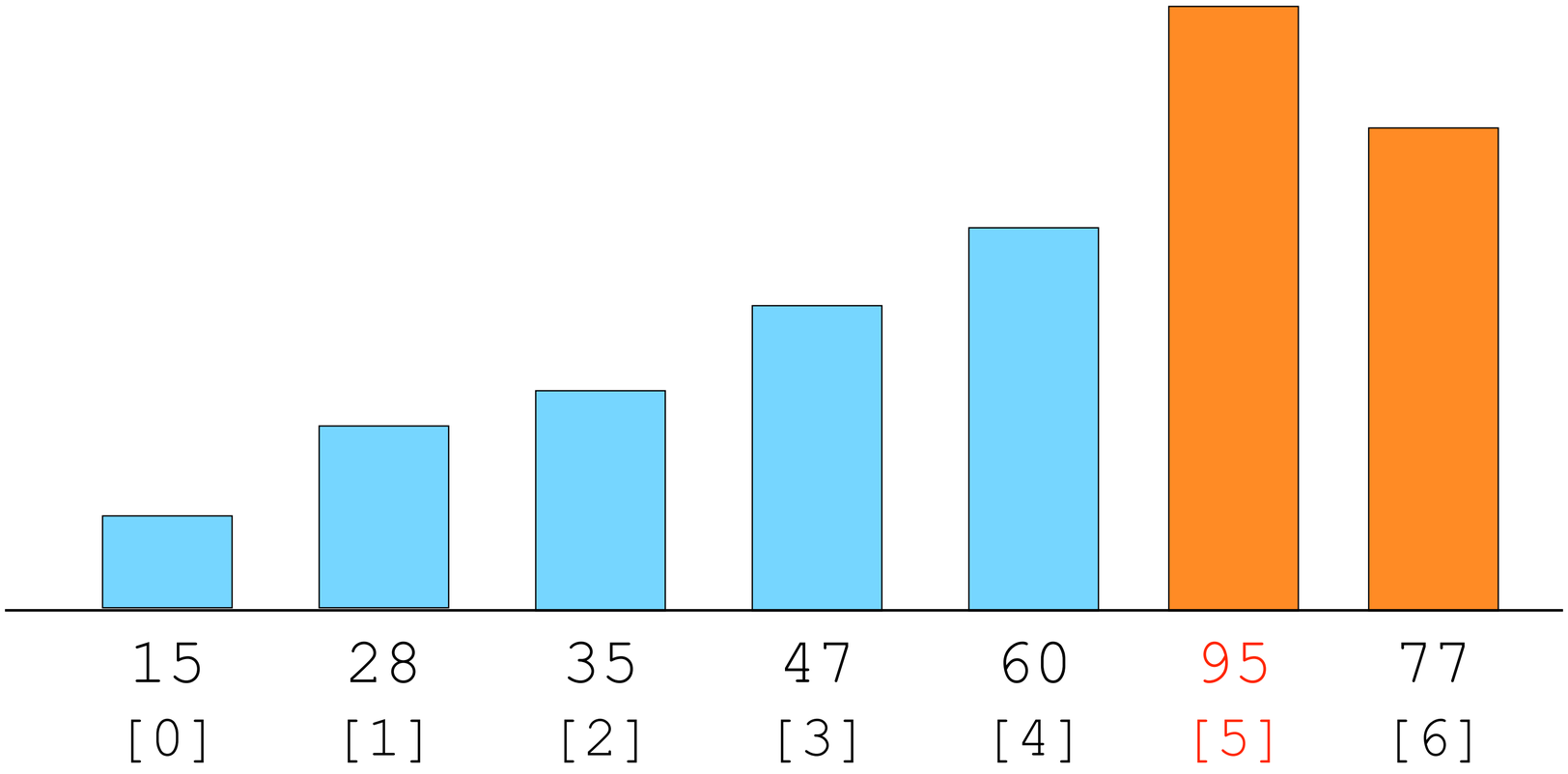
6



简单选择排序

min

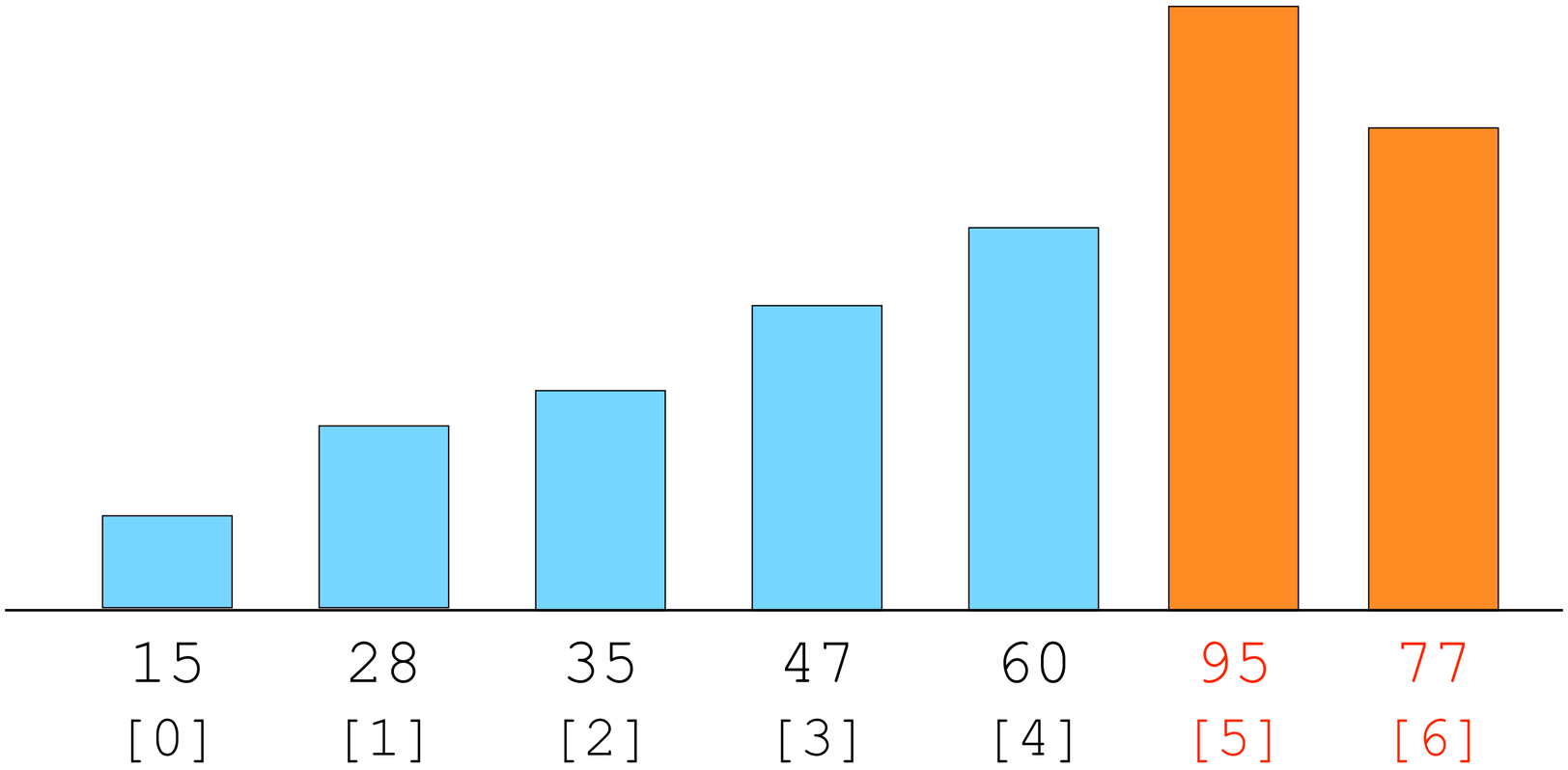
5



简单选择排序

min

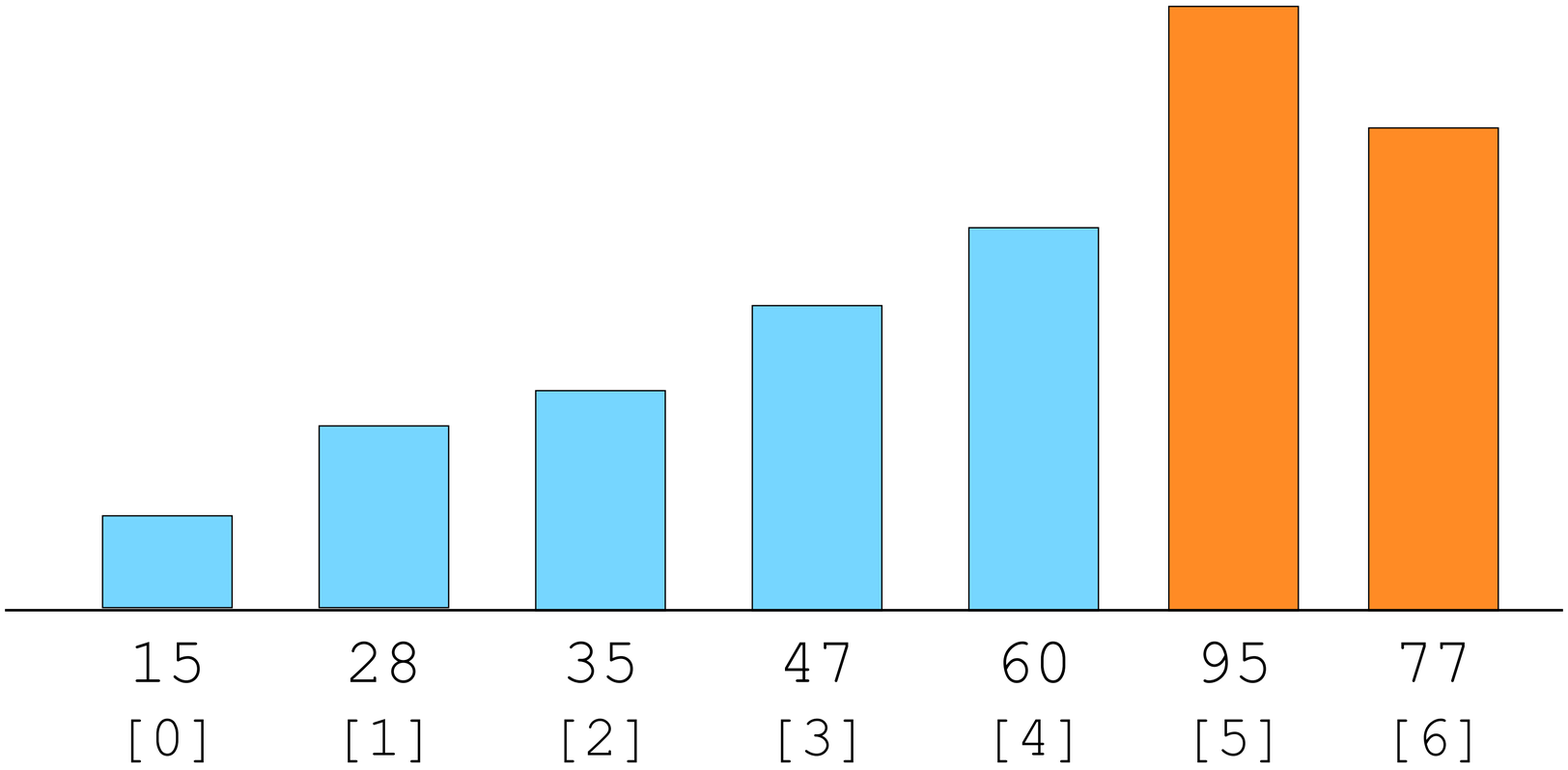
5



简单选择排序

min

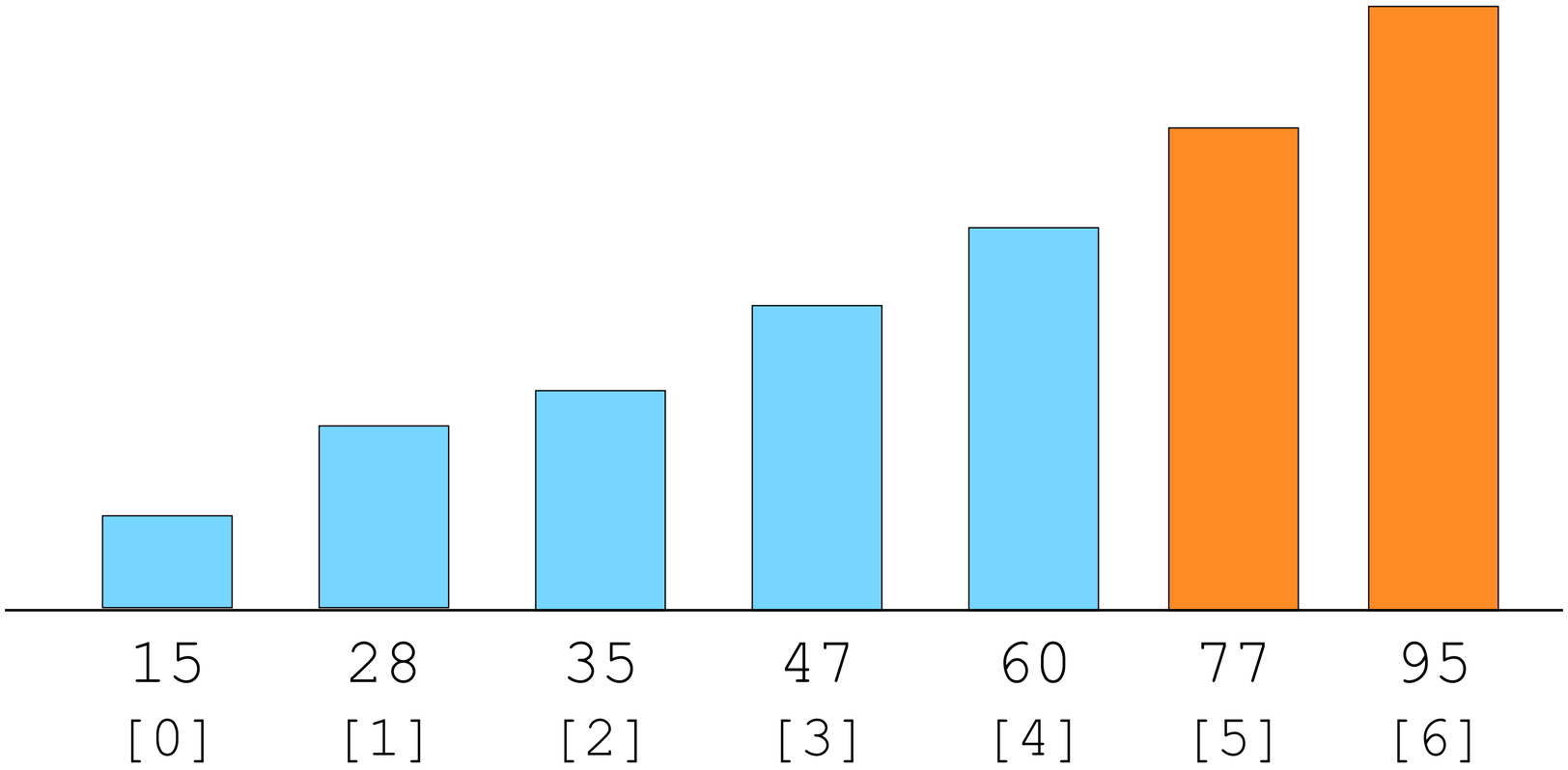
6



简单选择排序

min

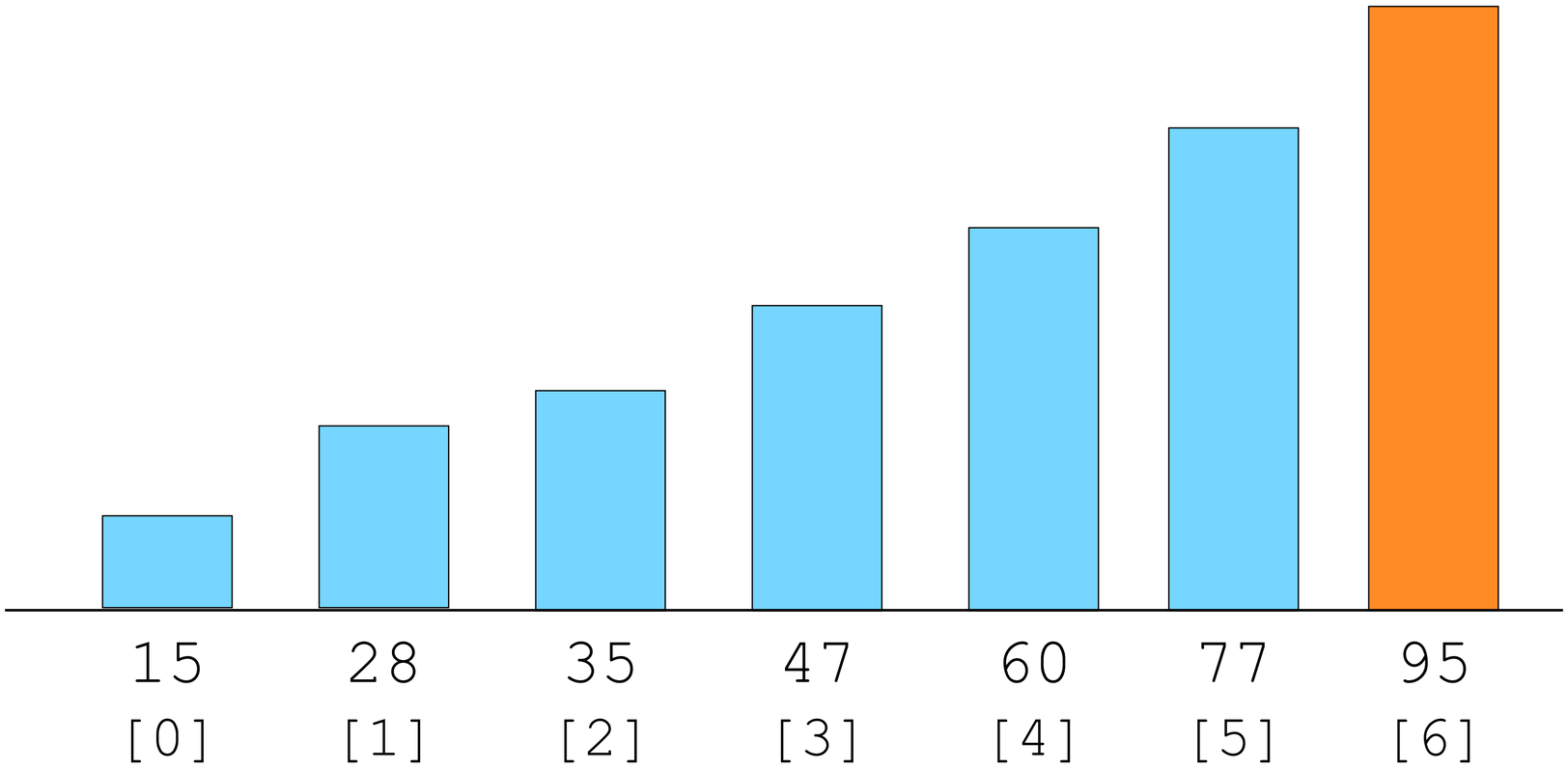
6



简单选择排序

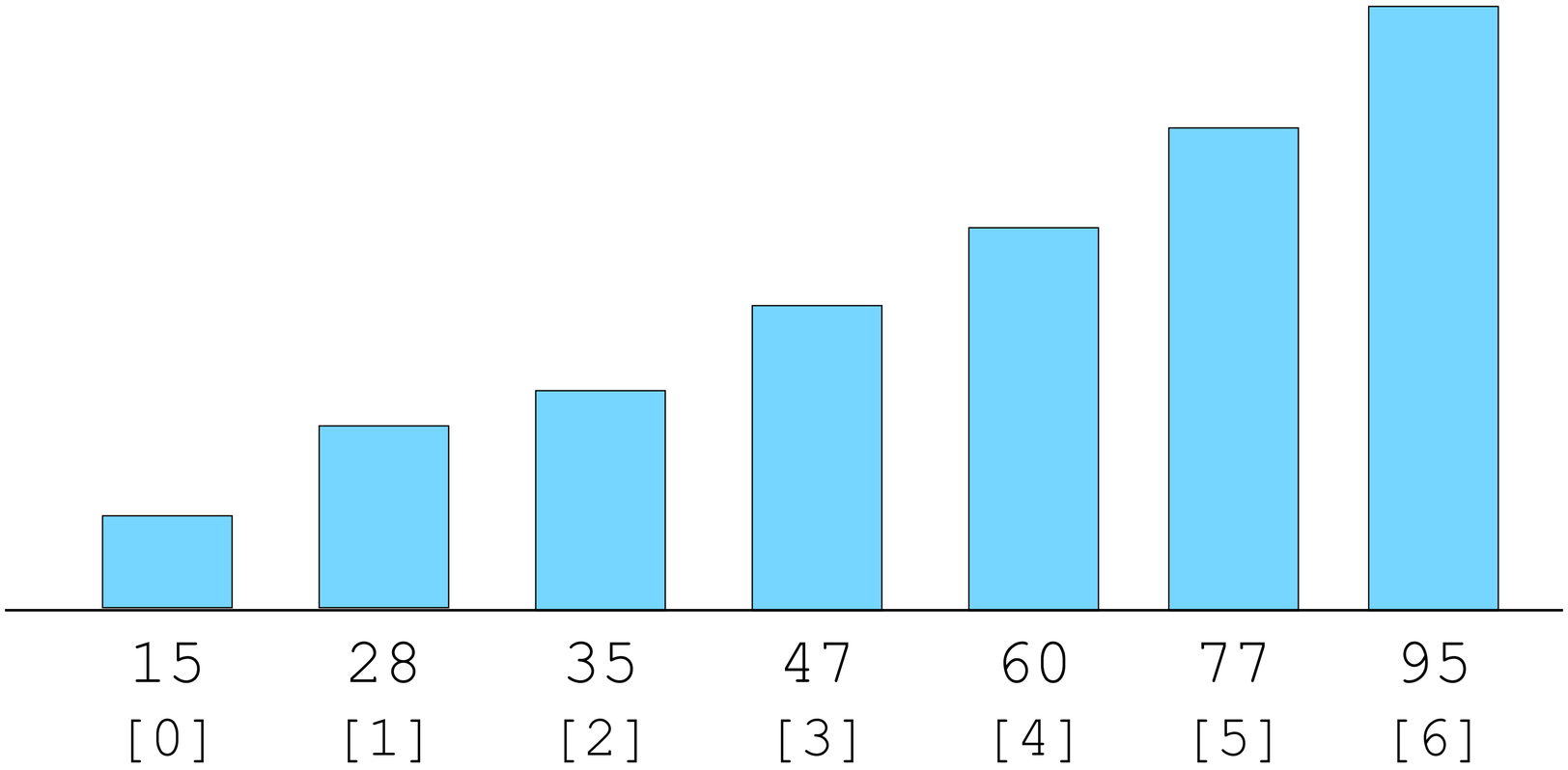
min

6



简单选择排序

排序 / test4.c



简单选择排序

维度	描述
时间复杂度	$O(n^2)$
空间复杂度	O(1)
算法稳定性	不稳定
适用场景	在小规模数据排序时，选择排序简单直观，便于快速实现。

堆与堆排序

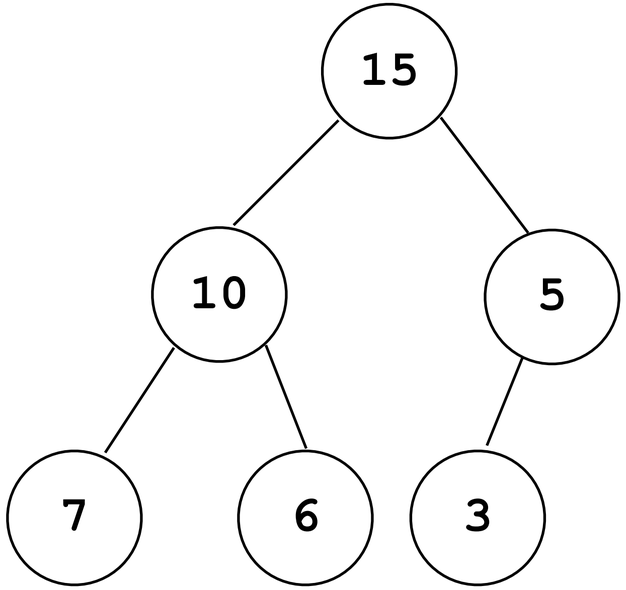
堆排序（Heap Sort）是一种基于完全二叉树结构的选择排序，它利用堆这种特殊的数据结构来选出当前无序部分的最大（或最小）元素，进而完成排序操作。

堆

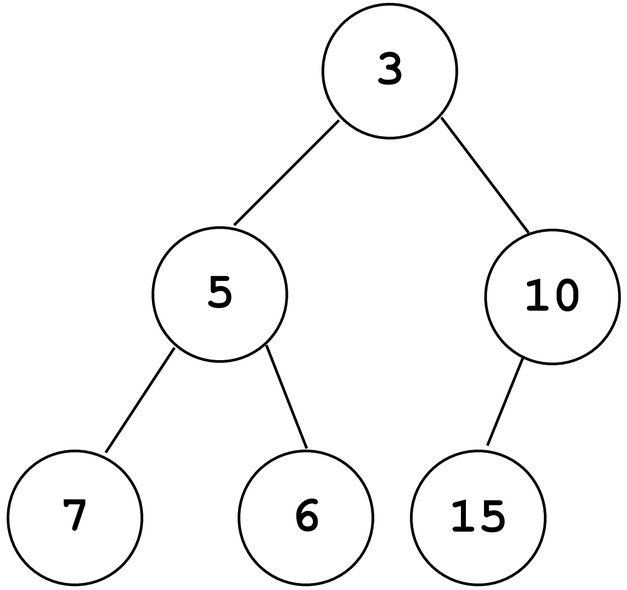
这里的“堆”不是平时程序中提到的“内存堆区”，而是一种满足特定性质的树状结构。在堆排序中，我们通常使用的是**大根堆 / 最大堆**（每个节点都大于等于其子节点）或**小根堆 / 最小堆**（每个节点都小于等于其子节点）。

堆

这里的“堆”不是平时程序中提到的“内存堆区”，而是一种满足特定性质的树状结构。在堆排序中，我们通常使用的是**大根堆 / 最大堆**（每个节点都大于等于其子节点）或**小根堆 / 最小堆**（每个节点都小于等于其子节点）。

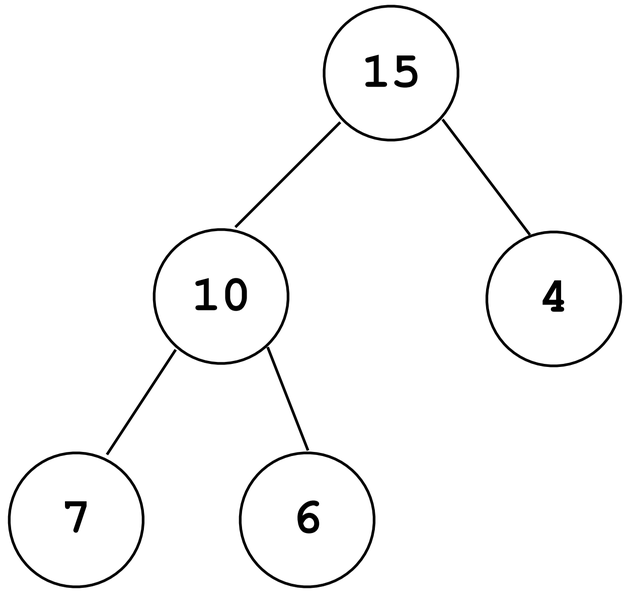


大根堆 / 最大堆



小根堆 / 最小堆

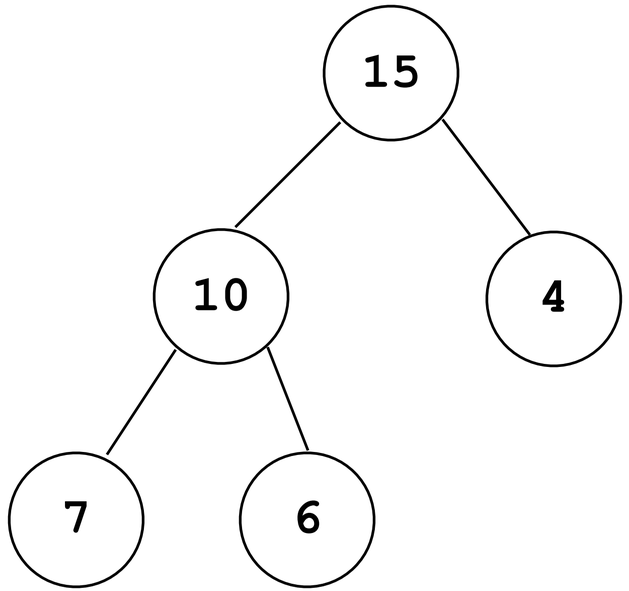
堆的插入操作



堆的插入操作

1.保证形态为完全二叉树的基础上插入

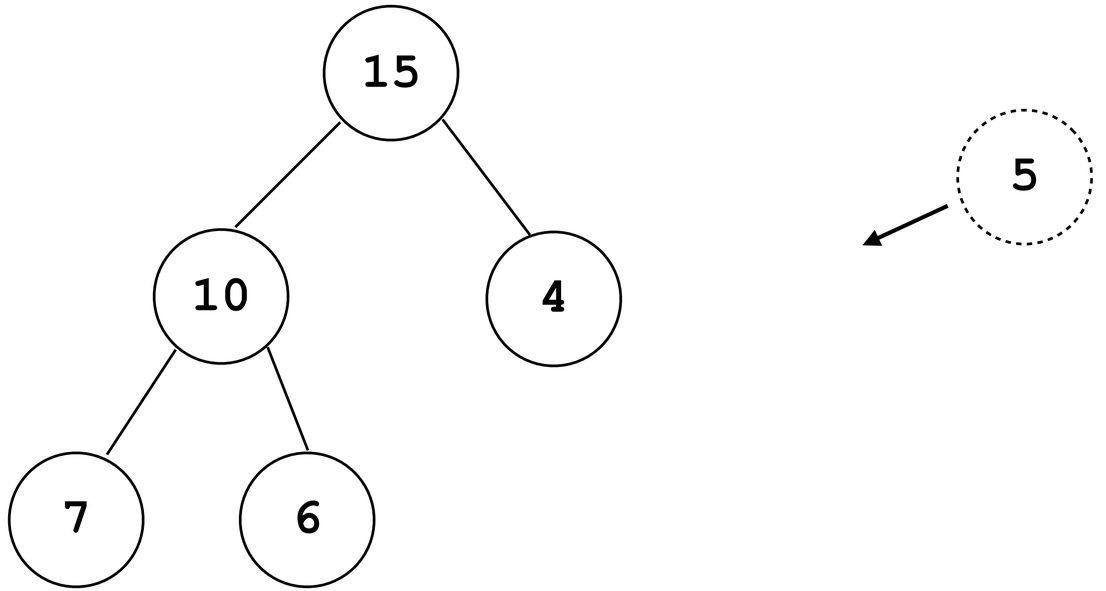
2.逐层调整



堆的插入操作

1.保证形态为完全二叉树的基础上插入

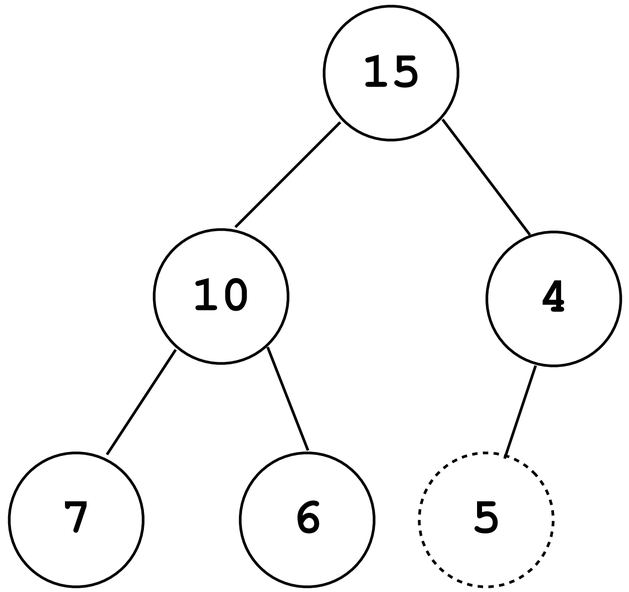
2.逐层调整



堆的插入操作

1.保证形态为完全二叉树的基础上插入

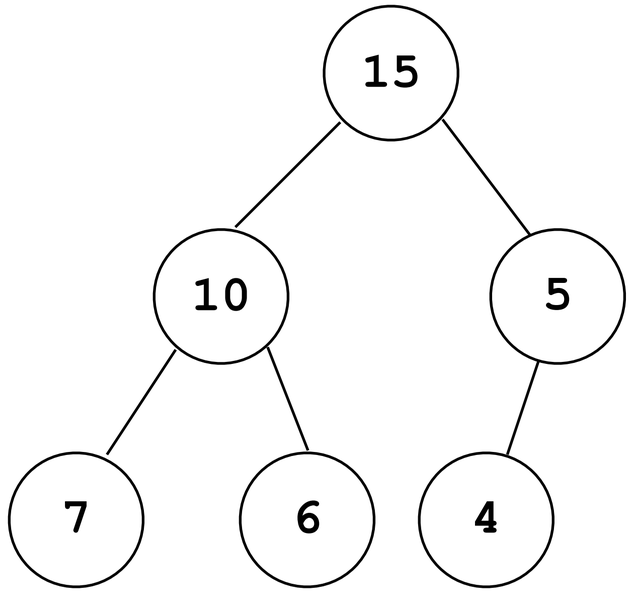
2.逐层调整



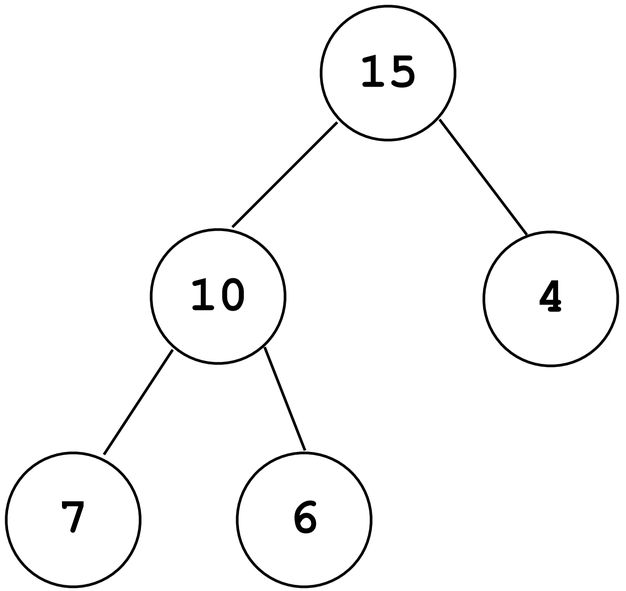
堆的插入操作

1.保证形态为完全二叉树的基础上插入

2.逐层调整



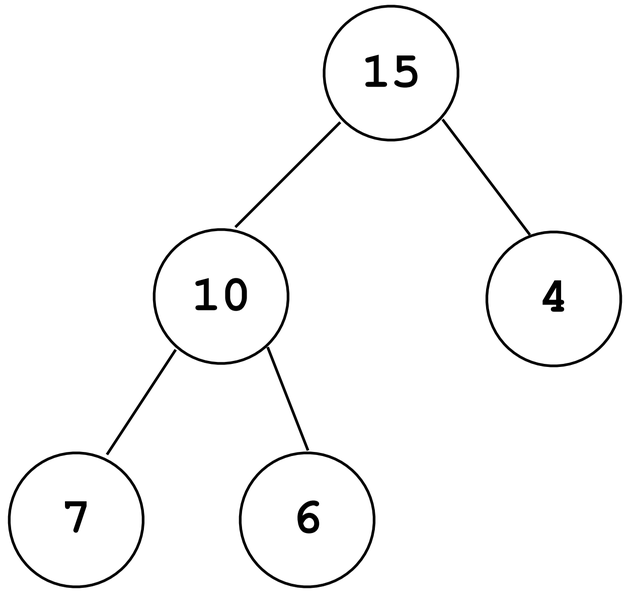
堆的删除操作



堆的删除操作

1.用最后一个结点替换要删除的结点

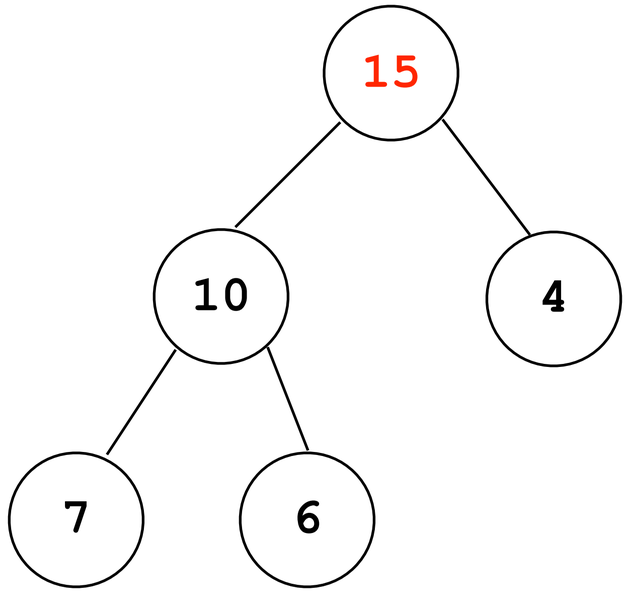
2.逐层调整



堆的删除操作

1.用最后一个结点替换要删除的结点

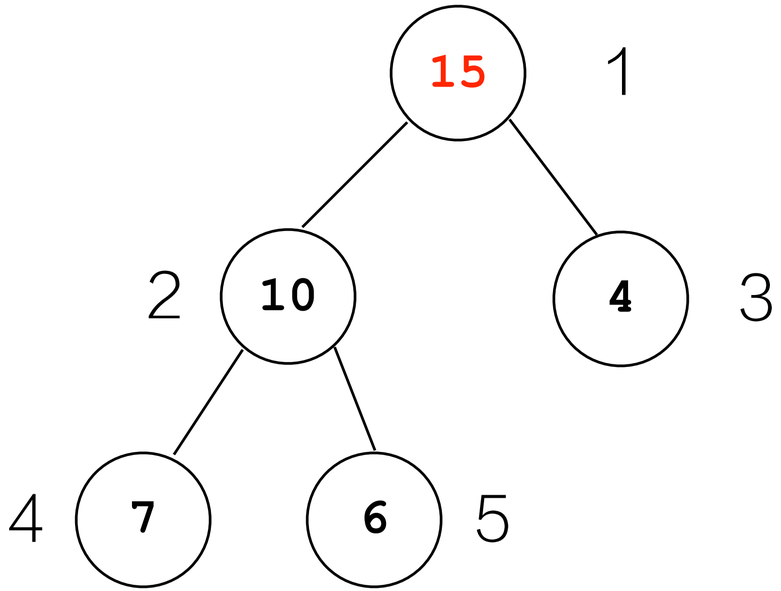
2.逐层调整



堆的删除操作

1.用最后一个结点替换要删除的结点

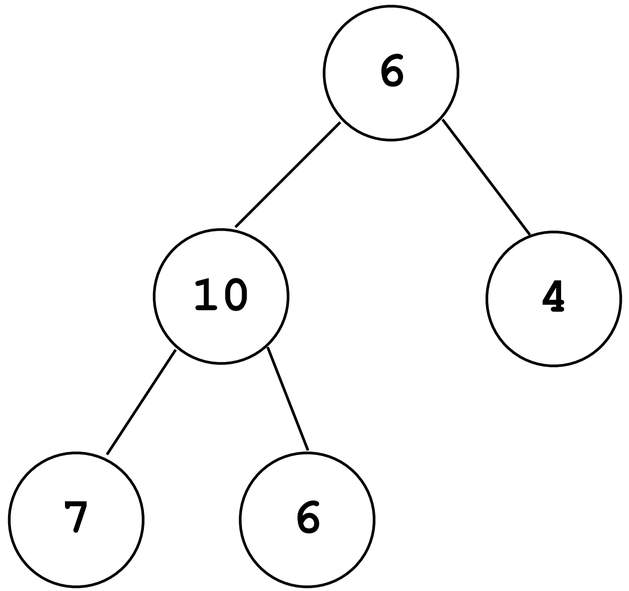
2.逐层调整



堆的删除操作

1.用最后一个结点替换要删除的结点

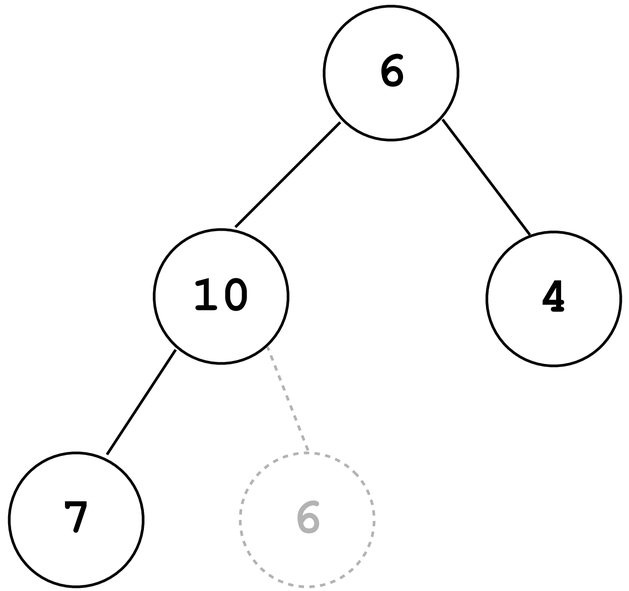
2.逐层调整



堆的删除操作

1.用最后一个结点替换要删除的结点

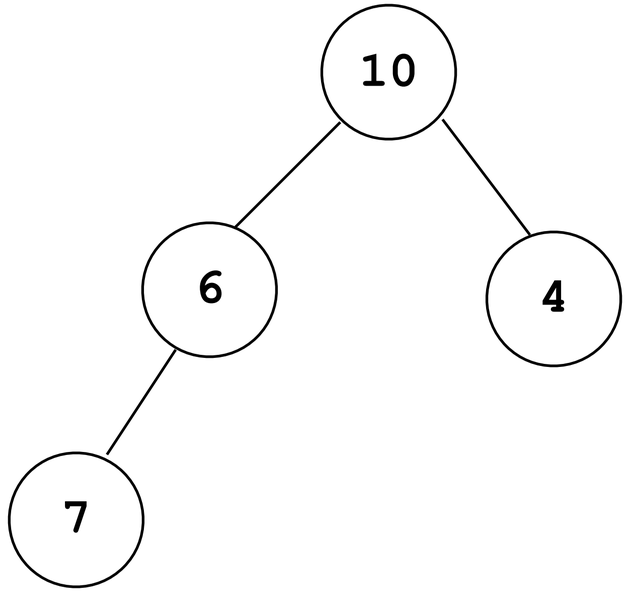
2.逐层调整



堆的删除操作

1.用最后一个结点替换要删除的结点

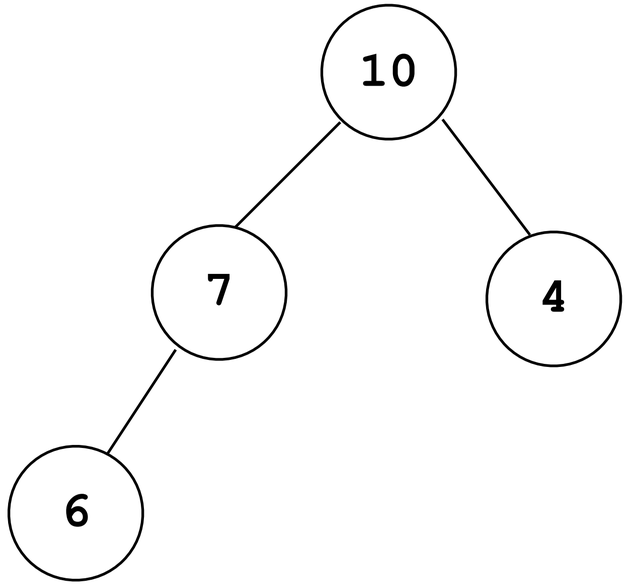
2.逐层调整



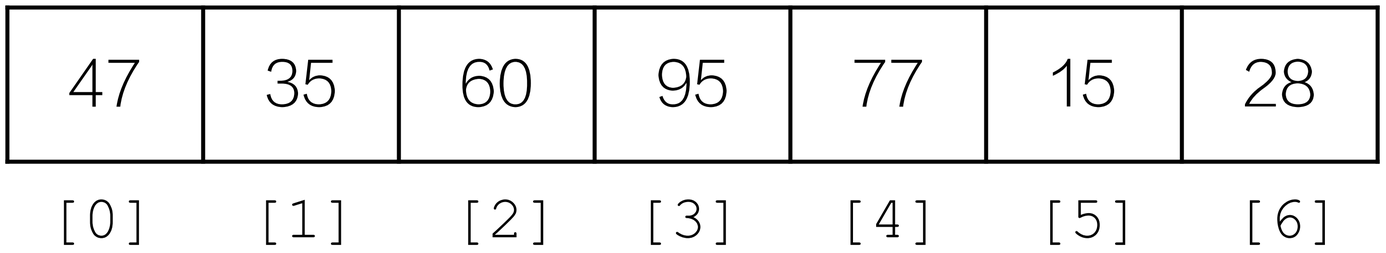
堆的删除操作

1.用最后一个结点替换要删除的结点

2.逐层调整



堆排序



堆排序

使用顺序存储方式构建二叉树

47	35	60	95	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]

堆排序

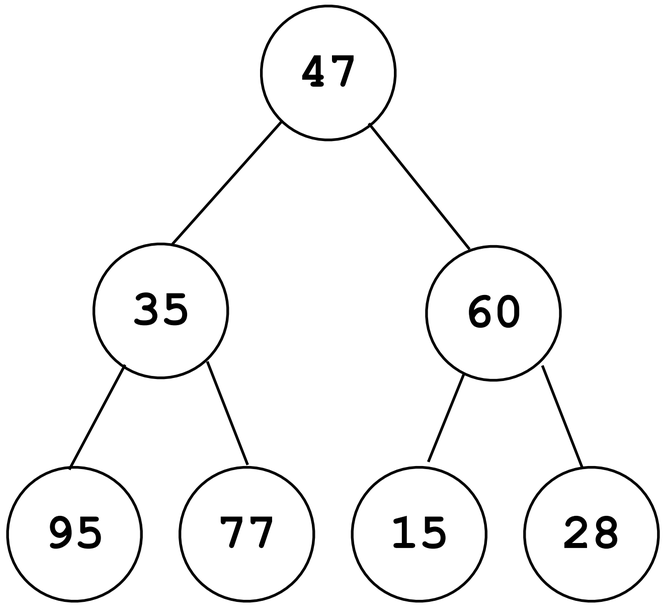
使用顺序存储方式构建二叉树

-1	47	35	60	95	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

堆排序

根据待排序的数组，构建二叉树

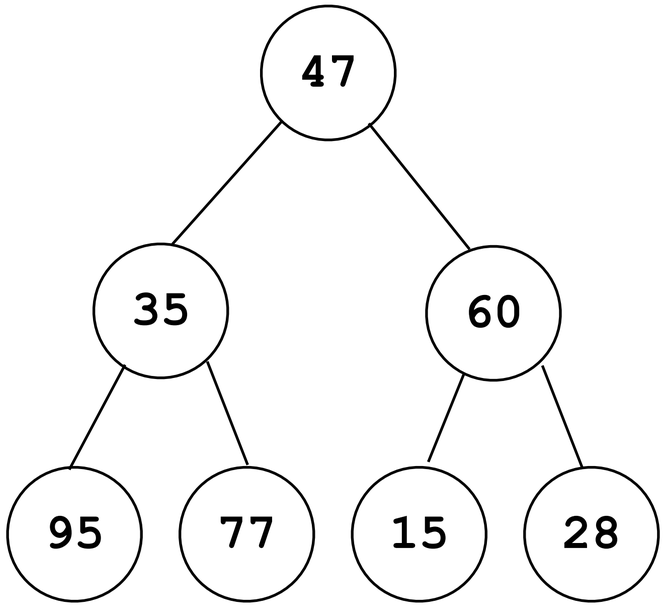
-1	47	35	60	95	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

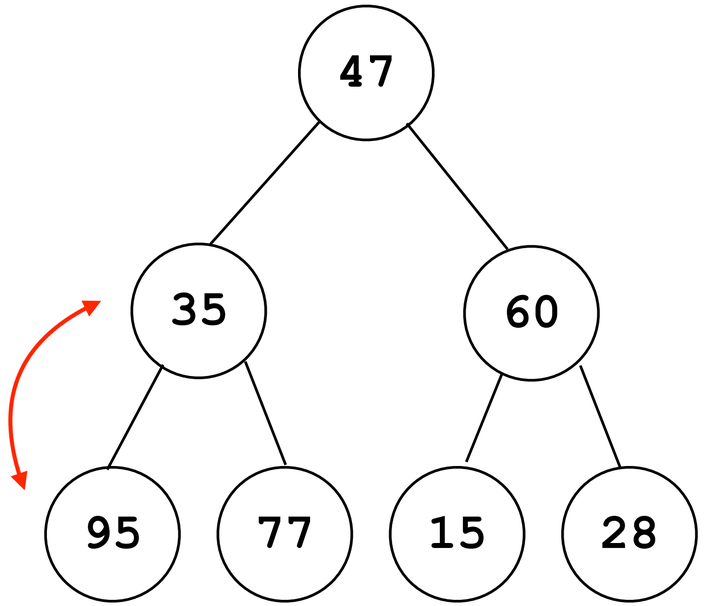
-1	47	35	60	95	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

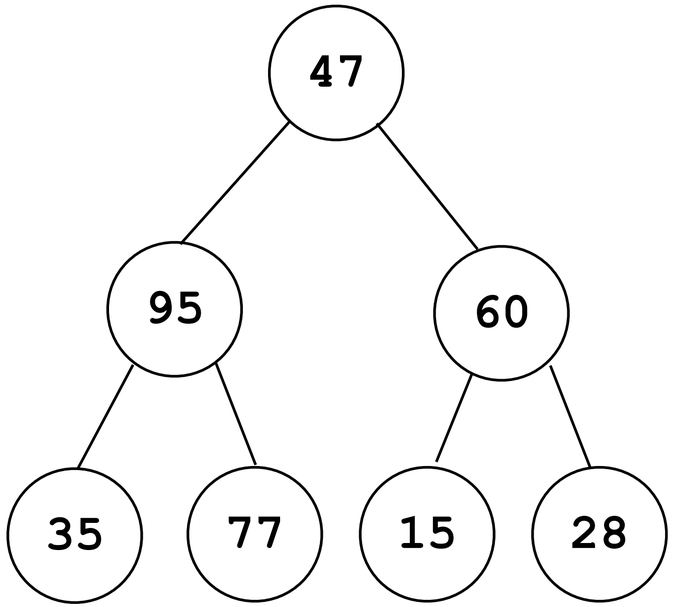
-1	47	35	60	95	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

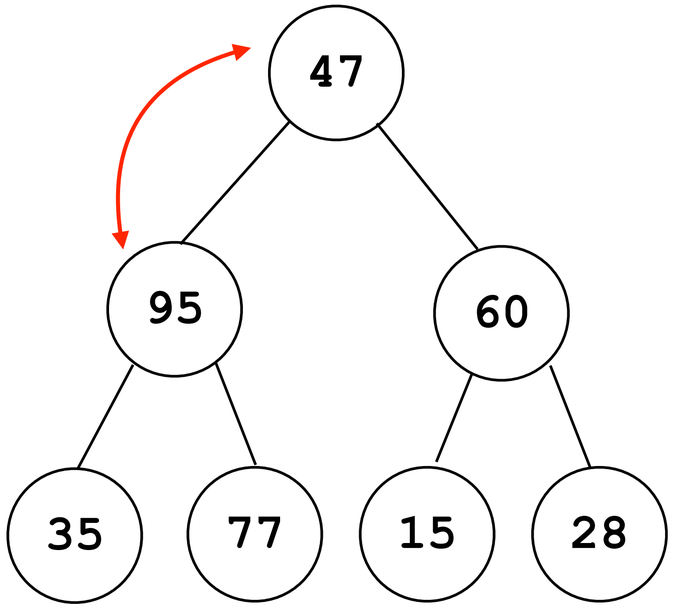
-1	47	95	60	35	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

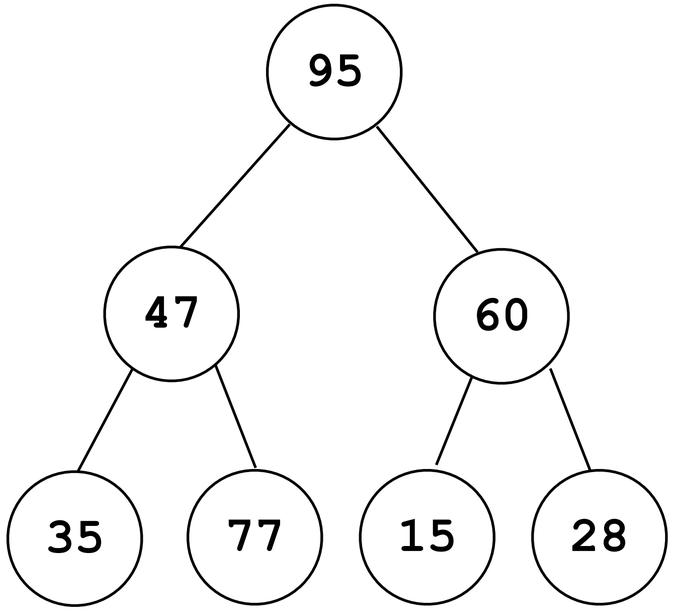
-1	47	95	60	35	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

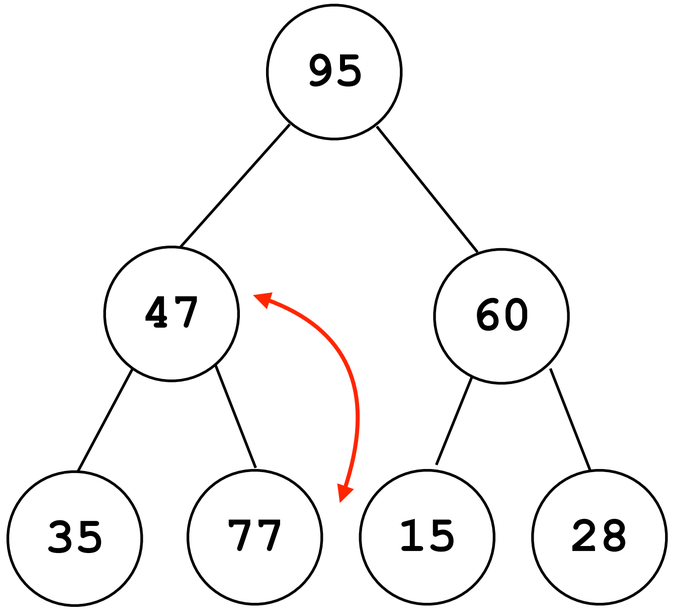
-1	95	47	60	35	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

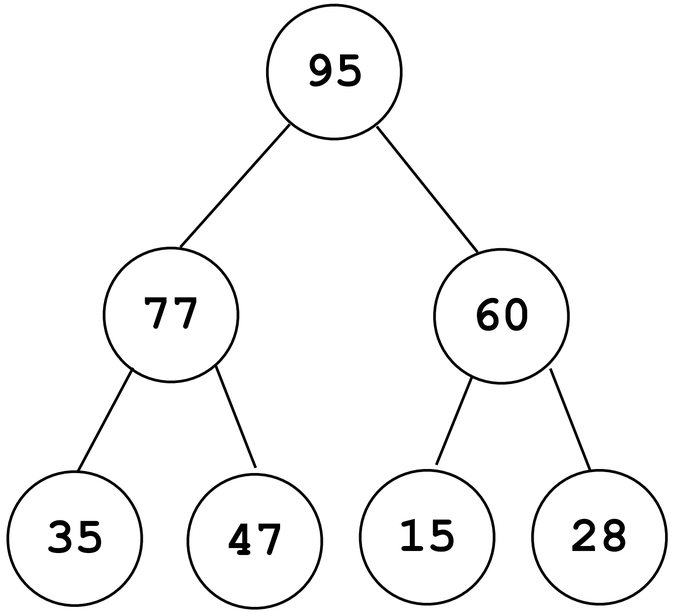
-1	95	47	60	35	77	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

建堆：从最后一个非叶子节点开始向前遍历，每个节点都执行一次向下调整，最终构成一个大根堆。

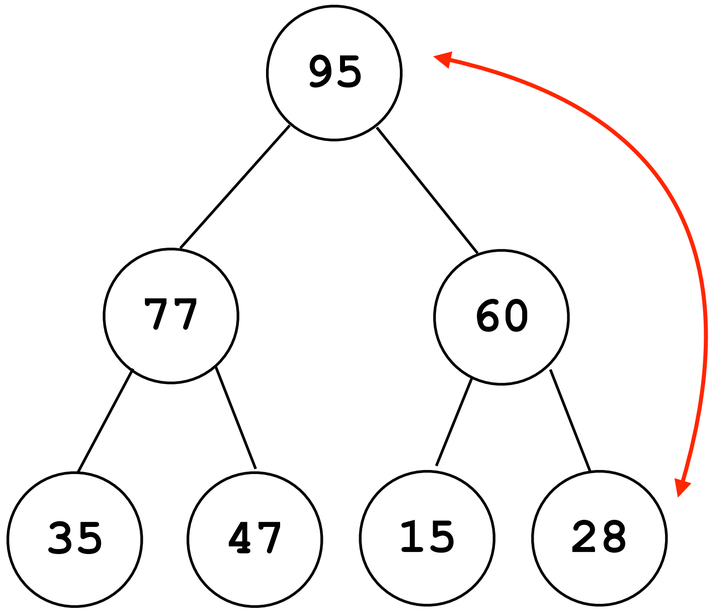
-1	95	77	60	35	47	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

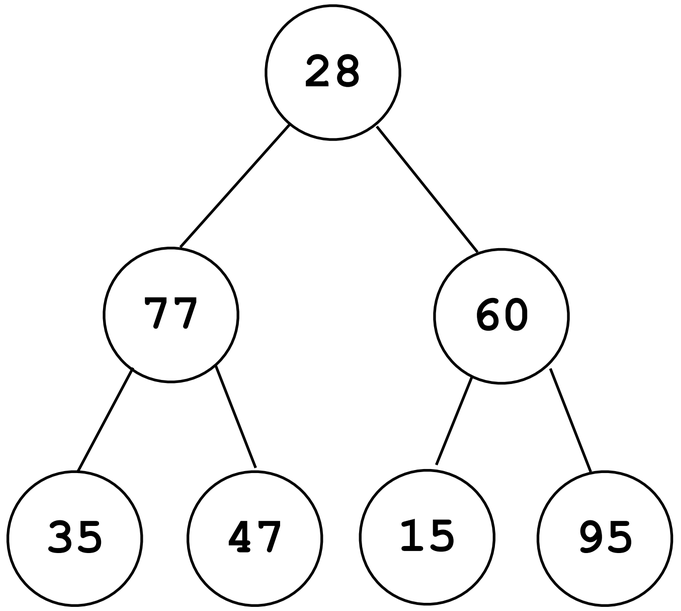
-1	95	77	60	35	47	15	28
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

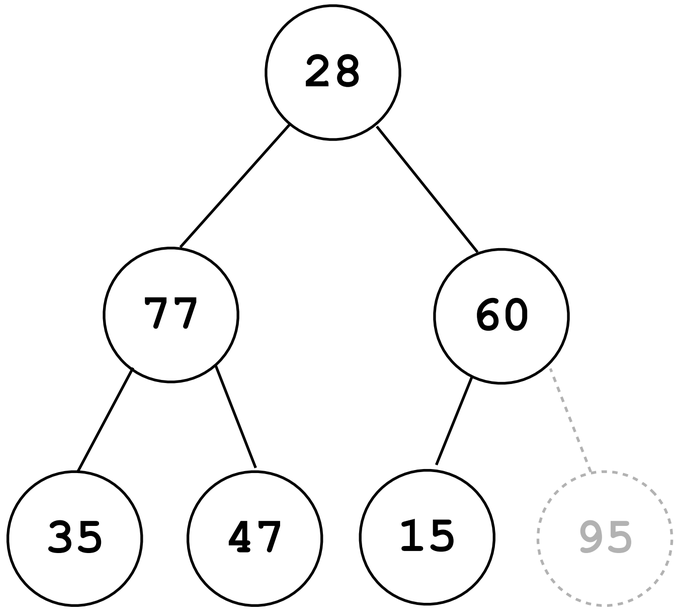
-1	28	77	60	35	47	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

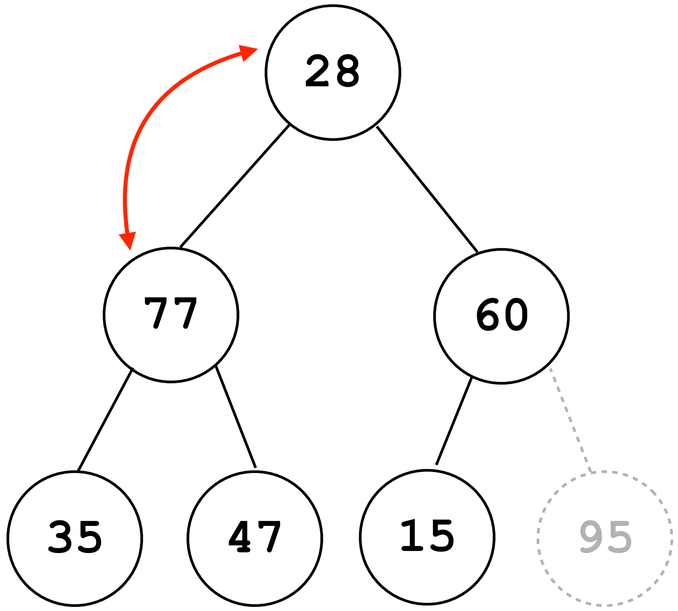
-1	28	77	60	35	47	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

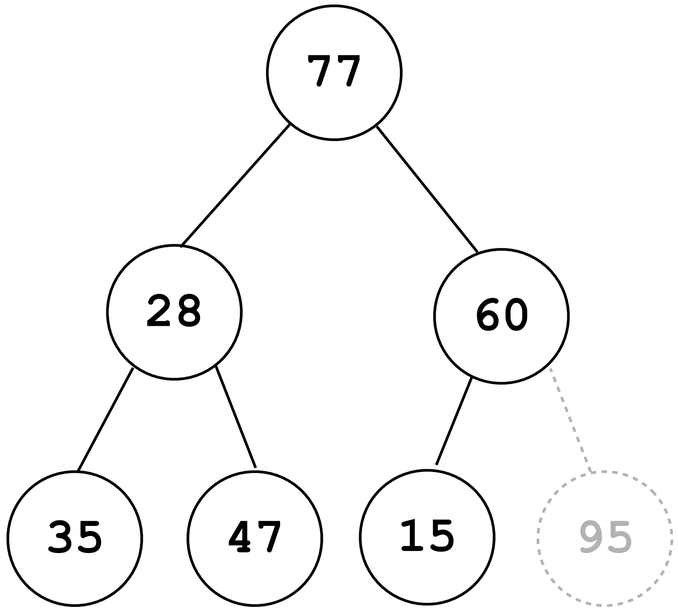
-1	28	77	60	35	47	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

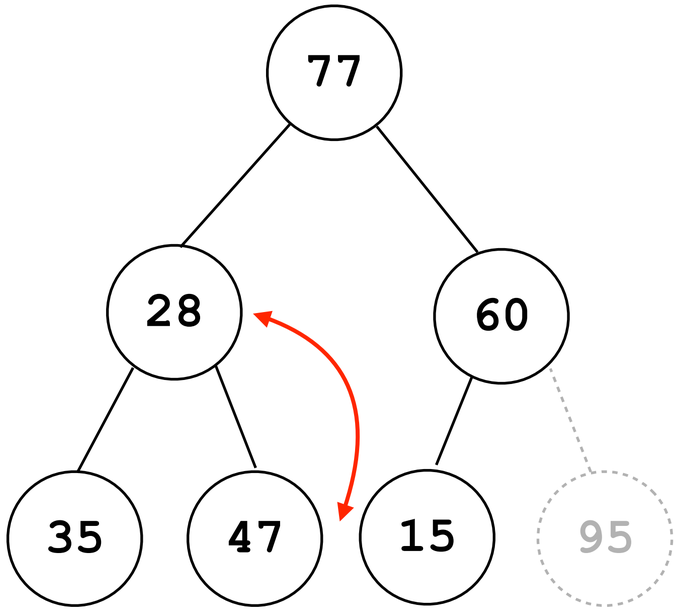
-1	77	28	60	35	47	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

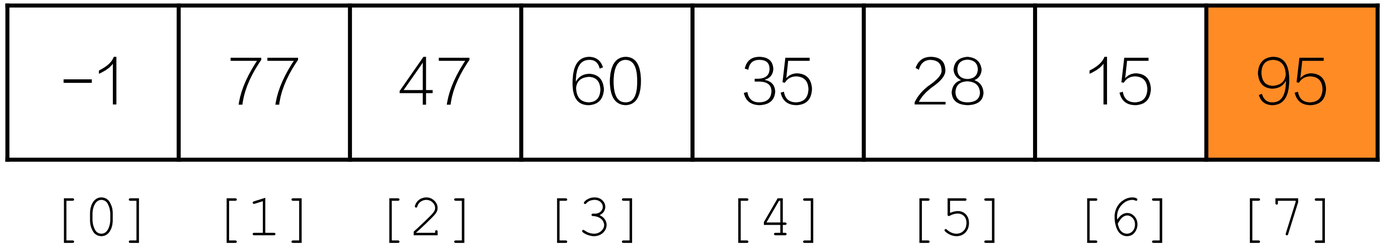
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

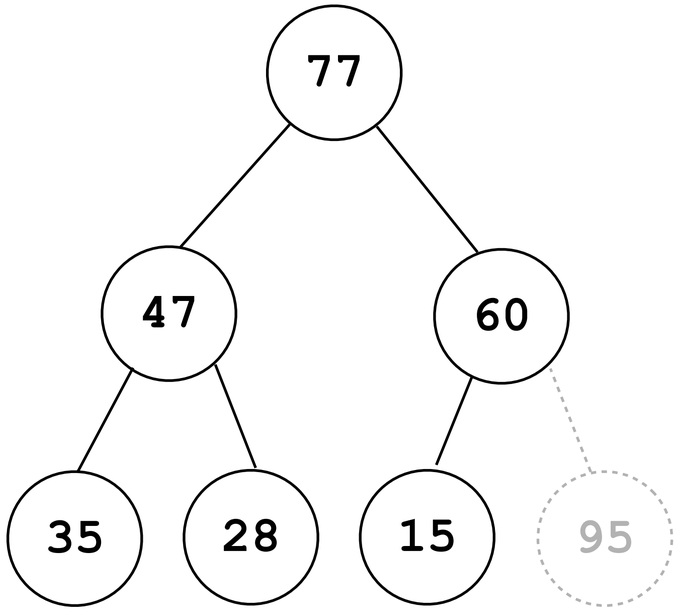
-1	77	28	60	35	47	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

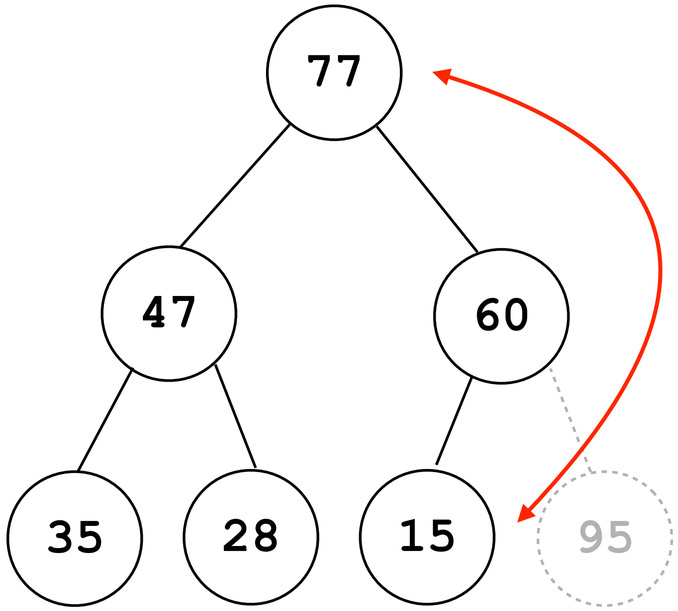




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

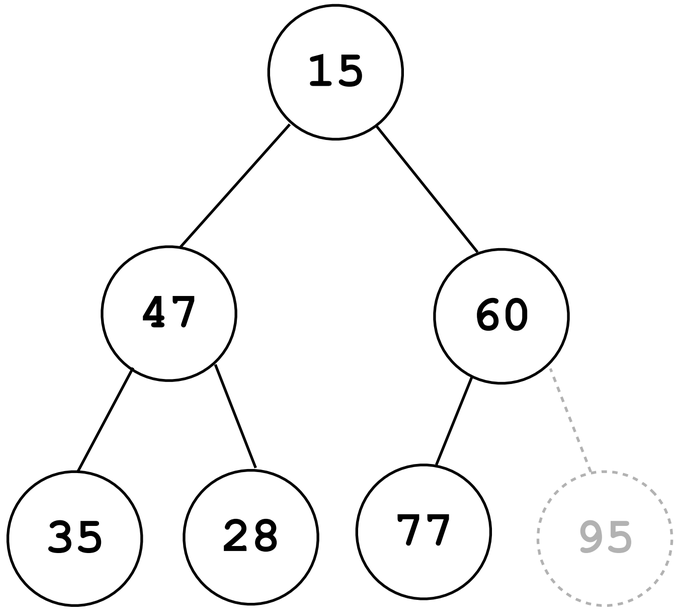
-1	77	47	60	35	28	15	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

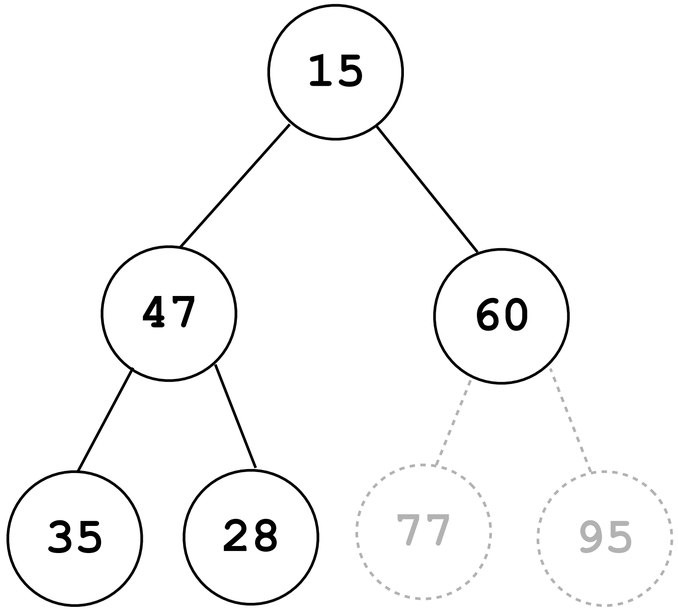
-1	15	47	60	35	28	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

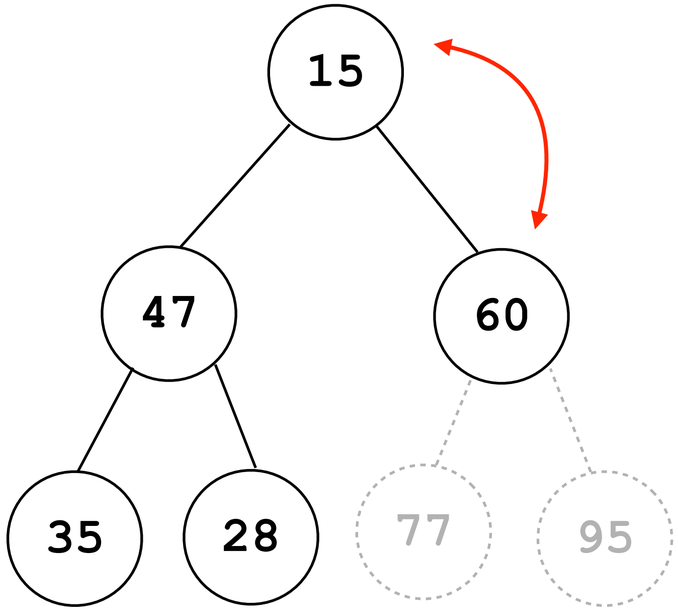
-1	15	47	60	35	28	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

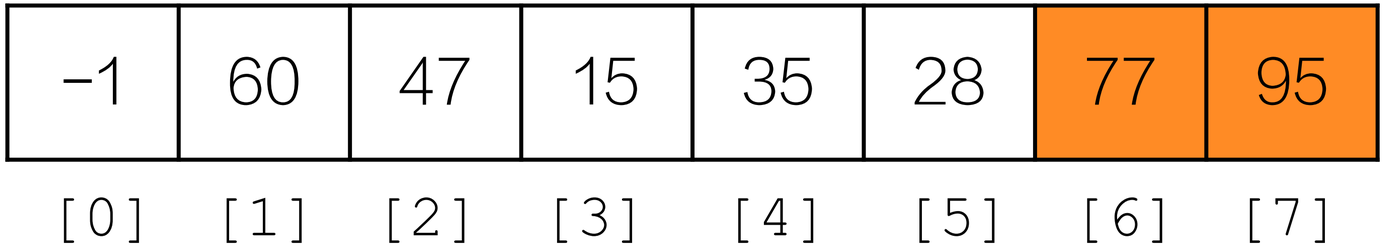
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

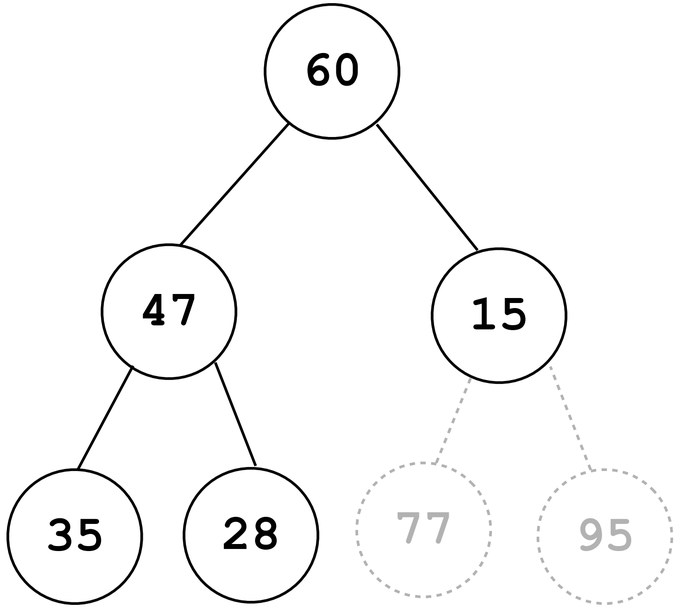
-1	15	47	60	35	28	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

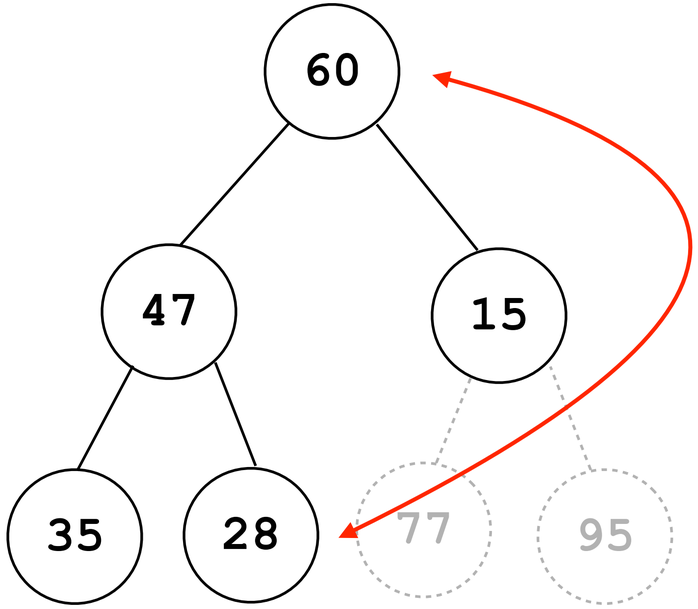




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

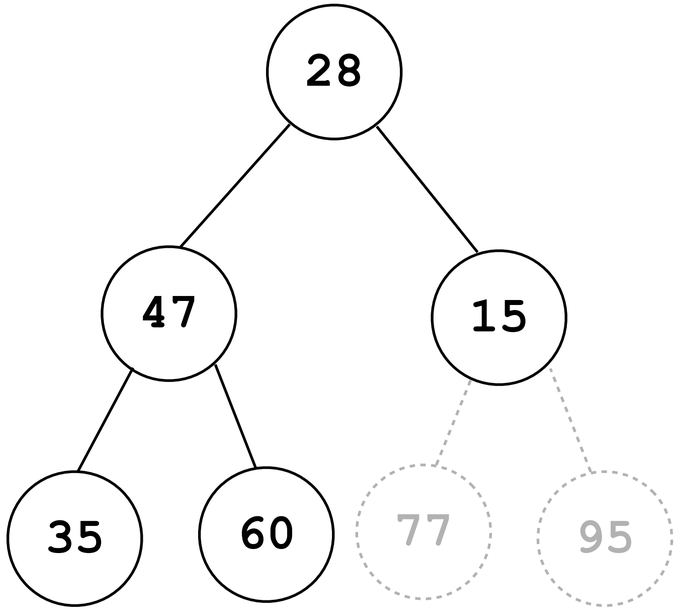
-1	60	47	15	35	28	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

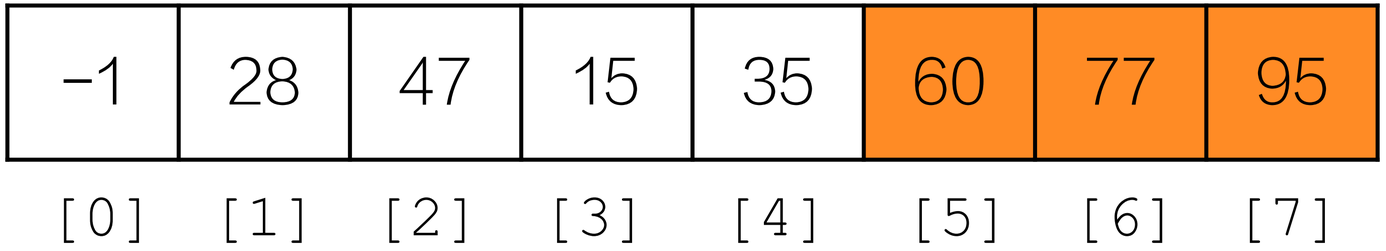
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

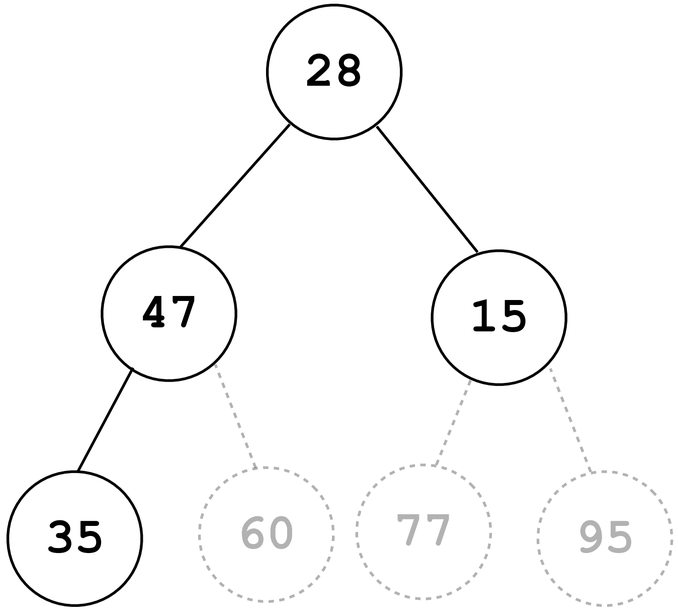
-1	28	47	15	35	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

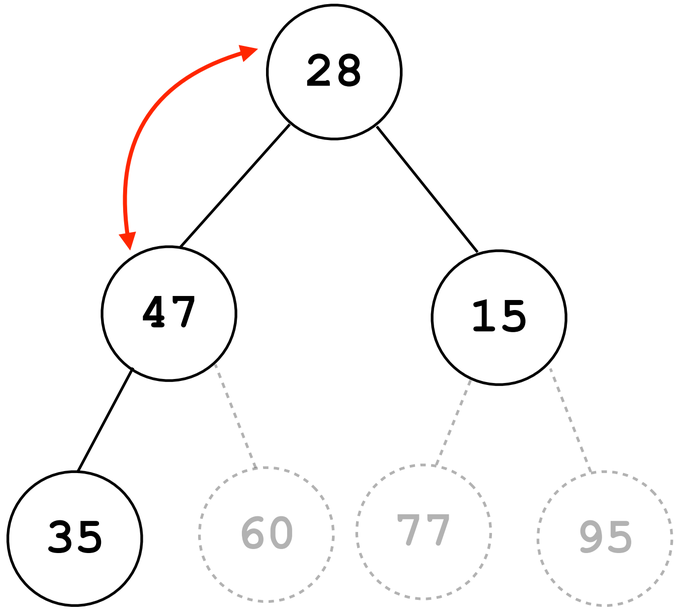




堆排序

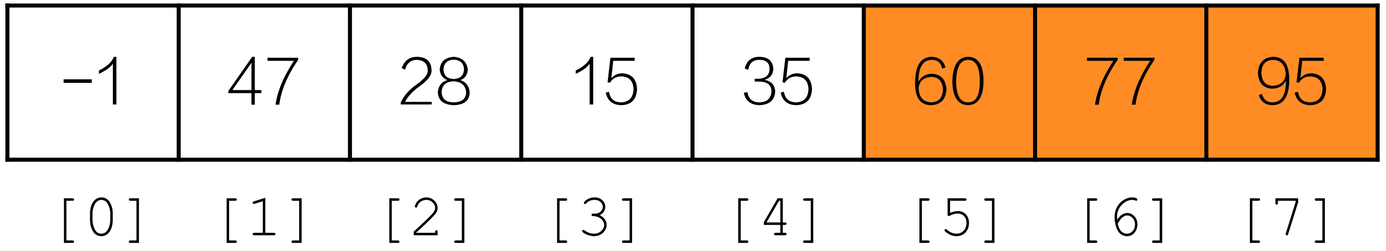
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

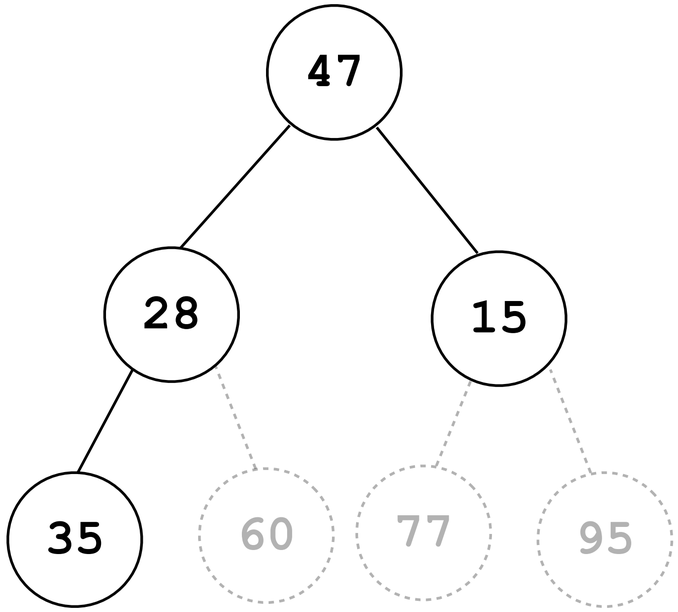
-1	28	47	15	35	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

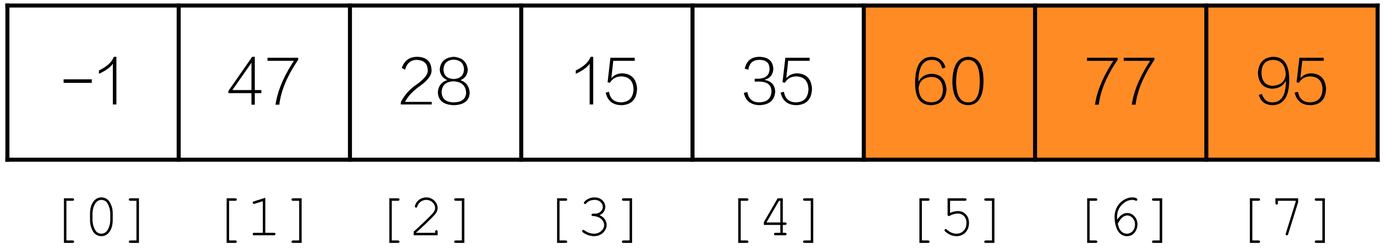
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

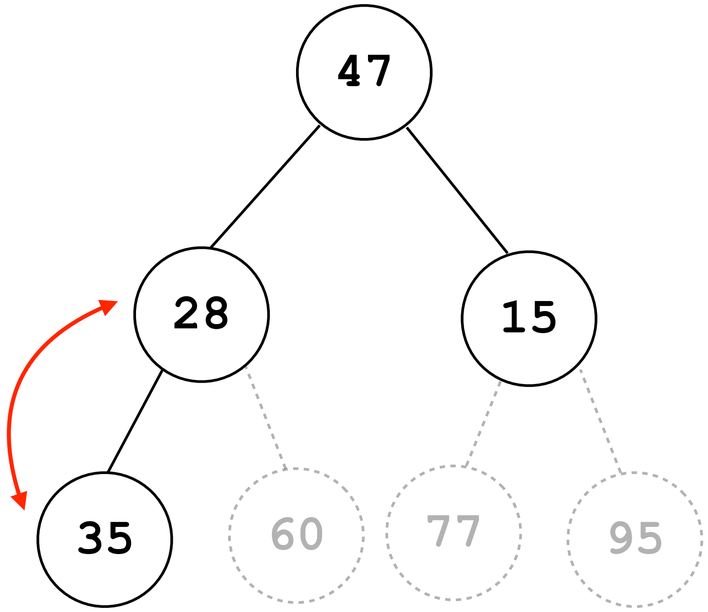




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

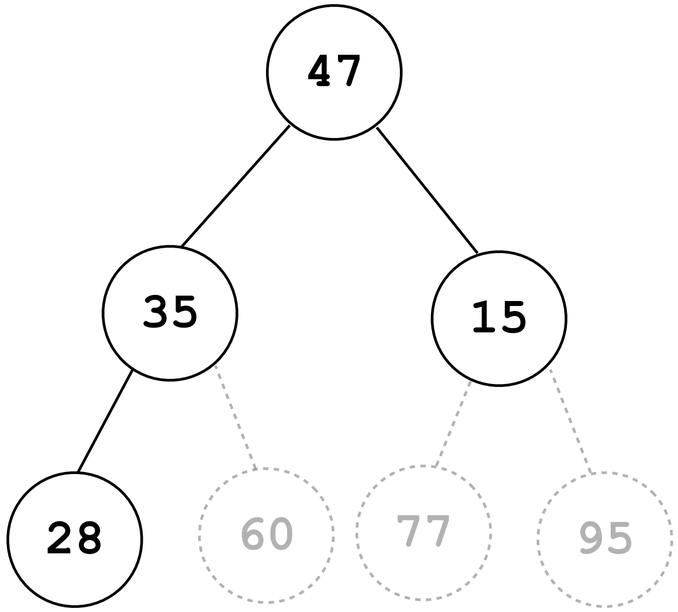




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

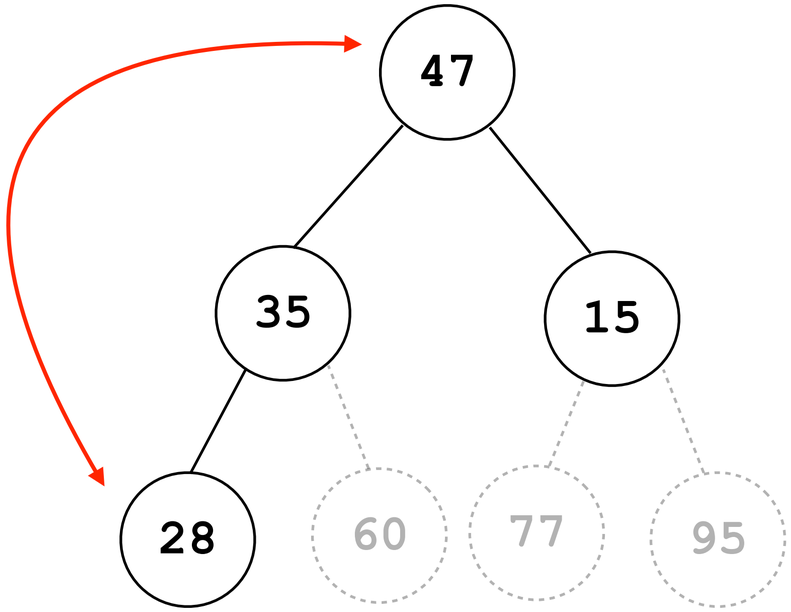
-1	47	35	15	28	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

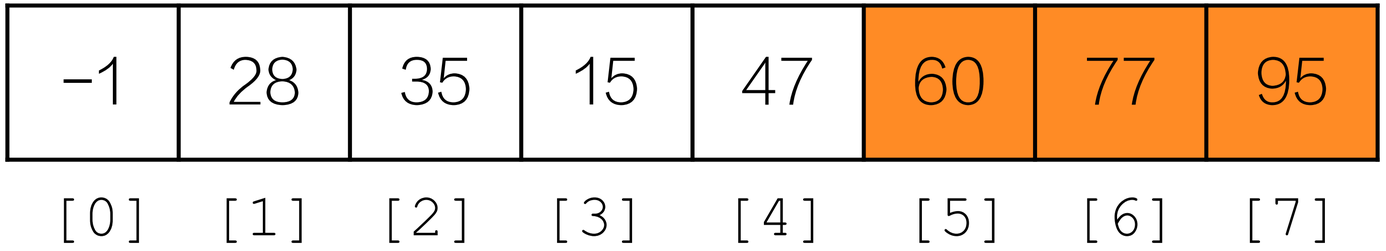
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

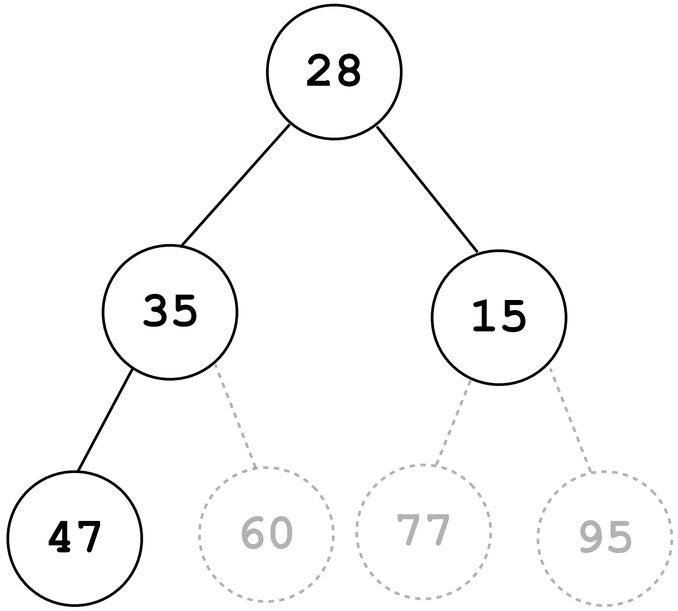
-1	47	35	15	28	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

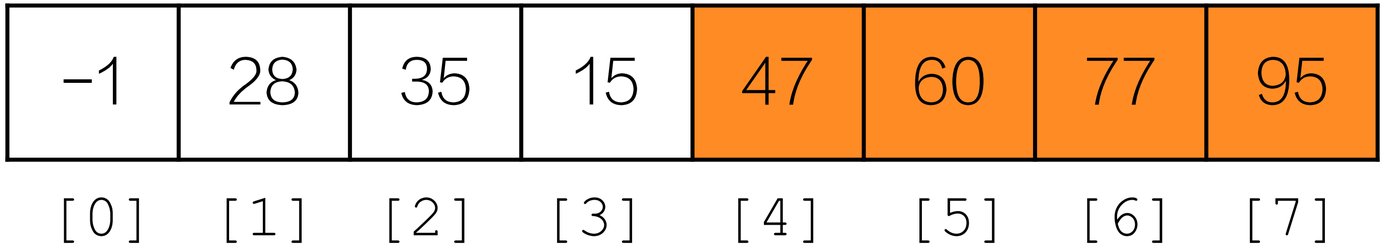
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

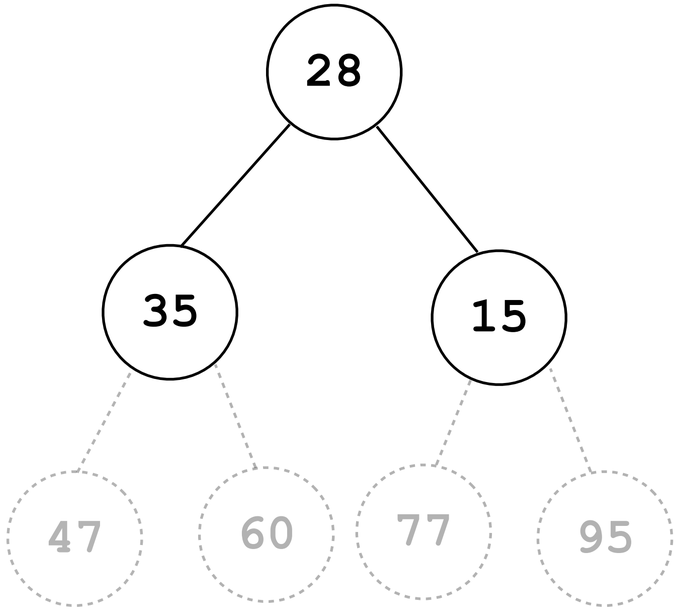




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

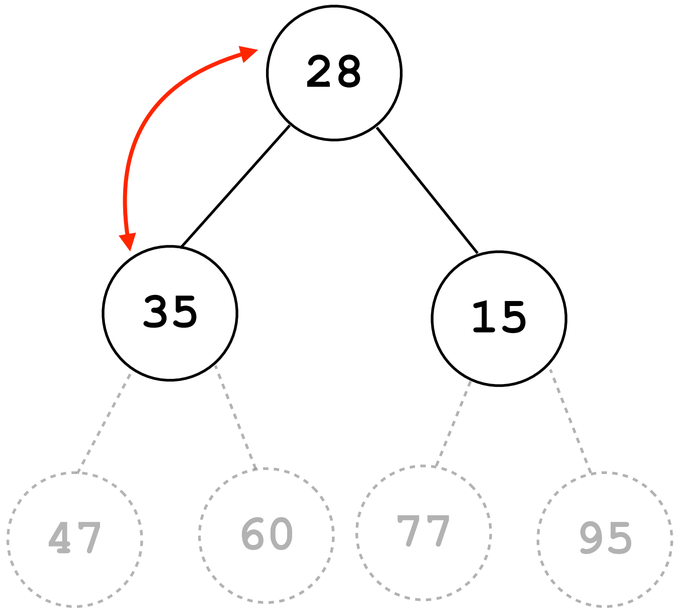




堆排序

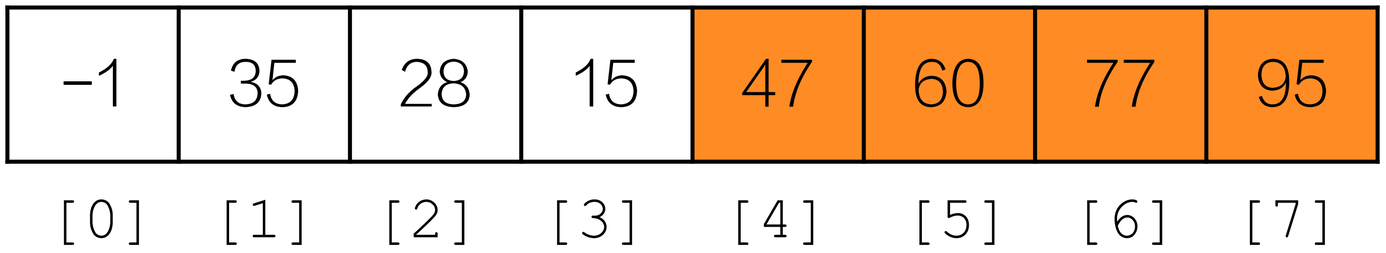
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

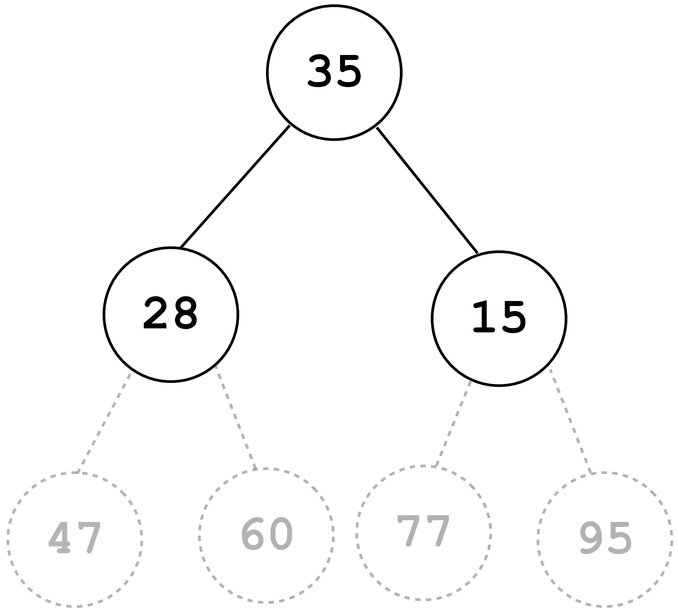
-1	28	35	15	47	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

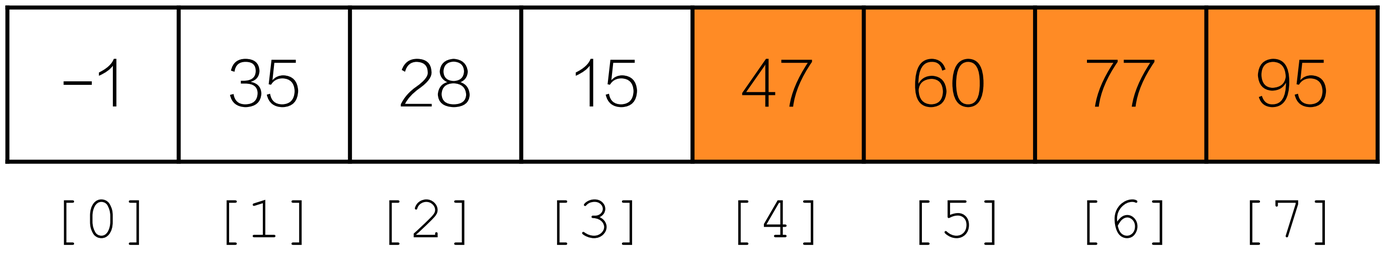
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

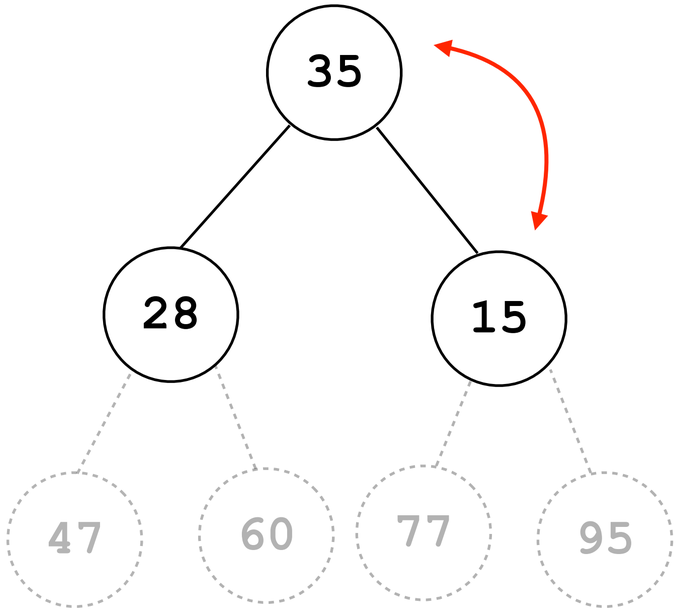




堆排序

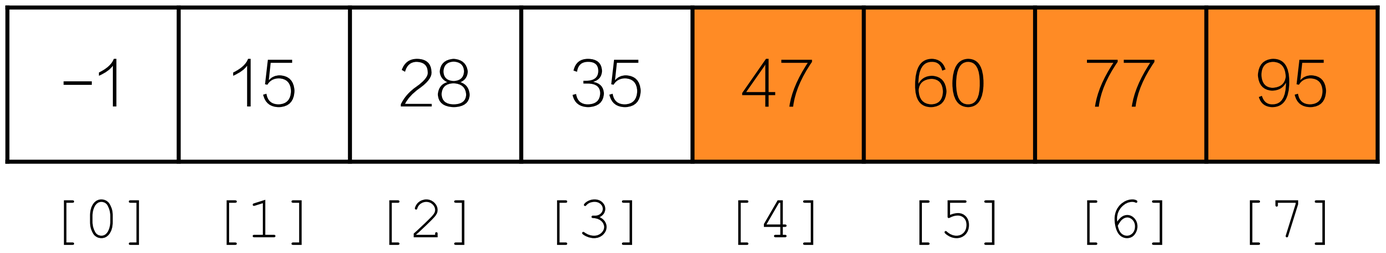
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

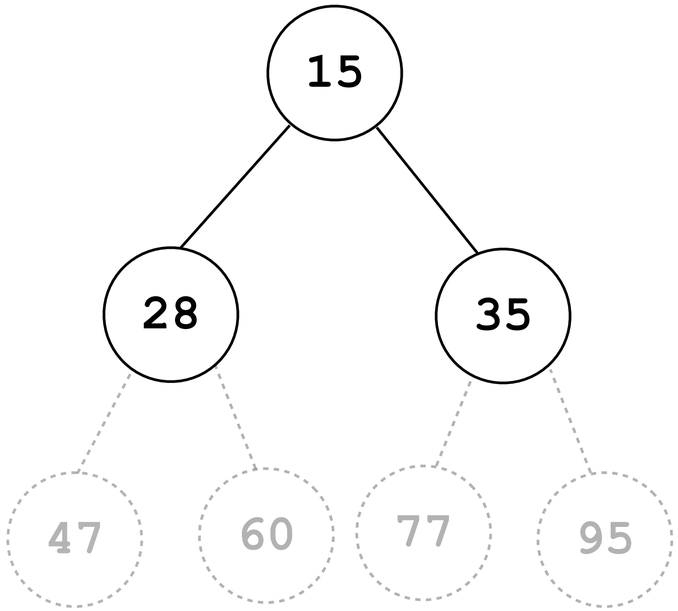




堆排序

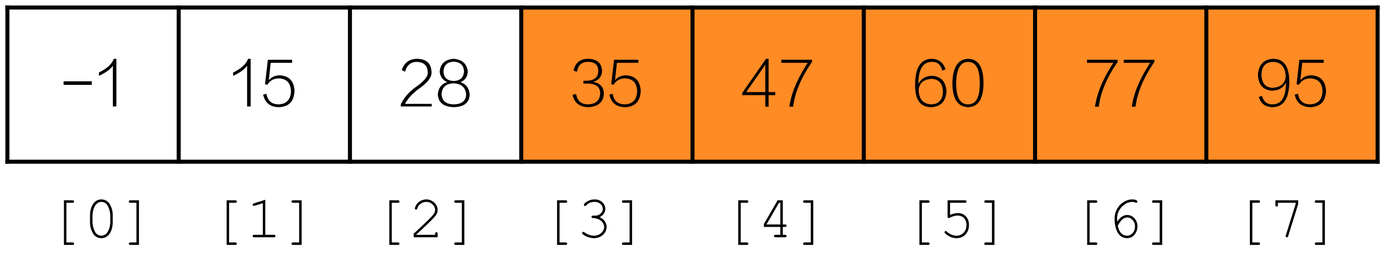
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

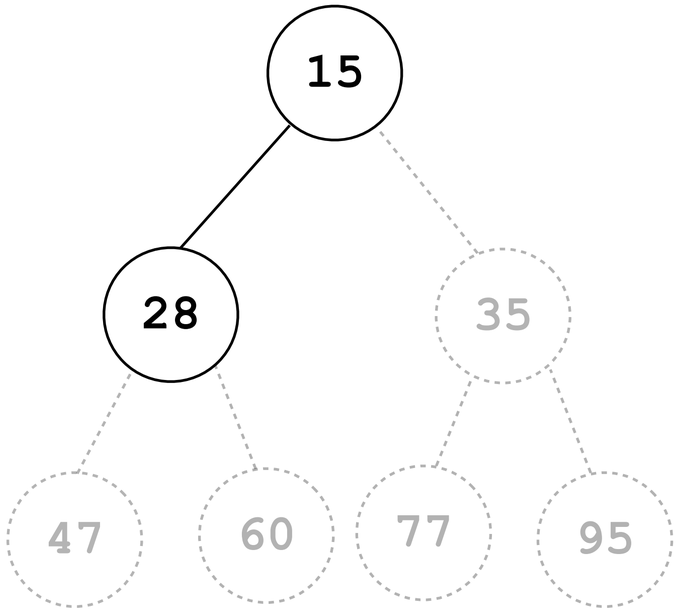




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

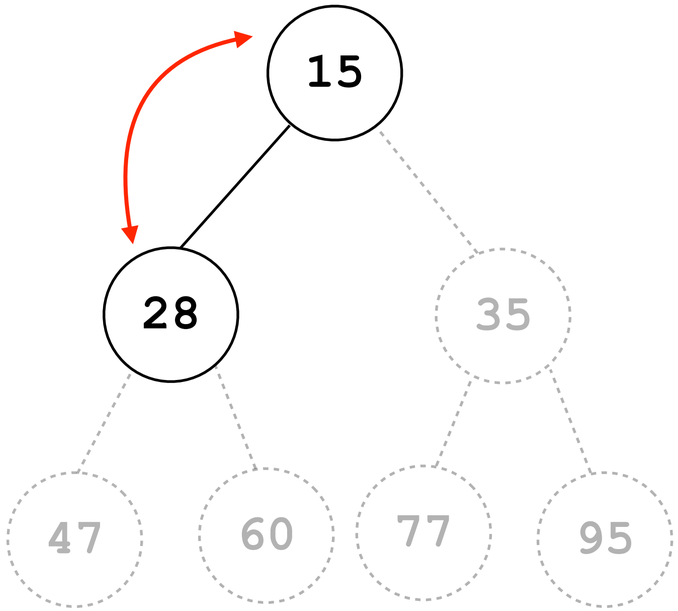




堆排序

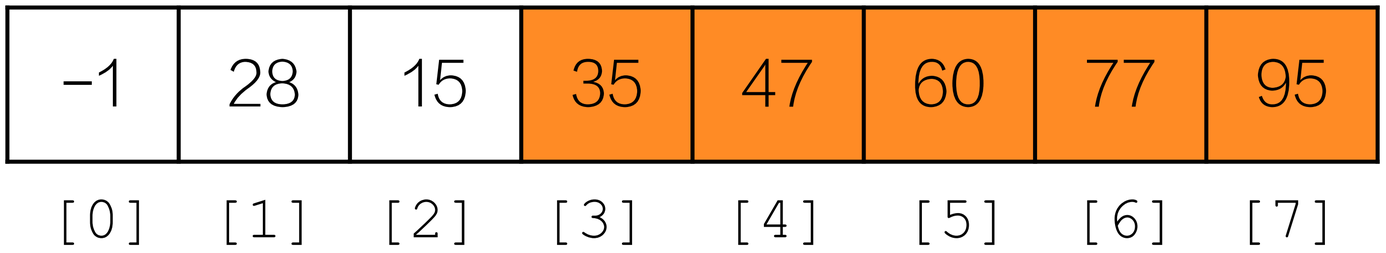
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

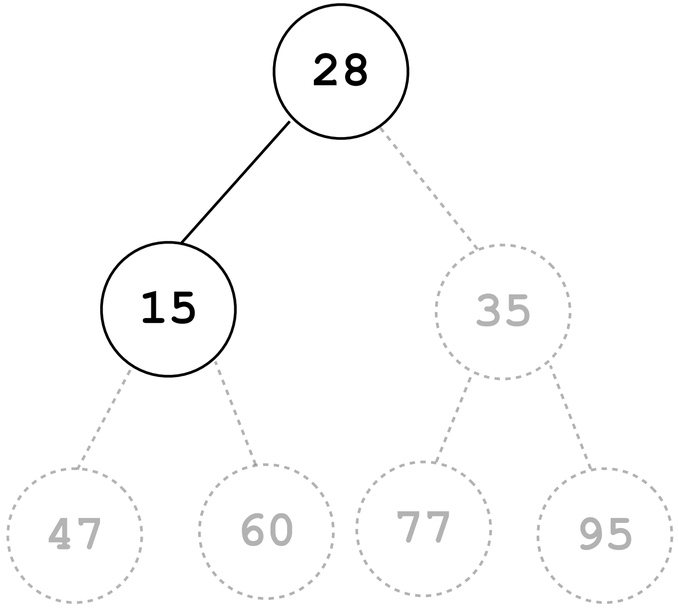
-1	15	28	35	47	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



堆排序

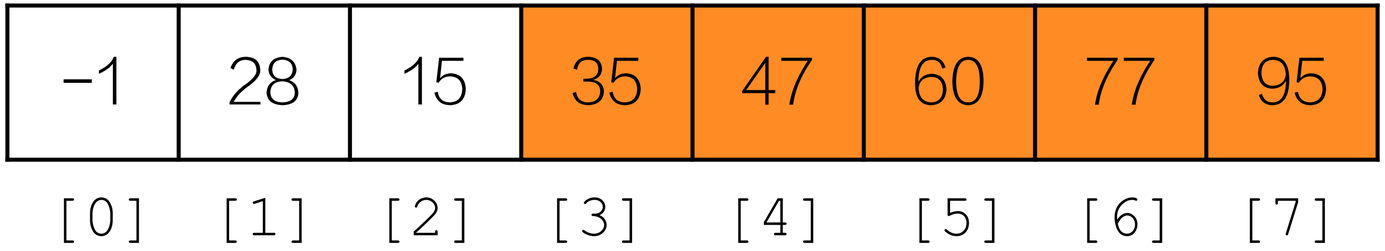
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

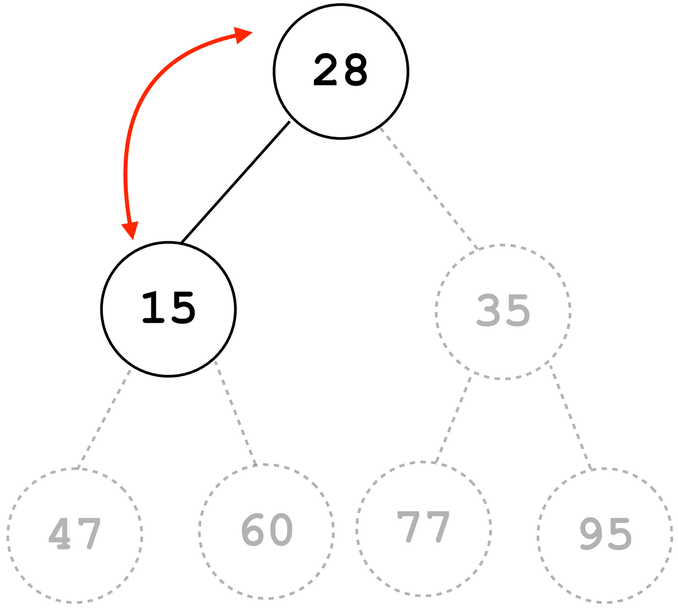




堆排序

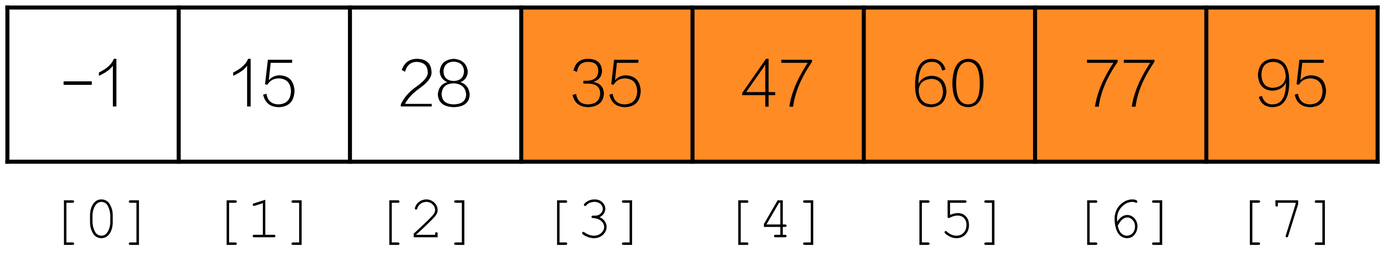
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

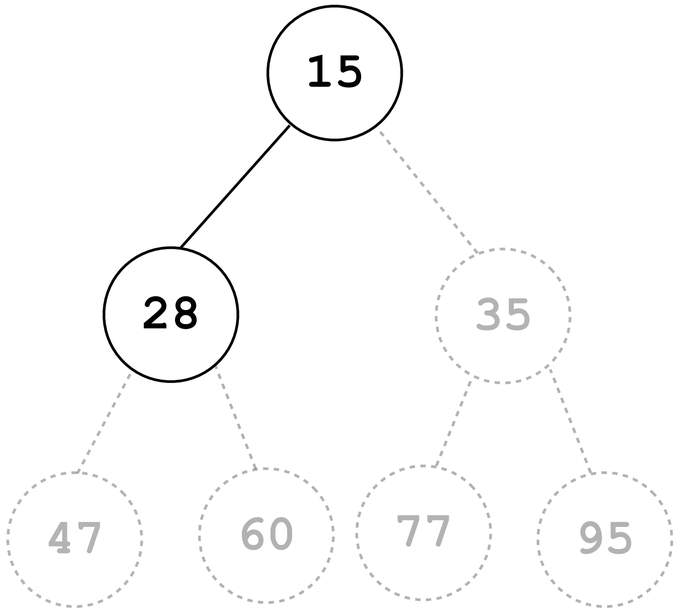




堆排序

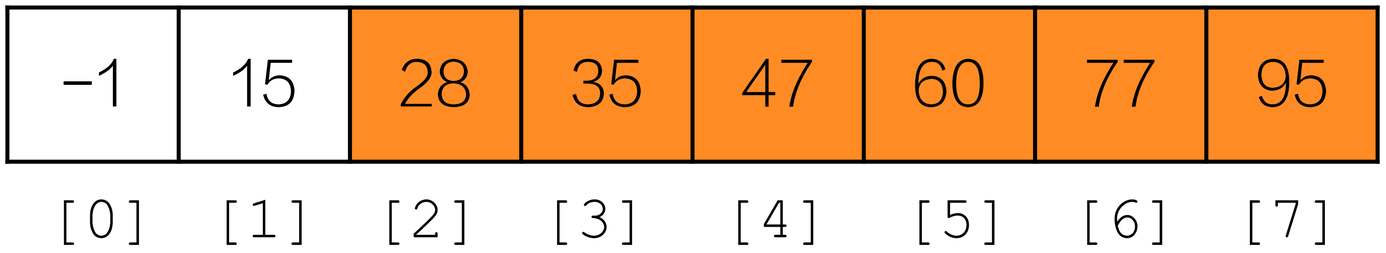
排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

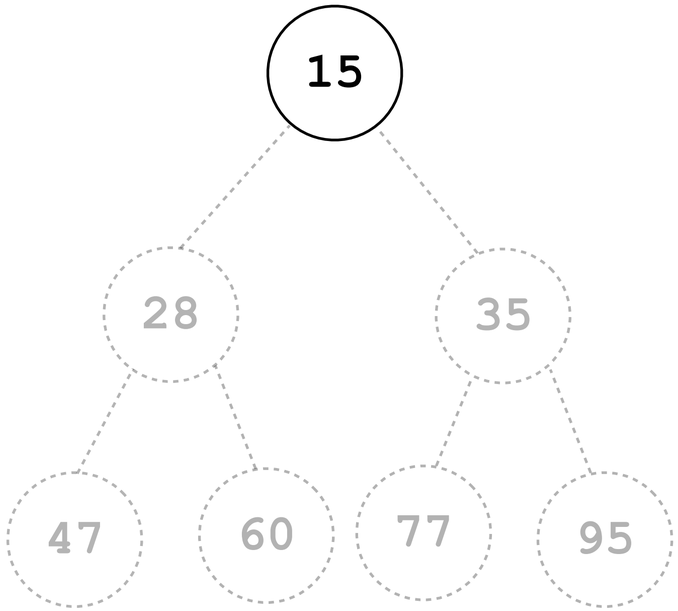




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

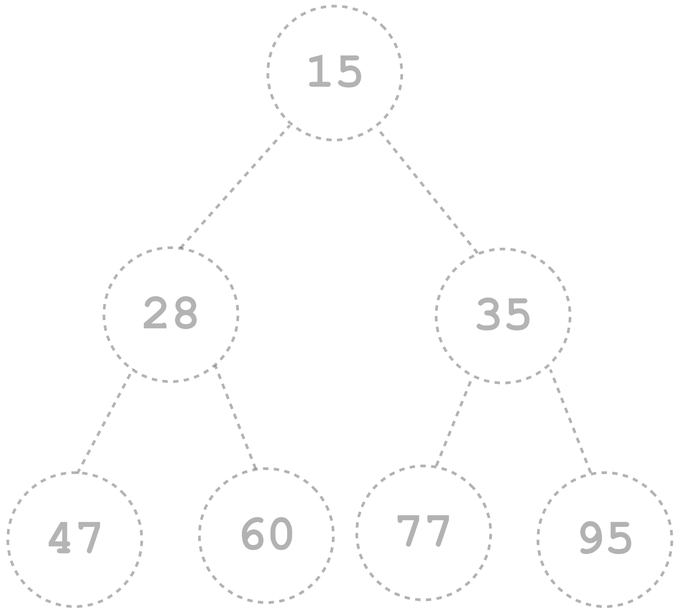




堆排序

排序：不断将堆顶（最大值）放到数组末尾，并重新调整剩下部分为大根堆。

-1	15	28	35	47	60	77	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]



真题解析

[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19

[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19

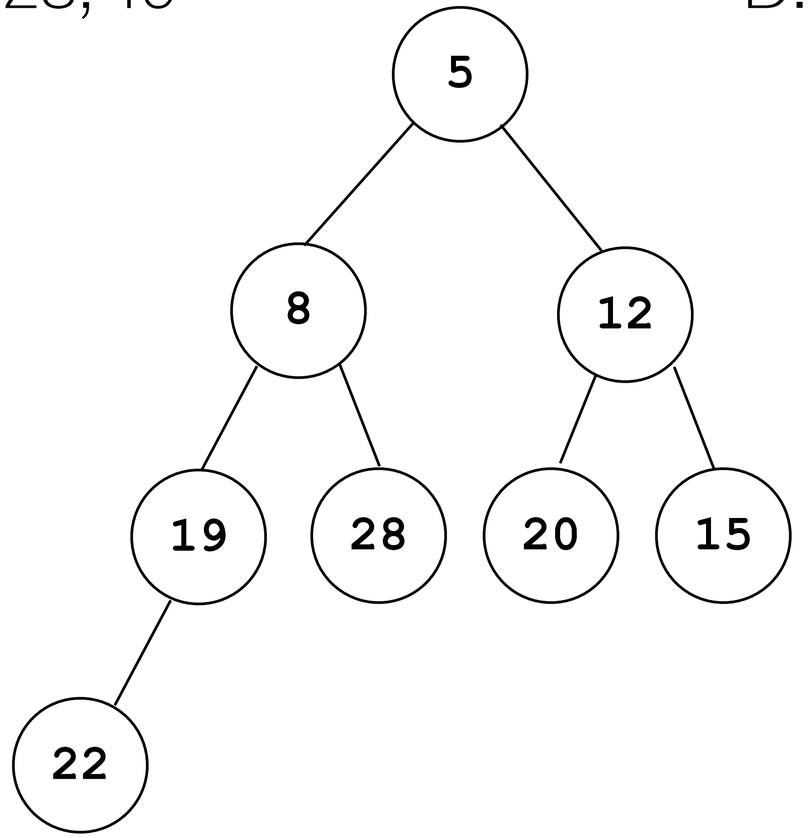
[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19



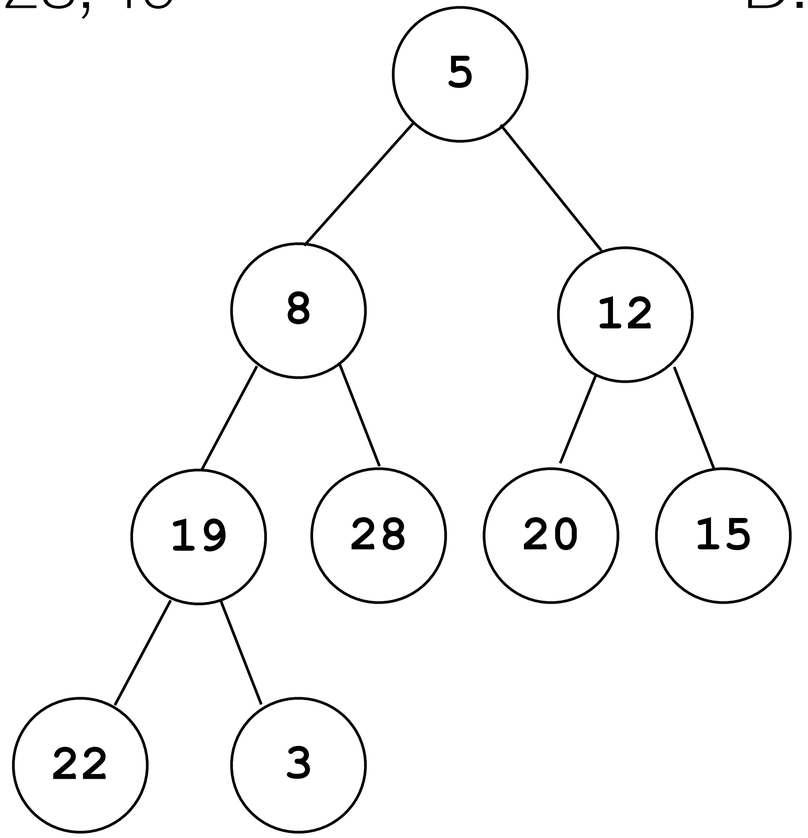
[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19



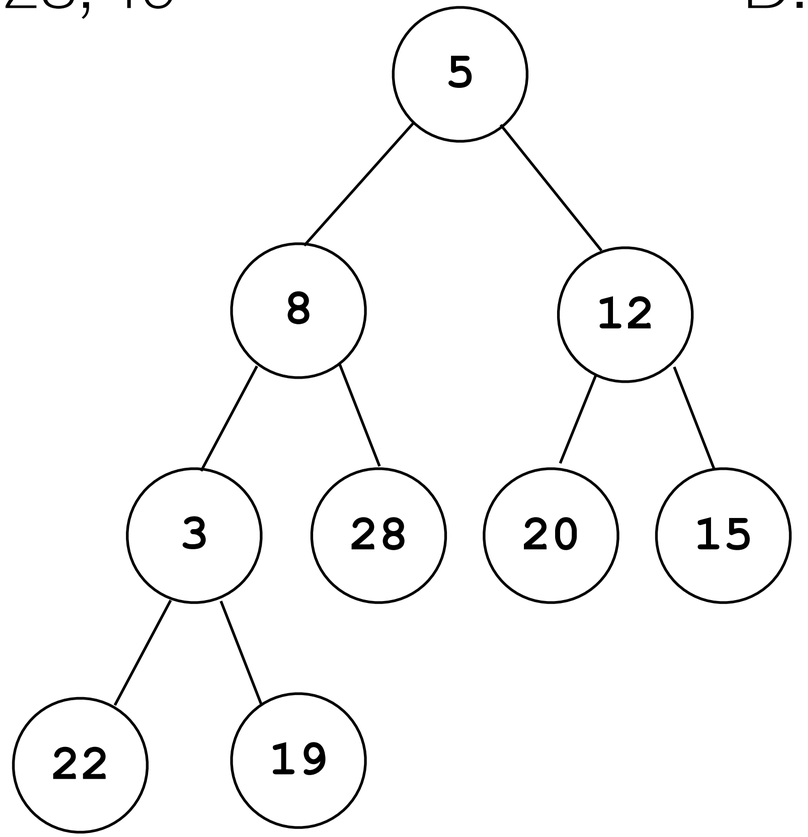
[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19



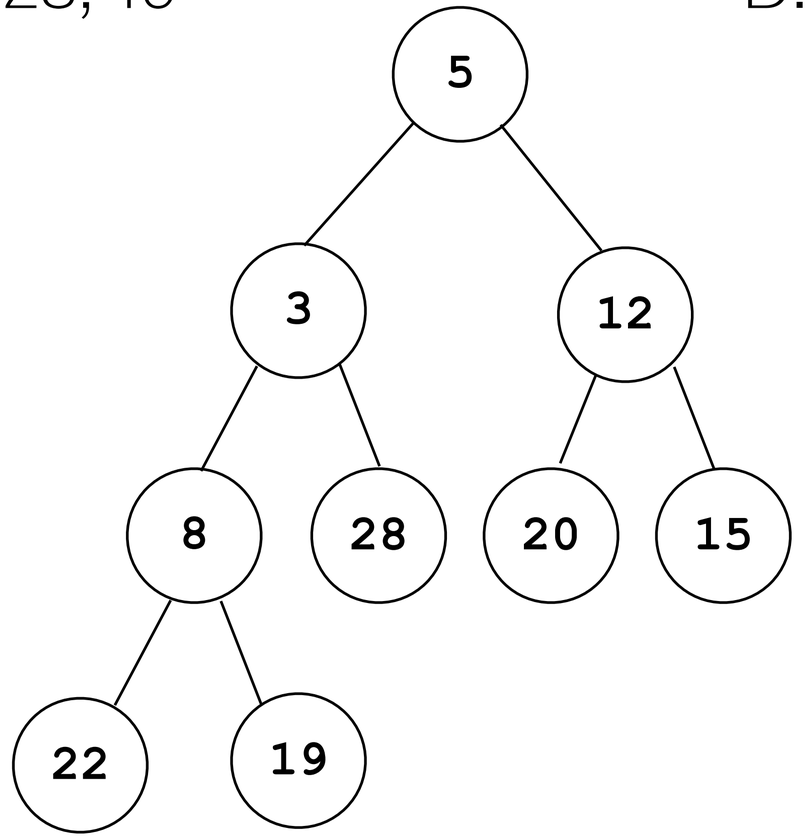
[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19



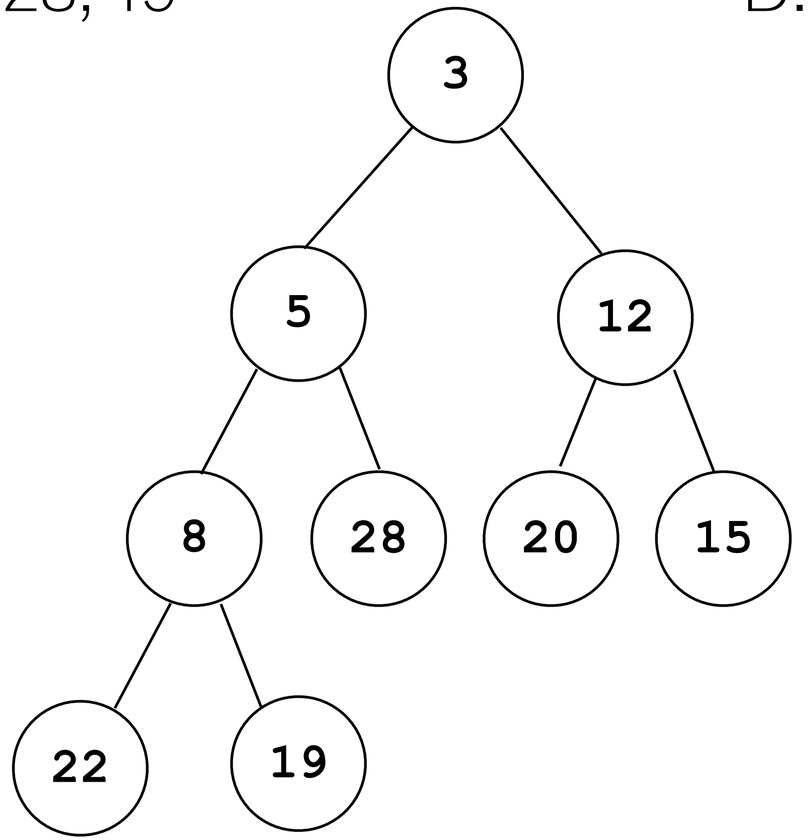
[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19



真题解析

[2009] 9. 已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

A. 3, 5, 12, 8, 28, 20, 15, 22, 19

B. 3, 5, 12, 19, 20, 15, 22, 8, 28

C. 3, 8, 12, 5, 20, 15, 22, 28, 19

D. 3, 12, 5, 8, 28, 20, 15, 22, 19

真题解析

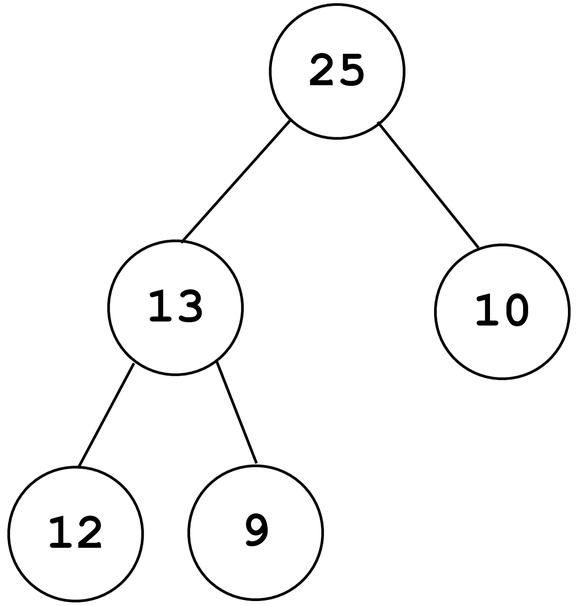
[2011] 11. 已知序列 25, 13, 10, 12, 9 是大根堆， 在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是_____。

A. 1　　B. 2　　C. 4　　D. 5

真题解析

[2011] 11. 已知序列 25, 13, 10, 12, 9 是大根堆， 在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是_____。

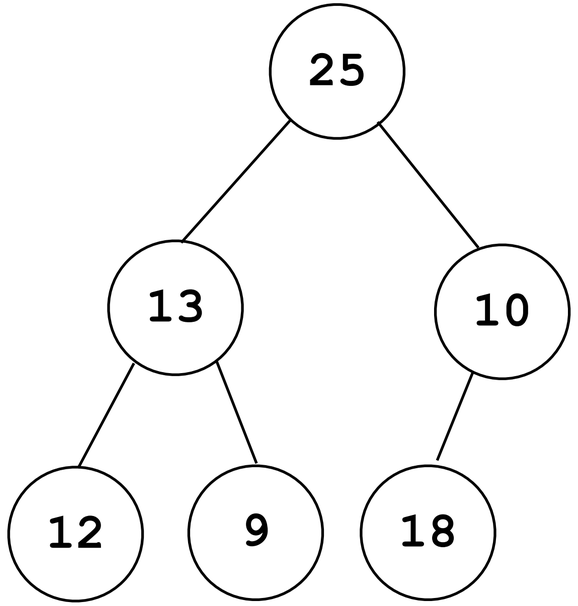
A. 1　　B. 2　　C. 4　　D. 5



真题解析

[2011] 11. 已知序列 25, 13, 10, 12, 9 是大根堆， 在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是_____。

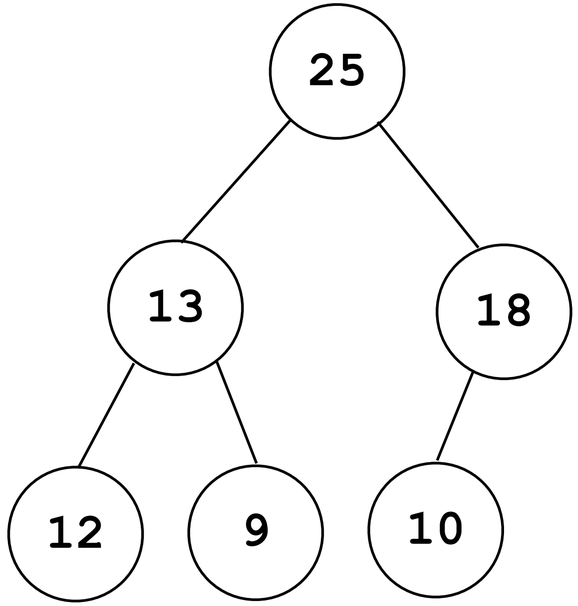
A. 1　　B. 2　　C. 4　　D. 5



真题解析

[2011] 11. 已知序列 25, 13, 10, 12, 9 是大根堆， 在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是_____。

A. 1　　B. 2　　C. 4　　D. 5



真题解析

[2011] 11. 已知序列 25, 13, 10, 12, 9 是大根堆， 在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是_____。

A. 1　　B. 2　　C. 4　　D. 5

真题解析

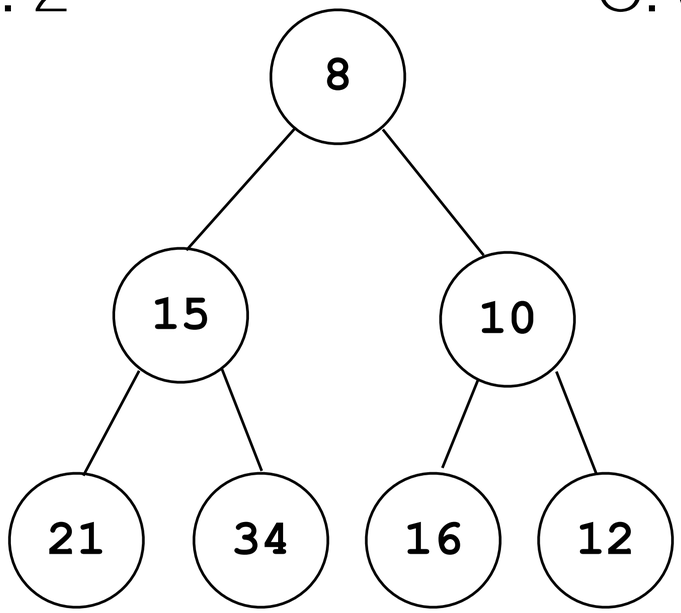
[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

A. 1　　B. 2　　C. 3　　D. 4

真题解析

[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

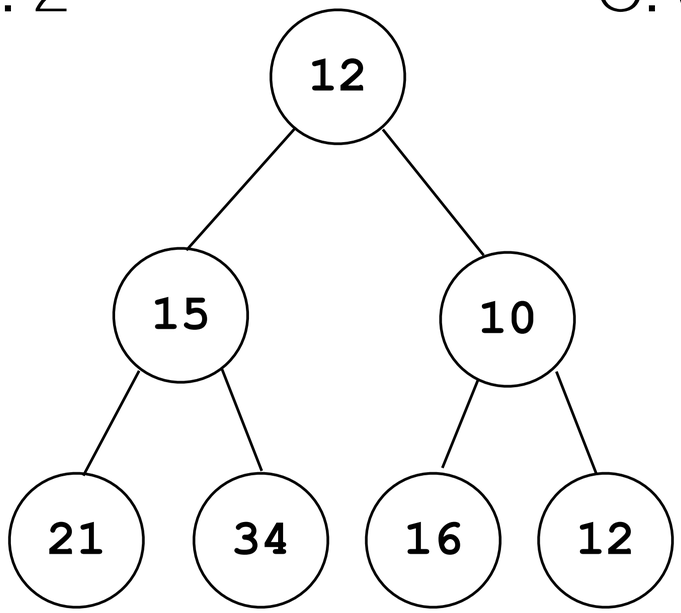
A. 1　　B. 2　　C. 3　　D. 4



真题解析

[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

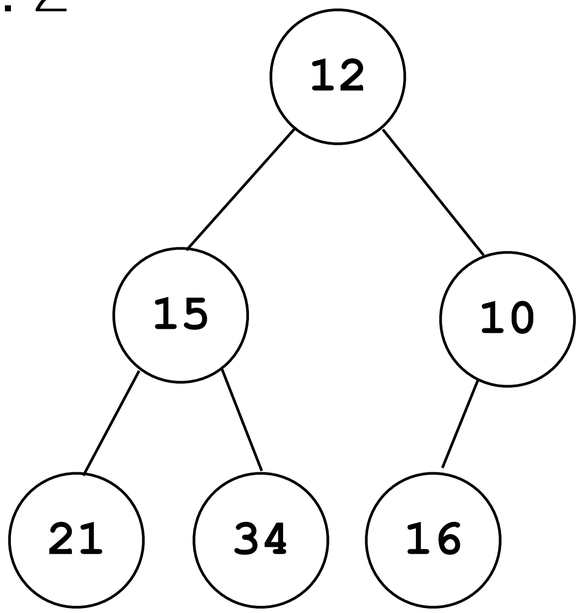
A. 1　　B. 2　　C. 3　　D. 4



真题解析

[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

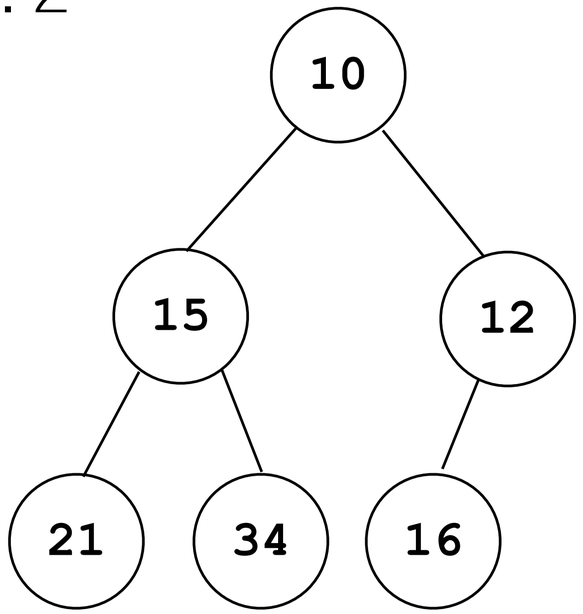
A. 1　　B. 2　　C. 3　　D. 4



真题解析

[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

A. 1　　B. 2　　C. 3　　D. 4



真题解析

[2015] 10. 已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是_____。

A. 1　　B. 2　　C. 3　　D. 4

真题解析

[2018] 10. 在将数据序列（6, 1, 5, 9, 8, 4, 7）建成大根堆时，正确的序列变化过程是_____。

A. 6, 1, 7, 9, 8, 4, 5 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

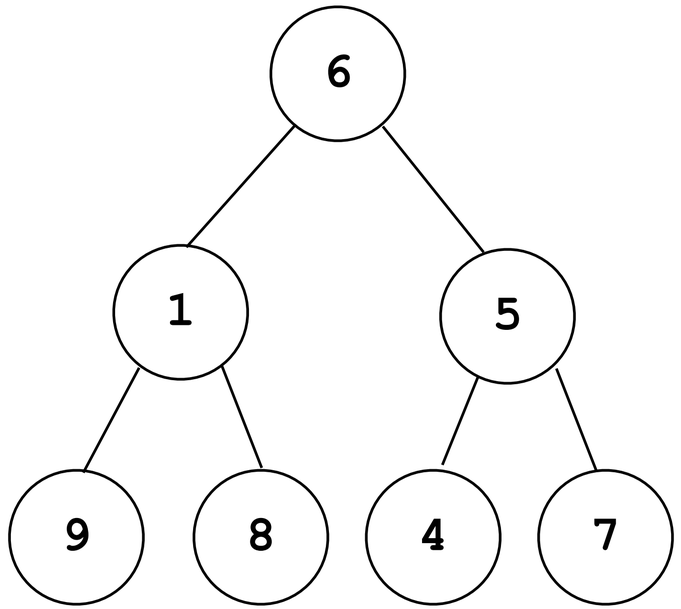
B. 6, 9, 5, 1, 8, 4, 7 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 -> 9, 6, 5, 1, 8, 4, 7 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 -> 7, 1, 6, 9, 8, 4, 5 -> 7, 9, 6, 1, 8, 4, 5 -> 9, 7, 6, 1, 8, 4, 5 -> 9, 8, 6, 1, 7, 4, 5

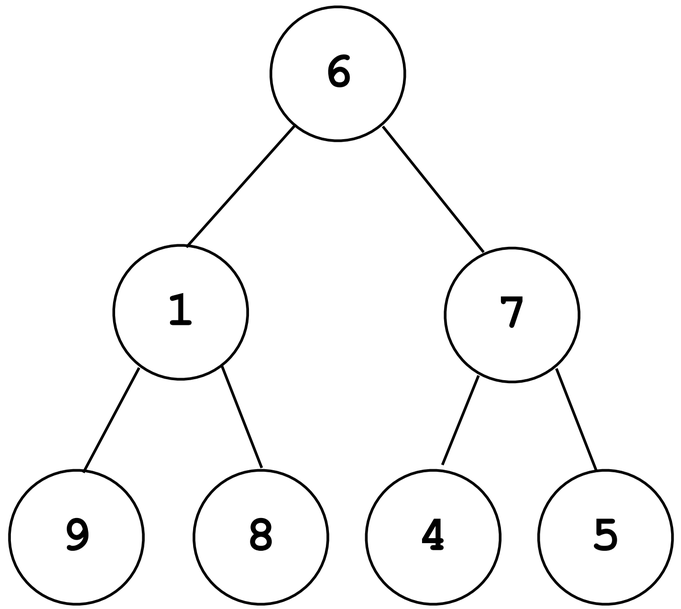
真题解析

[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



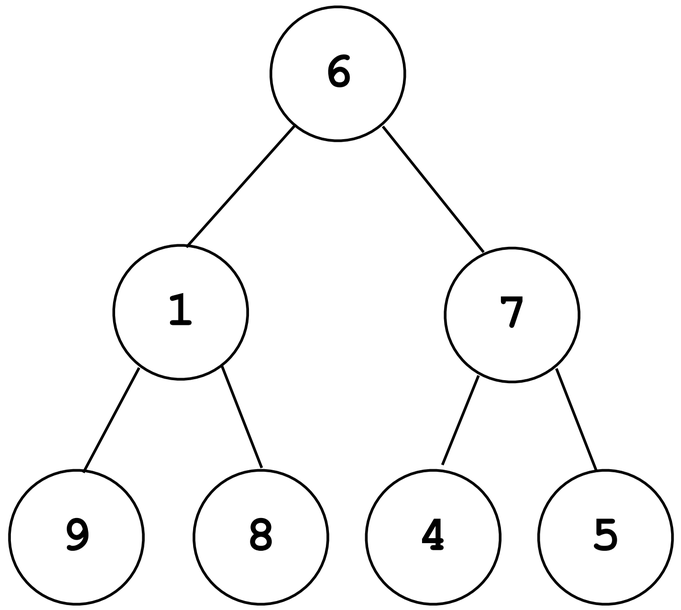
真题解析

[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



真题解析

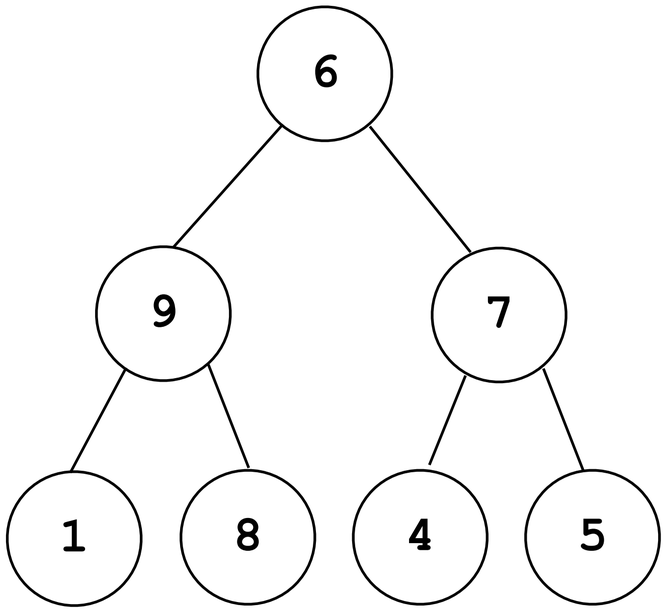
[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5

真题解析

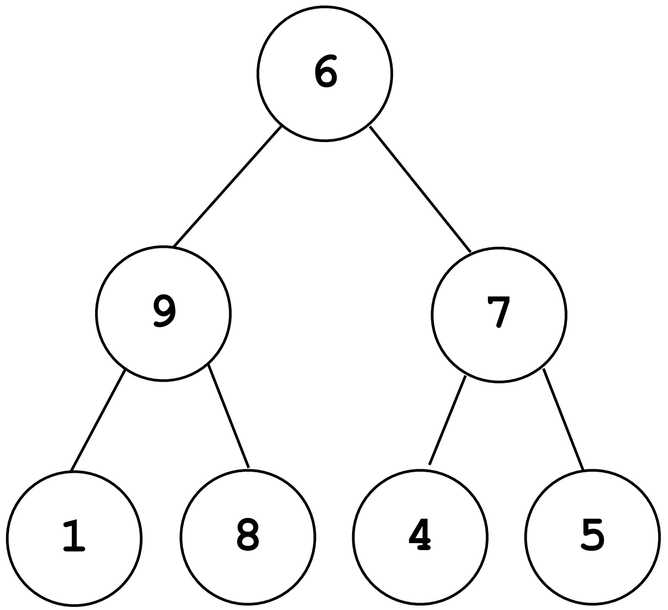
[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5

真题解析

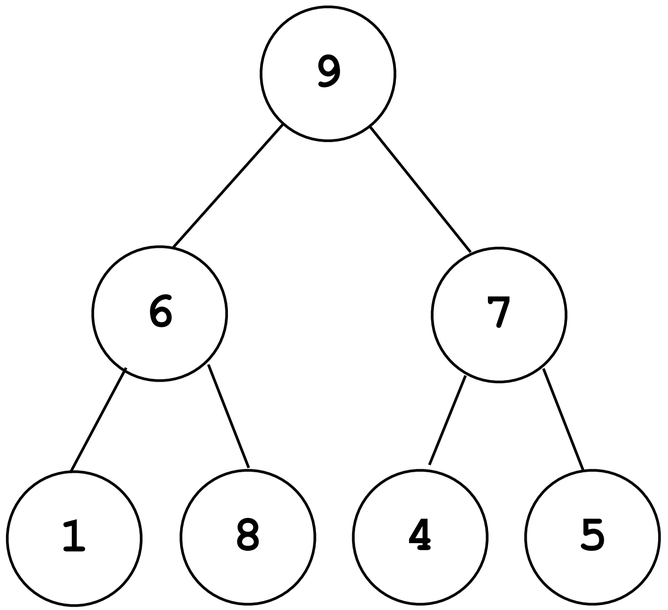
[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5　　6, 9, 7, 1, 8, 4, 5

真题解析

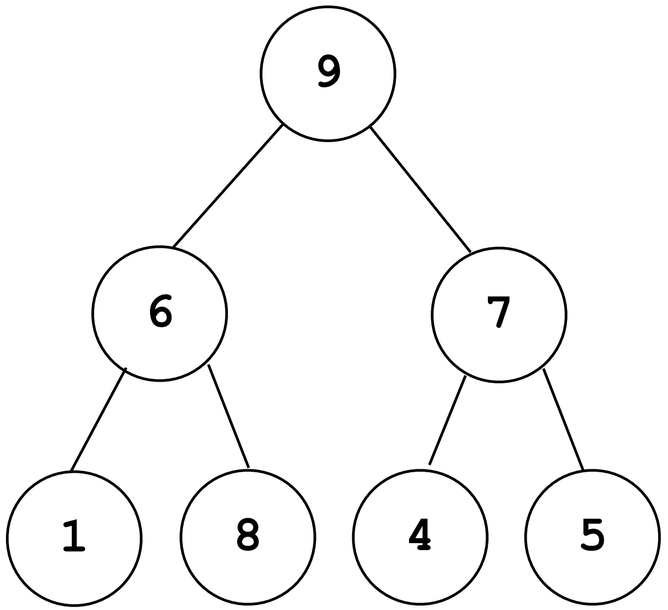
[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5　　6, 9, 7, 1, 8, 4, 5

真题解析

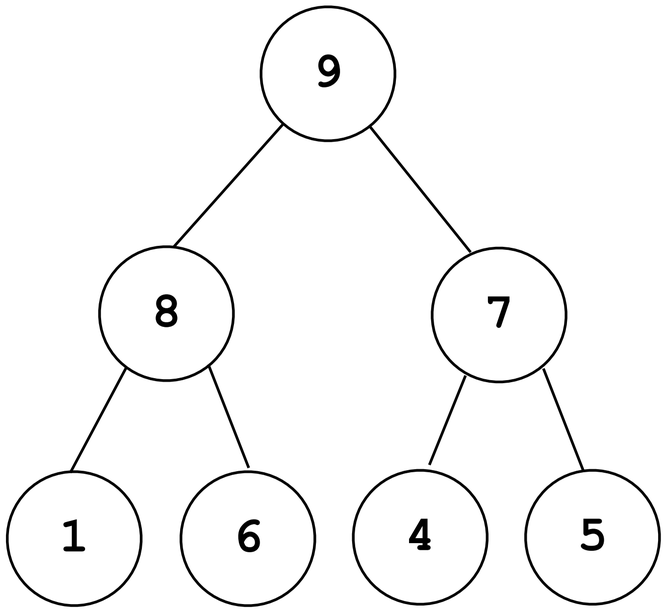
[2018] 10. 在将数据序列（ 6, 1, 5, 9, 8, 4, 7 ）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5　　6, 9, 7, 1, 8, 4, 5　　9, 6, 7, 1, 8, 4, 5

真题解析

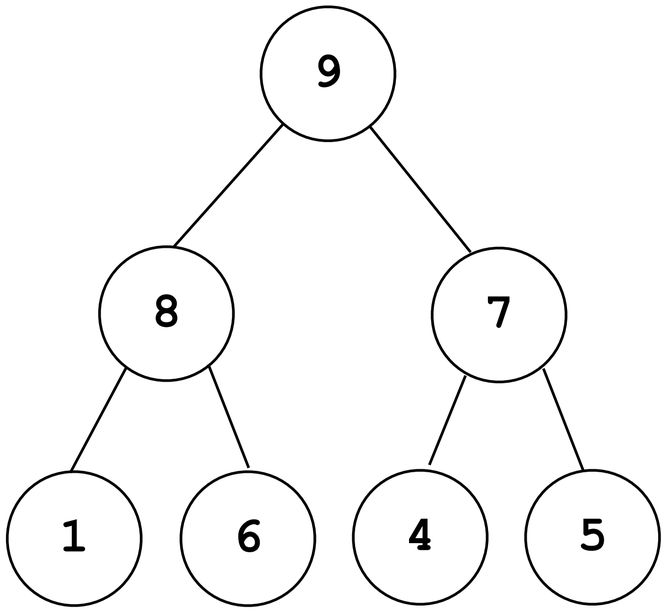
[2018] 10. 在将数据序列（6, 1, 5, 9, 8, 4, 7）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5　　6, 9, 7, 1, 8, 4, 5　　9, 6, 7, 1, 8, 4, 5

真题解析

[2018] 10. 在将数据序列（6, 1, 5, 9, 8, 4, 7）建成大根堆时，正确的序列变化过程是_____。



6, 1, 7, 9, 8, 4, 5　　6, 9, 7, 1, 8, 4, 5　　9, 6, 7, 1, 8, 4, 5　　9, 8, 7, 1, 6, 4, 5

真题解析

[2018] 10. 在将数据序列（6, 1, 5, 9, 8, 4, 7）建成大根堆时，正确的序列变化过程是_____。

A. 6, 1, 7, 9, 8, 4, 5 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

B. 6, 9, 5, 1, 8, 4, 7 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 -> 9, 6, 5, 1, 8, 4, 7 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 -> 7, 1, 6, 9, 8, 4, 5 -> 7, 9, 6, 1, 8, 4, 5 -> 9, 7, 6, 1, 8, 4, 5 -> 9, 8, 6, 1, 7, 4, 5

6, 1, 7, 9, 8, 4, 5　6, 9, 7, 1, 8, 4, 5　9, 6, 7, 1, 8, 4, 5　9, 8, 7, 1, 6, 4, 5

真题解析

[2018] 10. 在将数据序列（6, 1, 5, 9, 8, 4, 7）建成大根堆时，正确的序列变化过程是_____。

A. 6, 1, 7, 9, 8, 4, 5 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

B. 6, 9, 5, 1, 8, 4, 7 -> 6, 9, 7, 1, 8, 4, 5 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 -> 9, 6, 5, 1, 8, 4, 7 -> 9, 6, 7, 1, 8, 4, 5 -> 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 -> 7, 1, 6, 9, 8, 4, 5 -> 7, 9, 6, 1, 8, 4, 5 -> 9, 7, 6, 1, 8, 4, 5 -> 9, 8, 6, 1, 7, 4, 5

真题解析

[2020] 9. 下列关于大根堆（至少含 2 个元素）的叙述中，正确的是_____。

I. 可以将堆视为一棵完全二叉树

II. 可以采用顺序存储方式保存堆

III. 可以将堆视为一棵二叉排序树

IV. 堆中的次大值一定在根的下一层

A. 仅 I、II　　B. 仅 II、III　　C. 仅 I、II、IV　　C. III 和 IV

真题解析

[2020] 9. 下列关于大根堆（至少含 2 个元素）的叙述中，正确的是_____。

I. 可以将堆视为一棵完全二叉树

II. 可以采用顺序存储方式保存堆

III. 可以将堆视为一棵二叉排序树

IV. 堆中的次大值一定在根的下一层

A. 仅 I、II　　B. 仅 II、III　　C. 仅 I、II、IV　　C. III 和 IV

真题解析

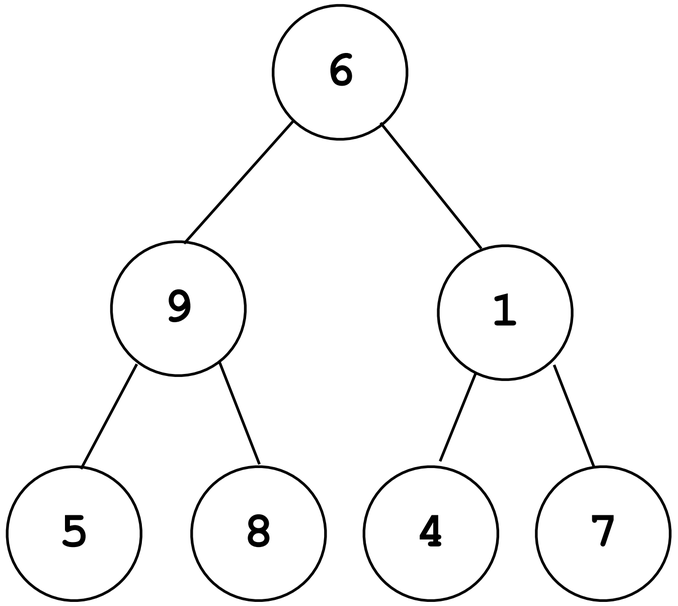
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1

真题解析

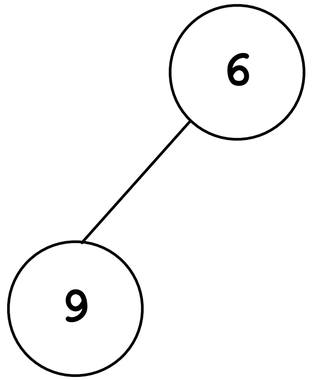
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

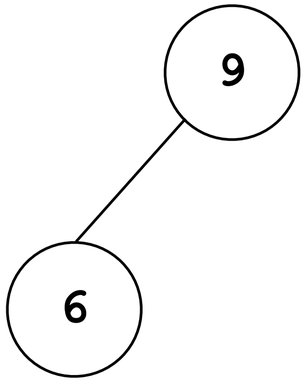
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



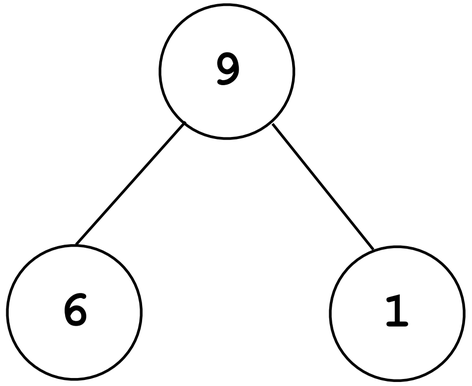
真题解析

[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1



D. 9, 6, 7, 5, 8, 4, 1

真题解析

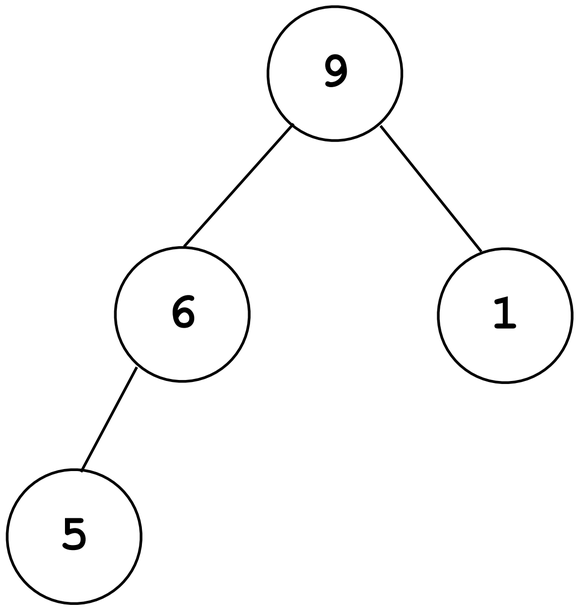
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

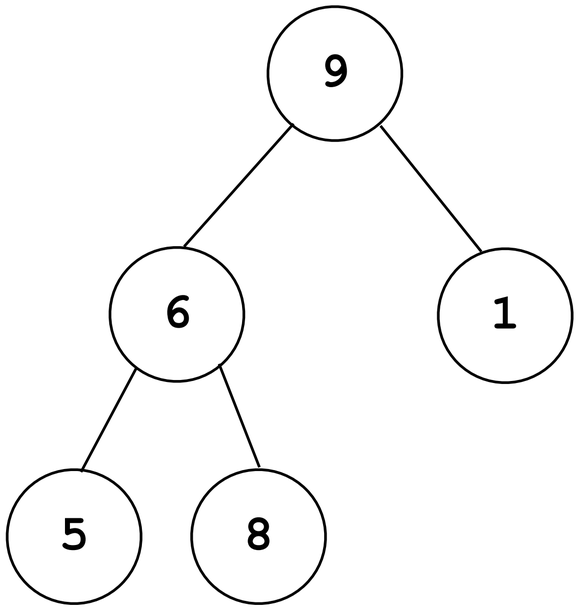
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

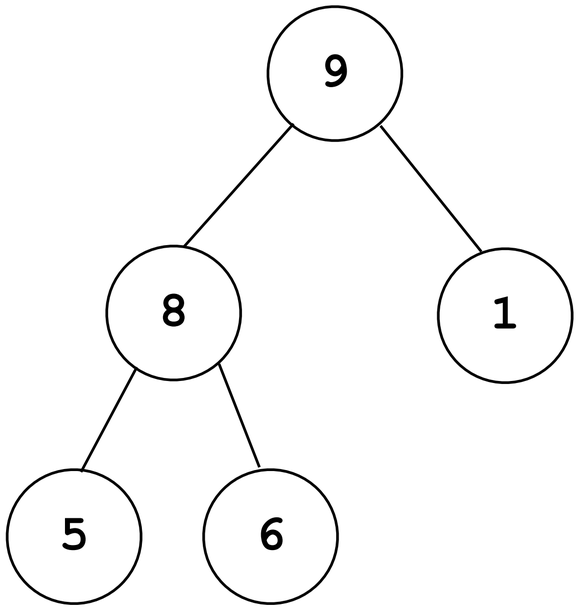
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

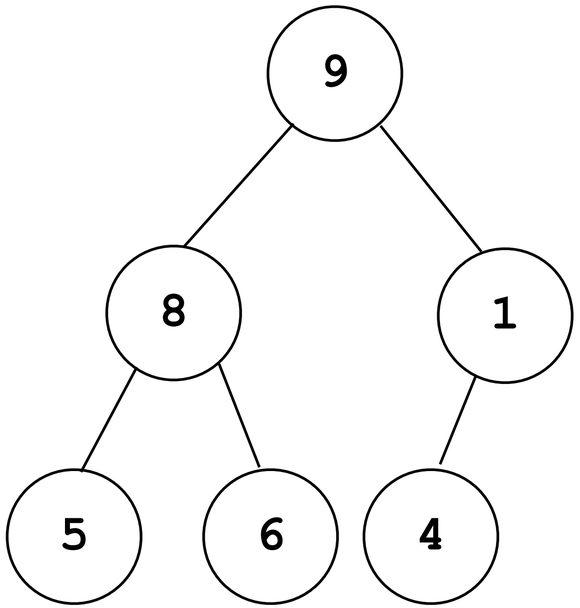
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

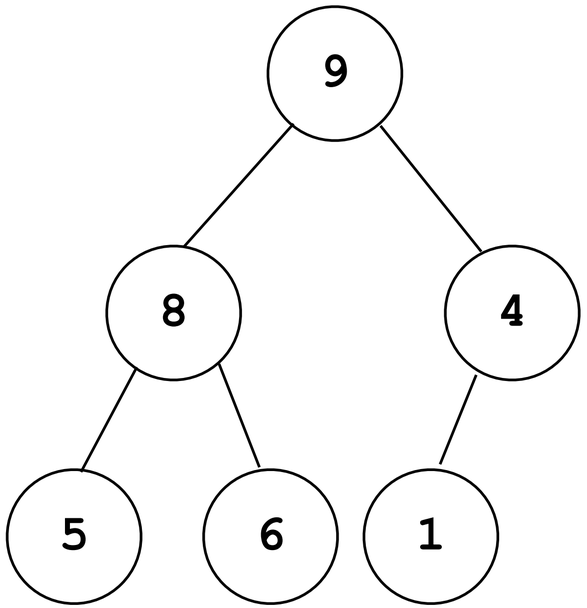
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

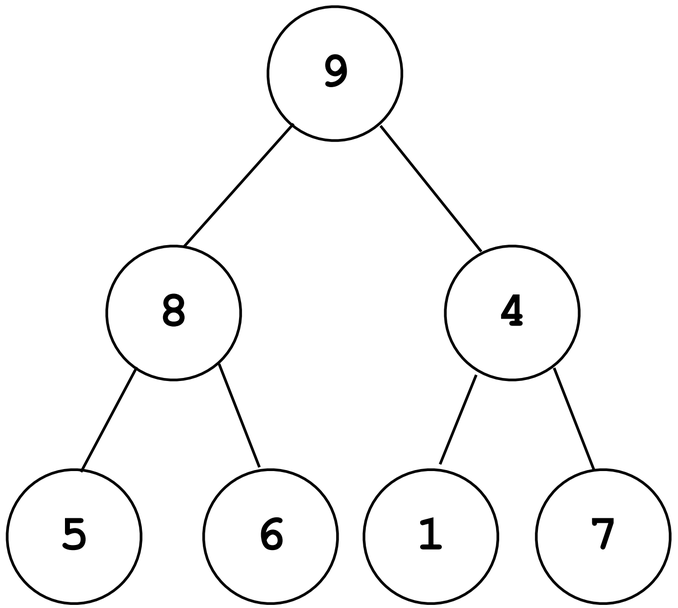
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

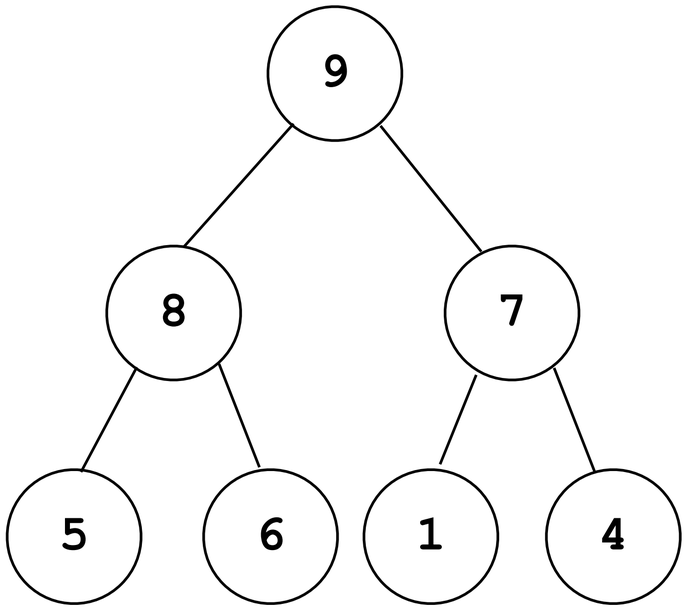
[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1



真题解析

[2021] 11. 将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是_____。

A. 9, 8, 7, 6, 5, 4, 1

B. 9, 8, 7, 5, 6, 1, 4

C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1

高效的排序算法

希尔排序

归并排序

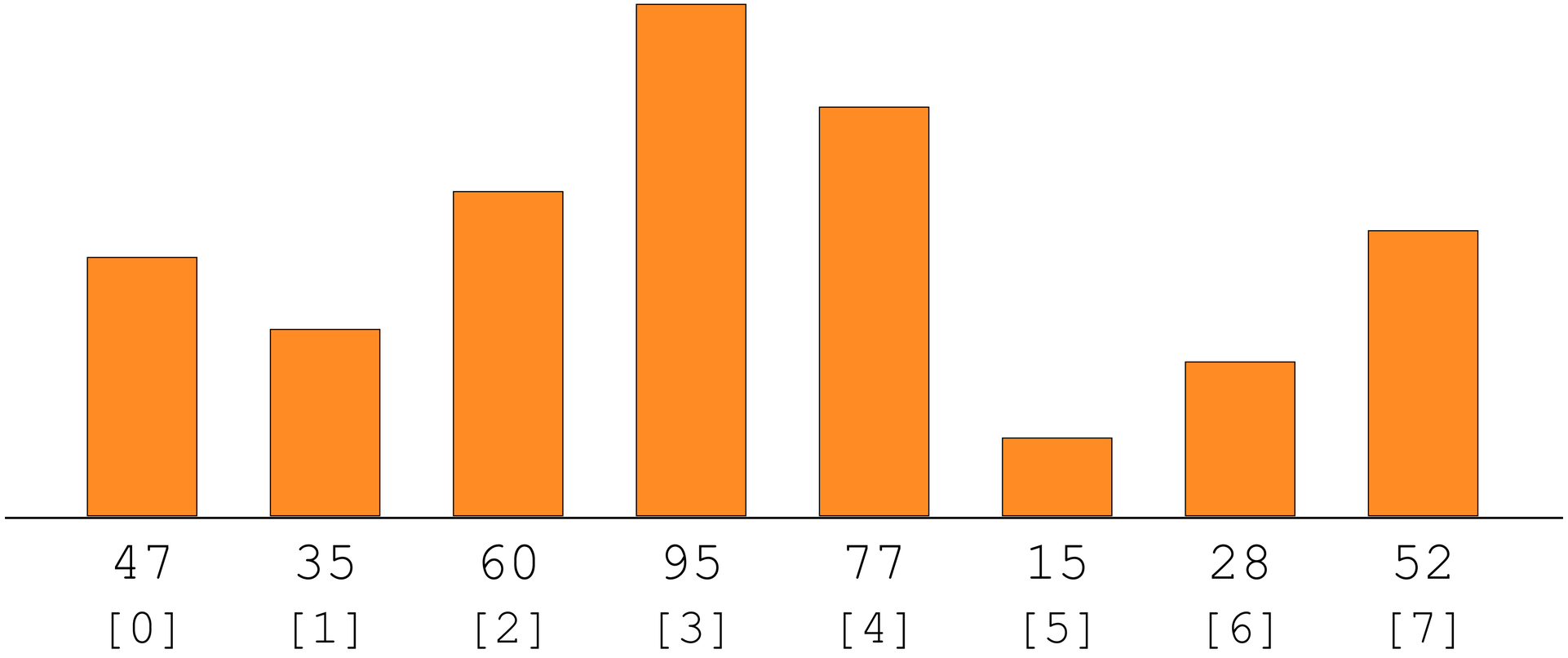
快速排序

基数排序

希尔排序

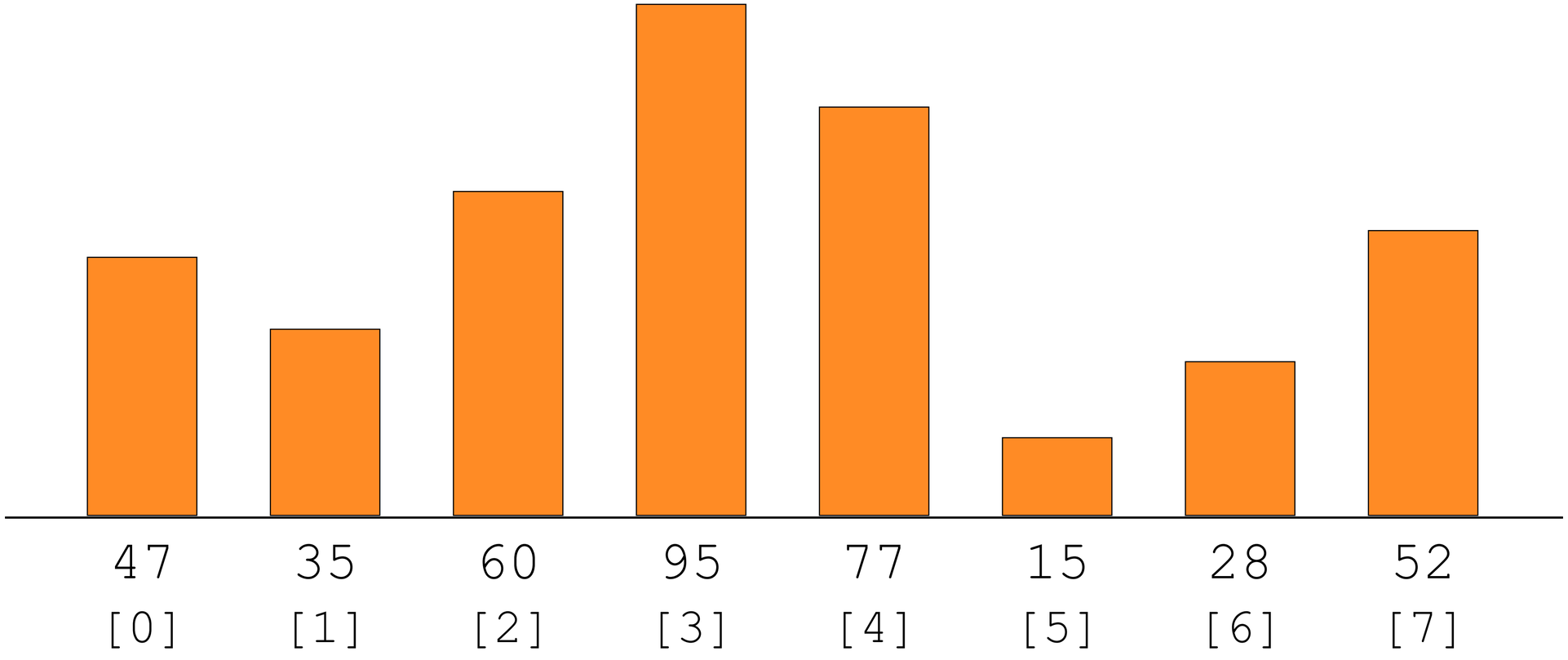
希尔排序的灵感就来自于，能不能先让数据“基本有序”，再进行排序。它的核心思想是：先按一定“步长”分组进行排序，逐步减小步长，最终步长为 1 时完成最终的排序。

希尔排序



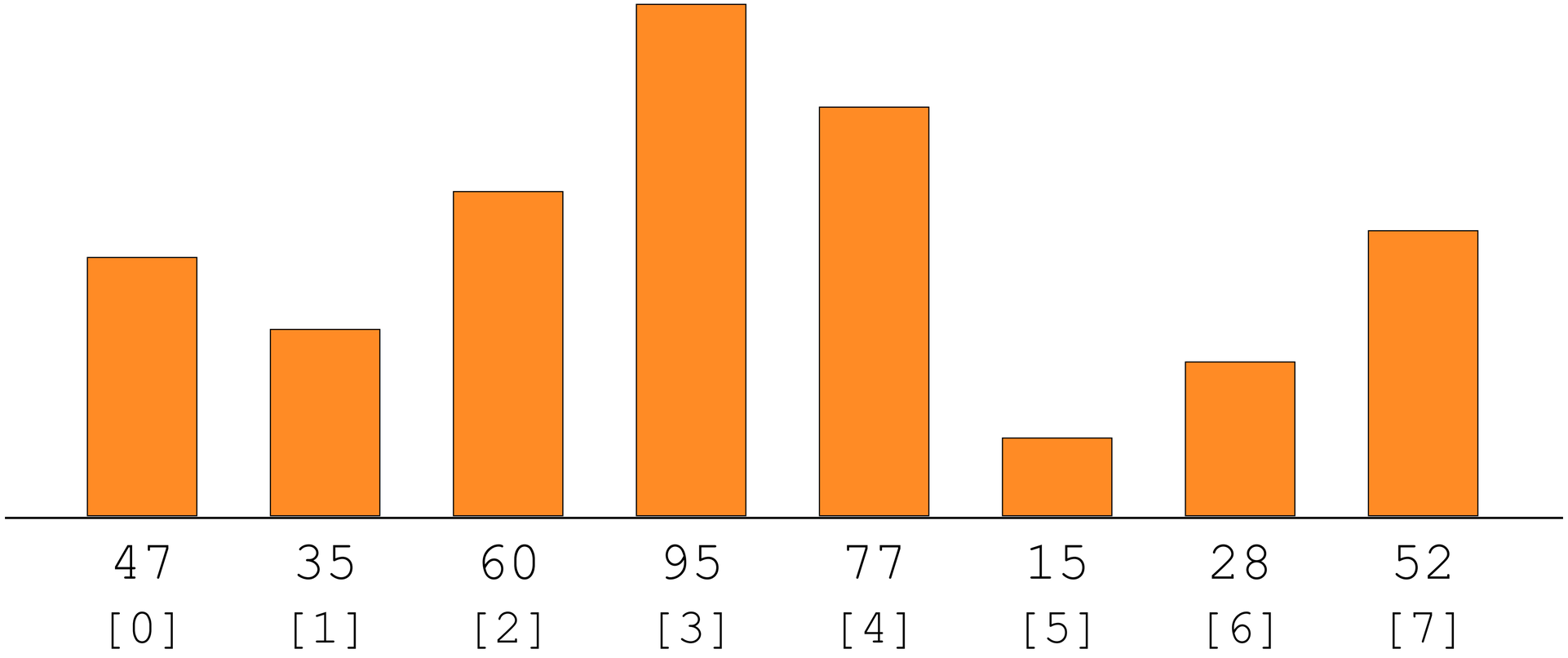
希尔排序

step



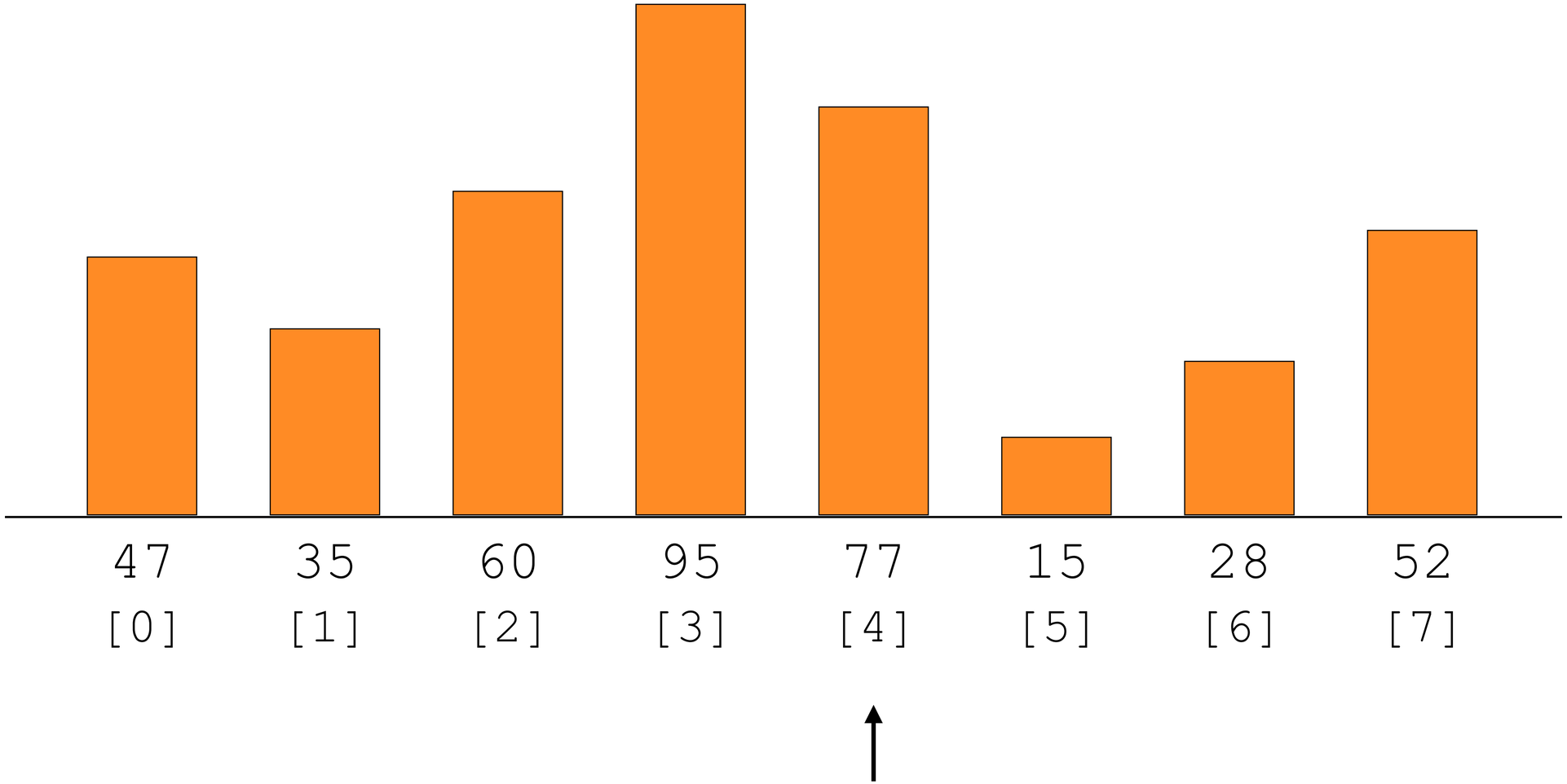
希尔排序

step 4



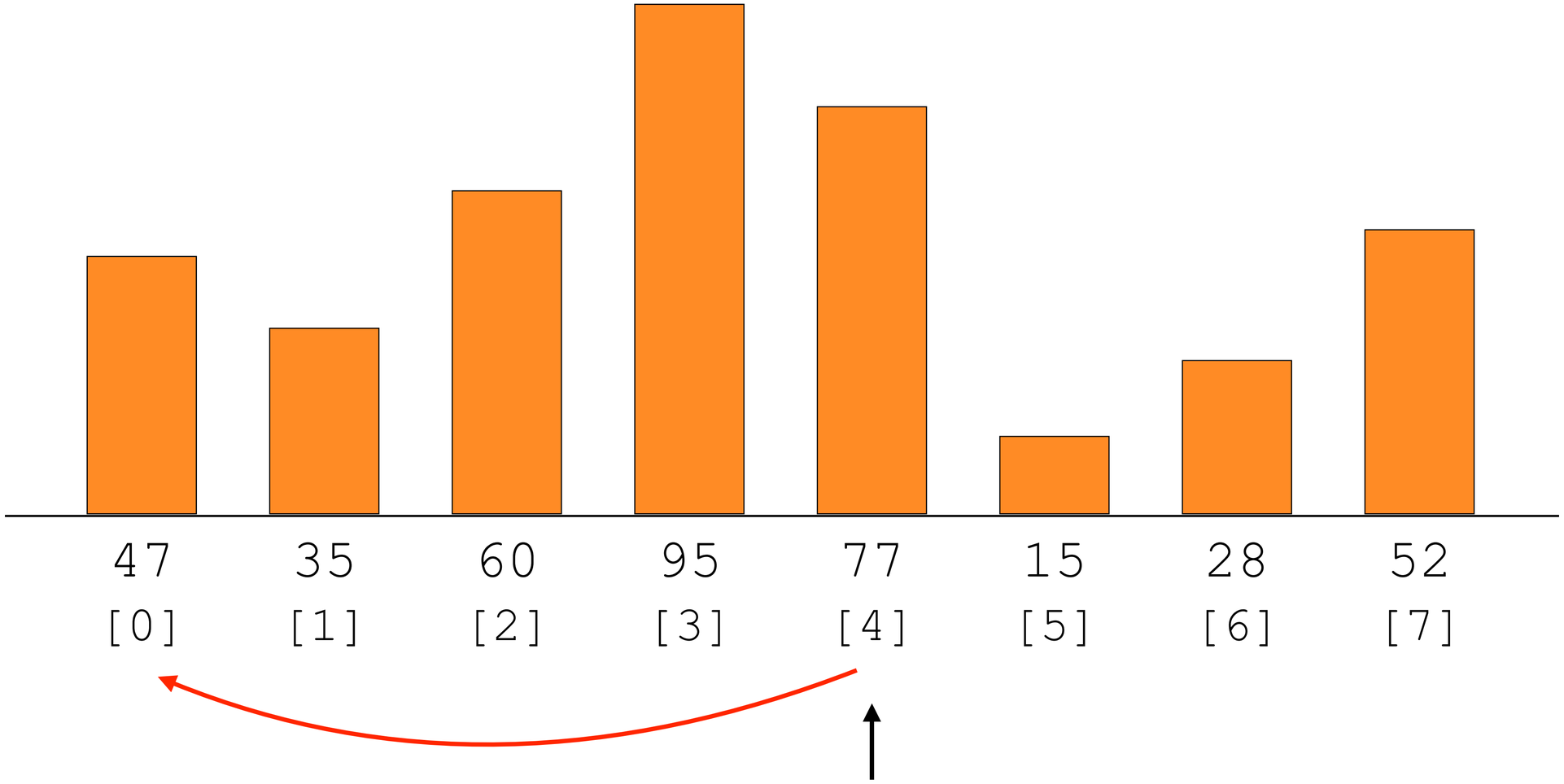
希尔排序

step 4



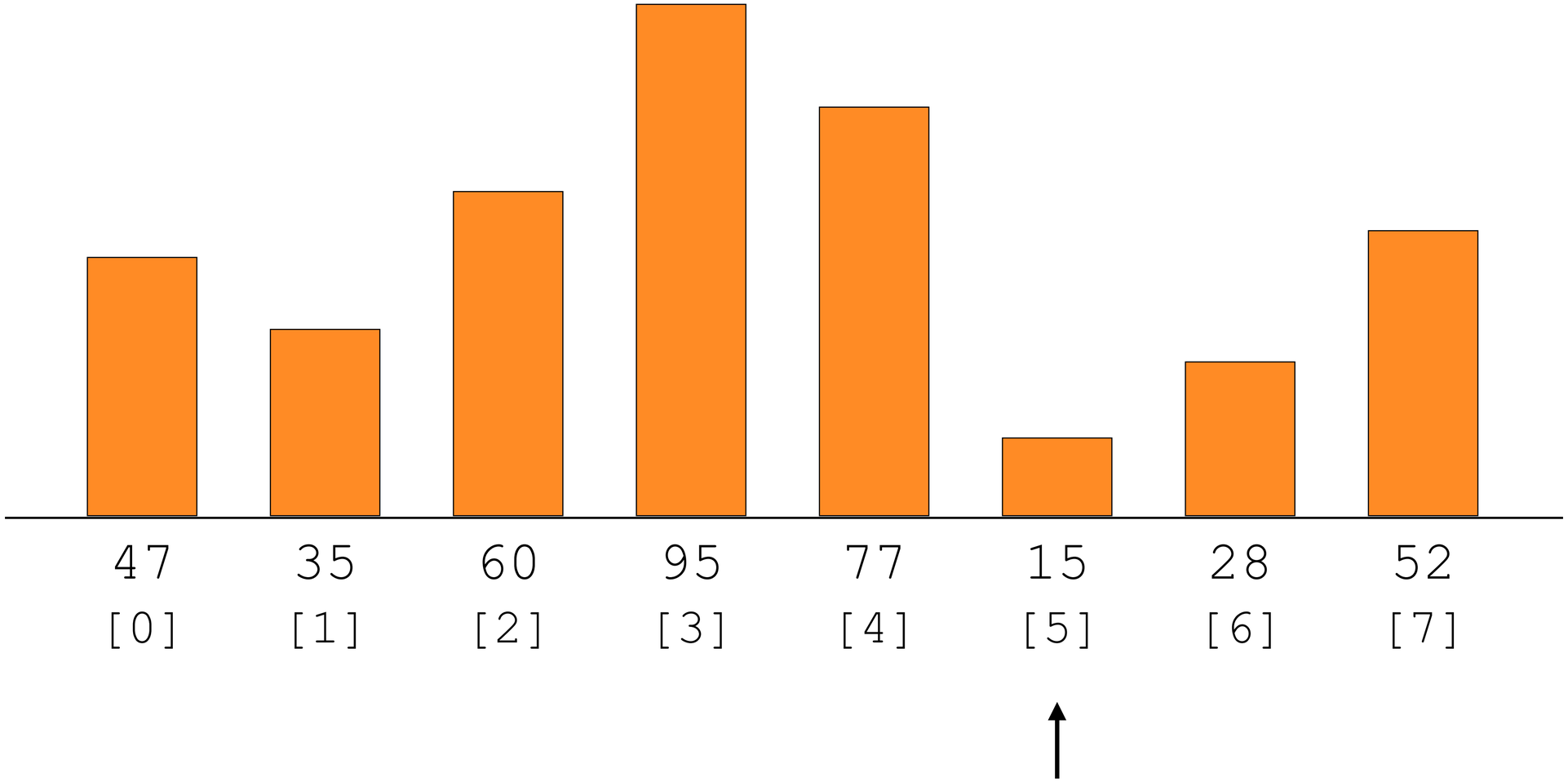
希尔排序

step 4



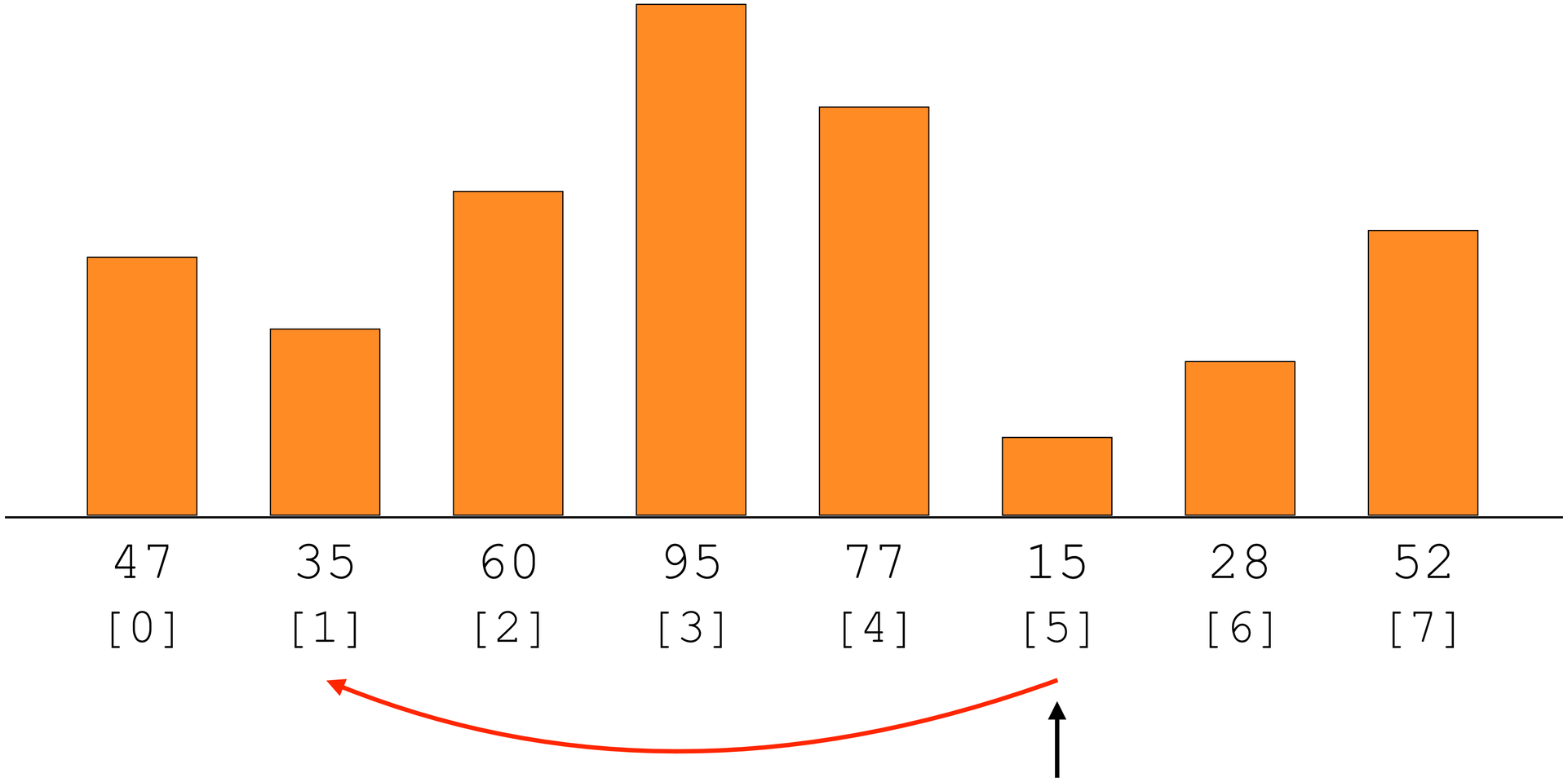
希尔排序

step 4



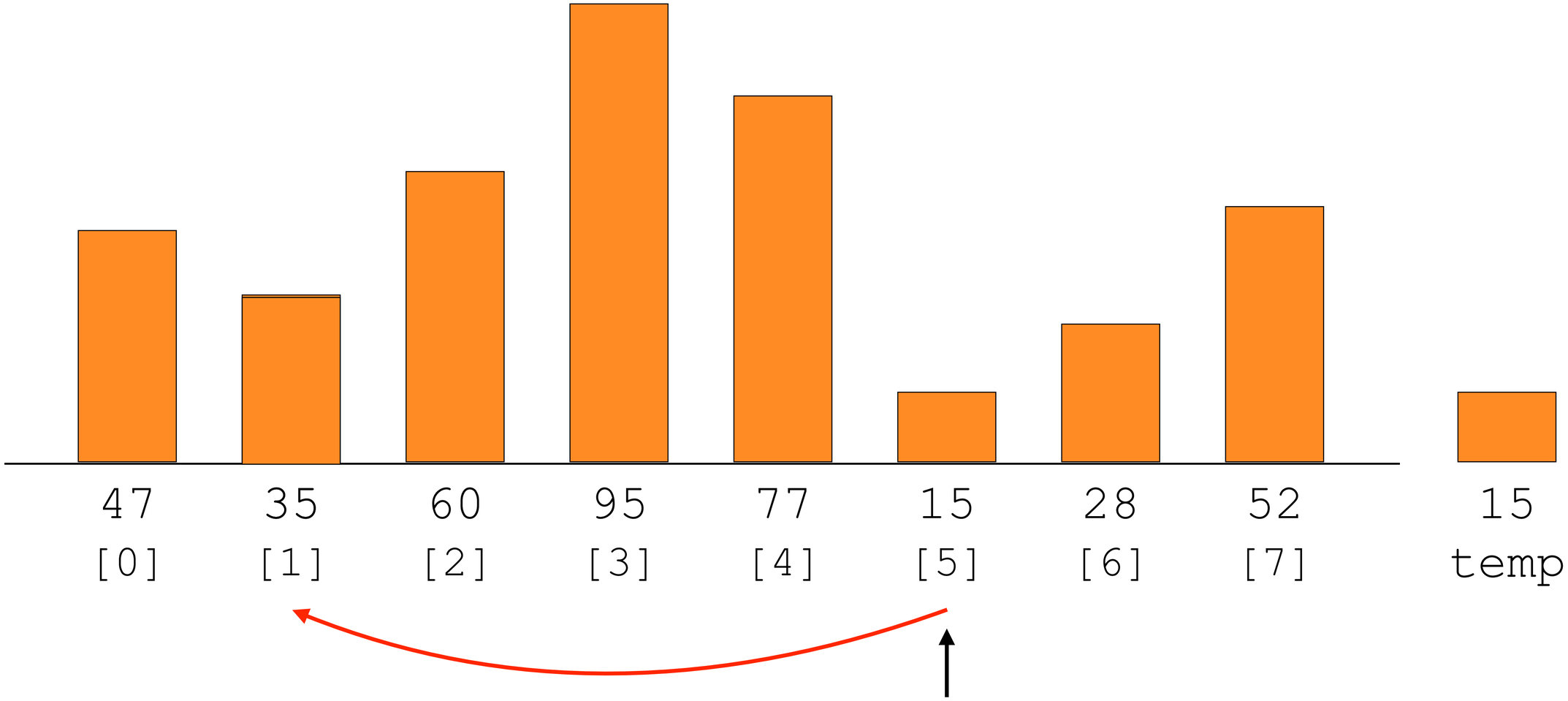
希尔排序

step 4



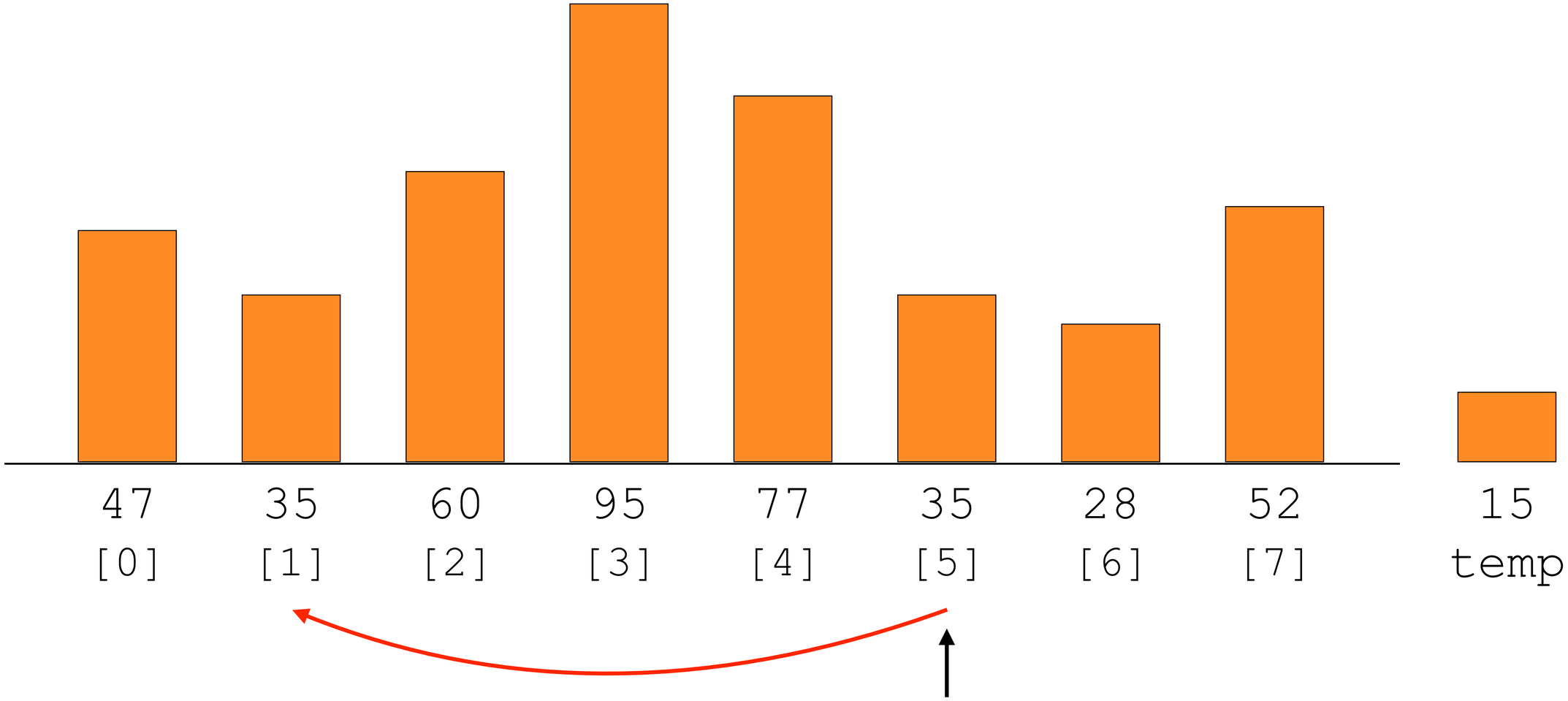
希尔排序

step 4



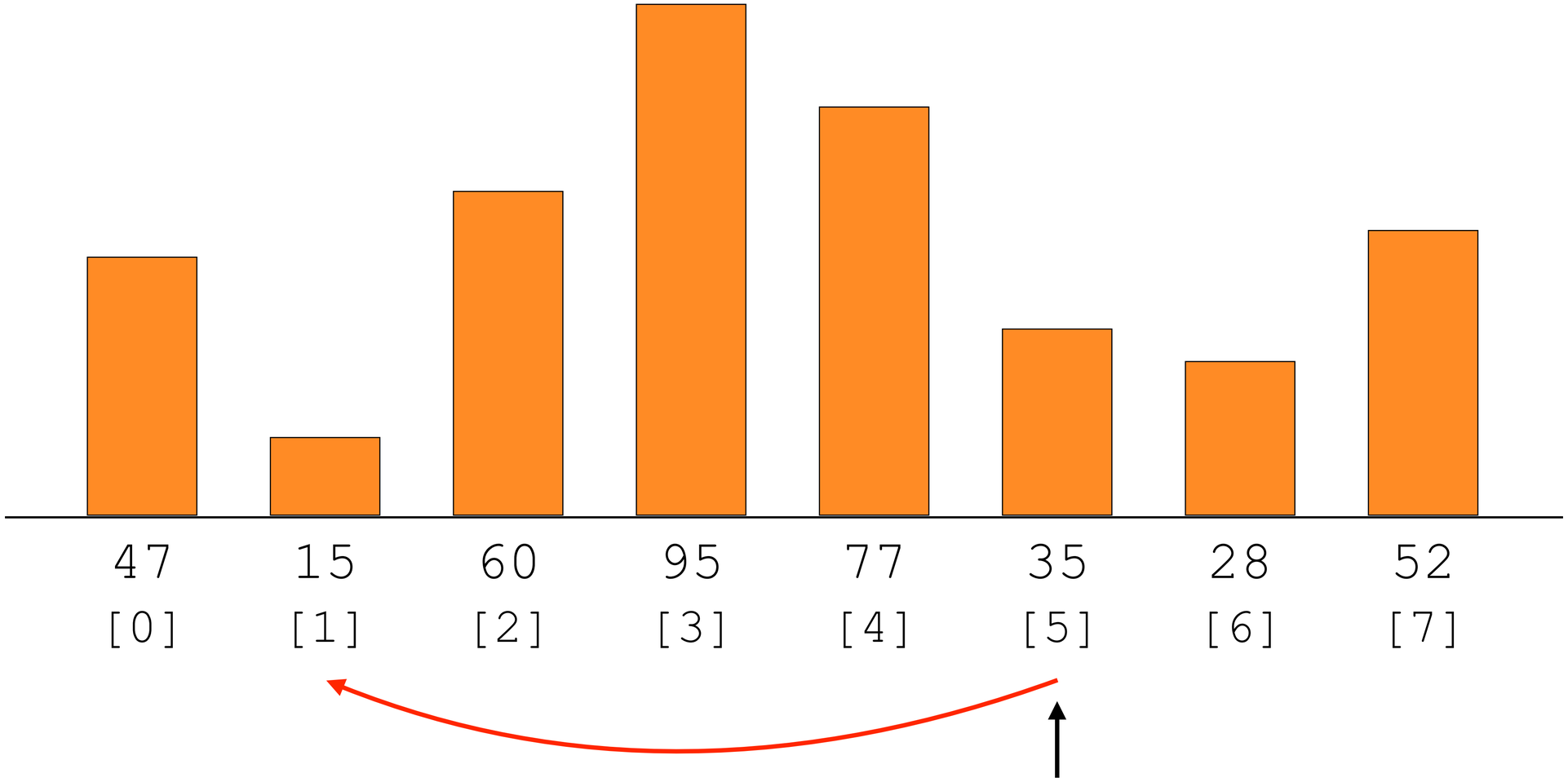
希尔排序

step 4



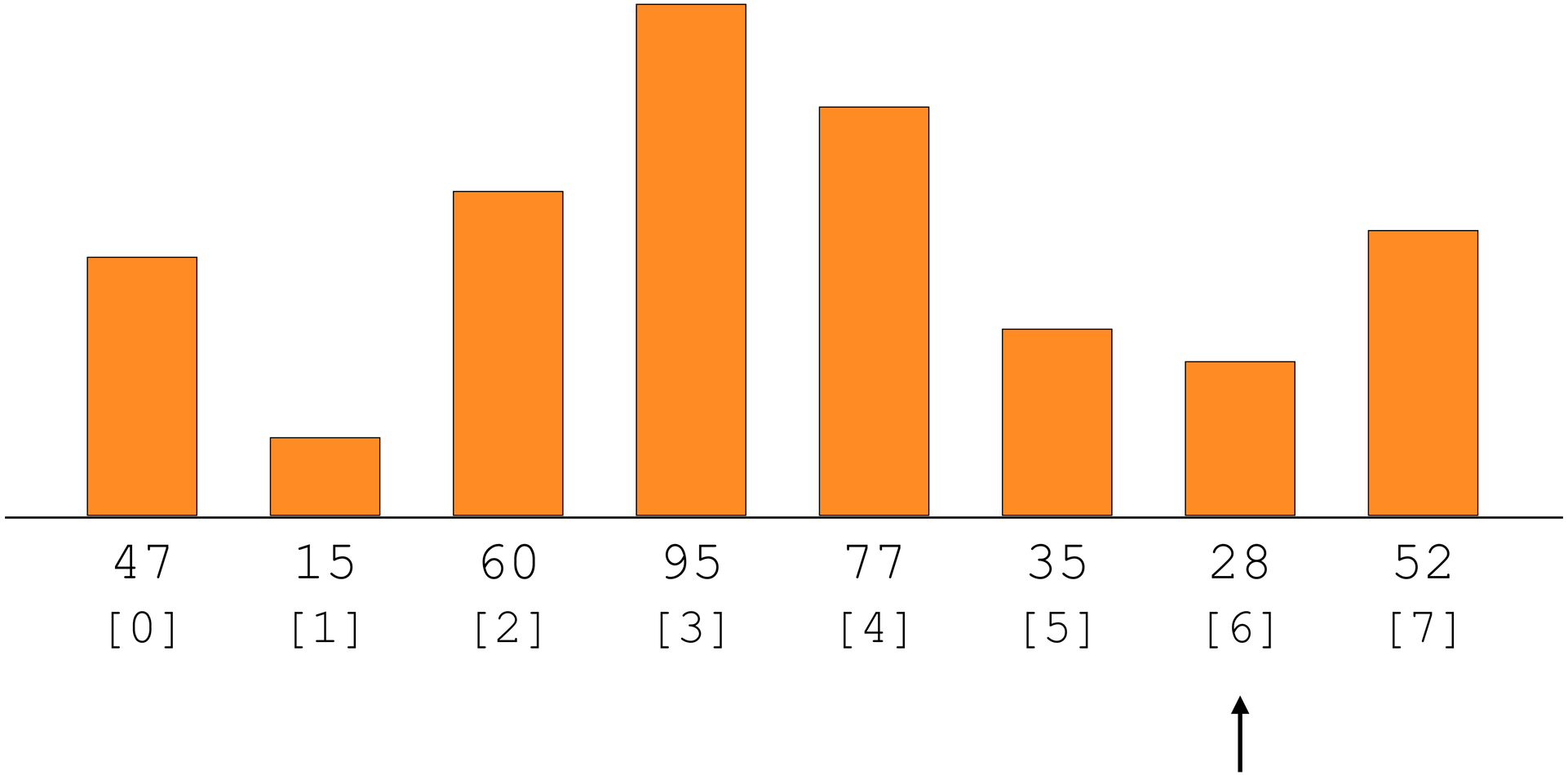
希尔排序

step 4



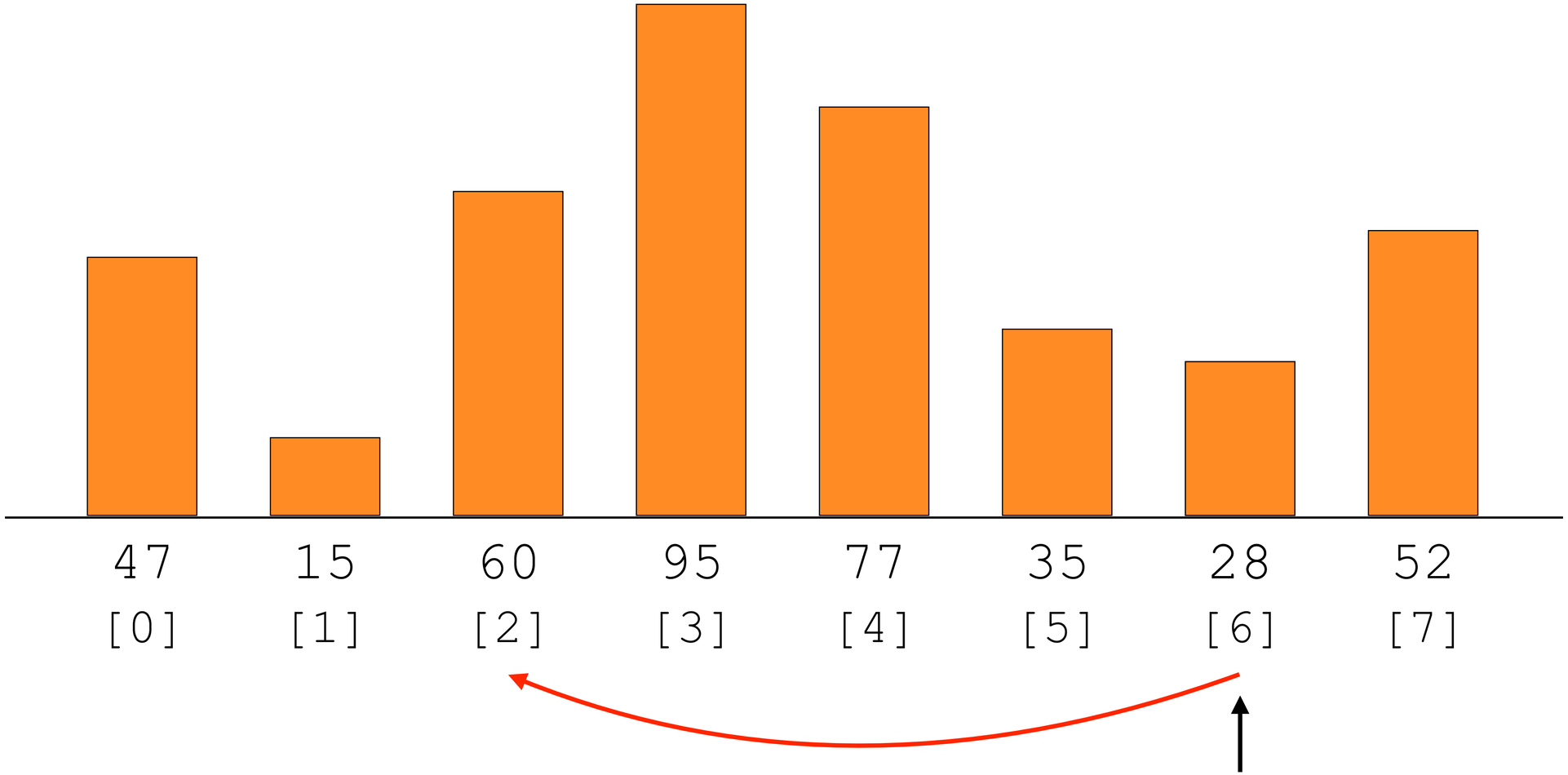
希尔排序

step 4



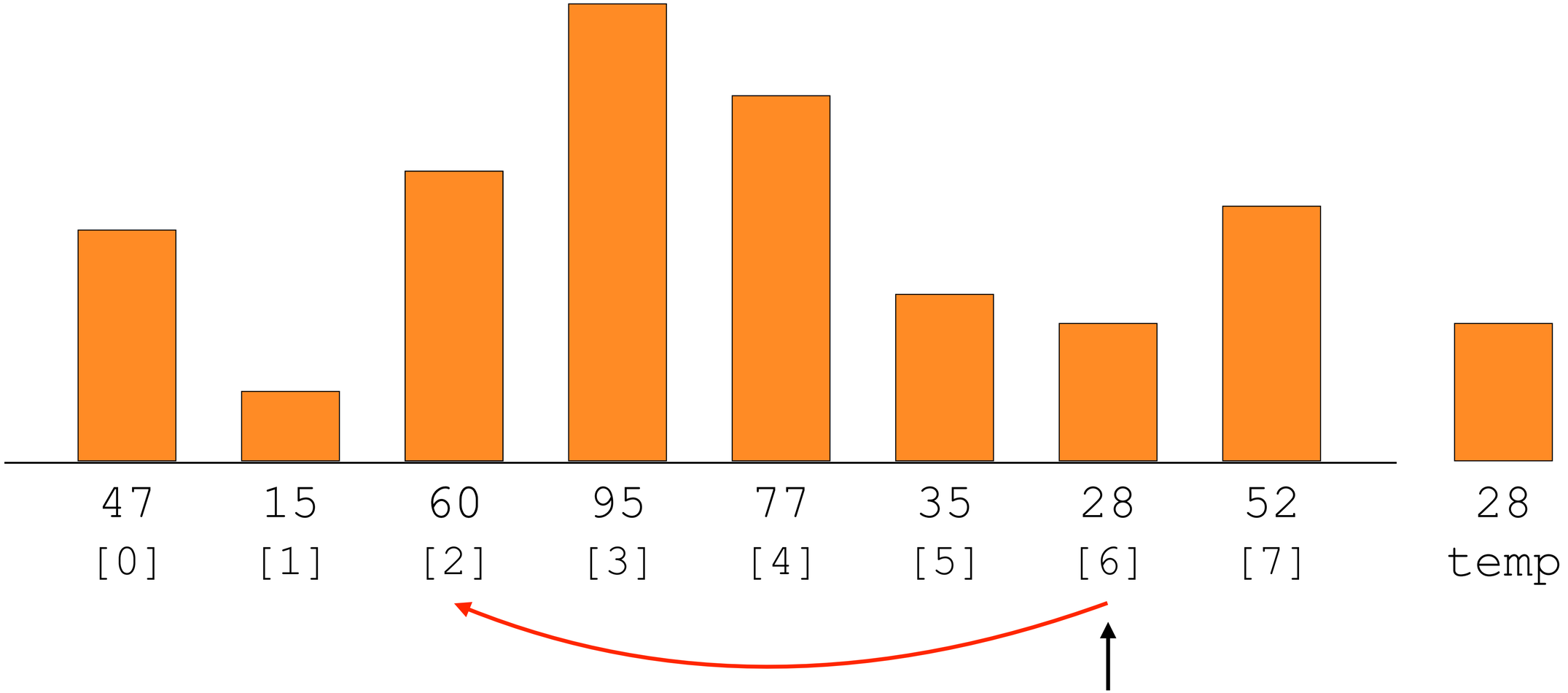
希尔排序

step 4



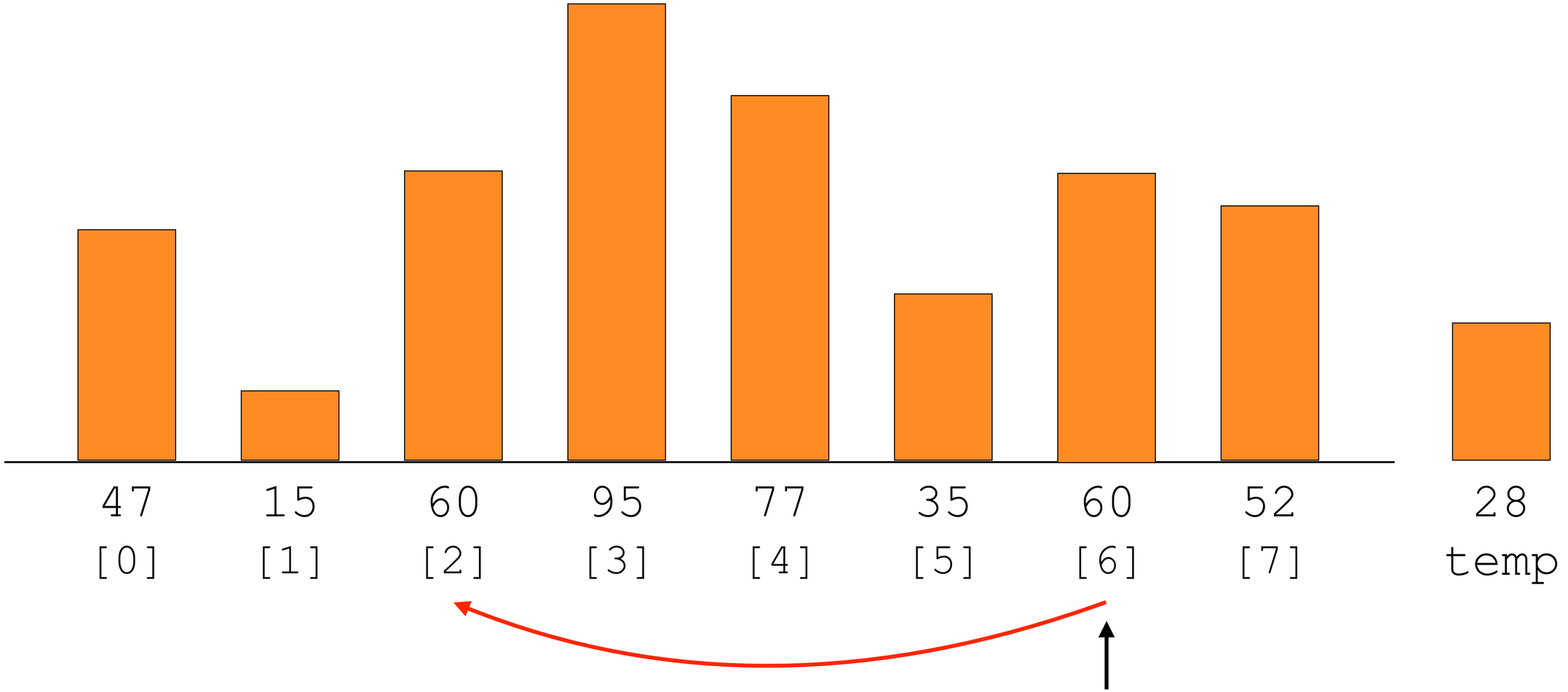
希尔排序

step 4



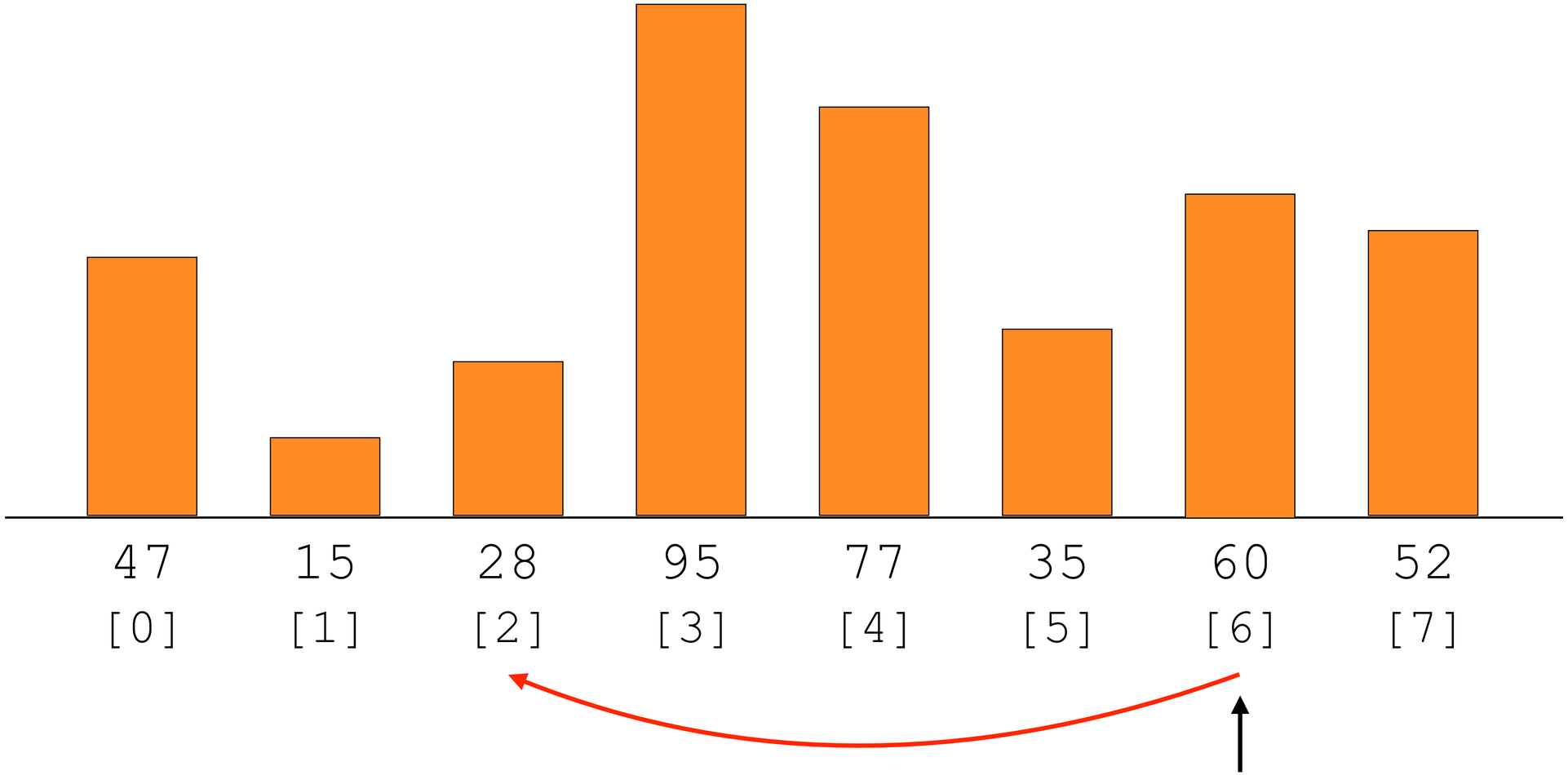
希尔排序

step 4



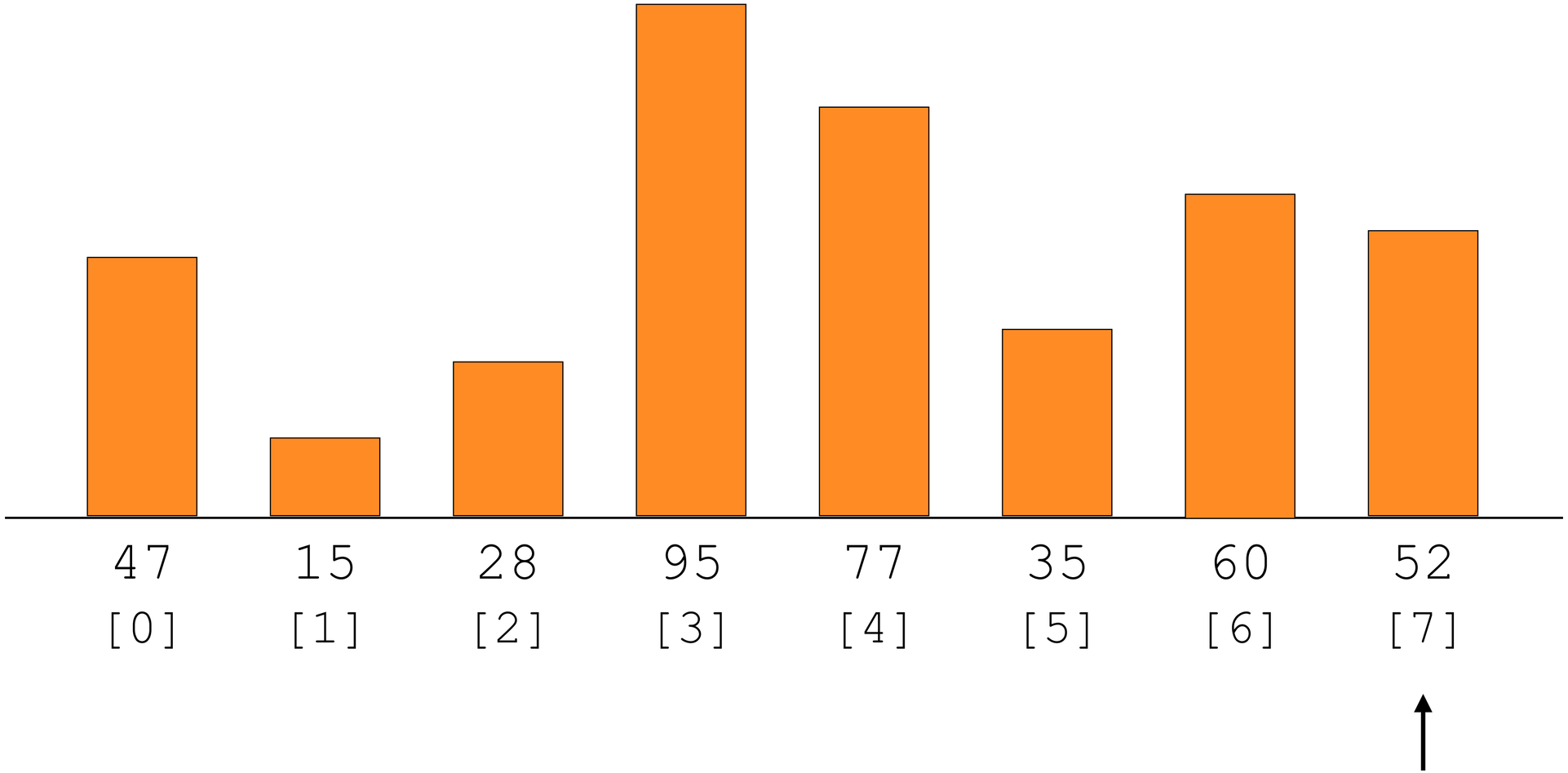
希尔排序

step 4



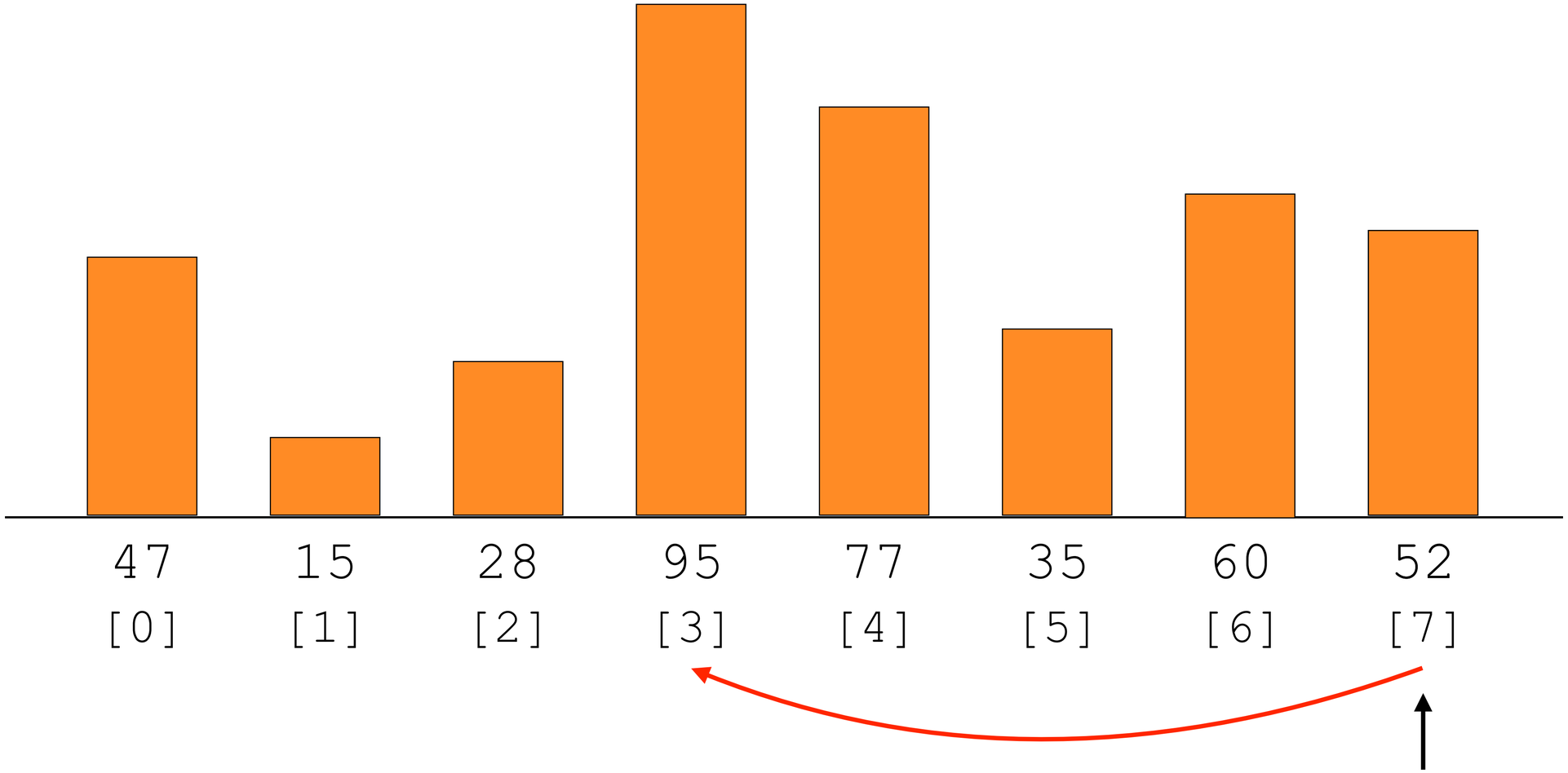
希尔排序

step 4



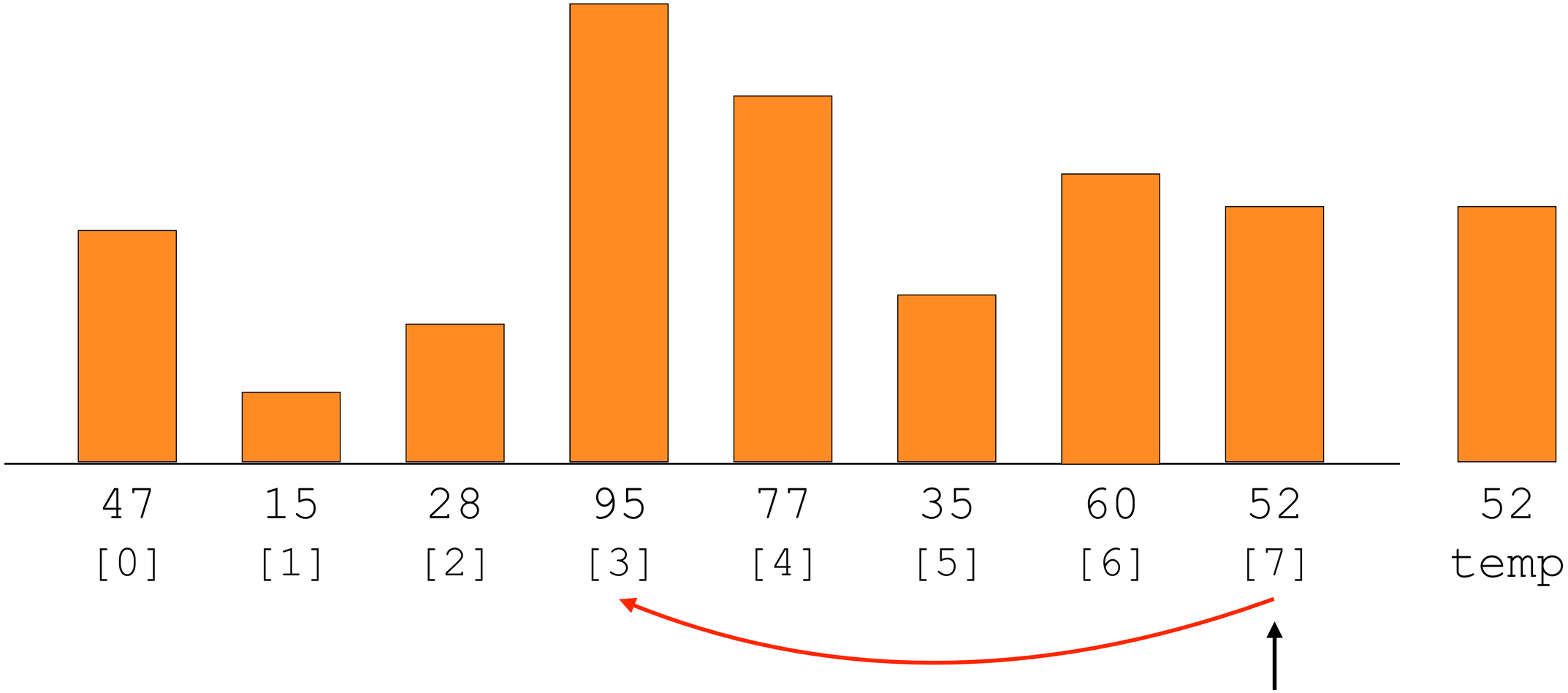
希尔排序

step 4



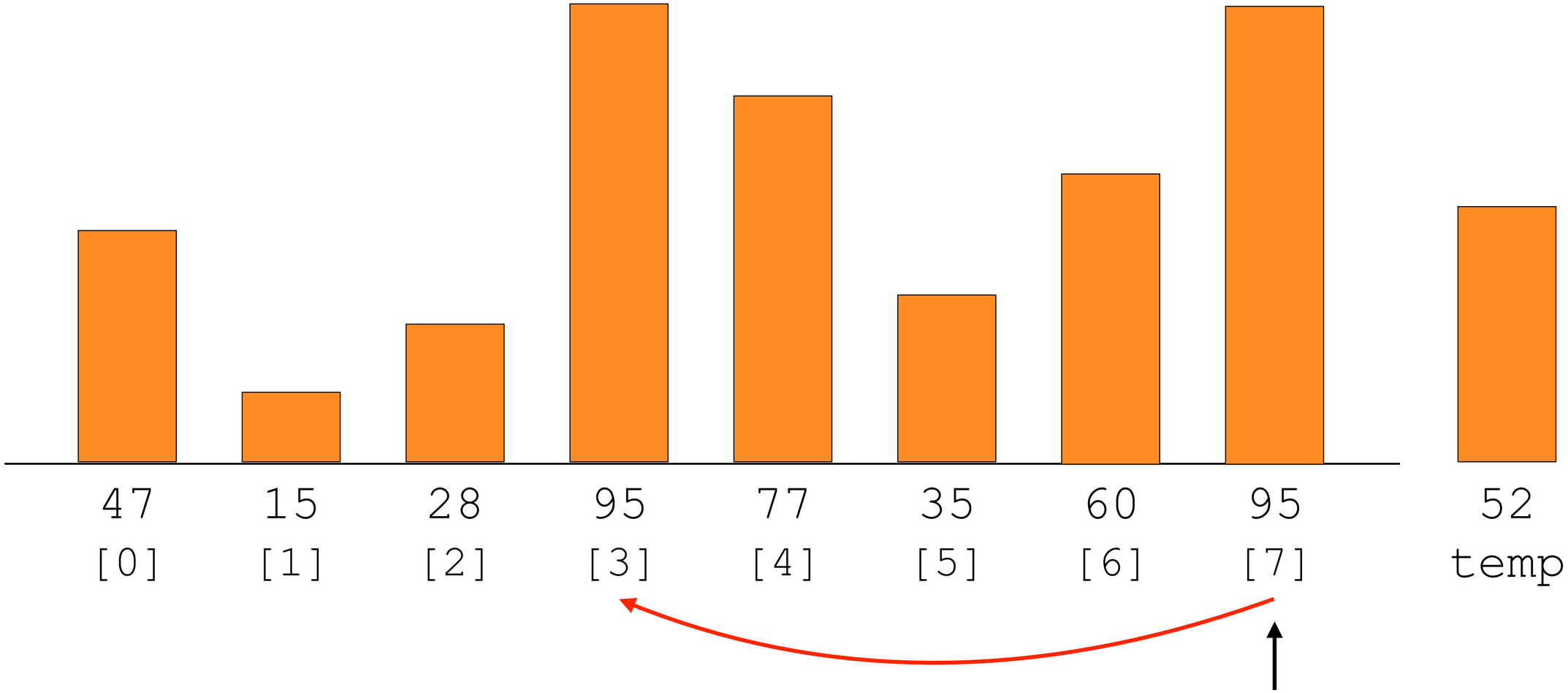
希尔排序

step 4



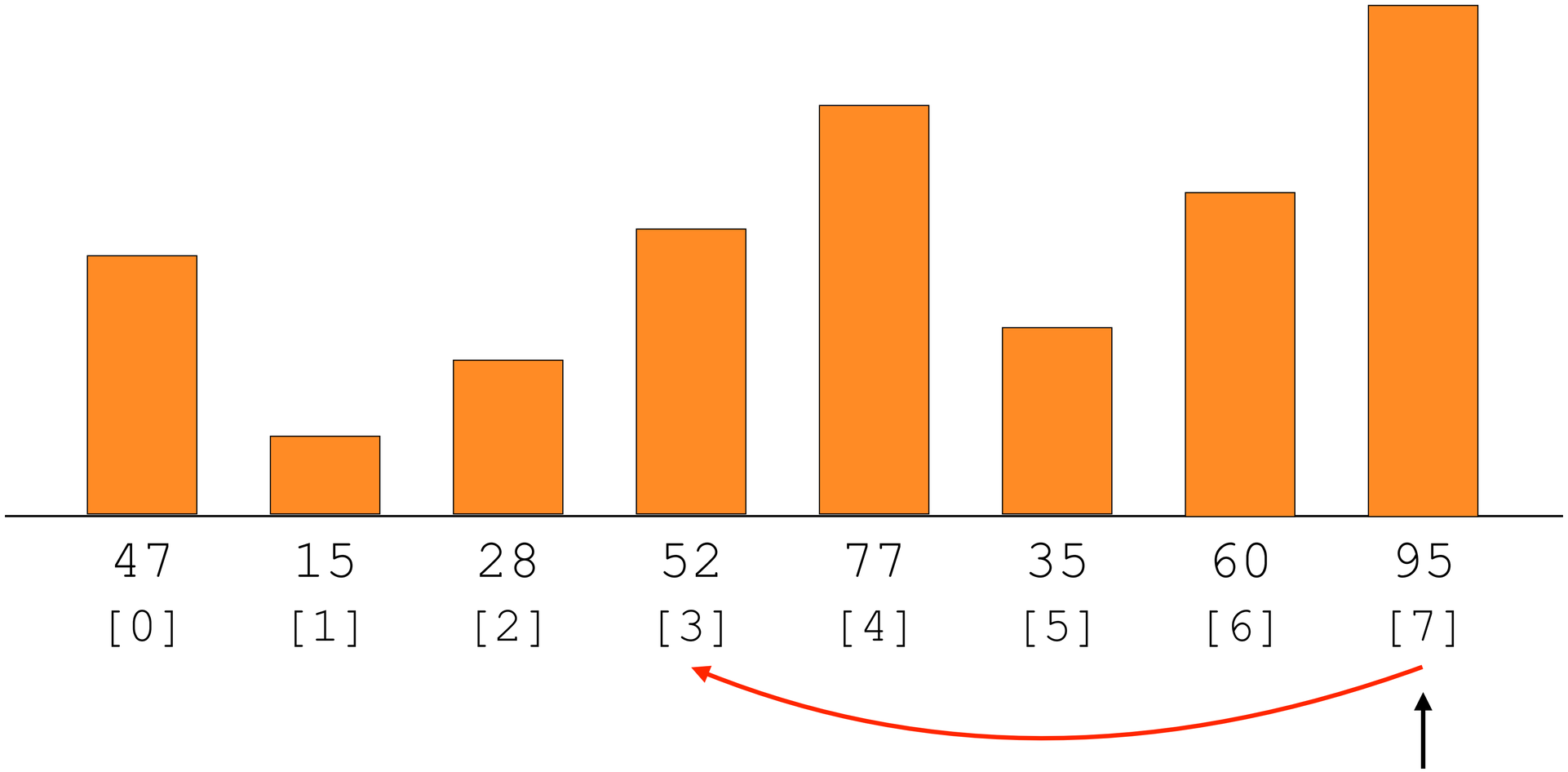
希尔排序

step 4



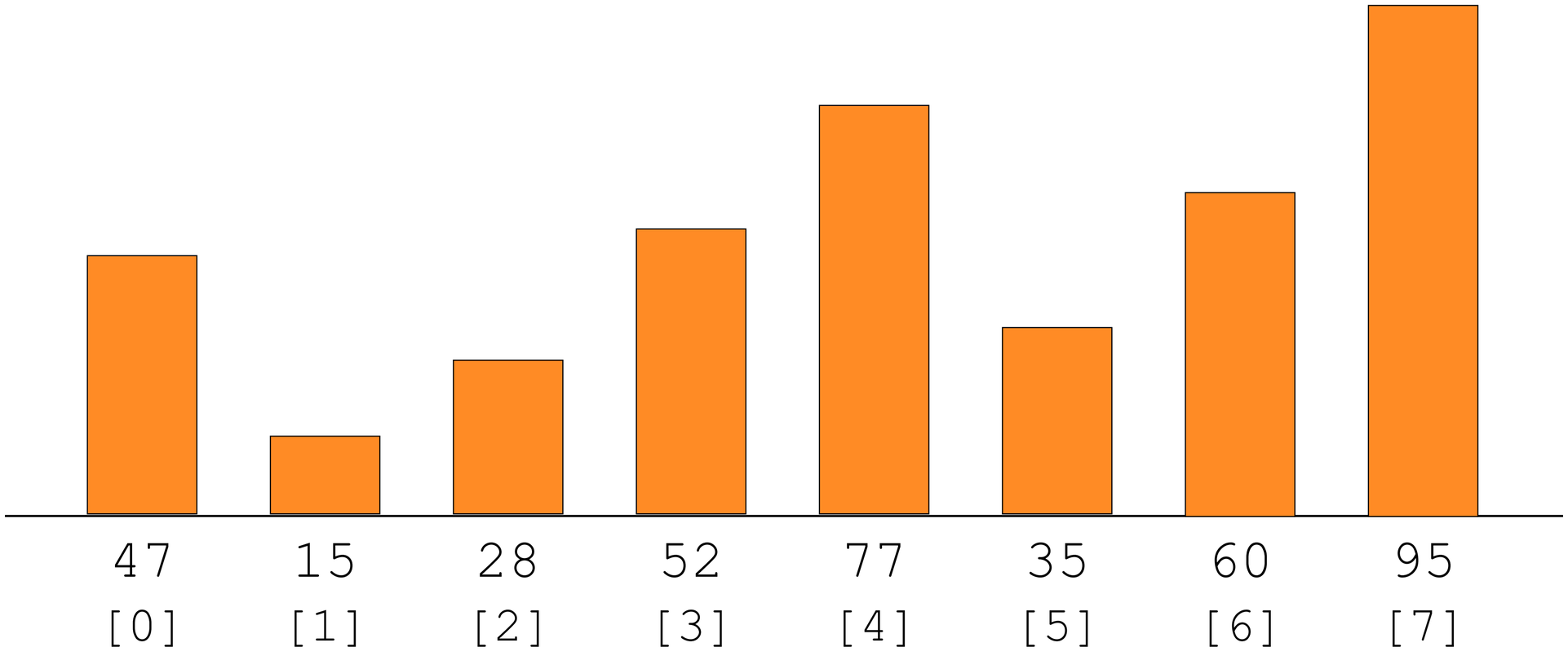
希尔排序

step 4



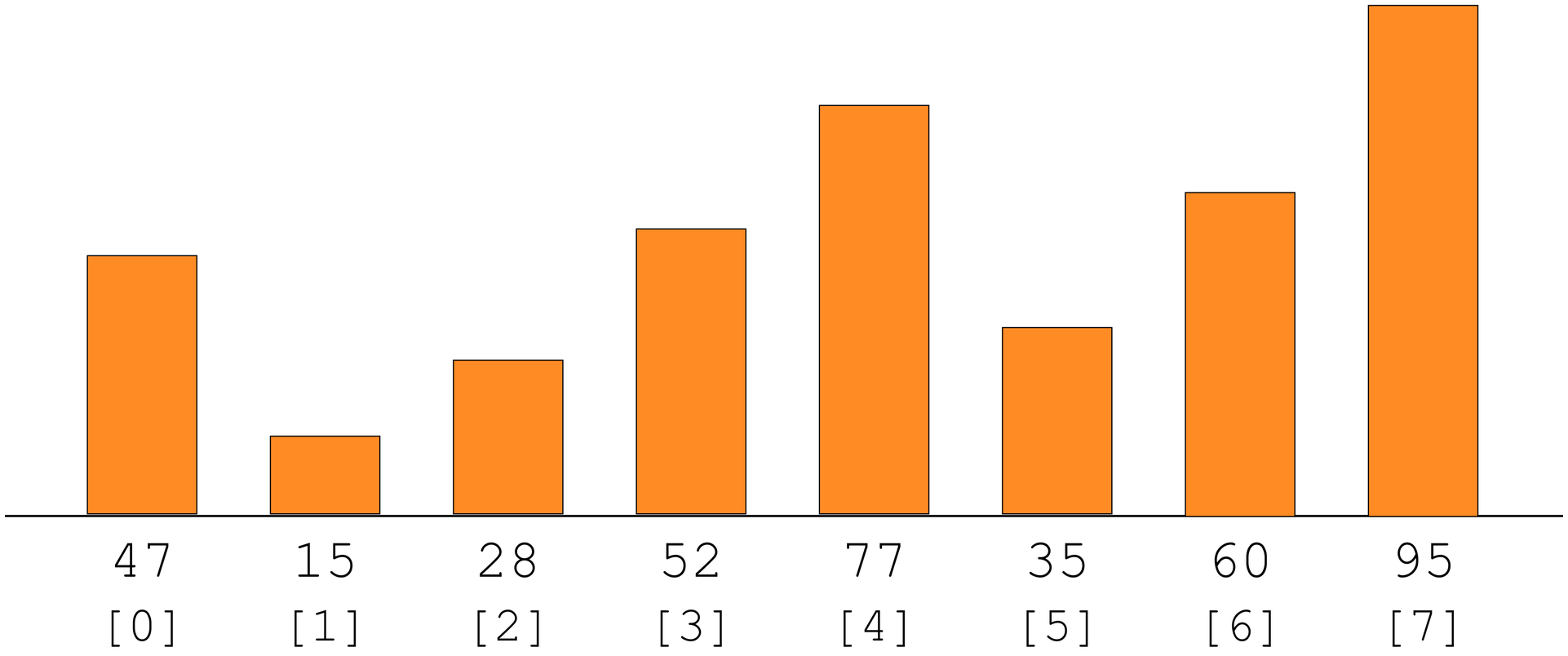
希尔排序

step 4



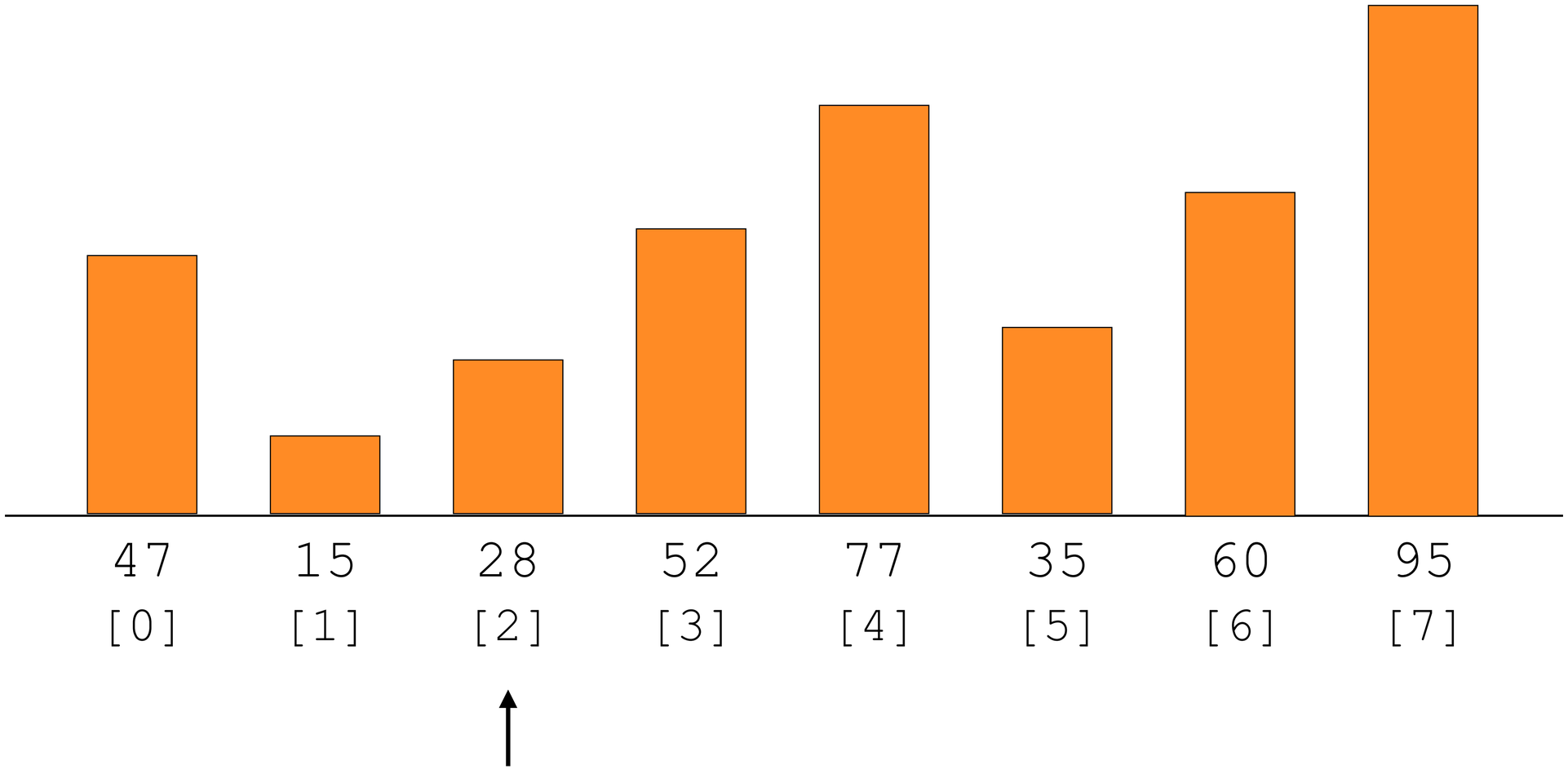
希尔排序

step 2



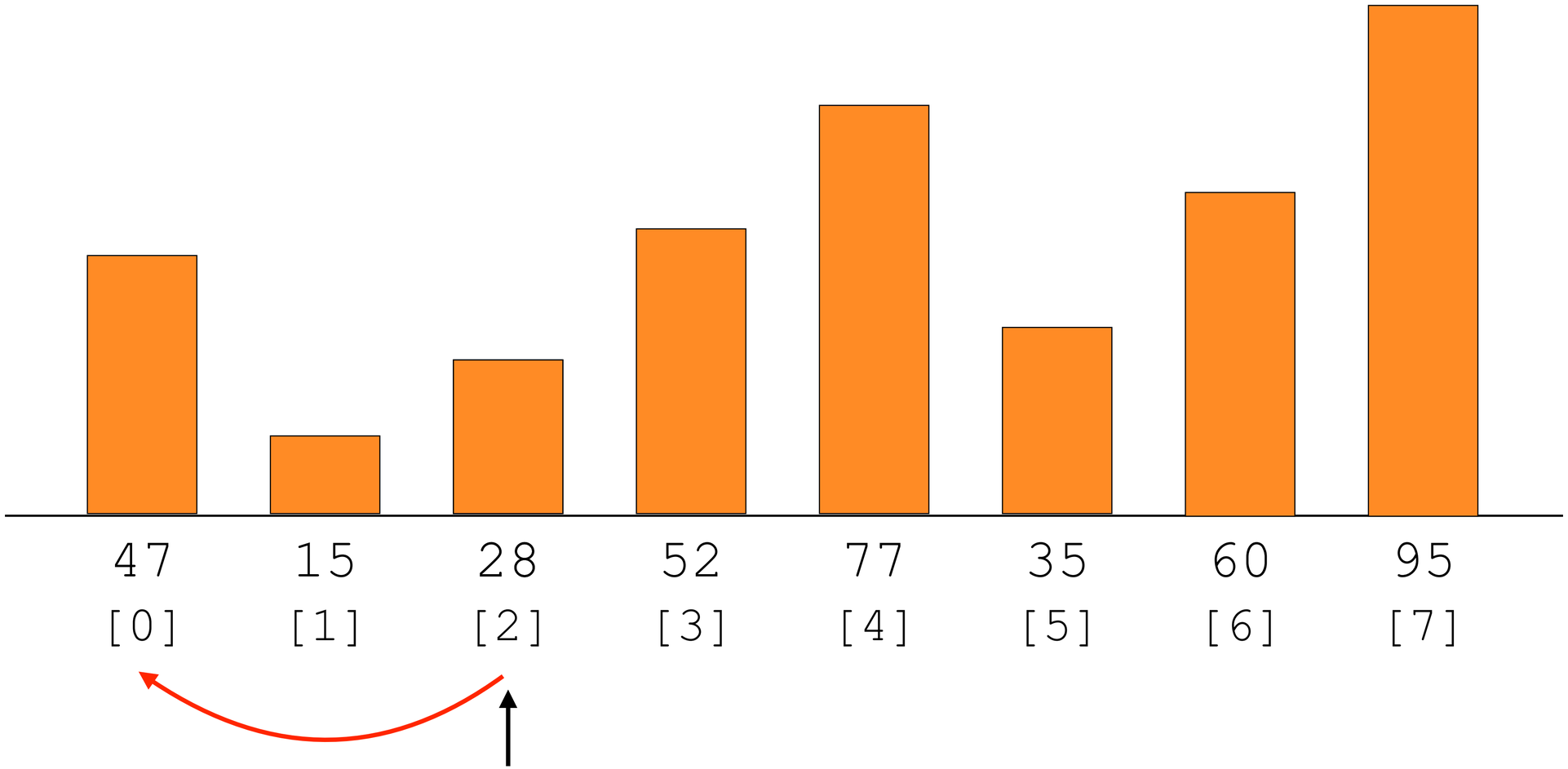
希尔排序

step 2



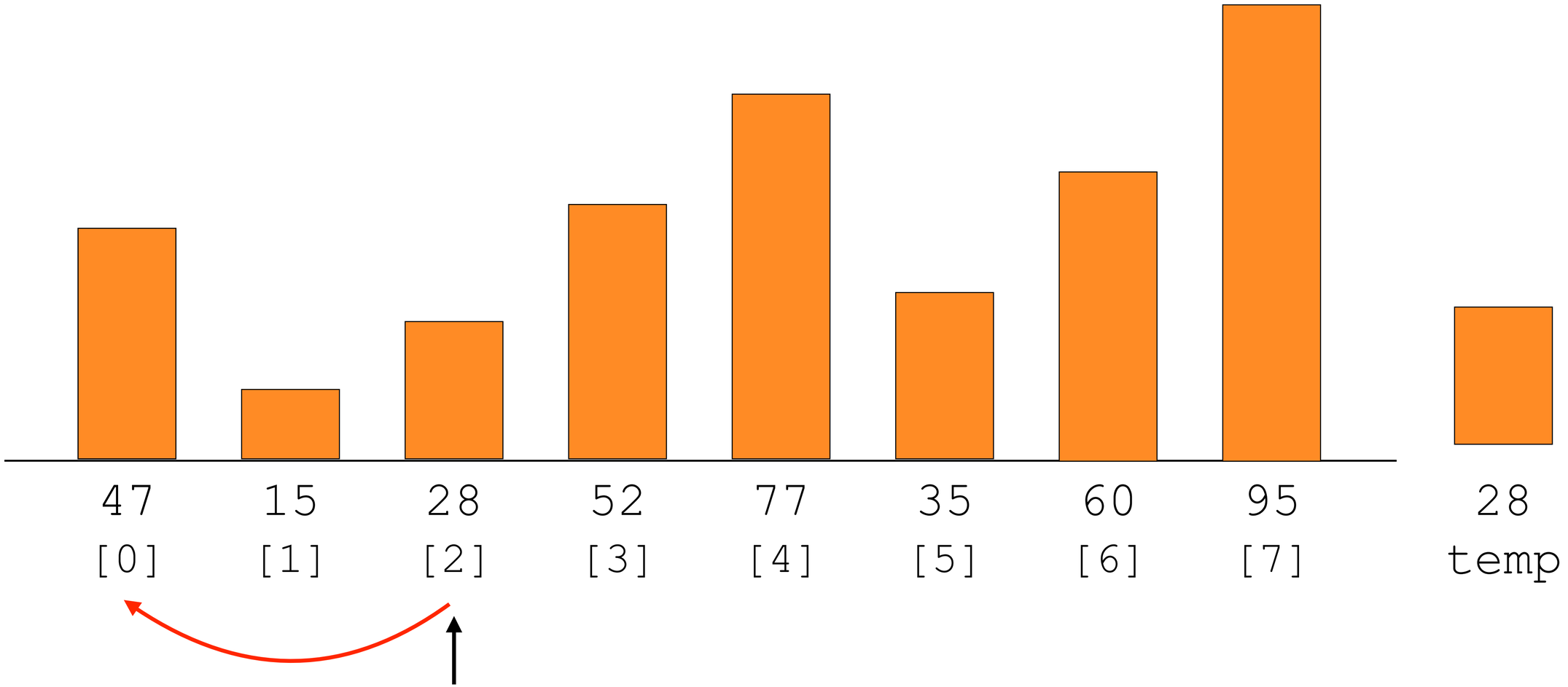
希尔排序

step 2



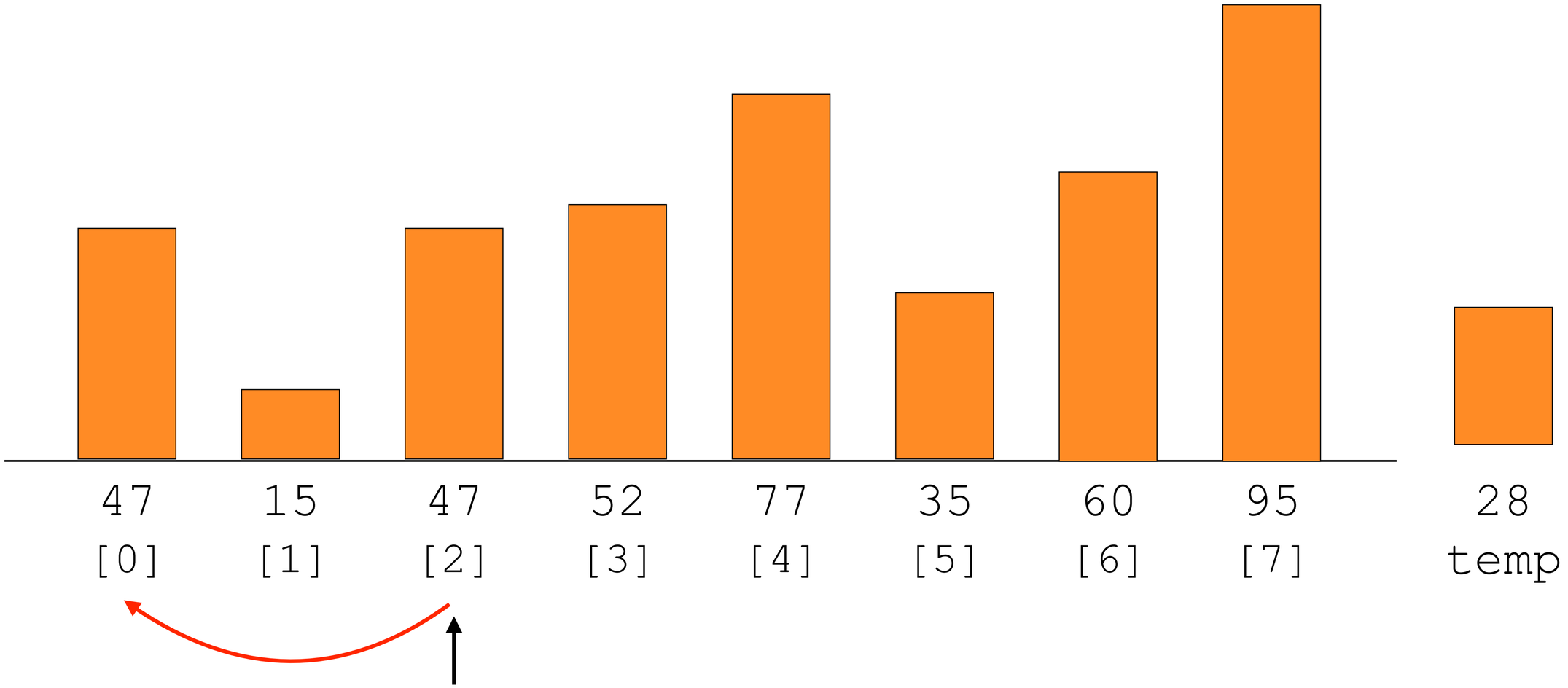
希尔排序

step 2



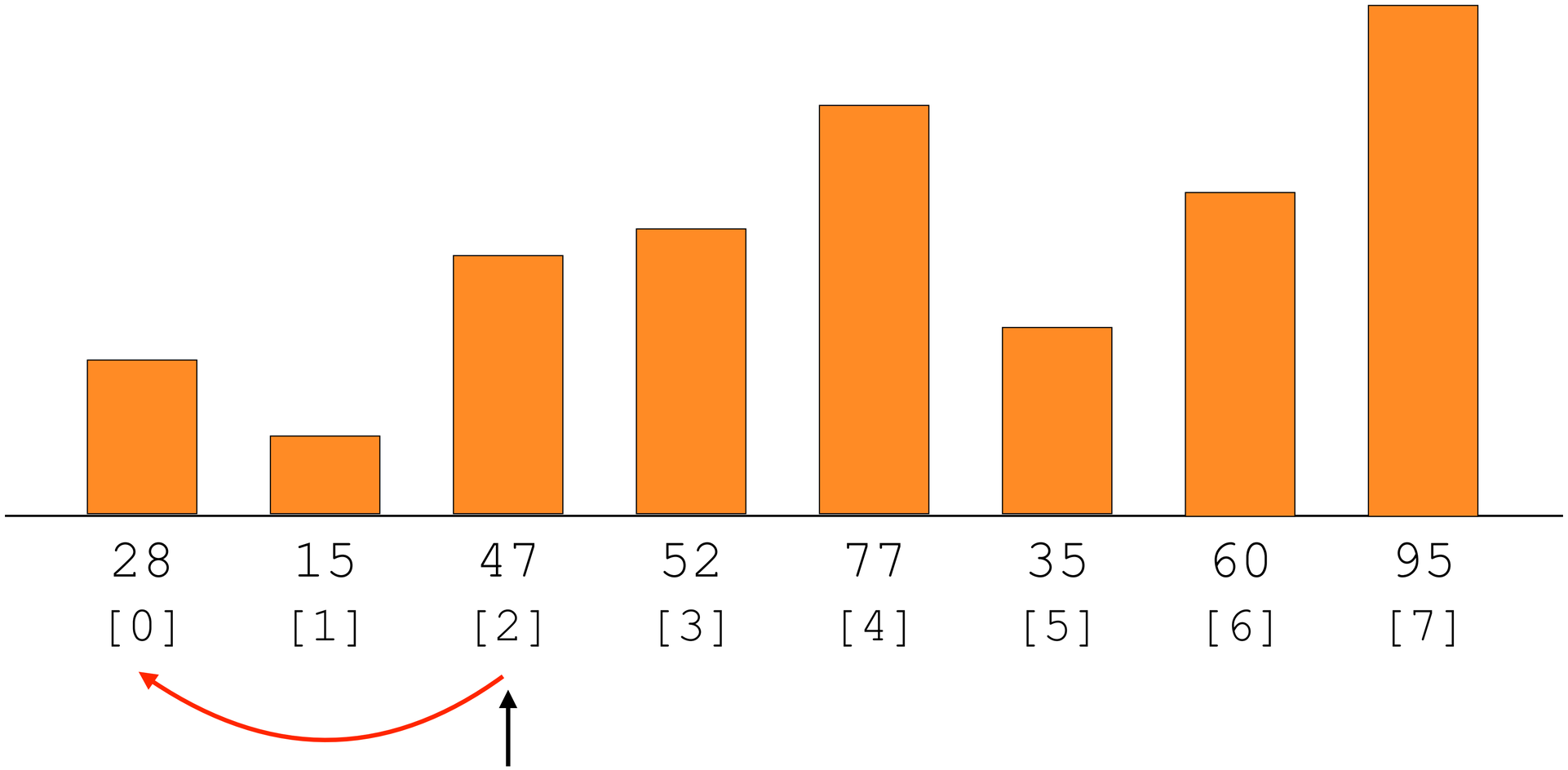
希尔排序

step 2



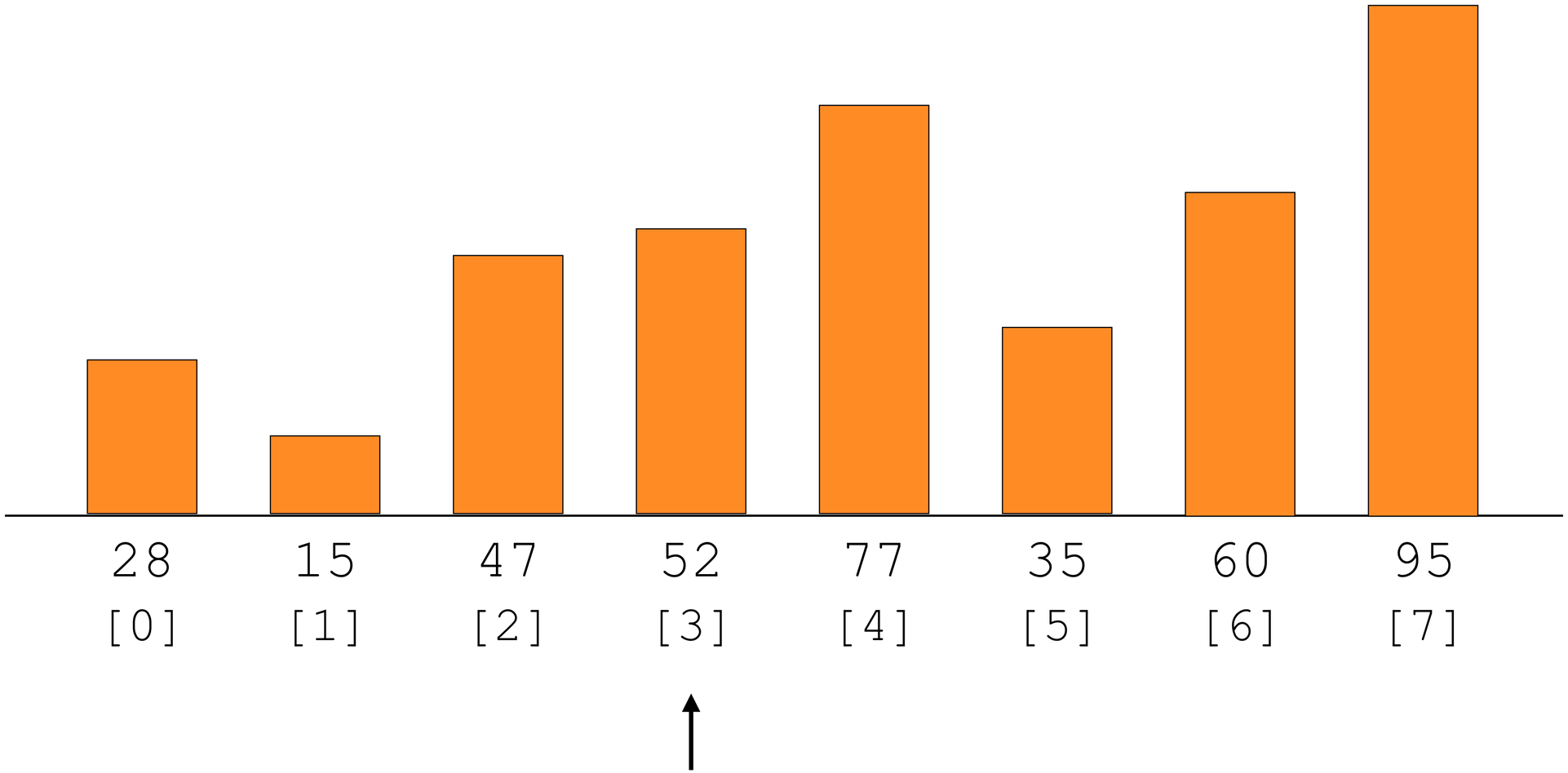
希尔排序

step 2



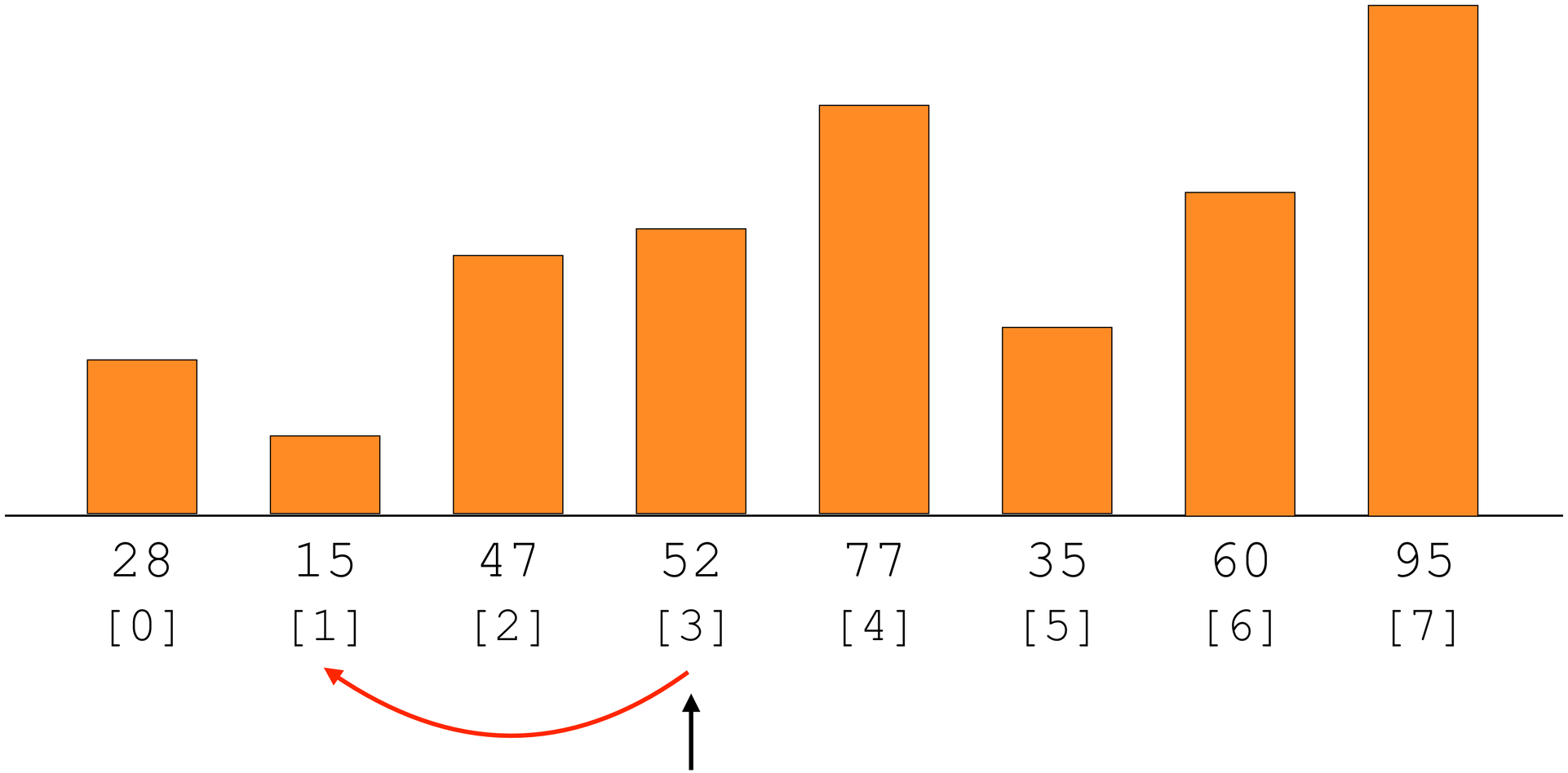
希尔排序

step 2



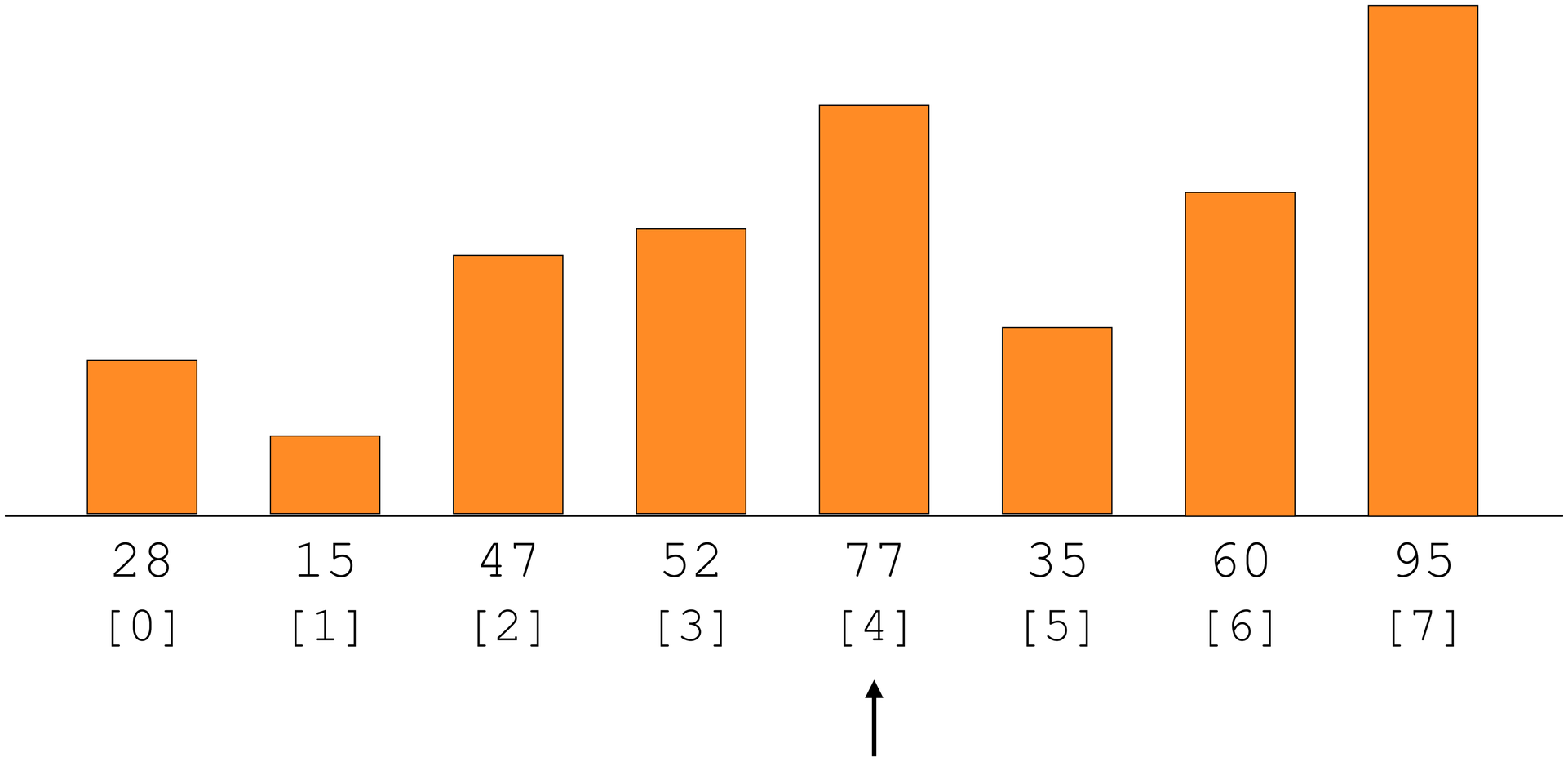
希尔排序

step 2



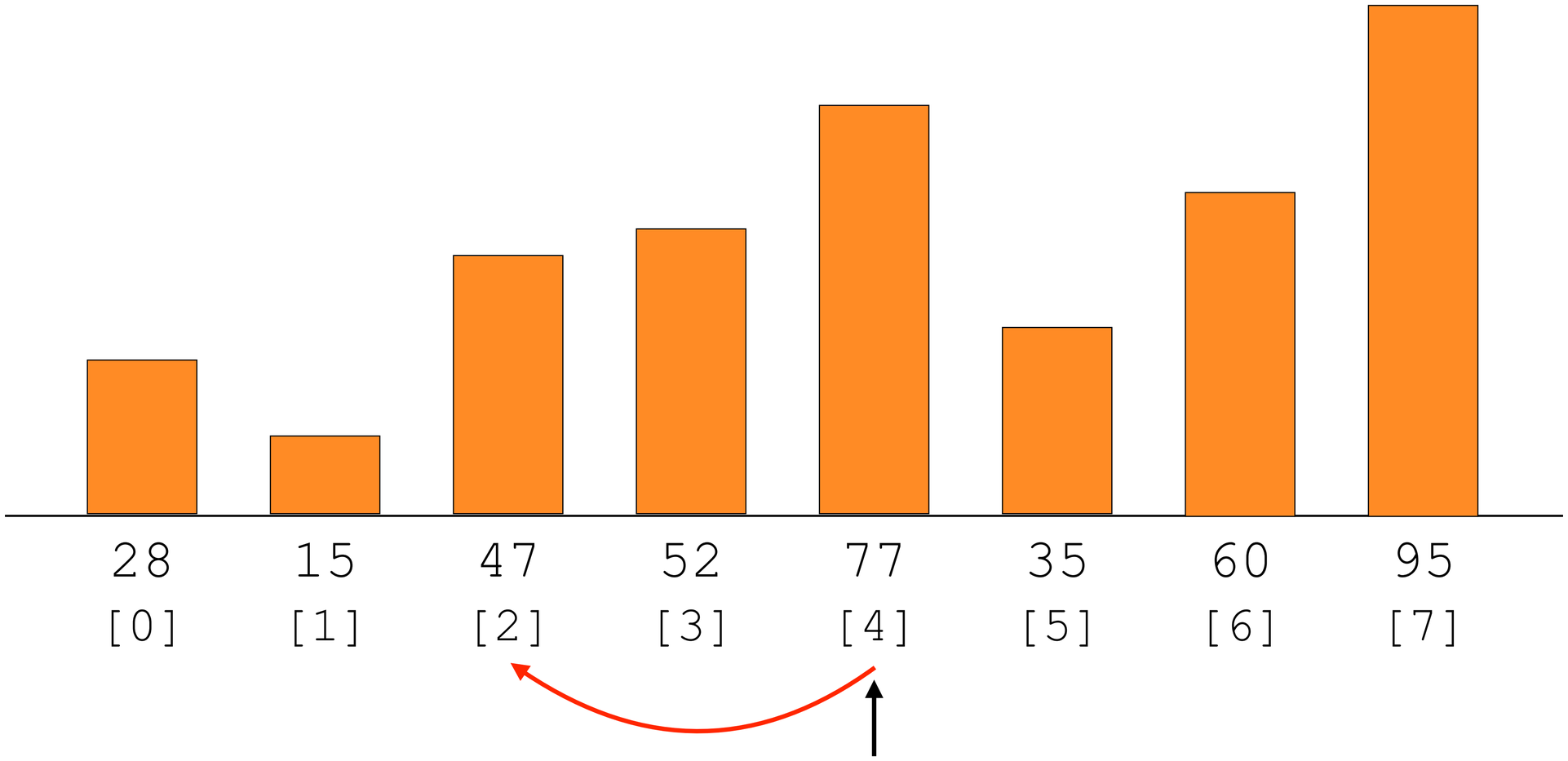
希尔排序

step 2



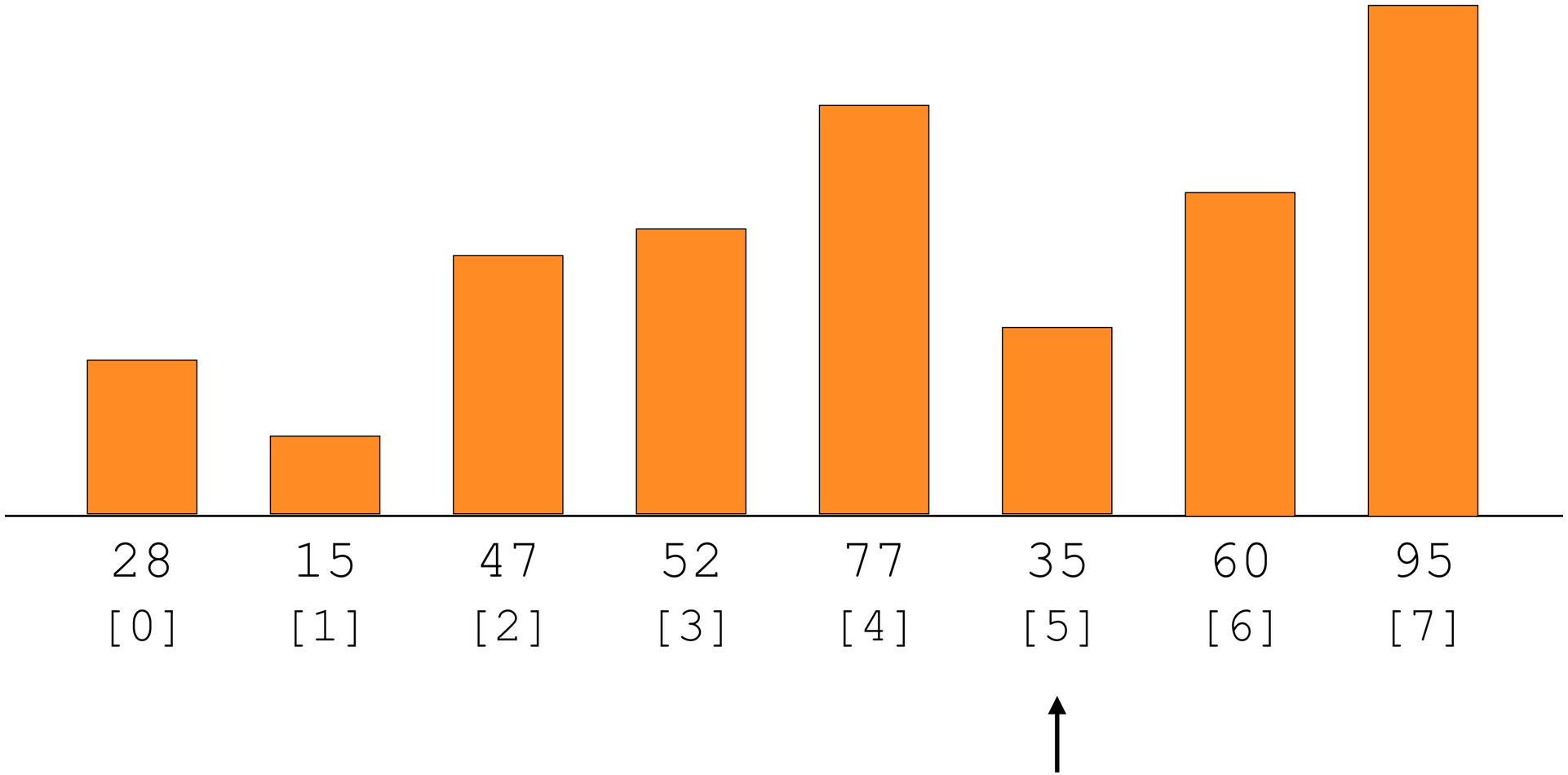
希尔排序

step 2



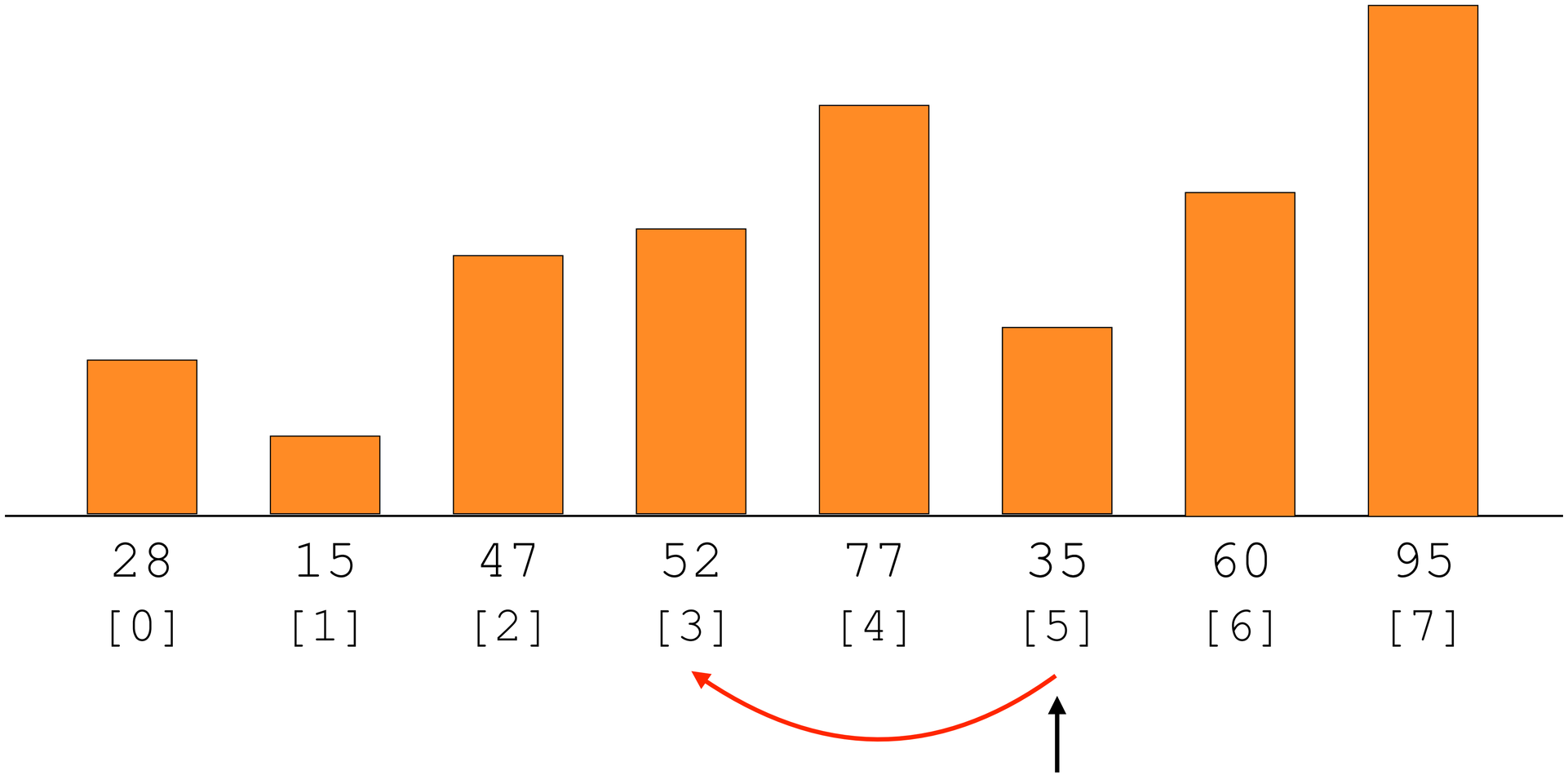
希尔排序

step 2



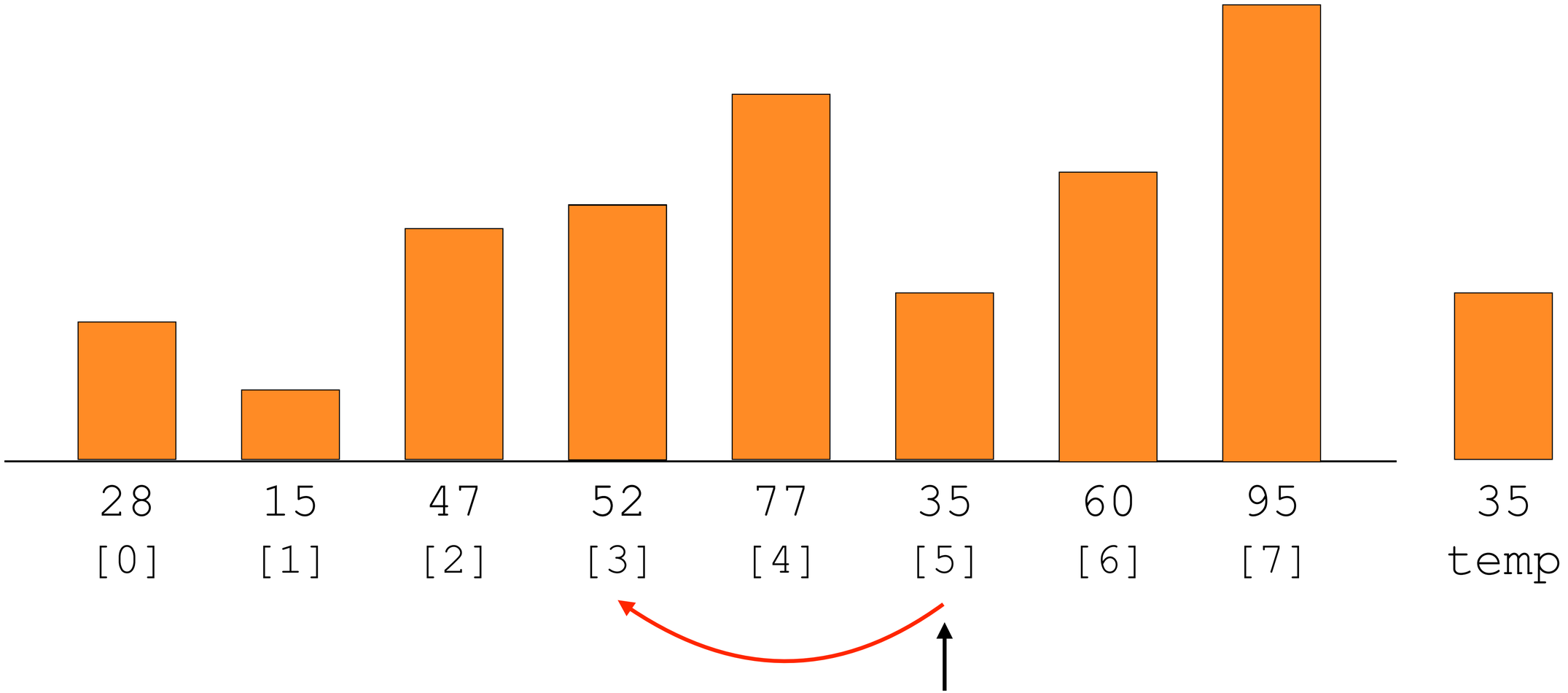
希尔排序

step 2



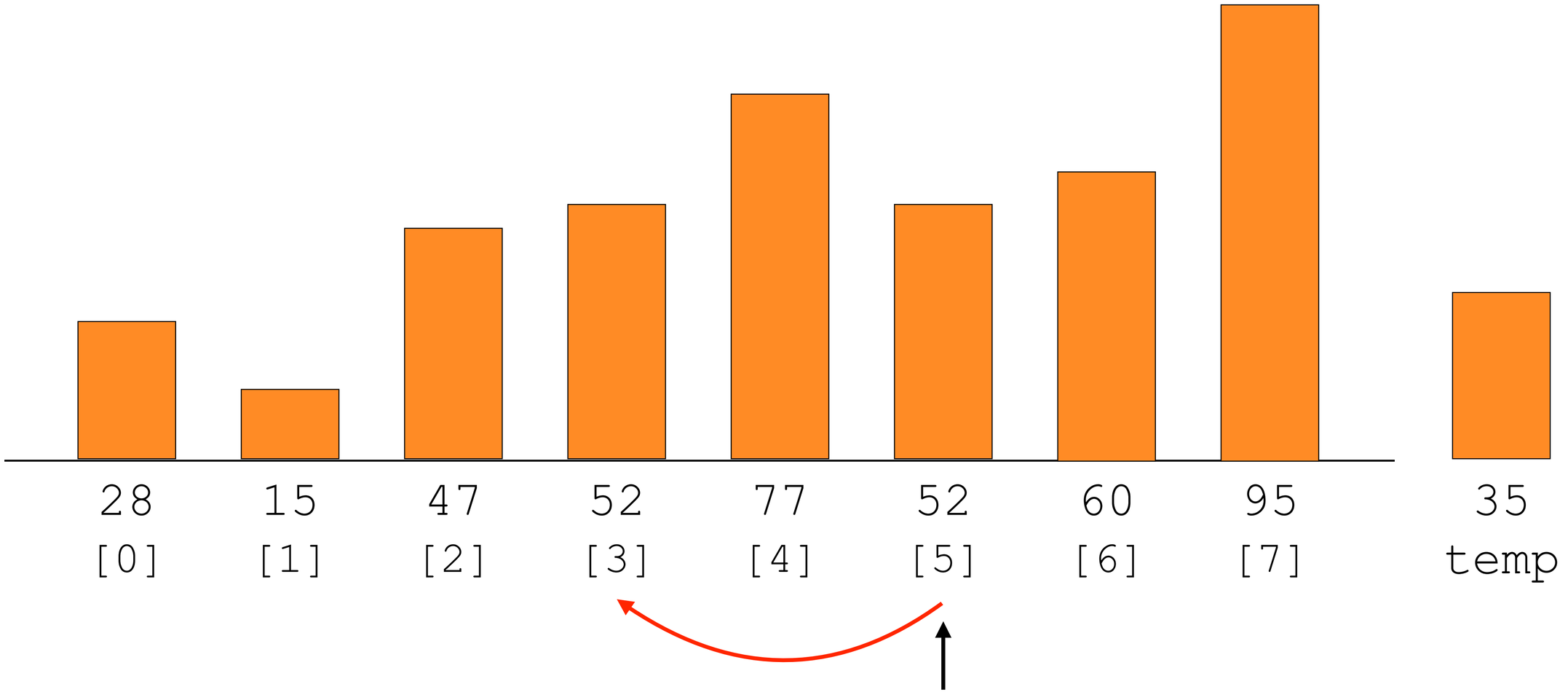
希尔排序

step 2



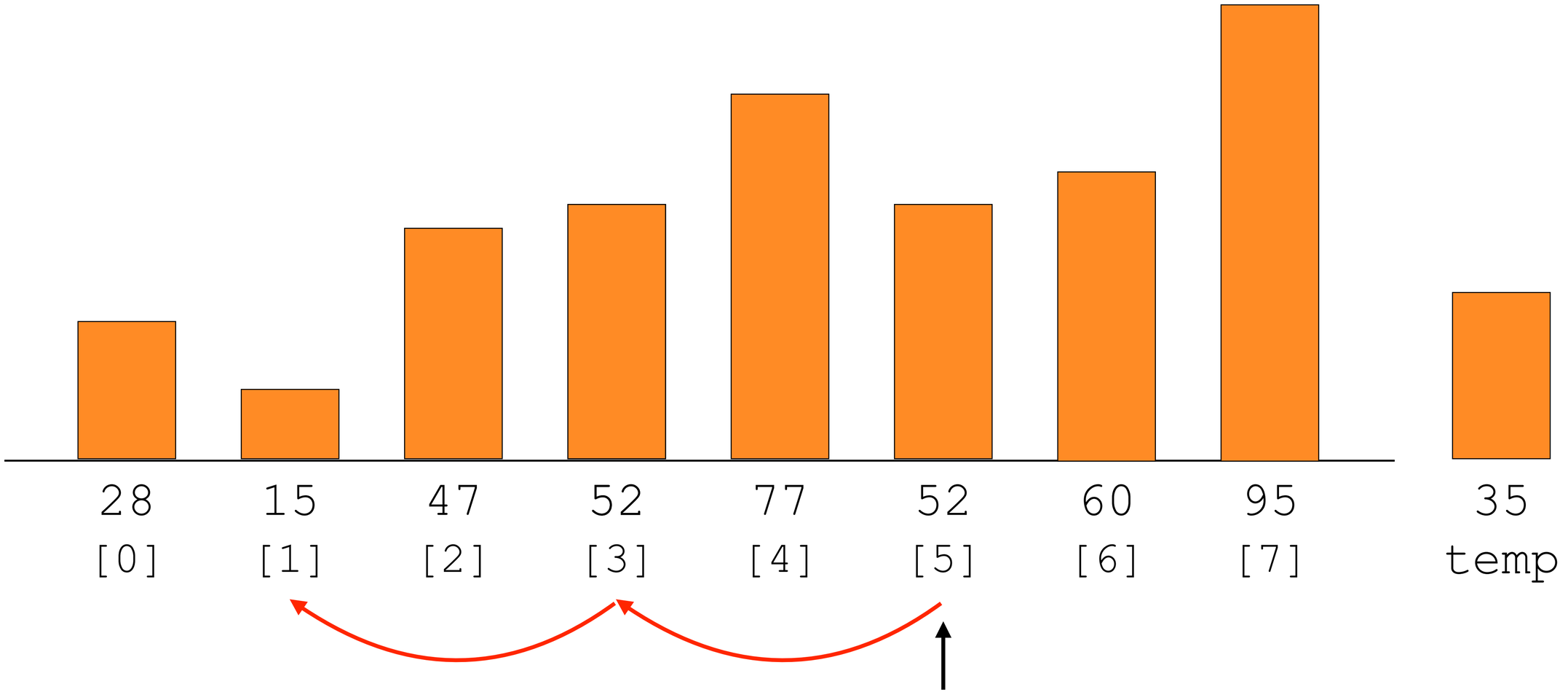
希尔排序

step 2



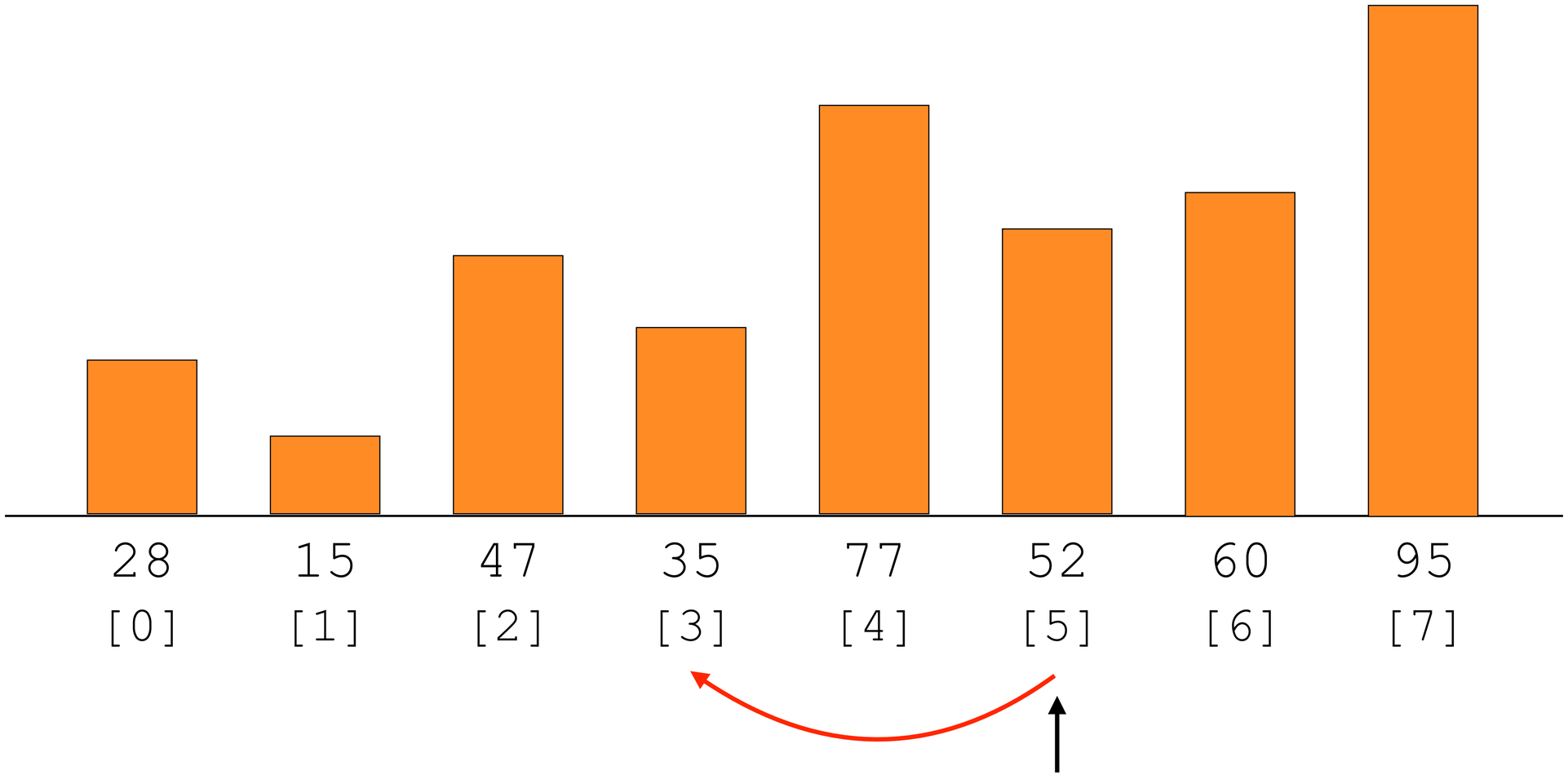
希尔排序

step 2



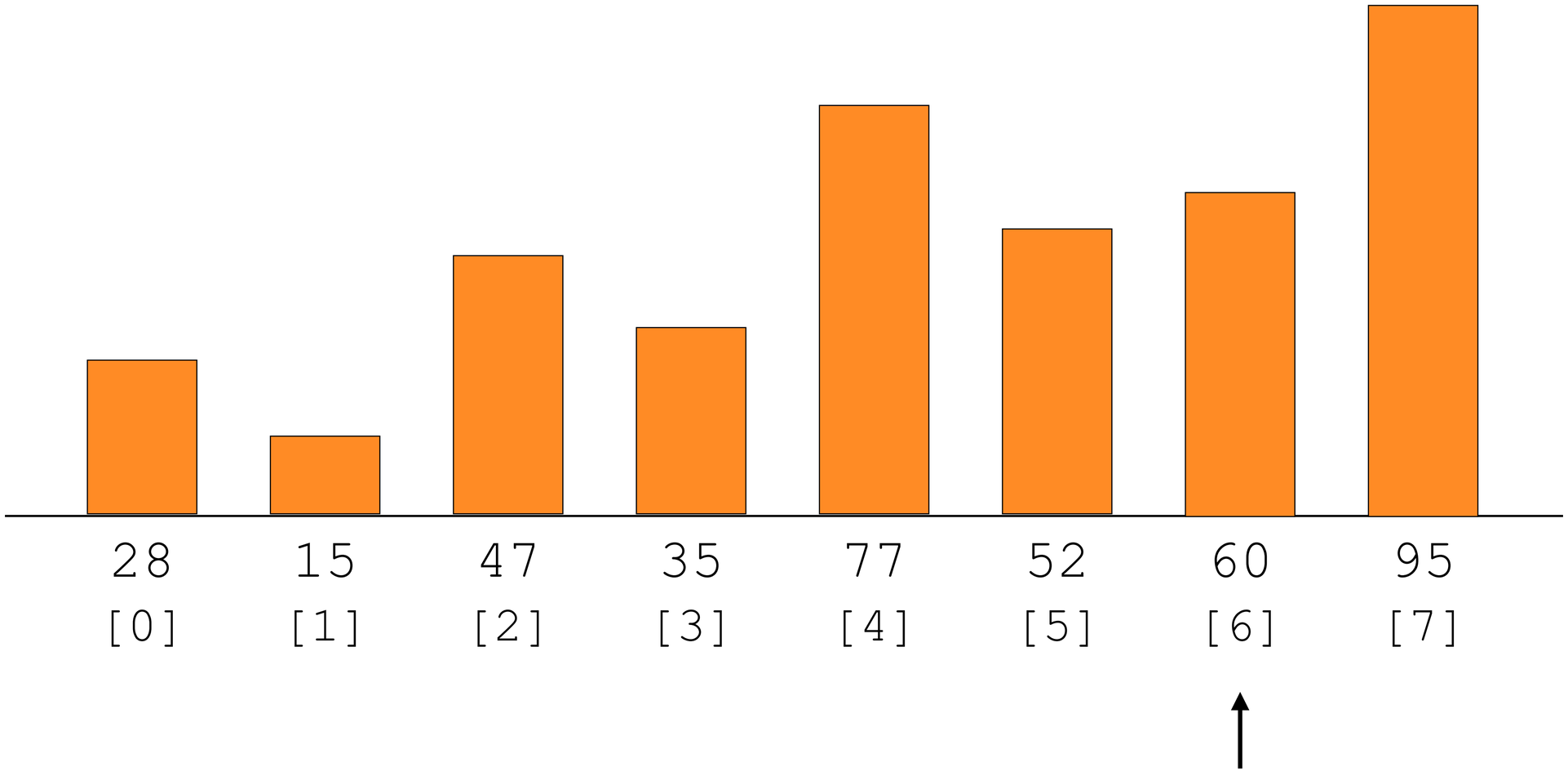
希尔排序

step 2



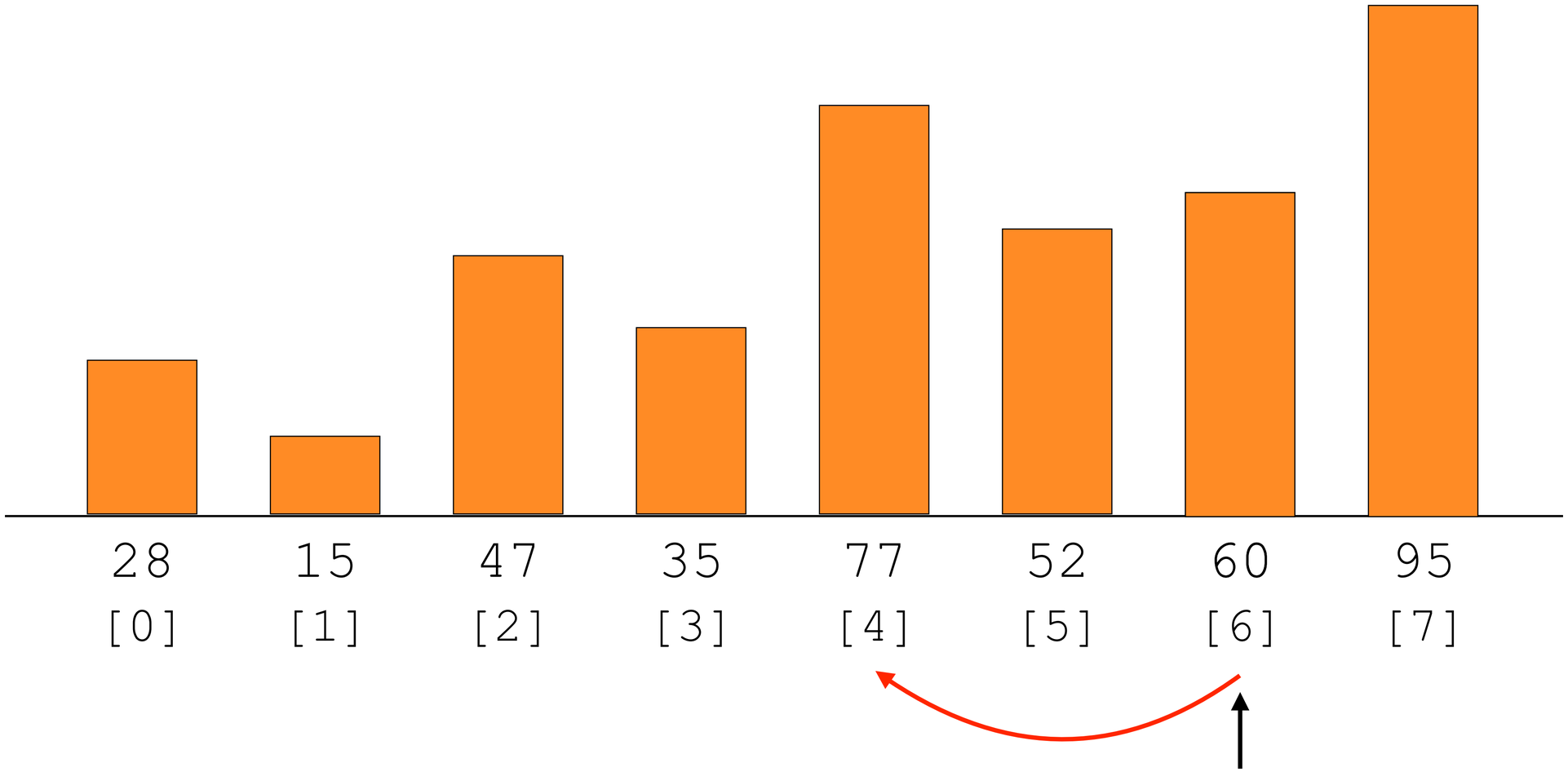
希尔排序

step 2



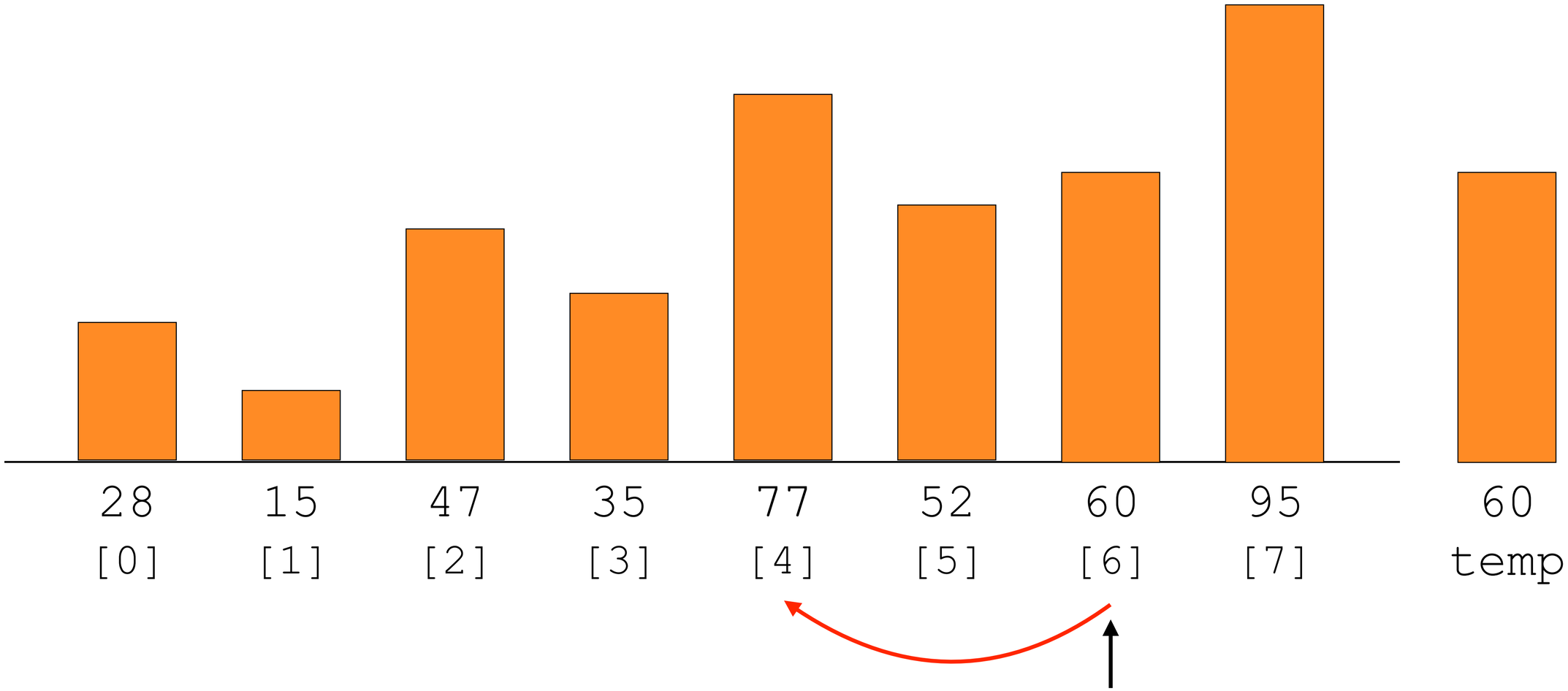
希尔排序

step 2



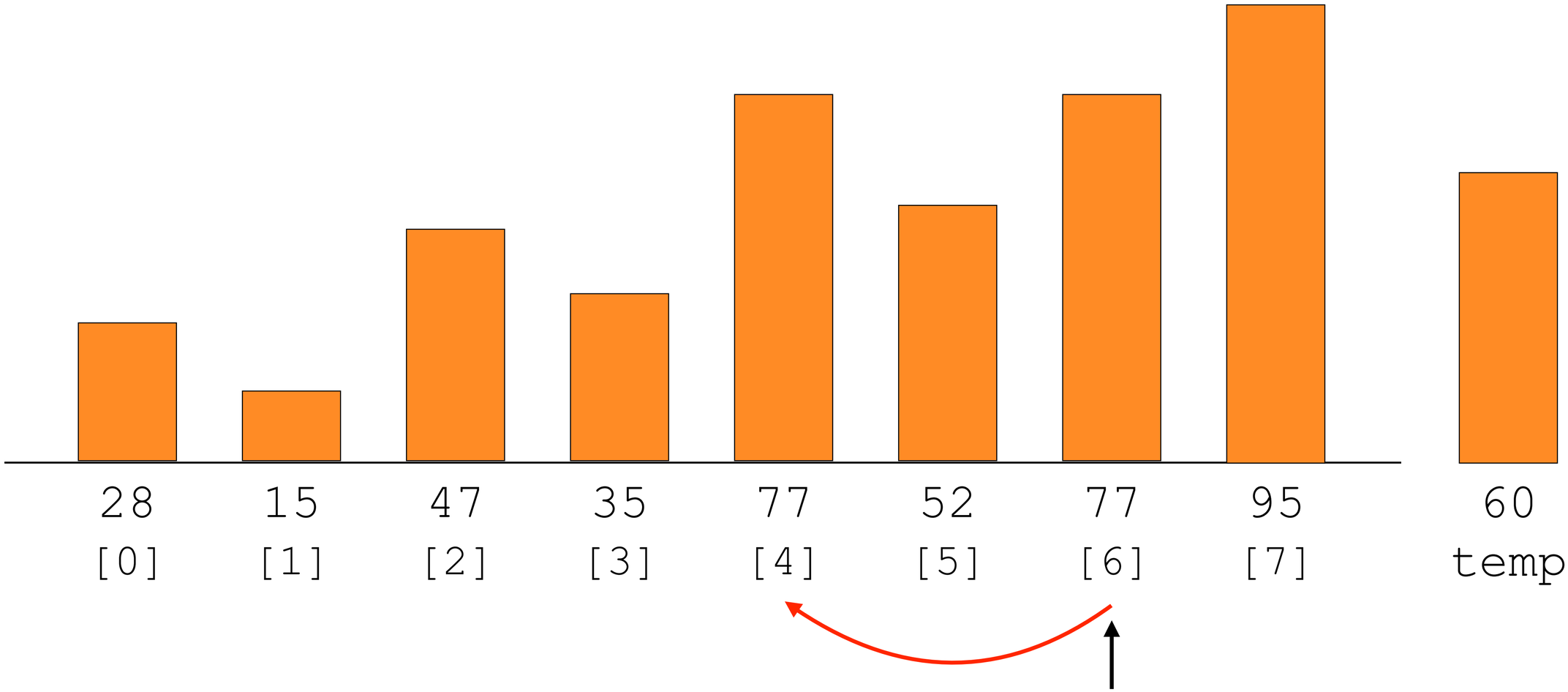
希尔排序

step 2



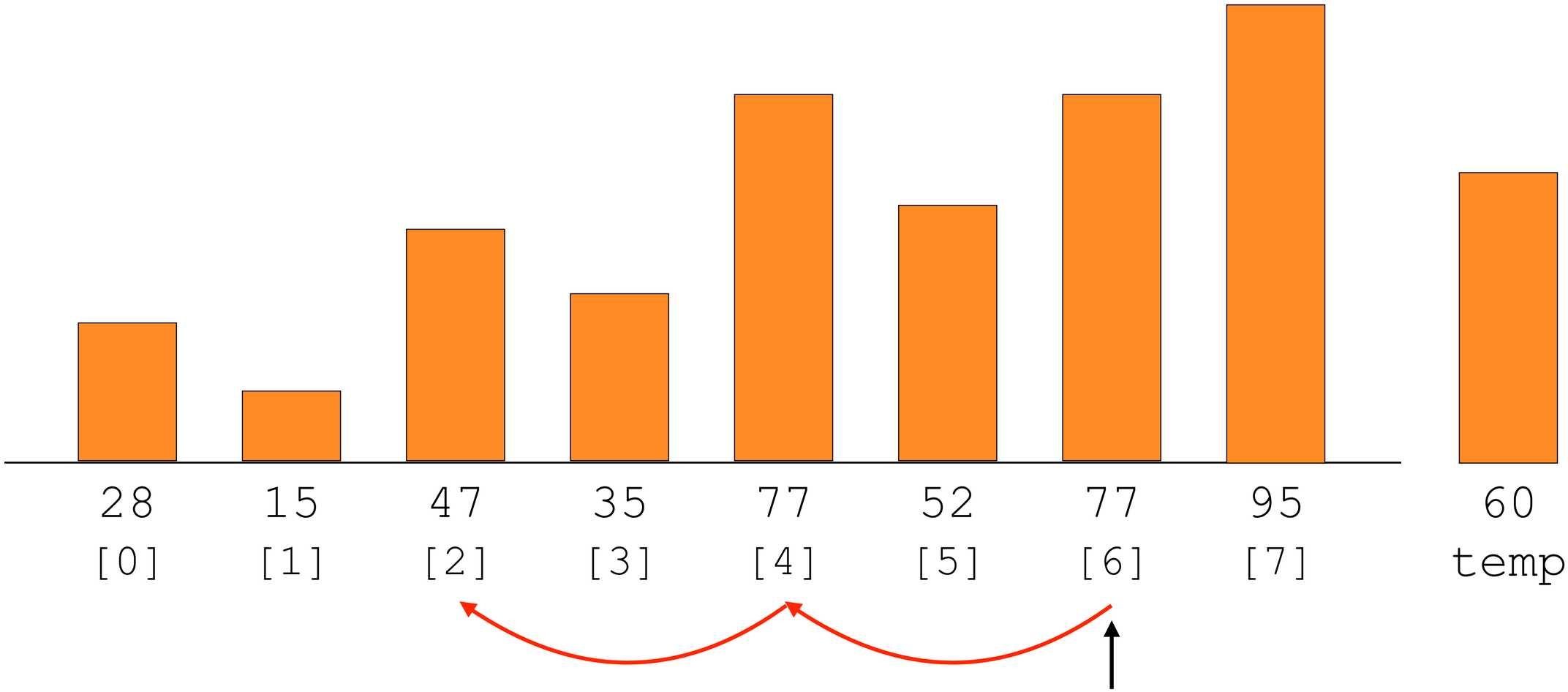
希尔排序

step 2



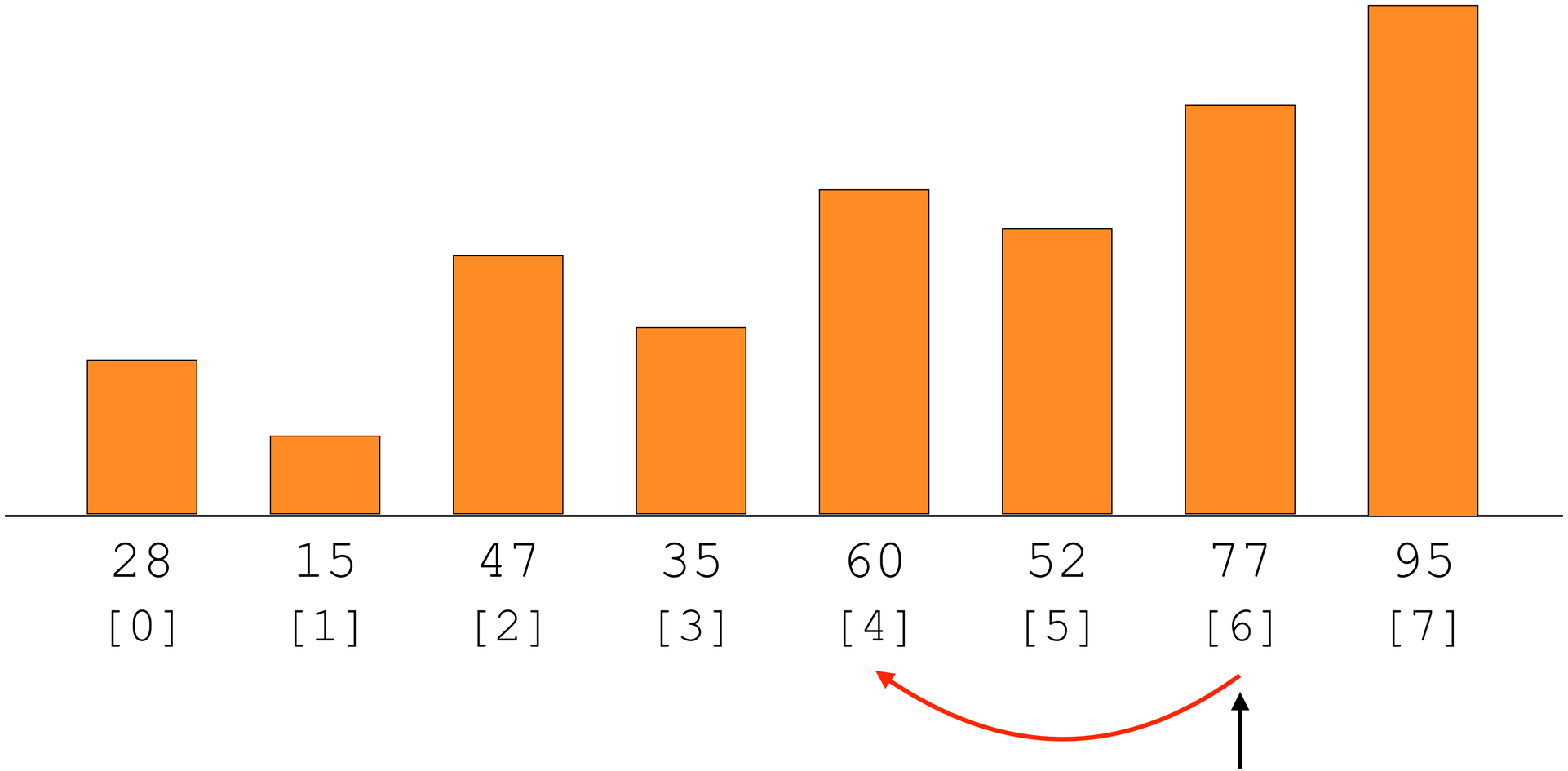
希尔排序

step 2



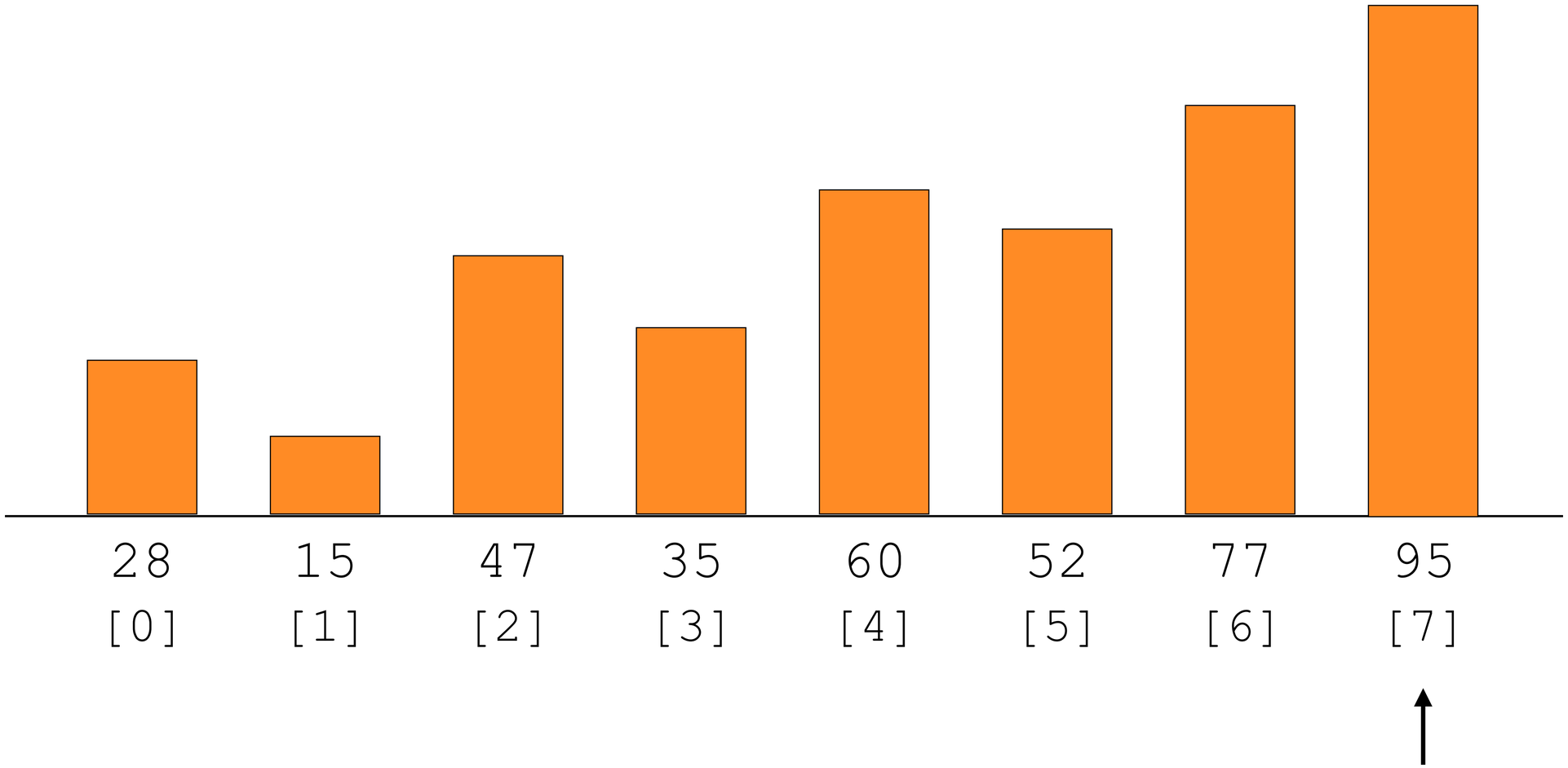
希尔排序

step 2



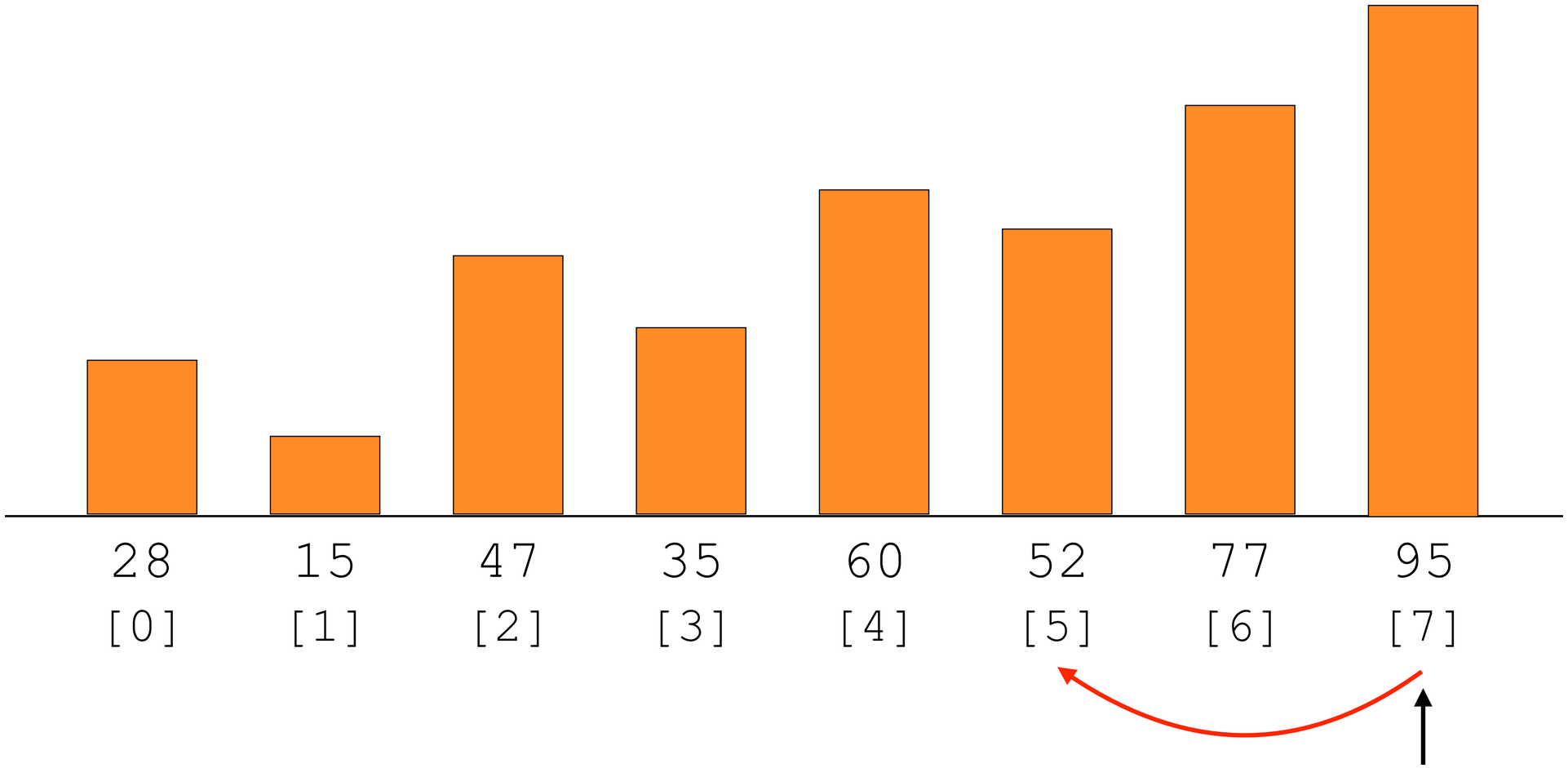
希尔排序

step 2



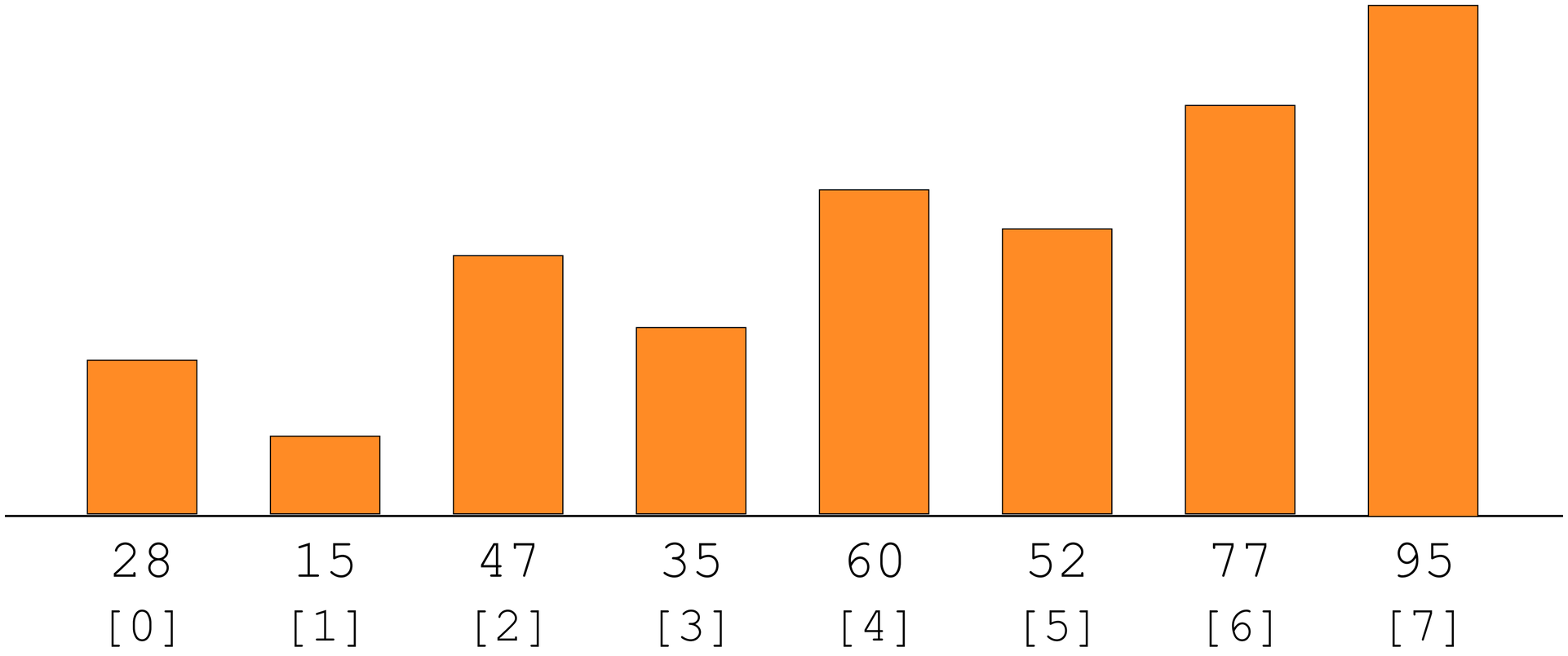
希尔排序

step 2



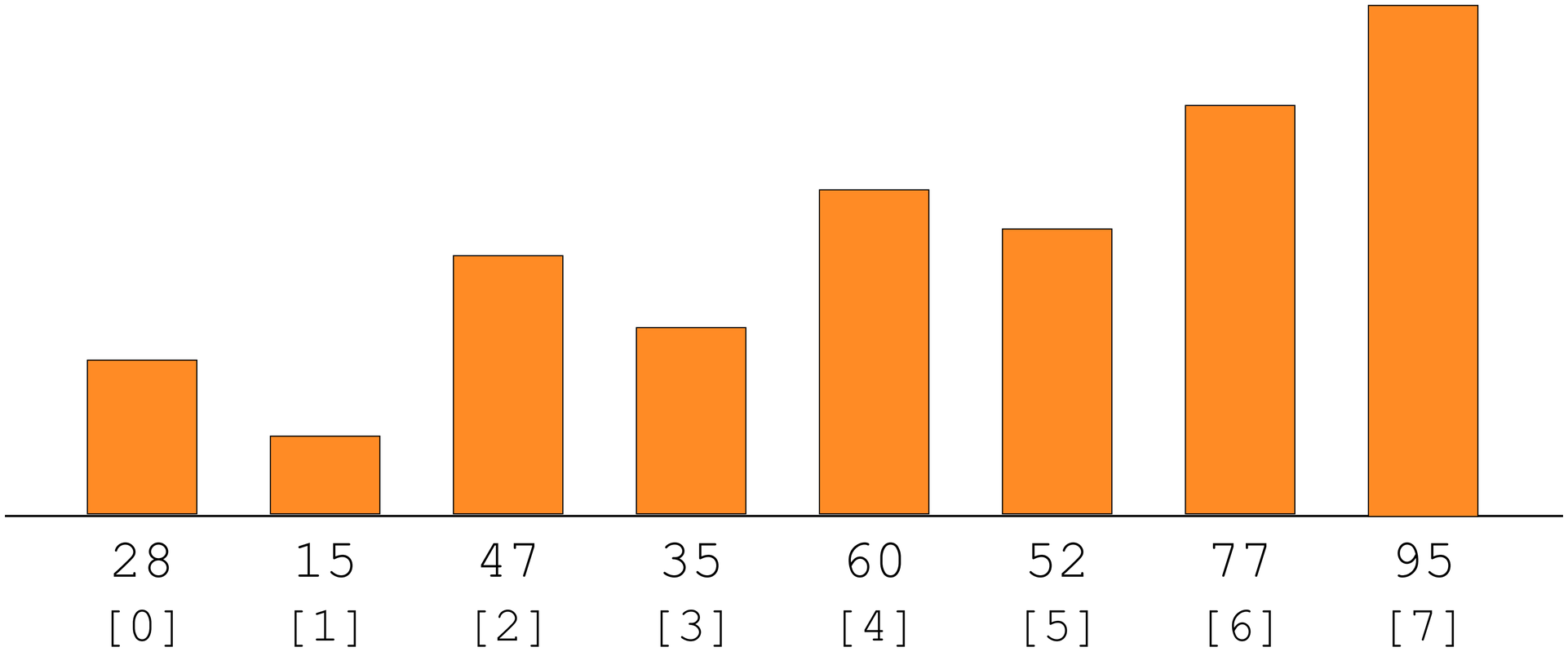
希尔排序

step 2



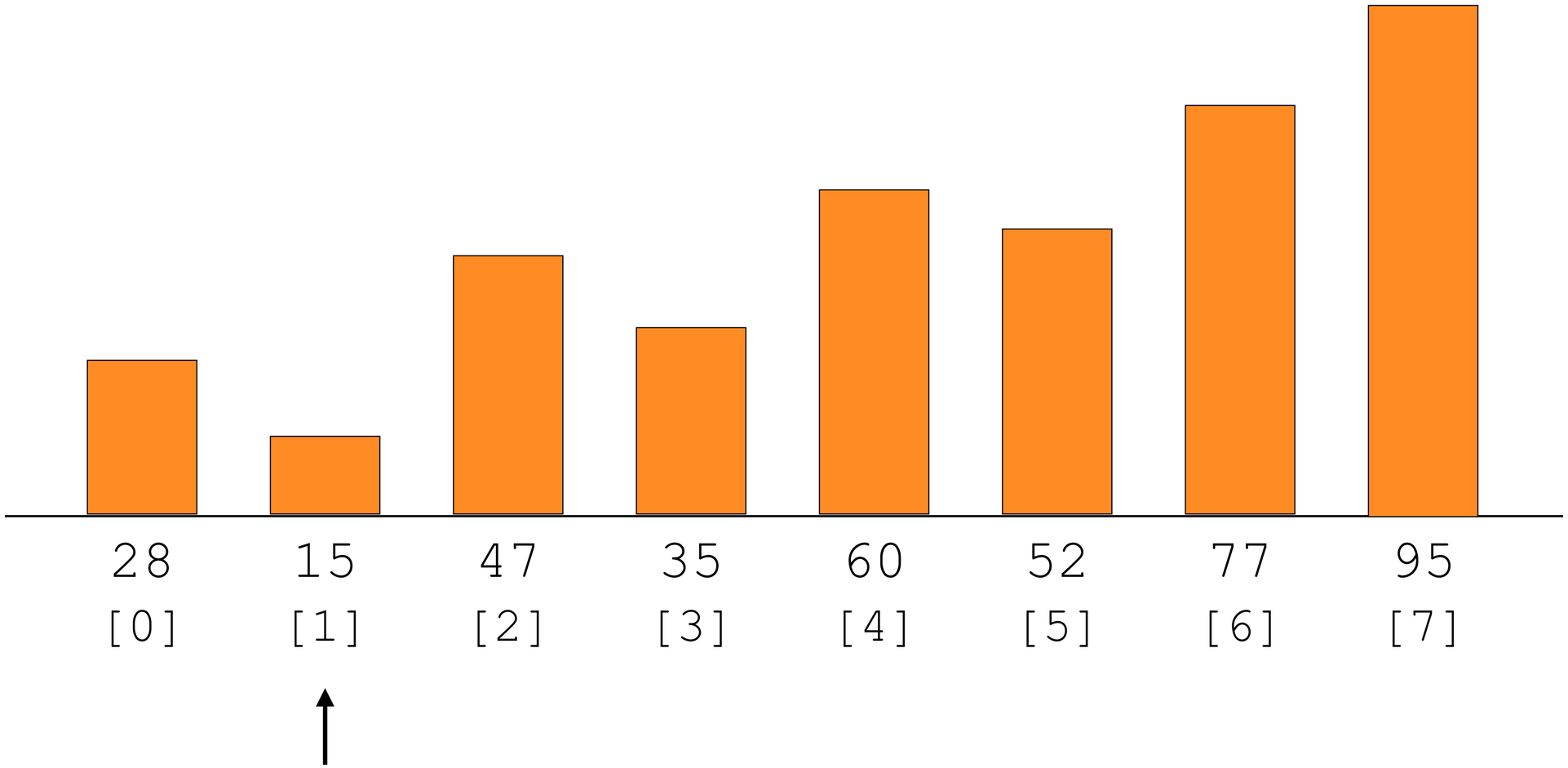
希尔排序

step 1



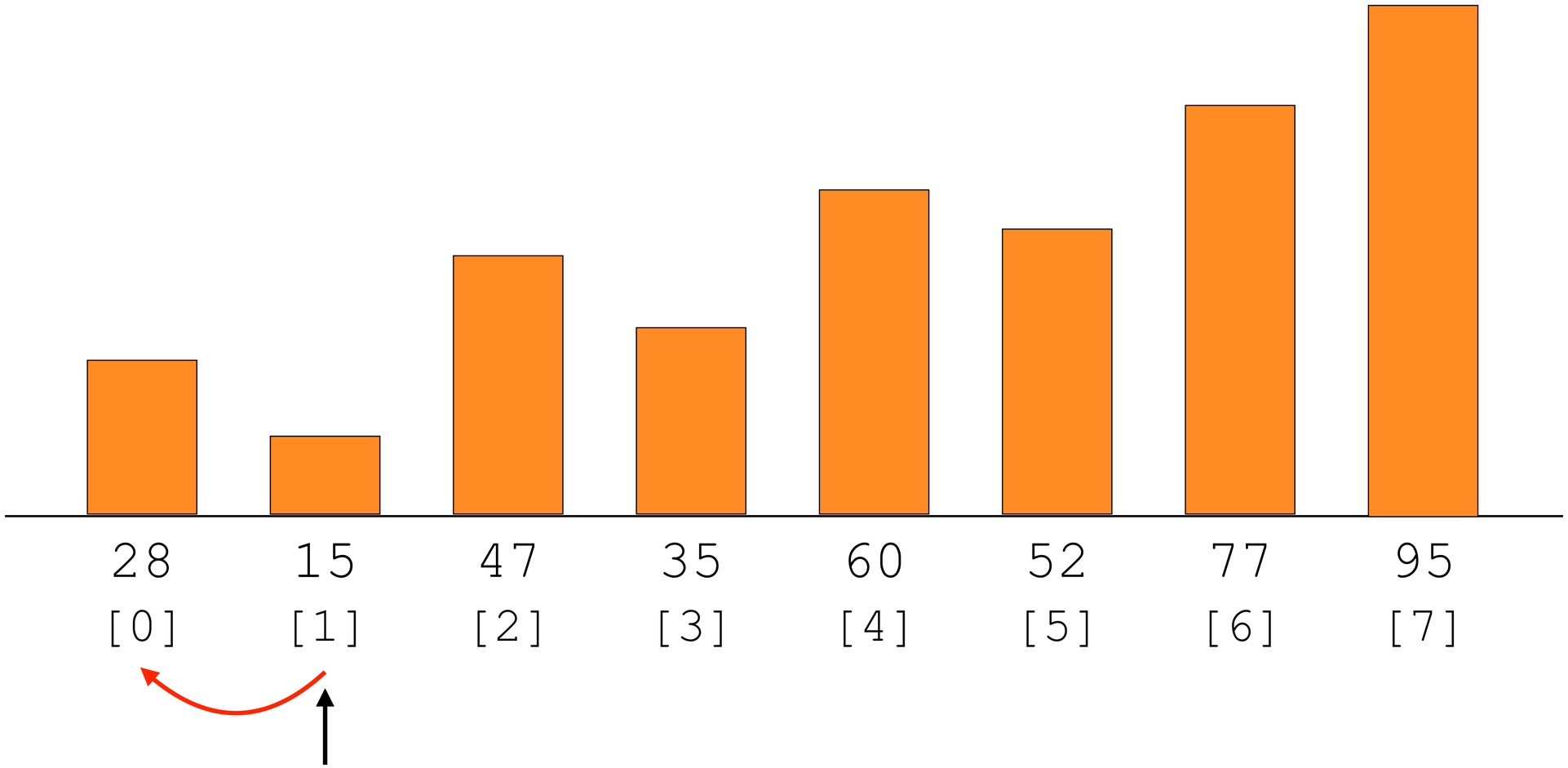
希尔排序

step 1



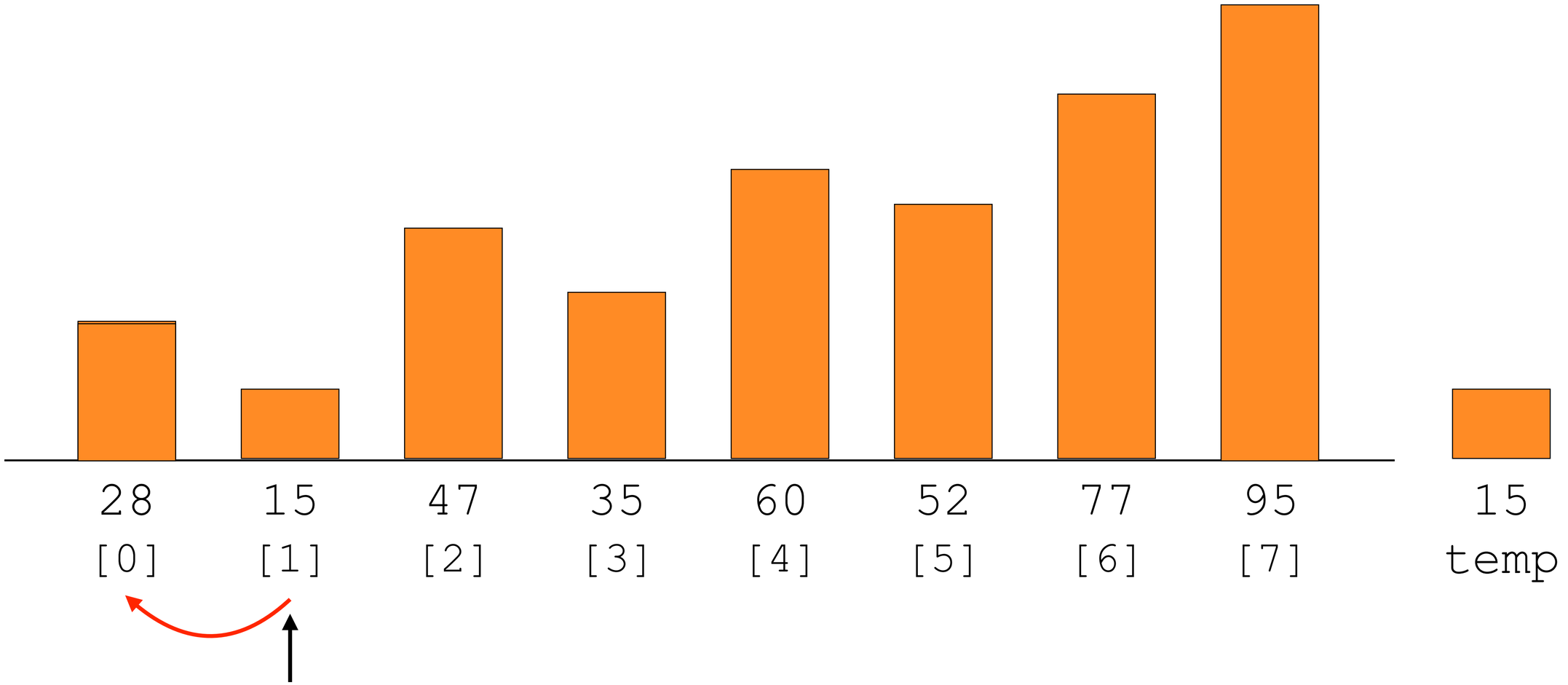
希尔排序

step 1



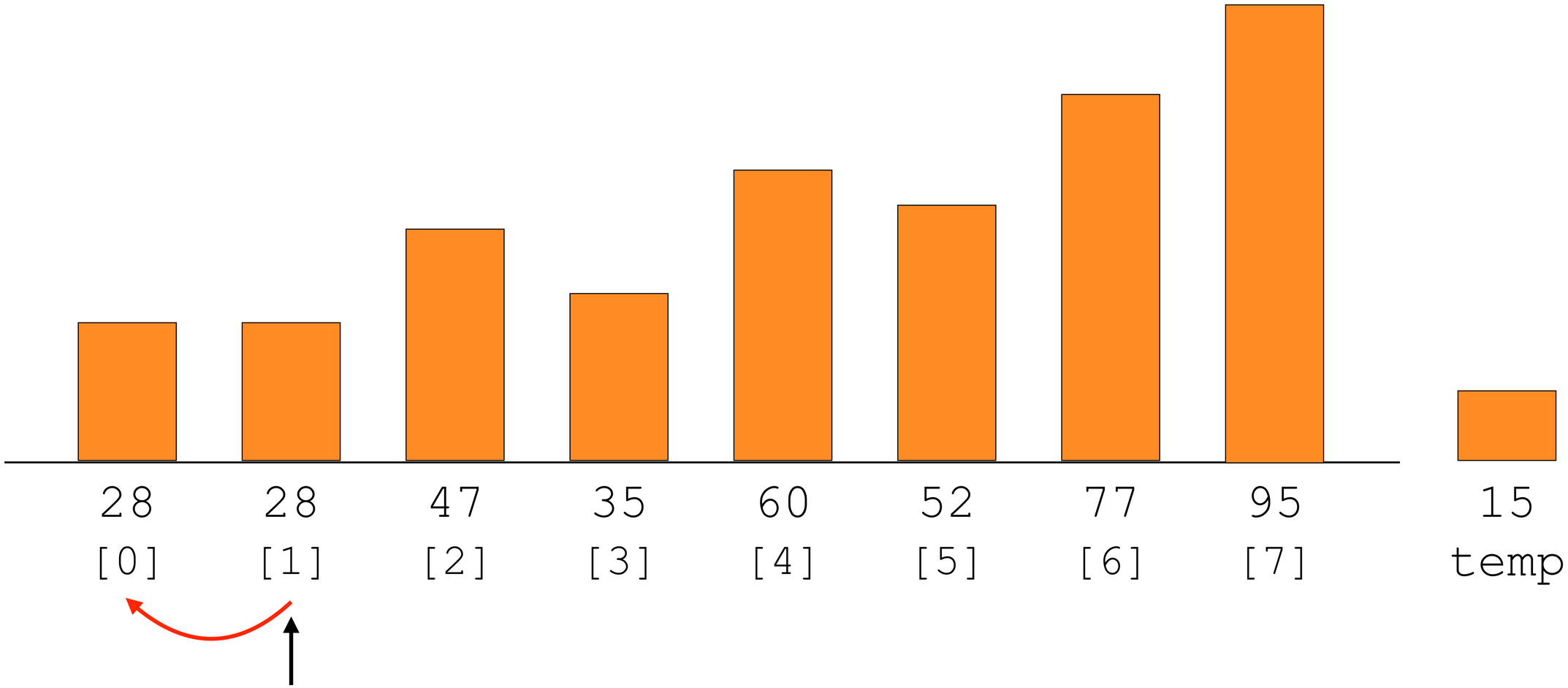
希尔排序

step 1



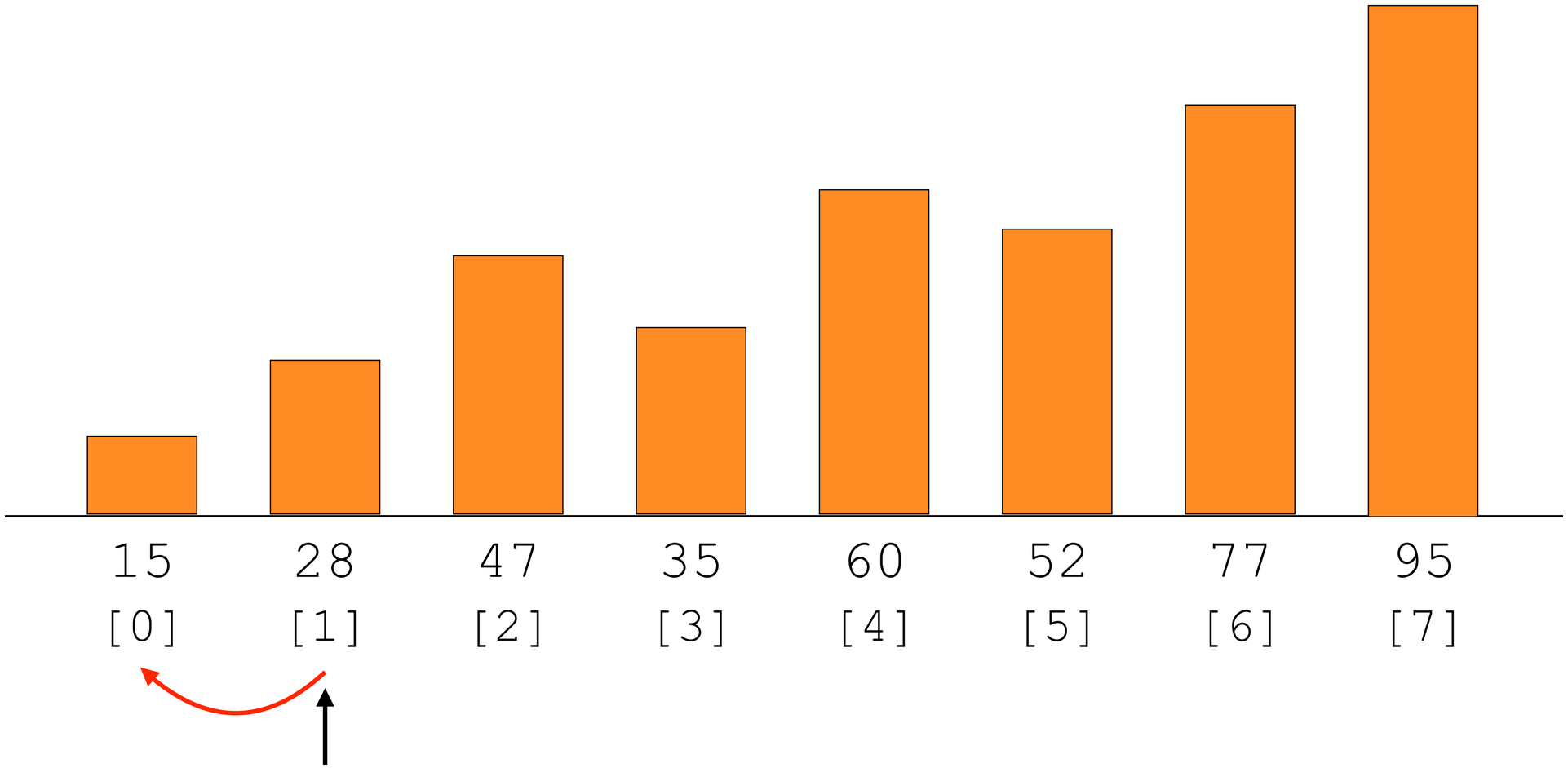
希尔排序

step 1



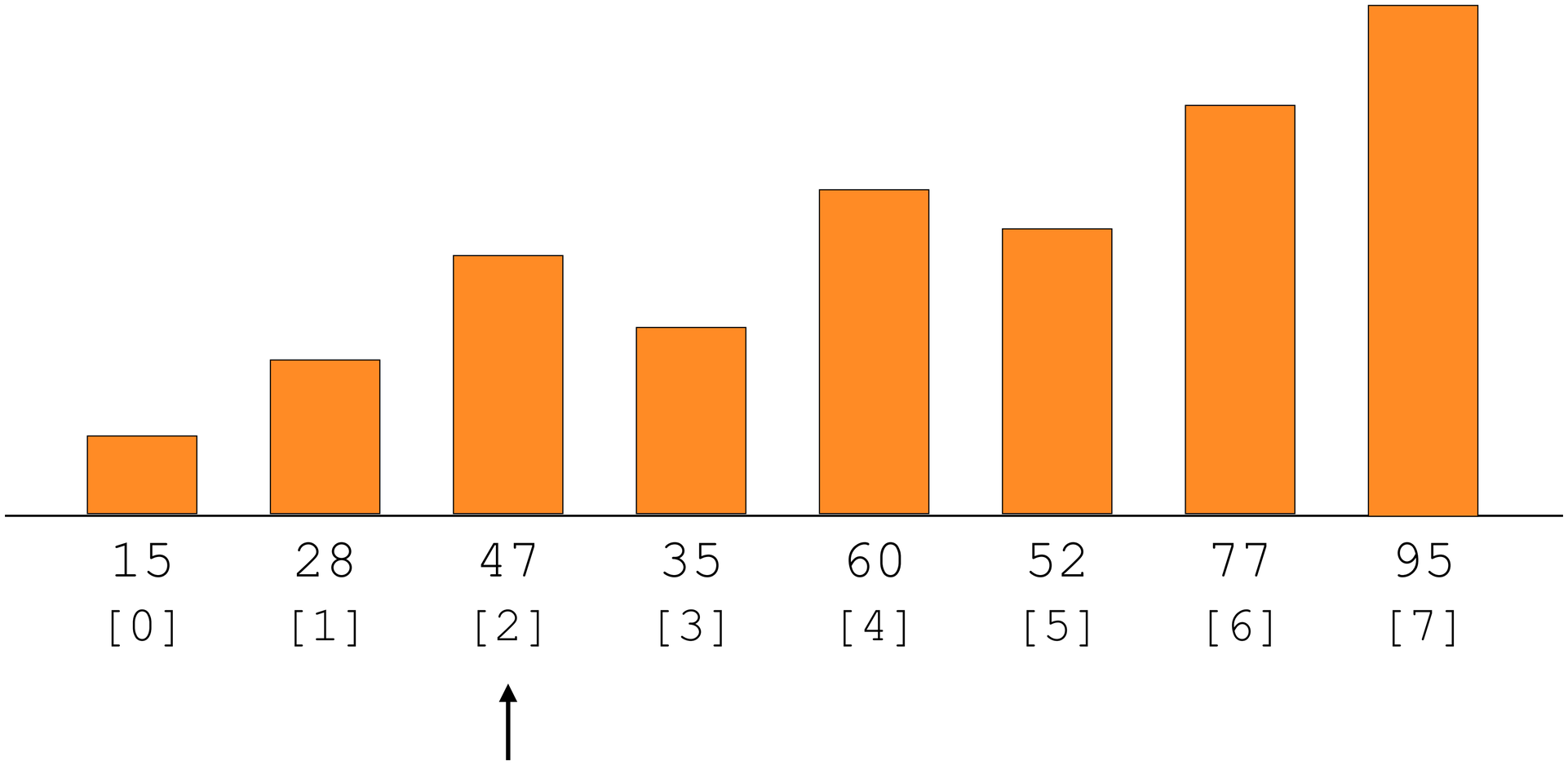
希尔排序

step 1



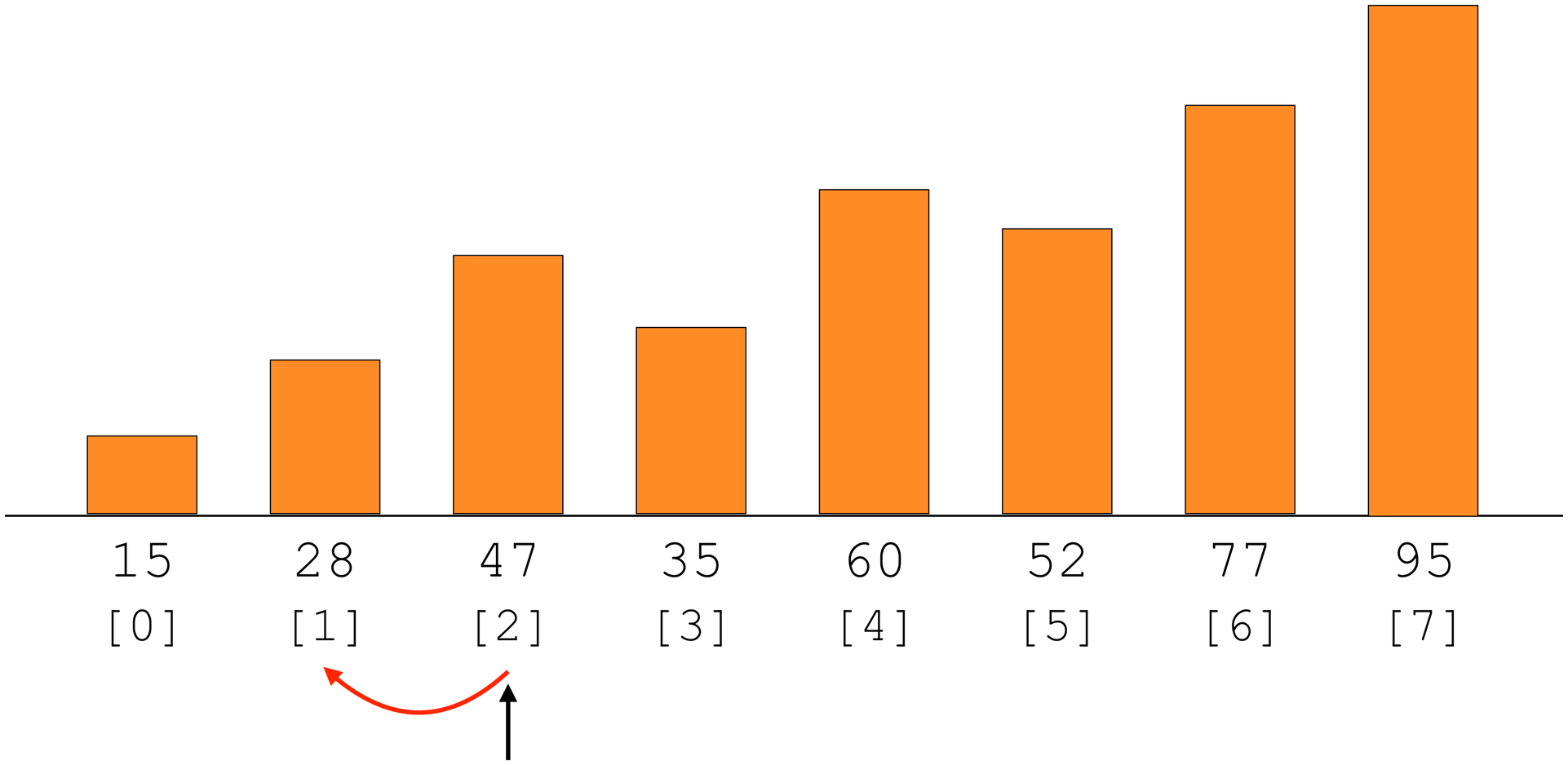
希尔排序

step 1



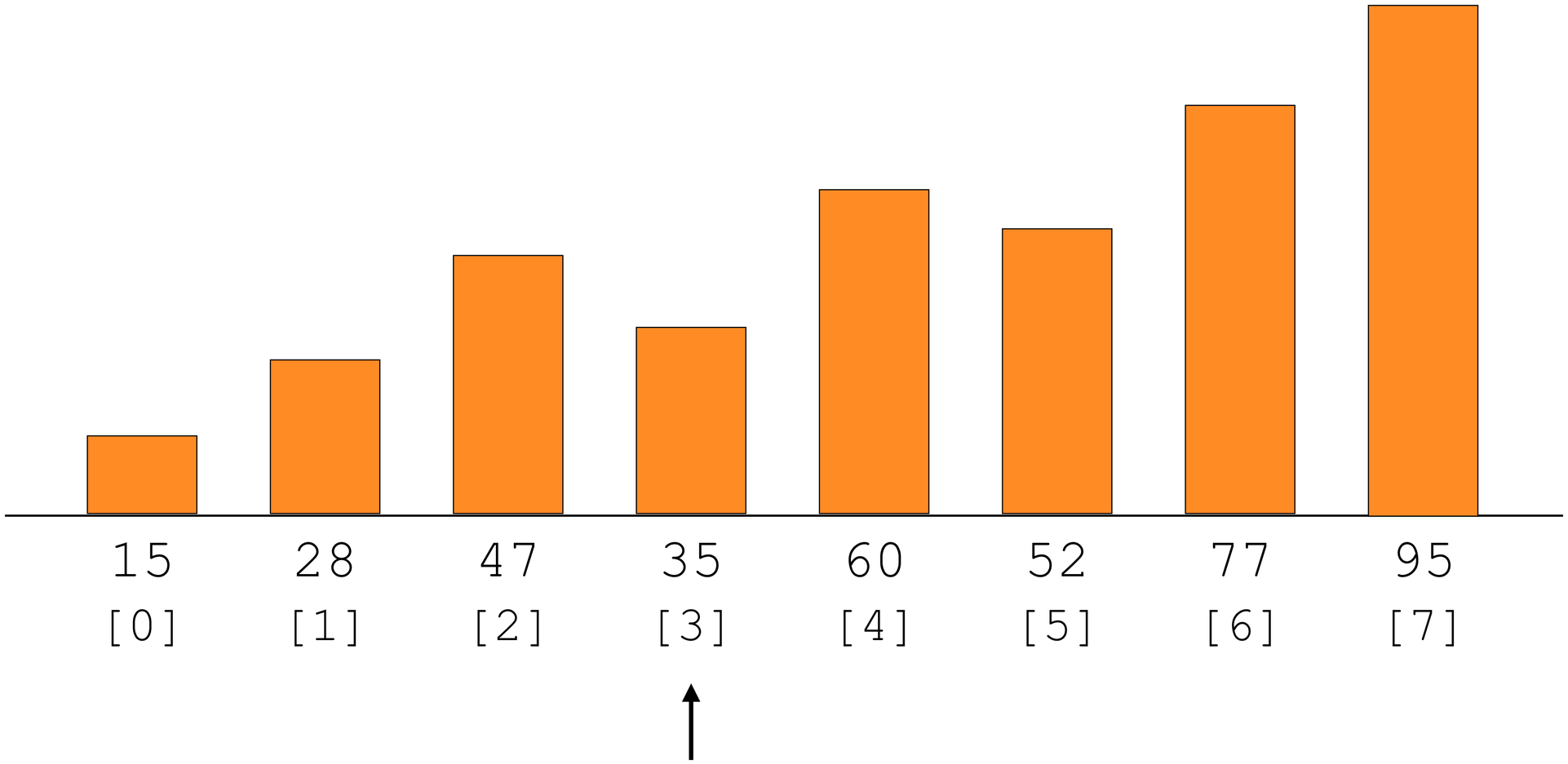
希尔排序

step 1



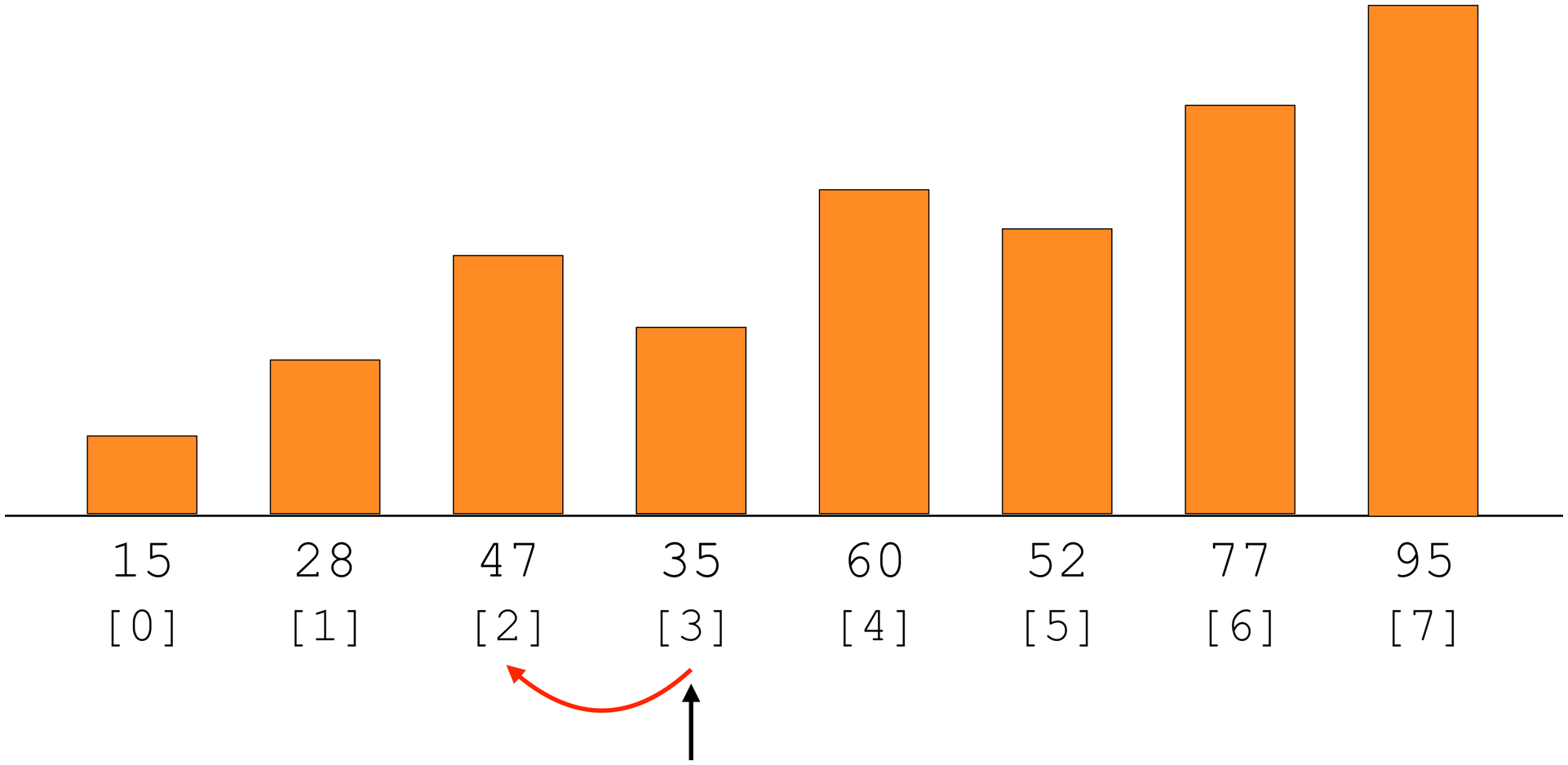
希尔排序

step 1



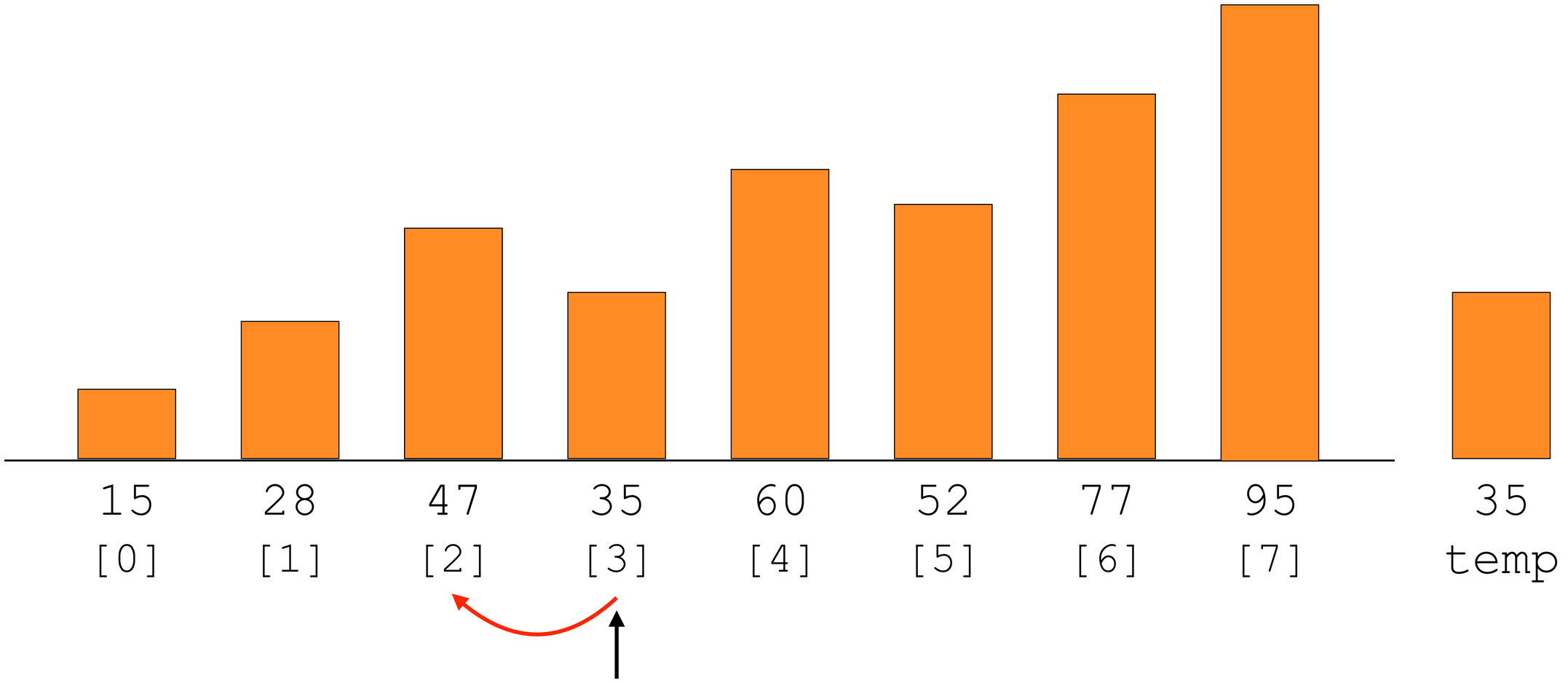
希尔排序

step 1



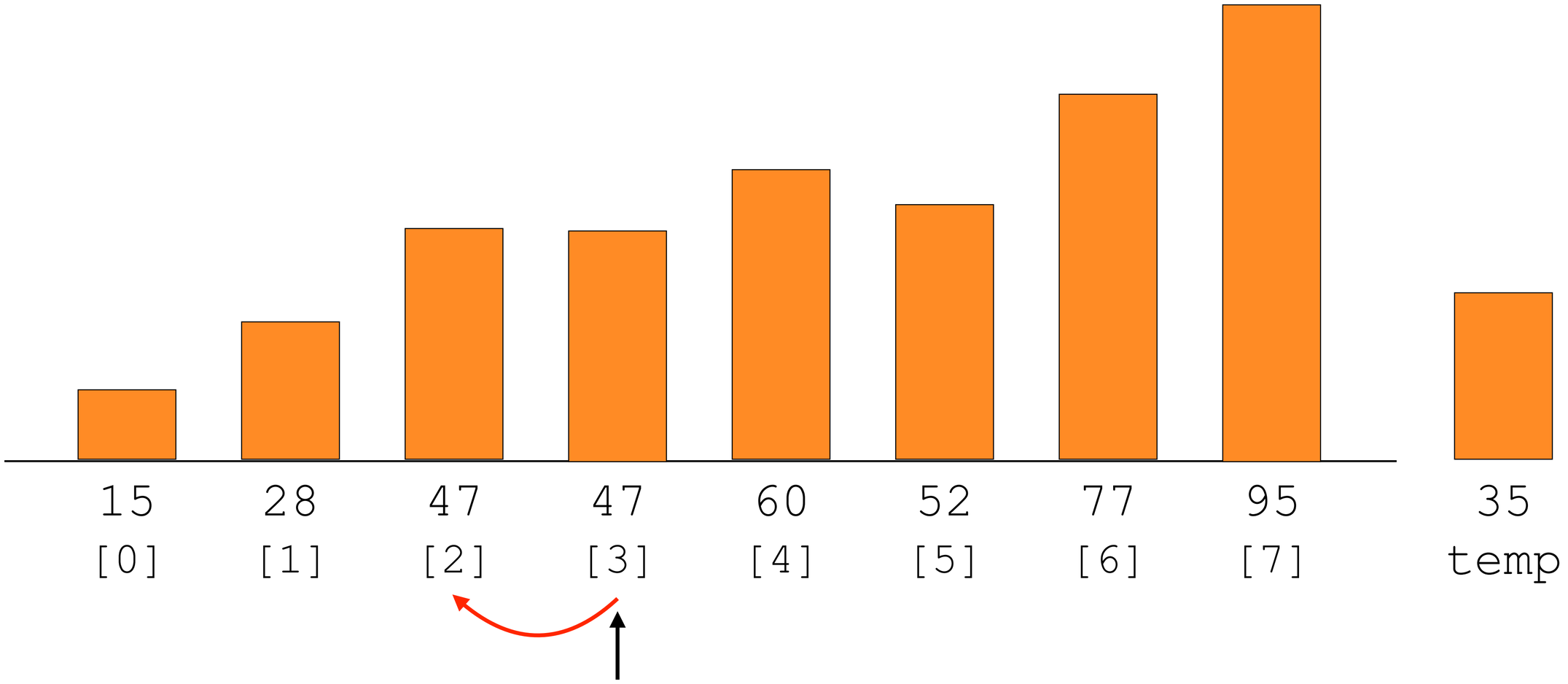
希尔排序

step 1



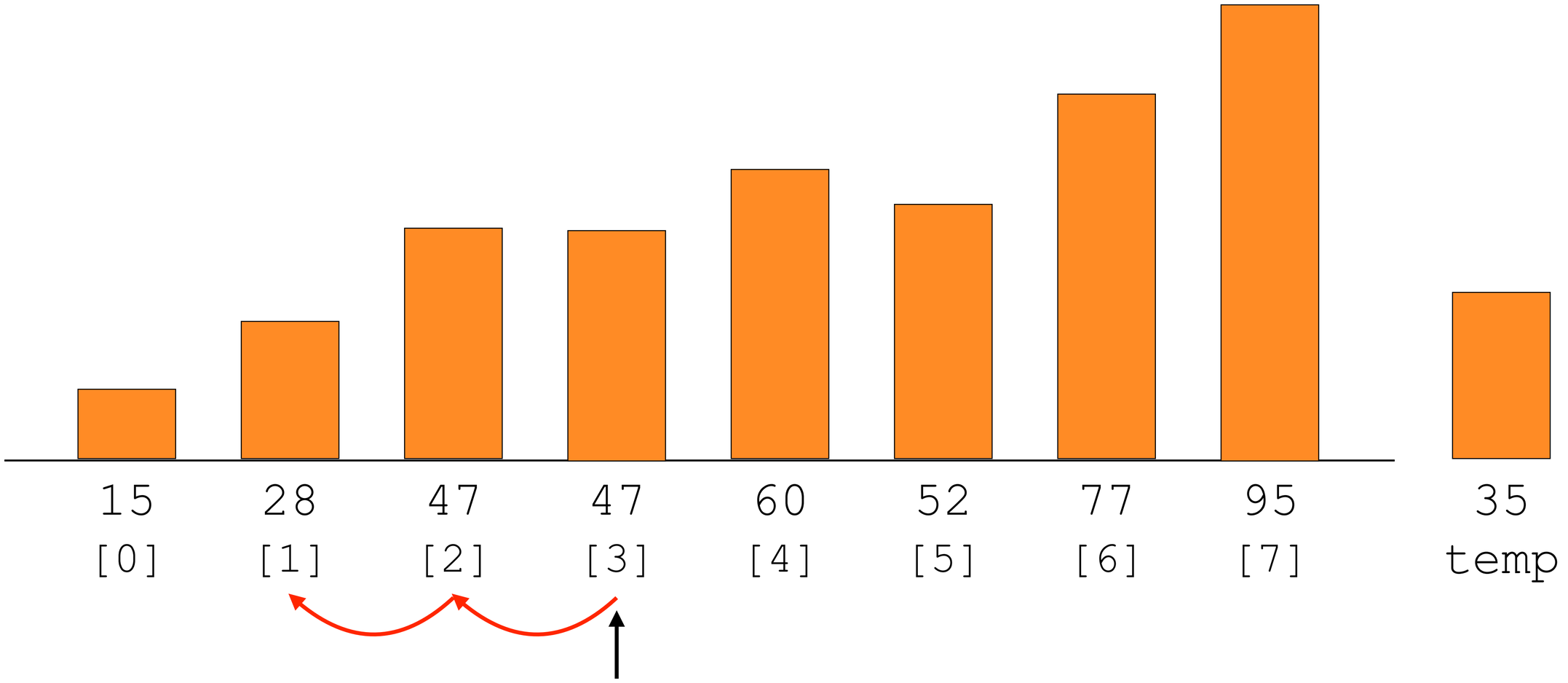
希尔排序

step 1



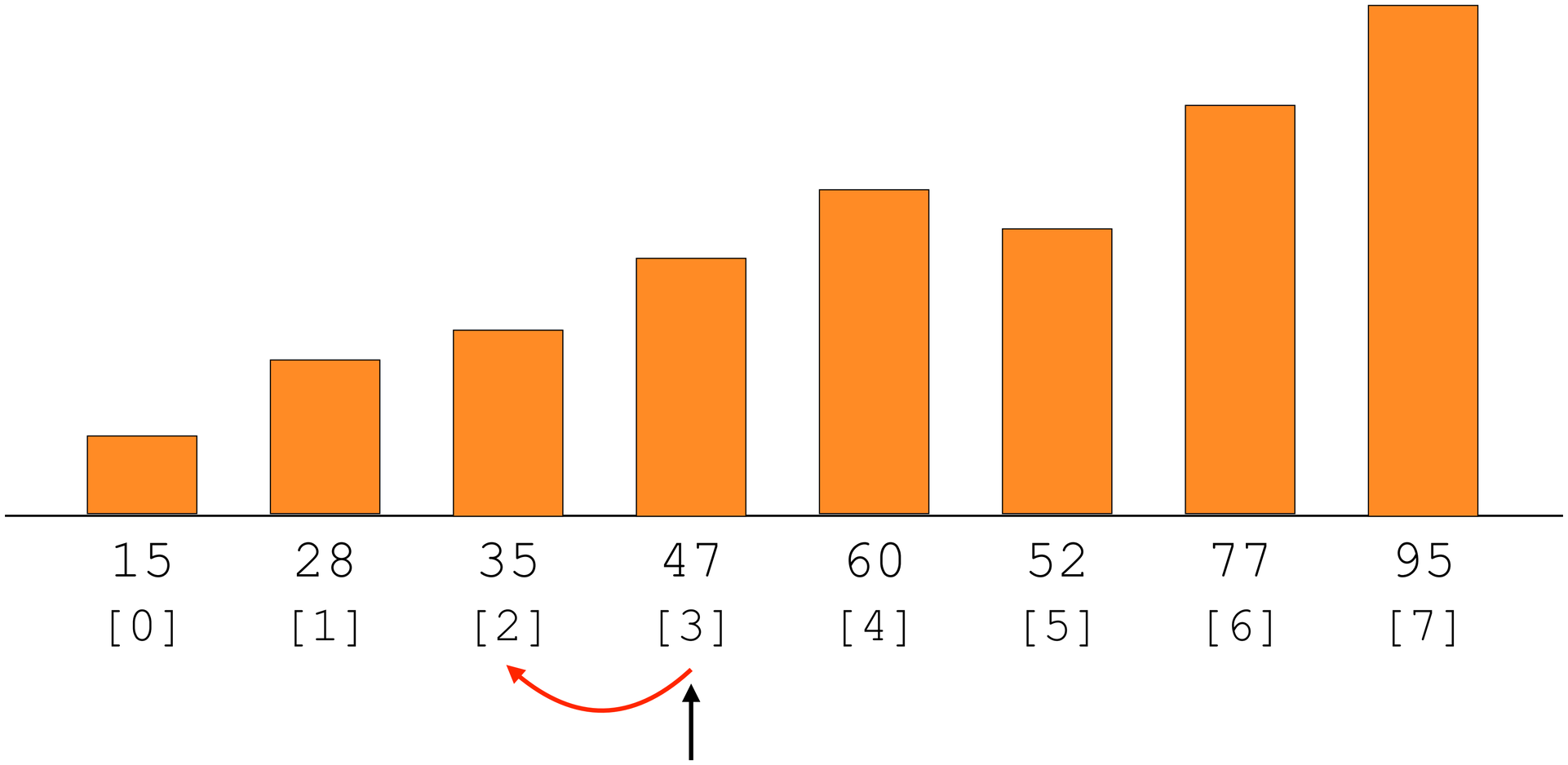
希尔排序

step 1



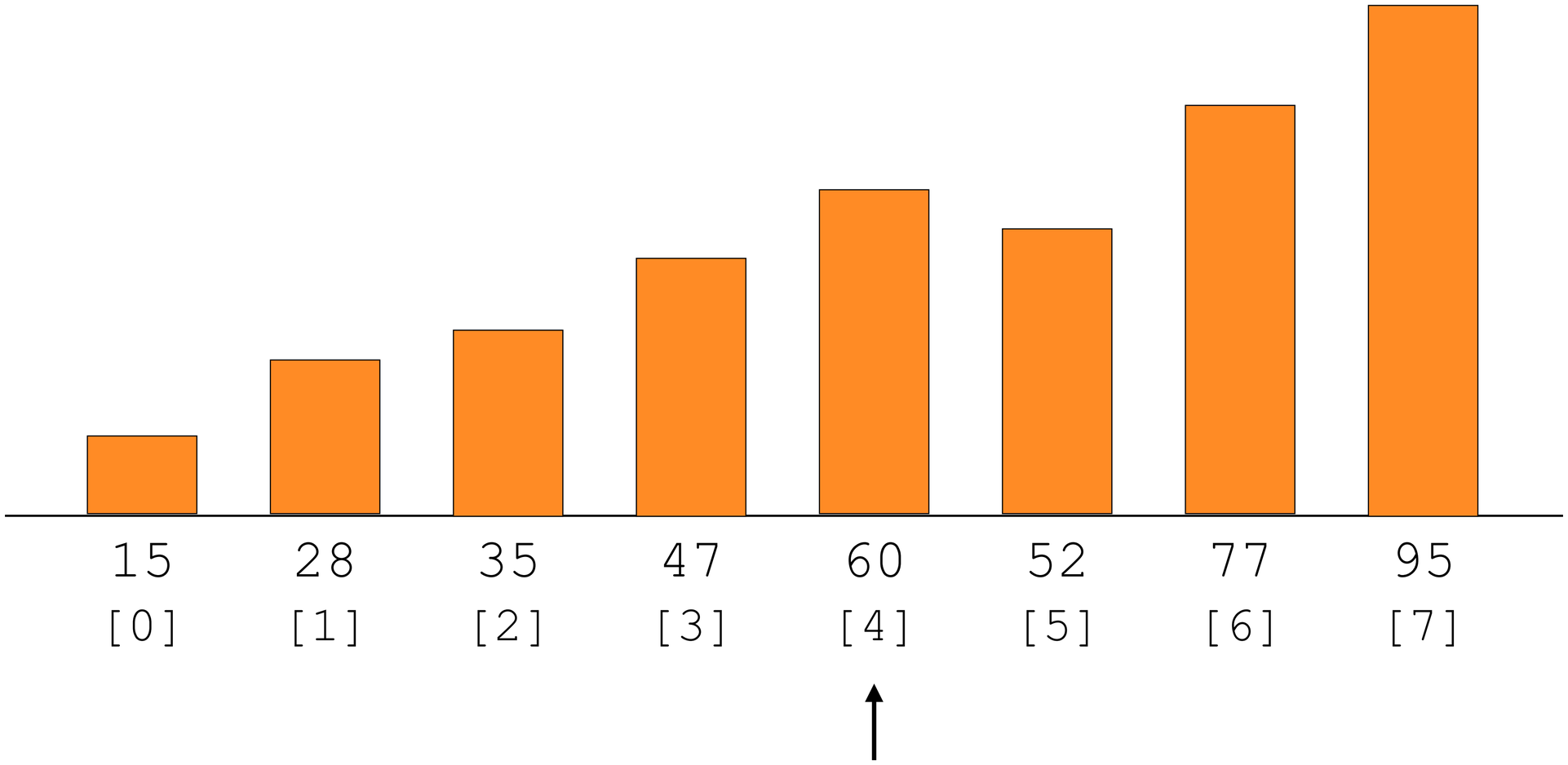
希尔排序

step 1



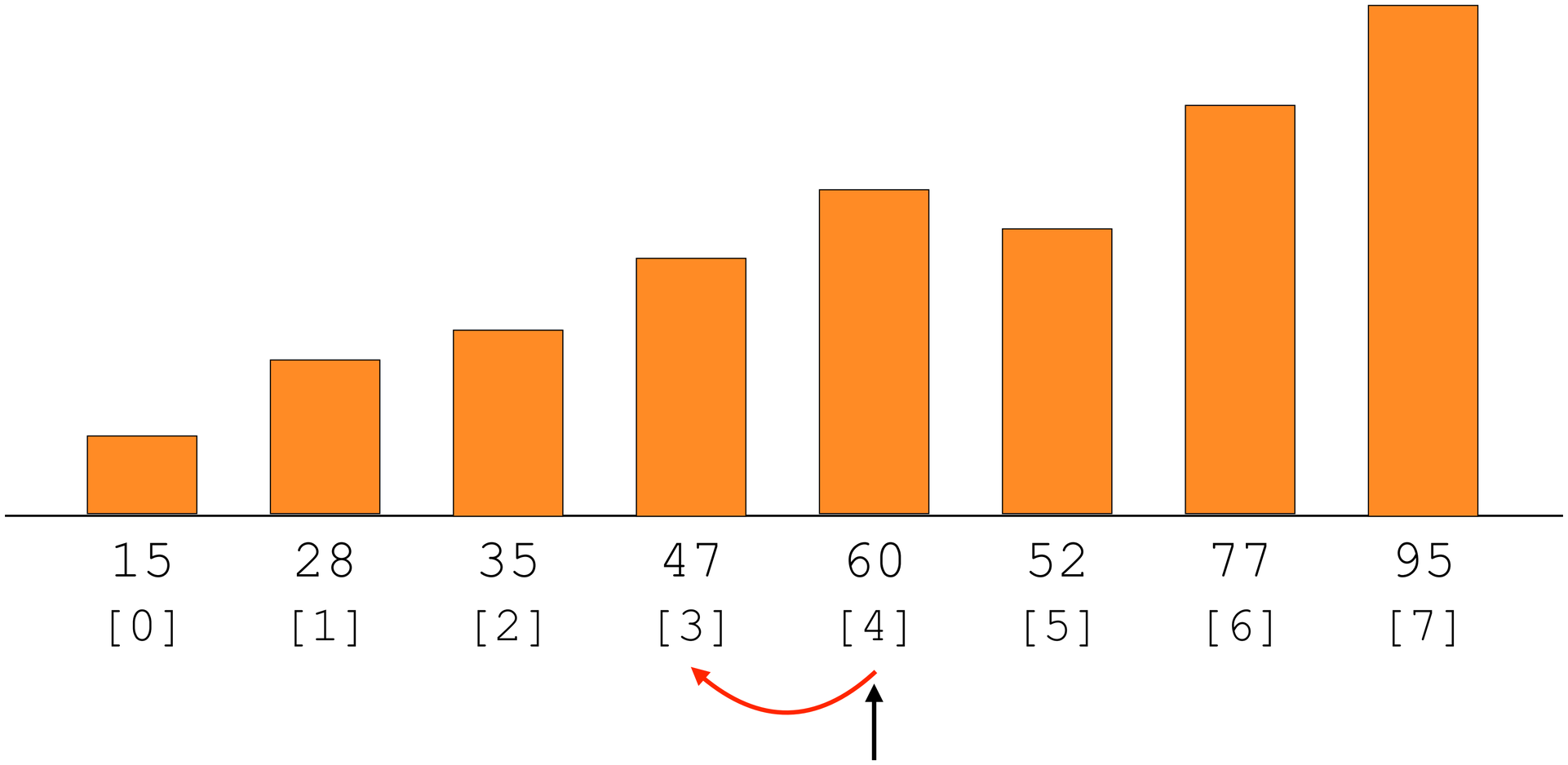
希尔排序

step 1



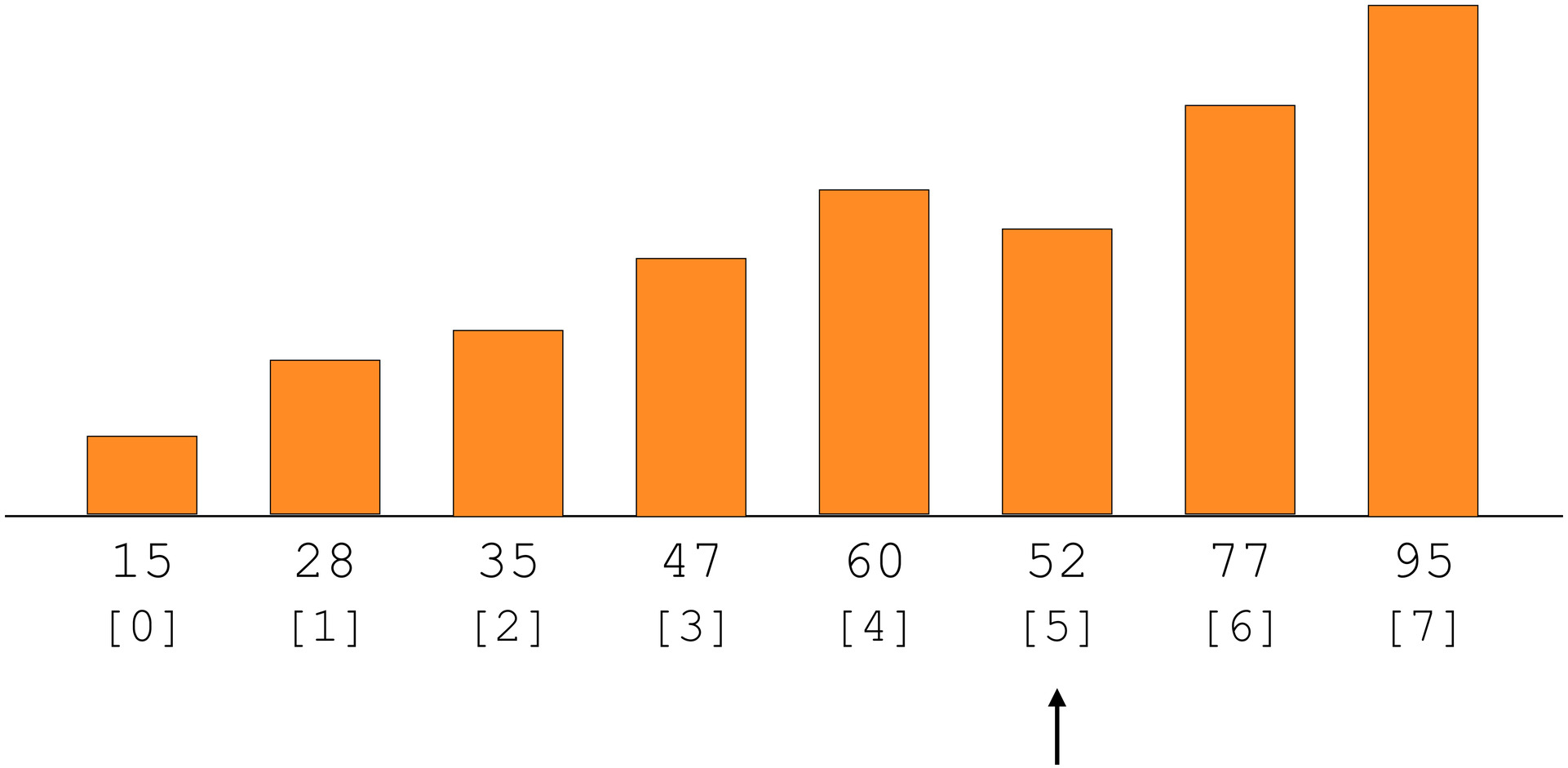
希尔排序

step 1



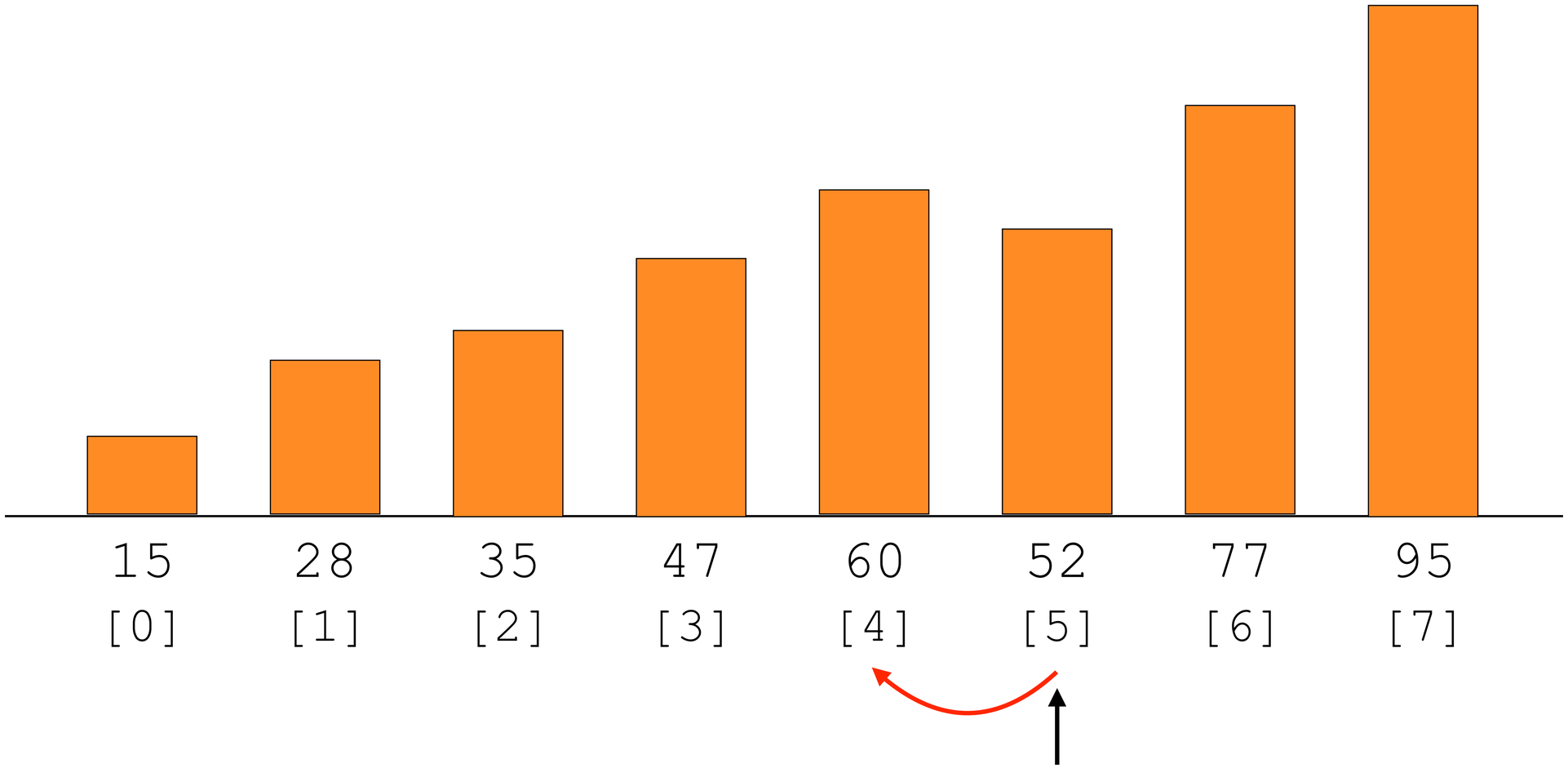
希尔排序

step 1



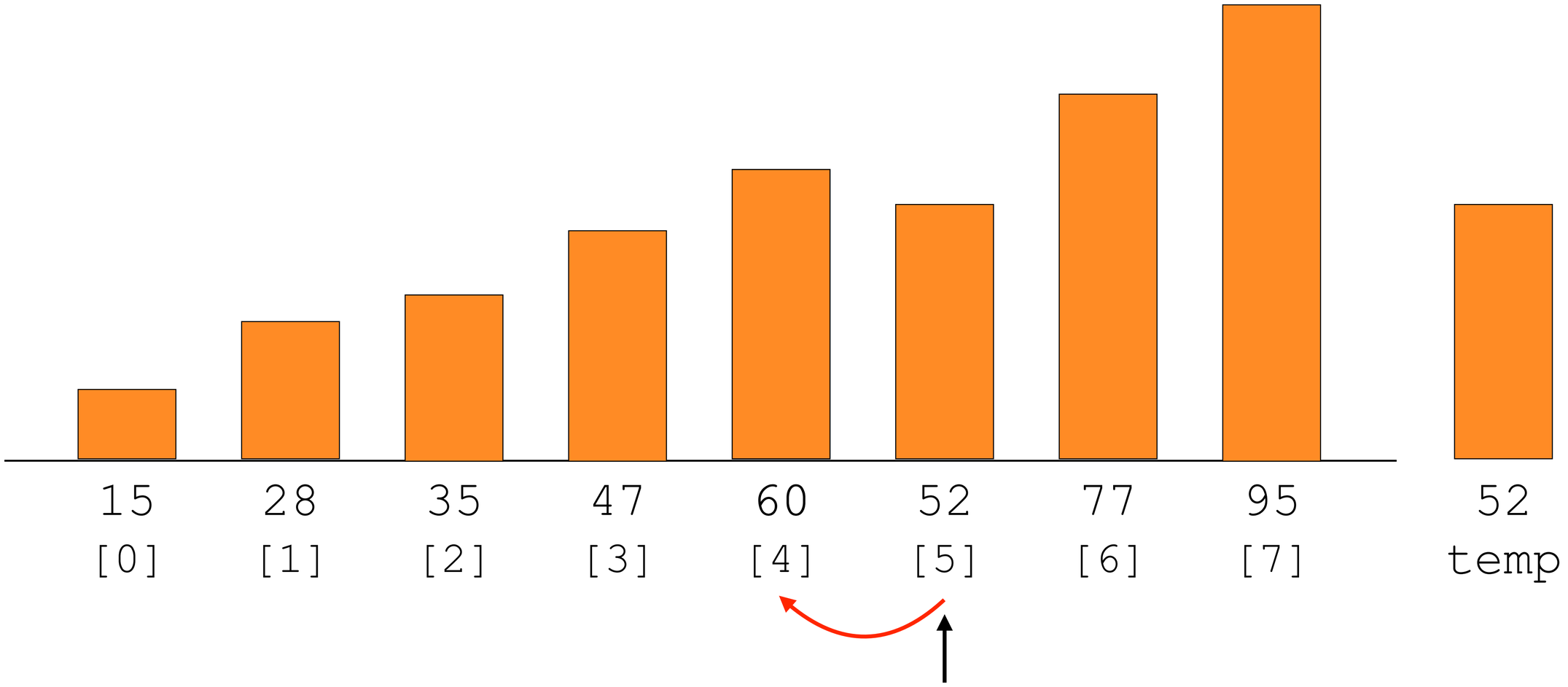
希尔排序

step 1



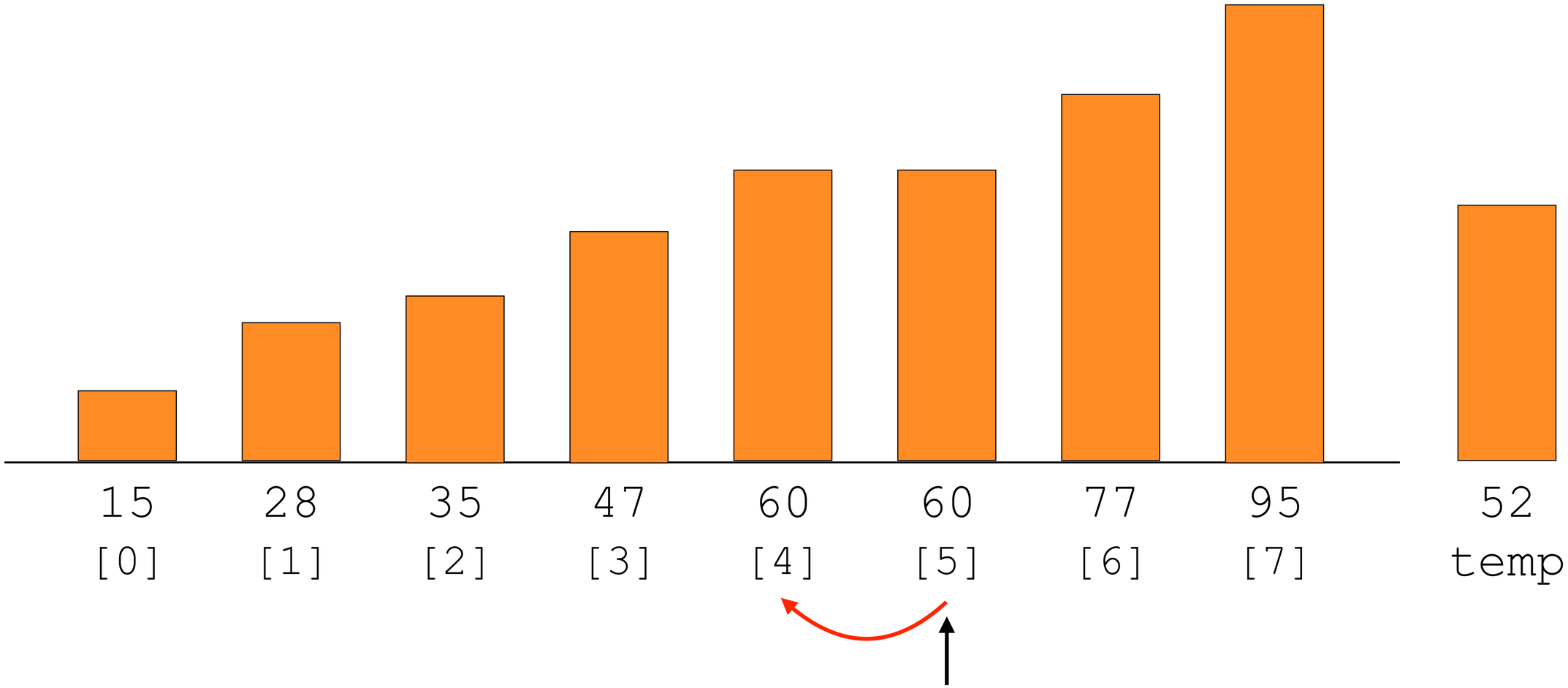
希尔排序

step 1



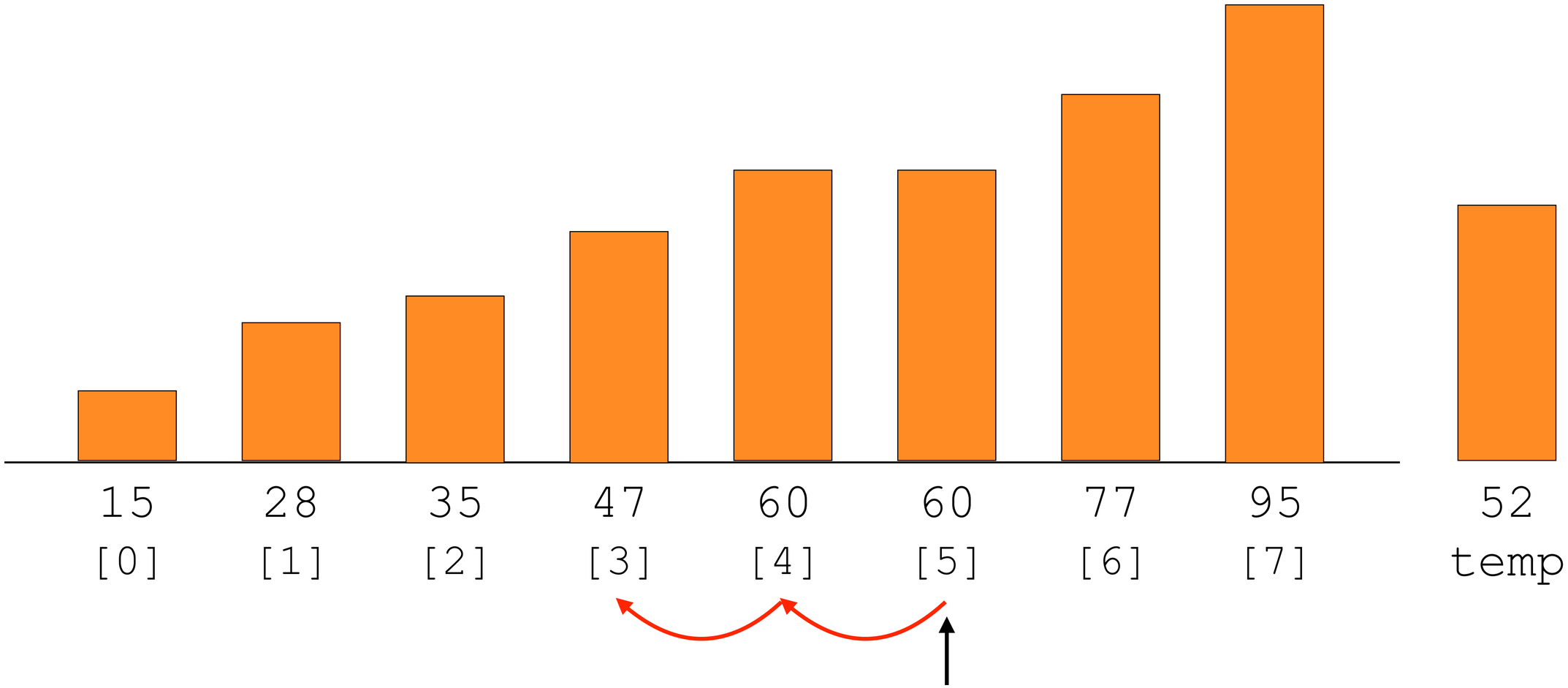
希尔排序

step 1



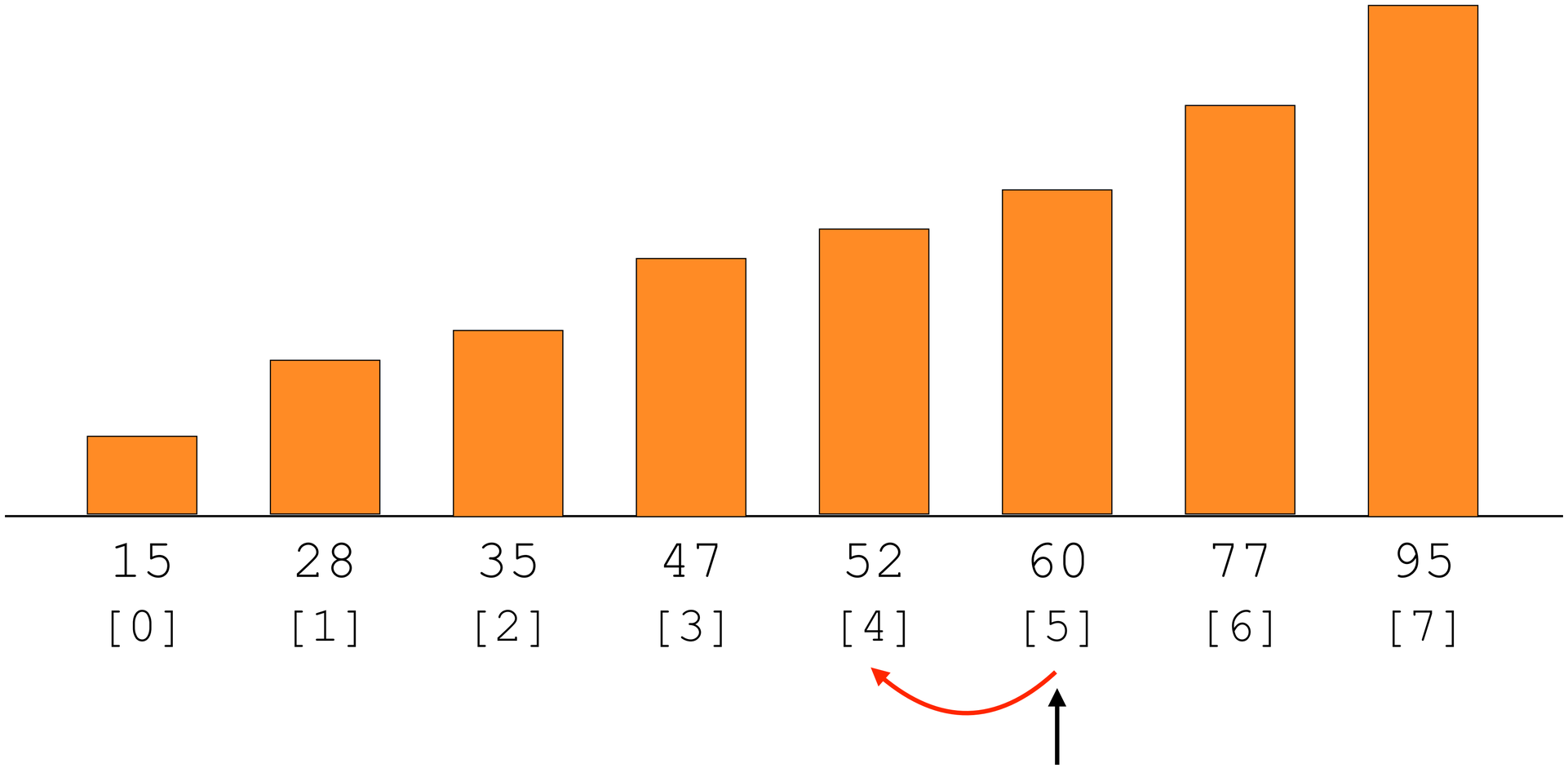
希尔排序

step 1



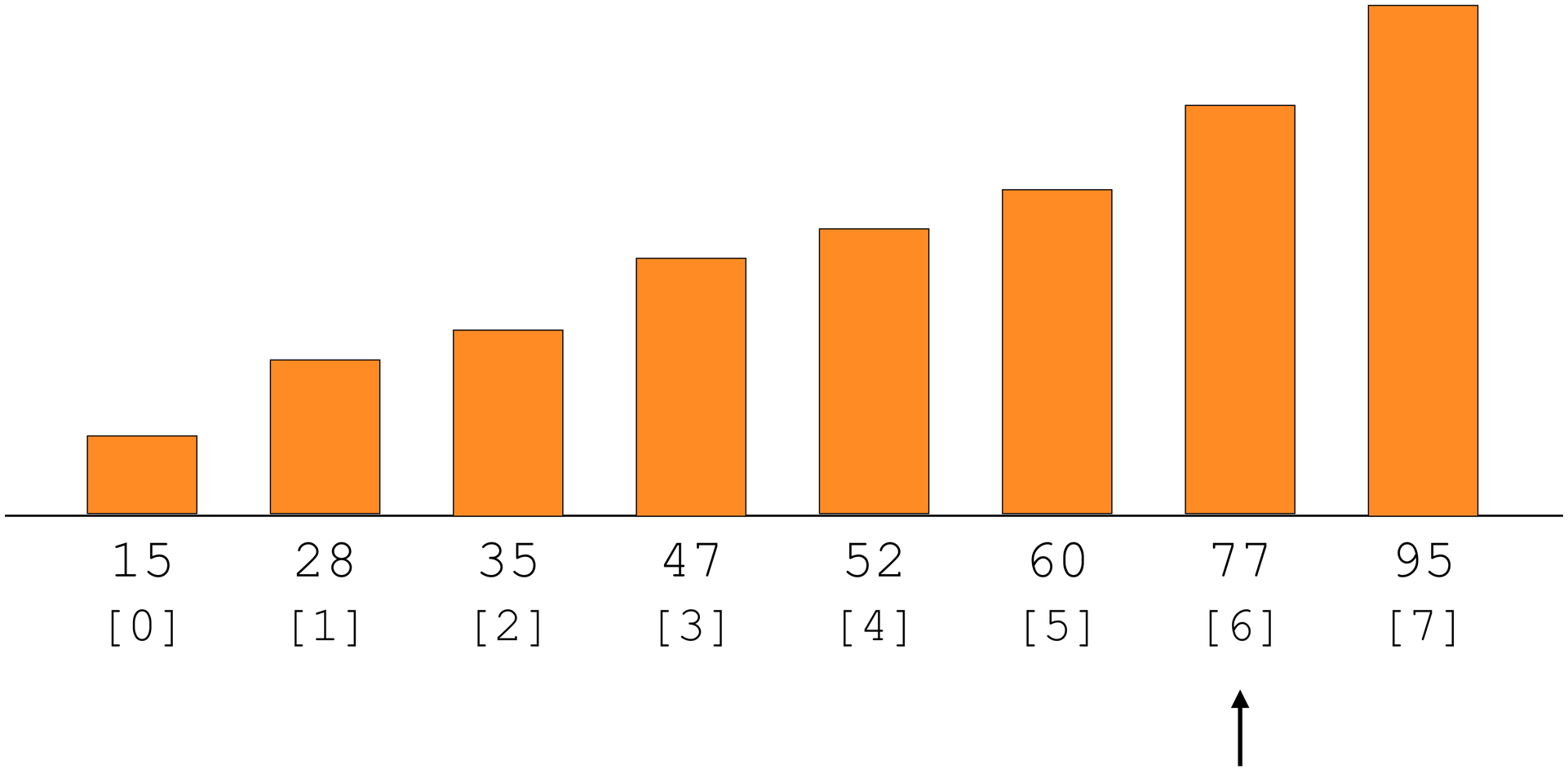
希尔排序

step 1



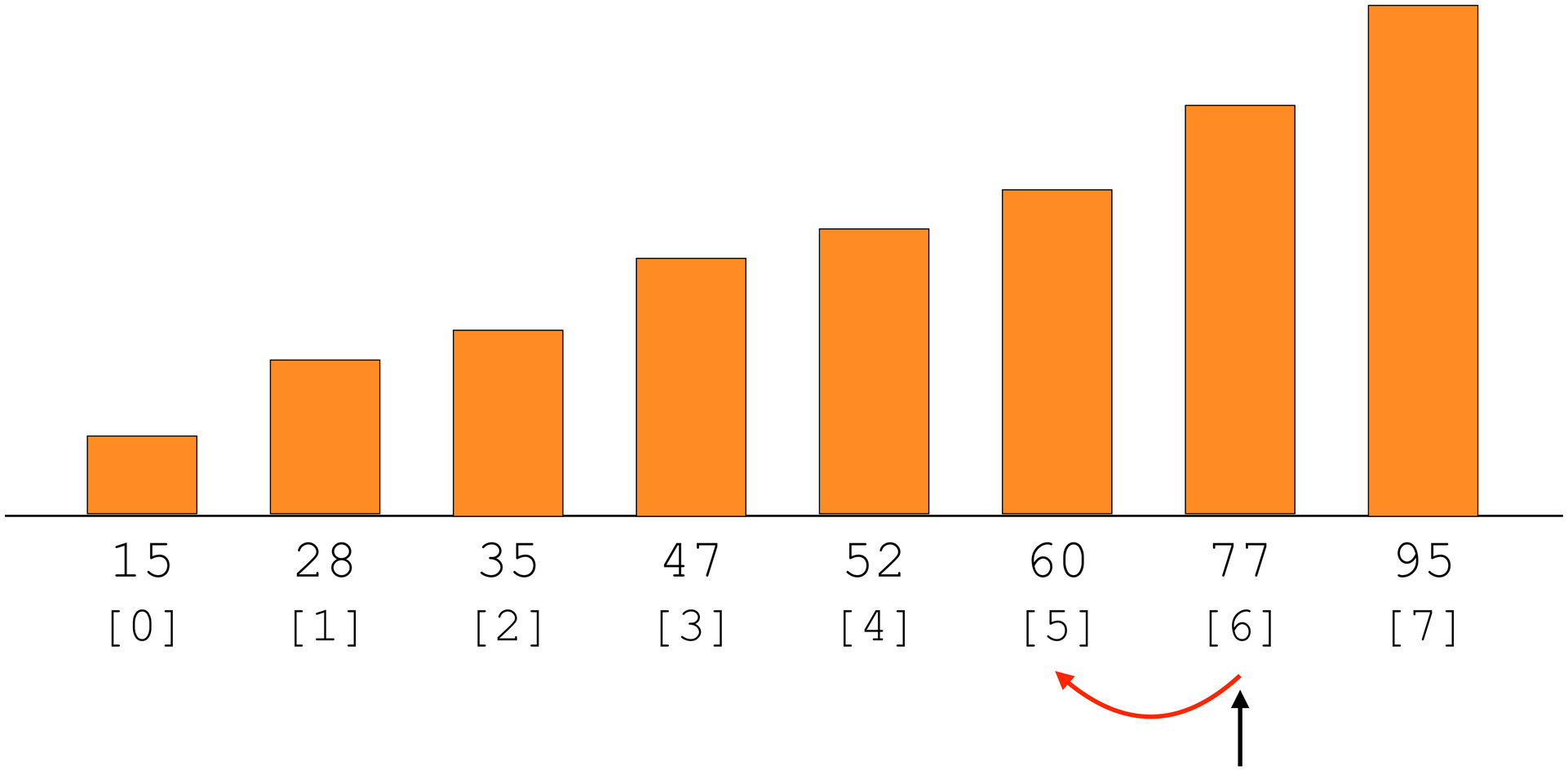
希尔排序

step 1



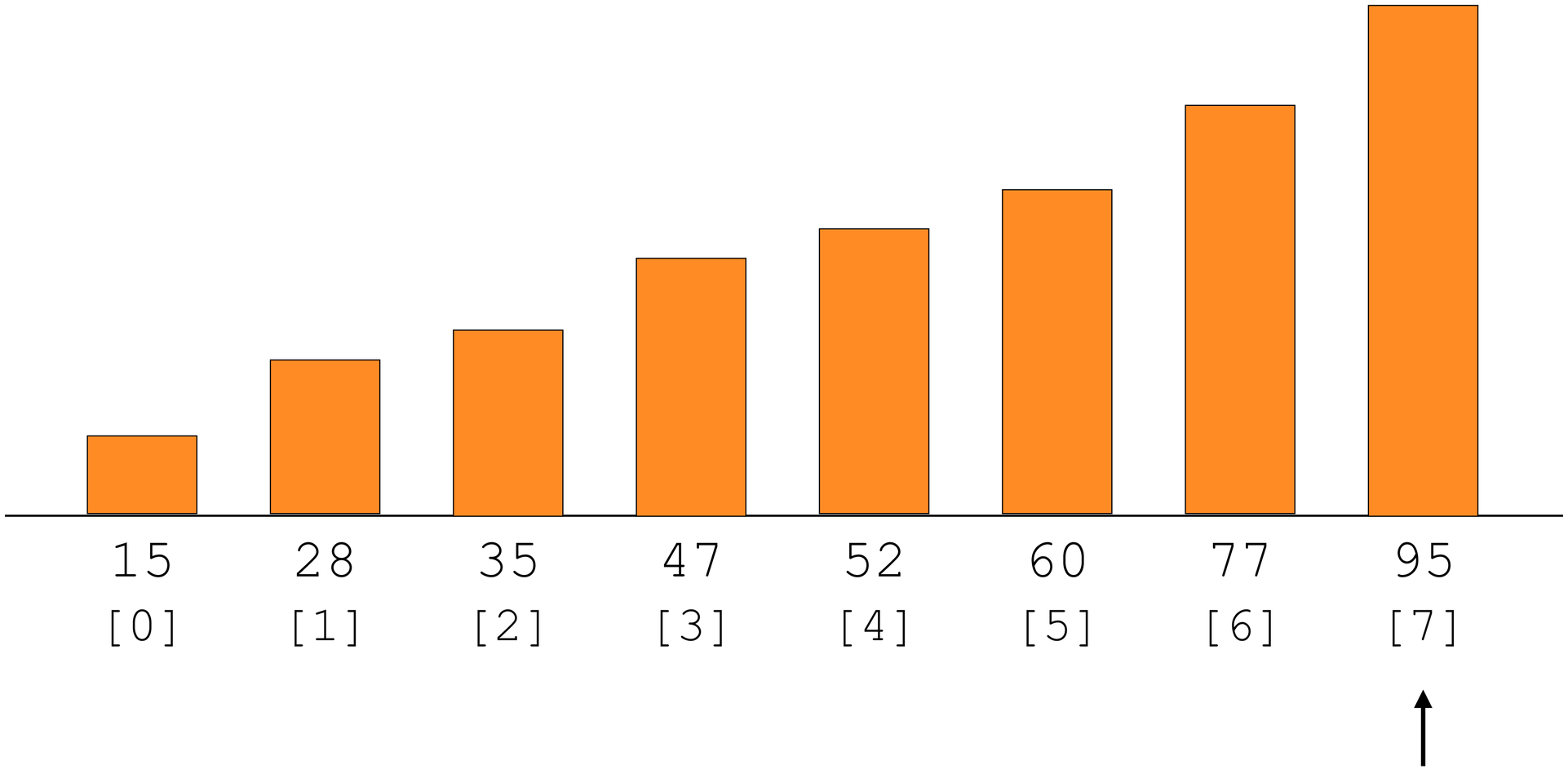
希尔排序

step 1



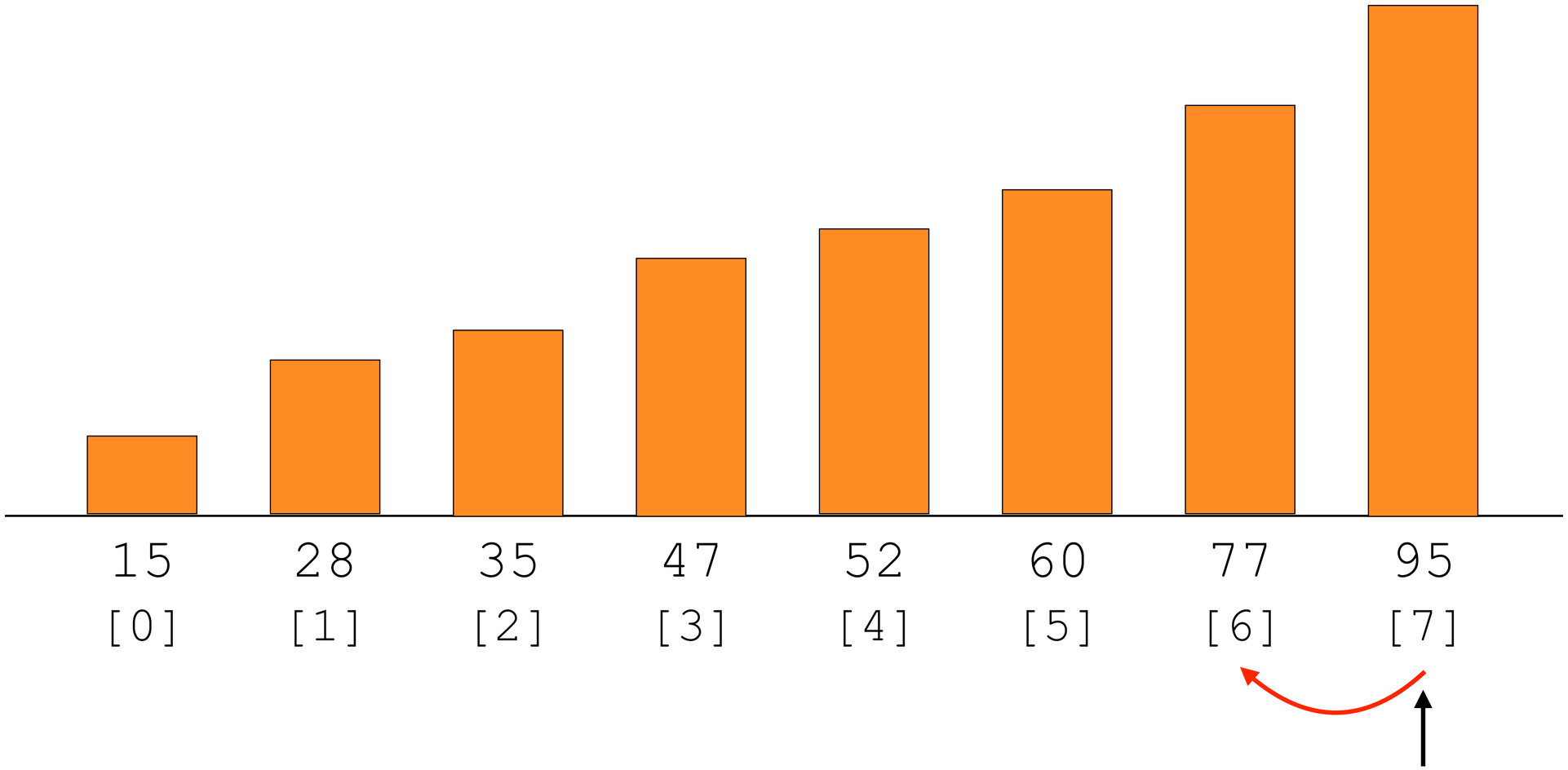
希尔排序

step 1



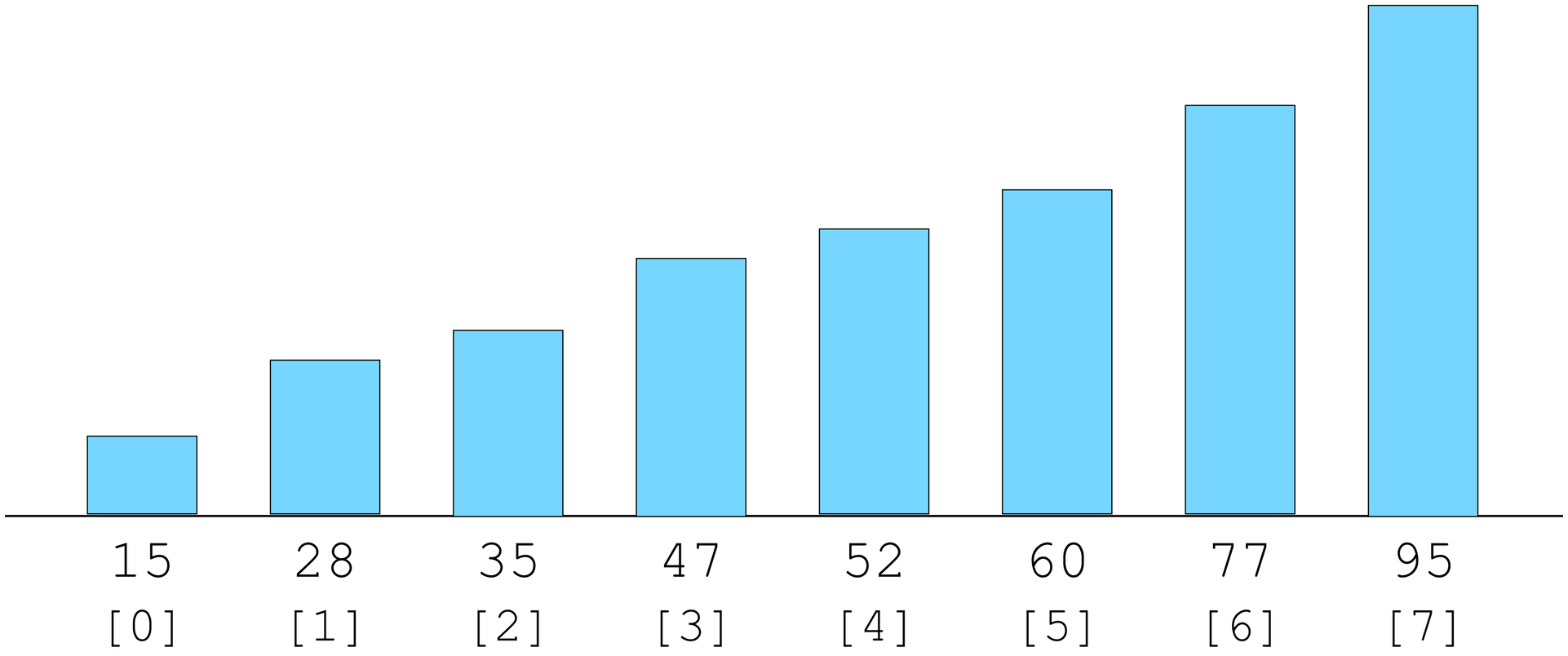
希尔排序

step 1



希尔排序

排序 / test6.c



希尔排序

步长的选取非常关键，目前还是一个数学难题，没有找到一种最好的步长序列

注意：最后一个步长必须为 1

希尔排序

维度	描述
时间复杂度	$O(n^{1+k})$ $(0<k<1)$
空间复杂度	O(1)
算法稳定性	不稳定
适用场景	适合中等规模数据排序，在不追求最优但希望实现简单、高效时是一个不错的选择

归并排序

归并排序是一种典型的“分治合”思想的应用。简单来说，它的做法是：把一个大问题，拆成小问题分别解决，再把解决后的结果合并起来。

分：就是把数组分成两半，一直分，直到每个子数组只剩一个元素。

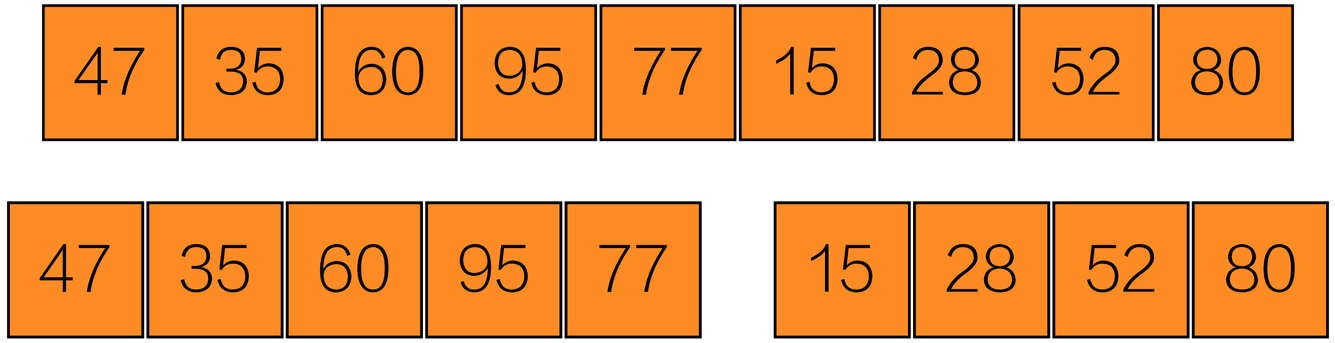
治：就是递归地对每个子数组排序。

合：把两个有序的子数组合并成一个有序数组。

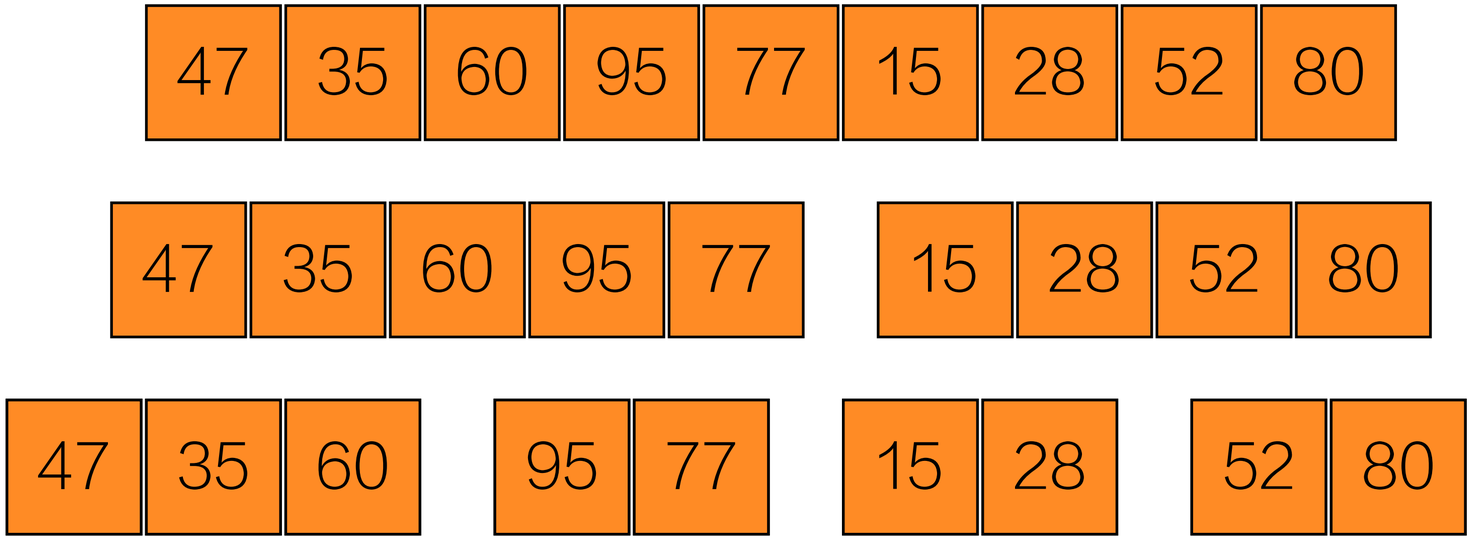
归并排序



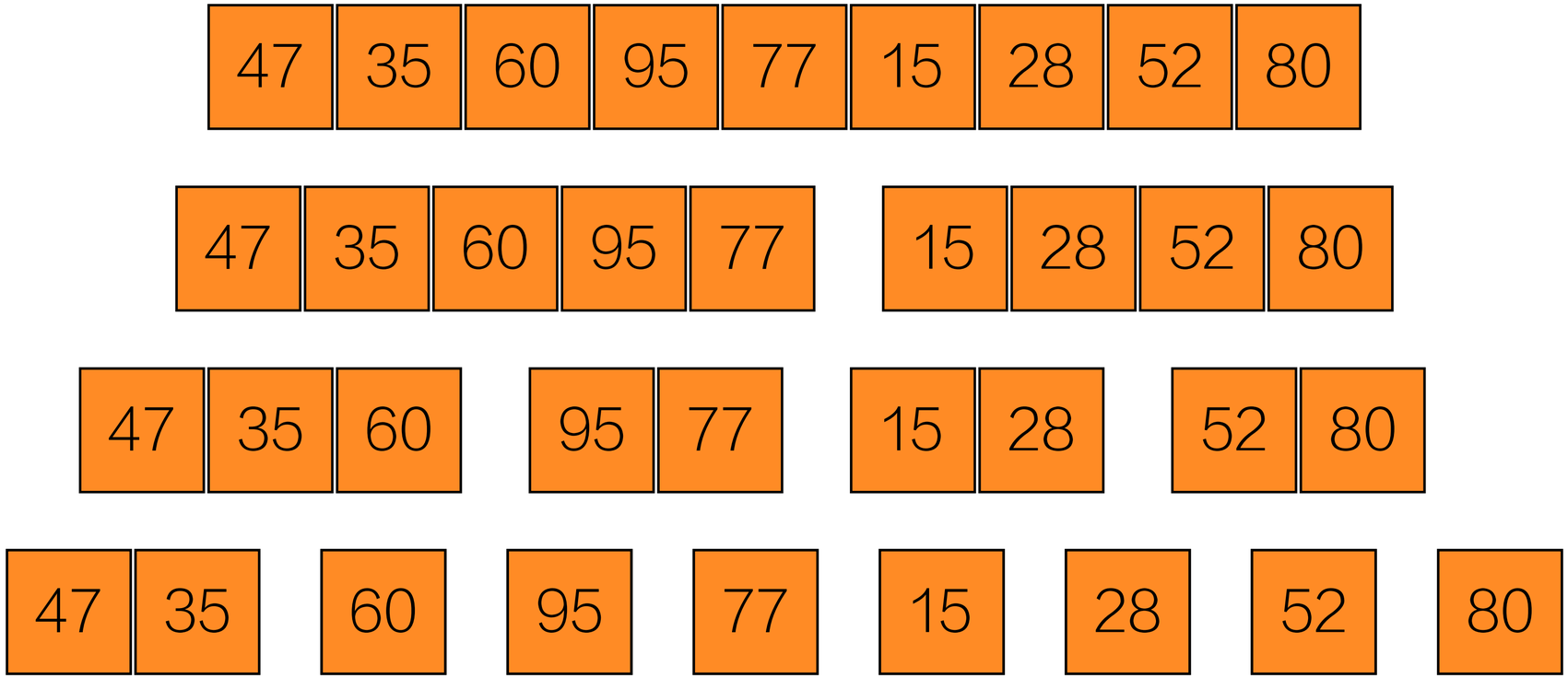
归并排序



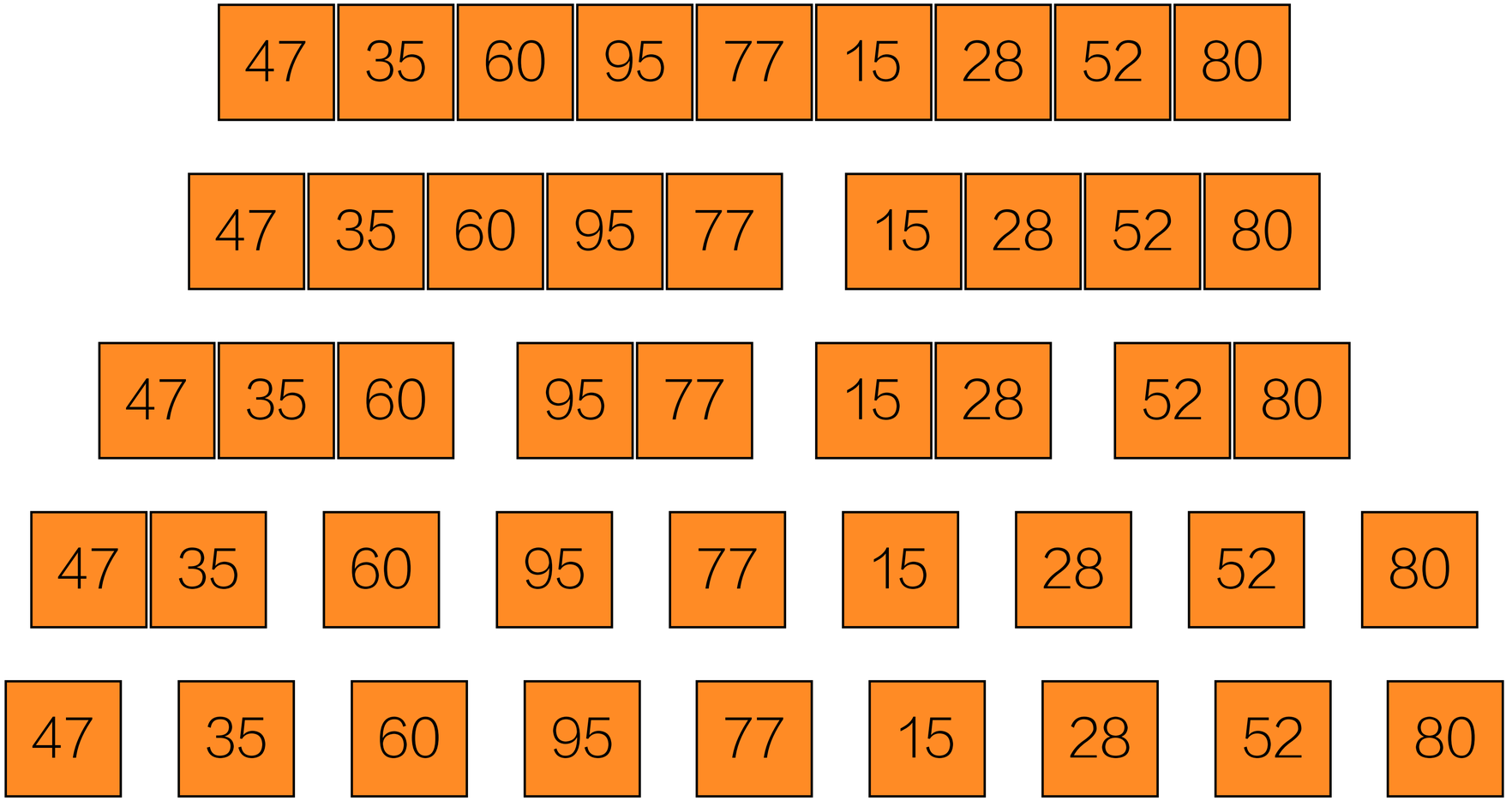
归并排序


47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

归并排序


47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

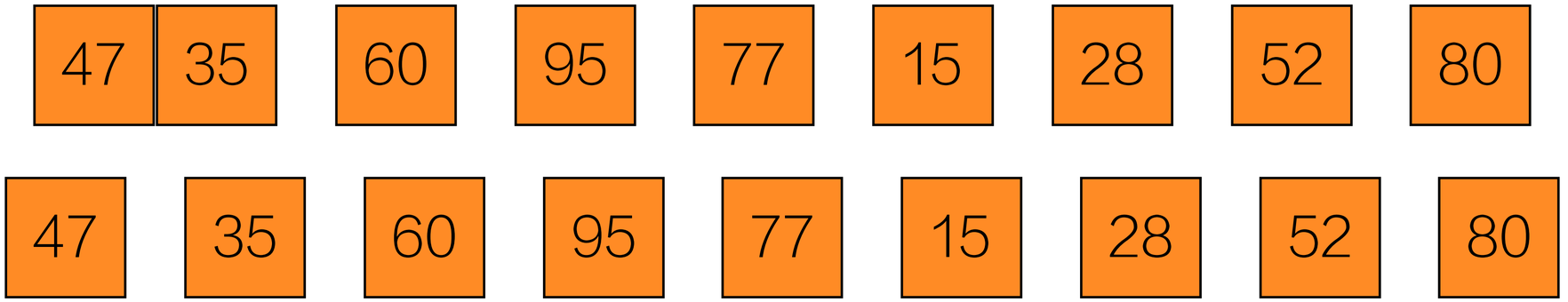
归并排序


47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

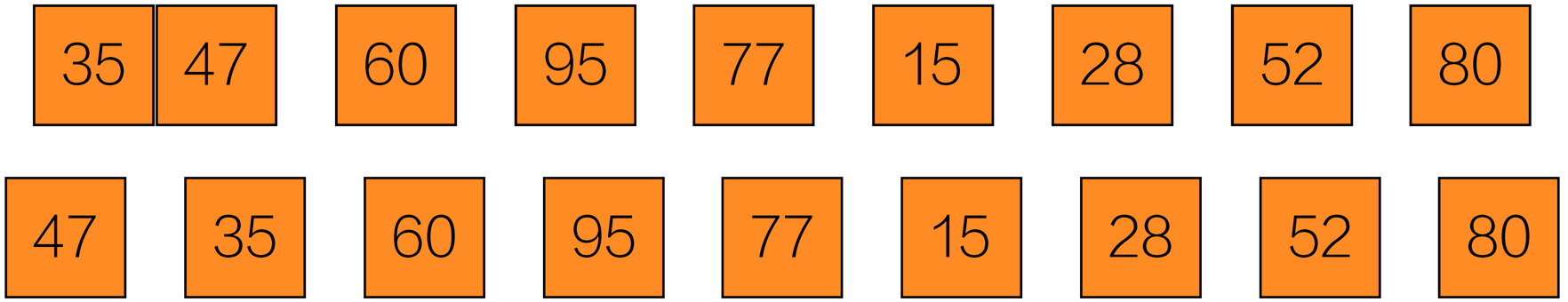
归并排序



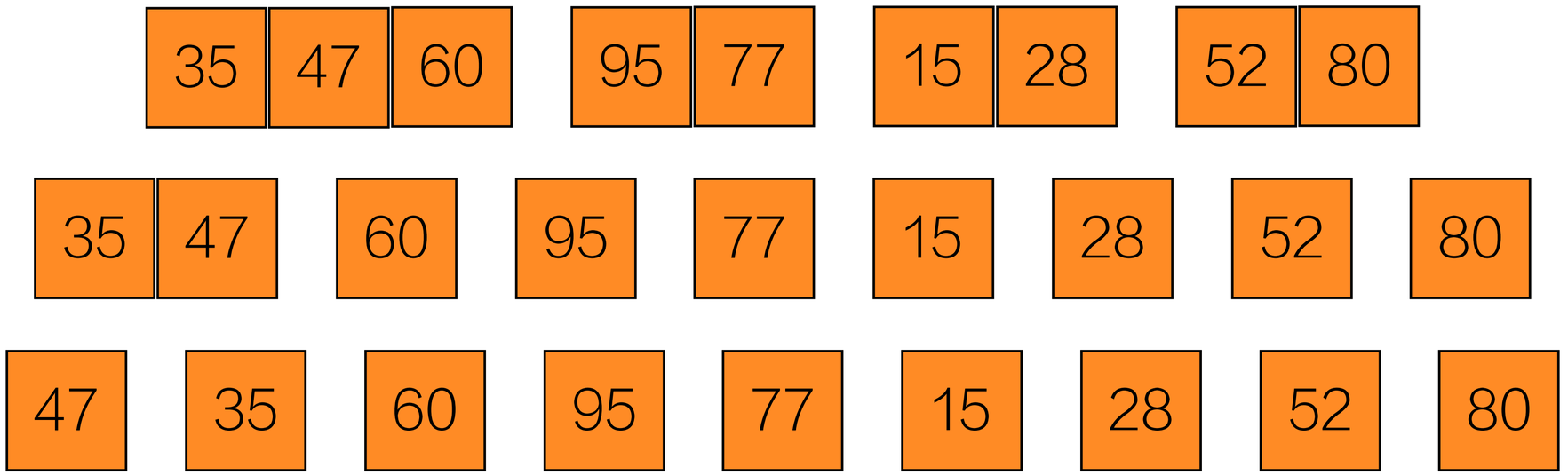
归并排序



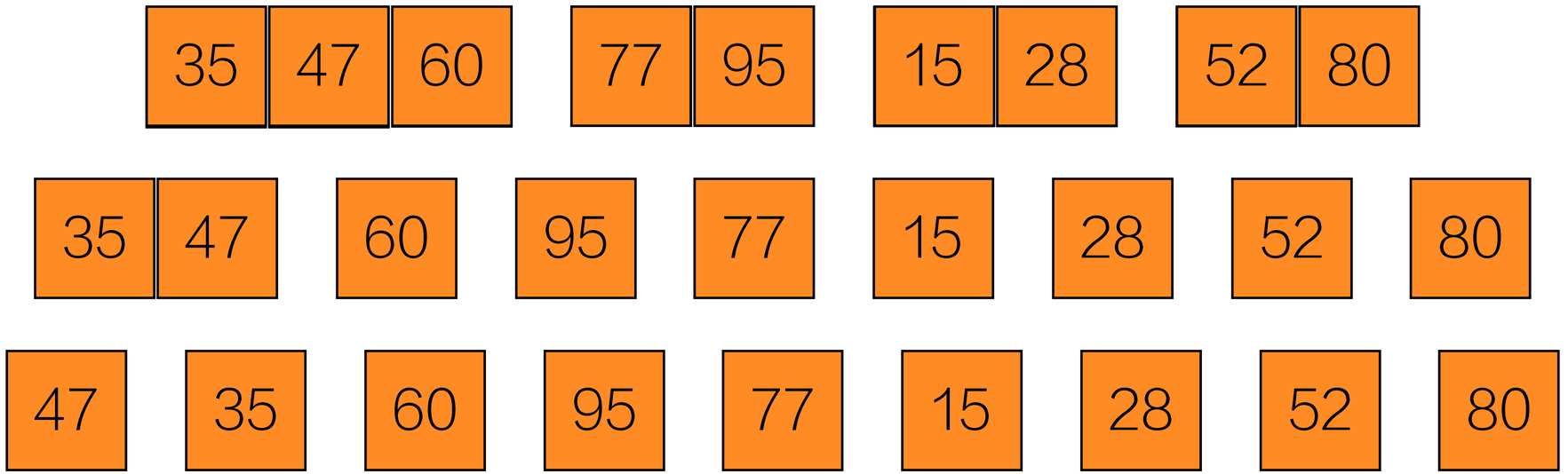
归并排序



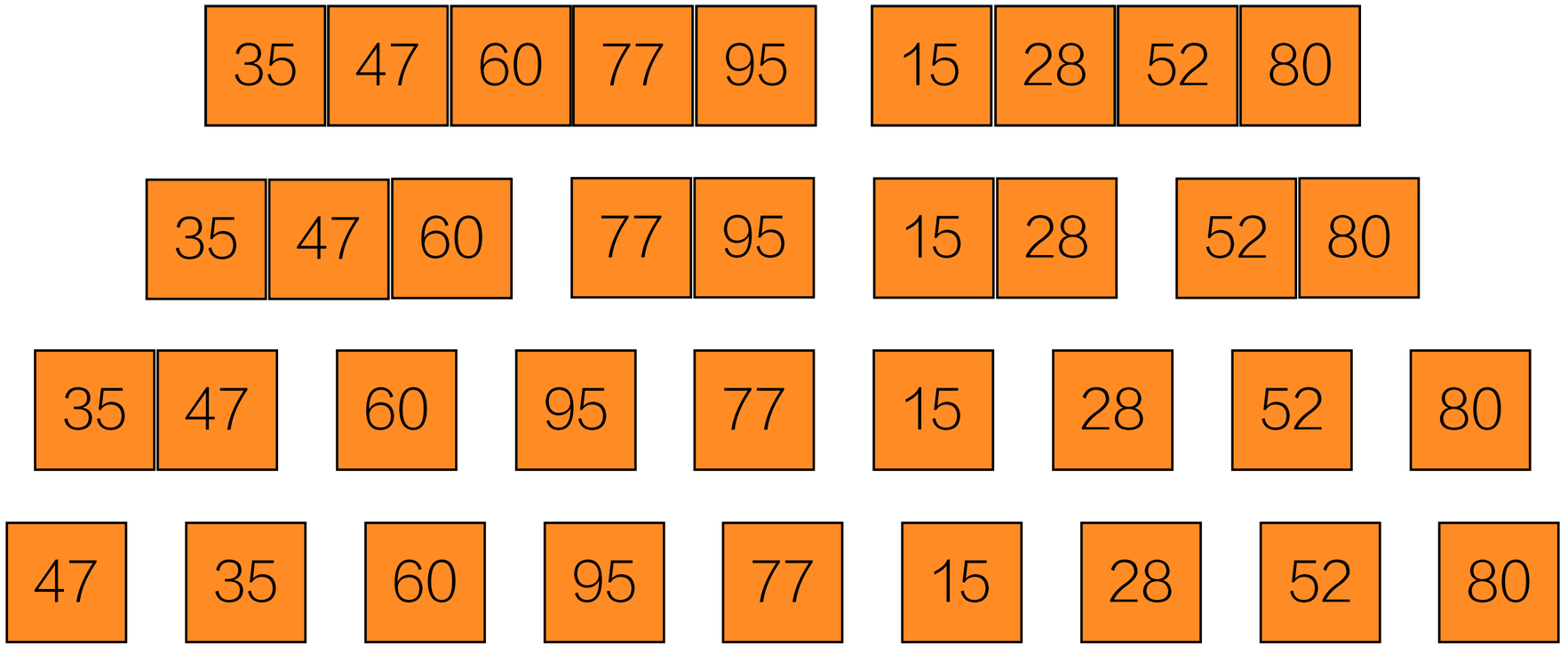
归并排序


35 47 60 95 77 15 28 52 80
35 47 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

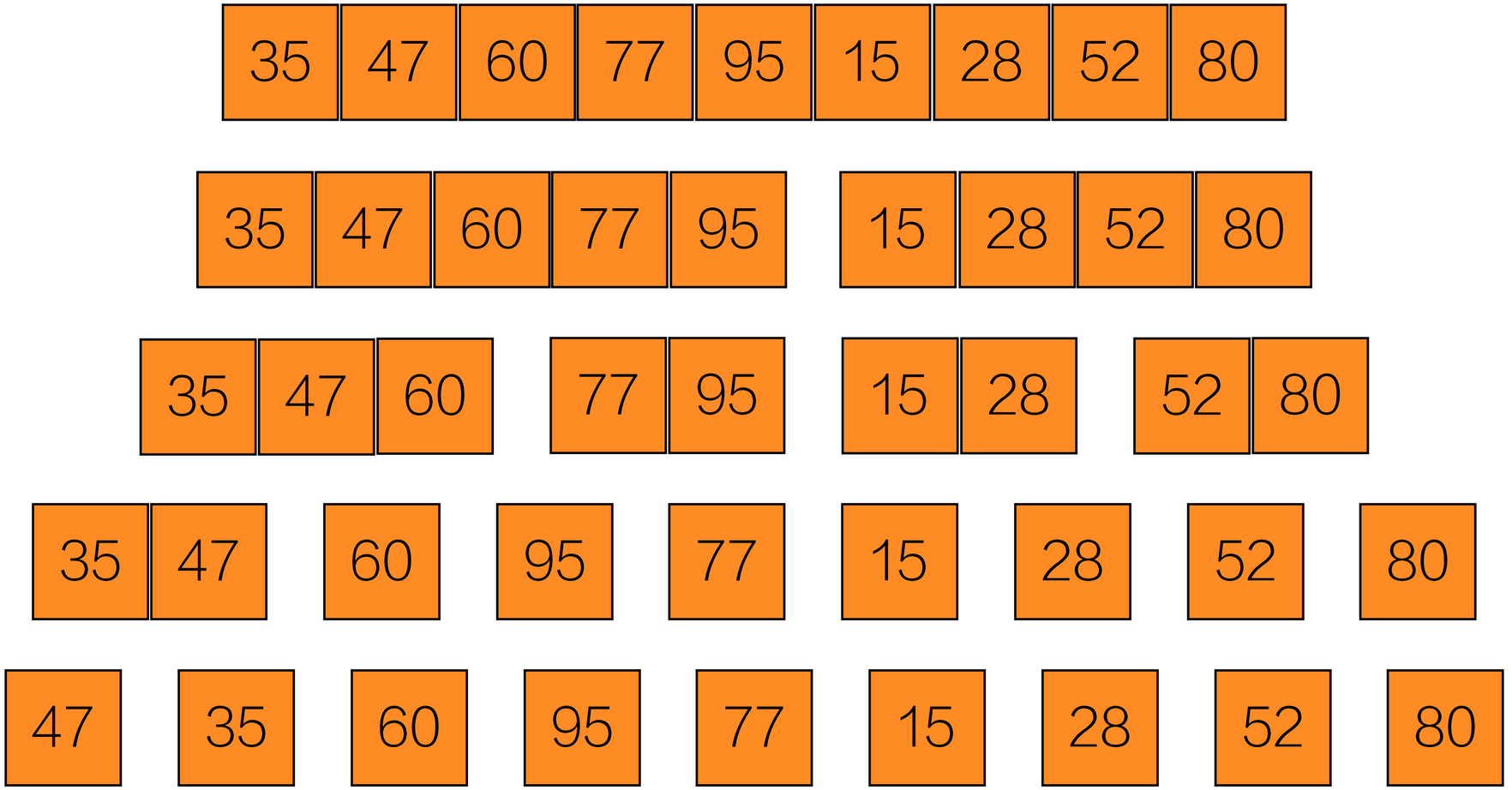
归并排序


35 47 60 77 95 15 28 52 80
35 47 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

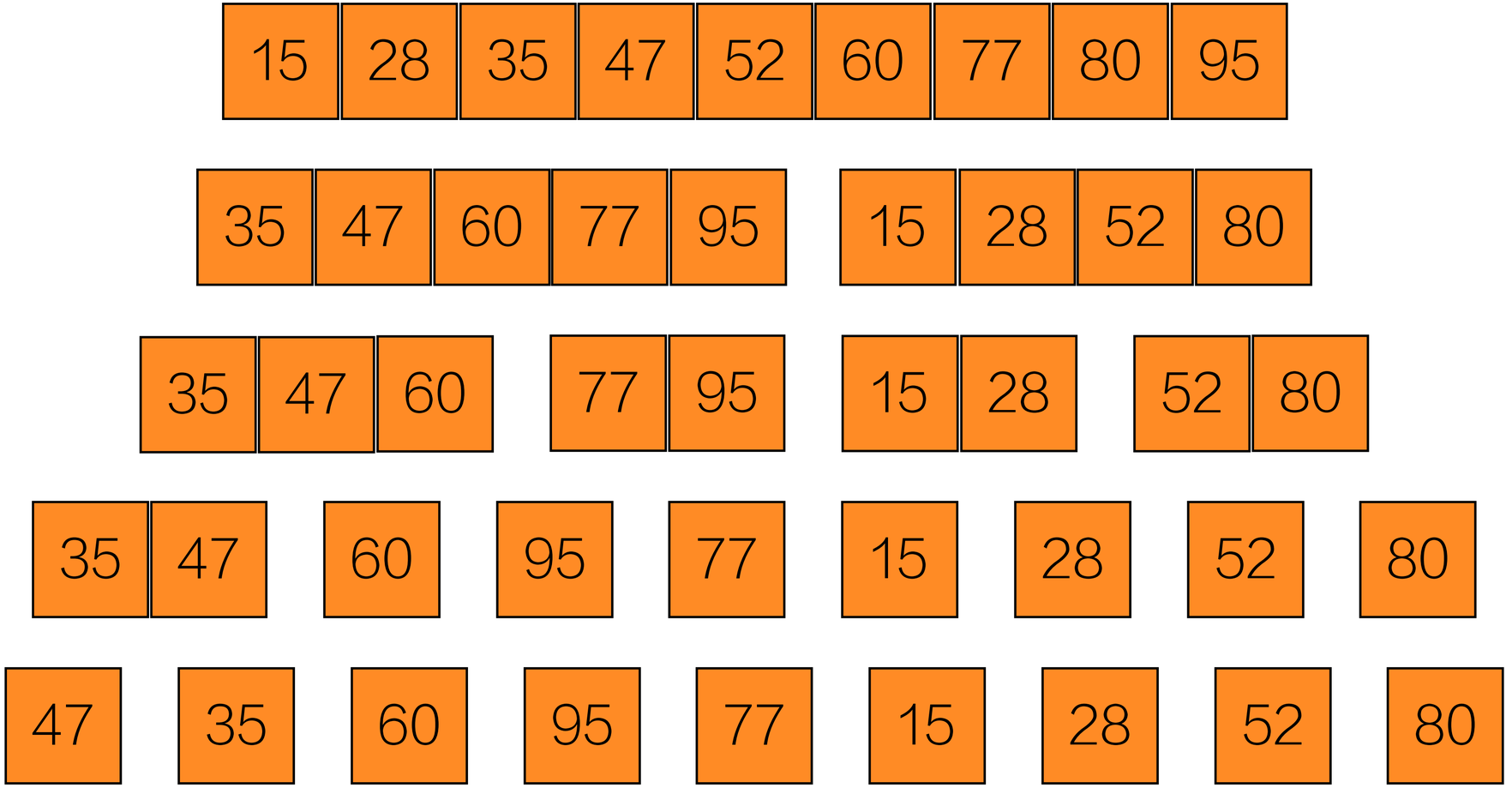
归并排序


35 47 60 77 95 15 28 52 80
35 47 60 77 95 15 28 52 80
35 47 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

归并排序


35 47 60 77 95 15 28 52 80
35 47 60 77 95 15 28 52 80
35 47 60 77 95 15 28 52 80
35 47 60 77 95 15 28 52 80
35 47 60 95 77 15 28 52 80
47 35 60 95 77 15 28 52 80

归并排序


15 28 35 47 52 60 77 80 95
35 47 60 77 95
15 28 52 80
35 47 60
77 95
15 28
52 80
35 47
60
95
77
15
28
52
80
47
35
60
95
77
15
28
52
80

归并排序



归并排序



```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

data

47	35	60	95	77	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

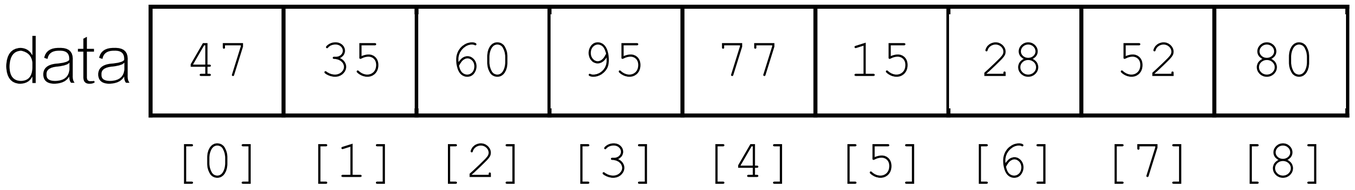
```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```



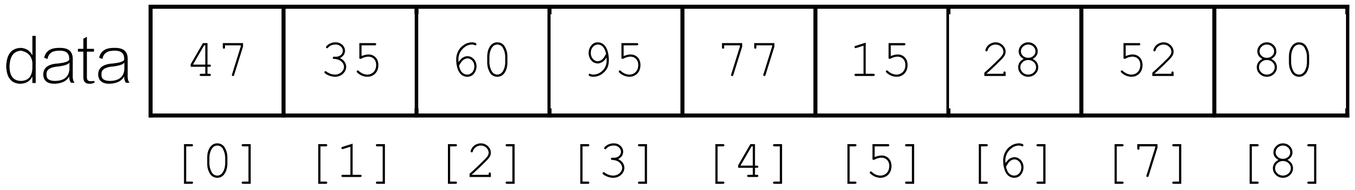
```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

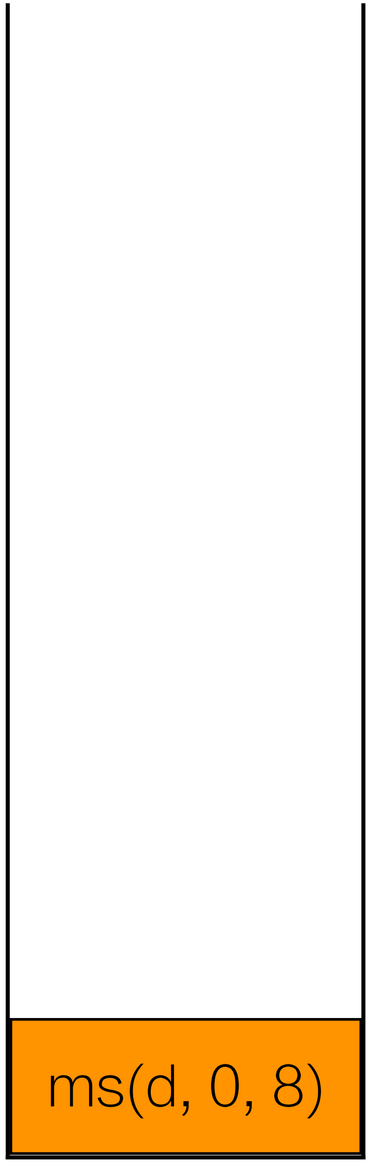
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：8　mid：

data	47	35	60	95	77	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：8　mid：4

data

47	35	60	95	77	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：8　mid：4

data	47	35	60	95	77	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

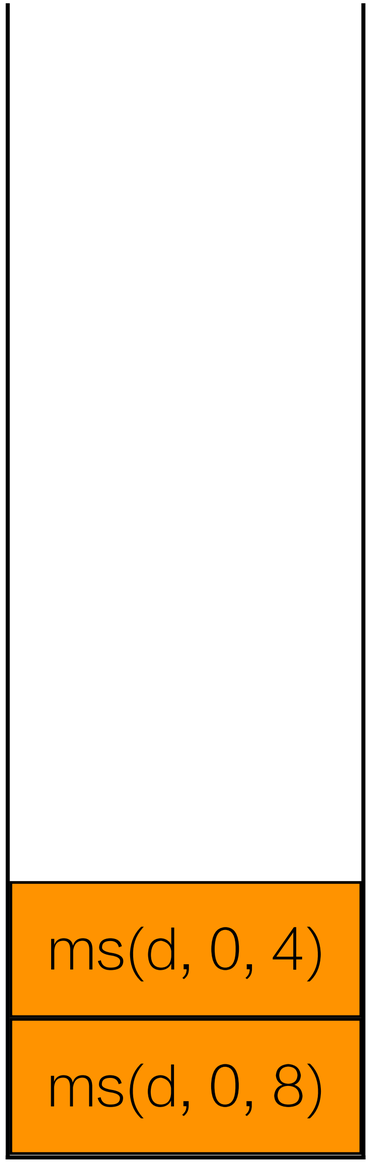
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

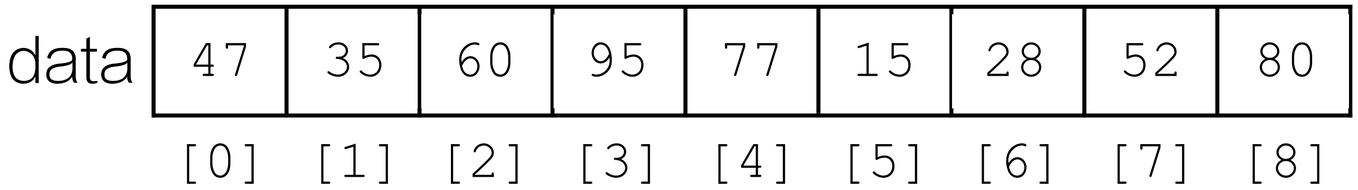
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：4　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

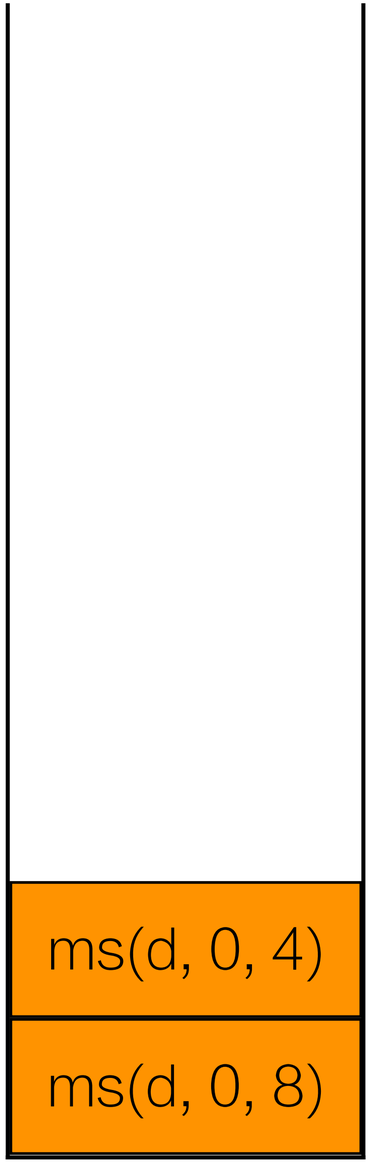
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

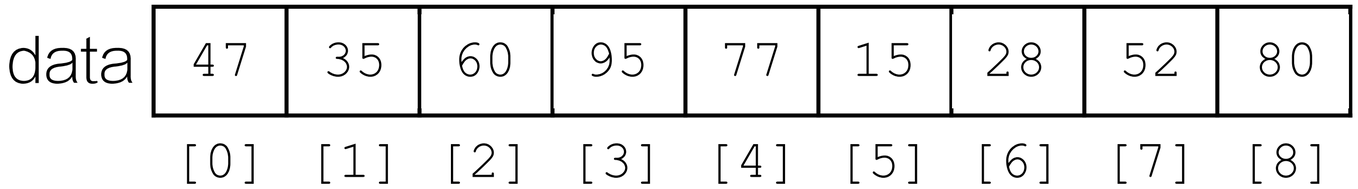
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：4　mid：2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

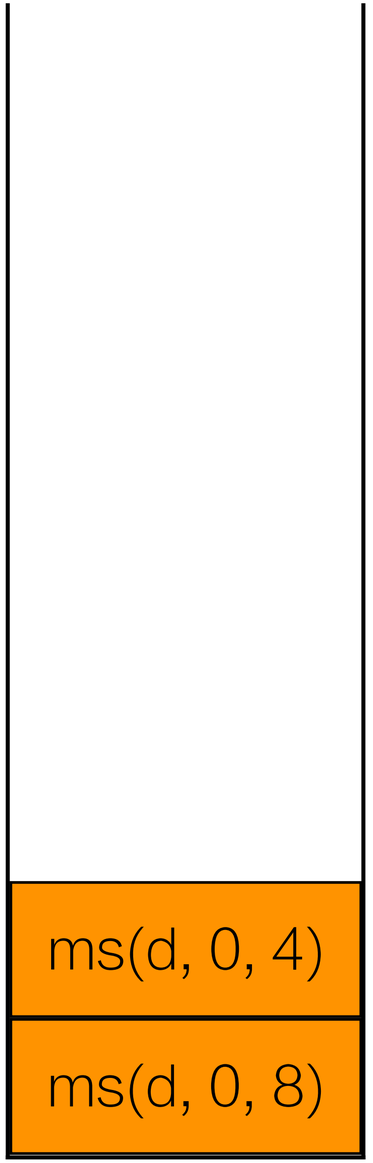
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

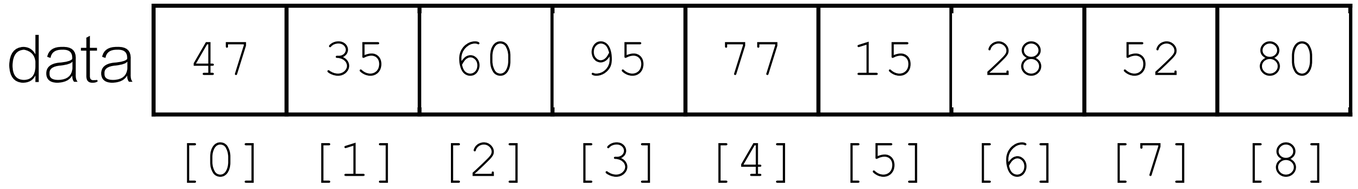
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：4　mid：2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

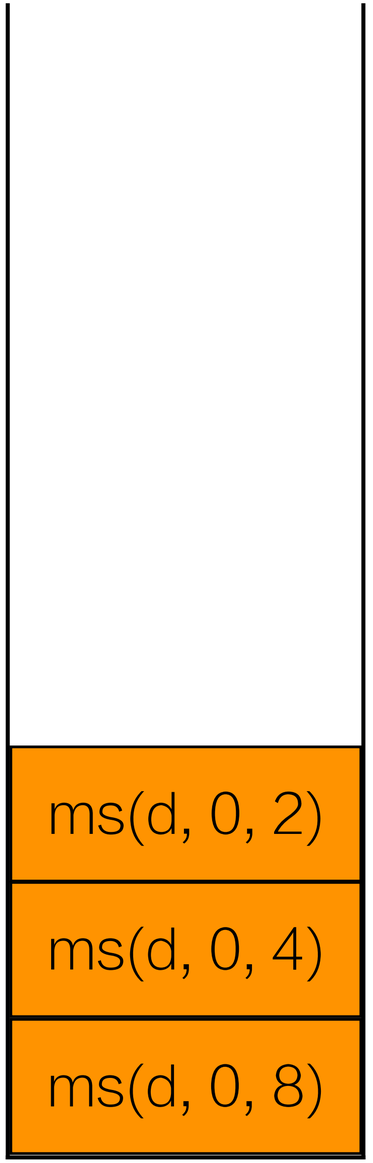
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

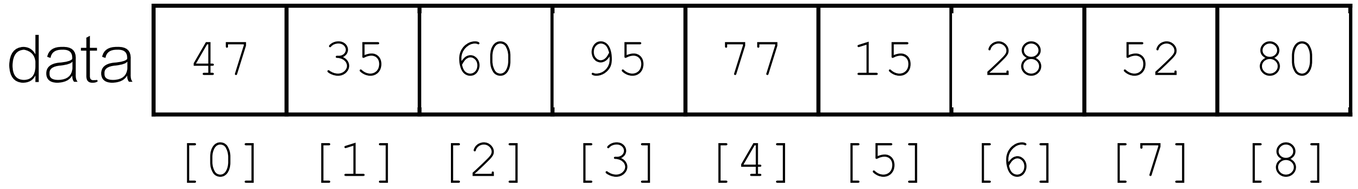
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：2　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

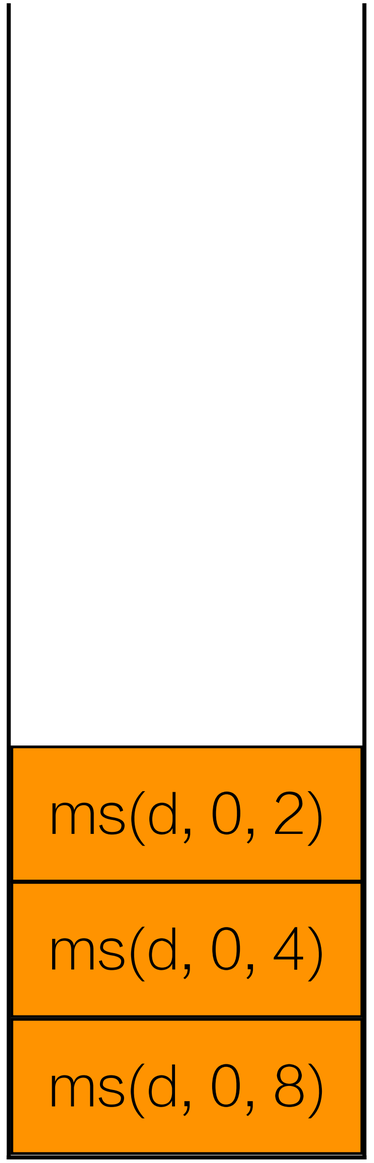
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

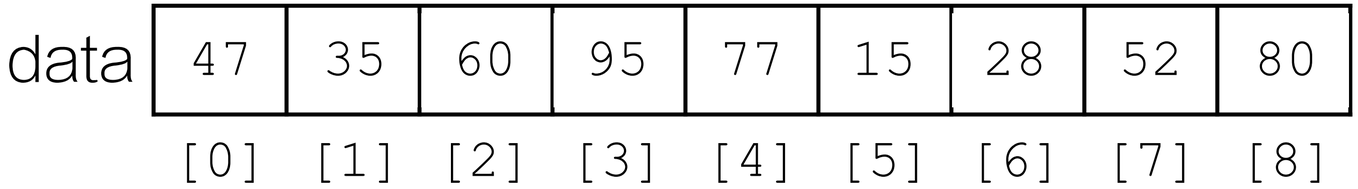
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：2　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

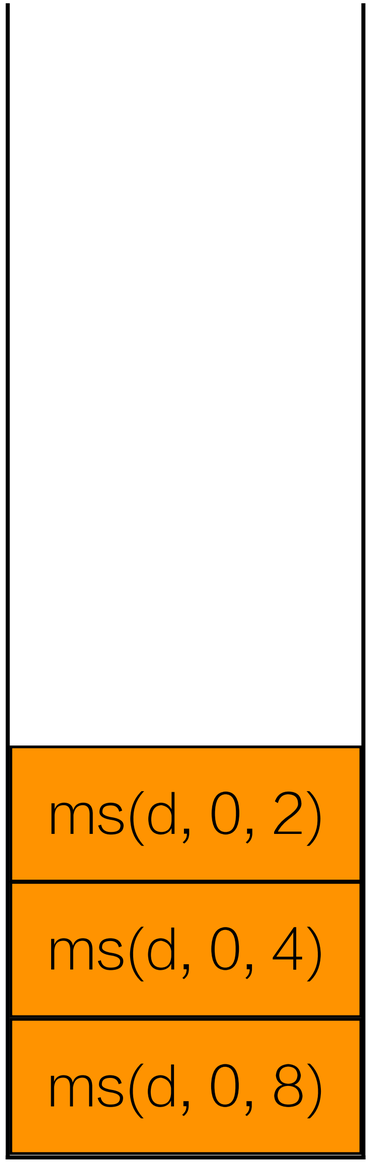
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

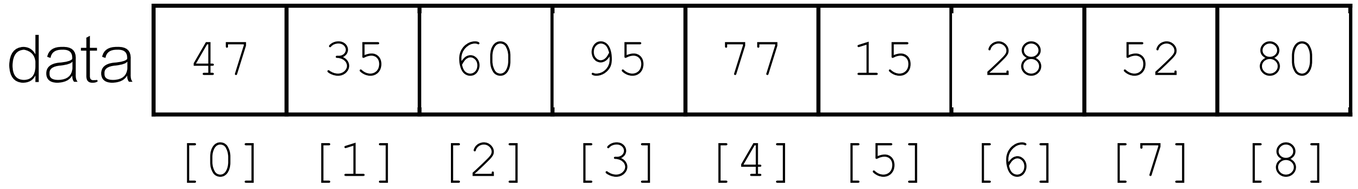
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：2　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

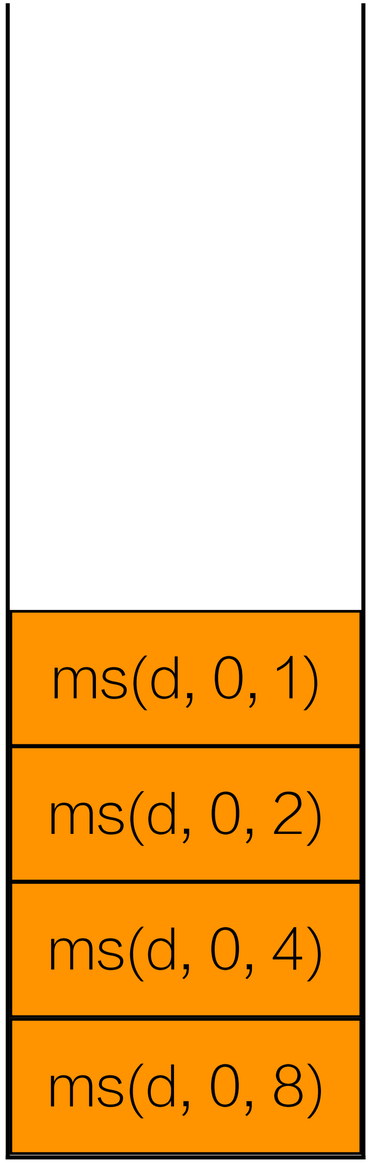
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

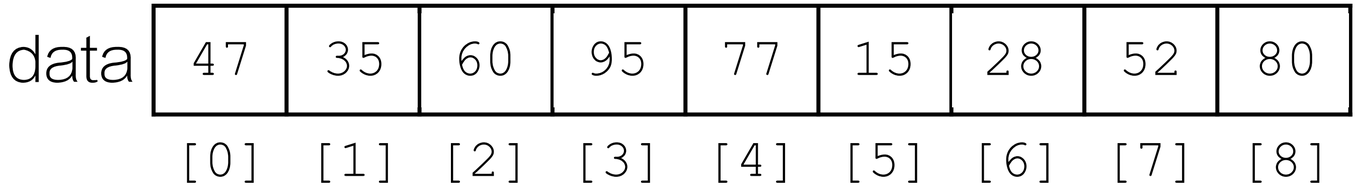
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

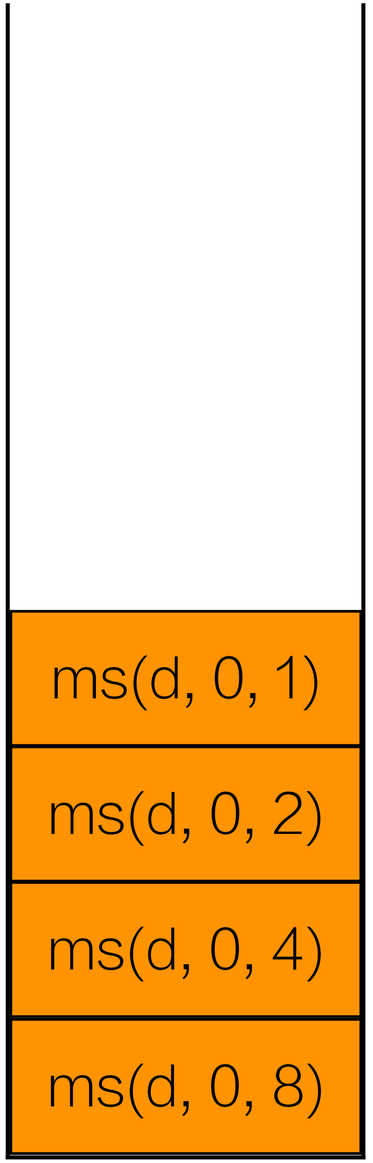
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

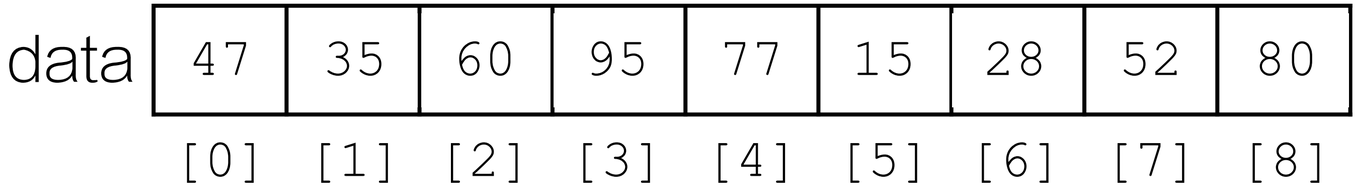
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

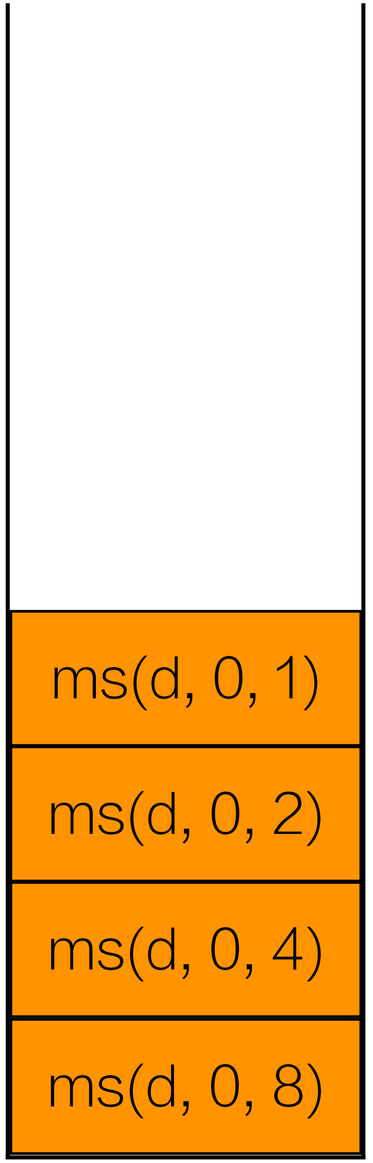
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

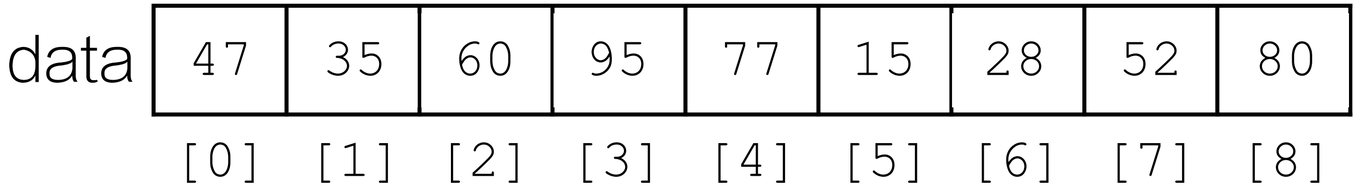
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

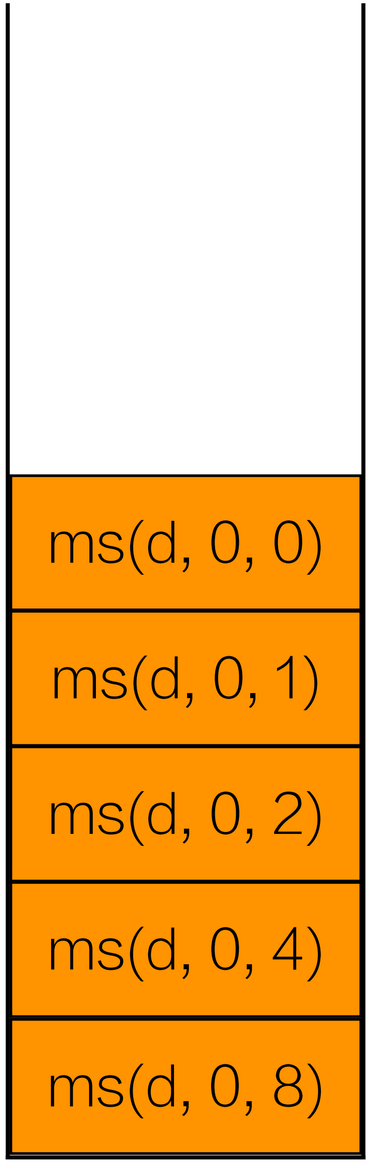
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

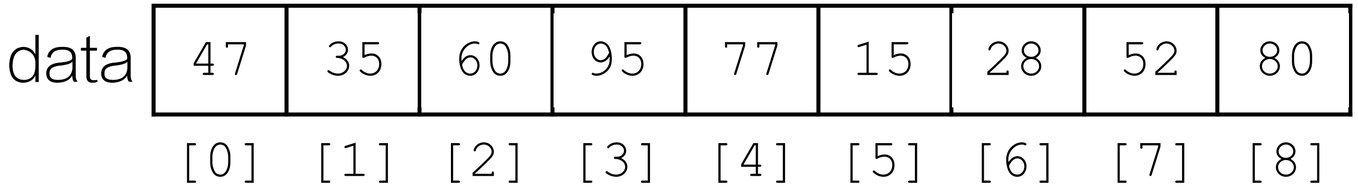
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：0　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

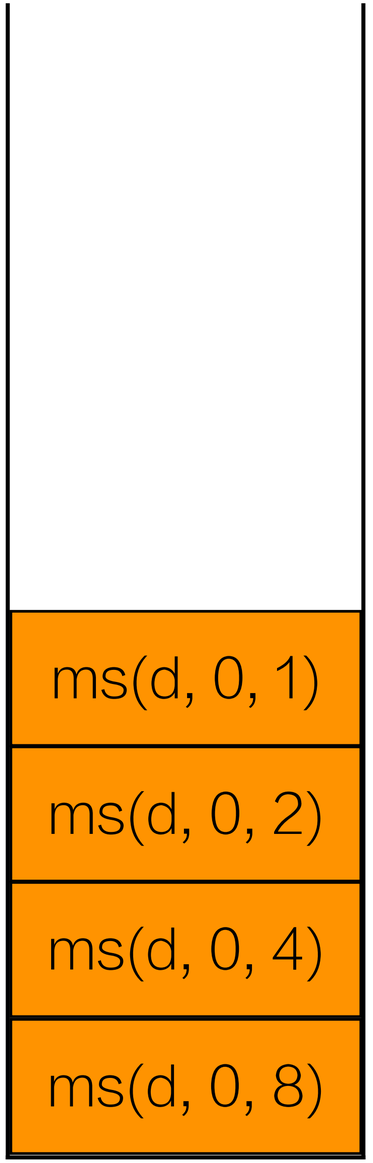
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

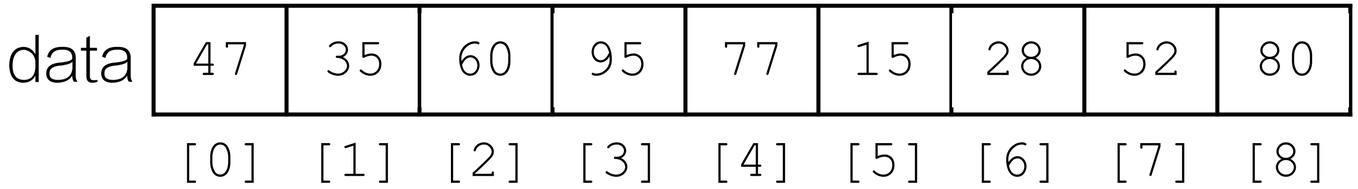
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

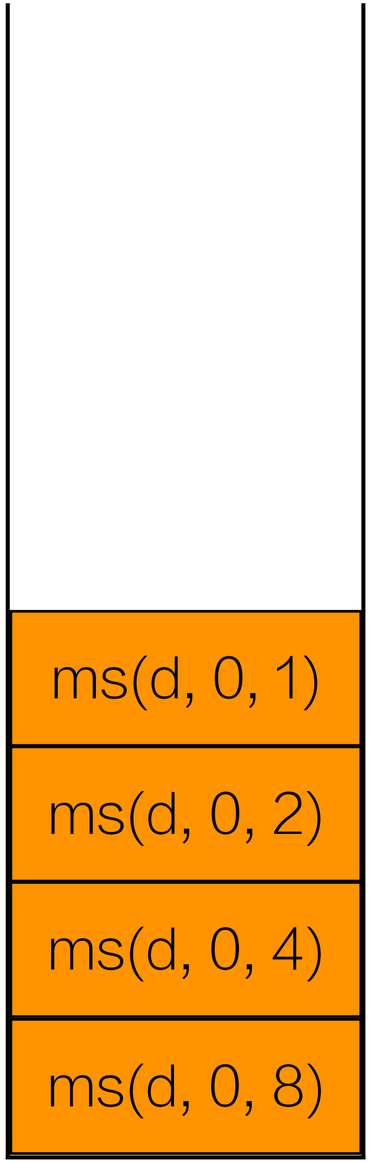
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

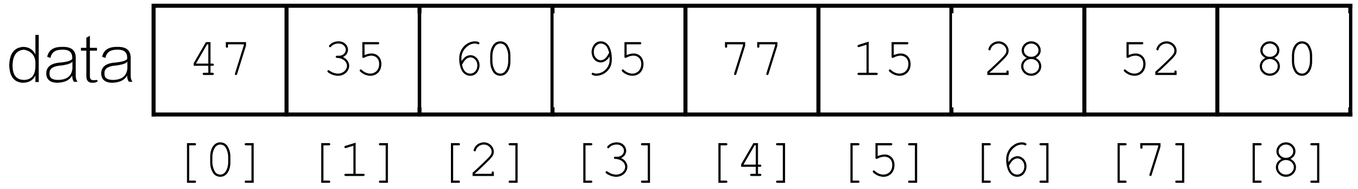
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

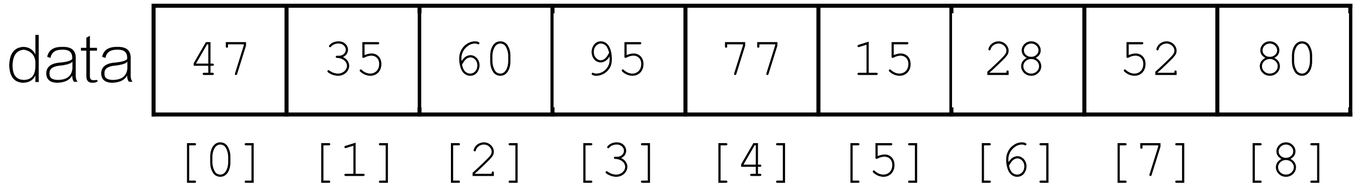
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：1　right：1　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

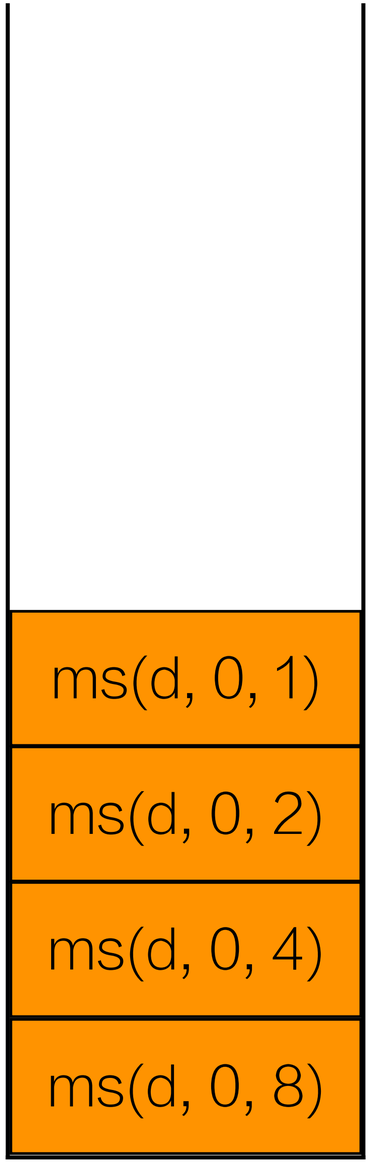
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

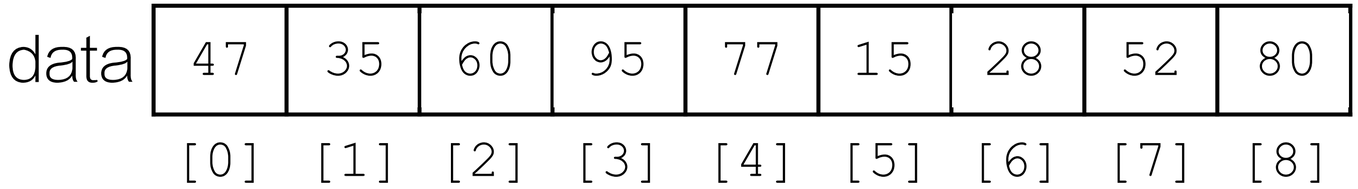
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

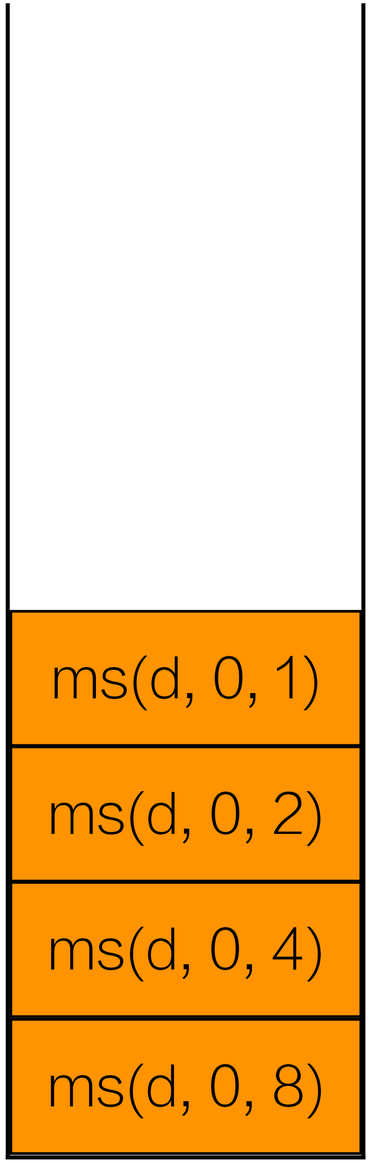
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

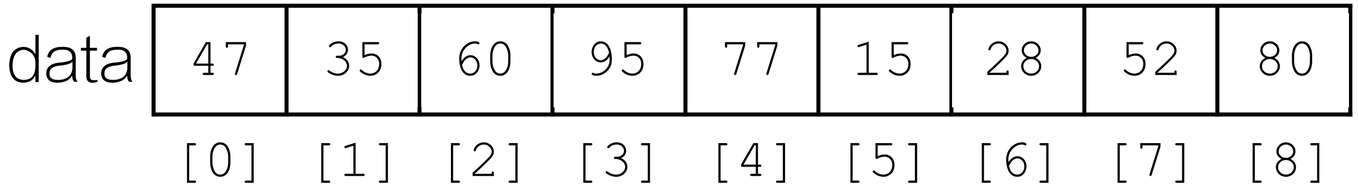
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　right：1　mid：0





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

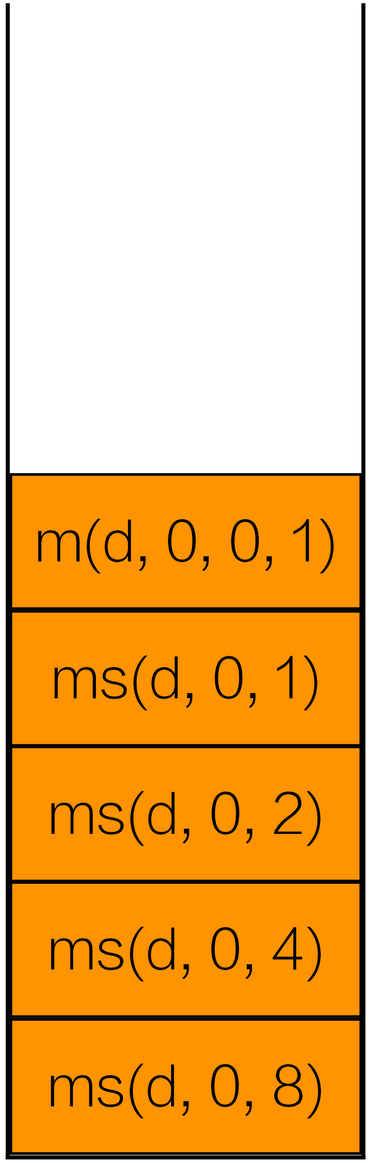
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

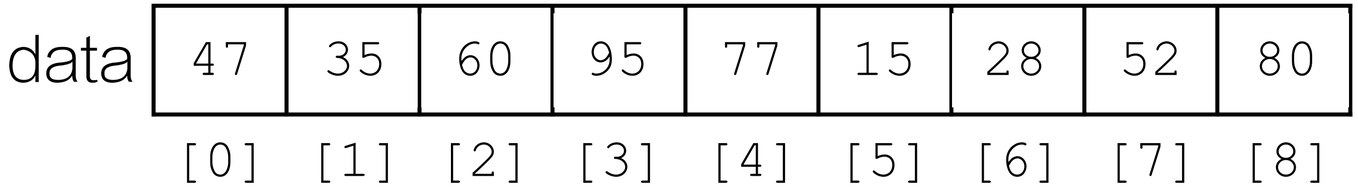
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　mid：0　　right：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

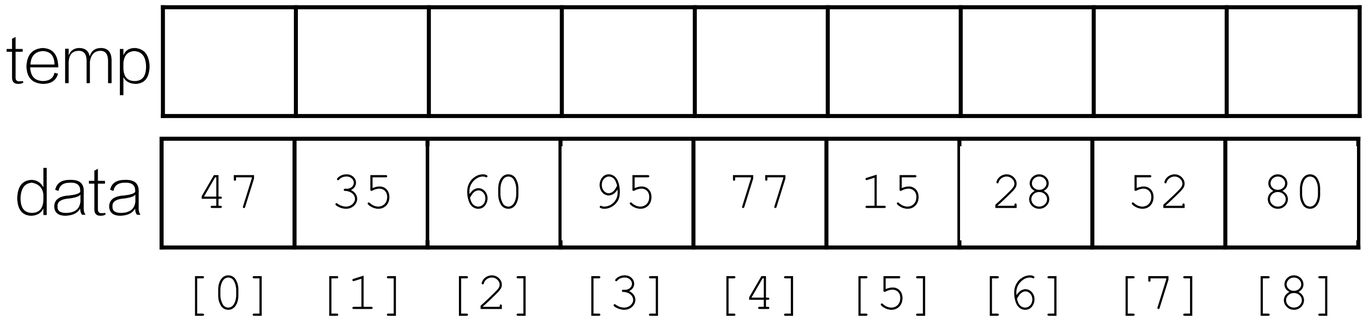
        // 对左半部分排序
        mergeSort(data, left, mid);

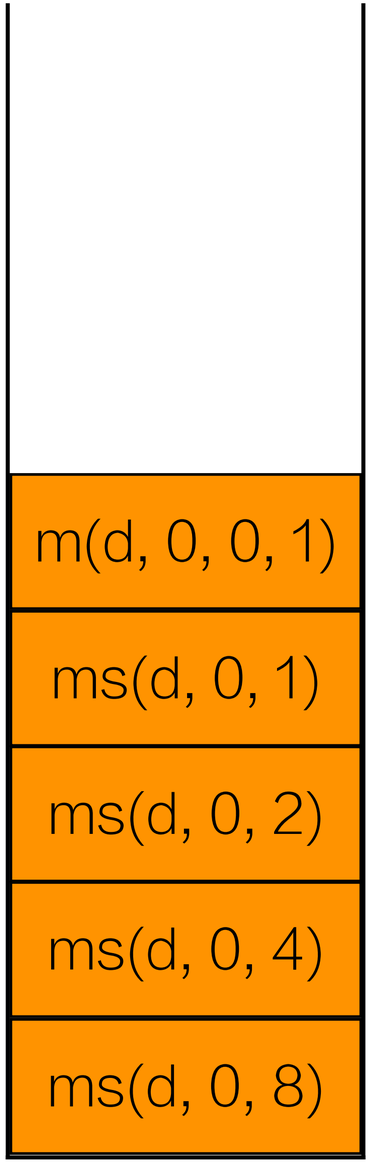
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 0　　j: 1　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

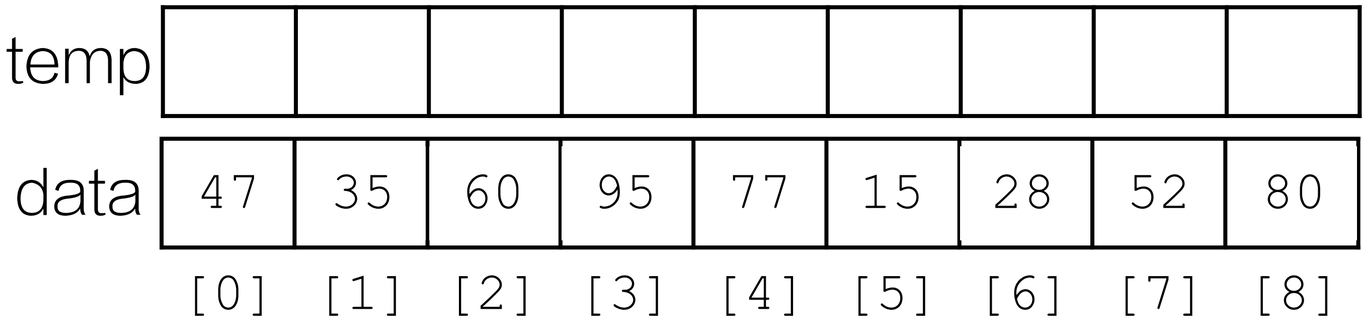
        // 对左半部分排序
        mergeSort(data, left, mid);

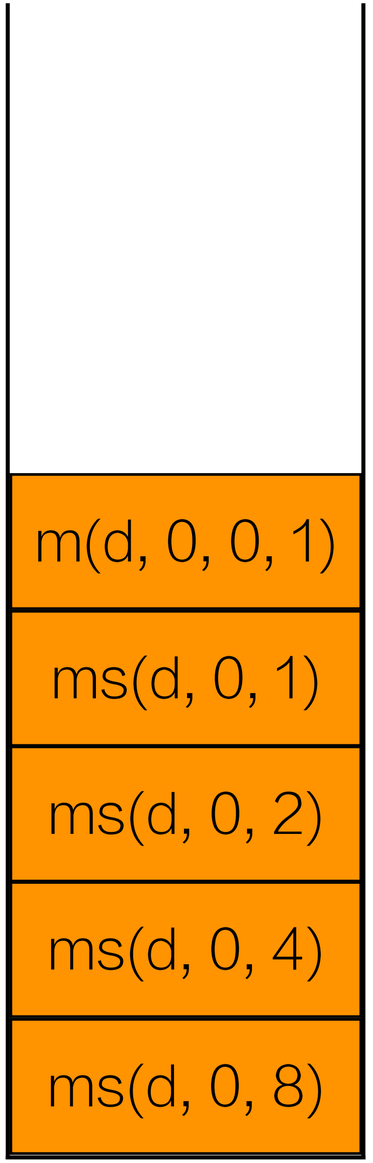
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 0　　j: 1　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

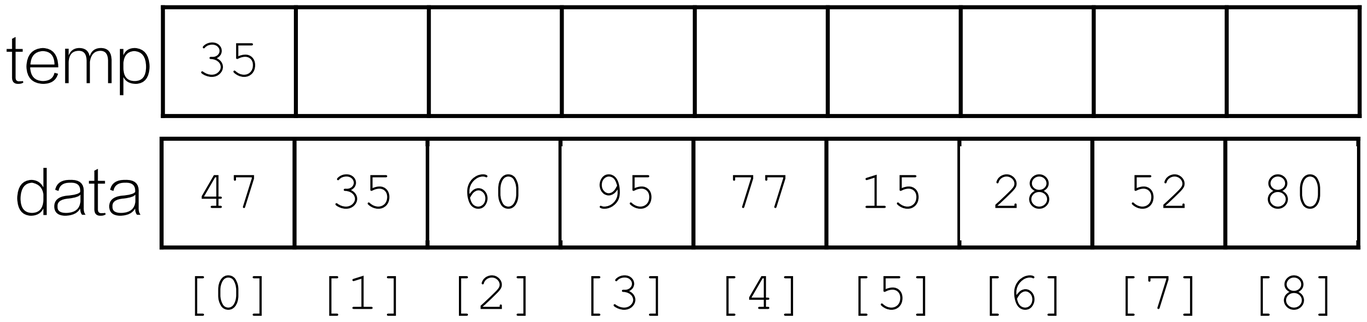
        // 对左半部分排序
        mergeSort(data, left, mid);

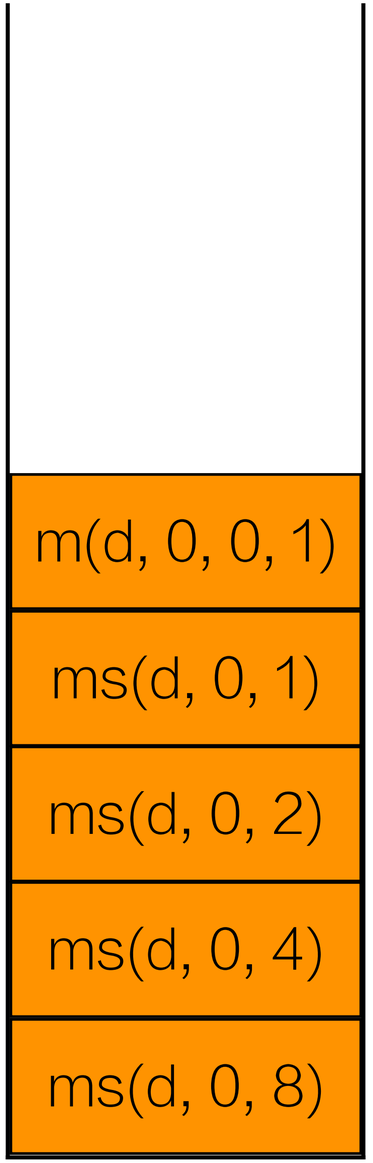
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 0　　j: 2　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 0　　j: 2　　k: 1

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35								
data	47	35	60	95	77	15	28	52	80

m(d, 0, 0, 1)

ms(d, 0, 1)

ms(d, 0, 2)

ms(d, 0, 4)

ms(d, 0, 8)

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

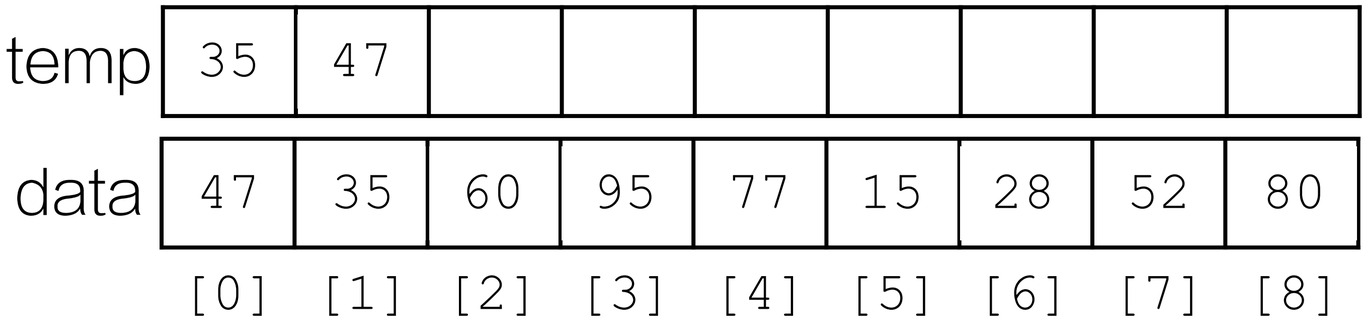
        // 对左半部分排序
        mergeSort(data, left, mid);

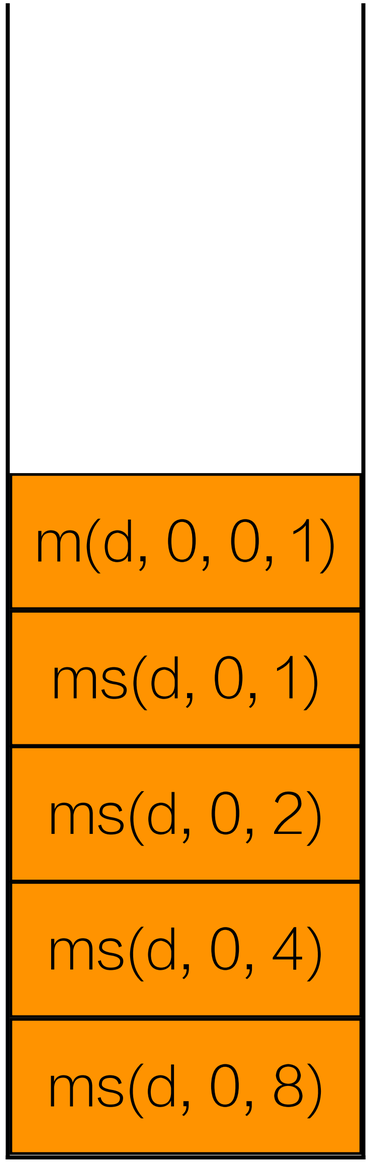
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 1　　j: 2　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

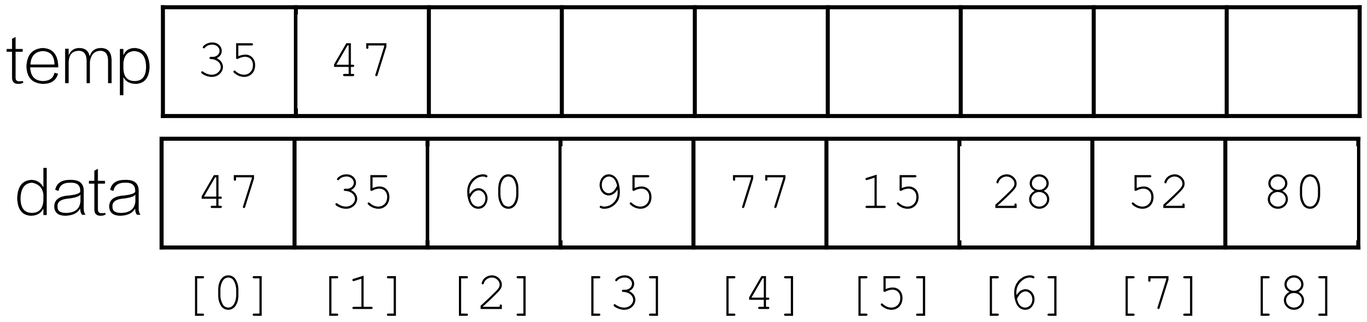
        // 对左半部分排序
        mergeSort(data, left, mid);

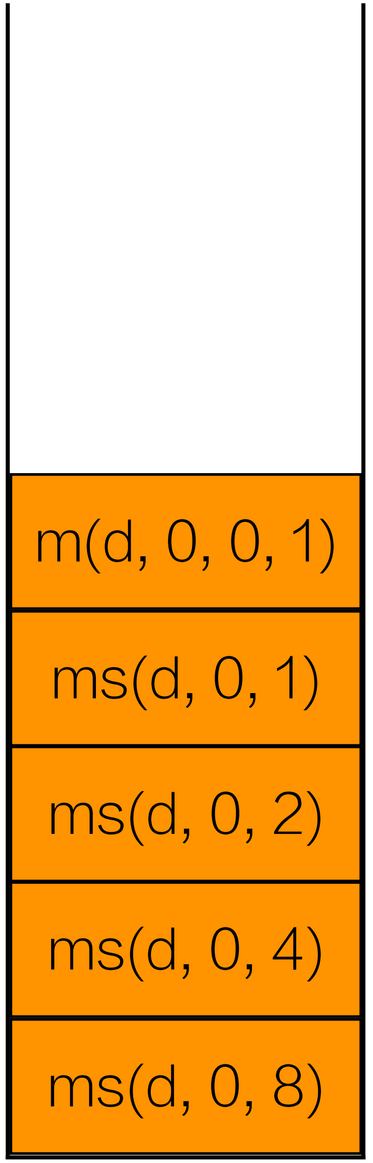
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 1　　j: 2　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 1　　j: 2　　k: 2

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35	47							
data	47	35	60	95	77	15	28	52	80

m(d, 0, 0, 1)

ms(d, 0, 1)

ms(d, 0, 2)

ms(d, 0, 4)

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

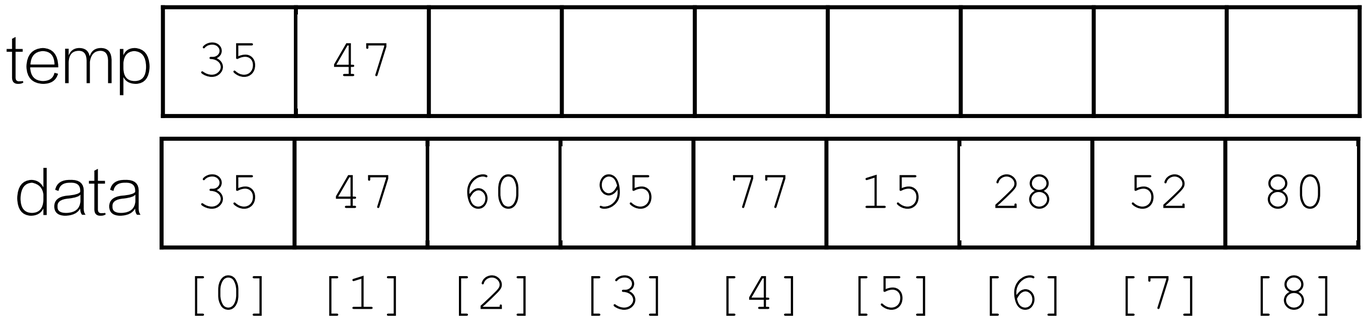
        // 对左半部分排序
        mergeSort(data, left, mid);

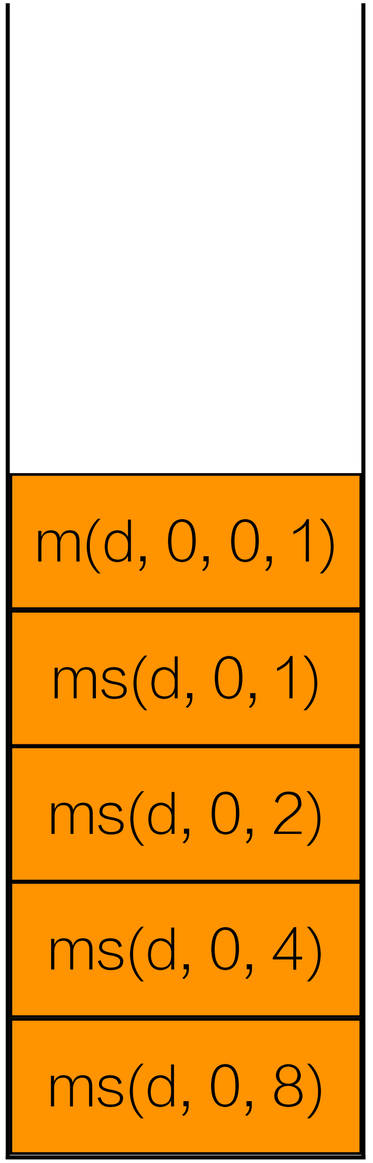
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 0　　right: 1

i: 1　　j: 2　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

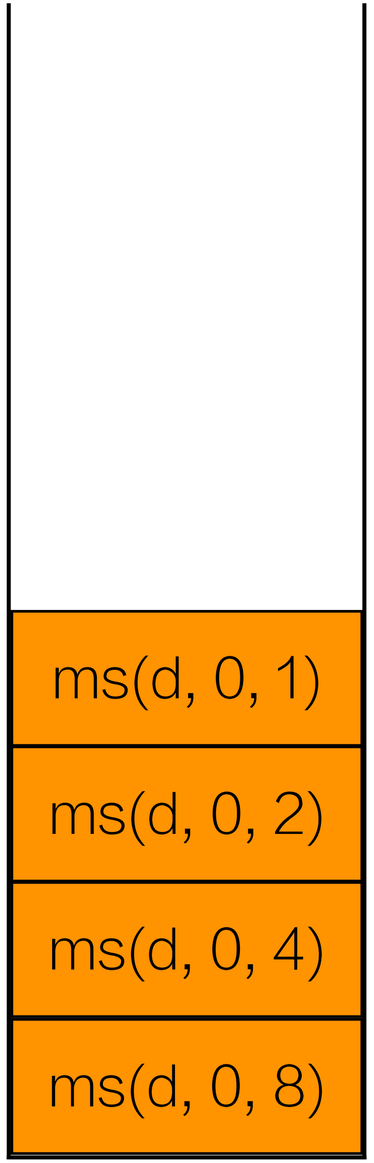
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

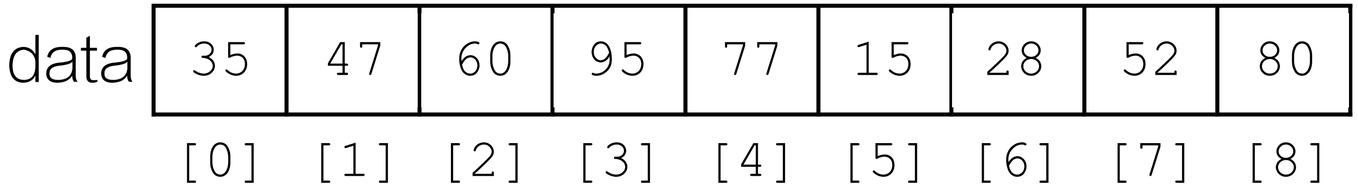
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：1　　mid：0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

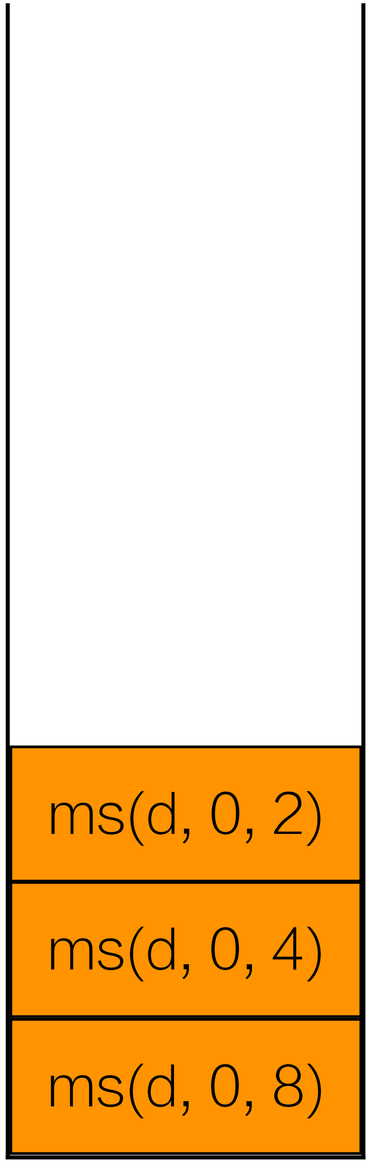
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

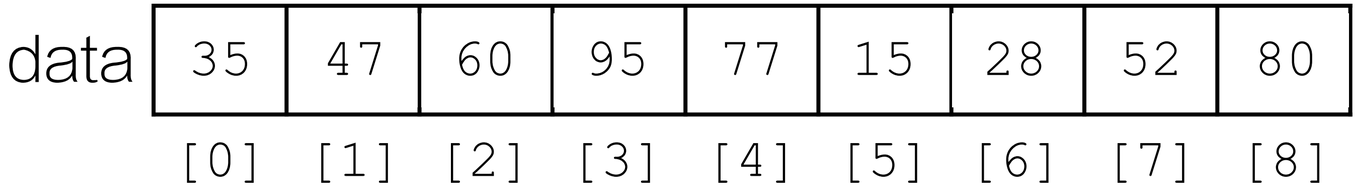
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：2　　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

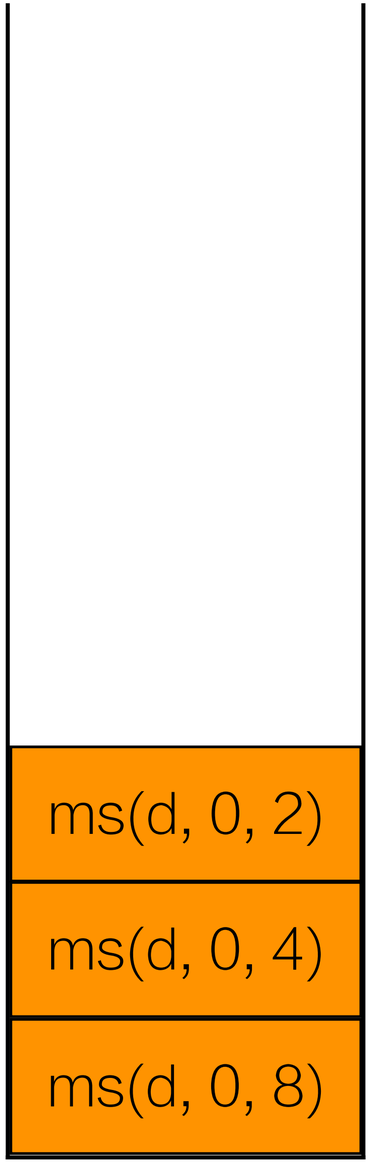
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

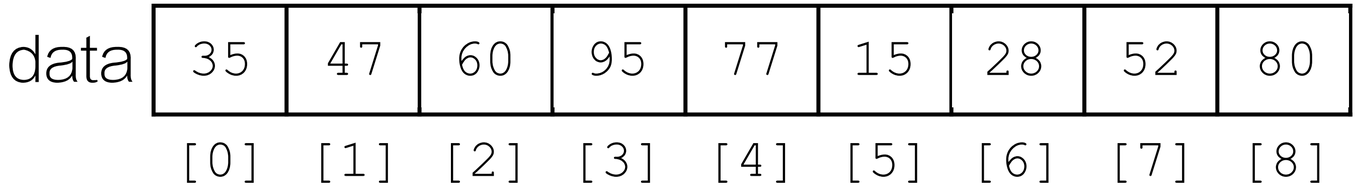
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：2　　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

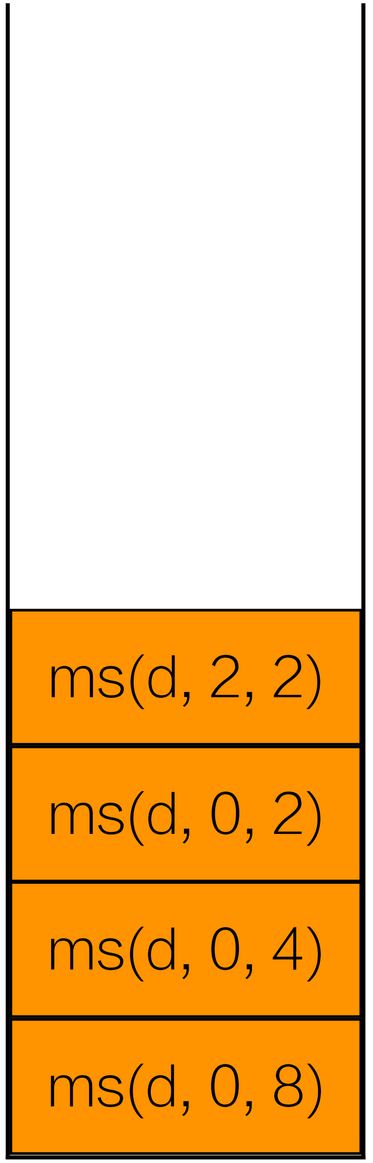
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

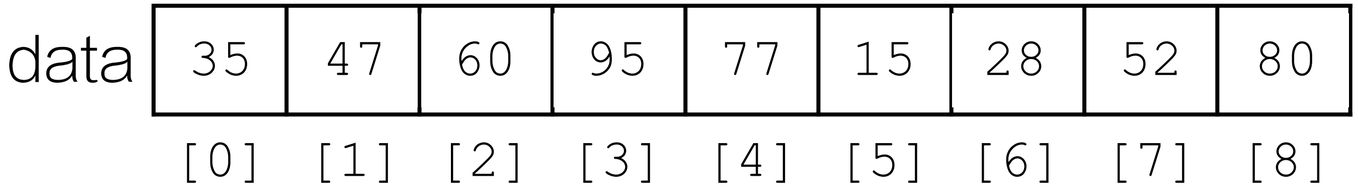
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：2　　right：2　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

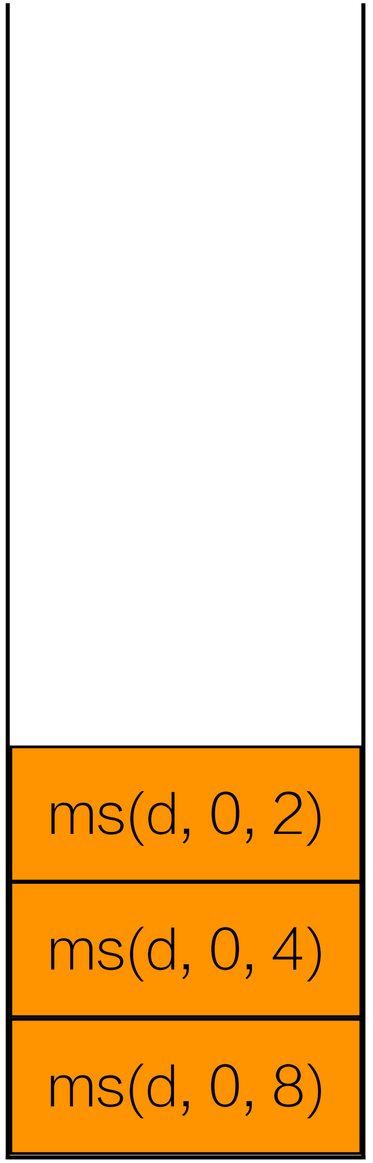
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

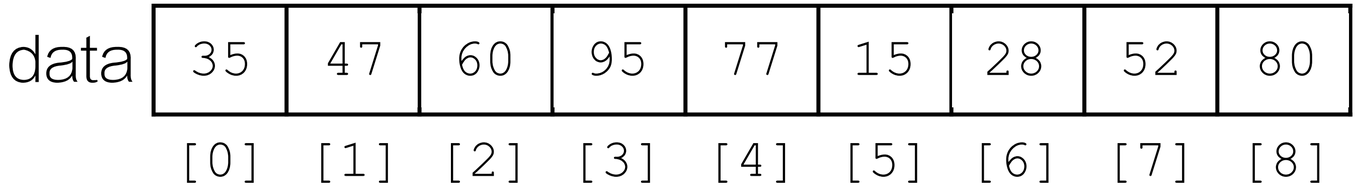
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：2　　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

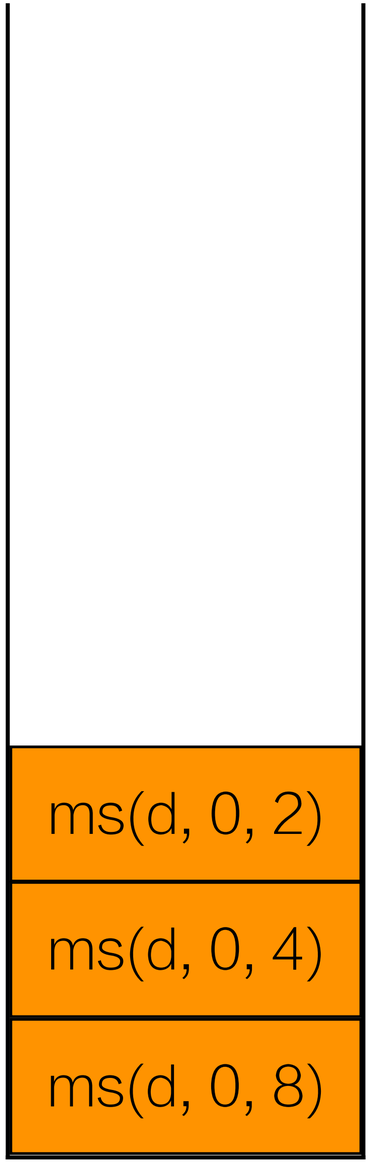
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

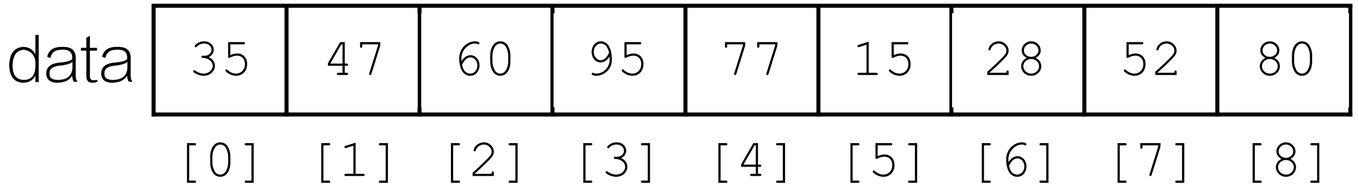
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：2　　mid：1





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

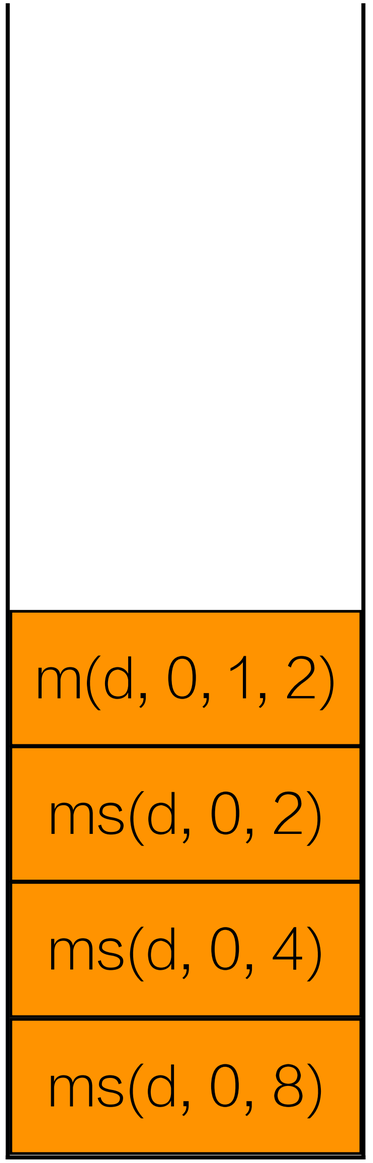
```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

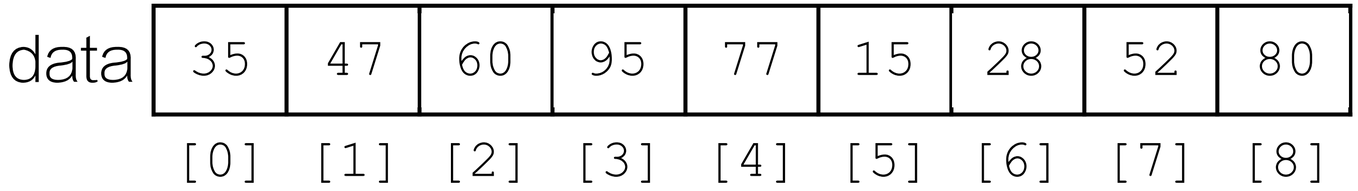
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　mid：1　　right：2





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

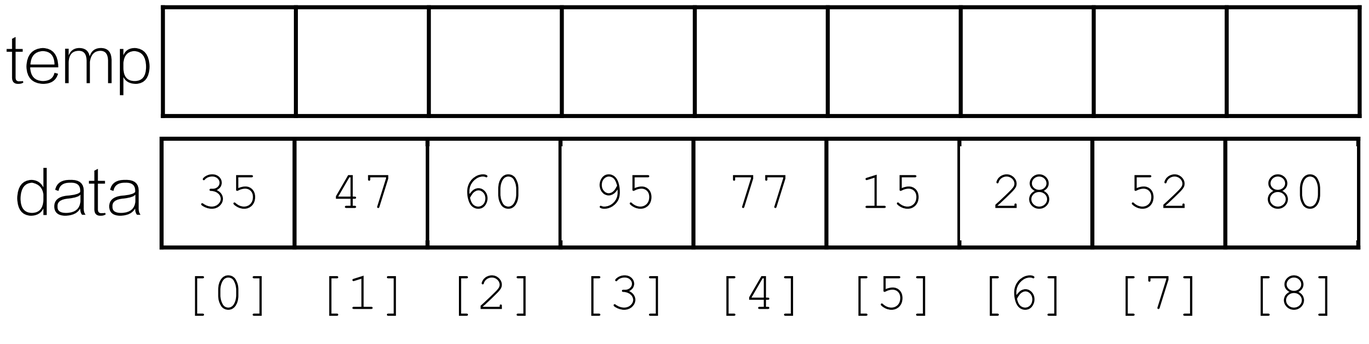
        // 对左半部分排序
        mergeSort(data, left, mid);

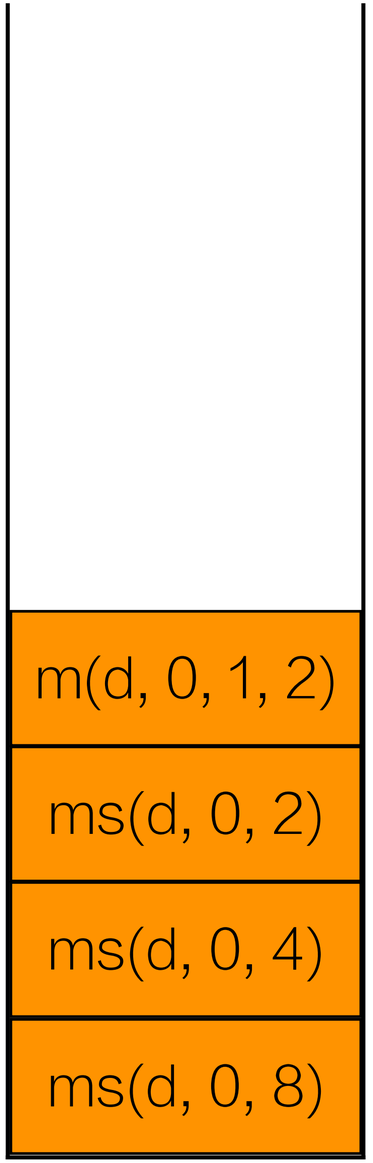
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 0　　j: 2　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

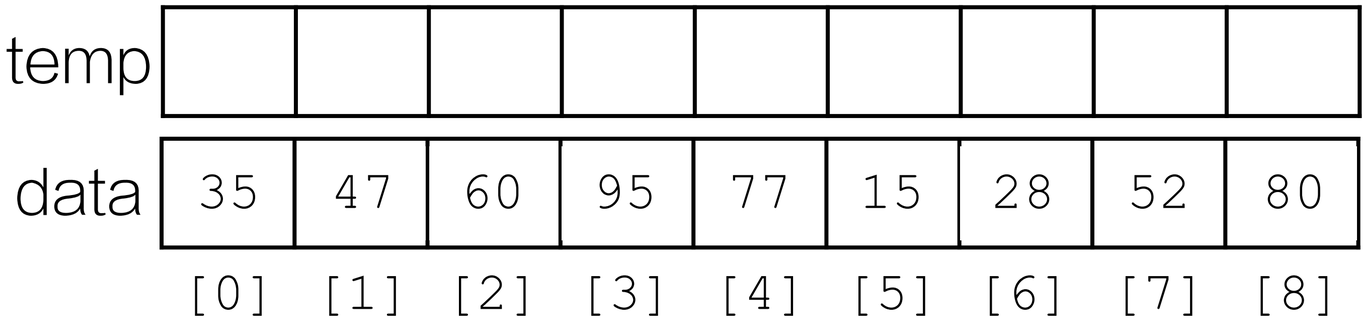
        // 对左半部分排序
        mergeSort(data, left, mid);

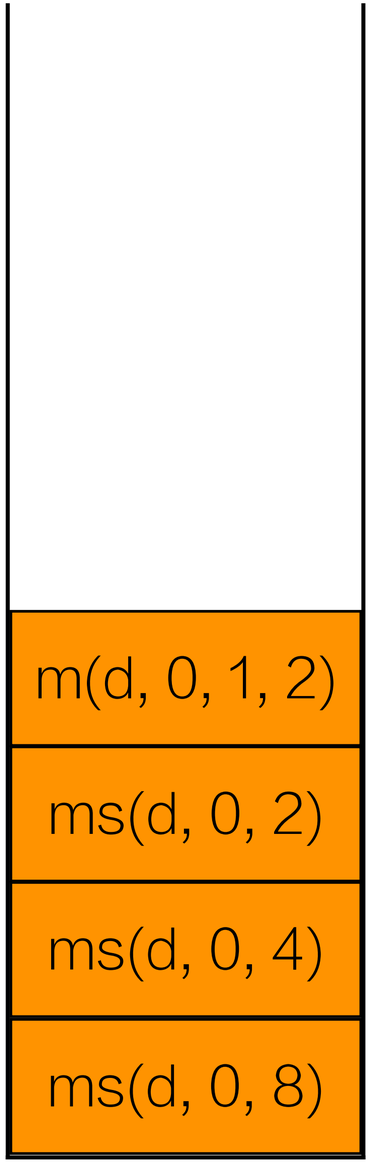
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 0　　j: 2　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

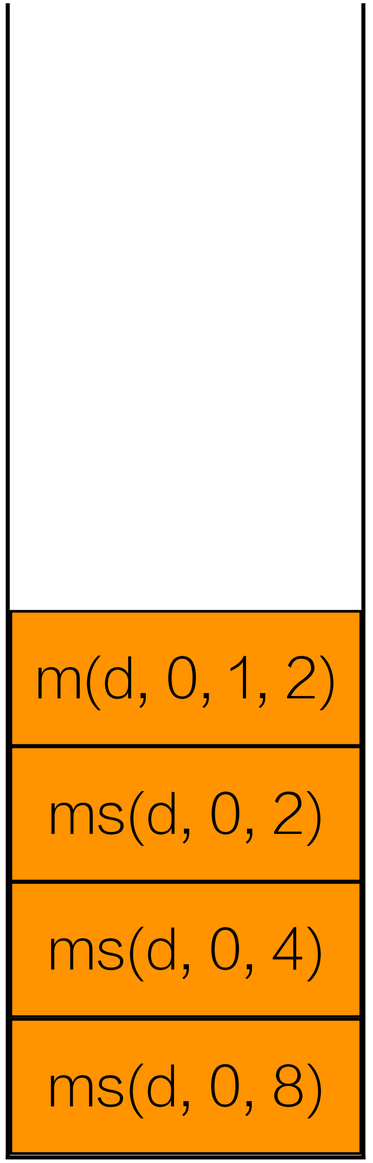
        // 对左半部分排序
        mergeSort(data, left, mid);

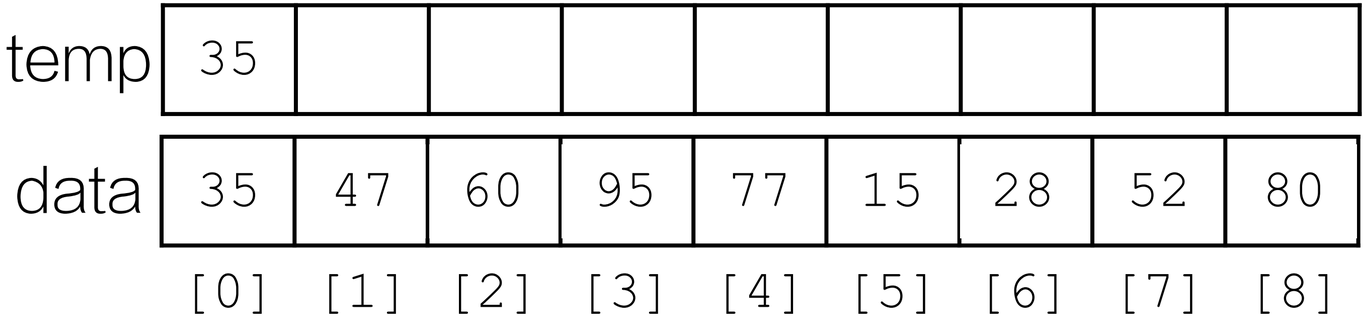
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 1　　j: 2　　k: 1





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 2　　j: 2　　k: 2

m(d, 0, 1, 2)

ms(d, 0, 2)

ms(d, 0, 4)

ms(d, 0, 8)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35	47							
data	35	47	60	95	77	15	28	52	80

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

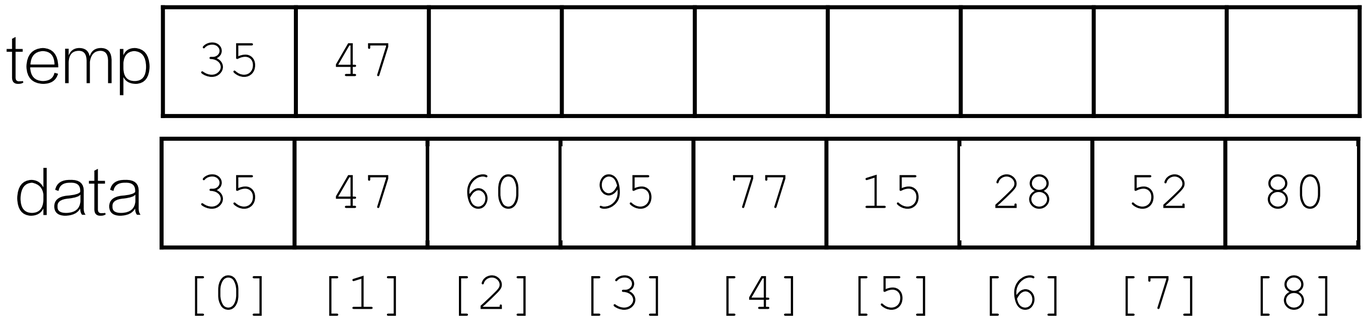
        // 对左半部分排序
        mergeSort(data, left, mid);

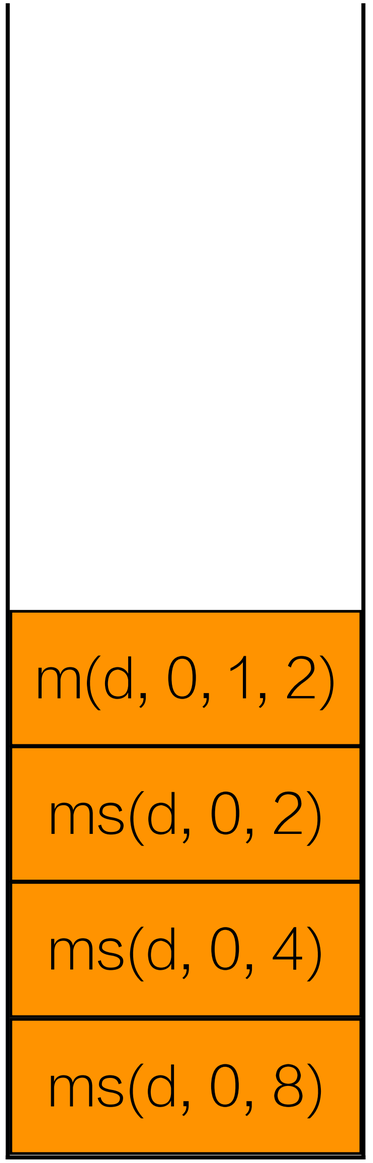
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 2　　j: 2　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

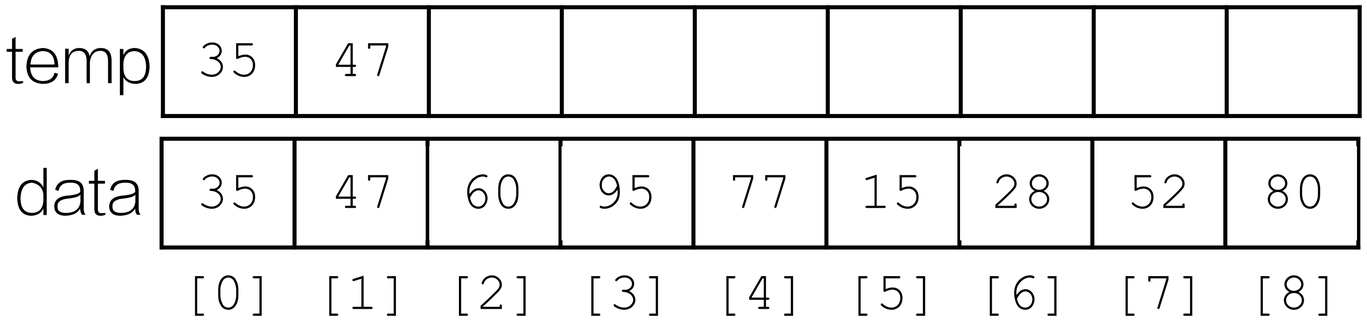
        // 对左半部分排序
        mergeSort(data, left, mid);

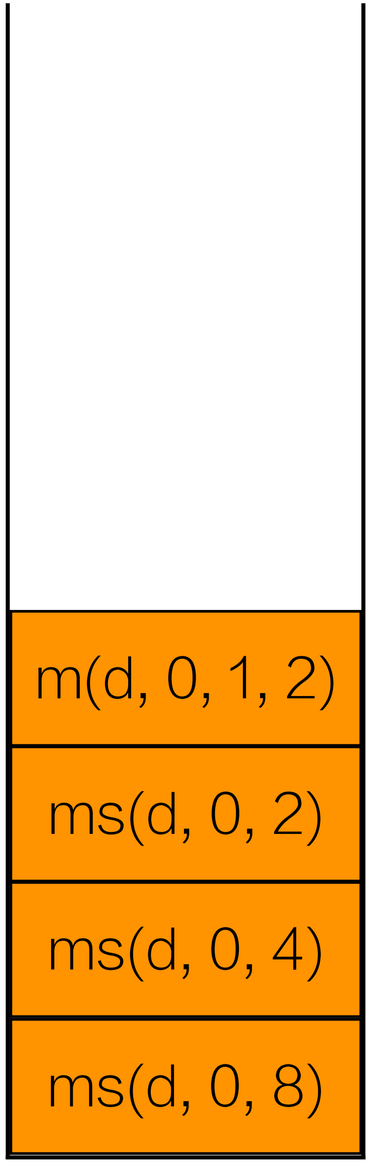
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 2　　j: 2　　k: 2





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

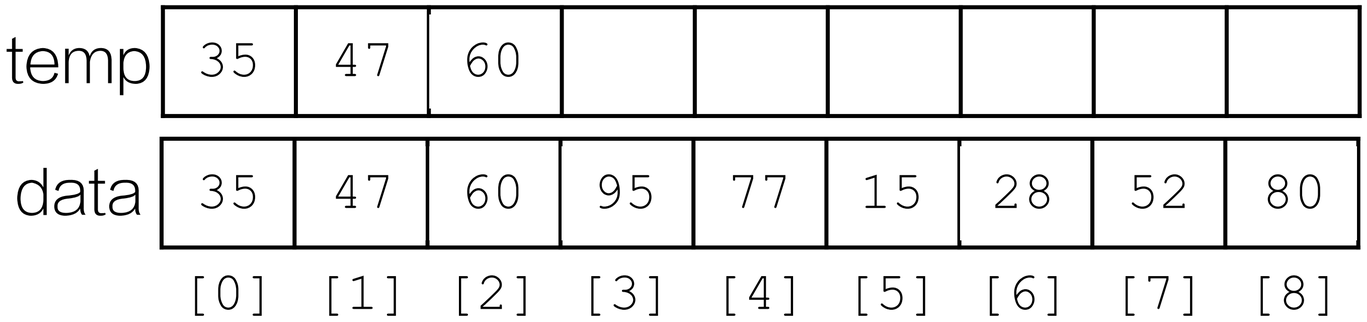
        // 对左半部分排序
        mergeSort(data, left, mid);

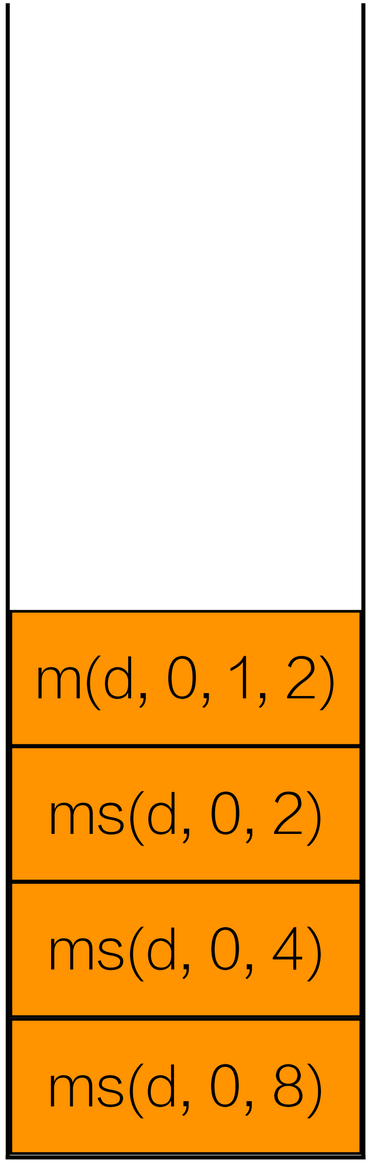
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 2　　j: 3　　k: 3





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

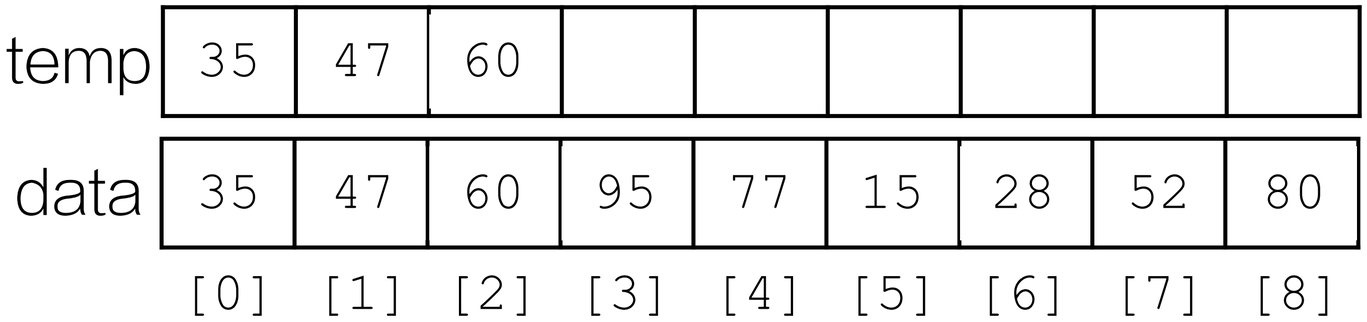
        // 对左半部分排序
        mergeSort(data, left, mid);

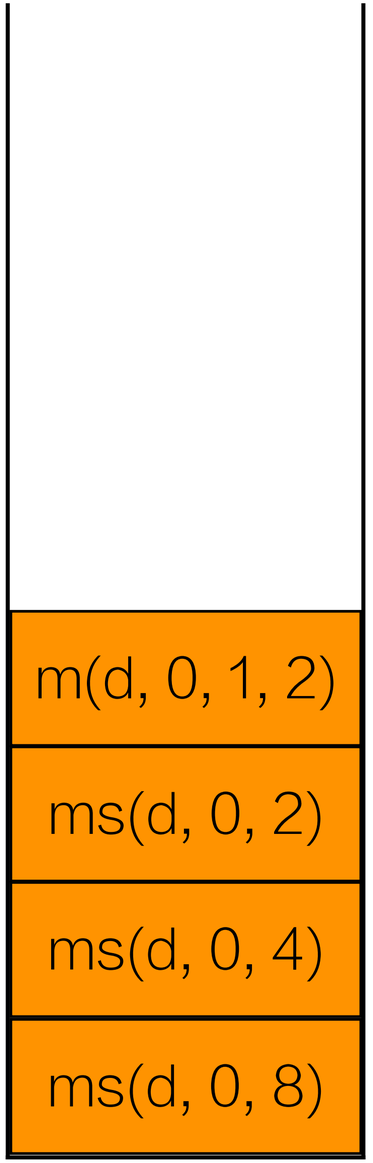
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 1　　right: 2

i: 2　　j: 3　　k: 3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

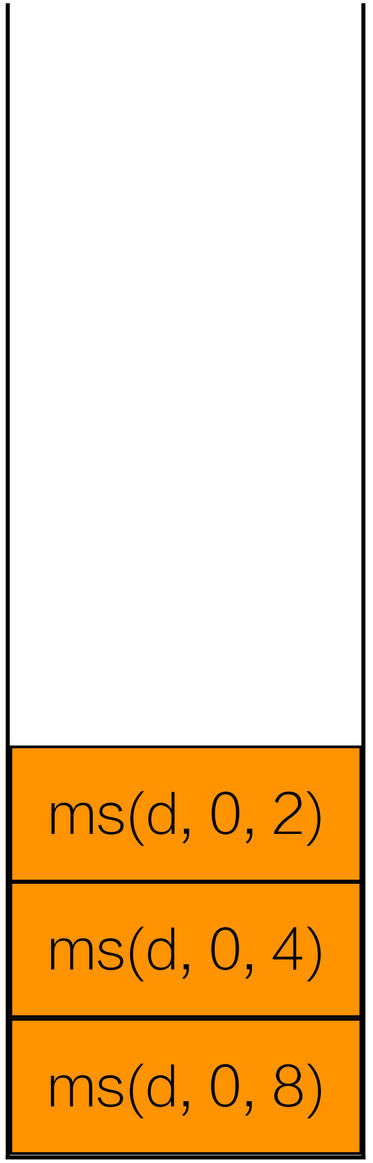
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

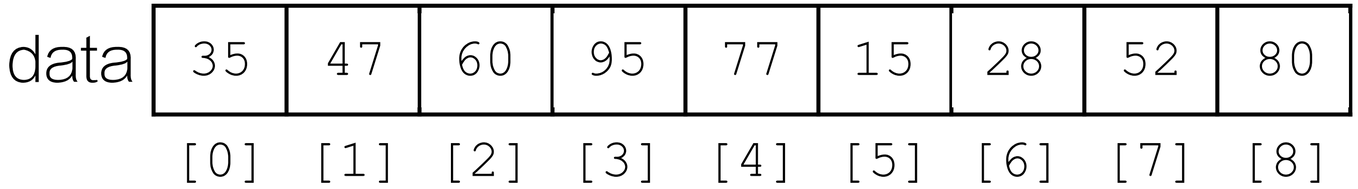
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：2　　mid：1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

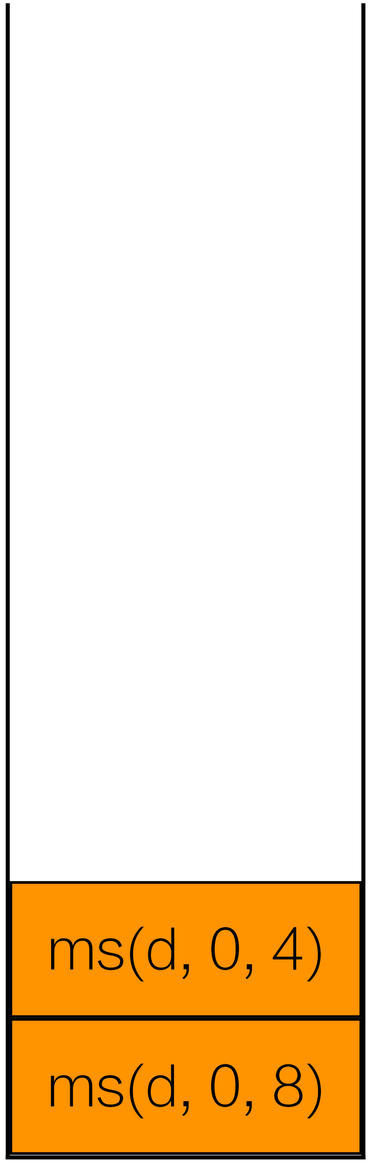
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

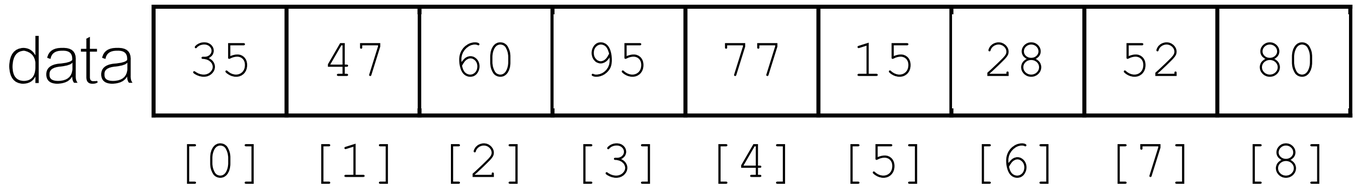
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：4　　mid：2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

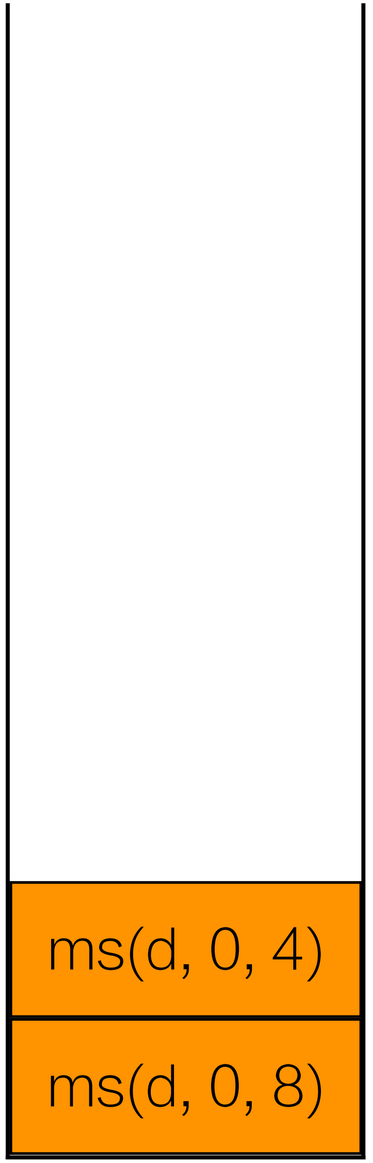
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：4　　mid：2



data	35	47	60	95	77	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

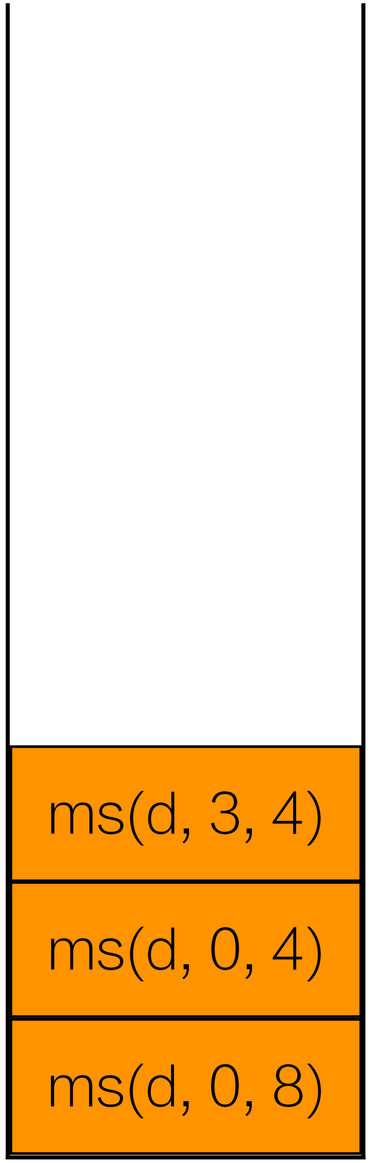
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：



data

35	47	60	95	77	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

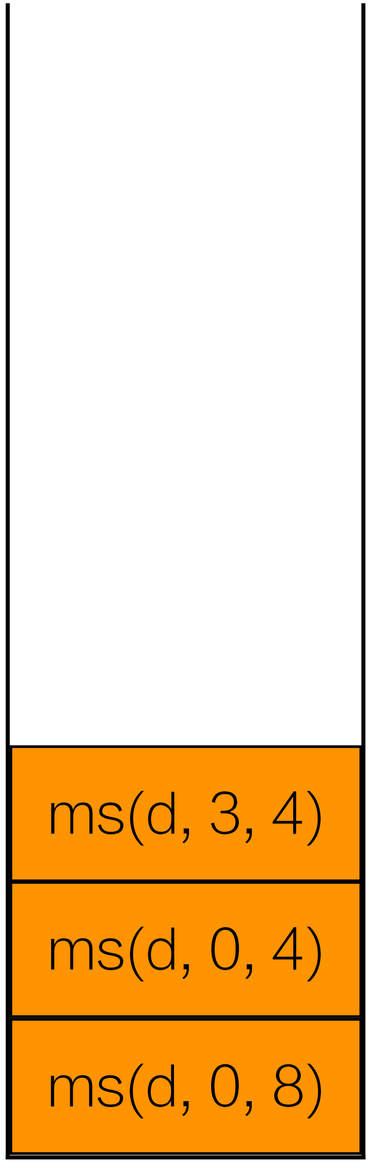
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

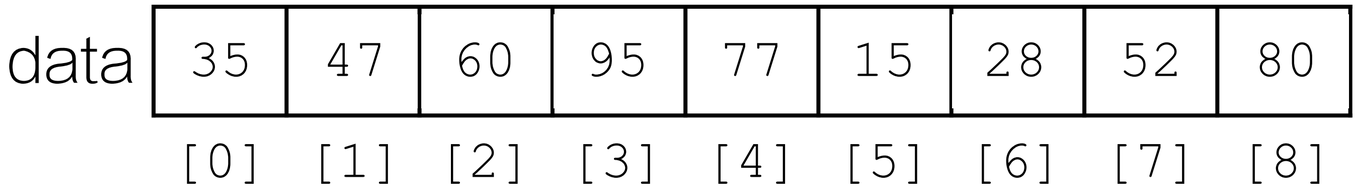
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

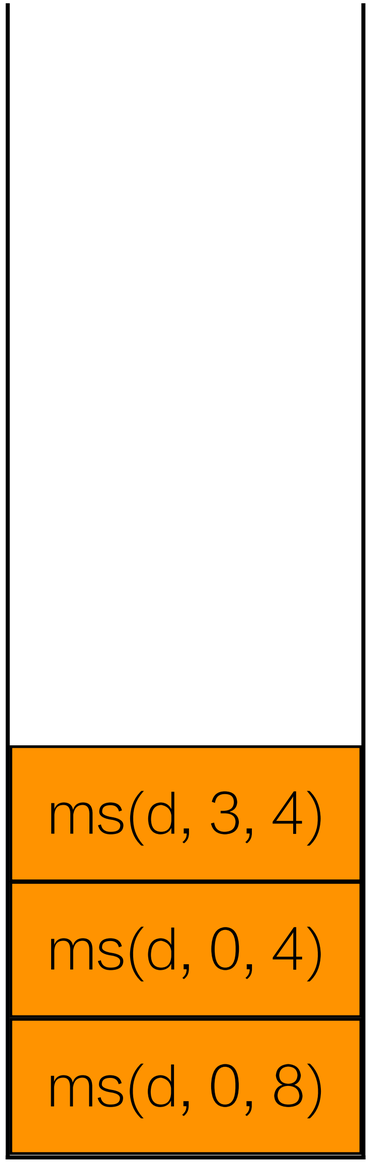
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

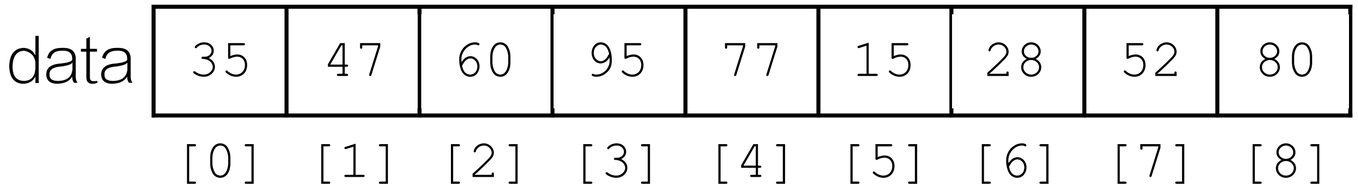
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

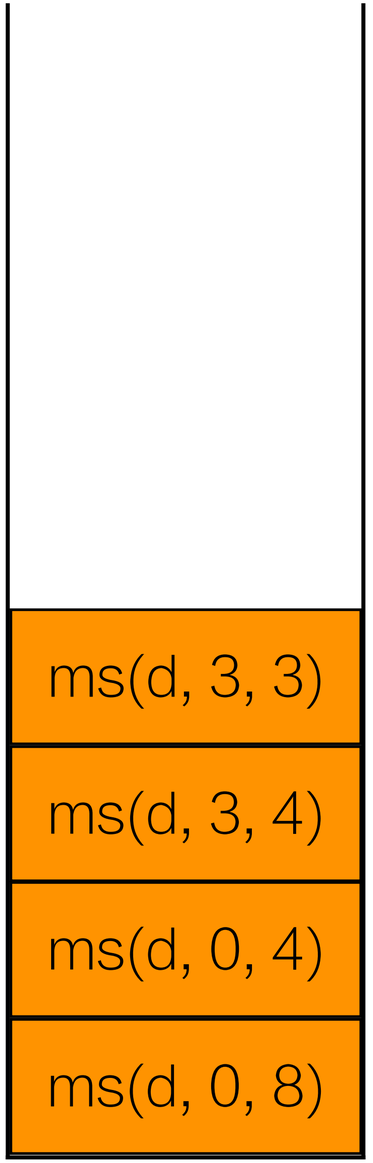
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

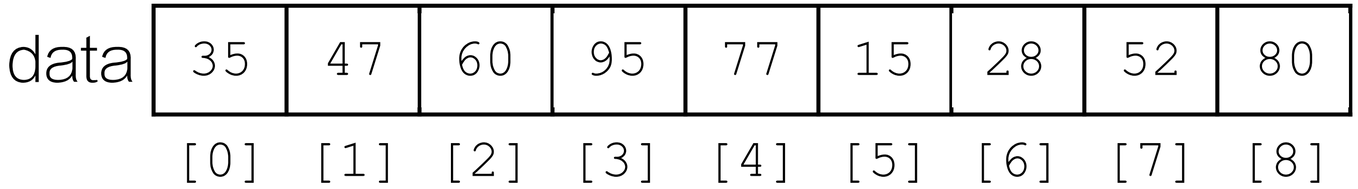
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：3　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

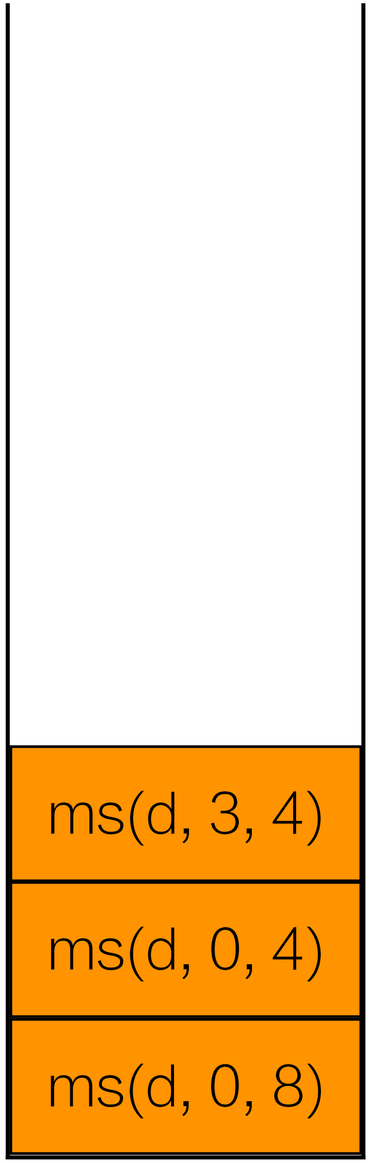
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

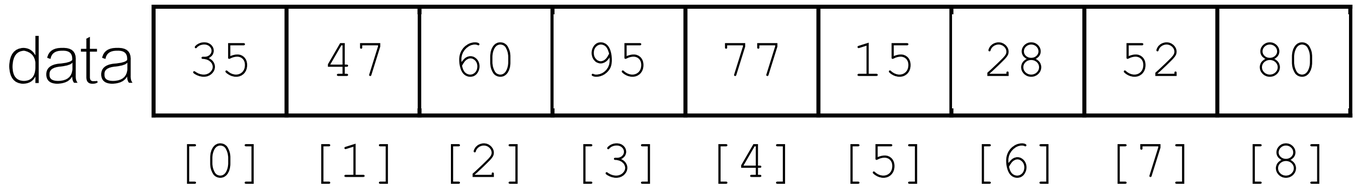
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

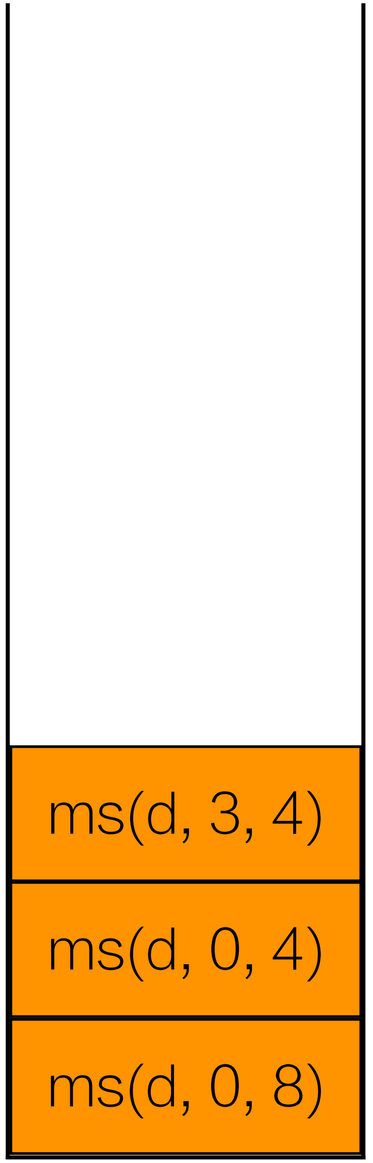
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

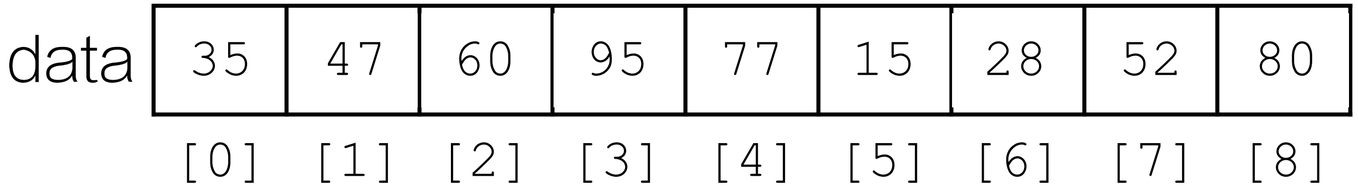
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

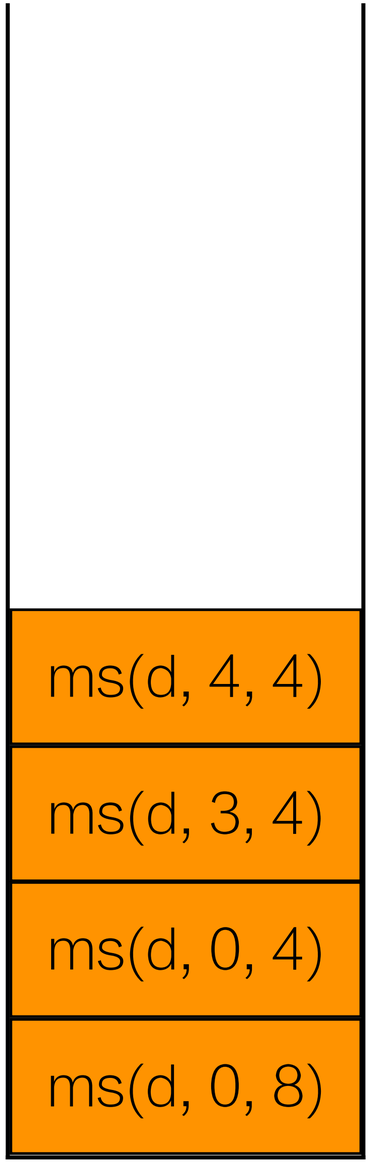
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

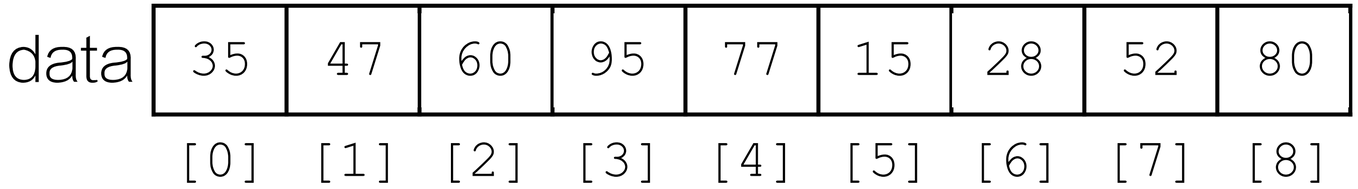
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：4　　right：4　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

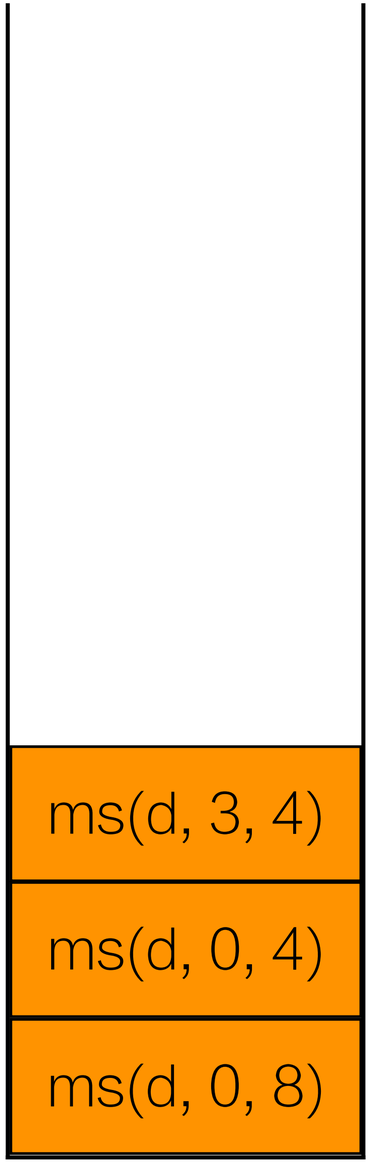
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

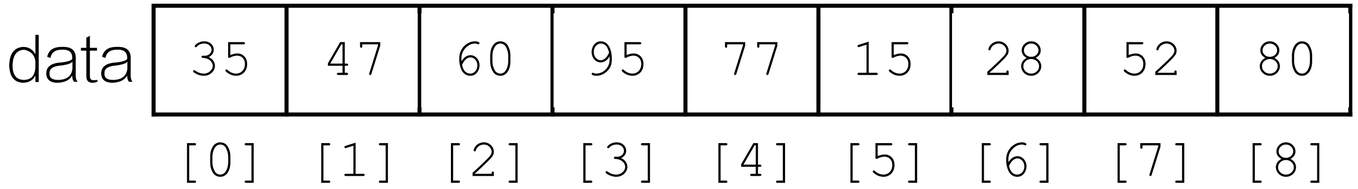
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

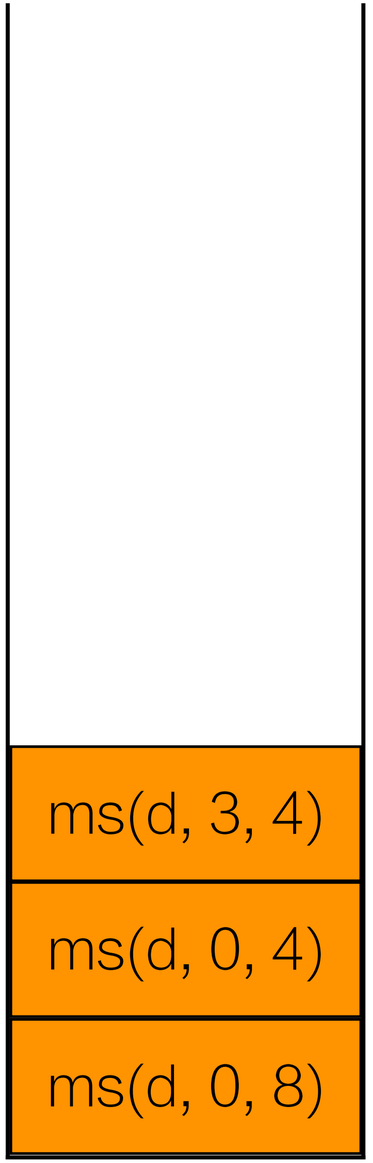
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3



data

35	47	60	95	77	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

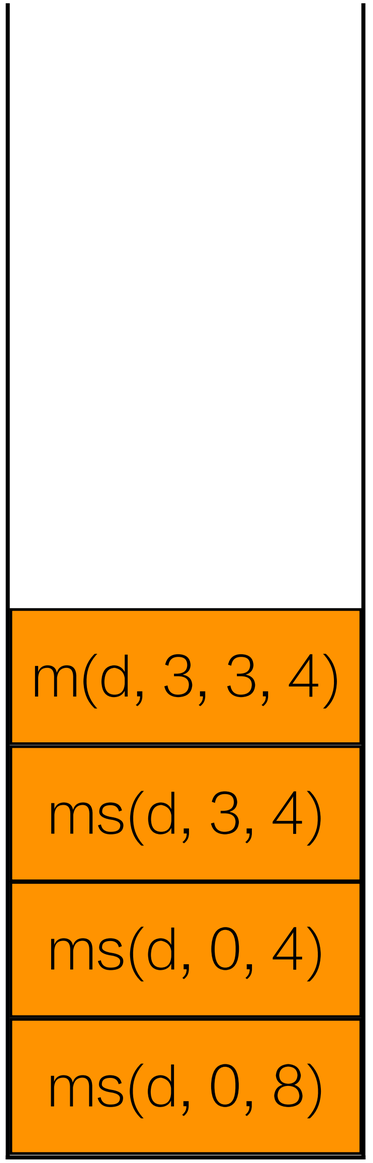
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　mid：3　　right：4



data	35	47	60	95	77	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

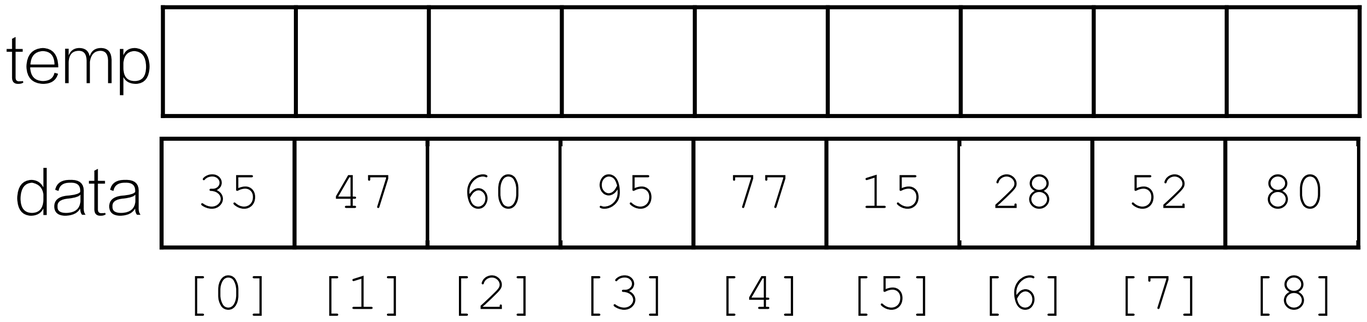
        // 对左半部分排序
        mergeSort(data, left, mid);

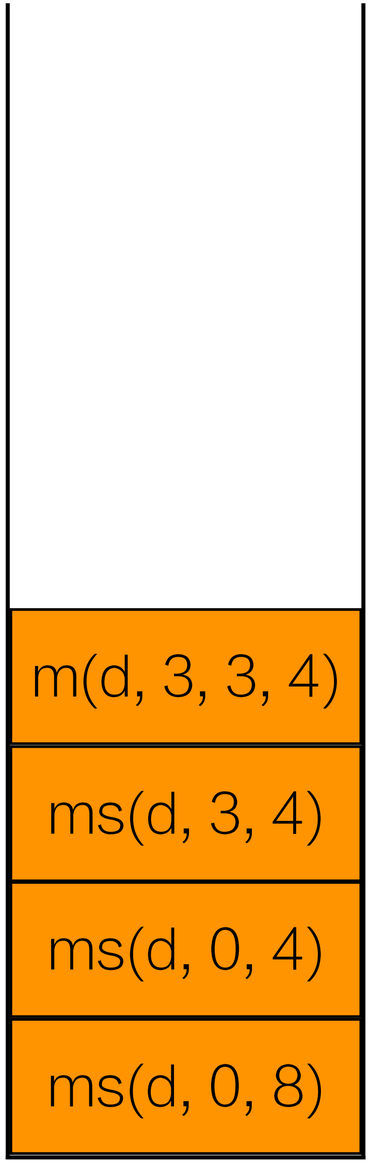
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 3　　j: 4　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

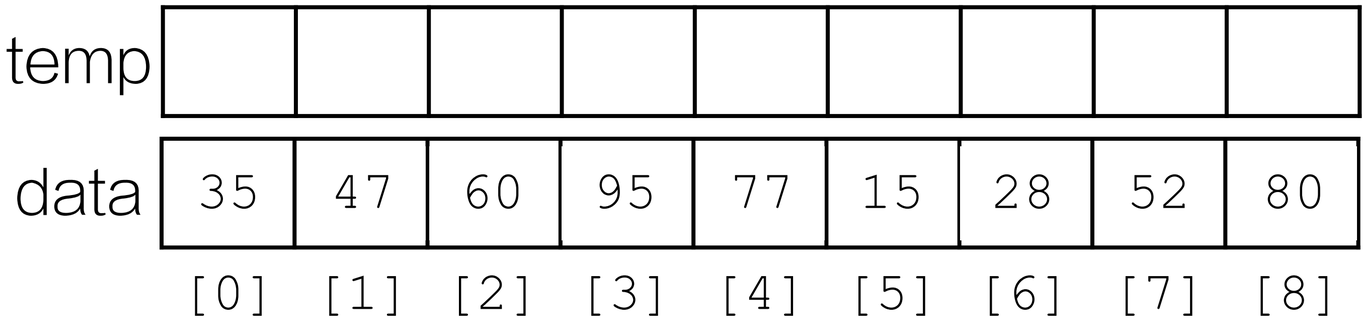
        // 对左半部分排序
        mergeSort(data, left, mid);

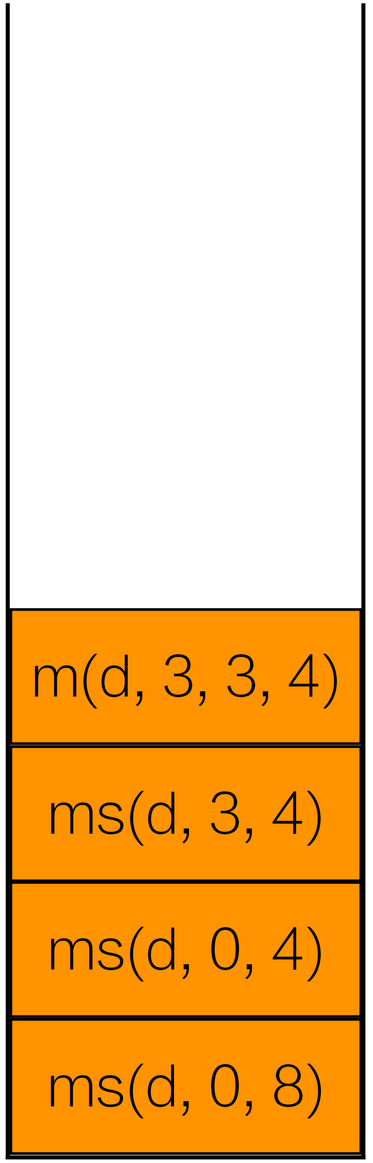
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 3　　j: 4　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

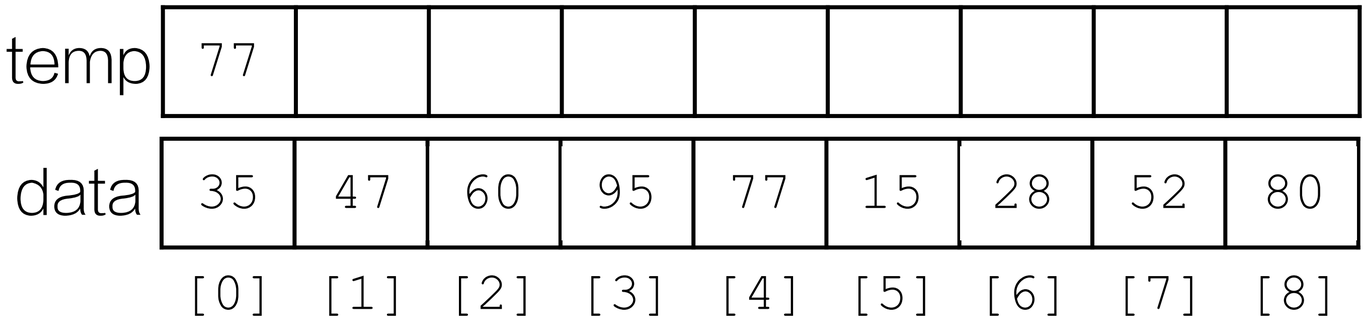
        // 对左半部分排序
        mergeSort(data, left, mid);

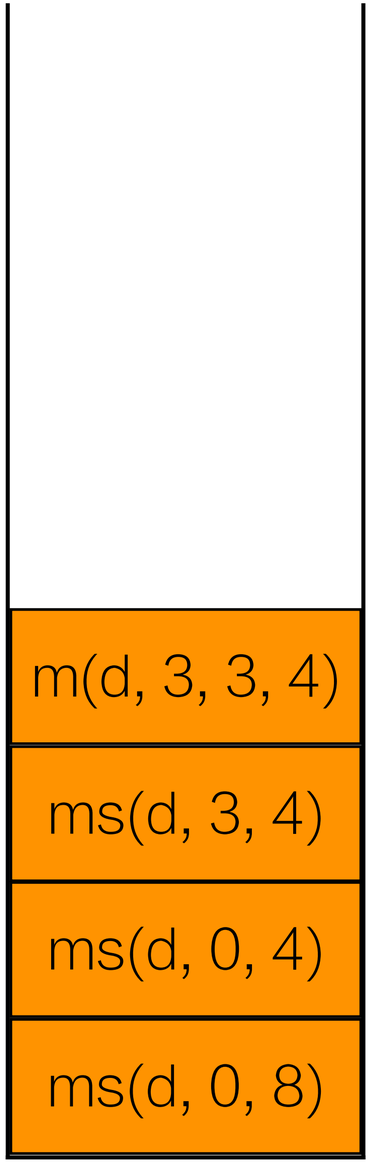
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 3　　j: 5　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

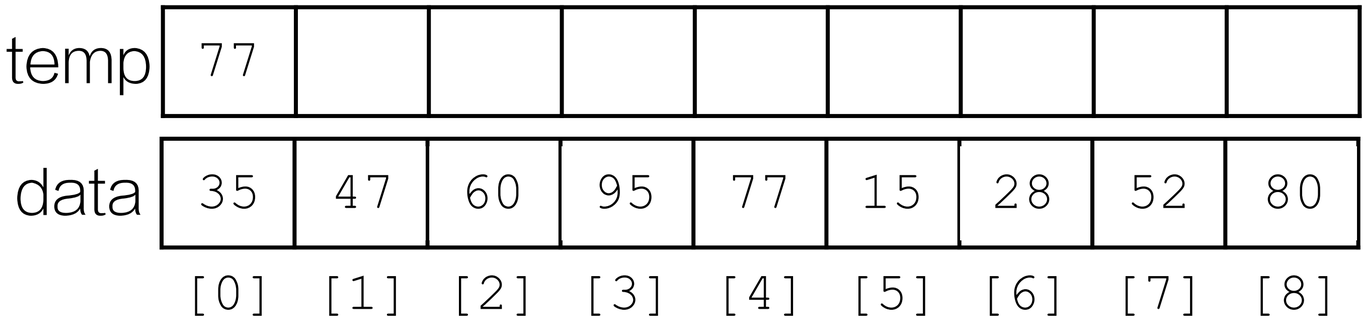
        // 对左半部分排序
        mergeSort(data, left, mid);

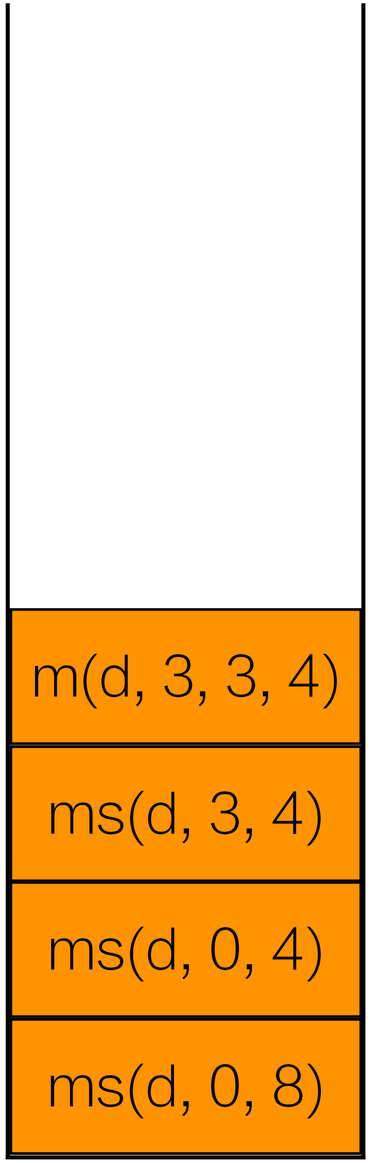
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 3　　j: 5　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

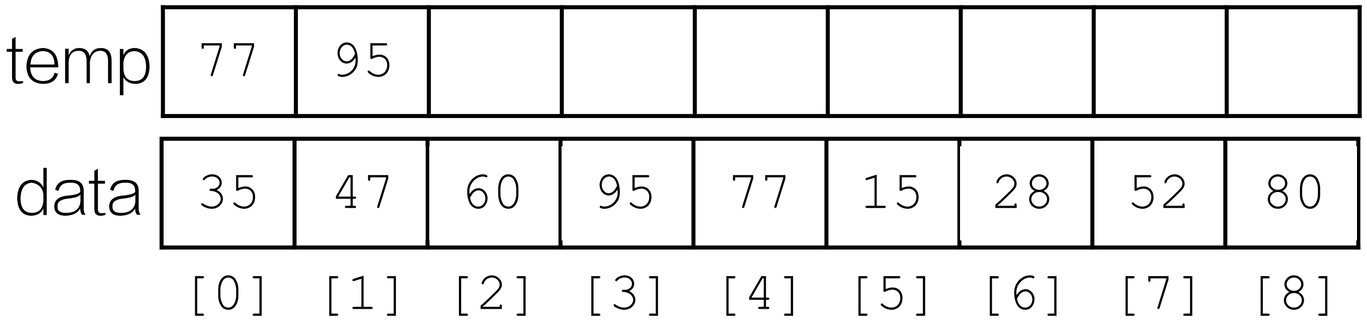
        // 对左半部分排序
        mergeSort(data, left, mid);

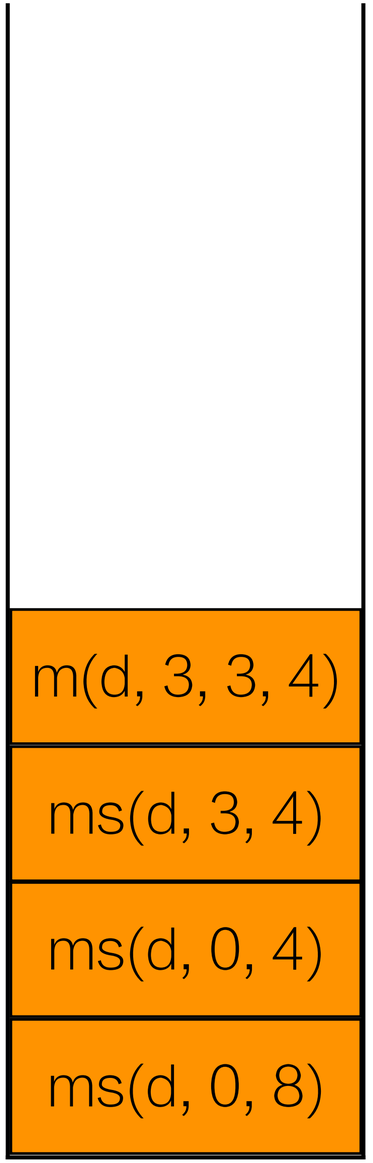
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 4　　j: 5　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

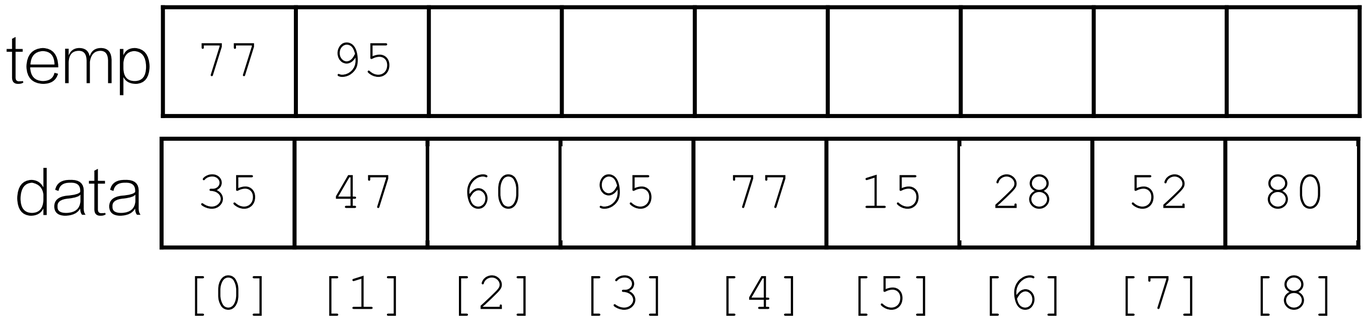
        // 对左半部分排序
        mergeSort(data, left, mid);

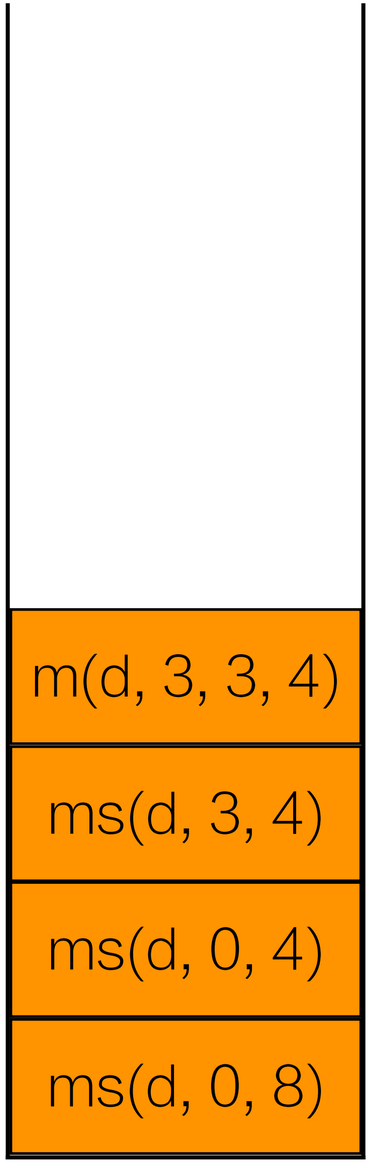
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 4　　j: 5　　k: 2





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

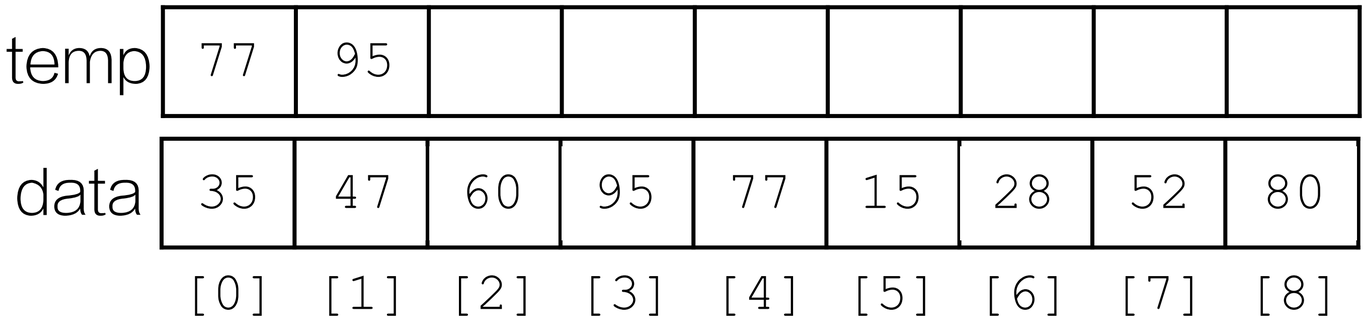
        // 对左半部分排序
        mergeSort(data, left, mid);

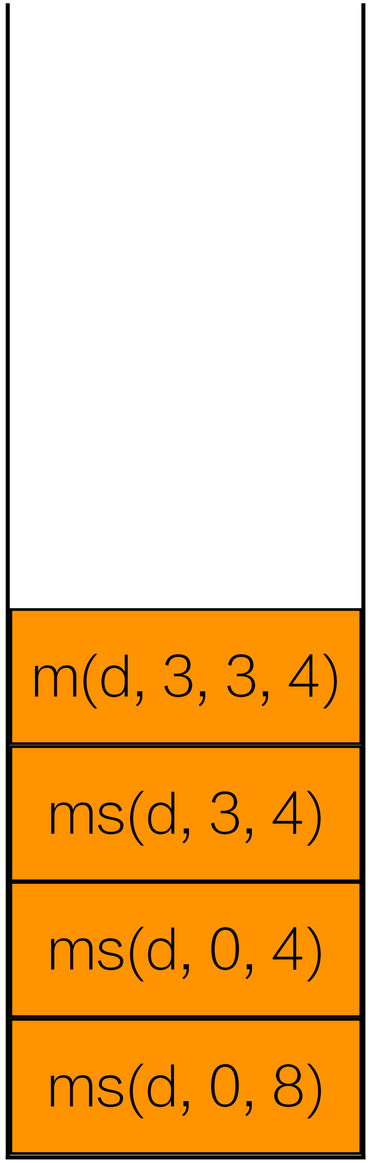
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 4　　j: 5　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

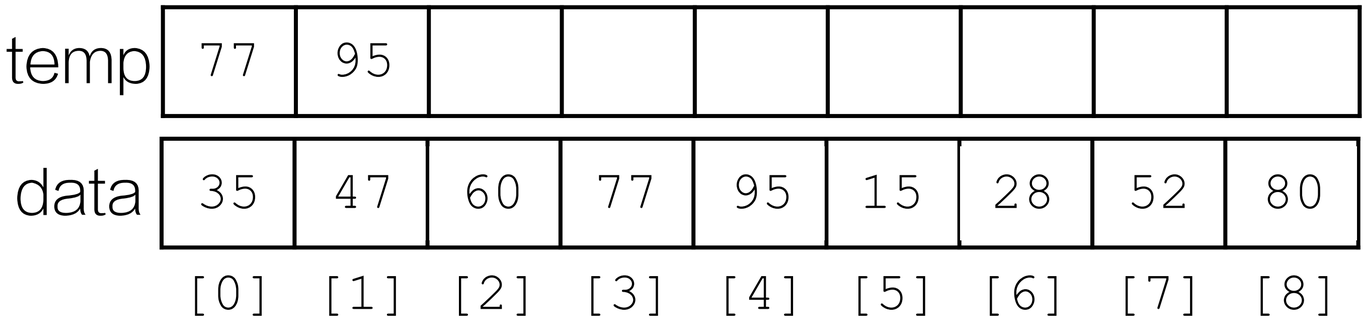
        // 对左半部分排序
        mergeSort(data, left, mid);

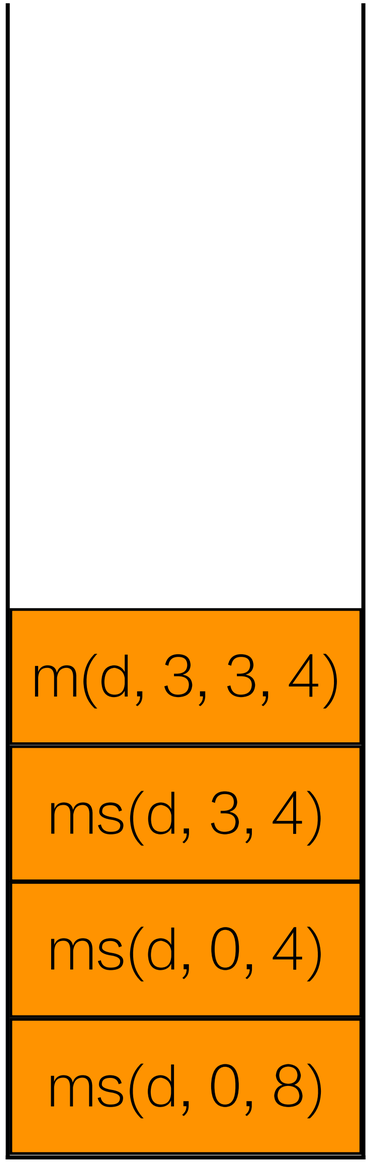
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 3　　mid: 3　　right: 4

i: 4　　j: 5　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

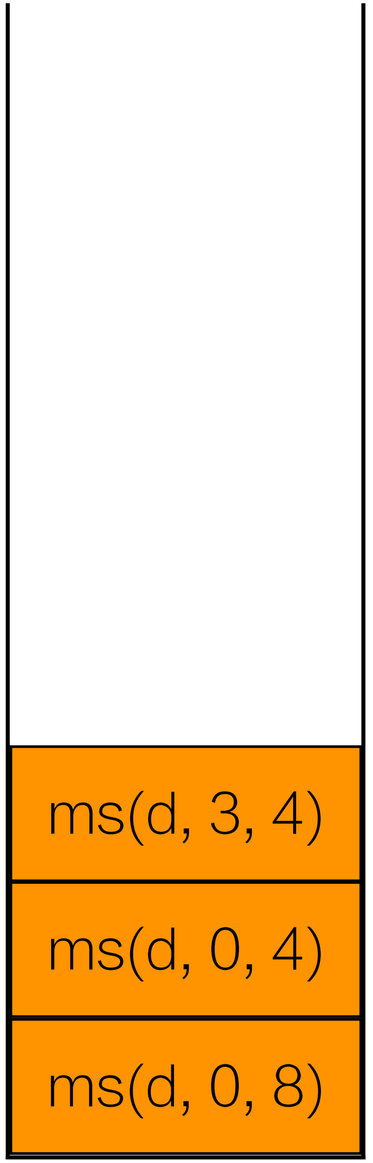
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

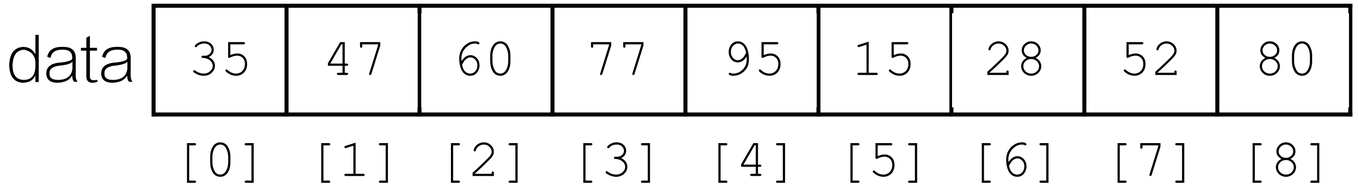
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：3　　right：4　　mid：3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

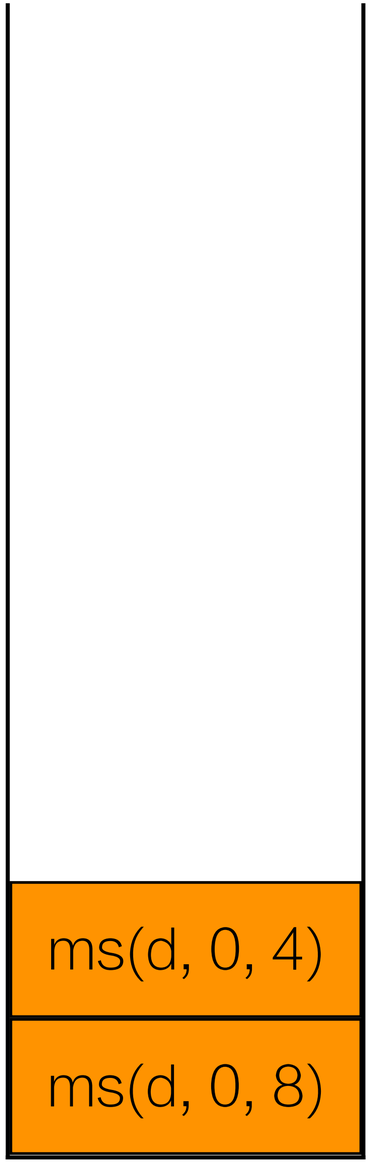
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

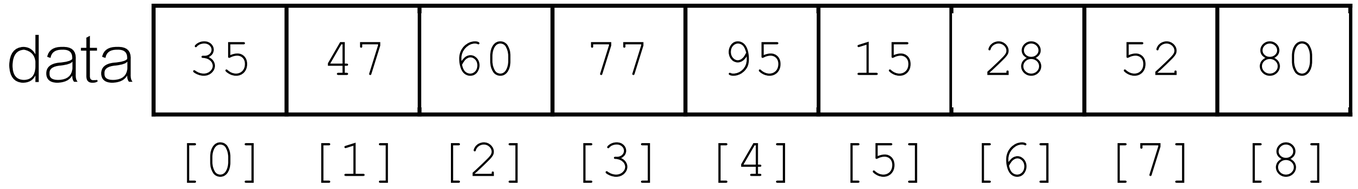
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：4　　mid：2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

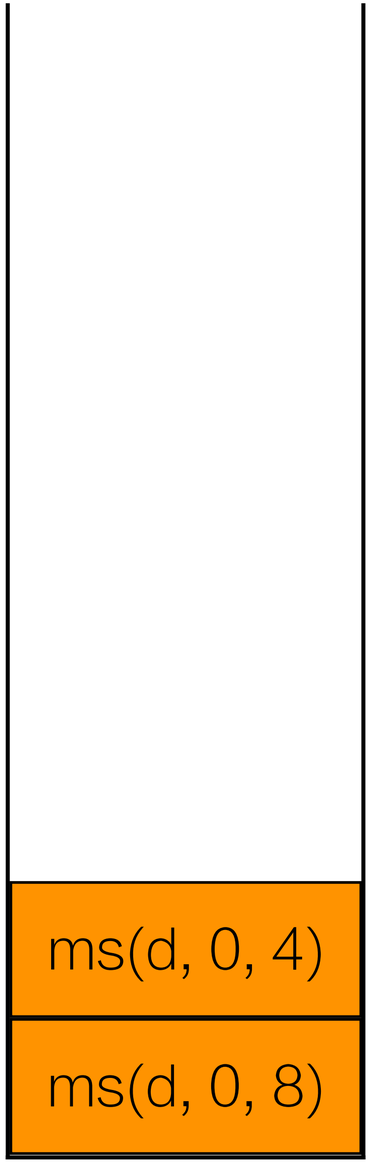
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

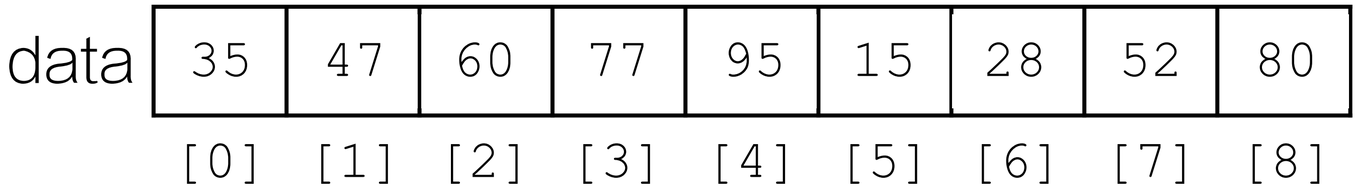
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：4　　mid：2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

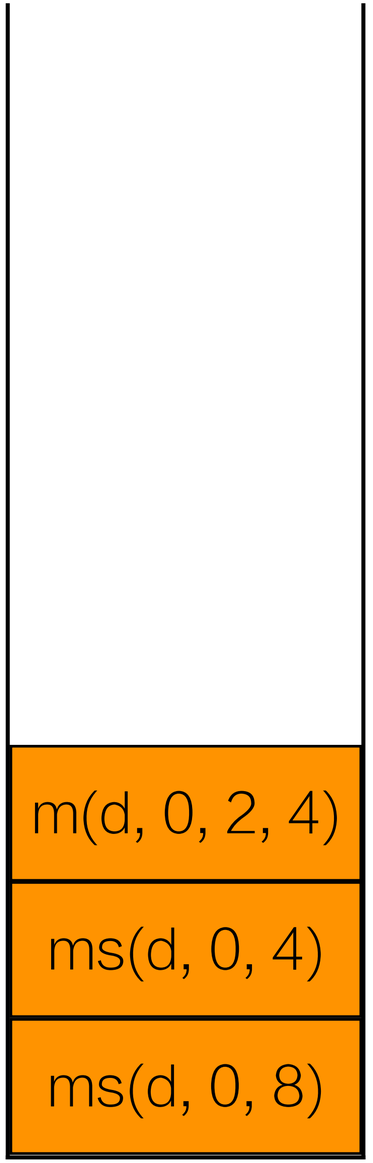
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

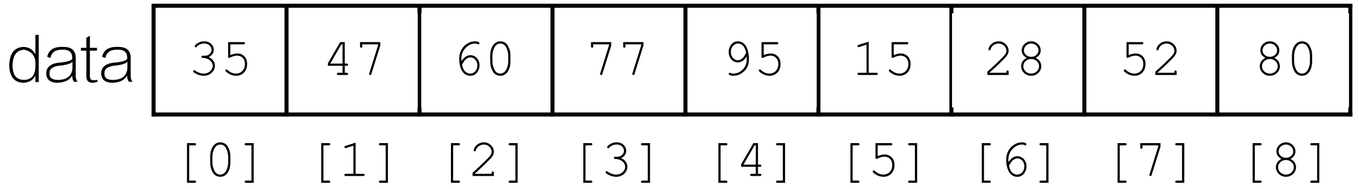
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　mid：2　　right：4





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

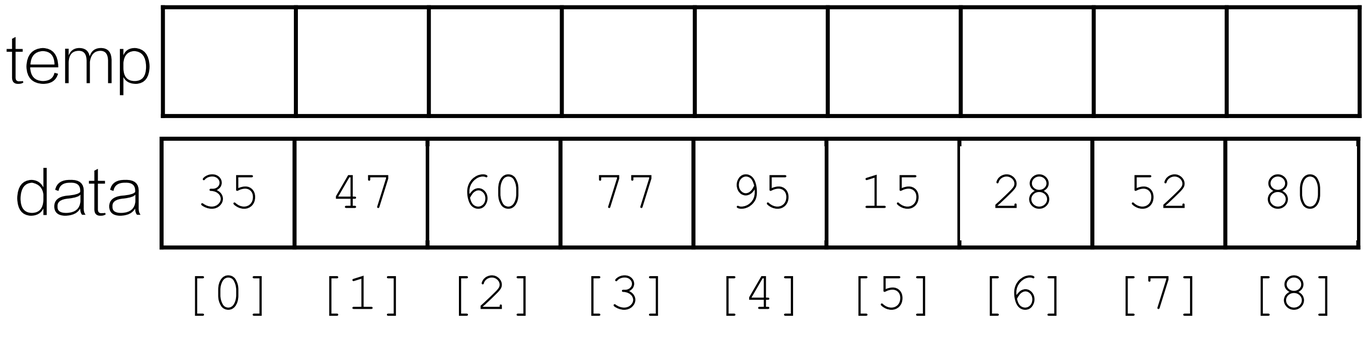
        // 对左半部分排序
        mergeSort(data, left, mid);

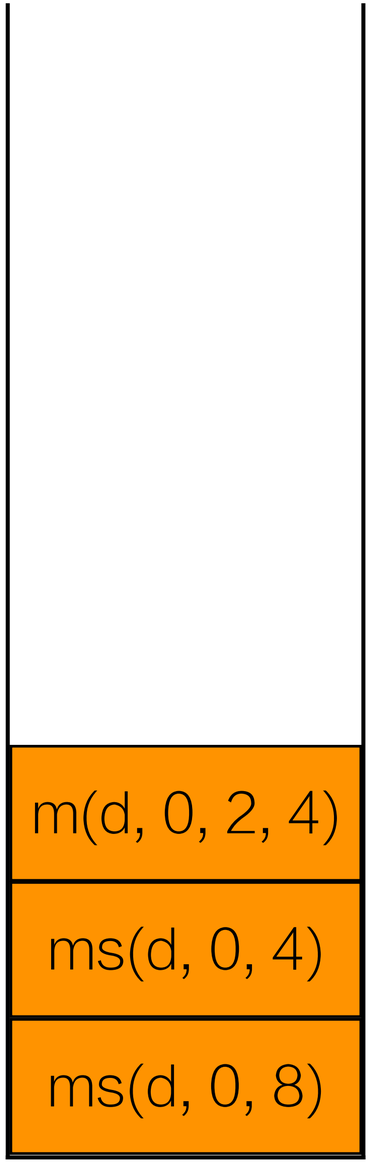
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 0　　j: 3　　k: 0





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

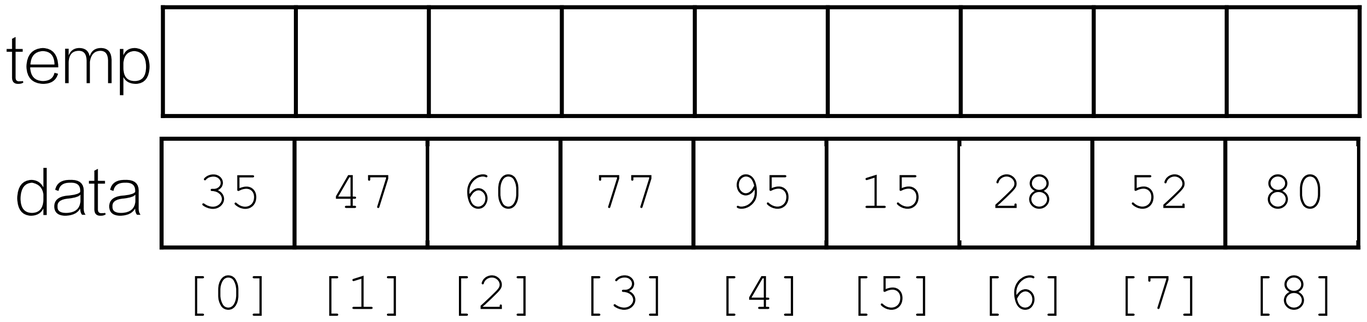
        // 对左半部分排序
        mergeSort(data, left, mid);

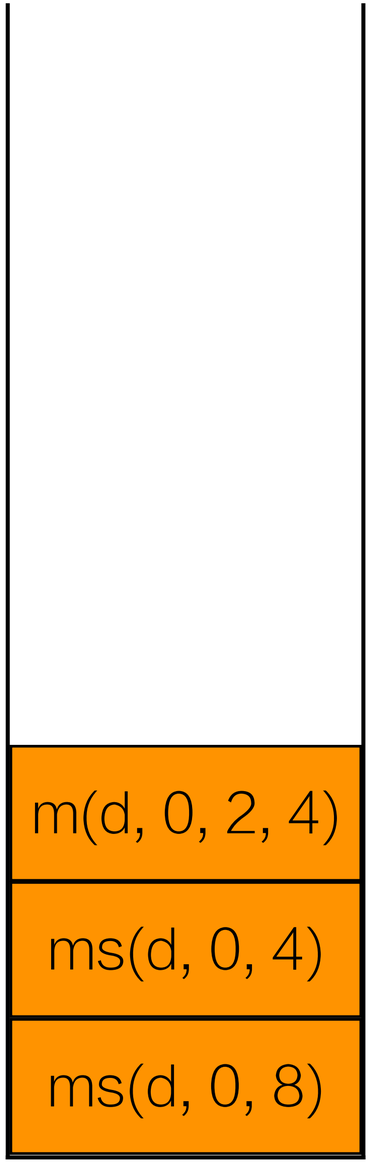
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 0　　j: 3　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

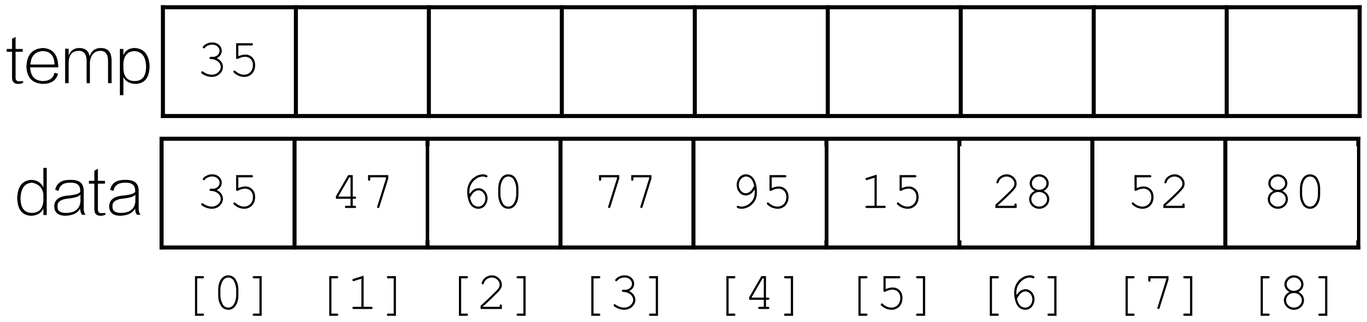
        // 对左半部分排序
        mergeSort(data, left, mid);

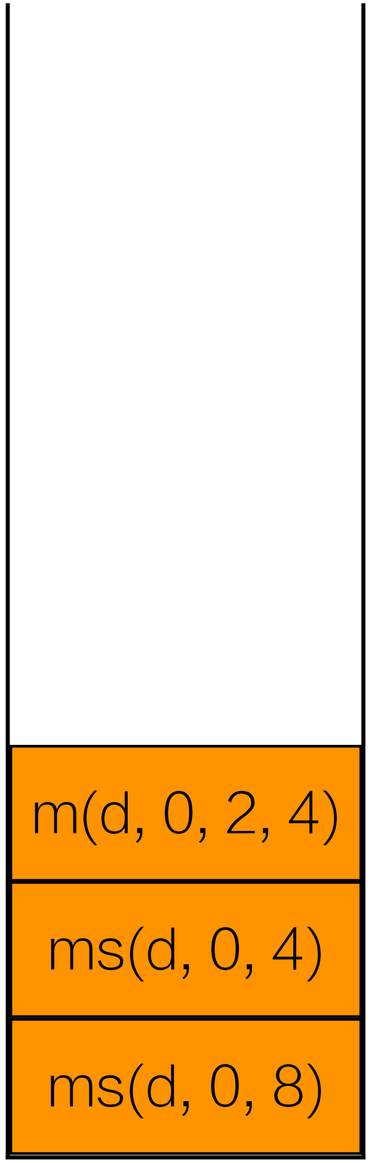
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 1　　j: 3　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

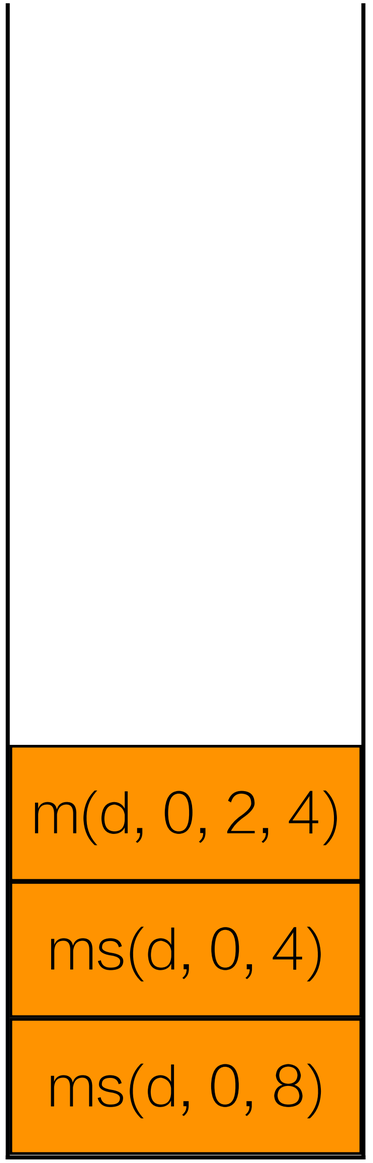
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 2　　j: 3　　k: 2

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35	47							
data	35	47	60	77	95	15	28	52	80



```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

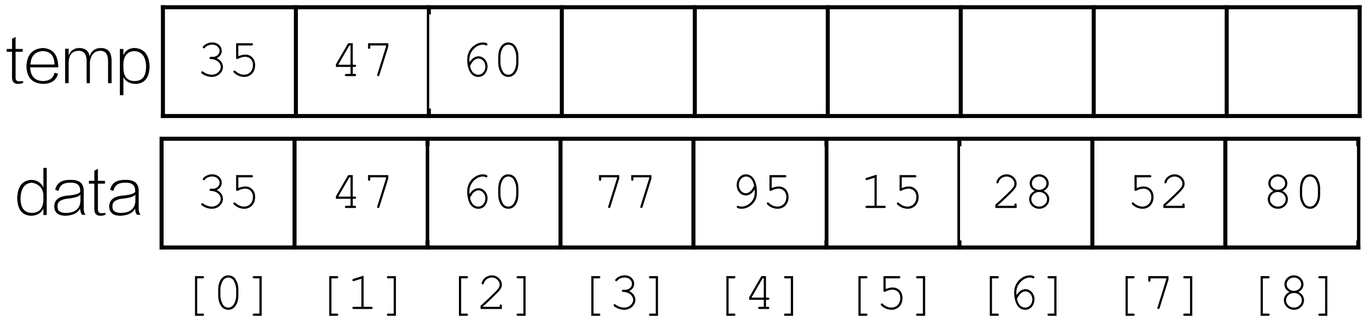
        // 对左半部分排序
        mergeSort(data, left, mid);

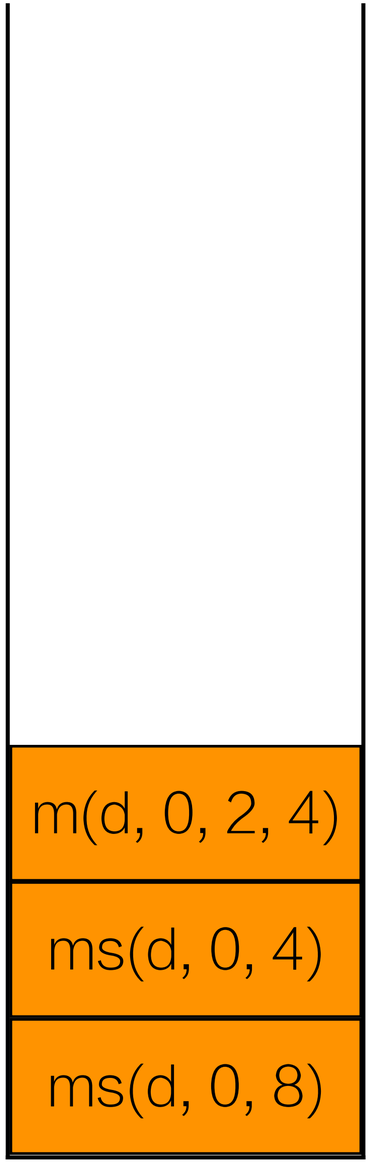
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 3　　k: 3





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

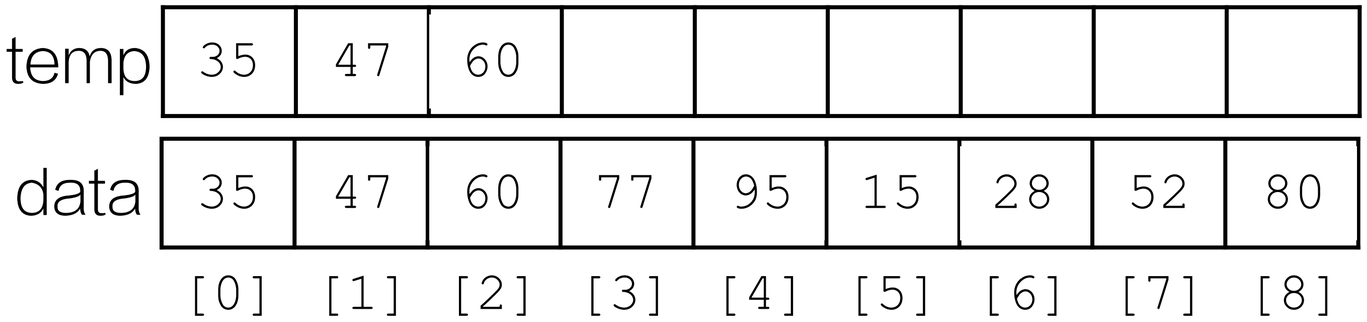
        // 对左半部分排序
        mergeSort(data, left, mid);

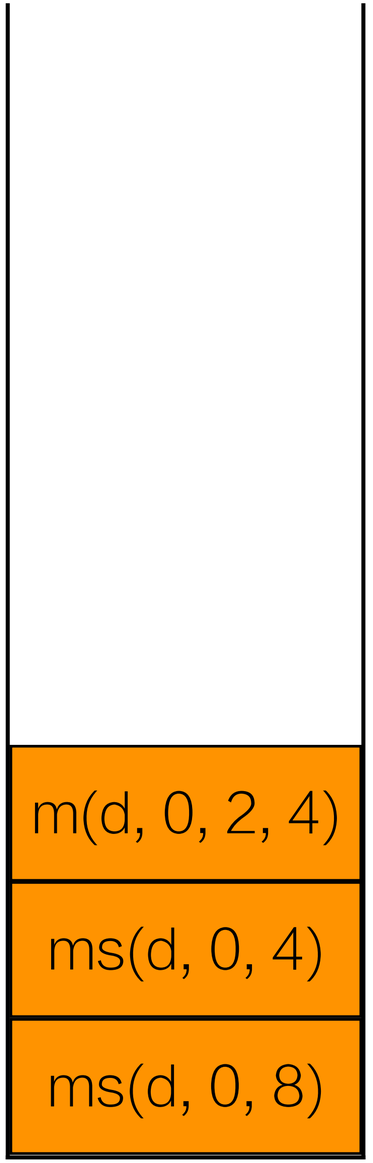
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 3　　k: 3





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

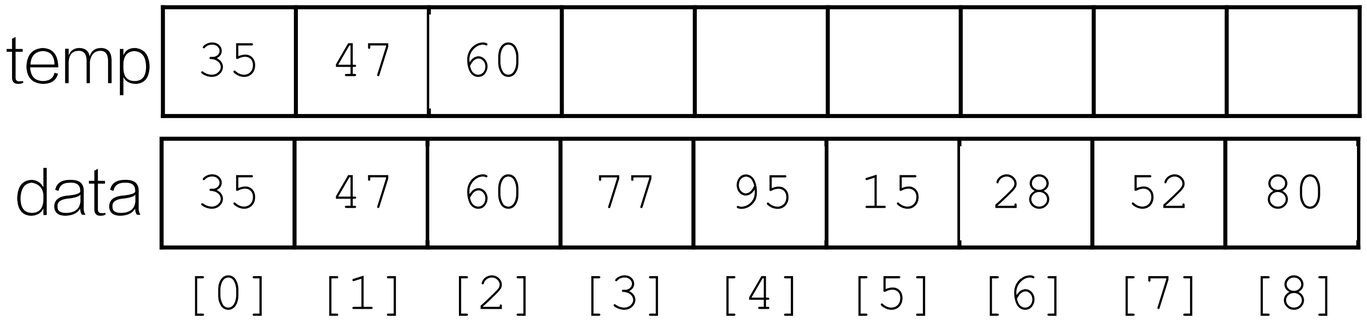
        // 对左半部分排序
        mergeSort(data, left, mid);

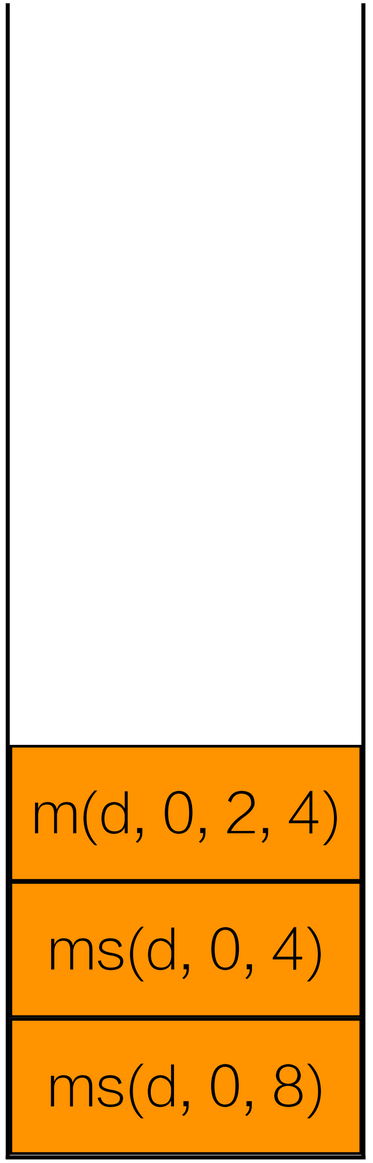
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 3　　k: 3





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 4　　k: 4

m(d, 0, 2, 4)

ms(d, 0, 4)

ms(d, 0, 8)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35	47	60	77					
data	35	47	60	77	95	15	28	52	80

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

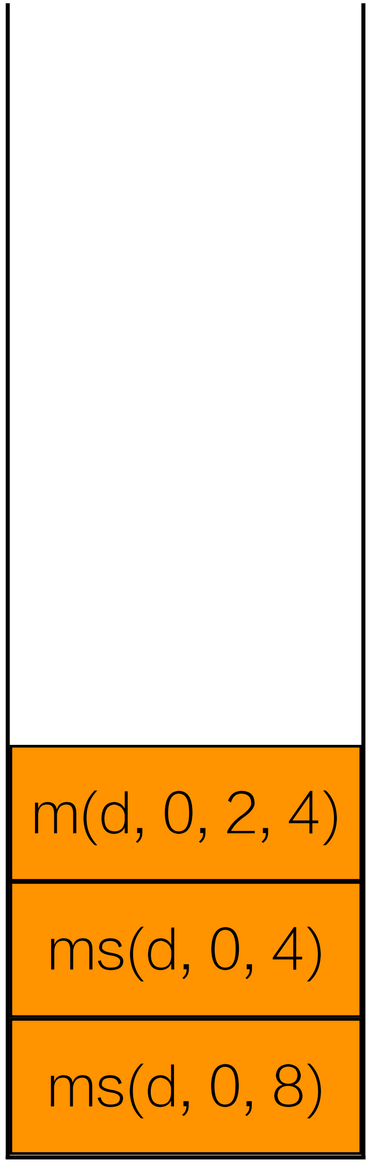
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 5　　k: 5

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	35	47	60	77	95				
data	35	47	60	77	95	15	28	52	80



```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

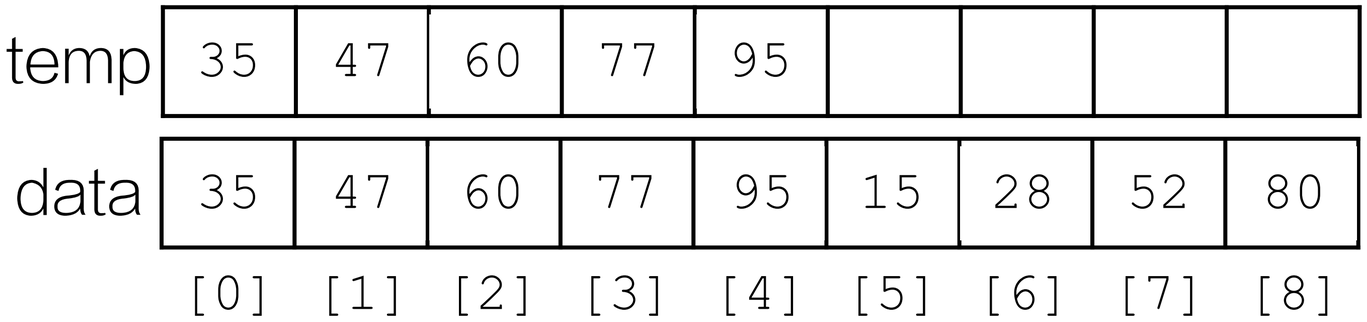
        // 对左半部分排序
        mergeSort(data, left, mid);

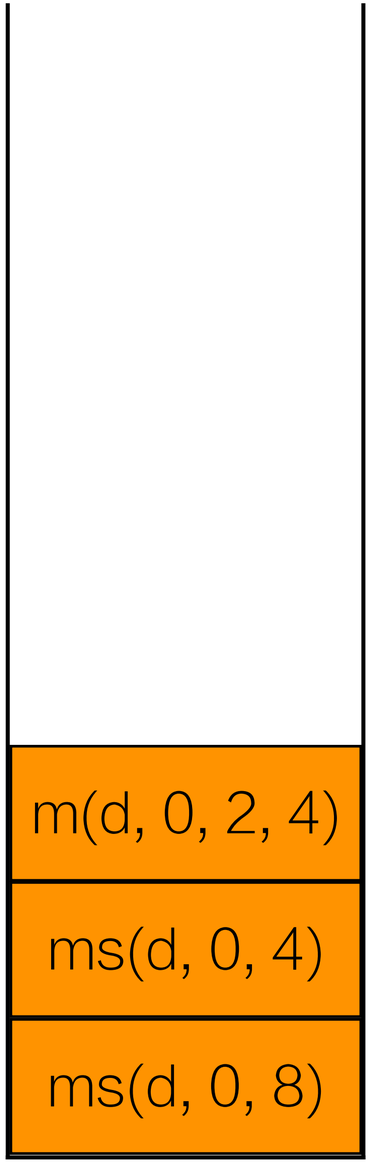
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 2　　right: 4

i: 3　　j: 5　　k: 5





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

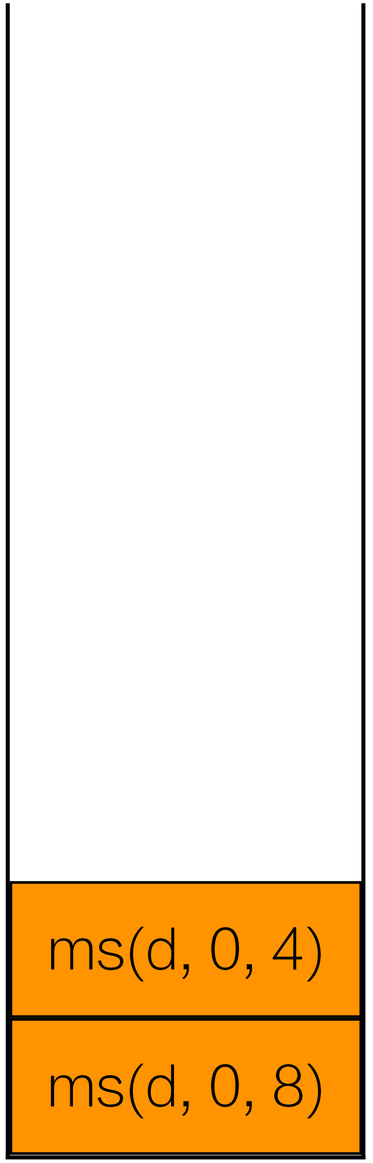
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

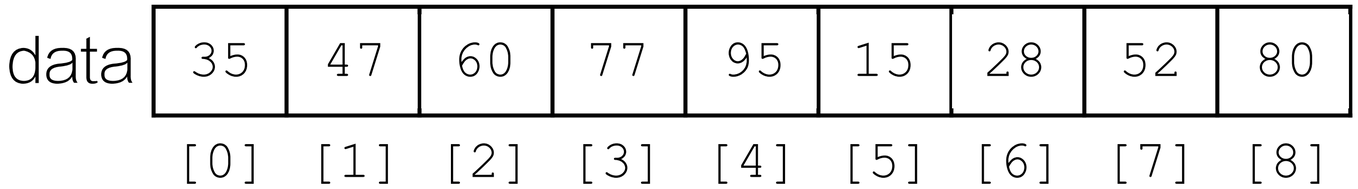
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：4　　mid：2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：8　　mid：4

data

35	47	60	77	95	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：8　　mid：4

data	35	47	60	77	95	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

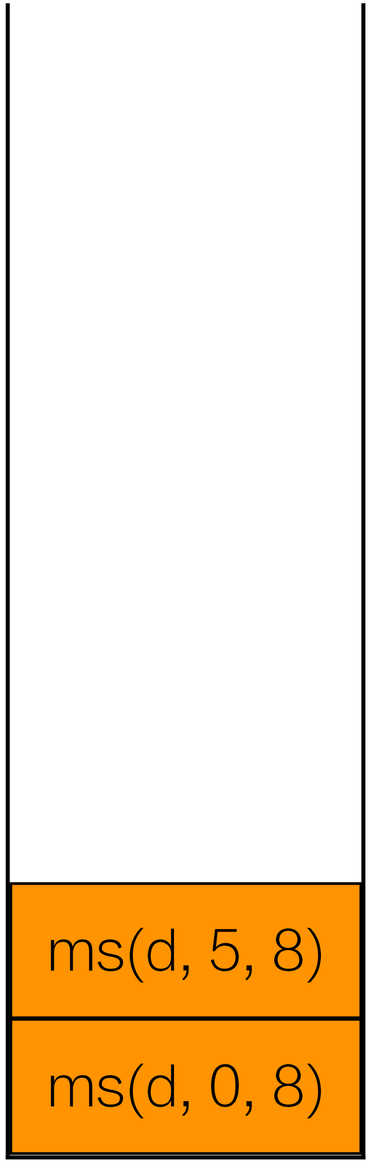
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

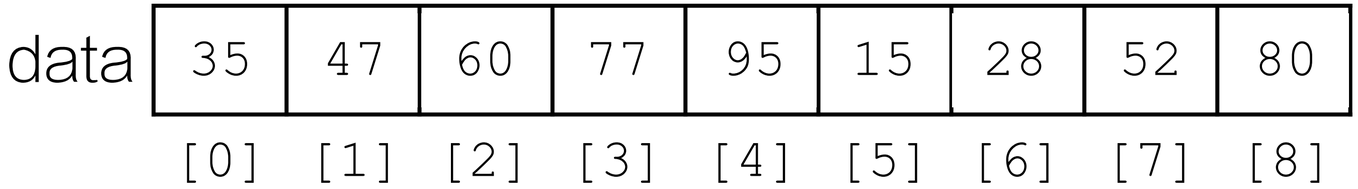
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

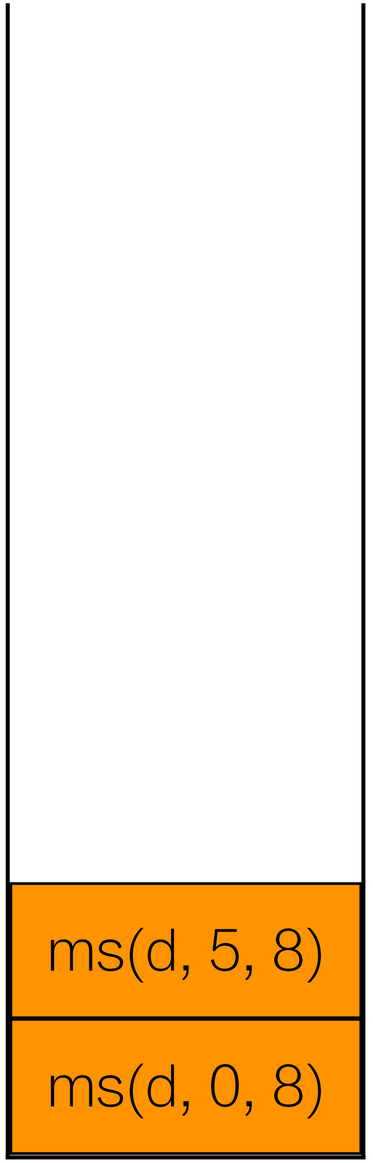
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

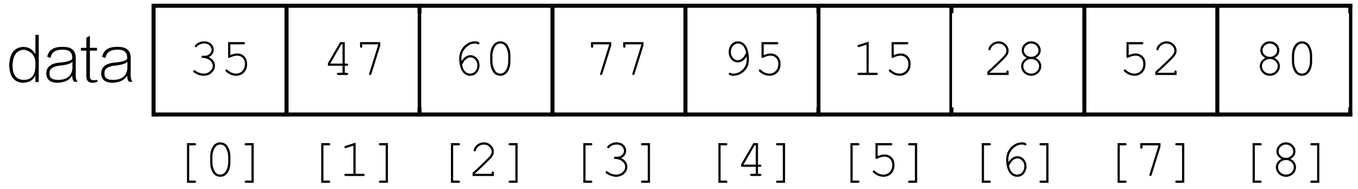
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

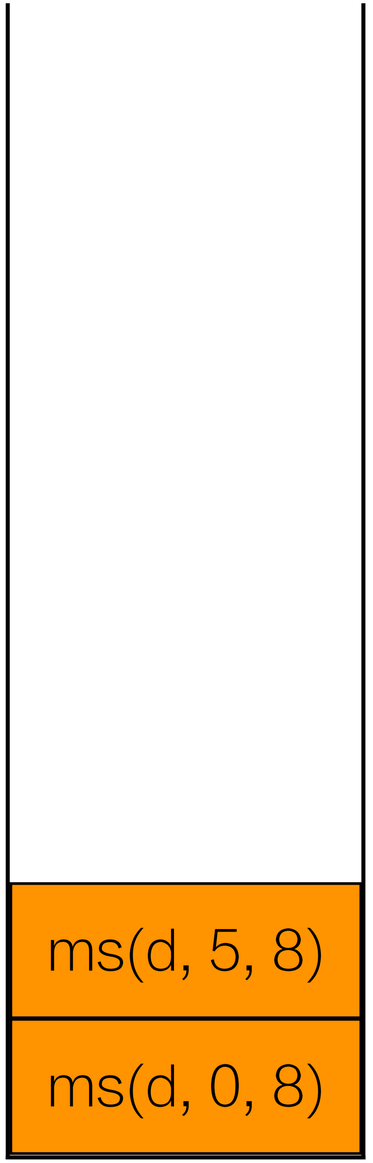
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6



data

35	47	60	77	95	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

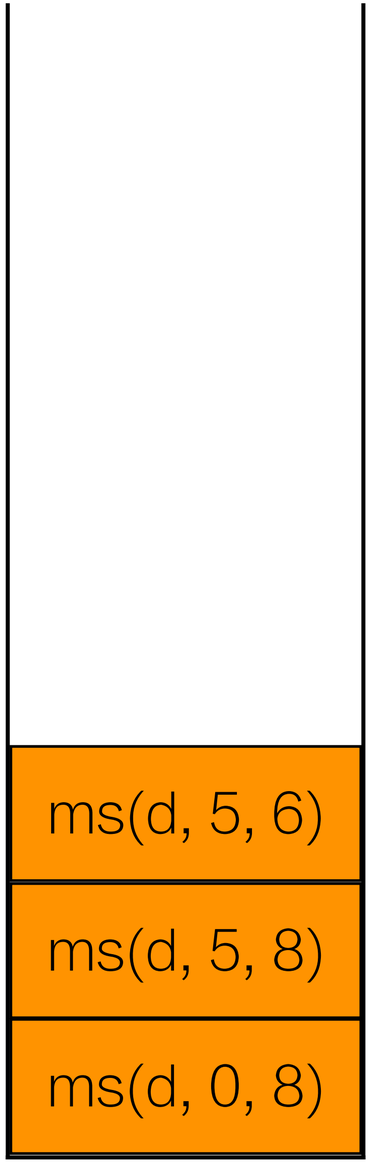
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

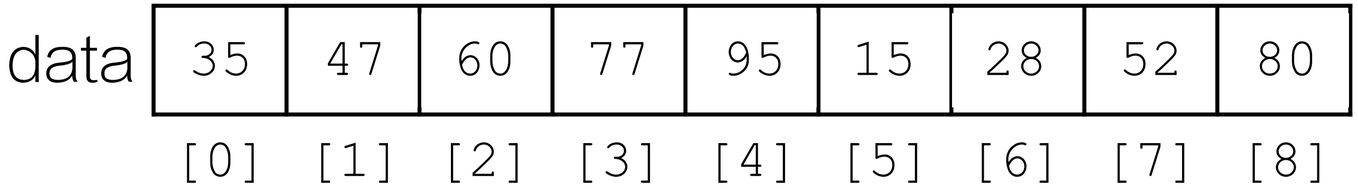
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

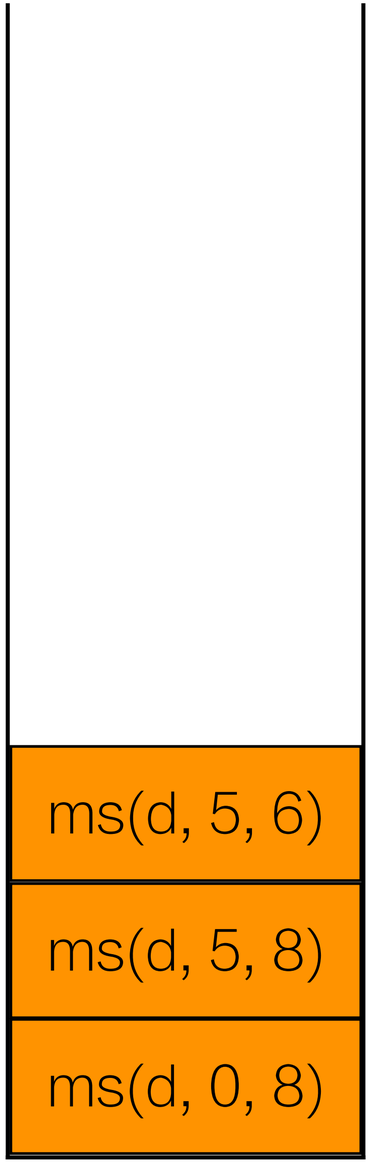
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

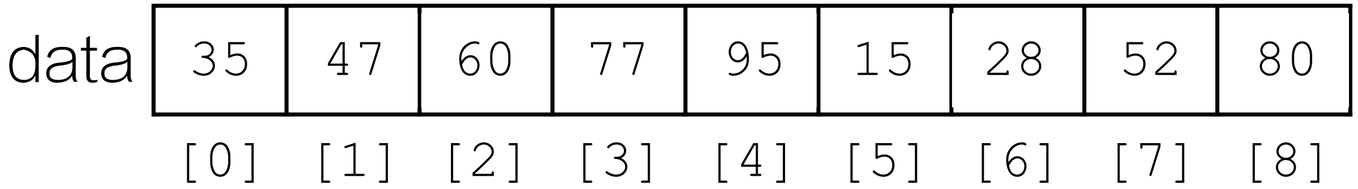
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

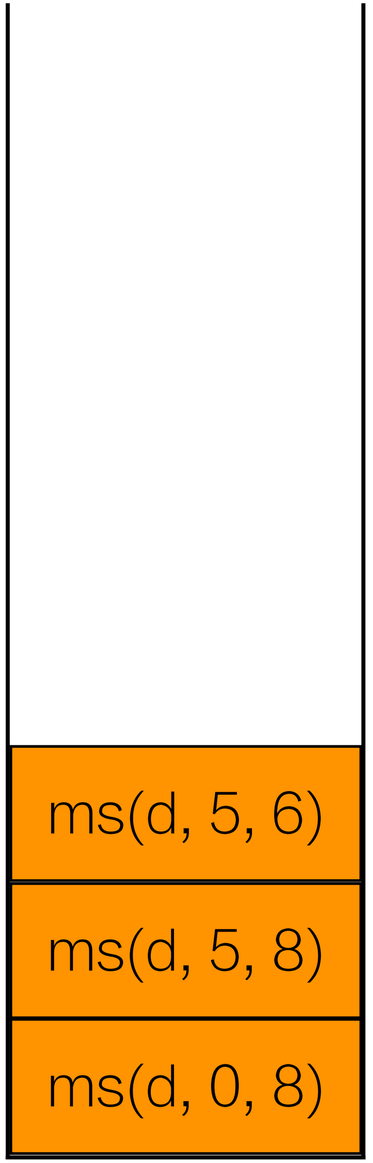
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

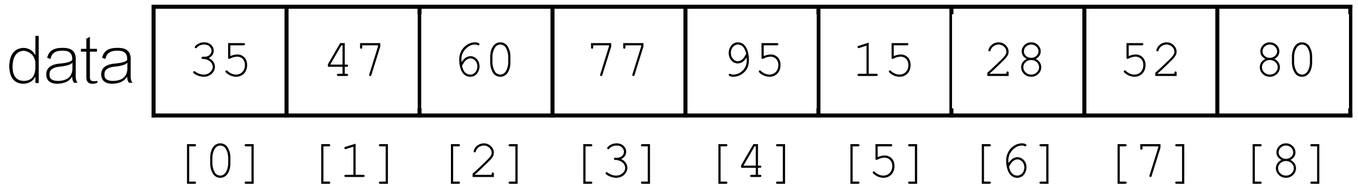
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

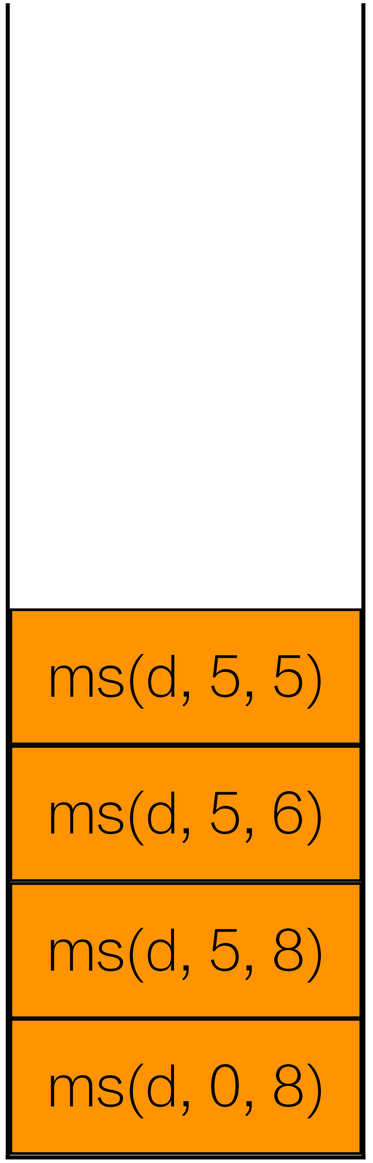
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

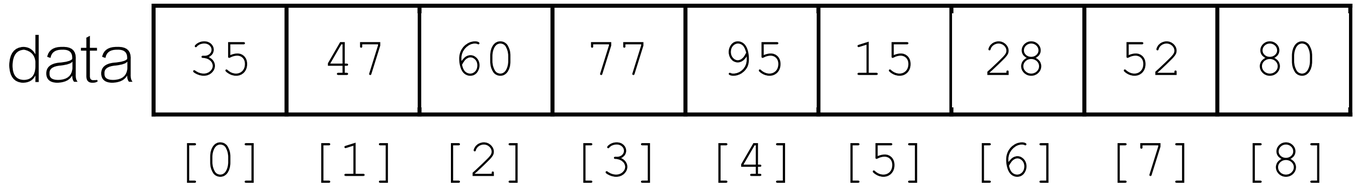
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：5　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

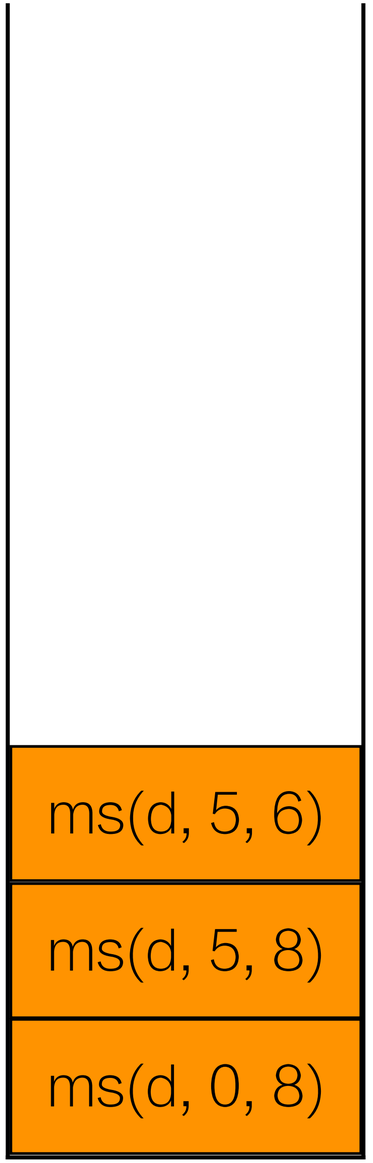
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

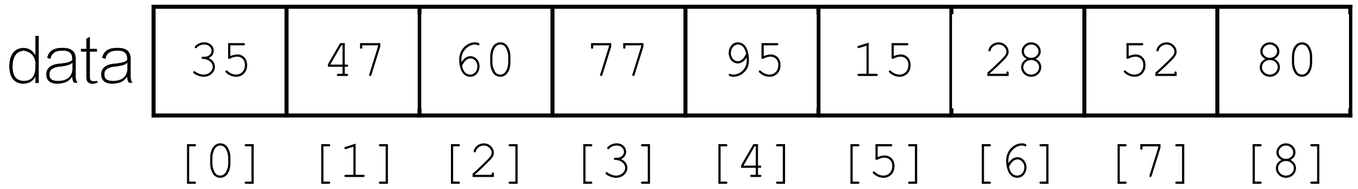
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

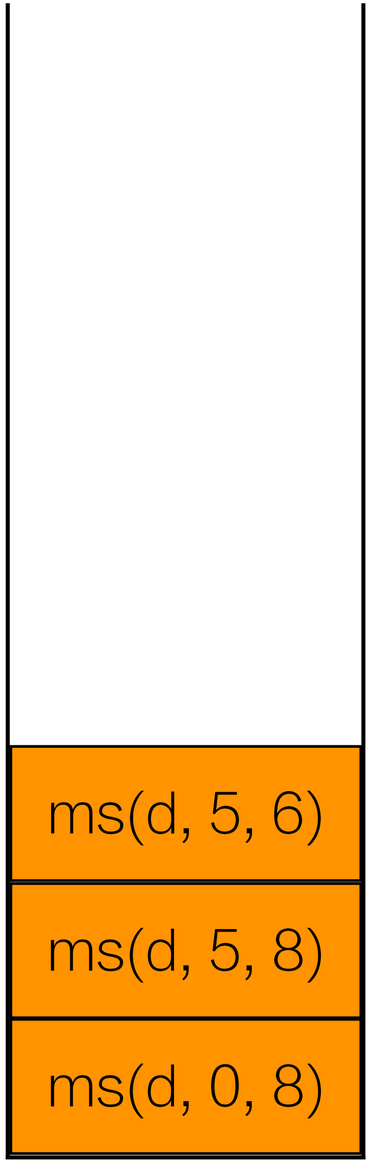
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5



data

35	47	60	77	95	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

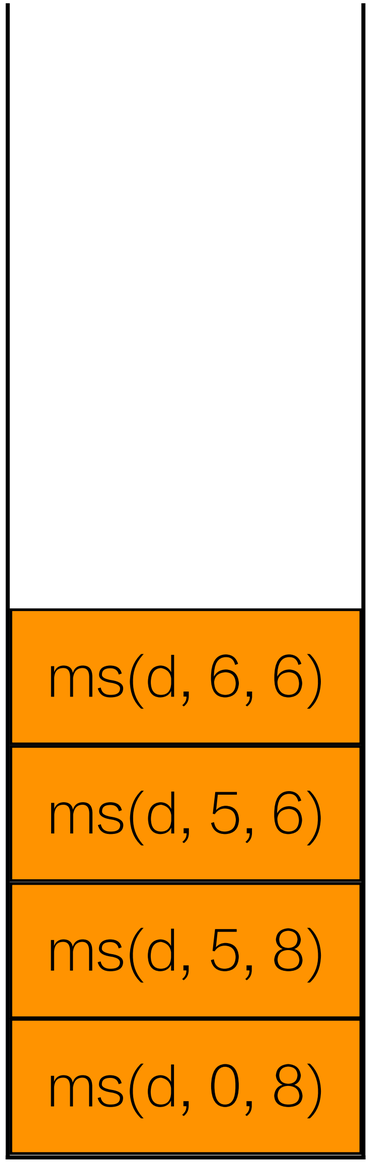
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

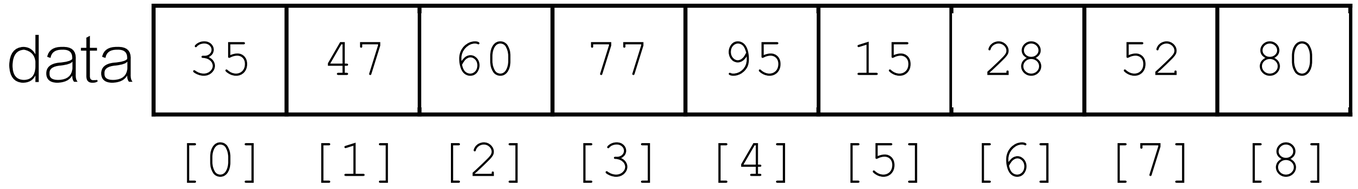
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：6　　right：6　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

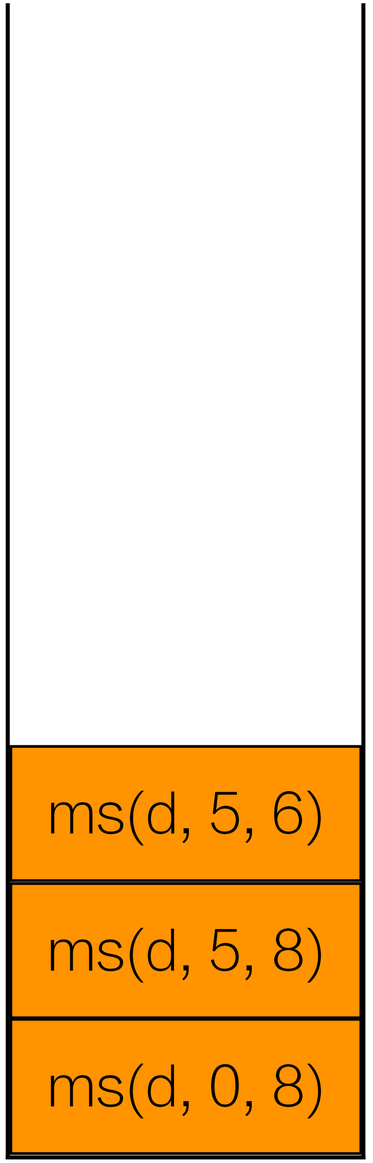
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5



data

35	47	60	77	95	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

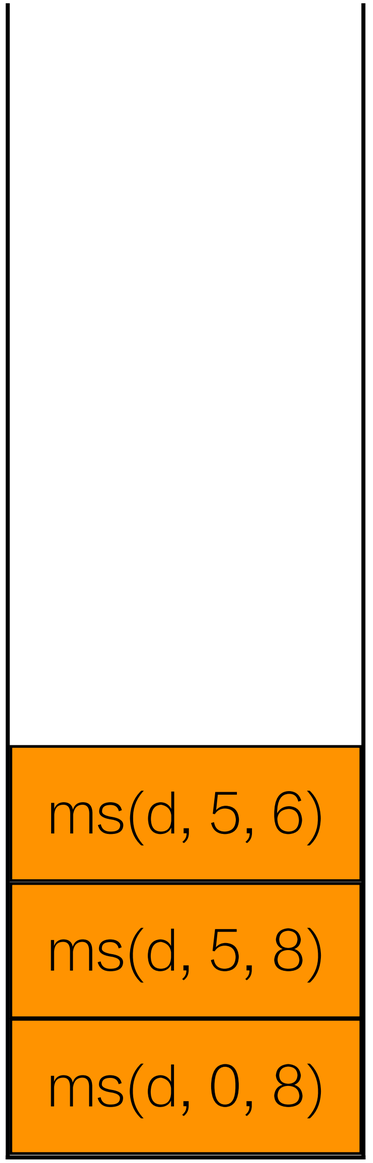
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

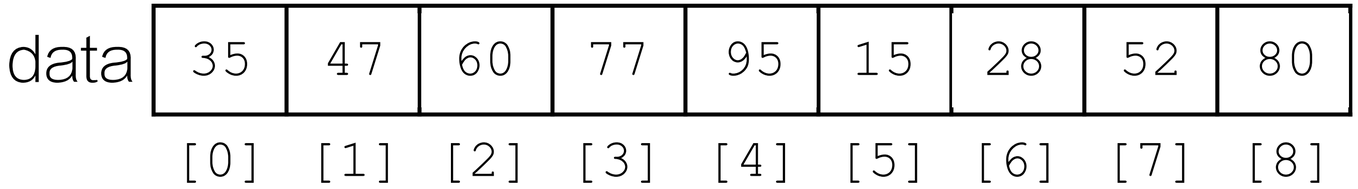
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

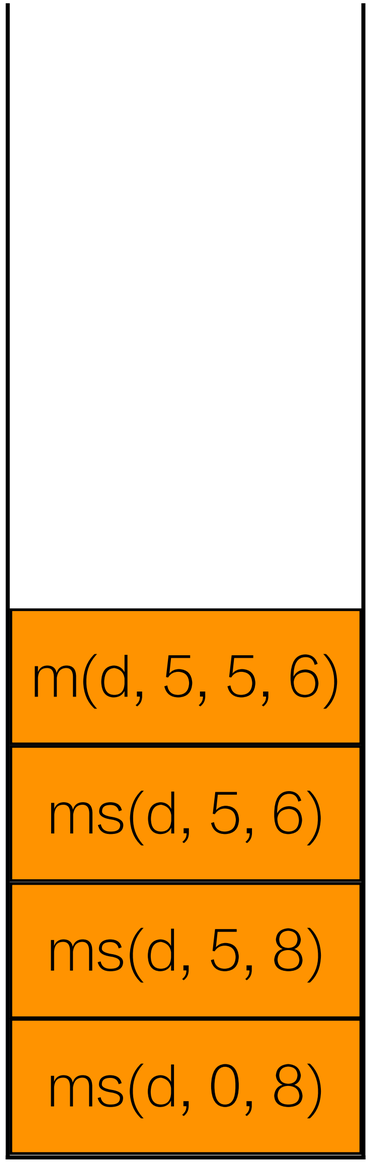
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

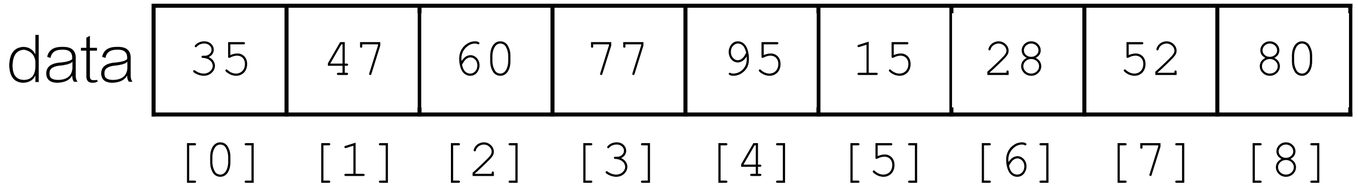
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　mid：5　　right：6





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

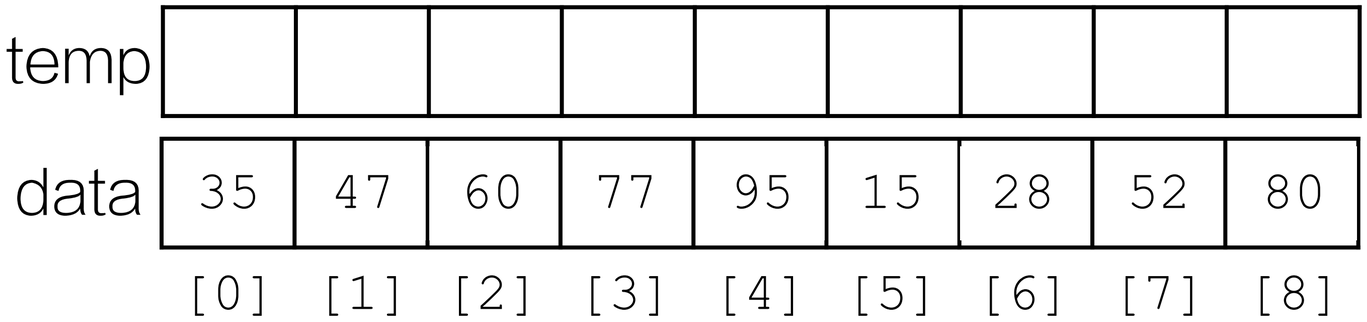
        // 对左半部分排序
        mergeSort(data, left, mid);

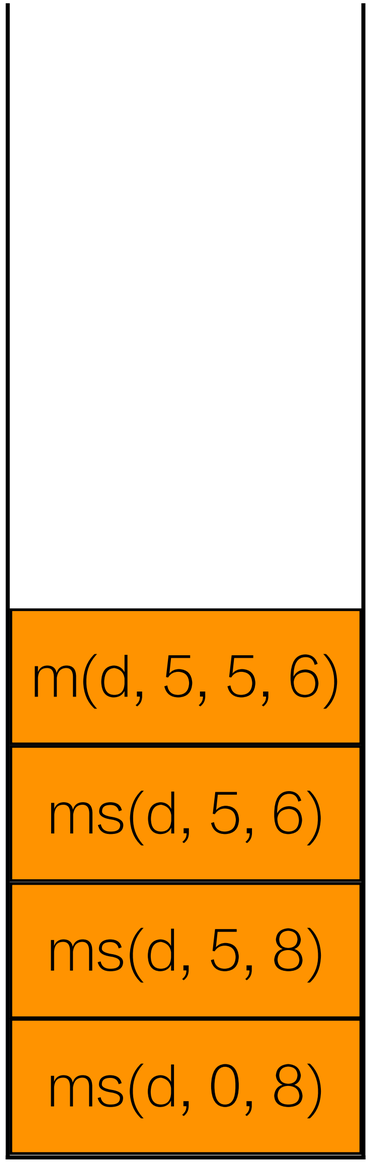
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 5　　j: 6　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

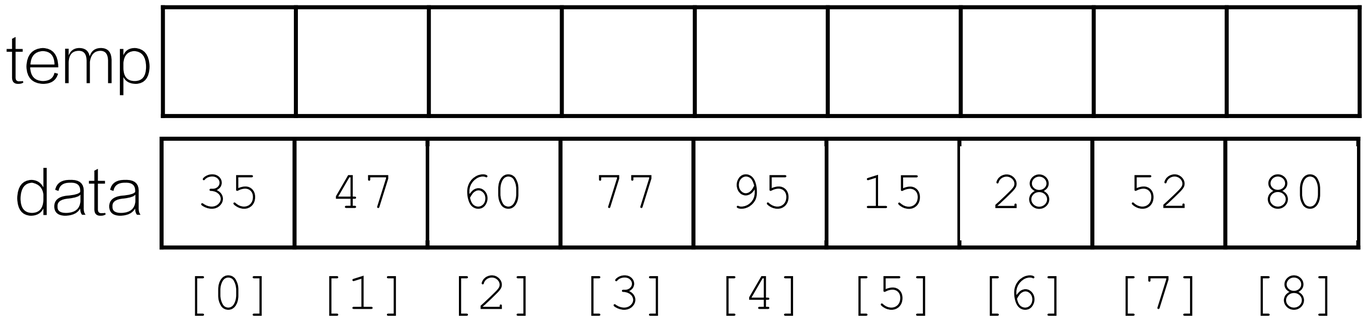
        // 对左半部分排序
        mergeSort(data, left, mid);

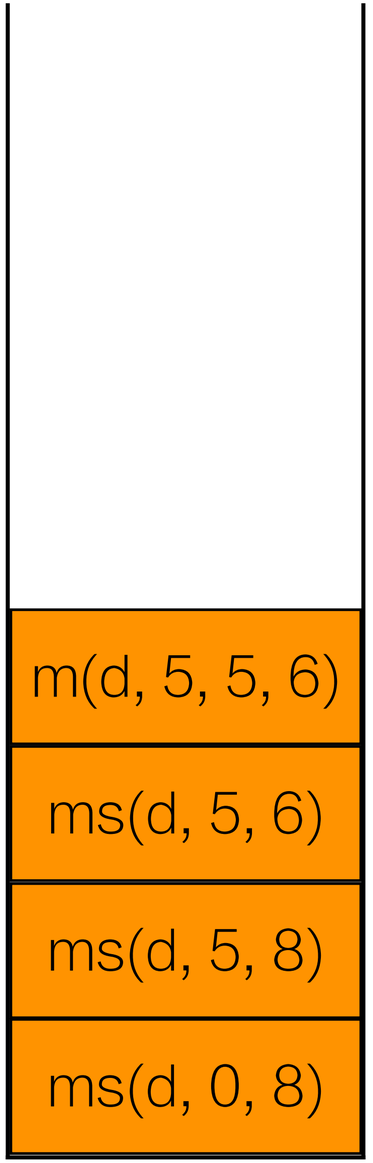
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 5　　j: 6　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

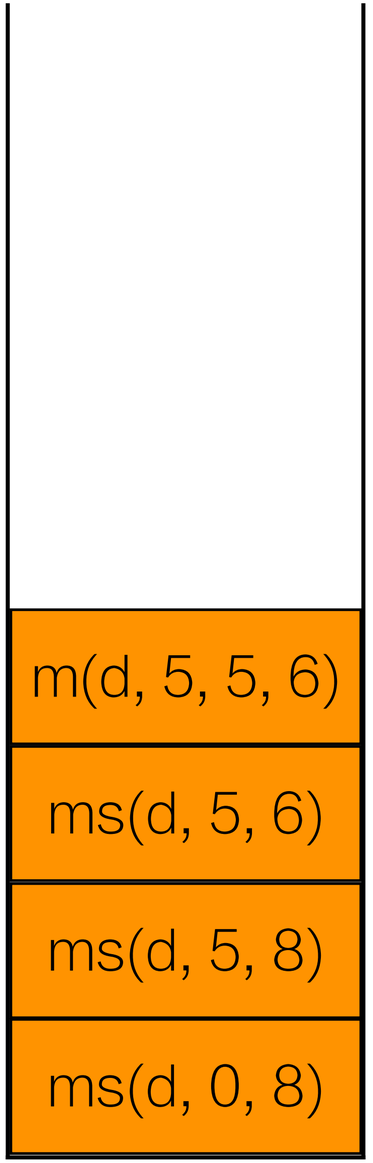
        // 对左半部分排序
        mergeSort(data, left, mid);

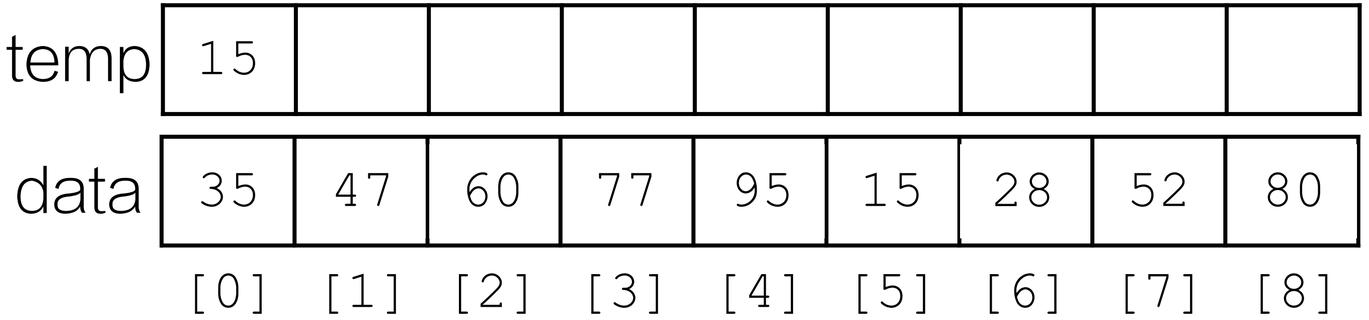
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 6　　j: 6　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

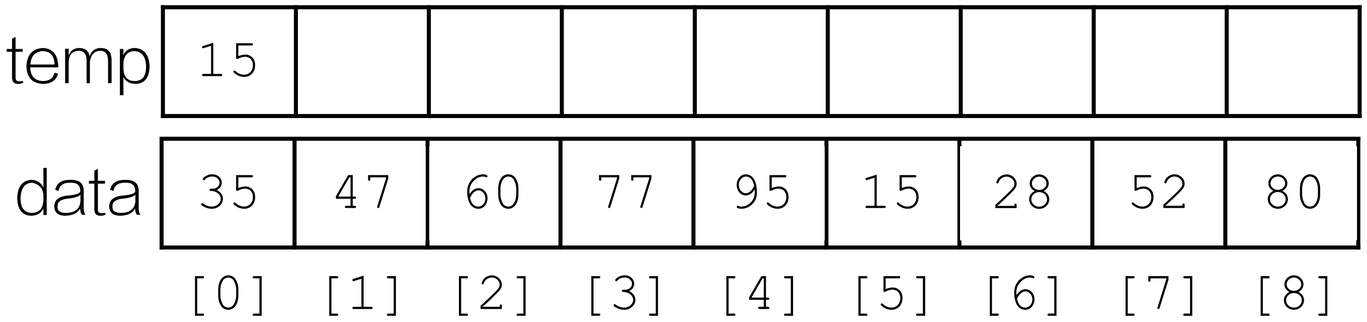
        // 对左半部分排序
        mergeSort(data, left, mid);

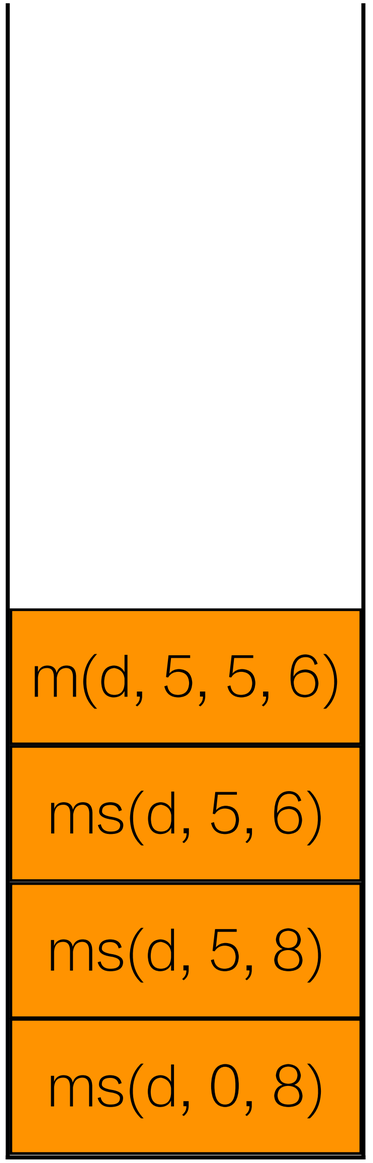
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 6　　j: 6　　k: 1





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

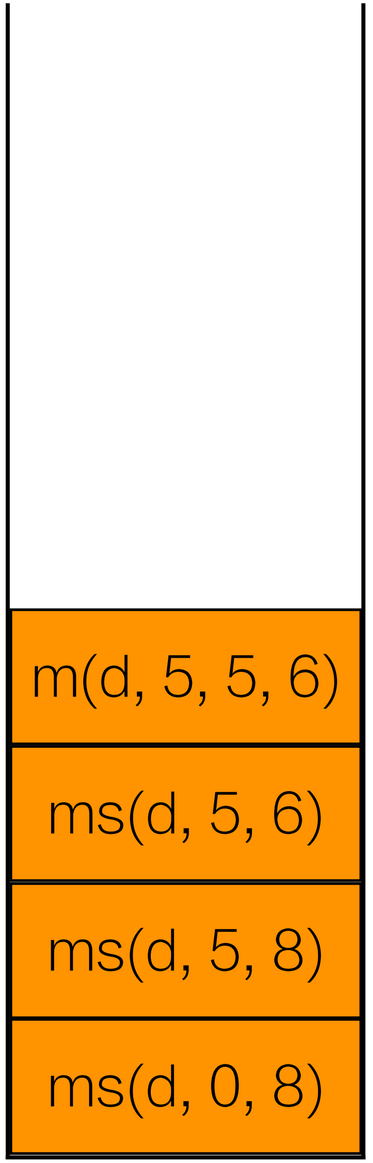
        // 对左半部分排序
        mergeSort(data, left, mid);

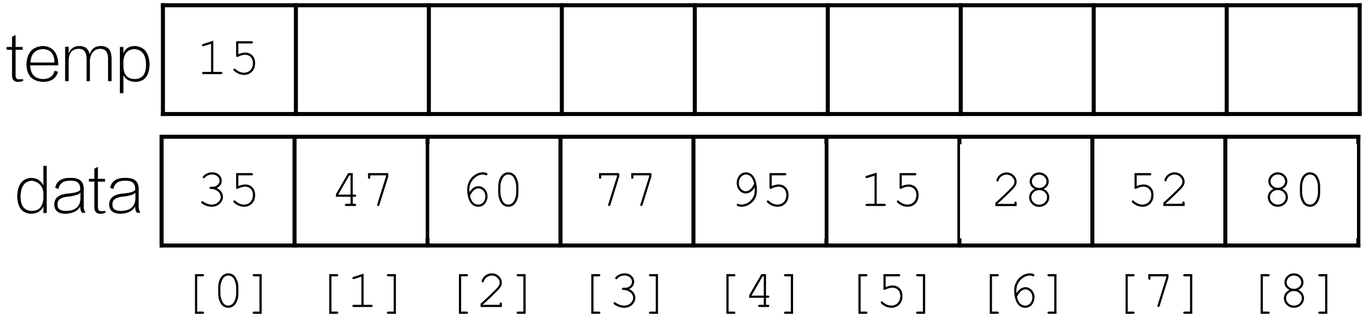
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 6　　j: 6　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

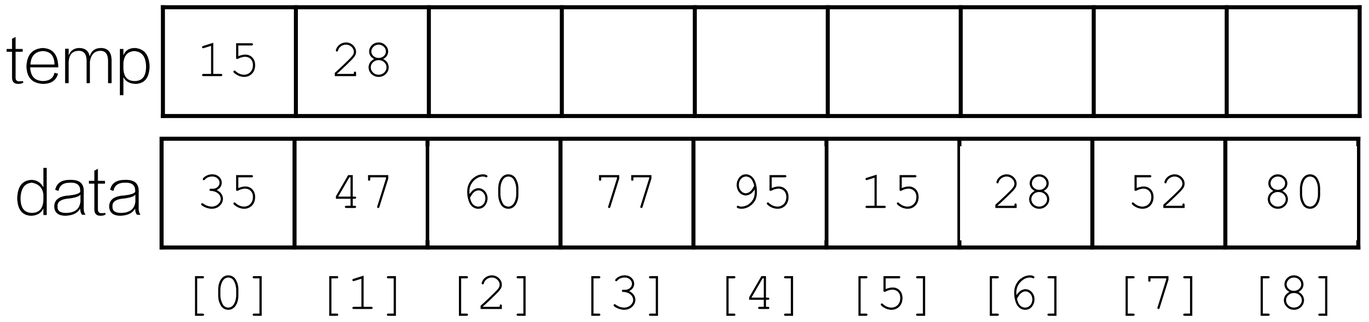
        // 对左半部分排序
        mergeSort(data, left, mid);

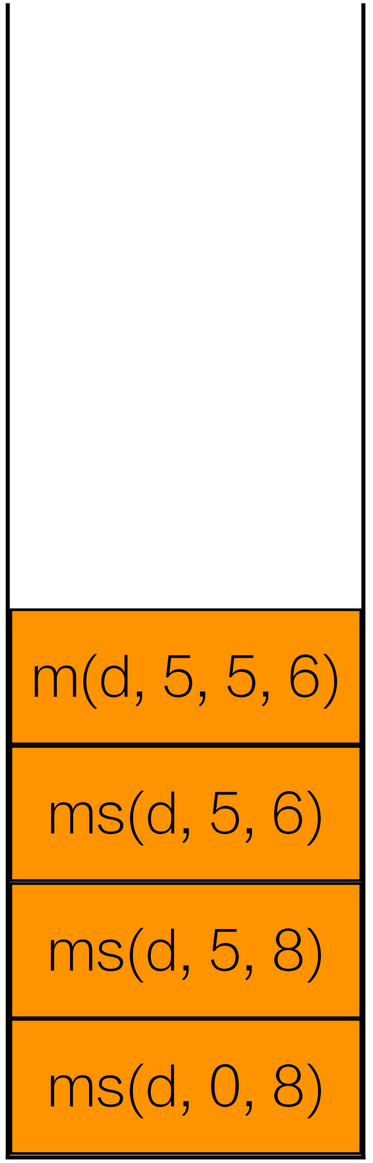
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 6　　j: 7　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

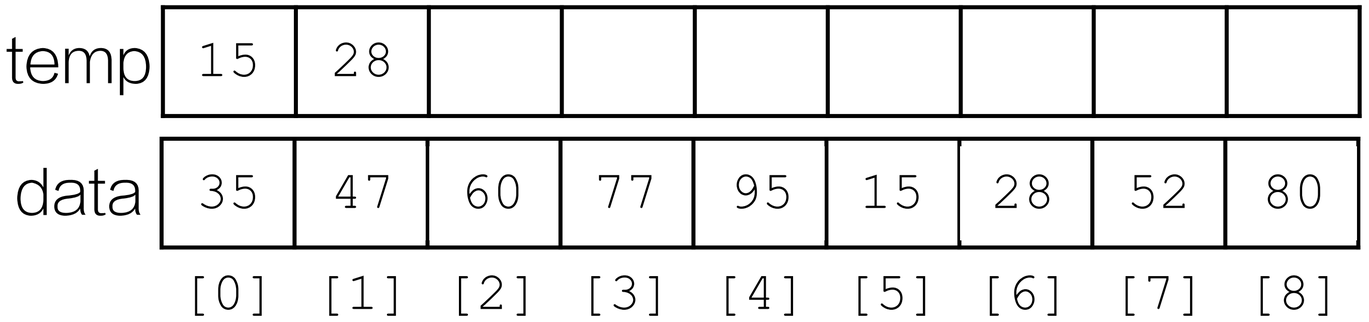
        // 对左半部分排序
        mergeSort(data, left, mid);

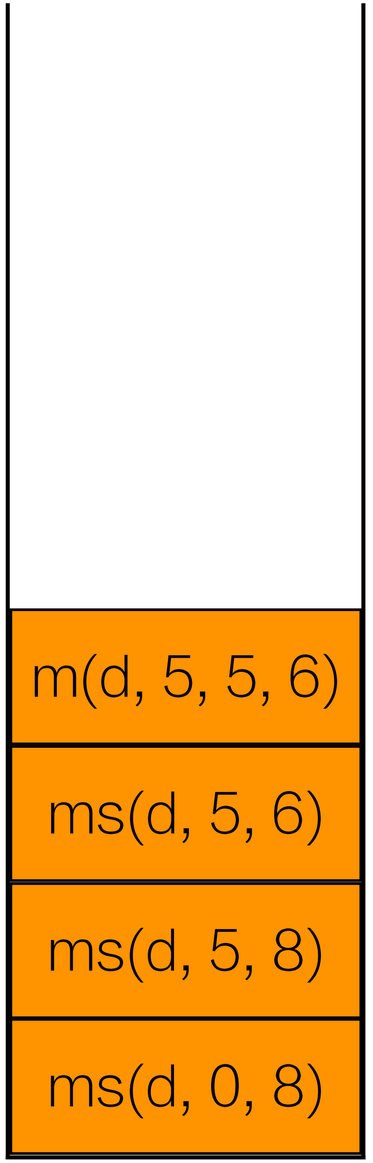
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 5　　right: 6

i: 6　　j: 7　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

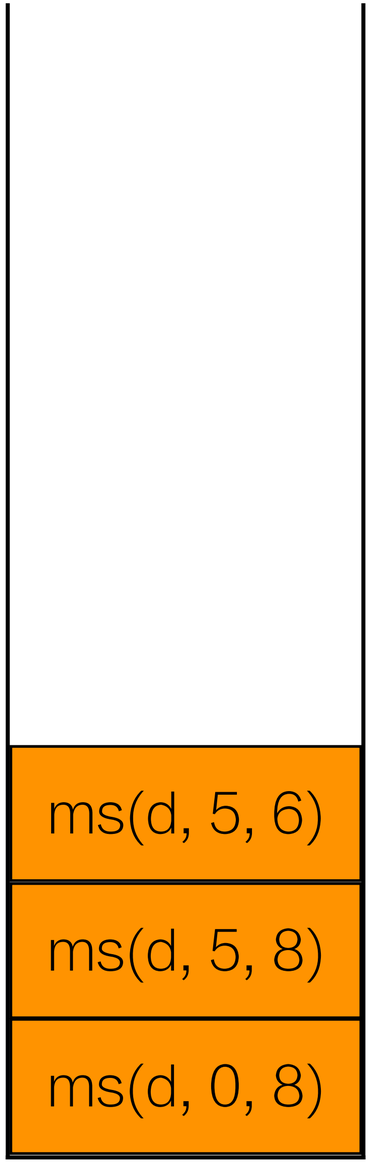
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

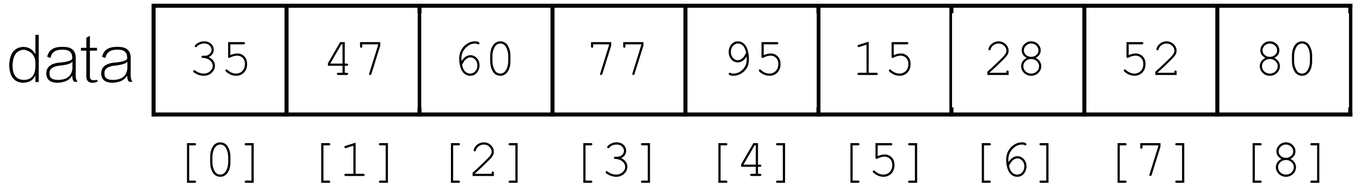
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：6　　mid：5





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

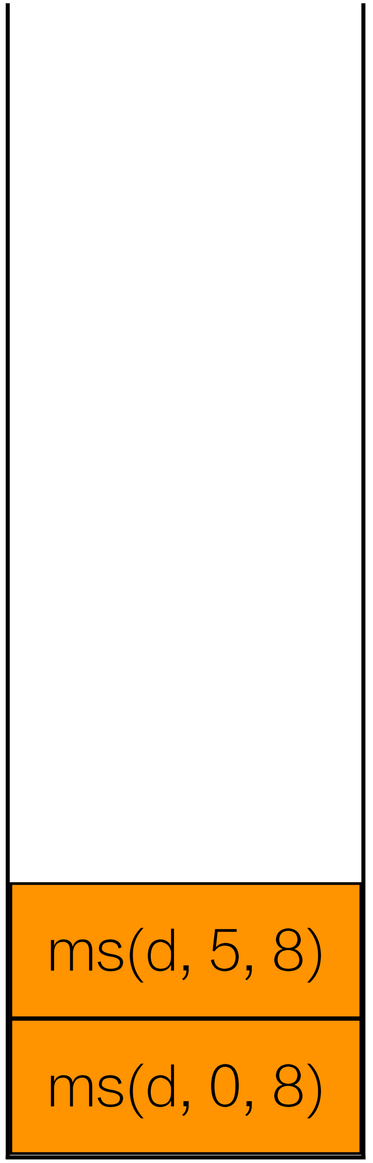
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6



data

35	47	60	77	95	15	28	52	80
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

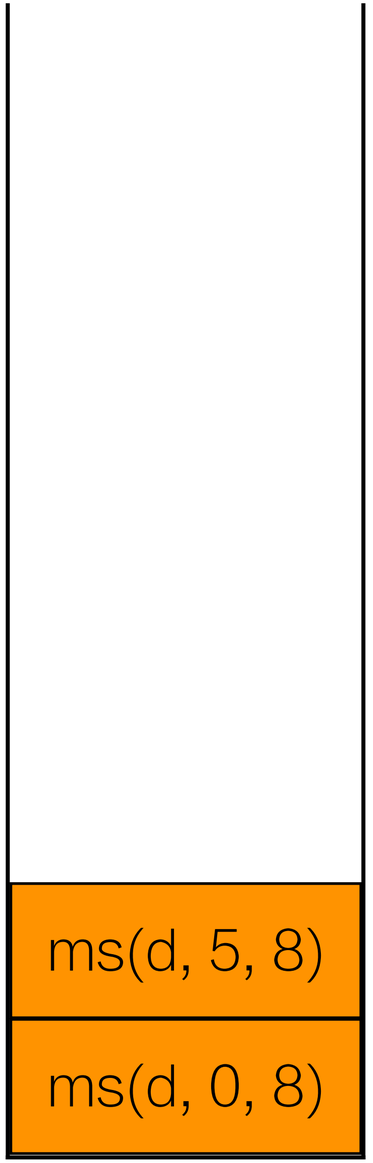
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

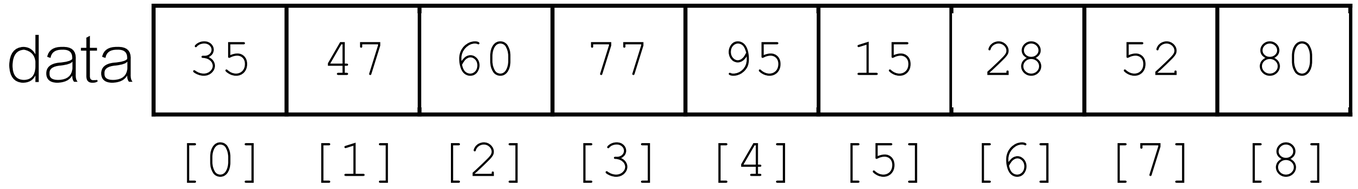
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

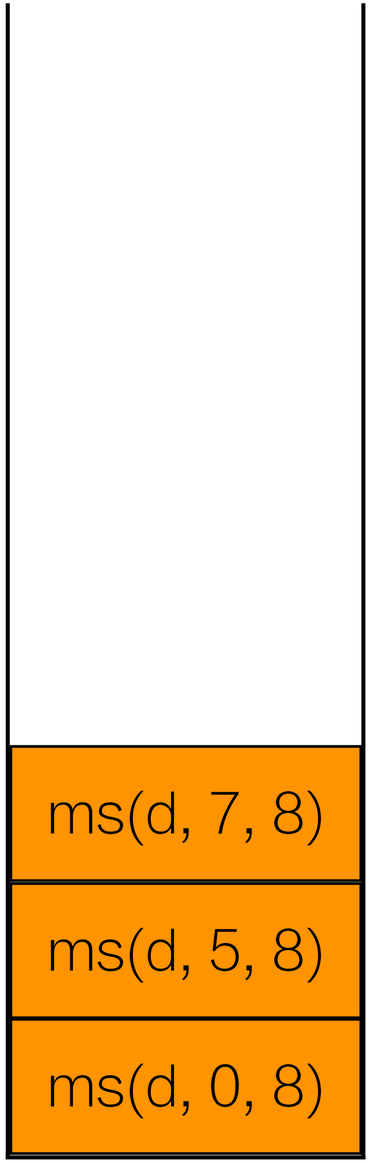
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

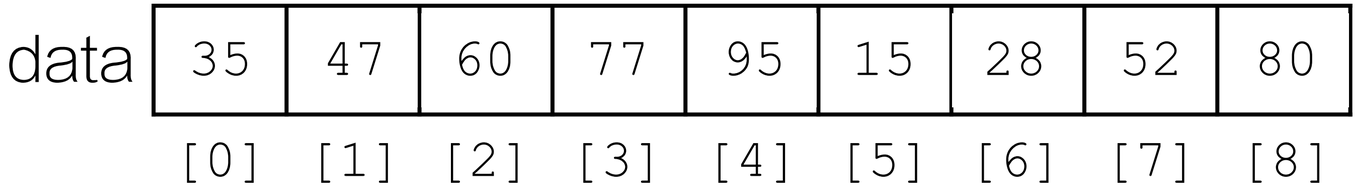
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

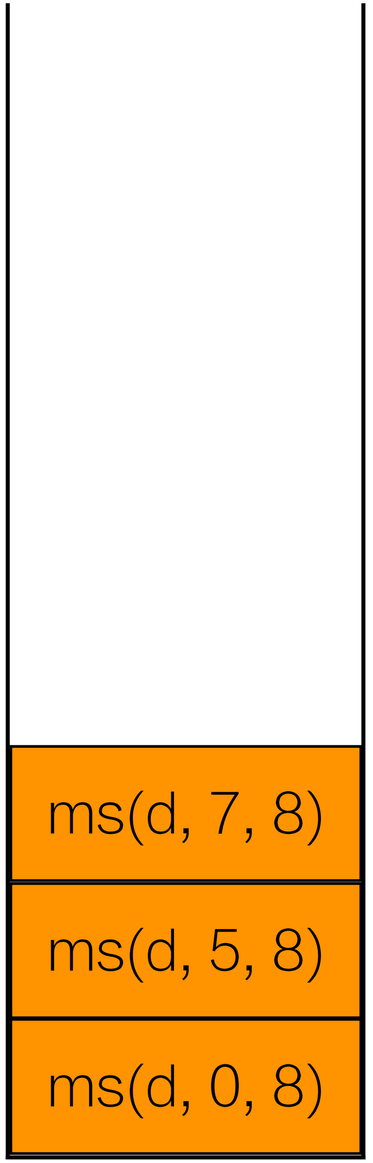
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

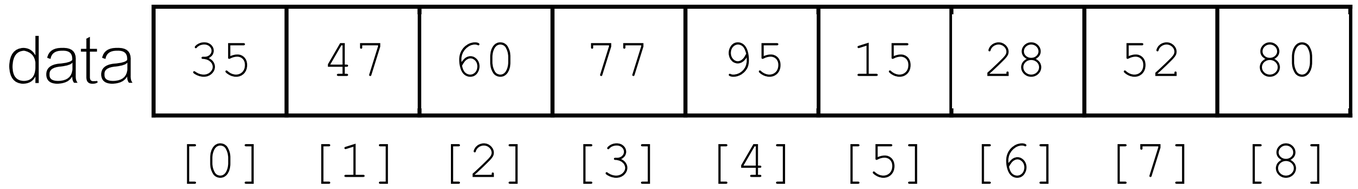
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

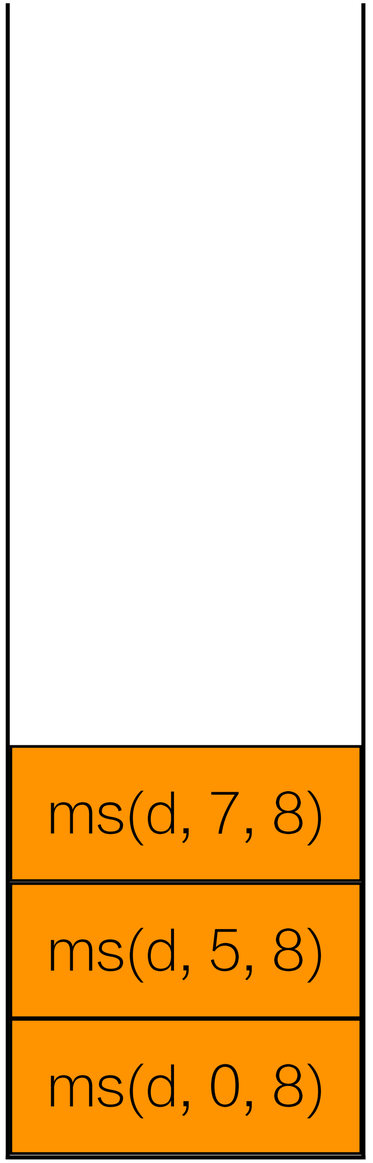
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

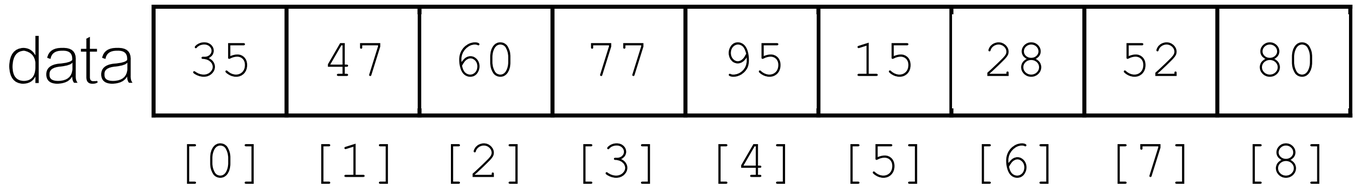
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

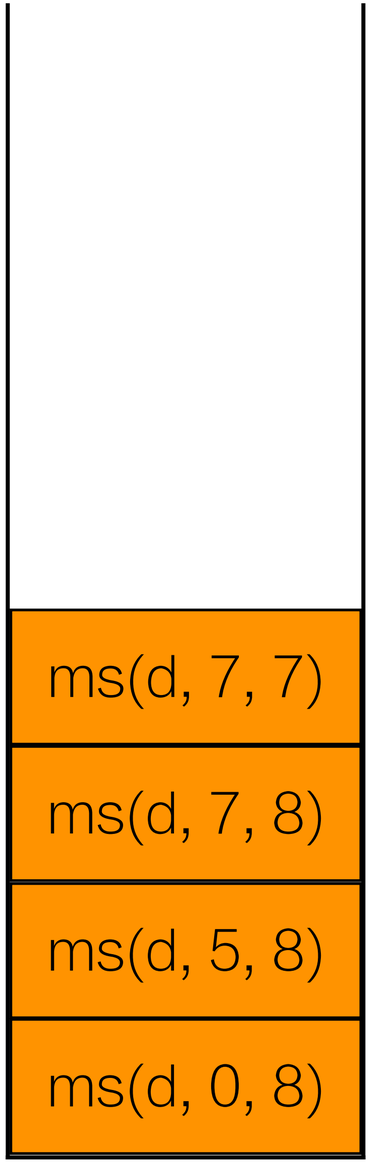
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

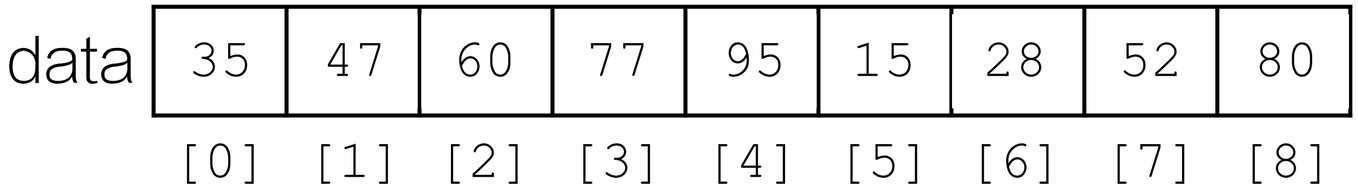
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：7　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

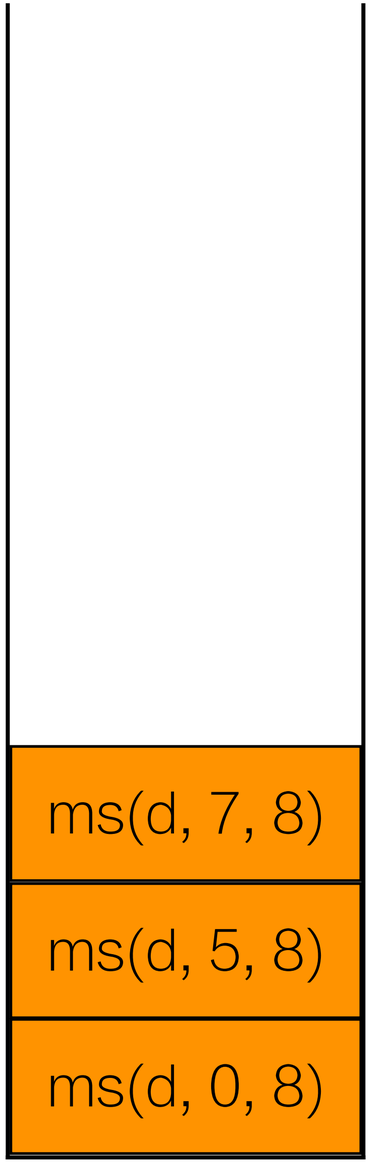
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

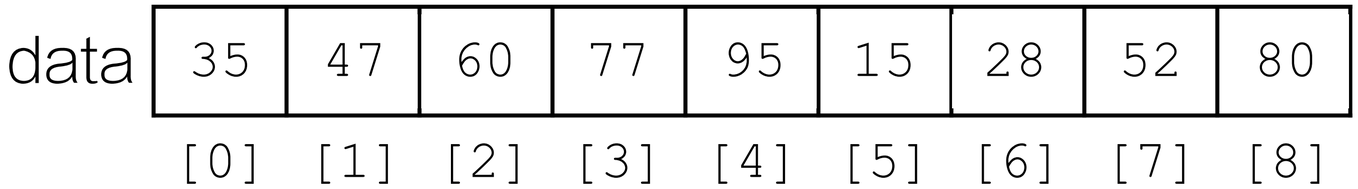
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

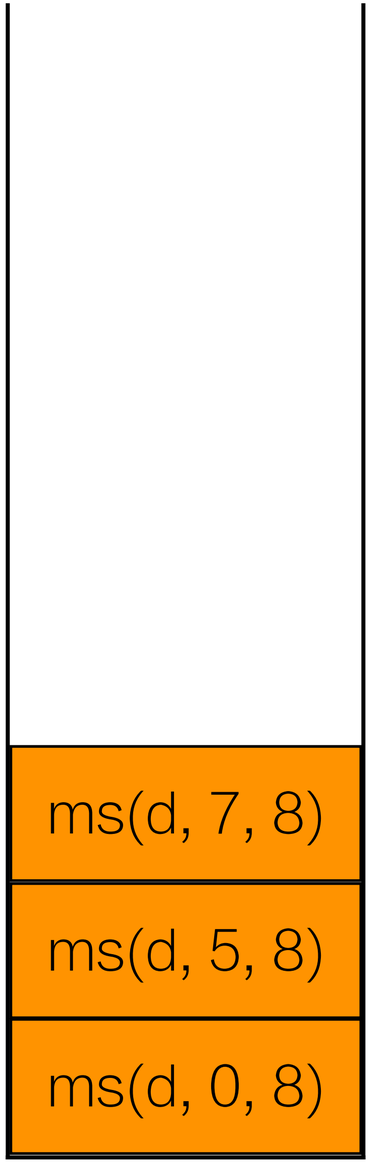
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

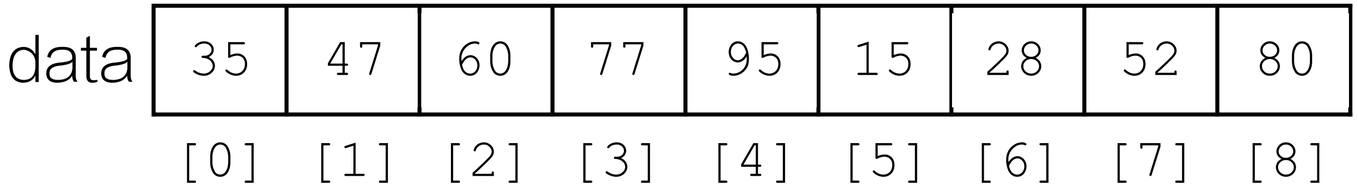
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

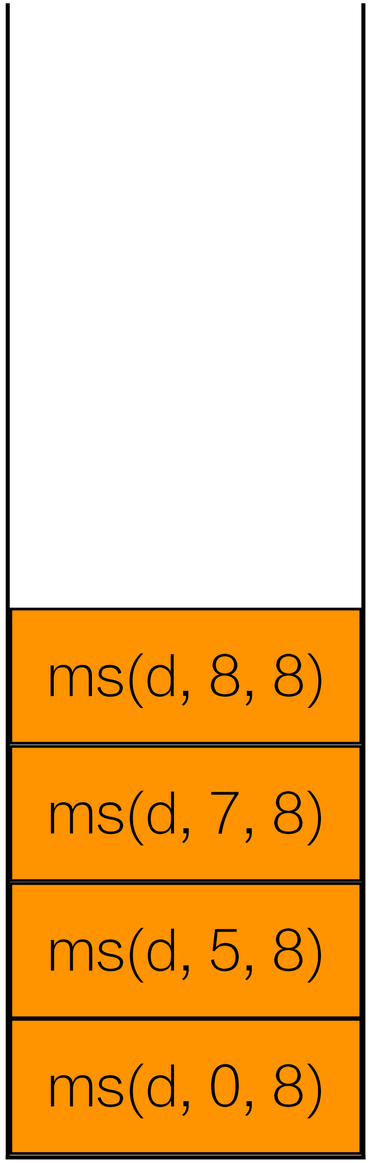
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

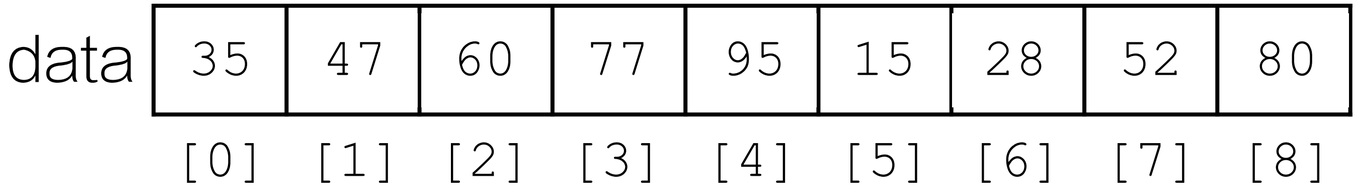
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：8　　right：8　　mid：





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

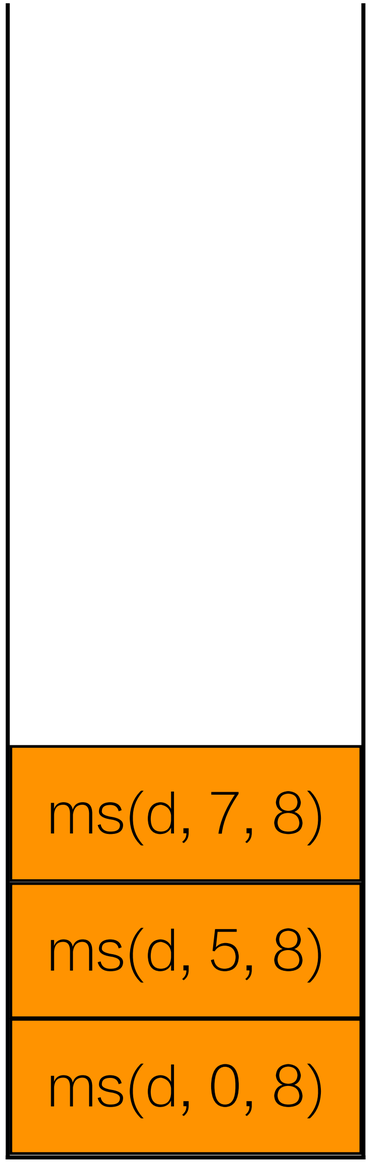
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

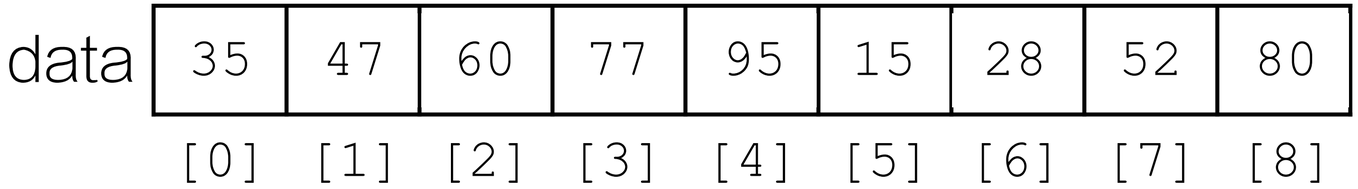
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

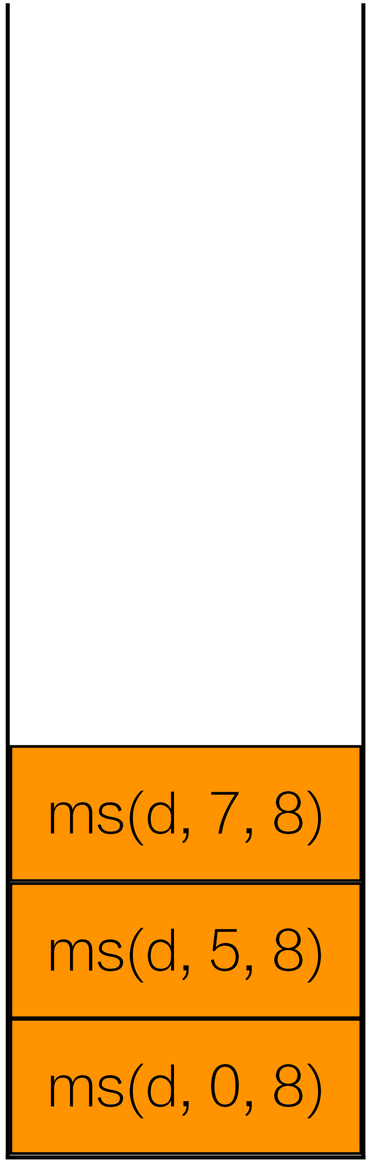
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

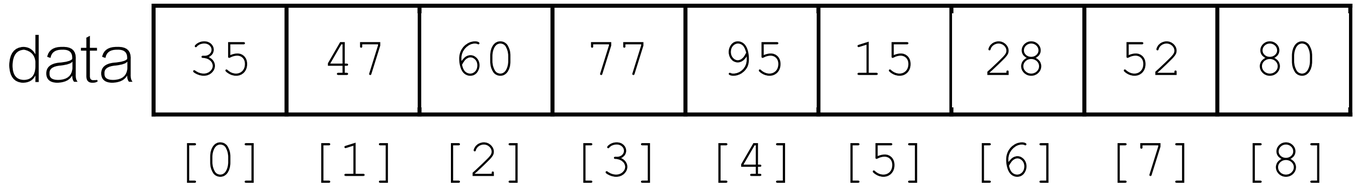
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

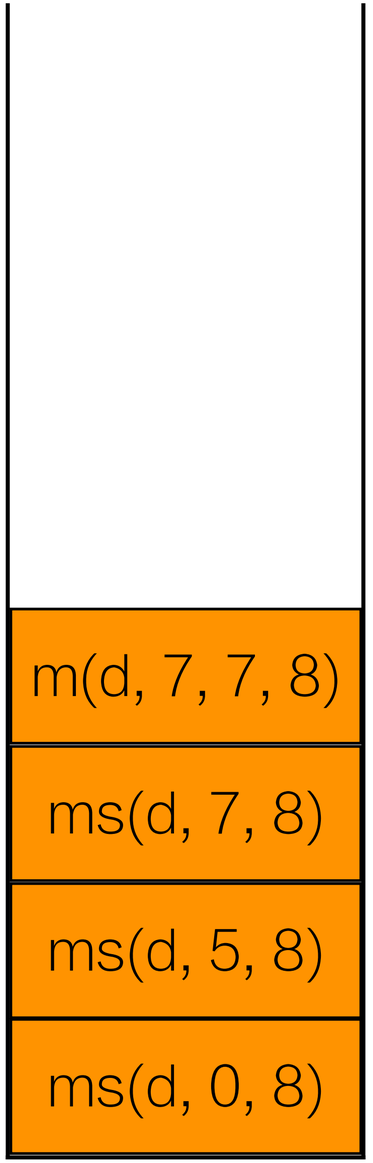
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

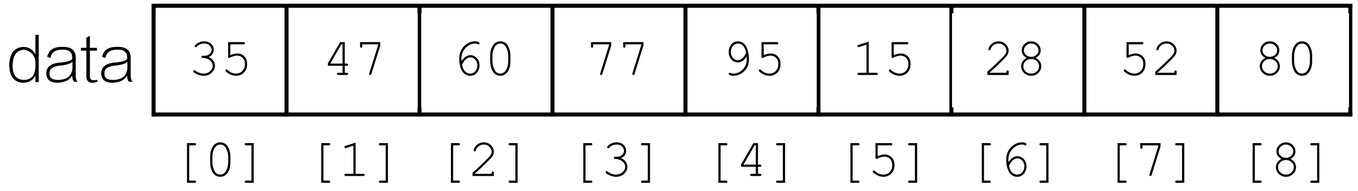
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　mid：7　　right：8





```java
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```java
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

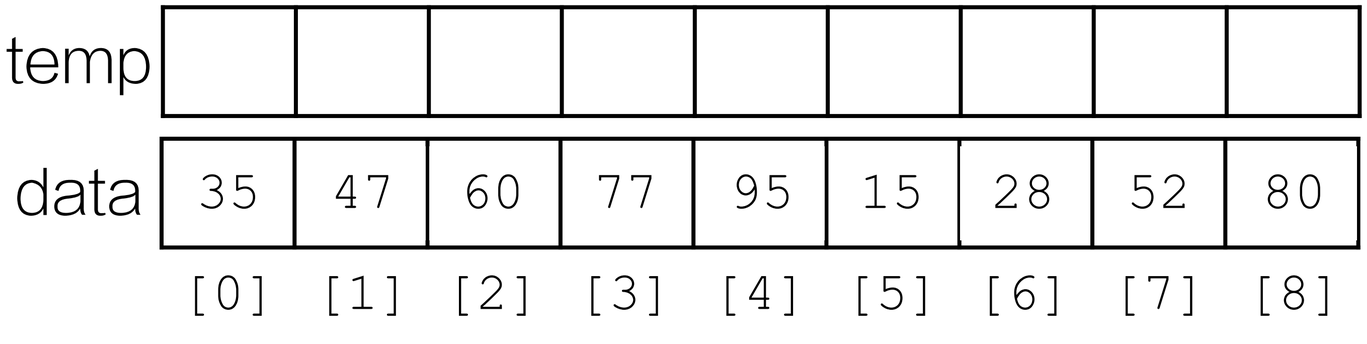
        // 对左半部分排序
        mergeSort(data, left, mid);

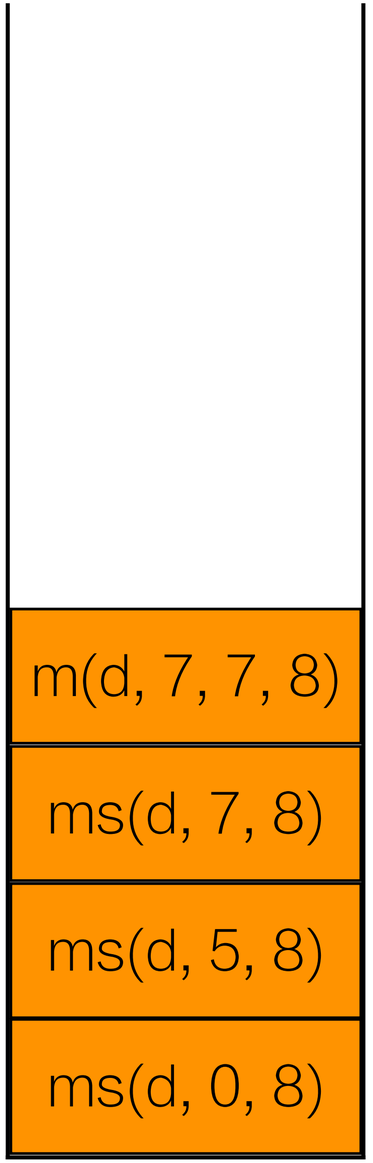
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 7　　j: 8　　k: 0





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

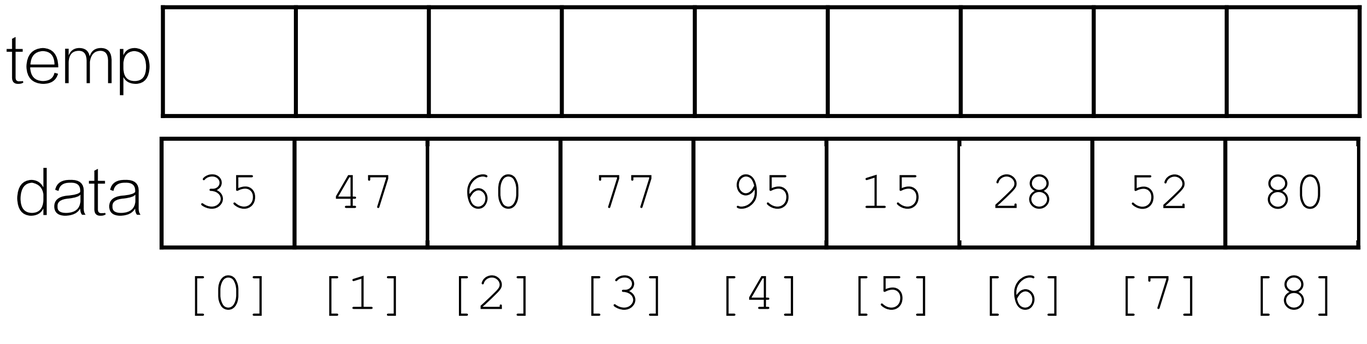
        // 对左半部分排序
        mergeSort(data, left, mid);

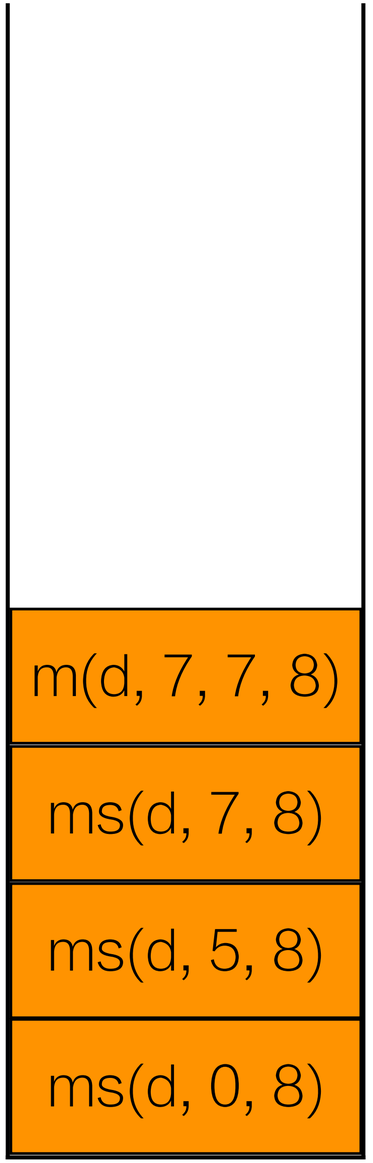
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 7　　j: 8　　k: 0





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

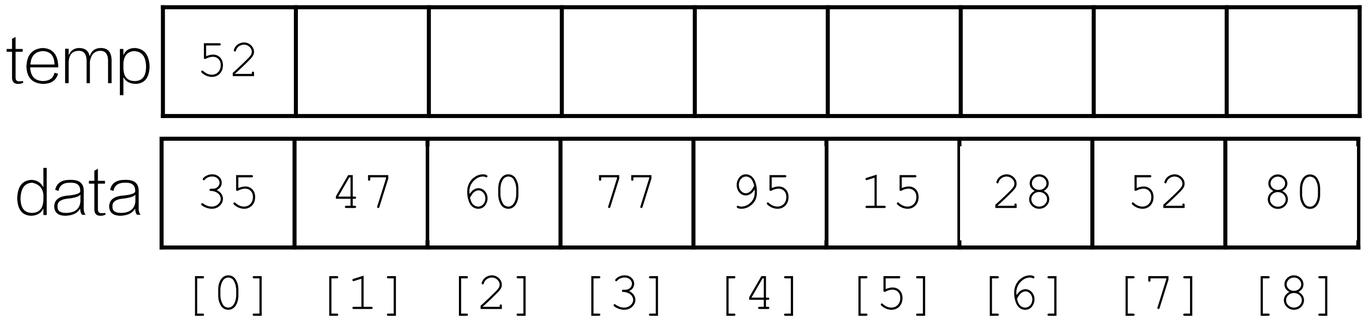
        // 对左半部分排序
        mergeSort(data, left, mid);

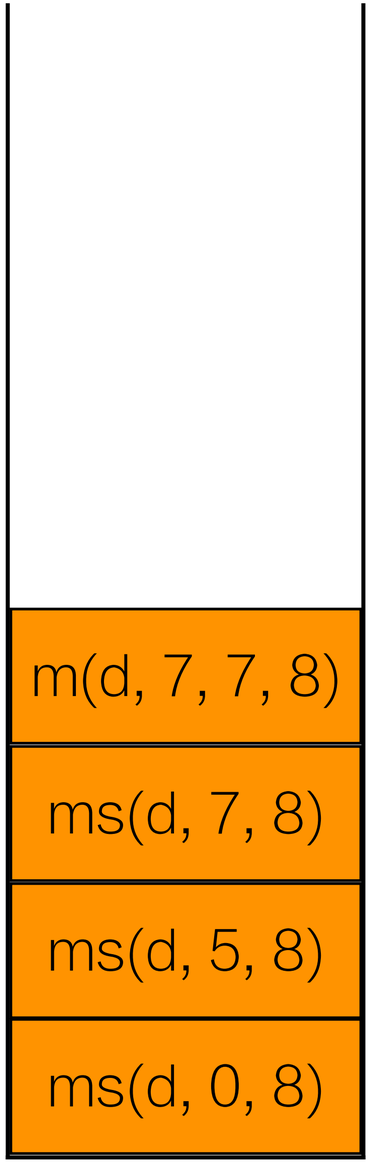
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 8　　j: 8　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

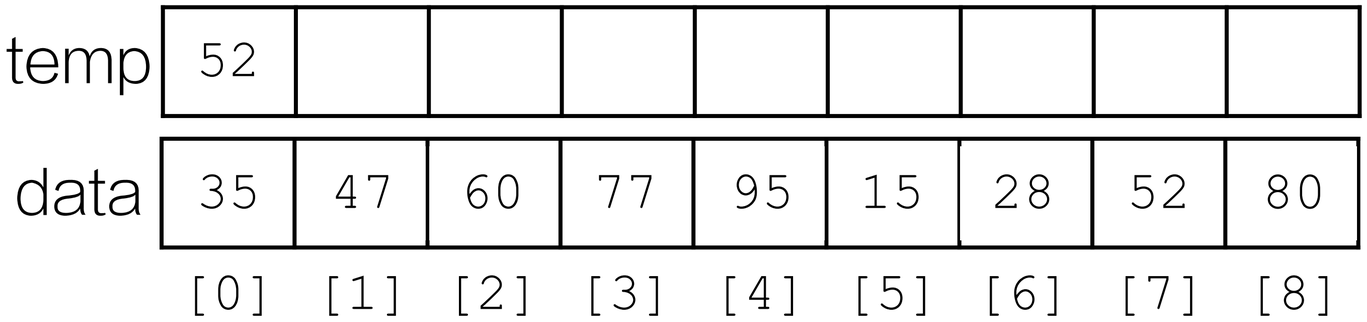
        // 对左半部分排序
        mergeSort(data, left, mid);

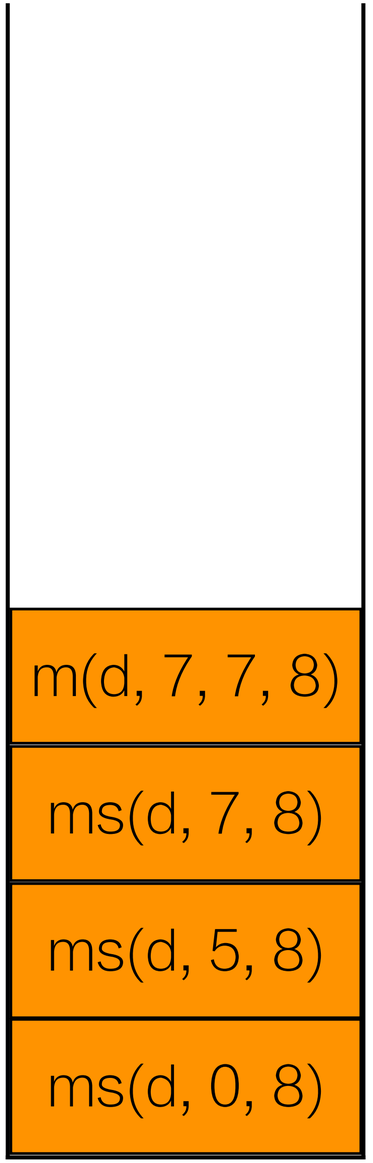
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 8　　j: 8　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

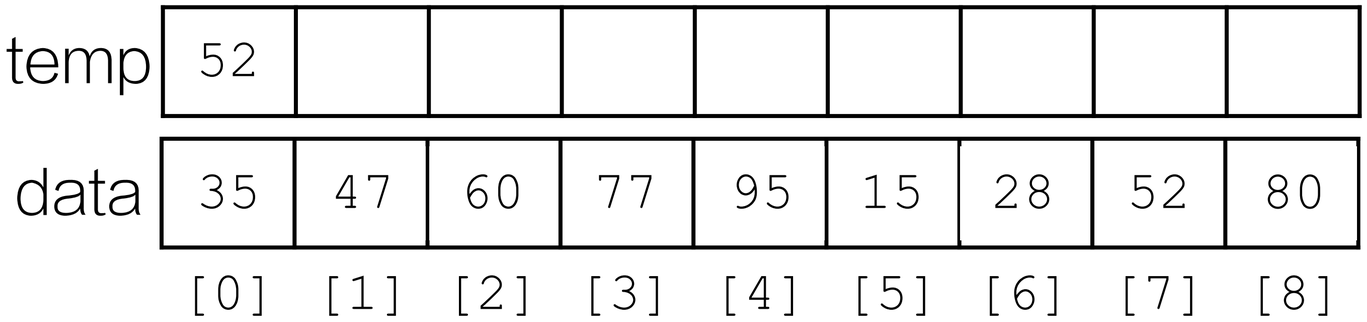
        // 对左半部分排序
        mergeSort(data, left, mid);

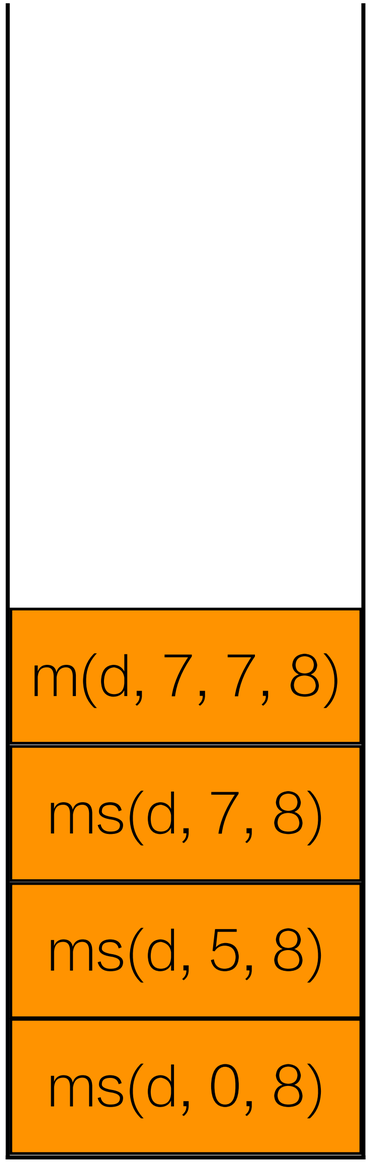
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 8　　j: 8　　k: 1





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

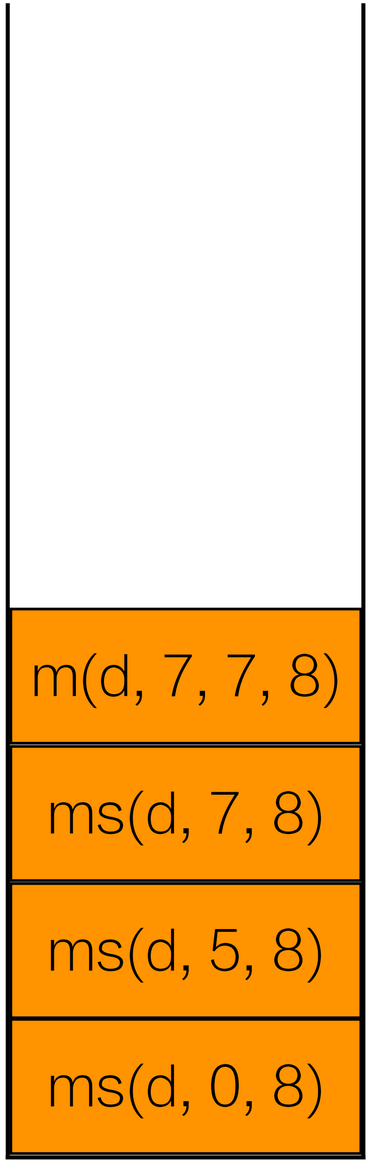
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 8　　j: 9　　k: 2



	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	52	80							
data	35	47	60	77	95	15	28	52	80

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

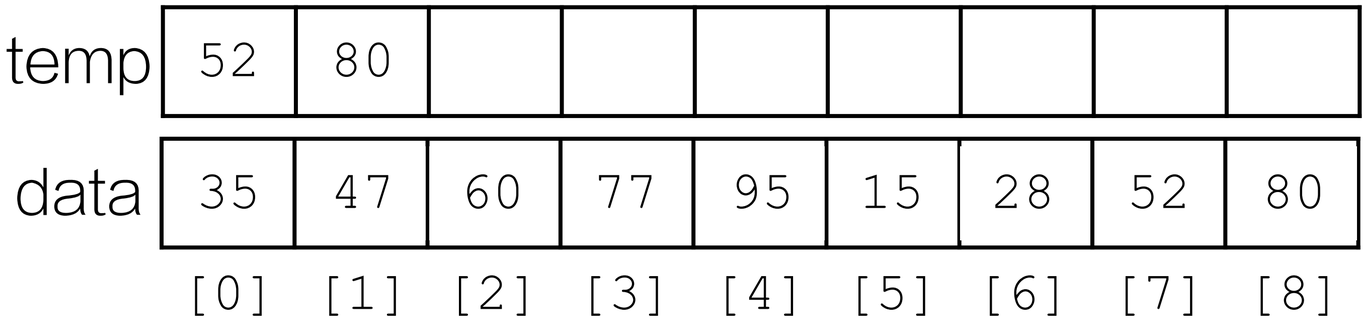
        // 对左半部分排序
        mergeSort(data, left, mid);

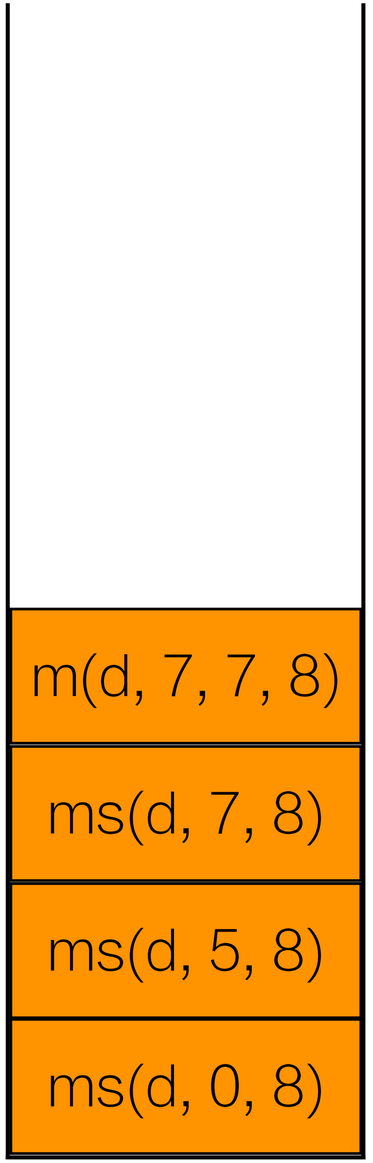
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 7　　mid: 7　　right: 8

i: 8　　j: 9　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

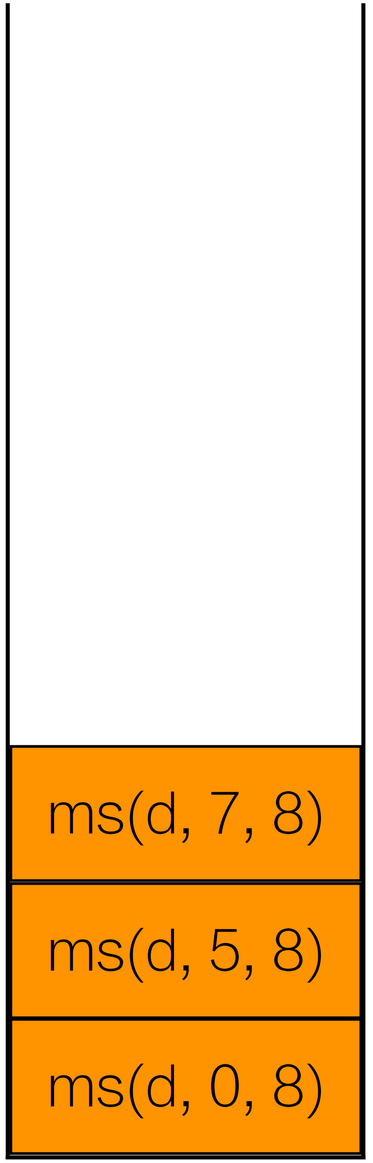
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

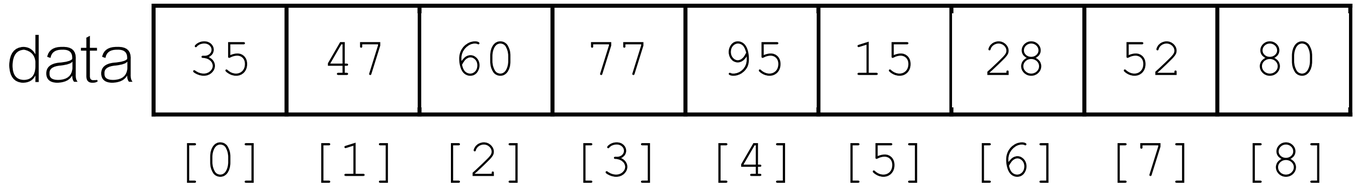
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：7　　right：8　　mid：7





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

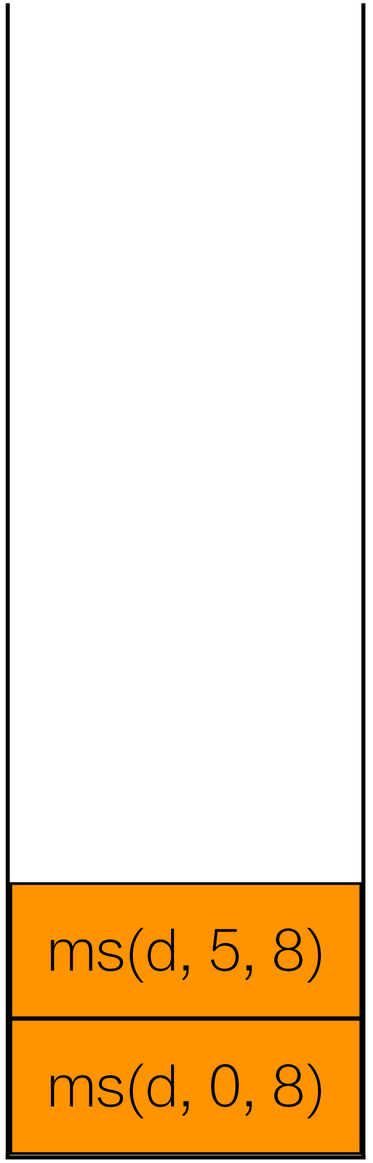
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

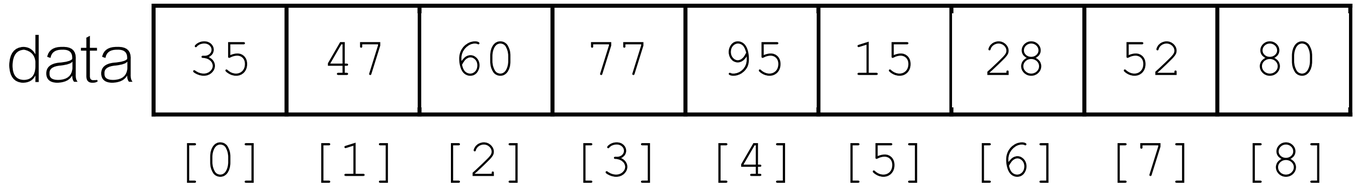
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

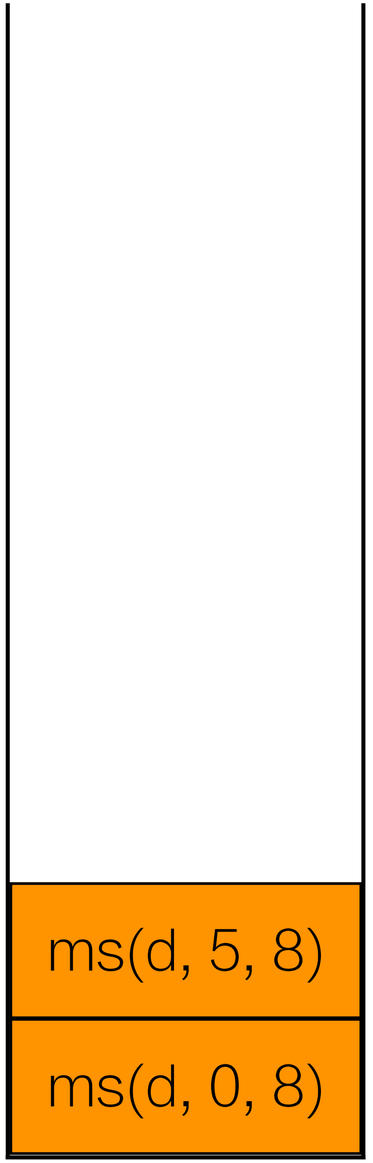
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

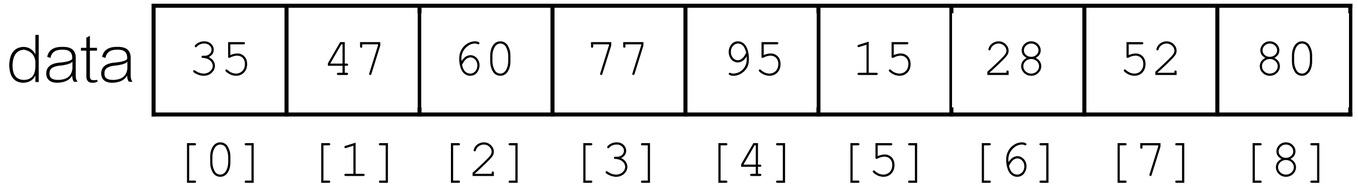
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

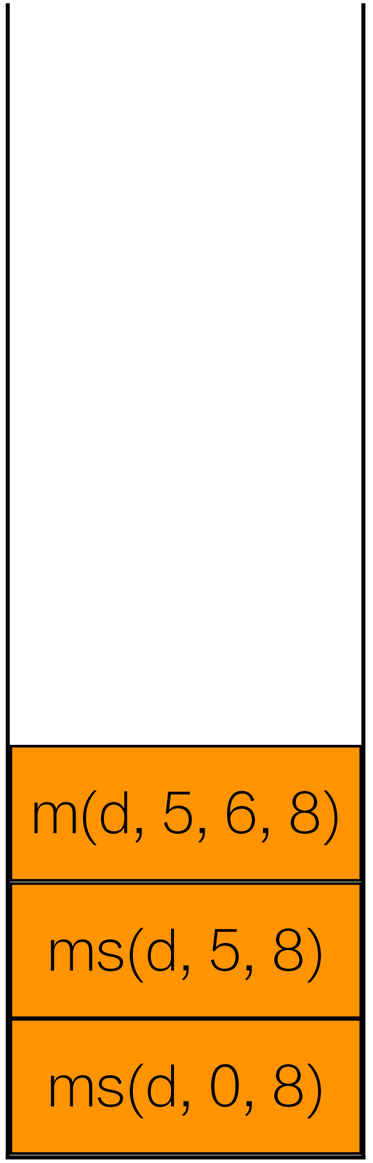
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

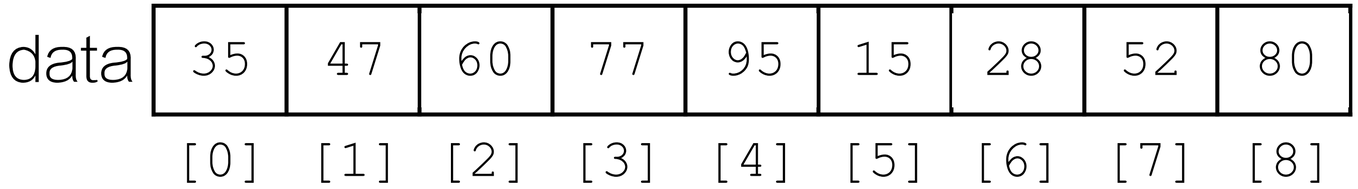
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　mid：6　　right：8





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

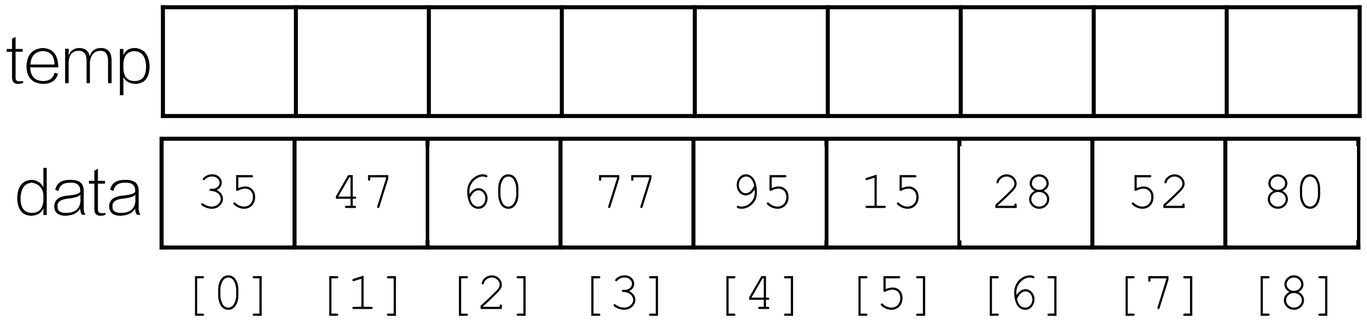
        // 对左半部分排序
        mergeSort(data, left, mid);

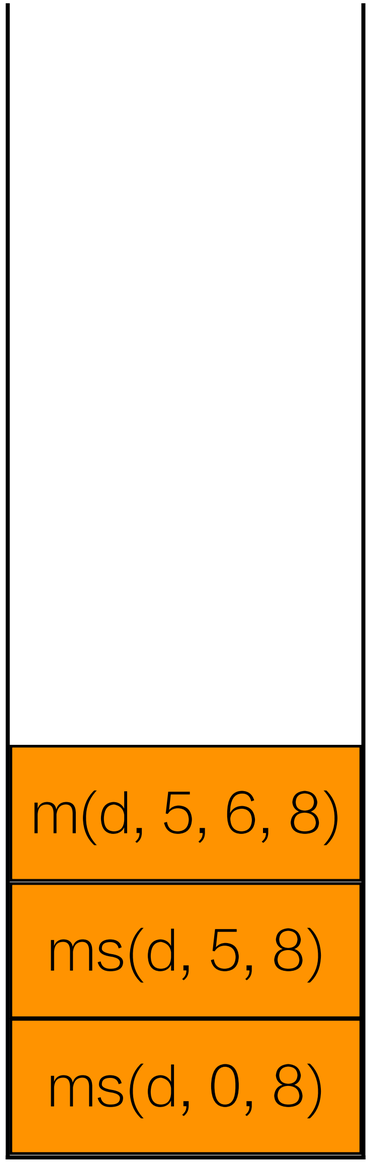
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 5　　j: 7　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

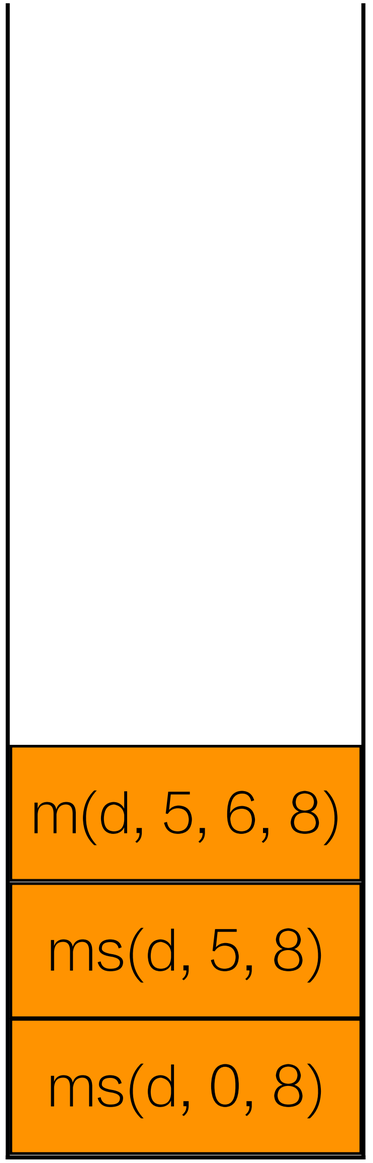
        // 对左半部分排序
        mergeSort(data, left, mid);

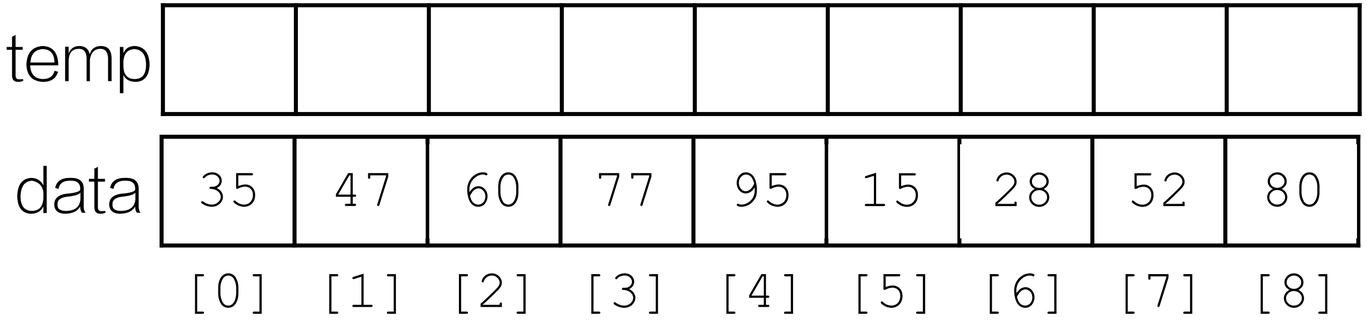
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 5　　j: 7　　k: 0





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

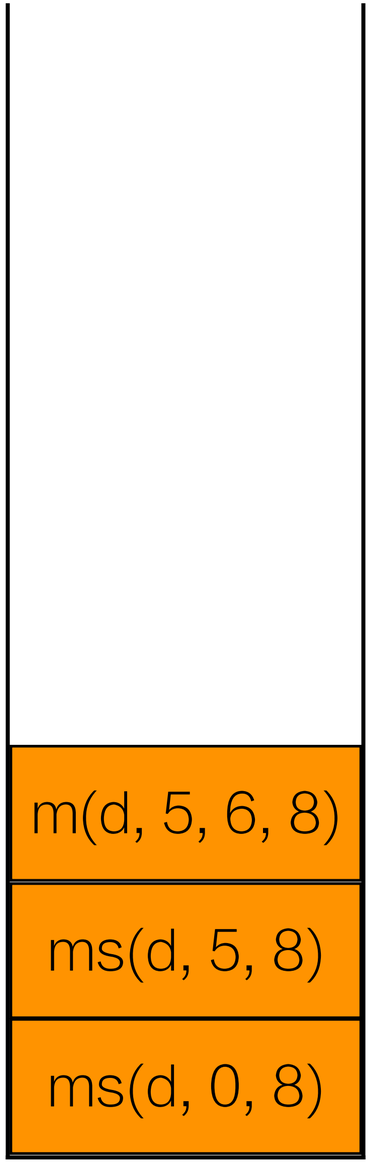
        // 对左半部分排序
        mergeSort(data, left, mid);

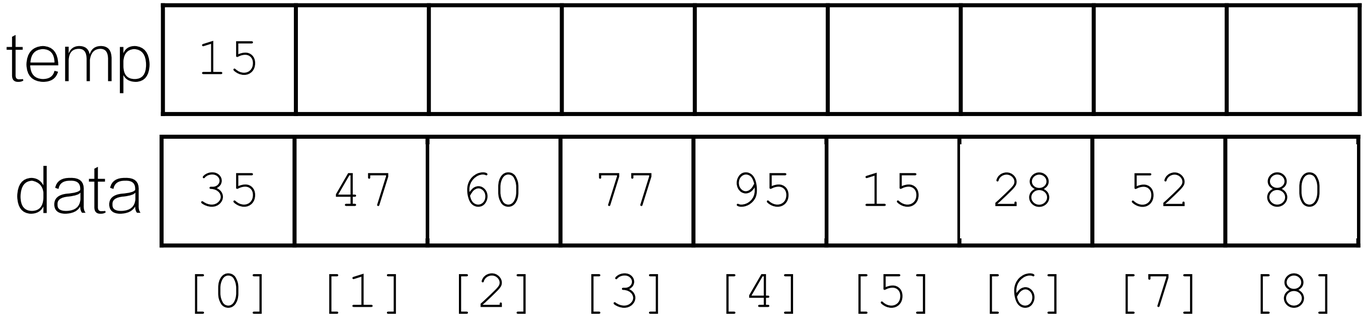
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 6　　j: 7　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

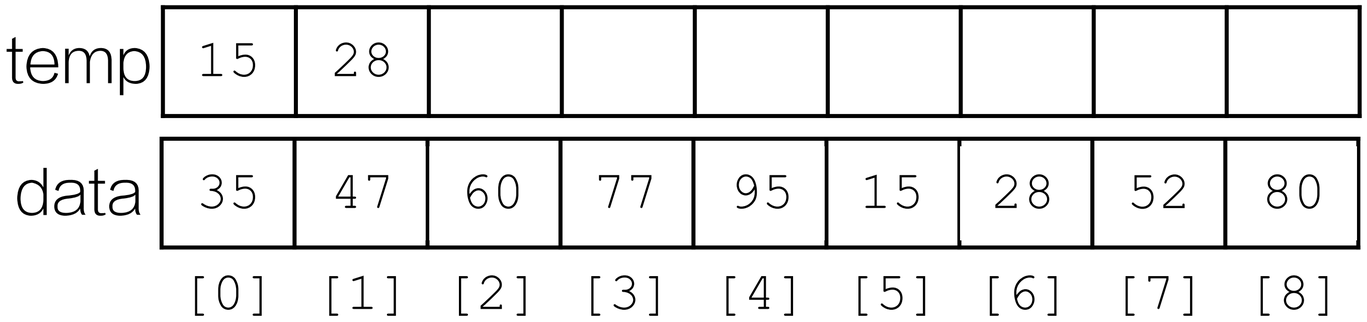
        // 对左半部分排序
        mergeSort(data, left, mid);

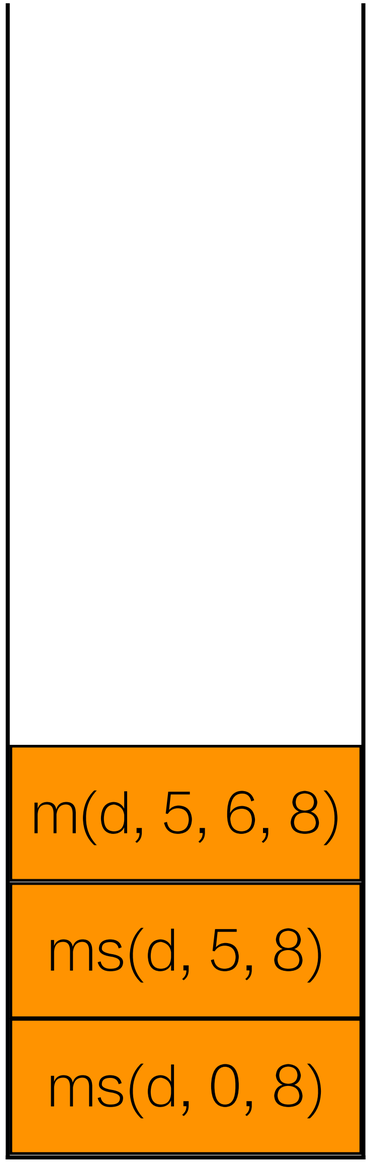
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 7　　j: 7　　k: 2





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

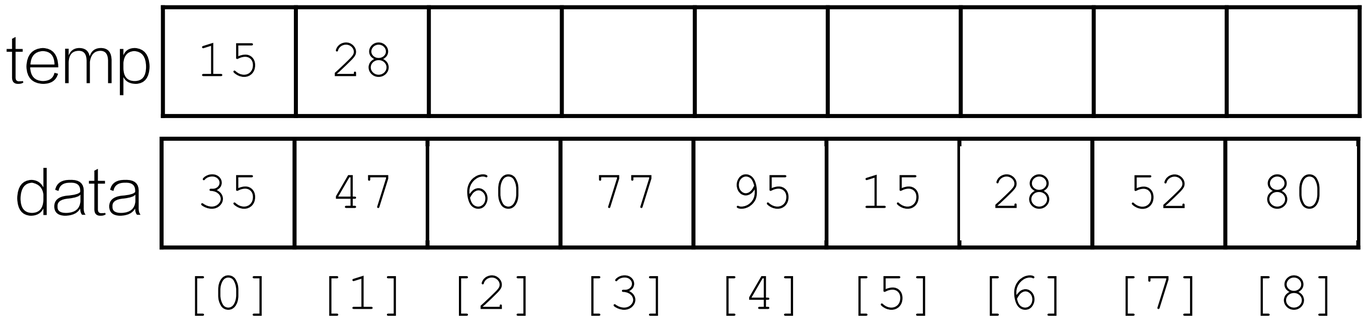
        // 对左半部分排序
        mergeSort(data, left, mid);

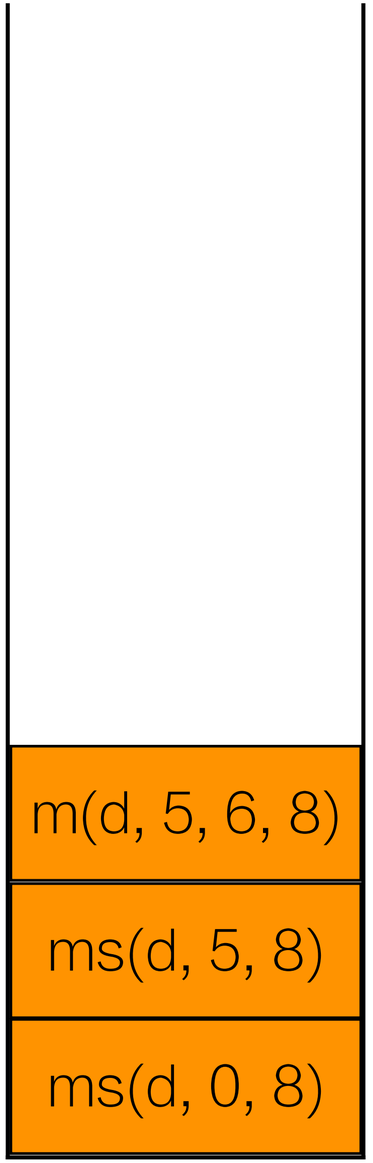
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 7　　j: 7　　k: 2





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

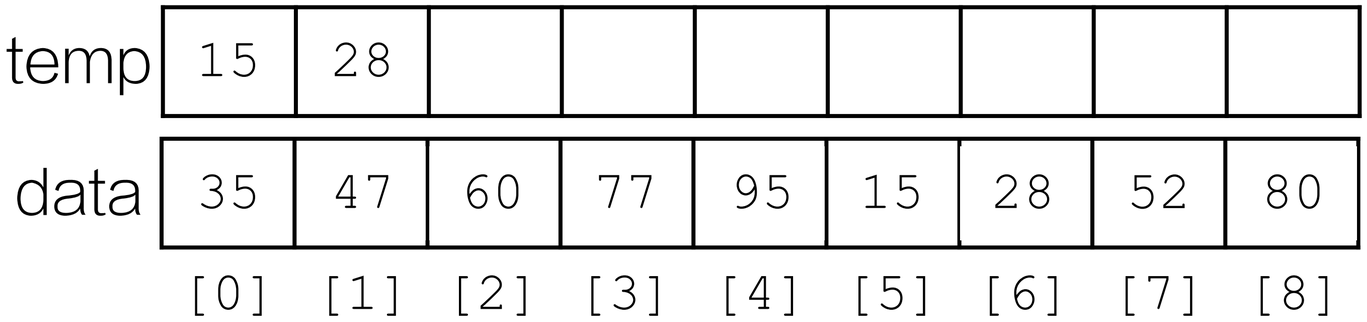
        // 对左半部分排序
        mergeSort(data, left, mid);

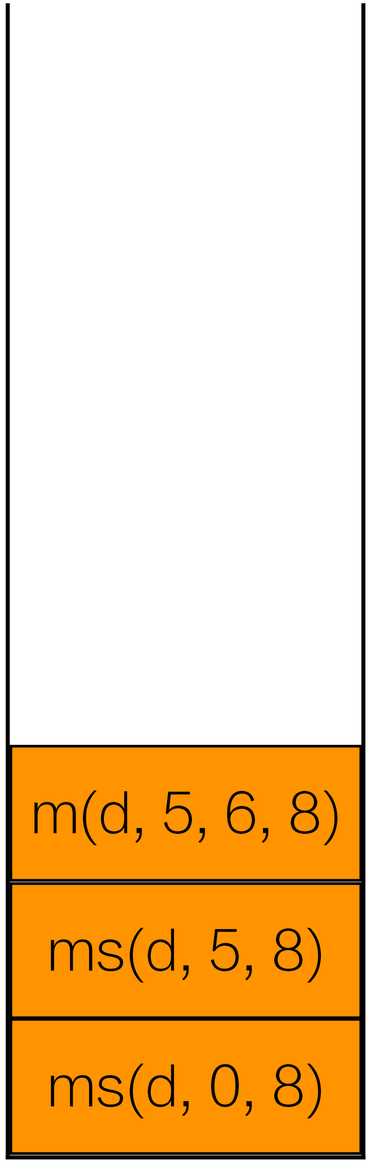
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 7　　j: 7　　k: 2





```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

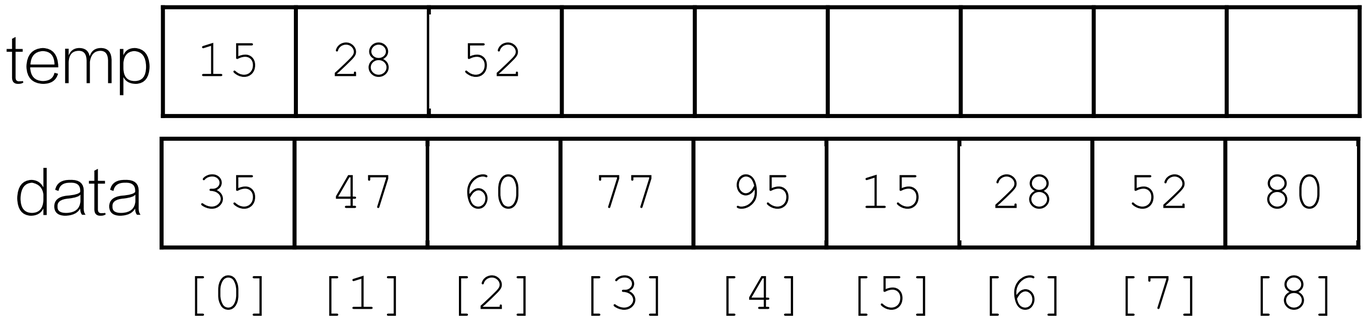
        // 对左半部分排序
        mergeSort(data, left, mid);

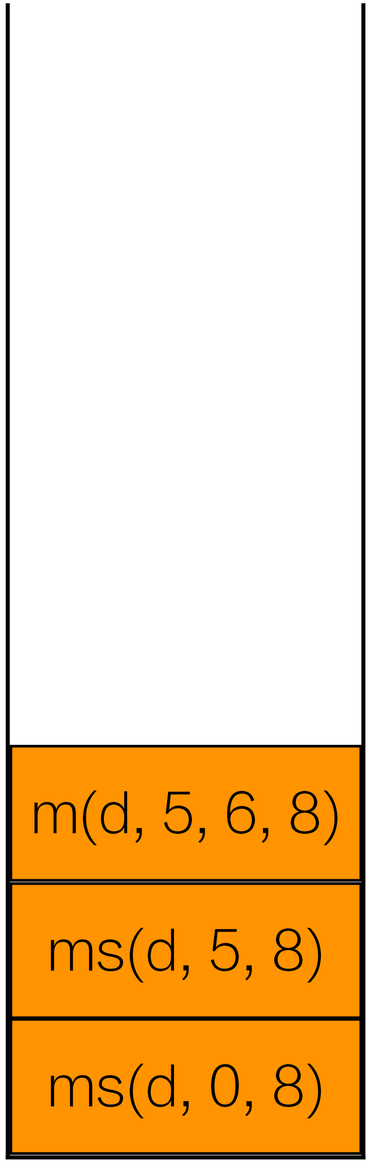
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 7　　j: 8　　k: 3





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

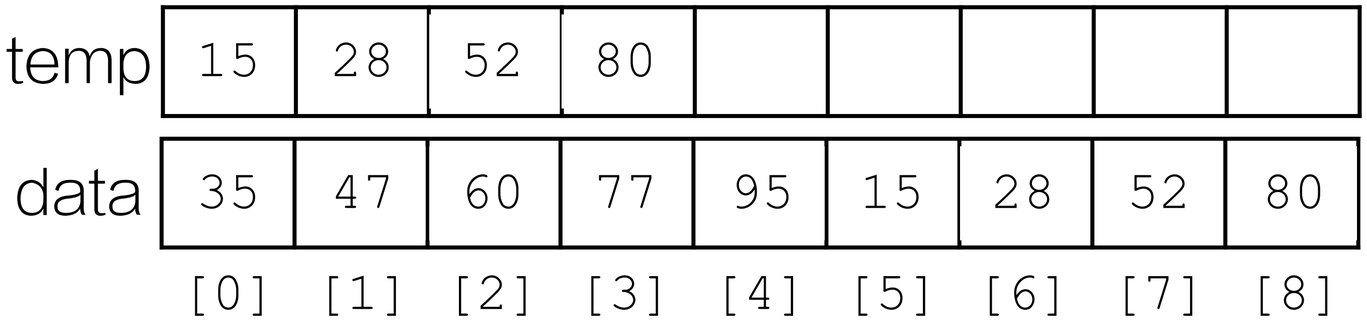
        // 对左半部分排序
        mergeSort(data, left, mid);

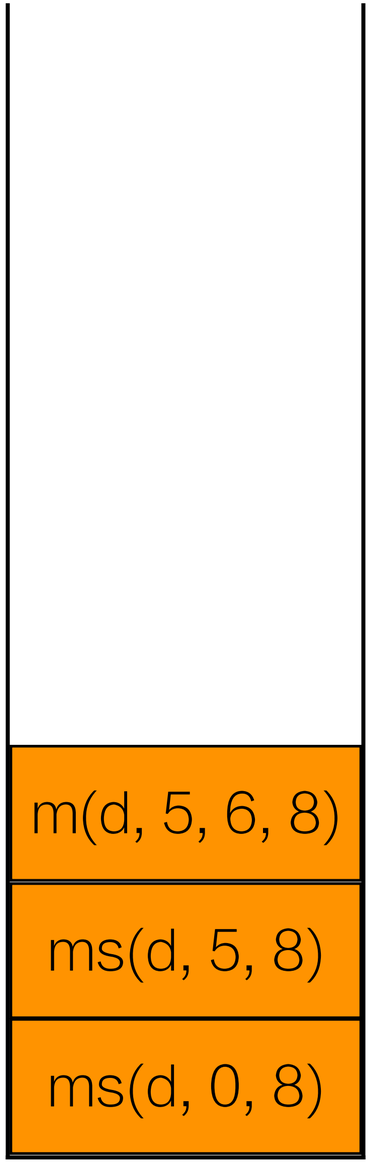
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 5　　mid: 6　　right: 8

i: 7　　j: 9　　k: 4





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

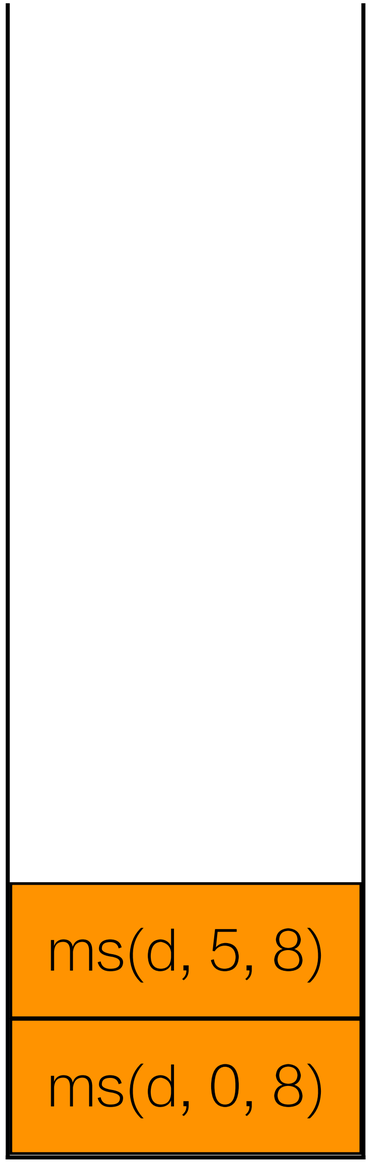
```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

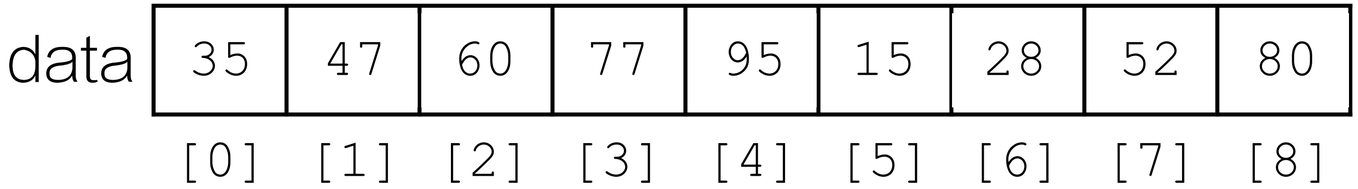
        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：5　　right：8　　mid：6





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：8　　mid：4

data	35	47	60	77	95	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：8　　mid：4

data	35	47	60	77	95	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

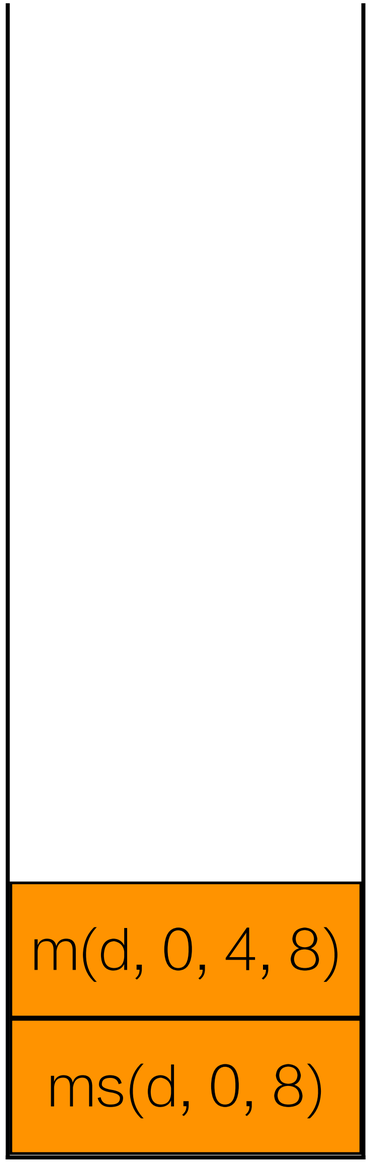
```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　mid：4　　right：8



data	35	47	60	77	95	15	28	52	80
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

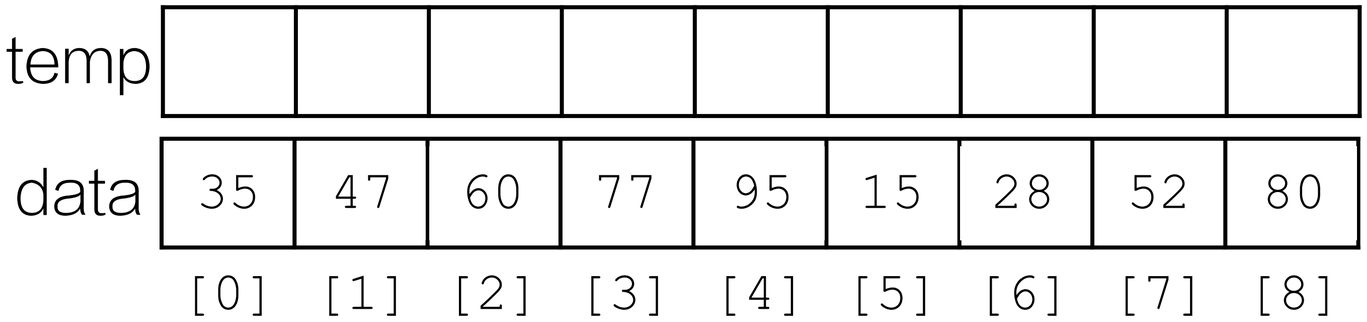
        // 对左半部分排序
        mergeSort(data, left, mid);

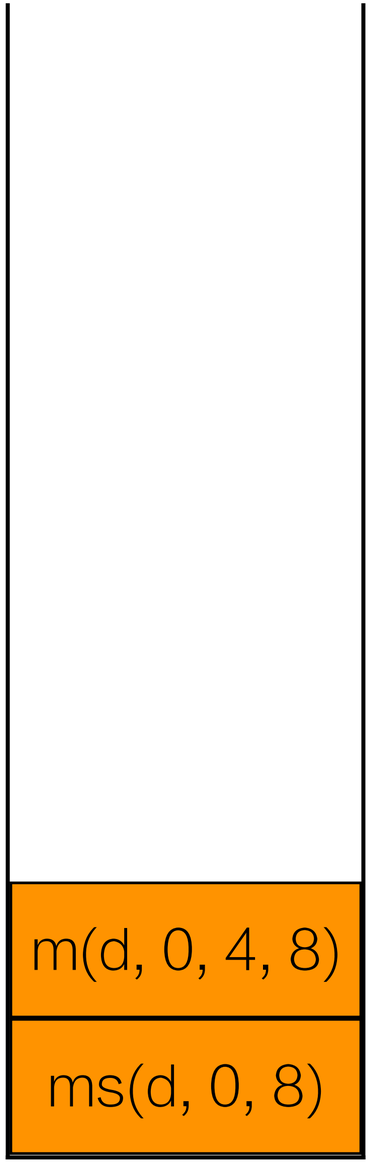
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 0　　j: 5　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

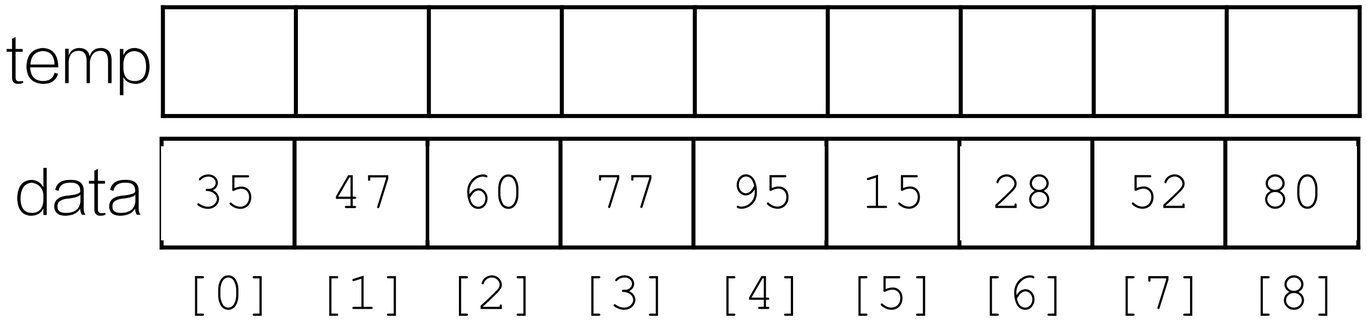
        // 对左半部分排序
        mergeSort(data, left, mid);

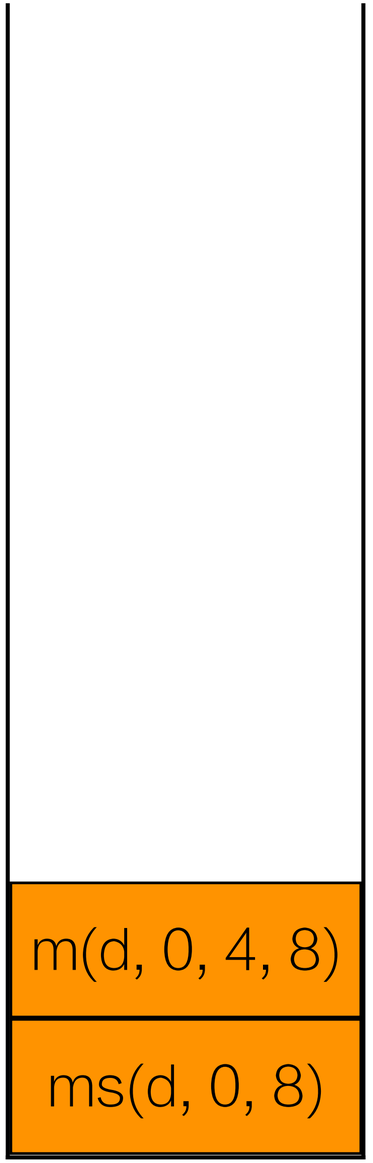
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 0　　j: 5　　k: 0





```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

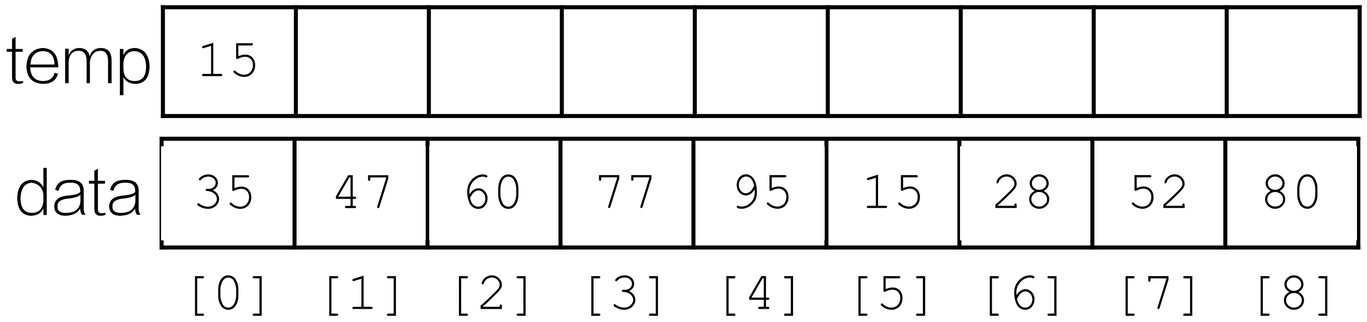
        // 对左半部分排序
        mergeSort(data, left, mid);

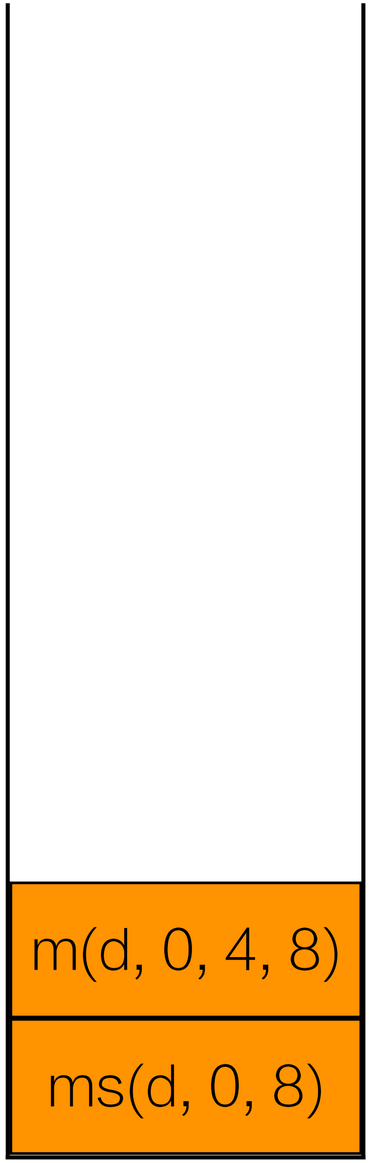
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 0　　j: 6　　k: 1





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 0　　j: 7　　k: 2

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28							
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 1　　j: 7　　k: 3

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35						
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

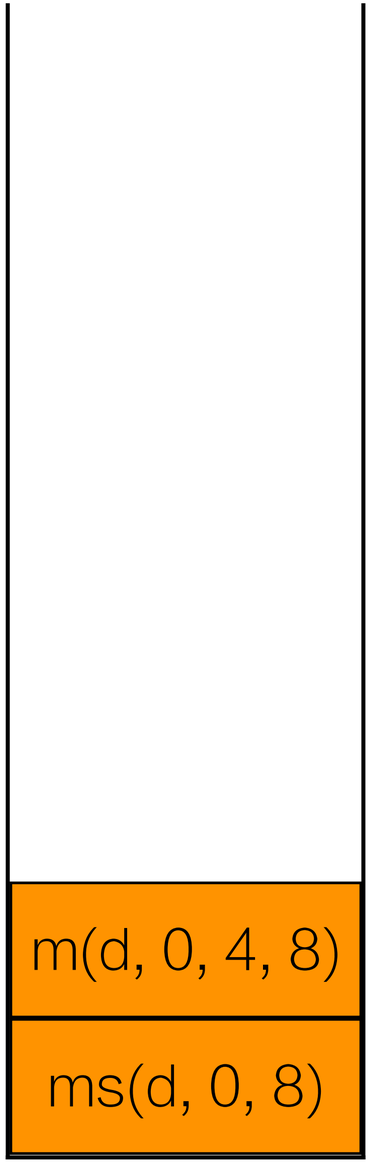
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 2　　j: 7　　k: 4

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47					
data	35	47	60	77	95	15	28	52	80



```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 2　　j: 8　　k: 5

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52				
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 3　　j: 8　　k: 6

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60			
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

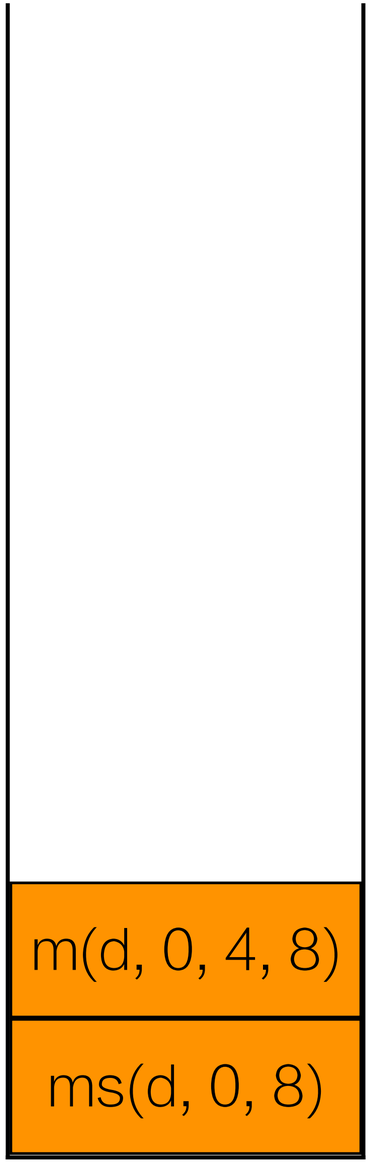
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 4　　j: 8　　k: 7

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77		
data	35	47	60	77	95	15	28	52	80



```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 4　　j: 9　　k: 8

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 4　　j: 9　　k: 8

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 5　　j: 9　　k: 9

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	95
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 5　　j: 9　　k: 9

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	95
data	35	47	60	77	95	15	28	52	80

m(d, 0, 4, 8)

ms(d, 0, 8)

```cpp
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```cpp
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

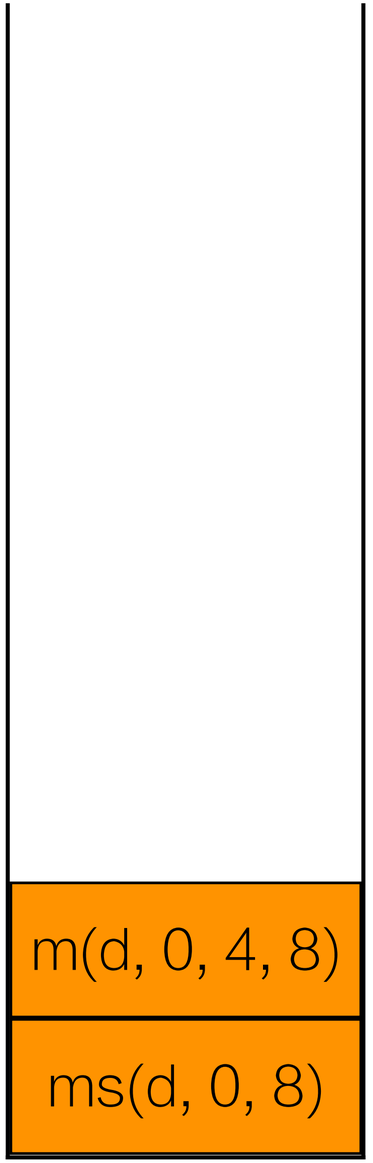
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 5　　j: 9　　k: 9

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	95
data	35	47	60	77	95	15	28	52	80



```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

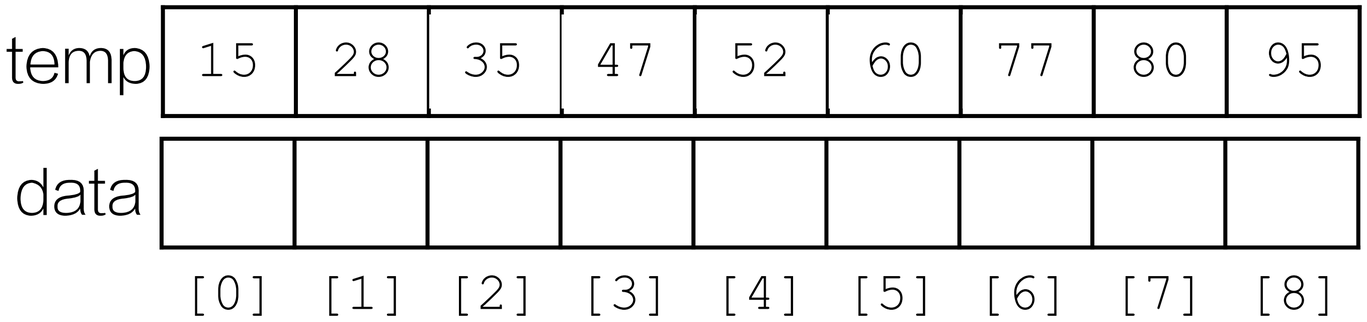
        // 对左半部分排序
        mergeSort(data, left, mid);

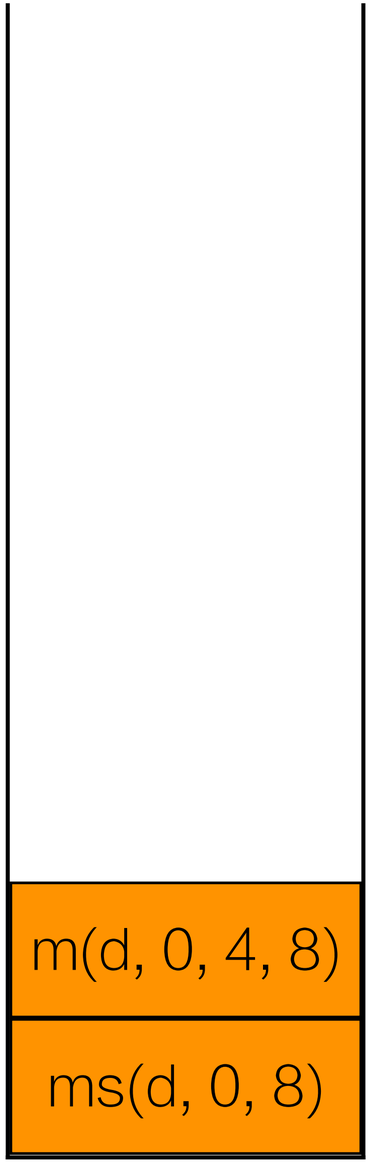
        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 5　　j: 9　　k: 9





```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;       // 左半部分起始索引
    int j = mid + 1;    // 右半部分起始索引
    int k = 0;          // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left: 0　　mid: 4　　right: 8

i: 5　　j: 9　　k: 9

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
temp	15	28	35	47	52	60	77	80	95
data	15	28	35	47	52	60	77	80	95

m(d, 0, 4, 8)

ms(d, 0, 8)

```
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

left：0　　right：8　　mid：4

data	15	28	35	47	52	60	77	80	95
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

ms(d, 0, 8)

```c
void merge(int* data, int left, int mid, int right)
{
    int temp[100]; // 临时数组，用于存放合并结果
    int i = left;      // 左半部分起始索引
    int j = mid + 1;   // 右半部分起始索引
    int k = 0;         // temp 数组下标
    // 将两个有序部分按照大小关系依次放入 temp 中
    while (i <= mid && j <= right)
    {
        if (data[i] <= data[j])
        {
            temp[k++] = data[i++];
        }
        else
        {
            temp[k++] = data[j++];
        }
    }
    // 将剩余的左边部分拷贝到 temp 中（如果有）
    while (i <= mid)
    {
        temp[k++] = data[i++];
    }
    // 将剩余的右边部分拷贝到 temp 中（如果有）
    while (j <= right)
    {
        temp[k++] = data[j++];
    }
    // 将 temp 中的结果拷贝回原数组中，完成合并
    for (int t = 0; t < k; t++)
    {
        data[left + t] = temp[t];
    }
}
```

```c
void mergeSort(int* data, int left, int right)
{
    if (left < right) // 至少两个元素才需要排序
    {
        int mid = (left + right) / 2;

        // 对左半部分排序
        mergeSort(data, left, mid);

        // 对右半部分排序
        mergeSort(data, mid + 1, right);

        // 将排好序的左右两部分合并
        merge(data, left, mid, right);
    }
}
```

data	15	28	35	47	52	60	77	80	95
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

归并排序

维度	描述
时间复杂度	O(nlogn)
空间复杂度	O(n)
算法稳定性	稳定
适用场景	大数据量排序、对稳定性有要求的场景等

快速排序

快速排序是一种非常高效的排序算法，采用了分治思想（Divide and Conquer）。其基本策略是：

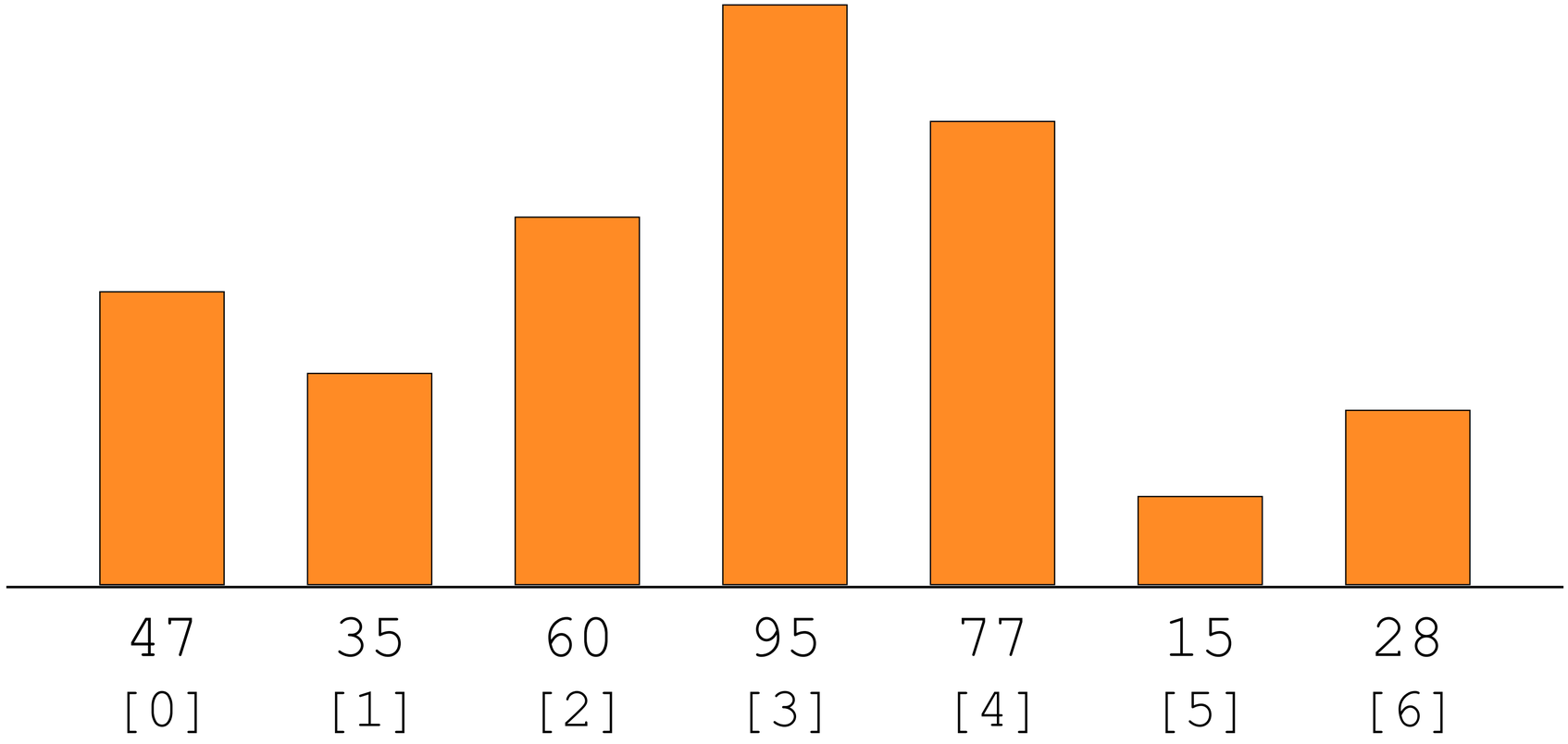
1. 从序列中选择一个“基准元素”

2. 将序列划分为两个子序列，所有比基准元素小的放到左边，比基准元素大的放到右边。

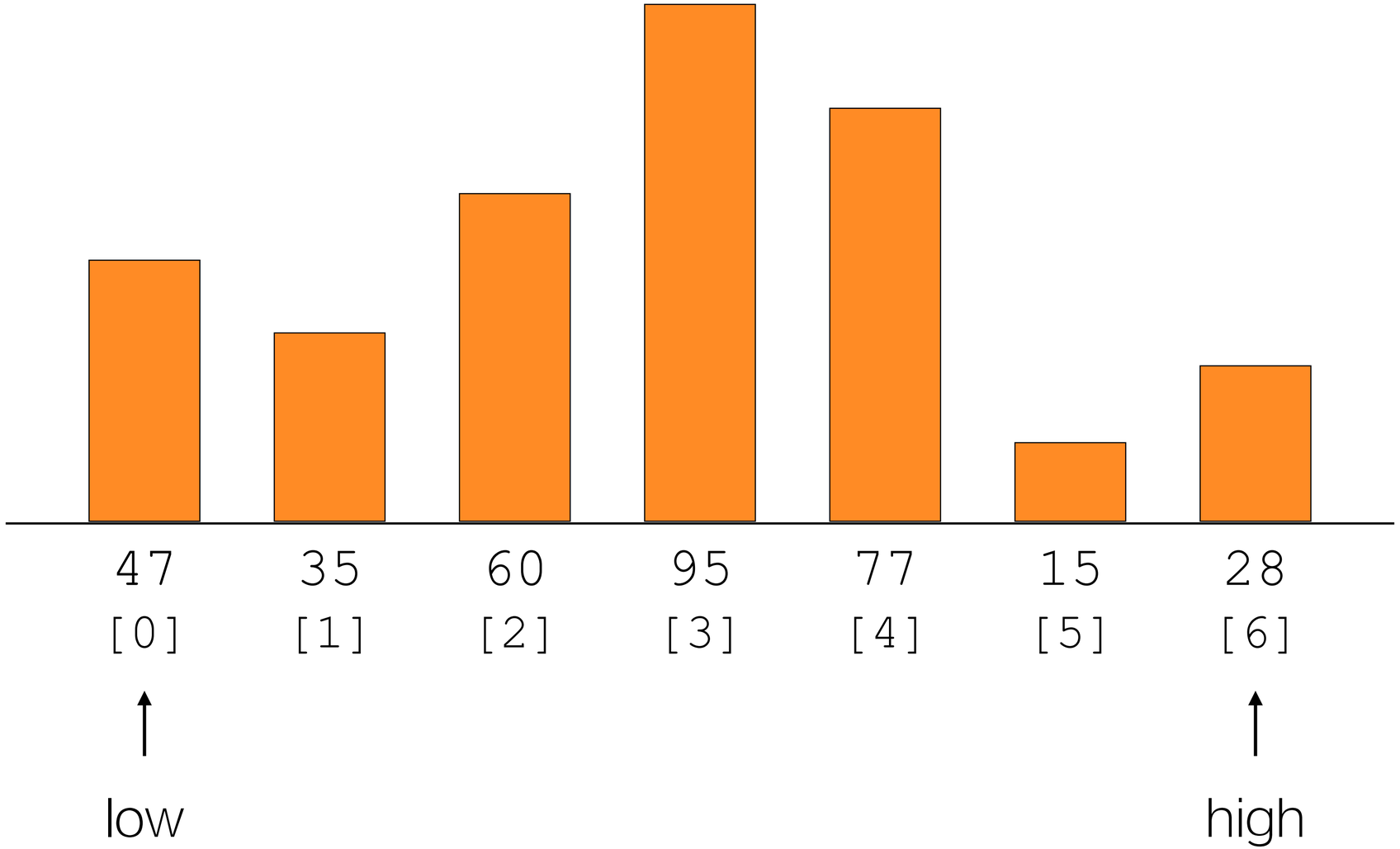
3. 对左右两个子序列递归进行快速排序

4. 直到子序列长度为 1 或 0，排序完成

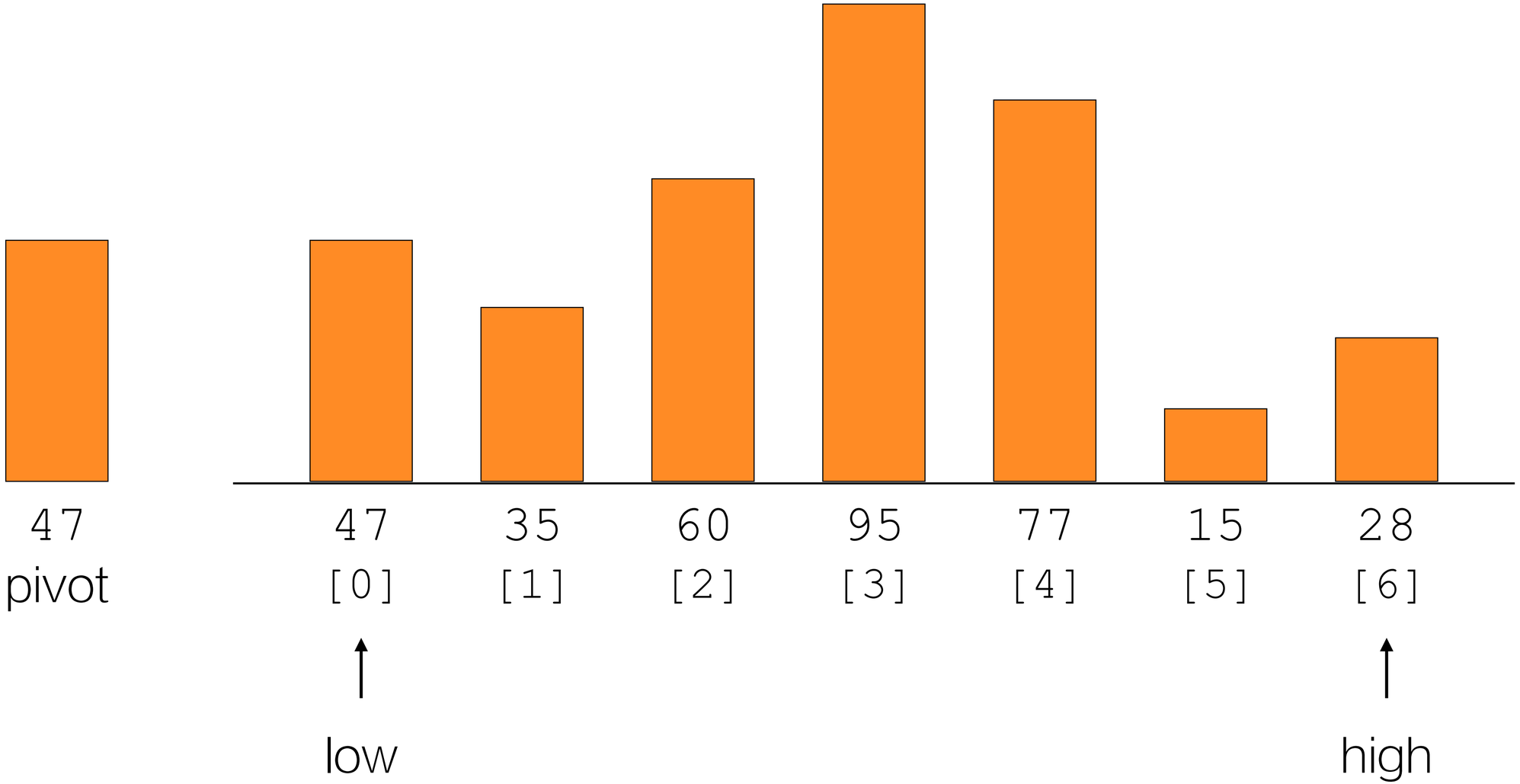
快速排序



快速排序

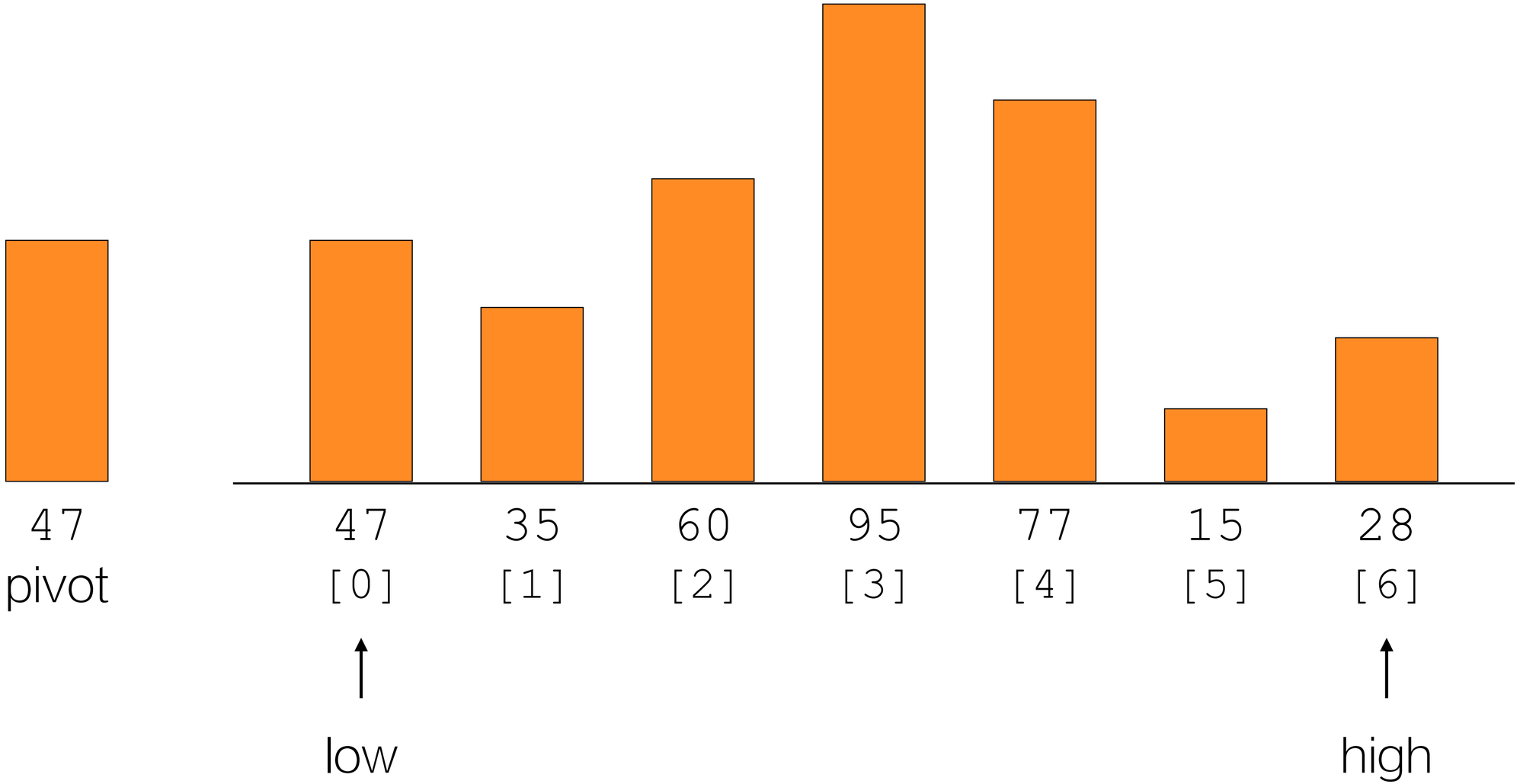


快速排序

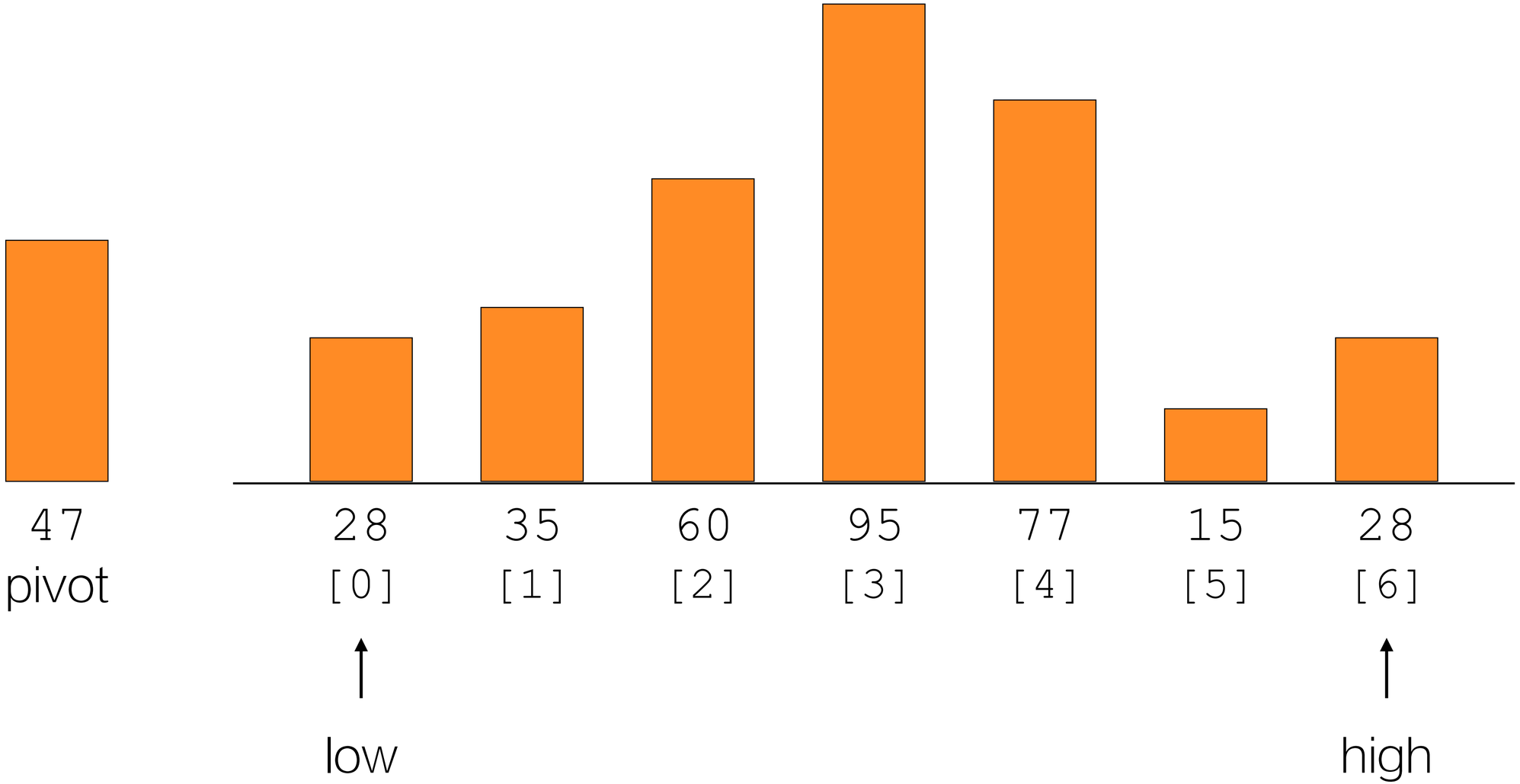


快速排序

data[high] > pivot ?

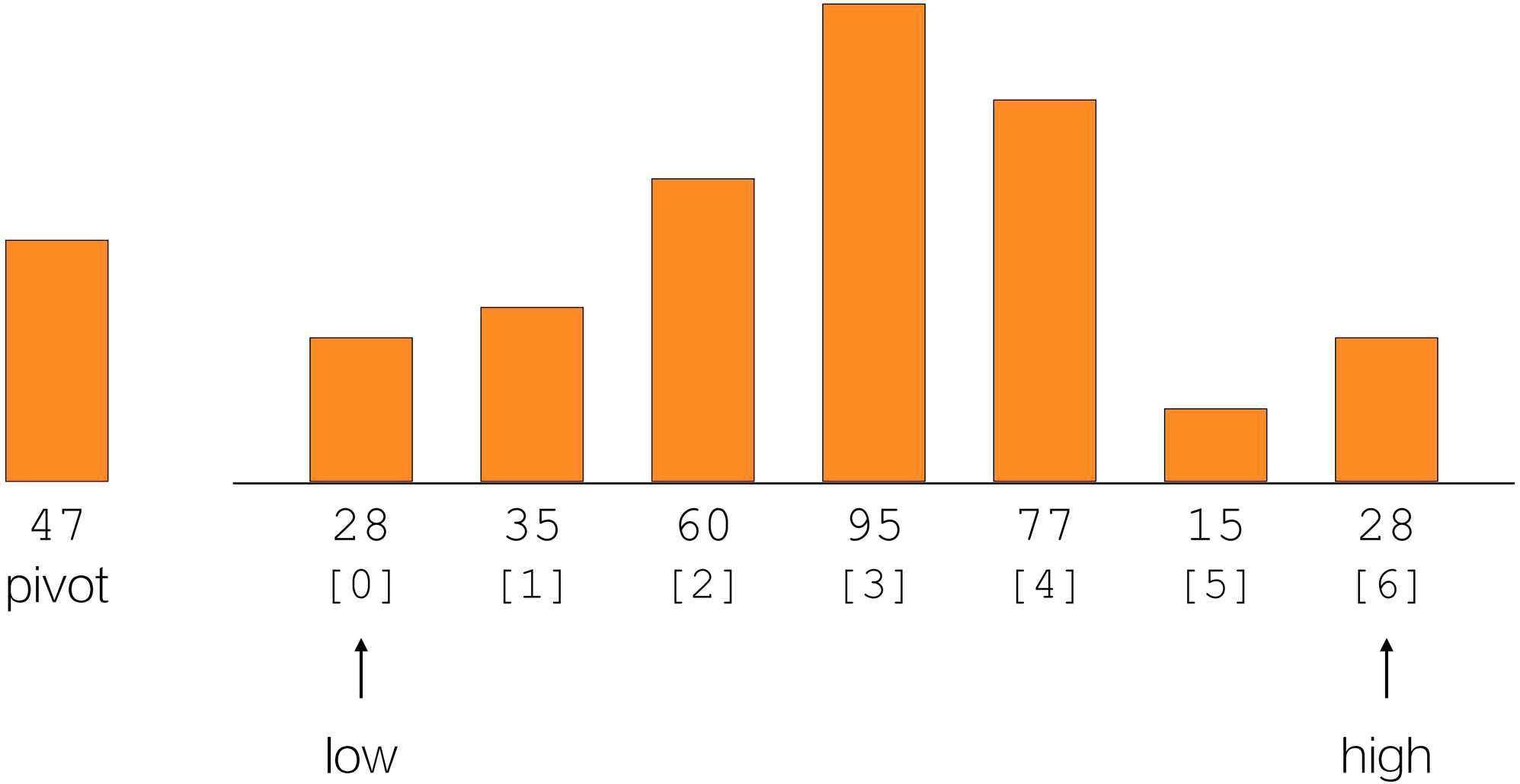


快速排序



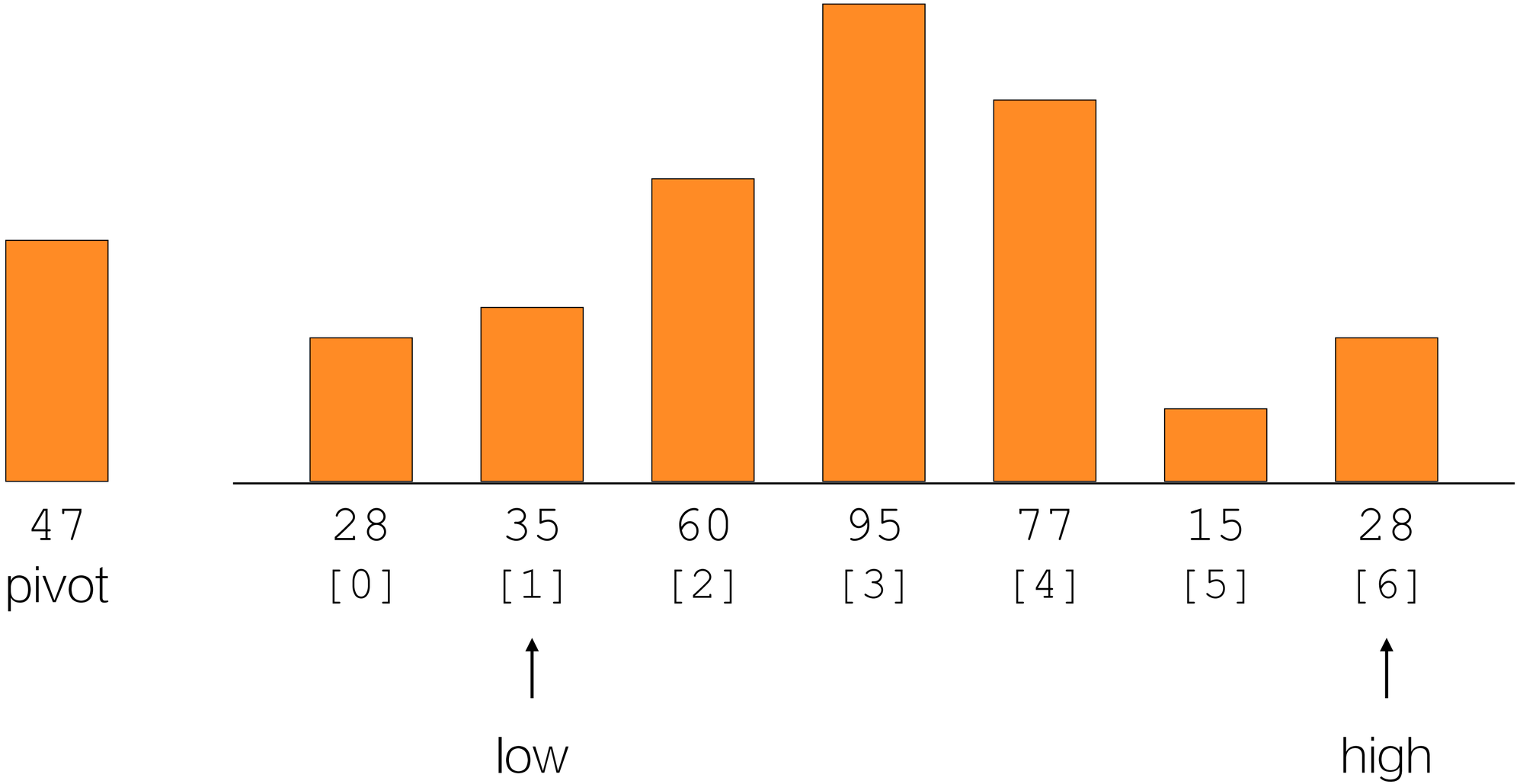
快速排序

data[low] < pivot ?



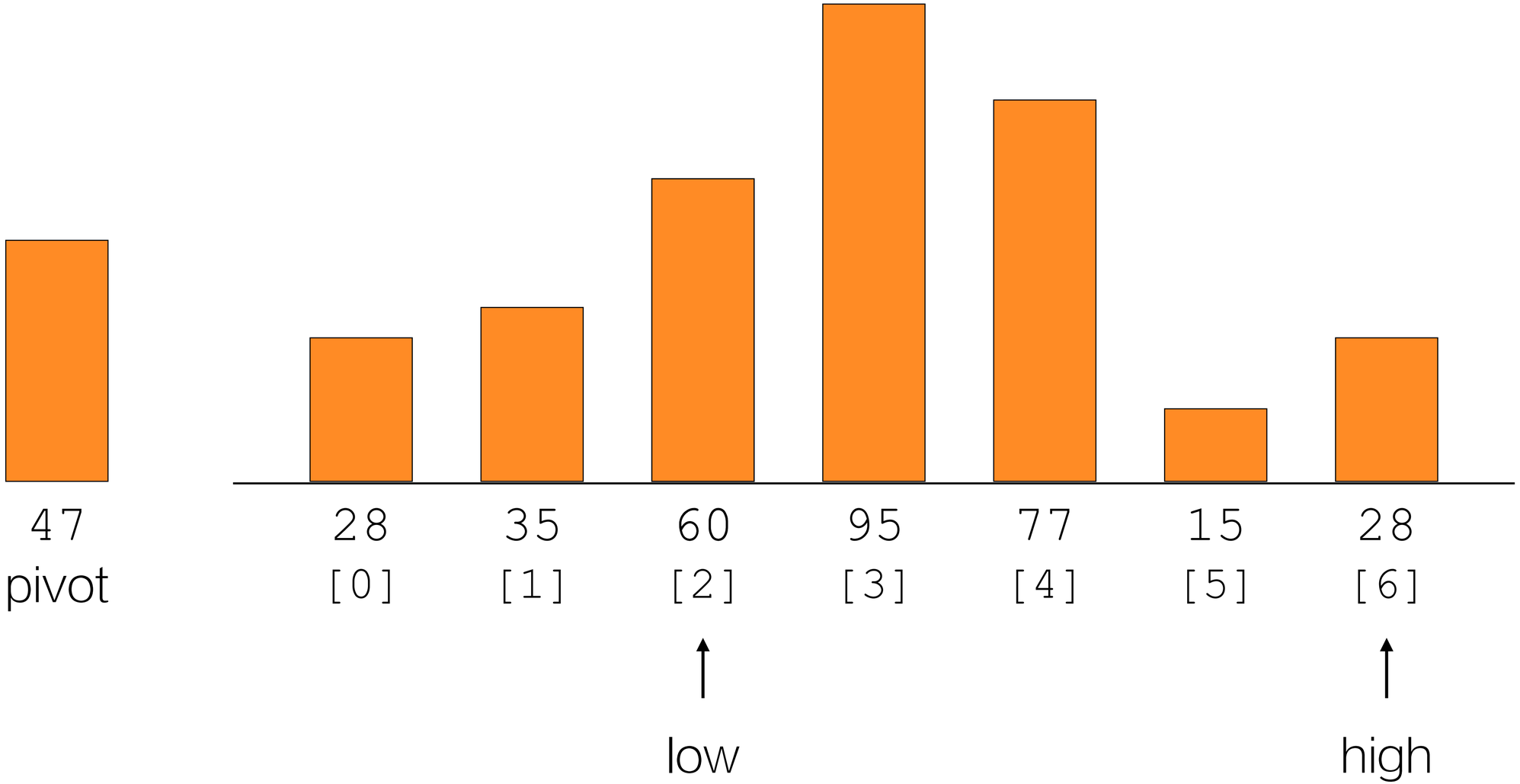
快速排序

data[low] < pivot ?

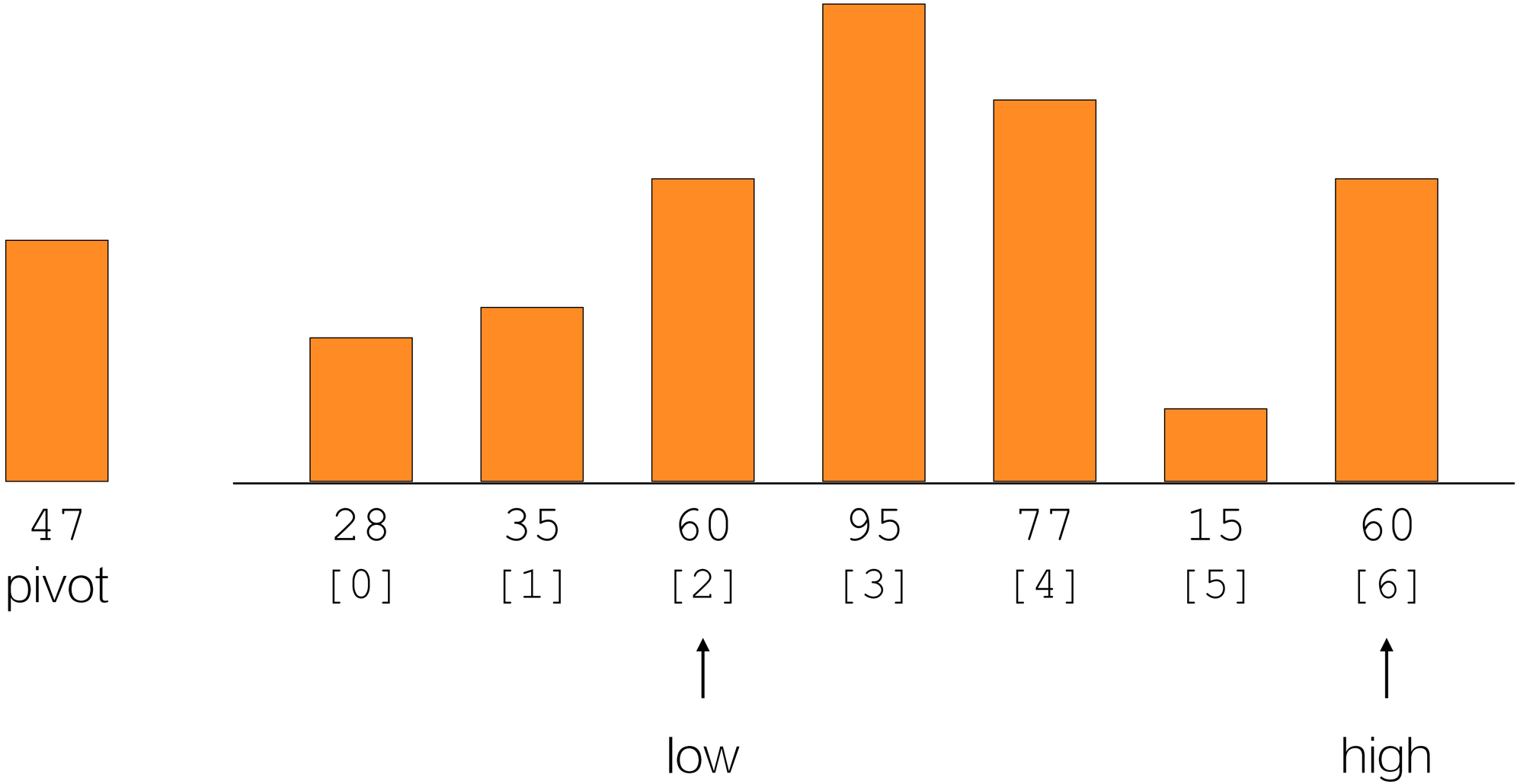


快速排序

data[low] < pivot ?

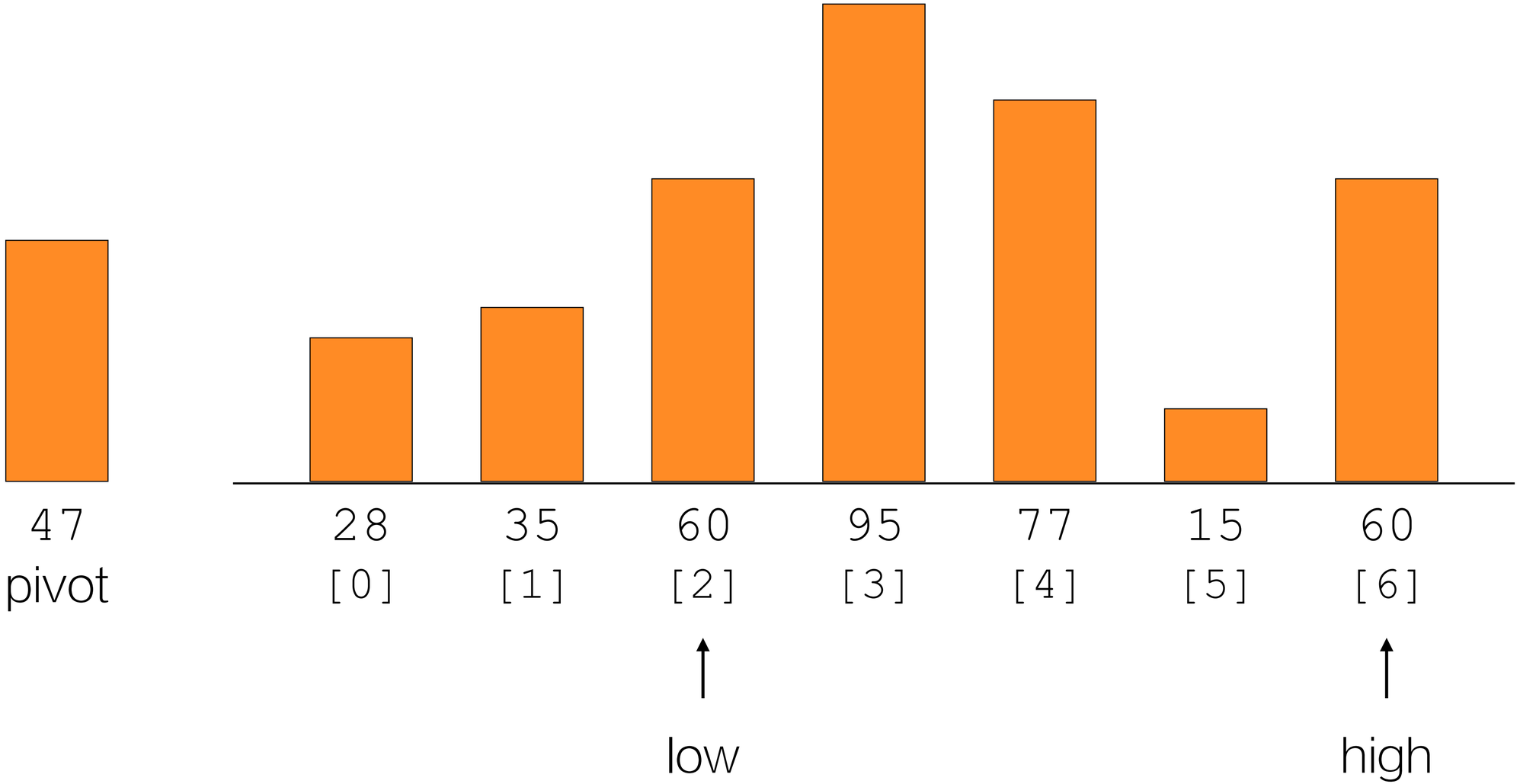


快速排序



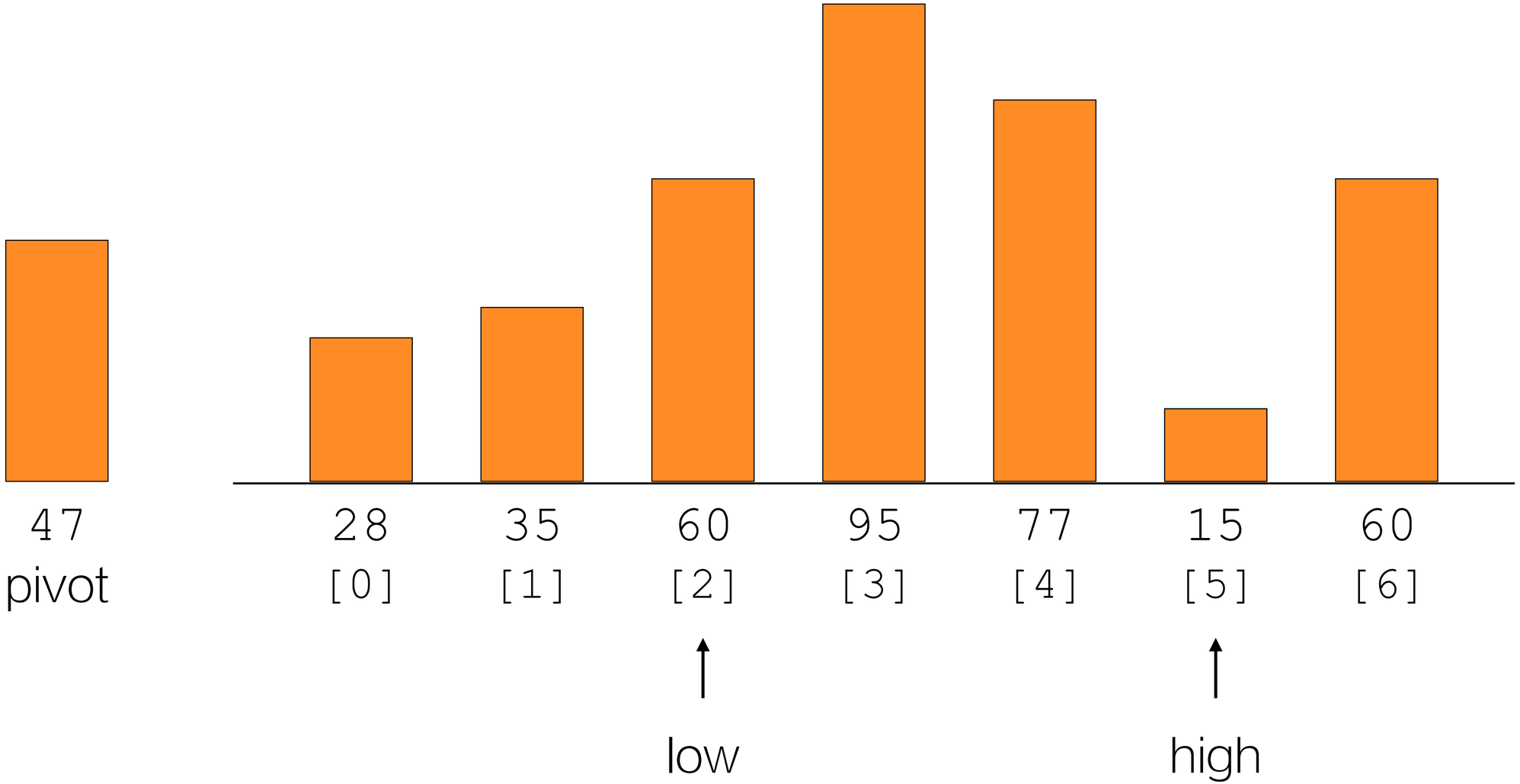
快速排序

data[high] > pivot ?

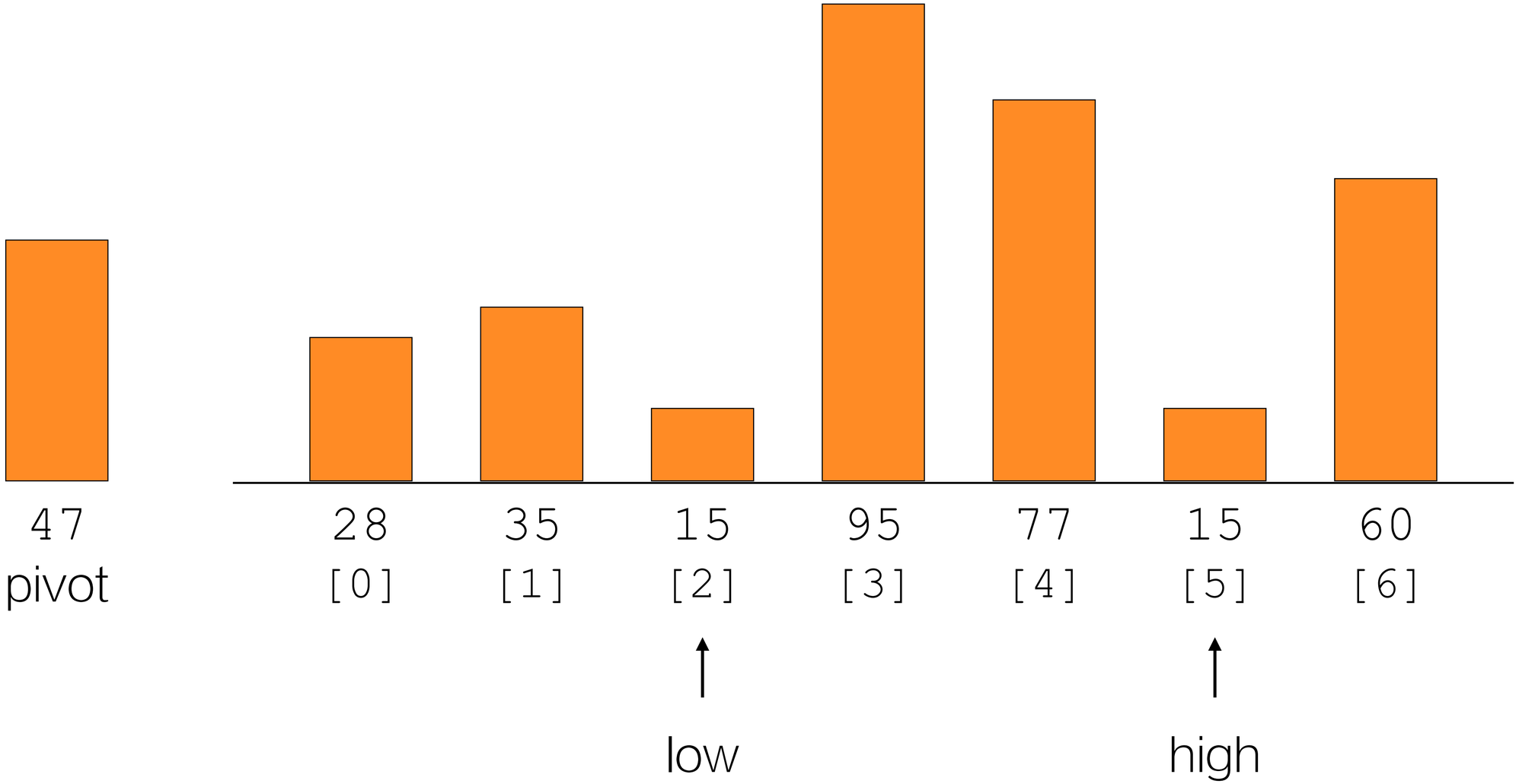


快速排序

data[high] > pivot ?

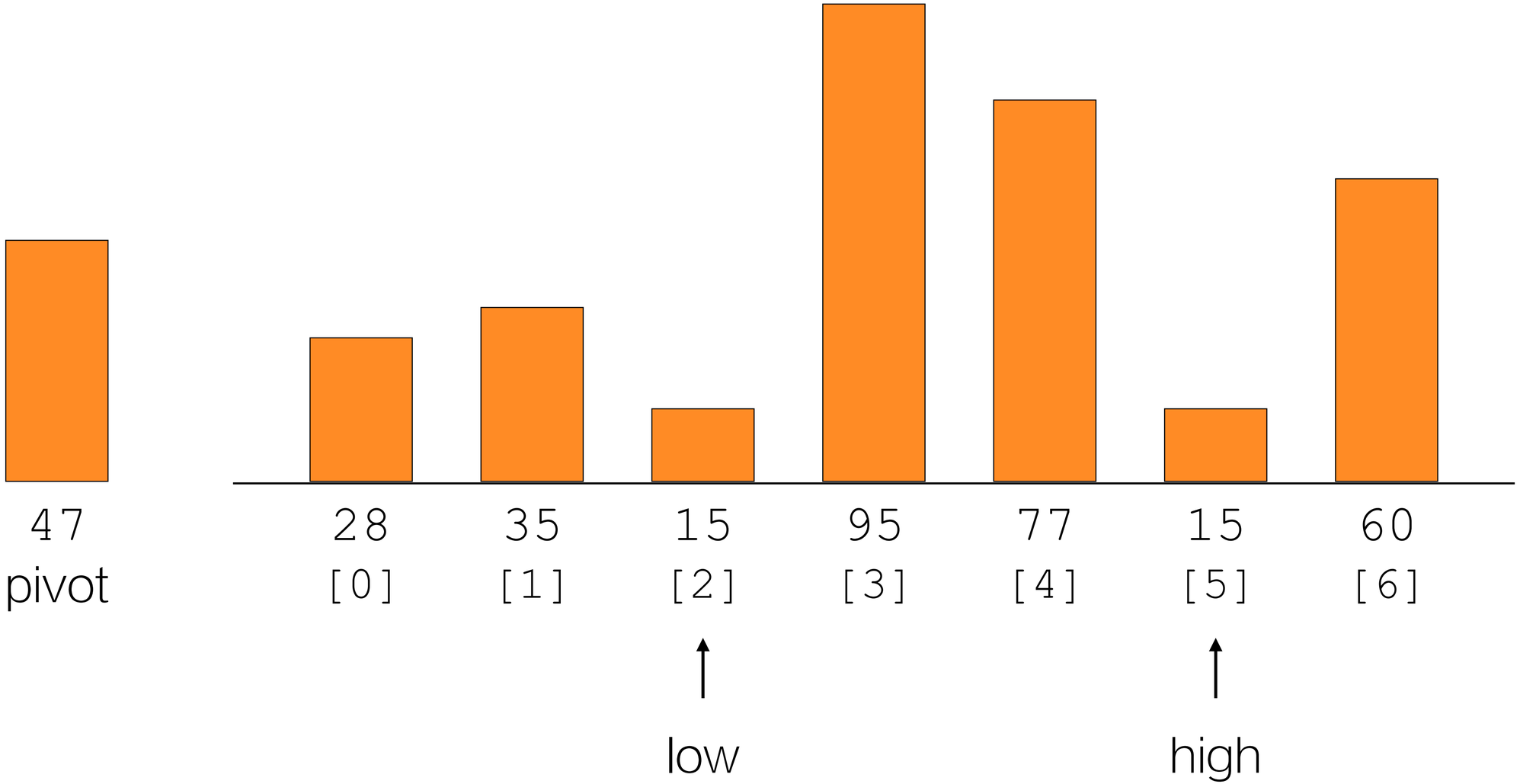


快速排序



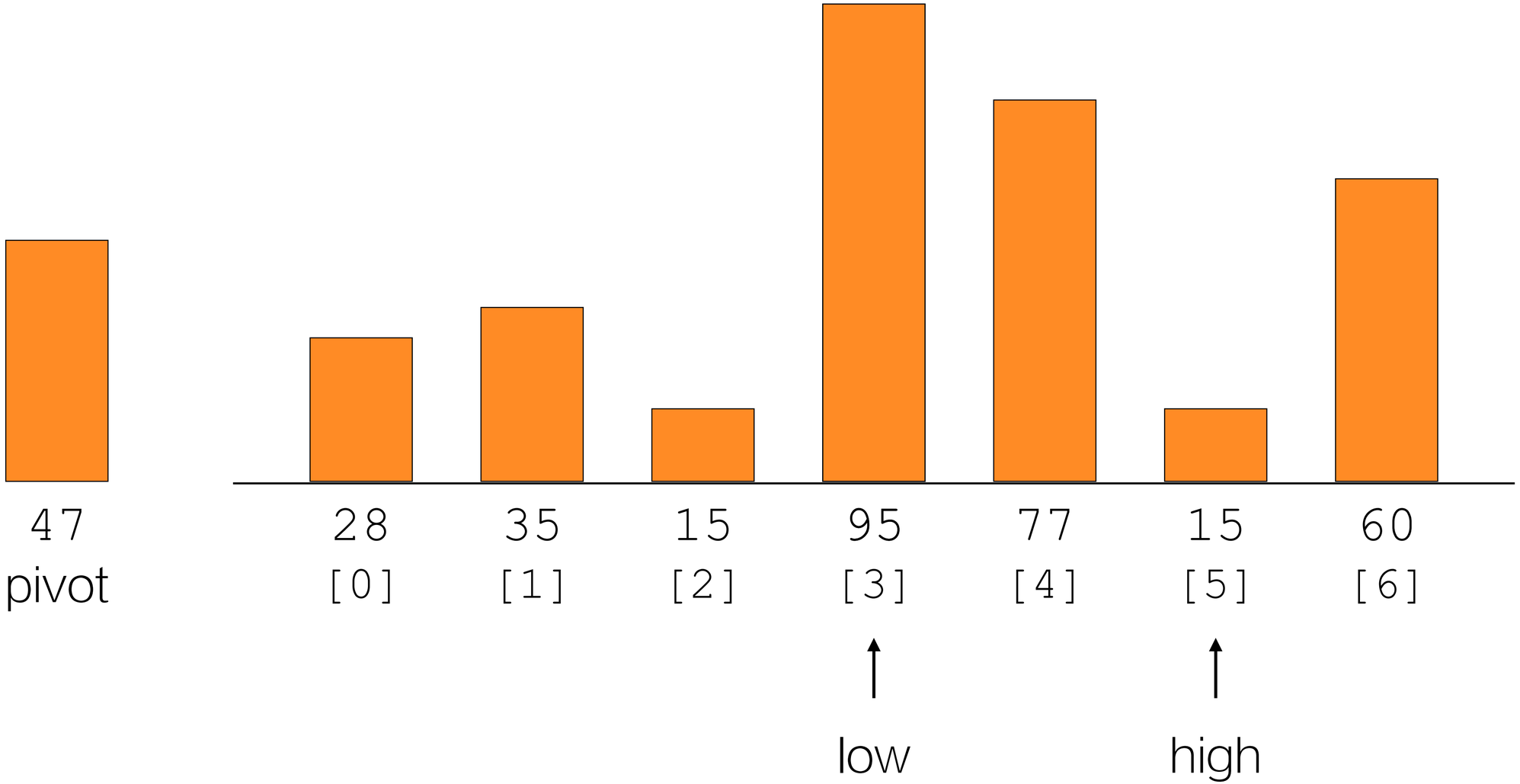
快速排序

data[low] < pivot ?

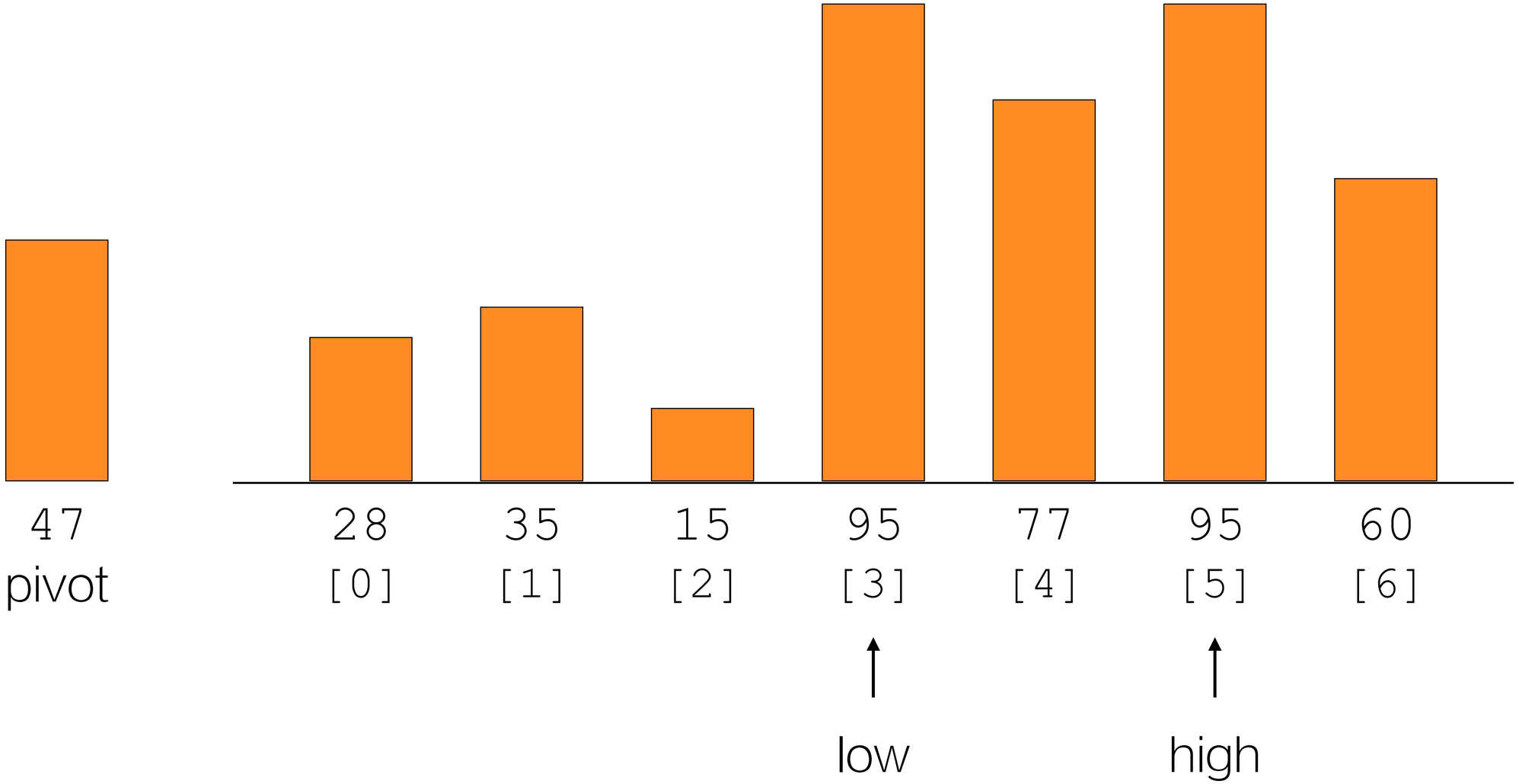


快速排序

data[low] < pivot ?

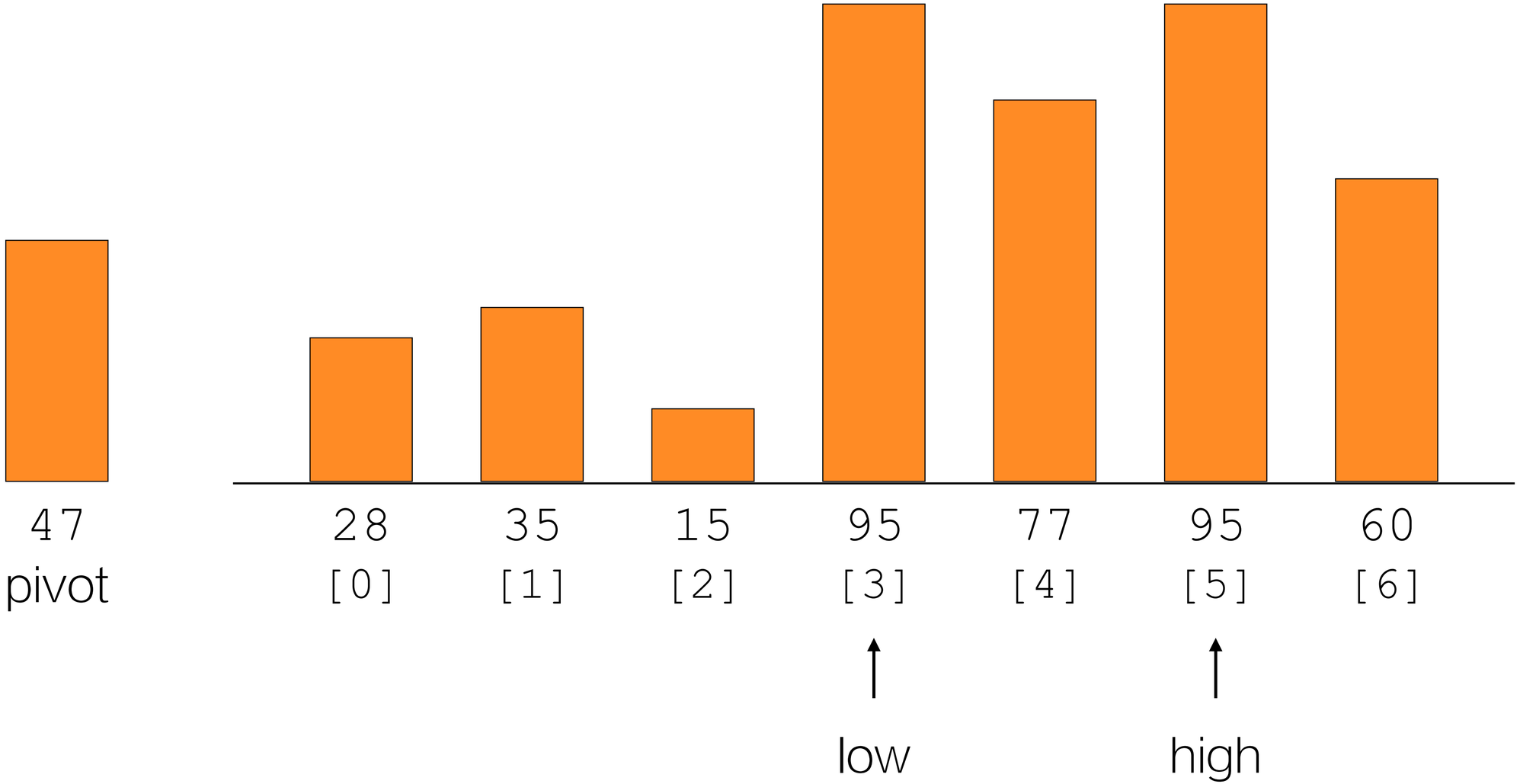


快速排序



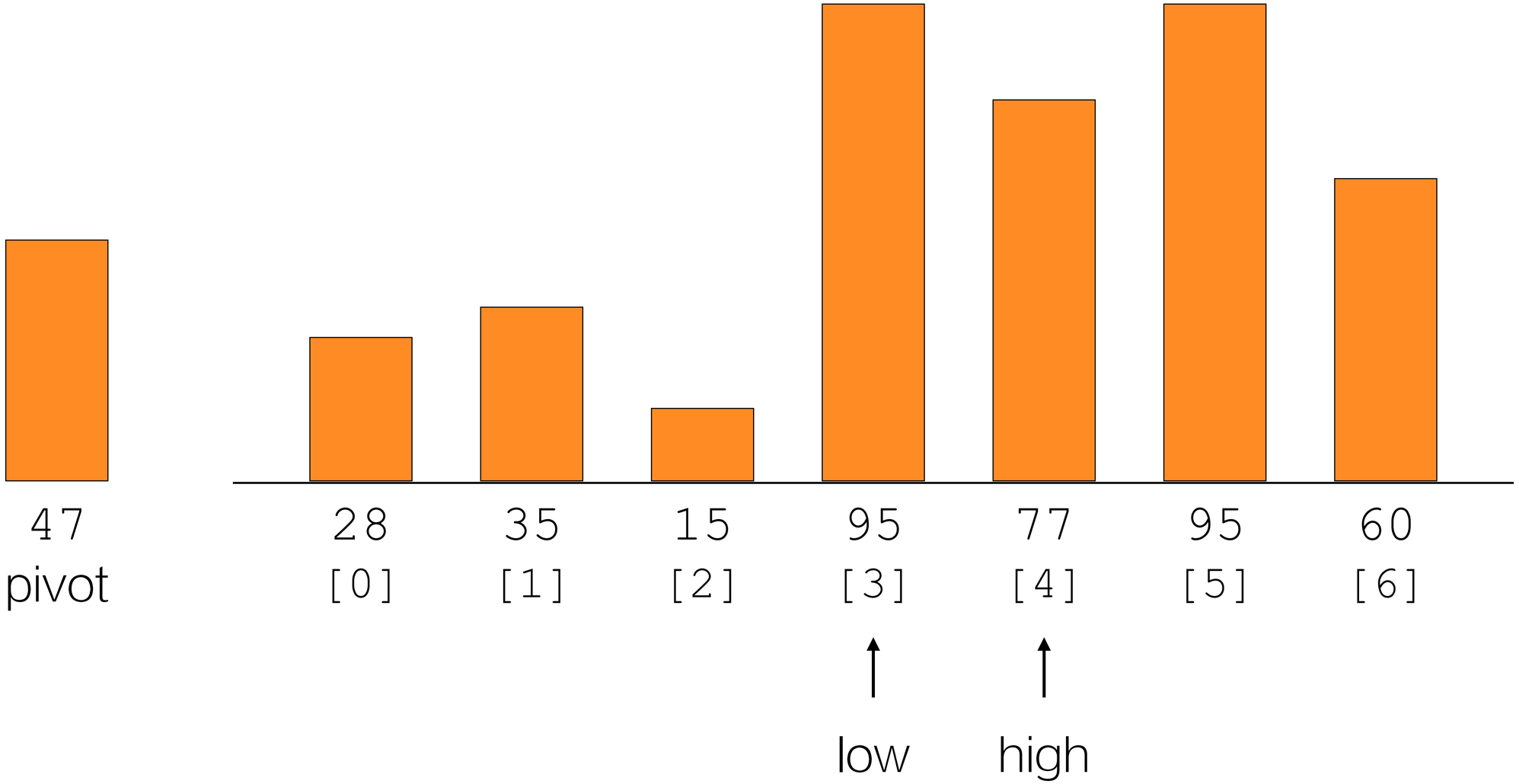
快速排序

data[high] > pivot ?

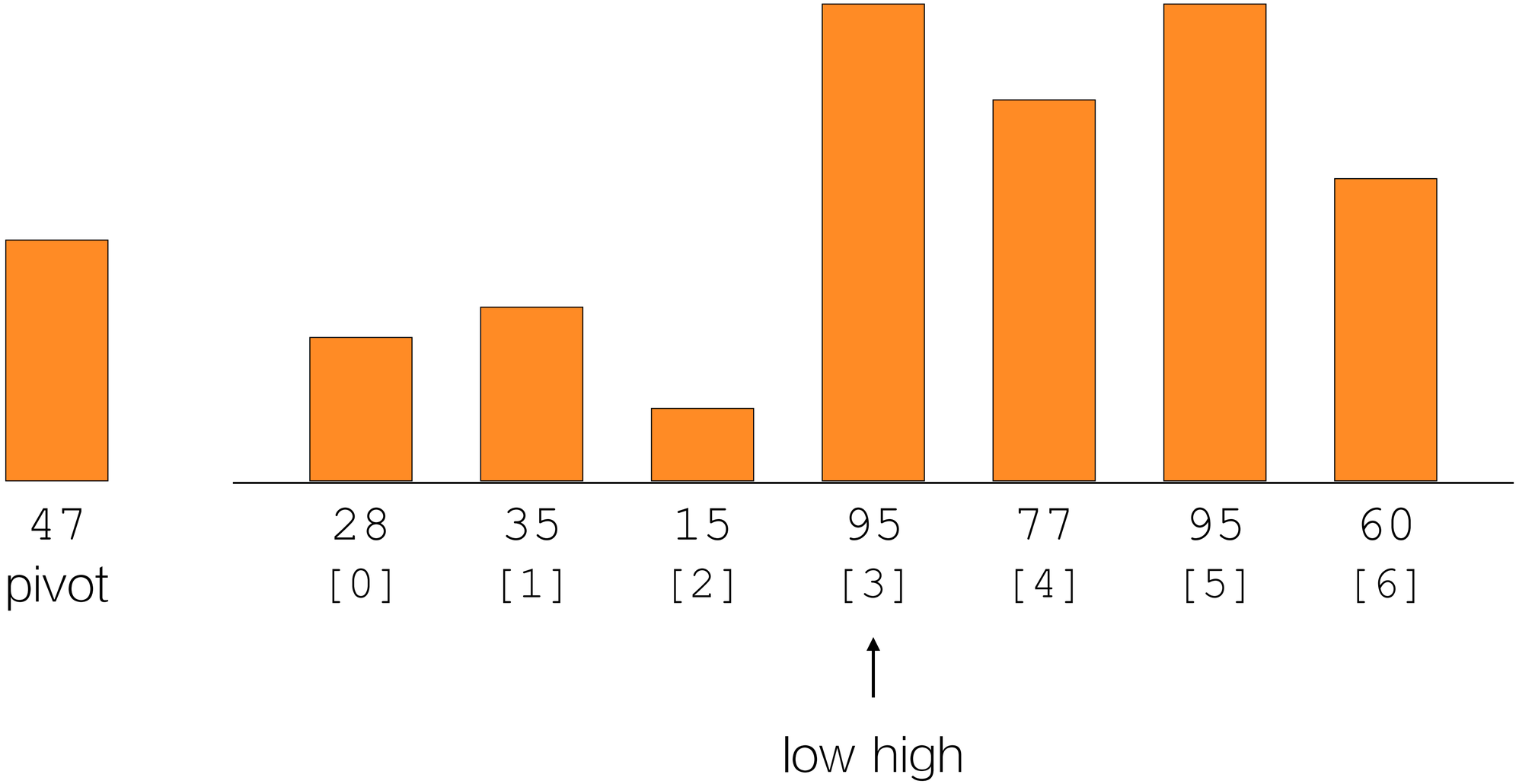


快速排序

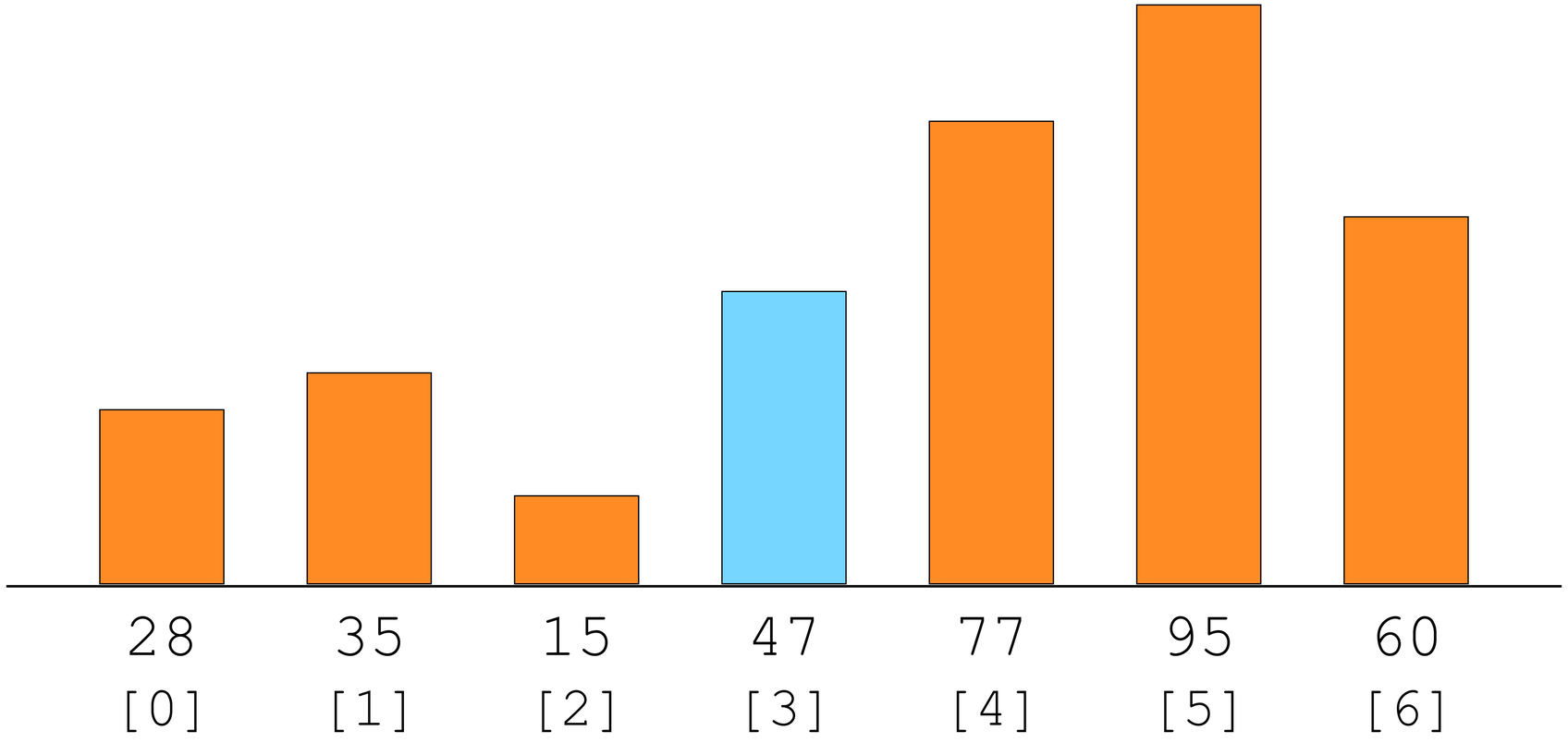
data[high] > pivot ?



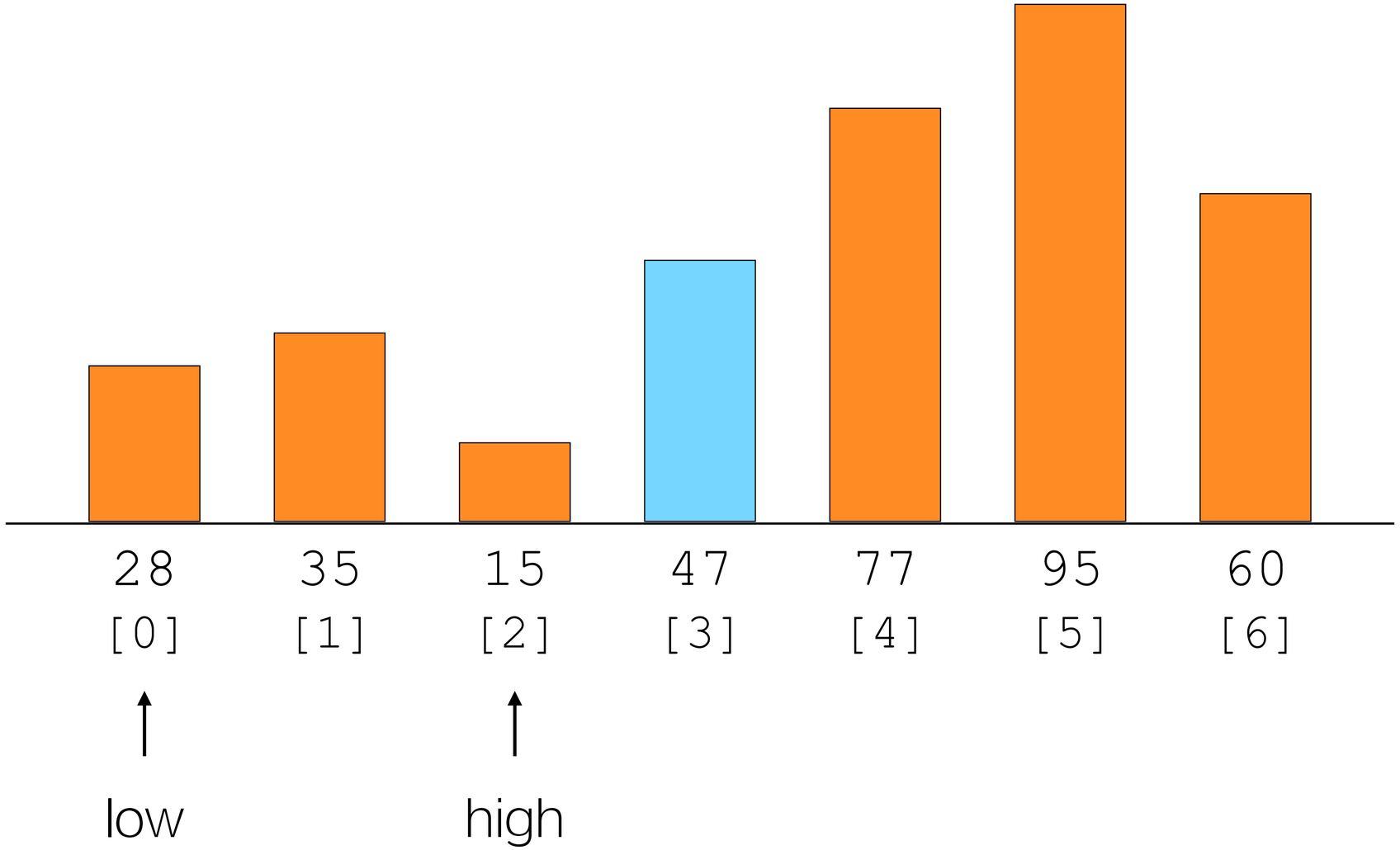
快速排序



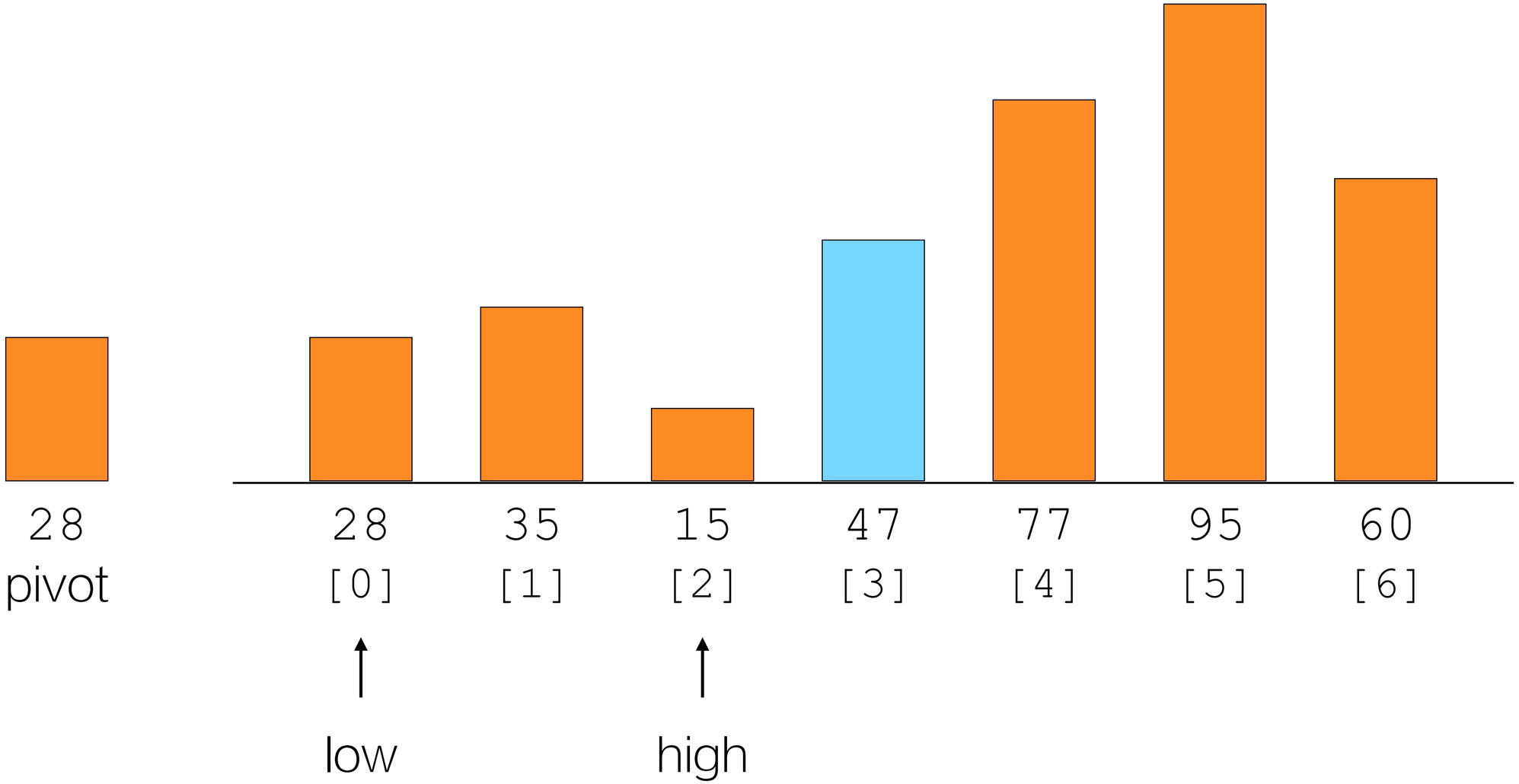
快速排序



快速排序

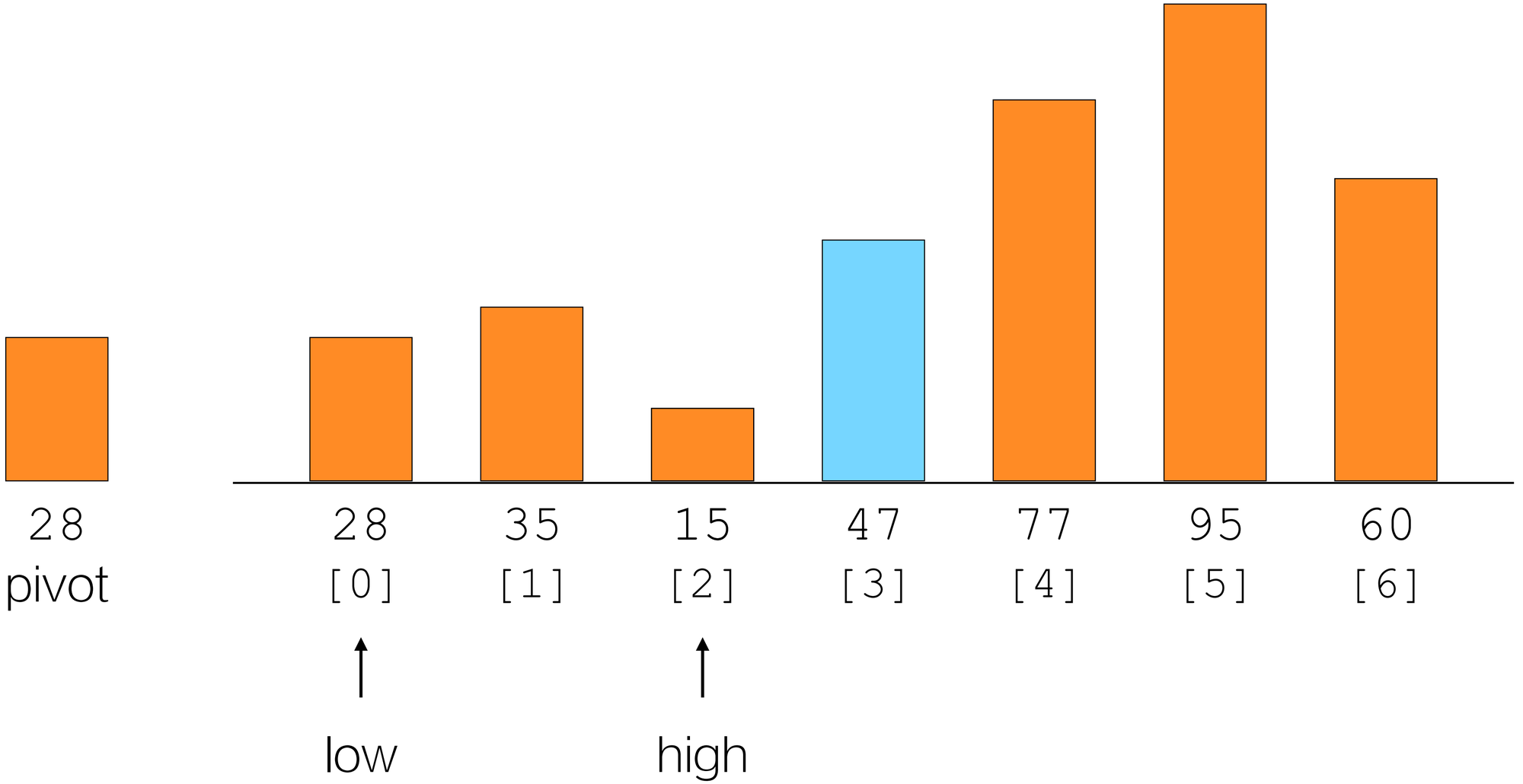


快速排序

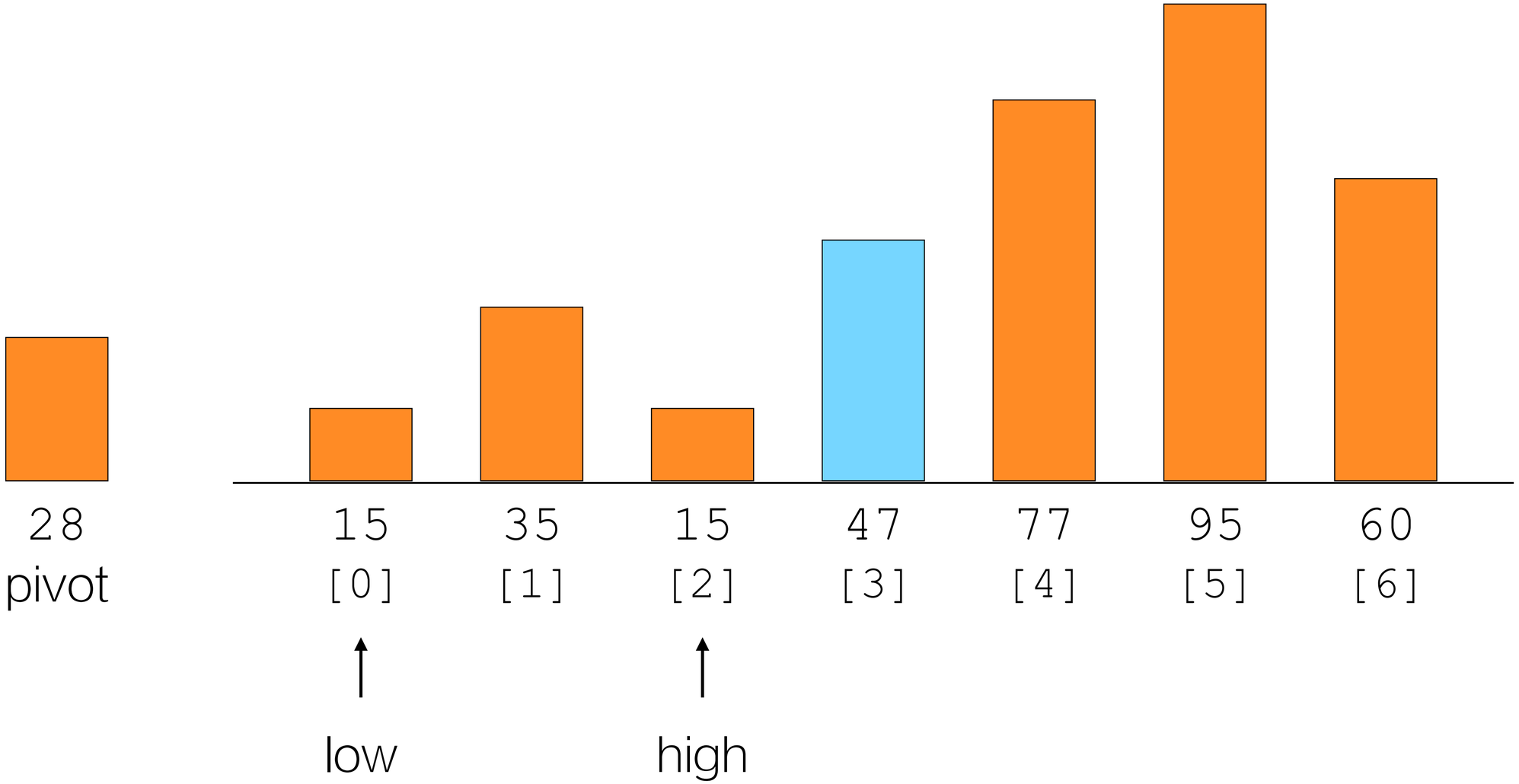


快速排序

data[high] > pivot ?

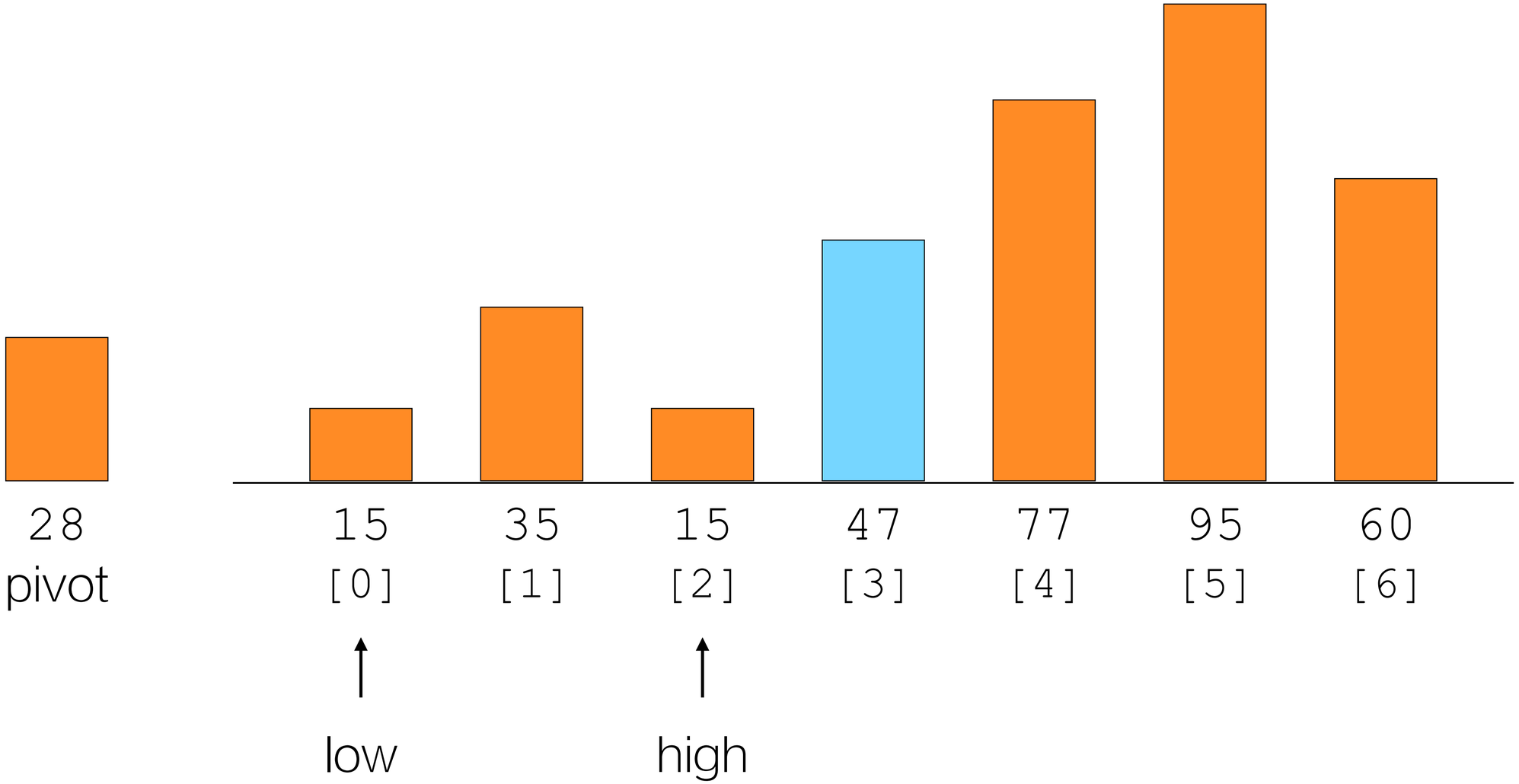


快速排序



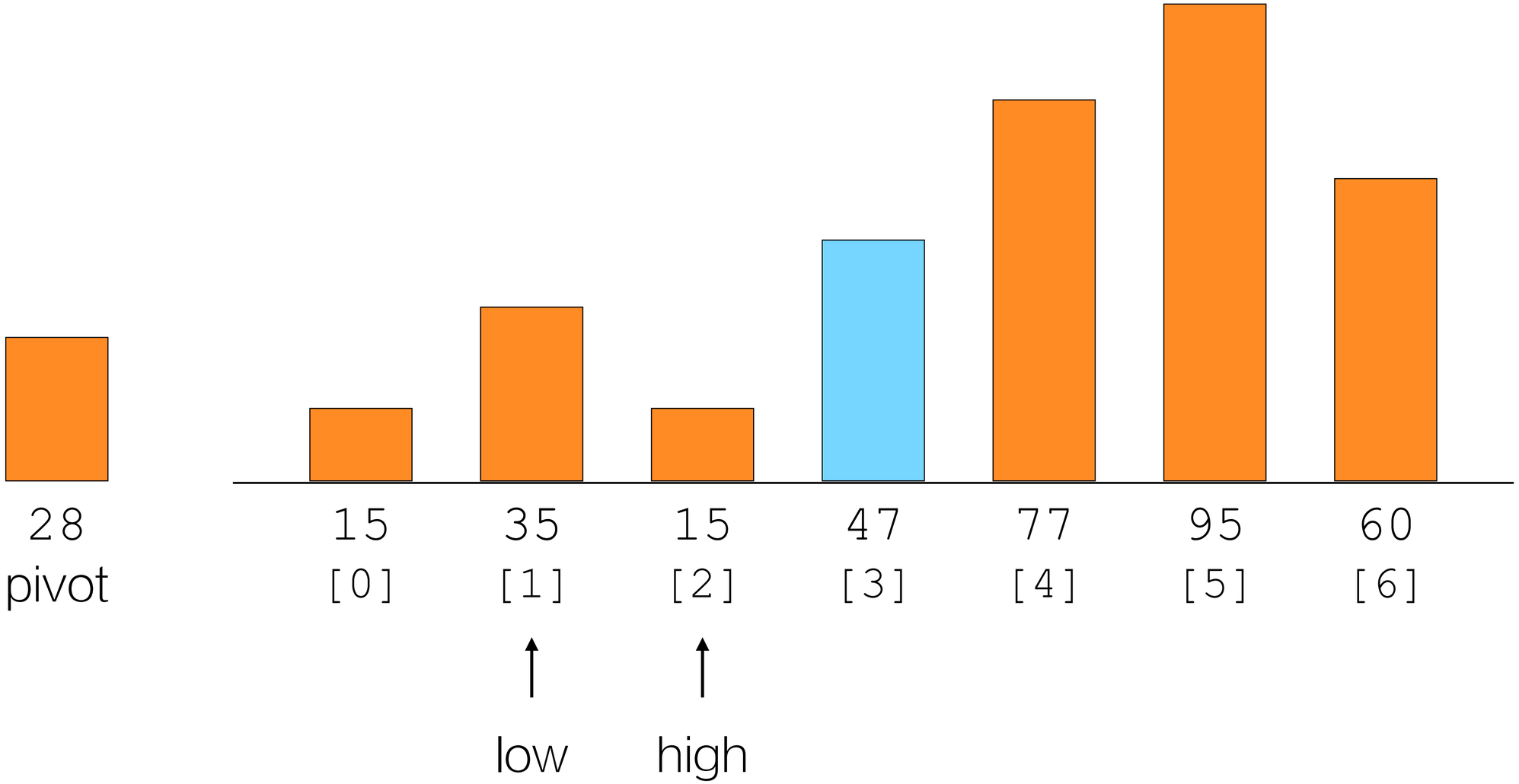
快速排序

data[low] < pivot ?

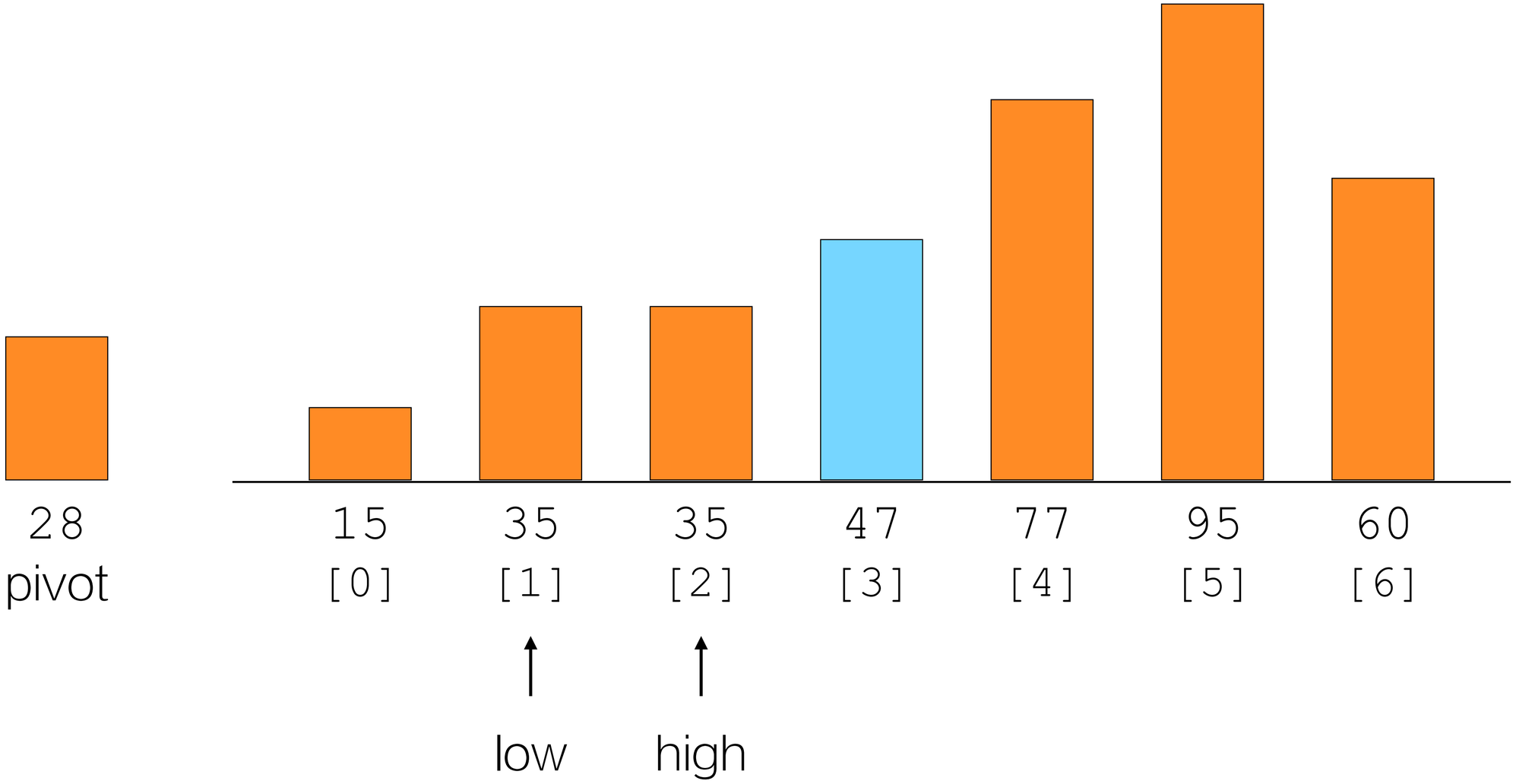


快速排序

data[low] < pivot ?

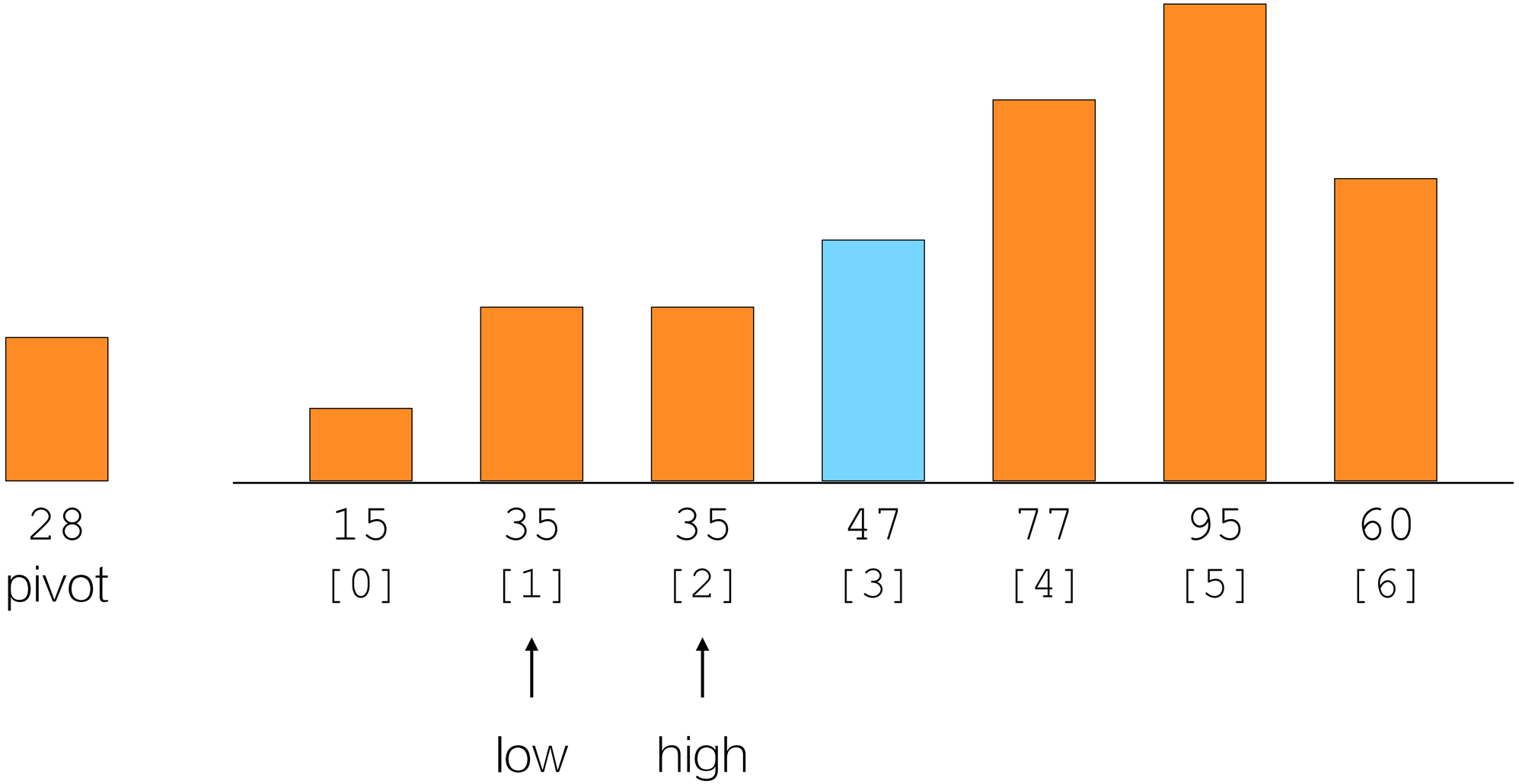


快速排序



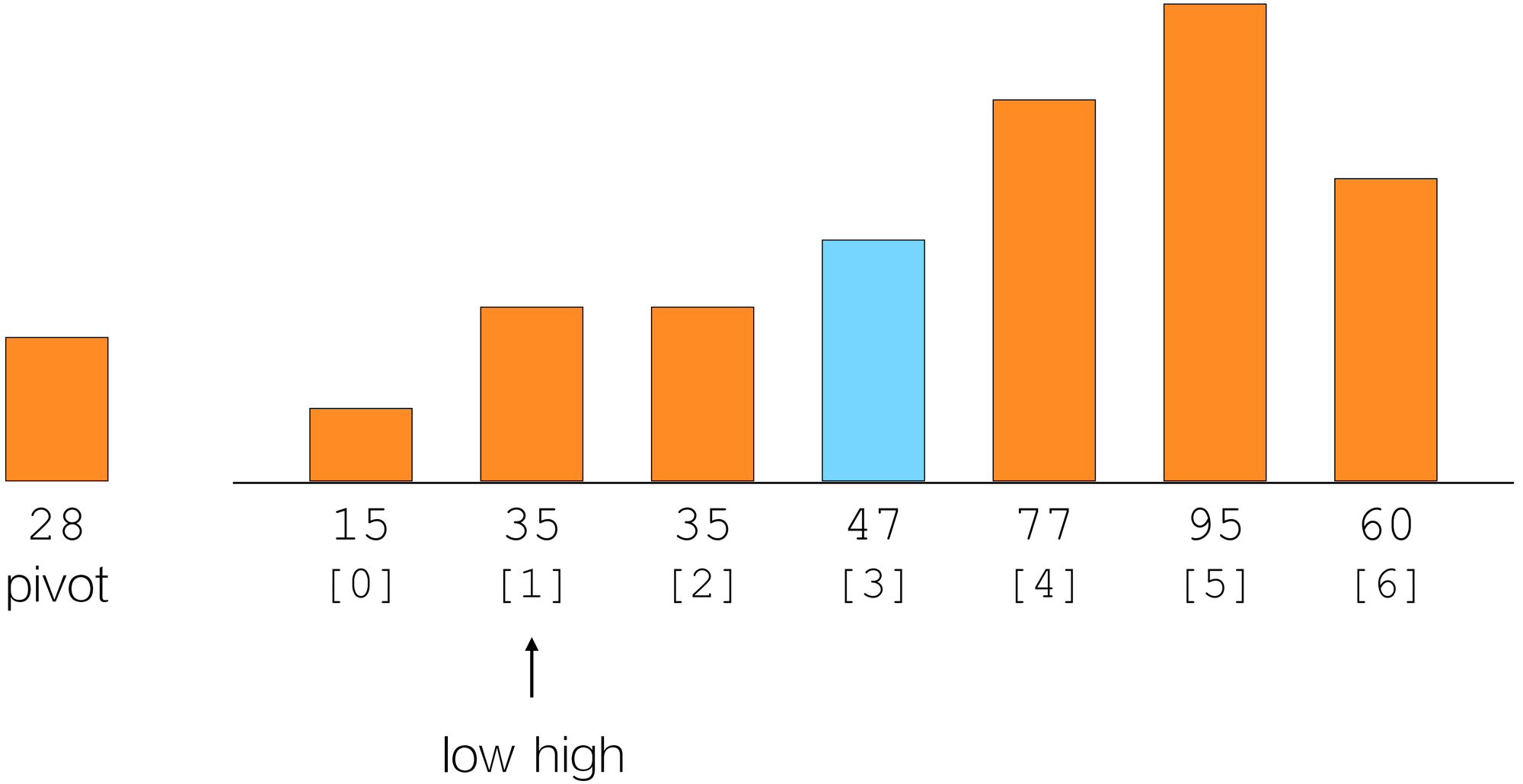
快速排序

data[high] > pivot ?

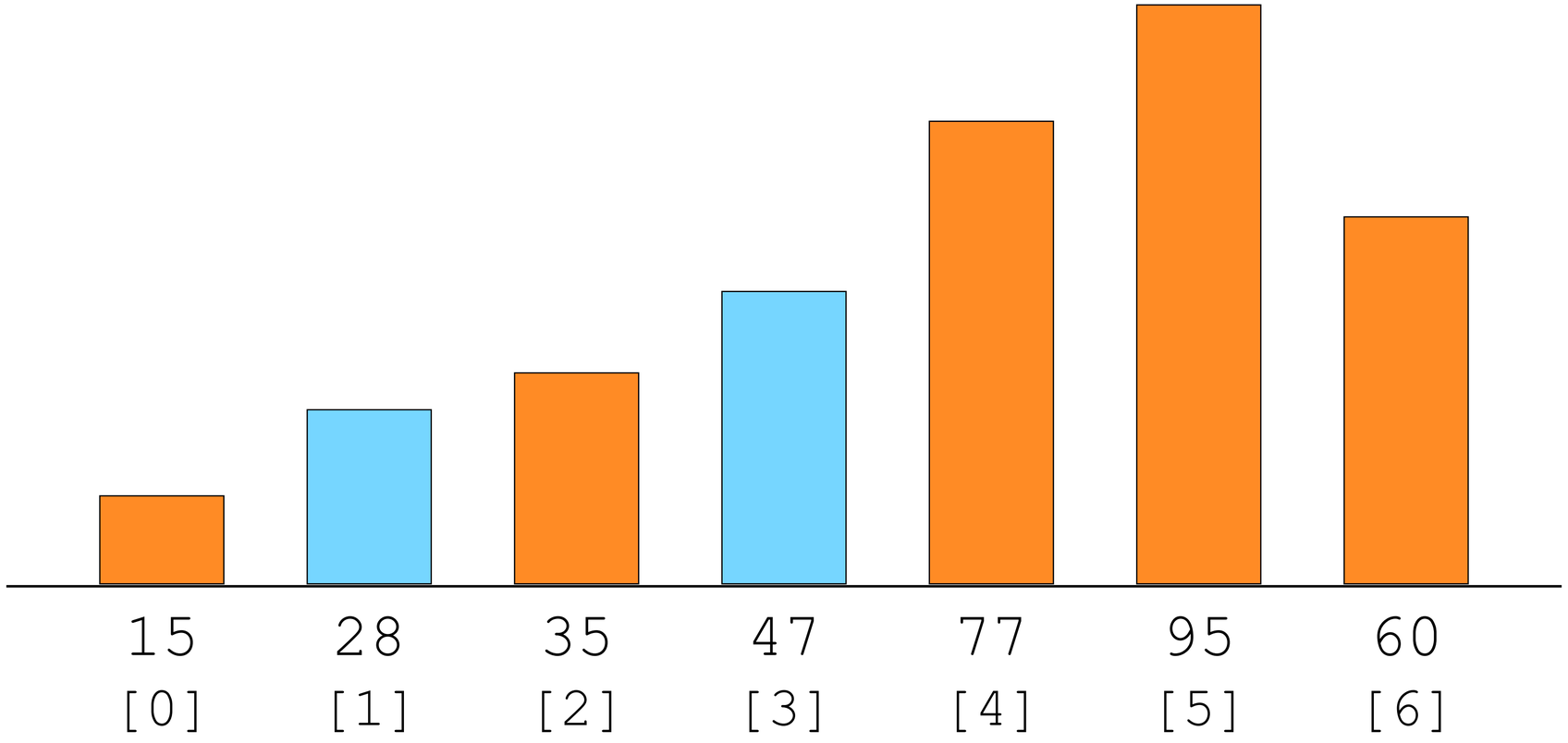


快速排序

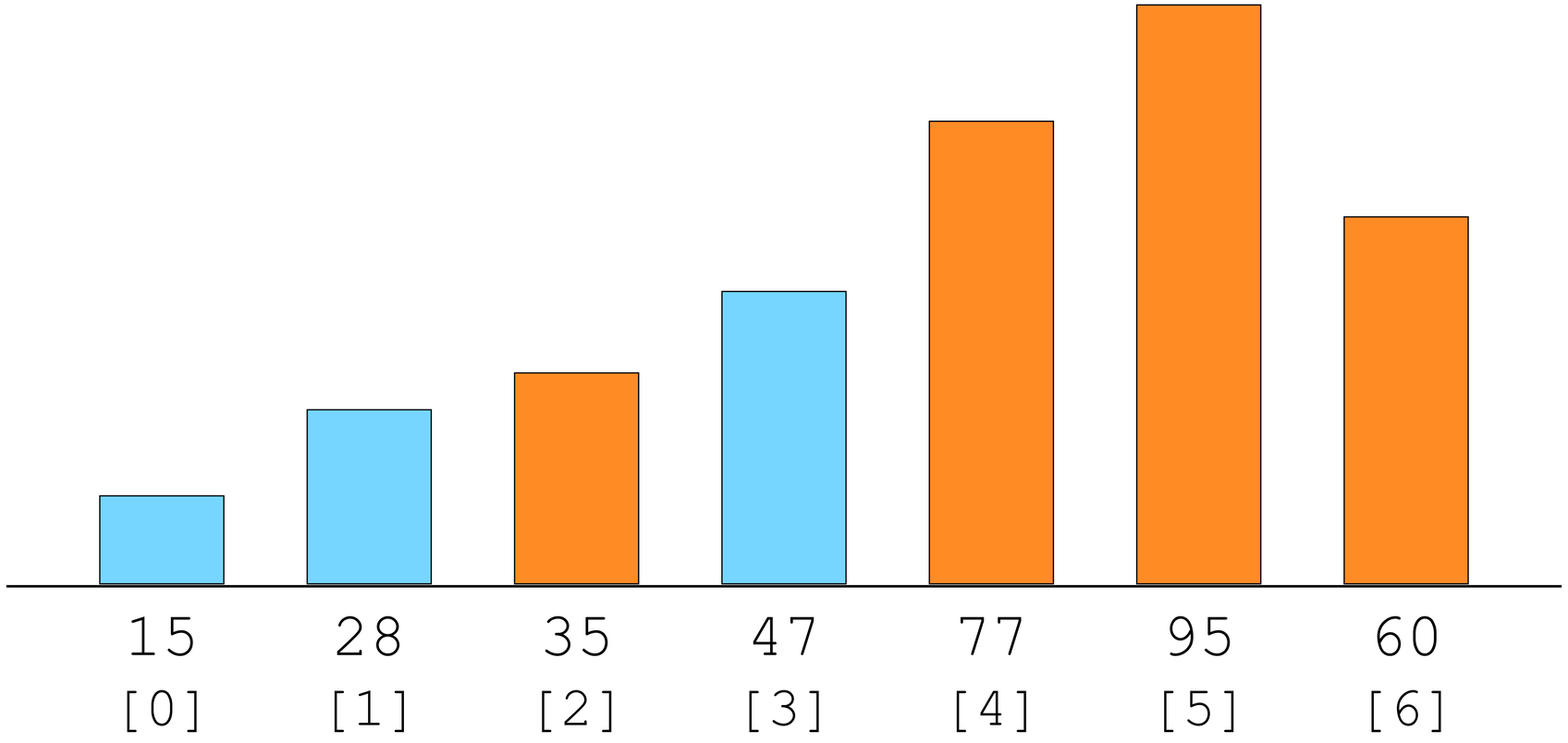
data[high] > pivot ?



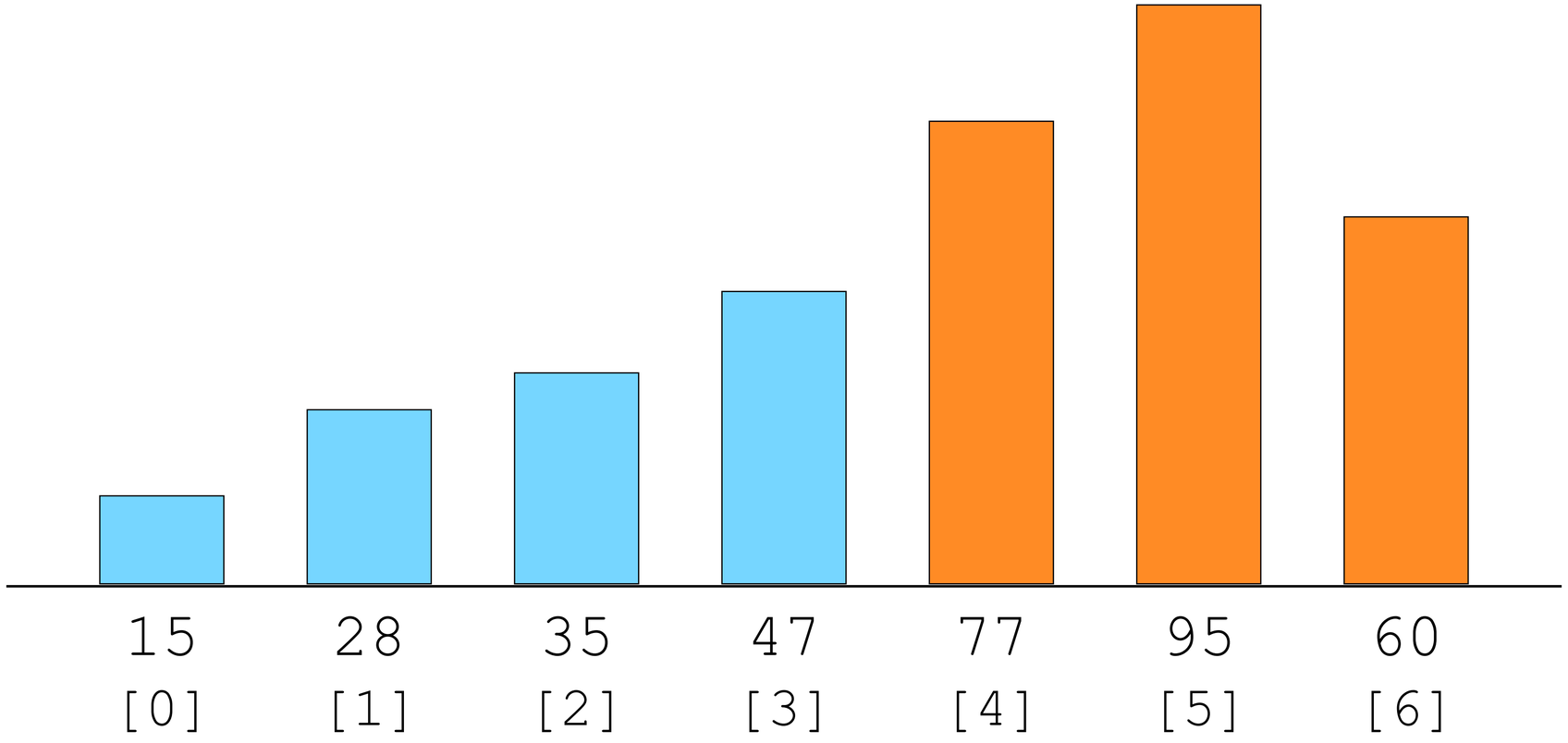
快速排序



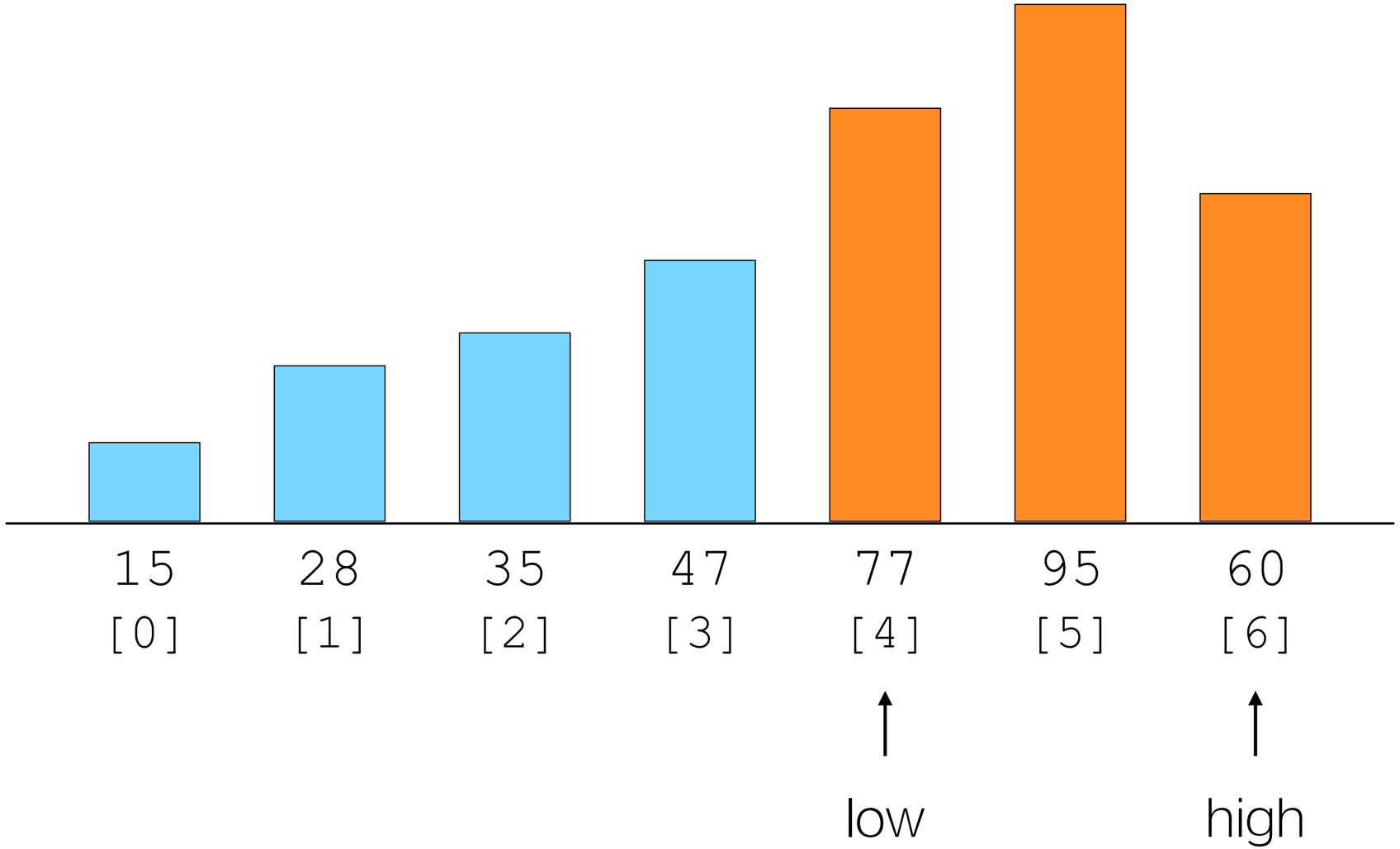
快速排序



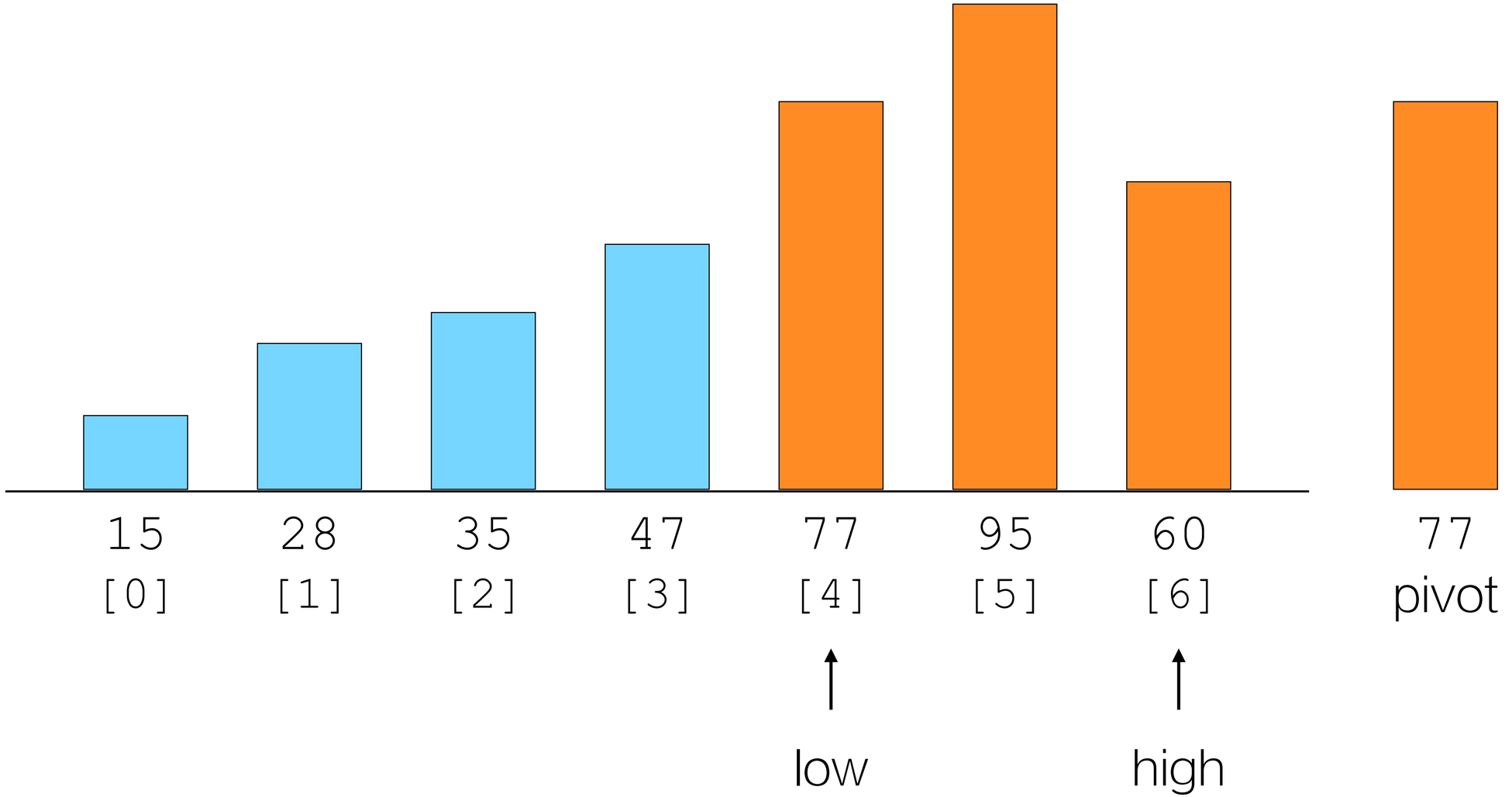
快速排序



快速排序

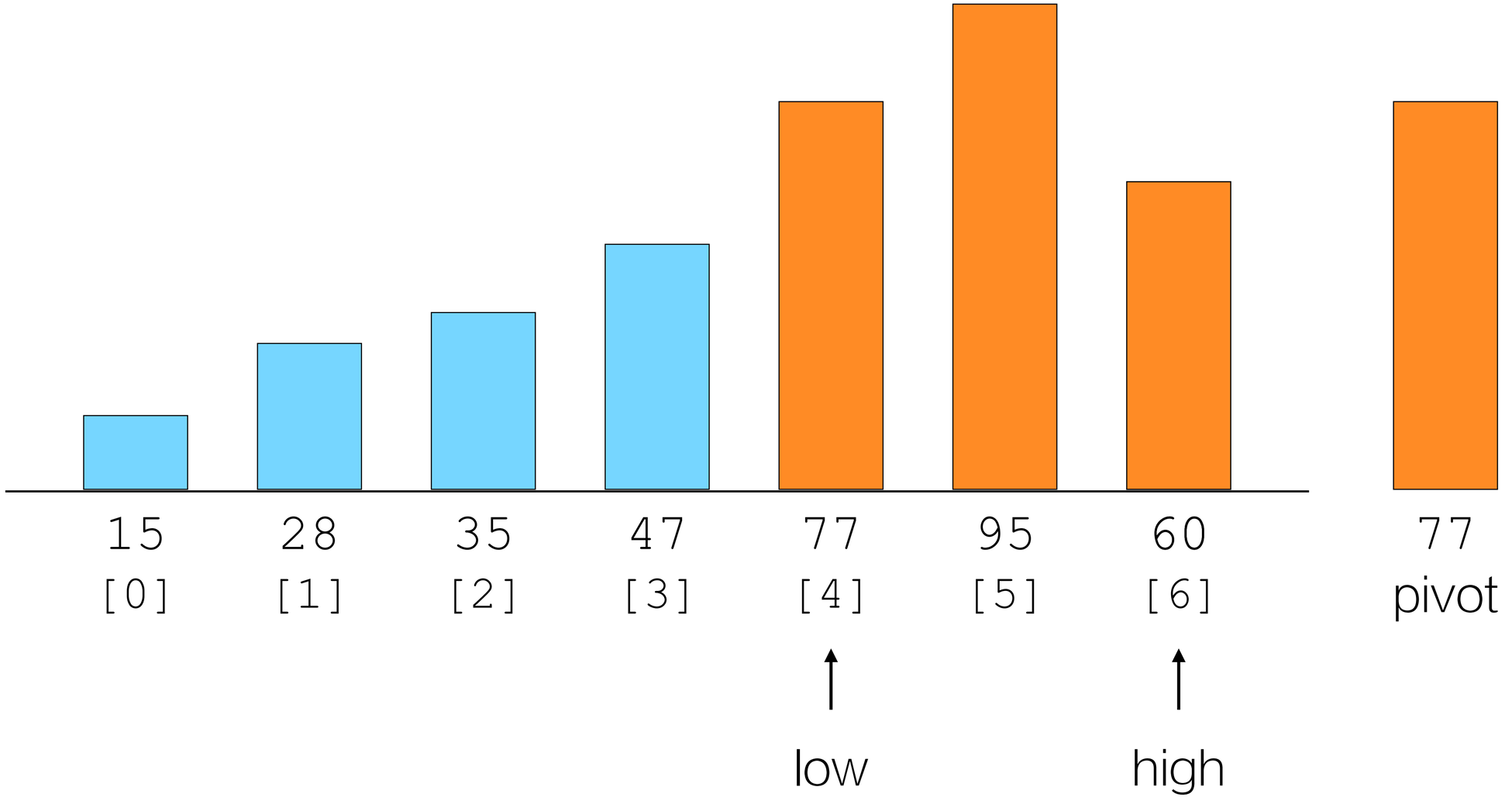


快速排序

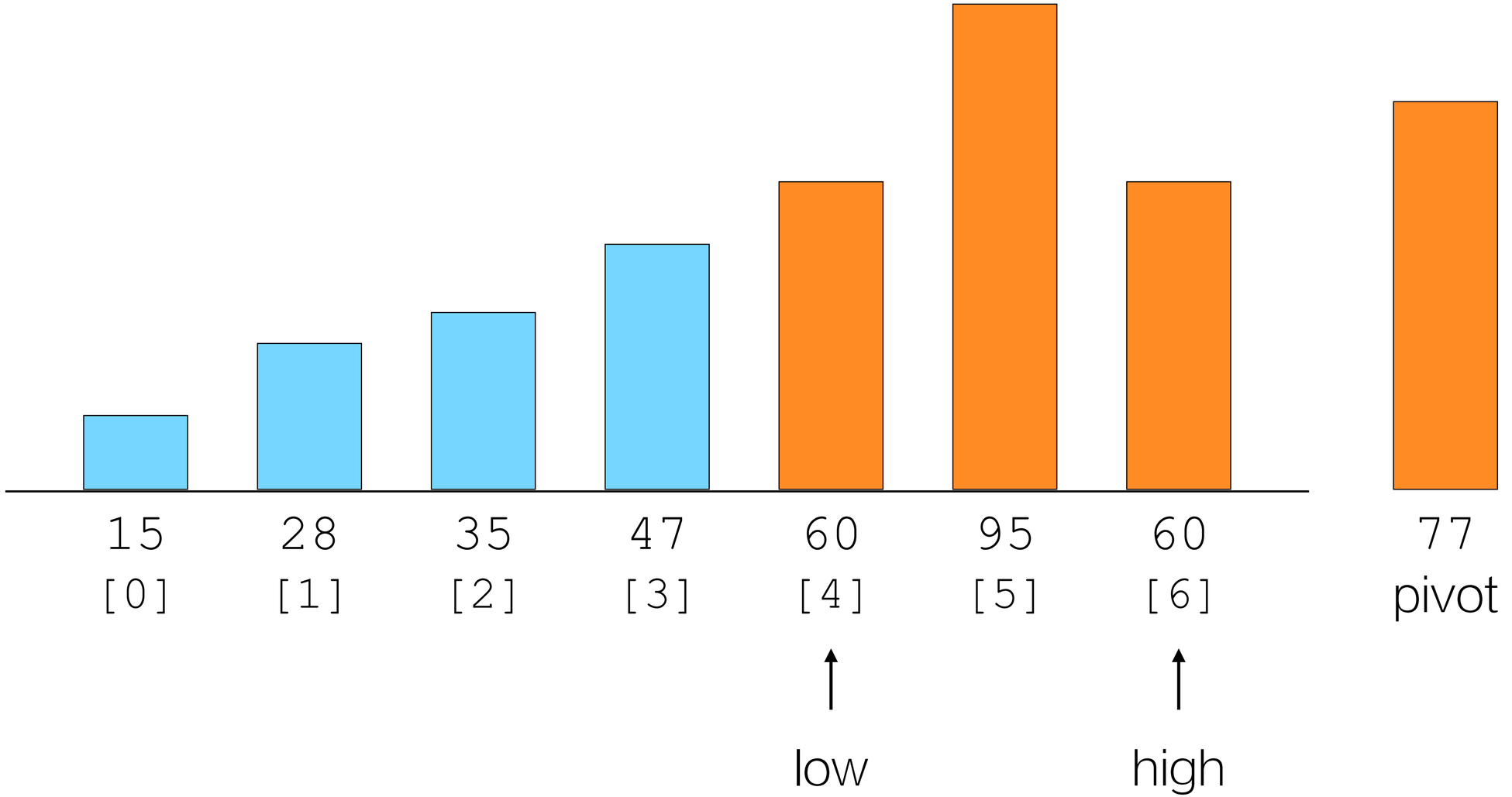


快速排序

data[high] > pivot ?

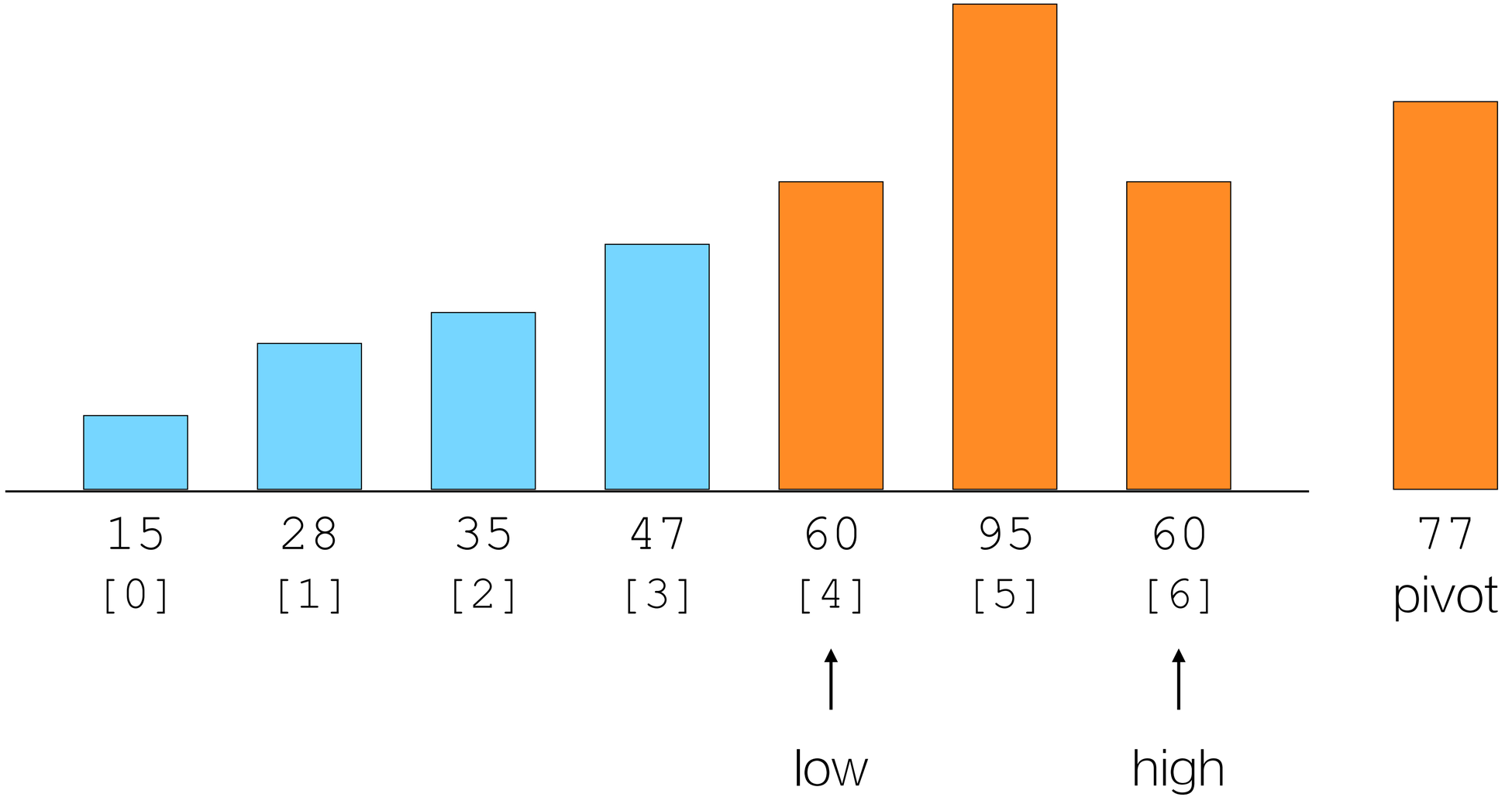


快速排序



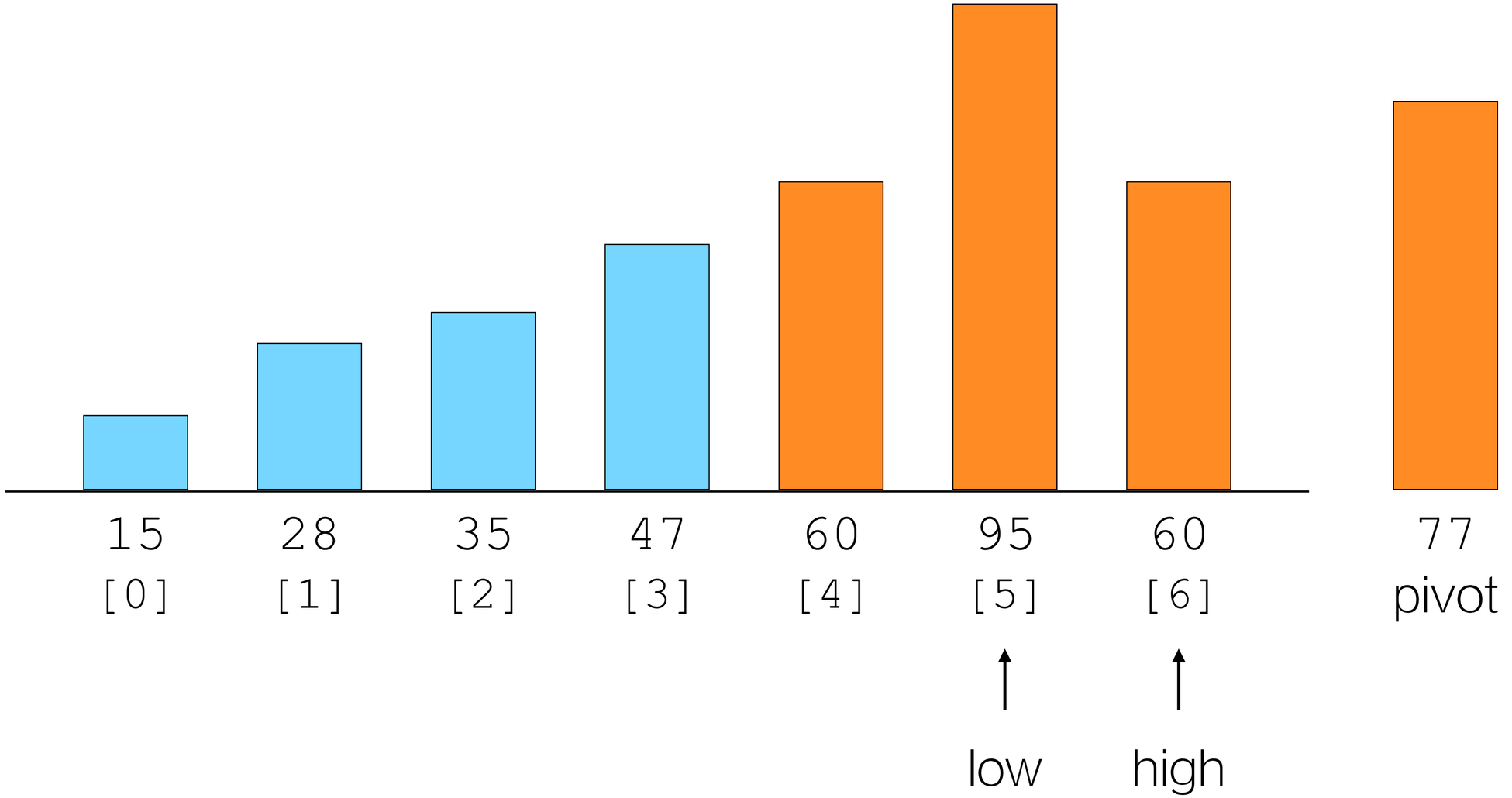
快速排序

data[low] < pivot ?

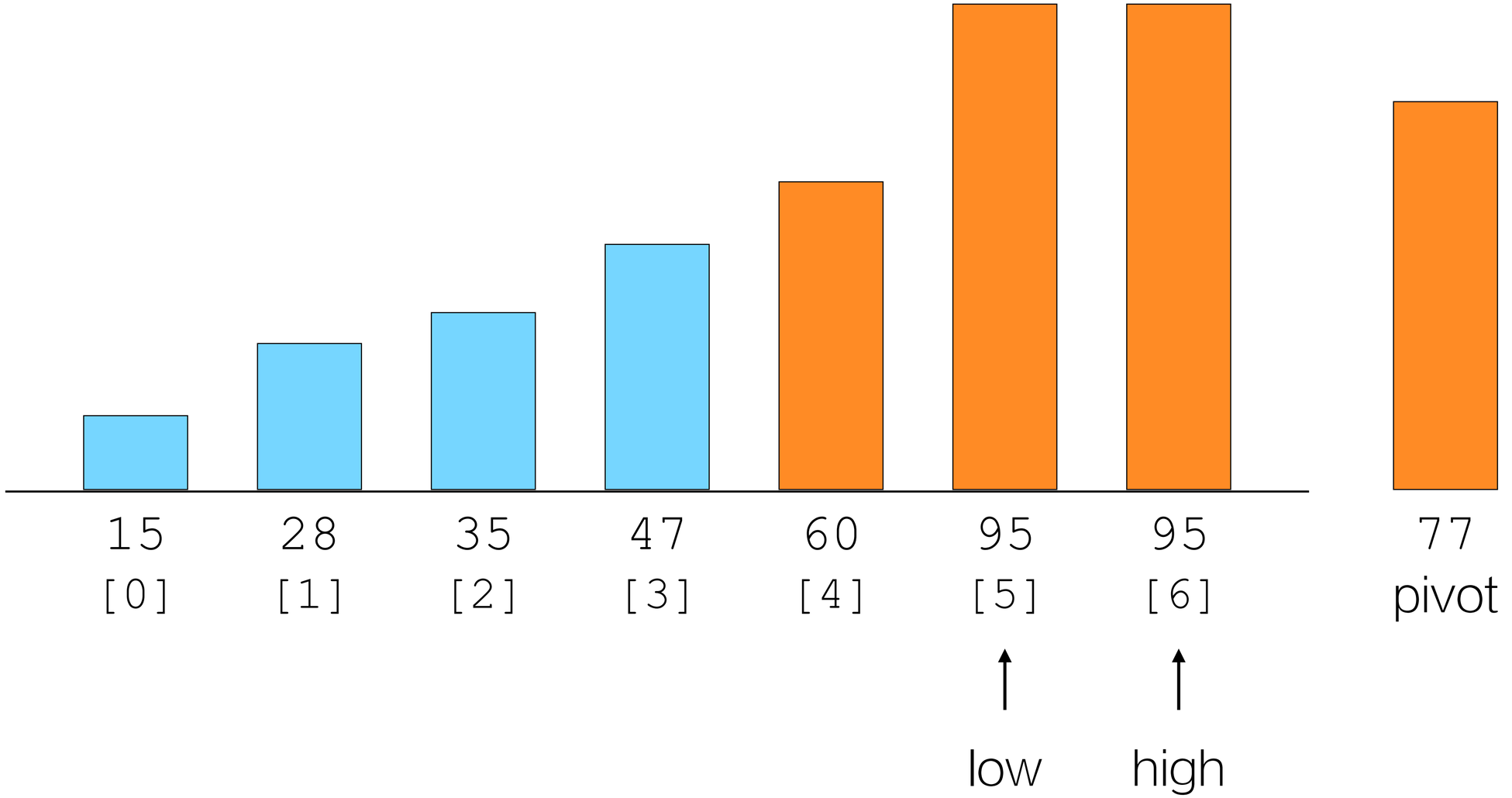


快速排序

data[low] < pivot ?

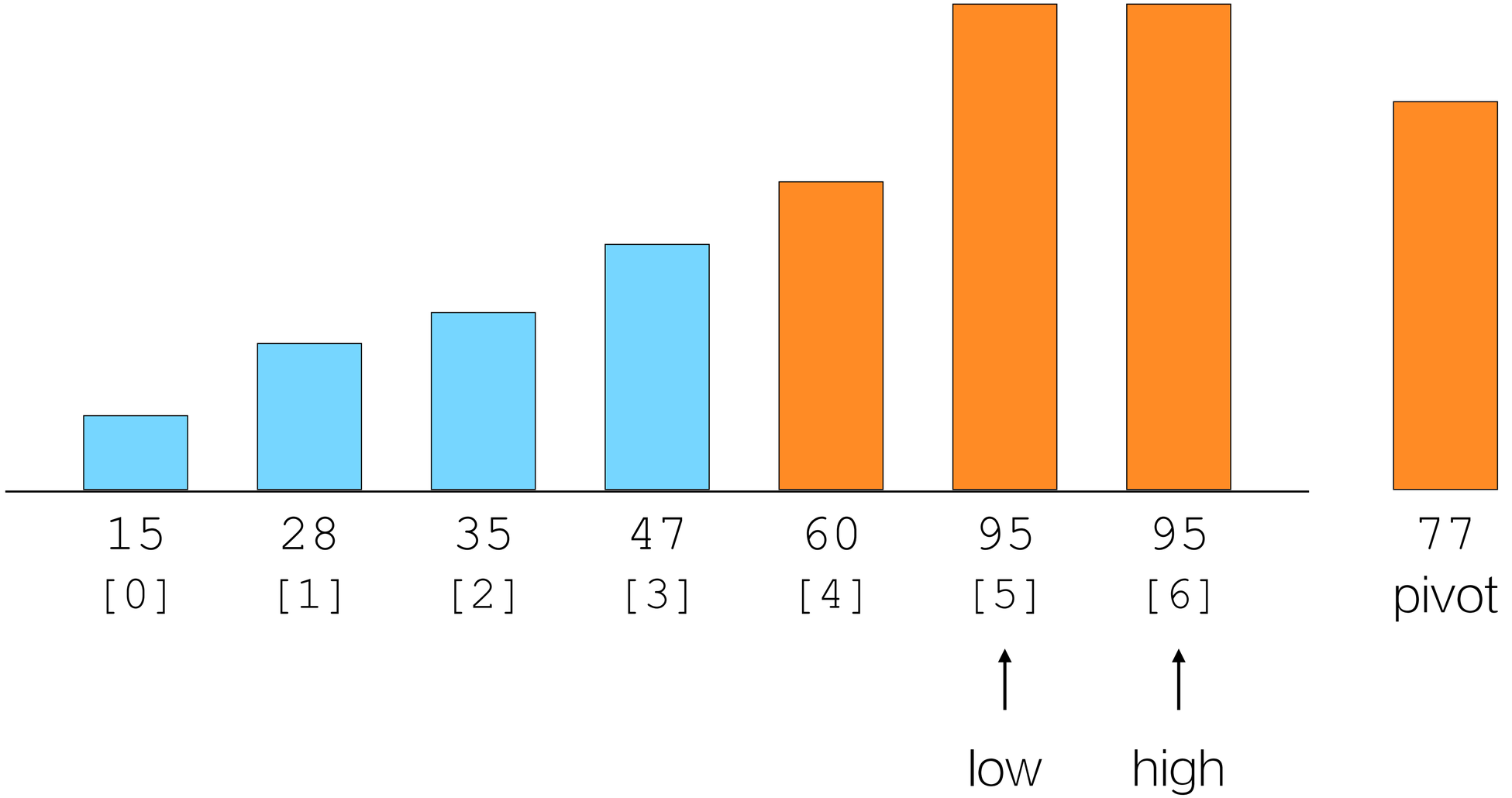


快速排序



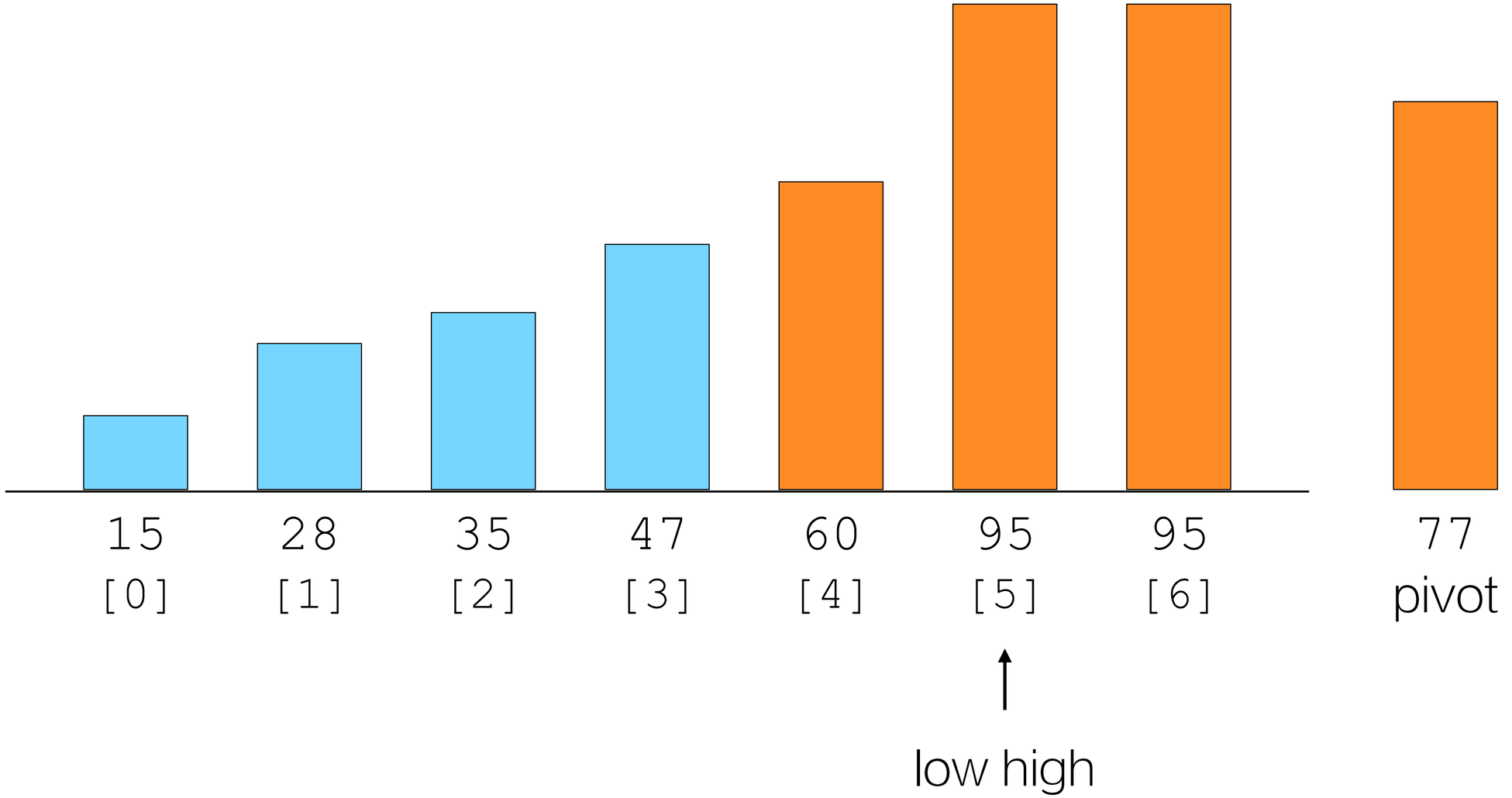
快速排序

data[high] > pivot ?

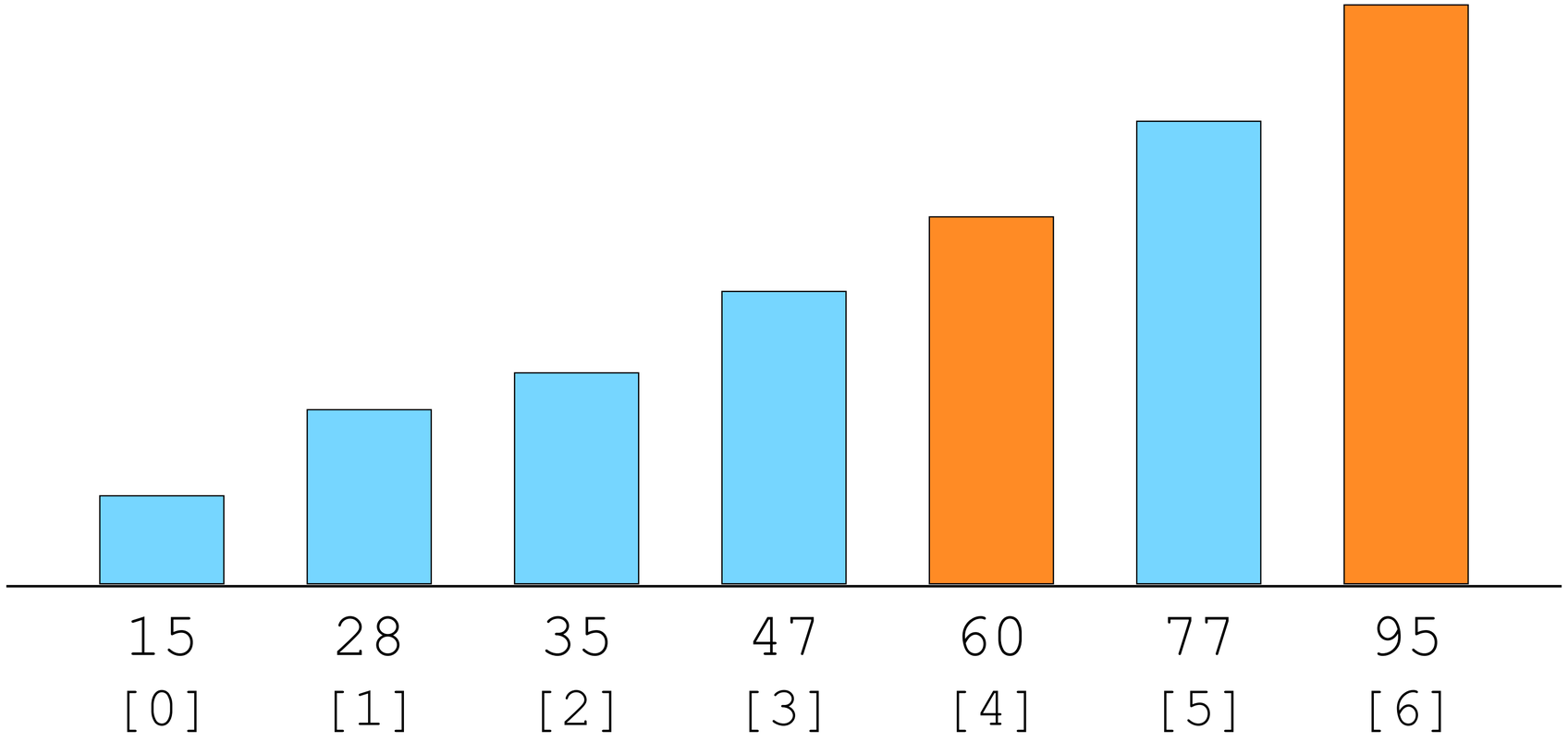


快速排序

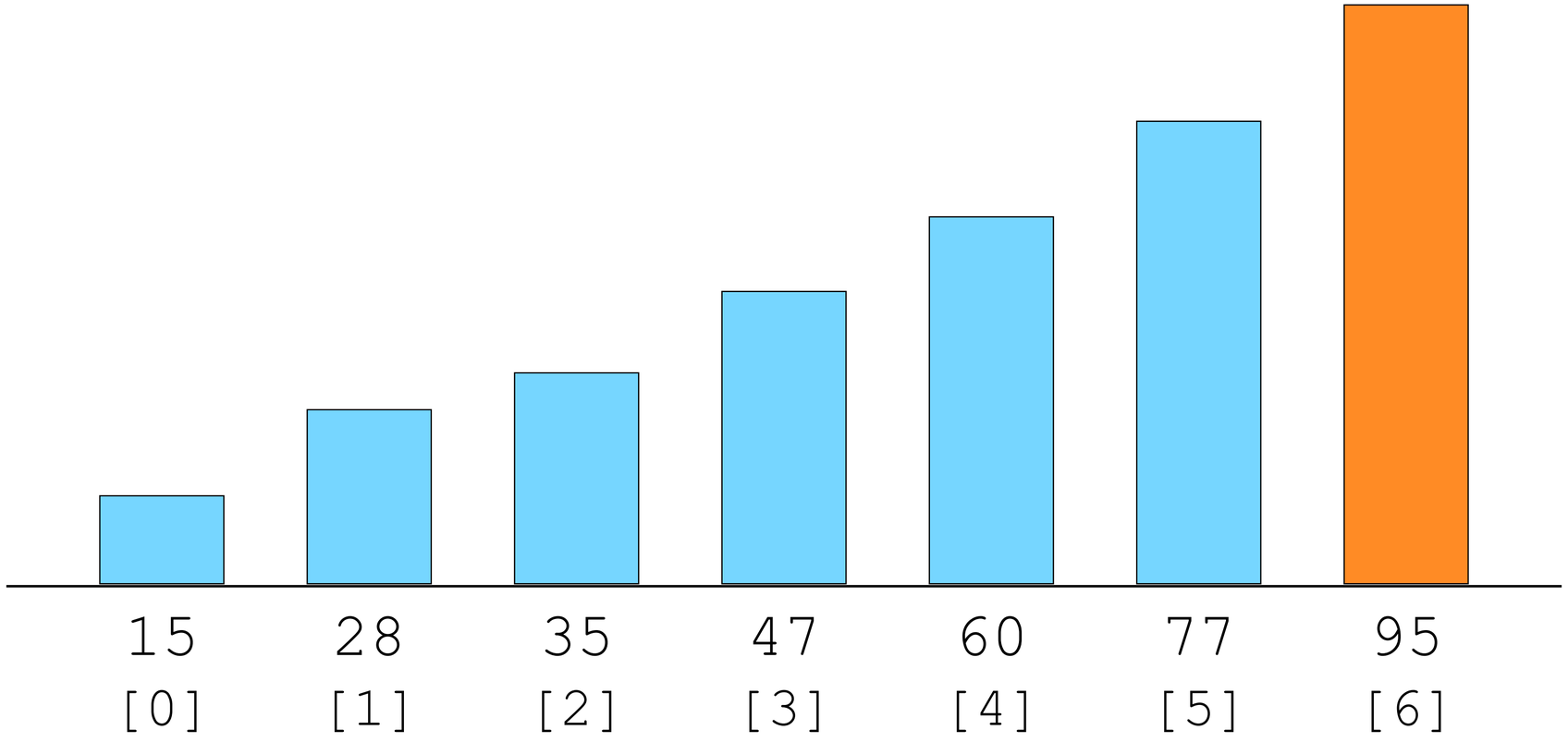
data[high] > pivot ?



快速排序

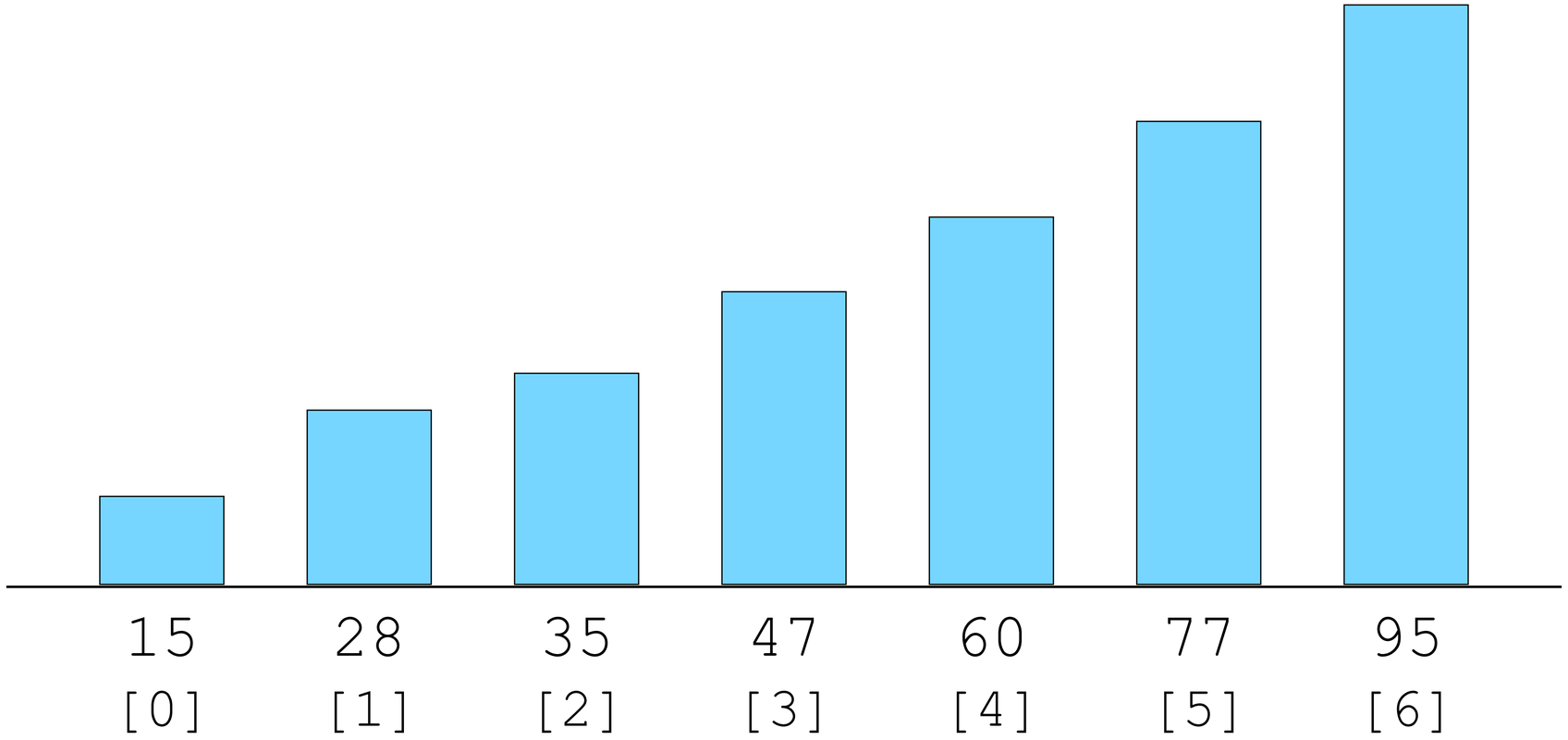


快速排序



快速排序

排序 / test8.c



快速排序

维度	描述
时间复杂度	$O(n^2)$
空间复杂度	O(log n)（递归栈空间）
算法稳定性	不稳定
适用场景	大多数场景通用、时间效率优先、不关注稳定性的场景

基数排序

基数排序是一种“分配式”的排序方法，适用于整数排序，尤其在处理大批量数据时表现亮眼。它的核心思想是：把整数按“位”分配到不同的桶中，再按位合并，通过多轮这样的分配和合并，最终完成排序。

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据个位分类	0	1	2	3	4	5	6	7	8	9

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

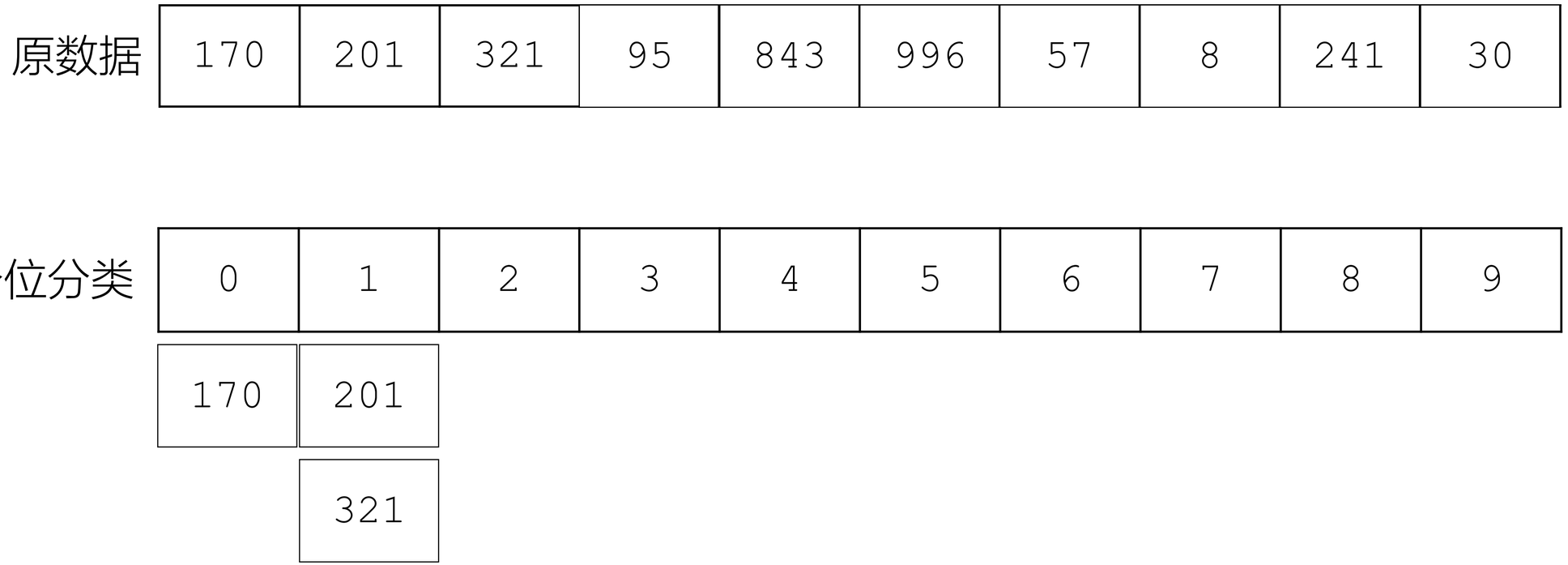
根据个位分类	0	1	2	3	4	5	6	7	8	9
	170									

基数排序

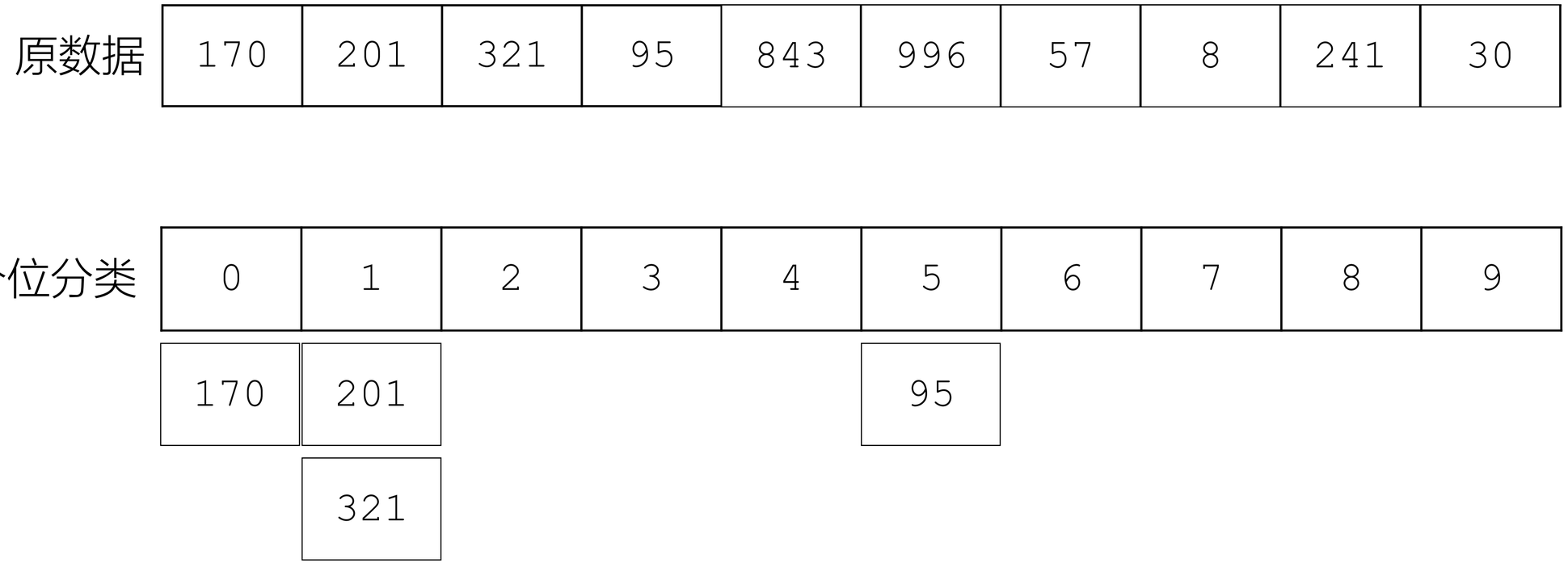
原数据	170	201	321	95	843	996	57	8	241	30

根据个位分类	0	1	2	3	4	5	6	7	8	9
	170	201								

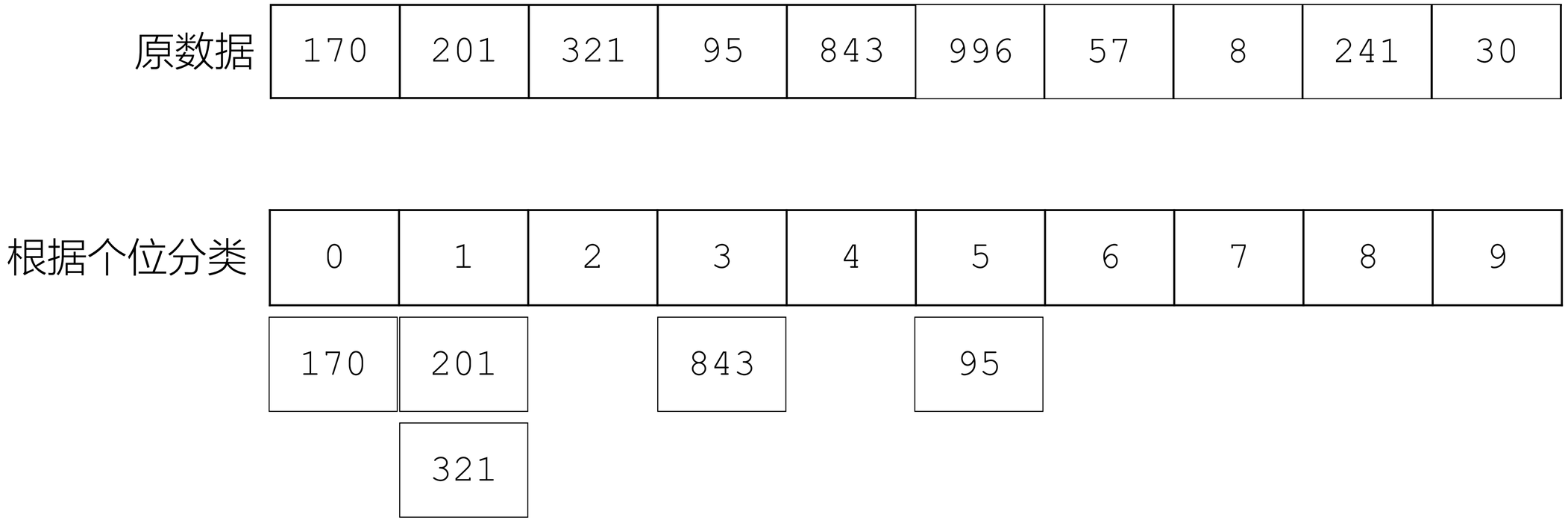
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
321

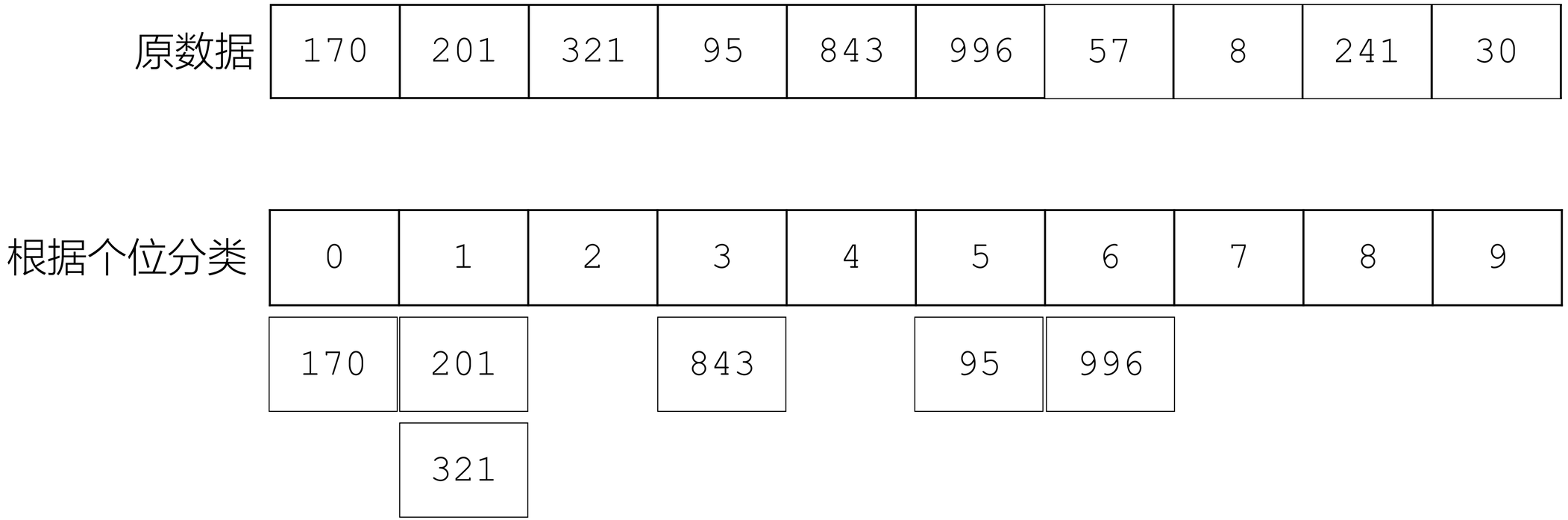
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
95
321

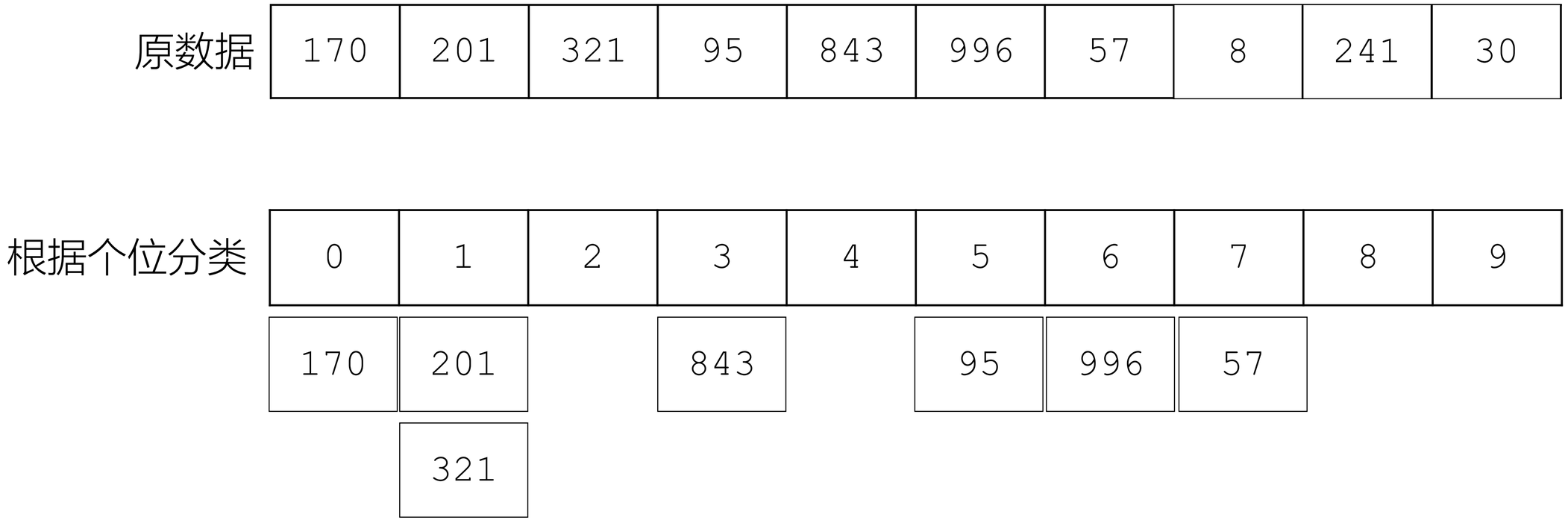
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
321

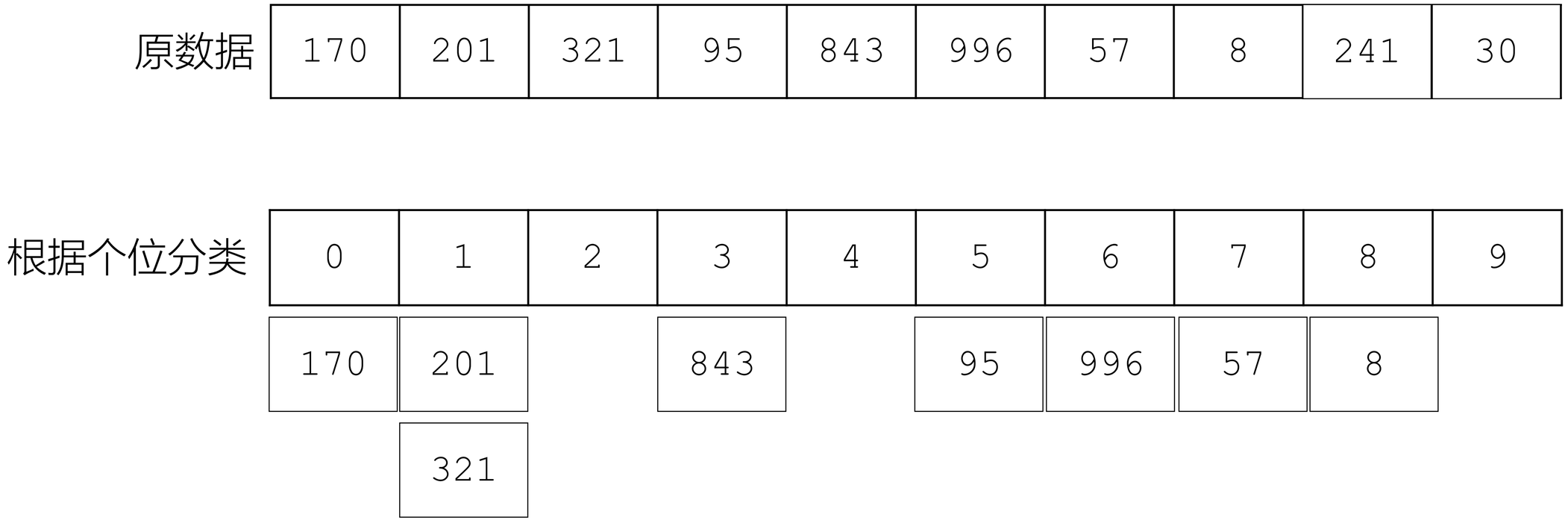
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
321

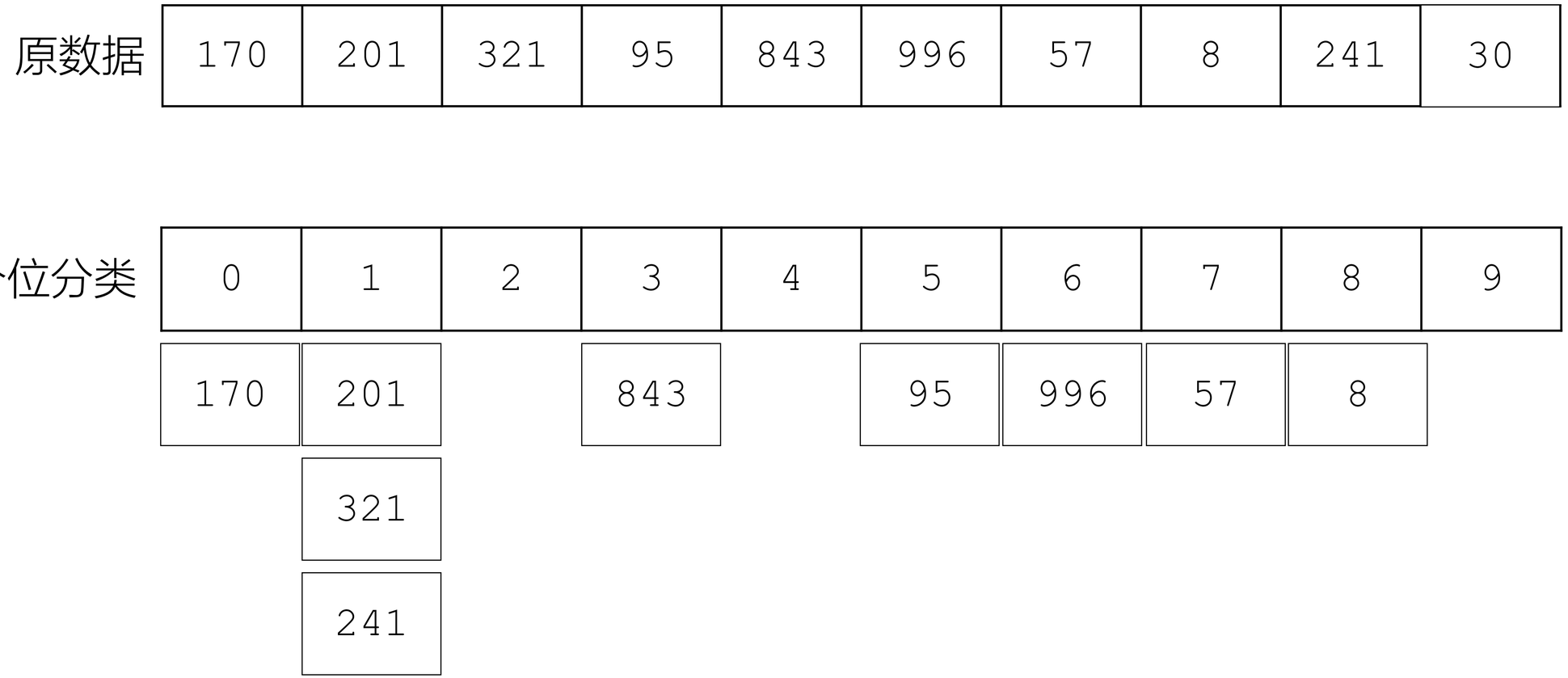
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
57
321

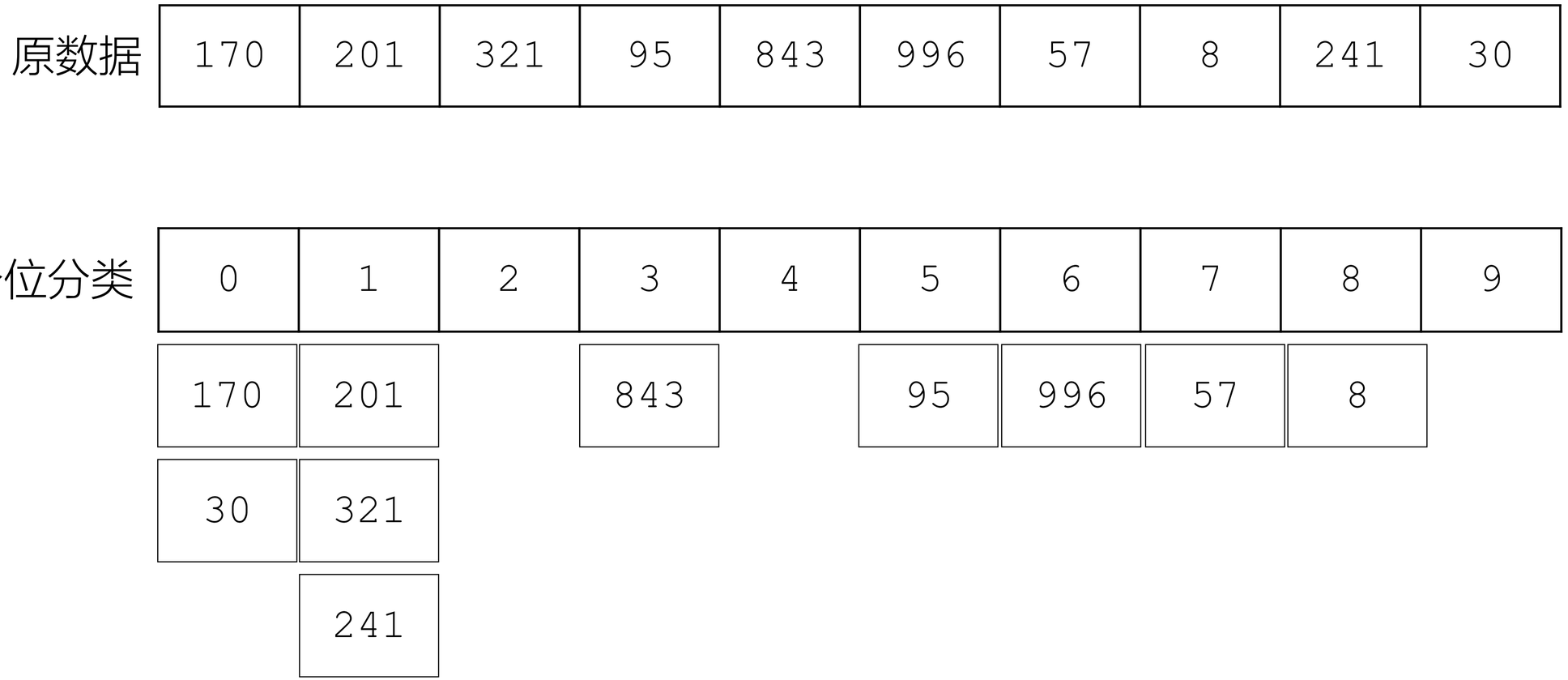
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
57
8
321

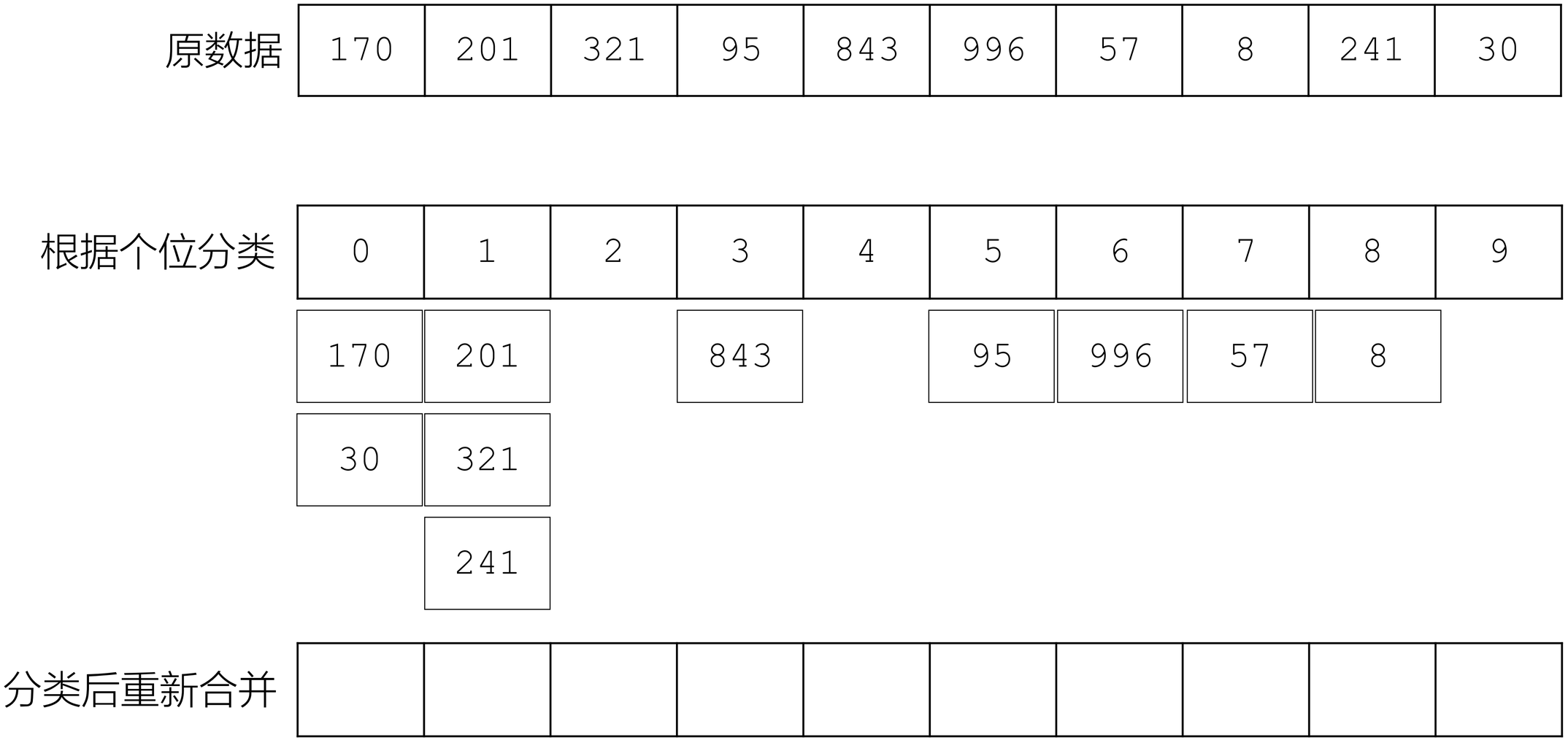
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
57
8
321
241

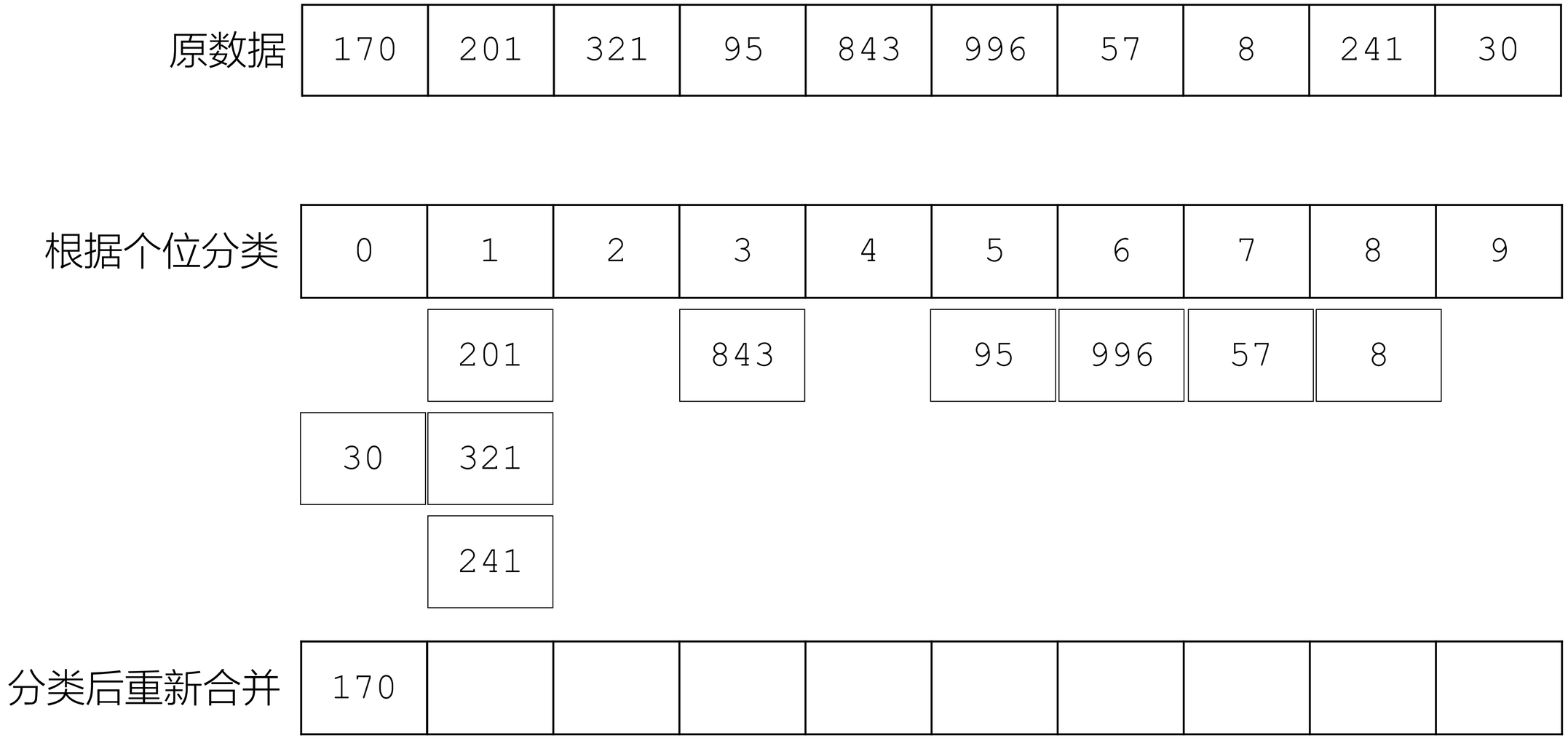
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
57
8
30
321
241

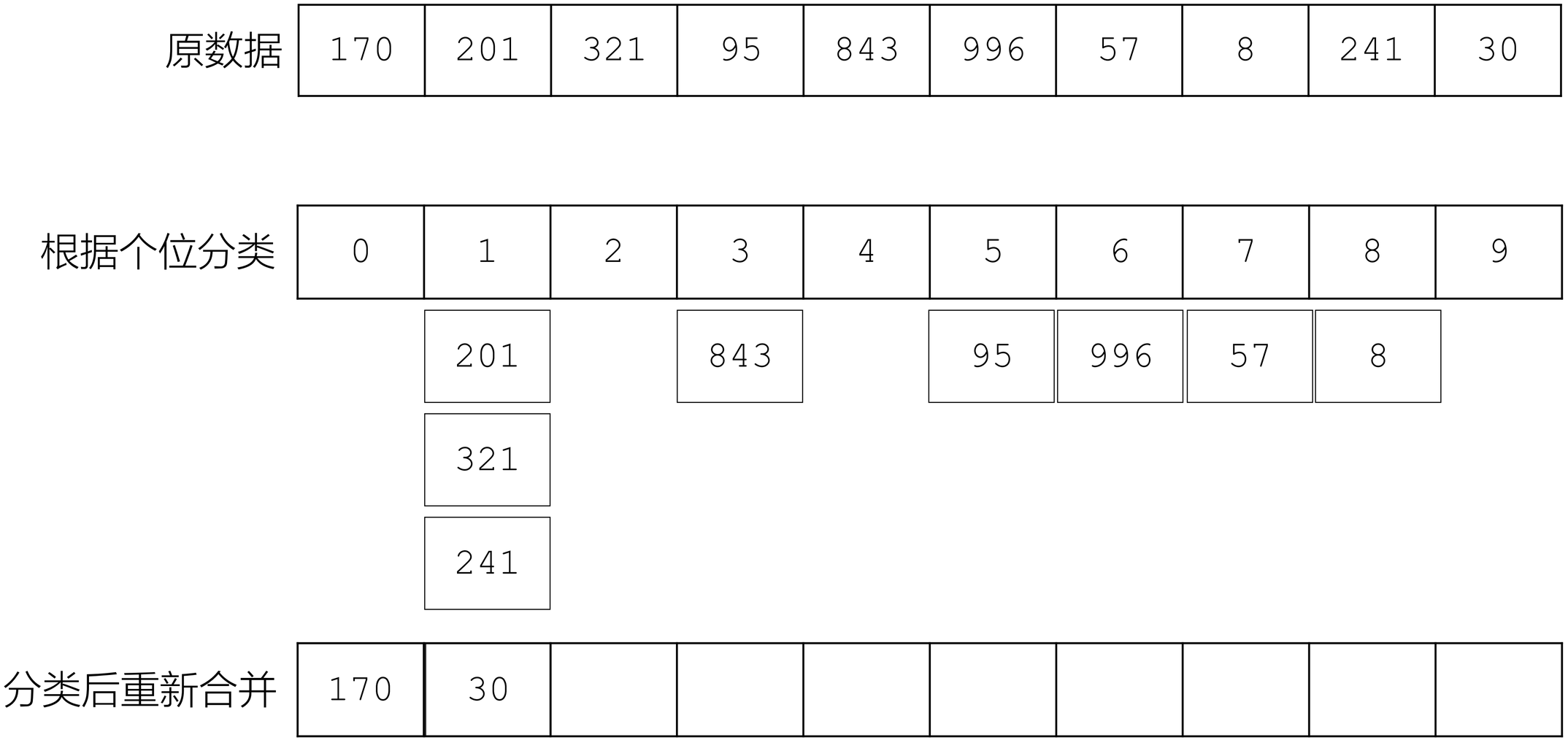
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
170
201
843
95
996
57
8
30
321
241
分类后重新合并

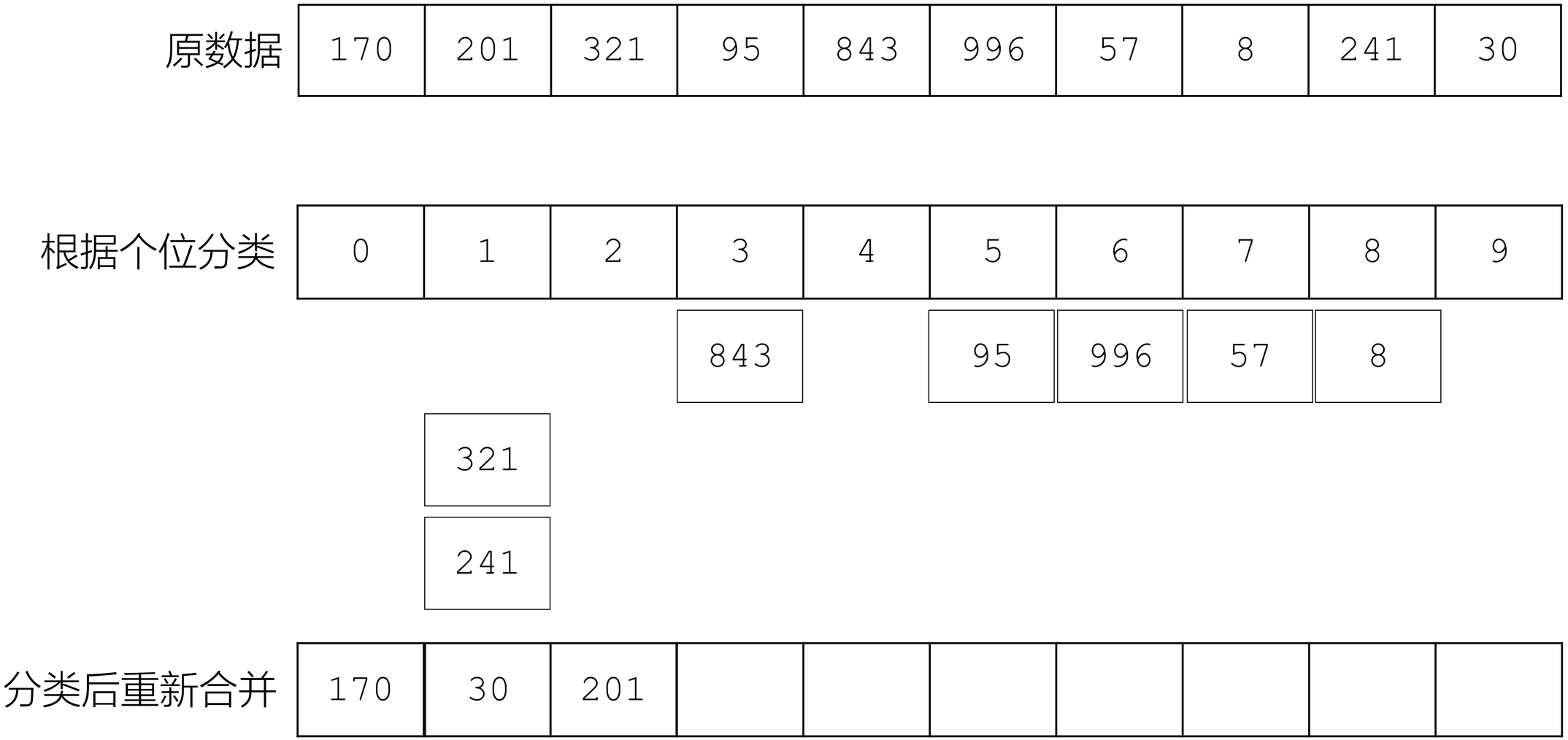
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
201
843
95
996
57
8
30
321
241
分类后重新合并
170

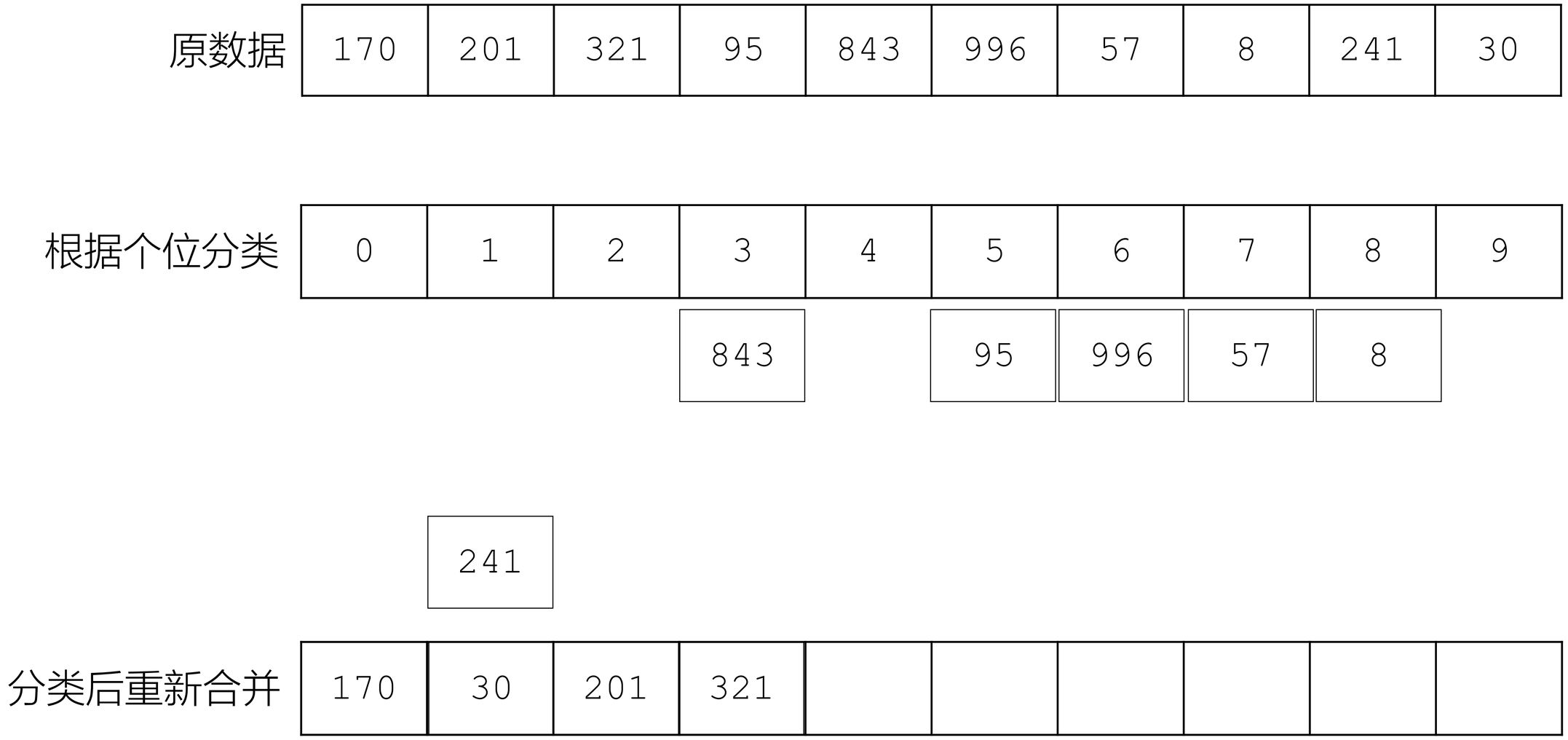
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
201
843
95
996
57
8
321
241
分类后重新合并
170
30

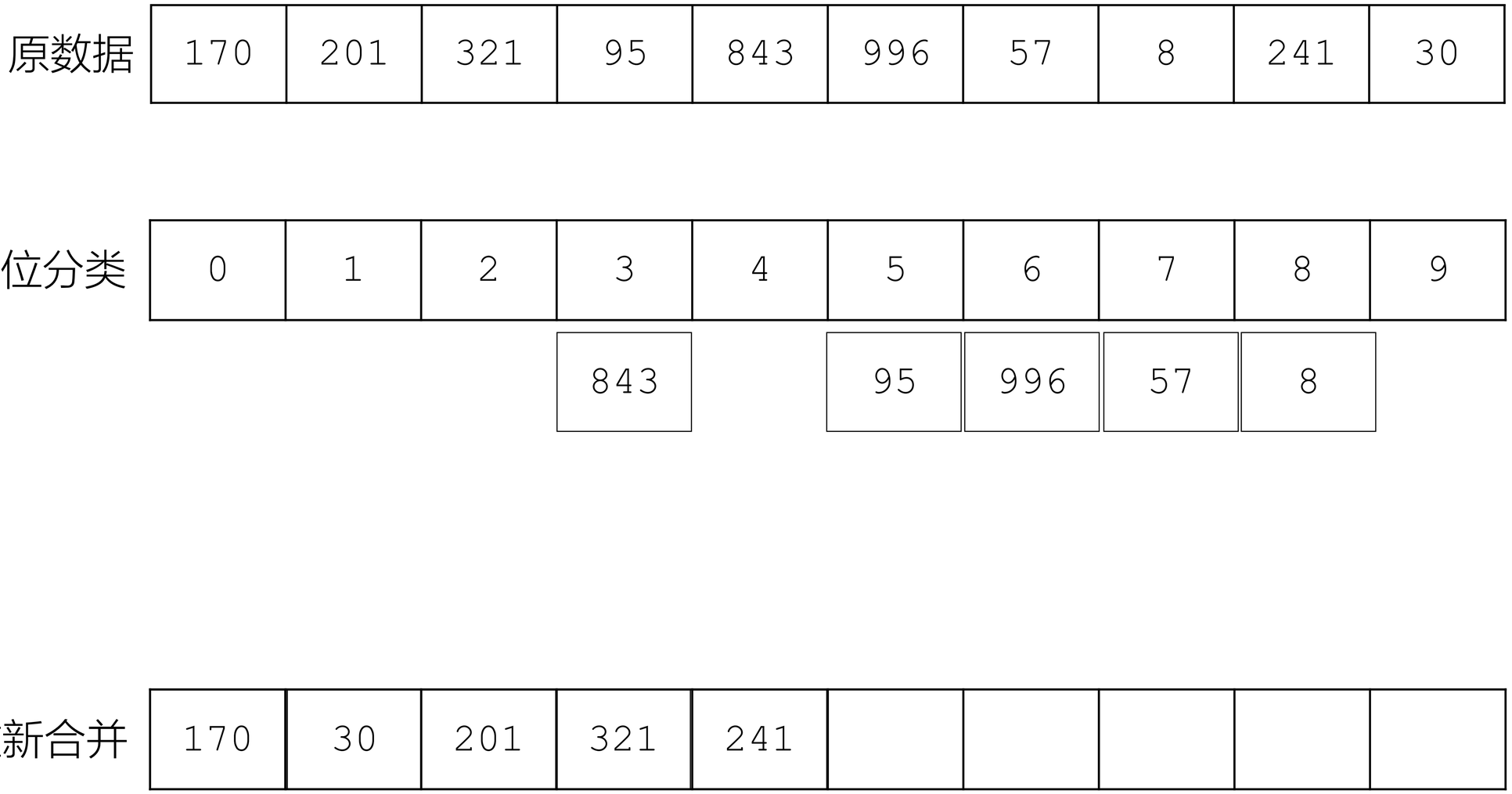
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
843
95
996
57
8
321
241
分类后重新合并
170
30
201

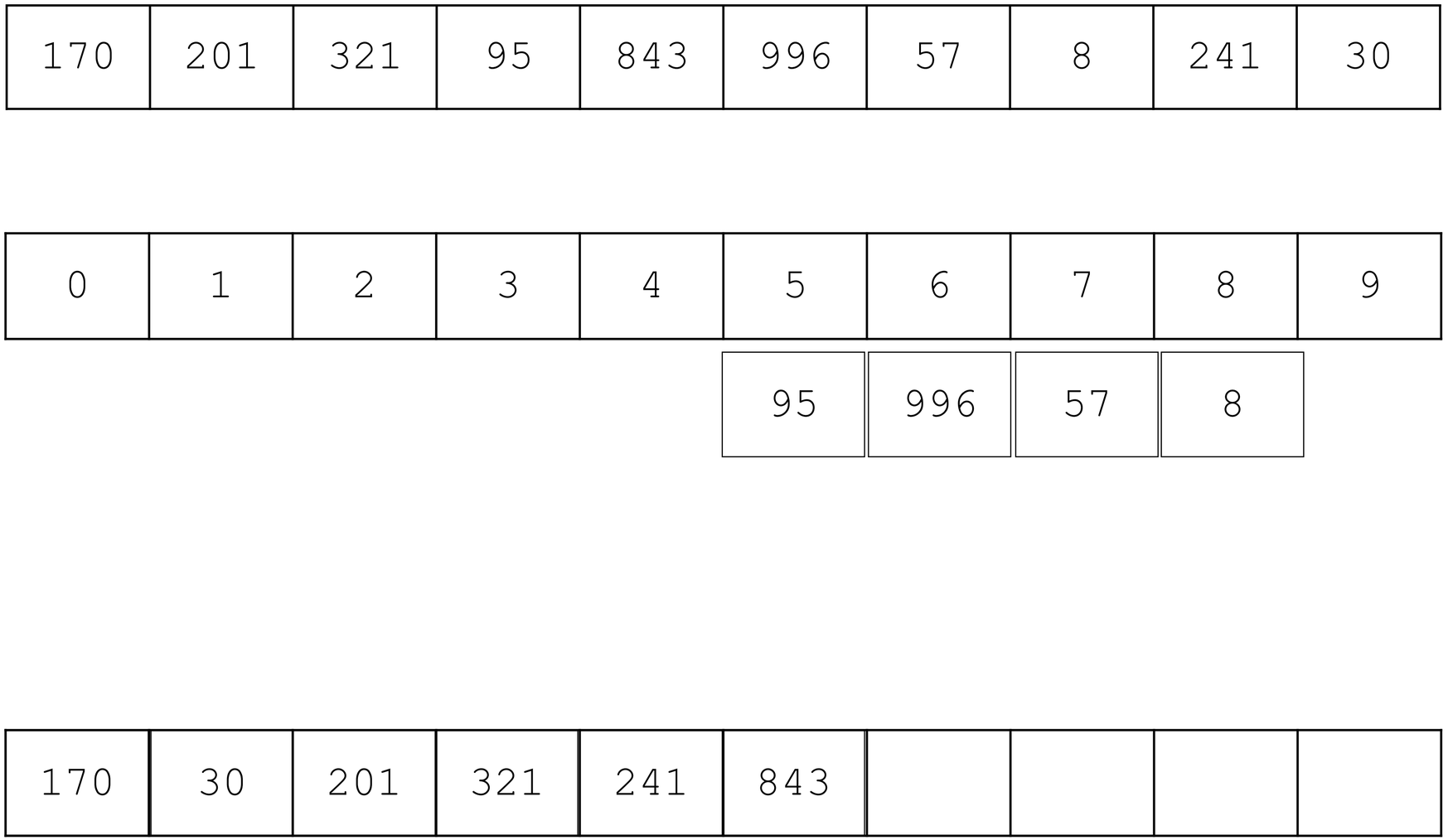
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
843
95
996
57
8
241
分类后重新合并
170
30
201
321

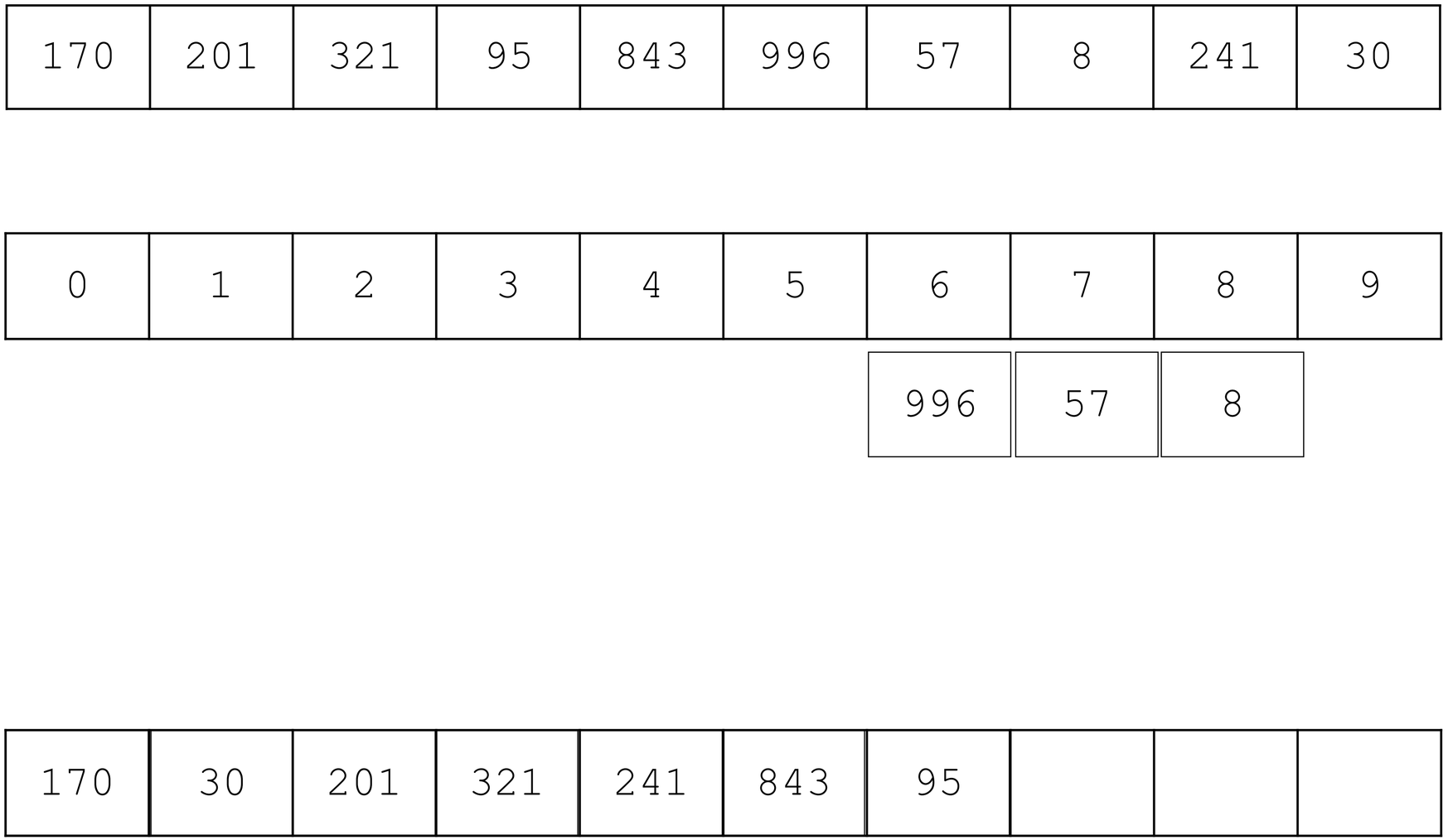
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
843
95
996
57
8
分类后重新合并
170
30
201
321
241

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
95
996
57
8
分类后重新合并
170
30
201
321
241
843

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据个位分类
0
1
2
3
4
5
6
7
8
9
996
57
8
分类后重新合并
170
30
201
321
241
843
95

基数排序

原数据

170	201	321	95	843	996	57	8	241	30

根据个位分类

0	1	2	3	4	5	6	7	8	9
							57	8	

分类后重新合并

170	30	201	321	241	843	95	996		

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据个位分类	0	1	2	3	4	5	6	7	8	9
									8	

分类后重新合并	170	30	201	321	241	843	95	996	57	

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据个位分类	0	1	2	3	4	5	6	7	8	9

分类后重新合并	170	30	201	321	241	843	95	996	57	8

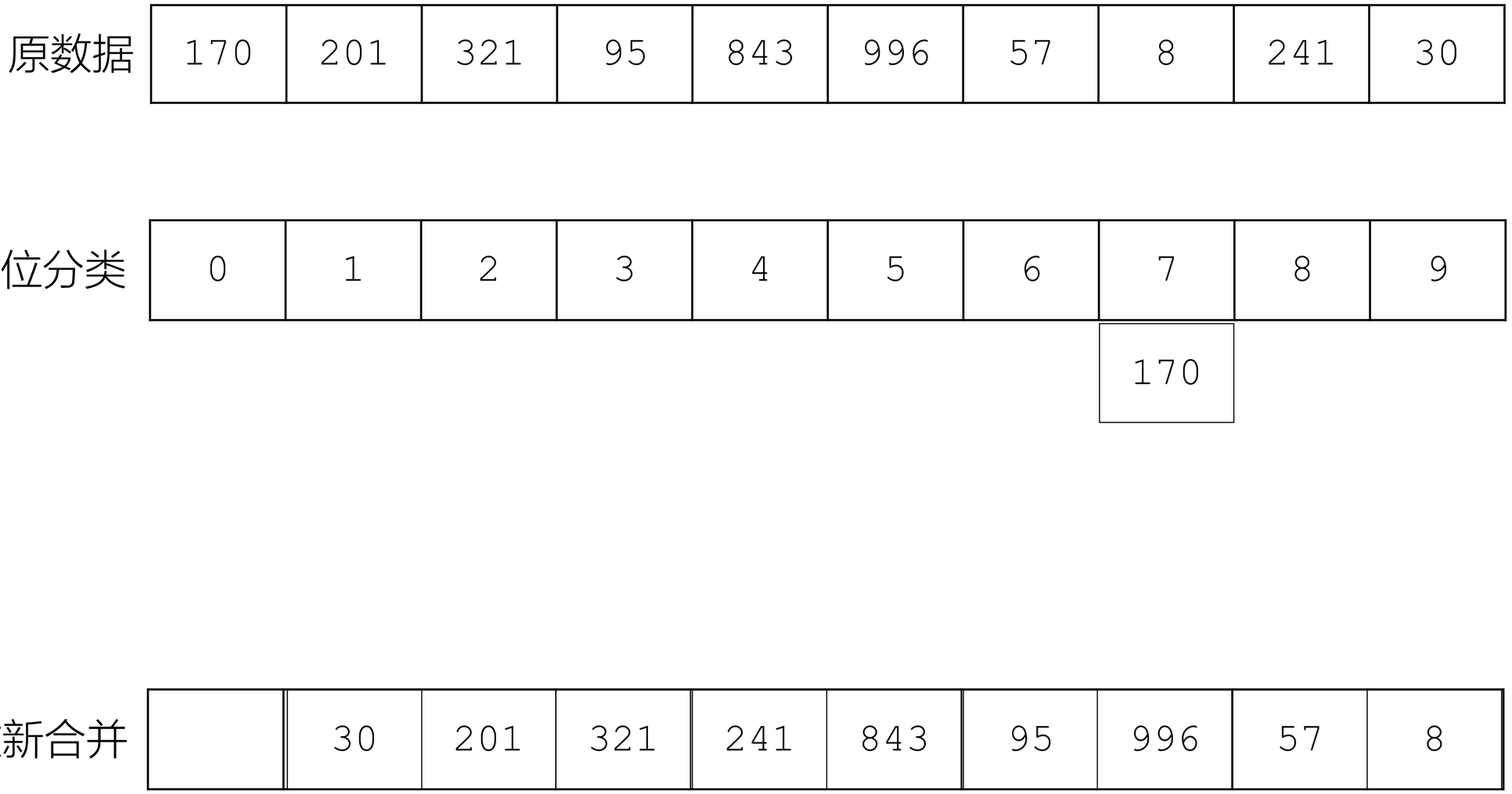
基数排序

原数据	170	201	321	95	843	996	57	8	241	30

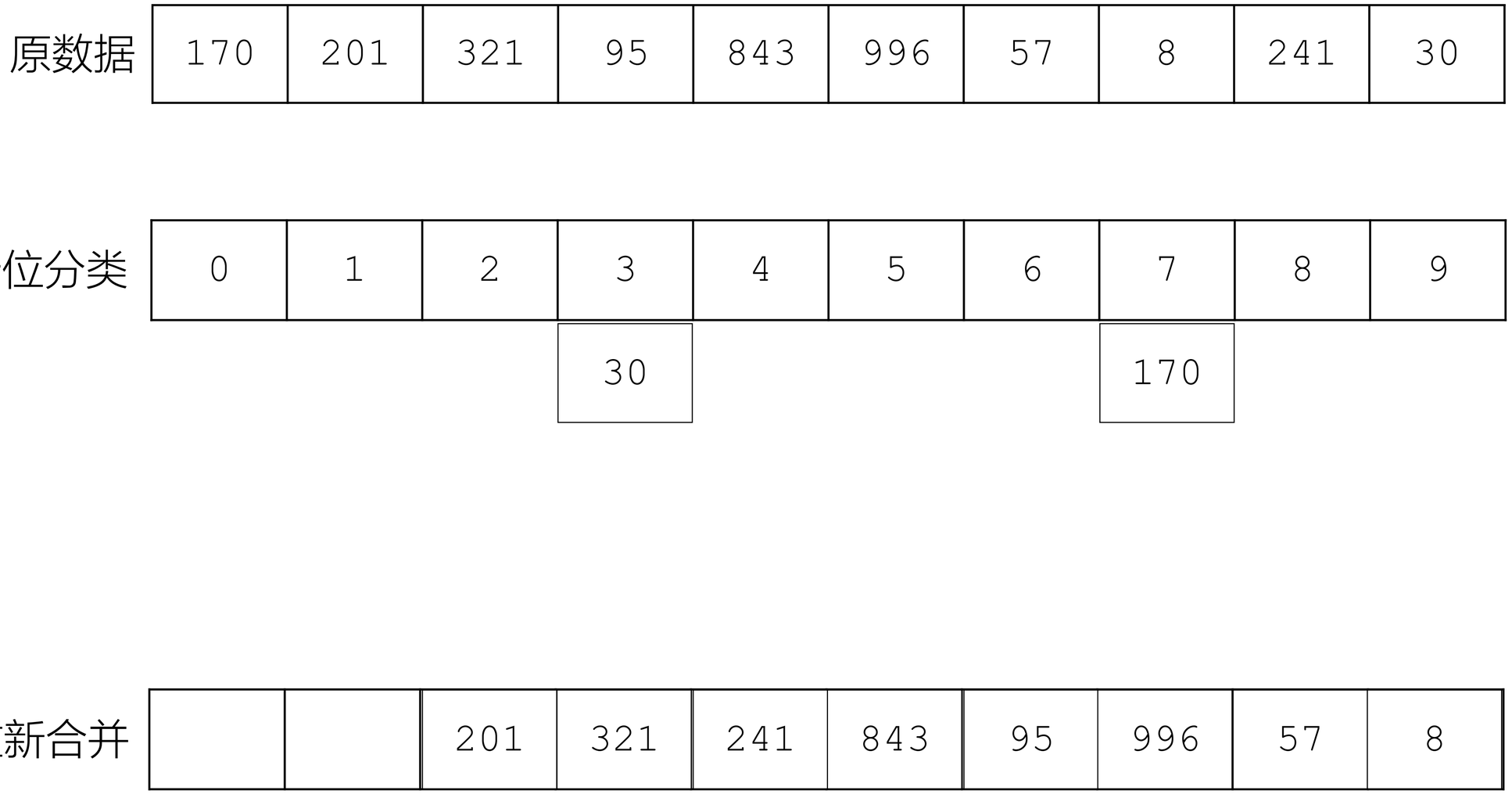
根据十位分类	0	1	2	3	4	5	6	7	8	9

分类后重新合并	170	30	201	321	241	843	95	996	57	8

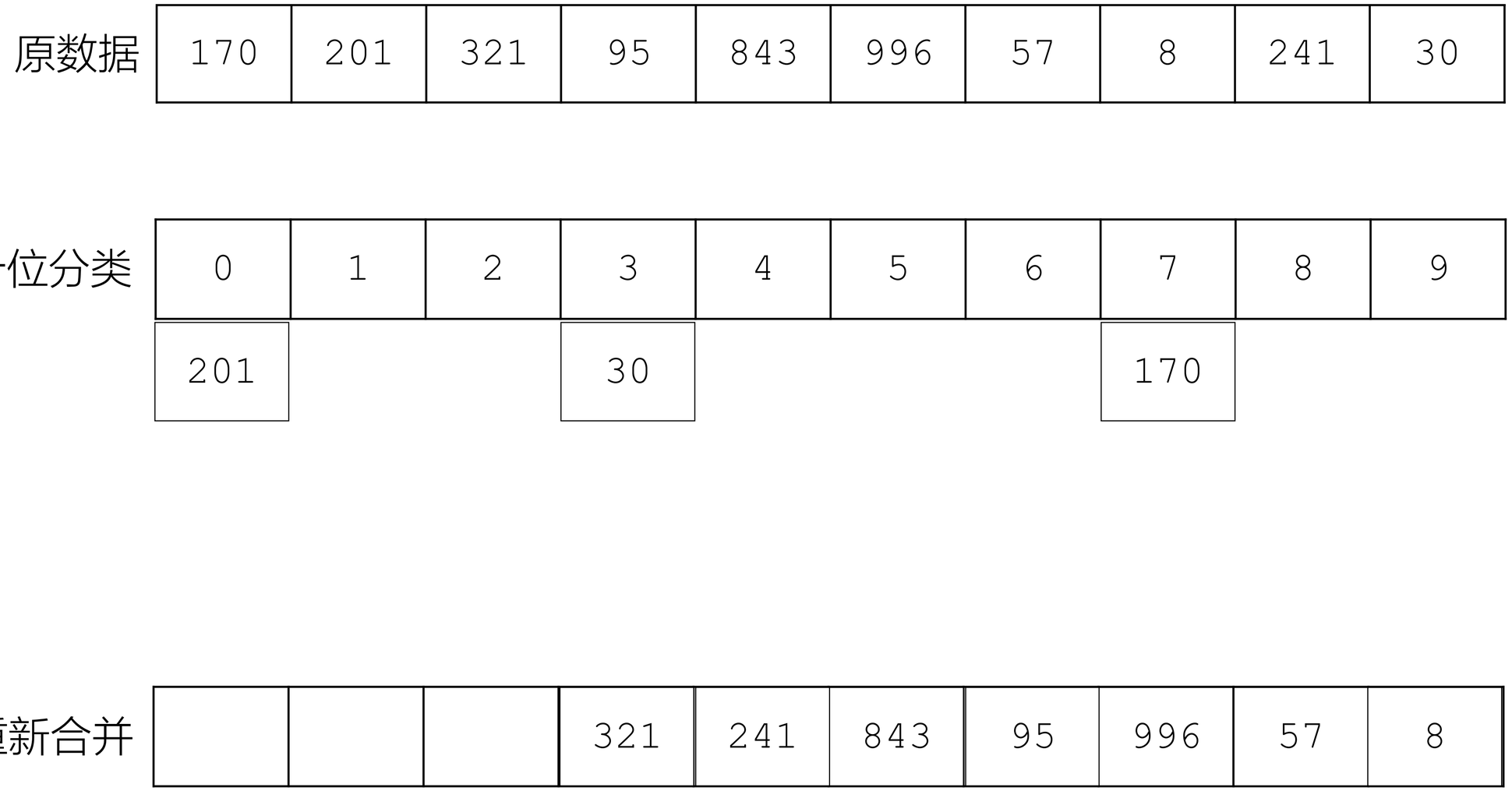
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
170
分类后重新合并
30
201
321
241
843
95
996
57
8

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
30
170
分类后重新合并
201
321
241
843
95
996
57
8

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
201
30
170
分类后重新合并
321
241
843
95
996
57
8

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据十位分类	0	1	2	3	4	5	6	7	8	9
	201		321	30				170		

分类后重新合并					241	843	95	996	57	8

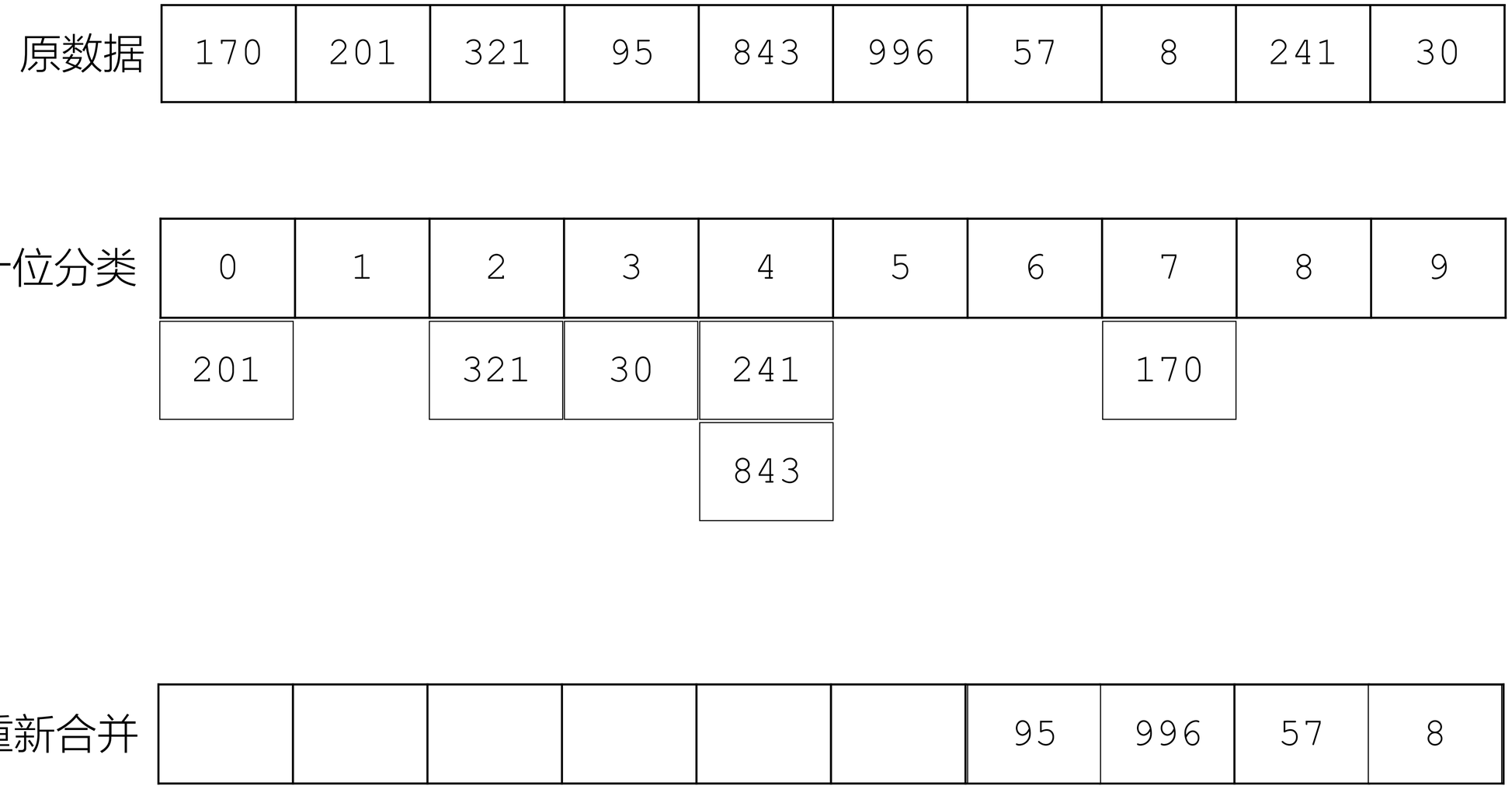
基数排序

原数据	170	201	321	95	843	996	57	8	241	30

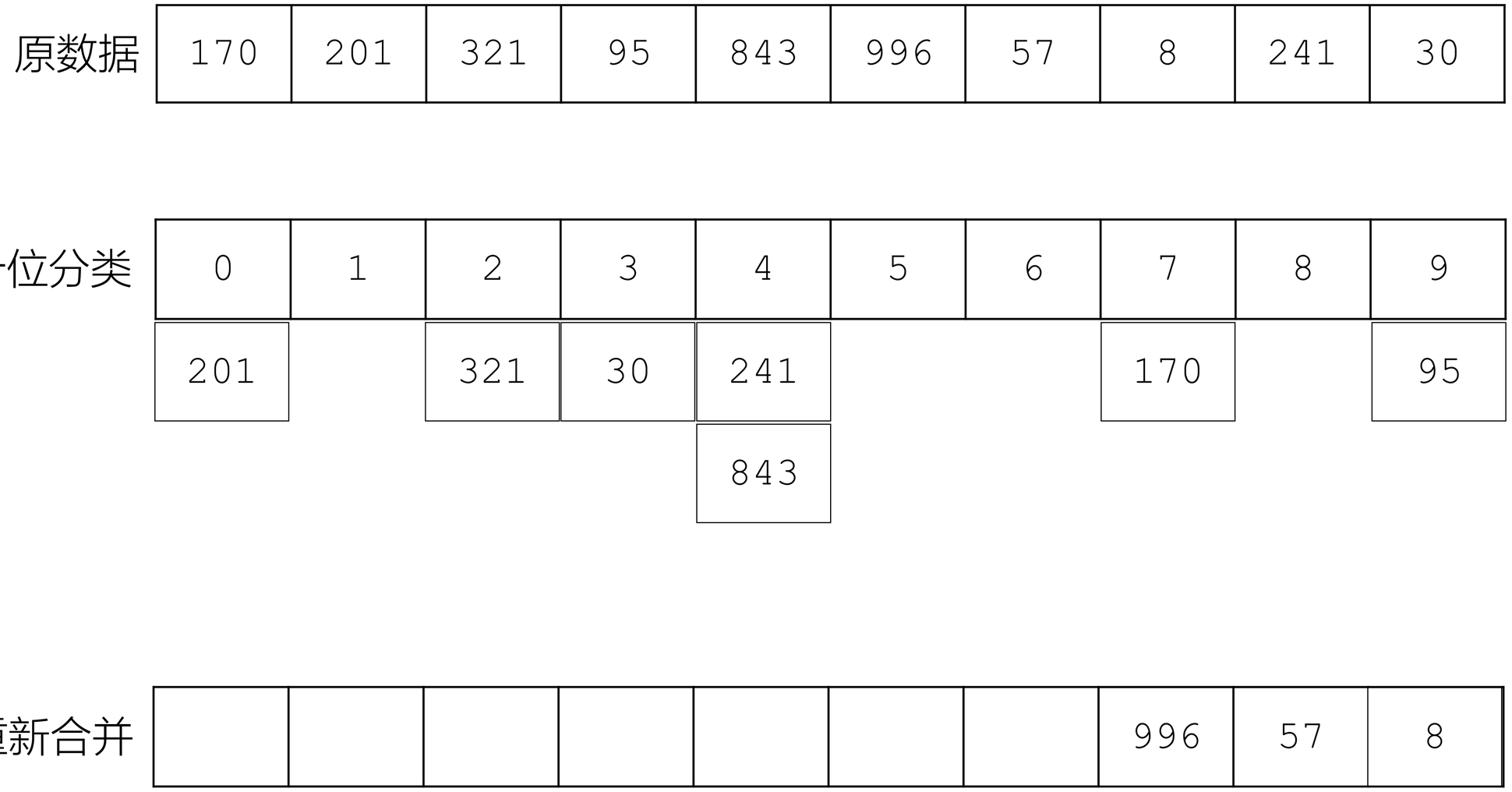
根据十位分类	0	1	2	3	4	5	6	7	8	9
	201		321	30	241			170		

分类后重新合并						843	95	996	57	8

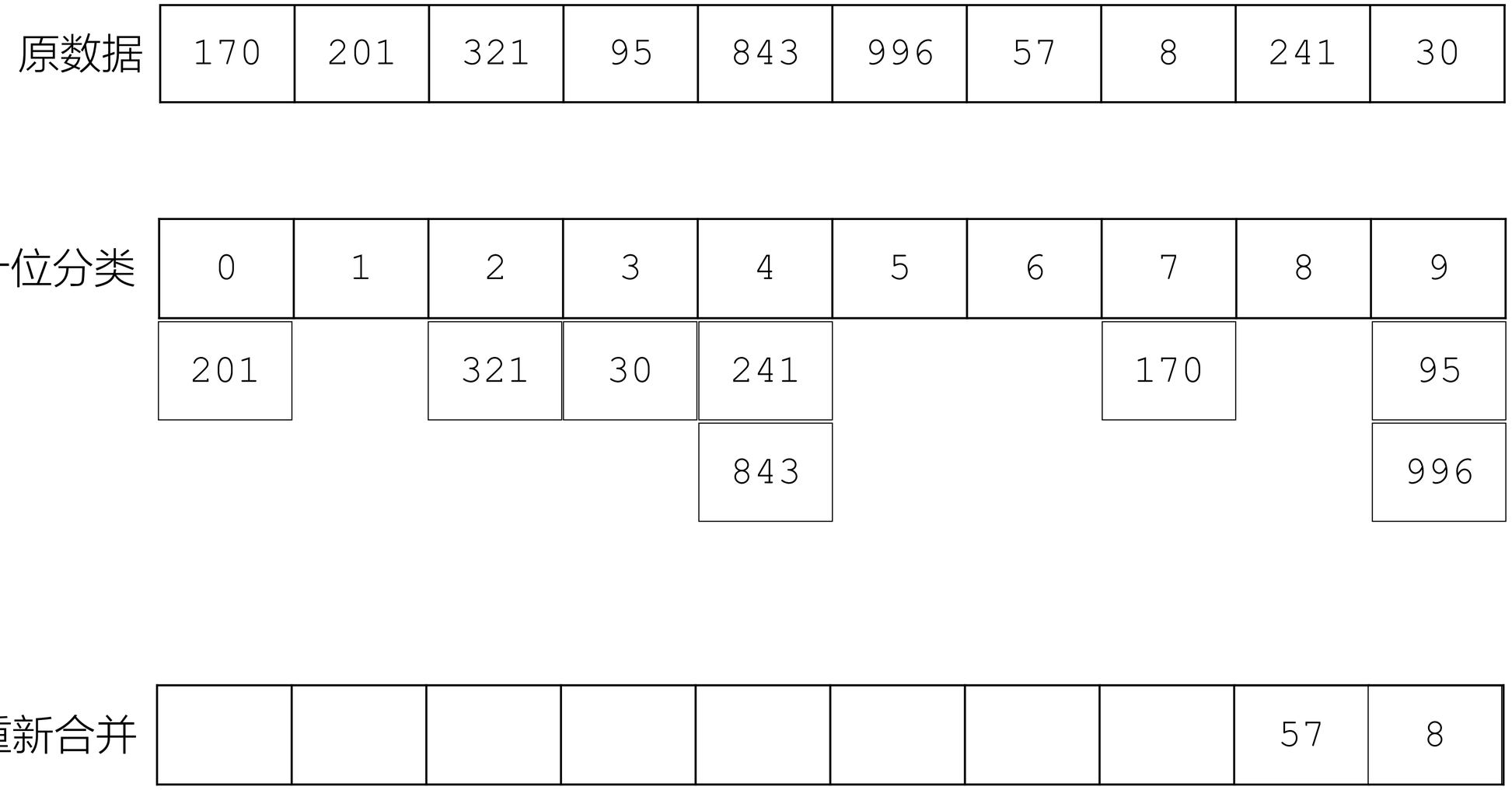
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
201
321
30
241
170
843
分类后重新合并
95
996
57
8

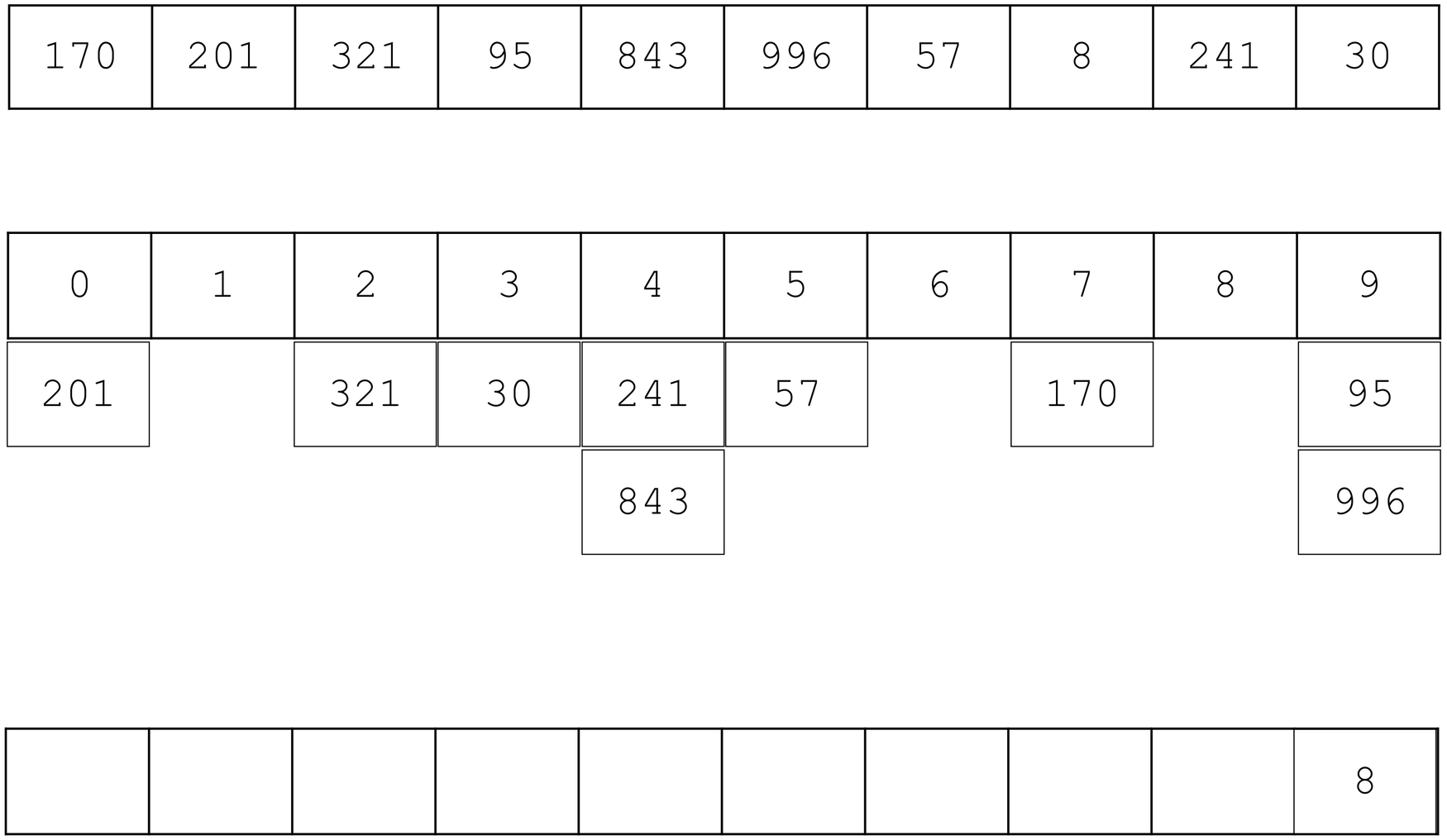
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
201
321
30
241
170
95
843
分类后重新合并
996
57
8

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
201
321
30
241
170
95
843
996
分类后重新合并
57
8

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
201
321
30
241
57
170
95
843
996
分类后重新合并
8

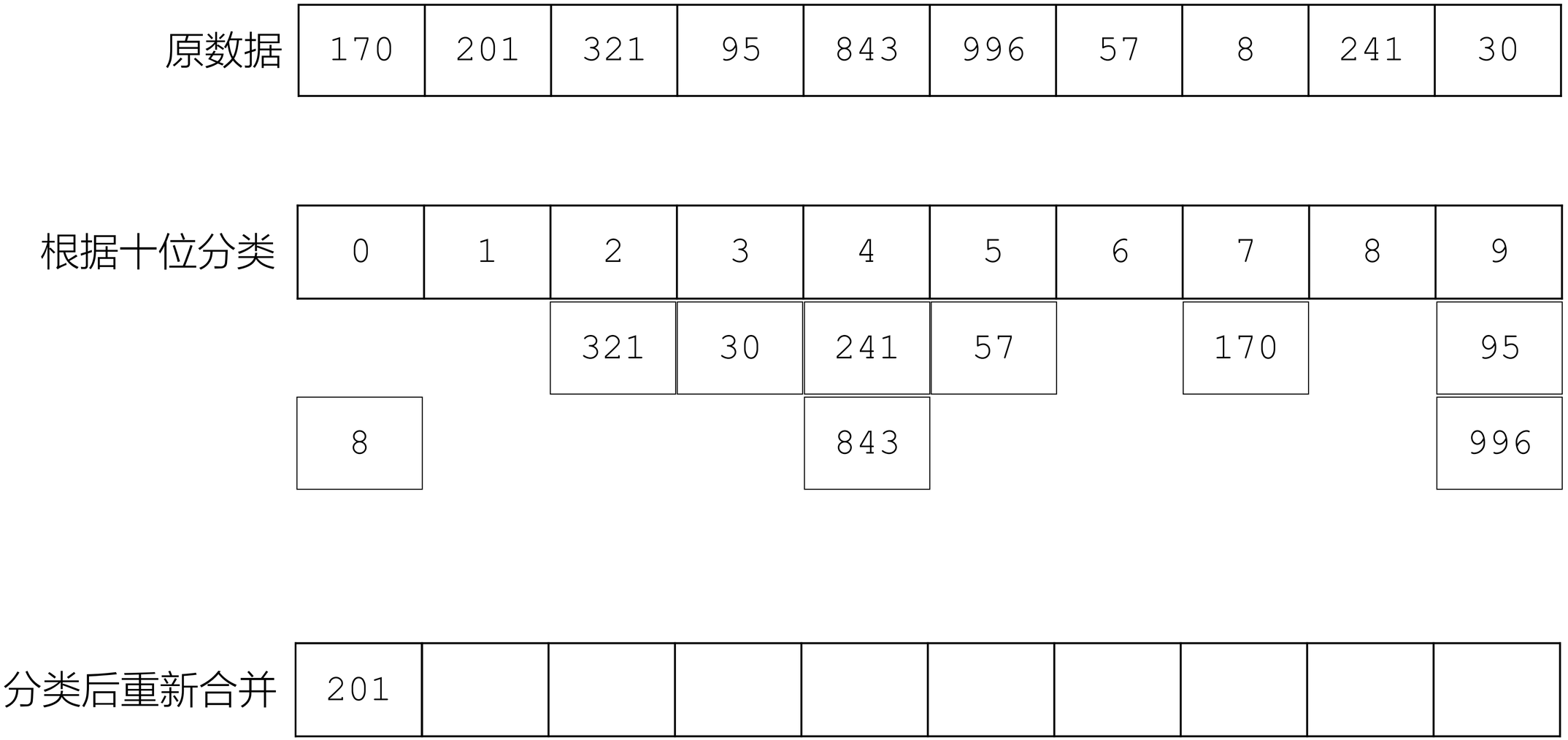
基数排序

原数据	170	201	321	95	843	996	57	8	241	30

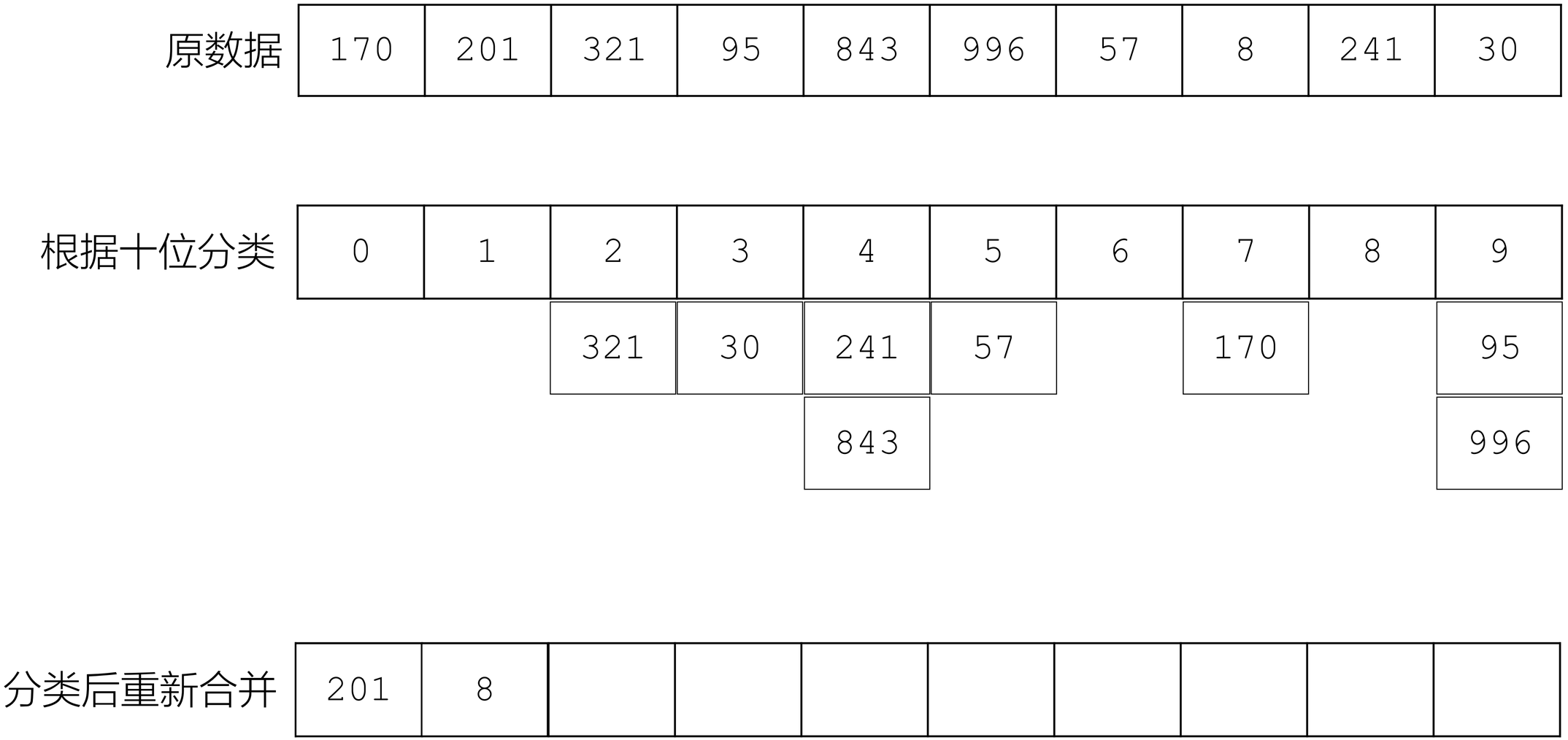
根据十位分类	0	1	2	3	4	5	6	7	8	9
	201		321	30	241	57		170		95
	8				843					996

分类后重新合并										

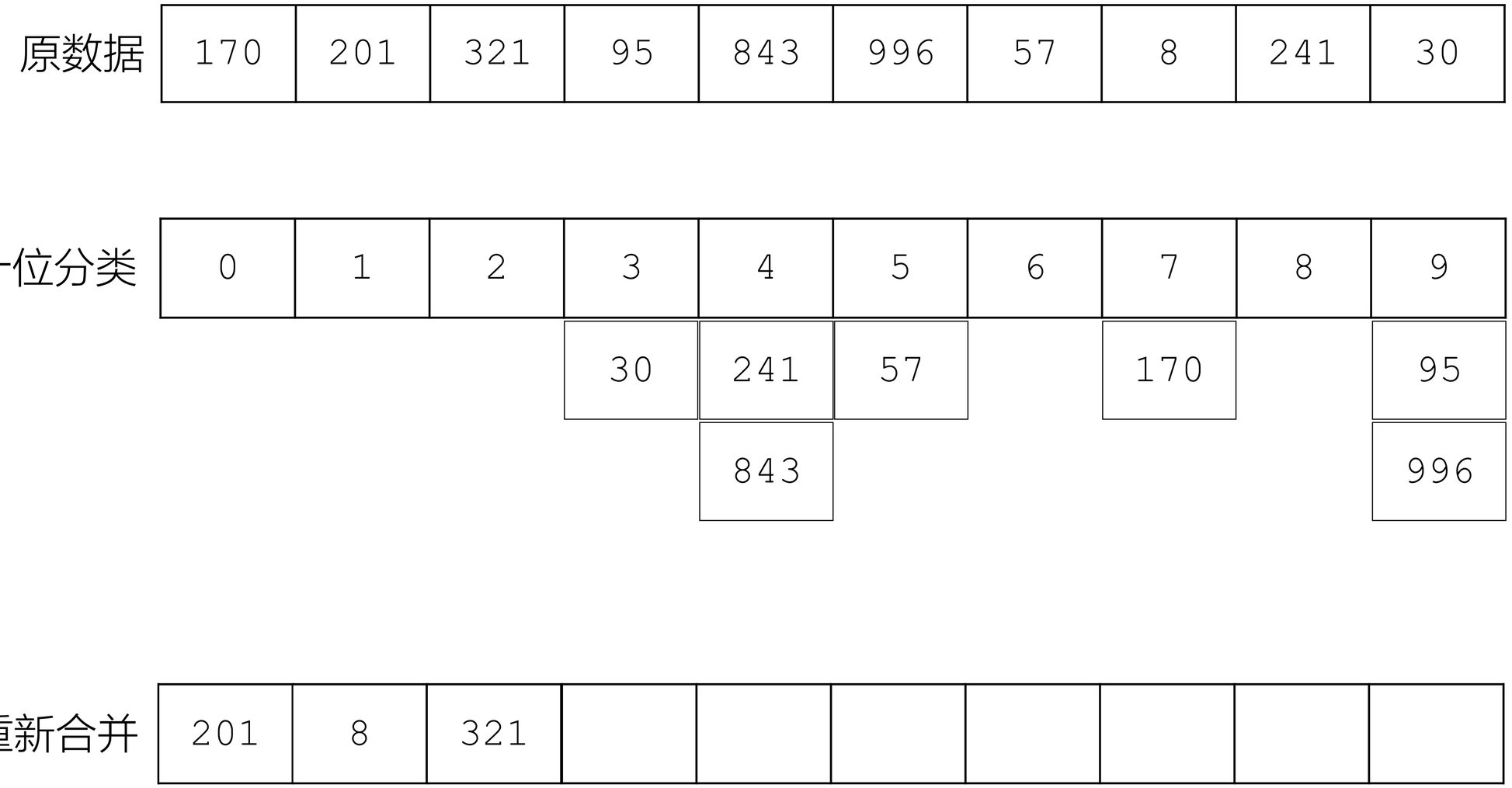
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
321
30
241
57
170
95
8
843
996
分类后重新合并
201

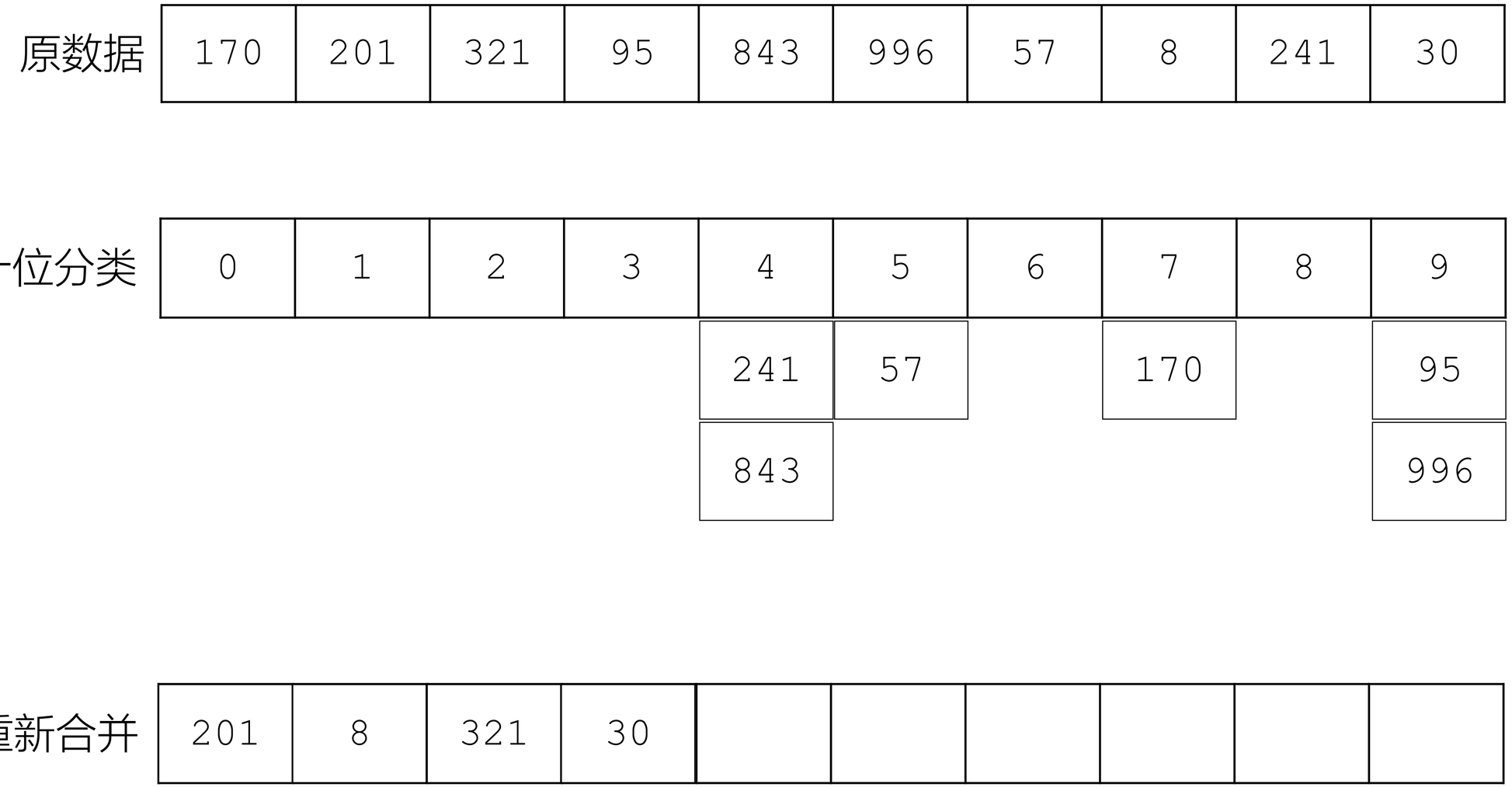
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
321
30
241
57
170
95
843
996
分类后重新合并
201
8

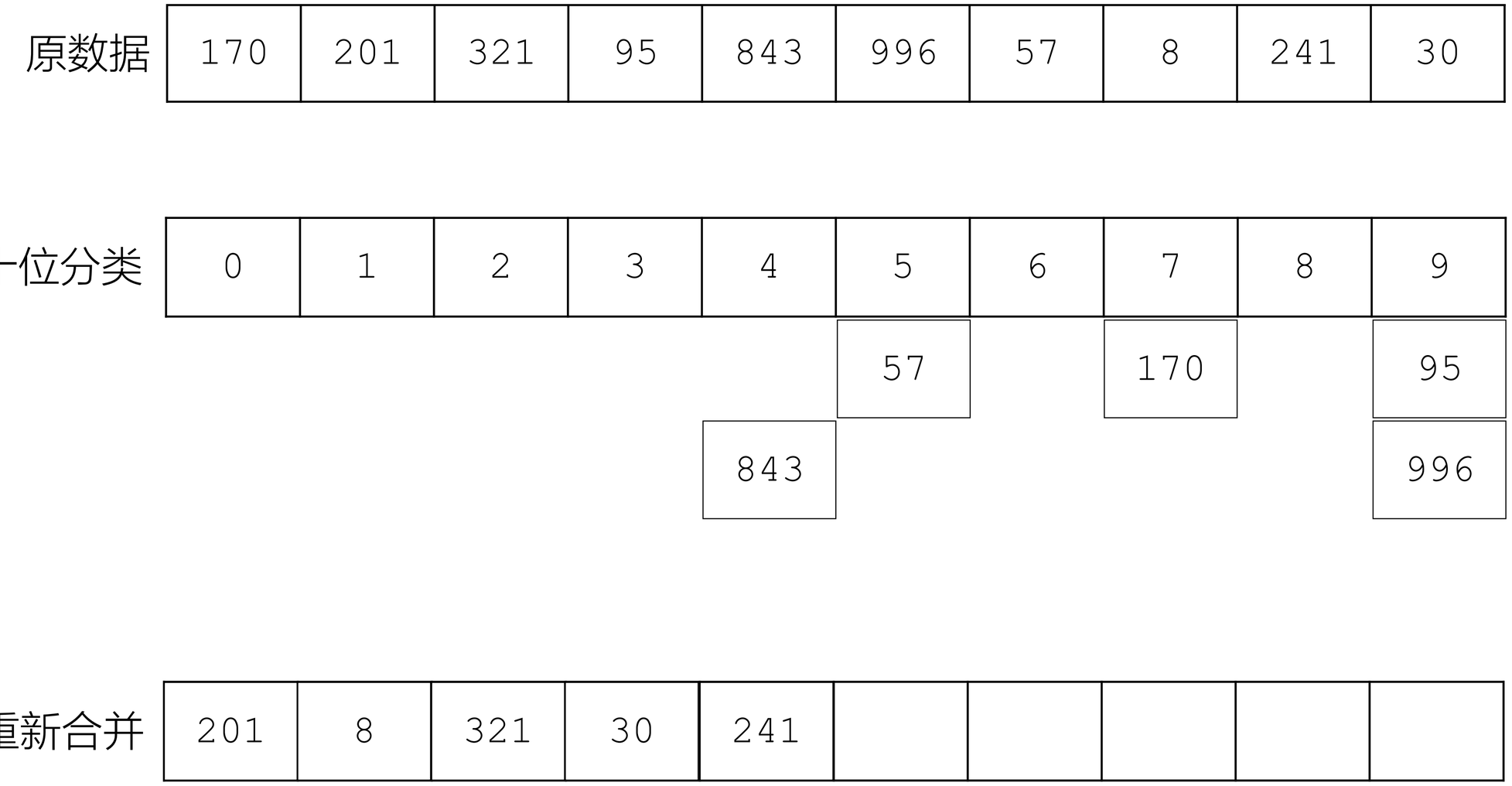
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
30
241
57
170
95
843
996
分类后重新合并
201
8
321

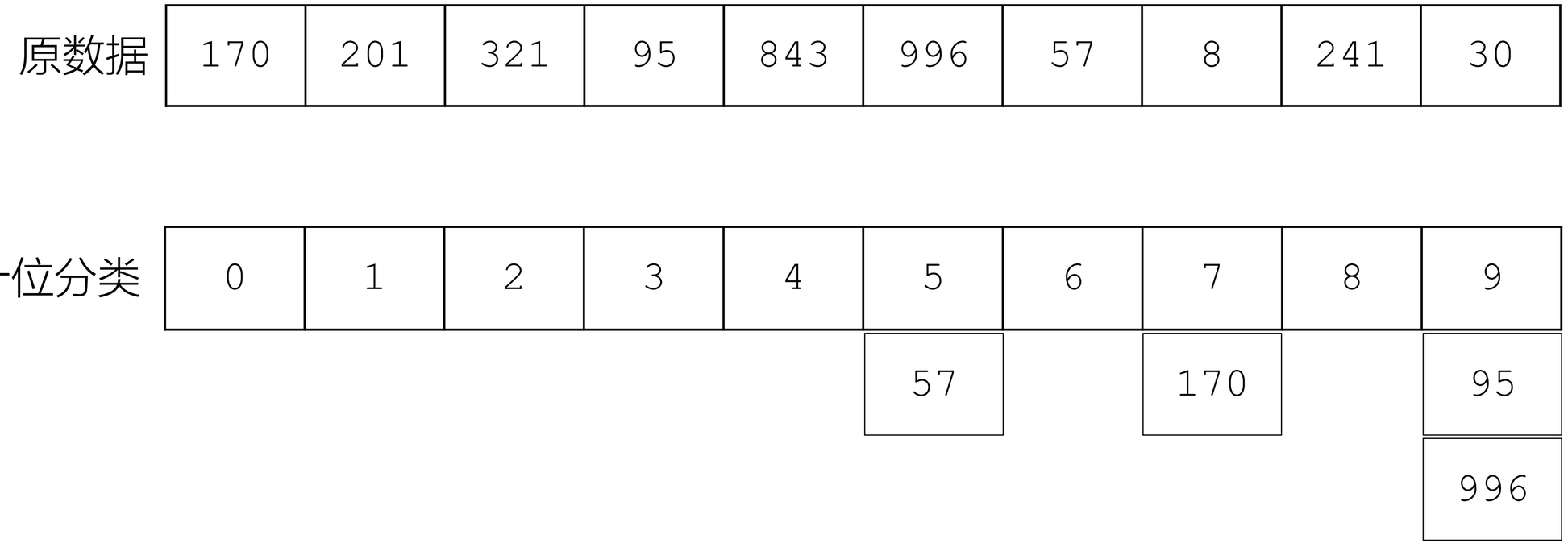
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
241
57
170
95
843
996
分类后重新合并
201
8
321
30

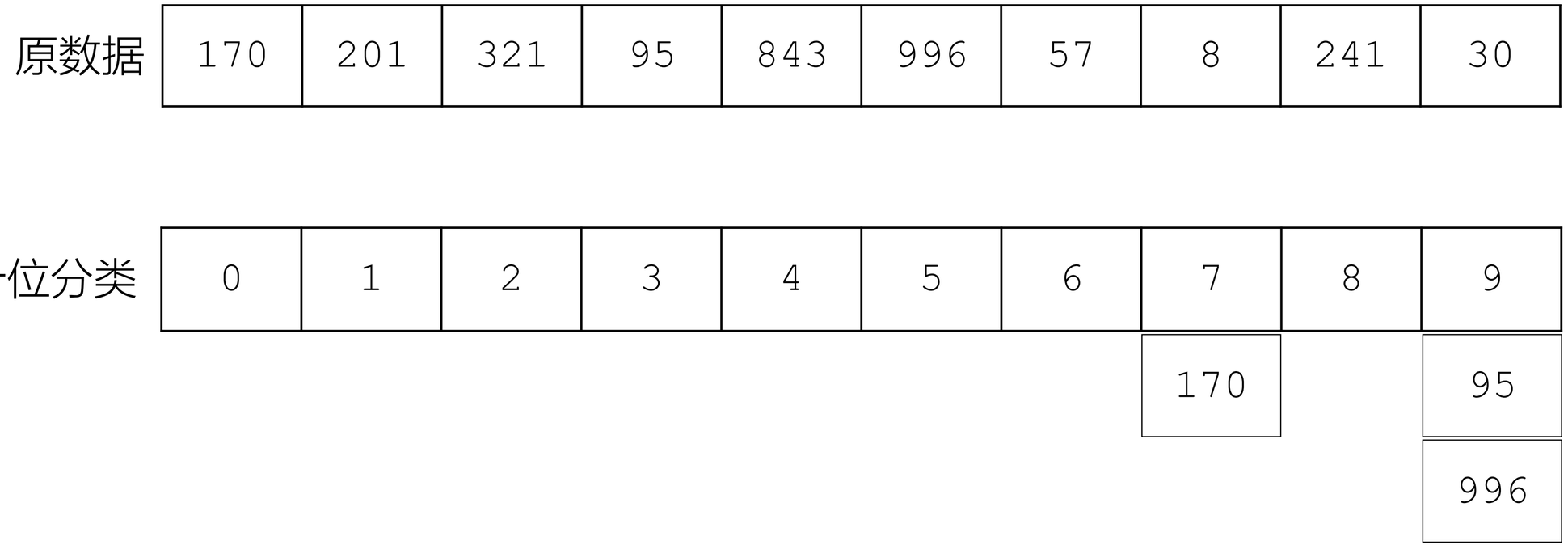
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
57
170
95
843
996
分类后重新合并
201
8
321
30
241

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
57
170
95
996
分类后重新合并
201
8
321
30
241
843

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据十位分类
0
1
2
3
4
5
6
7
8
9
170
95
996
分类后重新合并
201
8
321
30
241
843
57

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据十位分类	0	1	2	3	4	5	6	7	8	9
										95
										996

分类后重新合并	201	8	321	30	241	843	57	170		

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据十位分类	0	1	2	3	4	5	6	7	8	9

996

分类后重新合并	201	8	321	30	241	843	57	170	95	

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据十位分类	0	1	2	3	4	5	6	7	8	9

分类后重新合并	201	8	321	30	241	843	57	170	95	996

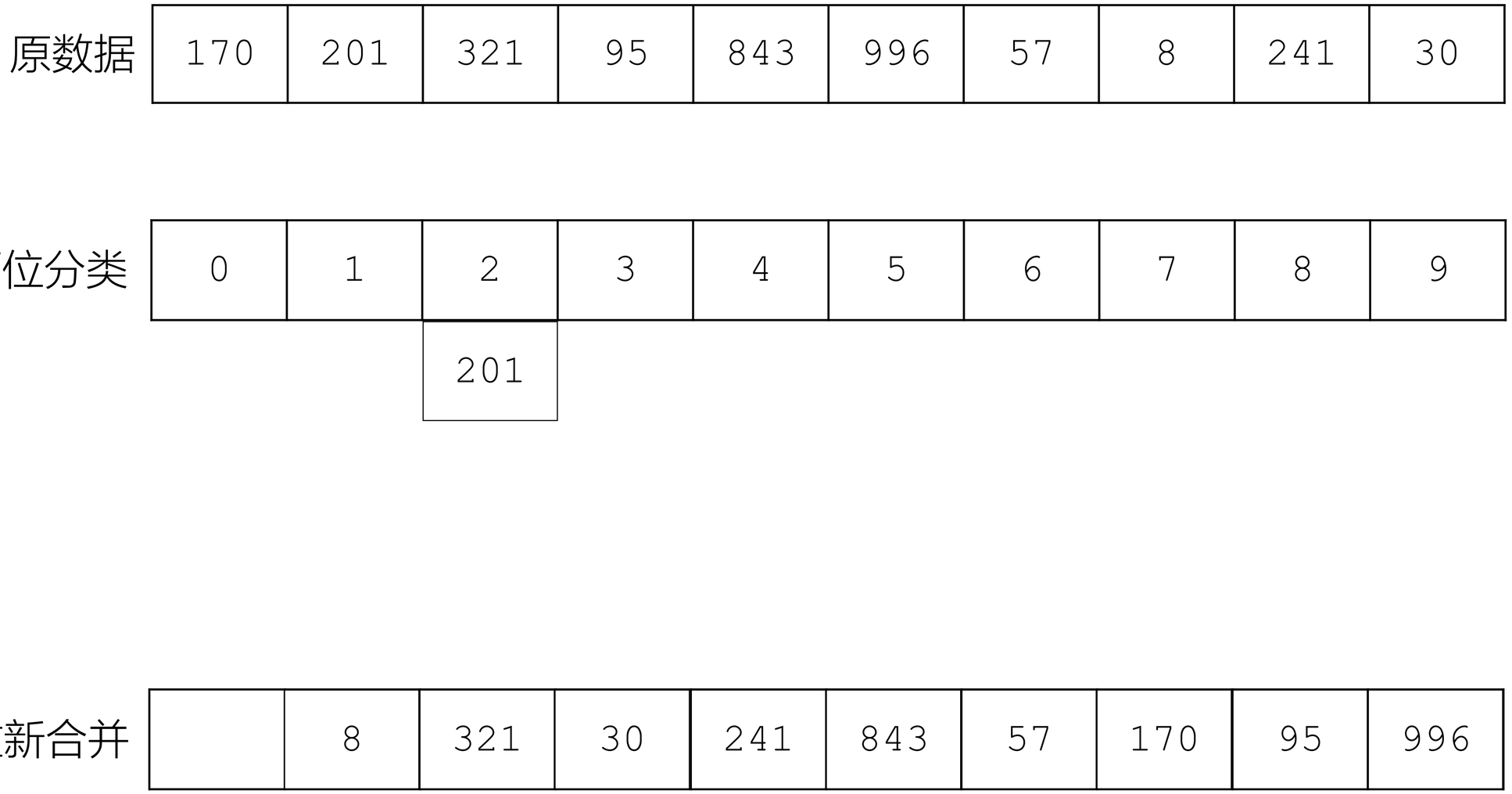
基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据百位分类	0	1	2	3	4	5	6	7	8	9

分类后重新合并	201	8	321	30	241	843	57	170	95	996

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
201
分类后重新合并
8
321
30
241
843
57
170
95
996

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据百位分类	0	1	2	3	4	5	6	7	8	9
	8		201							

分类后重新合并			321	30	241	843	57	170	95	996

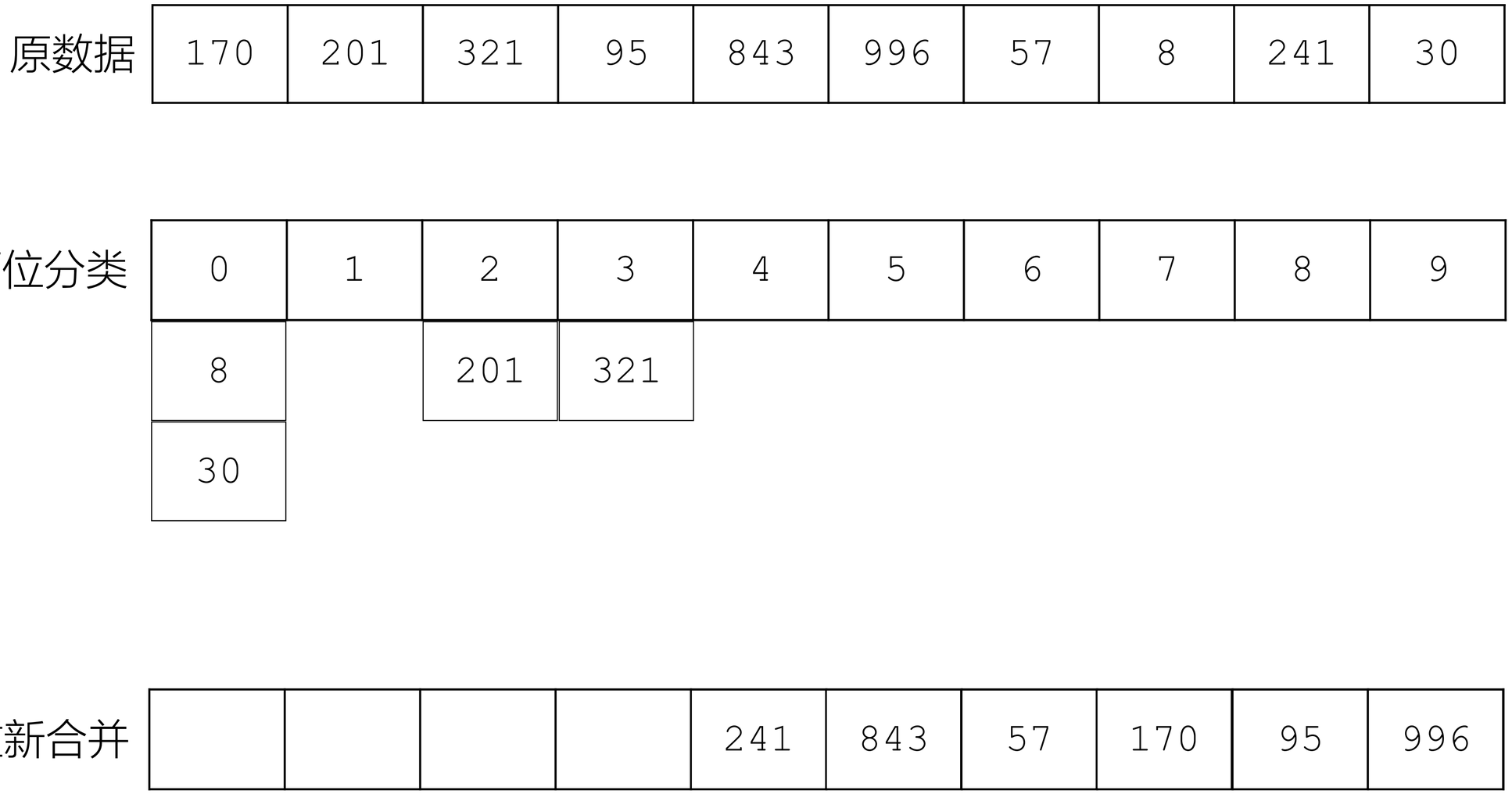
基数排序

原数据	170	201	321	95	843	996	57	8	241	30

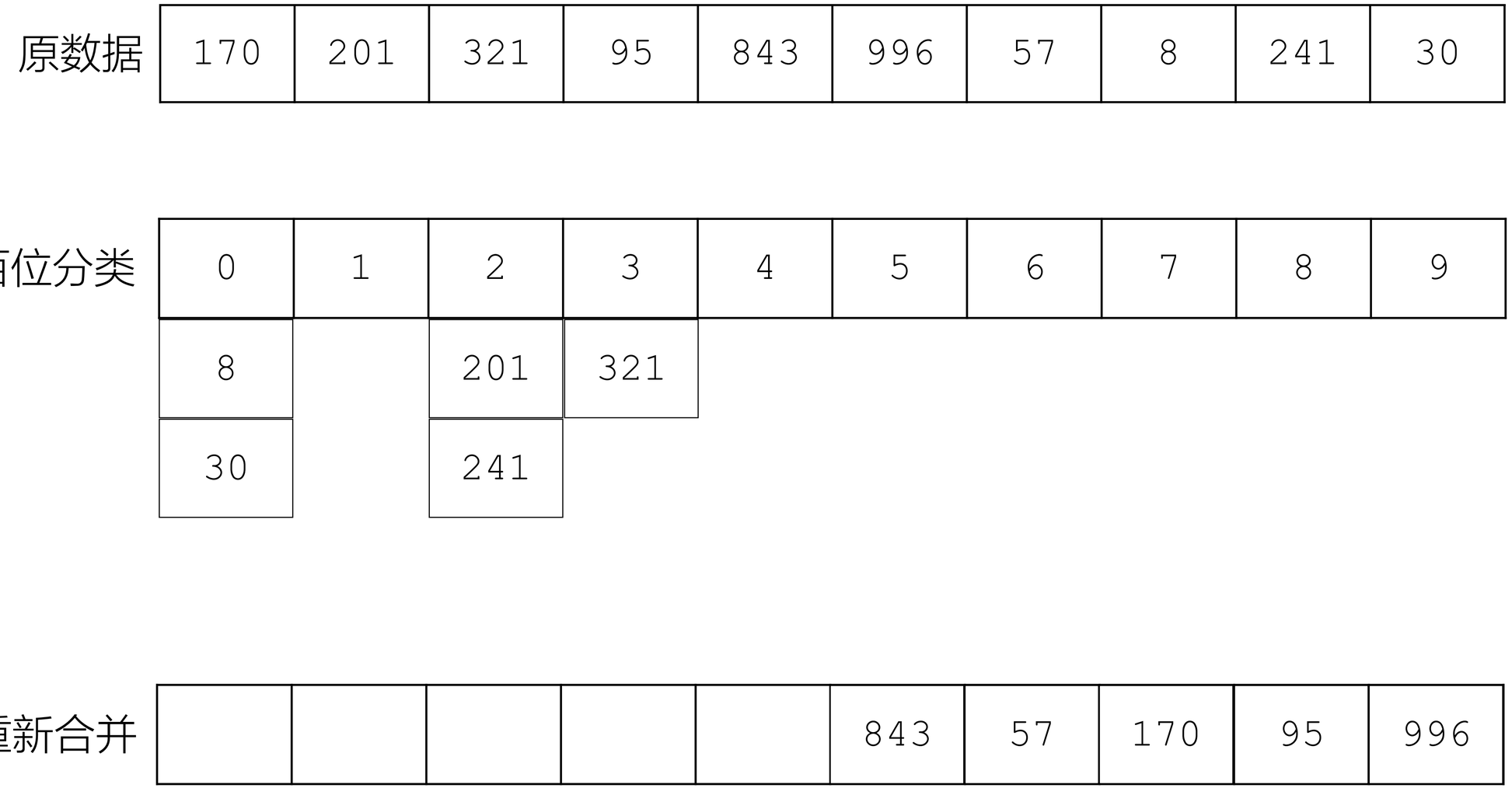
根据百位分类	0	1	2	3	4	5	6	7	8	9
	8		201	321						

分类后重新合并				30	241	843	57	170	95	996

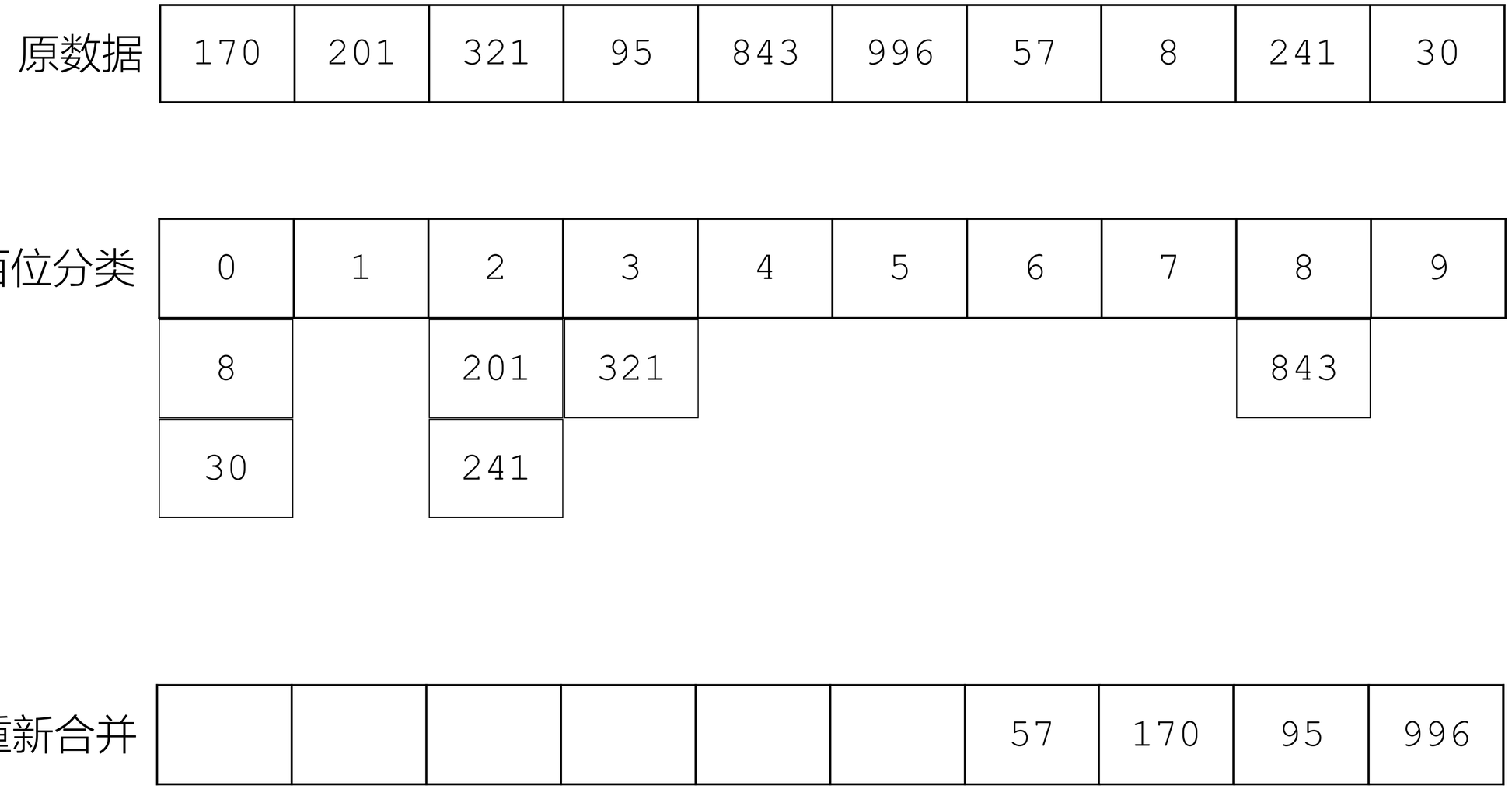
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
8
201
321
30
分类后重新合并
241
843
57
170
95
996

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
8
201
321
30
241
分类后重新合并
843
57
170
95
996

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
8
201
321
843
30
241
分类后重新合并
57
170
95
996

基数排序

原数据

170	201	321	95	843	996	57	8	241	30

根据百位分类

0	1	2	3	4	5	6	7	8	9
8		201	321					843	
30		241							
57									

分类后重新合并

							170	95	996

基数排序

原数据

170	201	321	95	843	996	57	8	241	30

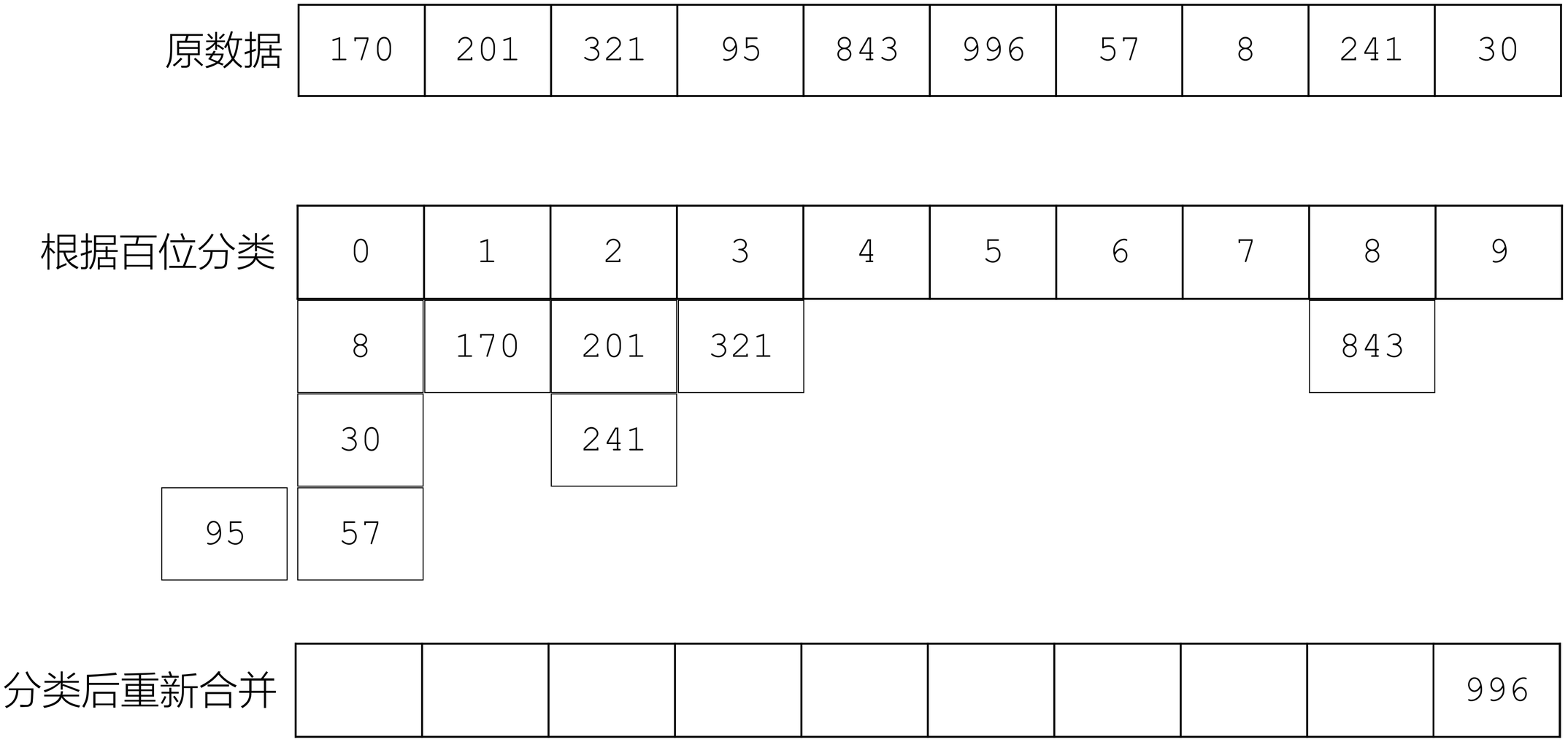
根据百位分类

0	1	2	3	4	5	6	7	8	9
8	170	201	321					843	
30		241							
57									

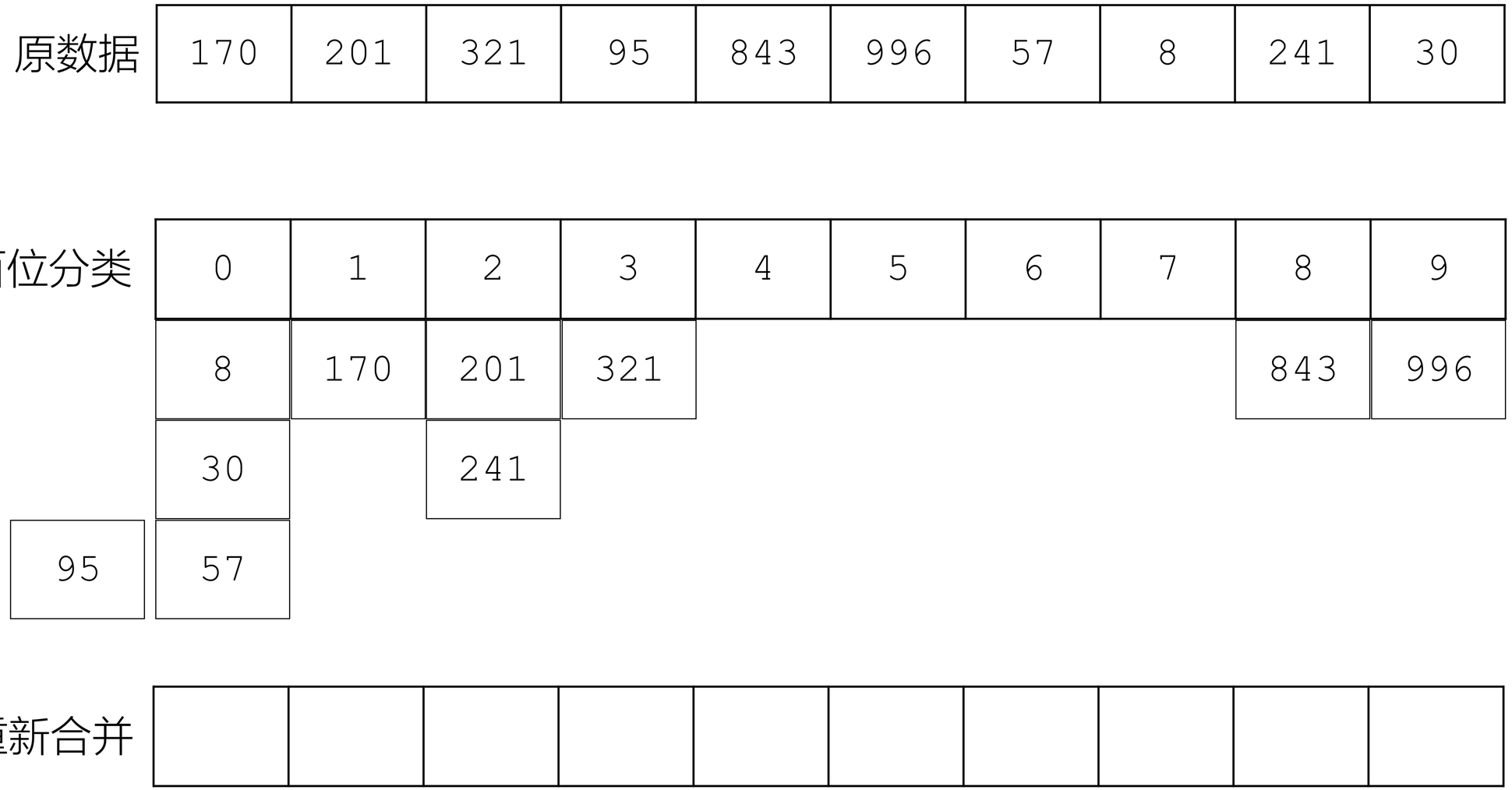
分类后重新合并

								95	996

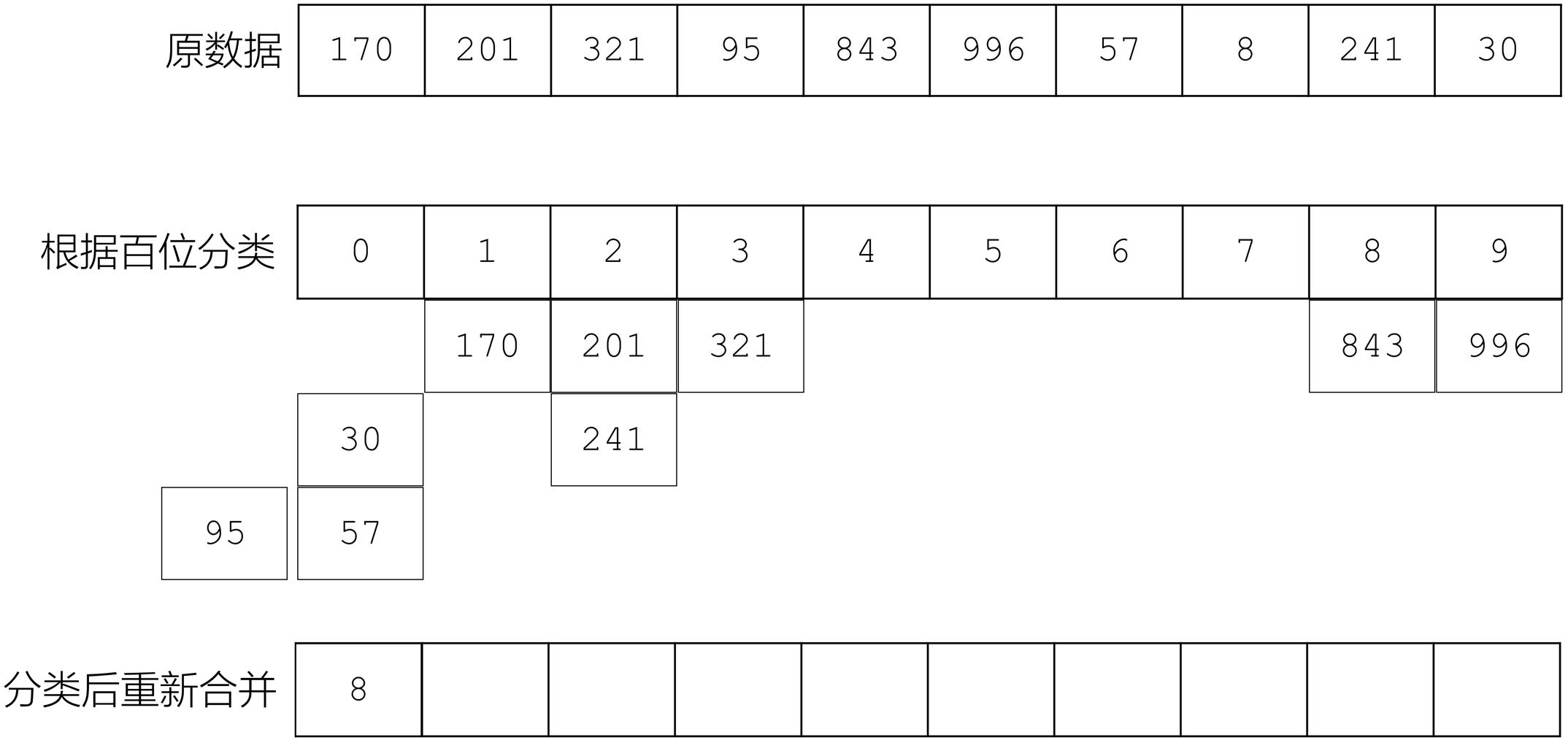
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
8
170
201
321
843
30
241
95
57
分类后重新合并
996

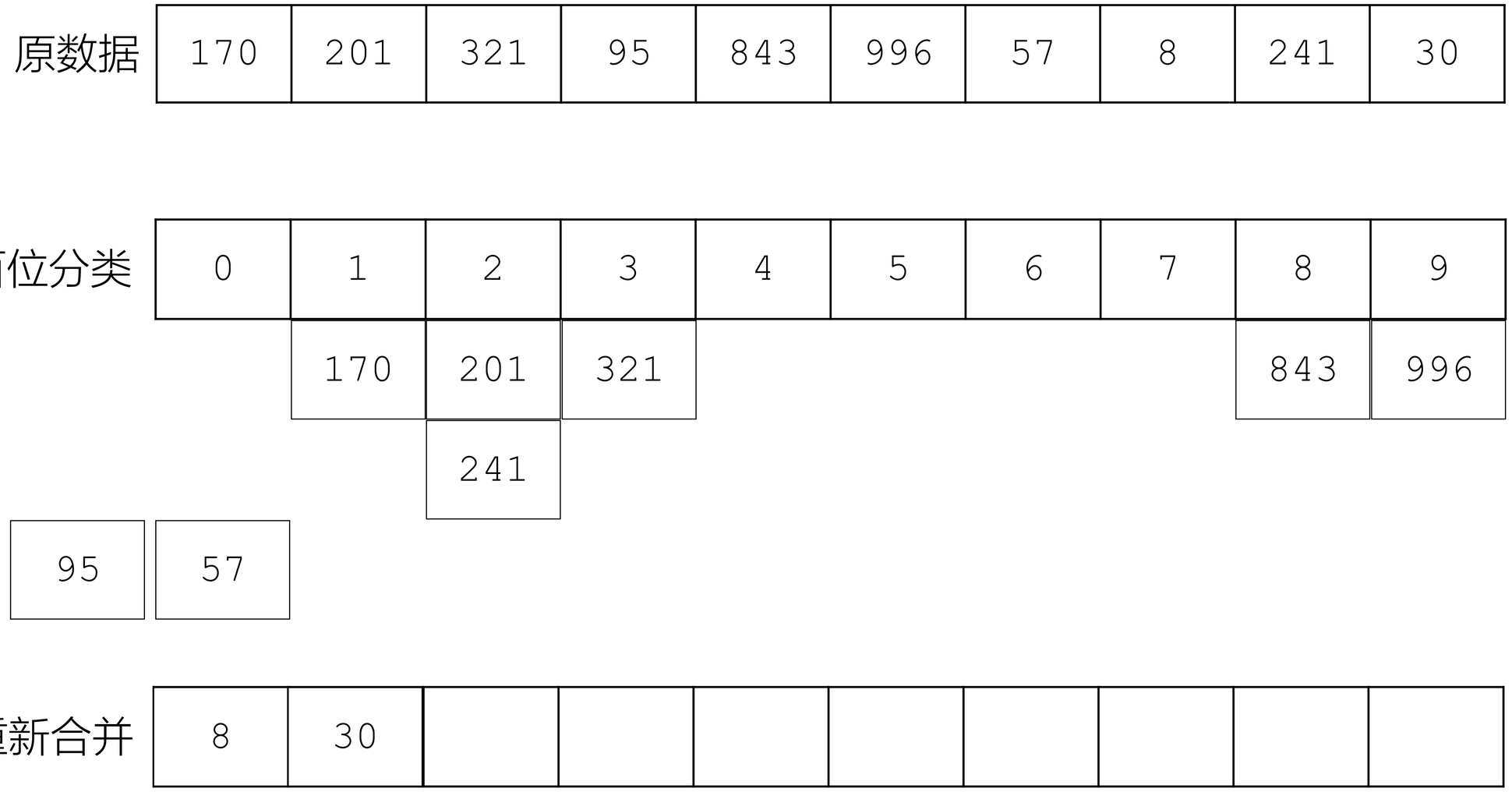
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
8
170
201
321
843
996
30
241
95
57
分类后重新合并

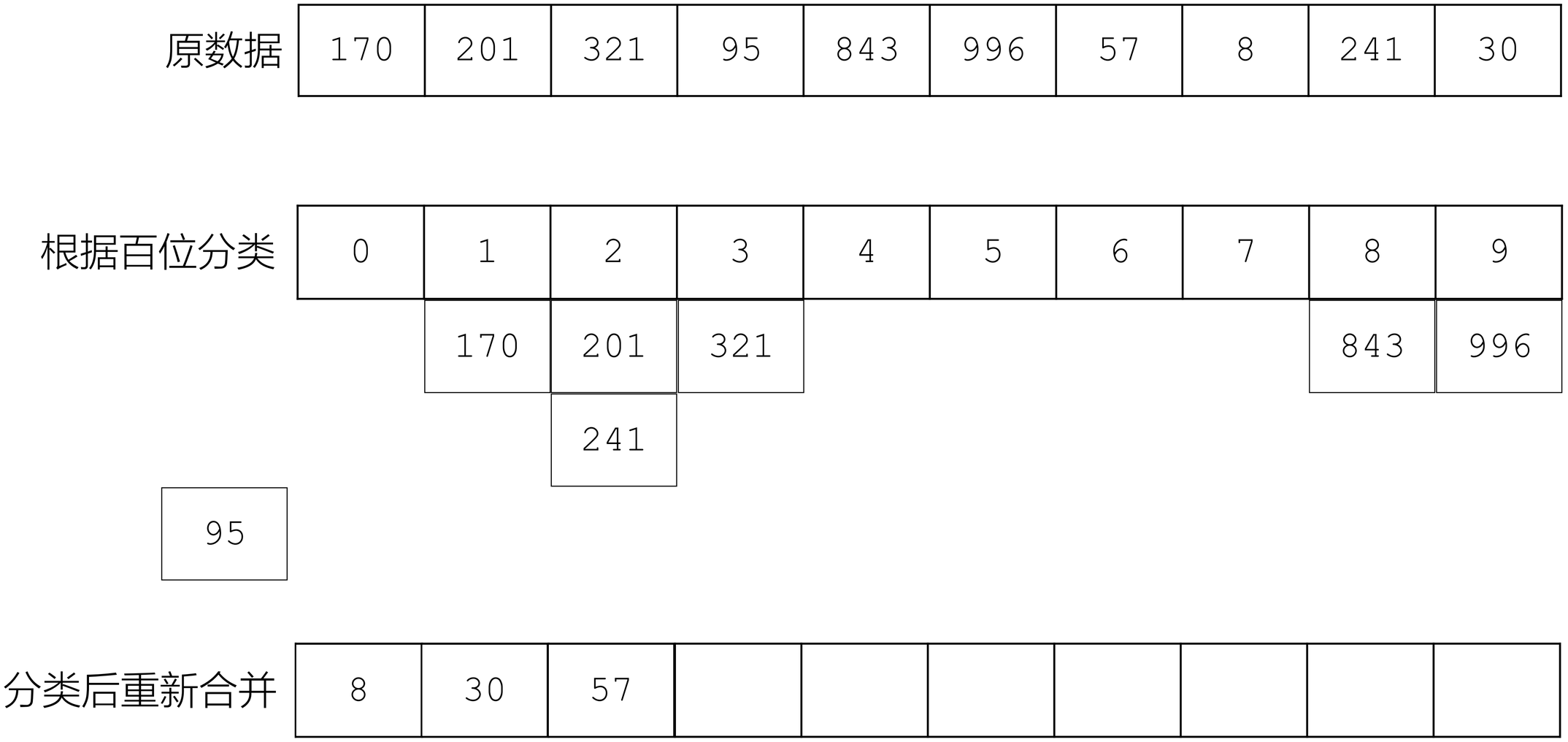
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
170
201
321
843
996
30
241
95
57
分类后重新合并
8

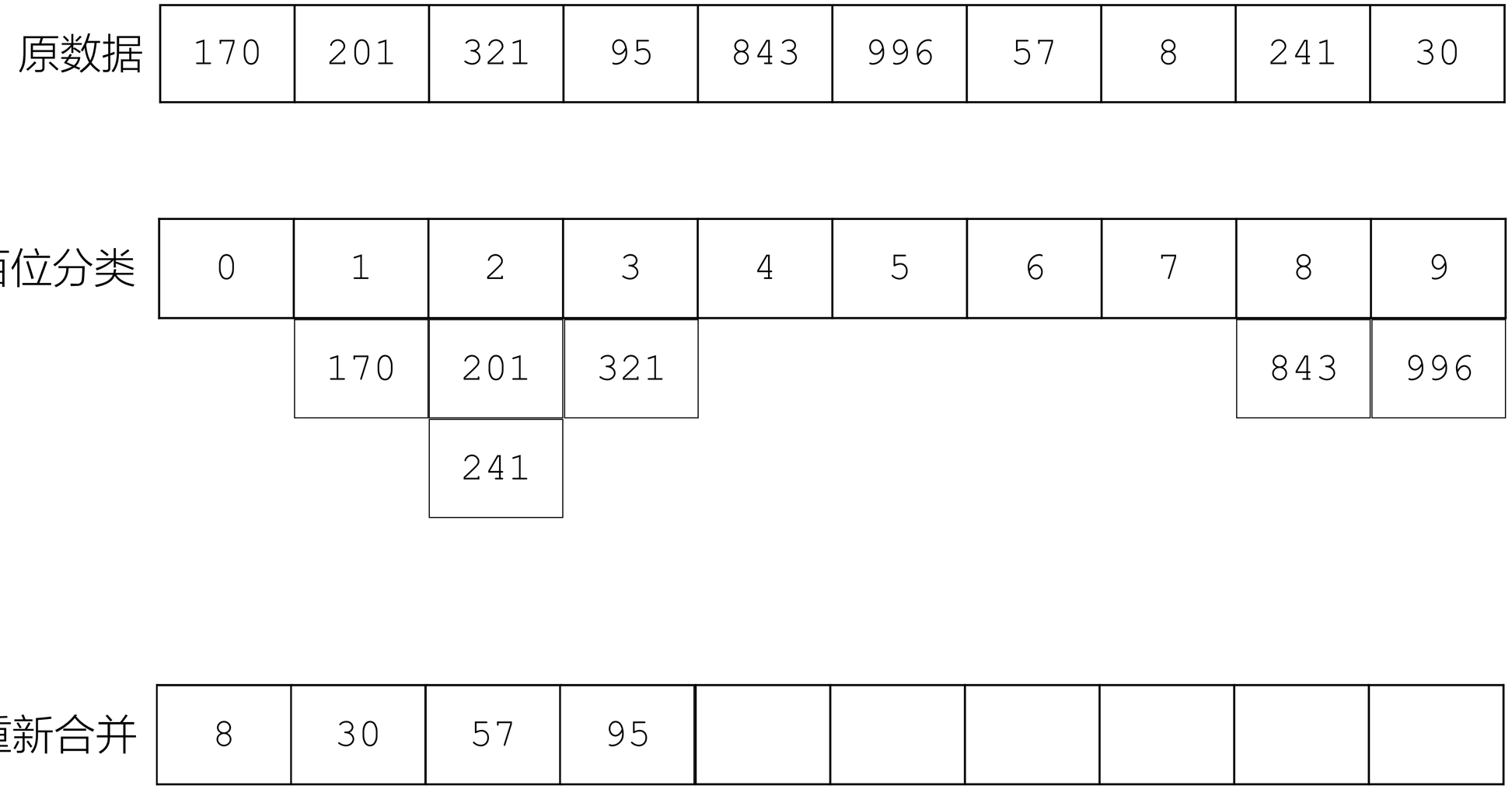
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
170
201
321
843
996
241
95
57
分类后重新合并
8
30

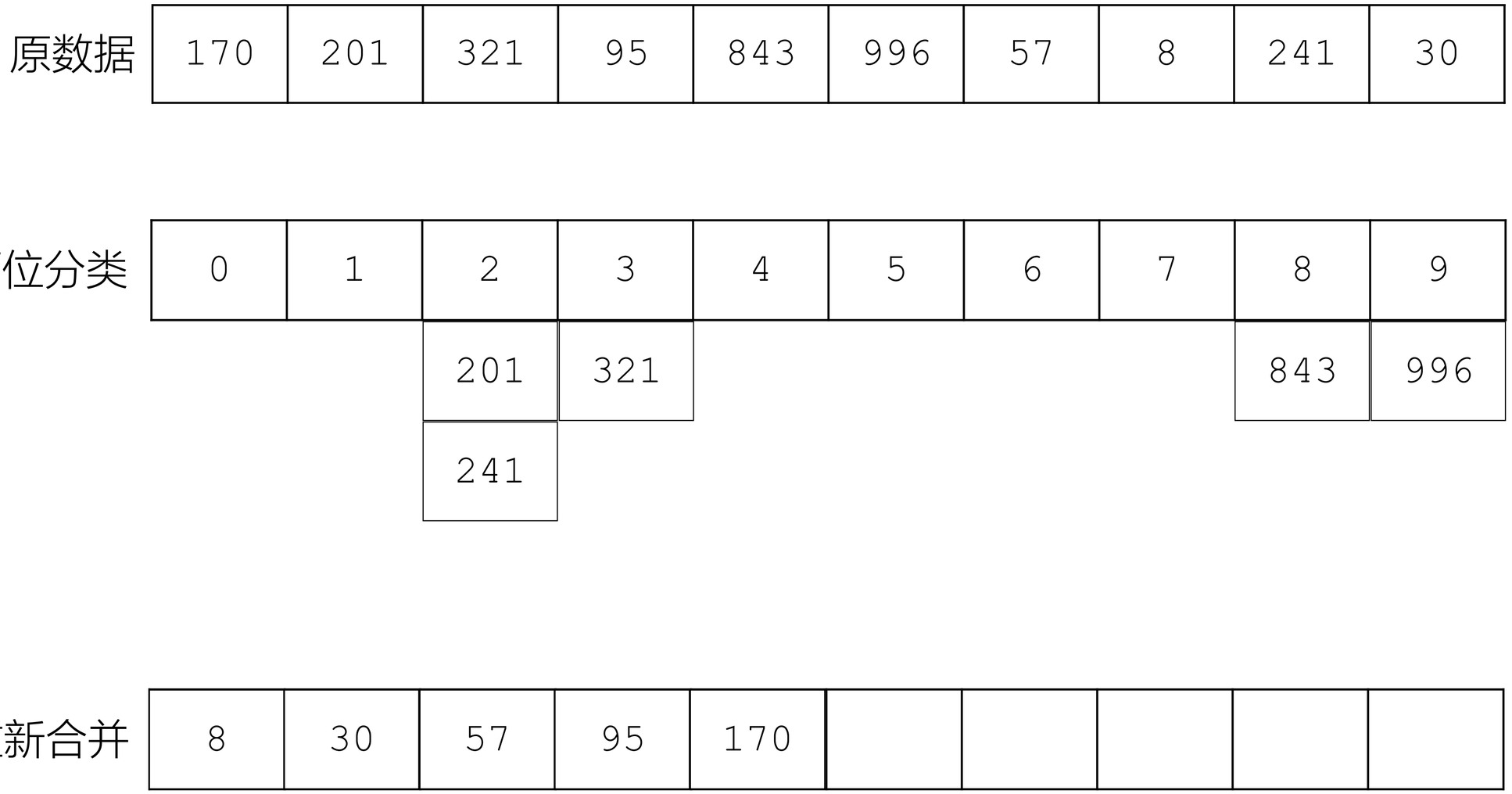
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
170
201
321
843
996
241
95
分类后重新合并
8
30
57

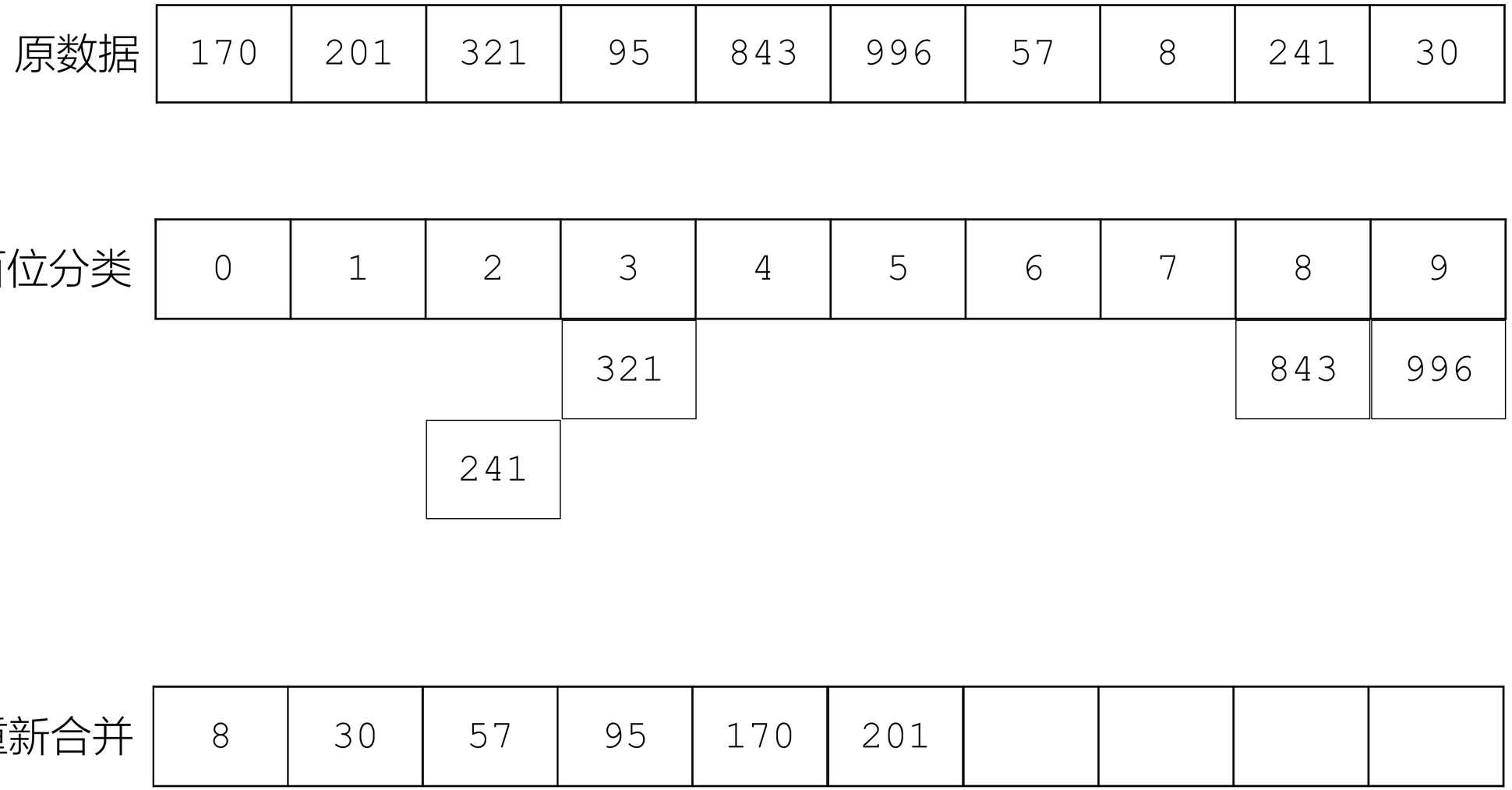
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
170
201
321
843
996
241
分类后重新合并
8
30
57
95

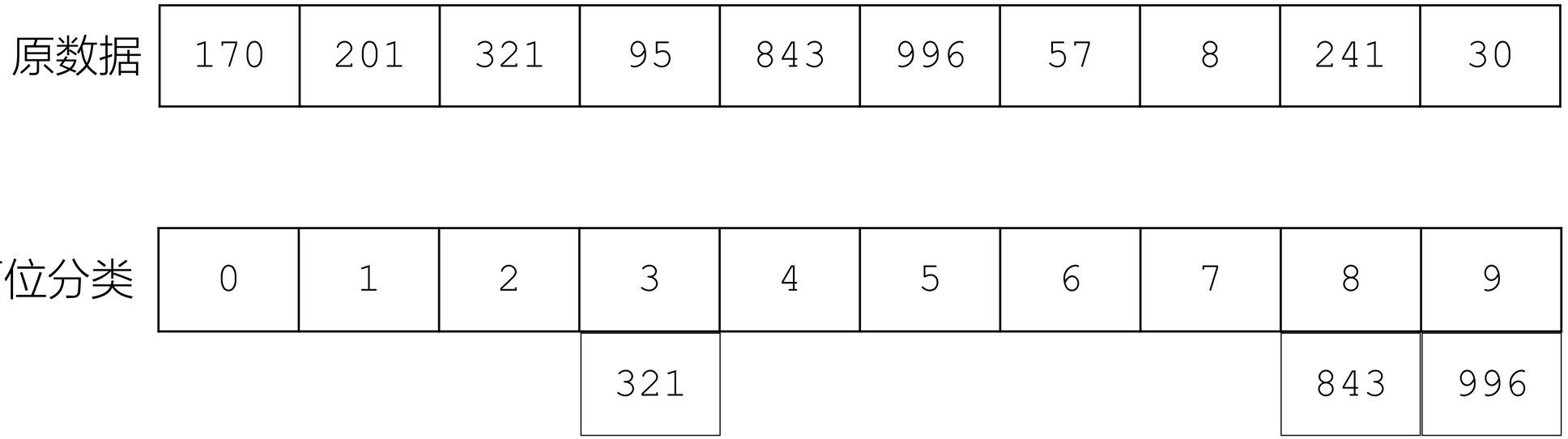
基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
201
321
843
996
241
分类后重新合并
8
30
57
95
170

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
321
843
996
241
分类后重新合并
8
30
57
95
170
201

基数排序


原数据
170
201
321
95
843
996
57
8
241
30
根据百位分类
0
1
2
3
4
5
6
7
8
9
321
843
996
分类后重新合并
8
30
57
95
170
201
241

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据百位分类	0	1	2	3	4	5	6	7	8	9
									843	996

分类后重新合并	8	30	57	95	170	201	241	321		

基数排序

原数据	170	201	321	95	843	996	57	8	241	30

根据百位分类	0	1	2	3	4	5	6	7	8	9
										996

分类后重新合并	8	30	57	95	170	201	241	321	843	

基数排序

排序 / test9.c

原数据	170	201	321	95	843	996	57	8	241	30

根据百位分类	0	1	2	3	4	5	6	7	8	9

分类后重新合并	8	30	57	95	170	201	241	321	843	996

基数排序

维度	描述
时间复杂度	$O(d \cdot n)$，其中 d 是最大数的位数，n 是数据量
空间复杂度	$O(n + RADIX)$，需要额外的输出数组和计数桶
算法稳定性	稳定
适用场景	适合整数、定长字符串，尤其是大数据量、数据分布均匀且位数不高的场景

真题解析

[2009] 10. 若数据元素序列 11, 12, 13, 7, 8, 9, 23, 4, 5 是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是_____。

A. 冒泡排序　　B. 插入排序　　C. 选择排序　　D. 二路归并排序

真题解析

[2009] 10. 若数据元素序列 11, 12, 13, 7, 8, 9, 23, 4, 5 是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是_____。

A. 冒泡排序　　B. 插入排序　　C. 选择排序　　D. 二路归并排序

真题解析

[2010] 10. 采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是_____。

A. 递归次数与初始数据的排列次序无关

B. 每次划分后，先处理较长的分区可以减少递归次数

C. 每次划分后，先处理较短的分区可以减少递归次数

D. 递归次数与每次划分后得到的分区的处理顺序无关

真题解析

[2010] 10. 采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是_____。

A. 递归次数与初始数据的排列次序无关

B. 每次划分后，先处理较长的分区可以减少递归次数

C. 每次划分后，先处理较短的分区可以减少递归次数

D. 递归次数与每次划分后得到的分区的处理顺序无关

真题解析

[2010] 11. 对一组数据（2, 12, 16, 88, 5, 10）进行排序，若前三趟排序结果如下：

第一趟排序结果：2, 12, 16, 5, 10, 88

第二趟排序结果：2, 12, 5, 10, 16, 88

第三趟排序结果：2, 5, 10, 12, 16, 88

则采用的排序方法可能是_____。

A. 冒泡排序　　B. 希尔排序　　C. 归并排序　　D. 基数排序

真题解析

[2010] 11. 对一组数据（2, 12, 16, 88, 5, 10）进行排序，若前三趟排序结果如下：

第一趟排序结果：2, 12, 16, 5, 10, 88

第二趟排序结果：2, 12, 5, 10, 16, 88

第三趟排序结果：2, 5, 10, 12, 16, 88

则采用的排序方法可能是_____。

A. 冒泡排序　　B. 希尔排序　　C. 归并排序　　D. 基数排序

真题解析

[2011] 10. 为了实现快速排序，待排序序列宜采用的存储方式是_____。

A. 顺序存储　　B. 散列存储　　C. 链式存储　　D. 索引存储

真题解析

[2011] 10. 为了实现快速排序，特排序序列宜采用的存储方式是_____。

A. 顺序存储　　B. 散列存储　　C. 链式存储　　D. 索引存储

真题解析

[2012] 10. 在内部排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序方法中，每一趟排序结束都至少能够确定一个元素最终位置的方法是_____。

I. 简单选择排序　　II. 希尔排序　　III. 快速排序

IV. 堆排序　　V. 二路归并排序

A. 仅 I、III、IV　　B. 仅 I、III、V　　C. 仅 II、III、IV　　D. 仅 III、IV、V

真题解析

[2012] 10. 在内部排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序方法中，每一趟排序结束都至少能够确定一个元素最终位置的方法是_____。

I. 简单选择排序　　II. 希尔排序　　III. 快速排序

IV. 堆排序　　V. 二路归并排序

A. 仅 I、III、IV　　B. 仅 I、III、V　　C. 仅 II、III、IV　　D. 仅 III、IV、V

真题解析

[2012] 11. 对一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是_____。

A. 排序的总趟数

B. 元素的移动次数

C. 使用辅助空间的数量

D. 元素之间的比较次数

真题解析

[2012] 11. 对一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是_____。

A. 排序的总趟数

B. 元素的移动次数

C. 使用辅助空间的数量

D. 元素之间的比较次数

真题解析

[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119

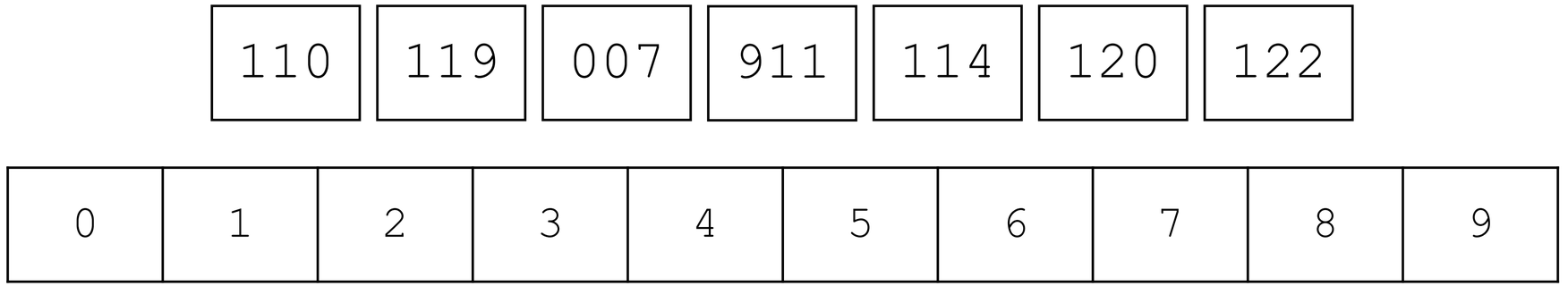
[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119



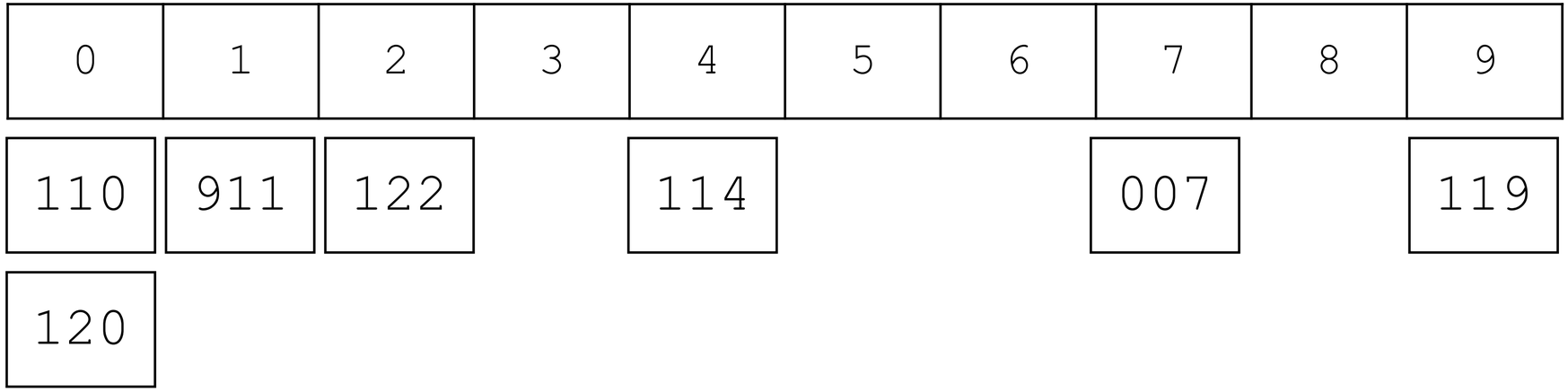
[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119



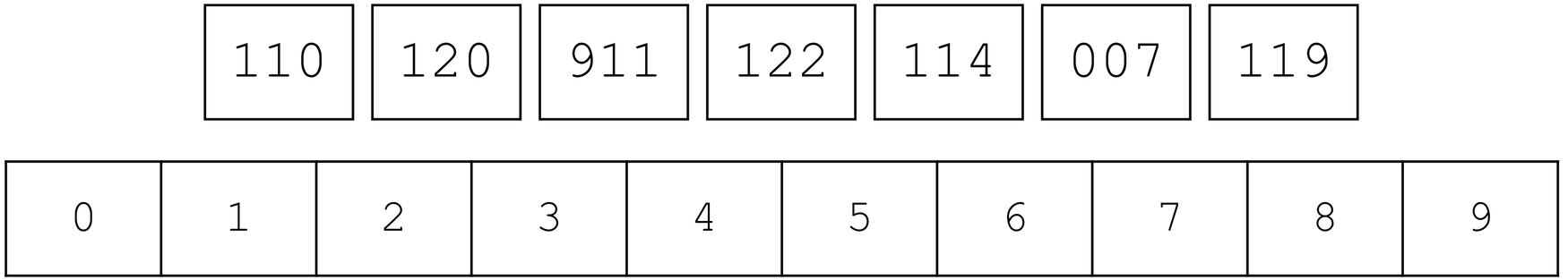
[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119



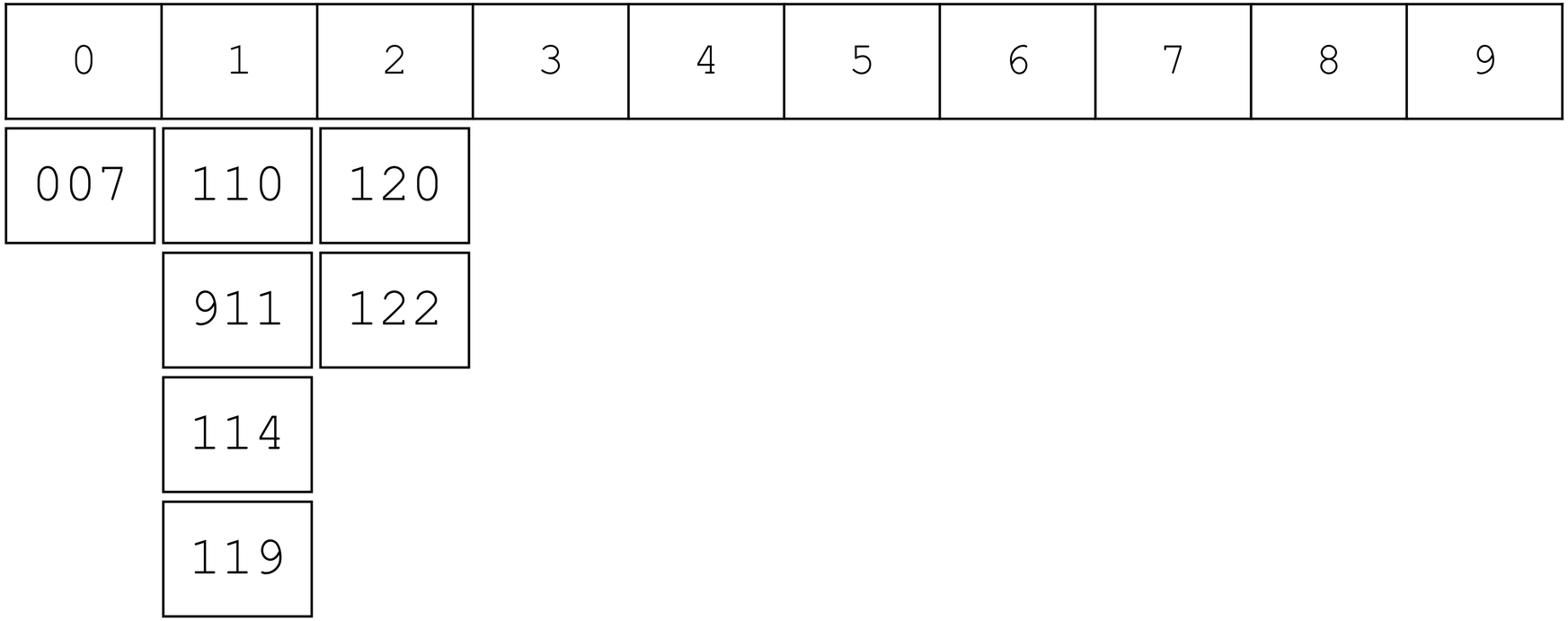
[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119



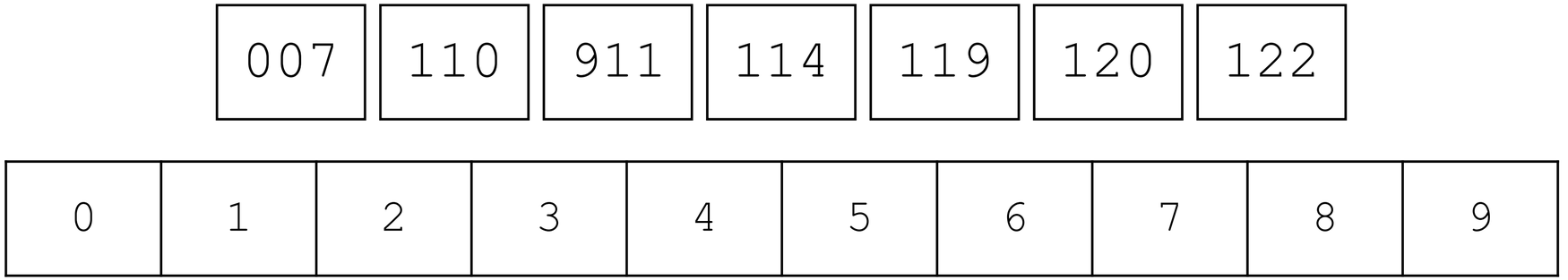
[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119



真题解析

[2013] 11. 对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是_____。

A. 007, 110, 119, 114, 911, 120, 122

B.007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
9	1	4	13	7	8	20	23	15

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
9	1	4	13	7	8	20	23	15

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
9	1	4	13	7	8	20	23	15

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
9	1	4	13	7	8	20	23	15

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
9	1	4	13	7	8	20	23	15

真题解析

[2014] 10. 用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是_____。

A. 2　　B. 3　　C. 4　　D. 5

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

_____ X _____

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

x _____

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

x _____ y _____

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

x _____ y _____

_____ x

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

x _____ y _____

_____ y _____ x

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

___ z ___ x ___ y ___

x _____ y _____

_____ y _____ x

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2014] 11. 下列选项中，不可能是快速排序第 2 趟排序结果的是_____。

A. 2, 3, 5, 4, 6, 7, 9　　B. 2, 7, 5, 6, 4, 3, 9　　C. 3, 2, 5, 4, 7, 6, 9　　D. 4, 2, 3, 5, 7, 6, 9

真题解析

[2015] 9. 下列排序算法中，元素的移动次数与关键字的初始排列次序无关的是_____。

A. 直接插入排序　　B. 起泡排序　　C. 基数排序　　D. 快速排序

真题解析

[2015] 9. 下列排序算法中，元素的移动次数与关键字的初始排列次序无关的是_____。

A. 直接插入排序　　B. 起泡排序　　C. 基数排序　　D. 快速排序

真题解析

[2015] 11. 希尔排序的组内排序采用的是_____。

A. 直接插入排序　　B. 折半插入排序　　C. 快速排序　　D. 归并排序

真题解析

[2015] 11. 希尔排序的组内排序采用的是_____。

A. 直接插入排序　　B. 折半插入排序　　C. 快速排序　　D. 归并排序

真题解析

[2017] 10. 在内部排序时，若选择了归并排序而没有选择插入排序，则可能的理由是_____。

I. 归并排序的程序代码更短　　II. 归并排序的占用空间更少

III. 归并排序的运行效率更高

A. 仅 II　　B. 仅 III　　C. 仅 I、II　　D. 仅 I、III

真题解析

[2017] 10. 在内部排序时，若选择了归并排序而没有选择插入排序，则可能的理由是_____。

I. 归并排序的程序代码更短　　II. 归并排序的占用空间更少

III. 归并排序的运行效率更高

A. 仅 II　　B. 仅 III　　C. 仅 I、II　　D. 仅 I、III

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

第一趟排序后结果：1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

第一趟排序后结果：1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

第一趟排序后结果：1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

第一趟排序后结果：1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9

真题解析

[2018] 10. 对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是_____。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

真题解析

[2019] 7. 选择一个排序算法时，除算法的时空效率，下列因素中，还需要考虑的是____。

I. 数据的规模　　II. 数据的存储方式　　III. 算法的稳定性　　IV. 数据的初始状态

A. 仅 III

B. 仅 I、II

C. 仅 II、III、IV

D. I、II、III、IV

真题解析

[2019] 7. 选择一个排序算法时，除算法的时空效率，下列因素中，还需要考虑的是____。

I. 数据的规模　　II. 数据的存储方式　　III. 算法的稳定性　　IV. 数据的初始状态

A. 仅 III

B. 仅 I、II

C. 仅 II、III、IV

D. I、II、III、IV

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2019] 10. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是____。

A. 5, 2, 16, 12, 28, 60, 32, 72

B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60

D. 5, 2, 12, 28, 16, 32, 72, 60

真题解析

[2020] 11.对大部分元素已有序的数组进行排序时，直接插入排序比简单选择排序效率更高，其原因是_____。

I. 直接插入排序过程中元素之间的比较次数更少

II. 直接插入排序过程中所需要的辅助空间更少

III. 直接插入排序过程中元素的移动次数更少

A. 仅 I　　B. 仅 III　　C. 仅 I、II　　D. I、II、III

真题解析

[2020] 11.对大部分元素已有序的数组进行排序时，直接插入排序比简单选择排序效率更高，其原因是_____。

I. 直接插入排序过程中元素之间的比较次数更少

II. 直接插入排序过程中所需要的辅助空间更少

III. 直接插入排序过程中元素的移动次数更少

A. 仅 I　　B. 仅 III　　C. 仅 I、II　　D. I、II、III

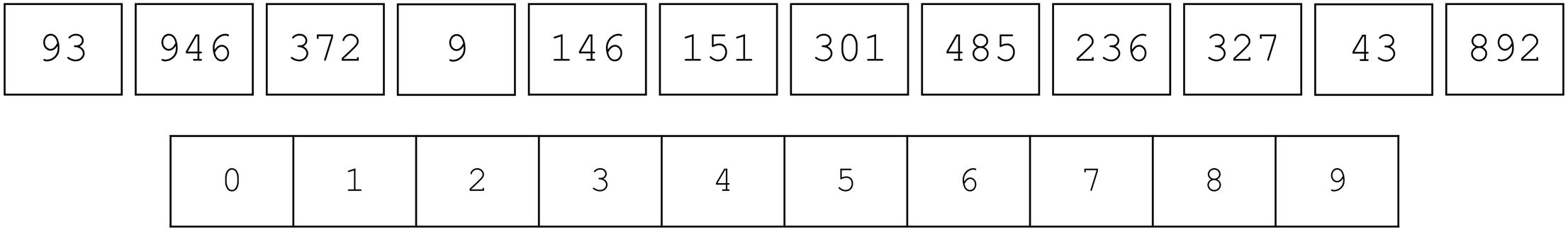
真题解析

[2021] 10.设数组 S[] = {93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第 1 趟分配、收集后，元素 372 之前、之后紧邻的元素分别是_____。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301

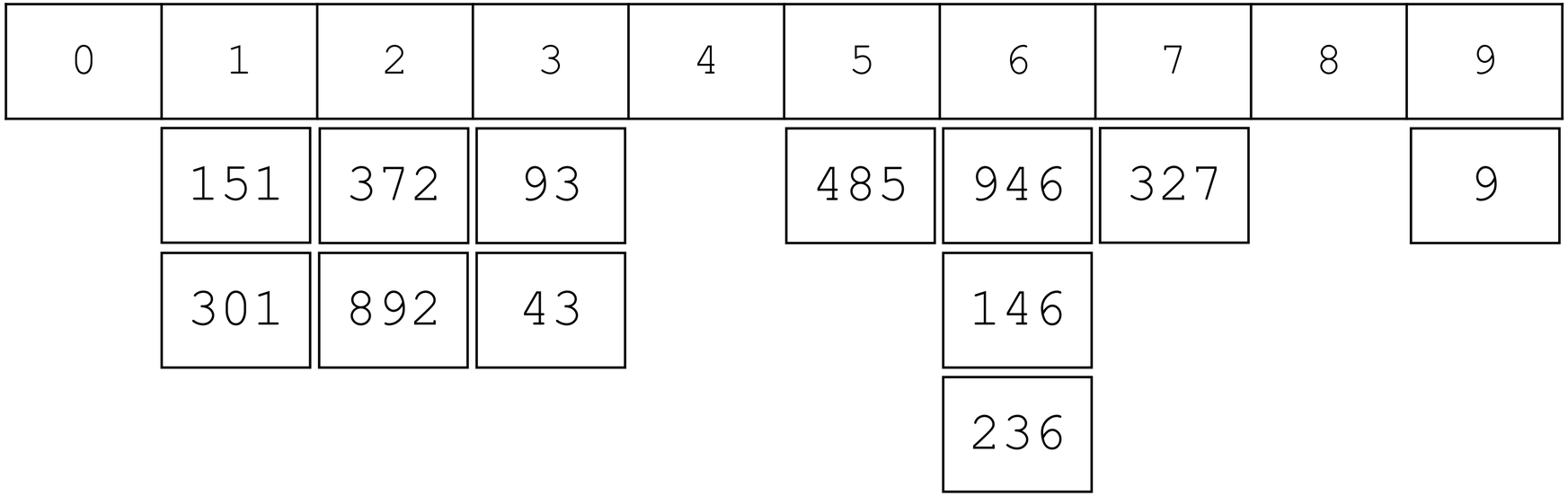
[2021] 10.设数组 S[] = {93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第 1 趟分配、收集后，元素 372 之前、之后紧邻的元素分别是_____。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301



[2021] 10.设数组 S[] = {93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第 1 趟分配、收集后，元素 372 之前、之后紧邻的元素分别是_____。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301



真题解析

[2021] 10.设数组 S[] = {93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第 1 趟分配、收集后，元素 372 之前、之后紧邻的元素分别是_____。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301

外部排序

基本思想

所需时间

多路平衡归并

转换-选择排序

最佳归并树

外部排序

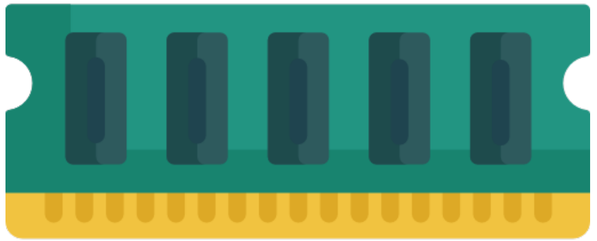
冒泡排序、插入排序、归并排序、快速排序等，这些算法有一个共同特点——它们都假设数据可以全部装入内存中进行处理。

然而，在实际应用中经常会遇到这样的场景：数据量非常庞大，远远超过了主存所能容纳的范围，比如排序一个包含数十亿条记录的日志文件，这时候常规的排序算法就无能为力了。

为了解决这一问题，需要采用外部排序（External Sorting）技术。

外部排序

外部排序是一种借助外部存储设备（如磁盘）完成排序的策略。它的基本思想是：将大文件分成多个能装入内存的小块，每一块在内存中完成排序，然后再将这些有序小块归并为一个整体有序文件。



内部排序



外部排序

外部排序

假设有 5400 条记录$A_1, A_2, A_3, \ldots, A_{5400}$需要排序，而计算机可用内存最多可以容纳 900 条记录。

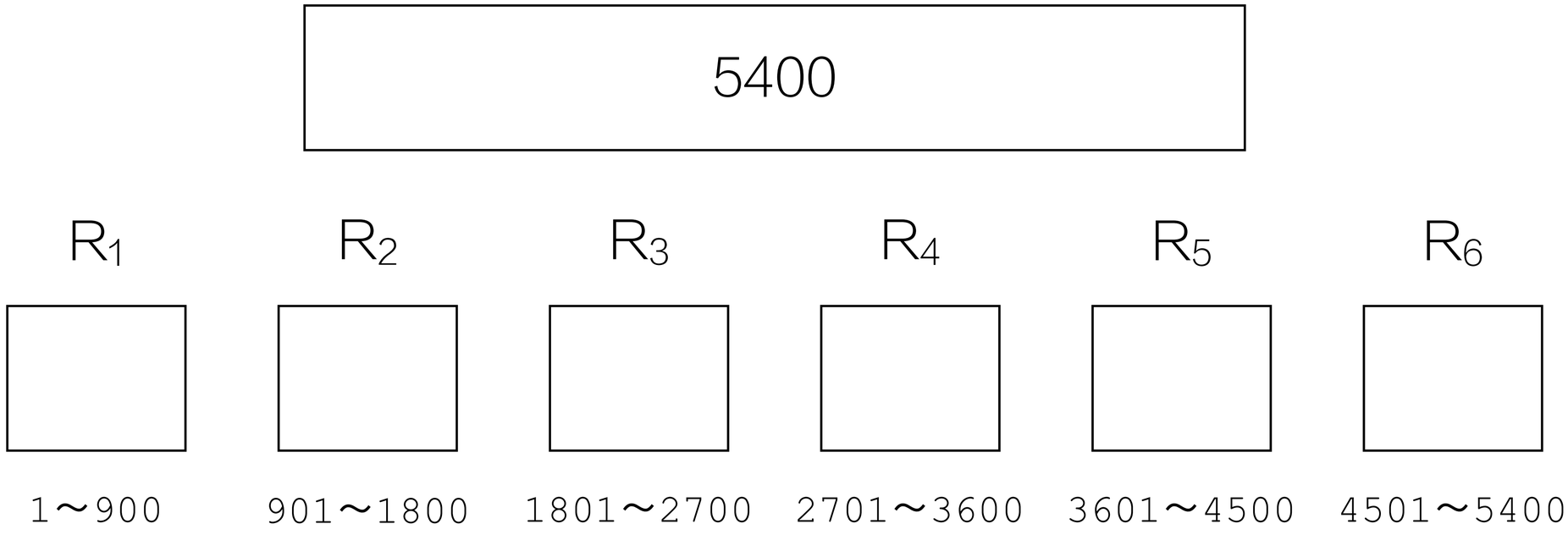
外部排序

假设有 5400 条记录$A_1, A_2, A_3, \ldots, A_{5400}$需要排序，而计算机可用内存最多可以容纳 900 条记录。



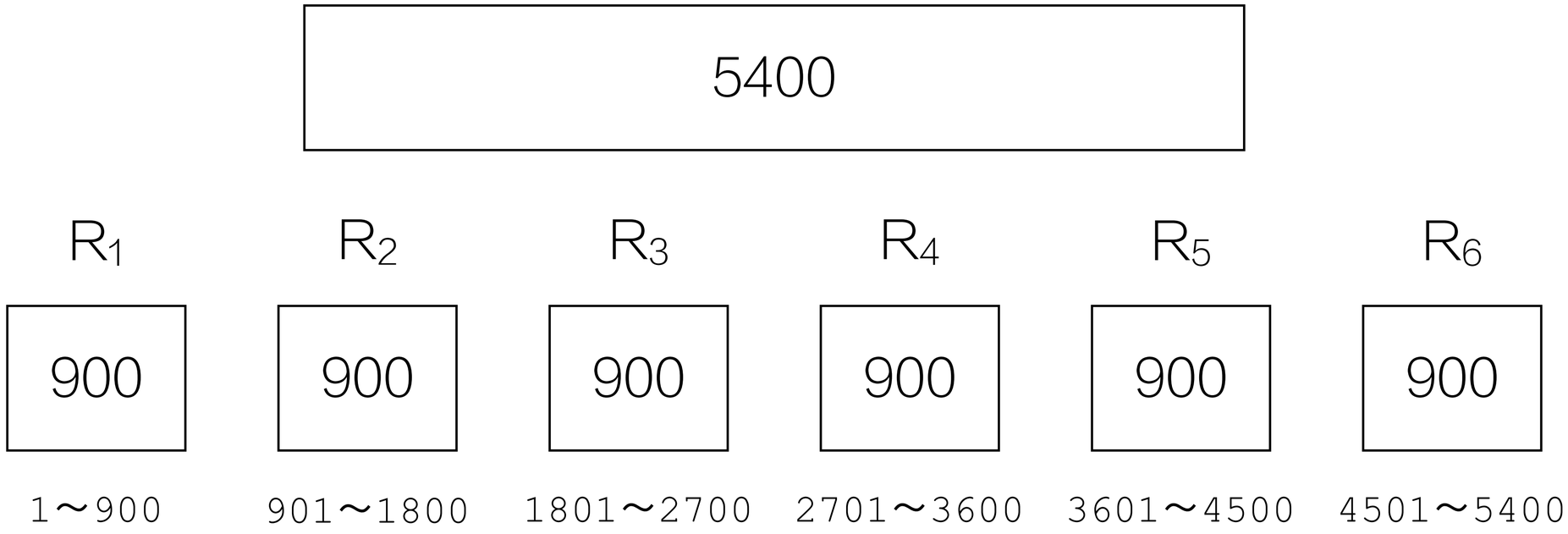
外部排序

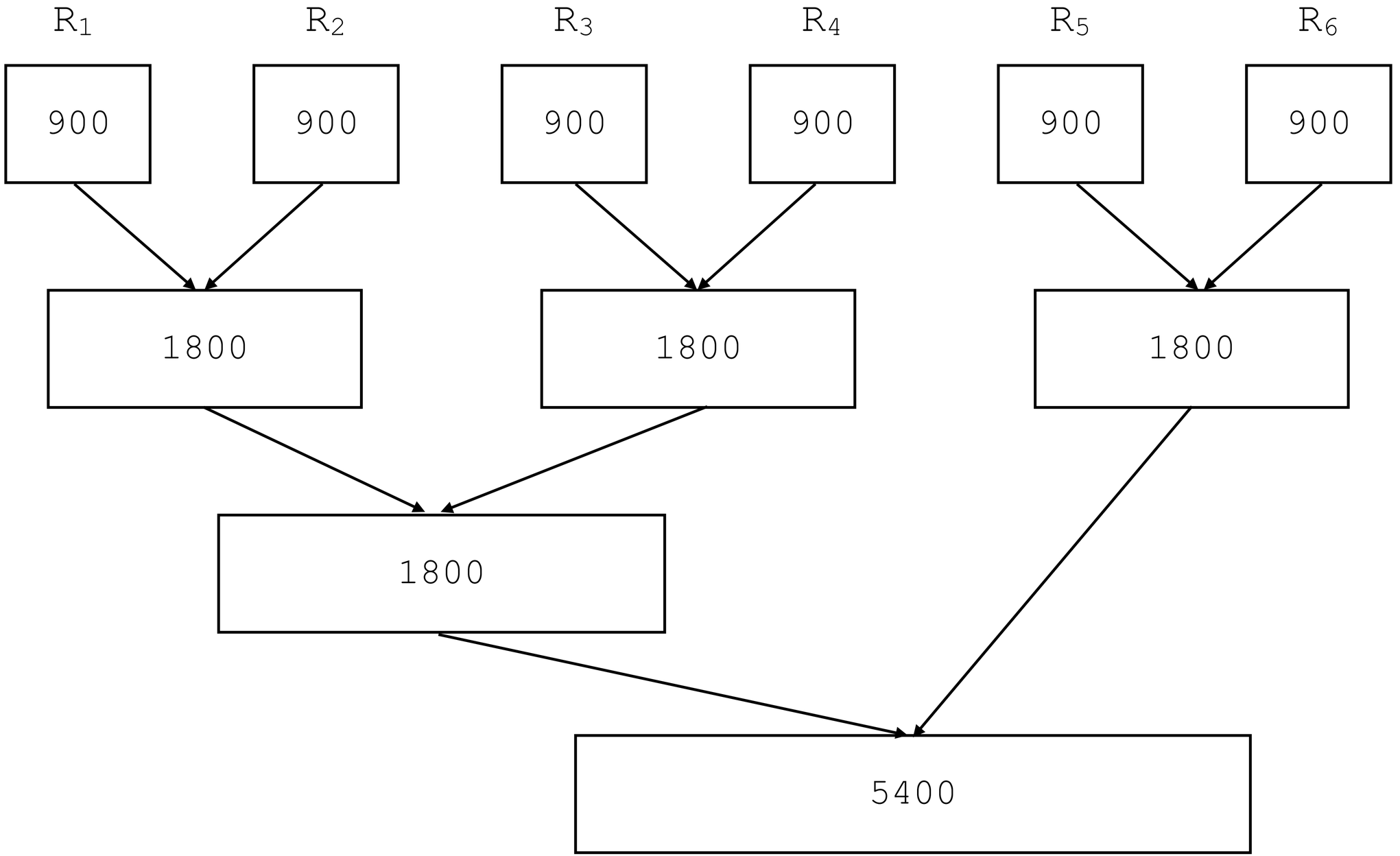
假设有 5400 条记录$A_1, A_2, A_3, \ldots, A_{5400}$需要排序，而计算机可用内存最多可以容纳 900 条记录。



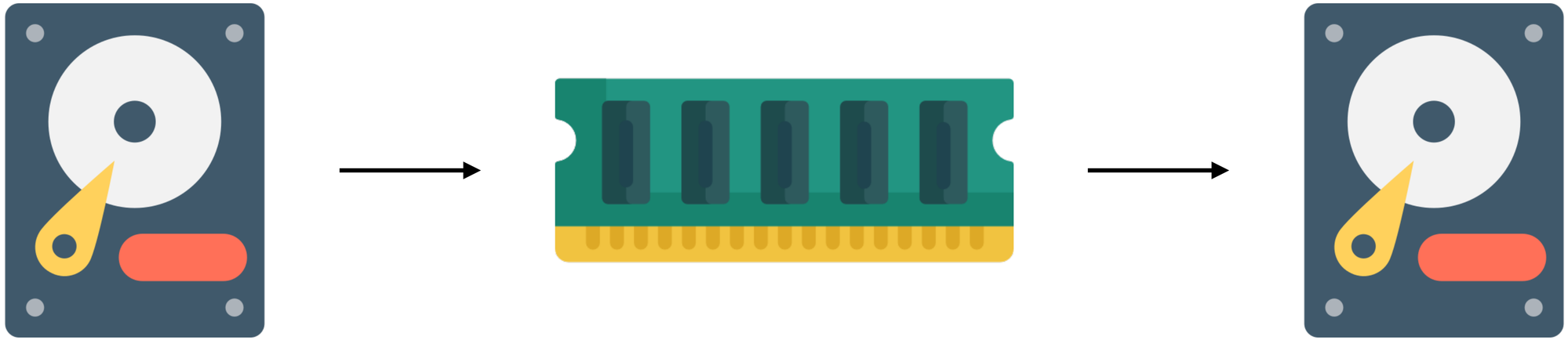
外部排序

假设有 5400 条记录$A_1, A_2, A_3, \ldots, A_{5400}$需要排序，而计算机可用内存最多可以容纳 900 条记录。

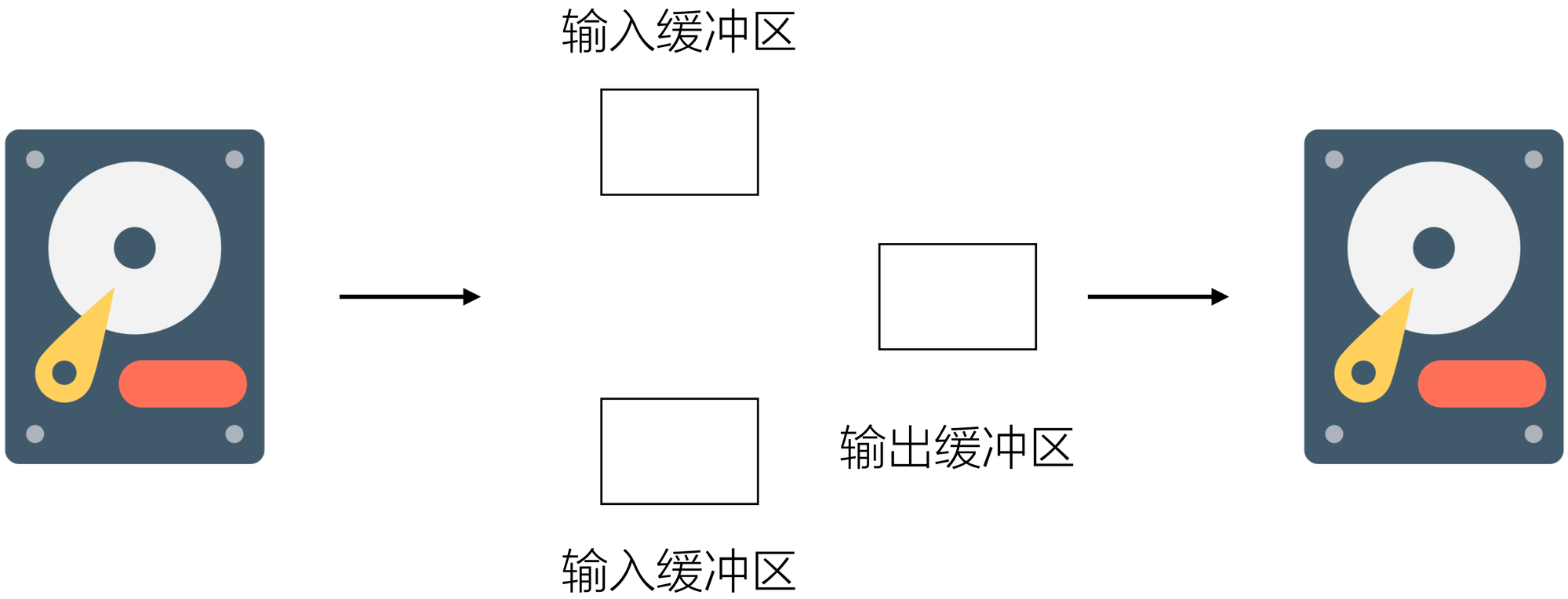



R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
1800
1800
1800
1800
5400

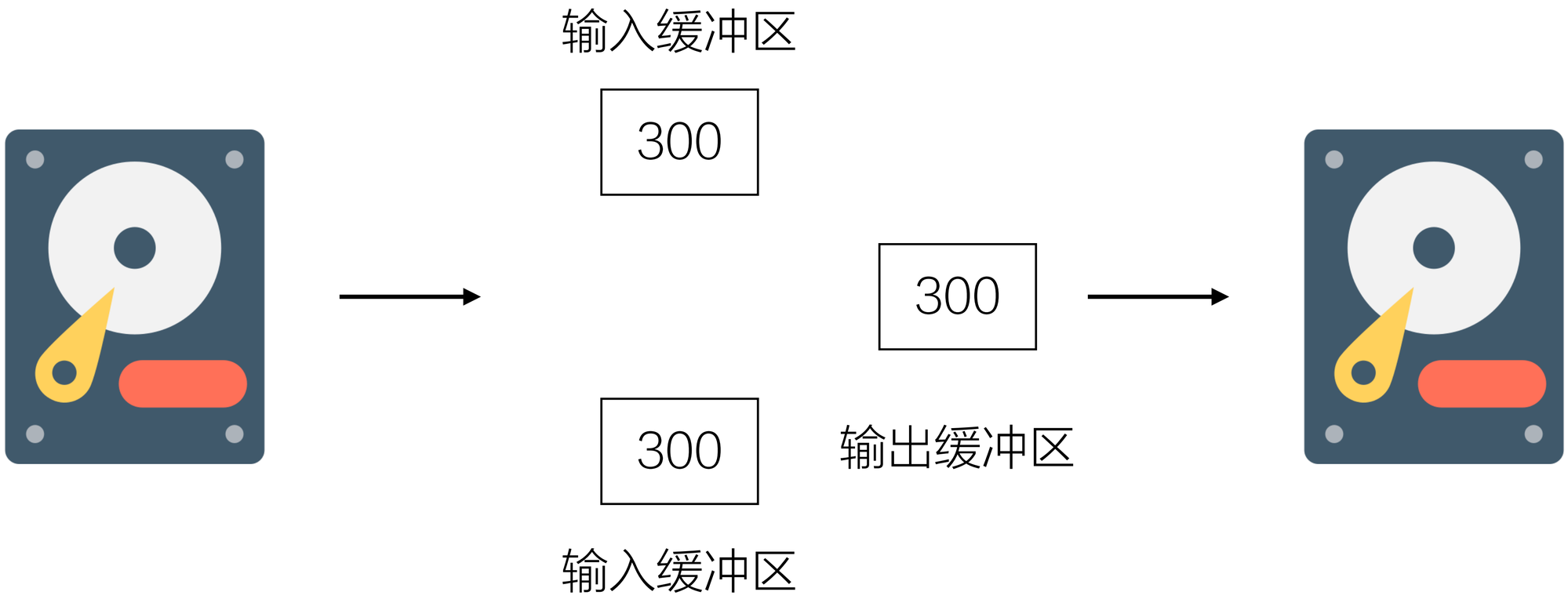
外部排序



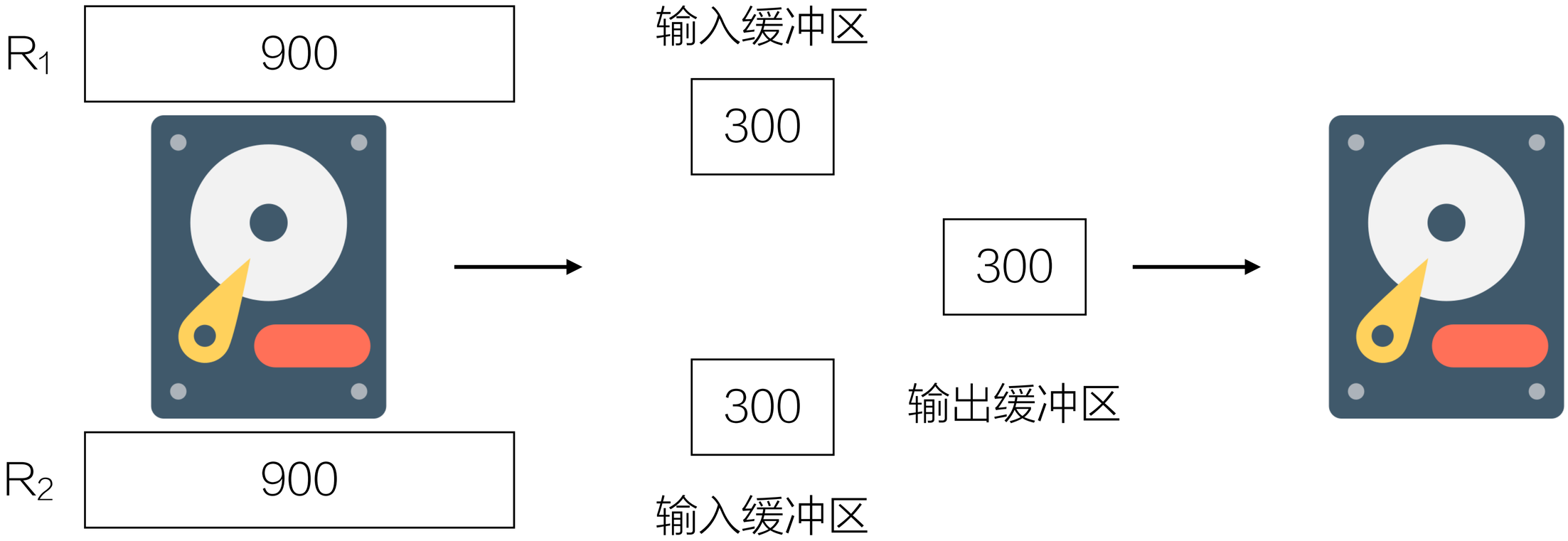
外部排序



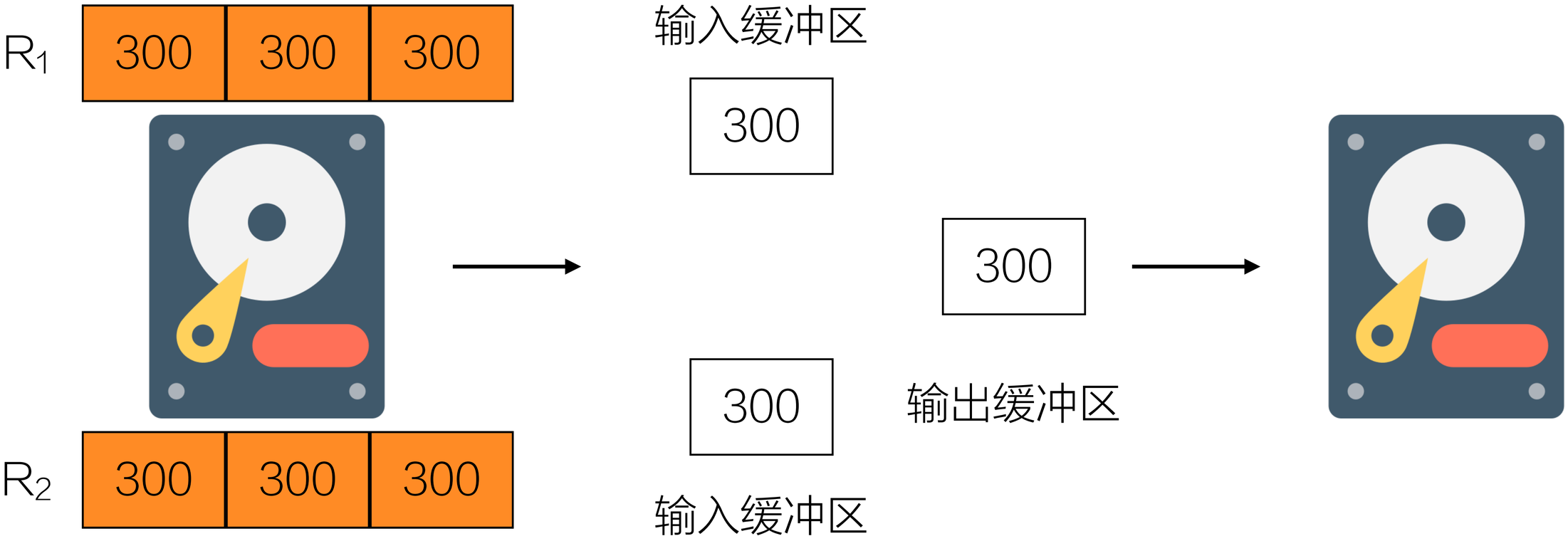
外部排序



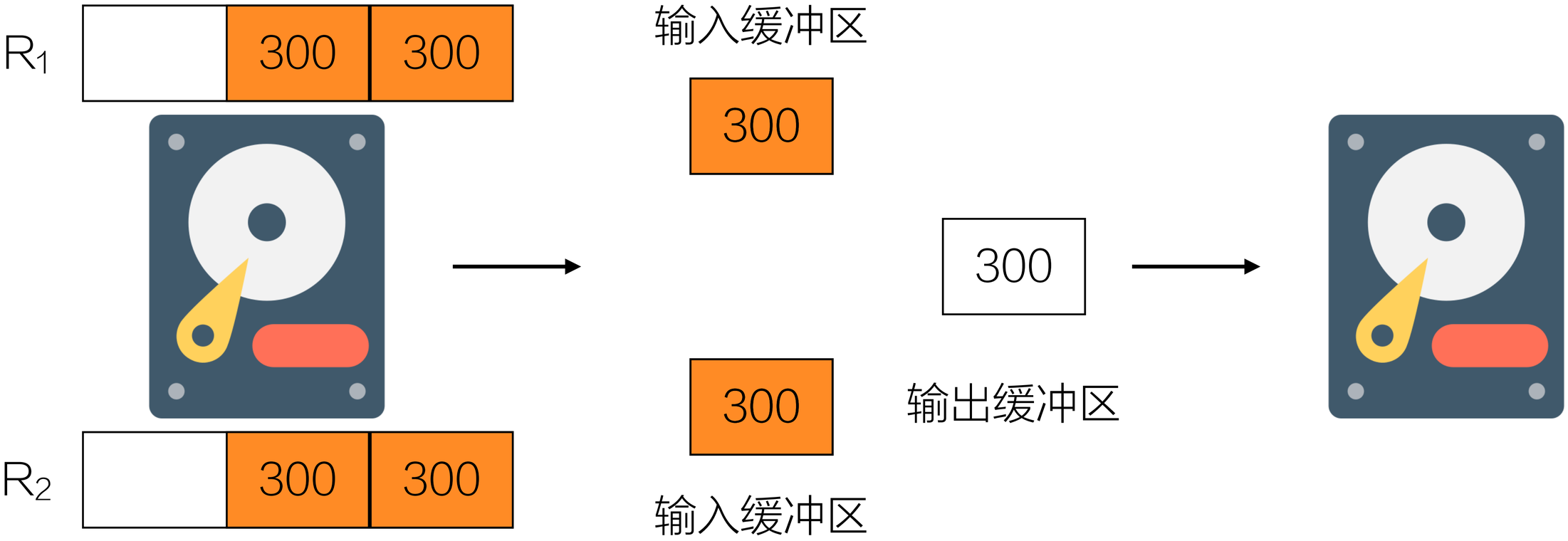
外部排序



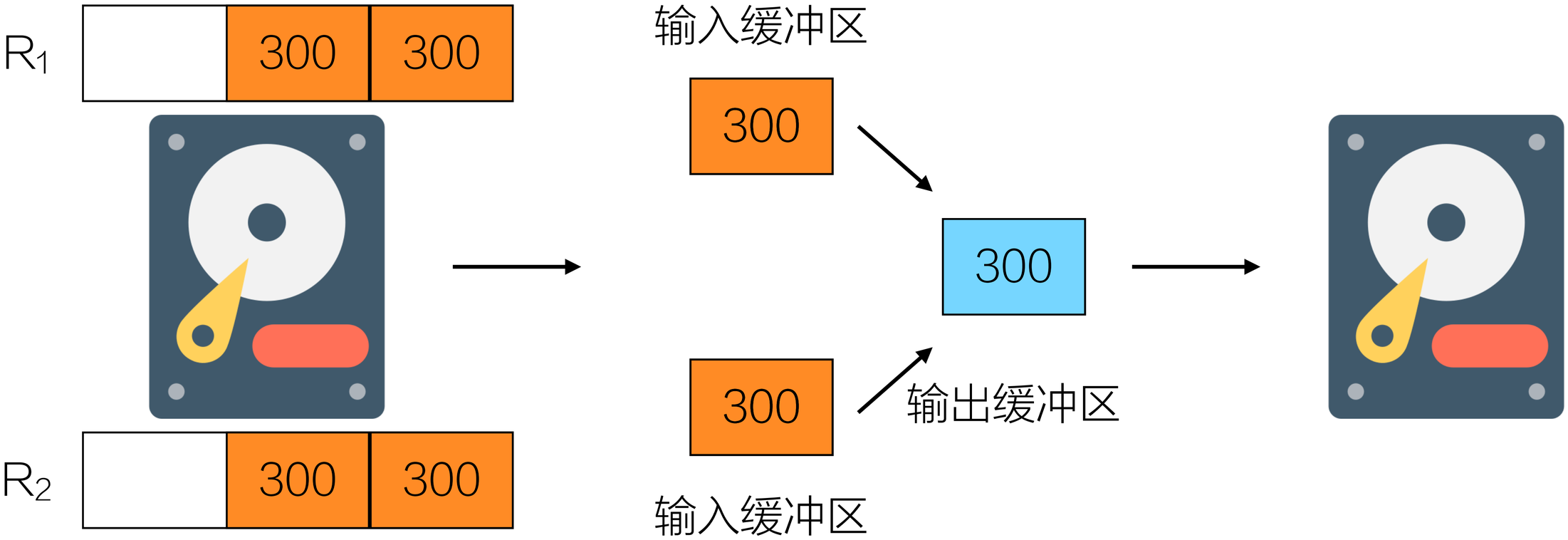
外部排序



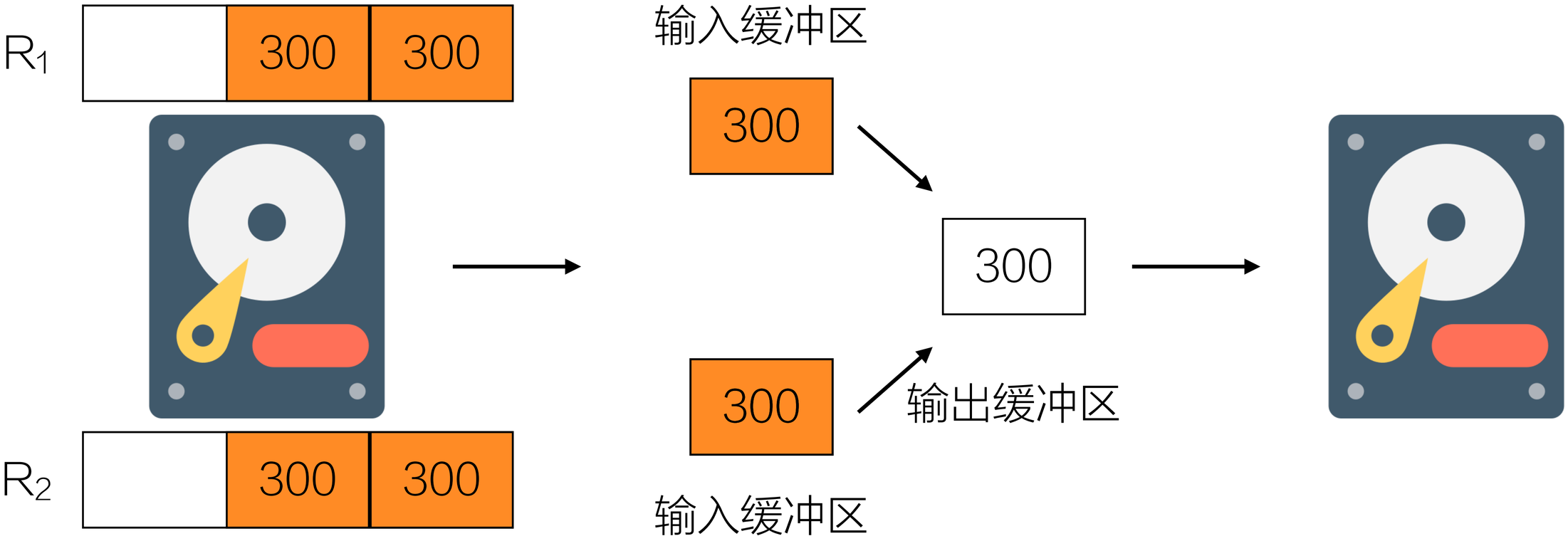
外部排序



外部排序



外部排序



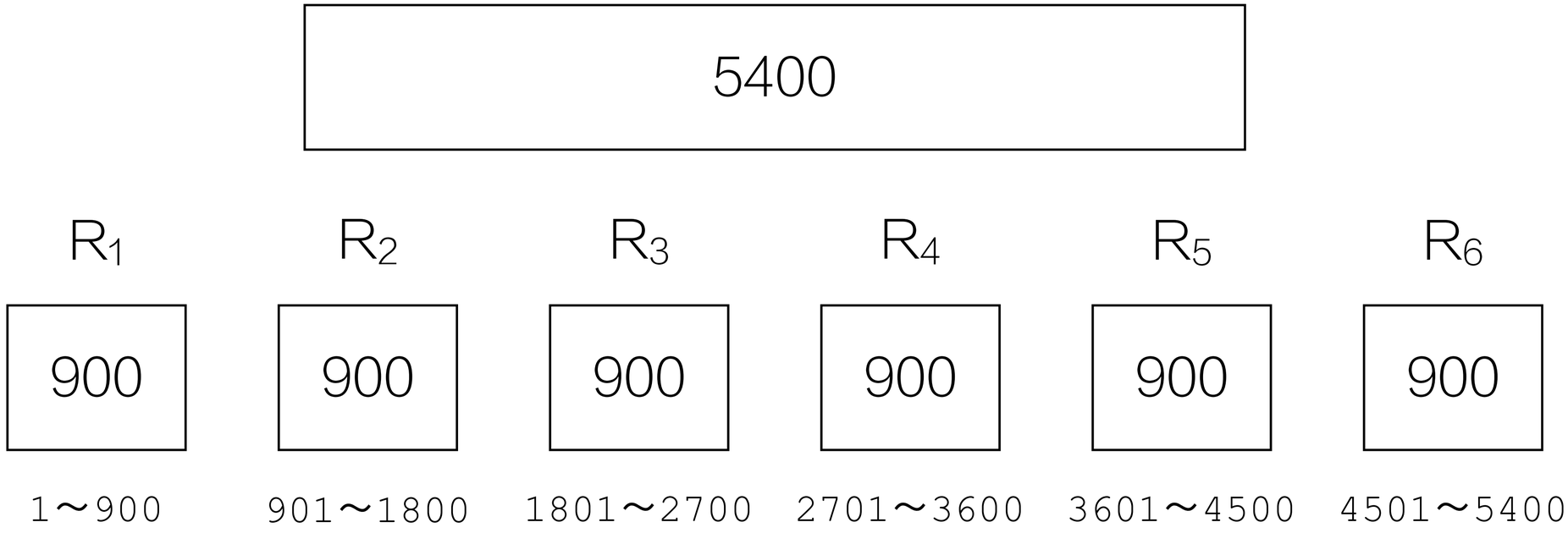
外部排序所需时间计算

$$\text{外部排序所需时间} = \begin{array}{c} \text{内部排序时间} + \text{归并排序时间} \\ + \\ (\text{磁盘与内存 读 / 写 时间}) \end{array}$$

外部排序所需时间计算-内部排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

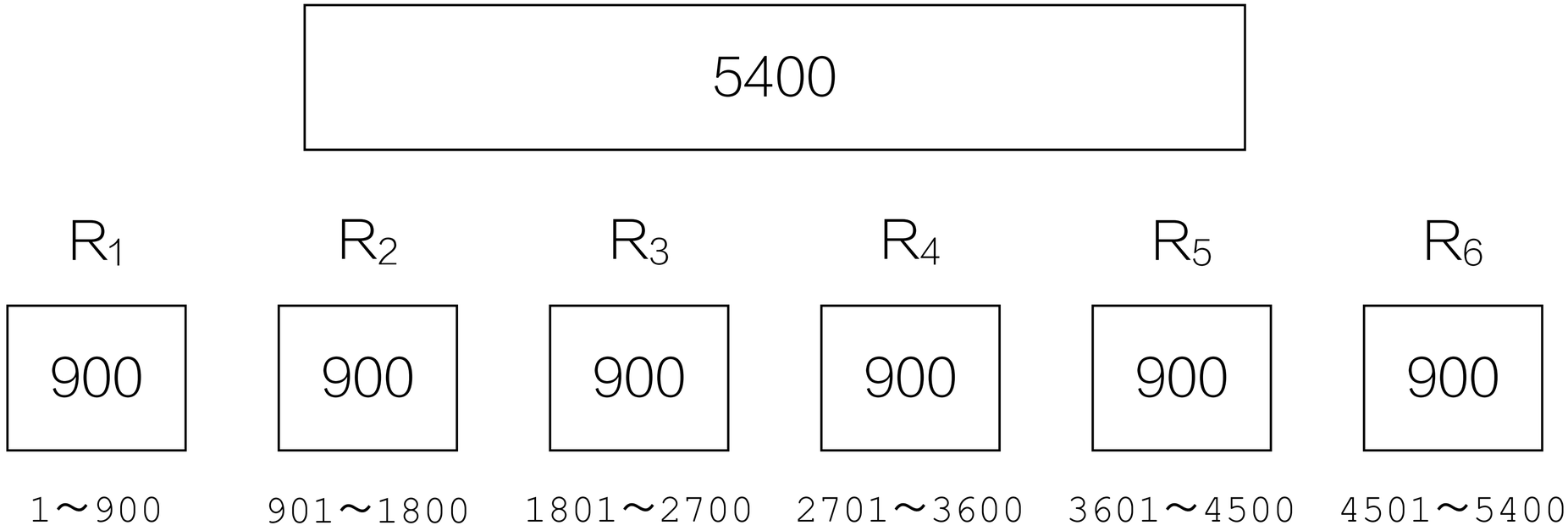
t_s：对 900 个关键字进行内部排序所需的时间



外部排序所需时间计算-内部排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

t_s：对 900 个关键字进行内部排序所需的时间

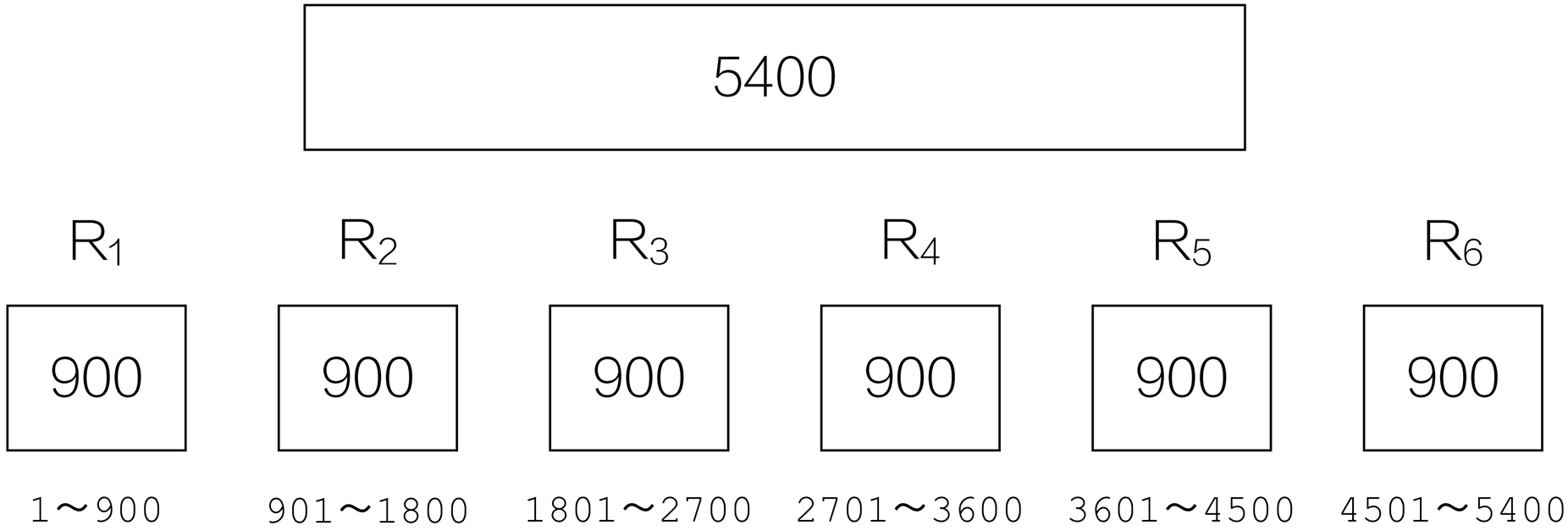


读：$18t_{IO}$

外部排序所需时间计算-内部排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

t_S：对 900 个关键字进行内部排序所需的时间

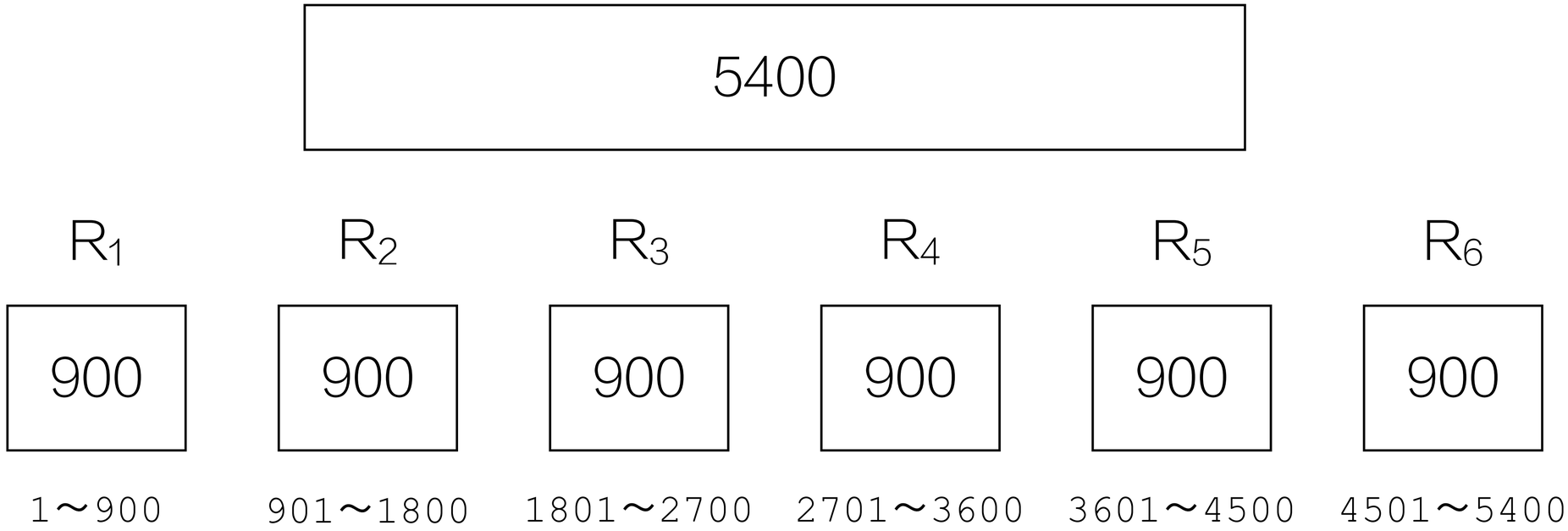


读：$18t_{IO}$　内部排序：$6t_S$

外部排序所需时间计算-内部排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

t_S：对 900 个关键字进行内部排序所需的时间

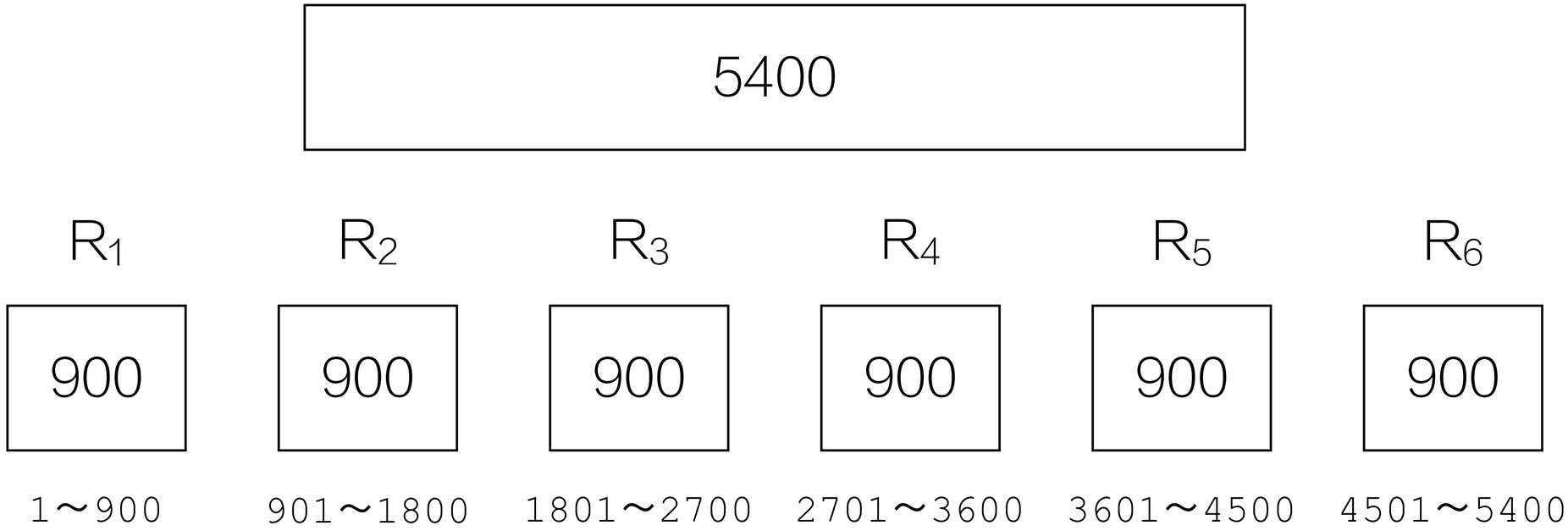


读：$18t_{IO}$　内部排序：$6t_S$　写：$18t_{IO}$

外部排序所需时间计算-内部排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

t_s：对 900 个关键字进行内部排序所需的时间

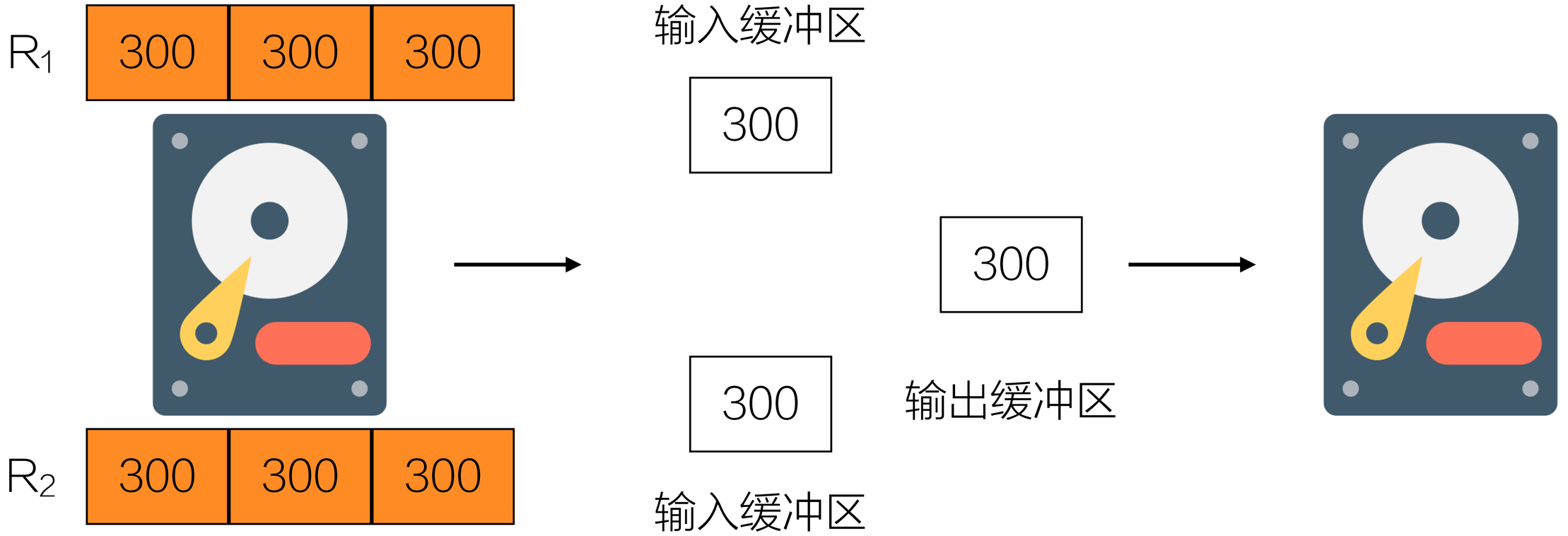


$36t_{IO} + 6t_s$

外部排序所需时间计算-归并排序时间

t_{IO}：读 / 写 300 个关键字所需的时间

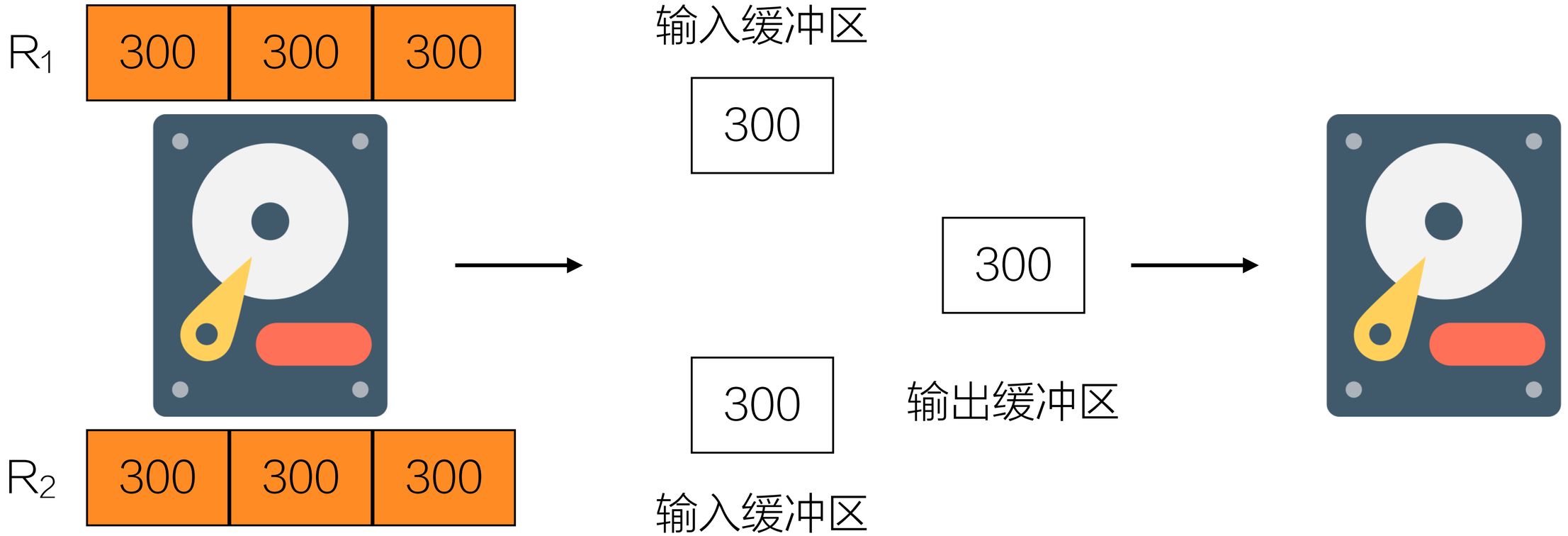
n t_m：将 n 个关键字从输入缓冲区归并到输出缓冲区所需的时间



外部排序所需时间计算-归并排序时间

t_{IO}：读 / 写 300 个关键字所需的时间　读：$6t_{IO}$

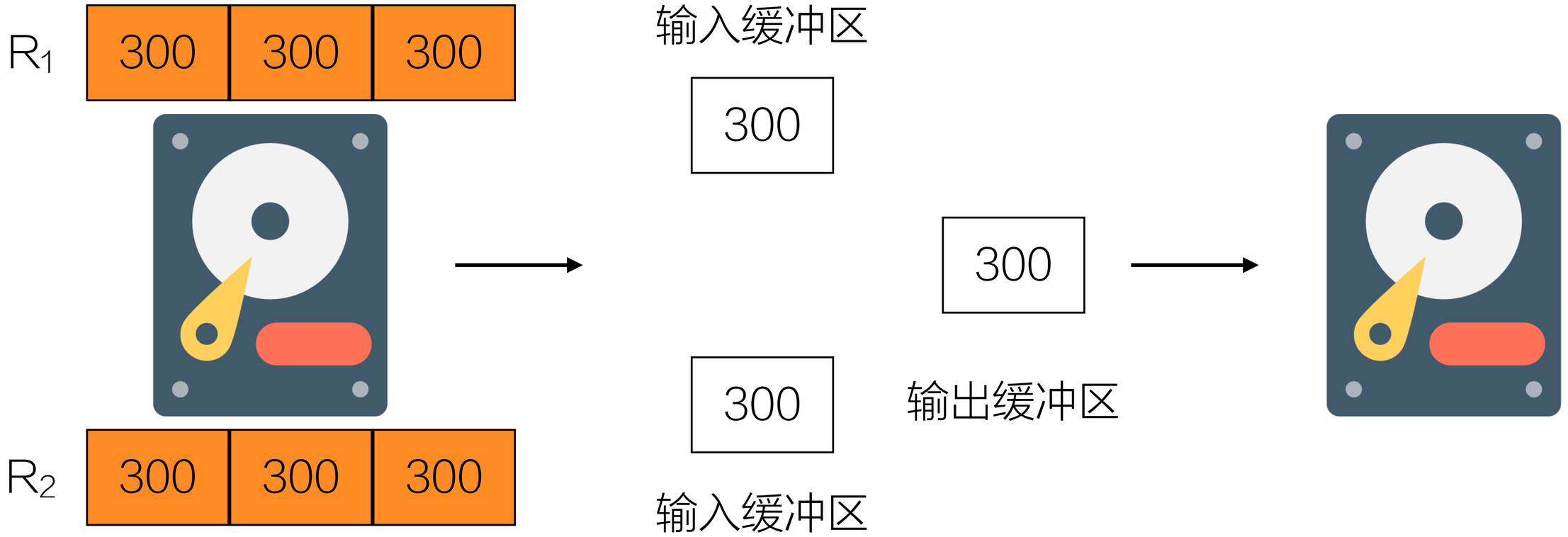
n t_m：将 n 个关键字从输入缓冲区归并到输出缓冲区所需的时间



外部排序所需时间计算-归并排序时间

t_{IO}：读 / 写 300 个关键字所需的时间　读：$6t_{IO}$　输入-> 输出缓冲区：$1800t_m$

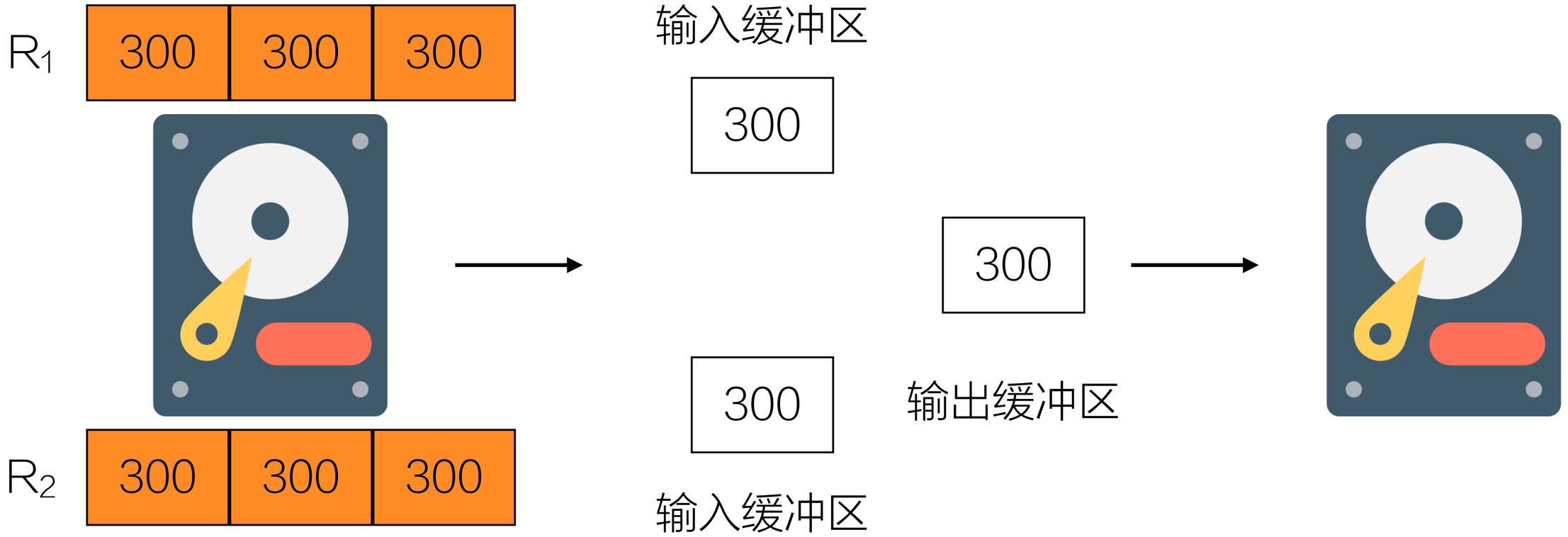
n t_m：将 n 个关键字从输入缓冲区归并到输出缓冲区所需的时间



外部排序所需时间计算-归并排序时间

t_{IO}：读 / 写 300 个关键字所需的时间　读：$6t_{io}$　输入-> 输出缓冲区：$1800t_m$　写：$6t_{io}$

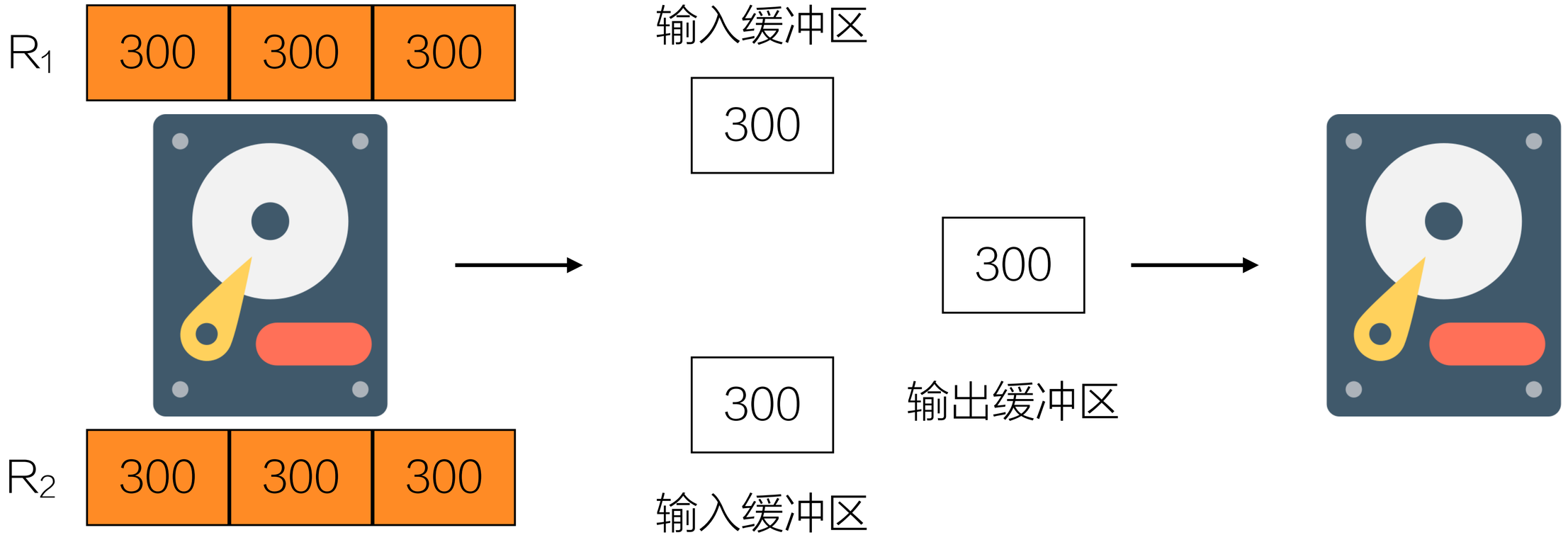
n t_m：将 n 个关键字从输入缓冲区归并到输出缓冲区所需的时间

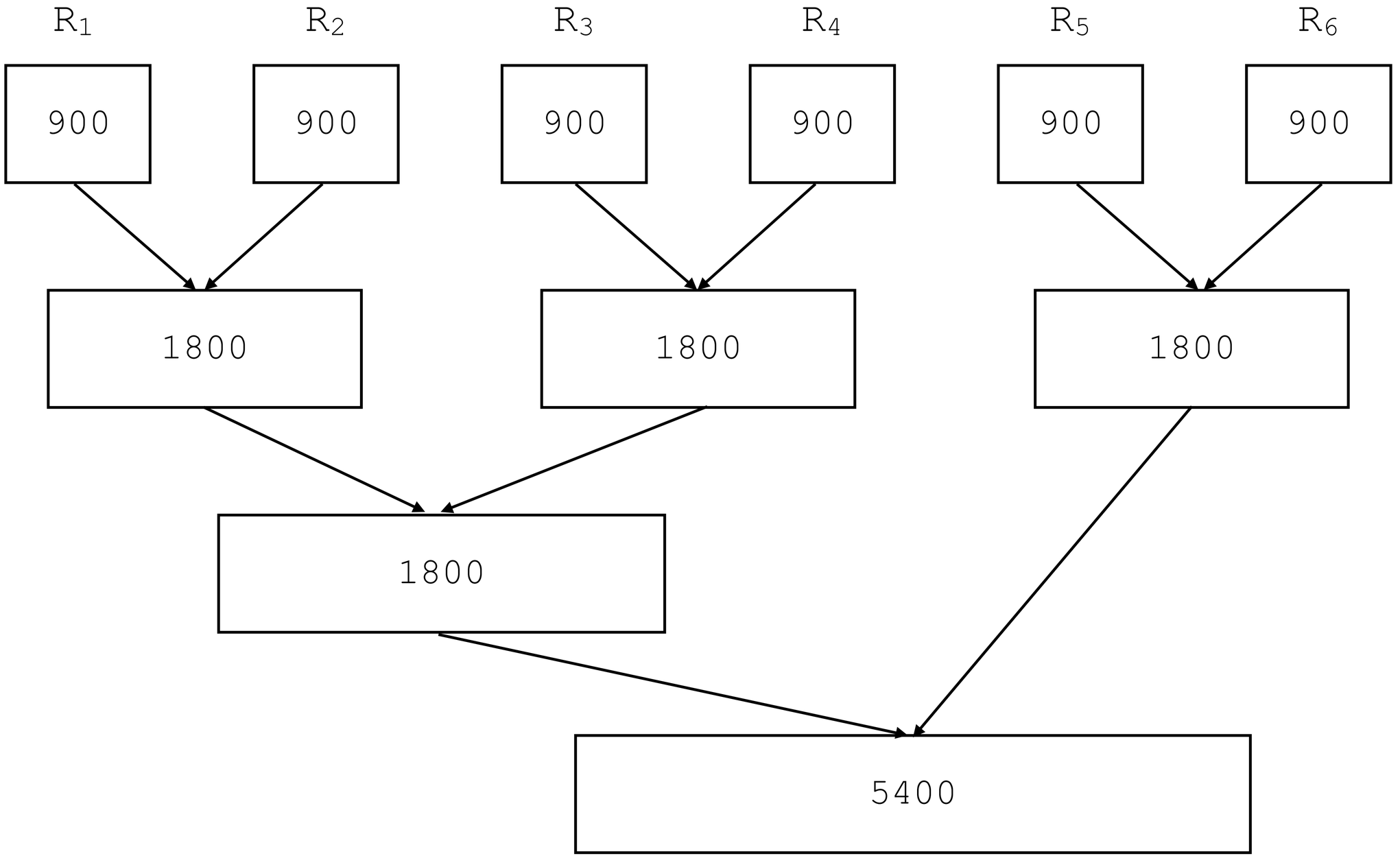


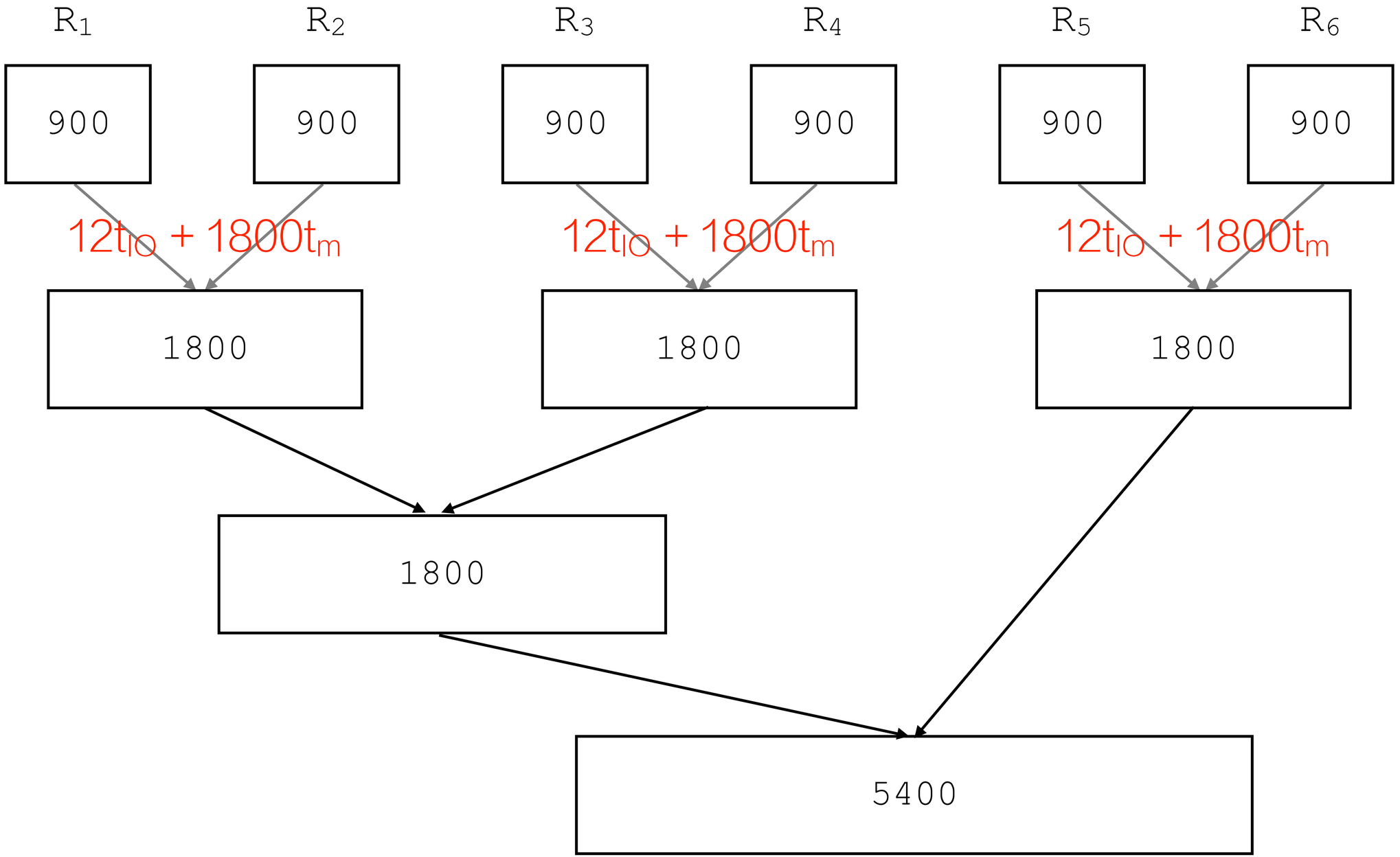
外部排序所需时间计算-归并排序时间

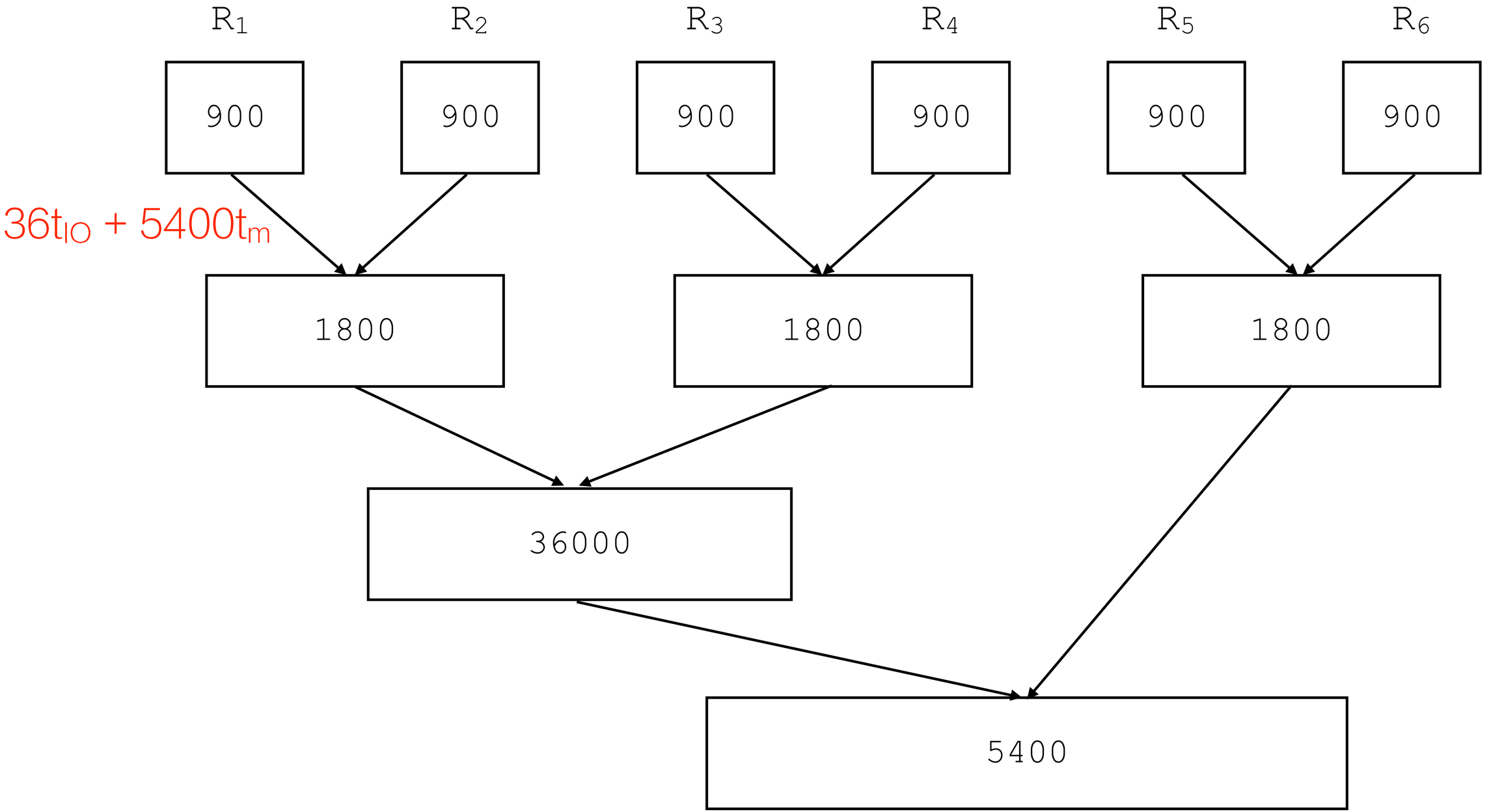
t_{IO}：读 / 写 300 个关键字所需的时间　　$12t_{IO} + 1800t_m$

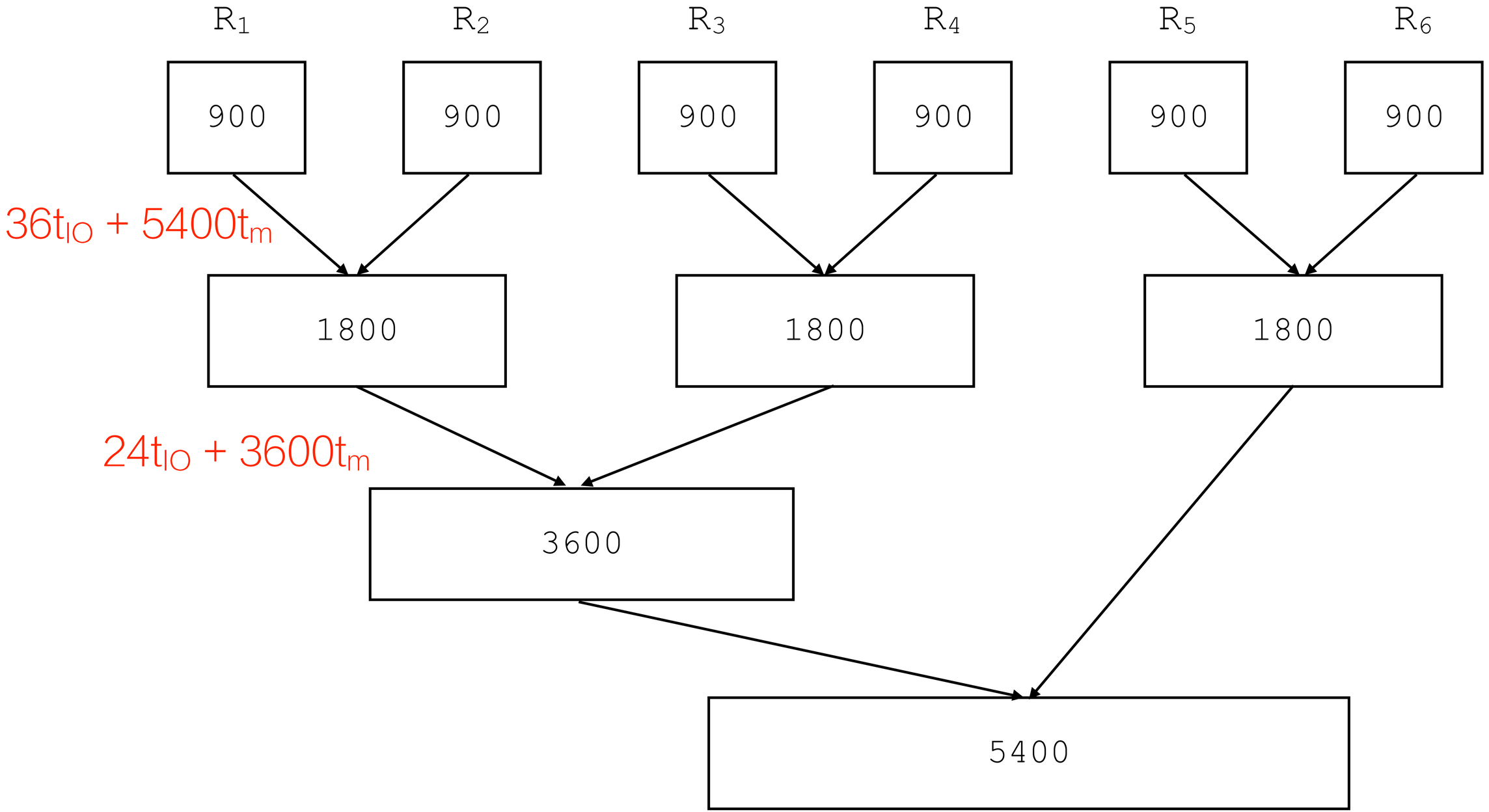
n t_m：将 n 个关键字从输入缓冲区归并到输出缓冲区所需的时间

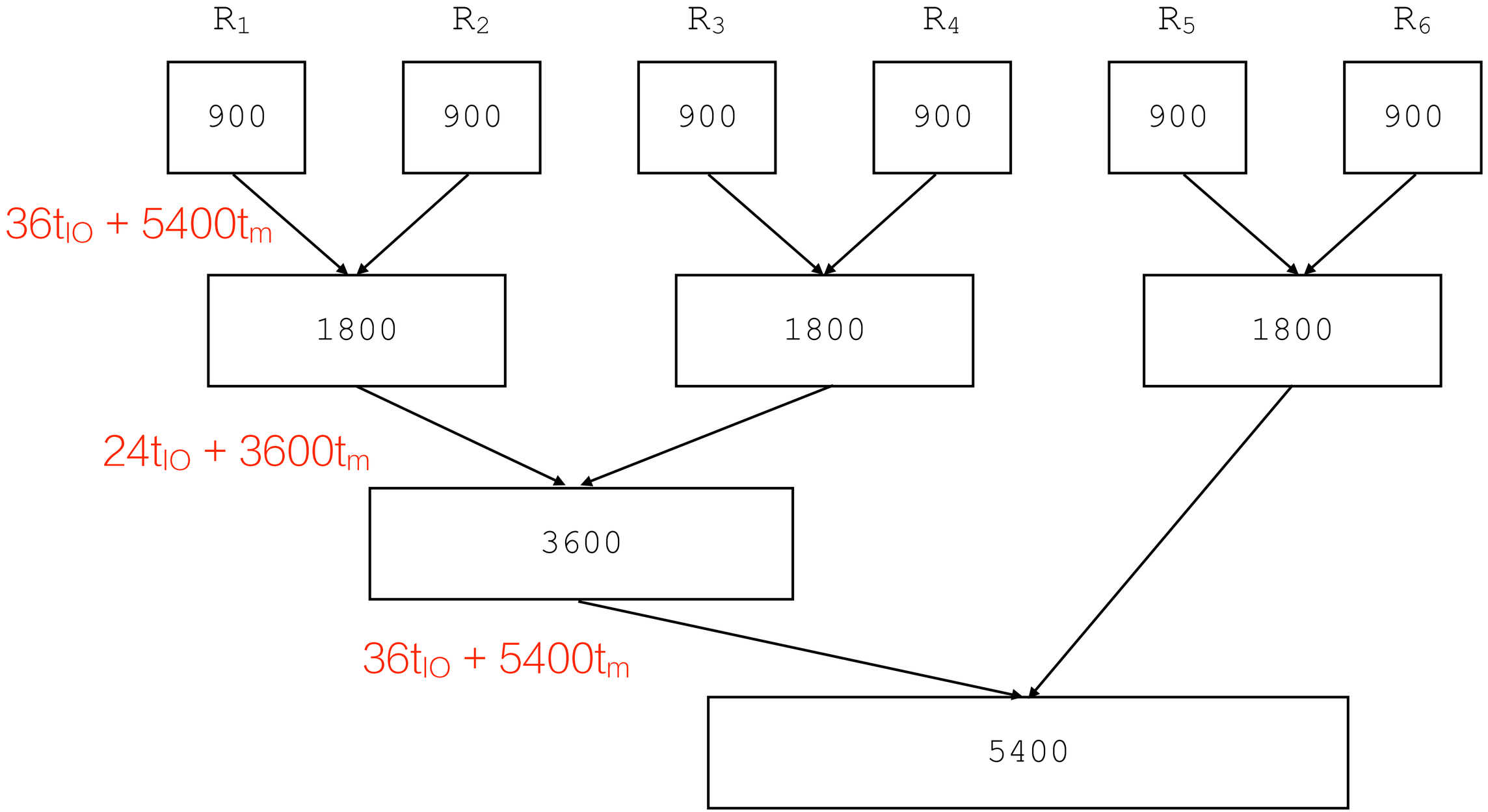


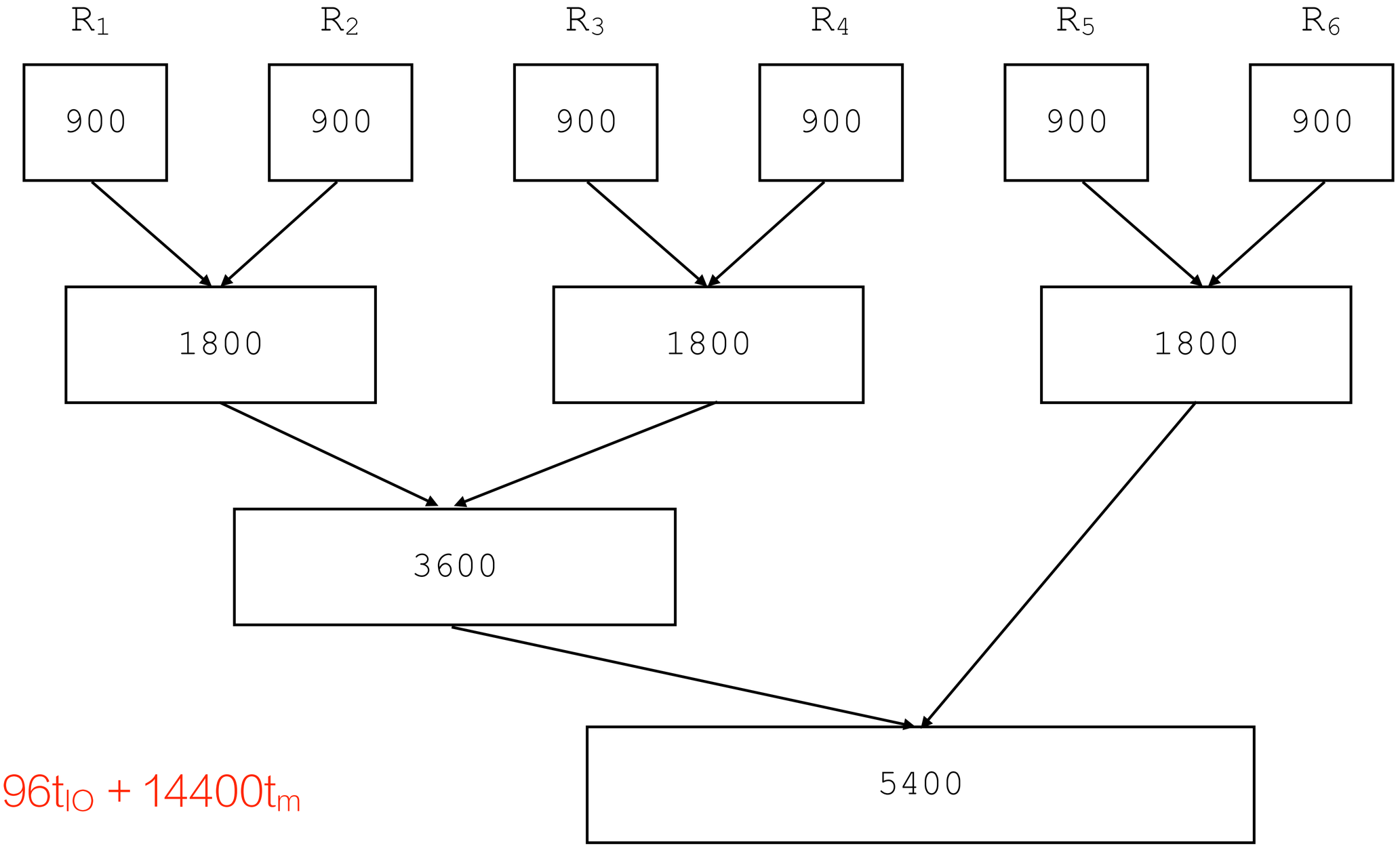

R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
1800
1800
1800
1800
5400


R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
12tIO + 1800tm
12tIO + 1800tm
12tIO + 1800tm
1800
1800
1800
1800
5400


R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
36tIO + 5400tm
1800
1800
1800
36000
5400


R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
36tIO + 5400tm
1800
1800
1800
24tIO + 3600tm
3600
5400


R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
36tIO + 5400tm
1800
1800
1800
24tIO + 3600tm
3600
36tIO + 5400tm
5400



$96t_{IO} + 14400t_m$

外部排序所需时间计算

$$\text{外部排序所需时间} = \begin{array}{c} \text{内部排序时间} + \text{归并排序时间} \\ + \\ (\text{磁盘与内存 读 / 写 时间}) \end{array}$$

外部排序所需时间计算

外部排序所需时间 = $36t_{IO} + 6t_s$ 内部排序时间 + $96t_{IO} + 14400t_m$ 归并排序时间 + （磁盘与内存 读 / 写 时间）

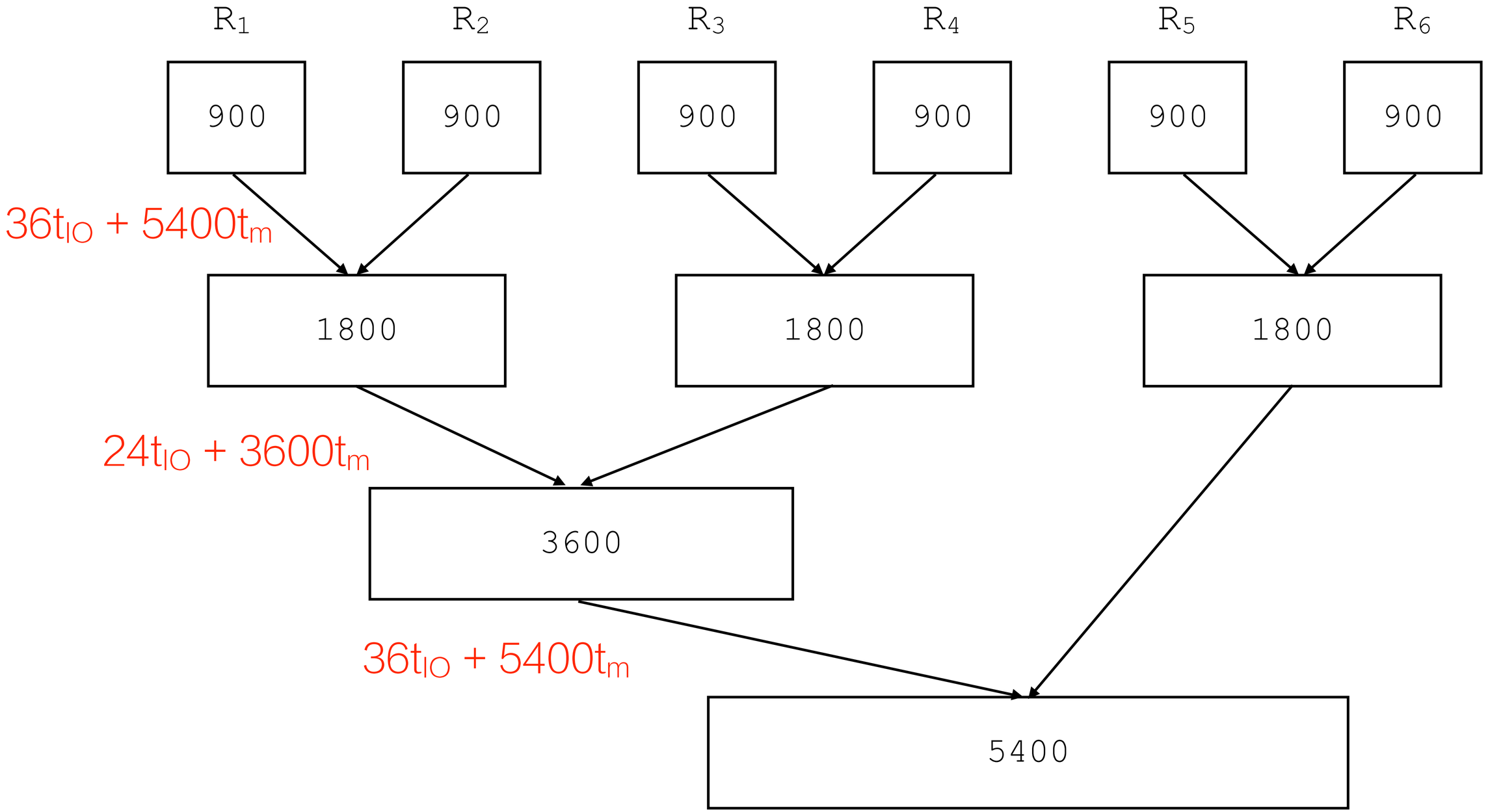
外部排序所需时间计算

$$\text{外部排序所需时间} = \underbrace{\text{内部排序时间} + \text{归并排序时间}}_{\text{（磁盘与内存 读 / 写 时间）}} = 132t_{IO} + 14400T_m + 6t_s$$

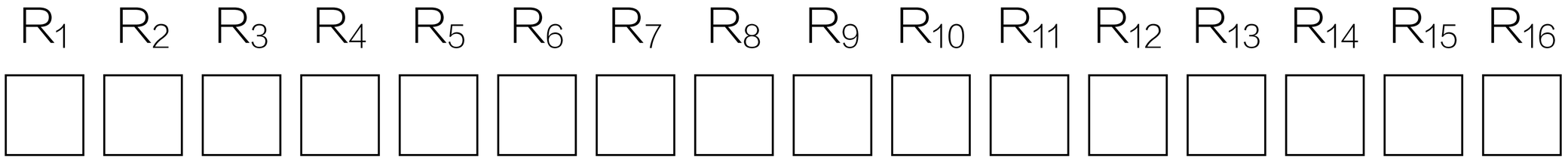
外部排序所需时间计算

$$\text{外部排序所需时间} = \underset{\text{（磁盘与内存 读 / 写 时间）}}{\text{内部排序时间} + \text{归并排序时间}} = 132t_{IO} + 14400T_m + 6t_s$$

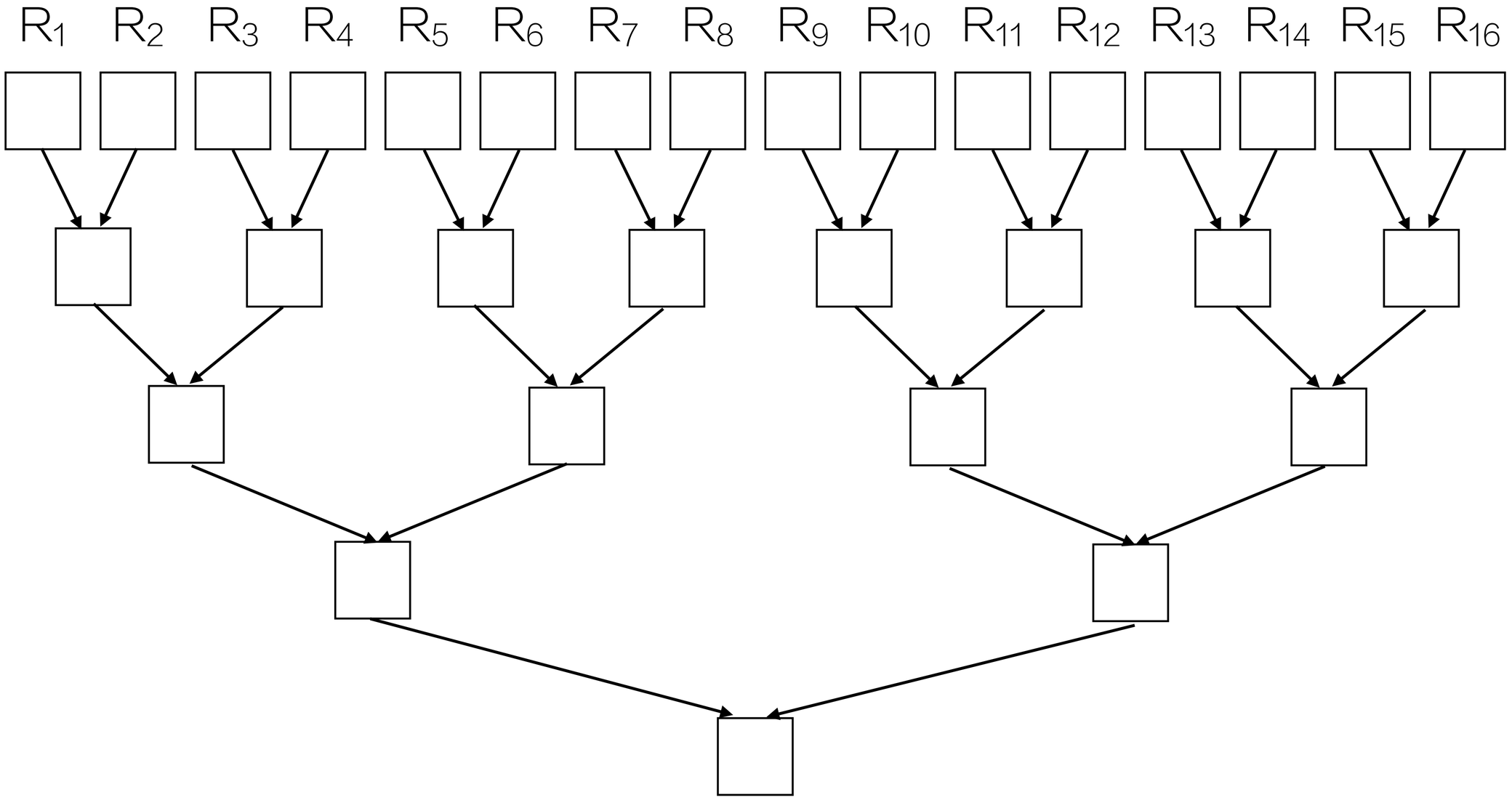
由于外存访问的速度远低于内存，一次完整的外部排序往往需要大量的磁盘读写过程，I/O 时间通常占据了整个排序过程的绝大部分


R1
R2
R3
R4
R5
R6
900
900
900
900
900
900
36tIO + 5400tm
1800
1800
1800
24tIO + 3600tm
3600
36tIO + 5400tm
5400

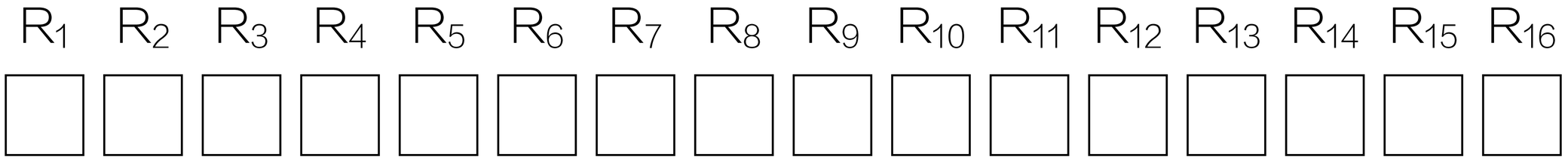
K 路归并



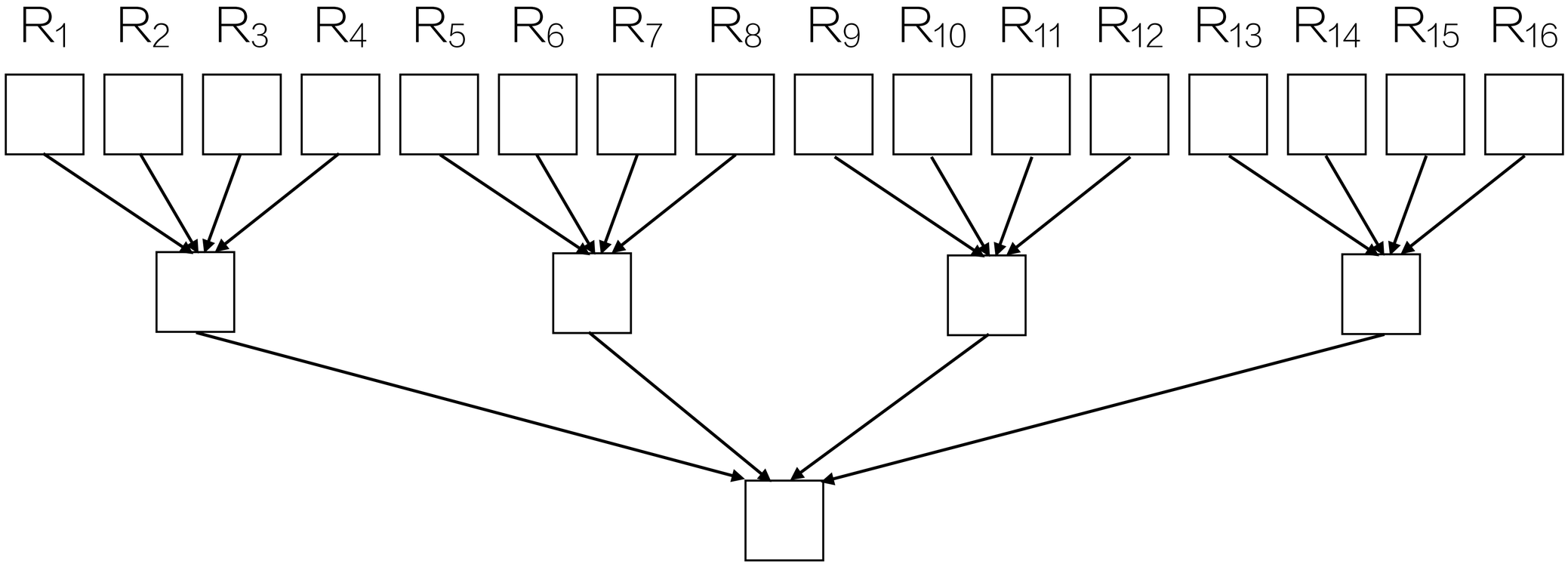
K 路归并



K 路归并

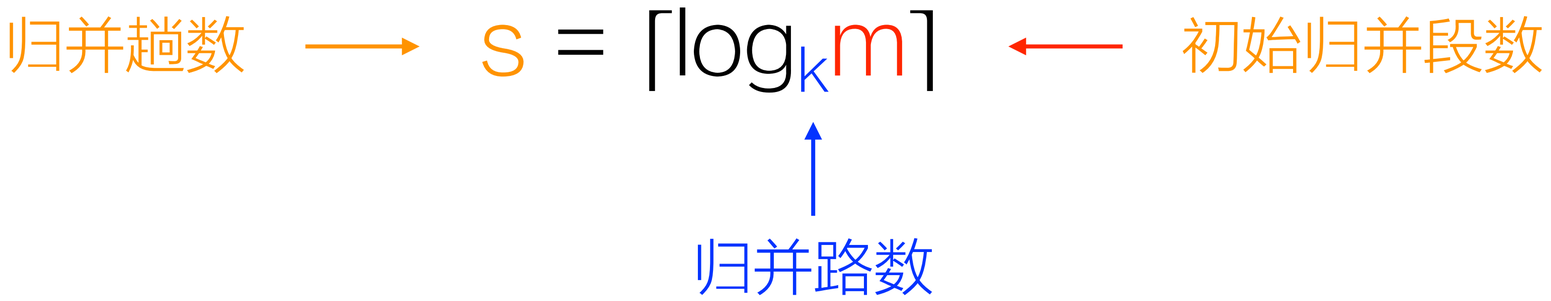


K 路归并



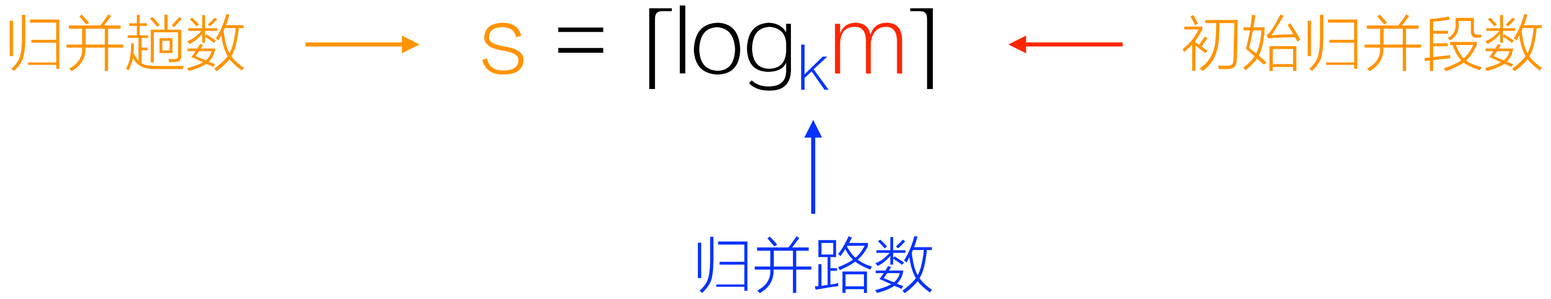
K路归并

对于同一个文件，归并路数越大，归并趟数越小，所需磁盘读写次数也越少



K路归并

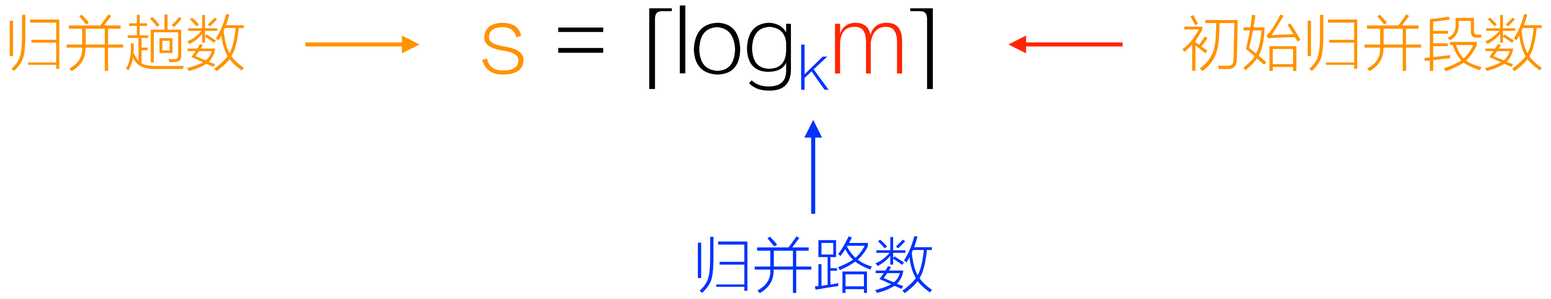
对于同一个文件，归并路数越大，归并趟数越小，所需磁盘读写次数也越少



提升外部排序效率：增加归并路数 k，减少初始归并段个数 m

K路归并

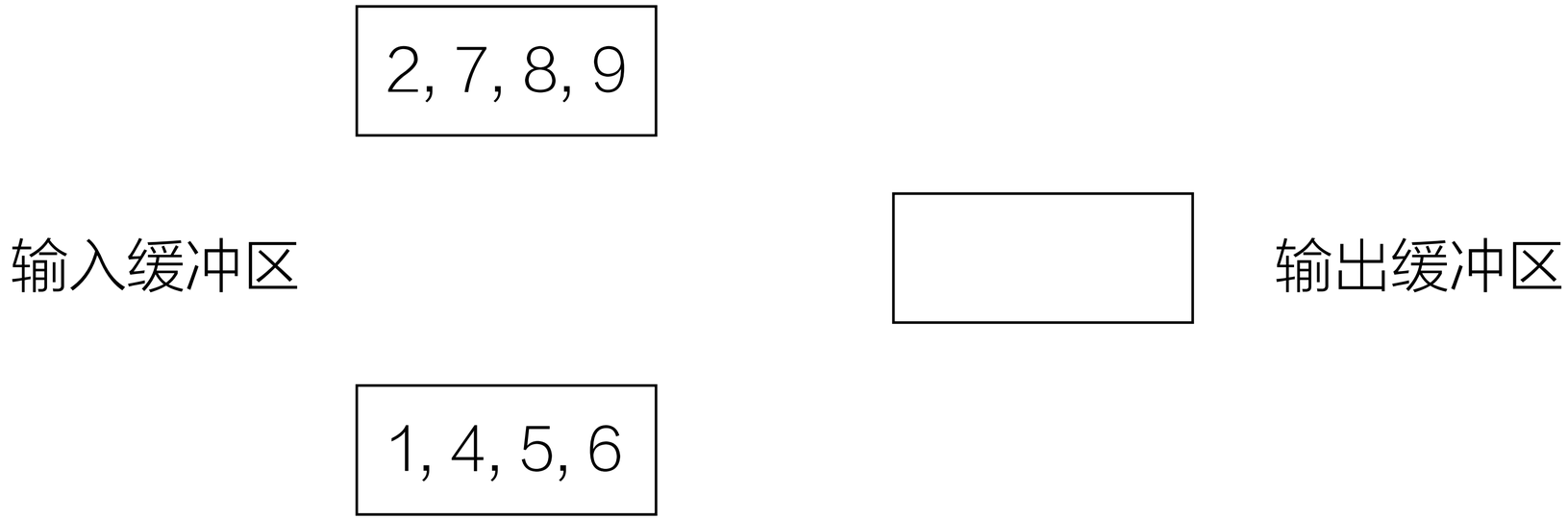
对于同一个文件，归并路数越大，归并趟数越小，所需磁盘读写次数也越少



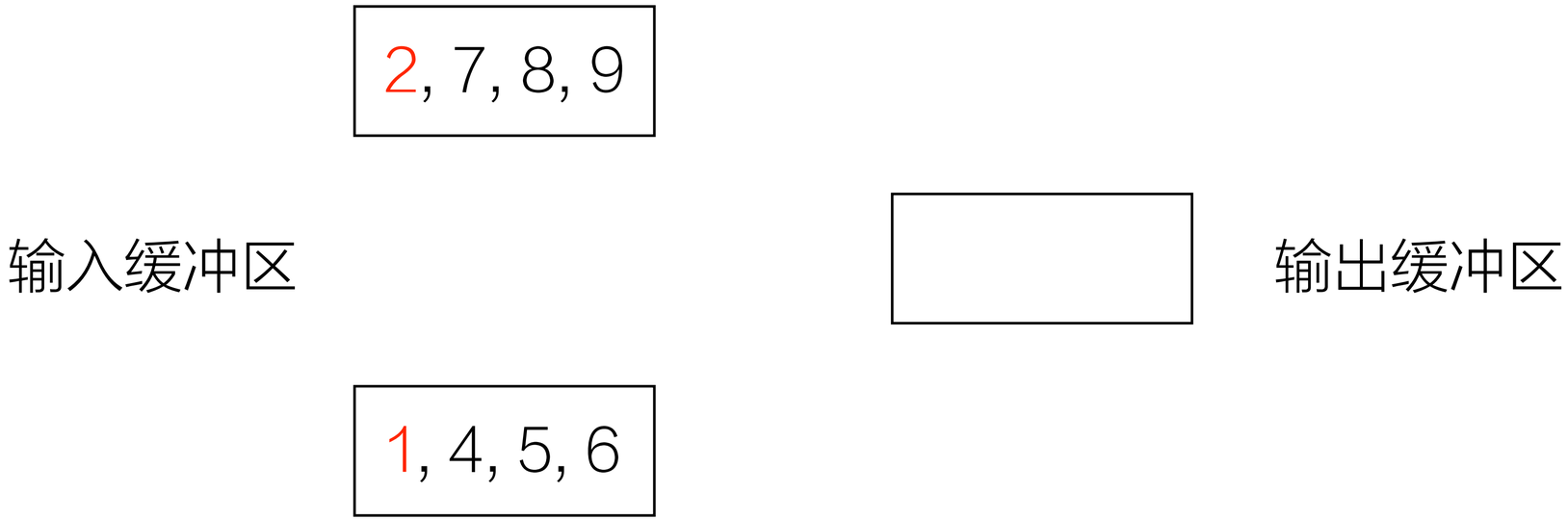
提升外部排序效率：增加归并路数 k，减少初始归并段个数 m

k 越大，所需要的内存空间也越大，缓冲区的个数变多，生成归并段也会变长，归并段的个数就会下降

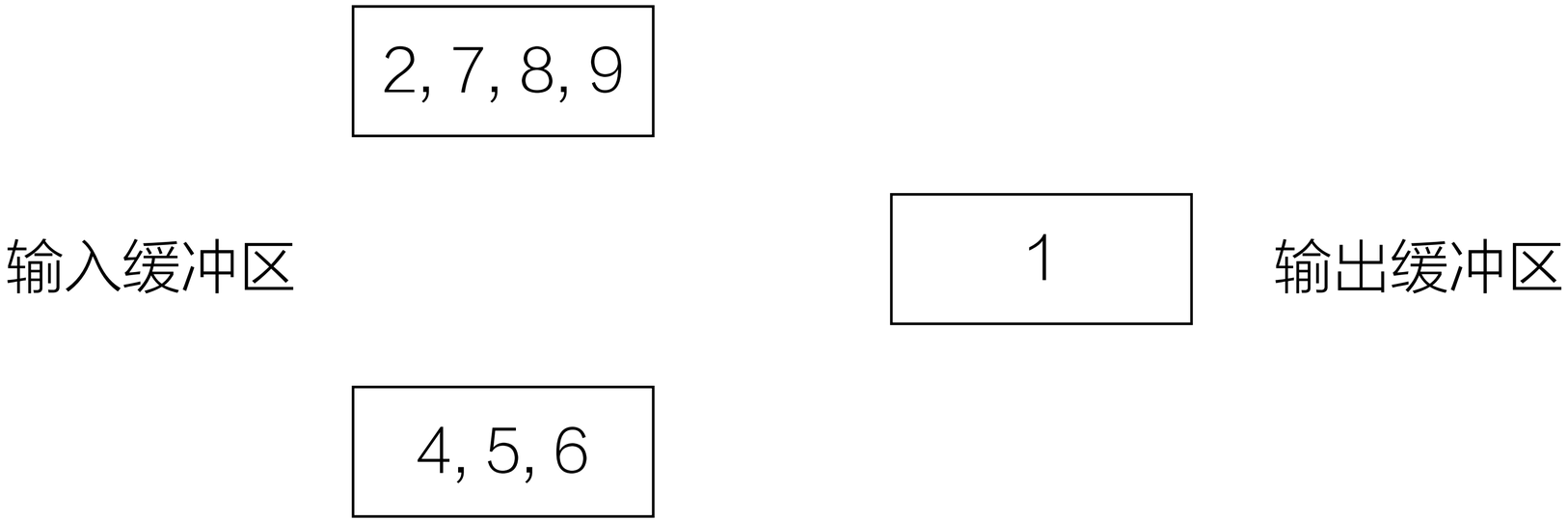
多路平衡归并



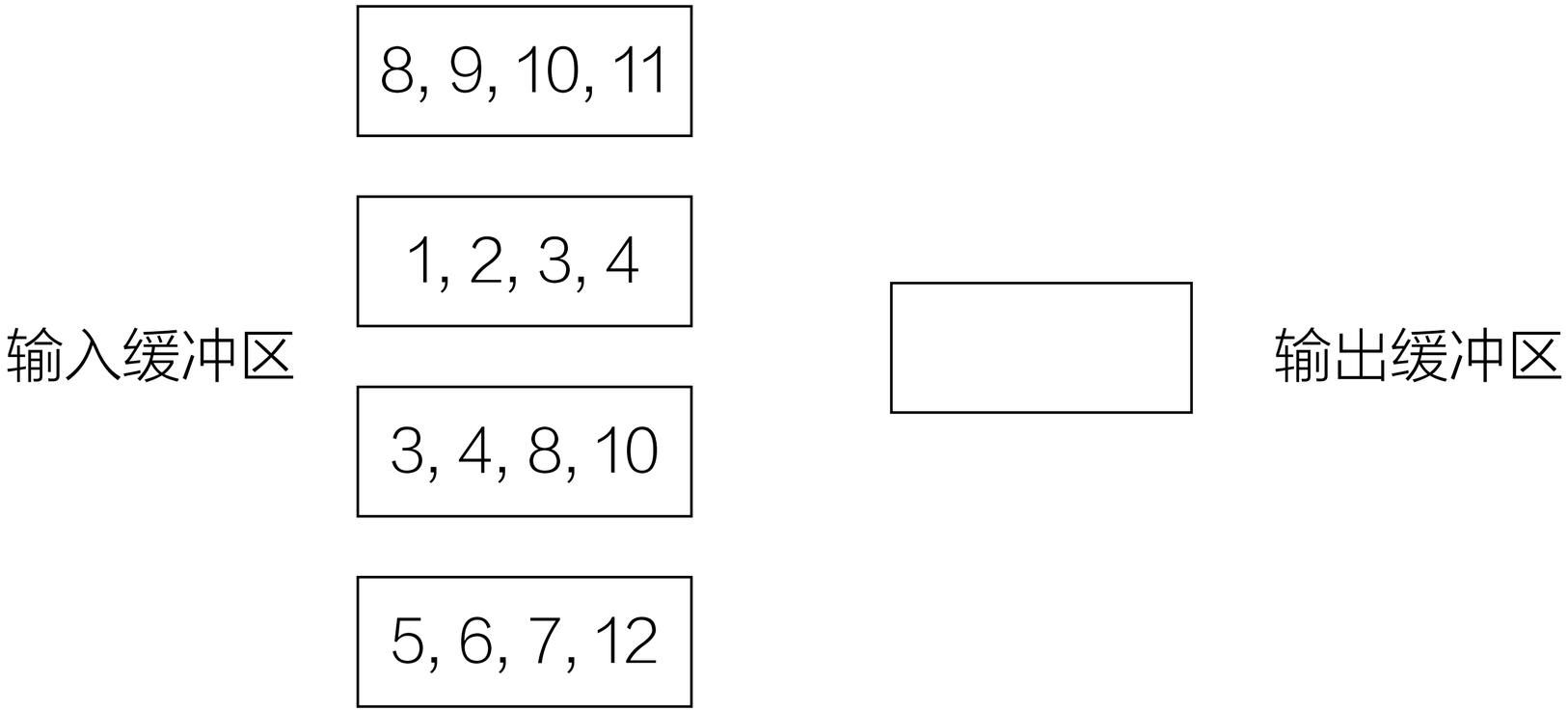
多路平衡归并



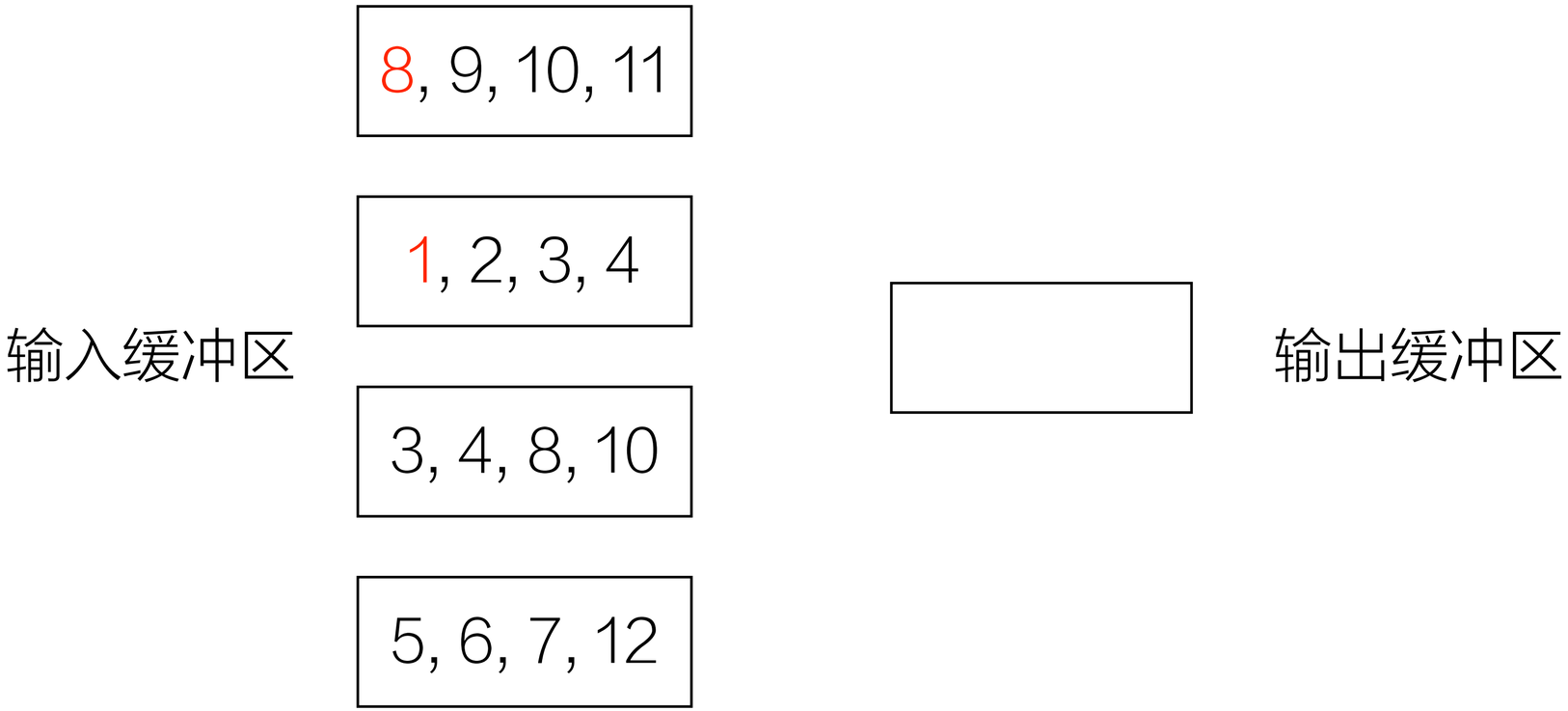
多路平衡归并



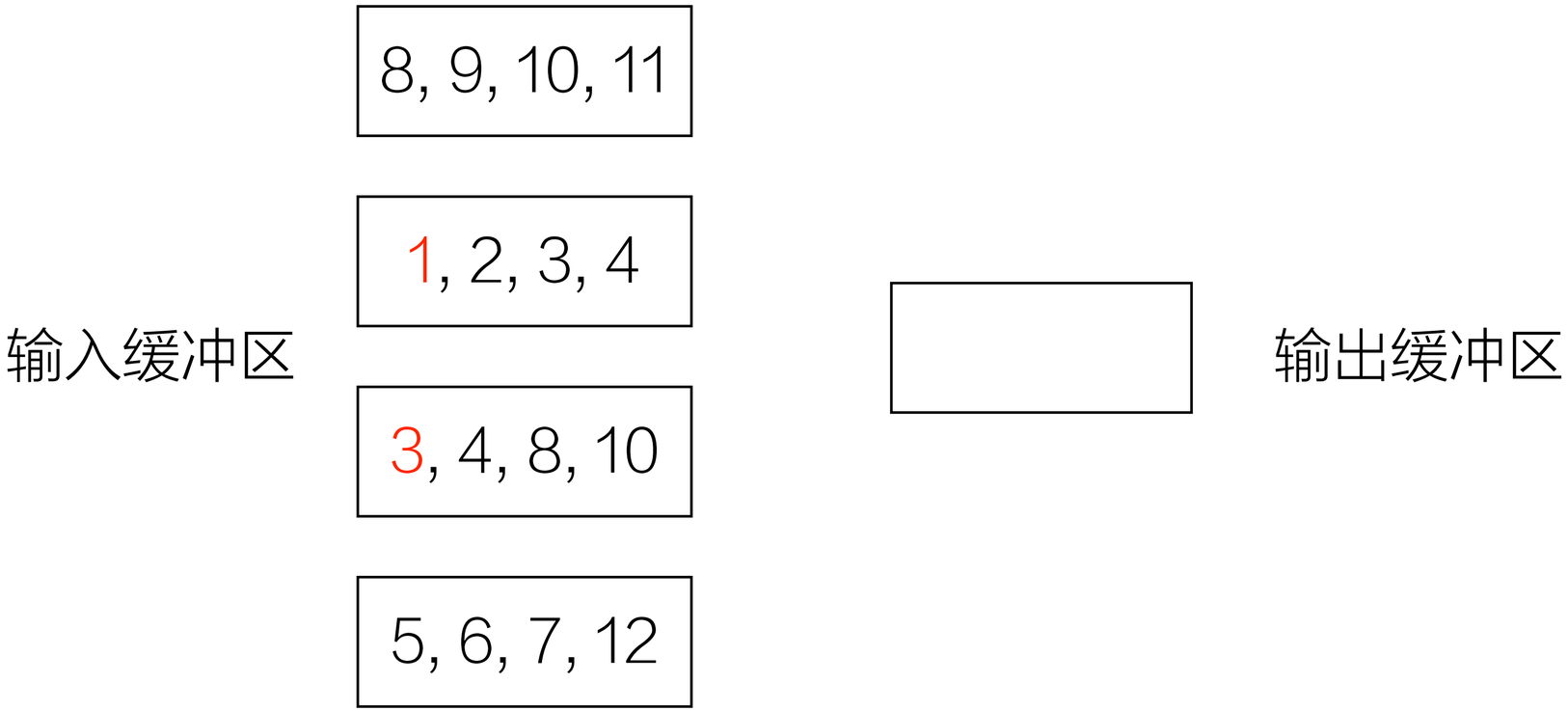
多路平衡归并



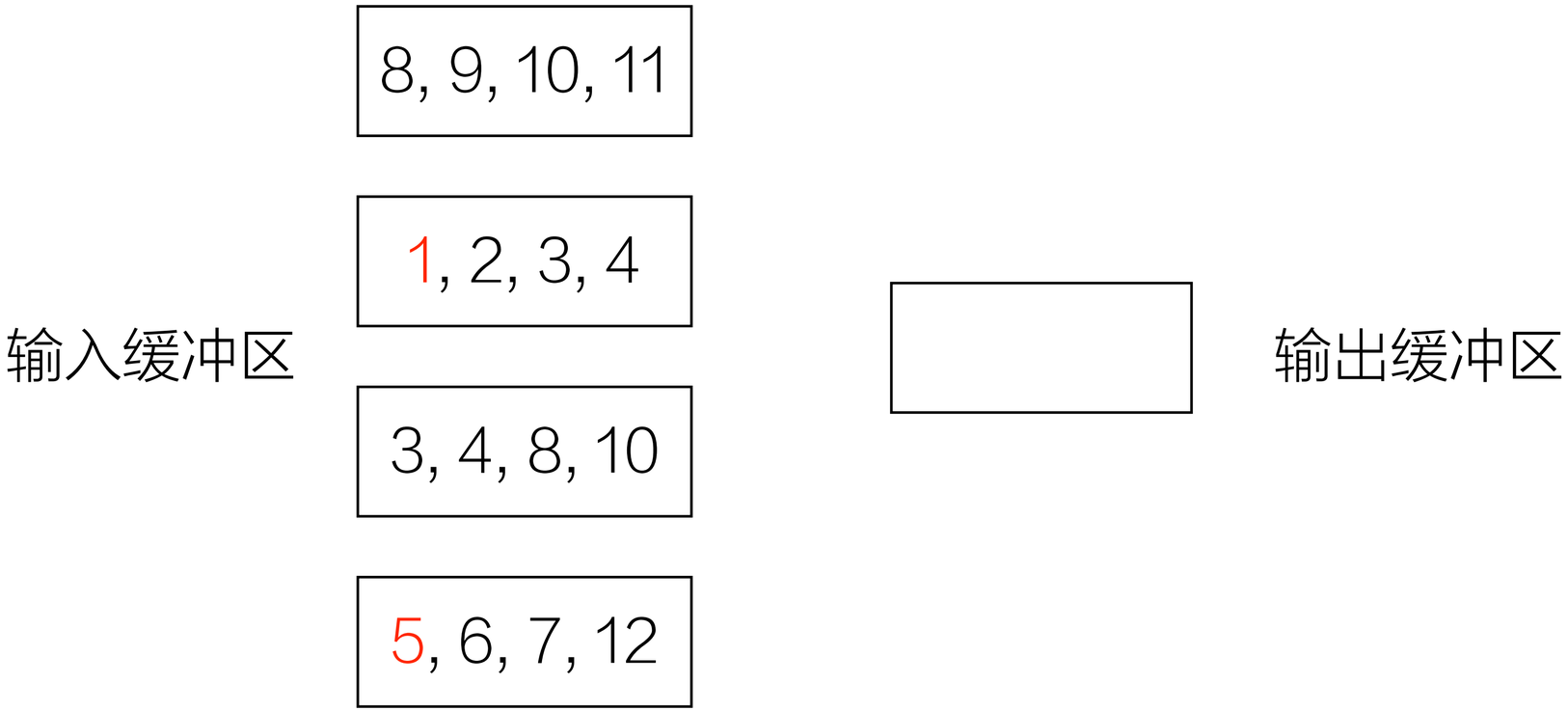
多路平衡归并



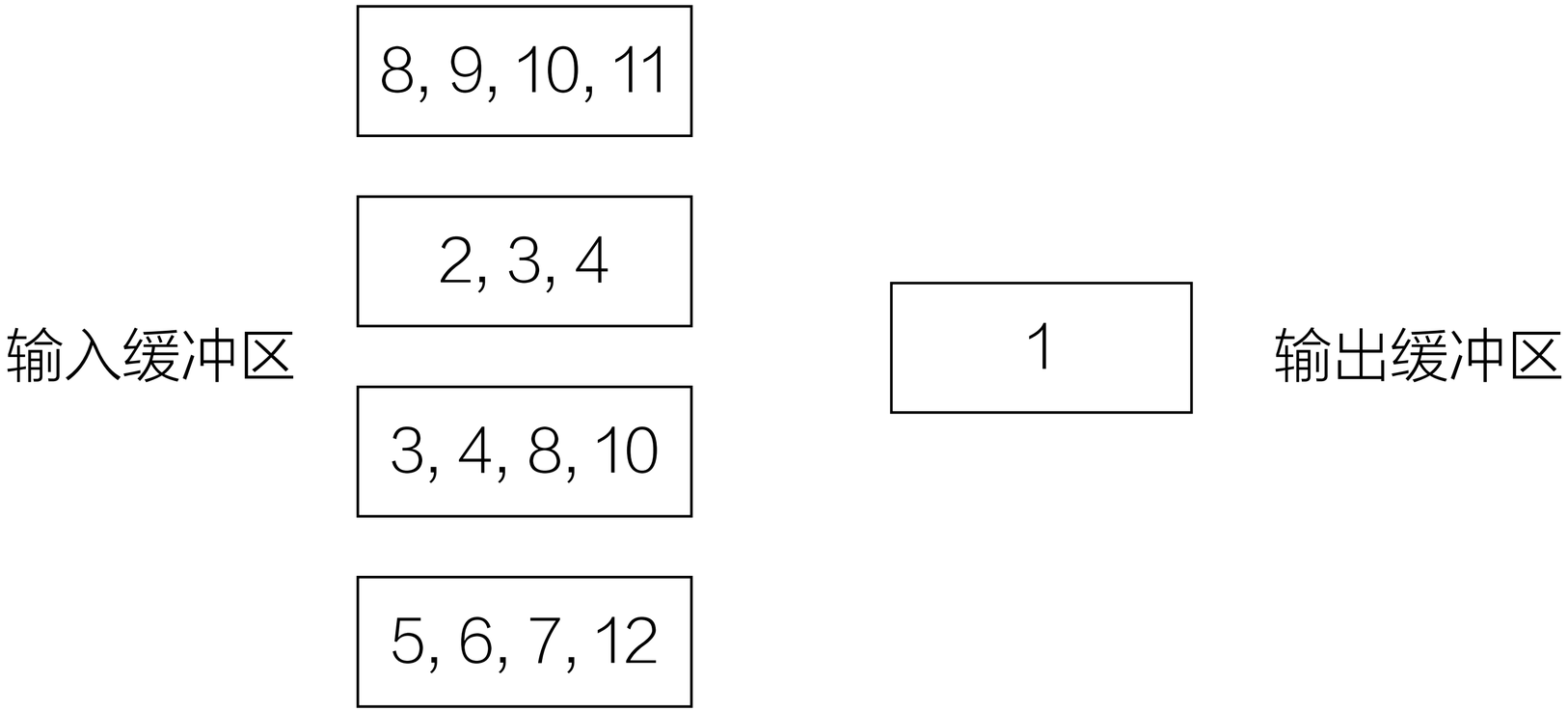
多路平衡归并



多路平衡归并

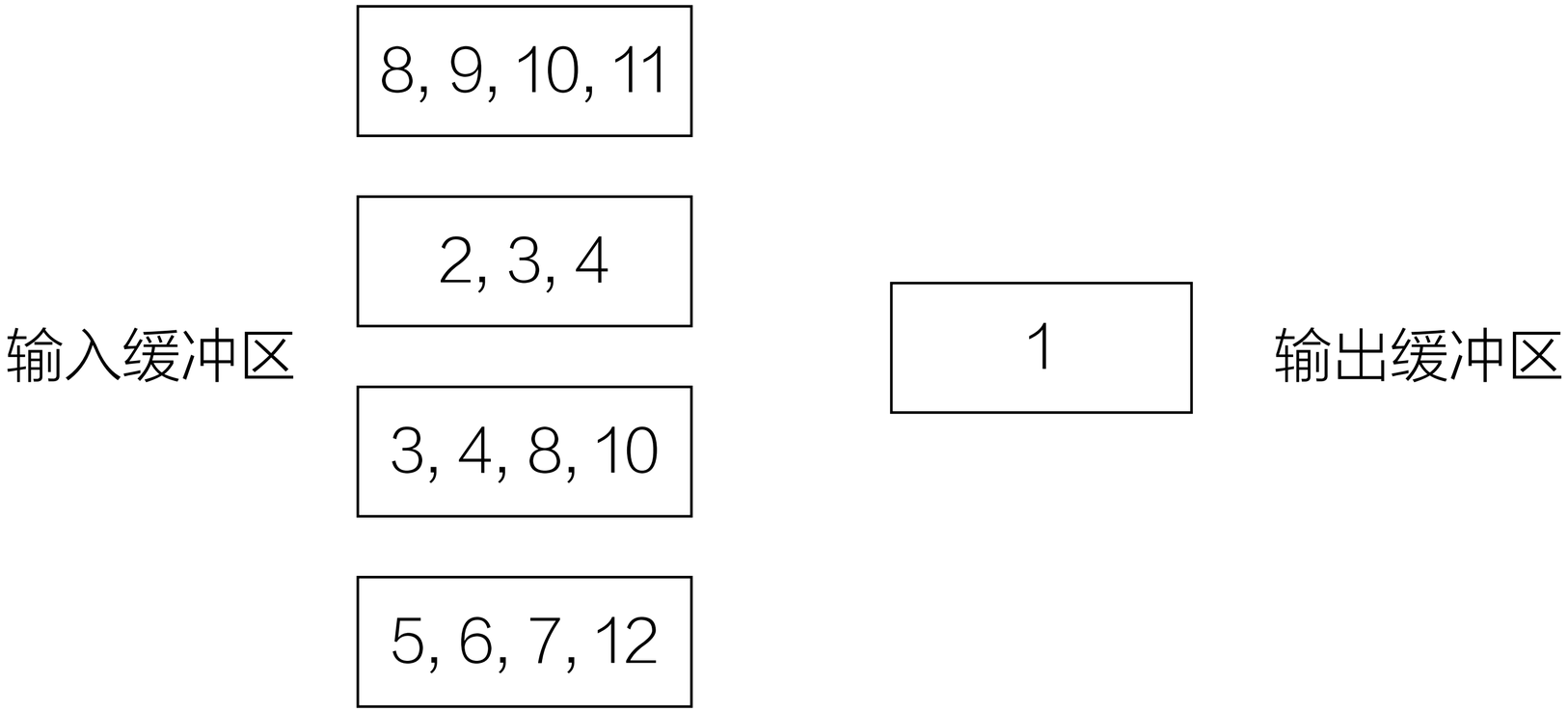


多路平衡归并



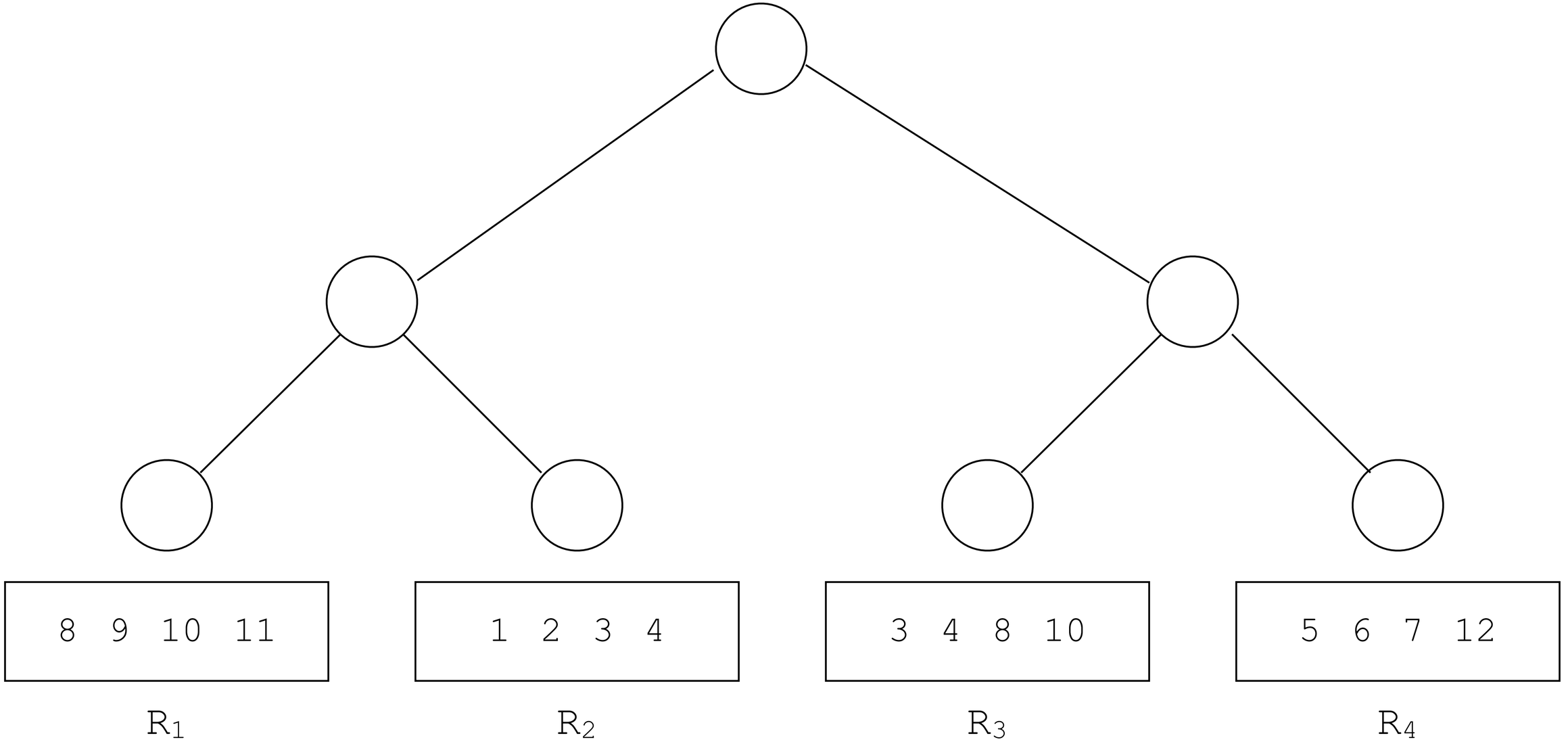
2 路比较 1 次，4 路比较 3 次

多路平衡归并

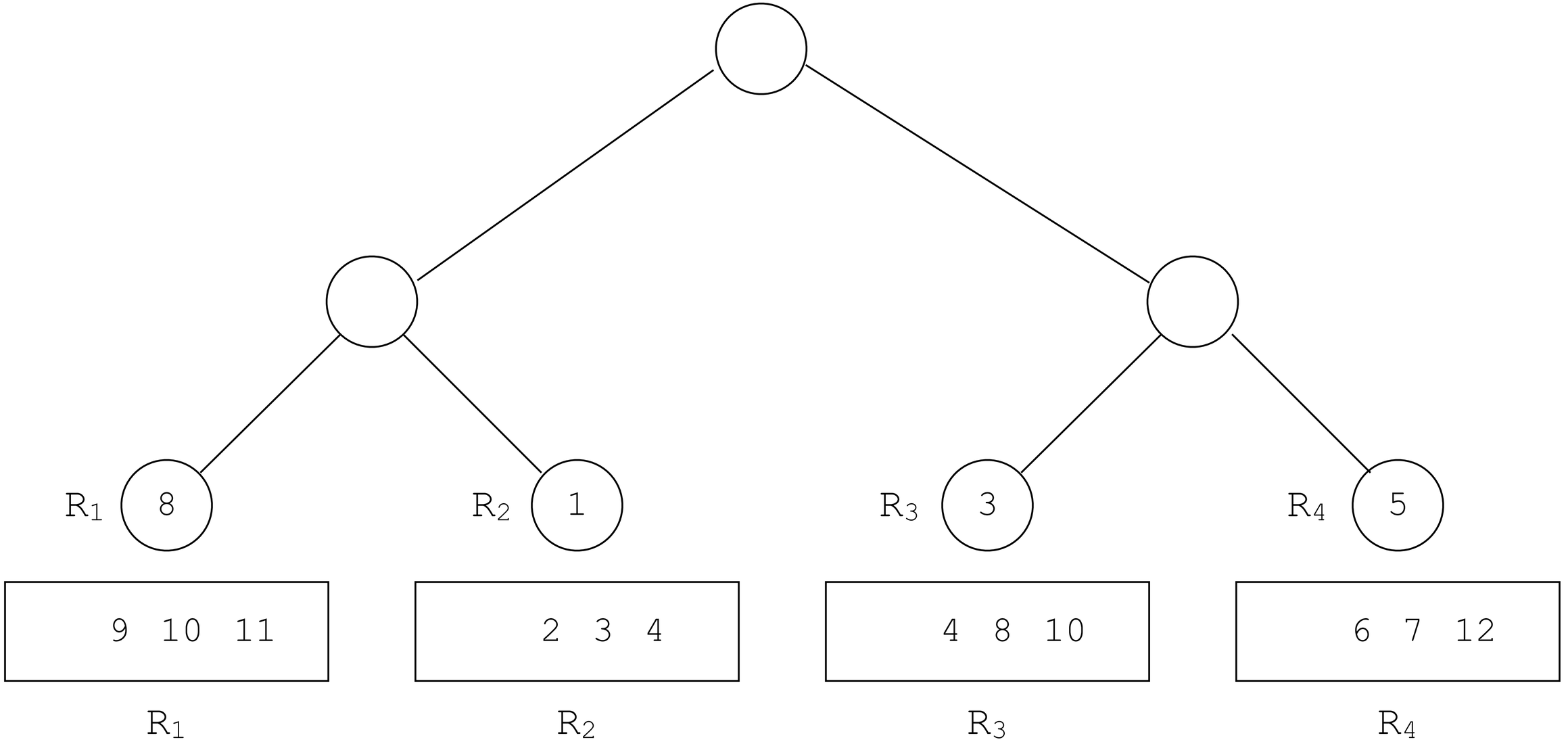


K 路比较 k-1 次，一趟 O(n(k-1))

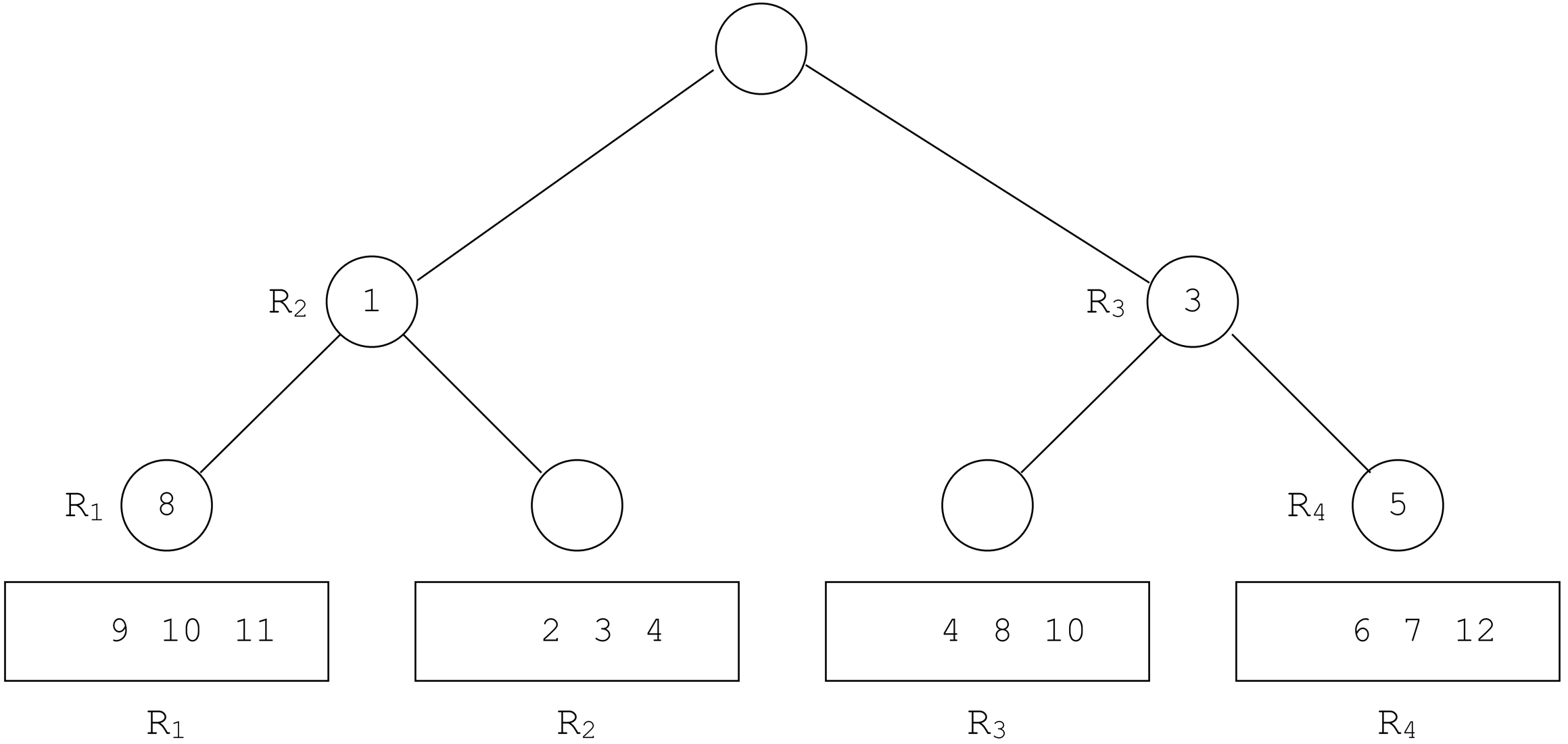
多路平衡归并–败者树



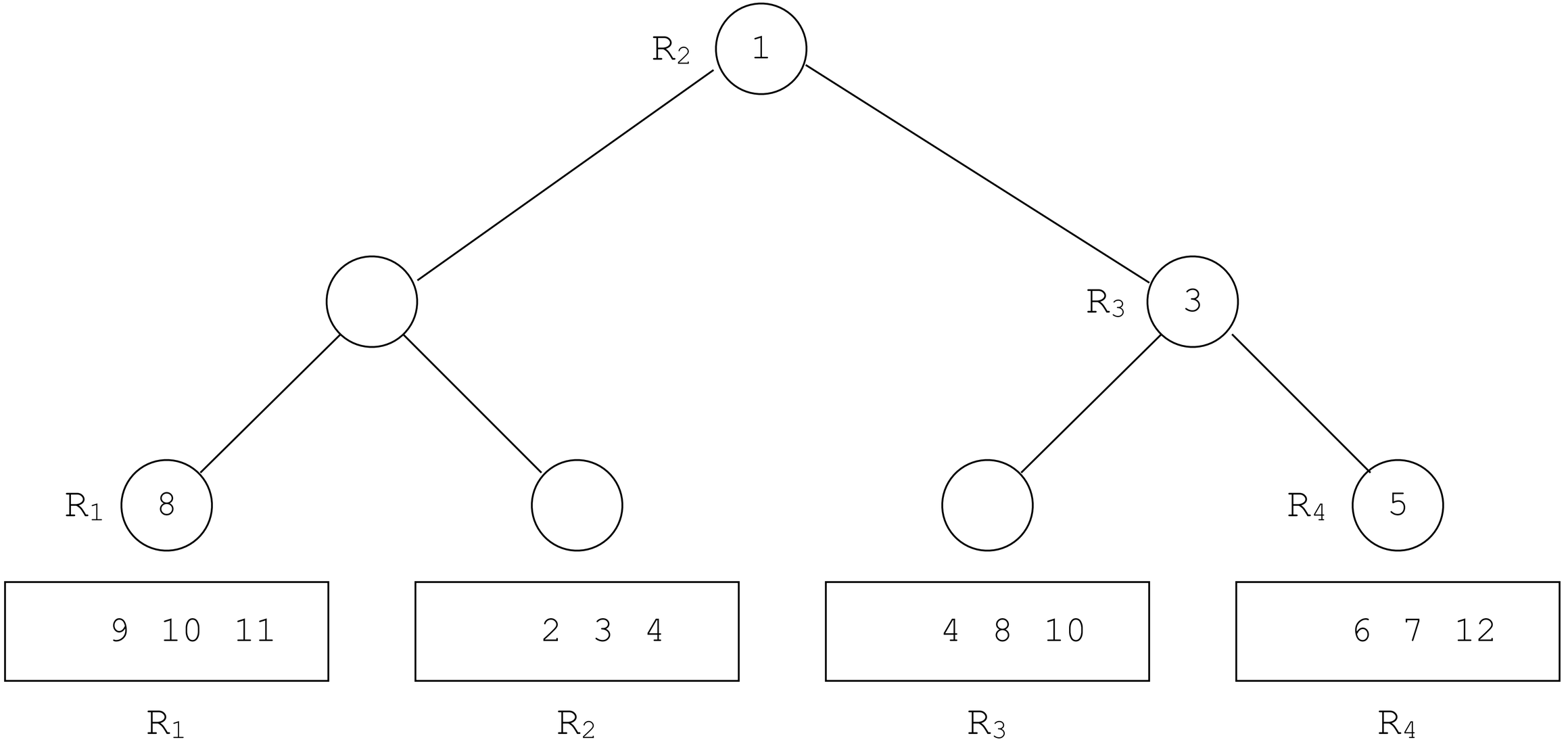
多路平衡归并–败者树



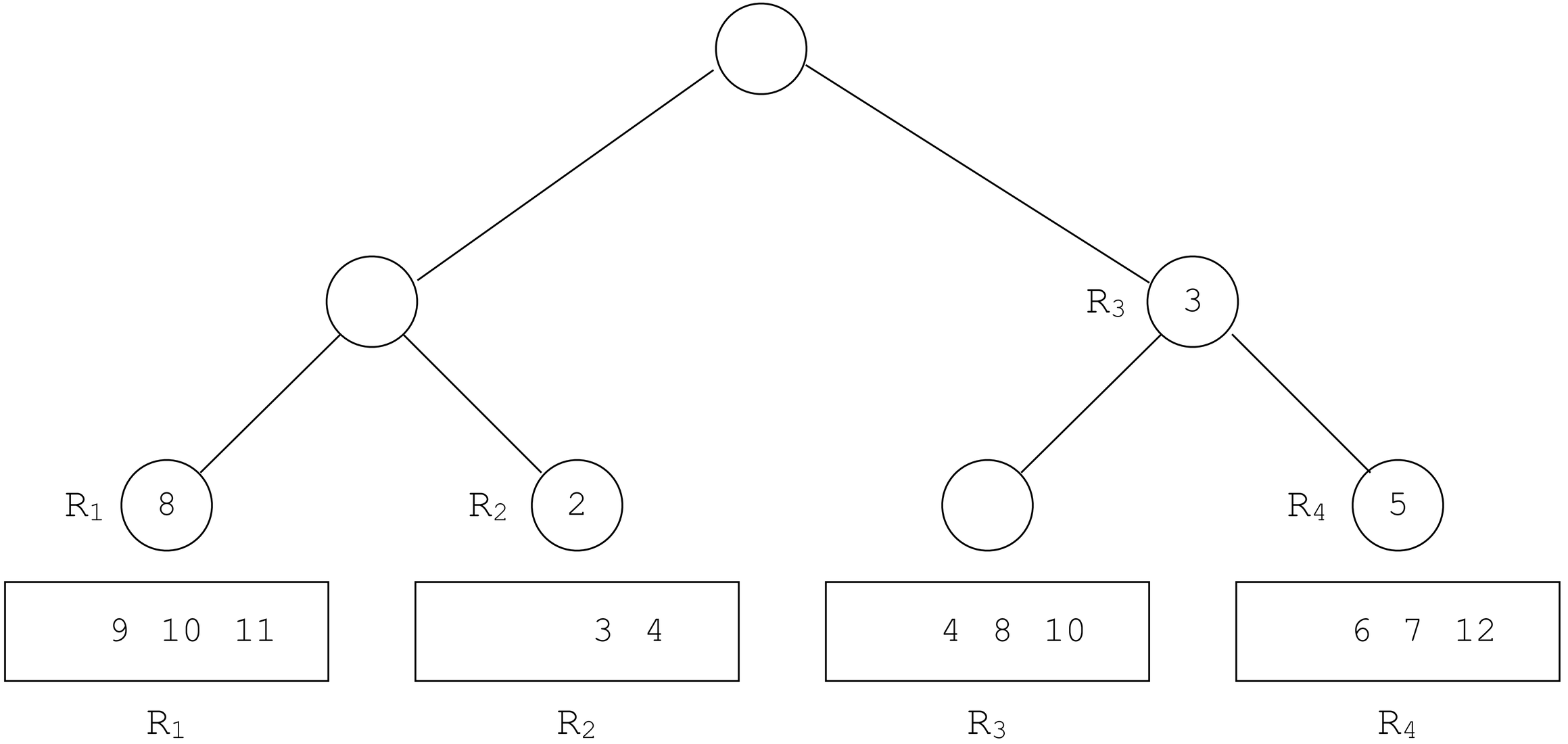
多路平衡归并-败者树



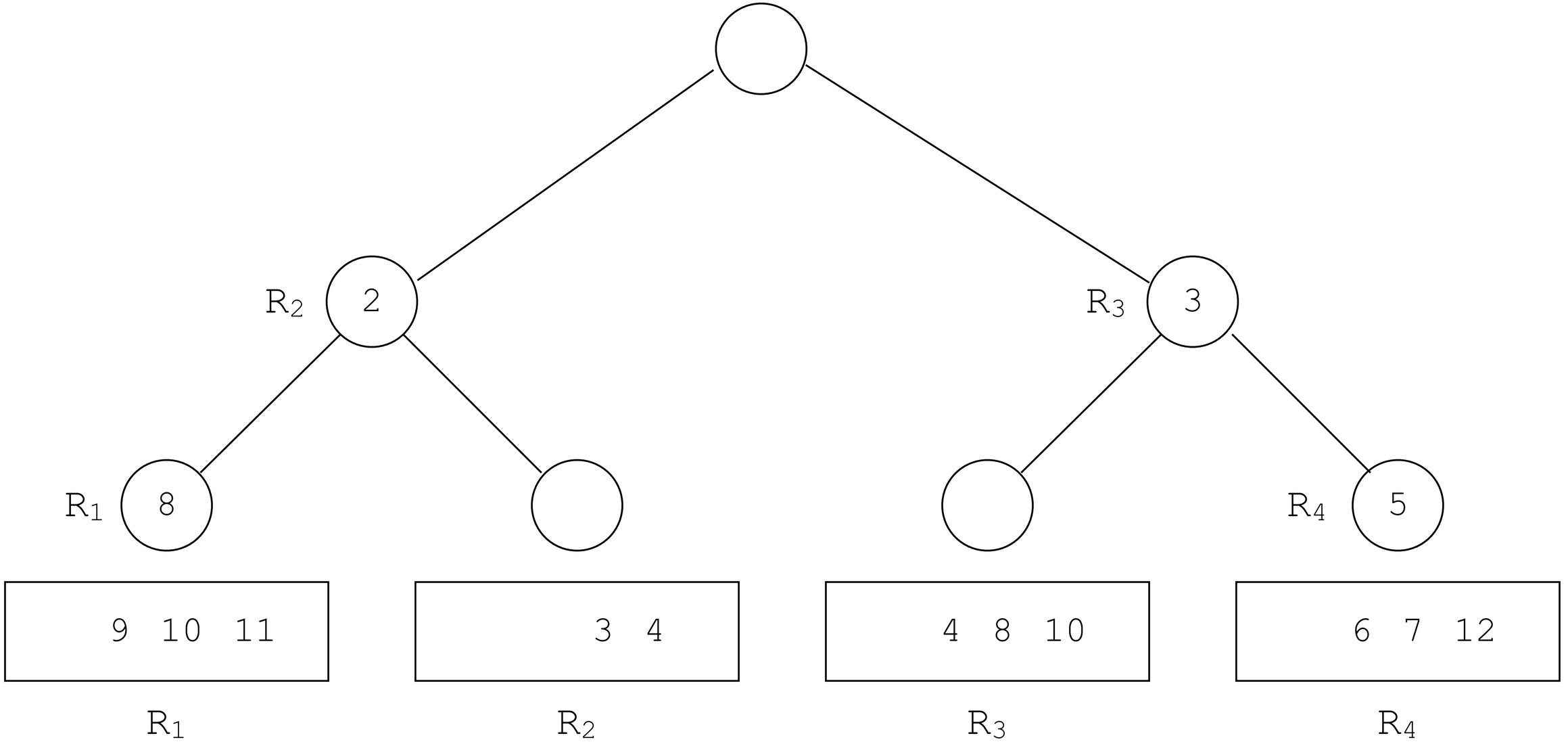
多路平衡归并-败者树



多路平衡归并-败者树

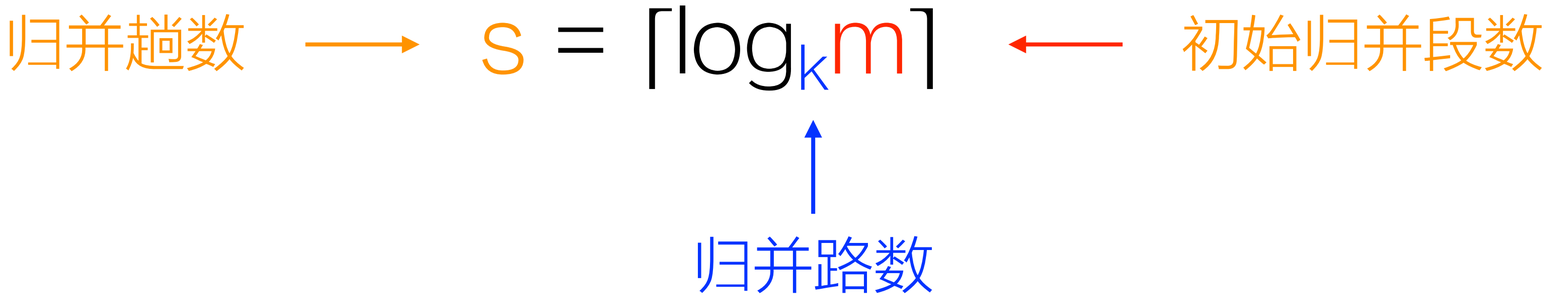


多路平衡归并-败者树



K路归并

对于同一个文件，归并路数越大，归并趟数越小，所需磁盘读写次数也越少



提升外部排序效率：增加归并路数 k，减少初始归并段个数 m

k 越大，所需要的内存空间也越大，缓冲区的个数变多，生成归产段也会变长，归并段的个数就会下降

置换–选择排序

49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62

置换–选择排序

49 47 34 20　　24 15 53 61　　17 32 25 26　　30 46 47 62

置换–选择排序

输入文件（File Input）：`49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62`

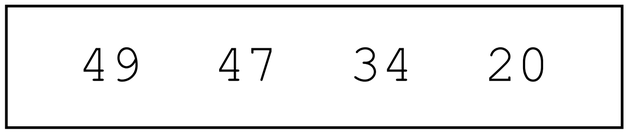


内存工作区（Work Area），假设容量为 4

输出文件（File Output）：

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62

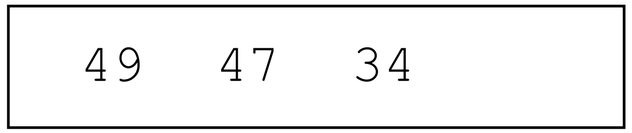


内存工作区（Work Area），假设容量为 4

输出文件（File Output）：

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



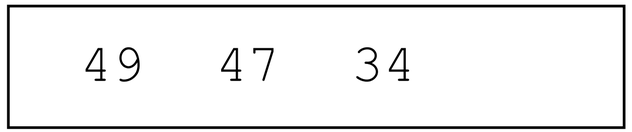
内存工作区（Work Area），假设容量为 4

输出文件（File Output）：

归并段 1：20

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

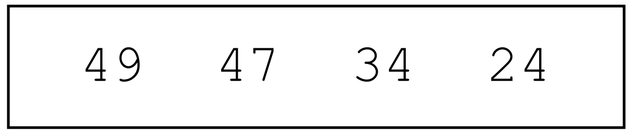
Min = 20

输出文件（File Output）：

归并段 1：20

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

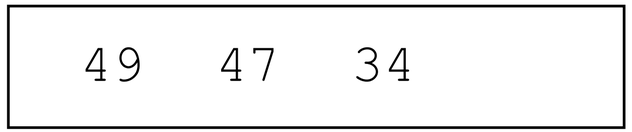
Min = 20

输出文件（File Output）：

归并段 1：20

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

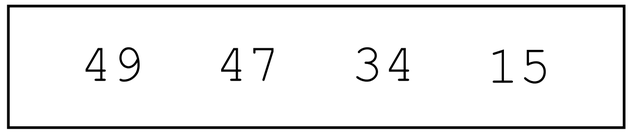
Min = 24

输出文件（File Output）：

归并段 1：20 24

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

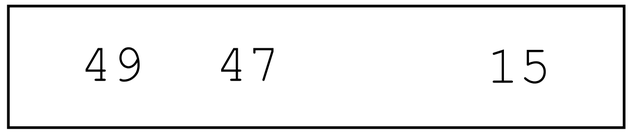
Min = 24

输出文件（File Output）：

归并段 1：20 24

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

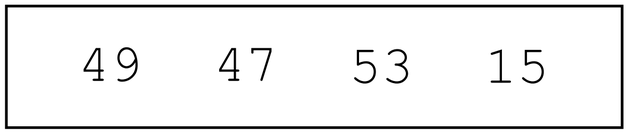
Min = 34

输出文件（File Output）：

归并段 1：20 24 34

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 34

输出文件（File Output）：

归并段 1：20 24 34

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

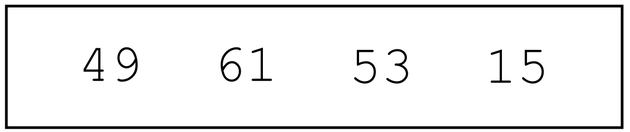
Min = 47

输出文件（File Output）：

归并段 1：20 24 34 47

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

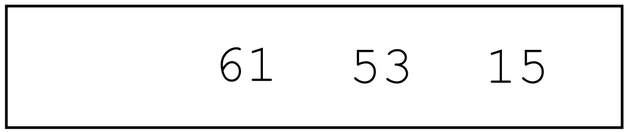
Min = 47

输出文件（File Output）：

归并段 1：20 24 34 47

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

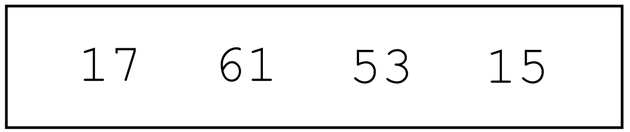
Min = 49

输出文件（File Output）：

归并段 1：20 24 34 47 49

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

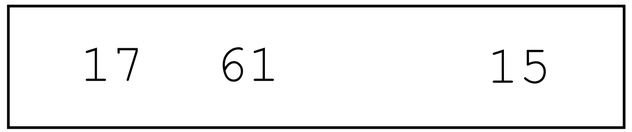
Min = 49

输出文件（File Output）：

归并段 1：20 24 34 47 49

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

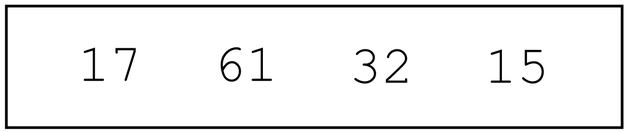
Min = 53

输出文件（File Output）：

归并段 1：20 24 34 47 49 53

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 53

输出文件（File Output）：

归并段 1：20 24 34 47 49 53

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 61

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 61

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min =

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 15

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 15

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 17

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 17

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 25

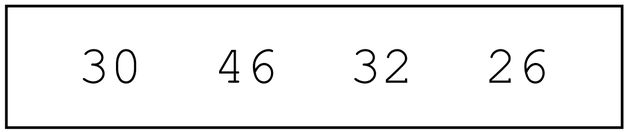
输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 25

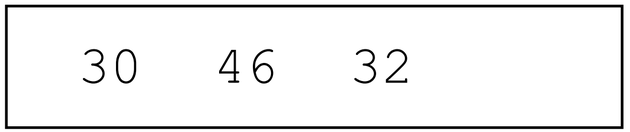
输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 26

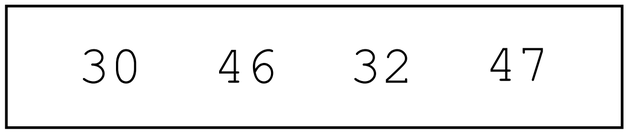
输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25 26

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 26

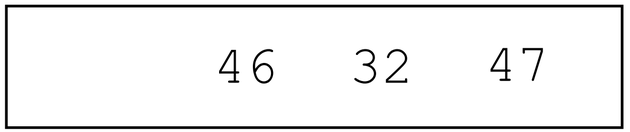
输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25 26

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 30

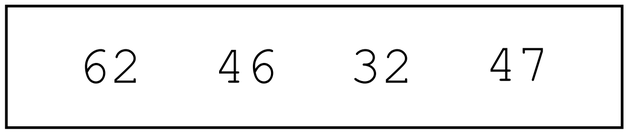
输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25 26 30

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



内存工作区（Work Area），假设容量为 4

Min = 30

输出文件（File Output）：

归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25 26 30

置换–选择排序

输入文件（File Input）：49 47 34 20 24 15 53 61 17 32 25 26 30 46 47 62



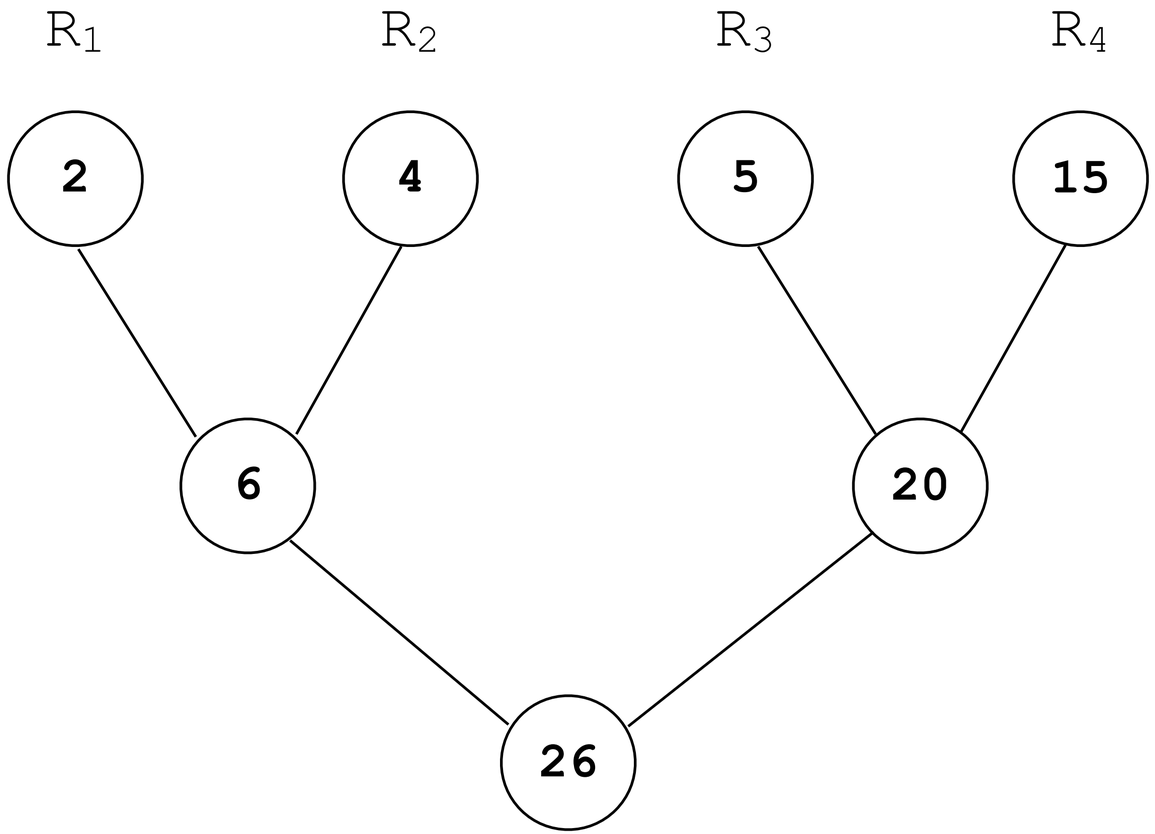
内存工作区（Work Area），假设容量为 4

输出文件（File Output）：

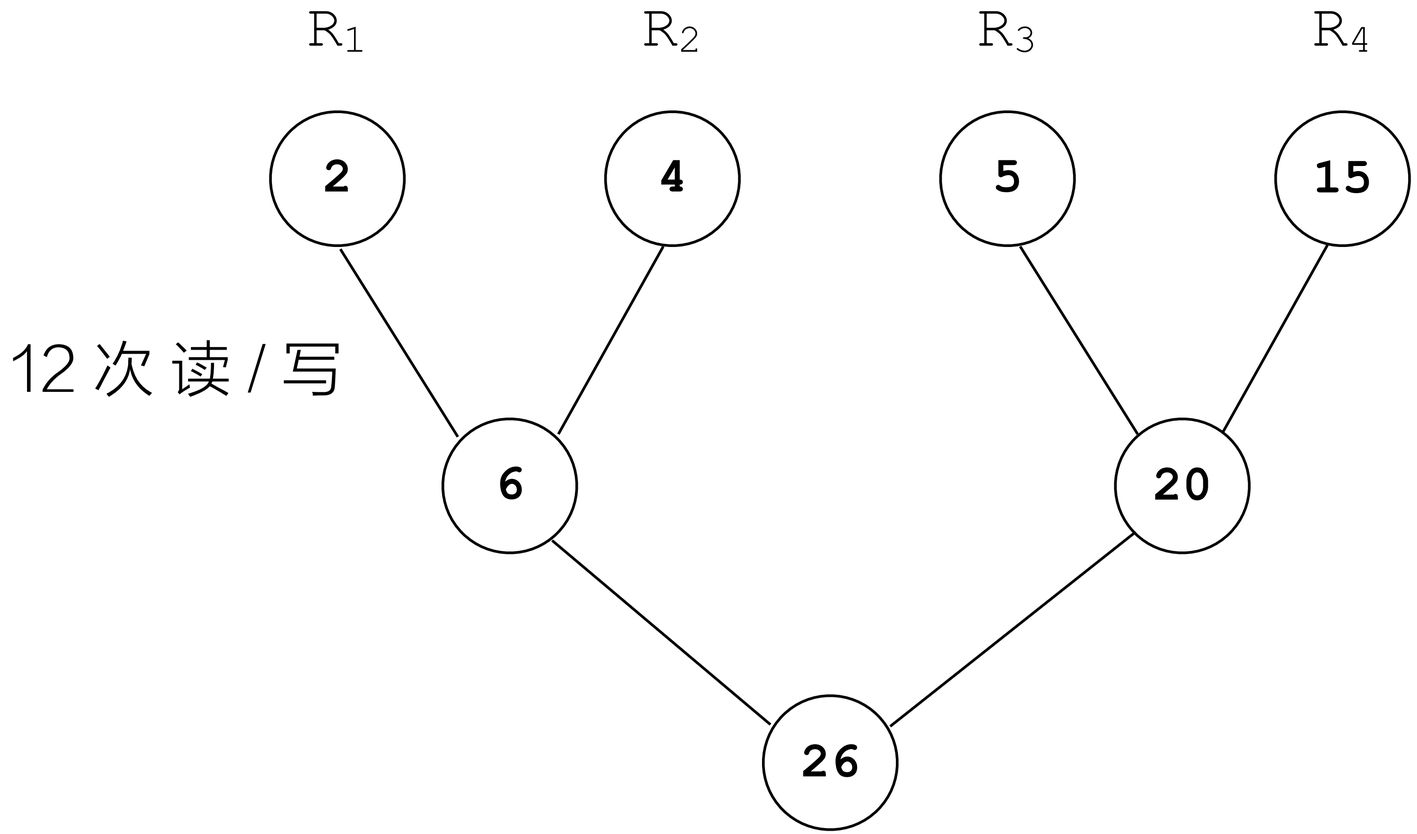
归并段 1：20 24 34 47 49 53 61

归并段 2：15 17 25 26 30 32 46 47 62

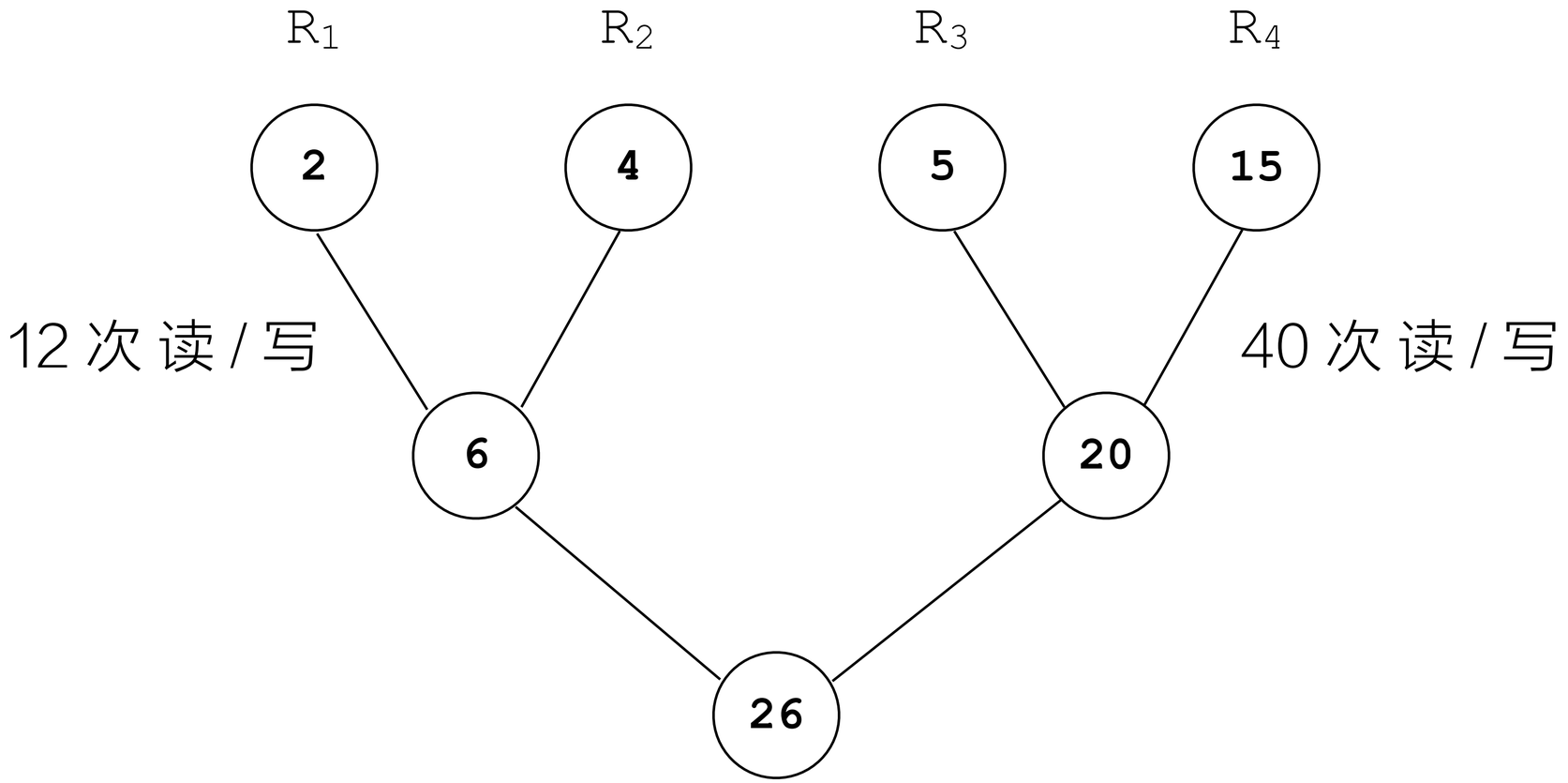
最佳归并树



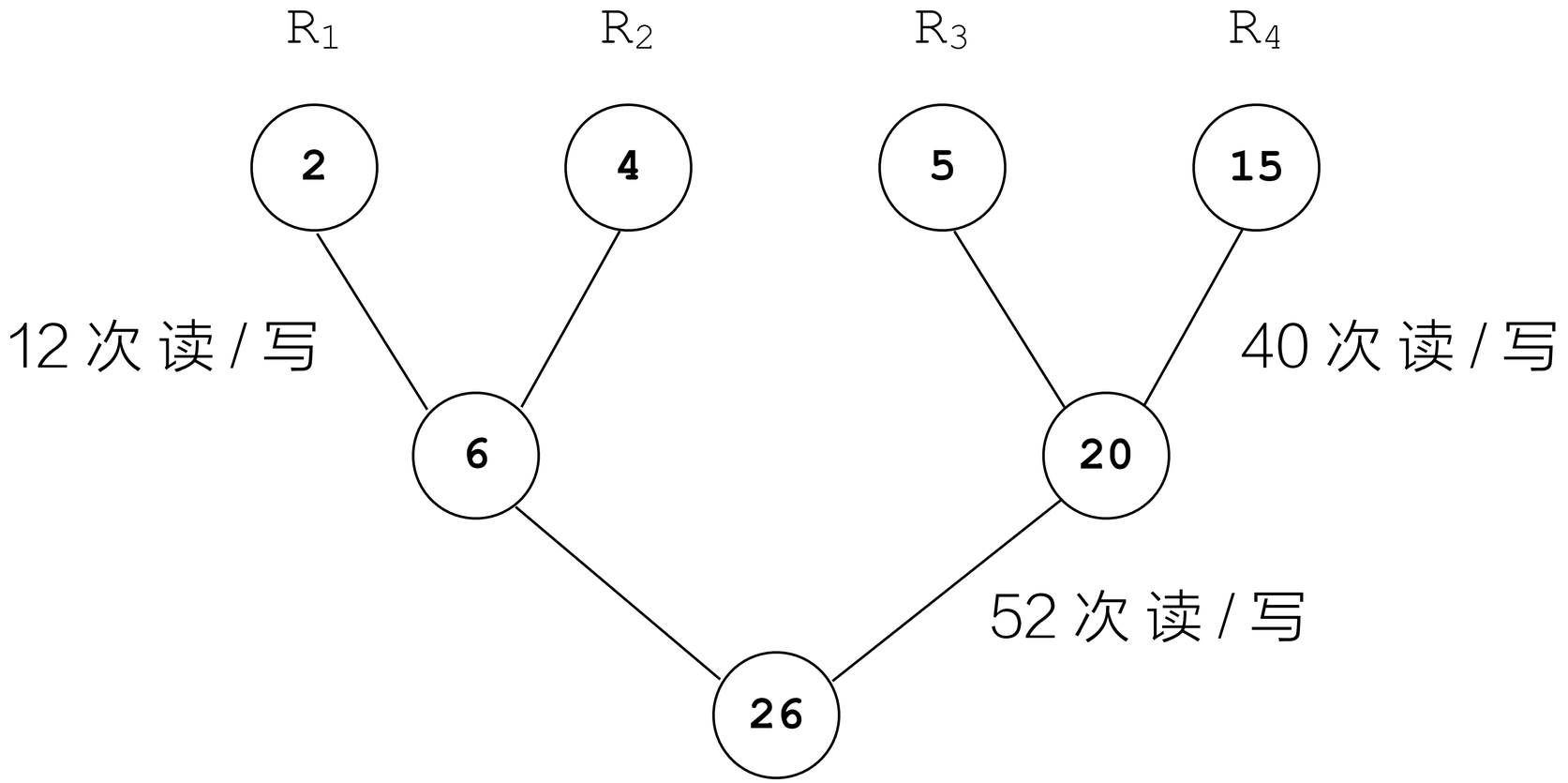
最佳归并树



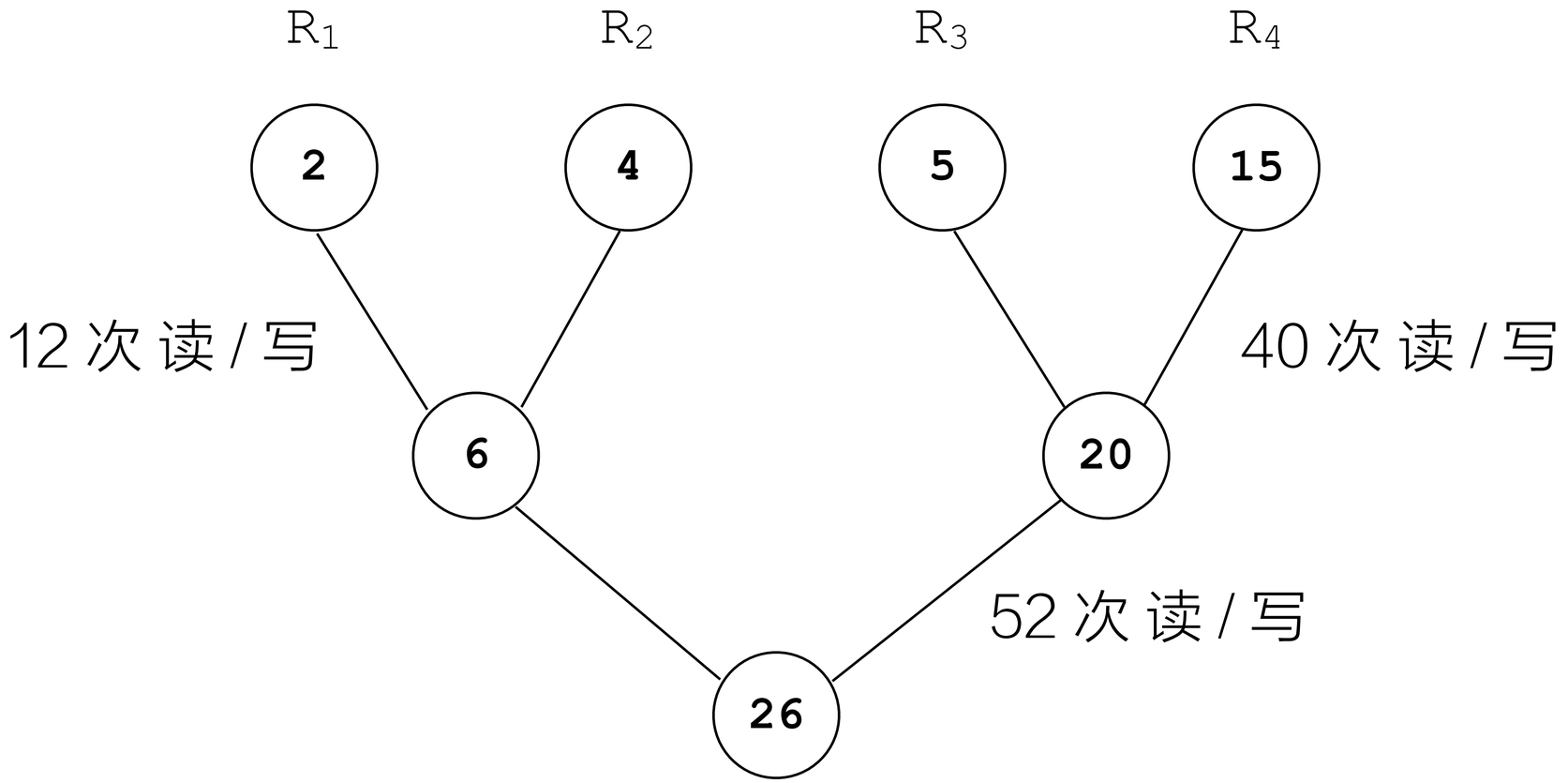
最佳归并树



最佳归并树

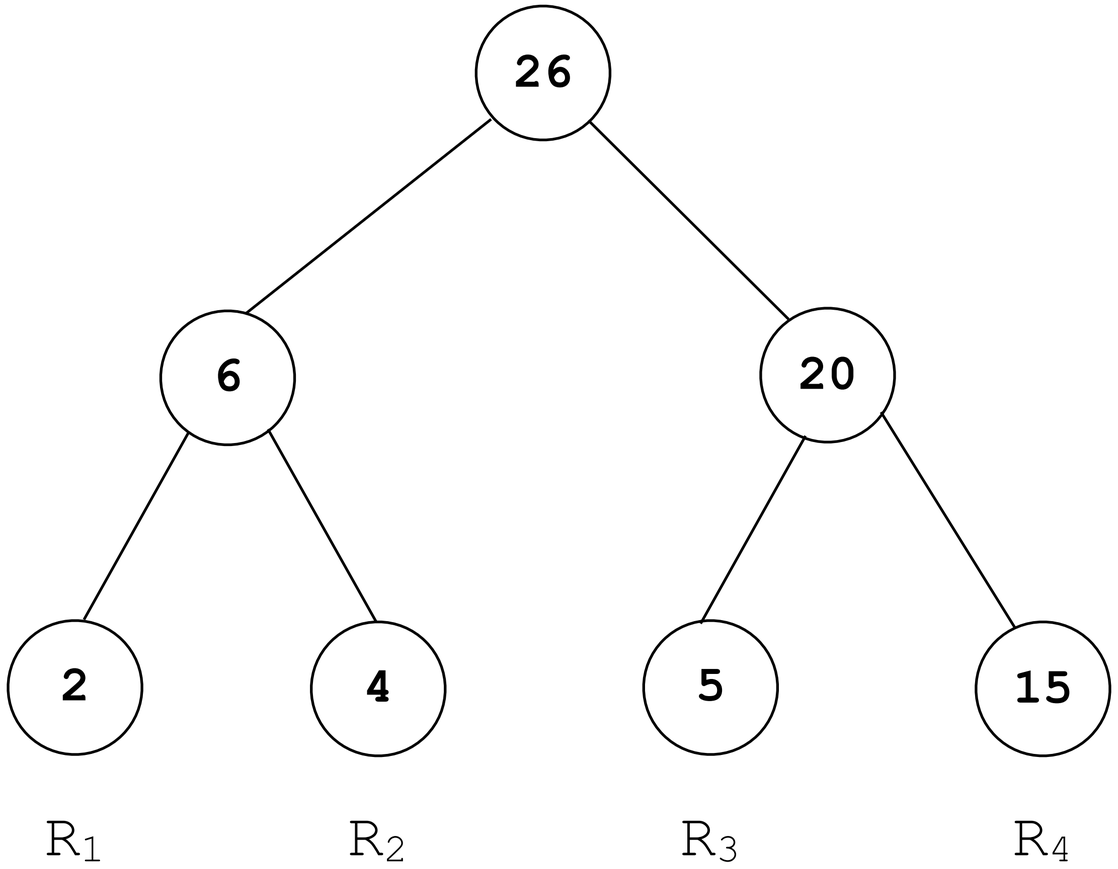


最佳归并树



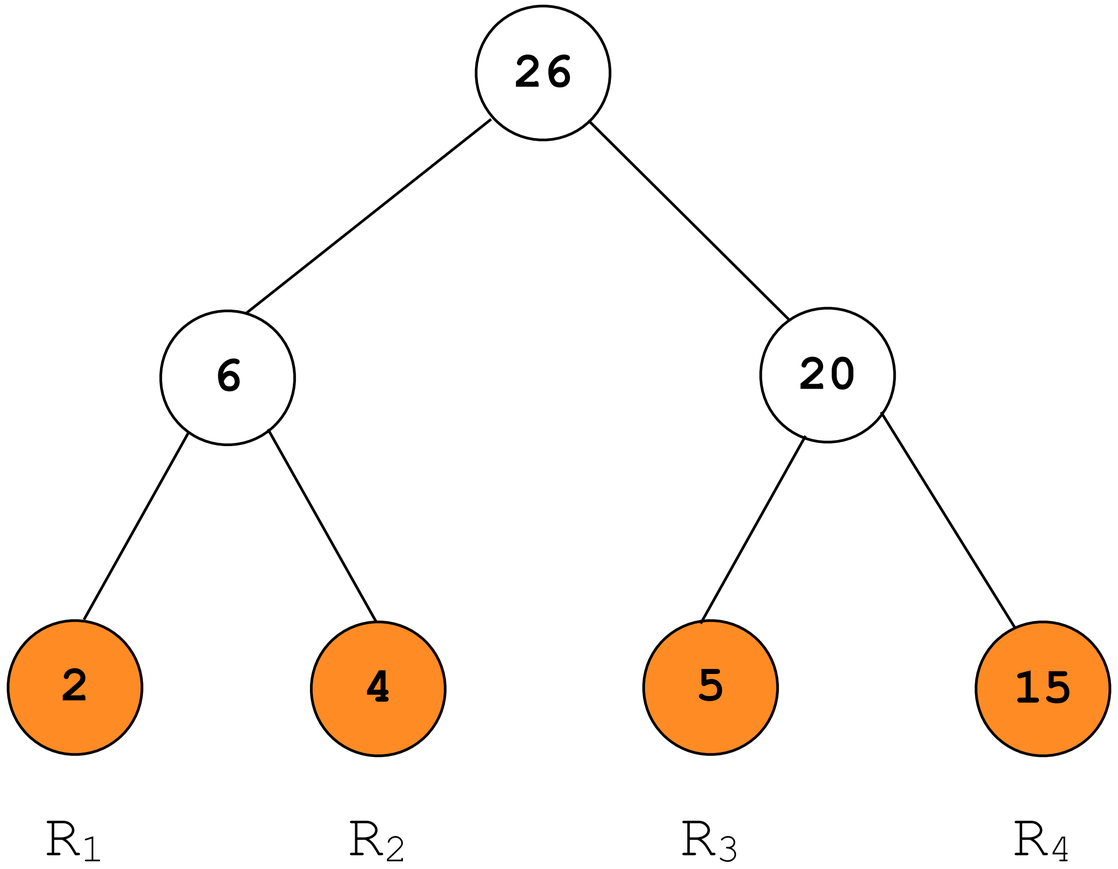
总读写次数：104

最佳归并树



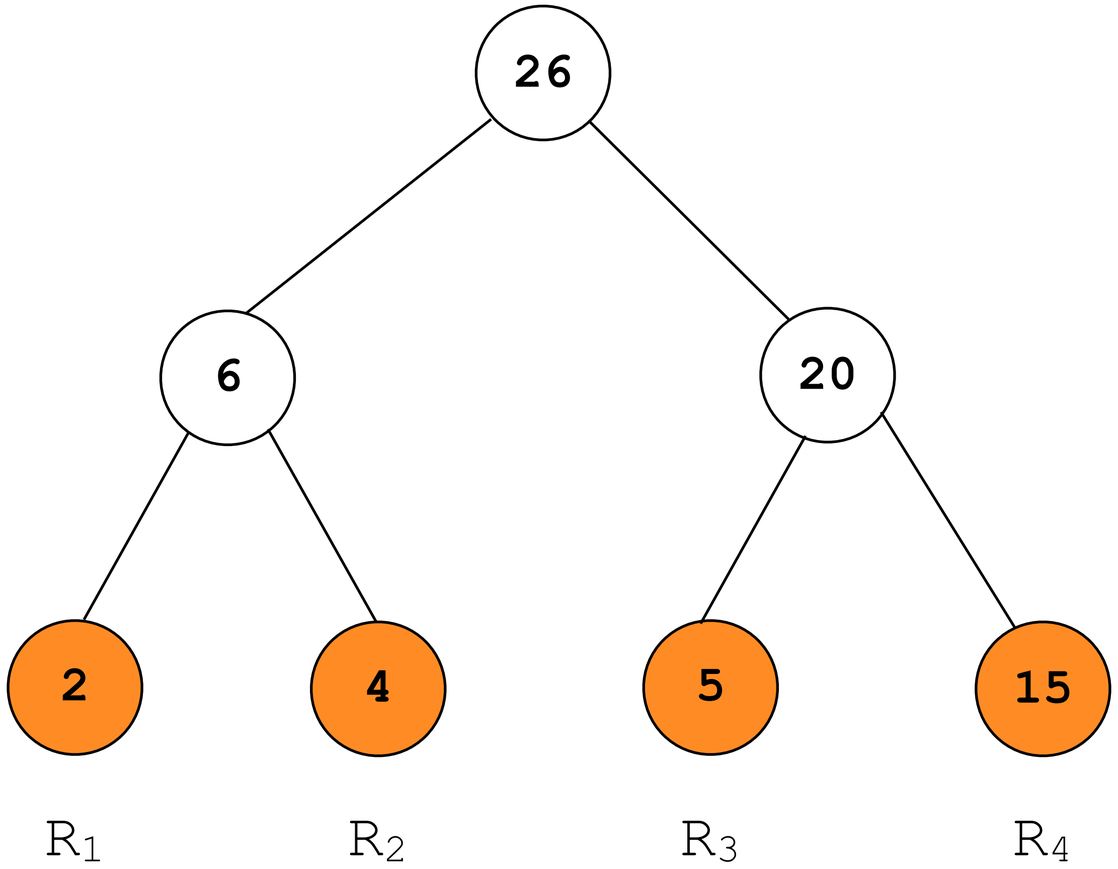
总读写次数：104

最佳归并树



总读写次数：104

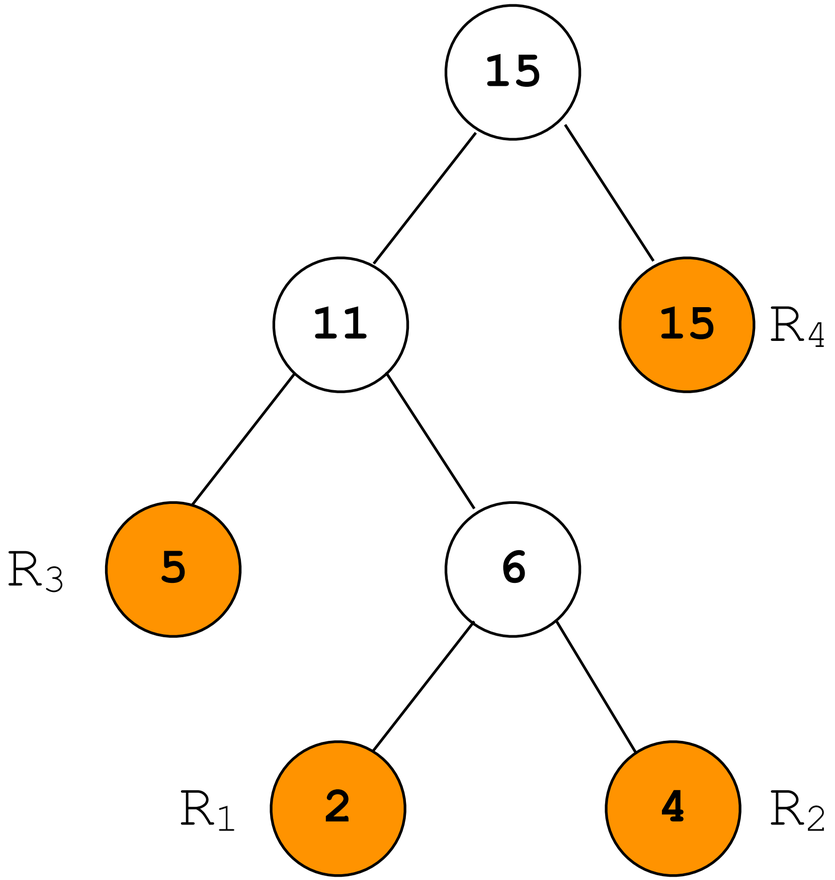
最佳归并树



总读写次数：104

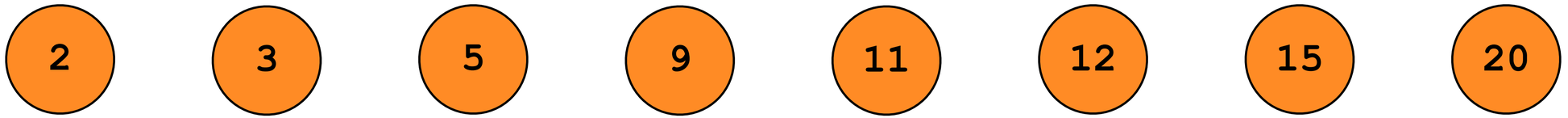
带权路径长度 WPL：2*2+4*2+5*2+15*2=52

最佳归并树



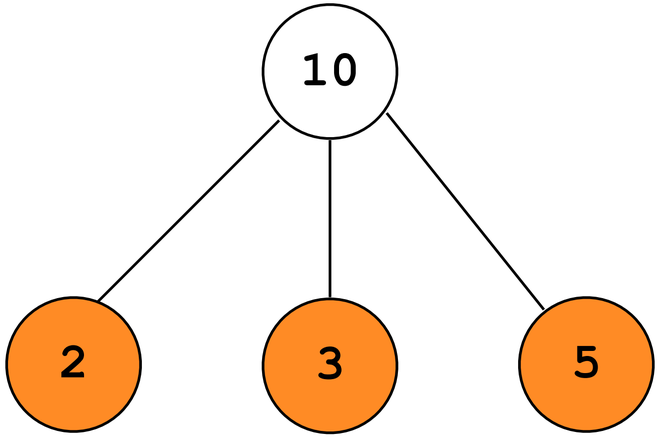
带权路径长度 WPL：2*3+4*3+5*2+15*1=43

最佳归并树

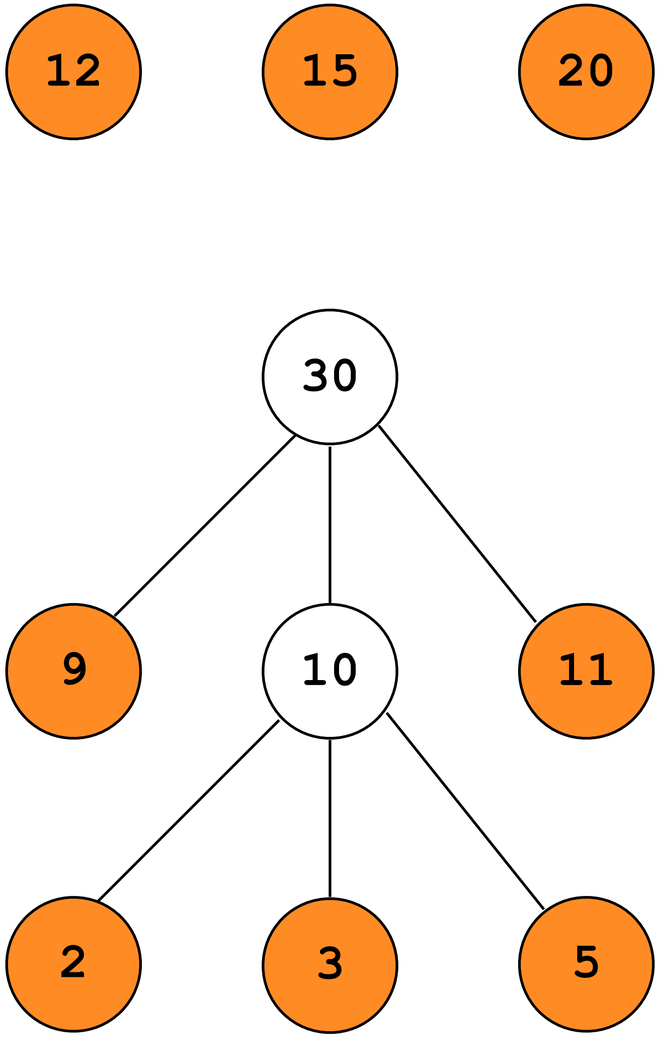


最佳归并树

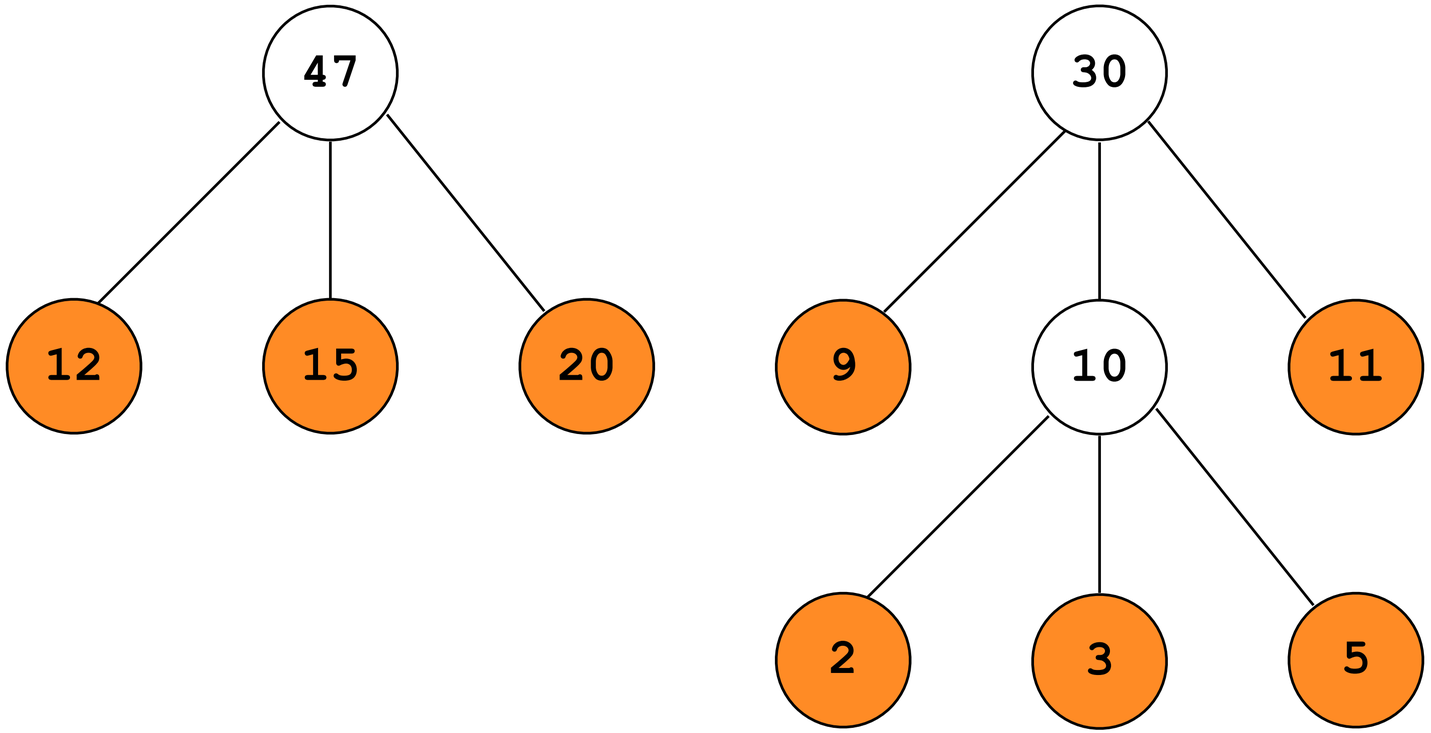




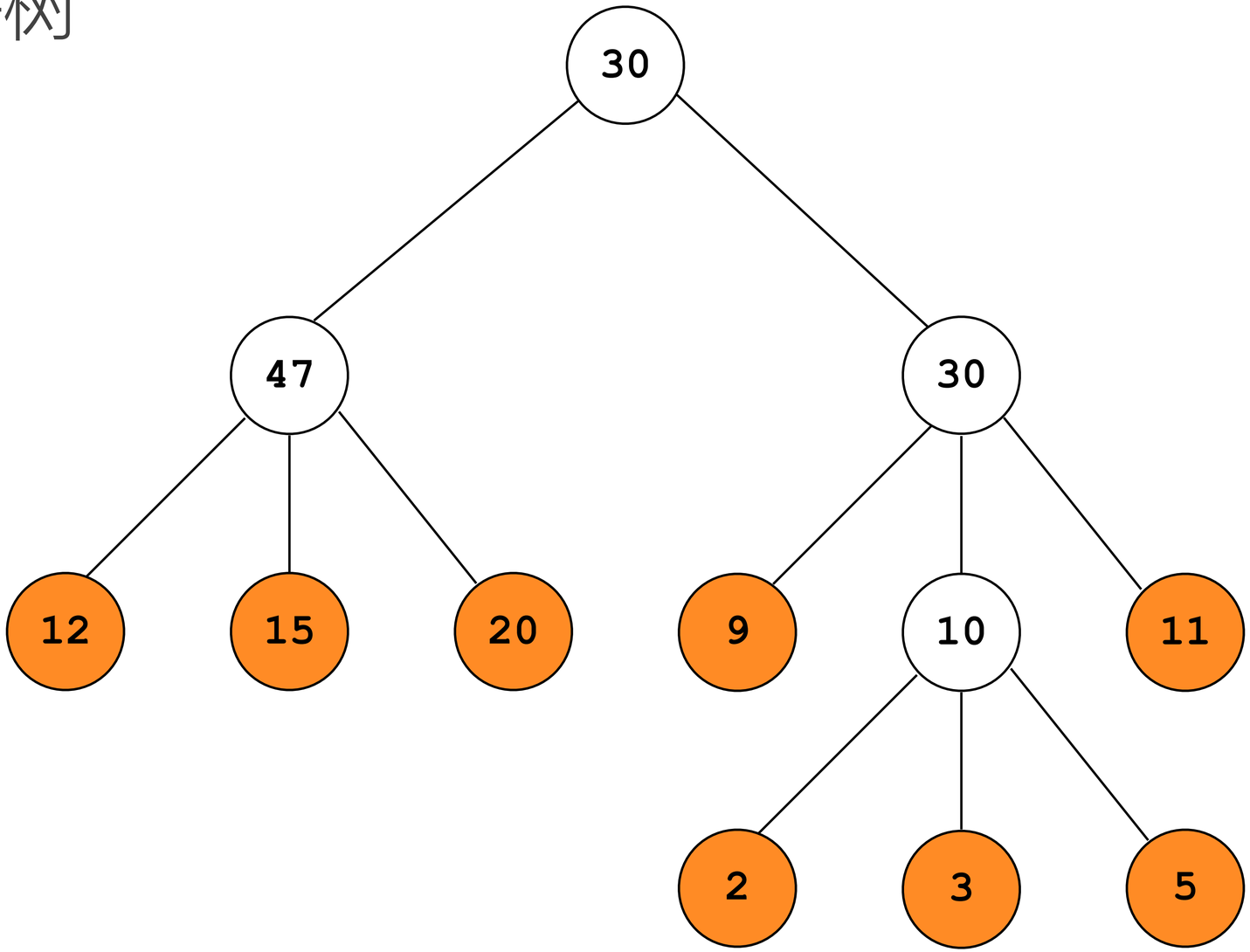
最佳归并树



最佳归并树

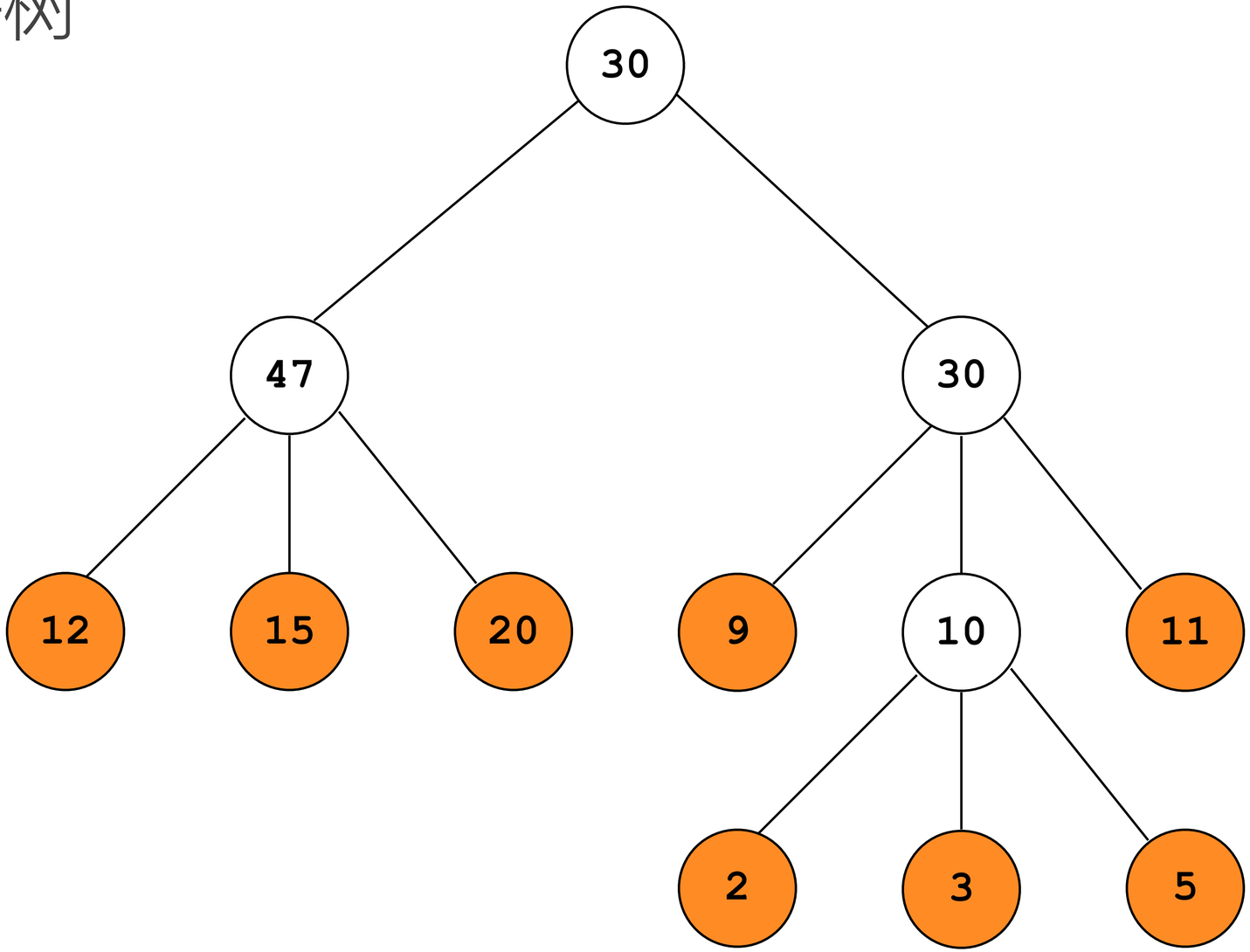


最佳归并树



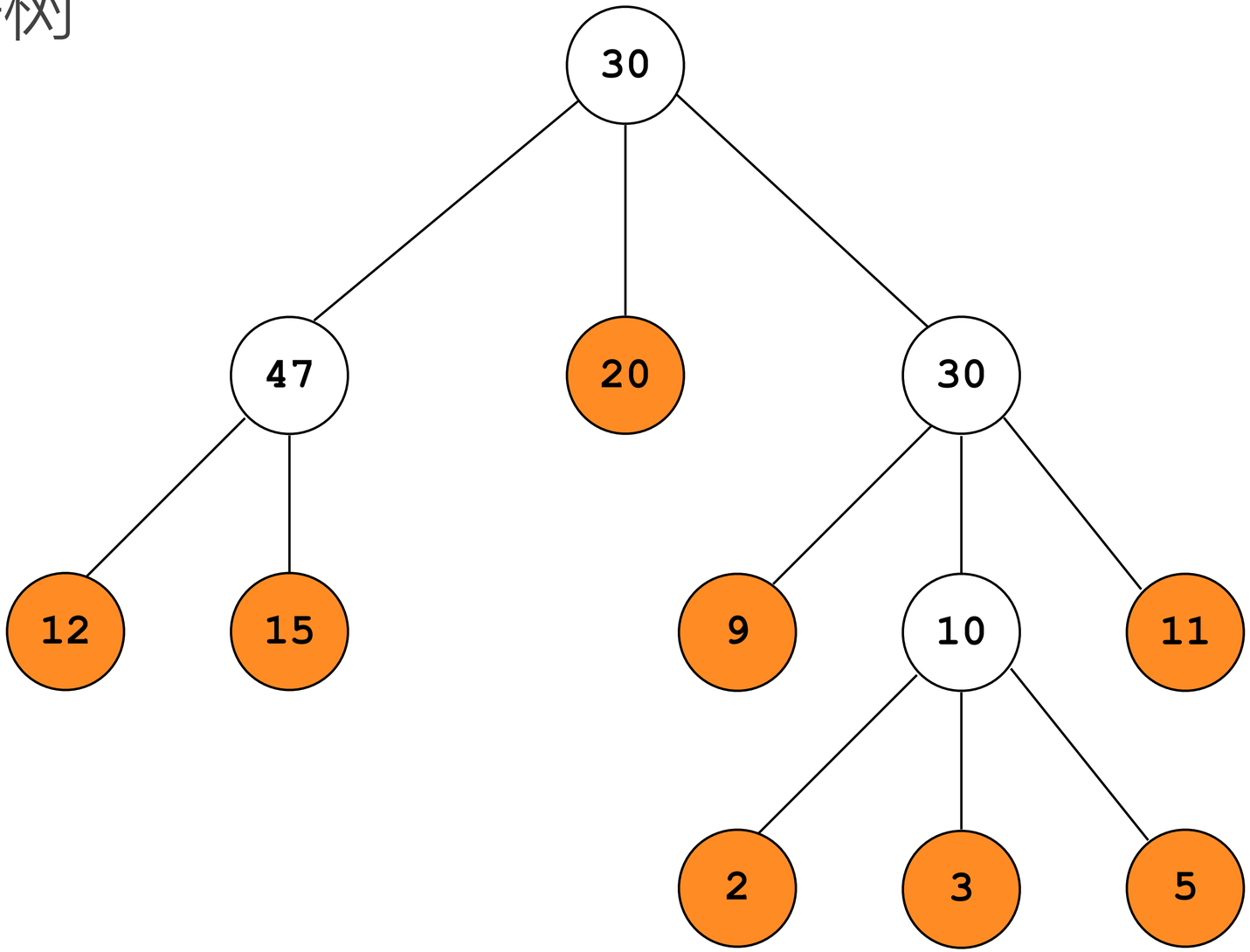
最佳归并树

并不是最优的

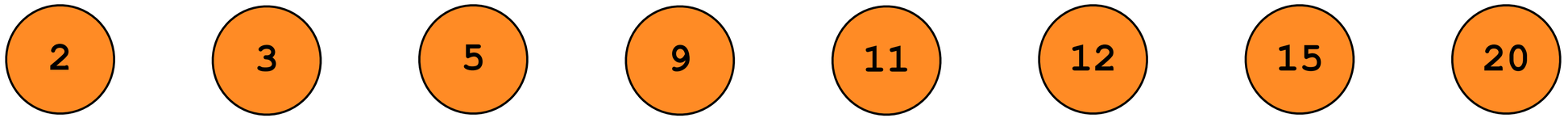


最佳归并树

并不是最优的



最佳归并树

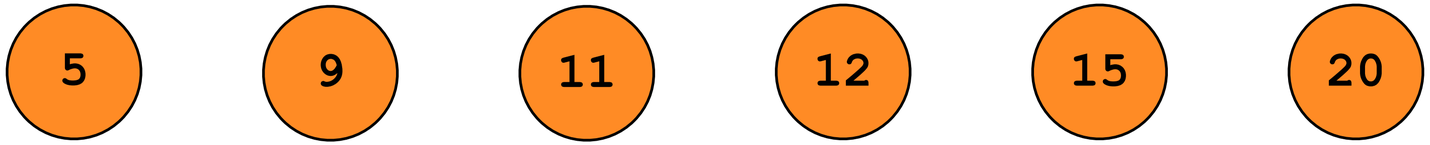


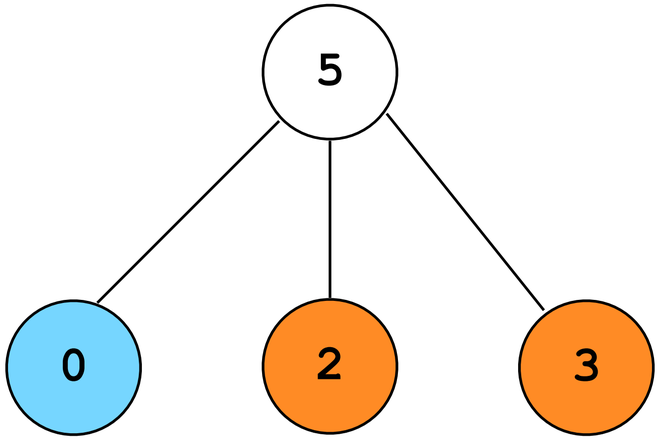
最佳归并树



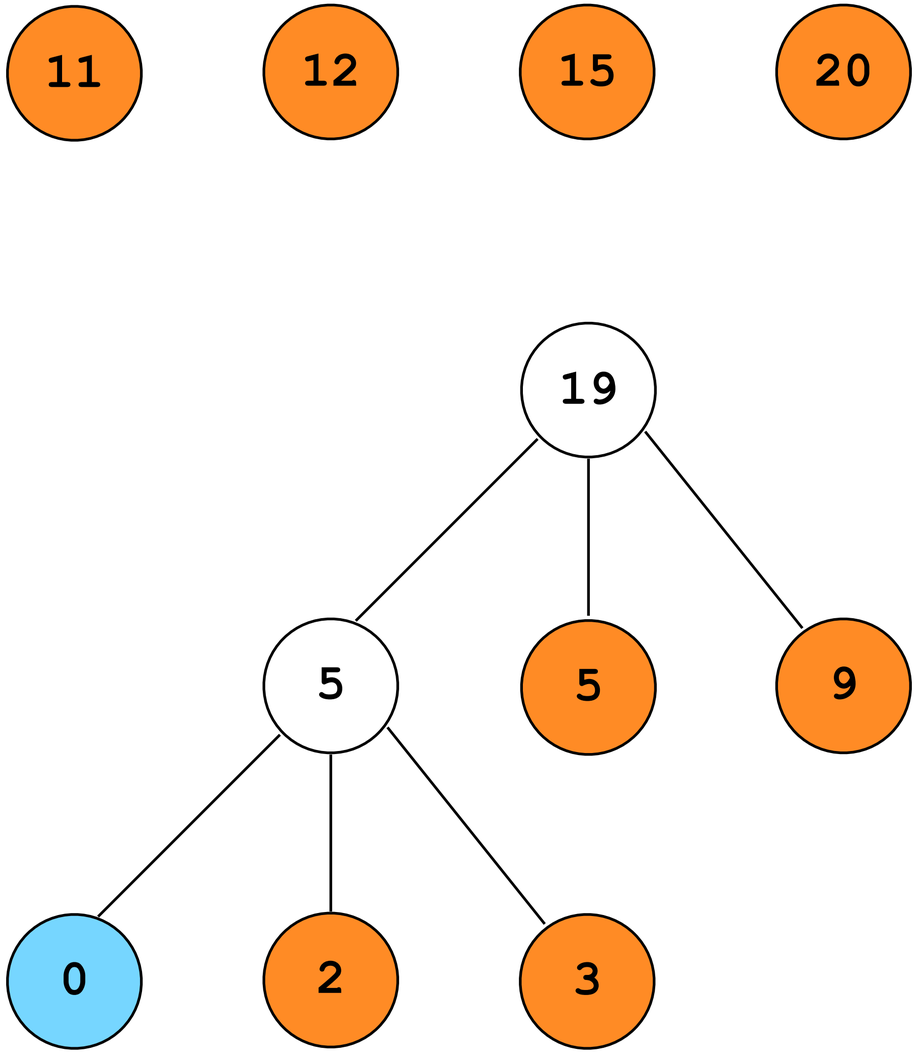
虚段

最佳归并树

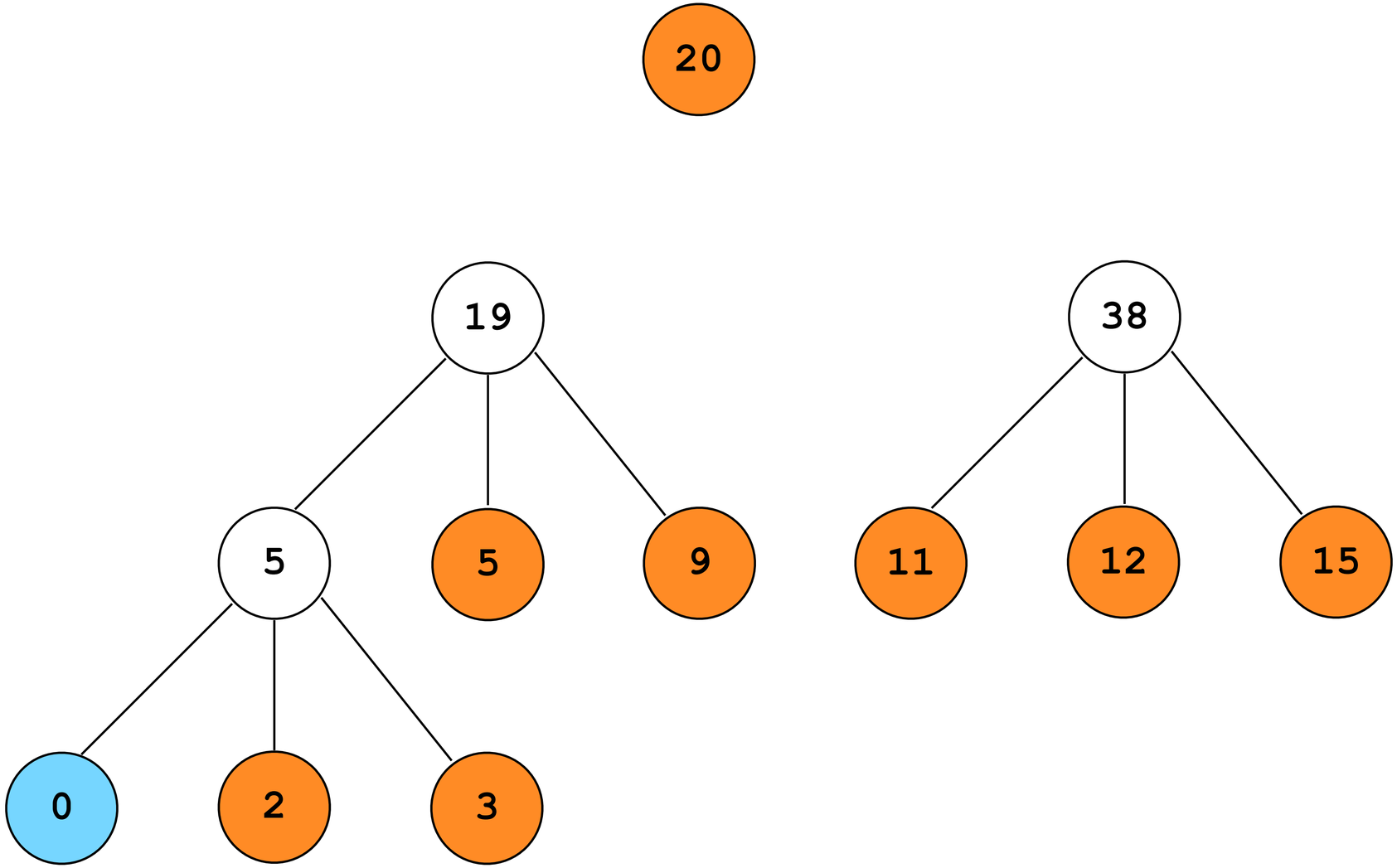




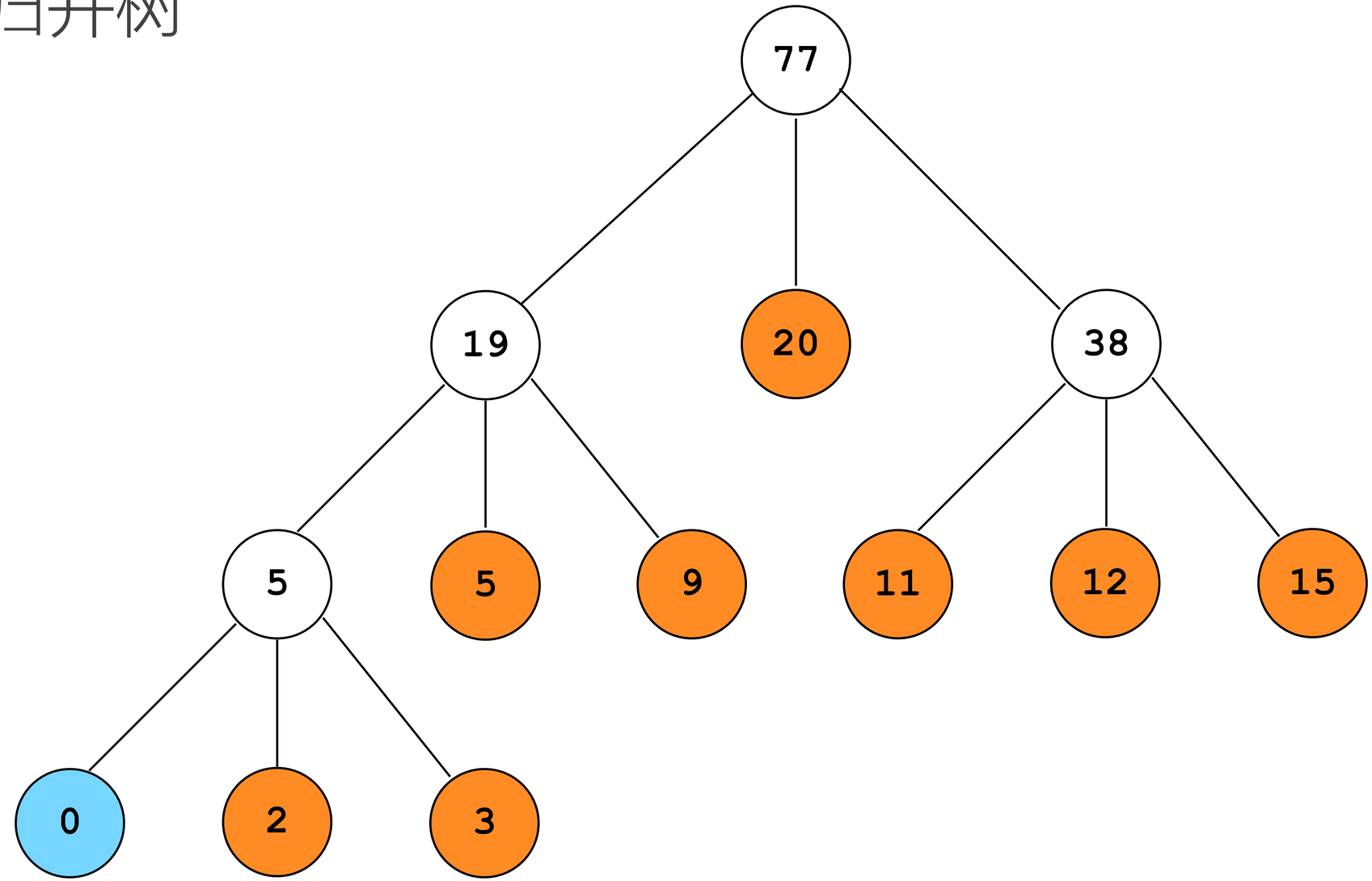
最佳归并树



最佳归并树



最佳归并树



最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$　　$kn_k + 1 = n$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

$kn_k + 1 = n$

$kn_k + 1 = n_0 + n_k$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

$$kn_k + 1 = n$$

$$kn_k + 1 = n_0 +$$

$$kn_k - n_k = n_0 - 1$$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

$$kn_k + 1 = n$$

$$kn_k + 1 = n_0 +$$

$$kn_k - n_k = n_0 - 1$$

$$n_k(k-1) = n_0 - 1$$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

$$kn_k + 1 = n$$

$$kn_k + 1 = n_0 +$$

$$kn_k - n_k = n_0 - 1$$

$$n_k(k-1) = n_0 - 1$$

$$n_k = (n_0 - 1) / (k - 1)$$

最佳归并树-虚段

k 叉最佳归并树，度为 k 的结点个数为 n_k，度为 0 的个数为 n_0

结点总数：$n = n_0 + n_k$

结点总度数：$kn_k = n - 1$

$$kn_k + 1 = n$$

$$kn_k + 1 = n_0 +$$

$$kn_k - n_k = n_0 - 1$$

$$n_k(k-1) = n_0 - 1$$

$$n_k = (n_0 - 1) / (k - 1)$$

n0：初始归并段的个数 + 虚段的个数

真题解析

[2016] 11. 对 10 TB 的数据文件进行排序，应使用的方法是_____。

A. 希尔排序　　B. 堆排序　　C. 快速排序　　D. 归并排序

真题解析

[2016] 11. 对 10 TB 的数据文件进行排序，应使用的方法是_____。

A. 希尔排序　　B. 堆排序　　C. 快速排序　　D. 归并排序

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

$$n_k = (n_0 - 1) / (k - 1)$$

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

$$n_k = (n_0 - 1) / (k - 1)$$

$$n_k = (n_0 - 1) / (12 - 1)$$

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

$$n_k = (n_0 - 1) / (k - 1)$$

$$n_k = (n_0 - 1) / (12 - 1)$$

$$n_k = (n_0 - 1) / 11$$

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

$$n_k = (n_0 - 1) / (k - 1)$$

$$n_k = (n_0 - 1) / (12 - 1)$$

$$n_k = (n_0 - 1) / 11$$

$$n_k = ((120 + x) - 1) / 11$$

真题解析

[2019] 11. 设外存上有 120 个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是_____。

A. 1　　B. 2　　C. 3　　D. 4

真题解析

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

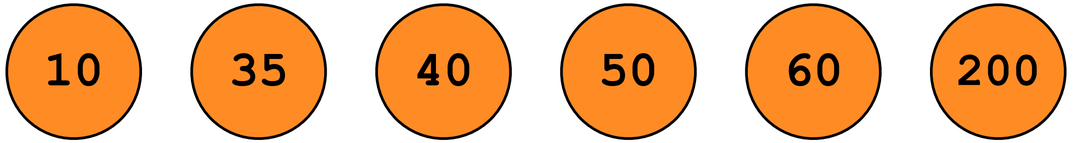
1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

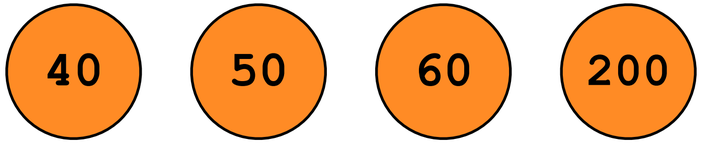
2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

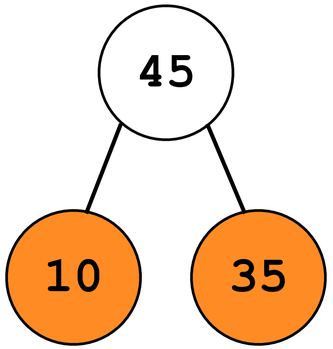


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



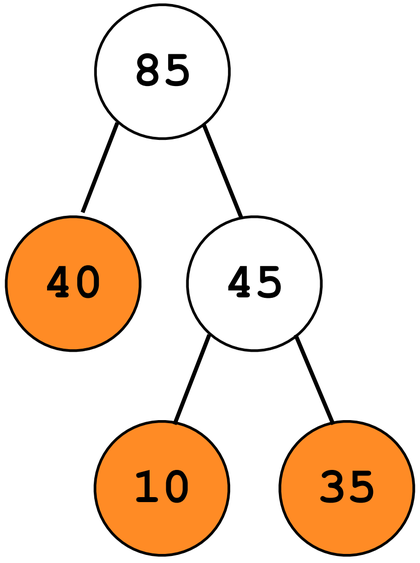


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

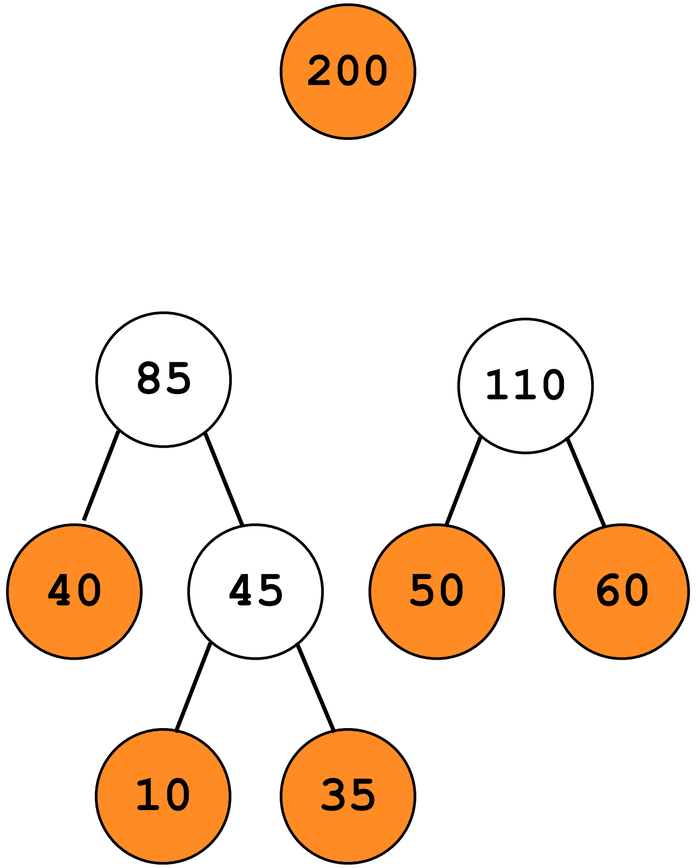




[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

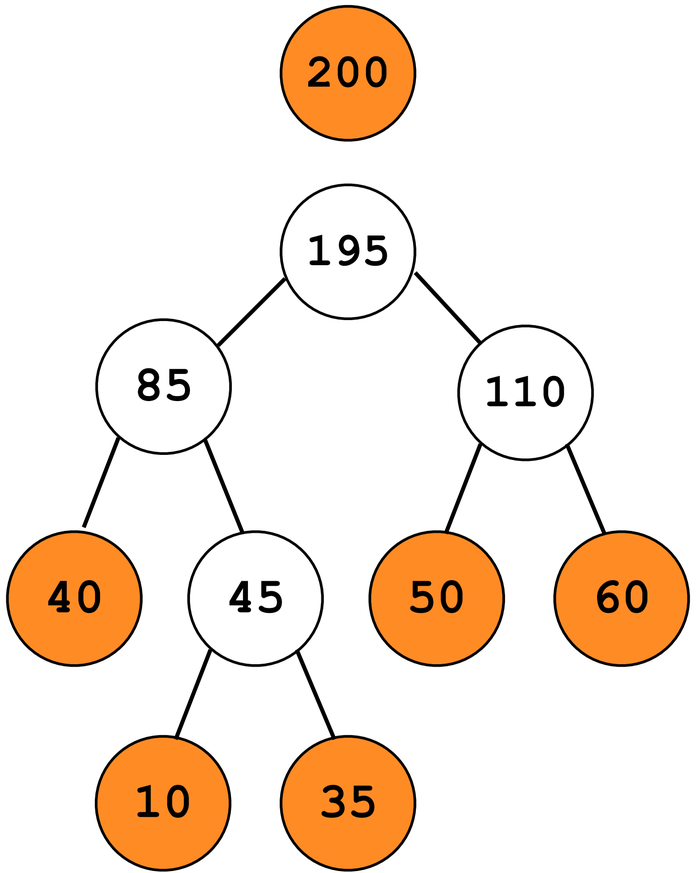
2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

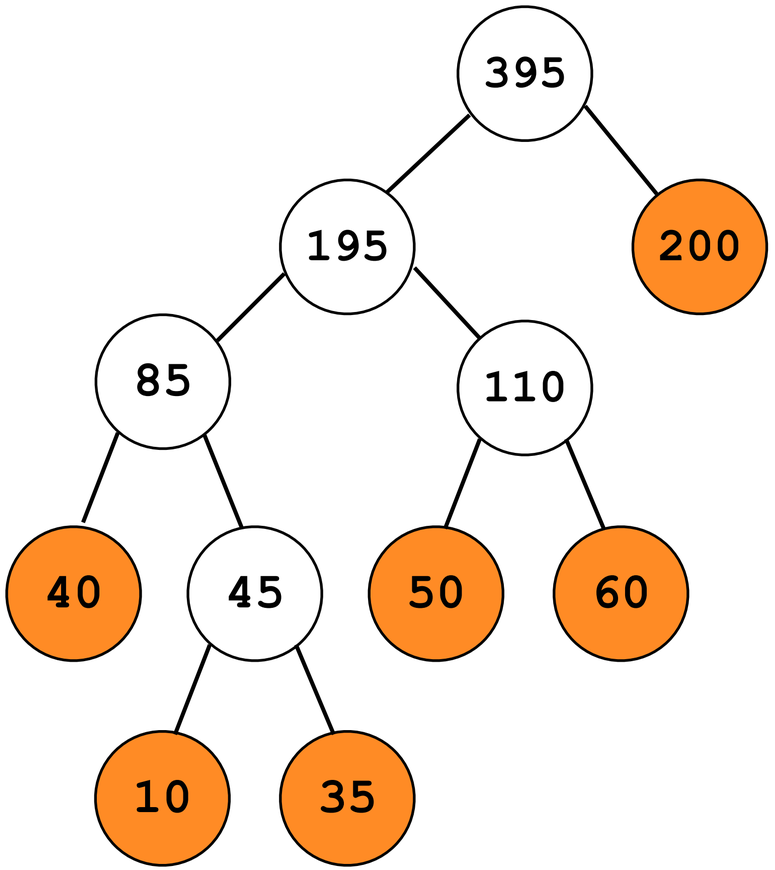
2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

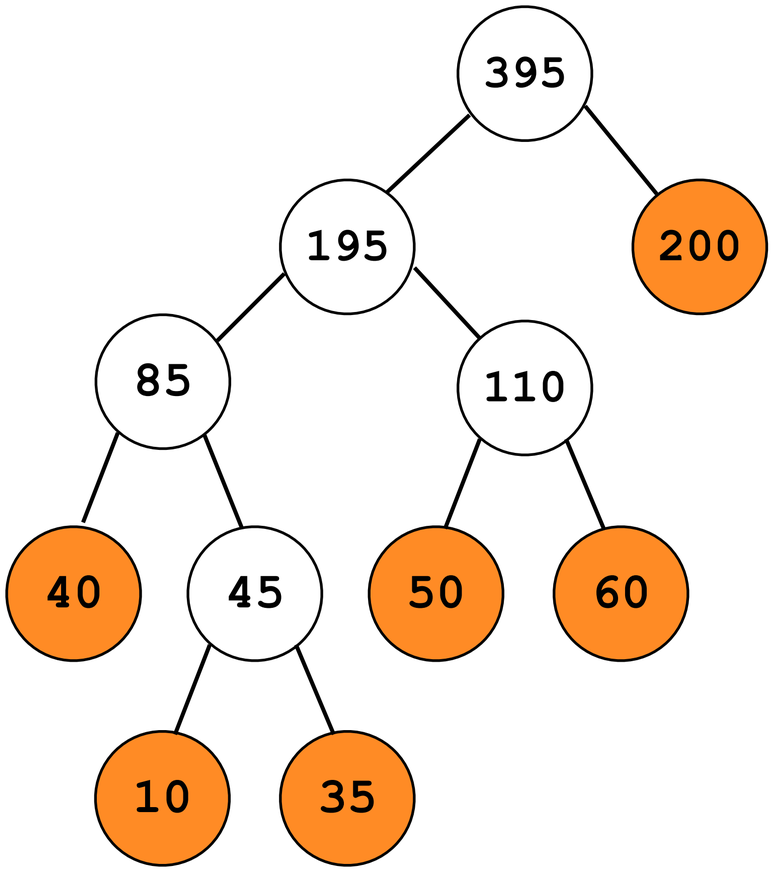
2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1 ）给出完整的合并过程，并求出最坏情况下比较的总次数。

2 ）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



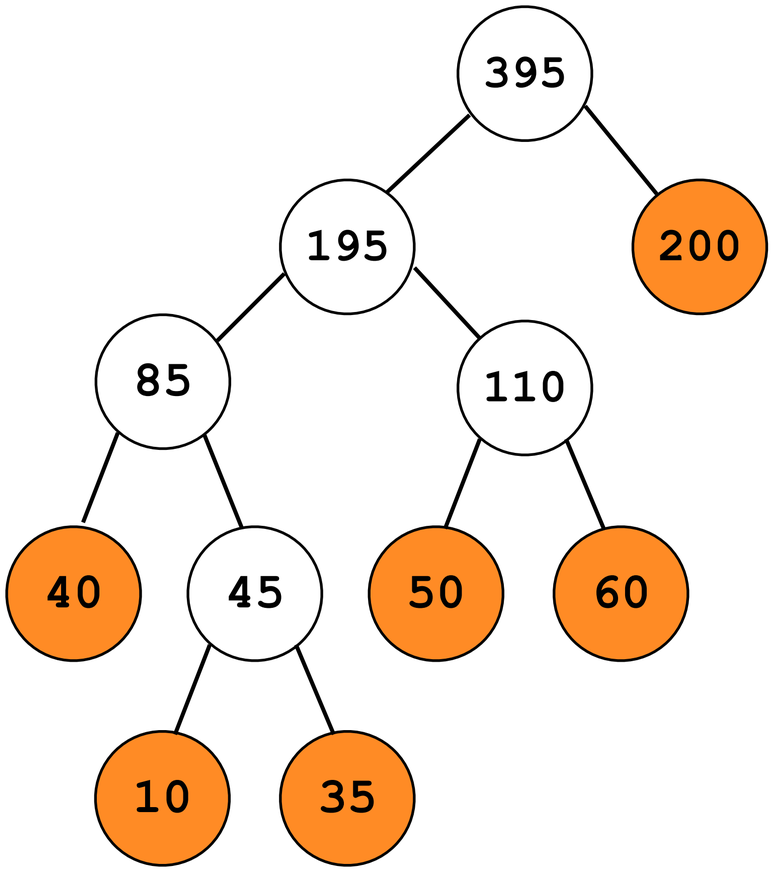
1, 2, 3, 4, 5

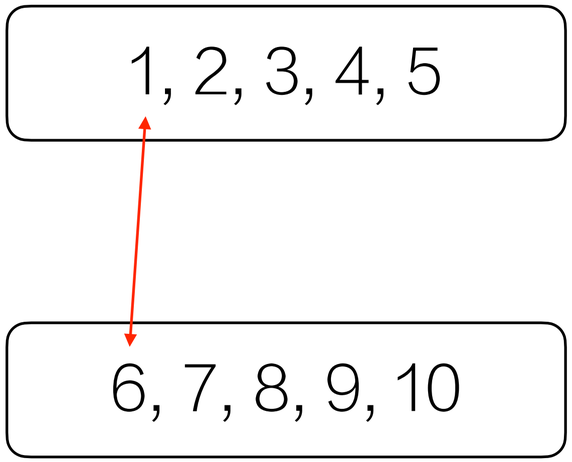
6, 7, 8, 9, 10

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

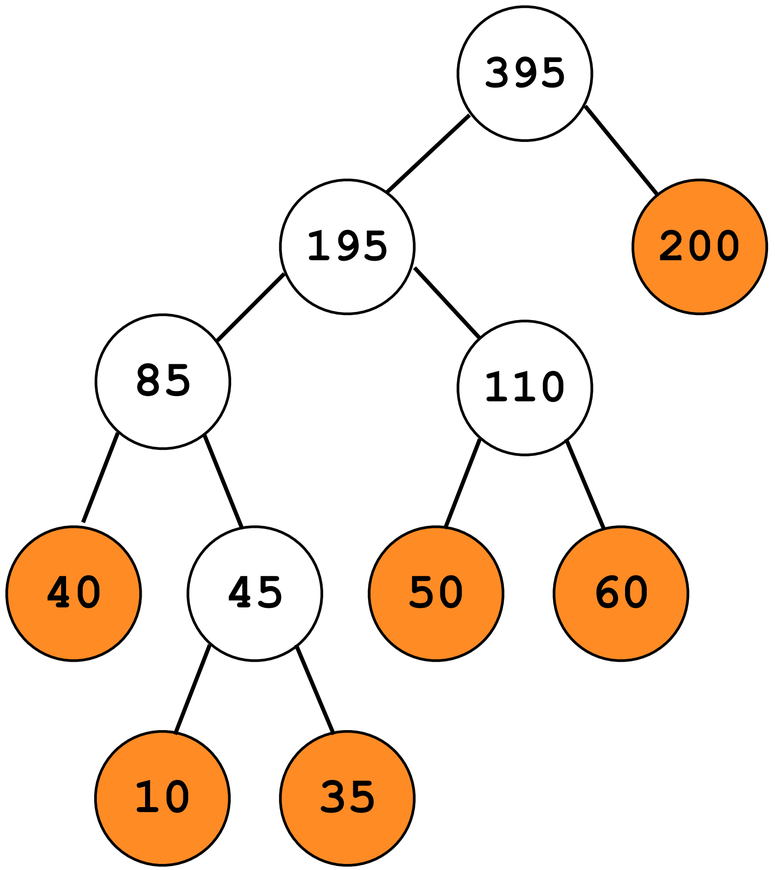


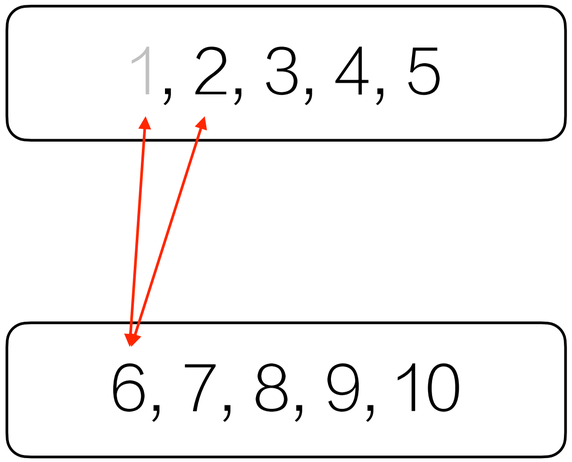


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

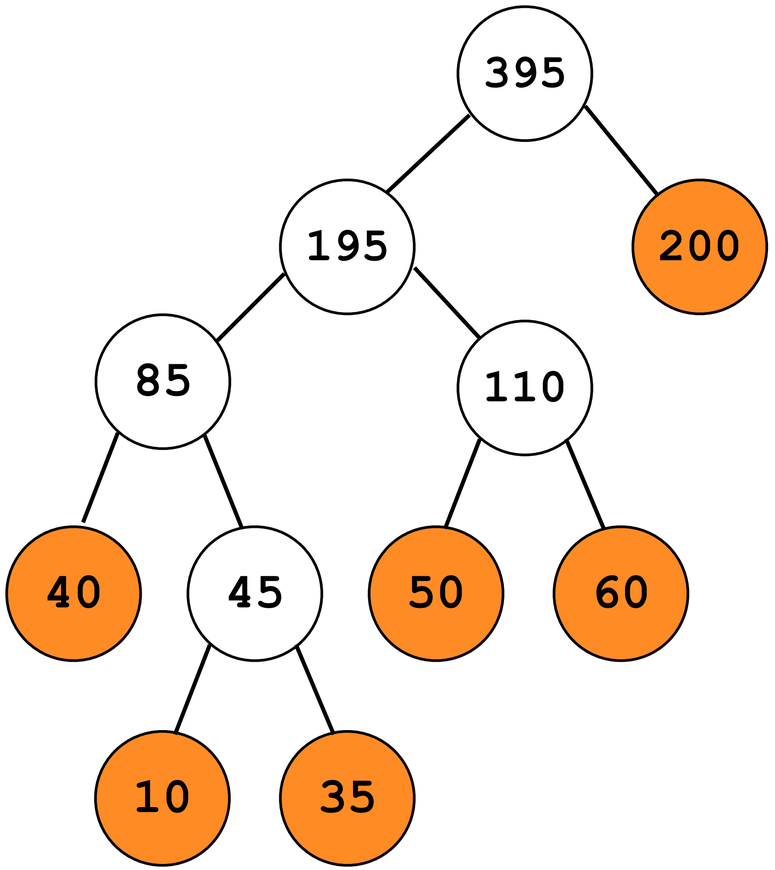


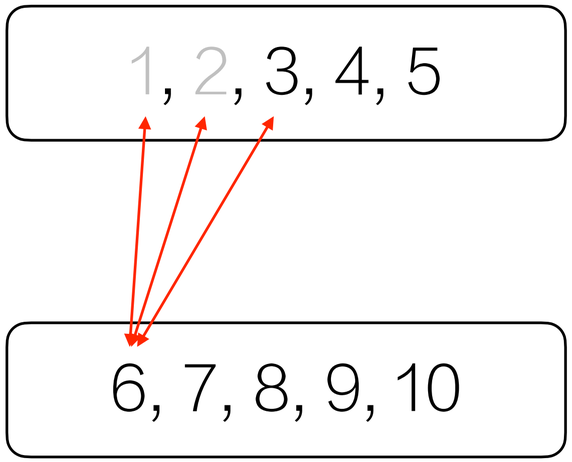


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

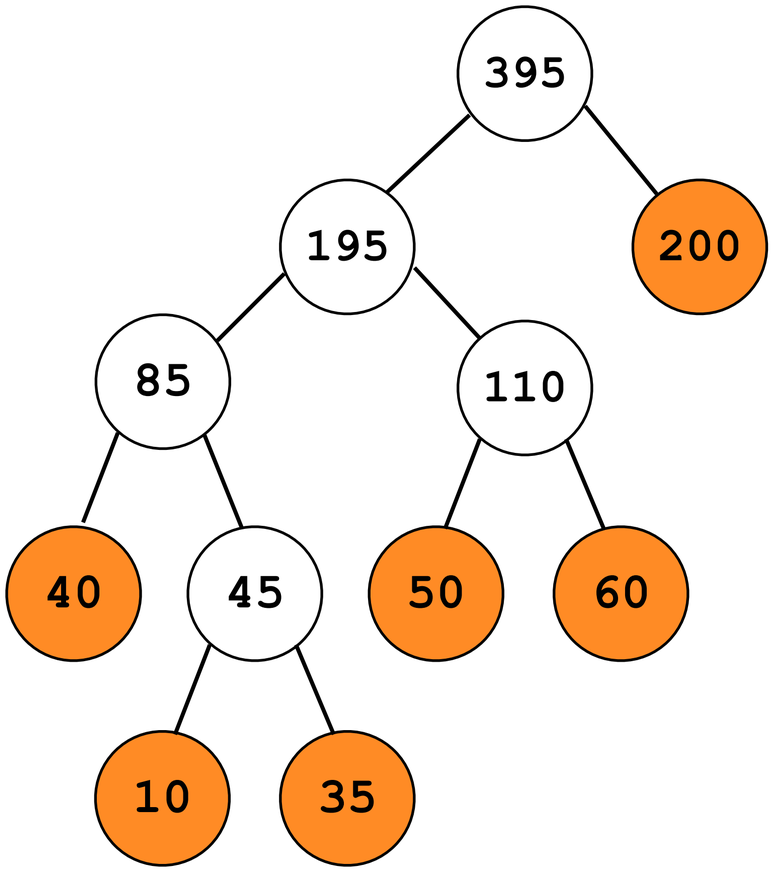


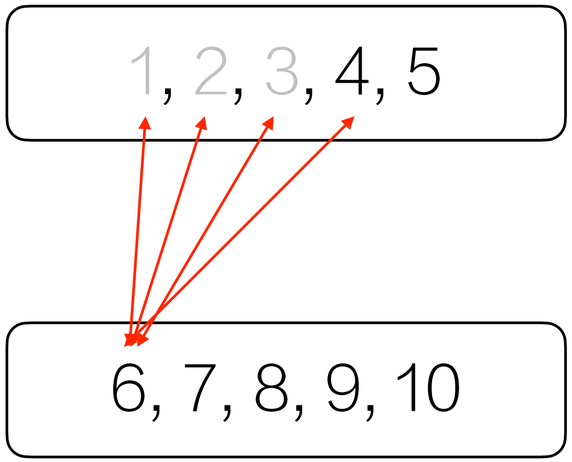


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

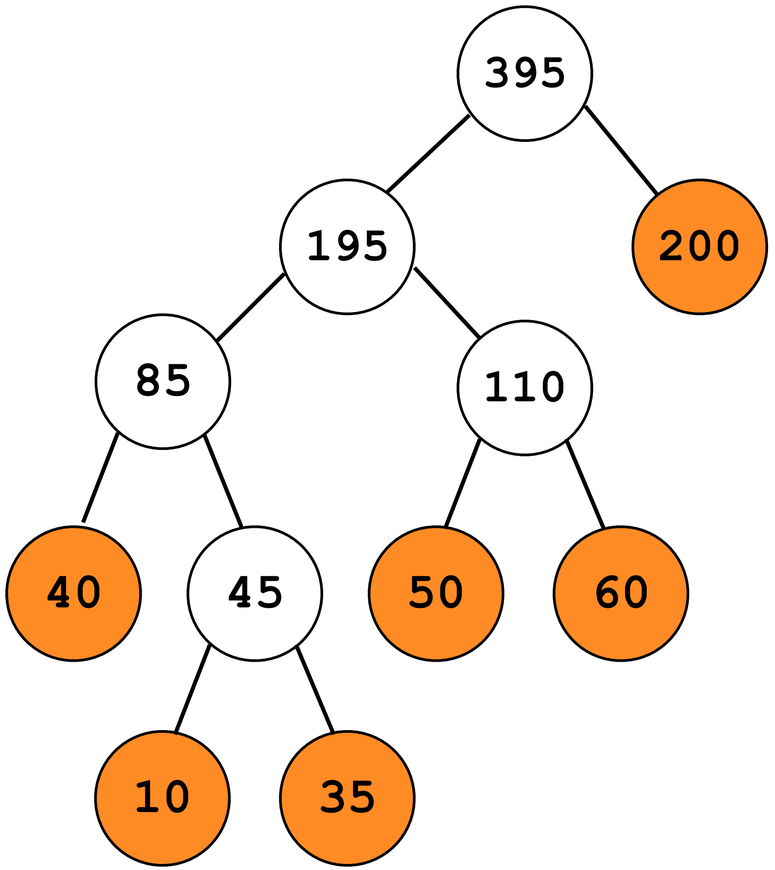


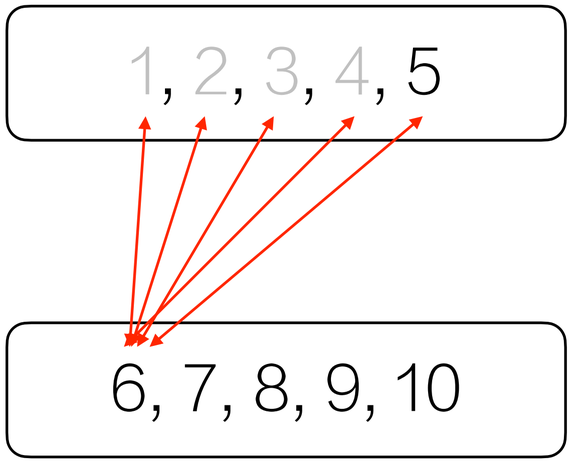


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

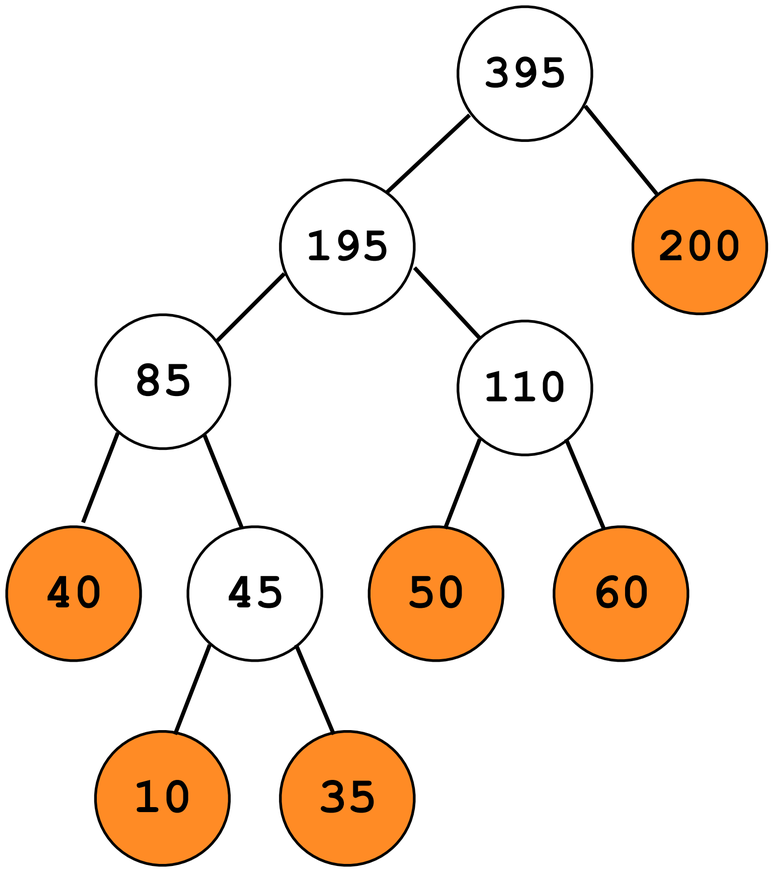


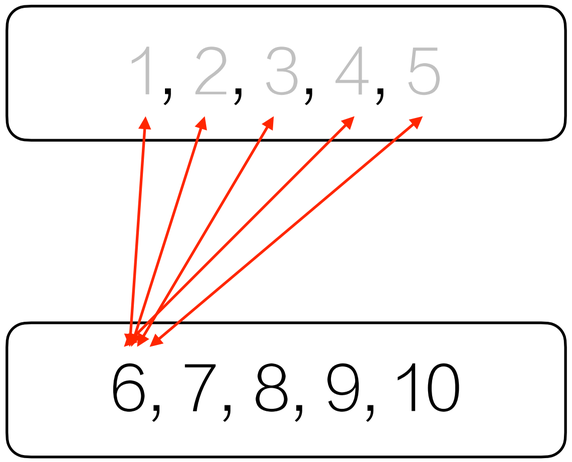


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

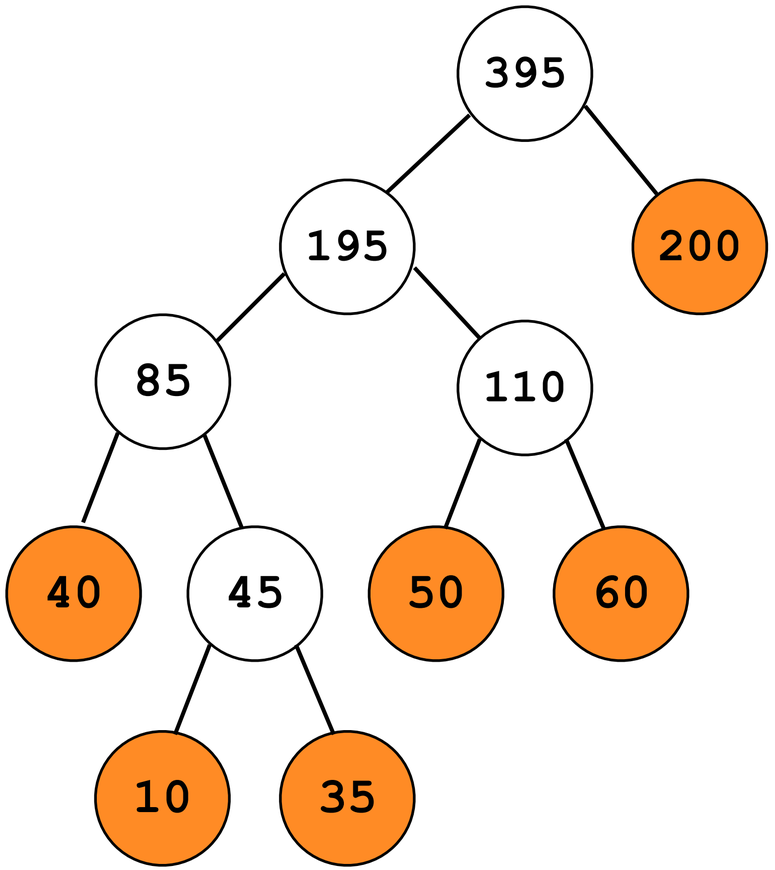




[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



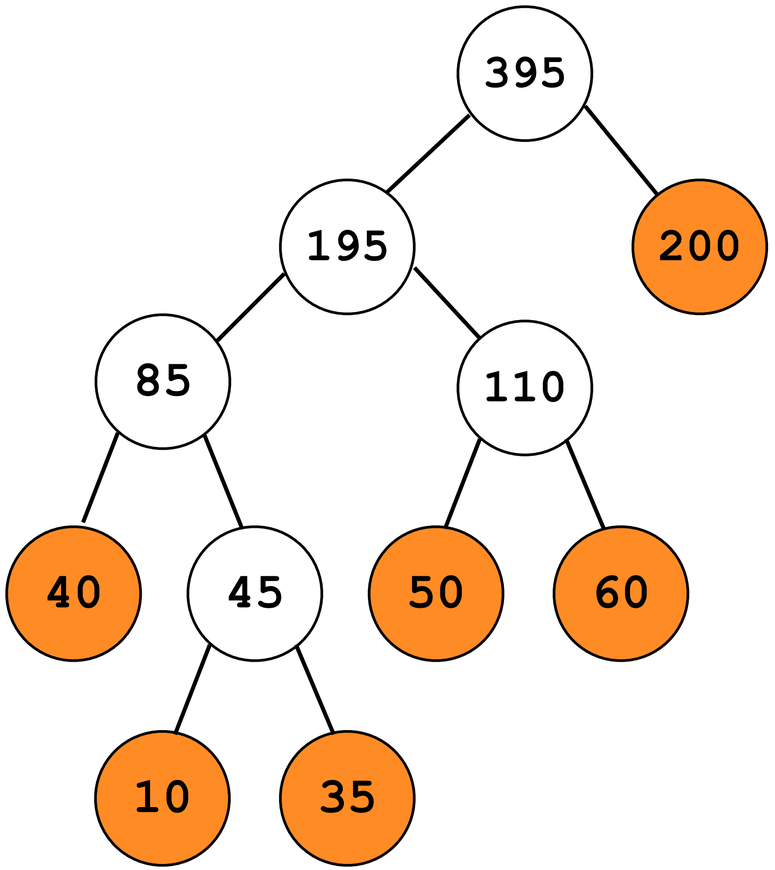
1, 3, 5, 7, 9

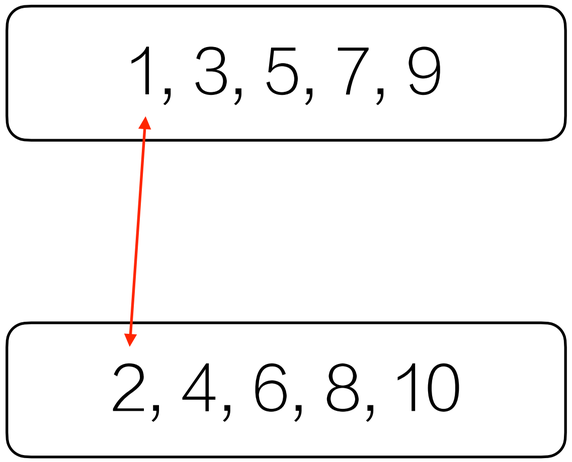
2, 4, 6, 8, 10

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

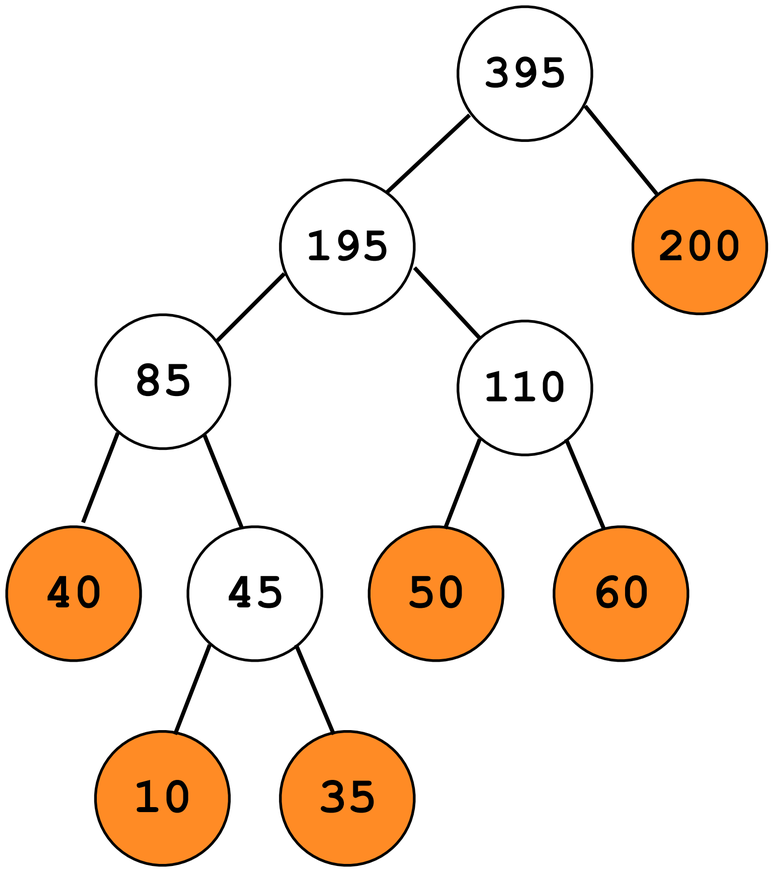


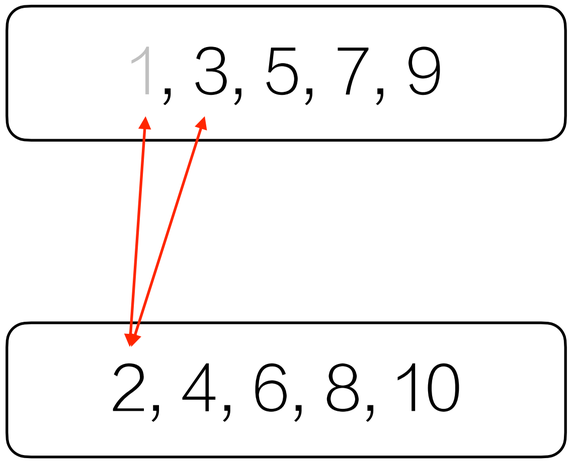


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

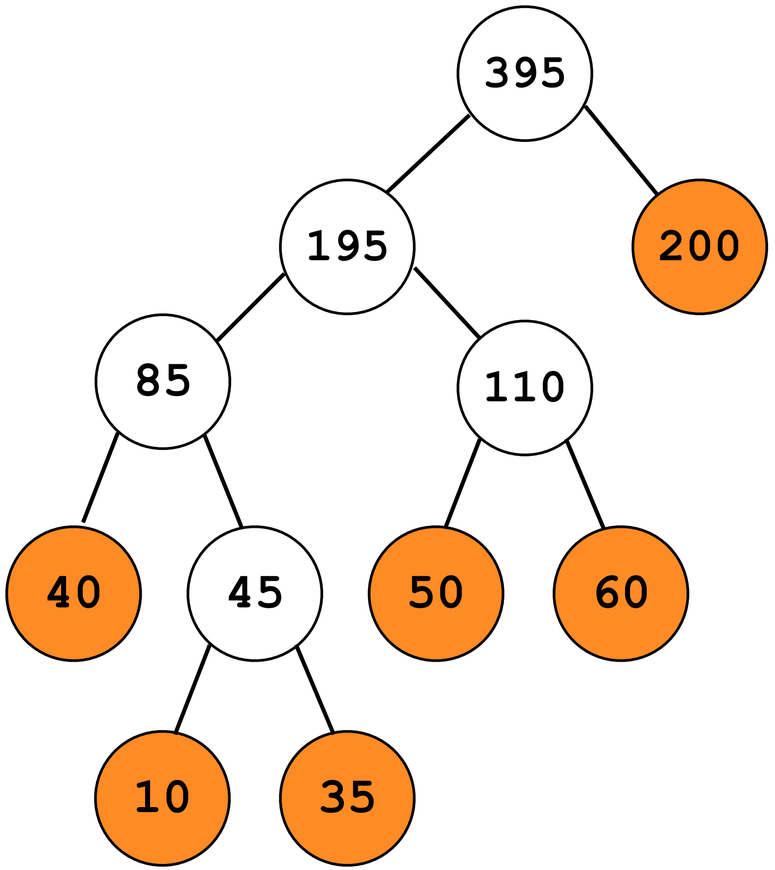


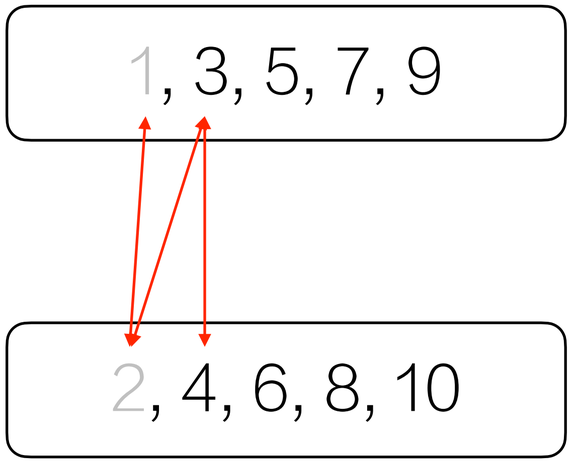


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

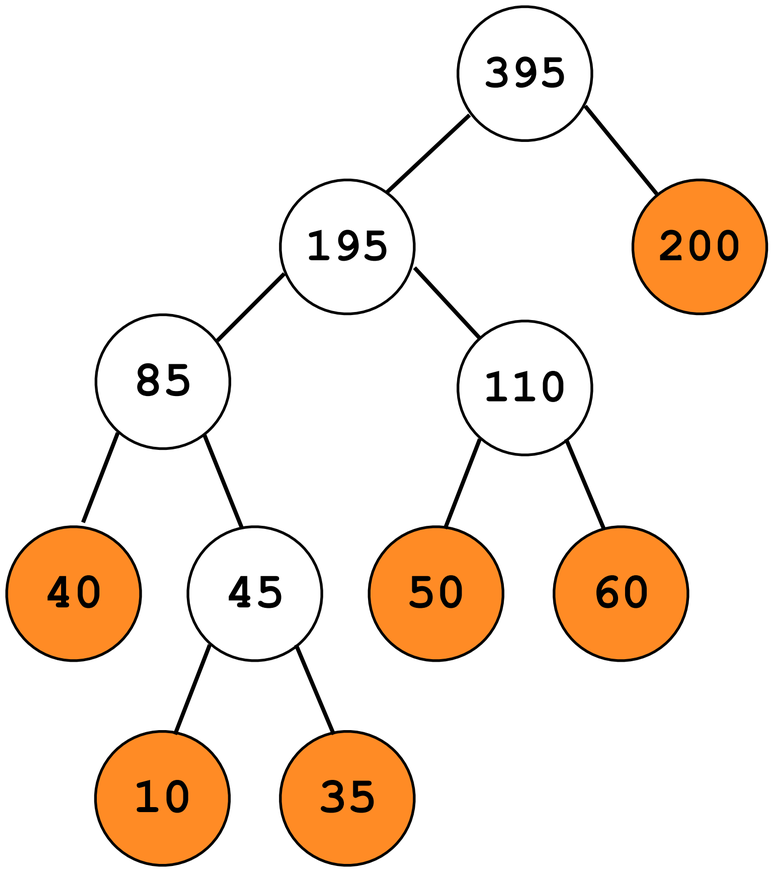


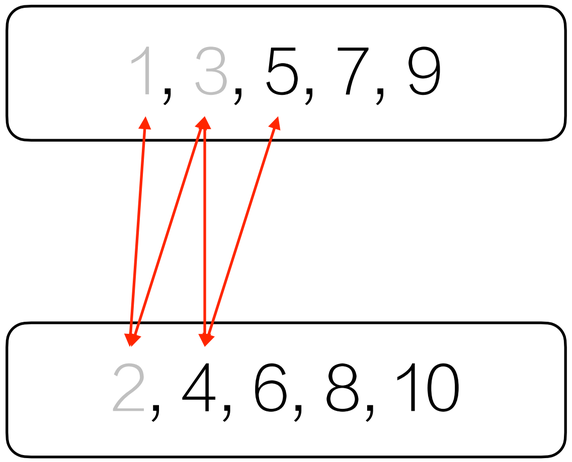


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

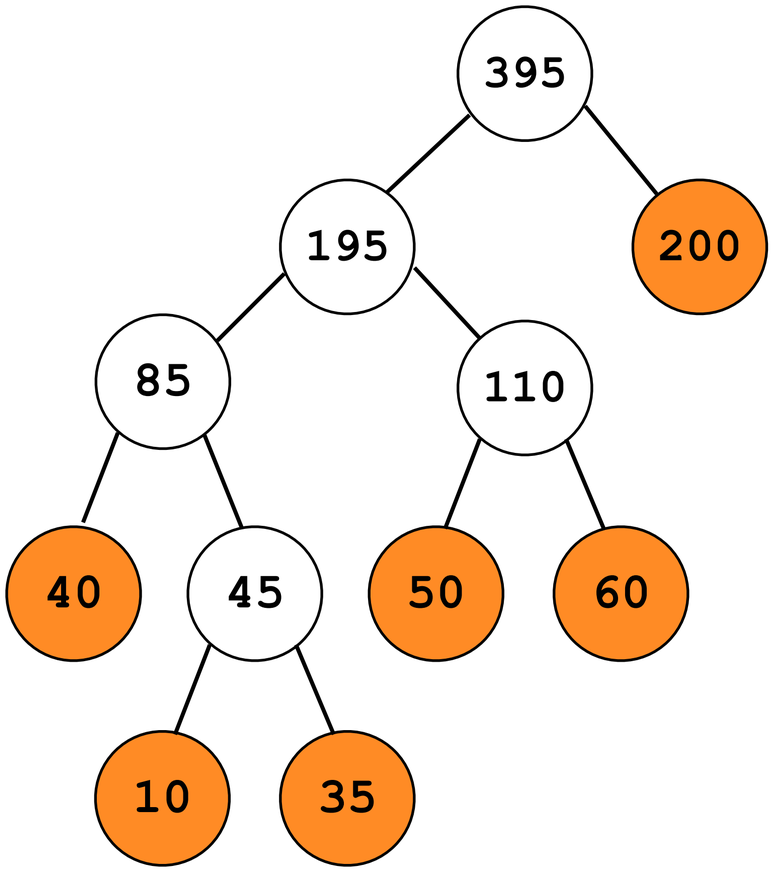


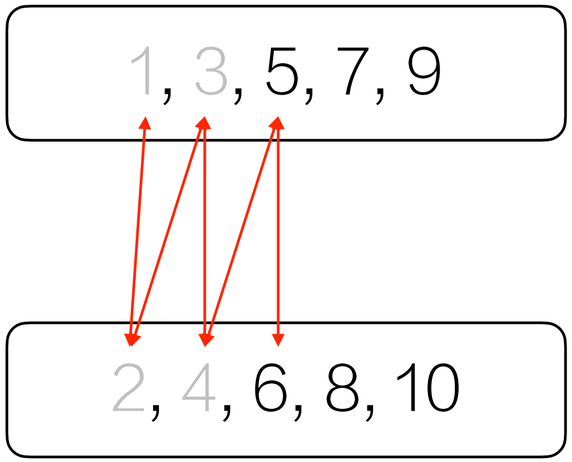


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

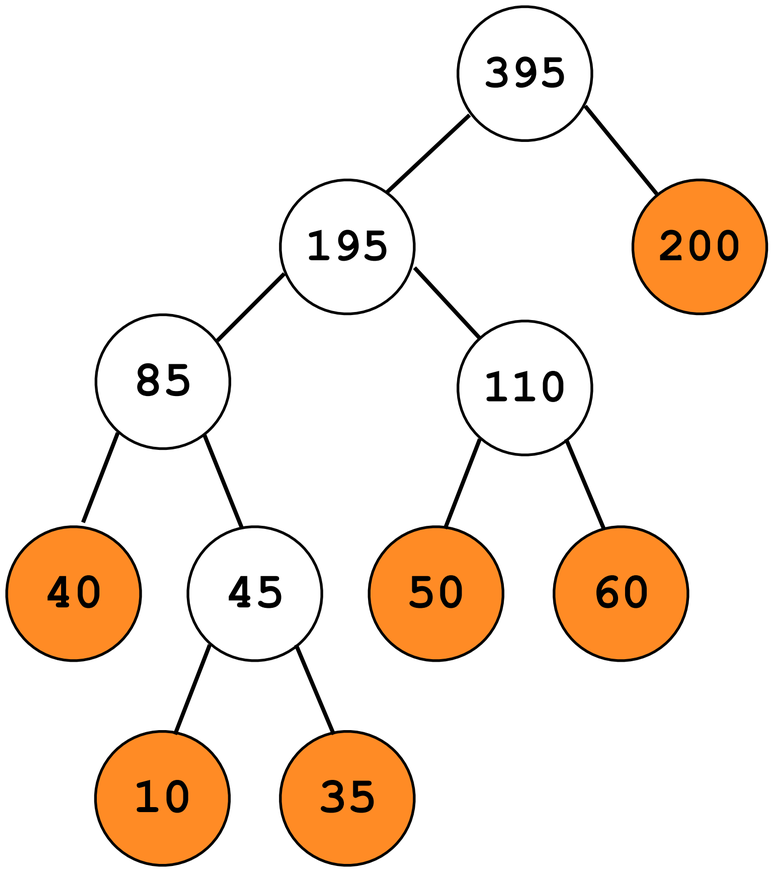


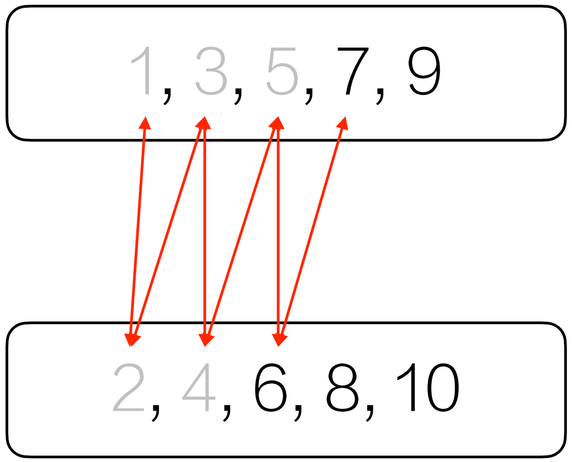


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

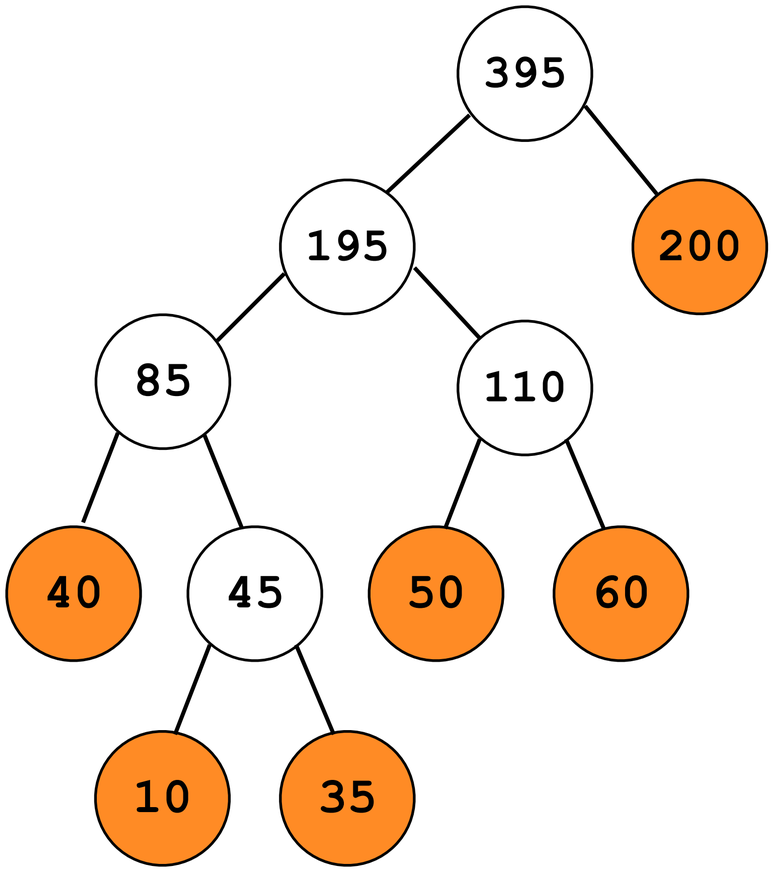


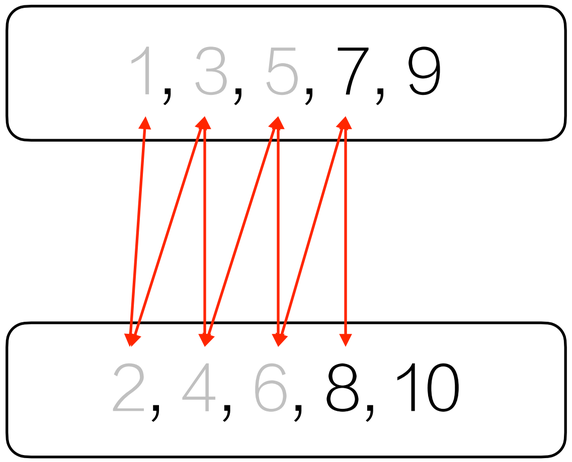


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

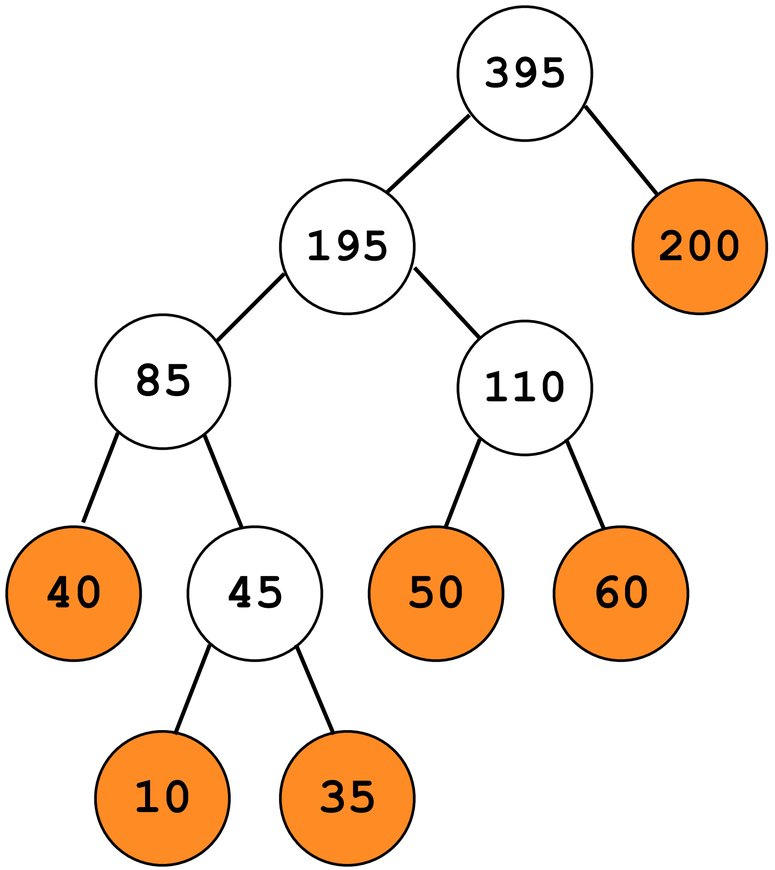


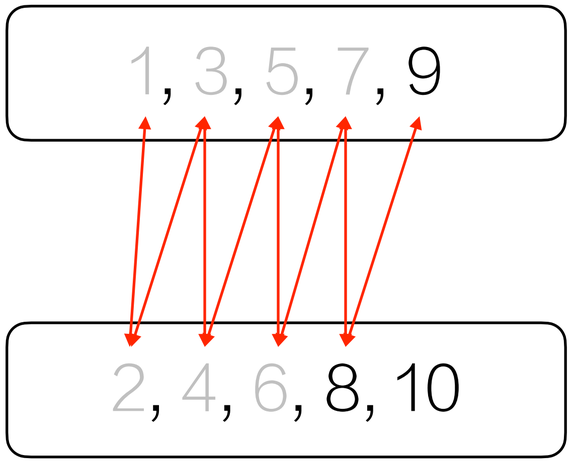


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

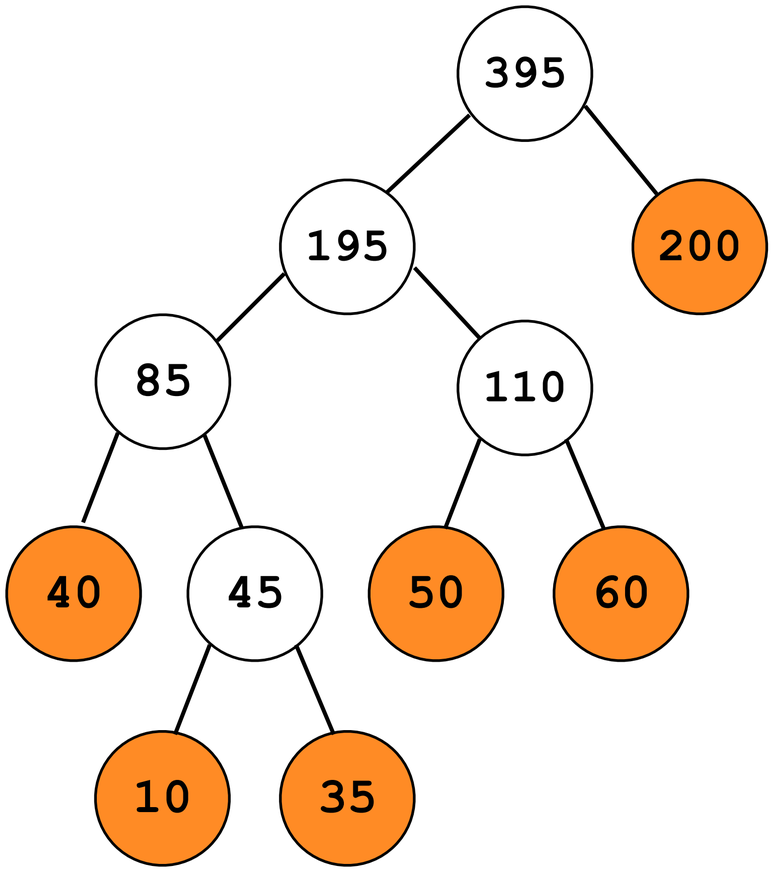


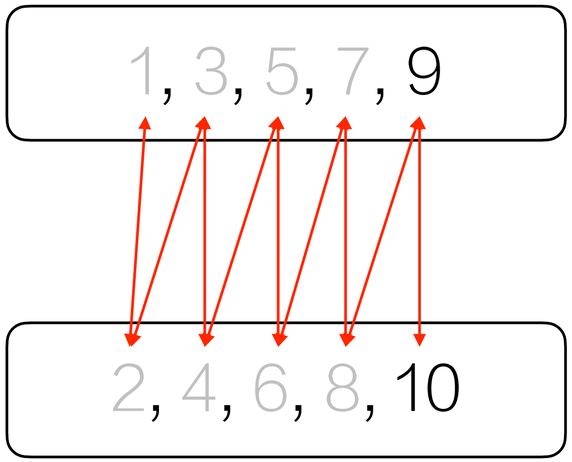


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。

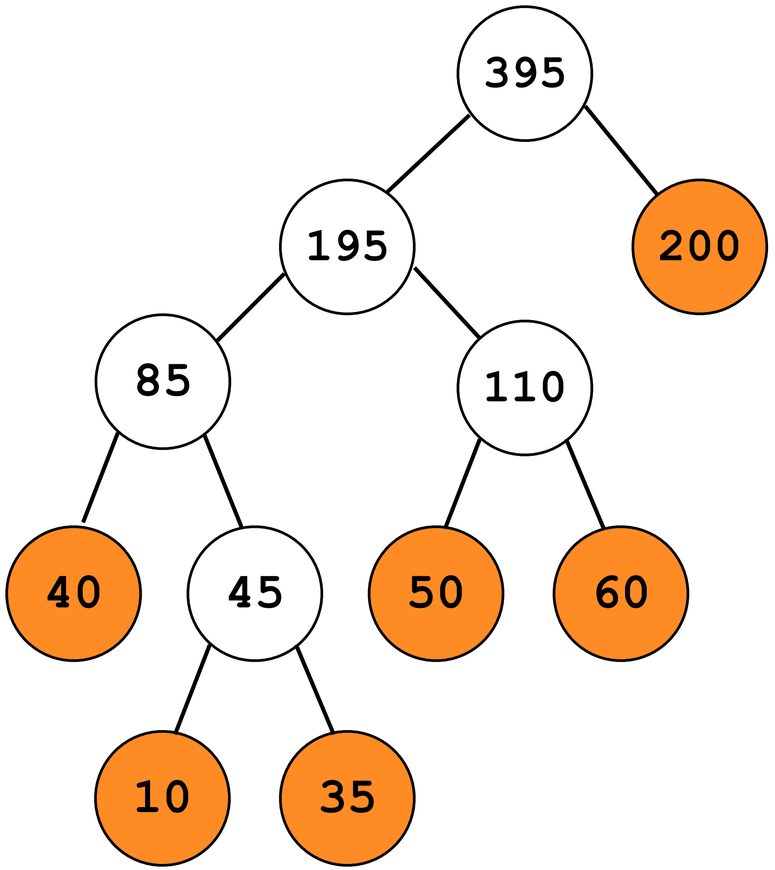


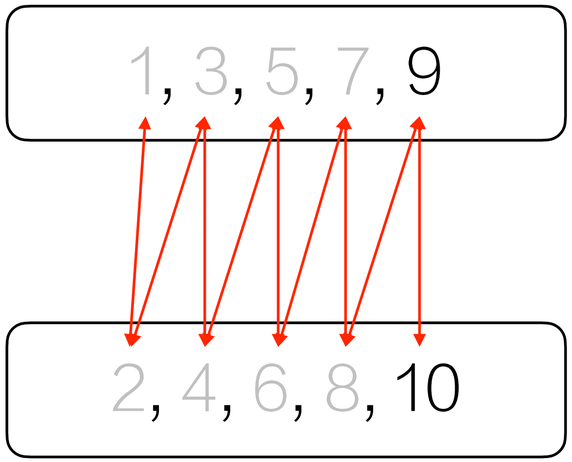


[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。



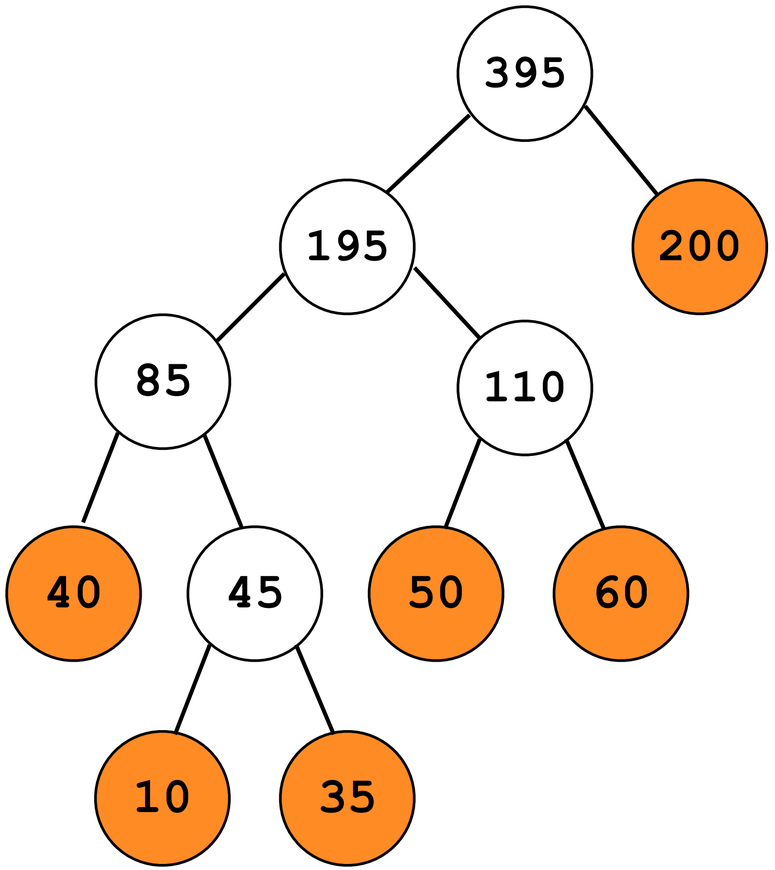


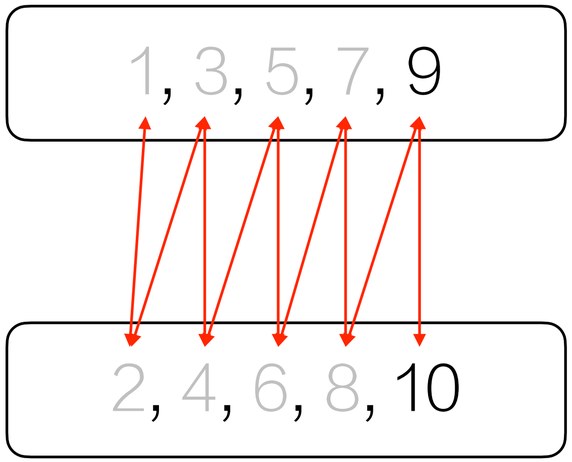
共 10 个元素比较 9 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





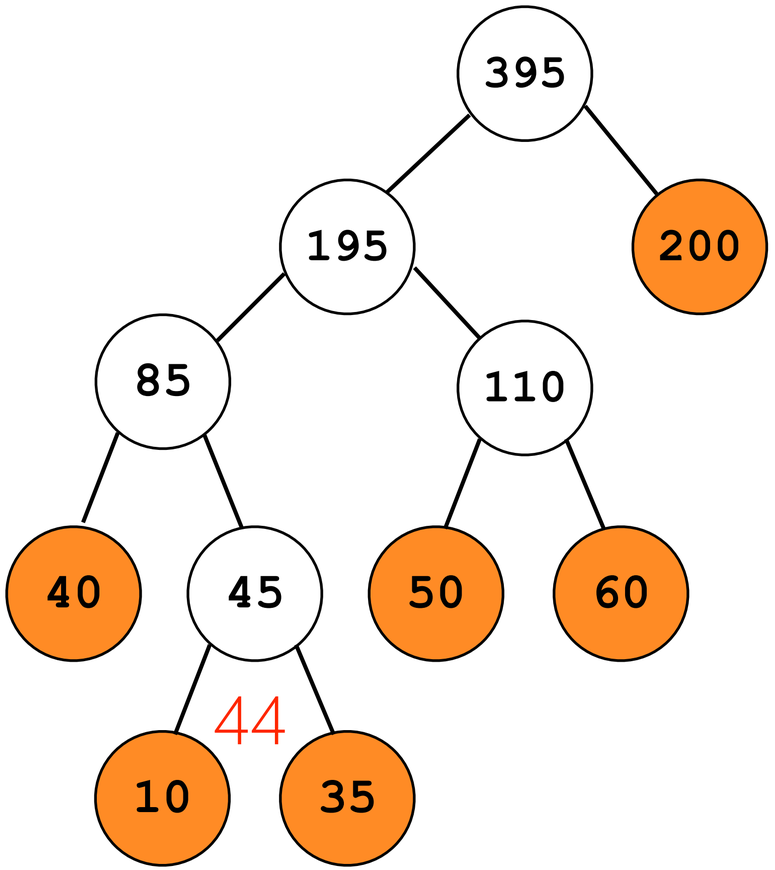
共 10 个元素比较 9 次

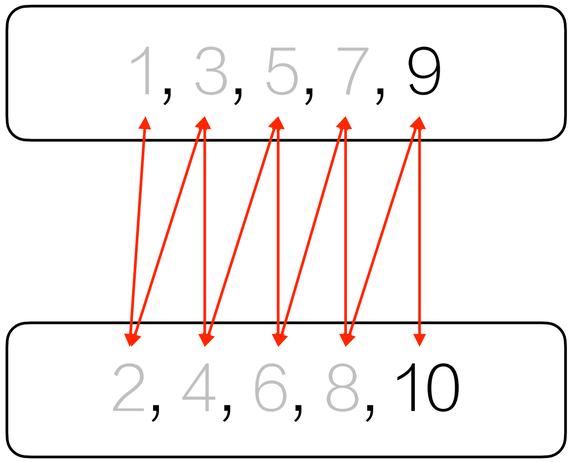
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





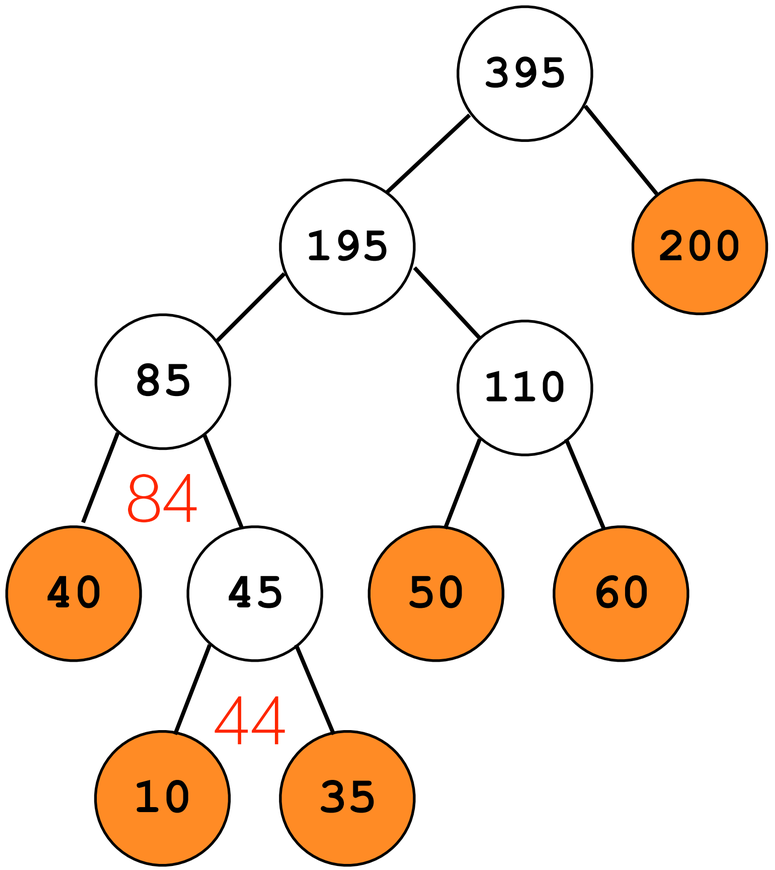
共 10 个元素比较 9 次

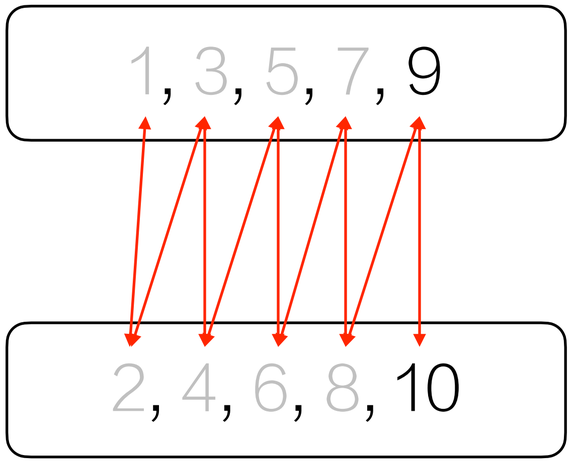
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





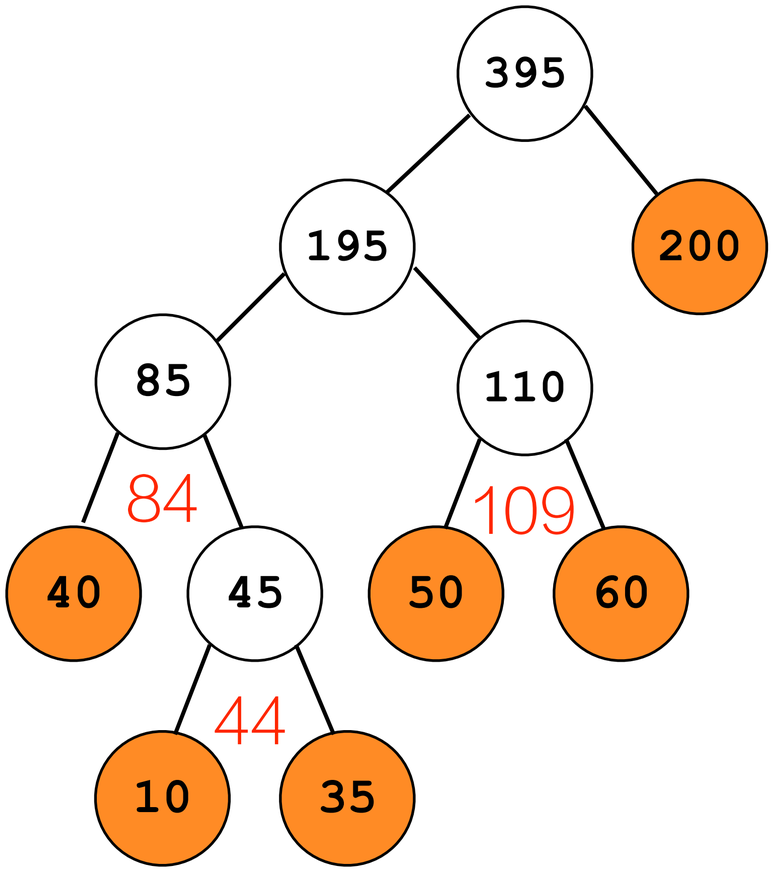
共 10 个元素比较 9 次

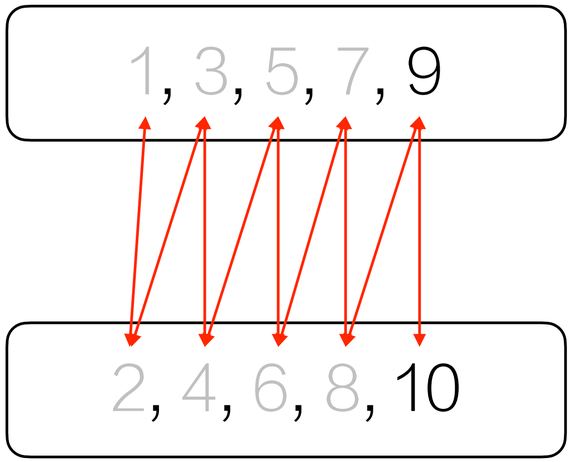
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





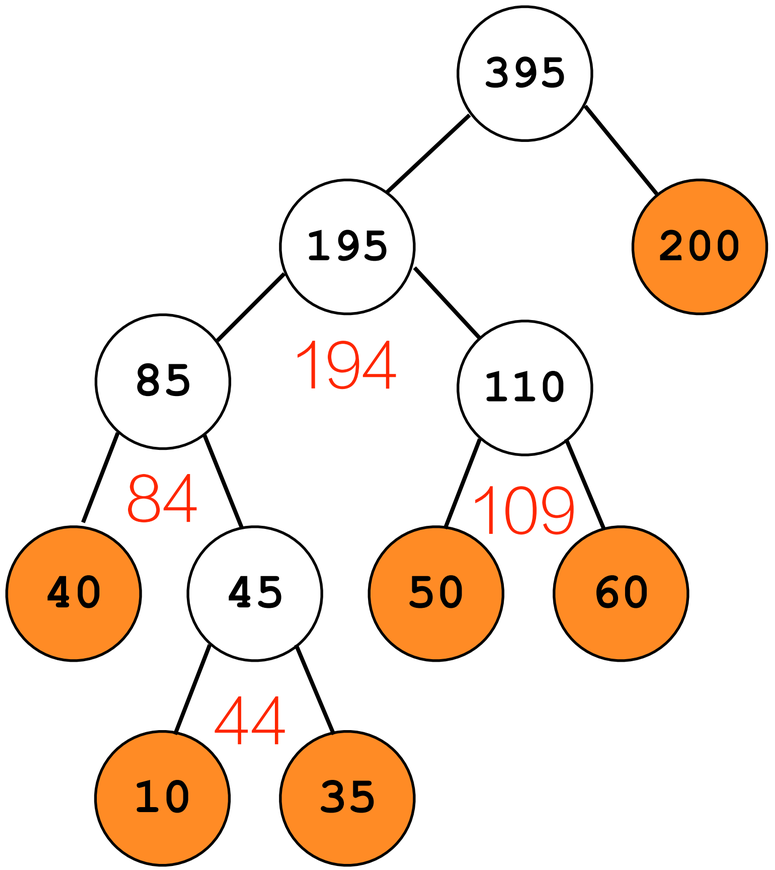
共 10 个元素比较 9 次

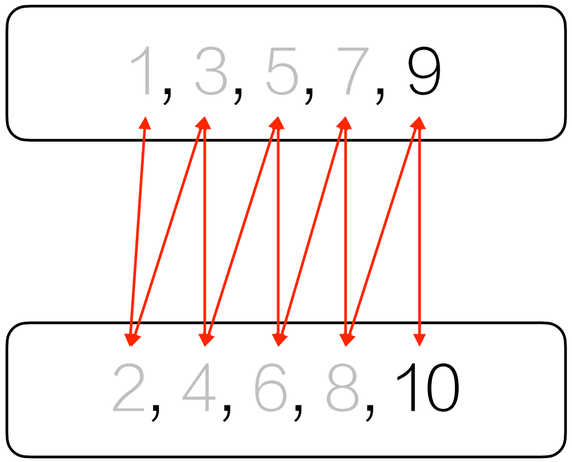
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





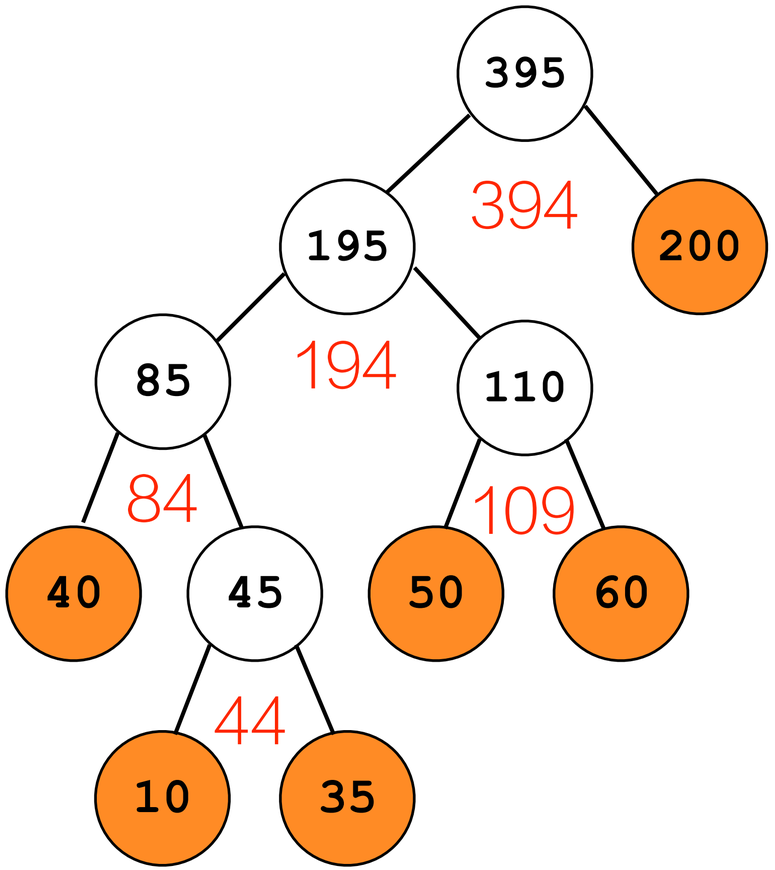
共 10 个元素比较 9 次

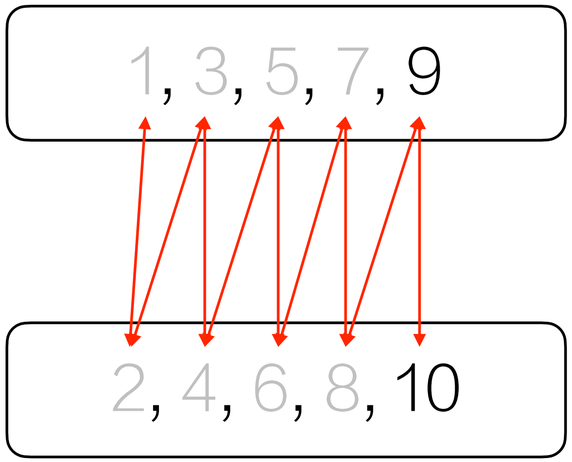
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





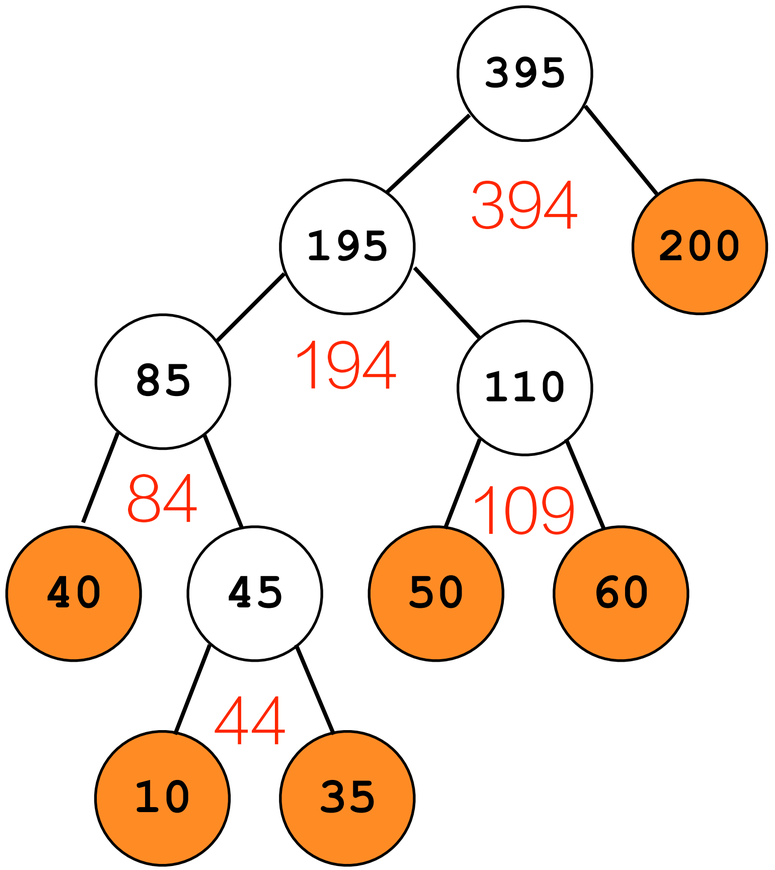
共 10 个元素比较 9 次

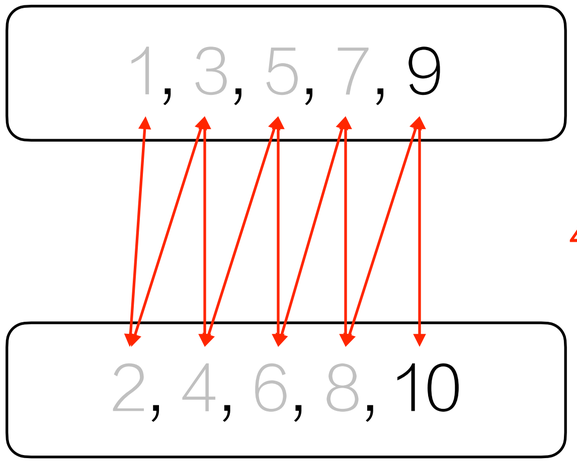
共 n 个元素比较 n−1 次

[2012] 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 N(N≥2) 个不等长升序表的合并策略，并说明理由。





44+84+109+194+394=825

共 10 个元素比较 9 次

共 n 个元素比较 n−1 次

真题解析

[2013] 42.（ 10分 ）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

[2013] 42.（ 10分 ）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

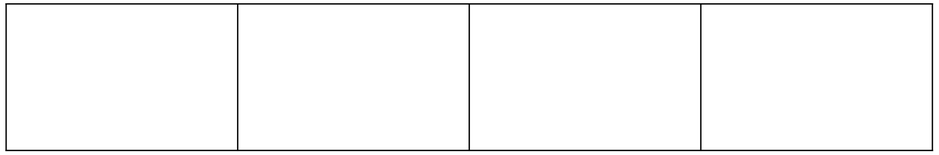
1 ）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2 ）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？



[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

do	for	repeat	while

[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

do	for	repeat	while
0.35	0.15	0.15	0.35

[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

do	while	for	repeat
0.35	0.35	0.15	0.15

[2013] 42.（ 10分 ）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

do	while	for	repeat
0.35	0.35	0.15	0.15

ASL=1*0.35+2*0.35+3*0.15+4*0.15=2.1

[2013] 42.（ 10分 ）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

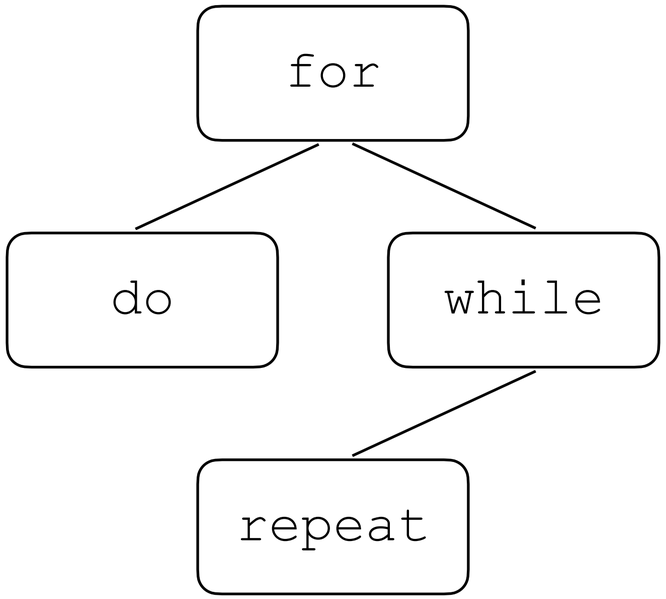
2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？



[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

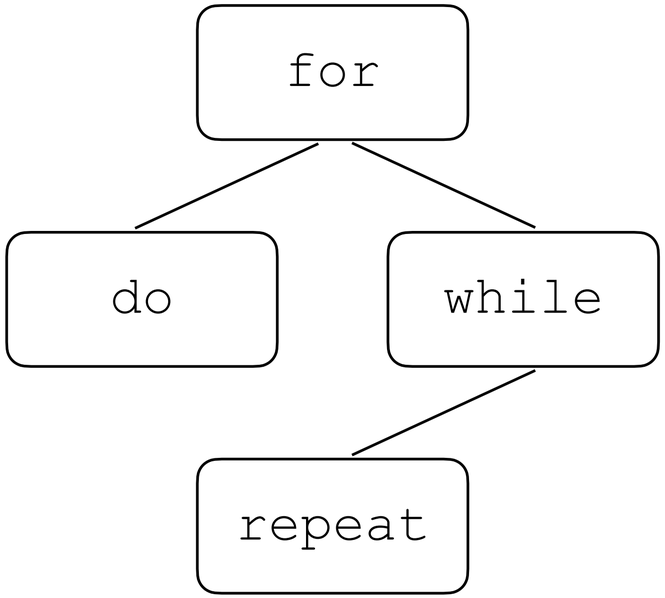
2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？



[2013] 42.（10分）设包含 4 个数据元素的集合 S={"do", "for", "repeat", "while"}，各元素的查找概率依次为 p1=0.35, p2=0.15, p3=0.15, p4=0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？



ASL=1*0.15+2*0.35+2*0.35+3*0.15=2

真题解析

[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小正整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

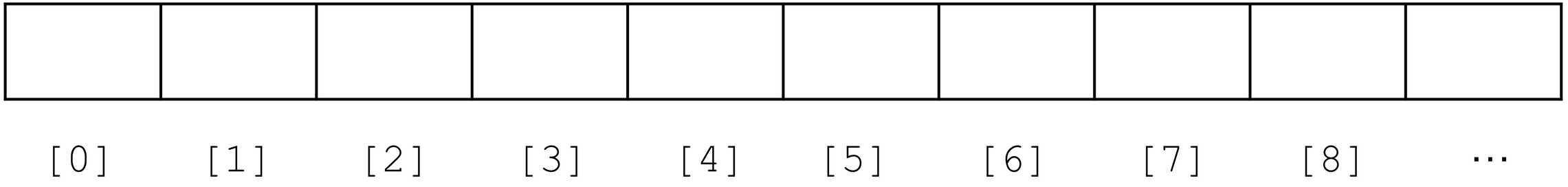
真题解析

[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。



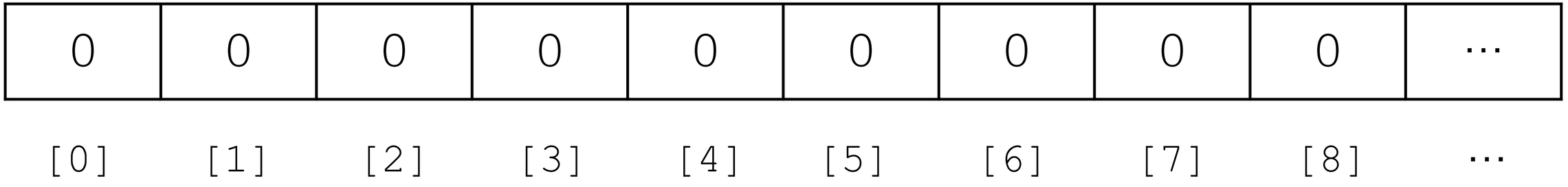
真题解析

[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。



真题解析

[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

{-5, 3, 2, 3}

0	0	0	0	0	0	0	0	0	…
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	…

真题解析

[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小正整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

{-5, 3, 2, 3}

0	0	1	1	0	0	0	0	0	…
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	…

真题解析

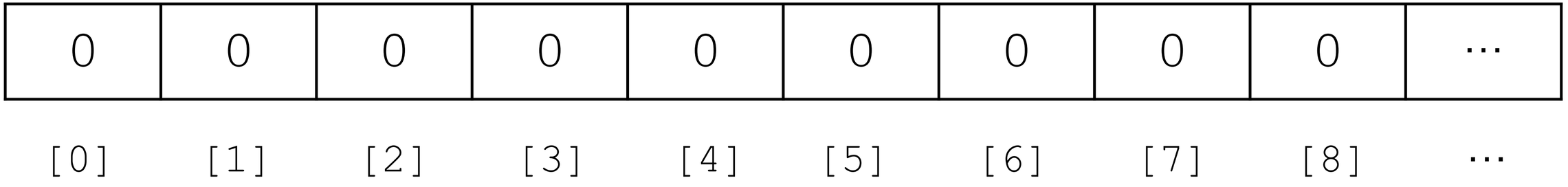
[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

{1, 2, 3}



真题解析

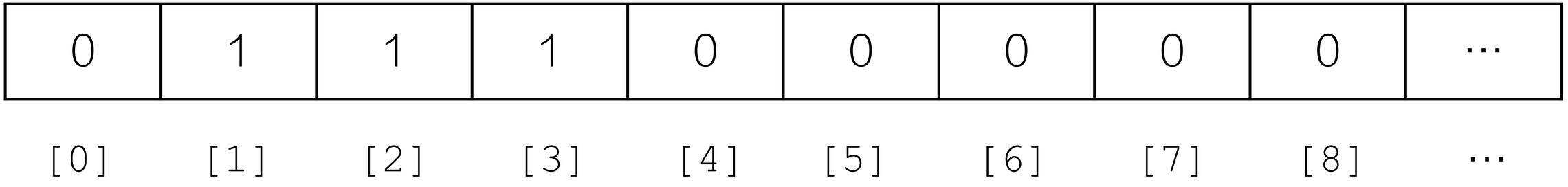
[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

{1, 2, 3}



真题解析

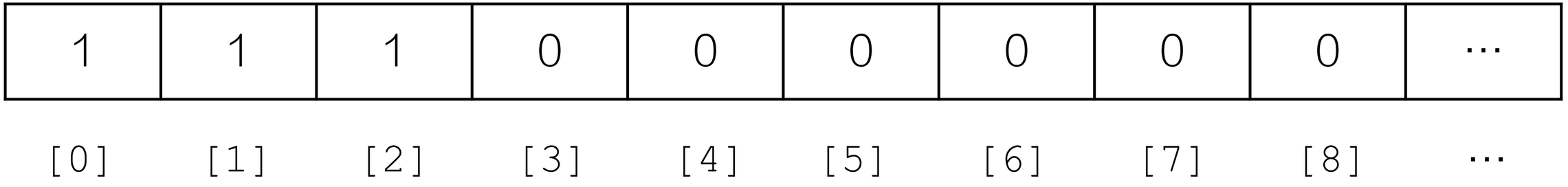
[2018] 41.（13分）给定一个含 n（n≥1）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如数组 {-5, 3, 2, 3} 中未出现的最小下整数是 1；数组 {1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本思想。

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

{1, 2, 3}



[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

请回答下列问题：

1 ）若有 int a[] = {25, −10, 25, 10, 11, 19}, b[6];，则调用 cmpCountSort(a, b, 6)后数组 b 中的内容是什么？

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	0	0	0	0	0	0

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	0	0	0	0	0	0

i:0　　j:1　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	1	0	0	0	0	0

i:0　　j:1　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	1	0	0	0	0	0

i:0　　j:2　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	2	0	0	0	0	0

i:0　　j:2　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	2	0	0	0	0	0

i:0　　j:3　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	3	0	0	0	0	0

i:0　　j:3　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	3	0	0	0	0	0

i:0　　j:4　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	4	0	0	0	0	0

i:0　　j:4　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	4	0	0	0	0	0

i:0　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	0	0	0	0

i:0　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	0	0	0	0

i:1　　j:2　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	1	0	0	0

i:1　　j:2　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	1	0	0	0

i:1　　j:3　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:1　　j:3　　n:6
b							
count	5	0	1	1	0	0	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:1　　j:4　　n:6
b							
count	5	0	1	1	0	0	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	1	1	1	0

i:1　　j:4　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	1	1	1	0

i:1　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	1	1	1	1

i:1　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:2　　j:3　　n:6
b							
count	5	0	1	1	1	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:2　　j:3　　n:6
b							
count	5	0	2	1	1	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:2　　j:4　　n:6
b							
count	5	0	2	1	1	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	3	1	1	1

i:2　　j:4　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	3	1	1	1

i:2　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:2　　j:5　　n:6
b							
count	5	0	4	1	1	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	4	1	1	1

i:3　　j:4　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:3　　j:4　　n:6
b							
count	5	0	4	1	2	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:3　　j:5　　n:6
b							
count	5	0	4	1	2	1	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19	i:3　　j:5　　n:6
b							
count	5	0	4	1	2	2	
	[0]	[1]	[2]	[3]	[4]	[5]	

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	4	1	2	2

i:4　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						
count	5	0	4	1	2	3

i:4　　j:5　　n:6

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a	25	-10	25	10	11	19
b						
count	5	0	4	1	2	3
	[0]	[1]	[2]	[3]	[4]	[5]

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b						25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10					25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10				25	25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10			25	25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11		25	25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11	19	25	25
count	5	0	4	1	2	3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:0 j:1 2 3 4 5

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:0 j:1 2 3 4 5 5

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:1　　j:2 3 4 5　　5

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:1　　j:2 3 4 5　　5+4

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:2 j:3 4 5 5+4

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:2 j:3 4 5

5+4+3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:3 j:4 5

5+4+3

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:3 j:4 5

5+4+3+2

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:4　　j:5

5+4+3+2

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

i:4　　j:5

5+4+3+2+1

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

$(n-1) + (n-2) \cdots + 1$

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

2）若 a 中含有 n 个元素，则算法执行过程中，元素之间的比较次数是多少?

$$(n-1) + (n-2) \cdots + 1 = n(n-1) / 2$$

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11	19	25	25
count	5	0	4	1	2	3

3）该算法是稳定的吗？若是，则阐述理由；否则，修改为稳定排序算法。

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11	19	25	25
count	5	0	4	1	2	3

3）该算法是稳定的吗？若是，则阐述理由；否则，修改为稳定排序算法。

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])     count[j]++;
            else              count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11	19	25	25
count	5	0	4	1	2	3

3）该算法是稳定的吗？若是，则阐述理由；否则，修改为稳定排序算法。

[2021] 42.（ 8分 ）已知某排序算法如下：

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count=(int*)malloc(sizeof(int)*n);    //C++语言：count=new int[n];
    for(i=0;i<n;i++) count[i]=0;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]<a[j])    count[j]++;
            else             count[i]++;
    for(i=0;i<n;i++)    b[count[i]]=a[i];
    free(count);    //C++语言：delete count;
}
```

a[i] <= a[j]

	[0]	[1]	[2]	[3]	[4]	[5]
a	25	-10	25	10	11	19
b	-10	10	11	19	25	25
count	5	0	4	1	2	3

3）该算法是稳定的吗？若是，则阐述理由；否则，修改为稳定排序算法。